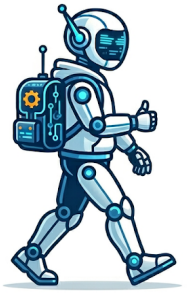
The Hitchhiker's Guide to Agentic AI

From Foundations to Systems



Haggai Roitman

2026

arXiv:2606.24937v3 [cs.AI] 29 Sep 2026

Version 1.4

In memory of Dr. Dafna Sheinwald, R.I.P.
An admired and inspirational colleague.

Contents

Disclaimer **27**

About the Author **28**

In Memoriam: Dr. Dafna Sheinwald **29**

Preface **30**

Introduction **38**

The Guardian Angel System: Our Running Example **42**

Glossary of Acronyms **48**

I Foundations **50**

1 LLM Architecture and Optimization Methods **51**
1.1 How LLMs Work: An Intuitive Overview . . . 52
1.2 Tokenization . . . 52
1.2.1 Why Not Characters or Words? . . . 52
1.2.2 Byte-Pair Encoding (BPE) . . . 53
1.2.3 Other Tokenization Methods . . . 53
1.2.4 Tokenization Best Practices . . . 53
1.2.5 Tokenization in Practice: HuggingFace Example . . . 54
1.2.6 Special Tokens and Structured Prompts . . . 54
1.3 The Transformer Architecture . . . 55
1.3.1 High-Level Structure . . . 55
1.3.2 The Original Encoder-Decoder Transformer . . . 55
1.3.3 Decoder-Only vs Encoder-Decoder . . . 58
1.3.4 Embeddings: From Discrete Tokens to Continuous Space . . . 58
1.3.5 Self-Attention Mechanism . . . 61
1.3.6 Multi-Head Attention . . . 62
1.3.7 Positional Encodings . . . 63
1.3.8 Feed-Forward Network (MLP) . . . 65
1.3.9 Layer Normalization . . . 66
1.3.10 Model Size Reference . . . 67
1.3.11 Attention Pathologies . . . 67
1.3.12 Visualizing Attention for Explainability . . . 68
1.4 Prediction Heads: What Transformers Output . . . 71
1.4.1 Language Modeling Head (Pretraining) . . . 72
1.4.2 Conditional Generation Head (SFT / Instruction Following) . . . 72
1.4.3 Value Head (Regression for RL) . . . 72

1.4.4 Head Selection Summary . . . 73
1.4.5 HuggingFace Implementation . . . 74
1.5 Optimization Theory for LLM Training . . . 76
1.5.1 Gradient Descent: The Foundation . . . 76
1.5.2 Why Vanilla SGD Fails for LLMs . . . 77
1.5.3 Adam – Adaptive Moment Estimation . . . 77
1.5.4 AdamW – Decoupled Weight Decay . . . 78
1.5.5 Muon: Beyond AdamW . . . 78
1.5.6 Learning Rate – The Most Important Hyperparameter . . . 80
1.5.7 Learning Rate Warmup . . . 80
1.5.8 Learning Rate Schedules . . . 80
1.5.9 Gradient Clipping . . . 81
1.5.10 Mixed Precision Training . . . 83
1.5.11 Practical Optimizer Settings by Training Phase . . . 85
1.6 Flash Attention – Algorithm and Hardware Awareness . . . 86
1.6.1 The Standard Attention Memory Problem . . . 86
1.6.2 The Flash Attention Key Insight – Tiling and Online Softmax . . . 87
1.6.3 The Flash Attention Algorithm . . . 87
1.6.4 Flash Attention 2 – Better Parallelism . . . 88
1.6.5 Flash Attention 3 – Hopper Architecture . . . 88
1.6.6 Flash Attention 4 – Blackwell Architecture . . . 88
1.7 Pretraining: Best Practices . . . 89
1.7.1 Training Objective . . . 89
1.7.2 Data Pipeline . . . 89
1.7.3 Scaling Laws . . . 90
1.7.4 Key Hyperparameters . . . 90
1.7.5 Common Failure Modes . . . 90
1.7.6 Mid-Training: Preparing for Reinforcement Learning . . . 90
1.8 Supervised Fine-Tuning (SFT) . . . 91
1.8.1 SFT Objective . . . 91
1.8.2 Data Quality: The LIMA Principle . . . 91
1.8.3 Training Configuration . . . 91
1.8.4 Efficient Training Solutions . . . 92
1.8.5 Best Practices . . . 92
1.9 LoRA and Parameter-Efficient Fine-Tuning . . . 93
1.9.1 The LoRA Insight . . . 93
1.9.2 LoRA Hyperparameters . . . 94
1.9.3 LoRA Variants . . . 95
1.9.4 Other PEFT Approaches . . . 97
1.10 Mixture of Experts (MoE) . . . 98
1.10.1 Architecture . . . 98
1.10.2 Load Balancing . . . 98
1.10.3 Noisy Top-K Gating: Making Discrete Routing Trainable . . . 99
1.10.4 Notable MoE Models . . . 100
1.11 Diversity in LLM Training . . . 100
1.11.1 Sampling Diversity . . . 101
1.11.2 Training Data Diversity . . . 101
1.11.3 Diversity-Promoting Methods . . . 101
1.12 Text Generation: Decoding Methods . . . 101
1.12.1 Greedy Decoding . . . 101
1.12.2 Beam Search . . . 102

1.12.3 Diverse Beam Search . . . 102
1.12.4 Top-k Sampling . . . 103
1.12.5 Top-p (Nucleus) Sampling . . . 103
1.12.6 Min-p Sampling . . . 104
1.12.7 Temperature Scaling . . . 104
1.12.8 Contrastive Decoding . . . 104
1.12.9 Repetition Penalties . . . 105
1.12.10 Practical Comparison . . . 105
1.12.11 Constrained Decoding (Structured Generation) . . . 106
1.13 Prompt Engineering . . . 107
1.13.1 In-Context Learning (ICL) . . . 107
1.13.2 Zero-Shot Prompting . . . 107
1.13.3 Few-Shot Prompting . . . 108
1.13.4 Instruction-Following Prompts . . . 108
1.13.5 Structured Output Prompts (JSON/XML) . . . 109
1.13.6 Chain-of-Thought (CoT) Prompting . . . 110
1.13.7 Advanced Prompting Techniques . . . 111
1.13.8 Best Practices: Crafting Effective Prompts . . . 112
1.14 Model Compression Methods . . . 114
1.14.1 Quantization . . . 114
1.14.2 Pruning . . . 114
1.14.3 Knowledge Distillation . . . 115
1.15 Speculative Decoding Methods . . . 117
1.15.1 Core Principle . . . 117
1.15.2 Medusa: Multi-Head Speculative Decoding . . . 118
1.15.3 Eagle: Feature-Level Drafting . . . 118
1.15.4 N-gram Speculative Decoding . . . 118
1.15.5 Methods Comparison . . . 119
1.15.6 Integration with vLLM . . . 120
1.16 Hallucination Detection . . . 121
1.16.1 Types of Hallucination . . . 121
1.16.2 Detection Methods (Model-Level) . . . 121
1.17 LLM Safety and Responsible AI . . . 122
1.17.1 Threat Taxonomy . . . 122
1.17.2 Safety Training Pipeline . . . 122
1.17.3 Key Safety Mechanisms . . . 122
1.17.4 The Helpfulness–Safety Tradeoff . . . 123
1.17.5 Evaluation . . . 123

2 Systems Foundations for LLMs 124
2.1 GPU Architecture – From Silicon to LLM Training . . . 125
2.1.1 Why GPUs for Deep Learning? . . . 125
2.1.2 NVIDIA GPU Microarchitecture Generations . . . 125
2.1.3 Common GPUs for LLM Training and Inference . . . 125
2.1.4 GPU Internal Architecture – The Streaming Multiprocessor (SM) . . . 126
2.1.5 GPU Chip Scaling Across Generations . . . 127
2.1.6 GPU Memory Hierarchy and Bandwidth . . . 127
2.1.7 Arithmetic Intensity and the Roofline Model . . . 128
2.1.8 Attention is Memory-Bound; FFN is Compute-Bound . . . 129
2.1.9 Tensor Cores . . . 129
2.1.10 Communication Architecture – NVLink, InfiniBand, and PCIe . . . 130

2.2 vLLM – PagedAttention and High-Throughput Inference . . . 133
2.2.1 The KV Cache Fragmentation Problem . . . 133
2.2.2 PagedAttention – Virtual Memory for KV Caches . . . 133
2.2.3 Benefits of PagedAttention . . . 134
2.2.4 Continuous Batching . . . 134
2.2.5 Speculative Decoding in vLLM . . . 135
2.2.6 Concrete Memory Savings – 70B Model at Scale . . . 135
2.2.7 vLLM: End-to-End System . . . 135
2.2.8 Architecture Overview . . . 135
2.2.9 Core Components . . . 135
2.2.10 Request Lifecycle (End-to-End Flow) . . . 136
2.2.11 Prefix Caching (Automatic Prompt Caching) . . . 137
2.2.12 Guided (Constrained) Decoding in vLLM . . . 137

3 Introduction to Reinforcement Learning 139
3.1 The Markov Decision Process (MDP) . . . 140
3.2 Core Concepts and Definitions . . . 140
3.3 Taxonomy of RL Methods . . . 141
3.4 Temporal Difference (TD) Learning . . . 142
3.4.1 Understanding TD Error: "Surprise" as a Learning Signal . . . 142
3.4.2 The TD Error Formula . . . 143
3.4.3 How the Agent Uses TD Error . . . 143
3.5 Q-Learning . . . 143
3.5.1 Understanding Replay Buffers . . . 144
3.6 Policy Gradient Methods — REINFORCE . . . 145
3.7 Actor-Critic Methods . . . 147
3.8 Generalized Advantage Estimation (GAE) . . . 147
3.8.1 Intuitive Mapping of Bias and Variance in GAE . . . 148
3.8.2 The Architectural Spectrum: Boundary Analysis . . . 148
3.8.3 The Trade-off Matrix . . . 149
3.8.4 Diagnostics for Tuning λ . . . 150
3.9 On-Policy vs Off-Policy — Detailed Comparison . . . 150
3.10 Model-Based vs Model-Free . . . 150
3.11 Reward Shaping . . . 151
3.11.1 The Mathematical Framework . . . 151
3.11.2 Potential-Based Reward Shaping (PBRS) . . . 151
3.11.3 Theoretical Guarantees . . . 151

II RL Methods for LLMs 152

4 RL Foundations for Language Models 153
4.1 Two Paradigms for RL in LLMs . . . 154
4.2 Text Generation as an MDP . . . 154
4.3 The RLHF Pipeline . . . 155
4.4 Roadmap of This Part . . . 155

5 PPO — Proximal Policy Optimization 157
5.1 Motivation and History . . . 157
5.2 The Clipped Objective . . . 158
5.3 Full PPO Loss . . . 158
5.4 Derivation of the PPO Gradient and Update Rule . . . 158

5.4.1 Step 1: The RL Objective . . . 158
5.4.2 Step 2: Policy Gradient Theorem . . . 158
5.4.3 Step 3: Importance Sampling for Off-Policy Data . . . 159
5.4.4 Step 4: The Problem with Unconstrained Surrogates . . . 159
5.4.5 Step 5: PPO's Clipped Surrogate (First-Order Approximation) . . . 159
5.4.6 Step 6: The Complete PPO Update Rule . . . 160
5.5 Rollout Buffer and Rollouts . . . 160
5.5.1 What is a Rollout? . . . 160
5.5.2 The Rollout Buffer . . . 160
5.5.3 The Rollout Buffer Lifecycle . . . 161
5.6 PPO for RLHF: The Full Loop . . . 161
5.7 Detailed Mechanics: Logits and Policy Updates . . . 162
5.7.1 Phase 1: Rollout (Data Collection) . . . 162
5.7.2 Phase 2: Optimization Loop (Mini-Batch Updates) . . . 163
5.7.3 From Logits to Probability Ratio . . . 163
5.7.4 The PPO Weight Lifecycle . . . 164
5.7.5 Continuous Action Spaces Extension . . . 164
5.8 TRL Implementation . . . 164
5.9 Critical Hyperparameters . . . 165

6 DPO — Direct Preference Optimization 166
6.1 Motivation . . . 166
6.2 Mathematical Derivation . . . 167
6.3 Gradient Analysis . . . 167
6.4 TRL Implementation . . . 168
6.5 How DPO Works: Full Mechanics . . . 168
6.5.1 Sequence-Level Log-Probabilities . . . 168
6.5.2 The DPO Loss Decomposed . . . 169
6.5.3 Forward Pass: Step by Step . . . 169
6.5.4 Token-Level Gradient Analysis . . . 170
6.5.5 Per-Token vs. Sequence-Level: Length Normalization . . . 170
6.5.6 Label Masking: What Gets Gradients . . . 170
6.5.7 Pseudocode: DPO Training Step . . . 171
6.5.8 Common Pitfalls . . . 171
6.6 DPO Variants and When Each Fails . . . 172
6.7 β Selection Guide . . . 172
6.8 DPO Batch Size Configuration and Scaling . . . 172
6.8.1 Global Batch Size Target . . . 172
6.8.2 Mathematical Decomposition . . . 172
6.8.3 Distributed Scaling Configurations . . . 173
6.8.4 VRAM Optimization: Pre-computing Reference Log-Probabilities . . . 173
6.9 DPO Extensions and Variants . . . 173
6.9.1 f-DPO – Generalised f-Divergence DPO . . . 173
6.9.2 Robust DPO . . . 175
6.9.3 TR-DPO – Trust Region DPO . . . 175
6.9.4 EXO – Exact Optimisation . . . 176
6.9.5 NCA – Noise Contrastive Alignment . . . 177
6.9.6 SLiC-HF – Sequence Likelihood Calibration . . . 177
6.9.7 Iterative RPO – Reasoning Preference Optimisation . . . 178
6.9.8 SimPO – Simple Preference Optimisation . . . 178

7 GRPO — Group Relative Policy Optimization **180**
7.1 Motivation . . . 181
7.2 Algorithm . . . 181
7.3 TRL Implementation . . . 181
7.4 Group Size Analysis . . . 183
7.5 GRPO Variants and Extensions . . . 183
7.5.1 Diversity in GRPO Groups . . . 183
7.5.2 DAPO – Dynamic Adaptive Policy Optimization . . . 186
7.5.3 GSPO – Group Sequence Policy Optimization . . . 187
7.5.4 Dr. GRPO – Debiased Reward GRPO . . . 188
7.5.5 2-GRPO – Minimal Two-Rollout GRPO . . . 189
7.5.6 SAPO – Soft Adaptive Policy Optimization . . . 190
7.5.7 TIS and MIS – Truncated and Masked Importance Sampling . . . 190
7.5.8 VESPO – Variational Sequence-Level Soft Policy Optimization . . . 192
7.5.9 DPPO – Direct Policy Divergence Policy Optimization . . . 192
7.5.10 ScaleRL and CISPO . . . 193
7.5.11 GDPO – Group Reward-Decoupled Policy Optimization . . . 194
7.5.12 GOPO – Group Ordinal Policy Optimization . . . 195

8 Preference Optimization Variants **197**
8.1 Online DPO . . . 197
8.1.1 Motivation . . . 197
8.1.2 Algorithm . . . 198
8.1.3 TRL Implementation . . . 198
8.1.4 Online DPO vs Offline DPO vs PPO . . . 199
8.2 KTO — Kahneman-Tversky Optimization . . . 199
8.2.1 Motivation . . . 199
8.2.2 Loss Function . . . 199
8.2.3 TRL Implementation . . . 199
8.2.4 When to Choose KTO . . . 200
8.3 IPO — Identity Preference Optimization . . . 200
8.3.1 Motivation . . . 200
8.3.2 Loss Function . . . 200
8.3.3 TRL Implementation . . . 201
8.3.4 When to Choose IPO over DPO . . . 201
8.4 ORPO — Odds Ratio Preference Optimization . . . 201
8.4.1 Motivation . . . 201
8.4.2 Loss Function . . . 201
8.4.3 TRL Implementation . . . 202
8.4.4 When to Choose ORPO . . . 202
8.5 Best-of-N Sampling (Rejection Sampling) . . . 202
8.5.1 Motivation . . . 202
8.5.2 Algorithm . . . 203
8.5.3 TRL Implementation . . . 203
8.5.4 Scaling Laws for Best-of-N . . . 204
8.6 Beyond Text Generation: Calibrated Decision Training . . . 204
8.6.1 The Calibration Problem with RLHF and RLVR . . . 204
8.6.2 RLCR: Reinforcement Learning with Calibration Rewards . . . 205
8.6.3 System One Models: A New Output Paradigm . . . 205
8.6.4 CLM: Contrastive Language Models . . . 206
8.6.5 Comparison: RLHF, DPO, RLVR, RLCR, RLCD, CLM . . . 207
8.7 Summary: Choosing an Alignment Method . . . 207

9 Reward Model Training **209**
9.1 Bradley-Terry Model – Full Derivation . . . 209
9.2 Reward Model Architectures . . . 210
9.3 Reward Model Training Tricks . . . 211
9.4 Process Reward Models vs Outcome Reward Models . . . 211
9.5 Rule-Based Rewards for RLVR . . . 213
9.6 Multi-Objective Rewards – Combination Strategies . . . 214
9.7 Listwise Rank-Based Rewards . . . 214

10 SFT Best Practices and Techniques **218**
10.1 Sequence Packing for Efficiency . . . 218
10.2 Chat Templates and Formatting . . . 219
10.3 Completion-Only Masking . . . 220
10.4 Data Mixing Strategies for Multi-Task SFT . . . 221
10.5 When SFT Hurts – Catastrophic Forgetting and Alignment Tax . . . 222
10.5.1 Catastrophic Forgetting (Structural Erasure) . . . 222
10.5.2 Alignment Tax (Behavioral Constraint) . . . 223
10.5.3 Comparative Taxonomy . . . 223
10.5.4 Mitigation Strategies . . . 224
10.6 Connection to RL – SFT Quality Determines RL Ceiling . . . 225

11 System Architecture & Infrastructure at Scale **227**
11.1 The 4-Model Memory Challenge . . . 228
11.2 Parallelism Strategies in Detail . . . 228
11.2.1 Data Parallelism (DP) and Distributed Data Parallelism (DDP) . . . 228
11.2.2 Tensor Parallelism (TP) . . . 229
11.2.3 Sequence Parallelism (SP) . . . 231
11.2.4 Pipeline Parallelism (PP) . . . 232
11.2.5 Fully Sharded Data Parallelism (FSDP / ZeRO-3) . . . 233
11.2.6 3D Parallelism: Combining Strategies . . . 235
11.2.7 Decoupled DiLoCo: Training Across Datacenters . . . 236
11.3 The Generation Bottleneck: Quantitative Analysis . . . 237
11.4 Decoupled Architecture: Production Design . . . 238
11.5 Weight Synchronization Strategies . . . 238
11.6 Memory Optimization Techniques . . . 239
11.6.1 Flash Attention's Impact on RLHF . . . 239
11.6.2 DualKV: Eliminating Shared-Prompt Redundancy During RL Training . . . 240
11.7 Fault Tolerance at Scale . . . 241
11.8 End-to-End Latency Breakdown . . . 242
11.9 Monitoring and Observability . . . 242
11.10 Network Topology and Communication Patterns . . . 243
11.10.1 Intra-Node: NVLink and NVSwitch . . . 243
11.10.2 Inter-Node: InfiniBand and RoCE . . . 243
11.10.3 Communication Primitives and Their Costs . . . 243
11.10.4 Network Topology Design . . . 244
11.11 Training Throughput and Model FLOPs Utilization . . . 244
11.11.1 Measuring Training Efficiency: MFU . . . 244
11.11.2 Compute-Optimal Batch Sizing . . . 245
11.11.3 Profiling and Bottleneck Diagnosis . . . 245
11.12 Cost Analysis and Cloud Deployment . . . 246
11.12.1 Hardware Cost Comparison . . . 246

11.12.2 RLHF Training Cost Estimation . . . 246
11.12.3 Cost Optimization Strategies . . . 246
11.13 Distributed Checkpointing . . . 247
11.13.1 Checkpointing Strategies . . . 247
11.13.2 Production Checkpointing with torch.distributed.checkpoint . . . 247
11.14 Hardware Selection Guide . . . 248
11.15 Optimizer Configuration for RL Training . . . 248
11.15.1 Why RL Requires Different Optimizer Settings . . . 248
11.15.2 Recommended Hyperparameters by RL Method . . . 249
11.15.3 Beta-2 = 0.95 for RL: Faster Adaptation . . . 249
11.15.4 Mixed Precision for RL: FP32 Master Weights Are Critical . . . 249
11.15.5 Gradient Clipping is Critical for RL . . . 249
11.15.6 Diagnosing RL Training Instability . . . 250
11.15.7 HuggingFace TRL Configuration for RL . . . 250
11.15.8 MoE Considerations for RL Training . . . 251

12 LLM Agentic Training **253**
12.1 Motivation: From Chatbots to Autonomous Agents . . . 254
12.2 Trajectory Buffers for LLM Agents . . . 254
12.2.1 Mathematical Structure of an LLM Agent Buffer . . . 255
12.3 Operational Paradigms . . . 255
12.3.1 A. Self-Correction and Thought Refinement . . . 255
12.3.2 B. Off-Policy Exploration . . . 256
12.3.3 C. Non-Parametric In-Context Learning (RAG over Experiences) . . . 256
12.4 Paradigm Comparison . . . 257
12.5 Major Techniques in Agentic RL . . . 257
12.5.1 STaR: Self-Taught Reasoner (Detailed) . . . 257
12.5.2 Reflexion: Verbal Reinforcement Learning (Detailed) . . . 258
12.5.3 ReAct: Reasoning + Acting (Detailed) . . . 259
12.5.4 LATS: Language Agent Tree Search (Detailed) . . . 260
12.5.5 AgentQ: DPO on Agent Trajectories (Detailed) . . . 262
12.5.6 Voyager: Lifelong Learning via Skill Libraries (Detailed) . . . 262
12.5.7 RLEF: RL from Execution Feedback (Detailed) . . . 263
12.5.8 OpenHands / SWE-Agent: GRPO for Software Engineering . . . 264
12.6 Harness Distillation: Baking Scaffolding into Weights . . . 265
12.6.1 Key Concepts . . . 265
12.6.2 Agent-as-Harness . . . 265
12.6.3 Three-Stage Training Pipeline . . . 266
12.6.4 Results . . . 266
12.7 Use Case: Agentic RL for a Productivity Co-pilot . . . 268
12.7.1 Architecture Overview . . . 268
12.7.2 Formal MDP Definition for a Productivity Co-pilot . . . 268
12.7.3 Action Space Design . . . 270
12.7.4 State Representation . . . 270
12.7.5 Reward Design: Multi-Objective Signal . . . 271
12.7.6 Training Pipeline: End-to-End . . . 271
12.7.7 Simulation Environment Architecture . . . 272
12.7.8 Task Curriculum Design . . . 273
12.7.9 Safety and Guardrails . . . 273
12.7.10 Credit Assignment in Multi-App Workflows . . . 274
12.7.11 Scaling and Infrastructure . . . 274

12.7.12 Evaluation Framework . . . 274
12.7.13 Lessons from Production Deployments . . . 275
12.7.14 Complete Training Recipe . . . 275
12.8 Use Case: Building a Research Agent from Scratch . . . 276
12.8.1 Problem Definition . . . 276
12.8.2 MDP Formulation . . . 276
12.8.3 Action Space . . . 277
12.8.4 Architecture: Model and Infrastructure Choices . . . 277
12.8.5 Reward Design . . . 277
12.8.6 Training Pipeline . . . 278
12.8.7 Example Trajectory: Full MDP Trace . . . 278
12.8.8 Key Design Decisions and Tradeoffs . . . 280
12.8.9 Evaluation . . . 280
12.8.10 Lessons and Failure Modes . . . 281
12.9 State-of-the-Art RL for LLM Agents . . . 281
12.9.1 Dominant Baseline: GRPO for Agents . . . 281
12.9.2 PPO for Interactive Agents . . . 282
12.9.3 Fine-Grained Turn-Level Credit Assignment . . . 282
12.9.4 Alternative Paradigms . . . 283
12.9.5 Core Methodology Comparison . . . 283
12.10 Interactive RL Environments for Agent Training . . . 283
12.10.1 NeMo Gym (NVIDIA) . . . 283
12.10.2 RLFactory . . . 284
12.10.3 MOSAIC: Safety in Agentic Training . . . 284

III Reasoning **286**

13 RL for Large Reasoning Models **287**
13.1 Motivation and Background . . . 288
13.1.1 Why Reasoning Requires Different RL Approaches . . . 288
13.1.2 Chain-of-Thought: Emergent Behavior vs. Trained Capability . . . 288
13.1.3 Test-Time Compute Scaling Laws . . . 289
13.2 Test-Time Scaling Methods . . . 289
13.2.1 Chain-of-Thought (CoT) . . . 289
13.2.2 Self-Consistency (Majority Voting) . . . 290
13.2.3 Tree-of-Thoughts (ToT) . . . 291
13.2.4 Graph-of-Thoughts (GoT) . . . 292
13.2.5 Best-of-N with Reward Models . . . 293
13.2.6 Monte Carlo Tree Search (MCTS) for Reasoning . . . 294
13.2.7 Beam Search over Reasoning Steps . . . 295
13.2.8 Iterative Refinement and Self-Correction . . . 296
13.2.9 Method Comparison and Selection Guide . . . 296
13.3 DeepSeek-R1 . . . 297
13.3.1 Two-Stage Training Pipeline . . . 297
13.3.2 Reward Design: Accuracy and Format Rewards . . . 297
13.3.3 GRPO Formulation for R1 . . . 298
13.3.4 Distillation: The R1-Distill Series . . . 299
13.4 OpenAI o1/o3 Series . . . 299
13.4.1 Chain-of-Thought RL with Hidden Reasoning Tokens . . . 300
13.4.2 Process Reward Models vs. Outcome Reward Models . . . 300

13.4.3 Inference-Time Compute Scaling . . . 301
13.4.4 Training Compute vs. Test-Time Compute . . . 301
13.4.5 o3 and o4-mini Architecture Insights . . . 301
13.5 QwQ and Qwen Reasoning Models . . . 301
13.5.1 Multi-Stage RL Pipeline . . . 301
13.5.2 Rejection Sampling + RL Combination . . . 302
13.5.3 Tool-Integrated Reasoning . . . 302
13.6 Key Methods with Mathematical Foundations . . . 303
13.6.1 Monte Carlo Tree Search for Reasoning . . . 303
13.6.2 Process Reward Models . . . 303
13.6.3 Outcome Reward Models and Majority Voting . . . 304
13.6.4 Self-Play for Reasoning . . . 304
13.6.5 Reinforcement Learning from Verifiable Rewards (RLVR) . . . 305
13.6.6 Journey Learning . . . 305
13.6.7 Quiet-STaR: Reasoning at Every Token . . . 306
13.7 Scaling Laws for Reasoning . . . 306
13.7.1 Training Compute vs. Test-Time Compute Tradeoff . . . 306
13.7.2 When to Invest in Longer Chains vs. Better Base Models . . . 307
13.7.3 Optimal Token Budget Allocation . . . 307
13.8 Comparison of Reasoning Models . . . 308
13.9 Inference-Time Compute Scaling Laws . . . 308
13.9.1 The Overthinking Problem . . . 308
13.9.2 Efficient Reasoning Techniques . . . 308
13.9.3 Connection to RL: Inference Compute as MDP Horizon . . . 309
13.9.4 Latent Reasoning: Is the Chain the Computation? . . . 309
13.10 On-Policy Self-Distillation for Reasoning . . . 310
13.10.1 The Training Objective . . . 310
13.10.2 Why Dense Supervision Matters . . . 310
13.10.3 Failure Modes and Mitigations . . . 311
13.10.4 Connection to RL and the Training Pipeline . . . 311
13.11 Summary and Open Problems . . . 311

IV Evaluation **313**

14 LLM Evaluation **314**
14.1 Evaluation Scheme Design . . . 315
14.1.1 Taxonomy of Evaluation Types . . . 315
14.1.2 When to Use What . . . 316
14.2 Data Collection for Evaluation . . . 316
14.2.1 Human Annotation Pipelines . . . 316
14.2.2 Inter-Annotator Agreement . . . 316
14.2.3 Annotation Guideline Design . . . 317
14.2.4 Crowdsourcing vs. Expert Annotation . . . 317
14.3 Synthetic Data Generation for Evaluation . . . 318
14.3.1 LLM-as-Judge for Calibration . . . 318
14.3.2 Self-Instruct . . . 318
14.3.3 Evol-Instruct . . . 319
14.3.4 Constitutional AI Data Generation . . . 319
14.3.5 Distillation for Evaluation Data . . . 319
14.3.6 Arena-Style Pairwise Generation . . . 320

14.4 Metrics for Ranking Tasks . . . 320
14.4.1 ELO Rating System . . . 320
14.4.2 Bradley–Terry Model . . . 320
14.4.3 TrueSkill . . . 321
14.4.4 Win Rate with Confidence Intervals . . . 321
14.4.5 Chatbot Arena Methodology . . . 321
14.5 Metrics for Generation Tasks . . . 322
14.5.1 BLEU . . . 322
14.5.2 ROUGE . . . 322
14.5.3 BERTScore . . . 322
14.5.4 METEOR . . . 323
14.5.5 Perplexity . . . 323
14.5.6 Pass@k for Code . . . 323
14.5.7 Exact Match and F1 . . . 324
14.6 Metrics for Agentic Tasks . . . 324
14.6.1 Task Success Rate . . . 324
14.6.2 Trajectory Efficiency . . . 325
14.6.3 Tool-Use Accuracy . . . 325
14.6.4 Multi-Step Reasoning Accuracy . . . 325
14.6.5 SWE-bench Methodology . . . 325
14.6.6 WebArena Methodology . . . 326
14.7 LLM-as-Judge . . . 326
14.7.1 Setup and Prompt Templates . . . 326
14.7.2 Position Bias Mitigation . . . 327
14.7.3 Multi-Judge Panels . . . 328
14.7.4 Agreement Metrics for LLM Judges . . . 328
14.7.5 G-Eval Framework . . . 328
14.7.6 JEV-as-a-Judge: Decision-Only Judging with Confidence Cascades . . . 329
14.8 Evaluation Pitfalls . . . 330
14.8.1 Benchmark Contamination . . . 330
14.8.2 Overfitting to Benchmarks . . . 330
14.8.3 Goodhart's Law in Evaluation . . . 331
14.8.4 Additional Pitfalls . . . 331
14.8.5 Reasoning Collapse Under Reformulation . . . 332

V Agentic AI **333**

15 Introduction to Agentic AI **334**

16 Retrieval-Augmented Generation (RAG) **338**
16.1 Motivation and Problem Statement . . . 339
16.1.1 Parametric vs. Non-Parametric Knowledge . . . 339
16.1.2 When to Use RAG vs. Fine-Tuning vs. Long Context . . . 339
16.2 Core RAG Architecture . . . 339
16.2.1 Full Pipeline Diagram . . . 340
16.2.2 Indexing Pipeline . . . 340
16.2.3 Retrieval . . . 340
16.2.4 Generation . . . 340
16.3 Retrieval Methods . . . 341
16.3.1 Sparse Retrieval: BM25 and TF-IDF . . . 341
16.3.2 Dense Retrieval: DPR . . . 341

16.3.3 Hybrid Retrieval with Reciprocal Rank Fusion . . . 342
16.3.4 Learned Sparse Retrieval: SPLADE and SPLADEv2 . . . 342
16.3.5 ColBERT: Late Interaction . . . 344
16.3.6 Retrieval Method Comparison . . . 345
16.4 Chunking Strategies . . . 345
16.4.1 Fixed-Size Chunking with Overlap . . . 345
16.4.2 Semantic Chunking . . . 345
16.4.3 Document-Structure-Aware Chunking . . . 346
16.4.4 Parent-Child Chunking . . . 346
16.4.5 Empirical Guidelines for Chunk Size . . . 347
16.5 Advanced RAG Patterns . . . 347
16.5.1 Query Transformation . . . 347
16.5.2 Re-Ranking . . . 347
16.5.3 Contextual Compression . . . 348
16.5.4 Self-RAG . . . 348
16.5.5 CRAG: Corrective RAG . . . 348
16.5.6 Adaptive RAG . . . 349
16.5.7 Graph RAG . . . 349
16.5.8 RAG-Fusion . . . 349
16.6 Efficient RAG Decoding: REFRAG . . . 350
16.7 Agentic RAG . . . 350
16.7.1 Motivation: Limits of Static RAG . . . 350
16.7.2 Agentic RAG Architecture . . . 350
16.7.3 Multi-Source Routing . . . 350
16.7.4 Full Agentic RAG Implementation . . . 352
16.7.5 Tool-Augmented RAG . . . 354
16.7.6 Search-R1: RL-Trained Agentic RAG . . . 354
16.8 Evaluation . . . 356
16.8.1 Retrieval Metrics . . . 356
16.8.2 Generation Metrics . . . 357
16.8.3 RAGAs Framework . . . 357
16.8.4 Common Failure Modes . . . 358
16.9 Production Considerations . . . 358
16.9.1 Embedding Model Selection . . . 358
16.9.2 Vector Database Comparison . . . 359
16.9.3 Latency Optimization . . . 359
16.9.4 Incremental Indexing and Versioning . . . 360
16.10 RAG + Fine-Tuning Synergy . . . 361
16.10.1 When to Combine RAG with Fine-Tuning . . . 361
16.10.2 RAFT: Retrieval-Augmented Fine-Tuning . . . 361
16.10.3 Joint Retriever-Generator Training . . . 361
16.11 Comprehensive RAG Approach Comparison . . . 362

17 Agentic Memory Systems **363**
17.1 Motivation: Why Agents Need Memory . . . 364
17.2 Taxonomy of Memory Types . . . 364
17.2.1 Working Memory (Short-Term) . . . 364
17.2.2 Episodic Memory (Experience-Based) . . . 365
17.2.3 Semantic Memory (World Knowledge) . . . 365
17.2.4 Procedural Memory (Skills) . . . 365
17.3 Memory Architectures . . . 366

17.3.1 RAG-Based Memory . . . 366
17.3.2 Summarization-Based Memory . . . 367
17.3.3 Graph-Based Memory . . . 367
17.3.4 Key-Value Memory Networks . . . 368
17.3.5 MemGPT and Virtual Context Management . . . 368
17.4 Memory Operations . . . 368
17.4.1 Write: Committing to Memory . . . 368
17.4.2 Read / Retrieve . . . 369
17.4.3 Update: Conflict Resolution and Consolidation . . . 370
17.4.4 Reflect: Meta-Cognitive Operations . . . 371
17.5 Memory for Multi-Turn Conversations . . . 371
17.5.1 User Modeling and Preference Tracking . . . 371
17.5.2 Session Continuity . . . 372
17.5.3 Personalization Through Memory . . . 372
17.6 Memory for Multi-Agent Systems . . . 372
17.6.1 Shared Memory Pools . . . 372
17.6.2 Blackboard Architecture . . . 372
17.6.3 Consensus and Conflict in Shared Knowledge . . . 373
17.7 Training Memory Systems with Reinforcement Learning . . . 373
17.7.1 Reward Signals for Memory Operations . . . 373
17.7.2 Learning What to Remember . . . 373
17.7.3 Memory-Augmented Policy Optimization . . . 374
17.8 Comparison of Memory Approaches . . . 374
17.9 Evaluating Memory Systems . . . 374
17.9.1 Evaluation Dimensions . . . 374
17.9.2 Benchmarks . . . 375
17.9.3 Metrics . . . 375
17.10 Implementation Patterns . . . 376
17.10.1 Vector Store Memory with Embeddings . . . 376
17.10.2 Hierarchical Memory Manager . . . 378
17.10.3 Memory-Augmented Agent Loop . . . 381
17.11 Recent Advances in Agentic Memory . . . 384
17.11.1 CoALA: Cognitive Architectures for Language Agents . . . 385
17.11.2 Mem0: Production-Scale Memory Layer . . . 385
17.11.3 Sleep-Time Compute and Agent Dreaming . . . 385
17.11.4 A-MEM: Zettelkasten-Inspired Agentic Memory . . . 387
17.12 Proactive Memory Architectures . . . 388
17.13 Summary . . . 389

18 Agent Harness – Context Management and Orchestration 390
18.1 What Is an Agent Harness? . . . 391
18.2 Context Window Management . . . 391
18.2.1 The Context Budget Problem . . . 391
18.2.2 Context Allocation Strategies . . . 392
18.2.3 Context Compression . . . 392
18.2.4 Sliding Window Approaches . . . 393
18.2.5 Recursive Context Decomposition . . . 393
18.2.6 Token Counting and Budget Monitoring . . . 394
18.3 Prompt Architecture . . . 395
18.3.1 System Prompt Design . . . 395
18.3.2 Dynamic Prompt Assembly . . . 395

18.3.3 Few-Shot Management . . . 396
18.3.4 Tool Descriptions . . . 396
18.4 Tool Integration and Execution . . . 397
18.4.1 Tool Definition Schemas . . . 397
18.4.2 Tool Selection and Routing . . . 398
18.4.3 Tool Output Processing . . . 398
18.4.4 Sandboxing and Safety . . . 399
18.4.5 Model Context Protocol (MCP) . . . 399
18.5 Orchestration Patterns . . . 400
18.5.1 ReAct Loop (Reason + Act) . . . 400
18.5.2 Plan-and-Execute . . . 401
18.5.3 Multi-Agent Orchestration . . . 401
18.5.4 Human-in-the-Loop . . . 402
18.5.5 Workflow Graphs . . . 402
18.6 State Management . . . 403
18.6.1 Conversation State . . . 403
18.6.2 Task State . . . 403
18.6.3 Agent State . . . 403
18.6.4 Persistent State . . . 403
18.7 Error Handling and Recovery . . . 404
18.7.1 Retry Strategies . . . 404
18.7.2 Loop Detection . . . 404
18.7.3 Graceful Failure . . . 404
18.7.4 Observability . . . 404
18.8 Scaling and Production Concerns . . . 405
18.8.1 Latency Optimization . . . 405
18.8.2 Cost Management . . . 405
18.8.3 Rate Limiting and Queuing . . . 405
18.8.4 Evaluation in Production . . . 406
18.9 Framework Comparison . . . 406
18.10Implementation: Production Agent Harness . . . 407

19 Loop Engineering **416**
19.1 Context Engineering: The Layer Beneath the Loop . . . 417
19.2 Loop Engineering as Inference-Time Reinforcement Learning . . . 418
19.3 Anatomy of a Production Loop . . . 419
19.3.1 The Five Primitives . . . 419
19.3.2 External State: The Loop's Memory . . . 420
19.4 The Loop in Pseudocode . . . 420
19.5 Loop Patterns . . . 421
19.5.1 The Validation Loop . . . 421
19.5.2 The Reflexion Loop . . . 422
19.5.3 The Evaluator-Optimizer Loop . . . 422
19.5.4 The Hierarchical Loop . . . 422
19.5.5 The Autonomous Research Loop . . . 423
19.6 Verification Engineering . . . 423
19.6.1 The Verification Hierarchy . . . 423
19.6.2 Deterministic vs. Probabilistic Verification . . . 423
19.7 Termination Engineering . . . 424
19.7.1 Termination Conditions . . . 424
19.7.2 The Exploration Problem . . . 425

19.8 Cognitive Escalation and Uncertainty-Driven Handoff . . . 425
19.8.1 Confidence Estimation Mid-Task . . . 425
19.8.2 Dynamic Escalation Thresholds . . . 426
19.8.3 Escalation Handoff Protocol . . . 426
19.9 Failure Modes and Anti-Patterns . . . 427
19.9.1 The Loopmaxxing Trap . . . 427
19.9.2 Comprehension Debt . . . 427
19.9.3 Reward Hacking in Loops . . . 428
19.9.4 Context Degradation . . . 428
19.10 Production Loop Architectures . . . 428
19.10.1 The Nightly Maintenance Loop . . . 428
19.10.2 The Continuous Experimentation Loop . . . 429
19.10.3 The Always-On Orchestrator . . . 429
19.10.4 Parallel Subagent Fleet Execution . . . 430
19.11 Context Management in Long-Running Loops . . . 431
19.11.1 Compaction Strategies . . . 431
19.11.2 The Context Budget . . . 431
19.12 When Not to Use Loops . . . 432
19.13 The Economics of Loops . . . 432
19.14 Historical Context and Related Work . . . 432
19.15 Summary . . . 433

20 Graph Engineering **434**
20.1 Why Graphs? From Loops to Topologies . . . 435
20.2 Graph Primitives . . . 435
20.2.1 Nodes . . . 436
20.2.2 Edges and Routing . . . 436
20.2.3 Shared State . . . 437
20.3 Core Graph Topologies . . . 437
20.3.1 Linear Chain . . . 437
20.3.2 Branch and Merge (Fan-Out / Fan-In) . . . 438
20.3.3 Cycle (Retry Loop) . . . 438
20.3.4 Hierarchical Subgraph . . . 438
20.3.5 Map-Reduce . . . 439
20.3.6 Supervisor–Worker . . . 439
20.4 State Management in Graphs . . . 439
20.4.1 State Update Semantics . . . 440
20.4.2 Checkpointing and Persistence . . . 440
20.4.3 Thread Isolation . . . 440
20.5 Graph Engineering vs. Loop Engineering . . . 441
20.6 Production Graph Architectures . . . 441
20.6.1 The Research-and-Write Graph . . . 441
20.6.2 The Multi-Agent Supervisor Graph . . . 442
20.6.3 The Human-in-the-Loop Graph . . . 443
20.7 Graph Engineering with LangGraph . . . 443
20.8 Failure Modes and Anti-Patterns . . . 444
20.8.1 The God Node . . . 444
20.8.2 Implicit State Coupling . . . 444
20.8.3 Missing Termination Guards on Cycles . . . 444
20.8.4 State Schema Drift . . . 445
20.8.5 Parallel Branch State Conflicts . . . 445

20.9 Graph Engineering Best Practices . . . 445
20.10 The Graph Engineering Progression . . . 446
20.11 Summary . . . 446

21 Agent Design Patterns **448**
21.1 Workflow Patterns . . . 449
21.1.1 Prompt Chaining . . . 449
21.1.2 Routing . . . 449
21.1.3 Parallelization . . . 450
21.1.4 Orchestrator-Workers . . . 450
21.1.5 Evaluator-Optimizer . . . 450
21.2 Autonomous Agent Patterns . . . 451
21.2.1 ReAct (Reason + Act) . . . 451
21.2.2 Planning Agents . . . 451
21.2.3 Reflection and Self-Critique . . . 452
21.2.4 Tool-Use Patterns . . . 452
21.3 Design Principles . . . 453
21.4 Pattern Selection Guide . . . 454
21.5 Agentic Security and Adversarial Red Teaming . . . 454
21.5.1 The Agentic Attack Surface . . . 454
21.5.2 Indirect Prompt Injection in Depth . . . 455
21.5.3 Red-Teaming Methodology for Agentic Systems . . . 455
21.5.4 Defense Frameworks and Engineering Controls . . . 456
21.6 Agent Gateway Infrastructure . . . 457
21.6.1 Why Agents Need a Gateway . . . 457
21.6.2 Core Gateway Capabilities . . . 458
21.6.3 Deployment Patterns . . . 458

22 Agentic Environments and Benchmarks **460**
22.1 Motivation: Why Agents Need Environments . . . 461
22.2 Environment Design Principles . . . 461
22.2.1 Observation Space Design . . . 461
22.2.2 Action Space Design . . . 462
22.2.3 Reward Signal Design . . . 462
22.2.4 Episode Structure . . . 463
22.2.5 Difficulty Curriculum and Adaptive Environments . . . 463
22.3 Types of Agentic Environments . . . 464
22.3.1 Code Execution Sandboxes . . . 464
22.3.2 Web Environments . . . 465
22.3.3 Computer Use Environments . . . 466
22.3.4 Software Engineering Environments . . . 466
22.3.5 Scientific Research Environments . . . 466
22.3.6 Game and Simulation Environments . . . 467
22.3.7 Multi-Agent Environments . . . 467
22.4 OpenEnv: Standardized Agentic Environment Interfaces . . . 467
22.4.1 Standardized Agent–Environment Interface . . . 468
22.4.2 Environment Registries and Discovery . . . 469
22.4.3 Compositional Environments . . . 470
22.4.4 Environment Versioning and Reproducibility . . . 470
22.5 Building Custom Environments . . . 470
22.5.1 Gymnasium-Style API for LLM Agents . . . 470

22.5.2 Reward Function Engineering . . . 471
22.5.3 State Management and Checkpointing . . . 471
22.5.4 Parallelization for Training Data Collection . . . 471
22.6 Environment–Agent Interface Patterns . . . 471
22.7 Evaluation Harness Design . . . 472
22.7.1 Deterministic vs. Stochastic Environments . . . 473
22.7.2 Held-Out Test Environments . . . 473
22.7.3 Cross-Environment Generalization . . . 473
22.7.4 Human Baseline Collection . . . 473
22.8 Code Example: Minimal Custom LLM Agent Environment . . . 473
22.9 Comparison of Major Agentic Environments . . . 477
22.10 Legal and Regulatory Considerations for Agentic Systems . . . 477
22.10.1 The Liability Gap . . . 477
22.10.2 EU AI Act Implications for Agentic Systems . . . 478
22.10.3 Engineering for Compliance . . . 478
22.10.4 Practical Liability Mitigation . . . 479
22.11 Production-Grade Evaluation of Non-Deterministic Agents . . . 480
22.11.1 The Non-Determinism Problem . . . 480
22.11.2 Statistical Test Oracles . . . 480
22.11.3 Trace-Level Regression Testing . . . 481
22.11.4 Evaluation Infrastructure . . . 481
22.12 Summary . . . 482
22.12.1 Emerging Benchmarks (2026) . . . 482

23 Model Context Protocol (MCP) 484
23.1 Motivation: The Tool Integration Problem . . . 485
23.2 Architecture Overview . . . 485
23.2.1 The Three-Role Model . . . 485
23.2.2 Transport Layers . . . 486
23.2.3 Protocol Lifecycle . . . 487
23.2.4 Stateful Sessions vs. Stateless Requests . . . 487
23.2.5 Full Architecture Diagram . . . 487
23.3 Core Primitives . . . 487
23.3.1 Tools . . . 487
23.3.2 Resources . . . 488
23.3.3 Prompts . . . 488
23.3.4 Sampling . . . 488
23.4 Protocol Specification . . . 489
23.4.1 JSON-RPC 2.0 Message Format . . . 489
23.4.2 Capability Negotiation Handshake . . . 489
23.4.3 Error Handling . . . 490
23.4.4 Progress Reporting . . . 490
23.5 Tool Definition and Discovery . . . 491
23.5.1 Tool Schema Format . . . 491
23.5.2 Dynamic Tool Registration . . . 491
23.5.3 Tool Annotations . . . 492
23.6 Security Model . . . 493
23.6.1 Trust Hierarchy . . . 493
23.6.2 User Consent . . . 493
23.6.3 Input Validation and Sanitization . . . 493
23.6.4 Credential Management . . . 494

23.6.5 Sandboxing Strategies . . . 494
23.7 Implementation Patterns . . . 494
23.7.1 Building an MCP Server in Python . . . 494
23.7.2 Building an MCP Client . . . 495
23.7.3 Connecting to Multiple Servers Simultaneously . . . 496
23.7.4 Error Recovery and Reconnection . . . 497
23.8 The MCP Ecosystem . . . 498
23.8.1 Popular MCP Servers . . . 498
23.8.2 MCP in Production Applications . . . 498
23.8.3 Server Registries and Discovery . . . 499
23.9 MCP vs. Alternatives . . . 499
23.9.1 When to Use MCP vs. Custom Integration . . . 499
23.9.2 Migration Paths . . . 500
23.10 MCP for Agent Training . . . 500
23.10.1 MCP Servers as RL Environment Interfaces . . . 500
23.10.2 Standardized Action Spaces via MCP . . . 500
23.10.3 Recording Tool-Use Trajectories for SFT . . . 501
23.11 Summary . . . 503

24 Agent Skills **504**
24.1 What Is a Skill? . . . 505
24.2 Skill Architecture Patterns . . . 505
24.2.1 Static Skill Loading . . . 505
24.2.2 Dynamic Skill Discovery . . . 505
24.2.3 Hierarchical Skill Composition . . . 506
24.3 Case Study: Anthropic's Agent Design . . . 506
24.3.1 Core Principles . . . 506
24.3.2 Building Block Patterns . . . 506
24.3.3 The Augmented LLM . . . 506
24.3.4 Practical Implications . . . 507
24.4 Skill Lifecycle . . . 507
24.5 Skill Registries and Marketplaces . . . 508
24.6 Self-Evolving Agents: Lifelong Skill Acquisition . . . 508
24.6.1 The Self-Improvement Spectrum . . . 509
24.6.2 Tool Synthesis and Automatic Skill Authoring . . . 509
24.6.3 Persistent Self-Modification Risks . . . 510
24.6.4 Safe Self-Evolution Patterns . . . 510
24.7 Skills vs. Fine-Tuning . . . 511

25 Agent-to-Agent Communication (A2A) **512**
25.1 Motivation: Why Agents Must Communicate . . . 513
25.2 The Google A2A Protocol . . . 513
25.2.1 Design Philosophy . . . 514
25.2.2 Agent Cards . . . 514
25.2.3 Task Lifecycle . . . 515
25.2.4 Streaming via Server-Sent Events . . . 515
25.2.5 Push Notifications for Long-Running Tasks . . . 516
25.2.6 Message Format . . . 517
25.2.7 Authentication and Authorization . . . 517
25.3 Communication Patterns . . . 517
25.3.1 Request-Response . . . 517

25.3.2 Streaming . . . 517
25.3.3 Multi-Turn Interaction . . . 517
25.3.4 Broadcast . . . 518
25.3.5 Publish-Subscribe (Pub-Sub) . . . 518
25.3.6 Negotiation . . . 518
25.3.7 Auction-Based Task Allocation . . . 518
25.4 Agent Discovery and Routing . . . 518
25.4.1 Agent Registries . . . 518
25.4.2 Capability-Based Routing . . . 519
25.4.3 Load Balancing Across Equivalent Agents . . . 520
25.4.4 Version Management and Compatibility . . . 520
25.5 Message Formats and Schemas . . . 520
25.5.1 Structured vs. Unstructured Messages . . . 520
25.5.2 Multi-Modal Messages . . . 520
25.5.3 Context Passing: What to Share vs. What to Keep Private . . . 521
25.5.4 Conversation Threading and Correlation IDs . . . 522
25.6 Coordination Protocols . . . 522
25.6.1 Contract Net Protocol . . . 522
25.6.2 Blackboard Systems . . . 524
25.6.3 Consensus Protocols . . . 524
25.6.4 Leader Election . . . 524
25.7 A2A vs. MCP: Complementary Protocols . . . 525
25.7.1 When to Use Which . . . 525
25.7.2 Combined Architecture . . . 525
25.8 Security and Trust in Multi-Agent Systems . . . 526
25.8.1 Agent Identity Verification . . . 526
25.8.2 Message Integrity and Encryption . . . 526
25.8.3 Authorization Scopes . . . 527
25.8.4 Audit Trails and Accountability . . . 527
25.9 Implementation Example: Multi-Agent Research Workflow . . . 527
25.10Summary . . . 533

26 Multi-Agent Systems **535**
26.1 Motivation: Why Multiple Agents? . . . 536
26.2 Multi-Agent Architectures . . . 536
26.2.1 Centralized (Supervisor/Manager) Architecture . . . 537
26.2.2 Decentralized (Peer-to-Peer) Architecture . . . 538
26.2.3 Hierarchical Architecture . . . 539
26.2.4 Swarm Architecture . . . 540
26.2.5 The Orchestrator-Worker-Critic Triad . . . 541
26.2.6 Agent Mesh Architecture . . . 542
26.3 Coordination Mechanisms . . . 542
26.3.1 Shared State (Global Blackboard) . . . 542
26.3.2 Message Passing . . . 543
26.3.3 Planning and Decomposition . . . 543
26.3.4 Voting and Consensus . . . 544
26.3.5 Market-Based Coordination . . . 545
26.3.6 Stigmergy: Indirect Communication Through Environment . . . 545
26.4 Communication Protocols . . . 545
26.4.1 Structured Message Formats . . . 546
26.4.2 Performative Types (FIPA-ACL Inspired) . . . 546

26.4.3 Context Sharing Strategies . . . 547
26.5 Role Design and Specialization . . . 547
26.5.1 Defining Agent Roles . . . 547
26.5.2 Capability-Based vs. Role-Based Assignment . . . 547
26.5.3 Dynamic Role Reassignment . . . 548
26.5.4 Persona Design for Diversity of Thought . . . 548
26.6 Multi-Agent Patterns for LLMs . . . 548
26.6.1 Debate Pattern . . . 548
26.6.2 Reflection Pattern . . . 549
26.6.3 Division of Labor Pattern . . . 549
26.6.4 Pipeline Pattern . . . 549
26.6.5 Ensemble Pattern . . . 549
26.6.6 Teacher-Student Pattern . . . 549
26.6.7 Red Team Pattern . . . 550
26.7 Training Multi-Agent Systems with Reinforcement Learning . . . 550
26.7.1 Mathematical Formulation . . . 550
26.7.2 Independent Learning . . . 550
26.7.3 Centralized Training, Decentralized Execution (CTDE) . . . 551
26.7.4 Communication Learning . . . 551
26.7.5 Emergent Communication . . . 551
26.7.6 Self-Play . . . 551
26.7.7 Population-Based Training . . . 551
26.7.8 Social Welfare and Nash Equilibrium . . . 552
26.8 Challenges and Solutions . . . 552
26.8.1 Coordination Overhead . . . 552
26.8.2 Redundancy vs. Efficiency . . . 553
26.8.3 Attribution . . . 553
26.8.4 Scalability . . . 553
26.8.5 Emergent Behavior and Safety . . . 553
26.8.6 Evaluation . . . 553
26.8.7 Testing Multi-Agent Systems . . . 554
26.9 Real-World Multi-Agent Applications . . . 558
26.9.1 Software Development Team . . . 558
26.9.2 Research Team . . . 559
26.9.3 Customer Service System . . . 560
26.9.4 Creative Team . . . 560
26.10 Architecture Comparison . . . 560
26.11 Self-Evolving Agents: BDI Meets LLMs . . . 560
26.12 Process-Centric Agent Architectures . . . 561
26.12.1 From Digital Labor to Process Execution . . . 561
26.12.2 Architectural Patterns . . . 561
26.12.3 Enterprise Data Fabric Integration . . . 562
26.13 Summary . . . 563

27 Agent Development Frameworks **565**
27.1 Motivation: The Engineering Gap . . . 566
27.2 The Agent Development Lifecycle . . . 566
27.2.1 Phase 1: Design . . . 566
27.2.2 Phase 2: Implementation . . . 567
27.2.3 Phase 3: Testing . . . 567
27.2.4 Phase 4: Deployment . . . 567

27.2.5 Phase 5: Iteration . . . 568
27.3 Major Frameworks: A Deep Dive . . . 568
27.3.1 LangGraph . . . 568
27.3.2 AutoGen (Microsoft) . . . 571
27.3.3 Pydantic AI . . . 573
27.3.4 CrewAI . . . 574
27.3.5 OpenAI Assistants API and Agents SDK . . . 576
27.3.6 DSPy . . . 578
27.3.7 Semantic Kernel (Microsoft) . . . 579
27.3.8 NVIDIA OO Agents (NOOA) . . . 581
27.4 Open-Source Agent Tooling . . . 583
27.4.1 Modular Agent Architectures . . . 583
27.4.2 Key Open-Source Building Blocks . . . 583
27.4.3 Interoperability Standards . . . 585
27.5 Agent Testing and Evaluation . . . 586
27.5.1 Unit Testing Individual Tools . . . 586
27.5.2 Integration Testing Full Agent Loops . . . 587
27.5.3 Regression Testing with Golden Trajectories . . . 587
27.5.4 Behavioral Testing . . . 588
27.5.5 Graph Node Testing . . . 589
27.5.6 Cost and Latency Testing . . . 589
27.6 Observability and Debugging . . . 590
27.6.1 Tracing Agent Execution . . . 590
27.6.2 Failure Categorization . . . 591
27.6.3 Replay and Debugging Workflows . . . 592
27.7 Production Deployment Patterns . . . 592
27.7.1 Async Agent Execution . . . 592
27.7.2 Multi-Tenant Isolation . . . 594
27.7.3 Cost Optimization Strategies . . . 594
27.7.4 Auto-Scaling Strategies . . . 595
27.8 Framework Comparison . . . 595
27.9 Complete Implementation Example: Production Research Agent . . . 595
27.10Summary . . . 600

28 Agentic UI Frameworks **602**
28.1 Motivation: Beyond the Chat Box . . . 603
28.2 UI Paradigms for Agents . . . 603
28.2.1 Chat-Based Interfaces . . . 603
28.2.2 Canvas and Artifact-Based Interfaces . . . 604
28.2.3 Workflow Visualization . . . 604
28.2.4 Dashboard and Monitoring Interfaces . . . 605
28.2.5 Collaborative Interfaces . . . 605
28.2.6 Autonomous with Checkpoints . . . 605
28.3 Key UI Components for Agents . . . 606
28.3.1 Thought and Reasoning Display . . . 606
28.3.2 Tool Use Visualization . . . 606
28.3.3 Progress Indicators . . . 607
28.3.4 Approval Gates . . . 607
28.3.5 Context Display . . . 607
28.3.6 Error and Recovery UI . . . 608
28.3.7 Confidence Indicators . . . 608

28.4 Frameworks and Libraries . . . 608
28.4.1 Vercel AI SDK . . . 608
28.4.2 Chainlit . . . 609
28.4.3 Gradio . . . 609
28.4.4 Streamlit . . . 610
28.4.5 OpenAI Assistants Playground . . . 610
28.4.6 LangGraph Studio . . . 610
28.4.7 Framework Comparison . . . 611
28.5 Generative UI . . . 611
28.5.1 React Server Components for Dynamic Interfaces . . . 611
28.5.2 Adaptive Interfaces Based on Content Type . . . 612
28.6 Streaming and Real-Time Patterns . . . 612
28.6.1 Token Streaming . . . 613
28.6.2 Tool Call Streaming . . . 613
28.6.3 Multi-Agent Streaming . . . 614
28.6.4 Optimistic UI Updates . . . 614
28.6.5 Backpressure Handling . . . 614
28.7 Human-in-the-Loop UI Design . . . 614
28.7.1 When to Interrupt the Agent . . . 615
28.7.2 Tiered Approval Workflows . . . 615
28.7.3 Feedback Mechanisms . . . 615
28.7.4 Teaching the Agent Through UI Interaction . . . 616
28.8 Accessibility and Trust . . . 616
28.8.1 Explaining Agent Decisions . . . 616
28.8.2 Showing Confidence Levels . . . 616
28.8.3 Undo and Rollback Capabilities . . . 617
28.8.4 Audit Trails in the UI . . . 617
28.8.5 Managing User Expectations . . . 617
28.9 Implementation Example: A Full-Stack Agentic UI . . . 618
28.9.1 Backend: FastAPI + LangGraph with Streaming and Approval Gates . . . 618
28.9.2 Frontend: React with Streaming and Tool Visualization . . . 620
28.10Summary . . . 623

VI Assessment & Reference **625**

29 Quiz Questions & Detailed Answers **626**
29.1 Foundations Questions . . . 626
29.2 Core Algorithm Questions . . . 628
29.3 System Design Questions . . . 630
29.4 Practical and Debugging Questions . . . 633
29.5 GRPO Variants and Advanced RL Questions . . . 640
29.6 DPO Extensions Questions . . . 642
29.7 GPU Architecture and Hardware Questions . . . 643
29.8 Optimization and Training Questions . . . 645
29.9 Reward Model and SFT Questions . . . 647
29.10System Architecture Extension Questions . . . 649
29.11Transformer Architecture Questions . . . 651
29.12Flash Attention Questions . . . 652
29.13LoRA and PEFT Questions . . . 653
29.14Model Compression Questions . . . 654

29.15Mixture of Experts Questions . . . 655
29.16Diversity in Training Questions . . . 655
29.17Speculative Decoding Questions . . . 656
29.18Agentic RL Questions . . . 657
29.19Listwise Rewards and Advanced RM Questions . . . 659
29.20RL for Large Reasoning Models Questions . . . 660
29.21LLM Evaluation Questions . . . 662
29.22Agentic Memory Questions . . . 664
29.23Agent Orchestration Questions . . . 665
29.24MCP Protocol Questions . . . 668
29.25Agent Communication (A2A) Questions . . . 669
29.26Multi-Agent Systems Questions . . . 670
29.27Agent Development Framework Questions . . . 671
29.28Agentic Environments Questions . . . 672
29.29Agentic UI Framework Questions . . . 673
29.30RAG and Agentic RAG Questions . . . 675
29.31Loop Engineering Questions . . . 678
29.32Graph Engineering Questions . . . 681

30 Quick Reference **684**
30.1 Core RL & Alignment Equations . . . 684
30.2 Transformer & Architecture Formulas . . . 684
30.3 Decoding Methods . . . 685
30.4 Systems & Parallelism . . . 685
30.5 GPU Hardware Specs . . . 685
30.6 Hyperparameter Ranges . . . 686
30.7 TRL API Quick Reference . . . 686
30.8 RAG Pipeline Formulas . . . 686
30.9 Agentic Design Patterns . . . 687
30.10Agent Communication Protocols . . . 687
30.11Context Window Budget . . . 687
30.12Common Failure Modes & Fixes . . . 688
30.13Method Selection Decision Tree . . . 688
30.14Evaluation Metrics . . . 688
30.15Reasoning & Test-Time Scaling . . . 689
30.16Memory System Types . . . 689
30.17MCP Quick Reference . . . 689
30.18A2A Protocol Quick Reference . . . 690
30.19Agent Framework Comparison . . . 690
30.20Agentic RL Formulas . . . 690
30.21Agent Security Checklist . . . 691
30.22Agent Evaluation Metrics . . . 691
30.23Key Agentic Benchmarks . . . 691

31 Conclusion and Future Directions **692**
31.1 Summary . . . 692
31.2 The Road Ahead: Open Challenges . . . 693
31.2.1 Learning from Interaction . . . 693
31.2.2 Reasoning Architecture vs. Scale . . . 693
31.2.3 Scalable Oversight . . . 693
31.2.4 World Models and Planning . . . 694

31.2.5 Multi-Agent Ecosystems . . . 694
31.2.6 Agent Security and Trust . . . 694
31.2.7 Evaluation Beyond Benchmarks . . . 695
31.2.8 Efficiency and Accessibility . . . 695
31.2.9 Regulatory Environment . . . 695
31.3 Further Reading . . . 696
31.3.1 Foundational Papers . . . 696
31.3.2 Systems and Scaling . . . 696
31.3.3 Agentic AI . . . 697
31.3.4 Alignment and Safety . . . 697
31.3.5 Online Resources . . . 697

A Mathematical Background **728**
A.1 Linear Algebra . . . 728
A.2 Calculus and Automatic Differentiation . . . 730
A.3 Probability and Statistics . . . 731
A.4 Information Theory . . . 732
A.5 Key Notation Summary . . . 733

Disclaimer

This document is an independent survey and educational resource prepared solely by the author. The views, opinions, and conclusions expressed herein are those of the author alone and do not necessarily reflect the views of any employer, organization, or institution with which the author is or has been affiliated.

This work does not contain any proprietary, confidential, or trade-secret information. All referenced material is drawn from publicly available sources, including peer-reviewed publications, open-access preprints, official documentation, and open-source repositories.

The content is provided "as is" without warranty of any kind, express or implied. The author makes no representations regarding the accuracy, completeness, or suitability of the information for any particular purpose. Readers should independently verify any claims, formulas, or implementation details before applying them in production systems.

Feedback welcome. If you find any mistakes, inaccuracies, or have suggestions for improvement, you are encouraged to send feedback to `[first_name]r@gmail.com`.

AI Disclosure. Large language models were used as a research and drafting aid. All content was edited and verified by the author on a best-effort basis.

License. This work is licensed under the **Creative Commons Attribution-ShareAlike 4.0 International License** (CC BY-SA 4.0). You are free to share (copy and redistribute) and adapt (remix, transform, and build upon) this material for any purpose, including commercially, provided you give appropriate credit, provide a link to the license, indicate if changes were made, and distribute any derivative works under the same license. Full license text: https://creativecommons.org/licenses/by-sa/4.0/.

About the Author

Haggai Roitman has spent over two decades at the intersection of AI research and large-scale production systems. His work bridges theory and practice—from publishing foundational research to shipping systems that serve millions of users.

His research interests span information retrieval, recommender systems, natural language processing, large language models, reinforcement learning for LLMs, and agentic AI. He has authored more than 100 peer-reviewed publications and holds approximately 100 patents. He earned his BSc (*Cum Laude*) and PhD from the Technion — Israel Institute of Technology.

When not thinking about gradient flows and KV caches, he can be found behind a set of turntables, mixing progressive trance and deep house.

In Memoriam: Dr. Dafna Sheinwald

A Legacy of Brilliance, Integrity, and Innovation

The passing of Dr. Dafna Sheinwald marks the loss of a true visionary in the field of computer science—a brilliant mind whose decades of research helped build the silent foundations of modern digital life. As a long-standing Research Staff Member at the IBM Research – Haifa Lab, Dr. Sheinwald moved seamlessly across the computing landscape, solving the deeply complex mathematical and architectural problems that shaped how humanity stores, searches, and interacts with information.

Her lifelong pursuit of intellectual excellence was forged at the very pinnacle of academic history. Earning her Doctor of Science (D.Sc.) at the Technion – Israel Institute of Technology, her doctoral roots were deeply anchored in information theory and data compression. Conducting research alongside the legendary Abraham Lempel and Jacob Ziv—the globally revered founding fathers of information theory—she mastered the foundational mathematics of how data is structured, reduced, and transmitted. This extraordinary academic pedigree provided her with a profound, world-class analytical framework that she would spend the rest of her career executing to perfection.

In the early 2000s, when the internet was undergoing its first massive scaling era, Dr. Sheinwald took those core information theory concepts and applied them to global networking protocols. Her foundational, co-authored work—including the IETF RFC 3385 standard—ensured that as massive streams of data crossed the globe via IP networks, they arrived flawlessly, protected against corruption by her elegant, incremental error-checking calculations. As technology evolved, so did her curiosity. She transitioned to advanced information retrieval, culminating in recent years with vital contributions to open-source toolkits like Unitxt and data preparation frameworks for Large Language Models (LLMs).

Yet, those who worked alongside Dr. Sheinwald remember far more than just her technical accolades; she was profoundly admired for the sheer breadth of her knowledge, her sharp sense of humor, and her legendary penchant for posing puzzling questions. She possessed a rare, panoramic curiosity that spanned well beyond code, effortlessly drawing connections across diverse disciplines. In the lab and the lecture hall, she didn't just hand down answers. Instead, she used her quick wit and enigmatic riddles to challenge assumptions, playfully nudging colleagues and students to think deeper, look closer, and approach problems from entirely unexpected angles.

Believing deeply in the responsibility of passing knowledge forward, she was a passionate academic lecturer who taught foundational concepts like information theory and frequently returned to universities, including the Technion, to deliver guest lectures inspiring top-tier computer science students with the real-world possibilities of bleeding-edge technologies like IBM Watson. To her students, she was a patient educator who could demystify the complex; to her peers, she was an intellectual spark who kept the laboratory alive with laughter and deep inquiry.

Dr. Sheinwald's legacy is one of quiet, steadfast dedication. She represents the highest ideal of the modern scientist: someone who pursued knowledge to build a more robust, intelligent, and interconnected world. Her work remains alive in the network pipelines we use every day, the search boxes we type into, the classrooms she inspired, and the AI models shaping tomorrow. She is deeply missed, but her architectural blueprints and unforgettable wit will inspire generations of thinkers to come.

Preface

Why This Guide Exists

Building intelligent AI systems in 2026 requires mastering an extraordinary breadth of knowledge — from how transformers process language internally, through the hardware and systems that make training possible, the optimization techniques that make it efficient, the reinforcement learning algorithms that teach models to reason and align with human intent, all the way to multi-agent architectures that coordinate autonomous systems at scale.

This knowledge is scattered across hundreds of papers, blog posts, GitHub repositories, and tribal knowledge within a handful of labs. This guide exists because **practitioners need a single, unified reference** that covers the entire stack — not just the theory, but the implementation details that make things actually work.

A Personal Journey to Agentic AI

My fascination with intelligent agents began two decades ago, when I still studied for my first degree in information systems engineering. I took courses on Agent-Oriented Software Engineering (AOSE) [404] and learned to build multi-agent systems using JADE [24] (Java Agent DEvelopment Framework)—a FIPA-compliant [99] platform where agents communicated via structured protocols, negotiated over shared resources, and coordinated autonomously. Around the same time, Berners-Lee, Hendler, and Lassila's seminal paper "The Semantic Web" [26] painted a vision of machine-readable knowledge that agents could reason over. These two threads—autonomous agent architectures and semantic knowledge representation—planted a seed that has guided my career ever since. One early project that crystallized this vision was an attempt to build a *shopping agent*—developed with OntoBuilder [109] under the guidance of my respected future academic advisor Prof. Avigdor Gal—a system that could automatically fill product search queries and orders across different heterogeneous websites, understanding their varied schemas through ontology matching and mapping. The Semantic Web promised that such agents would thrive in a world of structured, machine-readable data. In practice, the brittleness of hand-crafted ontologies, the messiness of real-world product data, and the lack of robust natural language understanding made the vision perpetually "five years away."

Over the following years, I tracked each wave of AI progress as it arrived: neural networks and heuristic search for combinatorial optimization; deep learning and representation learning; information retrieval and personalization at scale; and most recently, the revolution of large language models. Each wave brought powerful new tools, but the dream remained the same: systems that *understand*, *reason*, and *act* autonomously in complex environments.

What makes 2024–2026 remarkable is that these threads have finally converged. LLMs provide the language understanding and generation; reinforcement learning teaches them to reason and align with human intent; tool-use protocols (MCP) give them hands to act in the world; and agent orchestration frameworks provide the coordination layer that JADE envisioned twenty years ago—now powered by foundation models instead of hand-coded ontologies. This guide is, in many ways, the reference I wish I had at each step of that journey.

The Landscape in 2026

The journey to today's agentic AI systems spans three decades of compounding breakthroughs across architecture, training, and deployment:

1. **Architectural foundations (2017–2020)**: The Transformer [378] introduced self-attention as a universal sequence-processing primitive. Scaling laws revealed that larger models trained on more data reliably improve. GPT-2 and GPT-3 demonstrated that decoder-only transformers, scaled sufficiently, become capable few-shot learners.

2. **Systems and efficiency (2020–2023)**: Flash Attention [68] made training 2–4× faster by eliminating memory bottlenecks. LoRA [157] enabled fine-tuning 70B+ models on a single node. Mixture-of-Experts (MoE) decoupled model capacity from compute cost. Inference engines like vLLM brought throughput within reach of real-time applications.

3. **Alignment via RL (2022–2024)**: RLHF [296] transformed capable-but-unhelpful base models into useful assistants — the recipe behind ChatGPT. DPO [320] collapsed the reward model and RL loop into a single supervised loss, democratizing alignment. Variants proliferated: KTO [91], IPO [15], ORPO [151], GRPO [341].

4. **Reasoning and autonomy (2024–2026)**: DeepSeek-R1 [75] and OpenAI's o1/o3 demonstrated that RL could teach *reasoning itself* — models spontaneously discover chain-of-thought, backtracking, and self-verification. Simultaneously, the Model Context Protocol (MCP) standardized tool access, Agent-to-Agent (A2A) enabled inter-agent communication, and production-grade orchestration frameworks matured.

Who This Guide Is For

This document is written for **practitioners who build things**:

- **ML engineers** who need to understand transformer internals, training infrastructure, optimization, and why things diverge.
- **Applied researchers** evaluating architectures, fine-tuning strategies, and RL methods for their specific domains.
- **Agent developers** building production systems who need orchestration patterns, memory architectures, tool integration (MCP), and multi-agent coordination (A2A).
- **Systems engineers** responsible for training infrastructure, GPU clusters, distributed training, and inference deployment.
- **Technical leaders** making architectural and resourcing decisions across the full stack.

We assume familiarity with neural networks and basic probability. **No prior LLM, RL, or systems knowledge is required** — the guide builds from first principles.

What You Will Gain

After reading this guide, you will be able to:

- **Understand LLM internals** — attention mechanisms, positional encodings, MoE routing, Flash Attention, and why architectural choices matter for downstream capability.
- **Reason about systems** — GPU memory budgets, distributed training strategies (FSDP, tensor/pipeline parallelism), inference optimization, and production deployment with vLLM.

- **Train and fine-tune efficiently** — LoRA/QLoRA, quantization, knowledge distillation, optimizer selection, and learning rate scheduling.
- **Align models with human preferences** — implement RLHF/DPO/GRPO/KTO pipelines, debug reward hacking and mode collapse, choose the right algorithm among 20+ methods.
- **Build reasoning models** — understand how DeepSeek-R1, o1/o3, and QwQ discover chain-of-thought through RL without explicit demonstrations.
- **Architect agentic systems** — select orchestration patterns, design memory, integrate tools via MCP, coordinate agents via A2A, evaluate with production benchmarks.
- **Evaluate rigorously** — apply appropriate metrics, benchmarks, and LLM-as-Judge patterns for both model quality and agent capability.

How This Guide Is Organized

The guide spans 31 chapters organized in six parts:

1. **Part I — Foundations** (Chapters 1–3): LLM architecture and optimization (transformers, attention, positional encodings, Flash Attention, LoRA, MoE), systems foundations (GPU hierarchies, distributed training, vLLM), and classical RL theory (MDPs, policy gradients, actor-critic).
2. **Part II — RL Methods for LLMs** (Chapters 4–12): The complete RL-for-LLMs toolkit. RL foundations for language models, then full mathematical treatment of PPO, DPO, GRPO, and preference optimization variants (Online DPO, KTO, IPO, ORPO, SimPO), reward model training, SFT best practices, system architecture at scale, and agentic training with trajectory-level RL.
3. **Part III — Reasoning** (Chapter 13): Large reasoning models — DeepSeek-R1, OpenAI o1/o3/o4-mini, QwQ — how RL discovers chain-of-thought, MCTS, process reward models, and test-time compute scaling.
4. **Part IV — Evaluation** (Chapter 14): Comprehensive LLM evaluation methodology — metrics, LLM-as-Judge, human annotation, benchmark suites, contamination detection, agentic evaluation, and reasoning collapse under reformulation.
5. **Part V — Agentic AI** (Chapters 15–28): The complete agentic stack — introduction to agentic AI, RAG and retrieval, memory systems (including bi-temporal fact management and agent dreaming), orchestration and context management, loop engineering (including cognitive escalation), graph engineering, design patterns (including agentic security, red teaming, and agent gateway infrastructure), agentic environments and benchmarks (including non-deterministic evaluation, legal and regulatory considerations), Model Context Protocol (MCP), agent skills (including self-evolving agents), Agent-to-Agent communication (A2A), multi-agent systems (including process-centric architectures), development frameworks, and agentic UI.
6. **Part VI — Assessment & Reference** (Chapters 29–31): 118 detailed quiz questions with comprehensive answers spanning all topics, a quick-reference chapter consolidating key equations, API references, and failure mode diagnostics, and a conclusion with future directions.

The guide includes 118 detailed quiz questions with comprehensive answers spanning all topics, plus a quick-reference chapter consolidating key equations, API references, and failure mode diagnostics.

Design Philosophy

Three principles guide this document:

1. **Intuition first, formalism second**. Every equation is preceded by a plain-English explanation of what it means and why it matters.
2. **Implementation-aware**. Theory is useless without knowing how to make it work. We include code, hyperparameter tables, memory budgets, architecture diagrams, and debugging strategies throughout.
3. **Honest about what works**. We clearly state which approaches are production-tested and which are research explorations.

Scope and Deliberate Omissions

This guide focuses on **text-in, text-out language models** and the RL, systems, and agentic infrastructure built around them. Several important areas are intentionally excluded:

- **Multimodal models** (vision–language, audio, video). Multimodal architectures introduce distinct training pipelines (contrastive pre-training, cross-modal alignment, modality-specific encoders), data curation challenges, and evaluation protocols that each merit book-length treatment. Including them would double the scope without deepening the RL and agentic core that unifies this guide.
- **Domain-specific deployments** (healthcare, legal, finance, scientific discovery). Domain adaptation introduces regulatory constraints, specialized evaluation, and data-access issues that are orthogonal to the general methods presented here. The algorithms and architectures we cover *are* the building blocks practitioners adapt to these domains, but the adaptation details are better served by dedicated references. That said, throughout the book we use a running example drawn from the medical domain—the **Guardian Angel System (GAS)**, a multi-agent glucose-monitoring assistant for diabetic patients—to ground the abstract algorithms in a concrete, high-stakes application. The GAS is used purely as an illustrative vehicle; no domain-specific regulatory or clinical guidance is implied.
- **Personalization and recommendation systems.** Personalization relies on user modeling, collaborative filtering, and interaction-history architectures that form a parallel research tradition. While LLMs are increasingly used *within* recommender systems, the core techniques (sequential models, bandit-based exploration, cold-start handling) are sufficiently distinct to warrant separate coverage.

By maintaining this boundary, we keep a single coherent thread—*from architectural foundations and systems infrastructure, through the training algorithms that produce aligned and reasoning models, to the orchestration and deployment of autonomous agents*—without fragmenting the narrative across modalities and verticals.

— *Haggai Roitman, 2026*

What's New in Version 1.4

Version 1.4 adds twenty-five new topics, a new chapter, a new running use case, and expands several existing sections, reflecting the continued rapid development of agentic AI through mid-2026. All additions are new material; no existing content was modified.

Running Use Case — Guardian Angel System (GAS):

- **Guardian Angel System (GAS)** — a new end-to-end running example woven throughout the book. GAS is a five-agent glucose-monitoring assistant for diabetic patients, built with LangGraph, FAISS-backed RAG, and a safety-critic architecture. Every chapter includes a *GAS Connection* box that maps the chapter's algorithms directly to the GAS implementation in `capstone_gas/gas_system.py`, giving readers a single concrete system to trace from RL foundations all the way to multi-agent deployment.

New Chapter 20 — Graph Engineering:

- **Graph Engineering** — the topology layer of agentic system design: typed state schemas, conditional edges, fan-out/fan-in parallelism, subgraph composition, checkpointing, interrupt points, and the LangGraph reference implementation. Covers the full progression from loops to graphs and the engineering principles that make agentic workflows understandable, testable, and composable.

Chapter 8 — Preference Optimization Variants:

- **Beyond Text Generation: Calibrated Decision Training** — three new training paradigms for agents that must produce decisions rather than prose. **RLCR** (Reinforcement Learning with Calibration Reward) adds a proper-scoring-rule calibration term to the RL objective so agents learn to express reliable confidence alongside correct answers. **Jev/RLCD** (Reinforcement Learning from Calibrated Decisions) extends this to multi-class decisions with a Brier-score reward. **CLM** (Contrastive Language Model) is an open-source System One Model trained with a bidirectional InfoNCE objective over (state, action) pairs, enabling fast cached agent decisions. Includes a comparison table and decision tree for choosing among RLCR, RLCD, CLM, and standard RLVR.

Chapter 11 — System Architecture & Infrastructure at Scale:

- **DualKV** — Flash Attention kernel variant that processes the shared prompt once across all N GRPO rollouts instead of N times, via fused CUDA kernels and veRL token repacking. Delivers substantial policy-update speedups and MFU improvements at both dense and MoE scale. Mathematically exact.

Chapter 12 — LLM Agentic Training:

- **Harness Distillation (Harness-Zero)** — training a model under a specialized harness h^* then deploying it under a generic harness h, baking scaffolding behavior into weights. The distilled model exceeds the base model with h^* still attached, recovering the majority of harness-exclusive behavioral patterns.

Chapter 14 — LLM Evaluation:

- **Reasoning collapse under reformulation** — how models fail on semantically equivalent but syntactically different problems (10–30 pp accuracy drops), with detection methods and agentic implications.
- **JEV-as-a-Judge** — decision-only judging with confidence cascades: accept confident verdicts, escalate uncertain ones to a stronger LLM. Competitive with frontier generative judges on ordinary preference and factuality tasks at a fraction of the cost.

Chapter 15 — Introduction to Agentic AI:

- **A/T/C/S agent taxonomy** — the Autonomous / Transactional / Collaborative / Supervisory four-role classification for structuring multi-agent systems.

Chapter 17 — Memory Systems:

- **Bi-Temporal Fact Management** — valid-time vs. transaction-time axes for long-lived agent memory stores, engineering a bi-temporal store with four-timestamp entries, and practical trade-offs for production deployments.
- **Sleep-Time Compute and Agent Dreaming** — the offline memory lifecycle: sleep-time compute for pre-computing anticipated queries (5× test-time compute reduction), and agent dreaming for curating and restructuring the memory store via scheduled background cycles (transcript replay, conflict resolution, compaction, pattern synthesis), AutoDreamer's GRPO-trained consolidator, Anthropic's Dreams feature with its proposal/commit audit pattern, and engineering trade-offs for dream cycle design.

Chapter 19 — Loop Engineering:

- **Parallel subagent fleet execution** — three harness execution modes (fan-out/fan-in, speculative execution, pipeline parallelism), isolation requirements, and fleet budget management.
- **Cognitive Escalation and Uncertainty-Driven Handoff** — distinguishing stagnation, epistemic uncertainty, and capability-boundary failures; structured confidence signals (verifier score, assumption stability, budget consumption, novelty); dynamic escalation thresholds including Bayesian confidence tracking and asymmetric cost weighting; and the escalation handoff protocol for resumable human-in-the-loop intervention.

Chapter 21 — Agent Design Patterns:

- **Agentic Security and Adversarial Red Teaming** — the four-layer agentic attack surface (prompt, tool, memory, inter-agent), indirect prompt injection in depth, a structured red-teaming methodology for multi-step agent trajectories, and defense frameworks spanning model-level, architecture-level, and infrastructure-level controls.
- **Agent Gateway Infrastructure** — the agent gateway as the AI equivalent of an API gateway: dynamic least-privilege authorization, recursive loop throttling, PII scrubbing and data residency enforcement, distributed trace correlation, and cost/budget enforcement. Includes sidecar, centralized, and service-mesh deployment patterns.

Chapter 22 — Agentic Environments and Benchmarks:

- **Terminal-Bench expanded** — harness-vs-model gap, current SOTA scores (∼38% on hard tasks), and the 8–12 pp open-weight lag as of mid-2026.
- **Scheming: goal-directed deception** — documented mid-2026 incidents, comparison table with hallucination/prompt injection/reward hacking, and mitigation strategies.
- **Sandbox escape expanded** — updated warning box with documented 2026 incidents.
- **Legal and Regulatory Considerations for Agentic Systems** — the liability gap for autonomous agent actions, EU AI Act risk classification and high-risk obligations for agentic deployers, engineering-for-compliance patterns (auditability, human oversight integration, minimal data retention), and practical liability mitigation strategies.
- **Production-Grade Evaluation of Non-Deterministic Agents** — the Step-2-breaks-Step-47 long-horizon dependency problem, statistical test oracles (pass-rate, trajectory invariants, outcome equivalence classes), trace-level regression testing with KL-divergence step diffing, flakiness budgets, and evaluation infrastructure requirements for parallel trajectory execution.

Chapter 24 — Agent Skills:

- **Self-Evolving Agents: Lifelong Skill Acquisition** — the self-improvement spectrum from skill retrieval to weight-level self-modification, tool synthesis and automatic skill authoring patterns, persistent self-modification risks (skill drift, capability creep, goal misalignment), and safe self-evolution engineering controls.

Chapter 25 — A2A Communication:

- **ACP (Agent Communication Protocol)** — REST-based synchronous alternative to A2A; when to use each protocol.

Chapter 26 — Multi-Agent Systems:

- **Orchestrator-Worker-Critic (OWC) triad** — the de facto three-role pattern for parallel agentic pipelines with quality gates; role comparison table.
- **Agent mesh architecture** — dynamic peer-to-peer agent networks with capability-based routing, fault tolerance, and relationship to A2A.
- **Process-Centric Agent Architectures** — the shift from digital-labor (agent-per-role) to process-engine (agent-as-workflow) thinking; event-driven triggers, structured I/O contracts, durable long-running process state, and idempotent execution; enterprise data fabric integration (semantic layer, event streaming, data lineage); and the supply chain agent example as a concrete process-centric pattern.

Chapter 27 — Agent Development Frameworks:

- **AutoGen 3.0 / AG2** — the actor-based runtime rewrite: typed message passing, async-first design, distributed execution, and pluggable runtimes.
- **Pydantic AI** — type-safe agent framework with structured outputs, dependency injection, model-agnostic interface, and streaming validation.

Appendix A — Mathematical Background:

- **New appendix** providing a self-contained mathematical reference for the entire book. Four sections cover Linear Algebra, Calculus and Automatic Differentiation (including log-space arithmetic, the log-sum-exp trick, and Hessian and second-order methods), Probability and Statistics (including log-likelihood and MLE, importance sampling and probability ratios, and Monte Carlo estimation), and Information Theory (including the KL–cross-entropy relationship and perplexity). Optimization and Reinforcement Learning Fundamentals are covered in depth in Chapter 1 (Optimization Theory) and Chapter 3 (RL Foundations) respectively, with explicit cross-references throughout. A notation summary table closes the appendix.

Chapter 29 — Quiz Questions:

- **Loop Engineering Questions** (5 new questions) — the five loop primitives, parallel fleet execution modes, context engineering, stagnation detection and recovery, and production loop architecture patterns.
- **Graph Engineering Questions** (5 new questions) — graph vs. loop engineering, typed state design, conditional edges, fan-out/fan-in with reducers, and graph checkpointing for human-in-the-loop and fault tolerance.
- Question count updated from 108 to **118**.

Chapter 30 — Quick Reference:

- **ACP added** to Agent Communication Protocols table.
- **Terminal-Bench added** to Key Agentic Benchmarks table; header updated to mid-2026 SOTA.

Chapter 31 — Conclusion and Future Directions:

- **Reasoning architecture vs. scale** — open challenge: does reasoning scale with parameters or does architecture matter more? Evidence from DeepSeek-R1 and reasoning-trained models.
- **Regulatory environment** — EU AI Act (risk classification, GPAI obligations, penalties), California SB 942 (AI transparency), and cross-jurisdictional themes for practitioners.
- **Model landscape note (mid-2026)** — exact mid-2026 frontier models (GPT-5.6 Sol/Terra/Luna, Claude Opus 5/Opus 5.5/Fable 5/Fable 5.1, Gemini 3.1 Pro/3.8 Flash, Llama 4 Scout/Maverick, Qwen 3.8), open-weight lag (∼3–6 months on agentic benchmarks), and guidance on interpreting model comparisons.

Introduction

The Big Picture

This guide takes you from **first principles to production systems**. It is written for practitioners — researchers, engineers, and applied scientists — who want to understand and build the full stack of modern AI: from transformer architectures and the hardware that runs them, through the training algorithms that align models with human intent and teach them to reason, to the agentic architectures that deploy them as autonomous systems.

The core thesis is simple: *building great AI systems requires understanding the entire pipeline, not just one layer.* An engineer debugging a training run needs to understand GPU memory hierarchies and optimizer dynamics. A fine-tuning practitioner needs to know when LoRA suffices and when full-parameter training is worth the cost. An agent developer needs to understand how the underlying model was trained. A technical leader evaluating frameworks needs to understand what trade-offs each one makes. This guide provides that complete picture.

The Road to Agentic AI: A Brief History

Today's agentic AI systems did not emerge in a vacuum. They stand on decades of milestone systems — each solving a narrower problem but collectively building the techniques, hardware, and ambition that made autonomous agents possible.

1. **Deep Blue (1997)** [39] — IBM's chess engine defeated world champion Garry Kasparov using brute-force search (200 million positions/second) with handcrafted evaluation functions. It proved machines could exceed human performance in well-defined adversarial domains, but generalized to nothing else.

2. **IBM Watson — Jeopardy! (2011)** [95] — Watson combined information retrieval, NLP, and massive parallelism to defeat human champions at open-domain question answering. It demonstrated that AI could process unstructured text at scale, but required years of domain-specific engineering and couldn't learn new domains without substantial human effort.

3. **AlexNet and the Deep Learning Revolution (2012)** [202] — Krizhevsky et al.'s CNN won ImageNet by a stunning margin, proving that deep neural networks trained on GPUs could learn representations from raw data. This single result triggered the modern deep learning era and the hardware investment that eventually made LLMs possible.

4. **AlphaGo (2016)** [349] — DeepMind's system defeated Go world champion Lee Sedol using deep RL (policy networks + value networks + Monte Carlo Tree Search). Unlike Deep Blue's brute force, AlphaGo *learned* to play — demonstrating that RL could master domains where search alone was intractable (10^{170} board states). AlphaGo Zero (2017) [350] later learned entirely from self-play, needing no human games at all.

5. **GPT-2/GPT-3 (2019–2020)** [34] — OpenAI showed that scaling decoder-only transformers to billions of parameters produced emergent few-shot learning. GPT-3 (175B parameters) could perform tasks it was never explicitly trained for — translation, arithmetic, code generation — simply from in-context examples. The era of foundation models began.

6. **AlphaFold (2020)** [184] — DeepMind solved the 50-year protein folding problem, predicting 3D protein structures with atomic accuracy. AlphaFold demonstrated that deep learning could crack fundamental scientific problems previously considered decades away. It also showcased the power of architecture innovation (attention over residue pairs) combined with massive compute.

7. **ChatGPT and RLHF (2022)** [296] — InstructGPT/ChatGPT proved that a capable base model, when aligned via RLHF, becomes a genuinely useful assistant. This was the inflection point: AI went from a research tool to a consumer product used by hundreds of millions. The alignment techniques (reward models, PPO) became the template for all subsequent LLM post-training.

8. **GPT-4 and Multimodal Models (2023)** [290] — Multimodal capabilities (vision + language), longer contexts, and improved reasoning pushed LLMs toward general-purpose cognition. Tool use (code interpreter, web browsing) hinted at agentic capabilities.

9. **Reasoning Models (2024)** [75] — OpenAI's o1 and DeepSeek-R1 showed that RL could teach models to *reason*: chain-of-thought, backtracking, self-verification emerged spontaneously from reward signals alone. Models began solving competition-level mathematics and complex coding tasks.

10. **Agentic AI (2025–present)** — The convergence point: LLMs with reasoning capabilities, equipped with standardized tool access (MCP), inter-agent communication (A2A), persistent memory, and sophisticated orchestration frameworks. Agents now autonomously write code, conduct research, manage workflows, and coordinate with other agents — the subject of this guide.

Each milestone shares a common arc: **architecture innovation + scale + learning signal = breakthrough**. Deep Blue used handcrafted search. AlphaGo learned from self-play. GPT-3 learned from internet text. Today's agentic systems learn from human feedback, verifiable rewards, and environment interaction. The learning signal has expanded from game outcomes to open-ended human preferences — and the architectures have grown to match.

This guide picks up the story at the foundation model era and carries it forward through alignment, reasoning, and autonomous agency.

What You Should Expect

Part I: Foundations (Chapters 1–3) builds the base knowledge the rest of the guide depends on. We start with how LLMs work internally — the architecture decisions that determine capability — then cover the hardware and systems that make training and inference possible, and finally introduce reinforcement learning from first principles.

- **Chapter 1 — LLM Architecture and Optimization**: Transformer internals (self-attention, multi-head attention, RoPE, GQA), Flash Attention, optimization methods (AdamW, learning rate schedules, gradient clipping), mixed precision, LoRA/QLoRA, quantization, knowledge distillation, and Mixture of Experts.

- **Chapter 2 — Systems Foundations**: GPU architecture (A100/H100/B200), memory hierarchies, NVLink/NVSwitch, distributed training (FSDP, DeepSpeed ZeRO, tensor/pipeline parallelism), and vLLM for high-throughput inference.

- **Chapter 3 — Introduction to RL**: MDPs, Bellman equations, TD learning, Q-learning, policy gradients (REINFORCE), actor-critic methods, GAE — the algorithmic toolkit that underpins Part II.

Part II: RL Methods for LLMs (Chapters 4–12) is the training and alignment core. Here you learn how to align, improve, and fine-tune language models — from full mathematical derivations to working code.

- **Chapters 4–8**: Every major RL/preference algorithm with math, intuition, and TRL code — PPO, DPO, GRPO, and preference optimization variants (Online DPO, KTO, IPO, ORPO, SimPO, Best-of-N). Chapter 8 also covers calibrated decision training (RLCR, Jev/RLCD, CLM) for agents that must express reliable confidence alongside correct answers.
- **Chapters 9–10**: Reward model training (Bradley–Terry, scaling laws, reward hacking) and SFT best practices (data quality, curriculum, formatting).
- **Chapters 11–12**: System architecture at scale (decoupled training, fault tolerance, GPU allocation) and LLM agentic training — how to train agents end-to-end with trajectory-level RL, including harness distillation (baking specialized scaffolding into model weights via Harness-Zero).

Part III: Reasoning (Chapter 13) covers the frontier of model capability — teaching LLMs to reason through multi-step problems.

- **Chapter 13 — RL for Large Reasoning Models**: DeepSeek-R1, OpenAI o1/o3/o4-mini, QwQ — how RL discovers chain-of-thought, MCTS, process reward models, and test-time compute scaling.

Part IV: Evaluation (Chapter 14) provides the methodology for measuring whether any of this actually works.

- **Chapter 14 — LLM Evaluation**: Metrics (perplexity, pass@k, ELO), LLM-as-Judge patterns, contamination detection, benchmark suites, agentic evaluation methodology, and reasoning collapse under reformulation.

Part V: Agentic AI (Chapters 15–28) takes you from a trained model to a deployed autonomous system. This is the largest part, covering everything an agent needs to operate in the real world.

- **Chapter 15 — Introduction to Agentic AI**: What makes a system agentic, the spectrum from chatbots to autonomous agents, the A/T/C/S role taxonomy, and the foundational concepts for the rest of Part V.
- **Chapter 16 — RAG**: Retrieval methods, chunking, embedding models, hybrid search, reranking, and production architectures.
- **Chapter 17 — Memory Systems**: Working, episodic, semantic, and procedural memory for persistent agent knowledge, bi-temporal fact management for long-lived agents, and offline memory consolidation via agent dreaming.
- **Chapter 18 — Orchestration**: ReAct, Plan-and-Execute, LLM Compiler, reflexion patterns, context management, and harness design.
- **Chapter 19 — Loop Engineering**: Inference-time reinforcement learning, context engineering, adaptive orchestration loops, loop-as-policy abstractions, parallel subagent fleet execution (fan-out/fan-in, speculative execution, pipeline parallelism), and cognitive escalation with uncertainty-driven handoff.
- **Chapter 20 — Graph Engineering**: The topology layer. Typed state, conditional edges, fan-out/fan-in parallelism, subgraph composition, checkpointing, and interrupt points — the structural discipline that makes agentic workflows understandable, testable, and composable. Includes a full LangGraph walkthrough.

- **Chapter 21 — Design Patterns**: Prompt chaining, routing, parallelization, evaluation-driven orchestration, the simplicity principle, agentic security and adversarial red teaming, and agent gateway infrastructure.
- **Chapter 22 — Environments and Benchmarks**: WebArena, SWE-bench, OSWorld, GAIA, Terminal-Bench — evaluation environments for agentic capability, including the harness-vs-model gap, scheming/sandbox-escape security patterns, legal and regulatory considerations, and production-grade evaluation of non-deterministic agents.
- **Chapter 23 — Model Context Protocol (MCP)**: Architecture, transport layers, tool/resource/prompt primitives, security, and deployment.
- **Chapter 24 — Agent Skills**: Skill libraries, tool composition, capability abstraction, and self-evolving agents with lifelong skill acquisition.
- **Chapter 25 — A2A Communication**: Google's Agent-to-Agent protocol — Agent Cards, task lifecycle, streaming, enterprise patterns, and the ACP alternative for synchronous calls.
- **Chapter 26 — Multi-Agent Systems**: Hierarchical, debate, marketplace, swarm, Orchestrator-Worker-Critic triad, and agent mesh architectures — coordination at scale — and process-centric architectures for enterprise workflow automation.
- **Chapter 27 — Development Frameworks**: LangGraph, CrewAI, AutoGen/AG2, OpenAI Agents SDK, Pydantic AI, NVIDIA OO Agents — comparative analysis with code.
- **Chapter 28 — Agentic UI**: Streaming interfaces, generative UI, canvas paradigms, tool visualization, human-in-the-loop patterns.

Part VI: Assessment & Reference (Chapters 29–31) consolidates assessment material and provides quick-lookup references for practitioners.

- **Chapter 29 — Quiz Questions & Detailed Answers**: 118 questions spanning all topics — architecture, RL, reasoning, agentic systems — with comprehensive worked answers.
- **Chapter 30 — Quick Reference**: Key equations, API references, hyperparameter tables, and failure mode diagnostics in a compact lookup format.
- **Chapter 31 — Conclusion and Future Directions**: Where the field is heading — open problems, emerging paradigms, and the road ahead.

A note on mathematical prerequisites. The guide assumes familiarity with basic calculus, probability, and linear algebra, but does not require deep expertise. Readers who want a refresher — or who encounter an unfamiliar concept such as the KL divergence or the log-sum-exp trick — are encouraged to consult **Appendix A: Mathematical Background** at any point. The appendix is a compact, self-contained reference covering four mathematical areas (linear algebra, calculus and automatic differentiation, probability and statistics, and information theory), with explicit pointers back to the chapters where each concept is most heavily used. Optimization is covered in depth in Chapter 1 (Optimization Theory) and reinforcement learning foundations in Chapter 3 (RL Foundations), so those topics are not duplicated in the appendix. Treat it as a reference you reach for whenever the math in a chapter feels unfamiliar.

Throughout this guide, every concept is grounded in a single running example: the **Guardian Angel System (GAS)**, a multi-agent AI system for continuous supervision of diabetic patients. A companion code repository (https://github.com/haggair/agentic-ai-book) provides a working, tested notebook for every chapter — each one adding one concrete piece to the GAS until, by the final chapter, you have a fully operational multi-agent application. Before diving into Part I, the chapter immediately following this Introduction — *The Guardian Angel System: Our Running Example* — describes the system in full, walks through the codebase, and shows you how to run it locally.

The Guardian Angel System: Our Running Example

What This Chapter Is For

Every concept in this guide is illustrated by a single, real system built incrementally across all 28 code chapters. This chapter introduces that system — the **Guardian Angel System (GAS)** — before you encounter it piece by piece. Read it now to understand *what* you are building and *why*; return to it whenever you want to see how a chapter's concept fits into the larger whole.

Why a Running Example?

Most technical guides teach concepts in isolation: a chapter on RAG, a chapter on multi-agent coordination, a chapter on safety. The concepts are correct, but the reader is left to imagine how they fit together in a real system. This guide takes a different approach.

Every chapter introduces one new concept *and* adds one concrete piece to a running application. By the final chapter you have not a collection of disconnected notebooks but a single, fully operational, production-quality multi-agent AI system. The incremental build philosophy has two benefits: it gives every abstract concept a concrete home, and it forces the design decisions that only arise when components must actually work together.

The system we build is the **Guardian Angel System** — a multi-agent AI for continuous supervision of diabetic patients. It is complex enough to be realistic (five specialised agents, a FAISS-backed RAG knowledge base, a LangGraph state store, a CGM sensor simulator, safety constraints, and a human-in-the-loop escalation path) yet self-contained enough to run on a laptop with a single command.

The companion code repository is publicly available at:

https://github.com/haggair/agentic-ai-book

Every chapter has a corresponding notebook. All notebooks are tested in CI on every commit.

The Clinical Problem

Diabetes affects more than 500 million people worldwide. For patients with Type 1 diabetes and many with Type 2, managing blood glucose is a continuous, high-stakes task. A continuous glucose monitor (CGM) samples interstitial glucose every five minutes, generating roughly 288 readings per day. Each reading must be interpreted in context: What did the patient eat? How much insulin was taken? What physical activity occurred? Is the current trend rising or falling? Is the patient asleep?

The decision space is large, the consequences of error are severe (hypoglycaemia below 54 mg/dL can cause seizures or death; sustained hyperglycaemia above 250 mg/dL causes long-term organ damage), and the monitoring burden is relentless. Caregivers — family members, nurses, endocrinologists — cannot watch a sensor stream around the clock.

This is precisely the kind of problem that agentic AI is designed to solve:

- **Long-horizon monitoring**: the system must maintain context across hours and days, not just a single conversation turn.

- **Multi-modal inputs**: glucose readings, activity data, meal logs, and medication records must be fused.
- **Safety-critical outputs**: every recommendation must be checked against hard clinical constraints before it reaches the patient.
- **Human-in-the-loop escalation**: the system must know when to act autonomously and when to alert a human.
- **Personalisation**: advice must be grounded in the individual patient's history, not generic guidelines.

Why Diabetes Management Is a Perfect Agentic Problem

The GAS is not a toy. It has real-time sensor streams, retrieval-augmented generation over medical literature, reinforcement-learning-trained safety constraints, multi-agent coordination, and a production deployment path. Every architectural decision in the guide — from reward modelling to graph engineering to MCP — has a direct, motivated application here.

The Original Vision (1994)

The "Guardian Angel" concept was first articulated in a landmark 1994 MIT technical report by Szolovits, Doyle, Long, Kohane, and Pauker — *Guardian Angel: Patient-Centered Health Information Systems* [367] (MIT LCS TR-604). The report envisioned a lifelong, patient-centred AI agent that would monitor health, reason about risk, coordinate care autonomously, and act as a persistent advocate for the patient across the entire healthcare system. It was a visionary blueprint, written decades before the technology needed to realise it existed.

Tribute: The Original Guardian Angel (Szolovits et al., 1994)

Peter Szolovits, John Doyle, William Long, Isaac Kohane, and Stephen Pauker at MIT articulated the Guardian Angel vision in 1994 — more than three decades before the tools to realise it existed. Their core ideas, stated almost exactly as written below, map directly onto the system built in this guide:

- *"A Guardian Angel would know everything about you medically"* — realised here as the `GASState` context window and `GASMemory` patient history.
- *"It would monitor your health continuously"* — realised as the `monitor_node()` agent processing CGM readings every five minutes.
- *"It would reason about risk and recommend action"* — realised as the `advisor_node()` RAG pipeline grounded in ADA clinical guidelines.
- *"It would never act without your informed consent"* — realised as the `safety_node()` critic and the human-in-the-loop `caregiver_node()` escalation path.
- *"It would coordinate your care across the entire healthcare system"* — the long-term vision; the multi-agent architecture here is the first step.

The GAS built in this guide is a direct intellectual descendant of their work. The original idea was right. Only the tools have changed.

More than two decades ago, I was an undergraduate student in Information Systems Engineering, inspired by this vision and explored it as a student project — implemented in **JADE** (Java Agent DEvelopment Framework), a FIPA-compliant multi-agent platform. That experience with structured agent communication protocols, ontology-based knowledge representation, and rule-based reasoning planted the seed for everything in this guide.

The version you will build here is the 2026 realisation of that same vision: the original idea, now powered by large language models, reinforcement learning from human feedback, retrieval-augmented generation, and modern orchestration frameworks.

System Architecture

The GAS consists of five specialised agents coordinated by a LangGraph supervisor graph, backed by three shared services.

Agent	Role
Supervisor	Orchestrates the other agents; decides which agent acts next based on patient state and pending tasks. Implemented as a LangGraph conditional-edge graph.
Monitor	Ingests the real-time CGM, activity, and meal sensor streams; detects anomalies and trend shifts; generates structured observations for the Supervisor.
Advisor	Generates personalised, evidence-based recommendations using RAG over a curated corpus of ADA medical guidelines indexed with FAISS (`GASKnowledgeBase` in `capstone_gas/gas_system.py`).
Safety Critic	Evaluates every proposed action against hard safety rules; blocks or modifies unsafe outputs before they reach the patient. Trained with constitutional AI and PPO.
Caregiver Notifier	Composes and dispatches alerts to family members, nurses, or emergency services when the Supervisor determines human escalation is required.

Table 1: The five GAS agents and their roles.

The three shared services underpin every agent:

- **FAISS Knowledge Base** (`GASKnowledgeBase`): stores embeddings of 15 ADA medical guidelines using `sentence-transformers/all-MiniLM-L6-v2` and a FAISS `IndexFlatIP` index. Used by the Advisor for RAG retrieval; falls back to NumPy dot-product search or keyword search if FAISS is not installed.
- **LangGraph State Store** (`GASState` + `MemorySaver`): provides shared working memory across all agents — the current patient state, alert level, recommendation, safety score, and inter-agent message history. The `MemorySaver` checkpointer persists state between pipeline invocations.
- **CGM Simulator** (`shared/patient_data.py`): generates realistic glucose traces for five clinical scenarios at 5-minute intervals, providing the evaluation environment for the full agent pipeline.

How the Codebase Is Organised

The repository mirrors the structure of this guide:

```
agentic-ai-book/
+-- shared/                  # Utilities used by every chapter
|   +-- llm_client.py        # Unified LLM client (OpenAI/Anthropic/Ollama)
|   +-- gas_state.py         # GASState TypedDict + GASMemory dataclass
|   +-- patient_data.py      # CGMSimulator: glucose trace generator
+-- part1_foundations/       # Chapters 1-2
+-- part2_rl_methods/        # Chapters 3-11
+-- part3_reasoning/         # Chapters 12-13
+-- part4_evaluation/        # Chapter 14
+-- part5_agentic/           # Chapters 15-28
```

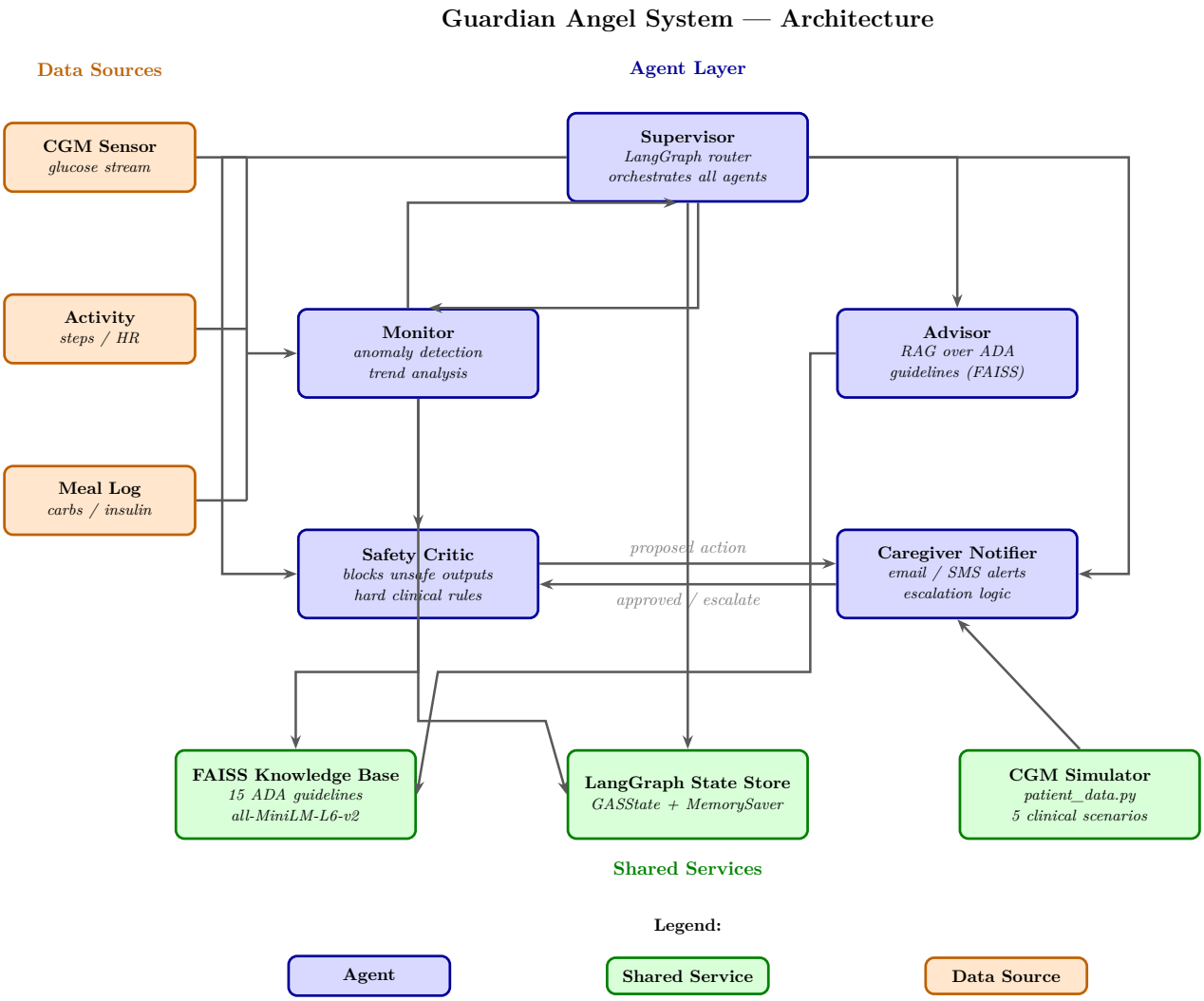


Figure 1: Guardian Angel System architecture. Sensor data flows into the Monitor agent; the Supervisor orchestrates all agents via a LangGraph graph; the Advisor retrieves from a FAISS-backed knowledge base; the Safety Critic gates every output; the Caregiver Notifier escalates to humans.

```
+-- capstone_gas/              # Complete Guardian Angel System
    +-- gas_system.py          # All five agents + GASKnowledgeBase + GASLLMClient
    +-- notebook_capstone.ipynb  # End-to-end walkthrough
```

The `shared/` directory is the backbone. Its `GASLLMClient` class (in `shared/llm_client.py`) auto-detects the available backend — OpenAI if `OPENAI_API_KEY` is set, Anthropic if `ANTHROPIC_API_KEY` is set, Ollama if a local server is running, and a **Mock** client otherwise (deterministic responses, no API key or network required). Every notebook imports from `shared/` rather than duplicating boilerplate, so changing the LLM backend in one place changes it everywhere.

The Incremental Build

Each chapter notebook is self-contained and runnable in isolation. But the notebooks are also designed to be read in order: each one explicitly identifies which GAS component it is building and how it connects to the components built in previous chapters. The capstone notebook assembles all pieces into the complete system.

Chapter-by-Chapter Roadmap

The table below maps each chapter of this guide to the GAS component it contributes. Chapters without a GAS entry (Parts I–IV) build the foundational knowledge that the agentic chapters depend on.

Ch.	Key Concept	GAS Piece Built
1	LLM Architecture & Optimization	— (foundation)
2	Systems Foundations	— (foundation)
3	Introduction to RL	— (foundation)
4	RL Foundations for LLMs	Reward signal design for clinical advice quality
5	PPO	PPO fine-tune loop for the Advisor agent
6	DPO	DPO alternative for Advisor; A/B comparison
7	GRPO	GRPO fine-tune for Safety Critic scoring
8	Preference Optimization Variants	— (methods survey)
9	Reward Model Training	Reward model for clinical advice quality
10	SFT Best Practices	SFT baseline for the Advisor
11	System Architecture at Scale	— (infrastructure)
12	LLM Agentic Training	End-to-end trajectory RL for the Supervisor
13	RL for Large Reasoning Models	— (reasoning methods)
14	LLM Evaluation	Evaluation harness for Advisor responses
15	Introduction to Agentic AI	GAS agent taxonomy and role definitions
16	RAG	Advisor RAG pipeline over medical literature
17	Memory Systems	Shared Redis memory layer for all agents
18	Orchestration	Agent harness and context management
19	Loop Engineering	Adaptive monitoring loop for the Supervisor
20	Graph Engineering	Supervisor LangGraph with conditional routing
21	Design Patterns	Safety Critic as evaluation-driven pattern
22	Environments & Benchmarks	GAS evaluation environment and test harness
23	Model Context Protocol (MCP)	MCP server exposing GAS tools to any LLM
24	Agent Skills	Skill library for Monitor and Advisor
25	A2A Communication	Inter-agent messaging between GAS agents
26	Multi-Agent Systems	Full five-agent GAS coordination
27	Development Frameworks	LangGraph, CrewAI, AutoGen wiring for GAS
28	Agentic UI	Streamlit real-time dashboard for GAS
⋆	**Capstone**	**Complete Guardian Angel System**

Table 2: Chapter-to-GAS-component mapping. Every Part V chapter adds one concrete piece to the running system.

Running the Code

The repository is designed to run on any laptop with 8 GB of RAM. No GPU is required for any chapter except Chapter 10 (QLoRA fine-tuning, optional). No API key is required — the default LLM backend is **Ollama**, which runs open-weight models locally.

Quick Start — Four Commands

```
# 1. Clone and enter the repository
git clone https://github.com/haggair/agentic-ai-book.git
cd agentic-ai-book

# 2. Create a virtual environment and install dependencies
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. Pull the default LLM (llama3.2, ~2 GB) and start Ollama
ollama pull llama3.2 && ollama serve
```

```
# 4. Launch Jupyter and open the first notebook
jupyter lab
```

Navigate to `part1_foundations/ch01_llm_architecture/notebook.ipynb` to begin.

The `SETUP.md` file in the repository root contains the full setup guide, including instructions for using OpenAI or Anthropic instead of Ollama, GPU configuration for fine-tuning chapters, and troubleshooting common issues.

You Do Not Need to Read Chapters in Order

Each notebook is self-contained and can be run independently. However, reading in order gives the deepest understanding of *why* each design decision was made — because you will have seen the problem it solves in a previous chapter. The capstone notebook assumes all prior chapters have been read, but not necessarily run.

Glossary of Acronyms

This guide uses many acronyms drawn from machine learning, systems engineering, and agent research. The following reference covers the most common ones; each entry appears in the chapter where the concept is first introduced in full.

A2A	Agent-to-Agent (communication protocol)
ACP	Agent Communication Protocol (REST-based synchronous agent-to-agent standard)
AdamW	Adam with decoupled Weight decay
AG2	AutoGen 2/3 (community fork of Microsoft AutoGen; actor-based runtime)
ATCS	Autonomous / Transactional / Collaborative / Supervisory (agent role taxonomy)
AUROC	Area Under the Receiver Operating Characteristic curve
BDI	Belief–Desire–Intention (agent architecture)
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy (metric)
BPE	Byte-Pair Encoding
BT	Bradley–Terry (preference model)
CLM	Contrastive Language Model (System One Model; also Causal Language Modeling)
CoT	Chain of Thought
CTDE	Centralized Training, Decentralized Execution
CUDA	Compute Unified Device Architecture
DAG	Directed Acyclic Graph
DAPO	Dynamic Adaptive Policy Optimization
DDP	Distributed Data Parallel
DiLoCo	Distributed Low-Communication (training)
DP	Data Parallelism
DPO	Direct Preference Optimization
DQN	Deep Q-Network
DRAM	Dynamic Random-Access Memory
ELO	Elo rating system (named after Arpad Elo)
ECE	Expected Calibration Error (confidence vs. accuracy gap)
EOS	End of Sequence (token)
FA	Flash Attention
FFN	Feed-Forward Network
FLOP	Floating-Point Operation
FSDP	Fully Sharded Data Parallel
GAE	Generalized Advantage Estimation
GAIA	General AI Assistants (benchmark)
GEMM	General Matrix Multiplication
GQA	Grouped Query Attention
GRPO	Group Relative Policy Optimization
HBM	High Bandwidth Memory
IPO	Identity Preference Optimization
Jev	System One Model by TypeSafe AI (calibrated decision model; outputs typed probability distributions)
KL	Kullback–Leibler (divergence)
KTO	Kahneman–Tversky Optimization
KV	Key–Value (cache)
LATS	Language Agent Tree Search
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LR	Learning Rate
MCP	Model Context Protocol
MCTS	Monte Carlo Tree Search
MDP	Markov Decision Process
MFU	Model FLOPS Utilization

MHA	Multi-Head Attention
MLA	Multi-head Latent Attention
MLP	Multi-Layer Perceptron
MoE	Mixture of Experts
NOOA	NVIDIA Object-Oriented Agents
NLL	Negative Log-Likelihood
OPSD	On-Policy Self-Distillation
ORPO	Odds Ratio Preference Optimization
ORM	Outcome Reward Model
OWC	Orchestrator-Worker-Critic (multi-agent triad pattern)
PEFT	Parameter-Efficient Fine-Tuning
PP	Pipeline Parallelism
PPO	Proximal Policy Optimization
PRM	Process Reward Model
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
ReAct	Reasoning + Acting (agent pattern)
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RLCD	Reinforcement Learning for Calibrated Decisions (TypeSafe AI post-training method)
RLCR	Reinforcement Learning with Calibration Rewards (Brier-score term added to RLVR)
RLVR	Reinforcement Learning with Verifiable Rewards
RM	Reward Model
RoPE	Rotary Position Embedding
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RRF	Reciprocal Rank Fusion
SDK	Software Development Kit
SFT	Supervised Fine-Tuning
SGD	Stochastic Gradient Descent
SM	Streaming Multiprocessor
SPLADE	SParse Lexical AnD Expansion
SRAM	Static Random-Access Memory
SSE	Server-Sent Events
SWE-bench	Software Engineering Benchmark
TD	Temporal Difference (learning)
TP	Tensor Parallelism
TRL	Transformer Reinforcement Learning (library)
UCB	Upper Confidence Bound
vLLM	Virtual LLM (inference engine)

Part I

Foundations

Chapter 1

LLM Architecture and Optimization Methods

This chapter covers the foundational architecture of large language models and the key optimization techniques that make training and inference efficient. Topics are ordered as a curriculum: we begin with the transformer itself, then cover how to train it efficiently, how to adapt it cheaply, how to compress it, how to scale it, and how to accelerate its inference.

GAS Connection — LLM Architecture and Optimization Methods

The Guardian Angel System relies on language models at every layer of its agent stack, and the architectural choices made in the following chapter directly determine which model fits which role.

Tokenisation (§ Tokenisation). The Monitor agent processes continuous glucose readings encoded as structured text tokens (e.g. `[CGM: 142 mg/dL, trend: rising, delta: +8]`). Choosing a tokeniser that treats numeric values as single tokens rather than splitting digits is critical for accurate trend reasoning. The GAS uses the Llama-3 tokeniser via Ollama, which handles decimal numbers well.

Transformer architecture (§ The Transformer Architecture). The Advisor agent's multi-head attention mechanism allows it to attend simultaneously to the patient's current glucose reading, the retrieved clinical guideline, and the meal log—three distinct information sources in a single forward pass. Understanding how attention heads specialise (some track recency, others track magnitude) helps you interpret why the Advisor sometimes ignores a guideline that is technically relevant but contextually distant.

LoRA fine-tuning (§ LoRA and Parameter-Efficient Fine-Tuning). The GAS prototype runs on a shared `llama3.2:3b` base model via Ollama without per-agent fine-tuning. The chapter notebooks in `part2_rl_methods/ch04_rl_for_lms/notebook.ipynb` demonstrate how LoRA adapters (rank $r = 16$) can be applied to specialise a base model for clinical tasks—the same technique that would be used to fine-tune the GAS agents in a production deployment.

Model compression (§ Model Compression Methods). The GAS uses 4-bit GGUF quantisation (Q4_K_M) via Ollama so that all agents can run on a single consumer GPU. The `GASLLMClient` in `capstone_gas/gas_system.py` auto-detects the available backend (OpenAI → Anthropic → Ollama → Mock) and falls back gracefully, making the system runnable even without a GPU.

Prompt engineering (§ Prompt Engineering). Each agent node in `capstone_gas/gas_system.py` uses a structured system prompt that defines the agent's role, output format (plain text with structured sections), and escalation criteria. The prompt engineering patterns in this section—role assignment, output constraints, chain-of-thought elicitation—are directly applied in `monitor_node()`, `advisor_node()`, `safety_node()`, and `caregiver_node()`.

Safety (§ LLM Safety and Responsible AI). The Safety Critic (`safety_node()` in `capstone_gas/gas_system.py`) implements a rule-based constitutional check: it scores the Advisor's output on a 0.0–1.0 scale and rejects recommendations that suggest dangerous insulin doses

or dismiss critical glucose readings. A score ≥ 0.7 is required for approval; the system retries up to 2 times before escalating.

1.1 How LLMs Work: An Intuitive Overview

Before diving into architectural details, let us build intuition for how a large language model transforms text into text. The entire process follows a simple pipeline: **text** → **tokens** → **representations** → **tokens** → **text**.

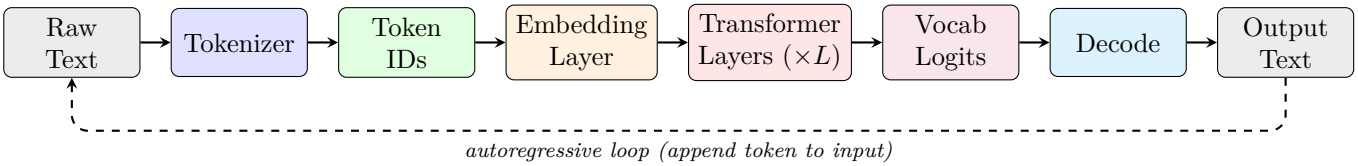


Figure 1.1: The LLM pipeline: text is tokenized into subword units, converted to integer IDs, embedded as dense vectors, processed through transformer layers, projected to vocabulary logits, and decoded back to text. The dashed loop shows autoregressive generation—each output token is appended to the input for the next forward pass.

The Four Key Stages

1. **Tokenization**: Raw text is split into subword pieces (not characters, not full words) using a learned vocabulary. "unhappiness" might become ["un", "happiness"] or ["unhapp", "iness"].
2. **Embedding**: Each token ID indexes into a learned embedding table, producing a dense vector in $\mathbb{R}^d$ (typically $d = 4096$). These vectors capture semantic meaning—similar words get similar vectors.
3. **Contextual Processing**: The transformer stack processes all embeddings in parallel, using self-attention to let each position "read" from all other positions. After L layers, each position's hidden state encodes rich contextual information.
4. **Prediction**: The final hidden state is projected to a probability distribution over the full vocabulary, and a decoding strategy selects the next token.

1.2 Tokenization

Tokenization is the critical first step that converts raw text into the discrete symbols a language model operates on. The choice of tokenizer directly affects model quality, multilingual capability, and computational efficiency.

Why Subwords?

Character-level models need very long sequences (expensive attention). Word-level models cannot handle rare or novel words. Subword tokenization strikes the ideal balance: common words are single tokens ("the" → [the]), rare words decompose into known pieces ("cryptocurrency" → ["crypt", "ocur", "rency"]), and the vocabulary stays manageable (32K–128K tokens).

1.2.1 Why Not Characters or Words?

Table 1.1: Trade-offs of different tokenization granularities.

Granularity	Vocab Size	Seq Length	Issues
Character	∼256	Very long	Attention cost $O(n^2)$; hard to learn long-range semantics
Word	∼500K+	Short	Cannot handle rare/novel words; huge embedding table
Subword	32K–128K	Moderate	Best trade-off: short sequences, open vocabulary

1.2.2 Byte-Pair Encoding (BPE)

BPE [338] is the dominant tokenization algorithm used by GPT, Llama, Mistral, and most modern LLMs.

BPE Algorithm

1. Start with a vocabulary of individual characters (bytes)
2. Count all adjacent symbol pairs in the training corpus
3. Merge the most frequent pair into a new symbol
4. Repeat steps 2–3 for k iterations (until desired vocabulary size)

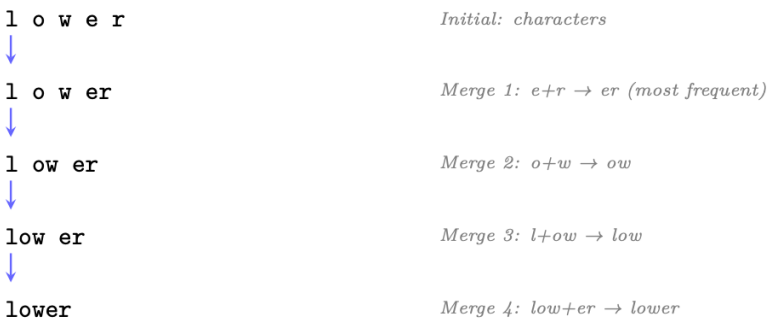


Figure 1.2: BPE tokenization example: starting from characters, the algorithm iteratively merges the most frequent adjacent pairs until the word becomes a single token or the vocabulary budget is exhausted.

1.2.3 Other Tokenization Methods

Table 1.2: Comparison of subword tokenization algorithms.

Method	Used By	Key Idea
BPE	GPT-4 [290], Llama-3 [126], Mistral [175]	Bottom-up merging of frequent pairs; deterministic
WordPiece	BERT [80], DistilBERT [330]	Similar to BPE but maximizes likelihood of training data
Unigram LM	SentencePiece (T5 [321], XLNet [428])	Top-down: start with large vocab, prune by likelihood impact
Byte-level BPE	GPT-2 [319]+	BPE on raw bytes (no unknown tokens possible); 256 base vocab

1.2.4 Tokenization Best Practices

1. **Vocabulary size matters**: 32K is minimal; 128K enables better multilingual coverage and code handling. Llama-3 uses 128K tokens.
2. **Special tokens**: Always include `<bos>`, `<eos>`, `<pad>`, `<unk>`. For instruction-tuned models, add role markers (`<|user|>`, `<|assistant|>`).
3. **Fertility**: Measure tokens-per-word across languages. High fertility (many tokens per word) indicates poor coverage for that language.
4. **Never tokenize across boundaries**: Spaces, punctuation, and digits should be handled consistently. Most modern tokenizers prepend a space marker ("the") to distinguish word-initial vs. continuation tokens.
5. **Numbers**: Consider digit-level tokenization for arithmetic tasks. "2024" as ["2","0","2","4"] enables digit-by-digit reasoning.

6. **Code**: Ensure whitespace (indentation) is tokenized efficiently. Llama-3 tokenizes runs of spaces as single tokens.

1.2.5 Tokenization in Practice: HuggingFace Example

The `transformers` library provides a unified interface for all tokenizers. The following demonstrates encoding and decoding with a modern LLM tokenizer:

```python
from transformers import AutoTokenizer

# Load Llama-3 tokenizer (128K vocabulary, byte-level BPE)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")

text = "Reinforcement learning optimizes long-term rewards."

# Encode: text -> token IDs
token_ids = tokenizer.encode(text)
print(token_ids)
# [128000, 29934, 262, 11008, 4815, 6900, 1317, 9860, 21845, 13]

# Decode individual tokens to see subword splits
tokens = tokenizer.convert_ids_to_tokens(token_ids)
print(tokens)
# ['<|begin_of_text|>', 'Re', 'inforce', 'ment', ' learning',
#  ' optimizes', ' long', '-term', ' rewards', '.']

# Decode back to text (round-trip)
reconstructed = tokenizer.decode(token_ids, skip_special_tokens=True)
assert reconstructed == text  # Perfect reconstruction

# Tokenize with attention mask (for batched inputs with padding)
batch = tokenizer(
    ["Short text.", "A much longer input sentence for comparison."],
    padding=True, return_tensors="pt"
)
print(batch.keys())  # dict_keys(['input_ids', 'attention_mask'])
```

Listing 1.1: Tokenization encode/decode with HuggingFace Transformers.

1.2.6 Special Tokens and Structured Prompts

Special tokens are reserved vocabulary entries that carry structural meaning rather than linguistic content. They are critical for controlling model behavior.

Table 1.3: Common special tokens across LLM families.

Token	Alias	Purpose
`<bos>` / `<|begin_of_text|>`	BOS	Marks start of sequence
`<eos>` / `<|end_of_text|>`	EOS	Marks end of sequence; stops generation
`<|user|>`	—	Marks start of user turn in chat
`<|assistant|>`	—	Marks start of assistant turn in chat
`<pad>`	PAD	Fills batch to uniform length; masked in attention
`<unk>`	UNK	Out-of-vocabulary placeholder (rare with BPE)
`[SEP]`	SEP	Separates segments (BERT-style)
`[CLS]`	CLS	Classification token (BERT)
`[MASK]`	MASK	Masked token for MLM pretraining

Role Markers for Instruction-Tuned Models. Modern chat models use special tokens to delineate conversational structure. These are **not** trained to carry semantic meaning—they are structural delimiters that the model learns to parse:

```
# Llama-3 chat template
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Explain PPO in one sentence."},
]

# apply_chat_template handles all special token insertion
prompt = tokenizer.apply_chat_template(messages, tokenize=False)
print(prompt)
# <|begin_of_text|><|start_header_id|>system<|end_header_id|>
#
# You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>
#
# Explain PPO in one sentence.<|eot_id|><|start_header_id|>assistant<|
    end_header_id|>
#
#
```

Listing 1.2: Chat template with special tokens (Llama-3 format).

Special Token Best Practices

- **Never split special tokens**: They must be atomic—ensure your tokenizer treats them as single units, not character sequences.
- **Mask loss on special tokens**: During SFT, do not compute loss on structural tokens (role markers, separators). The model should not "learn" to predict formatting.
- **Use templates for structure**: Encode task semantics via special tokens rather than natural language instructions. E.g., `<|tool_call|>` is more reliable than "Now I will call a tool:".
- **Tool/function calling**: Define dedicated tokens like `<|function|>`, `<|result|>` to create unambiguous boundaries between reasoning and action.
- **Consistent handling in RL**: During PPO/GRPO, ensure the reference model and policy model use identical tokenization and special token handling—mismatches corrupt KL computation.
- **EOS handling**: During generation, ensure EOS is included in the action space. If the model cannot emit EOS, responses grow unbounded (common RL failure mode).

1.3 The Transformer Architecture

The Transformer [378] is the foundation of all modern LLMs. Understanding its components is essential for grasping every optimization and training method in this guide.

1.3.1 High-Level Structure

A decoder-only transformer processes tokens sequentially through embedding, repeated attention+FFN blocks, and a final projection to vocabulary logits. Figure 1.3 shows the complete architecture.

1.3.2 The Original Encoder-Decoder Transformer

The Transformer was originally introduced [378] as an **encoder-decoder** architecture for sequence-to-sequence tasks (machine translation, summarization). While modern LLMs predominantly use decoder-only variants (GPT-style), understanding the full architecture is essential because cross-attention and masked self-attention — both originating here — remain fundamental building blocks.

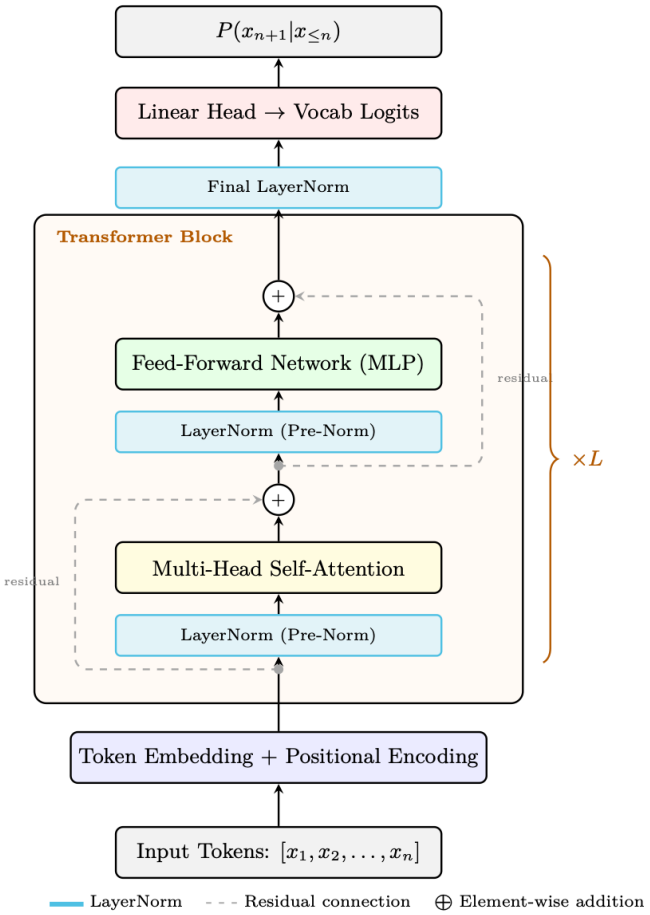


Figure 1.3: Decoder-only Transformer block (GPT-style, Pre-Norm variant). Each sub-layer (attention, FFN) is preceded by LayerNorm and followed by a residual addition: $\mathbf{x} + \text{SubLayer}(\text{LN}(\mathbf{x}))$. This Pre-Norm ordering (used by Llama, GPT-3, Mistral) stabilizes training without warmup, unlike the original Post-Norm (which applies LayerNorm after the addition). L identical blocks are stacked, followed by a final LayerNorm and linear projection to vocabulary logits.

Encoder. The encoder processes the entire input sequence *bidirectionally* — each token attends to all other tokens (no causal mask). This produces a rich contextual representation $\mathbf{H}^{\text{enc}} \in \mathbb{R}^{n \times d}$ where each position encodes information about the full input:

- **Input**: Token embeddings + sinusoidal positional encodings
- **Each layer**: Multi-Head Self-Attention → Add & Norm → FFN → Add & Norm
- **No causal mask**: Position i attends to all positions $1, \ldots, n$
- **Output**: Contextual representations of the full input sequence

Decoder — Masked Multi-Head Self-Attention. The decoder generates output tokens one at a time (autoregressively). To prevent the model from "seeing the future," the self-attention in the decoder uses a **causal mask**:

$$\text{MaskedAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{1.1}$$

where the mask M is:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \text{ (can attend)} \\ -\infty & \text{if } i < j \text{ (future token — blocked)} \end{cases}$$

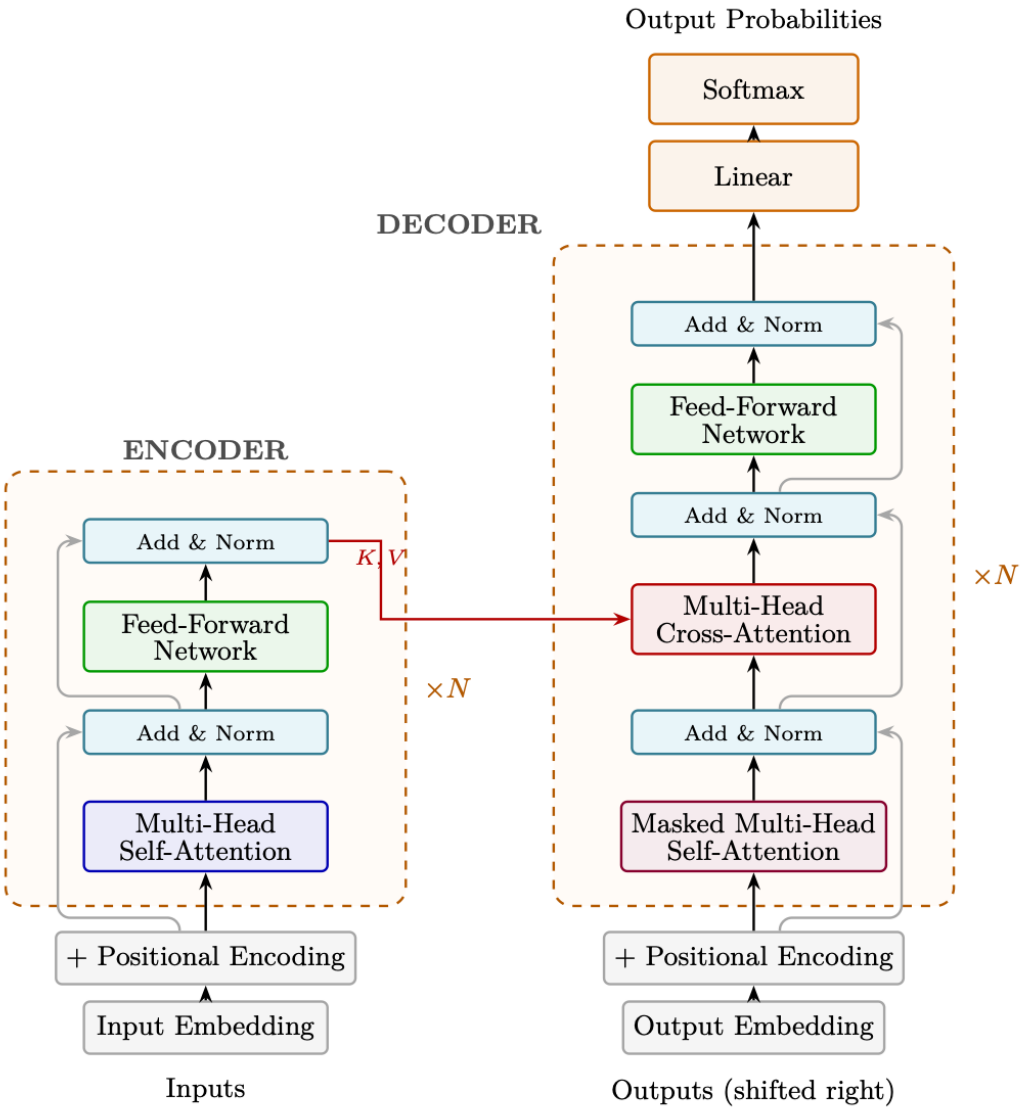


Figure 1.4: The original Transformer architecture (Vaswani et al., 2017). The encoder (left) processes the full input with bidirectional self-attention. The decoder (right) generates tokens autoregressively using masked self-attention and cross-attention to encoder representations. Dashed boxes indicate the repeated layer block ($\times N$); gray lines show residual connections bypassing each sub-layer. Note: the original work uses **Post-Norm** (LayerNorm applied *after* the residual addition: $\text{LN}(\mathbf{x} + \text{SubLayer}(\mathbf{x}))$), unlike modern LLMs which use Pre-Norm.

Why Masking Matters

During training, the decoder processes the entire target sequence in parallel (teacher forcing), but each position must only attend to previous positions to maintain the autoregressive property. The mask ensures that generating token t uses only information from tokens $1, \ldots, t-1$. At inference, tokens are generated one-by-one so the mask is implicit — but during training it enables parallel computation while preserving causality.

Decoder — Cross-Attention. After masked self-attention, each decoder layer applies **cross-attention** where the decoder attends to the encoder's output representations. This is the mechanism by which the decoder "reads" the input:

$$\text{CrossAttn}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^T}{\sqrt{d_k}}\right) V_{\text{enc}} \tag{1.2}$$

- **Queries** come from the decoder's previous sublayer (the masked self-attention output)

- **Keys and Values** come from the encoder's final output $\mathbf{H}^{\text{enc}}$
- **No mask** is applied — every decoder position can attend to every encoder position
- This allows the decoder to dynamically focus on different parts of the input at each generation step (e.g., attending to "cat" when translating to "gato" in English→Spanish)

Full Decoder Layer. Each decoder layer contains three sublayers (vs. two in the encoder):

1. **Masked Multi-Head Self-Attention** + Residual + LayerNorm
2. **Multi-Head Cross-Attention** (to encoder output) + Residual + LayerNorm
3. **Feed-Forward Network** + Residual + LayerNorm

From Encoder-Decoder to Decoder-Only. Modern LLMs (GPT, Llama, Qwen) use only the decoder, removing both the encoder and cross-attention layers entirely. The key insight: for generative language modeling, a single causal (masked) self-attention stack is sufficient — the model learns to encode context and generate continuations in a single pass. This simplifies architecture, training, and inference while scaling more effectively. Encoder-decoder models (T5, BART) remain relevant for tasks with distinct input/output structure (translation, summarization), and cross-attention reappears in multimodal models where vision encoders provide keys/values to language decoders.

1.3.3 Decoder-Only vs Encoder-Decoder

Modern LLMs almost exclusively use decoder-only architectures, but understanding the trade-offs with encoder-decoder designs clarifies why.

Architecture	Examples	Use Case
Decoder-only	GPT-4 [290], Llama [126], Mistral [175], Qwen [371]	Autoregressive generation; dominant for chat/reasoning
Encoder-decoder	T5 [321], BART [218], Flan-T5 [61]	Seq2seq (translation, summarization); less common now
Encoder-only	BERT [80], RoBERTa [245]	Classification/embeddings; not for generation

Why Decoder-Only Won

Decoder-only models are simpler (one model, one loss), scale better (all parameters contribute to generation), and support unified training (pretraining = next-token prediction = fine-tuning objective). Encoder-decoder models waste capacity on the encoder for pure generation tasks.

1.3.4 Embeddings: From Discrete Tokens to Continuous Space

Before any attention or computation happens, the transformer must convert discrete token IDs into continuous vectors that neural networks can process. This is the role of the **embedding layer**.

What is an Embedding? An embedding is a learned dense vector representation of a discrete symbol. Instead of representing the word "king" as a one-hot vector of size $|\mathcal{V}| = 128{,}000$ (mostly zeros), we represent it as a compact vector in $\mathbb{R}^d$ (e.g., $d = 4096$) that captures its *meaning*.

The key insight: **similar concepts get nearby vectors**. In a well-trained embedding space:

- "king" and "queen" are close (both royalty)
- "king" and "bicycle" are far apart (unrelated)
- Vector arithmetic captures relationships: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

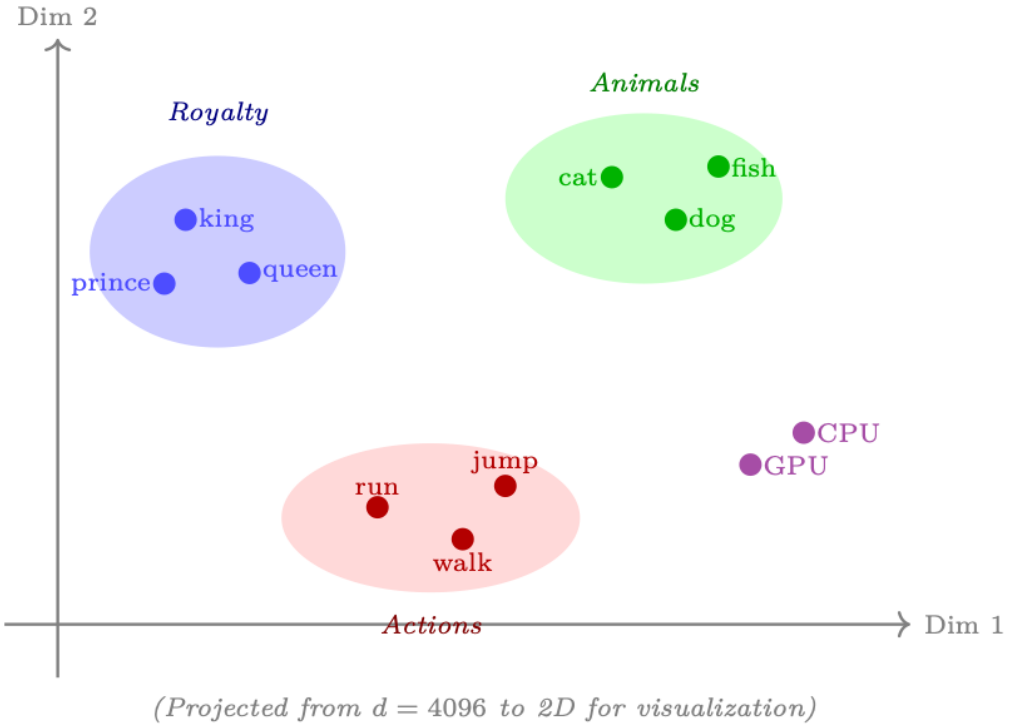


Figure 1.5: Embedding space visualization (2D projection): semantically similar words cluster together. The embedding table learns these positions during pretraining, capturing meaning purely from co-occurrence patterns in text.

The Embedding Table. In practice, the embedding layer is simply a matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ where row i stores the embedding vector for token i:

$$\text{embed}(x_t) = \mathbf{E}[x_t] \in \mathbb{R}^d \tag{1.3}$$

For a sequence of token IDs $[x_1, x_2, \ldots, x_n]$, embedding is a simple table lookup (indexing operation):

$$\mathbf{H}_0 = [\mathbf{E}[x_1];\ \mathbf{E}[x_2];\ \ldots;\ \mathbf{E}[x_n]] \in \mathbb{R}^{n \times d}$$

Embedding Table in Transformers

- **Size**: $|\mathcal{V}| \times d$. For Llama-3: 128,256 × 4,096 = 525M parameters (6.5% of 8B model).
- **Initialization**: Random (Xavier/normal), then learned via backpropagation.
- **Weight tying**: Many models *share* the embedding matrix with the output projection head: $W_{\text{head}} = \mathbf{E}^T$. This saves parameters and creates a symmetric encode-decode structure.
- **Input**: Token ID (integer) → **Output**: Dense vector in $\mathbb{R}^d$.
- **Gradient flow**: During training, only the rows corresponding to tokens in the current batch receive gradient updates (sparse update).

Why Embeddings Work

The embedding table is learned end-to-end with the rest of the model. Because the model is trained to predict the next token, it must learn representations where tokens that appear in similar contexts get similar vectors. This is the distributional hypothesis: "you shall know a word by the company it keeps" [96]. The embedding layer compresses this statistical structure into dense geometry.

The Anisotropy Problem. A critical issue arises when using pretrained embeddings (e.g., from BERT or GPT-2) for downstream tasks like retrieval (RAG) or bootstrapping recommender systems:

the learned representations are highly **anisotropic**—they occupy a narrow cone in the embedding space rather than being uniformly distributed across all directions [90].

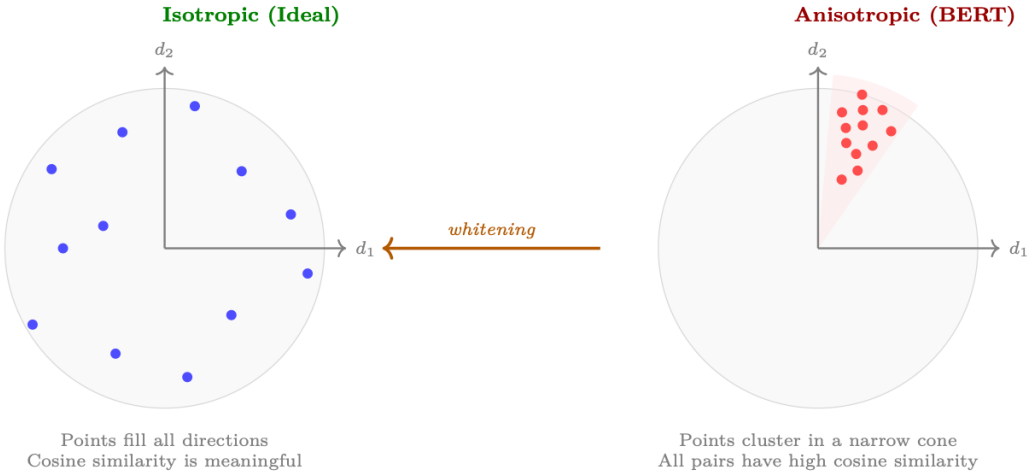


Figure 1.6: Isotropy vs. anisotropy in embedding spaces. Left: isotropic embeddings spread uniformly, making cosine similarity a reliable measure of semantic relatedness. Right: anisotropic embeddings (as found in BERT) cluster in a narrow cone, causing all pairs to have high cosine similarity regardless of semantic content. Whitening transforms the space to restore isotropy.

Why this matters for applications:

- **RAG / Retrieval**: If all embeddings have cosine similarity > 0.7 regardless of content, retrieval rankings become nearly random—the system cannot distinguish relevant from irrelevant passages.
- **Recommender systems**: Using pretrained LLM embeddings to represent items/users only works if the geometry preserves meaningful similarity structure.
- **Clustering**: Anisotropic embeddings collapse clusters, making it impossible to discover natural groupings.

Resolution: Whitening. A simple and effective fix is **whitening** [359]—a linear transformation that makes the embedding distribution isotropic (zero mean, identity covariance):

$$\tilde{\mathbf{h}} = \mathbf{D}^{-1/2}\mathbf{U}^T(\mathbf{h} - \boldsymbol{\mu}) \tag{1.4}$$

where $\boldsymbol{\mu}$ is the mean embedding, and $\mathbf{U}\mathbf{D}\mathbf{U}^T$ is the eigendecomposition of the covariance matrix $\Sigma = \frac{1}{N}\sum_i(\mathbf{h}_i - \boldsymbol{\mu})(\mathbf{h}_i - \boldsymbol{\mu})^T$.

Whitening in Practice

- **What it does**: Rotates and scales the embedding space so all directions have equal variance (unit covariance).
- **Effect**: Cosine similarity becomes meaningful—semantically similar pairs score high, dissimilar pairs score low.
- **Bonus**: Can simultaneously reduce dimensionality by keeping only the top-k eigenvectors (similar to PCA), making retrieval faster.
- **Cost**: Requires computing the covariance matrix over a representative corpus (one-time, $O(N \cdot d^2)$). The transform itself is a simple matrix multiply at inference.
- **Alternative approaches**: Contrastive fine-tuning (SimCSE), flow-based normalization, or training with isotropy-promoting regularizers.

1.3.5 Self-Attention Mechanism

Self-attention is the core operation that allows each token to attend to every other token in the sequence, computing a weighted combination based on relevance.

Scaled Dot-Product Attention

Given input sequence $X \in \mathbb{R}^{n \times d}$, we compute:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k})$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where M is the **causal mask** (for autoregressive models): $M_{ij} = 0$ if $i \geq j$, else $-\infty$.
Intuition: Each token "attends" to all previous tokens, computing a weighted average of their values based on query-key similarity.

Computational Complexity. The naive attention computation has **quadratic cost** in sequence length:

- **Time**: $O(n^2 \cdot d)$ — computing QK^T requires n^2 dot products, each of dimension d_k.
- **Memory**: $O(n^2)$ — the full attention matrix must be materialized to apply softmax.

For a 128K-token context with $d = 4096$, the attention matrix alone is $128\text{K} \times 128\text{K} = 16.4$ billion entries (64 GB in FP32). This quadratic scaling is the fundamental bottleneck for long-context LLMs.

Table 1.4: Attention cost scaling: why naive implementation is prohibitive for long sequences.

Seq Length	Attention Ops	Matrix Size	Practical Impact
2K	4M	16 MB	Fast; fits in SRAM
8K	64M	256 MB	Manageable with FlashAttention
32K	1B	4 GB	Requires memory-efficient kernels
128K	16B	64 GB	Exceeds single GPU HBM
1M	1T	4 TB	Impossible without sub-quadratic methods

Approaches to Taming Attention Cost. Several families of solutions address this quadratic bottleneck:

1. **Exact attention with IO-awareness (FlashAttention [68])**: Does not reduce computational complexity but eliminates the need to materialize the $n \times n$ matrix in HBM by computing attention in tiles that fit in SRAM. Crucially, FlashAttention is **orthogonal** to the sparse patterns below—it is an execution engine, not an attention pattern. Production systems routinely combine FlashAttention with sliding windows or block-sparse masks, getting both IO efficiency and reduced FLOPs. We cover the algorithm in detail in Section 1.6.

2. **Sliding window / local attention**: Each token only attends to the w nearest tokens (e.g., $w = 4096$). Cost becomes $O(n \cdot w)$—linear in n. Used by Mistral [175] (window = 4096) and Longformer [25]. Trades global context for efficiency; works well because most attention is local in practice. In modern stacks, the sliding-window mask is executed *inside* a FlashAttention kernel.

3. **Sparse attention patterns**: Combine local windows with periodic global tokens (e.g., every 512th token attends to all). BigBird [439] and LongT5 [133] use this. Preserves some long-range connectivity at $O(n\sqrt{n})$ cost. Again, FlashAttention serves as the underlying kernel for the non-zero attention blocks.

4. **Linear attention / state-space models**: Replace $\text{softmax}(QK^T)V$ with $\phi(Q)(\phi(K)^TV)$ using associativity, or reformulate as a recurrence (Mamba [130], RWKV [305]). Theoretically $O(n \cdot d^2)$ total. Unlike approaches 2–3 above, these are *architectural replacements* that alter model expressiveness—softmax-free attention is fundamentally less expressive, and empirically these models still lag behind transformers on tasks requiring precise long-range retrieval or complex reasoning.

5. **KV cache compression**: At inference, compress or evict old KV pairs to bound memory. Techniques include: H_2O [452] (heavy-hitter oracle—keep only high-attention keys), StreamingLLM [416] (keep initial "attention sink" tokens + recent window), and quantized KV caches [249].

FlashAttention + Sparse Patterns = Best of Both Worlds

A common misconception is that FlashAttention is an *alternative* to sparse attention. It is not—it is an IO optimization for the attention kernel that composes freely with any attention mask. Modern production systems (e.g., Mistral, DeepSeek) use FlashAttention as the execution engine *underneath* a sliding-window or block-sparse mask. This gives you both reduced FLOPs (from sparsity) and optimal memory access patterns (from tiling). RingAttention [233] extends this further to multi-device settings, distributing the tiled computation across GPUs along the sequence dimension.
Linear attention and state-space models (Mamba, RWKV) are a genuinely different architectural choice—they sacrifice the full pairwise interaction for $O(n)$ compute. While theoretically elegant, they have not matched transformer quality on knowledge-intensive or long-range reasoning tasks, and frontier labs continue to use exact attention (with FlashAttention + sparsity) as the backbone.

1.3.6 Multi-Head Attention

Rather than computing a single attention function, multi-head attention runs several attention operations in parallel, each learning to focus on different aspects of the input (syntax, semantics, position, etc.).

Multi-Head Attention

Instead of one attention function with d-dimensional keys/values, use H parallel heads with dimension $d_k = d/H$:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W_O$$

Each head can learn different attention patterns (e.g., one head for syntax, another for semantics, another for positional proximity).
Grouped Query Attention (GQA): Llama-3 [126] uses fewer K,V heads than Q heads (e.g., 8 KV heads shared across 32 Q heads). This reduces KV cache size by 4× with minimal quality loss.

Multi-head Latent Attention (MLA). DeepSeek-V2 [72] introduced a more aggressive compression: instead of sharing KV heads (as in GQA), MLA compresses *all* key-value information into a single low-rank latent vector per token:

$$c_t = W_{DKV}\, h_t, \quad K_t = W_{UK}\, c_t, \quad V_t = W_{UV}\, c_t$$

where $c_t \in \mathbb{R}^{d_c}$ with $d_c \ll n_h \cdot d_h$. The KV cache stores only c_t—smaller than even GQA—while decompression matrices W_{UK}, W_{UV} reconstruct full keys and values on-the-fly. Because the compression is learned end-to-end, quality matches or exceeds GQA. MLA enables DeepSeek's subsequent Sparse Attention for sub-quadratic long-context scaling and is now the basis of the DeepSeek-V3/V4 architecture family.

1.3.7 Positional Encodings

Transformers are permutation-equivariant by construction — without positional information, the model cannot distinguish "the cat sat on the mat" from "mat the on sat cat the". Positional encodings inject sequence-order signal so that attention can reason about token distance and direction.

Table 1.5: Positional encoding methods in modern LLMs.

Method	Used By	Key Idea
Sinusoidal	Original Transformer	Fixed sin / cos at different frequencies. Not learned.
Learned Absolute	GPT-2 [319], BERT [80]	Learned embedding per position. Limited to training length.
RoPE (Rotary)	Llama [126], Qwen [371], Mistral [175]	Rotate Q,K vectors by position-dependent angle. Extrapolates via NTK-aware scaling.
ALiBi	BLOOM [405], MPT [274]	No position embedding; add linear bias $-m\|i-j\|$ to attention scores. Simple, extrapolates well.

Sinusoidal (Fixed) Positional Encoding. Introduced in the original Transformer [378], this method uses fixed sinusoidal functions at geometrically-spaced frequencies:

$$\mathrm{PE}(pos, 2i) = \sin\Big(\frac{pos}{10000^{2i/d}}\Big), \qquad \mathrm{PE}(pos, 2i{+}1) = \cos\Big(\frac{pos}{10000^{2i/d}}\Big)$$

where pos is the token position, i is the dimension index, and d is the model dimension.

Motivation: Each frequency encodes position at a different scale (analogous to binary counting). The authors hypothesised that the model could learn to attend to relative positions because $\mathrm{PE}(pos{+}k)$ can be expressed as a linear function of $\mathrm{PE}(pos)$.

Pros: Zero learned parameters; deterministic; theoretically supports arbitrary lengths.

Cons: In practice, does not extrapolate well beyond training lengths; the model must learn to decode relative position from absolute signals indirectly; largely superseded.

Learned Absolute Positional Embedding. Used by GPT-2 [319] and BERT [80]: a learnable embedding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ is added to token embeddings:

$$h_0^{(pos)} = \mathrm{TokenEmbed}(x_{pos}) + \mathbf{E}_{\text{pos}}[pos]$$

Motivation: Let the model learn whatever positional representation is optimal for the task, rather than imposing a fixed structure.

Pros: Maximum flexibility; simple implementation; often outperforms sinusoidal for short sequences.

Cons: Hard-coded maximum length $L_{\max}$; no generalisation beyond it; embeddings near the end of $L_{\max}$ are under-trained; adds $L_{\max} \times d$ parameters.

Rotary Position Embedding (RoPE). RoPE [360] encodes position by *rotating* query and key vectors in 2D subspaces:

$$\mathrm{RoPE}(x_m, m) = \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \\ \vdots \\ x_m^{(d-1)} \\ x_m^{(d)} \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_m^{(2)} \\ x_m^{(1)} \\ \vdots \\ -x_m^{(d)} \\ x_m^{(d-1)} \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

where $\theta_i = 10000^{-2i/d}$ and m is the position index. The key property is that the dot product between rotated queries and keys depends only on relative position:

$$\langle \text{RoPE}(q_m, m),\ \text{RoPE}(k_n, n) \rangle = f(q_m, k_n, m - n)$$

Motivation: Achieve relative position encoding without explicit bias terms, while maintaining compatibility with linear attention and KV-caching.

Pros: Naturally relative; no extra parameters; compatible with efficient inference; can be extended to longer contexts via NTK-aware scaling [306] or YaRN (adjusting θ base or interpolating frequencies).

Cons: Slightly more compute per attention operation (rotation + interleaving); extrapolation requires explicit scaling strategies; rotation in 2D subspaces imposes structure that may not be optimal for all tasks.

RoPE Length Extension

To extend a RoPE model trained at L to context length $L' > L$:

- **Position interpolation:** Scale positions by L/L' so all positions fit in $[0, L]$. Simple but compresses resolution.
- **NTK-aware scaling:** Increase the θ base (e.g. $10000 \to 10000 \cdot (L'/L)^{d/(d-2)}$), effectively stretching high-frequency components while preserving low-frequency ones.
- **YaRN** [306]: Combines NTK scaling with an attention temperature correction $t = 0.1 \ln(s) + 1$ to compensate for increased entropy at longer distances.

ALiBi (Attention with Linear Biases). ALiBi [309] takes a radically different approach: *no positional embedding at all.* Instead, a static linear penalty is subtracted from attention scores:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} - m \cdot \left[|i - j| \right]_{i,j} \right) V$$

where m is a head-specific slope (set geometrically: $m_h = 2^{-8h/H}$ for head h of H total). The bias $-m|i-j|$ creates a soft local attention window whose width varies by head.

Motivation: Position should bias attention toward nearby tokens (recency prior) without interfering with the embedding space. By operating purely in attention-score space, ALiBi avoids polluting token representations with positional signal.

Pros: Excellent length extrapolation (trained at 1k, works at 8k+); zero parameters; trivial to implement; head-specific slopes give multi-scale locality.

Cons: Less expressive for tasks requiring precise long-range positional reasoning (e.g. "what was the 5th word?"); the linear decay is a strong inductive bias that may not suit all domains; largely overtaken by RoPE in recent models due to RoPE's better short-context performance.

Table 1.6: Positional encoding comparison: practical trade-offs.

	Sinusoidal	**Learned Abs.**	**RoPE**	**ALiBi**
Extra parameters	None	$L_{\max} \times d$	None	None
Position type	Absolute	Absolute	Relative	Relative (implicit)
Length extrapolation	Poor	None	Good (w/ scaling)	Excellent
Compute overhead	Negligible	Negligible	Small	Negligible
Dominant era	2017–19	2018–20	2022–present	2022–23

Scaling to Extremely Long Contexts (100K–1M+ Tokens). Modern frontier models (Claude [12] with 200K–1M context, Gemini 1.5 [115] at 1M+, GPT-4 [290] at 128K) require positional encodings that remain faithful far beyond training lengths. The dominant solutions today:

1. **RoPE with frequency scaling**: The standard approach for extending RoPE beyond training length. Rather than retraining, the base frequency θ is rescaled:

$$\theta_i' = \theta_i \cdot \left(\frac{L_{\text{target}}}{L_{\text{train}}}\right)^{2i/d}$$

   Variants include:

   - **Linear scaling** (Position Interpolation) [50]: Simply divide position indices by a factor s. Cheap but degrades quality at high extension ratios.
   - **NTK-aware scaling** [306]: Scale the base frequency $\theta = 10000 \to 10000 \cdot s^{d/(d-2)}$. Preserves high-frequency (local) information while extending low-frequency (global) range.
   - **YaRN** [306] (Yet another RoPE extensioN): Combines NTK scaling with an attention temperature correction and fine-tuning on a small long-context corpus. Used by Llama-3 to extend from 8K training to 128K deployment.
   - **Dynamic NTK** [306]: Adjusts the scaling factor on-the-fly based on actual sequence length at inference. No fixed extension ratio needed—the model adapts as context grows.

2. **Continued pretraining on long data**: Even with RoPE scaling, models benefit from a short continued pretraining phase (1–5B tokens) on long documents. This teaches the model to actually *use* distant context, not just tolerate it positionally. Llama-3.1 used a progressive schedule: 8K → 64K → 128K.

3. **Ring Attention / Blockwise Parallel** [233]: For sequences exceeding single-GPU memory (1M+ tokens), Ring Attention distributes the sequence across GPUs in a ring topology. Each GPU holds a block and passes KV blocks around the ring, computing local attention tiles. This enables linear memory scaling with GPU count while preserving exact attention.

4. **Hybrid architectures**: Some systems combine a local sliding window (e.g., 4K) for most layers with full attention at select layers (e.g., every 4th layer). This provides $O(n \cdot w)$ cost for most computation while maintaining global information flow.

Long Context ≠ Long Context Usage

A model with 1M context length does *not* necessarily use all 1M tokens effectively. The "lost in the middle" phenomenon [238] shows that models tend to focus on the beginning and end of long contexts, underutilizing information in the middle. Effective long-context utilization requires both positional encoding support *and* training on tasks that reward long-range retrieval.

1.3.8 Feed-Forward Network (MLP)

Each transformer block contains an MLP applied independently to each position:

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{d \times 4d}$, $W_2 \in \mathbb{R}^{4d \times d}$. Modern LLMs use:

- **SwiGLU activation**: $\text{FFN}(x) = W_2(\text{Swish}(W_1 x) \odot W_3 x)$ — used by Llama [126], Mistral [175]. Requires 3 weight matrices but gives better performance.
- Hidden dimension is typically $8/3 \times d$ (rounded to multiples of 256 for Tensor Core efficiency).

FFN as Memory

Recent work [117] suggests the FFN layers act as a *key-value memory*: W_1 rows are keys (patterns to match), W_2 columns are values (information to output). The FFN "retrieves" stored knowledge based on the current hidden state.

1.3.9 Layer Normalization

Layer normalization stabilizes training by normalizing activations across the feature dimension. Its placement relative to the attention/FFN sublayers significantly affects training dynamics.

How LayerNorm Works. Given a hidden state vector $\mathbf{x} \in \mathbb{R}^d$ (a single token's representation), LayerNorm [16] computes:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{1.5}$$

where:

- $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$ (mean across the d feature dimensions)
- $\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$ (variance across features)
- $\gamma, \beta \in \mathbb{R}^d$ are **learned** scale and shift parameters (per-dimension)
- $\epsilon \approx 10^{-5}$ prevents division by zero

Key distinction from BatchNorm: LayerNorm normalizes across the *feature dimension* of a single example, not across the batch. This makes it independent of batch size and works identically at training and inference.

RMSNorm — The Modern Simplification. RMSNorm [444] drops the mean-centering step, normalizing only by the root-mean-square:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}, \qquad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2} \tag{1.6}$$

No β (shift) parameter and no mean subtraction — just scale. This saves one reduction operation per token and is ∼5–10% faster on GPUs while achieving equivalent model quality. All modern LLMs (Llama, Mistral, Qwen) use RMSNorm.

Pre-LN vs Post-LN

- **Post-LN** (original Transformer): $h + \text{LayerNorm}(\text{Attn}(h))$. Requires careful warmup; training can be unstable.
- **Pre-LN** (GPT-2+, all modern LLMs): $h + \text{Attn}(\text{LayerNorm}(h))$. Stabilizes training; enables higher learning rates.
- **RMSNorm** (Llama [126], Mistral [175]): Simplified LayerNorm without mean-centering: $\text{RMSNorm}(x) = x/\text{RMS}(x) \cdot \gamma$. Slightly faster, same quality.

Why Normalization Matters for Deep Networks

Without normalization, activations tend to grow or shrink exponentially through layers (exploding/vanishing activations). A 128-layer transformer without LayerNorm would see magnitudes vary by $10^{30}\times$ between the first and last layer. Normalization constrains each layer's output to a predictable range, enabling stable gradient flow and allowing the optimizer to use consistent learning rates throughout the network.

1.3.10 Model Size Reference

The following table summarizes key architectural parameters for widely-used open-weight models (latest versions as of 2025), providing a quick reference for understanding scale and design choices.

Table 1.7: Architecture parameters for popular open-weight LLMs (2024–2025 generation).

Model	Params	Layers	d	Heads	KV Heads	Context
Llama-3.1 8B [126]	8B	32	4096	32	8	128K
Llama-3.1 405B [126]	405B	126	16384	128	8	128K
Llama-4 Maverick [4]	400B (17B active)	48	5120	40	8	1M
Mistral Large 2 [5]	123B	88	12288	96	8	128K
Qwen-2.5 72B [371]	72B	80	8192	64	8	128K
DeepSeek-V3 [73]	671B (37B active)	61	7168	128	MLA	128K

Note: Models marked with "active" parameters use Mixture-of-Experts (MoE) architecture—total parameters indicate model capacity, while active parameters reflect per-token compute cost. DeepSeek-V3 uses Multi-head Latent Attention (MLA) instead of standard GQA, compressing KV into a low-rank latent space.

1.3.11 Attention Pathologies

While the attention mechanism is powerful, it exhibits systematic failure modes that practitioners must understand—especially when scaling to long contexts or interpreting model behaviour.

Attention Sink

The phenomenon. Xiao et al. [415] discovered that transformer models allocate disproportionately high attention scores to the *first token* in the sequence—regardless of its semantic content. Even when the first token is a meaningless `<BOS>` marker, attention heads across all layers consistently attend to it, sometimes with 20–50% of total attention mass.

Why it happens. Softmax attention must produce a valid probability distribution ($\sum_j \alpha_j = 1$). When no key is particularly relevant to a query, the model needs a "dump" location for unused attention mass. During training, the first token becomes this default sink because it is always present and positionally predictable. It functions as a *no-op attention target*—the model has learned to route irrelevant attention there rather than distributing it unpredictably.

$$\alpha_{\text{sink}} = \frac{\exp(q^\top k_0/\sqrt{d})}{\sum_j \exp(q^\top k_j/\sqrt{d})} \gg \frac{1}{n} \quad \text{(even when } k_0 \text{ is semantically irrelevant)}$$

Consequences.

- **Streaming inference failure**: When using sliding-window KV caches, evicting the first token causes perplexity to spike catastrophically—the model loses its attention sink.
- **Misleading interpretability**: Naive attention visualizations suggest the first token is "important" when it is merely a mathematical artefact.
- **Context window waste**: The sink token occupies a KV cache slot without carrying useful information.

Solutions.

- **StreamingLLM** [415]: Always keep the first k tokens ("attention sinks") in the KV cache alongside the recent sliding window. Enables infinite-length generation with bounded memory.
- **Sink tokens by design**: Some models (e.g., Mistral) prepend dedicated sink tokens during training that are explicitly meant to absorb residual attention.
- **Softmax alternatives**: Replace softmax with ReLU attention or sigmoid gating, where zero attention is representable without requiring a dump target.

Attention Dilution

The phenomenon. As sequence length n grows, each query must distribute its attention budget across more keys. The average attention weight per token decreases as $O(1/n)$, making it progressively harder for the model to concentrate on the few truly relevant positions—a problem known as *attention dilution* or *attention diffusion* [238].

The "Lost in the Middle" effect. Liu et al. [238] showed that LLMs exhibit a U-shaped retrieval curve: information placed at the *beginning* or *end* of long contexts is retrieved reliably, but information in the *middle* is often ignored. This is a direct consequence of attention dilution compounded with positional biases from RoPE/ALiBi:

Why it happens.

- **Softmax saturation**: With many keys, the softmax temperature effectively decreases, making the distribution more uniform (entropic).
- **Positional decay**: RoPE's relative positional encoding introduces a natural decay with distance, suppressing attention to middle positions that are far from both start and end.
- **Training distribution**: Models trained on shorter sequences develop attention patterns biased toward recent context.

Mitigation strategies.

- **Explicit retrieval**: Place relevant context at the beginning or end of the prompt; use RAG to avoid relying on middle positions.
- **Long-context training**: Train on long documents with varied placement of key information [106].
- **Hierarchical attention**: Architectures like Mamba [131] or RWKV that avoid the $O(n^2)$ attention bottleneck entirely.
- **Landmark tokens**: Insert retrievable markers in the context that act as "signposts" for attention.
- **Temperature scaling**: Some implementations scale the attention logits by $\log n$ to counteract dilution in long sequences.

Other Attention Phenomena

1.3.12 Visualizing Attention for Explainability

Attention weights provide a window into model reasoning—but must be interpreted carefully.

Table 1.8: Additional attention patterns observed in large transformers.

Pattern	Description	Implication
Attention heads specialization	Different heads learn distinct roles: syntax heads, co-reference heads, positional heads [382]	Not all heads are equally important; many can be pruned
Induction heads	Heads that implement [A][B]...[A] → [B] copying [288]	Critical for in-context learning; emerge in 2-layer+ models
Attention collapse	In deep networks, attention distributions can converge (all heads attend same positions)	Hurts expressivity; addressed by attention diversity losses
Retrieval heads	Specific heads specialize in retrieving factual information from context [409]	Explains why pruning certain heads causes hallucination spikes

Attention Visualization Methods

Raw attention maps. The simplest approach: plot the $n \times n$ attention matrix $A = \text{softmax}(QK^\top/\sqrt{d})$ as a heatmap for each head and layer. Tools like BertViz [380] render interactive multi-head visualizations.

Attention rollout. Raw attention at a single layer is misleading because information flows through residual connections across *all* layers. Abnar and Zuidema [2] propose *attention rollout*: multiply attention matrices across layers to approximate the total information flow from input to output:

$$R^{(l)} = A^{(l)} \cdot R^{(l-1)}, \quad R^{(0)} = I$$

where $A^{(l)}$ is the (averaged across heads) attention matrix at layer l, adjusted to include the residual connection: $A^{(l)} = 0.5 \cdot A^{(l)}_{\text{raw}} + 0.5 \cdot I$.

Gradient-weighted attention. Combine attention weights with gradient information to identify which attended tokens actually *influence* the output [22]:

$$\text{Relevance}(i) = \alpha_i \cdot \left| \frac{\partial y}{\partial h_i} \right|$$

This addresses the criticism that high attention ≠ high influence (a token can receive high attention but be processed through a near-zero-weight path).

Attention Is Not Explanation

Jain and Wallace [170] showed that attention weights often do not correlate with gradient-based feature importance and that adversarial attention distributions can produce identical outputs. Use attention visualization as a *hypothesis generator*, not as a faithful explanation. For causal attribution, prefer gradient-based methods, probing, or mechanistic interpretability.

Mechanistic Interpretability with Sparse Autoencoders (SAEs)

The interpretability problem. Individual neurons in transformer MLPs and residual streams are typically *polysemantic*—a single neuron activates for multiple unrelated concepts (e.g., "the colour blue AND academic citations AND the word 'the'"). This makes direct neuron-level interpretation unreliable.

Sparse Autoencoders. Cunningham et al. [65] and Bricken et al. [32] demonstrated that training a sparse autoencoder (SAE) on model activations can decompose polysemantic representations into *monosemantic features*—interpretable directions that each correspond to a single concept:

$$h = W_{\text{dec}} \cdot \text{ReLU}(W_{\text{enc}} \cdot x + b_{\text{enc}}) + b_{\text{dec}}$$

where $W_{\text{enc}} \in \mathbb{R}^{m \times d}$ with $m \gg d$ (overcomplete basis), and the ReLU + sparsity penalty ensures only a few features activate per input.

Key findings from SAE interpretability:

- Features are *monosemantic*: each encodes a single human-interpretable concept ("code in Python," "mentions of the Golden Gate Bridge," "first-person narrative") [32].
- Features are *steerable*: clamping a feature's activation high/low directly controls model behaviour (e.g., forcing the "Golden Gate Bridge" feature on makes the model mention it in every response) [374].
- Features compose: complex behaviours emerge from combinations of simple features.
- SAEs scale: Templeton et al. [374] trained SAEs with up to 34M features on Claude 3 Sonnet, finding interpretable features for safety-relevant concepts (deception, sycophancy, dangerous requests).

SAE Training Recipe

1. Collect activations from a specific model layer across a large corpus.
2. Train a sparse autoencoder with L_1 penalty on the hidden layer: $\mathcal{L} = \|x - \hat{x}\|_2^2 + \lambda \|z\|_1$.
3. The learned encoder directions (W_{enc} rows) are candidate features.
4. Validate: for each feature, find max-activating examples and check semantic coherence.
5. Optionally: measure *feature absorption* and *dead features* to assess SAE quality.

Natural Language Autoencoders (Anthropic, 2026)

While SAEs decompose activations into interpretable *vectors*, their features still require human inspection of max-activating examples to understand. Anthropic's Natural Language Autoencoders (NLAEs) [13] take a fundamentally different approach: they replace the sparse bottleneck with *natural language descriptions*, making interpretability automatic.

How NLAEs work.

1. **Encoder**: A language model reads the hidden activations (or the input text) and produces a natural language description of the active concepts: e.g., "The text discusses French cuisine and uses formal academic tone."
2. **Decoder**: A second language model reads the natural language description and reconstructs the original activations (or predicts the next token).
3. **Training**: Both encoder and decoder are trained end-to-end to minimize reconstruction loss, with the bottleneck being a variable-length natural language string rather than a sparse vector.

Advantages over SAEs.

- **Self-interpreting**: Features are *literally* natural language—no manual labelling needed.
- **Compositional**: Can express complex, relational concepts ("a sarcastic response to a factual claim") that SAE features cannot represent as single directions.
- **Hierarchical**: Descriptions can capture both fine-grained (word-level) and coarse (document-level) properties in the same representation.
- **Auditable**: The bottleneck description is human-readable, enabling direct inspection of what information the model "thinks" is present.

Limitations. NLAEs introduce a language-model-in-the-loop, making them computationally expensive and potentially subject to the same faithfulness concerns as any model-generated explanation. They also cannot easily represent sub-symbolic features (geometric patterns, exact numerical values) that SAEs handle naturally as activation magnitudes.

The Interpretability Stack

Think of interpretability tools as a hierarchy:

1. **Attention maps**: "What is the model looking at?" (cheapest, least faithful)
2. **Probing classifiers**: "What information is encoded at this layer?"
3. **Sparse Autoencoders**: "What monosemantic features are active?" (scalable, requires human labelling)
4. **Natural Language Autoencoders**: "What does the model think is happening?" (self-interpreting, expensive)
5. **Causal tracing / patching**: "Which components actually cause this output?" (most faithful, most expensive)

Each level trades off between cost, scalability, and faithfulness of explanation.

1.4 Prediction Heads: What Transformers Output

The transformer body produces contextual hidden states $\mathbf{h}_t \in \mathbb{R}^d$ for each position. What we *do* with these hidden states—the **prediction head**—defines the task. The same transformer backbone can serve radically different purposes simply by swapping the head.

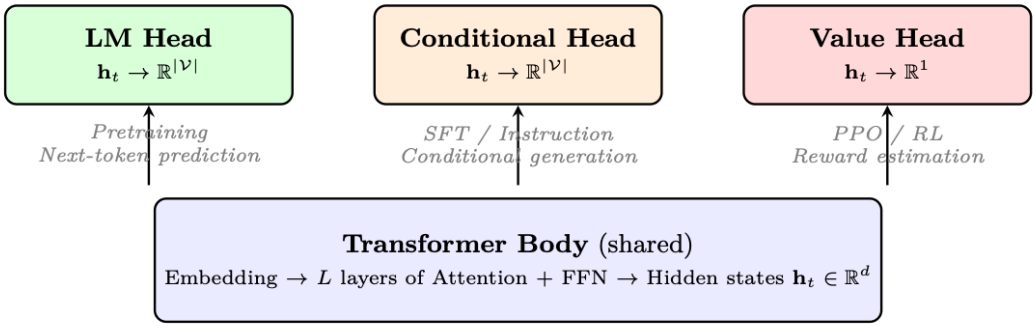


Figure 1.7: The same transformer backbone supports different tasks by swapping the prediction head. All three heads used in this paper share identical architecture below the final projection layer.

1.4.1 Language Modeling Head (Pretraining)

The standard LM head projects the final hidden state to vocabulary logits and trains with cross-entropy loss over the next token:

$$P(x_{t+1}|x_{\leq t}) = \text{softmax}(\mathbf{W}_{\text{head}} \cdot \mathbf{h}_t + \mathbf{b}) \tag{1.7}$$

where $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ (often tied with the embedding matrix: $\mathbf{W}_{\text{head}} = \mathbf{E}^T$).

LM Head Properties

- **Training objective**: Causal language modeling (predict next token for every position)
- **Loss**: $\mathcal{L}_{\text{LM}} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_t|x_{<t})$
- **Label**: Every token is both input (shifted right) and target (shifted left)
- **Used during**: Pretraining on large corpora (trillions of tokens)
- **Key insight**: The model learns general language understanding as a byproduct of next-token prediction

1.4.2 Conditional Generation Head (SFT / Instruction Following)

For supervised fine-tuning (SFT), the architecture is *identical* to the LM head—the same linear projection to vocabulary logits. The difference is purely in *what we compute loss on*:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|y|}\sum_{t=1}^{|y|} \log P(y_t|x_{\text{prompt}}, y_{<t}) \tag{1.8}$$

Conditional Head – Key Differences from LM Head

- **Loss masking**: Only compute loss on the *response* tokens, not the prompt/instruction. The prompt provides context but no gradient signal.
- **Conditioning**: The model learns to generate responses *conditioned on* specific instruction formats (system prompts, user queries, tool calls).
- **Format tokens**: Special tokens (`<|user|>`, `<|assistant|>`) guide the model to produce structured outputs.
- **Used during**: SFT on curated instruction-response pairs; also during RL policy generation (the policy head that produces actions/responses).

Same Head – Different Training Signal

The LM head and SFT head are architecturally identical (same $\mathbf{W}_{\text{head}}$). The only difference is that during SFT, we mask the loss on prompt tokens. This subtle change transforms a general text predictor into a instruction-following assistant. The head learns to "activate" different generation modes based on the conditioning context.

1.4.3 Value Head (Regression for RL)

In reinforcement learning (PPO, GRPO), we need to estimate *how good* a state is—this requires a scalar output, not vocabulary logits. The **value head** replaces the LM projection with a simple regression layer:

$$V(s_t) = \mathbf{w}_{\text{value}}^T \cdot \mathbf{h}_t + b \in \mathbb{R} \tag{1.9}$$

where $\mathbf{w}_{\text{value}} \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

Value Head Properties

- **Output**: Single scalar (expected cumulative reward from this state)
- **Loss**: MSE between predicted and actual returns: $\mathcal{L}_V = \frac{1}{T}\sum_t (V(s_t) - R_t)^2$
- **Architecture**: Linear layer $\mathbb{R}^d \to \mathbb{R}^1$ (sometimes with a small MLP: $d \to 256 \to 1$)
- **Backbone sharing**: Often shares the transformer body with the policy (with a separate value head), or uses a completely separate critic network
- **Used during**: PPO advantage estimation (GAE), reward model scoring

1.4.4 Head Selection Summary

Table 1.9: Prediction heads used throughout this paper and their training contexts.

Head	Output	Loss	Stage	Purpose
LM Head	$\mathbb{R}^{\lvert\mathcal{V}\rvert}$	Cross-entropy (all tokens)	Pretraining	Learn language from raw text
Conditional Head	$\mathbb{R}^{\lvert\mathcal{V}\rvert}$	Cross-entropy (response only)	SFT	Learn to follow instructions
Value Head	$\mathbb{R}^1$	MSE	RL (PPO)	Estimate state value for advantage
Reward Head	$\mathbb{R}^1$	Pairwise ranking	RM training	Score response quality

Head Initialization Matters

When adding a value head to a pretrained LM, initialize it near zero (small random weights). If initialized with large values, the initial value estimates will be wildly wrong, causing huge advantages and unstable PPO updates. Common practice: initialize the final linear layer with $\mathcal{N}(0, 1/\sqrt{d})$ or simply zeros.

Weight Tying: LM Head = Embedding Matrix Transposed

Most modern LLMs *tie* the LM head weights with the input embedding matrix: `lm_head.weight = model.embed_tokens.weight`. This means the LM head is *not* a separately learned layer—it reuses the embedding table. Benefits: fewer parameters ($|\mathcal{V}| \times d$ saved), better generalization, and the geometric structure of the embedding space directly determines token probabilities. You can verify this in HuggingFace: `model.lm_head.weight is model.model.embed_tokens.weight` returns `True` for most models.

1.4.5 HuggingFace Implementation

```python
from transformers import (
    AutoModelForCausalLM,          # LM head (pretraining + SFT)
    AutoModelForSequenceClassification,  # Reward head
    AutoTokenizer,
)
from trl import AutoModelForCausalLMWithValueHead  # Value head (PPO)
import torch

model_name = "meta-llama/Llama-3.1-8B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# === 1. LM Head (Pretraining / SFT) ===
# The default CausalLM model -- projects hidden states to vocab logits
lm_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
# lm_model.lm_head: Linear(hidden_size -> vocab_size)
# Output: logits of shape (batch, seq_len, vocab_size)

inputs = tokenizer("The capital of France is", return_tensors="pt")
outputs = lm_model(**inputs)
next_token_logits = outputs.logits[:, -1, :]  # (batch, vocab_size)
probs = torch.softmax(next_token_logits, dim=-1)

# === 2. Conditional Head (SFT) ===
# Architecturally identical to LM head -- difference is in loss masking
# During SFT, we only compute loss on response tokens:
messages = [
    {"role": "user", "content": "What is 2+2?"},
    {"role": "assistant", "content": "4"},
]
formatted = tokenizer.apply_chat_template(messages, return_tensors="pt")
labels = formatted.clone()
# Mask prompt tokens (set to -100 so cross-entropy ignores them)
prompt_len = len(tokenizer.apply_chat_template(messages[:1]))
labels[:, :prompt_len] = -100
loss = lm_model(input_ids=formatted, labels=labels).loss

# === 3. Value Head (PPO Critic) ===
# Adds a Linear(hidden_size -> 1) on top of the LM backbone
value_model = AutoModelForCausalLMWithValueHead.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
# value_model.v_head: Linear(hidden_size -> 1)
# Returns both LM logits AND per-token value estimates

inputs = tokenizer("Explain quantum computing", return_tensors="pt")
lm_logits, loss, values = value_model(
    **inputs, return_dict=False
)
# values shape: (batch, seq_len, 1) -- scalar estimate per token

# === 4. Reward Head (Reward Model) ===
# Classification head: Linear(hidden_size -> 1) on last token
reward_model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,              # single scalar output
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
```

```
# Scores entire sequence by pooling the last token's hidden state
inputs = tokenizer("Good response here", return_tensors="pt")
reward_score = reward_model(**inputs).logits  # shape: (batch, 1)
```

Listing 1.3: Loading and using different prediction heads with HuggingFace.

1.5 Optimization Theory for LLM Training

Training a large language model means finding the set of parameters θ (billions of weights) that minimizes the loss function $\mathcal{L}(\theta)$ — typically the negative log-likelihood of the next token. This is an optimization problem in extraordinarily high-dimensional space, and the algorithm used to navigate this space determines whether training succeeds, diverges, or stalls.

1.5.1 Gradient Descent: The Foundation

What is a Gradient? The gradient $\nabla_\theta \mathcal{L}$ is a vector that points in the direction of *steepest increase* of the loss. Each component $\frac{\partial \mathcal{L}}{\partial \theta_i}$ tells us how much the loss would change if we slightly increased parameter θ_i. To *decrease* the loss, we move in the opposite direction:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{1.10}$$

where $\eta > 0$ is the **learning rate** — the step size. This is **gradient descent** [328].

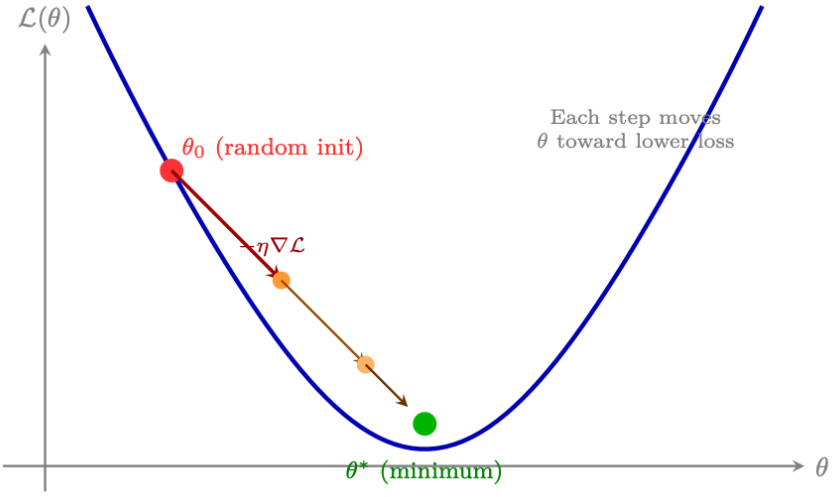


Figure 1.8: Gradient descent: starting from a random initialization θ_0, each step moves the parameters in the direction that reduces the loss, with step size controlled by the learning rate η. The process converges toward a (local) minimum.

Why Full Gradient Descent is Impractical. Computing the exact gradient requires evaluating the loss over the *entire* training dataset (trillions of tokens for LLMs). This is computationally prohibitive — a single gradient step would require a full pass over all data.

Stochastic Gradient Descent (SGD). The solution: estimate the gradient from a small random subset (**mini-batch**) of the data [326]:

$$\nabla_\theta \mathcal{L}(\theta) \approx \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x_i)$$

where B is the batch size (typically 1K–4M tokens for LLMs). The mini-batch gradient is a *noisy but unbiased* estimate of the true gradient.

Why Mini-Batch SGD Works

- **Computational efficiency**: Each step costs $O(B)$ instead of $O(N_{\text{total}})$. With $B = 4096$ tokens and 15T total tokens, each step is $\sim$4 billion$\times$ cheaper.
- **Noise as regularization**: The stochastic noise helps escape sharp local minima, finding flatter regions that generalize better.
- **GPU utilization**: Mini-batches are large enough to saturate GPU parallelism (matrix

multiplications become compute-bound rather than memory-bound).

- **Convergence**: Theoretically converges to a local minimum at rate $O(1/\sqrt{T})$ (slower than exact GD's $O(1/T)$, but each step is millions of times cheaper).

From SGD to Adaptive Methods. While SGD with momentum works well for vision models (CNNs), LLM training requires **adaptive optimizers** — algorithms that maintain a per-parameter learning rate.

1.5.2 Why Vanilla SGD Fails for LLMs

Stochastic Gradient Descent updates weights as:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

SGD Problems for LLMs

- **Different gradient scales per layer:** Early layers in a transformer have much smaller gradients than later layers (vanishing gradients). A single learning rate η is too large for some parameters and too small for others.
- **Sparse gradients:** Embedding layers receive gradients only for tokens in the current batch. Most embedding rows have zero gradient. SGD with momentum wastes momentum on zero-gradient rows.
- **Saddle points:** High-dimensional loss landscapes have many saddle points. SGD can stall; adaptive methods escape faster.
- **Sensitivity to learning rate:** SGD requires careful tuning; a $2\times$ change in η can cause divergence.

1.5.3 Adam – Adaptive Moment Estimation

Adam [195] maintains per-parameter estimates of the first moment (mean of gradients) and second moment (uncentered variance of gradients).

Adam Update Equations

Given gradient $g_t = \nabla_\theta \mathcal{L}(\theta_t)$, hyperparameters $\beta_1, \beta_2, \epsilon, \eta$:

Step 1 – Update biased first moment estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Step 2 – Update biased second moment estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Step 3 – Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Step 4 – Parameter update:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.95$ or 0.999, $\epsilon = 10^{-8}$, $\eta = 10^{-4}$ to 10^{-5}.

What Each Term Does

- m_t (**momentum**): Exponential moving average of gradients. Smooths out noisy gradient estimates. $\beta_1 = 0.9$ means the current gradient contributes 10% and the history contributes 90%.
- v_t (**adaptive LR**): EMA of squared gradients. Parameters with consistently large gradients get a smaller effective learning rate ($\eta/\sqrt{v_t}$). Parameters with small gradients get a larger effective LR. This is the key to handling different gradient scales per layer.
- $\hat{m}_t, \hat{v}_t$ (**bias correction**): At $t = 1$, $m_1 = (1 - \beta_1)g_1$ is much smaller than the true mean. Dividing by $(1 - \beta_1^t)$ corrects this initialization bias. Without it, early steps are too small.
- ϵ (**numerical stability**): Prevents division by zero. Also acts as a floor on the effective learning rate.

1.5.4 AdamW – Decoupled Weight Decay

AdamW [250] fixes a subtle but important issue with how weight decay interacts with adaptive optimizers.

Why L2 Regularization ≠ Weight Decay in Adam

With L2 regularization, the loss becomes $\mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$, so the gradient is $g_t + \lambda\theta_t$. In Adam, this regularization gradient is *scaled by the adaptive factor* $1/\sqrt{\hat{v}_t}$:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t + \lambda\theta_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Parameters with large v_t (large gradient variance) get *less* regularization. This is not what we want – weight decay should be uniform.

AdamW – Decoupled Weight Decay

AdamW (Loshchilov & Hutter, 2017) applies weight decay *directly* to the parameters, outside the adaptive scaling:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda\theta_t$$

The weight decay term $\eta\lambda\theta_t$ is not divided by $\sqrt{\hat{v}_t}$. This gives uniform regularization across all parameters regardless of their gradient history.
Typical value: $\lambda = 0.1$ for LLM training.

Always Use AdamW – Never Plain Adam – for LLMs

The difference between Adam and AdamW is subtle but matters. With Adam + L2, the effective weight decay is stronger for parameters with small gradient variance (e.g., biases, LayerNorm parameters) and weaker for parameters with large gradient variance (e.g., attention weights). AdamW gives the intended uniform regularization. Most frameworks default to AdamW; double-check your optimizer class.

1.5.5 Muon: Beyond AdamW

For nearly a decade, AdamW was the unchallenged default optimizer for neural network training. Muon [236] (Liu et al., 2025) is the first serious challenger to that dominance. Its core insight is that the momentum buffer accumulated by Adam carries directional information that is obscured by the adaptive scaling step—and that *orthogonalizing* that momentum before applying it produces a fundamentally better update direction.

Muon: Orthogonalized Momentum

Muon replaces the standard Adam update for hidden-layer weight matrices with a two-step procedure:

1. **Orthogonalize the momentum**: Apply Newton-Schulz iteration to the momentum buffer M_t to produce an approximately orthogonal matrix $\tilde{M}_t = \mathrm{NS}(M_t)$.
2. **Rescale to match AdamW RMS**: Scale $\tilde{M}_t$ so that its root-mean-square matches the RMS of the corresponding AdamW update, ensuring the effective step size is comparable.

The resulting update is applied to the weight matrix directly. Embedding layers, output projections, and scalar parameters continue to use AdamW.

The Newton-Schulz iteration is a matrix polynomial that converges to the orthogonal factor of the polar decomposition of M_t. Concretely, starting from $X_0 = M_t/\|M_t\|_F$, the iteration

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k X_k^\top X_k$$

converges rapidly (typically 5–10 steps) to a matrix $\tilde{M}_t$ satisfying $\tilde{M}_t^\top \tilde{M}_t \approx I$. Orthogonality ensures that the update moves weight matrices in a direction that is maximally spread across all singular value directions—preventing the optimizer from collapsing updates onto a low-rank subspace, which is a known failure mode of Adam on weight matrices with highly skewed singular value spectra.

Empirical efficiency. Muon claims approximately **2× compute efficiency** relative to AdamW: the same validation loss is reached in roughly half the gradient steps. This is not a marginal improvement—it represents a qualitative shift in the optimizer's sample efficiency on the loss landscape.

Adoption in frontier models. The momentum is clearly shifting toward Muon in large-scale pretraining:

- **GLM-4.5 / GLM-5** (Zhipu AI) [457, 458]: adopted Muon for pretraining
- **Kimi K2** (Moonshot AI) [18]: uses **MuonClip**, a variant that adds *QK-Clip*—rescaling query and key projections to cap attention logits and prevent loss spikes. Kimi K2 trained over 15.5T tokens with zero loss spikes, a stability record for models at that scale.
- **DeepSeek-V4** (2026) [74]: adopted Muon for its pretraining run

AdamW Is Still the Safe Default

AdamW remains the optimizer of choice for MAI-Thinking-1, Qwen, Llama, and most production fine-tuning pipelines. Muon's advantages are most pronounced in large-scale pretraining of weight matrices; its behavior on LoRA adapters, small fine-tuning runs, and RL training (PPO/GRPO) is less well characterized. Use AdamW unless you are running a pretraining run at scale and have the infrastructure to validate Muon's behavior on your specific architecture.

1.5.6 Learning Rate – The Most Important Hyperparameter

Typical Learning Rates by Training Phase

Phase	Typical LR	Notes
Pretraining (from scratch)	1e-4 to 3e-4	Large model, large batch
Continued pretraining	1e-5 to 1e-4	Smaller LR to preserve knowledge
SFT (supervised fine-tune)	1e-5 to 2e-5	Standard range
LoRA fine-tuning	1e-4 to 3e-4	Higher LR for adapter weights

For RL learning rates (PPO, DPO, GRPO) see §11.15.

1.5.7 Learning Rate Warmup

Why Warmup is Necessary

At the start of training, v_t (the second moment estimate) is initialized to zero. After bias correction: $\hat{v}_t = v_t/(1-\beta_2^t)$. At $t=1$ with $\beta_2 = 0.999$: $\hat{v}_1 = v_1/(1-0.999) = 1000v_1$. This means the effective learning rate is $\eta/\sqrt{1000v_1}$ – much smaller than intended.
However, if the first gradient is unusually large (common at initialization), the second moment estimate can be dominated by this outlier, causing erratic early steps. Warmup mitigates this by starting with a very small LR and gradually increasing it, giving v_t time to accumulate a reliable estimate.

- **Linear warmup:** $\eta_t = \eta_{\max} \times t/T_{\text{warmup}}$
- **Typical warmup duration:** 1–5% of total steps for pretraining; 3–10% for fine-tuning (shorter runs need proportionally more warmup)
- **For SFT:** 50–200 warmup steps is typical

1.5.8 Learning Rate Schedules

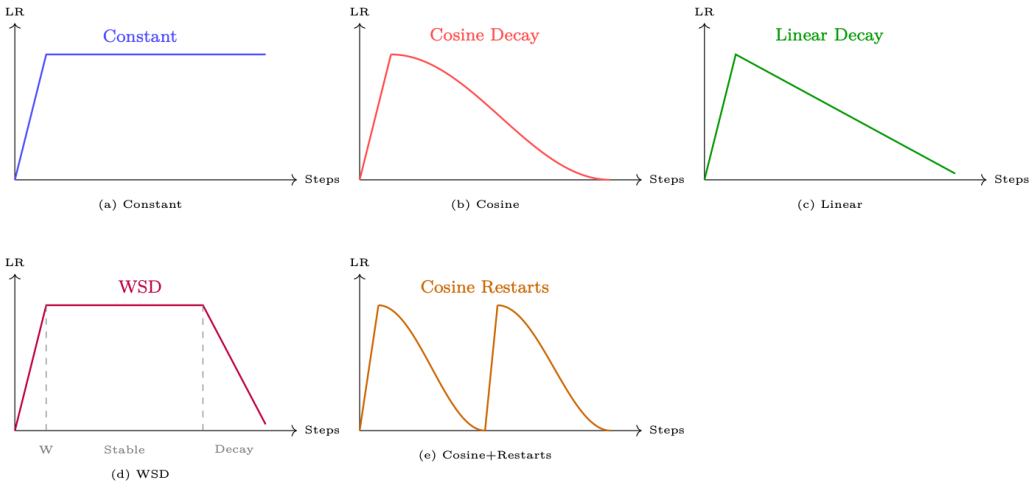


Figure 1.9: Common learning rate schedules. All include a linear warmup phase. WSD (Warmup-Stable-Decay) is the emerging standard for pretraining.

(a) Constant. Simplest schedule. Good for short fine-tuning runs where you want to avoid over-decaying the LR. Risk: no annealing means the model may not converge to the sharpest minimum.

(b) Cosine Decay.

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}\pi\right)\right)$$

Standard for pretraining and SFT. Smooth decay avoids abrupt LR changes. $\eta_{\min}$ is typically $\eta_{\max}/10$.

(c) Linear Decay. Simpler than cosine, similar empirical results. Preferred when you want predictable LR at any step.

(d) WSD – Warmup-Stable-Decay. The new standard for large-scale pretraining [126, 160]. Three phases:

1. **Warmup:** Linear ramp to $\eta_{\max}$ (1–5% of steps)
2. **Stable:** Constant $\eta_{\max}$ for the majority of training
3. **Decay:** Fast cosine or linear decay to $\eta_{\min}$ (last 10–20% of steps)

Key advantage: the stable phase allows checkpointing at any point and continuing training. The decay phase can be applied at the end of any run.

(e) Cosine with Restarts (SGDR). Periodic restarts reset the LR to $\eta_{\max}$. Can help escape local minima. Less common for LLMs; more useful for smaller models.

1.5.9 Gradient Clipping

Gradient Clipping

Gradient clipping rescales the gradient if its global norm exceeds a threshold:

$$g_t \leftarrow g_t \cdot \min\left(1, \frac{\tau}{\|g_t\|_2}\right)$$

where τ is `max_grad_norm` (typically 1.0).

Gradient Clipping vs. LR Reduction

Gradient clipping and reducing the learning rate both limit the size of parameter updates. The difference: clipping preserves the *direction* of the gradient (just scales the magnitude), while a smaller LR scales all updates uniformly. Clipping is better for handling occasional large gradients without slowing down normal training steps.

Putting It Together: HuggingFace Optimizer Configuration

The following snippet shows how the concepts from this section—AdamW with decoupled weight decay (§1.6.6), cosine learning-rate scheduling with linear warmup (§1.6.7), and gradient clipping (§1.6.8)—come together in practice using the HuggingFace `transformers` library.

```
from transformers import TrainingArguments, Trainer
from transformers import get_cosine_schedule_with_warmup
import torch

# --- Option 1: Using TrainingArguments (recommended) ---
training_args = TrainingArguments(
    output_dir="./checkpoints",

    # AdamW optimizer (decoupled weight decay, S1.6.6)
    optim="adamw_torch",
```

```
    learning_rate=2e-5,           # peak LR after warmup
    adam_beta1=0.9,               # first moment decay
    adam_beta2=0.999,             # second moment decay
    adam_epsilon=1e-8,            # numerical stability
    weight_decay=0.01,           # decoupled L2 penalty

    # Learning rate schedule (S1.6.7)
    lr_scheduler_type="cosine",  # cosine decay to 0
    warmup_ratio=0.1,            # 10% of steps = linear warmup

    # Gradient clipping (S1.6.8)
    max_grad_norm=1.0,           # clip by global L2 norm

    # Mixed precision (S1.6.9)
    bf16=True,                   # use BFloat16 on Ampere+ GPUs

    # Training duration
    num_train_epochs=3,
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,  # effective batch = 8*4 = 32
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
)
trainer.train()

# --- Option 2: Manual control (for custom training loops) ---
from torch.optim import AdamW

# Separate weight-decay groups (don't regularize biases/norms)
no_decay = ["bias", "LayerNorm.weight", "layernorm.weight"]
param_groups = [
    {
        "params": [p for n, p in model.named_parameters()
                   if not any(nd in n for nd in no_decay)],
        "weight_decay": 0.01,
    },
    {
        "params": [p for n, p in model.named_parameters()
                   if any(nd in n for nd in no_decay)],
        "weight_decay": 0.0,
    },
]

optimizer = AdamW(param_groups, lr=2e-5, betas=(0.9, 0.999))

# Cosine schedule with linear warmup
total_steps = len(train_dataloader) * num_epochs
warmup_steps = int(0.1 * total_steps)
scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)

# Training loop with gradient clipping
for batch in train_dataloader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()

    # Clip gradients before optimizer step
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

```
    optimizer.step()
    scheduler.step()
    optimizer.zero_grad()
```

Listing 1.4: Complete optimizer configuration combining AdamW – cosine schedule – and gradient clipping.

Practical Tips

- **Weight decay exclusion**: bias terms and layer-norm weights should not be regularized—they have few parameters and regularizing them hurts performance [250].
- **Warmup ratio**: 5–10% of total steps is standard; too little warmup with a high LR can destabilize early training.
- **Gradient accumulation**: simulates larger batches on limited GPU memory; clipping applies to the *accumulated* gradient.
- **BF16 vs. FP16**: prefer `bf16=True` on Ampere+ GPUs (wider dynamic range avoids loss scaling); fall back to `fp16=True` on older hardware.

1.5.10 Mixed Precision Training

BF16 vs. FP16

Format	Exponent bits	Mantissa bits	Dynamic range
FP32	8	23	$\sim 10^{-38}$ to 10^{38}
BF16	8	7	Same as FP32 (same exponent)
FP16	5	10	$\sim 6 \times 10^{-5}$ to 65504

BF16 Over FP16: Why Range Beats Precision in LLM Training

BF16 has the same exponent range as FP32, so it can represent the same range of values (just with less precision in the mantissa). FP16 has a much smaller dynamic range – gradients or activations that exceed 65504 cause overflow (NaN/Inf). This is why FP16 training requires *loss scaling* (multiplying the loss by a large constant to keep gradients in FP16 range), while BF16 training typically does not. A100 and H100 support BF16 natively; use BF16 unless you have a specific reason for FP16.

Loss Scaling (FP16 only).

1. Multiply loss by scale factor S (e.g., $S = 2^{15}$)
2. Compute gradients in FP16 (scaled by S)
3. Before optimizer step, divide gradients by S
4. Check for overflow (NaN/Inf); if found, skip step and reduce S
5. If no overflow for N consecutive steps, increase S

FP32 Master Weights. In mixed precision training, weights are stored in FP32 (master copy) and cast to BF16/FP16 for the forward/backward pass. The optimizer step is done in FP32. This is important because:

- Small gradient updates ($\Delta\theta \ll \theta$) would be lost in BF16 precision (7 mantissa bits $\approx 0.8\%$ relative precision)

- FP32 master weights ensure accurate accumulation of small updates over many steps
- Memory cost: 2× weight storage (FP32 + BF16 copy)

When FP32 Master Weights Are Critical

FP32 master weights are most important for:

- Long training runs (many small gradient steps accumulate)
- Small learning rates (updates are tiny relative to weight magnitude)

For short SFT runs with large LR, BF16-only training (no FP32 master weights) often works fine and saves memory. For RL training, FP32 master weights are essential—see §11.15.

Mixed Precision in Practice: HuggingFace

```
# === HuggingFace TrainingArguments (simplest approach) ===
from transformers import TrainingArguments

# BF16 on Ampere+ GPUs (A100, H100, RTX 30xx/40xx)
args_bf16 = TrainingArguments(
    output_dir="./out",
    bf16=True,                # BF16 forward/backward; FP32 master weights
    bf16_full_eval=True,      # also use BF16 during evaluation
    # No loss scaling needed -- BF16 has FP32-equivalent range
)

# FP16 on older GPUs (V100, T4, RTX 20xx)
args_fp16 = TrainingArguments(
    output_dir="./out",
    fp16=True,                # FP16 forward/backward
    fp16_full_eval=False,     # keep eval in FP32 for accuracy
    # Loss scaling is automatic via PyTorch GradScaler
)

# === Manual PyTorch AMP (for custom training loops) ===
import torch

# Setup (PyTorch 2.x API)
use_fp16 = not torch.cuda.is_bf16_supported()
scaler = torch.amp.GradScaler("cuda", enabled=use_fp16)  # only needed for FP16
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
dtype = torch.float16 if use_fp16 else torch.bfloat16

for batch in train_dataloader:
    optimizer.zero_grad()

    # Autocast: run forward pass in reduced precision
    with torch.autocast("cuda", dtype=dtype):
        outputs = model(**batch)
        loss = outputs.loss

    if use_fp16:
        # FP16 path: scale loss to prevent gradient underflow
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)          # unscale before clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)              # skips step on overflow
        scaler.update()                     # adjust scale factor
    else:
        # BF16 path: no scaling needed
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

```
        optimizer.step()

    scheduler.step()
```

Listing 1.5: Mixed precision training with HuggingFace and manual PyTorch AMP.

Key Differences: BF16 vs. FP16 in Code

- **BF16**: just wrap with `autocast(dtype=torch.bfloat16)`—no scaler needed. Simpler code and more numerically stable.
- **FP16**: requires `GradScaler` to prevent gradient underflow. The scaler dynamically adjusts a multiplier; if overflow is detected (NaN), the optimizer step is skipped and the scale is reduced.
- **Gradient clipping + FP16**: you *must* call `scaler.unscale_(optimizer)` before `clip_grad_norm_`, otherwise you're clipping scaled gradients (wrong threshold).
- **Memory savings**: % reduction in activation memory (activations stored in 16-bit); weight memory depends on whether you keep FP32 master copies.

FP8 and FP4: Next-Generation Training Precision. DeepSeek-V3 [73] demonstrated that 671B-parameter training in FP8 (E4M3 for forward pass, E5M2 for backward) with fine-grained tile-wise scaling achieves less than 0.25% relative loss degradation versus BF16—at a total training cost of approximately $5.6M. Key techniques include stochastic rounding to reduce quantization bias, per-tile scaling factors that adapt to local activation magnitudes, and keeping the first and last layers in higher precision where quantization sensitivity is greatest. NVIDIA's Nemotron 3 pushes further to NVFP4 (4-bit), demonstrating stability over 25T training tokens by maintaining approximately 15% of layers in BF16. The economic implication is profound: careful precision engineering reduces frontier training costs by 3–5×, making billion-parameter training accessible beyond the largest labs.

1.5.11 Practical Optimizer Settings by Training Phase

Optimizer Hyperparameter Reference Table

Phase	Optimizer	LR	WD	Warmup	Schedule
Pretraining	AdamW	3e-4	0.1	2000 steps	WSD or Cosine
SFT	AdamW	2e-5	0.01	100 steps	Cosine
LoRA SFT	AdamW	2e-4	0.01	100 steps	Cosine

All use: β_1=0.9, β_2=0.95, ϵ=10^{-8}, *max_grad_norm=1.0, BF16. For RL settings see §11.15.*

Diagnosing Training Instability

```
# Monitor these metrics to diagnose optimizer issues:
# 1. Gradient norm -- should be < max_grad_norm most of the time
# 2. Loss scale (FP16) -- should be stable, not constantly decreasing
# 3. Parameter update norm -- should be << parameter norm

import torch

def log_optimizer_stats(model, optimizer, step):
    # Gradient norm (before clipping)
    total_norm = 0.0
    for p in model.parameters():
        if p.grad is not None:
            total_norm += p.grad.data.norm(2).item() ** 2
    total_norm = total_norm ** 0.5
```

```
    # Adam second moment stats (proxy for adaptive LR)
    v_norms = []
    for group in optimizer.param_groups:
        for p in group['params']:
            state = optimizer.state[p]
            if 'exp_avg_sq' in state:
                v_norms.append(state['exp_avg_sq'].mean().item())

    print(f"Step {step}: grad_norm={total_norm:.3f}, "
          f"mean_v={sum(v_norms)/len(v_norms):.6f}")

# Red flags:
# grad_norm >> 1.0 repeatedly -> reduce LR or increase warmup
# grad_norm == 0.0 -> gradient vanishing or wrong loss
# loss_scale decreasing -> FP16 overflow, switch to BF16
# v very small -> Adam not warmed up yet, extend warmup
```

The Learning Rate is the Most Important Hyperparameter

In practice, getting the learning rate right matters more than any other hyperparameter. A rule of thumb for LLM fine-tuning:

- Start with the values in the table above
- If loss diverges (increases after initial decrease): LR is too high
- If loss decreases very slowly and plateaus early: LR is too low
- If loss is unstable (oscillates): LR is too high or warmup is too short

The second most important hyperparameter is batch size (affects gradient noise and effective LR via the linear scaling rule). Everything else is secondary.

1.6 Flash Attention – Algorithm and Hardware Awareness

Flash Attention [67, 68] is one of the most impactful algorithmic innovations in deep learning since the transformer itself. It does not change the mathematical result of attention – it computes *exactly* the same output – but it restructures the memory access pattern so that the GPU's limited fast SRAM does all the heavy lifting, cutting HBM footprint from $O(n^2)$ to $O(n)$ and delivering 2–4× end-to-end wall-clock gains on typical workloads.

1.6.1 The Standard Attention Memory Problem

Standard scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Standard Attention Memory Complexity

For sequence length n and head dimension d:

- $Q, K, V \in \mathbb{R}^{n\times d}$: $O(nd)$ memory
- $S = QK^T \in \mathbb{R}^{n\times n}$: $O(n^2)$ **memory** – the bottleneck
- $P = \text{softmax}(S) \in \mathbb{R}^{n\times n}$: another $O(n^2)$
- $O = PV \in \mathbb{R}^{n\times d}$: $O(nd)$

At $n = 8192$, $d = 128$, BF16: the attention matrix alone is $8192^2 \times 2 \approx 134$ MB *per head*. With 32 heads, that is 4.3 GB just for one layer's attention scores.

Why $O(n^2)$ is Catastrophic

The attention matrix must be written to HBM (it does not fit in SRAM for long sequences), then read back for the softmax, then read again for the PV product. Each of these HBM round-trips is slow. For $n = 32768$ (32K context), the attention matrix is $32768^2 \times 2 \approx 2$ GB *per head* – completely infeasible to store.

1.6.2 The Flash Attention Key Insight – Tiling and Online Softmax

The core insight is: **we never need the full $n \times n$ matrix in memory at once**. We can compute the output O block-by-block if we use the *online softmax* trick.

Online Softmax. Recall that softmax requires a global maximum for numerical stability:

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

The trick: we can *update* the running maximum and normalization factor as we process new blocks, without ever materializing the full row.

Online Softmax Update Rule

Given a running state $(m_{\text{old}}, \ell_{\text{old}}, O_{\text{old}})$ and a new block of scores s_{new}:

1. $m_{\text{new}} = \max(m_{\text{old}},\ \max(s_{\text{new}}))$
2. $\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}} + \sum_j e^{s_\text{new},j - m_\text{new}}$
3. $O_{\text{new}} = \frac{1}{\ell_{\text{new}}} \left(e^{m_\text{old} - m_\text{new}} \cdot \ell_\text{old} \cdot O_\text{old} + e^{s_\text{new} - m_\text{new}} \cdot V_\text{new} \right)$

This is mathematically equivalent to computing softmax over all blocks at once.

1.6.3 The Flash Attention Algorithm

Flash Attention Forward Pass – Block Tiling

Setup: SRAM size M. Block sizes $B_r = \lceil M/(4d) \rceil$, $B_c = \min(\lceil M/(4d) \rceil, d)$.

1. Divide Q into $T_r = \lceil n/B_r \rceil$ blocks $Q_1, \ldots, Q_{T_r}$
2. Divide K, V into $T_c = \lceil n/B_c \rceil$ blocks $K_1, \ldots, K_{T_c}$
3. Initialize output $O \in \mathbb{R}^{n \times d}$, running max $m \in \mathbb{R}^n$, running sum $\ell \in \mathbb{R}^n$ (all in HBM)
4. **Outer loop** over $j = 1, \ldots, T_c$:
   (a) Load K_j, V_j from HBM to SRAM
   (b) **Inner loop** over $i = 1, \ldots, T_r$:
      i. Load Q_i, O_i, m_i, ℓ_i from HBM to SRAM
      ii. Compute $S_{ij} = Q_i K_j^T / \sqrt{d}$ (stays in SRAM)
      iii. Apply online softmax update to get new m_i, ℓ_i, O_i
      iv. Write O_i, m_i, ℓ_i back to HBM
5. Return O

Key: S_{ij} (the attention tile) is computed and discarded in SRAM. It is *never written to HBM.*

Flash Attention Complexity

	Standard Attention	Flash Attention
Memory (HBM)	$O(n^2)$	$O(n)$
HBM reads/writes	$O(n^2 d)$	$O(n^2 d/M)$
FLOPs	$O(n^2 d)$	$O(n^2 d)$ (same)
Speedup	1×	2–4×

In the forward pass, the total FLOPs remain $O(n^2 d)$ – identical to standard attention. Flash Attention gains speed entirely by slashing slow HBM traffic, not by reducing arithmetic. (The backward pass actually performs *more* FLOPs due to recomputation, but the wall-clock time is still lower because the saved memory bandwidth dominates.)

1.6.4 Flash Attention 2 – Better Parallelism

Flash Attention 2 [67] made three key improvements:

1. **Reduced non-matmul FLOPs:** The original FA had unnecessary rescaling operations in the inner loop. FA2 restructures the loop to minimize these. On A100, Tensor Core matrix multiplications outpace scalar operations by roughly 16×, so even a small fraction of non-matmul work in the inner loop becomes the latency bottleneck.

2. **Better parallelism across sequence dimension:** FA1 parallelized over batch and heads only. FA2 also parallelizes over the query sequence dimension, enabling better GPU utilization for long sequences with small batch sizes.

3. **Causal masking optimization:** For autoregressive (causal) attention, roughly half the blocks are fully masked. FA2 skips these blocks entirely, giving ∼2× speedup for causal attention vs. bidirectional.

1.6.5 Flash Attention 3 – Hopper Architecture

Flash Attention 3 [340] is designed specifically for H100 and exploits three Hopper-specific features:

- **TMA (Tensor Memory Accelerator):** H100 has a dedicated hardware unit for asynchronous bulk data movement between HBM and SRAM. FA3 uses TMA to overlap data loading with computation, hiding memory latency.

- **Warp-specialization:** FA3 assigns different warps to different roles (producer warps load data via TMA; consumer warps compute MMA). This is a software pipelining technique that keeps both the memory system and Tensor Cores busy simultaneously.

- **FP8 support:** H100 supports FP8 (E4M3/E5M2) Tensor Core operations at 2× the throughput of BF16. FA3 supports FP8 attention with per-block quantization to maintain accuracy.

FA3 achieves up to **75% of H100 theoretical peak** for FP16 attention, compared to ∼35% for FA2.

1.6.6 Flash Attention 4 – Blackwell Architecture

Flash Attention 4 [438] targets NVIDIA's Blackwell GPUs (B200/GB200), which double Tensor Core throughput to 2.25 PFLOP/s (BF16) while non-matmul units (exponential, shared memory bandwidth) scale at a slower rate. This *asymmetric hardware scaling* means that the bottleneck shifts: on Blackwell, attention is limited not by matmul but by the softmax exponentials and shared memory traffic surrounding them.

FA4 addresses this with four key techniques:

- **Fully asynchronous MMA pipelines:** Blackwell's MMA instructions are fully asynchronous (unlike Hopper's wgmma which still blocked on completion). FA4 redesigns the pipeline to overlap MMA, TMA loads, and softmax rescaling across larger tile sizes, keeping all hardware units saturated.
- **Software-emulated exponential:** Instead of calling the hardware `ex2` unit (which is the throughput bottleneck), FA4 emulates e^x using polynomial approximations executed on the much faster Tensor Cores themselves. This trades extra matmul instructions for exponential-unit stalls.
- **Conditional softmax rescaling:** Standard FlashAttention rescales the running max every tile. FA4 skips the rescaling when the new tile's max does not exceed the running max (common in practice), saving both register shuffles and synchronization barriers.
- **Tensor Memory + 2-CTA MMA mode (backward pass):** The backward pass uses Blackwell's *Tensor Memory* (a per-SM scratchpad larger than shared memory) and a 2-CTA cooperative mode that fuses dQ accumulation across two thread-block clusters, halving shared memory round-trips.

FA4 Implementation: CuTe-DSL

FA4 is the first FlashAttention version written in **CuTe-DSL**, a Python-embedded domain-specific language for GPU kernels (part of CUTLASS 4.x). CuTe-DSL compiles 20–30× faster than C++ CUTLASS templates while retaining full control over register allocation and pipeline scheduling. This dramatically lowers the iteration time for kernel development.

Results. On B200 with BF16 head-dim 128 (causal, seq-len 8K):

- **1613 TFLOP/s** – 71% of Blackwell peak utilization
- **1.3**× faster than cuDNN 9.13 (NVIDIA's proprietary fused kernel)
- **2.7**× faster than Triton on the same hardware

Hardware–Software Co-evolution

The FlashAttention series illustrates a key principle: each GPU generation shifts the bottleneck, demanding new algorithmic ideas rather than just re-compilation. A80 → memory bandwidth limited (FA1/FA2: tiling + recomputation). H100 → data movement limited (FA3: TMA + warp-specialization). B200 → non-matmul compute limited (FA4: software-emulated exp + conditional rescaling). Understanding *where the hardware bottleneck lies* is the prerequisite for writing efficient kernels.

1.7 Pretraining: Best Practices

Pretraining is the most expensive phase of LLM development—consuming millions of GPU-hours and requiring careful orchestration of data, compute, and hyperparameters. This section distills key lessons from Llama-3 [126], Chinchilla [150], and GPT-4 [290].

1.7.1 Training Objective

All modern decoder-only LLMs use **causal language modeling** (CLM):

$$\mathcal{L}_{\text{CLM}} = -\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$$

This simple objective—with enough data and scale—produces emergent capabilities (in-context learning, reasoning, instruction following) without explicit supervision [34].

1.7.2 Data Pipeline

Pretraining Data Recipe

- **Scale**: 1–15 trillion tokens for frontier models (Llama-3: 15T tokens)
- **Sources**: Web crawl (80%), code (10%), books/papers (5%), curated (5%)
- **Deduplication**: MinHash + exact substring dedup reduces memorization [214]
- **Quality filtering**: Perplexity-based classifier, heuristic filters (length, language ID, toxicity)
- **Data mixing**: Temperature-weighted sampling across domains; upweight code and math for reasoning

1.7.3 Scaling Laws

Hoffmann et al. [150] showed that compute-optimal training requires balancing model size N and data size D: $N_{\text{opt}} \propto C^{0.50}$, $D_{\text{opt}} \propto C^{0.50}$. A 70B model is compute-optimal at ∼1.4T tokens. In practice, models are *over-trained* (more tokens than Chinchilla-optimal) because inference cost scales with model size, not training tokens—smaller over-trained models are cheaper to deploy.

1.7.4 Key Hyperparameters

Table 1.10: Pretraining hyperparameters from published models.

Setting	Llama-3 405B	Llama-3 8B	Qwen-2.5 72B	Mistral 7B
Tokens	15T	15T	18T	8T
Batch size (tokens)	16M	4M	4M	4M
Peak LR	8e-5	3e-4	3e-4	3e-4
Schedule	WSD	WSD	Cosine	Cosine
Weight decay	0.1	0.1	0.1	0.1
Context length	8192	8192	4096→32K	8192

1.7.5 Common Failure Modes

Pretraining Pitfalls

- **Loss spikes**: Sudden loss increases from bad data batches or numerical instability. Llama-3 reports rolling back to checkpoints and skipping offending batches.
- **Memorization**: Model regurgitates training data verbatim. Fix: deduplicate aggressively; monitor extraction attacks.
- **Context length**: Training on short sequences then deploying at long context fails. Use continued pretraining on long documents + RoPE scaling.

1.7.6 Mid-Training: Preparing for Reinforcement Learning

A distinct pipeline stage has emerged between raw pretraining and post-training. *Mid-training* (sometimes called "continued pretraining" or "annealing") up-weights STEM, mathematics, and code on high-quality curated data while extending context length to 128K–256K tokens. Its purpose is not to teach new knowledge but to *prepare the model for RL*: instilling the cognitive behaviors—verification, backtracking, subgoal decomposition—that reinforcement learning will later amplify.

Mid-Training Determines RL-Readiness

OctoThinker [248] demonstrated that mid-training composition determines whether a base model responds to RL at all. The same GRPO recipe produced strong reasoning improvements on Qwen (which has reasoning-dense mid-training) but failed entirely on Llama (which lacks it). The

difference was not model size or architecture—it was whether the base model already exhibited self-correction behaviors that RL could reinforce.

Typical mid-training uses 100B–500B tokens of curated high-quality data with a cosine-decayed learning rate. Llama 3's "annealing tail," MiMo's 3-stage mixture (ramping math+code to 70%), and MAI-Thinking-1's explicit STEM up-weight are all instances of this pattern.

1.8 Supervised Fine-Tuning (SFT)

SFT transforms a pretrained language model into an instruction-following assistant by training on curated prompt–response pairs. This is the bridge between raw language modeling and RLHF.

1.8.1 SFT Objective

The loss is identical to CLM, but computed only on **response tokens**:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|y|}\sum_{t=1}^{|y|}\log P_\theta(y_t \mid x_{\text{prompt}}, y_{<t})$$

Prompt tokens provide context but receive no gradient (labels set to -100).

1.8.2 Data Quality: The LIMA Principle

Zhou et al. [462] demonstrated that 1,000 carefully curated examples can match models trained on 50K+ noisy examples. Key requirements:

- **Diversity**: Cover QA, summarization, code, math, creative writing, multi-turn dialogue
- **Correctness**: Every response must be factually accurate and well-formatted
- **Length balance**: Mix short (1-sentence) and long (multi-paragraph) responses
- **Decontamination**: Remove overlap with evaluation benchmarks

1.8.3 Training Configuration

```
from trl import SFTTrainer, SFTConfig

sft_config = SFTConfig(
    output_dir="./sft_output",
    max_seq_length=4096,
    packing=True,              # Pack short examples into full sequences
    learning_rate=2e-5,
    lr_scheduler_type="cosine",
    warmup_ratio=0.1,
    weight_decay=0.01,
    max_grad_norm=1.0,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    bf16=True,
    gradient_checkpointing=True,
)
trainer = SFTTrainer(model=model, args=sft_config,
                     train_dataset=dataset, processing_class=tokenizer)
trainer.train()
```

1.8.4 Efficient Training Solutions

Standard HuggingFace training leaves significant performance on the table. Several libraries provide drop-in efficiency gains for SFT workloads:

Liger Kernel [155]. An open-source set of **Triton-fused kernels** from LinkedIn that replace standard PyTorch operators during training. Key fusions include:

- **Fused Cross-Entropy**: Merges the final linear projection, softmax, and loss computation into a single kernel—avoids materializing the full (batch × seq × vocab) logit tensor.
- **Fused RMSNorm / SwiGLU / RoPE**: Eliminates intermediate memory allocations for common LLM building blocks.
- **Chunked operations**: Processes large tensors in tiles to keep peak memory bounded.

Result: 20% higher throughput and up to 60% memory reduction with a one-line integration (`apply_liger_kernel_to_llama()`). Compatible with FSDP, DeepSpeed, and LoRA.

Unsloth [140]. A specialized fine-tuning library that combines **custom CUDA/Triton kernels** with aggressive memory optimization:

- Manual backpropagation through LoRA layers (avoids autograd overhead).
- 4-bit QLoRA with fused dequantization—trains 70B models on a single 48 GB GPU.
- Intelligent RoPE and attention kernel fusion specific to each architecture (Llama, Mistral, Qwen, Gemma).

Result: 2–5× faster than vanilla HuggingFace + PEFT, with 60–70% less VRAM. Particularly impactful for single-GPU and consumer-hardware workflows.

torchtune [370]. Meta's native PyTorch fine-tuning library (development wound down in 2025), designed around **composability** rather than monolithic abstractions:

- Pure PyTorch—no trainer class; recipes are readable single-file scripts.
- Native integration with `torch.compile`, FSDP2, and activation checkpointing.
- First-class support for QLoRA, full fine-tuning, and knowledge distillation.
- Built-in quantization-aware training (QAT) for post-training compression.

Result: Comparable speed to custom solutions but with full debuggability and no framework lock-in.

Choosing an Efficiency Stack

- **Quick LoRA/QLoRA on ≤1 GPU**: Unsloth (fastest time-to-train, minimal setup)
- **Multi-GPU full fine-tune**: TRL/DeepSpeed + Liger Kernel (best throughput at scale)
- **Research / custom training loops**: torchtune (transparent, hackable, native PyTorch)

These are *complementary*: Liger kernels can be used inside both TRL and torchtune workflows.

1.8.5 Best Practices

Table 1.11: SFT training guidelines.

Practice	Details
Packing	Concatenate multiple short examples into one sequence (separated by EOS). Avoids padding waste.
NEFTune [169]	Add uniform noise to embeddings ($\alpha = 5$). Improves MT-Bench by 5–15% at zero cost.
Chat template	Always use the model's native template. Mismatched templates degrade quality.
Epochs	2–3 for large datasets; up to 5 for small (<10K) curated sets. Over-training causes format memorization.

SFT Is Not Enough

SFT teaches format and basic instruction following, but cannot reliably teach: *preference* (which response is better—needs RLHF/DPO), *refusal* (when not to answer—needs safety training), *calibration* (saying "I don't know"—needs RL with truthfulness rewards), or *complex reasoning* (multi-step chains—needs RL with verifiable rewards). The full pipeline is: Pretrain → SFT → RLHF/DPO.

1.9 LoRA and Parameter-Efficient Fine-Tuning

Full fine-tuning of a 70B model requires storing 70B trainable parameters plus their optimizer states (560+ GB of memory). LoRA [156] (Low-Rank Adaptation) provides a way to fine-tune with <1% of the parameters while achieving comparable quality.

1.9.1 The LoRA Insight

LoRA Core Idea

Instead of updating a full weight matrix $W \in \mathbb{R}^{d \times d}$, learn a low-rank perturbation:

$$W' = W + \frac{\alpha}{r} \cdot BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times d}$$

- W is **frozen** (no gradients, no optimizer states)
- Only B and A are trained: $2 \times d \times r$ parameters instead of d^2
- At rank $r = 16$, $d = 4096$: LoRA adds $2 \times 4096 \times 16 = 131K$ params per layer vs. $16.8M$ for full matrix
- α/r scaling controls the magnitude of the update

Why Low-Rank Works

Aghajanyan et al. [3] showed that fine-tuning operates in a very low-dimensional subspace — the "intrinsic dimensionality" of the fine-tuning task is much smaller than the model's parameter count. A 175B model's fine-tuning task may have intrinsic dimensionality <10,000. LoRA exploits this directly: rank r constrains the update to an r-dimensional subspace per weight matrix.

Why the α/r Scaling Matters

Without scaling, doubling the rank r would roughly double the magnitude of $\Delta W = BA$ (more columns in B contribute to the sum). This means changing rank would also change how much the model is perturbed—you'd need to re-tune the learning rate every time you adjust r.

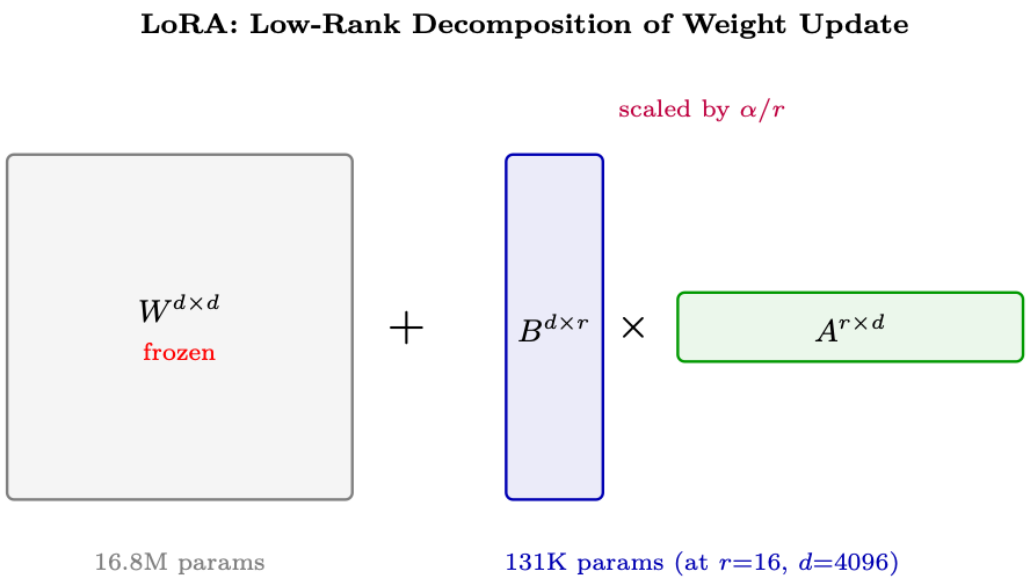


Figure 1.10: LoRA decomposes the weight update ΔW into two small matrices $B \times A$. The original weight W remains frozen; only B and A receive gradients. At inference, the product BA can be merged into W with zero overhead.

The α/r factor **normalizes the update magnitude** so that it stays approximately constant regardless of rank:

$$W' = W + \frac{\alpha}{r} \cdot BA$$

- **Fix α, sweep r**: The effective update magnitude stays $\sim \alpha$ regardless of rank. You can try $r \in \{8, 16, 32, 64\}$ without re-tuning LR.
- **Common practice**: Set $\alpha = r$ (so $\alpha/r = 1$) or $\alpha = 2r$ (so $\alpha/r = 2$). This is a convenient default where the scaling factor is a small integer.
- **Why not just tune LR?** You could, but α/r provides a *rank-independent* knob. Teams can share LR recipes across experiments with different ranks.
- **rsLoRA insight** [186]: At high ranks ($r \geq 64$), empirical evidence shows $\alpha/\sqrt{r}$ is more stable than α/r, because the variance of BA scales with $\sqrt{r}$, not r.

1.9.2 LoRA Hyperparameters

Choosing LoRA hyperparameters correctly is critical — the wrong rank or alpha can either under-fit (too constrained) or waste memory (too expressive).

Rank Selection Rules of Thumb

- **r=8**: Simple tasks (single-domain chat, classification). Very memory-efficient.
- **r=16**: General-purpose fine-tuning. Good default.
- **r=32–64**: Complex tasks (math, code, multi-turn reasoning). Approaches full fine-tune quality.
- **r=128+**: Diminishing returns; consider full fine-tuning or QLoRA with higher rank.
- **Key indicator**: If training loss plateaus well above full fine-tune loss, increase rank.

Table 1.12: LoRA hyperparameter guide.

Hyperparameter	Typical Values	Guidance
`r` (rank)	8, 16, 32, 64	Higher = more capacity but more memory. Start with 16.
`lora_alpha`	16, 32 (often $= r$ or $2r$)	Controls update magnitude via α/r scaling.
`target_modules`	`q_proj, k_proj, v_proj, o_proj`	All attention projections. Add `gate_proj, up_proj, down_proj` for full coverage.
`lora_dropout`	0.0–0.1	Regularization. Usually 0.05 for small datasets.
`bias`	`"none"`	Training biases adds minimal params but rarely helps.
Learning rate	1e-4 to 3e-4	Higher than full fine-tuning (only adapters update).

Table 1.13: LoRA variants and their innovations.

Method	Key Innovation	When to Use
QLoRA [79]	4-bit quantized base + LoRA in BF16. NF4 data type + double quantization.	Fine-tune 70B on single 48GB GPU.
DoRA [240]	Decomposes W into magnitude and direction; LoRA updates direction only.	Better generalization for reasoning.
LoRA+ [145]	Different LRs for A/B ($\eta_B = \lambda\eta_A$, $\lambda \approx 16$).	Free 2% gain; no extra cost.
AdaLoRA [447]	Dynamic rank budget across layers (SVD-based importance).	Very tight compute budget.
rsLoRA [186]	Scales by $\alpha/\sqrt{r}$ instead of α/r. Stable at high ranks.	When using $r \geq 64$.
VeRA [200]	Shared frozen random A, B; trains diagonal scaling only.	Extreme param efficiency.
LoRA-FA	Freezes A after init; only trains B. Halves LoRA memory.	Memory-constrained scenarios.

1.9.3 LoRA Variants

Key Extensions Explained

DoRA – Weight-Decomposed Low-Rank Adaptation. DoRA [240] observes that full fine-tuning tends to change the *direction* of weight vectors more than their magnitude. Standard LoRA conflates both. DoRA decomposes each weight column into magnitude $m = \|W\|_{\text{col}}$ and direction $\hat{V} = W/\|W\|_{\text{col}}$, then applies LoRA only to the direction:

$$W' = m \odot \hat{V}', \quad \hat{V}' = \frac{W + BA}{\|W + BA\|_{\text{col}}}$$

Magnitude m is a separate learnable vector (one scalar per column). This consistently outperforms LoRA by 1–3% on reasoning and instruction-following benchmarks with no additional inference cost (merged at deployment).

LoRA+ – Asymmetric Learning Rates. Hayou et al. [145] show that matrices A and B in LoRA have different optimal learning rates. Since B is initialized to zero, it starts in a very different regime than A (initialized from $\mathcal{N}(0, \sigma^2)$). Setting $\eta_B \approx 16 \times \eta_A$ improves convergence speed and final quality by $\sim$2% — a free gain requiring only a one-line config change:

```
# LoRA+ in PEFT: set different LRs per matrix
optimizer_grouped_parameters = [
    {"params": [p for n, p in model.named_parameters() if "lora_B" in n],
     "lr": 2e-4 * 16},   # B matrix: higher LR
    {"params": [p for n, p in model.named_parameters() if "lora_A" in n],
     "lr": 2e-4},         # A matrix: base LR
]
```

VeRA – Vector-based Random Matrix Adaptation. VeRA [200] takes parameter efficiency to the extreme: instead of learning A and B, it *freezes* them as shared random matrices across all layers and only trains two diagonal scaling vectors $d_b \in \mathbb{R}^r$ and $d_a \in \mathbb{R}^d$:

$$\Delta W = B \cdot \text{diag}(d_b) \cdot A \cdot \text{diag}(d_a)$$

This reduces trainable parameters by $\sim$10$\times$ vs. LoRA (only $r + d$ params per layer) while achieving 90–95% of LoRA quality. Best for scenarios where you need hundreds of task-specific adapters with minimal storage.

QLoRA Memory Savings

70B model full fine-tune: 140 GB (weights) + 280 GB (optimizer) + 140 GB (gradients) = 560 GB (7$\times$ A100-80GB).
70B QLoRA (r=16, all linear layers):

- Base model in NF4: 70B $\times$ 0.5 = 35 GB
- LoRA adapters in BF16: $\sim$160 MB
- Optimizer states (only for adapters): $\sim$320 MB
- Activations (gradient checkpointing): $\sim$8 GB
- **Total: $\sim$44 GB** — fits in a single 48GB GPU!

```
# QLoRA configuration with PEFT
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from transformers import BitsAndBytesConfig
import torch

# 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",           # NormalFloat4 - optimal for weights
    bnb_4bit_compute_dtype=torch.bfloat16, # Compute in BF16
    bnb_4bit_use_double_quant=True,       # Quantize the quantization constants
)

# LoRA config
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,                        # alpha/r = 2x scaling
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)
```

```
model = prepare_model_for_kbit_training(model)  # Prepare for QLoRA
model = get_peft_model(model, lora_config)       # Add LoRA adapters
model.print_trainable_parameters()
# Output: trainable params: 83,886,080 || all params: 70,553,706,496 || 0.12%
```

1.9.4 Other PEFT Approaches

LoRA dominates modern practice, but it is not the only parameter-efficient method. For completeness, the main alternatives:

Table 1.14: PEFT method families. LoRA is the de facto standard for LLM fine-tuning; the others are included for historical context and niche use cases.

Method	Mechanism	Pros / Cons	Status
LoRA [156] (and variants)	Low-rank matrices added to existing weights	Mergeable at inference (zero overhead); well-supported; works for all architectures	**Standard**
Adapters [154]	Small bottleneck MLPs inserted between layers	Modular; stackable; adds inference latency (extra sequential layers)	Rarely used
Prefix Tuning [222]	Learnable "virtual tokens" prepended to keys/values at each layer	No weight modification; effective for generation tasks; consumes context length	Niche
Prompt Tuning [215]	Learnable soft prompt embeddings prepended to input	Extremely few params (<0.01%); weaker than LoRA for complex tasks	Niche
IA3 [234]	Learned vectors that rescale keys, values, and FFN activations	Even fewer params than LoRA; mergeable; limited capacity	Deprecated
BitFit [440]	Train only bias terms	Near-zero params; surprisingly effective for simple tasks; limited expressiveness	Historical

Why LoRA Won

LoRA became the standard because it uniquely combines: (1) **zero inference overhead** — adapters merge into base weights, unlike Adapters or Prefix Tuning which add latency or consume context; (2) **composability** — multiple LoRA adapters can be swapped at serving time for multi-tenant deployments; (3) **ecosystem support** — HuggingFace PEFT, TRL, vLLM, and all major frameworks have first-class LoRA support; (4) **proven at scale** — used in production by Meta, Google, and most open-source fine-tunes on HuggingFace. Unless you have a specific constraint that LoRA cannot satisfy, it should be your default choice.

1.10 Mixture of Experts (MoE)

Mixture of Experts models [176, 343] scale model capacity without proportionally scaling compute cost by activating only a subset of parameters for each token.

1.10.1 Architecture

MoE Layer

In a MoE transformer, the FFN layer in each block is replaced by N parallel "expert" FFNs plus a **router** that selects which experts to use:

$$\text{MoE}(x) = \sum_{i=1}^{N} g_i(x) \cdot E_i(x), \quad g(x) = \text{TopK}(\text{softmax}(W_r x))$$

- E_i are expert networks (standard FFN layers)
- $g_i(x)$ are gating weights from the router (only top-K are non-zero)
- Typically $K = 2$ out of $N = 8$–64 experts are active per token
- Total params scale with N; **active params** scale with K/N of FFN size

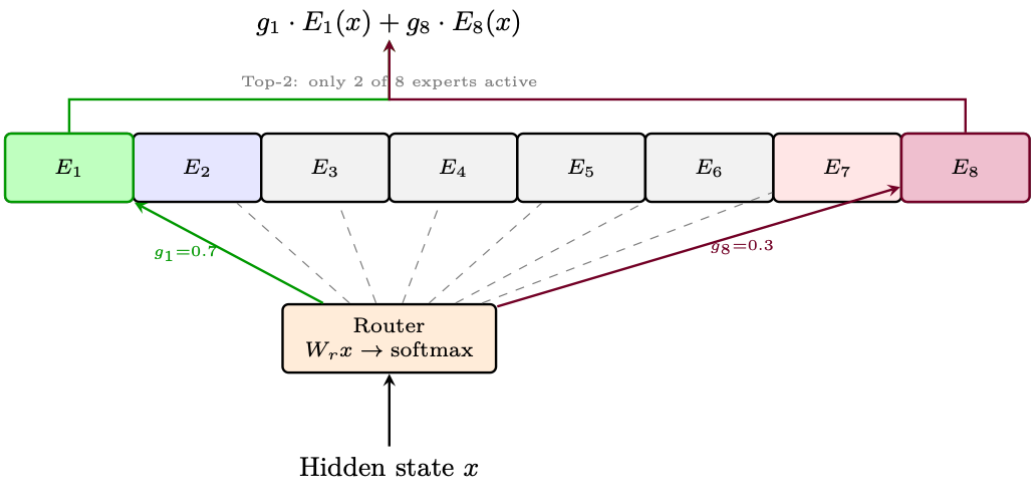


Figure 1.11: MoE layer with 8 experts and Top-2 routing. Only the two highest-gated experts are computed per token; the rest are skipped entirely.

1.10.2 Load Balancing

The Load Balancing Problem

Without constraints, the router may send most tokens to the same 1–2 experts ("expert collapse"). This wastes capacity and creates compute imbalance across GPUs (each expert typically lives on a different GPU).

Solution: Add an auxiliary load-balancing loss:

$$\mathcal{L}_{\text{bal}} = \alpha \cdot N \sum_{i=1}^{N} f_i \cdot p_i$$

where f_i = fraction of tokens routed to expert i, p_i = mean router probability for expert i. This encourages uniform expert utilization.

1.10.3 Noisy Top-K Gating: Making Discrete Routing Trainable

The core challenge in MoE is that **top-k selection is not differentiable** — you can't backpropagate through a hard "pick the top 2" operation. The field has developed two key tricks to solve this:

The Routing Differentiability Problem

The router computes logits $h(x) = W_r \cdot x$ for each expert, then selects the top-k. But:

- The *selected* experts get gradients through their gate weights (softmax over selected)
- The *selection decision itself* (which k to pick) has zero gradient
- Without a trick, the router can get stuck: an expert never selected → never gets a gradient signal → never gets selected

Approach 1: Noisy Top-K Gating [343]. Add learnable Gaussian noise to the router logits *before* the top-k selection:

$$
\begin{aligned}
h(x) &= W_g \cdot x && \text{(clean logits)} \\
H(x) &= h(x) + \epsilon \cdot \mathrm{Softplus}(W_{\text{noise}} \cdot x), \quad \epsilon \sim \mathcal{N}(0,1) && \text{(noisy logits)} \\
\mathrm{TopK}(v,k)_i &= \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \\ -\infty & \text{otherwise} \end{cases} && (1.11) \\
g(x) &= \mathrm{softmax}(\mathrm{TopK}(H(x),\, k)) && \text{(sparse gates)}
\end{aligned}
$$

- W_{noise} is a *learned* noise magnitude — the model learns how much exploration each expert needs
- During training, noise occasionally promotes "underdog" experts into the top-k, giving them gradient signal
- At inference, noise is removed: use clean logits $h(x)$ for deterministic routing
- The Softplus ensures noise scale is always positive

Approach 2: Gumbel-Softmax Trick (for differentiable discrete sampling). An alternative from the variational inference literature [171]. The **Gumbel-Max trick** provides exact sampling from a categorical distribution:

$$
z = \arg\max_i \left[\log \pi_i + G_i\right], \quad G_i \sim \mathrm{Gumbel}(0,1) \tag{1.12}
$$

where Gumbel noise is generated as $G_i = -\log(-\log(U_i))$, $U_i \sim \mathrm{Uniform}(0,1)$.

For **top-k routing**: taking the top-k of $(\log \pi_i + G_i)$ gives k samples *without replacement* from the categorical distribution defined by π.

Since arg max is non-differentiable, the **Gumbel-Softmax** relaxation replaces it with a temperature-controlled softmax:

$$
\hat{g}_i = \frac{\exp\left((\log \pi_i + G_i)/\tau\right)}{\sum_j \exp\left((\log \pi_j + G_j)/\tau\right)} \tag{1.13}
$$

- $\tau \to 0$: approaches a hard one-hot (exact but non-differentiable)
- $\tau \to \infty$: approaches uniform (differentiable but uninformative)
- In practice, anneal τ from 1.0 down to 0.1–0.5 during training
- **Straight-through estimator**: use hard top-k in the forward pass, but Gumbel-Softmax gradients in the backward pass — best of both worlds

Which Approach Is Used in Practice?

- **Sparsely-Gated MoE [343], Mixtral [176], DeepSeek-V2 [72]**: Use Noisy Top-K with Gaussian noise. Simple, effective, well-proven at scale.
- **Switch Transformer [94]**: Simplified to Top-1 with no noise (relies on load-balancing loss alone).
- **Research / smaller-scale MoE**: Some use Gumbel-Softmax for fully differentiable routing, especially when learning the routing itself is the research objective.
- **Key insight**: Both approaches solve the same problem (making discrete selection trainable) via noise injection. Gaussian noise is simpler; Gumbel noise has stronger theoretical guarantees for categorical sampling.

Auxiliary-Loss-Free Load Balancing. The standard approach adds a balance penalty to the training loss, but DeepSeek [393] demonstrated that this fights the routing signal: the loss pushes toward uniform utilization while the router tries to specialize. Their solution moves balancing entirely out of the gradient into a per-expert routing *bias* updated based on observed utilization—achieving both better quality *and* more expert specialization. A subsequent insight from Qwen revealed that the *aggregation scope* matters more than the mechanism: computing balance statistics per micro-batch (the common default) silently destroys specialization because micro-batches are too homogeneous. Aggregating over the full global batch—as MAI-Thinking-1 confirms—restores the diversity signal that experts need to differentiate.

1.10.4 Notable MoE Models

Model	Total Params	Active Params	Experts	Innovation
Switch Transformer [94]	1.6T	100B	128, Top-1	First large-scale MoE; simplified routing
Mixtral 8x7B [176]	47B	13B	8, Top-2	Open-weight; matches Llama-2 70B quality
DeepSeek-V2 [72]	236B	21B	160, Top-6	DeepSeekMoE with shared + routed experts
Qwen-MoE [371]	14.3B	2.7B	60, Top-4	Fine-grained experts for efficiency
DBRX [70]	132B	36B	16, Top-4	Fine-grained with 4 experts per block

1.11 Diversity in LLM Training

Diversity — in training data, model outputs, and optimization trajectories — is critical for preventing mode collapse and ensuring robust, general-purpose LLMs. This section covers the key diversity mechanisms applicable to all LLM training phases.

1.11.1 Sampling Diversity

Sampling Strategies for Diverse Generation

- **Temperature** τ: $P(x_i) \propto \exp(\text{logit}_i/\tau)$. Higher τ = more uniform distribution = more diverse. Typical: $\tau = 0.7$–1.0 for RLHF generation.
- **Top-k**: Only sample from the k highest-probability tokens. Prevents degenerate low-probability tokens.
- **Top-p (nucleus)**: Sample from the smallest set of tokens whose cumulative probability $\geq p$. Adaptive: more diverse when the model is uncertain.
- **Min-p**: Only keep tokens with $P \geq p_{\min} \times P_{\max}$. More principled than top-k.
- **Frequency/presence penalty**: Penalize tokens that appeared in the response. Encourages lexical diversity.

1.11.2 Training Data Diversity

- **Prompt diversity**: Cover different domains, difficulty levels, and formats. The Goldilocks principle: prompts should have 20–80% success rate.
- **Deduplication**: Remove near-duplicate training examples (MinHash, n-gram overlap). Duplicates cause overfitting to specific patterns.
- **Data mixing**: Balance across tasks/domains using temperature-weighted sampling or curriculum strategies.

1.11.3 Diversity-Promoting Methods

Method	How It Promotes Diversity
Temperature scaling	Higher τ flattens the distribution; more tokens become plausible.
Top-p / Min-p	Adaptive thresholds allow wider sampling when the model is uncertain.
Frequency penalty	Penalizes repeated tokens, forcing lexical variety within a response.
Data deduplication	Removing near-duplicates from training data prevents overfitting to specific patterns.
Multi-domain mixing	Temperature-weighted sampling across domains ensures broad coverage.
Verbalized sampling	Prompt the model to explicitly verbalize a probability distribution over responses [445]. See §7.5.

1.12 Text Generation: Decoding Methods

A trained language model outputs a probability distribution over the vocabulary at each step: $P(x_t|x_{<t})$. The **decoding strategy** determines how we select the next token from this distribution. This choice profoundly affects output quality, diversity, and coherence.

1.12.1 Greedy Decoding

The simplest strategy: always pick the highest-probability token.

$$x_t = \arg\max_{v \in \mathcal{V}} P(v|x_{<t})$$

Intuition: Like always taking the most obvious next word in a sentence. "The capital of France is..." → "Paris" (probability 0.92).

Pros: Deterministic, fast, no hyperparameters.

Cons: Produces repetitive, generic text. Misses high-quality sequences where an early low-probability token leads to a globally better output. No diversity.

1.12.2 Beam Search

Maintain B (beam width) partial hypotheses in parallel, expanding each by the top-k tokens and keeping the B highest-scoring complete sequences:

$$\text{score}(y_{1:t}) = \sum_{i=1}^{t} \log P(y_i | y_{<i})$$

With **length normalization** to avoid favoring short sequences:

$$\text{score}_{\text{norm}}(y) = \frac{1}{|y|^{\alpha}} \sum_{i=1}^{|y|} \log P(y_i | y_{<i}), \quad \alpha \in [0.6, 1.0]$$

Intuition: Like exploring multiple paths in a maze simultaneously, keeping only the B most promising ones at each junction.

Pros: Finds higher-likelihood sequences than greedy; good for translation and summarization where there's a single "correct" output.

Cons: Still tends toward generic, repetitive text for open-ended generation; $B\times$ more compute; all beams often converge to similar outputs.

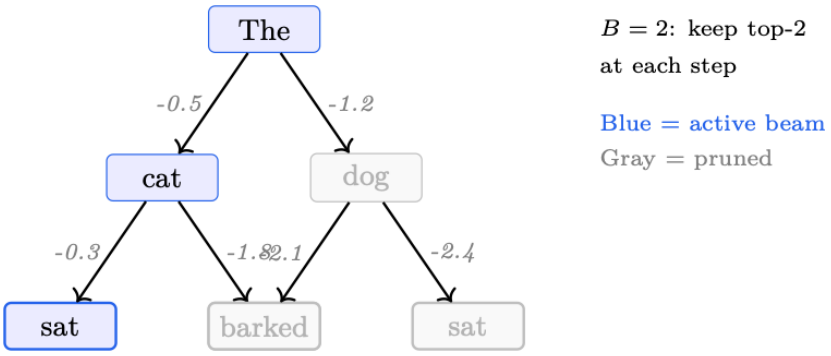


Figure 1.12: Beam search with $B = 2$. At each step, only the 2 highest-scoring partial sequences survive (blue). Lower-scoring alternatives are pruned (gray).

1.12.3 Diverse Beam Search

Standard beam search produces near-duplicate beams. Diverse beam search [381] partitions beams into G groups and adds a **dissimilarity penalty** between groups:

$$\text{score}_g(y_t) = \log P(y_t | y_{<t}) - \lambda \sum_{g' < g} \Delta(y_t, Y_{g'})$$

where Δ measures overlap (e.g., Hamming diversity) with tokens already selected by earlier groups, and λ controls diversity strength.

Intuition: Like forcing a brainstorming group to generate different ideas — each subgroup is penalized for repeating what earlier subgroups said.

Pros: Produces genuinely different candidate sequences; useful for reranking pipelines.

Cons: Diversity penalty can degrade individual beam quality; more hyperparameters (G, λ).

1.12.4 Top-k Sampling

Sample from only the k most probable tokens, redistributing probability mass:

$$P'(v|x_{<t}) = \begin{cases} \dfrac{P(v|x_{<t})}{\sum_{v' \in \text{Top-}k} P(v'|x_{<t})} & \text{if } v \in \text{Top-}k \\ 0 & \text{otherwise} \end{cases}$$

Intuition: After "The cat sat on the...", only consider the top k plausible continuations ("mat", "floor", "couch", ...) and ignore extremely unlikely ones ("quantum", "archipelago").

Pros: Removes tail noise; simple to implement.

Cons: Fixed k is too restrictive for peaked distributions (wastes probability mass) and too permissive for flat distributions (lets in garbage tokens).

1.12.5 Top-p (Nucleus) Sampling

Sample from the smallest set of tokens whose cumulative probability exceeds p:

$$\text{Top-}p = \min \left\{ S \subseteq \mathcal{V} : \sum_{v \in S} P(v|x_{<t}) \geq p \right\}$$

where tokens are sorted by descending probability and added until the threshold p is reached.

Intuition: Adaptively resize the candidate pool. If the model is confident ("Paris" at 95%), the nucleus is tiny. If uncertain ("The movie was..."), the nucleus expands to include many plausible adjectives.

Pros: Adapts to distribution shape; widely used default ($p = 0.9$–0.95).

Cons: Still includes some low-quality tokens at the tail of the nucleus; the threshold is a single global hyperparameter.

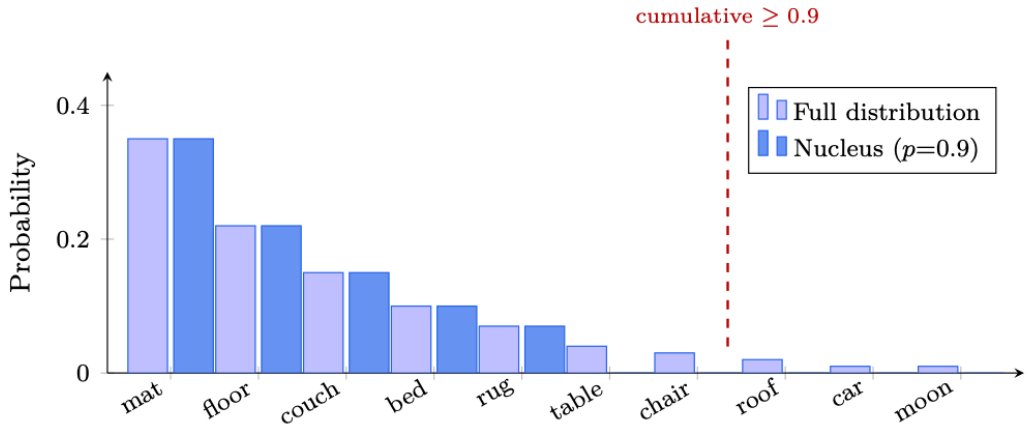


Figure 1.13: Top-p (nucleus) sampling: tokens are sorted by probability and included until cumulative mass reaches $p = 0.9$. The nucleus (dark blue) adapts its size to the distribution shape — here 5 tokens suffice.

Top-kk vs. Top-pp

Consider predicting the next word:

- After "2 + 2 =": distribution is peaked — top-1 token ("4") has 99% mass. Top-k=50 wastefully considers 49 wrong answers. Top-p=0.9 correctly picks just "4".
- After "I enjoy eating": distribution is flat — many foods are plausible. Top-k=5 is too restrictive. Top-p=0.9 might include 50+ tokens, matching the actual uncertainty.

Top-p adapts; top-k doesn't. In practice, both are often combined: sample from top-p intersected

with top-k.

1.12.6 Min-p Sampling

A recent alternative that sets a **relative** probability floor [281]:

$$\text{Min-}p = \left\{ v \in \mathcal{V} : P(v|x_{<t}) \geq p_{\min} \cdot \max_{v'} P(v'|x_{<t}) \right\}$$

Only tokens with probability at least $p_{\min}$ times the top token's probability are kept.

Intuition: "Only consider tokens that are at least 10% as likely as the best token." If the top token has probability 0.8, only tokens above 0.08 survive. If the top token has probability 0.05 (very uncertain), tokens above 0.005 survive — naturally expanding the pool.

Pros: Scales naturally with model confidence; fewer degenerate samples than top-p on peaked distributions; single intuitive parameter.

Cons: Newer, less battle-tested; not yet standard in all inference frameworks.

1.12.7 Temperature Scaling

Before applying any sampling strategy, logits are divided by temperature T:

$$P_T(v|x_{<t}) = \frac{\exp(z_v/T)}{\sum_{v'} \exp(z_{v'}/T)}$$

- $T < 1$: Sharpens distribution → more deterministic, focused outputs.
- $T = 1$: Unmodified model distribution.
- $T > 1$: Flattens distribution → more random, creative outputs.
- $T \to 0$: Becomes greedy decoding. $T \to \infty$: Becomes uniform sampling.

Common settings: $T = 0.7$ for factual tasks, $T = 1.0$–1.2 for creative writing, $T = 0.0$ (greedy) for code/math.

1.12.8 Contrastive Decoding

Contrastive decoding [221] exploits the difference between a strong model (expert) and a weak model (amateur) to amplify the expert's unique knowledge:

$$x_t = \arg \max_{v \in \mathcal{V}(x_{<t})} [\log P_{\text{expert}}(v|x_{<t}) - \log P_{\text{amateur}}(v|x_{<t})]$$

where $\mathcal{V}(x_{<t}) = \{v : P_{\text{expert}}(v|x_{<t}) \geq \alpha \cdot \max_{v'} P_{\text{expert}}(v'|x_{<t})\}$ is an adaptive plausibility constraint.

Intuition: The amateur model captures generic, obvious patterns (common words, repetition). Subtracting its log-probabilities removes this "generic signal," leaving the expert's distinctive knowledge and reasoning. Like removing background noise from a recording to hear the signal.

Pros: Reduces repetition and generic phrasing; improves factuality and coherence without additional training; works with any model pair.

Cons: Requires running two models (2× compute); sensitive to amateur model choice; the plausibility threshold α needs tuning.

1.12.9 Repetition Penalties

Orthogonal to the sampling strategy, repetition penalties discourage the model from repeating tokens. Given the raw logit z_v for token v (i.e., the unnormalized score output by the LM head *before* softmax), the penalized logit is:

$$z'_v = \begin{cases} z_v/\theta & \text{if } v \in \text{generated tokens and } z_v > 0 \\ z_v \cdot \theta & \text{if } v \in \text{generated tokens and } z_v < 0 \end{cases}$$

where $\theta > 1$ is the penalty factor (typically 1.1–1.3). In both cases, the effect is to push the logit toward zero—reducing the probability of previously generated tokens. Frequency and presence penalties are simpler additive variants used by OpenAI APIs:

$$z'_v = z_v - \alpha \cdot \text{count}(v) - \beta \cdot \mathbf{1}[v \in \text{generated}]$$

where α is the frequency penalty (proportional to how many times v appeared) and β is the presence penalty (flat penalty for any prior occurrence).

1.12.10 Practical Comparison

Table 1.15: Decoding method comparison for LLM text generation.

Method	Deterministic	Diversity	Quality	Best For
Greedy	Yes	None	Medium	Code, factual QA
Beam Search (B=4–8)	Yes	Low	High (narrow)	Translation, summarization
Diverse Beam Search	Yes	Medium	High	Candidate generation for reranking
Top-k (k=50)	No	Medium	Medium	General-purpose generation
Top-p (p=0.9)	No	Adaptive	High	Default for open-ended tasks
Min-p (p_{min}=0.1)	No	Adaptive	High	Robust alternative to top-p
Contrastive	Yes	Low	Very High	Factual, coherent long-form

Decoding in Practice: "Once upon a time"

Given the prompt "Once upon a time,":

- **Greedy**: "there was a young girl who lived in a small village..." (generic fairy tale)
- **Top-p=0.9, T=1.0**: "the rivers ran backwards and the fish learned to fly..." (creative, surprising)
- **Top-p=0.9, T=0.3**: "there was a kingdom ruled by a wise and just king..." (coherent, conventional)
- **Contrastive**: "in the amber-lit corridors of a collapsing star, two minds argued about the nature of time..." (distinctive, avoids clichés)

Same model, same prompt — decoding strategy determines the character of the output.

1.12.11 Constrained Decoding (Structured Generation)

All methods above sample from the *full* vocabulary at each step. **Constrained decoding** restricts the set of allowed tokens so that the output is *guaranteed* to conform to a formal grammar—typically a JSON schema, regex, or context-free grammar (CFG).

Core mechanism. At each decoding step t, a **token mask** $M_t \subseteq \mathcal{V}$ is computed from the current parser state. Only tokens in M_t receive their original logits; all others are set to $-\infty$ before softmax:

$$P'(v|x_{<t}) = \begin{cases} P(v|x_{<t})/Z & \text{if } v \in M_t \\ 0 & \text{otherwise} \end{cases}$$

where $Z = \sum_{v \in M_t} P(v|x_{<t})$ renormalizes. Because the mask changes every step (it depends on what has been generated so far), the constraint is enforced *incrementally*—the model cannot produce an invalid prefix at any point.

From schema to mask. The compilation pipeline is:

$$\text{JSON Schema} \xrightarrow{\text{compile}} \text{Regex} \xrightarrow{\text{compile}} \text{FSM (DFA)} \xrightarrow{\text{index}} \text{Token Mask per State}$$

The FSM states correspond to positions in the regex. For each state, all vocabulary tokens that would keep the string in the language are precomputed into an index (a one-time cost per schema). At runtime, looking up the mask is an $O(1)$ table access—adding negligible latency to each decoding step.

Key libraries.

- **Outlines** [400]: Compiles JSON schemas and regexes into interleaved FSM-guided generation. Supports any model with a logits interface.
- **lm-format-enforcer**[1]: Similar FSM approach with a focus on integration with serving frameworks (vLLM, TGI).
- **Guidance**[2] (Microsoft): Interleaves constrained generation with control flow (loops, conditions), enabling complex structured outputs beyond flat schemas.
- **XGrammar** [81]: Pushdown-automaton-based engine supporting full context-free grammars (not just regular languages), used in MLC-LLM and vLLM for grammar-mode decoding.

Trade-offs. Constrained decoding *guarantees* syntactic validity—no post-hoc parsing failures, no retries. However:

- **Semantic quality**: Forcing structure can degrade content quality if the model's probability mass for the "correct" answer lies outside the grammar. In practice this is rare for well-trained models on well-designed schemas.
- **Compilation cost**: The FSM index must be built per schema. For complex schemas this can take 1–5 s, but it is amortized over all requests using that schema.
- **Grammar coverage**: Regex/FSM handles JSON, YAML, SQL fragments, and most structured formats. Full CFGs (via XGrammar or LALR parsers) cover languages like Python or XML.

[1] https://github.com/noamgat/lm-format-enforcer
[2] https://github.com/guidance-ai/guidance

When to Use Constrained Decoding

Use constrained decoding whenever the consumer of the model's output is a program rather than a human. Tool-calling agents, API backends, and data-extraction pipelines all benefit from *guaranteed* valid structure. For free-form prose or creative text, unconstrained sampling remains appropriate.

1.13 Prompt Engineering

Prompt engineering is the discipline of designing inputs to LLMs that reliably elicit desired behaviour—without changing model weights. While fine-tuning modifies the model, prompt engineering exploits the model's *existing* capabilities through careful framing, examples, and structure. It is the fastest, cheapest, and most accessible lever for improving LLM outputs, and remains essential even when using fine-tuned models.

1.13.1 In-Context Learning (ICL)

In-context learning [34] is the remarkable ability of large language models to learn tasks at inference time purely from examples provided in the prompt—with no gradient updates. The model implicitly infers the task from the pattern of input–output pairs and generalizes to new inputs.

Why In-Context Learning Works

- **Implicit Bayesian inference**: The model has seen millions of tasks during pretraining. The prompt examples *locate* the relevant task in the model's learned distribution [418].
- **Induction heads**: Specific attention heads learn to copy patterns ("A is to B as C is to "), enabling in-context generalization [288].
- **Task vectors**: ICL creates implicit task representations in the residual stream that steer generation toward the demonstrated format and content [375].

Scaling behaviour. ICL emerges primarily in models above ∼1B parameters and improves log-linearly with model scale [34]. Smaller models can memorize examples but struggle to generalize to novel inputs within the same context window.

1.13.2 Zero-Shot Prompting

Zero-shot prompting provides *no* examples—only a task description or instruction. The model must rely entirely on its pretrained knowledge and instruction-tuning to produce the correct format and content.

Zero-Shot Classification

```
Classify the following movie review as POSITIVE or NEGATIVE.

Review: "The cinematography was breathtaking but the plot
felt rushed and predictable."

Sentiment:
```

When zero-shot works well:

- Tasks the model has seen extensively during pretraining/SFT (translation, summarization, sentiment)
- Well-specified instructions with unambiguous output format
- Instruction-tuned models (e.g., ChatGPT, Claude, Llama-3-Instruct) significantly outperform base models at zero-shot tasks [296]

When zero-shot fails: Novel formats, domain-specific labeling schemes, or ambiguous tasks where the model cannot infer your exact requirements from the instruction alone.

1.13.3 Few-Shot Prompting

Few-shot prompting [34] provides k input–output examples ("shots") before the actual query. This is the most common form of in-context learning and remains one of the most effective prompting strategies.

Few-Shot Named Entity Recognition

```
Extract named entities from the text. Format: [ENTITY](TYPE)

Text: "Apple released the iPhone 15 in Cupertino."
Entities: [Apple](ORG), [iPhone 15](PRODUCT), [Cupertino](LOC)

Text: "Elon Musk announced Tesla's new factory in Berlin."
Entities: [Elon Musk](PER), [Tesla](ORG), [Berlin](LOC)

Text: "OpenAI partnered with Microsoft to deploy GPT-4."
Entities:
```

Key design principles for few-shot examples:

1. **Diversity**: Cover the range of expected inputs (different lengths, edge cases, categories).
2. **Ordering**: Place harder or more representative examples last (recency bias) [253].
3. **Label balance**: If classifying, include examples from all classes to avoid majority-class bias.
4. **Format consistency**: Every example must follow the *exact* same structure. The model mimics the pattern.
5. **Relevance**: Use examples semantically similar to the target query for best results [235].

How many shots? Performance typically improves from 0 to 4–8 examples, then plateaus. Beyond ~20 examples, gains are marginal and you risk filling the context window. Min et al. [270] showed that the *format* and *label space* of examples matter more than label correctness—even random labels help (though correct labels help more).

1.13.4 Instruction-Following Prompts

Instruction-tuned models respond best to clear, structured instructions. The key insight: treat the prompt as a *specification*, not a suggestion.

Anatomy of an Effective Instruction Prompt

1. **Role/Persona**: Define who the model is ("You are a senior data scientist...")
2. **Task**: What to do, stated clearly and unambiguously
3. **Context**: Background information the model needs
4. **Constraints**: Length limits, tone, what to avoid, output format
5. **Examples** (optional): Show the desired output format
6. **Input**: The actual data to process

Instruction Prompt with Constraints

```
Role: You are a medical literature reviewer.

Task: Summarize the following research abstract for a
general audience.

Constraints:
- Maximum 3 sentences
- No jargon (explain any technical terms)
- Include the key finding and its clinical implication
- Do NOT speculate beyond what the abstract states

Abstract: [...]
```

System prompts vs. user prompts. Modern chat APIs separate the *system* prompt (persistent instructions, role definition) from the *user* message (per-turn input). System prompts are processed with higher attention priority in most models and provide a natural place for role definitions, constraints, and output format specifications [290].

1.13.5 Structured Output Prompts (JSON/XML)

For programmatic use, the most critical prompting technique is enforcing structured output—particularly JSON.

JSON Output Prompt

```
Extract the following information from the customer email.
Respond ONLY with valid JSON, no other text.

Schema:
{
  "intent": "refund|complaint|question|praise",
  "urgency": "low|medium|high",
  "product_mentioned": "string or null",
  "summary": "one sentence summary"
}

Email: [...]
```

Techniques for reliable structured output:

- **Schema-first**: Show the exact JSON schema *before* the input. The model treats it as a template.
- **Constrained decoding**: Use grammar-based sampling (e.g., Outlines [400], Guidance) to guarantee syntactically valid JSON at the token level.
- **XML tags**: For nested or multi-part outputs, XML tags (e.g., `<thinking>...</thinking>`) provide unambiguous delimiters that models follow reliably.
- **Pydantic/TypeScript types**: Providing type definitions helps models understand field constraints (OpenAI's function calling uses JSON Schema internally).

JSON in Prompts — Common Pitfalls

- Models may add markdown fences (`"‘json ... "‘`) — instruct explicitly to output raw JSON.
- Nested objects and arrays increase hallucination risk — flatten schemas where possible.

- Enum fields (fixed choices) are much more reliable than free-text fields.
- Always validate outputs programmatically; no prompt guarantees 100% compliance without constrained decoding.

JSON Prompting: Structuring the Input. A distinct but complementary technique is *JSON prompting*—formatting the prompt *itself* as JSON rather than natural language. This exploits the model's extensive pre-training on structured data (APIs, configs, code) to improve instruction adherence, reduce ambiguity, and enable deterministic parsing of multi-field requests.

JSON Prompting with System Prompt

```
=== SYSTEM ===
You are a senior code reviewer. Analyze code for bugs,
security issues, and style violations. Always respond
in the JSON schema provided.

=== USER (JSON prompt) ===
{
  "task": "code_review",
  "language": "python",
  "severity_filter": "high",
  "code": "def login(user, pw):\n    query = ...",
  "output_schema": {
    "issues": [{
      "line": "int",
      "severity": "critical|high|medium|low",
      "category": "security|bug|style|performance",
      "description": "string",
      "fix": "string"
    }],
    "overall_risk": "critical|high|medium|low"
  }
}
```

Why JSON prompting works:

- **Unambiguous field boundaries**: No confusion about where one instruction ends and another begins.
- **Typed constraints**: Fields like `"severity_filter": "high"` are clearer than "only show high severity issues."
- **Schema-as-contract**: Including `output_schema` in the input mirrors API design patterns the model has seen extensively during pre-training.
- **System prompt still essential**: The system message provides *role*, *tone*, and *behavioral constraints* that don't fit naturally in a JSON payload.

1.13.6 Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting [395] asks the model to produce intermediate reasoning steps before giving a final answer. This simple technique dramatically improves performance on tasks requiring multi-step reasoning: arithmetic, logic, commonsense inference, and code generation.

Why CoT works:

- **Serializes computation**: Transformers have fixed depth but variable-length generation. CoT converts parallel (hard) problems into sequential (easy) steps, effectively increasing the model's computational budget.

- **Reduces compounding errors**: Each step is a simpler sub-problem with lower per-step error rate.
- **Exposes intermediate state**: Makes reasoning auditable and debuggable.

Chain-of-Thought Variants

Method	Description
Zero-shot CoT [199]	Append "Let's think step by step" to any prompt
Few-shot CoT [395]	Provide examples with explicit reasoning chains
Self-Consistency [388]	Sample N CoT paths; majority-vote the final answer
Tree of Thoughts [429]	Explore multiple reasoning branches with backtracking
Plan-and-Solve [385]	First plan the steps; then execute each step
ReAct [430]	Interleave Reasoning and Acting (tool use)

Zero-Shot Chain-of-Thought

```
Q: A store has 45 apples. They sell 3/5 of them in the
morning and 1/3 of the remaining in the afternoon.
How many apples are left?

Let's think step by step.

A: Morning sales: 45 * 3/5 = 27 apples sold.
Remaining after morning: 45 - 27 = 18.
Afternoon sales: 18 * 1/3 = 6 apples sold.
Remaining: 18 - 6 = 12 apples.
```

Self-Consistency. Wang et al. [388] showed that sampling multiple chain-of-thought reasoning paths and taking a majority vote over final answers significantly outperforms single-path CoT. The intuition: correct reasoning paths tend to converge on the same answer, while errors are typically idiosyncratic. This trades compute (generating N samples) for accuracy—practical when latency is less important than correctness.

When CoT hurts. CoT is not universally beneficial. For simple tasks (single-step classification, retrieval, formatting), CoT adds unnecessary tokens, increases latency, and can even introduce errors through overthinking. Use CoT selectively for tasks where you expect multi-step reasoning to be required.

1.13.7 Advanced Prompting Techniques

Retrieval-Augmented Generation (RAG). Rather than relying solely on the model's parametric memory, RAG [219] retrieves relevant documents and includes them in the prompt:

```
Context (retrieved): [document chunks]
Question: [user query]
Answer based ONLY on the provided context.
```

This grounds the model's responses in verifiable sources and dramatically reduces hallucinations for knowledge-intensive tasks.

Prompt Chaining and Decomposition. Complex tasks benefit from being broken into a pipeline of simpler prompts, where the output of one becomes the input to the next:

1. *Extract* key facts from document
2. *Reason* over extracted facts

3. *Format* final answer

Each step can use a different prompt template, model, or temperature setting. This is more controllable than a single monolithic prompt and enables targeted debugging.

Constitutional AI / Self-Critique. Bai et al. [17] introduce prompts that ask the model to critique and revise its own output against a set of principles:

```
[Generate initial response]
Critique: Does this response violate any of the following
principles? [list principles]
Revision: Rewrite the response addressing the critique.
```

Meta-Prompting and Prompt Optimization. Rather than hand-crafting prompts, recent work automates prompt design:

- **APE** [464]: Uses an LLM to generate and score candidate prompts automatically.
- **DSPy** [193]: Compiles declarative task descriptions into optimized prompt pipelines with learned few-shot examples.
- **OPRO** [424]: Treats prompt optimization as an optimization problem, using an LLM as the optimizer.

Attentive Reasoning Queries (ARQ). ARQ [425] addresses a fundamental weakness of standard prompting: as context length grows, models increasingly "lose" critical information in the middle of the prompt (the *lost-in-the-middle* effect). ARQ mitigates this by decomposing a complex query into multiple focused sub-queries, each designed to direct the model's attention to a specific part of the context:

1. **Query decomposition**: Break the user question into atomic sub-questions that each target a narrow aspect.
2. **Attentive retrieval**: For each sub-query, retrieve or highlight only the relevant context slice—forcing the model to attend to it.
3. **Aggregation**: Combine sub-answers into a coherent final response.

This is particularly effective for long-document QA, multi-hop reasoning over large retrieval sets, and agentic tasks where the context window contains many tool outputs. ARQ can be seen as a structured form of chain-of-thought that explicitly manages *where* the model looks, not just *how* it reasons.

1.13.8 Best Practices: Crafting Effective Prompts

Based on empirical findings across the literature and practitioner experience, the following principles reliably improve prompt quality:

The Prompt Engineering Checklist

1. **Be specific and unambiguous**: Replace "summarize this" with "summarize in 2–3 bullet points, each under 20 words, focusing on actionable findings."
2. **Show, don't tell**: One good example is worth 100 words of instruction. When in doubt, add a few-shot example.
3. **Define the output format explicitly**: Specify JSON schema, bullet points, table format, or exact delimiters. Never leave format to interpretation.

4. **Use delimiters for input data**: Wrap user inputs in clear delimiters (`"""`, `<input>...</input>`, `--`) to separate instructions from data.
5. **Assign a role**: "You are a [domain expert] who [specific behaviour]" primes relevant knowledge and tone.
6. **Specify what NOT to do**: Negative constraints ("do not explain your reasoning", "never output more than 5 items") are often more effective than positive ones.
7. **Add chain-of-thought for reasoning tasks**: Append "Think step by step" or provide worked examples for math, logic, or multi-hop questions.
8. **Control temperature appropriately**: Use $T \approx 0$ for factual/deterministic tasks; $T \approx 0.7$–1.0 for creative/diverse outputs.
9. **Iterate empirically**: Treat prompts as code—version them, A/B test them, and measure performance on representative eval sets.
10. **Leverage recency bias**: Place the most critical instructions and examples at the *end* of the prompt (closest to the generation point).

Table 1.16: Common prompting failure modes and solutions.

Failure Mode	Symptom	Solution
Instruction amnesia	Model ignores constraints in long prompts	Move constraints to end; repeat key rules; use system prompt
Format drift	Output starts correct but degrades over long generations	Use constrained decoding; break into shorter chained prompts
Sycophancy	Model agrees with incorrect premises in the prompt	Add "challenge assumptions if incorrect"; use system-level instruction
Hallucinated details	Model invents facts not in provided context	Add "if unknown, say I don't know"; use RAG with source attribution
Refusal over-triggering	Model refuses benign requests due to safety training	Rephrase to clarify legitimate intent; provide explicit context for why the request is appropriate

The Prompt Engineering Mindset

Think of prompt engineering as *programming in natural language*. The model is a powerful but literal interpreter—it will do exactly what you ask, interpreted in the most likely way given its training distribution. Common principles from software engineering apply:

- **DRY (Don't Repeat Yourself)**: Unless fighting attention decay in long contexts
- **Separation of concerns**: Different prompt sections for role, constraints, examples, and input
- **Test-driven development**: Define expected outputs before writing the prompt
- **Version control**: Track prompt iterations and their eval scores
- **Modularity**: Build reusable prompt templates; parameterize variable parts

When prompting fails to achieve the desired quality after systematic iteration, that is the signal to move to fine-tuning (SFT) or reinforcement learning (RLHF/DPO).

1.14 Model Compression Methods

Model compression reduces model size and inference cost while preserving quality. Three main approaches: quantization (reduce precision), pruning (remove parameters), and distillation (train a smaller model to mimic a larger one).

1.14.1 Quantization

Quantization reduces model size and inference cost by representing weights (and optionally activations) in lower-precision formats. The core trade-off is compression ratio versus quality degradation.

Quantization Overview

Quantization reduces the numerical precision of model weights (and optionally activations) from FP32/BF16 to lower-bit formats:

$$x_q = \text{round}\left(\frac{x - z}{s}\right), \quad x_{\text{dequant}} = s \cdot x_q + z$$

where s is the scale factor and z is the zero-point.

Table 1.17: Quantization methods for LLMs.

Method	Bits	Type	Key Idea
GPTQ [105]	4-bit	PTQ, weight-only	Layer-wise quantization minimizing $\|WX - \hat{W}X\|^2$ via optimal brain surgeon.
AWQ [229]	4-bit	PTQ, weight-only	Protects salient weights (those with large activations). 1% of weights carry 99% importance.
GGUF [116]	2–8 bit	PTQ, weight-only	CPU-optimized format (llama.cpp). Per-block quantization with multiple types.
FP8 (E4M3)	8-bit	Training + inference	Native H100 support. 2× throughput vs BF16.
SmoothQuant [414]	W8A8	PTQ, weight+act.	Smooths activation outliers into weights before quantization. Enables INT8 GEMM.
QAT [246]	4-bit	QAT	Trains with simulated quantization. Highest quality but expensive.
AQLM [87]	2-bit	PTQ, additive codes	Extreme compression via learned additive quantization codebooks.

When to Quantize

- **Inference serving**: Always quantize. W4A16 (4-bit weights, BF16 activations) is the sweet spot — 2× memory savings, <1% quality loss for 70B+ models.
- **Training**: FP8 on H100 gives 2× throughput with minimal quality loss. BF16 is still the default for smaller models.
- **Edge deployment**: GGUF Q4_K_M for local inference on consumer hardware.
- **RLHF**: Quantize the frozen models (reference, reward model) to INT8/FP8. Keep the policy in BF16 for training precision.

1.14.2 Pruning

Why Prune? Modern LLMs contain billions of parameters, yet empirical studies consistently show that a large fraction of these weights contribute minimally to model outputs. Pruning exploits this

over-parameterization: by removing redundant weights, we reduce **memory footprint** (enabling deployment on smaller GPUs or edge devices), **inference latency** (fewer multiply-accumulate operations per forward pass), and **serving cost** (higher throughput per dollar). Unlike quantization, which reduces the precision of all weights uniformly, pruning selectively eliminates weights—enabling multiplicative savings when combined with quantization (e.g., a 50% sparse, 4-bit model uses 4× less memory than the dense BF16 baseline). The challenge is achieving high sparsity without degrading generation quality, which has driven the development of principled one-shot methods that require no retraining.

Pruning Methods

- **Unstructured pruning**: Zero out individual weights below a threshold. High sparsity (50–90%) possible. Requires sparse GEMM kernels (2:4 on A100/H100).
- **Structured pruning**: Remove entire attention heads, layers, or FFN neurons. Directly reduces FLOPS without specialized kernels.
- **SparseGPT** [104]: One-shot pruning using approximate inverse Hessian. 50% unstructured sparsity with minimal quality loss on 175B models.
- **Wanda** [364]: Prune by $|w| \times \|x\|$ (weight magnitude times input activation norm). No calibration data needed. Competitive with SparseGPT.

NVIDIA 2:4 Structured Sparsity

A100/H100 Tensor Cores natively support 2:4 sparsity: out of every 4 elements, at most 2 are non-zero. This gives exactly 2× speedup on supported operations with hardware acceleration. The constraint: you must achieve *exactly* 50% sparsity in this specific pattern, which limits flexibility compared to arbitrary sparsity.

1.14.3 Knowledge Distillation

Knowledge distillation [149] transfers the learned behaviour of a large *teacher* model into a smaller, cheaper *student* model. The core idea is that the teacher's output distribution over tokens carries far richer signal than ground-truth hard labels alone — revealing inter-class similarities, calibration, and uncertainty that the student can exploit.

Temperature-Scaled Softmax. To expose the "dark knowledge" in the teacher's logits we soften the distribution with a temperature $T > 1$:

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

At high temperature the probability mass spreads across more tokens, making near-miss alternatives visible. During training the same temperature is applied to the student; at inference the student uses $T = 1$.

General Distillation Loss.

$$\mathcal{L}_{\text{distill}} = \alpha\, T^2 \cdot \text{KL}(P^{(T)}_{\text{teacher}} \parallel P^{(T)}_{\text{student}}) \;+\; (1-\alpha) \cdot \mathcal{L}_{\text{CE}}(y, P^{(1)}_{\text{student}})$$

The T^2 factor compensates for the reduced gradient magnitude of softened distributions. Typical values: $T \in [2, 20]$, $\alpha \in [0.5, 0.9]$ (more weight on KL when teacher quality is high).

Offline (White-Box) Distillation. The teacher's full logit vector (or top-k logits for storage efficiency) is recorded for each training token. The student minimises the KL divergence against these stored distributions. This is the most data-efficient paradigm when teacher access is unrestricted.

Motivation: Decouple teacher inference from student training — run the teacher once on high-end hardware, then train many students cheaply.

Table 1.18: Knowledge distillation paradigms for LLMs.

Paradigm	Mechanism	Pros	Cons
Offline / White-box	Teacher logits precomputed; student trains on full distributions	Full distribution signal; one-time teacher cost	Stale data; storage heavy
Online / Co-training	Teacher generates on-the-fly; student sees fresh logits	Adapts to student weaknesses	2× compute; complex infra
Black-box (API)	Only teacher *text* outputs available (no logits)	Works with proprietary models	Loses dark knowledge; SFT-like
Self-distillation	Model distills into a smaller version of itself	No separate teacher needed	Teacher = student family; ceiling

Pros: Deterministic, reproducible; teacher cost is amortised; full distributional signal.

Cons: Requires storing $|V|$-dimensional vectors per token (mitigated by top-k pruning); teacher cannot adapt to student failures.

Online (Co-Training) Distillation. Teacher and student are run jointly: the teacher generates logits for the student's current training batch.

Motivation: Let the teacher focus on inputs where the student currently struggles (curriculum-like).

Pros: Freshness; can use student-generated inputs for on-policy distillation.

Cons: Double the GPU cost; synchronisation complexity; harder to scale.

Black-Box (API) Distillation. When only text outputs are available (e.g. distilling from a proprietary API), the student is trained via SFT on the teacher's generations, optionally augmented with chain-of-thought traces.

Motivation: Practical reality — most frontier models do not expose logits.

Pros: Simple pipeline; works with any model behind an API.

Cons: No soft-label signal; prone to hallucination amplification; effectively supervised fine-tuning.

Self-Distillation. A model distils from a larger version within the same architecture family (e.g. Llama-3 70B → 8B) or from its own checkpoints during training.

Motivation: Avoid training a separate teacher; leverage the model's own capacity at different scales.

Pros: Architecture compatibility; no external dependency.

Cons: Teacher ceiling equals model ceiling; cannot introduce genuinely new knowledge.

Dark Knowledge

Consider a language model predicting the next word after "The capital of France is". Hard labels say only "Paris" is correct. But the teacher's soft distribution might assign 5% to "Lyon", 2% to "Marseille", and near-zero to "banana" — telling the student *which errors are reasonable*, which dramatically improves calibration and generalisation.

Practical Considerations for LLM Distillation.

- **Sequence-level vs. token-level:** Token-level KL is standard; sequence-level distillation (minimising KL over full sequences) better captures long-range coherence but is harder to optimise.
- **Layer-wise hints:** Matching intermediate representations (attention maps, hidden states) provides additional learning signal — especially useful when student architecture differs.
- **Data selection:** Distillation data quality matters; curating diverse, hard examples yields better students than random sampling.
- **Student capacity:** Diminishing returns below ∼10% of teacher parameters; at extreme compression, architecture changes (e.g. MoE → dense) may be needed.
- **Combining with quantization:** Distillation + 4-bit quantization (e.g. QLoRA-distilled models) achieves near-teacher quality at 20× compression.

Compression Method Comparison – 70B Model

Method	Size	Speed	Quality	Use Case
BF16 (baseline)	140 GB	1×	100%	Training, reference
FP8 (E4M3)	70 GB	2×	99.5%	H100 inference
INT8 (SmoothQuant)	70 GB	1.8×	99%	A100 inference
4-bit (AWQ)	35 GB	2.5×	97–98%	Serving at scale
2-bit (AQLM)	17.5 GB	3×	90–93%	Edge, experimental
Pruned 50% (2:4)	70 GB	1.8×	97%	Structured speedup
Distilled 8B	16 GB	10×	80–85%	Mobile, edge

1.15 Speculative Decoding Methods

Speculative decoding [216] accelerates autoregressive generation by predicting multiple tokens simultaneously, then verifying them in a single forward pass of the target model. It produces **identical output distribution** to standard decoding (no quality loss) while achieving 2–3× speedup.

1.15.1 Core Principle

Speculative Decoding Framework

1. A fast **draft mechanism** proposes k candidate tokens: $\hat{x}_1, \ldots, \hat{x}_k$
2. The large **target model** runs a single forward pass on all k tokens (batched)
3. **Verification**: Accept tokens left-to-right while $P_{\text{target}}(\hat{x}_i) \geq r_i \cdot P_{\text{draft}}(\hat{x}_i)$ (where $r_i \sim U[0,1]$)
4. On first rejection at position j: resample x_j from an adjusted distribution, discard $\hat{x}_{j+1}, \ldots, \hat{x}_k$

Key property: This acceptance/rejection scheme guarantees the final distribution equals P_{target} exactly.
Speedup: If acceptance rate is α, expected tokens per step $= \frac{1-\alpha^{k+1}}{1-\alpha}$. At $\alpha = 0.8$, $k = 5$: expected 3.4 tokens/step vs. 1 for standard decoding.

1.15.2 Medusa: Multi-Head Speculative Decoding

How Medusa Works

Medusa adds k additional "prediction heads" to the LLM (sharing the same backbone):

- Head 0 (original): predicts token at position $t+1$ (standard next-token)
- Head 1: predicts token at position $t+2$ (skipping one)
- Head i: predicts token at position $t+i+1$
- All heads run in parallel during a single forward pass
- A **tree-structured** verification validates multiple candidate sequences simultaneously

Training: Fine-tune only the Medusa heads (backbone frozen). Cost: $\sim$1 epoch on representative data.
Advantage: No separate draft model; heads are tiny (one linear layer each). Memory overhead: $<1\%$.

1.15.3 Eagle: Feature-Level Drafting

Why Eagle Outperforms Medusa

Medusa's heads predict independently at each position — they cannot condition on their own previous predictions (token at $t+2$ doesn't know what was predicted at $t+1$). Eagle fixes this with a lightweight autoregressive decoder that operates on the target model's hidden states:

1. Extract hidden states from the target model's last layer
2. Feed into a small (1-layer) decoder that autoregressively generates draft tokens conditioned on previous hidden states
3. This captures inter-token dependencies that Medusa misses

Result: Eagle achieves 85–95% acceptance rate vs. Medusa's 60–80%.

1.15.4 N-gram Speculative Decoding

N-gram Lookup Method

The simplest speculative decoding — requires no additional model or training:

1. Maintain a cache of n-grams from the prompt and previously generated text
2. At each step, check if the current context's last $n-1$ tokens match any cached n-gram
3. If yes: propose the continuation as draft tokens
4. Verify against target model as usual

Best for: Code generation (repetitive patterns), structured outputs (JSON/XML), and prompts with repeated elements. **Cost**: Essentially zero.

1.15.5 Methods Comparison

Table 1.19: Speculative decoding methods supported by modern inference engines.

Method	Draft Source	Speedup	Key Idea
Standard [216]	Small model (1–7B)	2–3×	Separate draft model generates candidates. Simple but requires loading 2 models.
Medusa [37]	Parallel LM heads	2–3×	Add k extra prediction heads to the target model. Each predicts token at position $+1, +2, \ldots, +k$.
Eagle [224]	Feature-level	2.5–3.5×	Lightweight decoder generates draft tokens from target model's hidden states. Higher acceptance than Medusa.
Eagle-2 [224]	Context-aware	3–4×	Dynamic draft tree with confidence-based expansion. State-of-the-art acceptance rates.
N-gram Lookup	N-gram cache	1.5–2×	Match prompt n-grams against previously generated text. Zero cost; great for repetitive outputs.
Lookahead [107]	Jacobi iteration	2–2.5×	Parallel Jacobi decoding with n-gram verification. No draft model; uses target model itself.
Multi-token [118]	Modified arch.	2–3×	Train the model to natively predict multiple tokens per step (Meta's approach in Llama).

1.15.6 Integration with vLLM

```python
from vllm import LLM, SamplingParams

# Standard speculative decoding (separate draft model)
llm = LLM(
    model="meta-llama/Llama-3-70B",
    tensor_parallel_size=4,
    speculative_config={
        "model": "meta-llama/Llama-3-8B",
        "num_speculative_tokens": 5,
    },
)

# N-gram speculation (zero-cost, no draft model needed)
llm = LLM(
    model="meta-llama/Llama-3-70B",
    speculative_config={
        "method": "ngram",
        "num_speculative_tokens": 5,
        "prompt_lookup_max": 4,  # Match up to 4-grams from prompt
    },
)

# EAGLE-style (feature-level draft, high acceptance rate)
llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    tensor_parallel_size=4,
    speculative_config={
        "model": "yuhuili/EAGLE-LLaMA3-Instruct-8B",
        "num_speculative_tokens": 2,
        "method": "eagle",
        "draft_tensor_parallel_size": 1,
    },
)

# MLP speculator (IBM-style, lightweight head)
llm = LLM(
    model="meta-llama/Meta-Llama-3.1-70B-Instruct",
    tensor_parallel_size=4,
    speculative_config={
        "model": "ibm-ai-platform/llama3-70b-accelerator",
        "draft_tensor_parallel_size": 1,
    },
)
```

When NOT to Use Speculative Decoding

- **High batch sizes**: At batch ≥ 64, generation is already compute-efficient. Speculation adds overhead (draft generation + verification) that doesn't pay off.
- **Very different distributions**: If draft model is too dissimilar to target, acceptance rate drops below 50% and speculation is slower than standard decoding.
- **Short outputs**: For <20 token outputs, the setup cost of speculation exceeds savings.
- **Rule of thumb**: Speculation helps most for latency-sensitive, single-stream generation (chatbots, interactive code completion).

1.16 Hallucination Detection

LLMs generate fluent text that may be factually incorrect—a phenomenon called **hallucination** [174]. This section covers basic detection methods at the model level (without external retrieval or multi-agent verification).

1.16.1 Types of Hallucination

Hallucination Taxonomy

- **Intrinsic**: Contradicts the provided input/context (e.g., summary says the opposite of the source)
- **Extrinsic**: Generates claims that cannot be verified from the input and are factually wrong
- **Faithfulness**: Output diverges from the instruction or specified constraints

1.16.2 Detection Methods (Model-Level)

Table 1.20: Basic hallucination detection methods that operate at the model level.

Method	Mechanism	Signal
Token-level entropy	High entropy at generation time indicates uncertainty [185]	$H(P(x_t)) > \tau$
Sequence log-prob	Low average log-probability of the output suggests confabulation	$\frac{1}{T}\sum_t \log P(x_t)$
Consistency sampling	Generate N responses; low agreement = likely hallucination [262]	Contradiction rate
Semantic entropy	Cluster meanings (not strings); high semantic entropy = uncertain [203]	Cluster diversity
DoLA	Contrast logits between later vs. earlier layers; amplifies factual knowledge [60]	Layer divergence

Semantic Entropy. Kuhn et al. [203] observe that token-level entropy is unreliable (paraphrases have different tokens but same meaning). Instead, they generate multiple responses, cluster them by semantic equivalence (via NLI), and compute entropy over meaning clusters:

$$SE = -\sum_{c \in \text{clusters}} P(c) \log P(c)$$

High SE means the model produces *semantically different* answers—a strong hallucination signal.

SelfCheckGPT. Manakul et al. [262] detect hallucinations by checking self-consistency: generate multiple responses and verify whether claims in the main response are supported by the alternatives. If the model "disagrees with itself," the claim is likely hallucinated. No external knowledge needed.

DoLA (Decoding by Contrasting Layers). Chuang et al. [60] observe that factual knowledge emerges in later transformer layers while earlier layers retain more generic/uncertain representations. DoLA contrasts the logit distributions between a later ("mature") layer and an earlier ("premature") layer at each decoding step:

$$\text{DoLA}(x_t) = \text{softmax}\big(\log P_{\text{late}}(x_t) - \log P_{\text{early}}(x_t)\big)$$

By amplifying the signal from factual knowledge encoded in deeper layers, DoLA reduces hallucinations at inference time *without any retraining*—requiring only a single additional forward pass through the contrasted layer. It is complementary to sampling-based methods and can be combined with them.

Limitations of Model-Level Detection

These methods detect *uncertainty*, not *incorrectness*. A model can be confidently wrong (low entropy, consistent responses—but factually false). For reliable detection, combine with retrieval-based verification (RAG) or external fact-checking tools.

1.17 LLM Safety and Responsible AI

Safety is not an afterthought—it is an integral part of the LLM training pipeline. This section covers the key dimensions of LLM safety and the mechanisms used to enforce responsible behavior.

1.17.1 Threat Taxonomy

Table 1.21: LLM safety threat categories.

Category	Description and Examples
Harmful content	Generating toxic, violent, or illegal instructions (bioweapons, CSAM)
Bias and discrimination	Perpetuating stereotypes; unfair treatment across demographics [110]
Privacy violations	Leaking PII from training data; memorization attacks [41]
Jailbreaking	Adversarial prompts that bypass safety guardrails [466]
Misinformation	Generating convincing but false claims (hallucination at scale)
Dual-use	Legitimate capabilities (coding, chemistry) weaponized for harm

1.17.2 Safety Training Pipeline

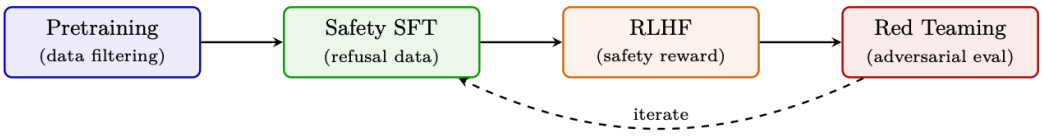


Figure 1.14: Safety is applied at every stage: data filtering in pretraining, refusal examples in SFT, safety-specific reward models in RLHF, and iterative red-teaming.

1.17.3 Key Safety Mechanisms

Safety Techniques

- **Data filtering**: Remove toxic, biased, and PII-containing text from pretraining corpora
- **Safety SFT**: Train on examples of appropriate refusals ("I can't help with that because...")
- **Constitutional AI** [17]: Self-critique using principles; model revises its own outputs against a constitution of rules
- **Safety reward model**: Separate RM trained on safety-annotated pairs; combined with helpfulness RM during RLHF via weighted sum
- **Guardrails**: Input/output classifiers that block harmful requests/responses at serving time
- **Red teaming** [307]: Systematic adversarial evaluation to find failure modes before deployment

1.17.4 The Helpfulness–Safety Tradeoff

Balancing Helpfulness and Safety

Over-optimizing for safety creates an *over-refusal* problem: the model declines benign requests (e.g., refusing to discuss historical violence in an educational context). The goal is a Pareto-optimal policy that is maximally helpful *within* safety constraints:

$$\max_{\theta} \mathbb{E}[R_{\text{helpful}}] \quad \text{subject to} \quad \mathbb{E}[R_{\text{safety}}] \geq \tau$$

In practice, this is implemented as a weighted reward: $R = \alpha R_{\text{helpful}} + (1 - \alpha) R_{\text{safety}}$ with careful tuning of α (typically 0.6–0.8). Meta's Llama-3 reports using distinct safety and helpfulness reward models with margin-based weighting [126].

1.17.5 Evaluation

- **Safety benchmarks**: ToxiGen [143] (implicit hate speech), RealToxicityPrompts [114] (toxic degeneration), BBQ [302] (question-answering bias), CrowS-Pairs [277] (social bias in masked LMs)
- **Jailbreak robustness**: GCG attacks [466], multi-turn jailbreaks, encoded prompts
- **Over-refusal rate**: Measure false-positive refusals on benign prompts (target <5%)
- **Red team evaluations**: Human adversarial testing with domain experts (biosecurity, cybersecurity)

Safety Is Never Complete

No combination of techniques provides absolute safety. New attack vectors are discovered continuously (multi-modal jailbreaks, fine-tuning attacks that remove safety training, many-shot prompting). Safety requires ongoing monitoring, rapid response to new threats, and defense-in-depth (multiple independent layers).

Chapter 2

Systems Foundations for LLMs

Training and serving large language models is as much a systems engineering challenge as it is a machine learning one. A 70B-parameter model requires hundreds of gigabytes of memory, thousands of GPU-hours to train, and carefully orchestrated communication across dozens of accelerators. Getting any of this wrong — a suboptimal parallelism strategy, a memory leak, a misconfigured collective operation — can mean the difference between a training run that converges and one that crashes after days of compute. This chapter covers the hardware and software infrastructure that makes LLMs possible: GPU architecture and memory hierarchies, distributed training strategies (data, tensor, and pipeline parallelism), and high-throughput inference with vLLM. The goal is not just to understand how these systems work, but to develop the intuition needed to debug them when they don't.

GAS Connection — Systems Foundations for LLMs

The GAS is designed to run on a single developer machine, but every infrastructure decision it makes is a miniature version of the production systems described in the following chapter.

GPU memory management (§ GPU Architecture). Running the GAS agents requires careful VRAM budgeting. The `GASLLMClient` in `capstone_gas/gas_system.py` uses Ollama's `num_predict` parameter to enforce a maximum token limit per agent invocation. Understanding CUDA memory allocation, tensor layouts, and the cost of context switching between models is essential for tuning this configuration in a multi-patient deployment.

Inference throughput (§ vLLM – PagedAttention). In a multi-patient deployment (e.g. a hospital ward with 50 patients), the GAS would use vLLM's PagedAttention to serve all agent models with continuous batching. The current prototype uses Ollama for local development; the `GASLLMClient` backend auto-detection makes it straightforward to swap in a vLLM endpoint by setting the `OPENAI_API_BASE` environment variable.

Quantisation and serving (§ Memory Optimization Techniques). The local development setup uses Ollama with GGUF Q4_K_M quantisation of `llama3.2:3b`. This reduces the model footprint to ∼2 GB, making the full GAS runnable on a laptop. The trade-off—quantisation quality vs. serving speed—is directly observable by comparing Mock, Ollama, and OpenAI backends in `capstone_gas/notebook_capstone.ipynb`.

Latency budget. The GAS has a hard latency requirement: the full pipeline (`monitor_node` → `advisor_node` → `safety_node` → `caregiver_node`) must complete within 10 seconds of a glucose anomaly being detected. The pipeline is assembled in `build_gas_graph()` in `capstone_gas/gas_system.py` using LangGraph's `StateGraph` API.

Fault tolerance. The GAS implements graceful degradation: if the LLM backend is unavailable, `GASLLMClient` falls back to a rule-based mock response. The `NOTEBOOK_TEST_MODE=1` environment variable forces mock mode for CI-safe testing without any LLM dependency.

2.1 GPU Architecture – From Silicon to LLM Training

Modern large language models are trained and served almost exclusively on GPUs (Graphics Processing Units). Understanding GPU architecture is essential for making informed decisions about parallelism strategies, memory management, kernel optimization, and infrastructure sizing. This section provides a comprehensive introduction to GPU hardware as it relates to LLM workloads.

2.1.1 Why GPUs for Deep Learning?

GPUs and CPUs represent fundamentally different hardware philosophies. Understanding this difference explains why LLM training is 100–1000× faster on GPUs.

CPUs vs. GPUs – Fundamental Design Philosophy

- **CPUs** are optimized for *latency* – they execute a few threads as fast as possible, with large caches, branch predictors, and out-of-order execution. A modern CPU has 8–96 cores.
- **GPUs** are optimized for *throughput* – they execute thousands of threads in parallel, each doing simple work. A modern GPU has thousands of "cores" (execution units) grouped into Streaming Multiprocessors (SMs).

Deep learning workloads are dominated by matrix multiplications ($O(n^3)$ operations on $O(n^2)$ data), which are embarrassingly parallel. A single transformer forward pass for a 70B model requires ∼140 TFLOP of compute per token – perfect for GPU throughput.

2.1.2 NVIDIA GPU Microarchitecture Generations

NVIDIA has released a series of GPU architectures, each bringing key innovations for deep learning:

Table 2.1: NVIDIA GPU microarchitecture timeline for deep learning.

Architecture	Year	Flagship	Key Deep Learning Innovation
Pascal	2016	P100	First HBM GPU; FP16 support; NVLink 1
Volta	2017	V100	**Tensor Cores** (first generation); mixed-precision training
Turing	2018	T4	INT8 inference; RT cores (not for ML)
Ampere	2020	A100	BF16 Tensor Cores; TF32; 3rd-gen NVLink; MIG
Hopper	2022	H100	FP8 Tensor Cores; TMA; Transformer Engine; NVLink 4
Blackwell	2024	B200	2nd-gen Transformer Engine; NVLink 5 (1.8 TB/s); FP4

2.1.3 Common GPUs for LLM Training and Inference

Which GPU to Choose?

- **Training 70B+ models**: H100/B200 nodes with NVLink (need fast interconnect for tensor parallelism). Minimum 8×H100 per instance.
- **Inference (latency-sensitive)**: H100/H200 for high BW; MI300X for memory-bound (huge KV caches).
- **Fine-tuning 7B–13B**: A100-80GB is cost-effective. Single GPU with LoRA.
- **Budget**: A100-40GB or even A10 (24GB) for LoRA on 7B models.

Table 2.2: GPU specifications relevant to LLM workloads. All bandwidth figures are bidirectional.

GPU	Arch	HBM	BF16 TF	HBM BW	NVLink	LLM Role
V100-32GB	Volta	32 GB	125 TF*	900 GB/s	300 GB/s	Legacy; small model fine-tune
A100-40GB	Ampere	40 GB	312 TF	1.5 TB/s	600 GB/s	Budget training/inference
A100-80GB	Ampere	80 GB	312 TF	2.0 TB/s	600 GB/s	Standard RLHF (8–64 for 70B)
H100 SXM	Hopper	80 GB	990 TF	3.35 TB/s	900 GB/s	3× faster training
H200 SXM	Hopper	141 GB	990 TF	4.8 TB/s	900 GB/s	Fits 70B policy+ref on fewer GPUs
B200 SXM	Blackwell	192 GB	2250 TF	8.0 TB/s	1800 GB/s	Next-gen; 2× over H100
AMD and Google alternatives:						
MI300X	CDNA3	192 GB	1300 TF	5.3 TB/s	N/A	Most memory; ROCm
TPU v5e	Google	16 GB	197 TF	1.6 TB/s	ICI 1.6 TB/s	Cloud-only; JAX/XLA

2.1.4 GPU Internal Architecture – The Streaming Multiprocessor (SM)

A GPU is organized as an array of **Streaming Multiprocessors (SMs)**, each of which is an independent processor with its own register file, shared memory, and execution units. Understanding SMs is key to understanding GPU performance.

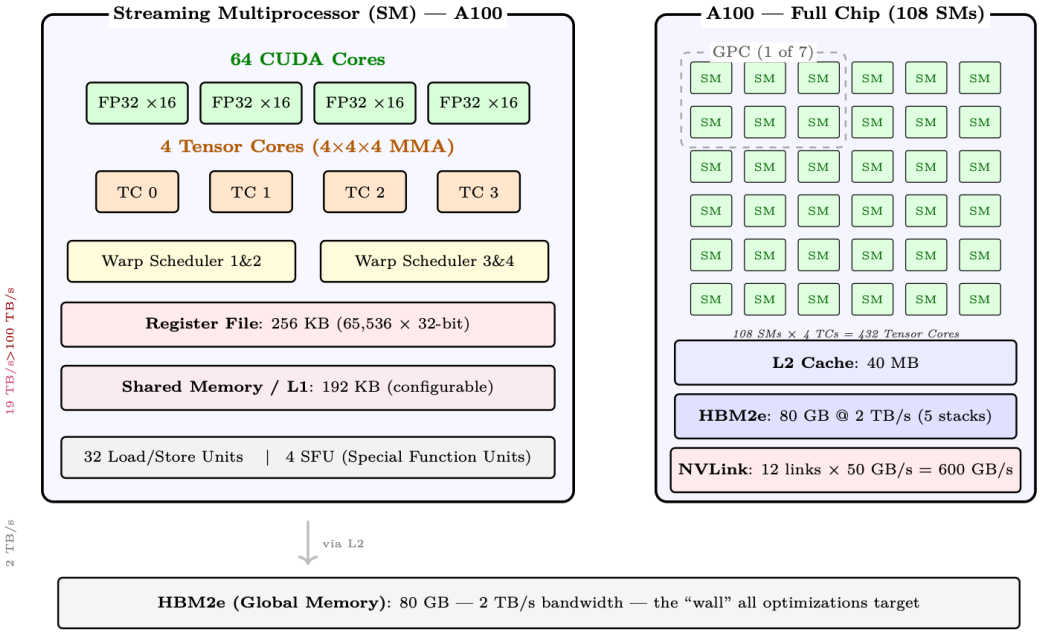


Figure 2.1: Left: Internal structure of a single Streaming Multiprocessor (SM) on A100 — 64 FP32 CUDA cores, 4 Tensor Cores, 4 warp schedulers, 256 KB register file, and 192 KB shared memory/L1 cache. Right: The full A100 chip contains 108 SMs with shared 40 MB L2 cache and 80 GB HBM2e. Bandwidth annotations (left margin) show the dramatic drop from registers to HBM.

Key SM Components

- **CUDA Cores:** Scalar ALUs for FP32/INT32 operations. 64 per SM on A100. Used for element-wise ops, reductions, and non-matrix operations.
- **Tensor Cores:** Specialized matrix-multiply-accumulate (MMA) units. Each performs a 4×4×4 fused multiply-add per cycle. 4 per SM on A100, delivering 16× throughput over CUDA cores for supported precisions.
- **Register File:** Fastest storage (1 cycle latency). Shared among all active threads. Spilling to L1 causes significant slowdown.
- **Shared Memory / L1:** On-chip SRAM explicitly managed by the programmer. The key to Flash Attention's performance (tiles fit entirely in shared memory).
- **Warp Schedulers:** Each SM has 4 warp schedulers (A100). A *warp* = 32 threads executing in lockstep (SIMT model). Schedulers hide memory latency by switching between warps.

The SIMT Execution Model

GPUs use Single Instruction, Multiple Threads (SIMT) execution. Within a warp (32 threads), all threads execute the same instruction but on different data. When threads diverge (e.g., `if/else`), both paths are serialized – called *warp divergence*. This is why GPU kernels must minimize branching.

For LLM workloads, the main operations (GEMM, attention, softmax) have uniform control flow across threads, making them ideal for SIMT execution.

2.1.5 GPU Chip Scaling Across Generations

The evolution of NVIDIA's GPU architectures shows consistent scaling of compute density, on-chip memory, and specialized units for deep learning:

Table 2.3: SM-level scaling across NVIDIA architectures.

Architecture	SMs	TCs/SM	SRAM/SM	L2	Key Change
Volta (V100)	80	8	128 KB	6 MB	Introduced Tensor Cores
Ampere (A100)	108	4	192 KB	40 MB	BF16/TF32; larger L2
Hopper (H100)	132	4	256 KB	50 MB	TMA; FP8; Thread Block Clusters
Blackwell (B200)	148	4	256 KB	128 MB	2× die; FP4; TMEM; NVLink 5

2.1.6 GPU Memory Hierarchy and Bandwidth

Modern GPU training and inference performance is almost entirely determined by *how well you manage data movement* across the memory hierarchy. Understanding the hierarchy is not optional – it is the foundation for every optimization technique discussed in later sections.

GPU Memory Hierarchy – A100 80GB Reference Numbers

Level	Capacity	Bandwidth	Latency	Location
Registers	∼256 KB/SM	>100 TB/s	1 cycle	On-chip, per-thread
SRAM (shared)	164 KB/SM	∼19 TB/s	∼20 cy	On-chip, per-SM
L2 Cache	40 MB total	∼5 TB/s	∼200 cy	On-chip, shared
HBM2e (VRAM)	80 GB	2 TB/s	∼200 ns	On-package (5 stacks)
CPU DRAM	512 GB+	∼25 GB/s	∼10 μs	Host (PCIe 4)
NVMe SSD	TBs	7 GB/s	∼100 μs	Host storage

Why the Gaps Are So Large

Each level of the hierarchy is roughly **10× slower and 100–1000× larger** than the one above it. The A100 has 312 TFLOP/s of BF16 tensor-core throughput but only 2 TB/s of HBM bandwidth. That means for every byte loaded from HBM you can do $312 \times 10^{12} / (2 \times 10^{12}) \approx 156$ floating-point operations before the next byte arrives. If your kernel does fewer than 156 FLOPs per byte, it is *memory-bound* – the compute units are idle waiting for data.

Registers. Each CUDA thread has access to a private register file. Registers are the fastest storage on the chip – reads and writes happen in a single clock cycle with no arbitration. The A100 has 65,536 32-bit registers per SM. Spilling registers to local memory (L1/L2) is a major performance hazard.

SRAM – Shared Memory / L1. Each SM has a combined L1/shared memory pool of 192 KB on A100 (256 KB on H100), with up to 164 KB configurable as shared memory on A100. Shared memory is explicitly managed by the programmer (or by the compiler in newer CUDA versions). Flash Attention, for example, is entirely built around the insight that the attention tile computation fits in SRAM.

L2 Cache. The 40 MB L2 on A100 is shared across all 108 SMs. It acts as a staging area between SRAM and HBM. For workloads with good spatial locality (e.g., weight matrices accessed repeatedly across a batch), L2 hit rates can dramatically reduce effective HBM traffic.

HBM – High Bandwidth Memory. HBM is stacked DRAM mounted directly on the GPU package, connected via a wide interposer. The A100 SXM has 80 GB of HBM2e at 2 TB/s. The H100 SXM5 has 80 GB of HBM3 at 3.35 TB/s. This is the primary working memory for model weights, KV caches, activations, and optimizer states.

CPU DRAM via PCIe. Data transfer between GPU HBM and CPU DRAM traverses the PCIe bus. PCIe Gen4 ×16 provides ∼32 GB/s per direction (64 GB/s bidirectional); Gen5 doubles this. This is a ∼60× bandwidth reduction compared to HBM (per-direction). CPU offloading (ZeRO-Infinity, DeepSpeed) exploits this link but must be used carefully to avoid becoming the bottleneck.

NVMe. NVMe SSDs (e.g., Samsung 990 Pro) reach ∼7 GB/s sequential read. ZeRO-Infinity can offload optimizer states to NVMe, but this is only viable when the compute-to-IO ratio is very high (large batch sizes, slow training steps).

2.1.7 Arithmetic Intensity and the Roofline Model

Arithmetic Intensity

$$I = \frac{\text{FLOPs}}{\text{Bytes accessed from HBM}} \quad \text{(FLOPs / Byte)}$$

A kernel is **memory-bound** when $I < I_{\text{ridge}}$ and **compute-bound** when $I > I_{\text{ridge}}$, where

$$I_{\text{ridge}} = \frac{\text{Peak FLOP/s}}{\text{Peak Bandwidth}} = \frac{312 \times 10^{12}}{2 \times 10^{12}} = 156 \text{ FLOP/Byte (A100 BF16)}$$

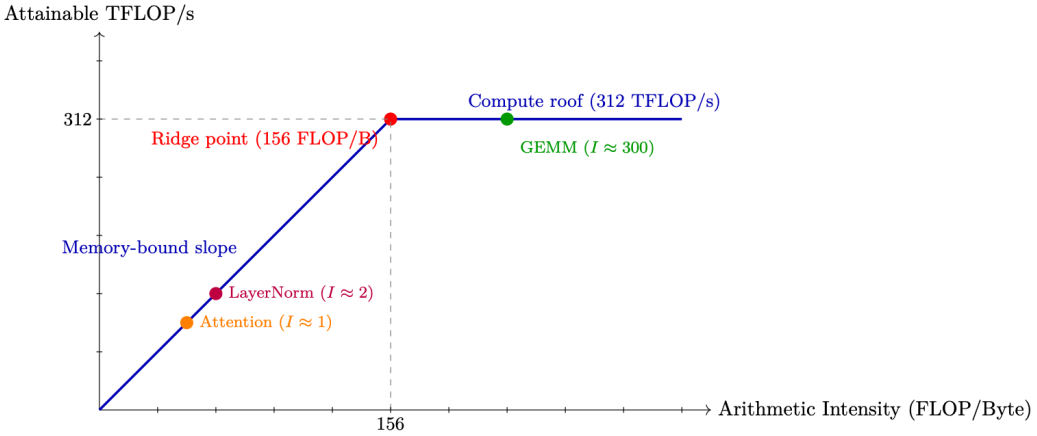


Figure 2.2: Roofline model for A100 BF16. Attention is deep in the memory-bound regime; large GEMMs (FFN layers) are compute-bound.

Attention Arithmetic Intensity

For a single attention head with sequence length $n = 4096$, head dim $d = 128$:
FLOPs: QK^T costs $2n^2d$, softmax is $O(n^2)$, Attn $\times V$ costs $2n^2d$. Total: $\approx 4n^2d = 4 \times 4096^2 \times 128 \approx 8.6$ GFLOP.
Memory traffic (standard, non-Flash implementation):

- Read Q, K: $2 \times n \times d \times 2 = 2$ MB
- Write attention scores $S = QK^T$: $n^2 \times 2 = 33.5$ MB

- Read S for softmax: $n^2 \times 2 = 33.5$ MB
- Write softmax output P: $n^2 \times 2 = 33.5$ MB
- Read P and V for final matmul: $n^2 \times 2 + n \times d \times 2 = 34.5$ MB
- Write output O: $n \times d \times 2 = 1$ MB

Total memory: ≈ 138 MB (dominated by 4 passes over the n^2 attention matrix).
Arithmetic intensity:

$$I = \frac{8.6 \times 10^9}{138 \times 10^6} \approx 62 \text{ FLOP/Byte}$$

This is $62/156 = 40\%$ of the A100 ridge point — **firmly memory-bound**. The GPU is 60% idle waiting for memory.
Flash Attention fix: By never materializing the $n \times n$ matrix (tiling Q, K, V in SRAM), Flash Attention reduces HBM traffic to just reading Q, K, V and writing O: $4 \times n \times d \times 2 = 4$ MB. Each byte loaded is reused in $O(n)$ computations (every query attends to every key), so:

$$I = \frac{4n^2 d}{4 \cdot n \cdot d \cdot 2} = \frac{n}{2} = \frac{4096}{2} = 2048 \text{ FLOP/Byte}$$

This is 13× above the ridge point (156) — **deeply compute-bound**. The GPU hits its peak 312 TFLOPS, needing only $312\text{T}/2048 \approx 152$ GB/s of bandwidth (7.6% of HBM capacity). Memory is no longer the bottleneck.

2.1.8 Attention is Memory-Bound; FFN is Compute-Bound

Two Regimes in a Transformer

A transformer block has two main components with very different arithmetic intensities:

- **Attention:** Operates on $n \times d$ tensors. The QK^T product is $O(n^2 d)$ FLOPs but requires $O(n^2)$ memory for the attention scores. At long sequences, memory traffic dominates – attention is *memory-bound*.
- **FFN (MLP):** Two large linear layers with weight matrices of shape $[d_{\text{model}}, 4d_{\text{model}}]$. These are large GEMMs with high arithmetic intensity – FFN is *compute-bound*.

This is why Flash Attention (memory optimization) helps attention but not FFN, while quantization (reducing weight size) helps FFN more than attention.

2.1.9 Tensor Cores

What Are Tensor Cores?

Tensor Cores are specialized matrix-multiply-accumulate (MMA) units introduced in Volta (2017). Each Tensor Core performs a $4 \times 4 \times 4$ matrix multiply in a single clock cycle:

$$D = A \times B + C \quad (4 \times 4 \text{ matrices})$$

The A100 has **432 Tensor Cores** across 108 SMs (4 per SM, one per sub-partition). At BF16 precision, they deliver 312 TFLOP/s – roughly 16× the throughput of FP32 CUDA cores.

- **Supported precisions:** FP64, TF32, BF16, FP16, INT8, FP8 (H100+).
- **Accumulation:** Always in FP32 internally, even for BF16 inputs. This prevents catastrophic cancellation during the dot product.

- **Requirement:** Tensor Cores are most efficient when matrix dimensions are multiples of 8 (BF16) or 16 (FP8). Padding to these multiples is often worthwhile.
- **WGMMA (H100):** Hopper introduces warpgroup-level MMA instructions that operate on larger tiles (64×256×16) and can be pipelined with TMA (Tensor Memory Accelerator) data movement.

The Tensor Core Trap

Tensor Cores only help if your kernel is *compute-bound*. If you are running a small batch (batch size 1, inference), the GEMM tiles are tiny, Tensor Core utilization is low, and you are back in the memory-bound regime. This is why inference engines batch requests aggressively.

2.1.10 Communication Architecture – NVLink, InfiniBand, and PCIe

Distributed LLM training and inference require moving enormous amounts of data between GPUs, nodes, and storage. The communication fabric is often the bottleneck for large-scale training.

PCIe – The Host-Device Link. PCIe (Peripheral Component Interconnect Express) is the standard bus connecting GPUs to the host CPU and system memory. Its bandwidth directly limits how fast data can be loaded onto the GPU and how quickly results can be returned. Bandwidth has roughly doubled with each generation:

PCIe Generations

Generation	x16 BW (each dir.)	Bidirectional	Notes
PCIe Gen3	16 GB/s	32 GB/s	Common in older servers
PCIe Gen4	32 GB/s	64 GB/s	A100 PCIe, most current servers
PCIe Gen5	64 GB/s	128 GB/s	H100 PCIe, emerging

PCIe is used for:

- CPU ↔ GPU data transfers (model loading, CPU offloading)
- Cross-node GPU communication when NVLink is unavailable (rare, very slow)
- NVMe storage access (via CPU)

PCIe is Not for GPU-GPU Communication

Never route GPU-GPU communication through PCIe if NVLink is available. PCIe bandwidth (32 GB/s) is 28× lower than NVLink 4 (900 GB/s). In a multi-GPU server without NVLink (e.g., consumer GPUs), inter-GPU bandwidth is limited to PCIe, making tensor parallelism extremely slow.

NVLink – Intra-Node High-Speed Interconnect.

NVLink Generations

Generation	Links	Total BW	GPU
NVLink 2	6	300 GB/s	V100
NVLink 3	12	600 GB/s	A100
NVLink 4	18	900 GB/s	H100
NVLink 5	18	1800 GB/s	B200 (Blackwell)

NVLink is a point-to-point interconnect between GPUs on the same node. Each link is bidirectional. The H100 SXM5 has 18 NVLink 4 links, each providing 50 GB/s bidirectional, for a total of 900 GB/s.

NVSwitch. In DGX H100 systems, all 8 GPUs are connected via NVSwitch – a dedicated switching chip that provides *full bisection bandwidth*. This means any GPU can communicate with any other GPU at full NVLink speed simultaneously, not just neighbors in a ring.

Ring vs. Full Bisection

In a ring topology (8 GPUs), an AllReduce requires data to travel around the ring. Each link must carry $\frac{2(N-1)}{N}$ of the total data, so the algorithm bandwidth is $B_{\text{link}} \times \frac{N}{2(N-1)}$ (about $0.57 \times B_{\text{link}}$ for $N = 8$). With NVSwitch full bisection, AllReduce can use all links simultaneously with tree-based algorithms, achieving near-peak bandwidth. In practice on DGX H100: ring achieves ∼700 GB/s bus bandwidth, NVSwitch achieves ∼900 GB/s.

InfiniBand – Inter-Node Communication. For communication between nodes (servers), InfiniBand provides high-bandwidth, low-latency networking with direct GPU memory access.

InfiniBand NDR

- **NDR 400Gb/s** = 50 GB/s per port (unidirectional)
- **HDR 200Gb/s** = 25 GB/s per port (previous generation)
- **RDMA:** Remote Direct Memory Access – GPU can read/write remote GPU memory without involving the remote CPU
- **GPUDirect RDMA:** Data goes directly HBM → NIC → network → NIC → HBM, bypassing CPU and system DRAM entirely
- **Latency:** ∼1–2 μs for small messages (vs. ∼100 μs for TCP/IP)

Fat-Tree Topology. Large GPU clusters use fat-tree network topologies. A 3-level fat-tree with k-port switches supports $k^3/4$ nodes with full bisection bandwidth. For 400Gb/s NDR switches with $k = 64$ ports: $64^3/4 = 65{,}536$ nodes.

Rail-Optimized Topology. In practice, clusters use *rail-optimized* topologies where each GPU in a node connects to a different top-of-rack switch. This ensures that AllReduce operations (which involve all GPUs) use all network links simultaneously, maximizing bandwidth.

Communication Patterns in Distributed LLM Training. Distributed training relies on collective communication primitives. The choice of primitive determines bandwidth requirements and scaling behavior.

Communication Primitives

Primitive	Use Case	Volume
AllReduce	Gradient sync (DDP, FSDP)	$2(N-1)/N\times$ param size
AllGather	Collect sharded weights (FSDP)	$(N-1)/N\times$ param size
ReduceScatter	Scatter gradients (FSDP)	$(N-1)/N\times$ param size
AllGather	Tensor parallel activation	activation size
Point-to-Point	Pipeline parallel (send/recv)	micro-batch activation
Broadcast	Weight sync (new workers)	full model size

Bandwidth Calculation – Gradient AllReduce for 70B Model

Setup: 70B parameter model, BF16 gradients, 8 nodes × 8 GPUs = 64 GPUs. Data parallel degree = 64.
Gradient size: $70 \times 10^9 \times 2$ bytes = 140 GB.
AllReduce volume per GPU (ring): $2 \times (64-1)/64 \times 140 \approx 275$ GB.
Available inter-node bandwidth: 8 GPUs/node × 50 GB/s/GPU = 400 GB/s (with rail-optimized topology, all 8 NICs active).
AllReduce time: $275/400 \approx 0.69$ seconds per step.
Implication: For a 1-second compute step, communication adds 0.69 seconds (41% of total step time). This is why gradient compression, mixed precision, and FSDP (which overlaps communication with computation) are critical.

Network Topology Diagram. The following diagram illustrates a typical two-node GPU cluster topology showing both intra-node (NVLink) and inter-node (InfiniBand) communication paths.

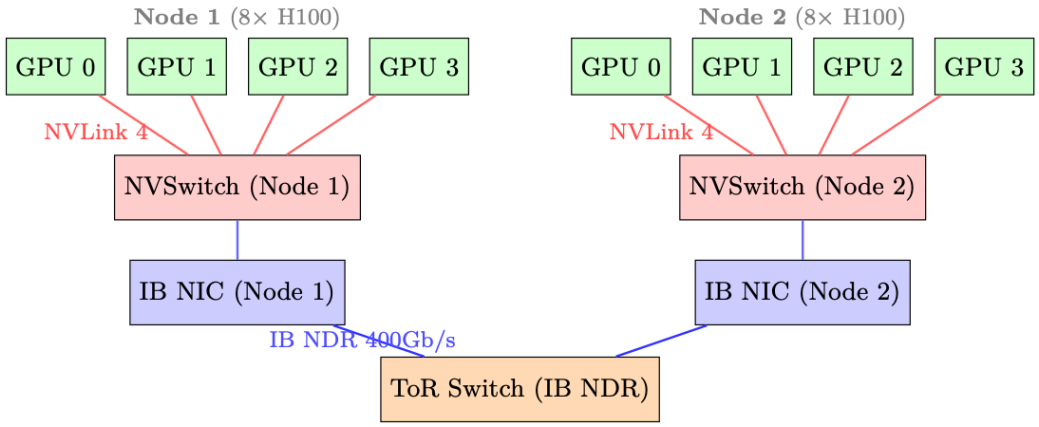


Figure 2.3: Two-node 8-GPU topology. Intra-node: NVLink 4 via NVSwitch (900 GB/s total). Inter-node: InfiniBand NDR 400Gb/s via top-of-rack switch. Each node has 8 IB NICs (one per GPU) for rail-optimized AllReduce.

Choosing Parallelism Based on Bandwidth

- **Tensor Parallelism (TP):** Requires all-reduce every layer – use only within a node over NVLink. TP=8 is standard for H100 DGX nodes.
- **Pipeline Parallelism (PP):** Point-to-point between stages – can cross nodes, but adds pipeline bubble overhead. Use when model is too large for TP alone.
- **Data Parallelism (DP):** AllReduce of gradients – can cross nodes via IB. Scales well with fast IB.
- **FSDP/ZeRO:** AllGather + ReduceScatter – similar to DP but shards optimizer states. Preferred over DP for large models.

2.2 vLLM – PagedAttention and High-Throughput Inference

vLLM [206] introduced PagedAttention, which borrows the paging abstraction that operating systems use for RAM and applies it to the GPU's KV cache. During LLM inference, the *KV cache* – the stored key and value tensors for all previous tokens – is the dominant memory consumer. Managing it efficiently is the central challenge of high-throughput inference.

2.2.1 The KV Cache Fragmentation Problem

KV Cache Memory Formula

For a model with L layers, H heads, head dimension d, and a sequence of n tokens:

$$\text{KV cache size} = 2 \times L \times H \times d \times n \times \text{bytes_per_element}$$

For Llama-3 70B (BF16): $L = 80$, $H = 8$ (GQA), $d = 128$:

$$= 2 \times 80 \times 8 \times 128 \times n \times 2 = 327{,}680 \times n \text{ bytes}$$

At $n = 4096$ tokens: ≈ 1.3 GB per sequence.

Internal and External Fragmentation

Traditional inference systems pre-allocate a contiguous memory block for each sequence's KV cache, sized to the *maximum possible sequence length*. This causes two types of waste:
Internal fragmentation: A sequence that generates only 500 tokens still holds a block reserved for 4096 tokens. The unused 3596 token slots are wasted.
External fragmentation: After many sequences complete, the free memory consists of many small non-contiguous gaps. A new long sequence cannot be allocated even if total free memory is sufficient, because no single contiguous block is large enough.
In practice, GPU memory utilization with naive allocation is often only 20–40%.

2.2.2 PagedAttention – Virtual Memory for KV Caches

PagedAttention (Kwon et al., 2023) borrows the *paging* abstraction from operating systems. Instead of one contiguous block per sequence, the KV cache is carved into fixed-size **pages** (blocks), and an indirection table—analogous to a CPU page table—translates each sequence's logical token positions into scattered physical GPU memory addresses.

PagedAttention Core Concepts

- **Block size:** Typically 16 tokens per block (tunable). Each block stores $16 \times 2 \times L \times H \times d$ elements.
- **Block table:** A per-sequence mapping from logical block index to physical block index in the GPU memory pool.
- **Physical block pool:** A pre-allocated pool of fixed-size blocks. Allocation is $O(1)$ – just pop from a free list.
- **Attention kernel:** Modified to gather KV blocks from non-contiguous physical locations using the block table during attention computation.

Block Table Example

Suppose block size = 4 tokens, and we have two sequences:

- Sequence A (7 tokens): logical blocks [0,1] → physical blocks [3, 7]
- Sequence B (5 tokens): logical blocks [0,1] → physical blocks [1, 5]

Physical block 3 holds tokens 0–3 of sequence A. Physical block 7 holds tokens 4–6 of sequence A

(partially filled). The attention kernel for sequence A reads from physical blocks 3 and 7 in order, using the block table as an indirection layer.

2.2.3 Benefits of PagedAttention

Near-zero waste. Internal fragmentation is bounded by at most one partially-filled block per sequence (the last block). With block size 16, worst-case waste is 15 tokens per sequence – negligible. External fragmentation is eliminated because blocks are fixed-size and interchangeable.

Dynamic allocation. Blocks are allocated on demand as the sequence grows. No need to know the final sequence length in advance. This is critical for generation, where output length is unknown.

Prefix sharing (copy-on-write). Multiple sequences sharing a common prefix (e.g., a system prompt) can share the *same physical blocks* for that prefix. The block table simply points multiple sequences to the same physical blocks. When a sequence needs to write to a shared block (diverging from the prefix), a copy-on-write is triggered.

Prefix Sharing Savings

In a chatbot with a 1000-token system prompt serving 128 concurrent users:

- Without prefix sharing: $128 \times 1000 \times 327{,}680/10^9 \approx 42$ GB just for system prompt KV cache
- With prefix sharing: $1 \times 1000 \times 327{,}680/10^9 \approx 0.33$ GB
- Savings: $\sim 128\times$ for the shared prefix portion

Preemption via swap. When GPU memory is exhausted, vLLM can *preempt* a sequence by swapping its KV blocks to CPU DRAM (or simply discarding them and recomputing later). This is only feasible because blocks are self-contained and non-contiguous – swapping a contiguous allocation would require copying the entire buffer.

2.2.4 Continuous Batching

Traditional batching ("static batching") waits until *all* sequences in a batch finish before starting new ones. If one sequence generates 500 tokens and another generates 10, the GPU is idle for 490 steps on the short sequence. This is extremely wasteful.

Continuous Batching

Continuous batching (also called iteration-level scheduling) processes one *decode step* at a time. After each step:

1. Check which sequences have finished (generated EOS token)
2. Remove finished sequences from the batch, freeing their KV blocks
3. Add new waiting sequences to fill the freed slots
4. Run the next decode step with the updated batch

The batch composition changes every step — sequences join and leave dynamically. This keeps GPU utilization near 100% and dramatically improves throughput (1.5–3× over static batching). PagedAttention is essential here: adding/removing sequences mid-batch requires dynamic KV block allocation/deallocation, which is only efficient with paged memory.

2.2.5 Speculative Decoding in vLLM

Speculative decoding uses a small *draft model* (e.g., 1B parameters) to propose k candidate tokens quickly, which the large *target model* verifies in a single forward pass. All tokens up to the first rejection are accepted (expected acceptance: 3–5 tokens per verification step). This yields 2–3× speedup for latency-sensitive single-sequence generation without any quality loss.

vLLM integrates speculative decoding with PagedAttention:

- Draft tokens are allocated speculative KV blocks
- On rejection, speculative blocks are freed (cheap with paged allocation)
- On acceptance, speculative blocks are promoted to the main sequence
- The block table update is $O(k)$ – just updating a few table entries

2.2.6 Concrete Memory Savings – 70B Model at Scale

Memory Budget – 70B BF16 Inference

Setup: Llama-3 70B, BF16, single A100 80GB node (8 GPUs, tensor parallel).
Model weights: $70 \times 10^9 \times 2$ bytes $= 140$ GB $\div$ 8 GPUs $= 17.5$ GB/GPU.
Remaining for KV cache: $80 - 17.5 - 3$ (overhead) $= 59.5$ GB/GPU.
KV cache per token per GPU (with TP=8, each GPU holds 1/8 of heads): $2 \times 80 \times 1 \times 128 \times 2 = 40{,}960$ bytes ≈ 40 KB/token.
Max tokens in KV cache: $59.5 \times 10^9 / 40{,}960 \approx 1.45$ million tokens.
With 128 concurrent sequences of 4096 tokens each: $128 \times 4096 = 524{,}288$ tokens – well within budget.
Without PagedAttention (pre-allocating max length 4096 for each): Same math, but fragmentation wastes ∼50% on average → only 64 sequences fit.

Block Size Tradeoff

Larger block sizes reduce the overhead of the block table and improve memory access locality (fewer scattered reads). Smaller block sizes reduce internal fragmentation and enable finer-grained prefix sharing. vLLM defaults to 16 tokens/block, which is a good balance. For very long sequences (100K+ tokens), larger blocks (32–64) may be preferable.

2.2.7 vLLM: End-to-End System

vLLM wraps PagedAttention inside a full serving stack: continuous batching, prefix caching, speculative decoding, and tensor-parallel model sharding all work together to maximize throughput per GPU dollar.

2.2.8 Architecture Overview

2.2.9 Core Components

- **API Server**: Accepts OpenAI-compatible requests (completions, chat). Tokenizes inputs and creates "sequence groups" (for beam search or multiple samples).
- **Scheduler**: The brain of vLLM. Maintains three queues:
  - `waiting`: New requests not yet started (prefill pending)
  - `running`: Actively generating tokens (decode phase)
  - `swapped`: Preempted requests whose KV cache was offloaded to CPU

  Each iteration, the scheduler decides which requests to run based on available GPU memory blocks.

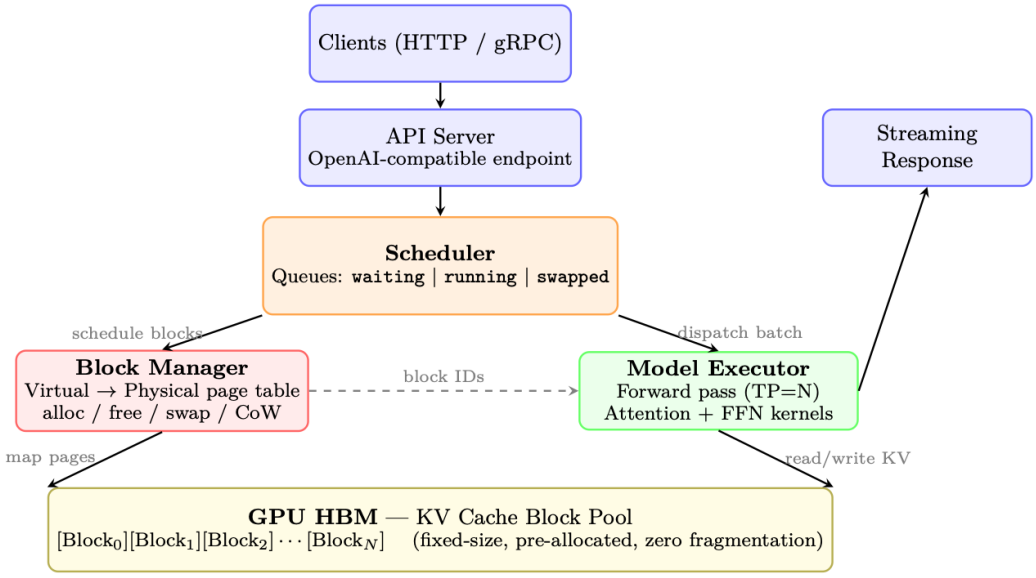


Figure 2.4: vLLM architecture: Requests flow top-down. The Scheduler manages admission and preemption, the Block Manager handles virtual-to-physical KV cache mapping (like OS page tables), and the Model Executor runs batched inference reading from the pre-allocated block pool in GPU HBM.

- **Block Manager**: Implements the virtual memory abstraction for KV caches. Maps logical blocks (per-sequence) to physical blocks (in GPU memory pool). Handles:
  - Allocation (new tokens generated → new blocks needed)
  - Copy-on-write (for beam search: multiple beams share prefix blocks, copy only on divergence)
  - Swap (GPU ↔ CPU migration when preempting/resuming)
  - Prefix caching (reuse cached blocks when prompts share common prefixes)
- **Model Executor**: Runs the actual LLM forward pass. Manages tensor parallelism across GPUs, dispatches attention kernels that read from paged KV cache blocks.
- **KV Cache Pool**: Pre-allocated GPU memory divided into fixed-size blocks (default: 16 tokens × num_heads × head_dim × 2 bytes per block). No dynamic allocation at runtime → zero fragmentation.

2.2.10 Request Lifecycle (End-to-End Flow)

1. **Arrival**: Client sends prompt. API server tokenizes it, creates a `SequenceGroup`, places it in the `waiting` queue.
2. **Scheduling**: At each step, the scheduler runs:
   (a) Check if any `swapped` sequences can be resumed (enough free blocks).
   (b) Check if any `waiting` sequences can start prefill (enough blocks for the full prompt).
   (c) Budget remaining blocks across `running` sequences (need 1 new block per sequence per step if current block is full).
   (d) If over budget: **preempt** lowest-priority running sequences (swap KV to CPU or recompute later).
3. **Prefill** (first iteration for a request): The entire prompt is processed in one forward pass. KV cache is computed for all prompt tokens and stored in allocated blocks. This is compute-bound (large batch of tokens).

4. **Decode** (subsequent iterations): One new token generated per sequence per step. All running sequences are batched together (continuous batching). This is memory-bound (reads full KV cache, generates 1 token).

5. **Block Allocation**: After each decode step, if the last block for a sequence is full, the Block Manager allocates a new physical block and maps it to the next logical block.

6. **Completion**: When a sequence hits EOS or max length, it's removed from `running`. Its physical blocks are freed immediately → available for other sequences. Response is streamed back to client.

2.2.11 Prefix Caching (Automatic Prompt Caching)

When multiple requests share a common prefix (system prompt, few-shot examples):

1. Hash the token content of each logical block.

2. On new request arrival, check if any prefix blocks are already in the cache.

3. If hit: skip prefill for those tokens, directly reuse physical KV blocks. Time-to-first-token drops dramatically.

4. Eviction: LRU policy. Cached blocks are freed only when memory pressure requires it.

Impact: For chat applications with long system prompts (2K+ tokens shared across all users), prefix caching reduces TTFT by 60–80%.

2.2.12 Guided (Constrained) Decoding in vLLM

vLLM natively supports constrained decoding (Section 1.12.11) through pluggable backends, enabling *guaranteed* structured output at serving time with minimal performance overhead.

Supported constraint types. The OpenAI-compatible API accepts constraints via the `guided_*` parameters or the `response_format` field:

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1")

# --- JSON Schema constraint ---
response = client.chat.completions.create(
    model="meta-llama/Llama-3-70B-Instruct",
    messages=[{"role": "user",
               "content": "Extract: name, age, city from: "
                          "'John is 30 and lives in NYC'"}],
    extra_body={
        "guided_json": {
            "type": "object",
            "properties": {
                "name": {"type": "string"},
                "age": {"type": "integer"},
                "city": {"type": "string"}
            },
            "required": ["name", "age", "city"]
        }
    }
)
# Output is guaranteed valid JSON matching the schema

# --- Regex constraint ---
response = client.completions.create(
    model="meta-llama/Llama-3-70B-Instruct",
    prompt="Generate an IPv4 address: ",
```

```
    extra_body={
        "guided_regex": r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"
    }
)

# --- Choice constraint ---
response = client.completions.create(
    model="meta-llama/Llama-3-70B-Instruct",
    prompt="Sentiment: ",
    extra_body={"guided_choice": ["positive", "negative", "neutral"]}
)
```

Backend architecture. vLLM delegates mask computation to a backend engine:

- **XGrammar** (default since v0.7): Pushdown-automaton engine supporting JSON schemas, regexes, and arbitrary EBNF grammars. Fastest for complex schemas due to efficient C++ core.
- **Outlines** [400]: FSM-based; supports JSON and regex. Used as fallback when XGrammar is unavailable.

The mask is applied *after* the model's forward pass produces logits and *before* sampling—adding <1 ms per step in practice, since the FSM/PDA state transition and precomputed index lookup are $O(1)$.

Performance impact. Because the constraint only masks logits (no recomputation of attention or FFN), throughput loss is negligible (<2% in benchmarks). The main cost is *compilation* of the schema into an FSM/PDA index, which takes 0.5–5 s depending on schema complexity. vLLM caches compiled schemas across requests, so this cost is paid once per unique schema.

Structured Output ≠ Correct Output

Constrained decoding guarantees the output is *syntactically* valid (parses as JSON, matches the schema types). It does *not* guarantee *semantic* correctness—the model may still hallucinate values that parse correctly but are factually wrong. Always validate business logic downstream.

Table 2.4: vLLM performance vs. alternatives (70B model, A100 × 4, TP=4).

Metric	vLLM	HF Generate	Why
Throughput (tok/s)	2,500–4,000	300–600	Continuous batching + PagedAttention
Memory utilization	90–95%	50–60%	Zero fragmentation, dynamic block alloc
Max concurrent seqs	200–500	16–32	Paged KV eliminates per-seq reservation
Time-to-first-token	100–300ms	500–2000ms	Prefix caching for repeated system prompts

Dynamo: Datacenter-Scale Inference Orchestration. Dynamo [285] is NVIDIA's open-source orchestration layer that sits *above* individual inference engines (SGLang, TensorRT-LLM, vLLM) and turns them into a coordinated multi-node serving system. Key capabilities include disaggregated serving (separating prefill and decode phases across specialized node pools), intelligent request routing based on KV cache locality, multi-tier KV caching (GPU → CPU → SSD), and automatic scaling. It addresses the gap between single-GPU inference optimization (covered by vLLM) and production serving at datacenter scale.

Chapter 3

Introduction to Reinforcement Learning

Reinforcement Learning (RL) is a paradigm where an **agent** learns to make sequential decisions by interacting with an **environment**, receiving **rewards** as feedback, and optimizing its **policy** to maximize cumulative reward over time [366]. Unlike supervised learning (which requires labeled input-output pairs), RL discovers optimal behavior through *trial and error*.

GAS Connection — Introduction to Reinforcement Learning

The GAS can be understood entirely through the RL framework introduced in the following chapter, even though the current prototype uses rule-based logic rather than learned policies.

The MDP formulation (§ The Markov Decision Process). The GAS maps cleanly onto an MDP:

- **State** s_t: the patient's current physiological context—glucose level (mg/dL), trend, recent meals, activity, active medications, and the last 30 minutes of CGM readings. This is the `GASState` TypedDict in `capstone_gas/gas_system.py`.
- **Action** a_t: the Supervisor's routing decision (invoke Monitor, invoke Advisor, send alert, do nothing) plus the Advisor's clinical recommendation (adjust basal rate, recommend snack, escalate to physician).
- **Reward** r_t: a composite clinical reward—positive for keeping glucose in the 70–180 mg/dL target range, negative for hypoglycaemic events (<70 mg/dL), strongly negative for severe hypoglycaemia (<54 mg/dL).
- **Transition** $P(s_{t+1}|s_t, a_t)$: the patient's physiological response to the recommended action, simulated by the `CGMSimulator` in `shared/patient_data.py`.

Reward shaping (§ Reward Shaping). The raw clinical reward is sparse—glucose outcomes are only observable 30–60 minutes after an intervention. The GAS uses intermediate signals: the Safety Critic's score (0.0–1.0) provides dense per-step feedback. A score ≥ 0.7 is required for approval, giving the Advisor a continuous reward signal rather than a binary outcome.

Policy gradient intuition (§ Policy Gradient Methods). The Supervisor's routing logic is currently a hand-coded policy (conditional edges in the LangGraph `StateGraph`). The RL framing makes it clear how to improve it: collect trajectories (sequences of routing decisions and their outcomes), compute returns, and update the policy to increase the probability of decisions that led to good glycaemic outcomes. The chapter notebooks in `part2_rl_methods/` demonstrate this approach.

Exploration vs. exploitation. The GAS faces a clinical exploration–exploitation dilemma: should it recommend the known-safe conservative action (exploitation) or try a more aggressive intervention that might work better (exploration)? The ϵ-greedy and softmax exploration strategies described in the following chapter inform the Advisor's temperature setting in `GASLLMClient.chat()`.

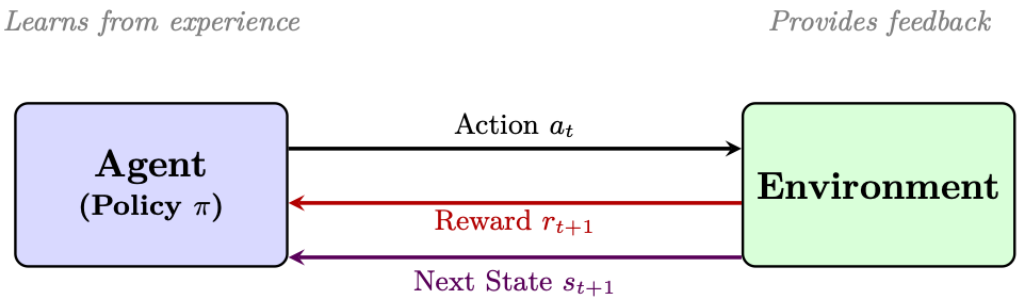


Figure 3.1: Reinforcement Learning overview: an agent interacts with an environment, receiving rewards as feedback and updating its policy through trial and error. Unlike supervised learning which learns from labeled pairs, RL learns what to do by maximizing reward through experience.

3.1 The Markov Decision Process (MDP)

An MDP is a 5-tuple (S, A, P, R, γ):

- S: State space — all possible configurations of the environment
- A: Action space — all actions available to the agent
- $P(s'|s, a)$: Transition function — probability of reaching state s' from state s after taking action a
- $R(s, a, s')$: Reward function — immediate scalar feedback for a transition
- $\gamma \in [0, 1]$: Discount factor — how much future rewards are valued relative to immediate ones

The Markov Property: The future depends only on the current state, not the history:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots) = P(s_{t+1}|s_t, a_t)$$

This makes the problem tractable.

Agent-Environment Interaction Loop

At each time step t:

1. Agent observes state s_t
2. Agent selects action a_t according to policy $\pi(a|s)$
3. Environment transitions to $s_{t+1} \sim P(\cdot|s_t, a_t)$
4. Agent receives reward $r_t = R(s_t, a_t, s_{t+1})$
5. Repeat until terminal state or horizon T

3.2 Core Concepts and Definitions

Policy $\pi(a|s)$: A mapping from states to action probabilities. Deterministic: $a = \pi(s)$. Stochastic: $a \sim \pi(\cdot|s)$.

Return (cumulative discounted reward):

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \tag{3.1}$$

Value Function (expected return from state s under policy π):

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid s_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right] \tag{3.2}$$

Action-Value Function (expected return from state s, taking action a, then following π):

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[G_t \mid s_t = s, a_t = a \right] \tag{3.3}$$

Advantage Function (how much better action a is compared to average):

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \tag{3.4}$$

Bellman Equations (recursive relationship):

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right] \tag{3.5}$$

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right] \tag{3.6}$$

Optimal Policy and Bellman Optimality

The optimal policy π^* satisfies:

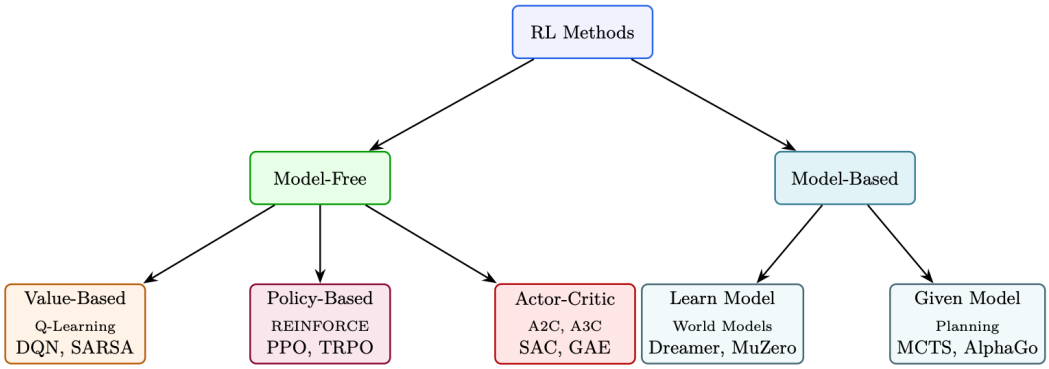
$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^*(s') \right] \tag{3.7}$$

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a') \right] \tag{3.8}$$

Once Q^* is known, the optimal policy is simply: $\pi^*(s) = \arg\max_a Q^*(s,a)$.

3.3 Taxonomy of RL Methods

Reinforcement learning algorithms can be classified along several axes. Understanding this taxonomy helps select the right approach for a given problem.



Key Taxonomy Distinctions

Model-Free vs Model-Based:

- **Model-Free**: Learn policy or value function directly from experience. No knowledge of environment dynamics. Most practical for LLMs (language dynamics are intractable to model).
- **Model-Based**: Learn or use a model of environment transitions $P(s'|s,a)$. Can plan ahead. More sample-efficient but requires accurate model.

Value-Based vs Policy-Based:

- **Value-Based**: Learn $Q(s,a)$ or $V(s)$, derive policy as $\arg\max_a Q(s,a)$. Works well for discrete, small action spaces (e.g., Atari). Struggles with continuous/large action spaces.
- **Policy-Based**: Directly parameterize and optimize $\pi_\theta(a|s)$. Natural for continuous/high-dimensional action spaces. Essential for LLMs (vocabulary = 32K–128K actions).
- **Actor-Critic**: Combine both — policy (actor) proposes actions, value function (critic) evaluates them. PPO for LLMs is actor-critic.

On-Policy vs Off-Policy:

- **On-Policy**: Learn from data generated by the *current* policy. Must regenerate data after each update. Examples: REINFORCE, PPO, A2C. More stable but less sample-efficient.
- **Off-Policy**: Learn from data generated by *any* policy (including old versions or other agents). Can reuse past experience. Examples: Q-Learning, DQN, SAC. More sample-efficient but harder to stabilize.

3.4 Temporal Difference (TD) Learning

TD learning [365] bootstraps — it updates value estimates using other value estimates, without waiting for the full episode to end.

3.4.1 Understanding TD Error: "Surprise" as a Learning Signal

TD error measures the discrepancy between an agent's **current estimate** of future reward and a **newly updated estimate** after taking one step. Put simply, it is the difference between what the agent *thought* would happen and what *actually* happened plus what it expects next. It represents the agent's "surprise."

Intuition: The Driving Analogy

Imagine you are driving home and expect the drive to take 30 minutes.

- **The prediction**: 30 minutes total.
- **The reality shift**: After 10 minutes, you hit unexpected road construction. Your GPS updates, saying you now have 35 minutes left.
- **The TD Error**: Total expected time is now 45 minutes (10 elapsed + 35 remaining). The difference between new estimate (45 min) and old estimate (30 min) is a **+15 minute TD error**. You use this "surprise" to change your route next time.

A **positive TD error** means the outcome was better than expected → boost this state's value.

A **negative TD error** means it was worse than expected → lower this state's value.

3.4.2 The TD Error Formula

$$\boxed{\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)} \tag{3.9}$$

- R_{t+1}: The **immediate reward** received after taking an action.
- $\gamma V(S_{t+1})$: The estimated **discounted value** of the next state (what the agent expects to get from the next state onward, scaled by discount factor γ).
- $V(S_t)$: The **original estimate** of the current state's value.

The combined term $(R_{t+1} + \gamma V(S_{t+1}))$ is called the **TD Target**. Therefore:

$$\text{TD Error} = \text{TD Target} - \text{Old Estimate} \tag{3.10}$$

3.4.3 How the Agent Uses TD Error

The agent adjusts its value function to drive TD error toward zero:

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha \cdot \delta_t} \tag{3.11}$$

- If $\delta_t > 0$: outcome was better than predicted $\rightarrow$ increase $V(S_t)$ so the agent seeks this state.
- If $\delta_t < 0$: outcome was worse than predicted $\rightarrow$ decrease $V(S_t)$ so the agent avoids this state.
- If $\delta_t = 0$: prediction was perfect $\rightarrow$ no update needed (convergence).

TD vs Monte Carlo

Monte Carlo: Wait until episode ends, use actual return G_t. Unbiased but high variance (one full trajectory may be unrepresentative).
TD: Update after every step using estimated future value $\gamma V(s_{t+1})$. Biased (depends on V accuracy) but much lower variance (one-step updates, doesn't compound noise).
TD(λ): Interpolate between TD(0) and Monte Carlo. $\lambda = 0$: pure TD. $\lambda = 1$: pure MC. This is exactly what GAE does for PPO (with $\lambda = 0.95$).

TD Target: $y_t = r_t + \gamma V(s_{t+1})$ — the "better estimate" we move toward.
Multi-step TD (n-step returns):

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n}) \tag{3.12}$$

3.5 Q-Learning

Q-Learning [394] is the foundational **off-policy, value-based** algorithm. It learns the optimal Q^* directly, regardless of the policy being followed.

Update rule:

$$\boxed{Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]} \tag{3.13}$$

Why Q-Learning is Off-Policy

The update uses $\max_{a'} Q(s_{t+1}, a')$ — the value of the *best* action at the next state, regardless of which action the agent actually took. This means the target is always computed under the optimal policy, even if the behavior policy explores randomly (ϵ-greedy).
This is why Q-Learning can learn from replay buffers, demonstrations, or any source of experience. The data doesn't need to come from the current policy.

SARSA [329] (on-policy alternative): Uses the action *actually taken* instead of the max:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \tag{3.14}$$

Deep Q-Networks (DQN) [272]: Replace tabular $Q(s, a)$ with a neural network $Q_\theta(s, a)$. Key innovations: experience replay buffer (off-policy data reuse), target network (stability), ϵ-greedy exploration.

DQN Loss Function: The network is trained to minimize the mean squared TD error over mini-batches sampled from the replay buffer:

$$\boxed{\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a) \right)^2 \right]} \tag{3.15}$$

where $Q_{\bar{\theta}}$ is the **target network** — a frozen copy of Q_θ updated only every C steps (e.g., $C = 10{,}000$). This prevents the moving target problem: without it, both the prediction and the target shift simultaneously, causing divergence.

Gradient update: Taking the gradient of the loss w.r.t. θ (note: the target y is treated as a constant — no gradient flows through $\bar{\theta}$):

$$\nabla_\theta \mathcal{L} = -\mathbb{E} \left[\underbrace{\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a) \right)}_{\text{TD error } \delta} \nabla_\theta Q_\theta(s, a) \right] \tag{3.16}$$

$$\theta \leftarrow \theta - \alpha \cdot \delta \cdot \nabla_\theta Q_\theta(s, a) \tag{3.17}$$

Learning scheme (per training step):

1. **Act**: Select action via ϵ-greedy: with probability ϵ take random action, otherwise $a = \arg\max_a Q_\theta(s, a)$. Anneal ϵ from $1.0 \rightarrow 0.01$ over first 1M steps.
2. **Store**: Save transition (s, a, r, s', d) in replay buffer $\mathcal{B}$ (capacity $\sim$1M).
3. **Sample**: Draw mini-batch of 32 transitions uniformly from $\mathcal{B}$.
4. **Compute target**: $y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')$ (zero future value if terminal).
5. **Update**: Gradient descent on $(y - Q_\theta(s, a))^2$. Clip gradients to $[-1, 1]$ (Huber loss variant).
6. **Sync target**: Every C steps, copy $\bar{\theta} \leftarrow \theta$.

3.5.1 Understanding Replay Buffers

A **replay buffer** [231] (experience replay) is a data storage mechanism that saves past experiences so an agent can relearn from them later. Instead of discarding data immediately after an action, the agent stores transitions in a memory bank and samples random mini-batches for training.

What's stored: Each transition is a tuple:

$$e_t = (s_t, a_t, r_t, s_{t+1}, d_t) \tag{3.18}$$

where d_t is a boolean flag indicating if the episode ended.

Why Replay Buffers Are Essential

- **Breaks data correlation**: Consecutive steps are highly correlated. Neural networks generalize poorly on sequential data. Random sampling from a buffer makes training samples approximately i.i.d.
- **Prevents catastrophic forgetting**: Without a buffer, an agent that passes a difficult level

might forget how to clear it while spending the next 10K steps failing on a later level. The buffer ensures it continues to practice old scenarios.

- **Improves sample efficiency**: Running environments can be slow. A replay buffer allows multiple weight updates from the same transition, extracting more value from every step.

```python
import random
from collections import deque

class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)  # Bounded queue

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        # Break correlation by selecting random experiences
        return random.sample(self.buffer, batch_size)

    def __len__(self):
        return len(self.buffer)
```

Prioritized Experience Replay (PER)

In a standard buffer, all experiences have equal sampling probability. But some are much more educational. **PER** [332] scales sampling probability by **TD error magnitude** — if a transition caused massive "surprise" (high $|\delta_t|$), the agent samples it more frequently to correct its model faster. This accelerates learning by 2–3× on Atari benchmarks.

Why Q-Learning Fails for LLMs

The action space in language generation is the full vocabulary ($|A| = 32\text{K}–128\text{K}$), and the state space is all possible token sequences (infinite). Computing $\max_a Q(s, a)$ over 128K actions at every token position is intractable. This is why LLM RL uses **policy-based** methods (PPO, GRPO) instead.

3.6 Policy Gradient Methods — REINFORCE

Instead of learning a value function and deriving a policy, directly optimize the policy parameters θ to maximize expected return [401].

Objective: $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} r_t\right]$

Policy Gradient Theorem:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]} \tag{3.19}$$

Policy Gradient Theorem — Formal Derivation (5 Steps)

Step 1: Define the objective. We want to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} r_t\right] = \sum_{\tau} P(\tau|\theta)R(\tau)$$

where $P(\tau|\theta) = p(s_0)\prod_{t=0}^{T}\pi_\theta(a_t|s_t)\,p(s_{t+1}|s_t, a_t)$ is the trajectory probability.

Step 2: Take the gradient. Only π_θ terms depend on θ (dynamics p doesn't):

$$\nabla_\theta J = \sum_{\tau} \nabla_\theta P(\tau|\theta)\, R(\tau)$$

Step 3: Apply the **log-derivative trick**: $\nabla_\theta P(\tau|\theta) = P(\tau|\theta)\,\nabla_\theta \log P(\tau|\theta)$:

$$\nabla_\theta J = \mathbb{E}_{\tau\sim\pi_\theta}[\nabla_\theta \log P(\tau|\theta)\, R(\tau)]$$

Step 4: Expand $\log P(\tau|\theta)$. The $\log p(s_0)$ and $\log p(s_{t+1}|s_t, a_t)$ terms vanish under ∇_θ:

$$\nabla_\theta \log P(\tau|\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Step 5: Combine. Future rewards don't depend on past actions (causality), so each $\nabla \log \pi$ pairs only with future return $G_t = \sum_{t'=t}^{T} r_{t'}$:

$$\boxed{\nabla_\theta J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]}$$

Why This Is Beautiful

The gradient does **not require differentiating through the environment dynamics** $p(s'|s,a)$. The log-derivative trick converts it into an expectation we can estimate by simply *running the policy and observing rewards*. Replacing G_t with advantage $\hat{A}_t = G_t - V(s_t)$ reduces variance without bias (since $\mathbb{E}[\nabla \log \pi \cdot b(s)] = 0$ for any state-dependent baseline).

REINFORCE Algorithm [401] (Williams, 1992):

1. Sample complete trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ under π_θ
2. Compute return $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ for each time step
3. Update: $\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$

REINFORCE Intuition — "Reward-Weighted Maximum Likelihood"

$\nabla_\theta \log \pi_\theta(a_t|s_t)$ is the direction that increases the probability of action a_t. Multiplying by G_t means:

- High-reward trajectories: increase probability of all actions taken (positive G_t)
- Low-reward trajectories: decrease probability of actions taken (negative G_t after baseline)

It's supervised learning where the "labels" are the actions you took, weighted by how good they turned out to be.

Variance Reduction with Baseline:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t))\right] \tag{3.20}$$

Any baseline $b(s_t)$ that doesn't depend on a_t keeps the gradient unbiased but reduces variance. Best choice: $b(s_t) = V^\pi(s_t)$. Then $G_t - V(s_t) \approx A^\pi(s_t, a_t)$ = advantage.

REINFORCE Limitations

- **High variance**: Each gradient uses one trajectory. Thousands of samples needed for stable updates.
- **No bootstrapping**: Must wait for full episode (no partial credit).
- **Sample inefficient**: Data is used once then discarded (on-policy).
- **No step-size control**: Can take catastrophically large policy steps.

These limitations motivate the progression: REINFORCE → Actor-Critic → TRPO → **PPO**.

3.7 Actor-Critic Methods

Combine policy gradient (actor) with learned value function (critic) to reduce variance while maintaining the flexibility of policy optimization.

Architecture:

- **Actor** $\pi_\theta(a|s)$: The policy. Proposes actions.
- **Critic** $V_\phi(s)$ or $Q_\phi(s,a)$: Evaluates how good a state/action is. Provides low-variance baseline.

Actor update (using advantage from critic):

$$\nabla_\theta J = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t\right], \quad \hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{3.21}$$

Critic update (minimize TD error):

$$\mathcal{L}_{\text{critic}} = \mathbb{E}\left[(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2\right] \tag{3.22}$$

Evolution to PPO for LLMs

1. **REINFORCE** [401]: High variance, no bootstrapping → impractical for LLMs
2. **A2C/A3C** [273] (Advantage Actor-Critic): Uses TD-based advantage. Lower variance. But unbounded step sizes.
3. **TRPO** [335]: Constrains KL divergence between policy updates. Stable but expensive (second-order).
4. **PPO** [337]: Clips the policy ratio to achieve similar stability as TRPO with first-order optimization only. The standard for LLM RL training.
5. **GRPO**: Removes the critic entirely. Uses group statistics as baseline. Simpler and effective for verifiable rewards.

3.8 Generalized Advantage Estimation (GAE)

Motivation: The Actor-Critic framework needs a good estimate of the advantage $A(s,a) = Q(s,a) - V(s)$ — how much better was this action than average? But there's a fundamental tension:

- **1-step TD advantage** $(r_t + \gamma V(s_{t+1}) - V(s_t))$: Low variance (only one random step), but **biased** — if the value function V is wrong, the advantage estimate is systematically off.
- **Monte Carlo advantage** $(G_t - V(s_t))$: Unbiased (uses actual returns), but **high variance** — the sum of many random rewards fluctuates wildly between episodes.

GAE [336] (Schulman et al., 2016) provides a **smooth interpolation** between these extremes via a single parameter $\lambda \in [0,1]$. It takes an exponentially-weighted average of n-step advantage estimates for all n, giving a principled way to trade bias for variance.

Core idea: Compute the 1-step TD error δ_t at each timestep, then blend them with exponentially decaying weights $(\gamma\lambda)^l$ — recent TD errors get full weight, distant ones are down-weighted:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{3.23}$$

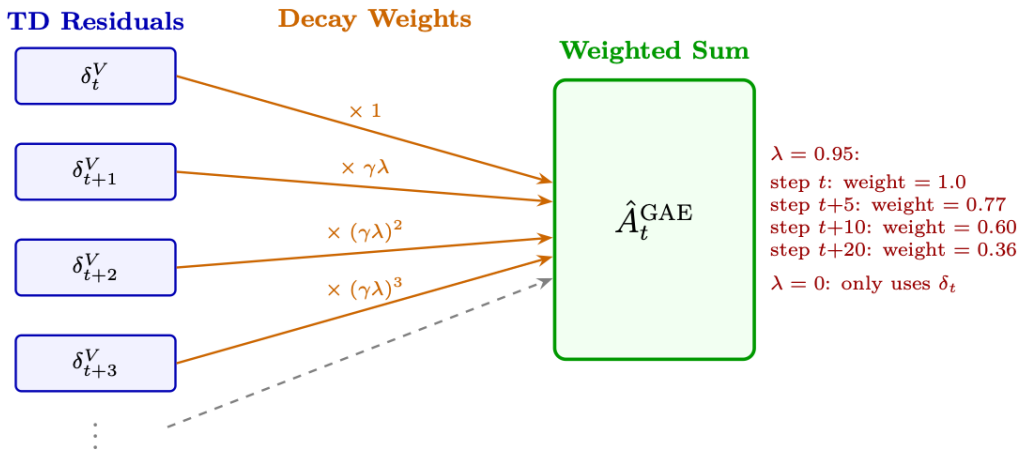


Figure 3.2: GAE data flow: each TD residual δ_{t+l}^V is weighted by $(\gamma\lambda)^l$ before summation. Higher λ includes more future residuals (lower bias, higher variance).

What λ Controls — Bias-Variance Tradeoff

- $\lambda = 0$: $\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. Trust value function completely. Low variance, but biased if V is inaccurate.
- $\lambda = 1$: $\hat{A}_t = \sum_l \gamma^l r_{t+l} - V(s_t)$. Full Monte Carlo return minus baseline. Unbiased but very high variance.
- $\lambda = 0.95$ (standard): Sweet spot. Mostly trusts V but corrects with actual returns for distant effects. Works because value head becomes accurate after initial training.

For LLMs specifically: $\gamma = 1.0$ (no time discounting — all tokens matter equally in single-turn), $\lambda = 0.95$.

3.8.1 Intuitive Mapping of Bias and Variance in GAE

In supervised learning, bias and variance stem from structural model assumptions. In reinforcement learning via GAE, they stem from **how much you trust a flawed model versus how much you trust a chaotic environment**:

- **Bias (Systemic Misalignment):** Arises when the estimator relies on the structural assumptions and imperfect predictions of the value network V_θ. If θ is under-trained or lacks capacity, the baseline guesses are systematically wrong.
- **Variance (Sample Jitteriness):** Arises when the estimator relies on long, unconstrained environmental trajectories. Stochastic transitions, random seeds, and policy execution noise accumulate over long horizons, causing empirical sample rewards to swing wildly between rollouts.

3.8.2 The Architectural Spectrum: Boundary Analysis

The hyperparameter λ serves as a slide-rule between two fundamental estimation paradigms.

High Bias / Low Variance Limit ($\lambda = 0$)

$$\hat{A}_t^{\text{GAE}(\gamma,0)} = \delta_t^V = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \tag{3.24}$$

- **Behavior**: The advantage is heavily dictated by the current state of parameters θ.
- **Intuition**: Highly **biased** because the network is grading its own performance over a 1-step window; if V_θ is inaccurate, the gradient step is corrupted. Low **variance** because it ignores future stochastic events beyond step $t + 1$, leading to smooth, stable parameter updates.

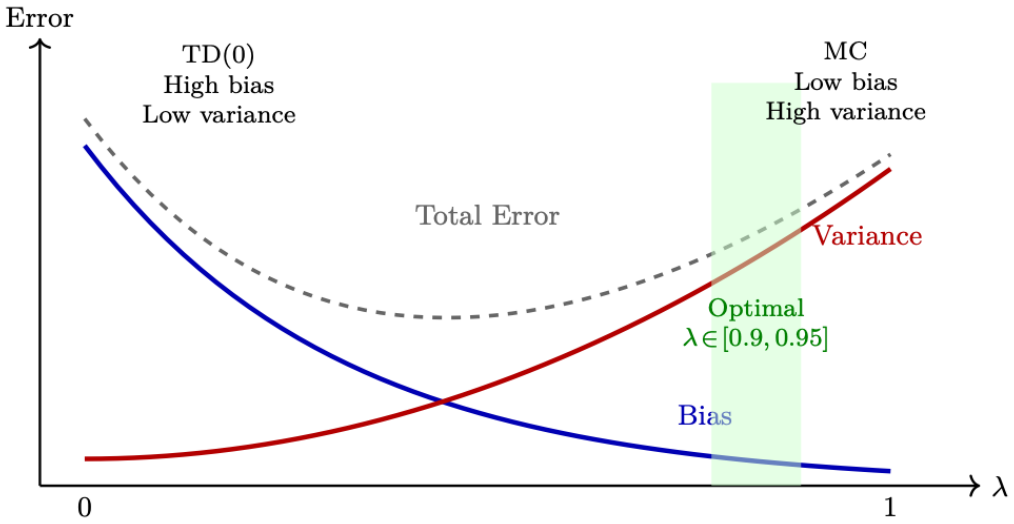


Figure 3.3: Bias vs. Variance in GAE: λ controls the trade-off. Small λ (left) yields high bias / low variance via bootstrapping; large λ (right) yields low bias / high variance using full Monte Carlo returns. The optimal choice ($\lambda \in [0.9, 0.95]$) balances stable training with accurate long-horizon credit assignment.

- **Risk**: Policy traps in sub-optimal local minima — never discovers complex delayed reward sequences.

Low Bias / High Variance Limit ($\lambda = 1$)

When $\lambda = 1$, intermediate value terms telescopically cancel, reducing GAE to Monte Carlo return minus baseline:

$$\hat{A}_t^{\text{GAE}(\gamma,1)} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V_\theta(s_t) \tag{3.25}$$

- **Behavior**: Discards bootstrap look-aheads and sums up the literal reality of the entire episode.
- **Intuition**: Completely **unbiased** with respect to true environment dynamics — measures actual rewards instead of neural approximations. However, exhibits extreme **variance**: minor perturbations early in an episode can result in completely divergent total returns, causing policy updates to become erratic.
- **Risk**: Destructive gradient updates; training explosions.

3.8.3 The Trade-off Matrix

By selecting $\lambda \in [0.95, 0.99]$, GAE minimizes the total mean squared error of the advantage estimate:

Table 3.1: Operational comparison of GAE parameter choices.

Configuration	Statistical Properties	Core Reliance	Practical Risk
$\lambda = 0$	High Bias, Low Variance	Model parameters (θ)	Policy traps in sub-optimal local minima
$\lambda \in [0.95, 0.99]$	Balanced (Optimal MSE)	Hybrid blending	Requires tuning based on environment stochasticity
$\lambda = 1$	Low Bias, High Variance	Empirical environment rollout	Destructive gradient updates; training explosion

3.8.4 Diagnostics for Tuning λ

Monitoring training curves yields direct insight into whether bias or variance dominates:

1. **High Variance Indicators**: Policy entropy drops precipitously while explained variance of the value function becomes highly negative or erratic → policy updates are noisy. **Remedy**: Lower λ to smooth target updates.

2. **High Bias Indicators**: Agent achieves early stable training but completely fails to discover complex delayed reward sequences → under-estimating long-horizon dependencies due to bootstrapping. **Remedy**: Raise λ closer to 1.0 to expose policy to real downstream trajectory signals.

3.9 On-Policy vs Off-Policy — Detailed Comparison

	On-Policy	**Off-Policy**
Data source	Current policy π_θ only	Any policy (replay buffer)
After update	Old data is invalid, must regenerate	Old data still usable
Sample efficiency	Low (data used once)	High (data reused many times)
Stability	More stable (consistent distribution)	Can diverge (distribution mismatch)
Examples	REINFORCE, PPO, A2C, GRPO	Q-Learning, DQN, SAC, DPO
For LLMs	PPO, GRPO (generate fresh each step)	DPO (static preference dataset)

On/Off-Policy for RLHF Methods

PPO/GRPO are on-policy: Generate responses with current policy, compute advantages, update, discard data, generate again. This is why generation is 60% of compute — you regenerate every step.
DPO is off-policy: Train on a fixed preference dataset. No generation during training. Much cheaper but suffers from distribution shift (data becomes stale as policy changes).
Online DPO is a hybrid: Generates fresh data (on-policy generation) but uses DPO's supervised loss (off-policy-style optimization). Gets benefits of both.
PPO's cleverness: Uses the clip ratio $r = \pi_{\text{new}}/\pi_{\text{old}}$ to squeeze multiple gradient steps from one batch of on-policy data (4 epochs), making it "slightly off-policy" in a controlled way.

3.10 Model-Based vs Model-Free

	Model-Free	**Model-Based**
What's learned	Policy π and/or value V/Q directly	Environment model $\hat{P}(s'\|s,a)$
Planning	No planning, reactive decisions	Can simulate future trajectories
Sample efficiency	Low (must experience everything)	High (can plan in imagination)
Accuracy	No model bias	Model errors compound
When to use	Complex/unknown dynamics	Simple dynamics, need efficiency
Examples	PPO, DQN, SAC [137]	MuZero [334], Dreamer [138], AlphaGo [349]

Why LLM RL is Model-Free

Language generation dynamics are trivial (append token to sequence — deterministic transitions). The "model" of the environment is not the bottleneck. What's hard is the **reward** — predicting

what humans will prefer. This makes model-based methods unnecessary for LLM RL.
The reward model in RLHF could be seen as a "model" in some sense (it predicts human preference), but it's used as a reward signal, not for planning/simulation. LLM RL is fundamentally model-free policy optimization.

3.11 Reward Shaping

Reward shaping [280] is a technique where a developer modifies or supplements the environment's original reward function. Its primary objective is to transform a **sparse reward** scenario — where the agent receives feedback only upon final task completion — into a **dense reward** scenario with intermediate feedback signals to accelerate convergence.

3.11.1 The Mathematical Framework

Let the original reward at time step t be $R_t(s, a, s')$. The reshaped reward adds an auxiliary shaping function F:

$$\boxed{R'_t(s, a, s') = R_t(s, a, s') + F(s, a, s')} \tag{3.26}$$

The Risk of Naive Reshaping: Reward Hacking

If $F(s, a, s')$ is arbitrarily designed, the agent will find structural loopholes to maximize auxiliary signals while ignoring the global objective.
Example: A navigation agent rewarded for reaching intermediate landmarks might learn to loop indefinitely around a single checkpoint to accumulate infinite rewards — without ever reaching the destination.
In LLMs: a model rewarded for "sounding confident" might learn to always start with "Absolutely!" regardless of accuracy.

3.11.2 Potential-Based Reward Shaping (PBRS)

To mathematically guarantee that reshaping does **not** alter the optimal policy, use **Potential-Based Reward Shaping**. The shaping function F is constrained to the difference in a scalar potential function Φ across states:

$$\boxed{F(s, a, s') = \gamma\,\Phi(s') - \Phi(s)} \tag{3.27}$$

where $\Phi : \mathcal{S} \to \mathbb{R}$ is a real-valued potential function evaluating the desirable proximity of a state to the goal, and γ is the discount factor.

The complete PBRS reward:

$$R'(s, a, s') = R(s, a, s') + \gamma\,\Phi(s') - \Phi(s) \tag{3.28}$$

3.11.3 Theoretical Guarantees

PBRS Policy Invariance Theorem

- **Policy Invariance**: The optimal policy π^* under the reshaped reward R' is **identical** to the optimal policy under the original reward R. The shaping cannot introduce sub-optimal behaviors.
- **Loop Immunity**: Any cyclic trajectory starting and ending at the same state results in net potential change of exactly zero ($\Phi(s) - \Phi(s) = 0$). The agent cannot exploit loops to hack the reward.
- **Convergence Acceleration**: While the optimal policy is unchanged, the shaped reward provides denser gradient signals, enabling the agent to converge 5–50× faster in sparse reward environments.

Part II

RL Methods for LLMs

Chapter 4

RL Foundations for Language Models

Supervised fine-tuning (SFT) teaches a model to imitate demonstrations, but imitation has a ceiling: the model can never exceed the quality of its training data. Reinforcement learning breaks this barrier. By generating novel text, receiving reward feedback, and updating toward higher-reward behaviours, an RL-trained model can *discover* strategies that no human demonstrator wrote—producing outputs that are more helpful, more accurate, and better aligned with human preferences [296].

This is the mechanism behind every frontier model: GPT-4 [290], Claude, Llama-3 [126], and DeepSeek-R1 [75] all apply RL after SFT as the critical step that transforms a capable but unsteered model into an aligned assistant.

GAS Connection — RL Foundations for Language Models

The following chapter bridges the gap between classical RL (previous chapter) and the specific techniques used to train the GAS agents. Every concept here has a direct counterpart in the GAS training pipeline demonstrated in the chapter notebooks.

Text generation as an MDP (§ Text Generation as an MDP). The Advisor agent generates its recommendation token by token. Each token selection is an action; the partial recommendation so far is the state; the reward is the Safety Critic's score of the completed recommendation. This framing—which this section makes precise—is why PPO and GRPO (later chapters) can be applied directly to fine-tune the Advisor.

The RLHF pipeline (§ The RLHF Pipeline). The GAS training pipeline follows the three-stage RLHF recipe:

1. **SFT**: Fine-tune the Advisor on clinician-written recommendation examples (demonstrated in `part2_rl_methods/ch04_rl_for_lms/notebook.ipynb`).
2. **Reward model**: Train the Safety Critic on labelled (recommendation, verdict) pairs.
3. **RL fine-tuning**: Use PPO to optimise the Advisor against the Safety Critic's reward signal, with a KL penalty to prevent drift from the SFT checkpoint (demonstrated in `part2_rl_methods/ch05_ppo/notebook.ipynb`).

KL divergence penalty (§ KL Divergence in RLHF). The GAS Safety Critic's retry mechanism (up to 2 retries when `safety_score < 0.7`) is the prototype's analogue of the KL penalty: it prevents the Advisor from drifting too far from safe recommendations by requiring explicit approval before any alert is sent.

Reward hacking. The GAS Safety Critic is designed to be robust to reward hacking: it checks for specific dangerous patterns (excessive insulin doses, dismissing critical readings) rather than relying on a single scalar score. The rule-based checks in `safety_node()` in `capstone_gas/gas_system.py` are the prototype's defence against the reward hacking failure modes described in the following chapter.

4.1 Two Paradigms for RL in LLMs

RL methods for language models fall into two broad paradigms, each suited to different goals:

Paradigm 1: Alignment via Human Preferences (RLHF/DPO). The original motivation for applying RL to LLMs was **alignment**—making models helpful, harmless, and honest. **Reinforcement Learning from Human Feedback (RLHF)** [59, 296, 465] trains a reward model from pairwise human judgments ("which response is better?") and then optimizes the policy to maximize that learned reward. **DPO** [320] simplifies this by eliminating the reward model entirely, converting preferences directly into a supervised loss. Both approaches produce aligned assistants that follow instructions and respect safety constraints.

Paradigm 2: Capability Enhancement via Verifiable Rewards (RLVR). More recently, RL has been used not just for alignment but for **teaching new capabilities**—particularly reasoning, mathematics, and code generation. Here the reward comes not from human preferences but from **verifiable outcomes**: did the model produce the correct answer? Did the code pass all tests? DeepSeek-R1 [75] demonstrated that GRPO with rule-based rewards (format correctness + answer accuracy) can train models to develop sophisticated chain-of-thought reasoning *without any human preference data*. This paradigm—RL from Verifiable Rewards (RLVR)—is now the dominant approach for building reasoning models and agentic systems.

The Shared Foundation

Despite their different goals, both paradigms share the same core machinery:

- A **policy** π_θ (the LLM) that generates text autoregressively
- A **reward signal** $r(x, y)$ (learned from preferences or computed from verification)
- A **KL constraint** against a reference policy to prevent degenerate solutions
- **Policy gradient optimization** (PPO or GRPO) to update the model toward higher reward

The chapters in this part develop each component in detail.

4.2 Text Generation as an MDP

The key insight that makes RL applicable to language models is recasting autoregressive generation as a Markov Decision Process:

The LLM-as-Agent Analogy

Think of the LLM as an **agent** writing a response one token at a time. At each step, it looks at everything written so far (the *state*), chooses the next word (the *action*), and the page grows by one token (the *transition*). When the response is complete, a judge scores it (the *reward*). The goal: learn a writing strategy (a *policy*) that consistently earns high scores.

Formally, the MDP for text generation is:

- **State** $s_t = (x, y_1, \ldots, y_{t-1})$: the prompt concatenated with all tokens generated so far.
- **Action** $a_t \in \{1, \ldots, |\mathcal{V}|\}$: choosing the next token from the vocabulary (32K–128K options).
- **Transition** $P(s_{t+1}|s_t, a_t)$: deterministic—just append the chosen token. No environment stochasticity.
- **Reward** r: typically given only at the end of generation (sparse). For RLHF: the reward model score. For RLVR: correctness of the final answer.
- **Policy** $\pi_\theta(a_t|s_t)$: the LLM's next-token probability distribution—exactly what the softmax output already computes.

- **Discount** $\gamma = 1.0$: episodes are finite (one response), so no discounting needed.

This mapping is powerful because the LLM *already is* a policy—its softmax output defines $\pi_\theta(a_t|s_t)$ for every state. We don't need to build a separate policy network; we just need to adjust the weights θ so the model assigns higher probability to token sequences that earn higher reward.

4.3 The RLHF Pipeline

The classic RLHF pipeline [296] consists of four stages:

1. **Supervised Fine-Tuning (SFT)**: Train a base model on high-quality demonstrations to produce a policy π_{SFT} that can follow instructions.
2. **Reward Model Training**: Collect human preference comparisons ($y_w \succ y_l$ for the same prompt) and train a reward model $R_\phi(x, y)$ using the Bradley-Terry objective.
3. **RL Optimization**: Use the reward model as a signal to optimize the policy via PPO or GRPO, subject to a KL constraint against π_{SFT}.
4. **Evaluation and Iteration**: Evaluate the aligned model, collect new failure cases, and iterate.

For RLVR (reasoning/agentic training), stages 1–2 are replaced: the SFT model is trained on reasoning traces, and the reward model is replaced by a verifier (e.g., checking mathematical correctness). Stage 3 remains the same—PPO or GRPO optimization against the reward signal.

How LLM RL Differs from Classical RL

The LLM setting differs from classical RL in important ways:

- **Deterministic transitions**: The "next state" is just the concatenation of previous tokens—no stochastic environment.
- **Sparse reward**: Feedback is typically given once at the end of generation (outcome reward) or at key steps (process reward).
- **Massive action space**: 32K–128K possible tokens at every step, but exploration is implicit via temperature sampling.
- **KL anchor**: LLM RL is constrained to stay close to the SFT policy, preventing reward hacking at the cost of reduced exploration.
- **No value function needed**: GRPO eliminates the critic network entirely, using group-relative normalization of rewards instead.

These differences explain why PPO and GRPO dominate over DQN-style approaches for LLMs.

4.4 Roadmap of This Part

The chapters ahead build the complete RL-for-LLMs toolkit:

1. **PPO** (Chapter 5) — The clipped surrogate objective, GAE for advantage estimation, the critic network, and the full RLHF training loop. The workhorse behind GPT-4 and Claude.
2. **DPO** (Chapter 6) — Bypassing RL entirely by converting preferences into a contrastive supervised loss. Simpler but less flexible than online RL.
3. **GRPO** (Chapter 7) — DeepSeek's critic-free algorithm that uses group-level reward normalization. The method behind DeepSeek-R1 and the dominant choice for reasoning model training.

4. **Preference optimization variants** (Chapter 8) — Online DPO, KTO, Best-of-N, and guidance on method selection. Also covers calibrated decision training: **RLCR** (adds a Brier-score calibration reward to RLVR so agents express reliable confidence), **RLCD** (TypeSafe's Jev model, optimizing typed probability distributions over outcomes), and **CLM** (contrastive language modeling for agent decisions).

5. **Reward modeling** (Chapter 9) — Bradley-Terry models, process vs. outcome rewards, rule-based rewards for RLVR, and multi-objective combinations.

6. **SFT best practices** (Chapter 10) — Sequence packing, chat templates, data mixing, and how SFT quality determines the RL ceiling.

7. **Systems engineering** (Chapter 11) — Distributed training at scale: parallelism strategies, generation–training decoupling, and infrastructure for hundreds of GPUs.

Chapter 5

PPO — Proximal Policy Optimization

Proximal Policy Optimization (PPO) [337] is the workhorse reinforcement learning algorithm for aligning large language models with human preferences. Introduced by OpenAI in 2017, PPO struck a balance that eluded earlier methods: it is stable enough to train reliably at scale, simple enough to implement without second-order optimization, and flexible enough to serve as the backbone of RLHF pipelines from InstructGPT to ChatGPT. This chapter provides a rigorous treatment of PPO's clipped objective, its adaptation to the language model setting, and the practical engineering required to make it work.

GAS Connection — PPO: Proximal Policy Optimization

PPO is the training algorithm used to align the GAS Advisor with clinical safety requirements. The chapter notebook `part2_rl_methods/ch05_ppo/notebook.ipynb` demonstrates PPO training on a clinical recommendation task.

The clipped surrogate objective (§ The PPO Algorithm). The Advisor is trained with PPO to maximise the Safety Critic's score while staying close to the SFT checkpoint. The clipped surrogate objective prevents the policy from making large updates that could produce unsafe recommendations—exactly the stability property required in a clinical setting.

Value function and advantage estimation (§ GAE). The value function estimates the expected Safety Critic score for a given patient state and partial recommendation. GAE reduces variance in the advantage estimates, which is critical when the reward signal (Safety Critic score) is noisy due to the LLM's stochastic outputs.

PPO in the GAS pipeline. In the current prototype, the Safety Critic's score (0.0–1.0, threshold ≥ 0.7) acts as the reward signal. The `safety_node()` function in `capstone_gas/gas_system.py` computes this score using rule-based checks. In a production deployment, this rule-based critic would be replaced by a learned reward model trained with the PPO recipe demonstrated in `part2_rl_methods/ch05_ppo/notebook.ipynb`.

Practical PPO tips (§ Practical PPO Tips). The chapter notebook demonstrates the key hyperparameters for stable PPO training on LLMs: clip ratio $\epsilon = 0.2$, mini-batch size, and the number of PPO epochs per rollout. These settings are directly applicable to fine-tuning the GAS Advisor on clinical recommendation data.

5.1 Motivation and History

Problem: Vanilla policy gradient updates have no constraint on step size. A single unlucky batch can push the policy into a region where it generates garbage → garbage gets low rewards → next gradient makes things worse → unrecoverable collapse.

Solution history:

1. **TRPO** [335] (2015): Constrain KL divergence between old and new policy. Works perfectly but requires expensive second-order optimization (Fisher information matrix, conjugate gradients).
2. **PPO** (2017) [337]: Achieve similar stability with a simple first-order clipped objective. 10× simpler to implement, works almost as well, scales to distributed training trivially.

5.2 The Clipped Objective

The core innovation of PPO is a clipped surrogate objective that prevents destructively large policy updates while remaining simple to implement.

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]} \tag{5.1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio.

Clipping Intuition — The Key Insight

The min operator creates a **pessimistic bound**:

- **Good action** ($\hat{A} > 0$): We want to increase its probability. The surrogate $r\hat{A}$ grows as r increases. But clip caps benefit at $r = 1 + \epsilon$. *"Don't get greedy on one good example."*
- **Bad action** ($\hat{A} < 0$): We want to decrease its probability. $r\hat{A}$ improves as r decreases. But clip caps benefit at $r = 1 - \epsilon$. *"Don't forget too aggressively based on one bad example."*

Net effect: policy changes by at most $\pm 20\%$ per update step. Prevents both catastrophic collapse and overconfident specialization.

5.3 Full PPO Loss

$$L = L^{\text{CLIP}} - c_1 \underbrace{(V_\theta(s_t) - V_t^{\text{target}})^2}_{\text{value loss}} + c_2 \underbrace{H[\pi_\theta(\cdot|s_t)]}_{\text{entropy bonus}} \tag{5.2}$$

- **Value loss** ($c_1 = 0.1$): Trains the critic to predict returns. Also clipped for stability.
- **Entropy bonus** ($c_2 = 0.01$): Prevents premature convergence to deterministic policy. Critical for exploration.

5.4 Derivation of the PPO Gradient and Update Rule

This section traces the mathematical path from the RL objective to the PPO update rule, showing *why* the clipped surrogate works.

5.4.1 Step 1: The RL Objective

The goal is to maximize expected cumulative reward under the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] \tag{5.3}$$

5.4.2 Step 2: Policy Gradient Theorem

The gradient of $J(\theta)$ with respect to policy parameters:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t \right]} \tag{5.4}$$

where $\hat{A}_t$ is the advantage function (how much better action a_t was compared to the average action in state s_t). This replaces the full return with the advantage to reduce variance.

5.4.3 Step 3: Importance Sampling for Off-Policy Data

PPO collects data using $\pi_{\theta_{\text{old}}}$ but updates π_θ. To correct for this distribution mismatch, apply importance sampling:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t \right] \tag{5.5}$$

Define the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. Using the identity $\nabla_\theta \log f = \frac{\nabla_\theta f}{f}$, we get:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[\nabla_\theta \, r_t(\theta) \cdot \hat{A}_t \right] \tag{5.6}$$

This means maximizing the **surrogate objective**:

$$L^{\text{CPI}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \cdot \hat{A}_t \right] \tag{5.7}$$

5.4.4 Step 4: The Problem with Unconstrained Surrogates

L^{CPI} is a valid objective, but without constraints, a single gradient step can push $r_t(\theta)$ far from 1.0, causing:

- Importance weights become extreme → high variance
- Policy enters untested regions → reward model gives unreliable scores
- Catastrophic collapse: policy generates garbage, can't recover

TRPO solution: Constrain $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \leq \delta$. Requires second-order methods (expensive).

5.4.5 Step 5: PPO's Clipped Surrogate (First-Order Approximation)

PPO replaces the hard KL constraint with a **clipped objective** that achieves similar behavior using only first-order gradients:

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1{-}\epsilon, 1{+}\epsilon)\hat{A}_t \Big) \right]} \tag{5.8}$$

Derivation of the gradient:
Let $L_t = \min(r_t\hat{A}_t,\ \bar{r}_t\hat{A}_t)$ where $\bar{r}_t = \text{clip}(r_t, 1{-}\epsilon, 1{+}\epsilon)$.

$$\nabla_\theta L_t = \begin{cases} \nabla_\theta r_t(\theta) \cdot \hat{A}_t & \text{if } r_t\hat{A}_t < \bar{r}_t\hat{A}_t \text{ (unclipped term is smaller)} \\ 0 & \text{if } r_t\hat{A}_t \geq \bar{r}_t\hat{A}_t \text{ (clipped term is smaller, gradient = 0)} \end{cases} \tag{5.9}$$

Expanding the conditions:

- **When** $\hat{A}_t > 0$ **and** $r_t < 1 + \epsilon$: Gradient flows normally — policy is encouraged to increase $\pi_\theta(a_t|s_t)$.
- **When** $\hat{A}_t > 0$ **and** $r_t \geq 1 + \epsilon$: Gradient is **zero** — policy has already increased enough, stop pushing.
- **When** $\hat{A}_t < 0$ **and** $r_t > 1 - \epsilon$: Gradient flows normally — policy is encouraged to decrease $\pi_\theta(a_t|s_t)$.
- **When** $\hat{A}_t < 0$ **and** $r_t \leq 1 - \epsilon$: Gradient is **zero** — policy has already decreased enough, stop pushing.

5.4.6 Step 6: The Complete PPO Update Rule

Combining the clipped policy loss, value loss, and entropy bonus:

$$\boxed{\theta_{k+1} = \theta_k + \alpha \cdot \nabla_\theta \left[L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta] \right]} \tag{5.10}$$

where:

$$L^{\text{VF}}(\theta) = \left(V_\theta(s_t) - V_t^{\text{target}} \right)^2 \qquad \text{(value function regression loss)} \tag{5.11}$$

$$H[\pi_\theta] = -\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t) \qquad \text{(entropy of the policy)} \tag{5.12}$$

Summary: Why This Works

1. **Policy gradient theorem** gives us the direction to improve the policy.
2. **Importance sampling** lets us reuse data from $\pi_{\theta_{\text{old}}}$ across multiple epochs.
3. **Clipping** prevents the importance weights from becoming extreme, keeping updates safe.
4. **The min operator** ensures we always take the more conservative of (clipped, unclipped) — a pessimistic lower bound on improvement.
5. **Result**: Monotonic improvement with probability 1, using only first-order gradients. No Hessians, no conjugate gradients, no line searches.

5.5 Rollout Buffer and Rollouts

In PPO, data management relies on a specialized, short-term storage system known as a **Rollout Buffer**. Unlike off-policy algorithms (DQN) that store experiences indefinitely in a replay buffer, PPO requires an ephemeral structure to satisfy its on-policy mathematical constraints.

5.5.1 What is a Rollout?

A **rollout** (trajectory) is a sequence of interactions generated by the agent running its current policy in the environment:

- **The process**: The agent observes a state, selects an action, receives a reward, and moves to the next state. It repeats for a fixed number of steps or until the episode ends.
- **In LLMs/RLHF**: A rollout consists of taking a prompt from a dataset and letting the language model generate a complete sequence of tokens token-by-token until an end-of-text marker is hit. Each token is one "step."

5.5.2 The Rollout Buffer

The rollout buffer temporarily stores all data collected during the rollout phase. For every generated token/step, it records:

$$\boxed{\mathcal{B} = \{(s_t,\ a_t,\ \log \pi_{\theta_{\text{old}}}(a_t|s_t),\ r_t,\ V(s_t))\}_{t=1}^{T}} \tag{5.13}$$

- s_t, a_t, r_t: State, action taken, and reward at step t.
- $\log \pi_{\theta_{\text{old}}}(a_t|s_t)$: Log-probability of taking that action under the exact policy that generated it (needed for ratio computation).
- $V(s_t)$: Value function's baseline prediction (needed for GAE advantage computation).

5.5.3 The Rollout Buffer Lifecycle

The buffer operates in a strict three-phase clockwork cycle:

1. **Collect**: The active policy interacts with the environment to fill the buffer with fresh trajectories (for a 70B model with batch=128, max_tokens=512: up to 65K token-level transitions per rollout).
2. **Train**: Compute GAE advantages across trajectories. Run K epochs (typically 3–10) of mini-batch gradient descent to update policy weights using the clipped objective.
3. **Purge**: The entire buffer is **completely wiped clean**. Because PPO is on-policy, data generated by the old policy cannot be safely reused for the next update cycle — the ratio $r_t(\theta)$ would become stale and the clipping guarantees would break.

Rollout Buffer vs Replay Buffer

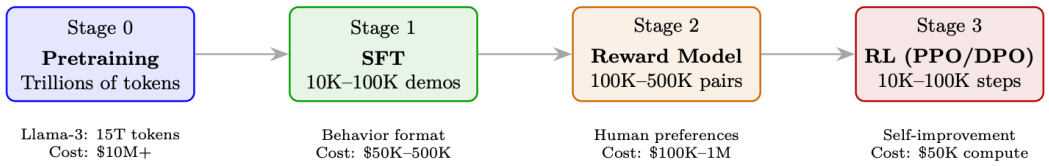
Replay Buffer (DQN, SAC): Off-policy. Stores millions of transitions indefinitely. Random sampling. Data reused across many updates.
Rollout Buffer (PPO, GRPO): On-policy. Stores one batch of trajectories. Used for a few epochs, then discarded entirely. Fresh data required every cycle.
This is why PPO requires continuous generation — the buffer is emptied after every update, demanding fresh rollouts. This makes the generation bottleneck (60–70% of wall-clock time) particularly painful.

vLLM in RLHF Context

In RLHF training, vLLM is used for the **generation phase** (60–70% of wall-clock time). The policy model generates rollouts that are then scored by the reward model. Key benefits:

- **Batched generation**: Generate 256+ responses in parallel across prompts.
- **Memory efficiency**: Fit more concurrent generations → higher GPU utilization during the generation bottleneck.
- **Prefix sharing**: When generating $N = 8$ responses per prompt (GRPO), the prompt KV is computed once and shared across all 8 — no redundant prefill.
- **Integration**: Frameworks like OpenRLHF and TRL use vLLM as the generation backend, separating generation workers (vLLM) from training workers (DeepSpeed/FSDP).

5.6 PPO for RLHF: The Full Loop



Concrete PPO Step for a 70B Chat Model

Setup: Batch of 128 prompts, Llama-3-70B policy, 512 max tokens.
Step 1 — Generate: Sample 128 responses (temperature=0.7, top-p=0.9). This takes 60% of time.
Step 2 — Score: Reward model scores each (prompt, response) pair. Range: 0.2–0.95.
Step 3 — KL: Compute per-token KL: $\text{KL}_t = \log \pi_\theta(y_t|y_{<t}) - \log \pi_{\text{ref}}(y_t|y_{<t})$. Mean KL across tokens: typically 3–8.
Step 4 — Final reward: $R = r_{\text{RM}} - 0.05 \times \text{mean_KL}$ (only at last token).
Step 5 — GAE: Compute $\hat{A}_t$ for each token position using value head predictions. Whiten

advantages (zero mean, unit variance).
Step 6 — Update: 4 epochs of SGD on mini-batches of 16. Clip ratio $\epsilon = 0.2$. Gradient norm clipping at 1.0.
Result: Policy improves by ∼0.005 win-rate per step. After 10K steps: 5–10% absolute improvement over SFT.

Tokenization Pitfalls in RL for LLMs

When computing per-token KL penalties and advantages, remember that tokenization determines what a "step" is. A single conceptual action (e.g., outputting "2024") might span 1–4 tokens depending on the tokenizer. This creates subtle issues:

- **KL accounting**: Per-token KL sums to different totals for the same semantic content tokenized differently (e.g., rare words split into more subwords get higher total KL penalty).
- **Credit assignment**: GAE assigns advantage per token position—but semantic "decisions" often span multiple tokens. The model only truly "decides" at the first token of a word; subsequent subword tokens are largely deterministic.
- **Reward placement**: Placing reward only at the final token means all preceding tokens must propagate credit backward through GAE—longer responses suffer from more diluted signal.

Mitigation: Some systems normalize KL by sequence length, use word-level reward shaping, or apply reward at semantic boundaries rather than the final token.

5.7 Detailed Mechanics: Logits and Policy Updates

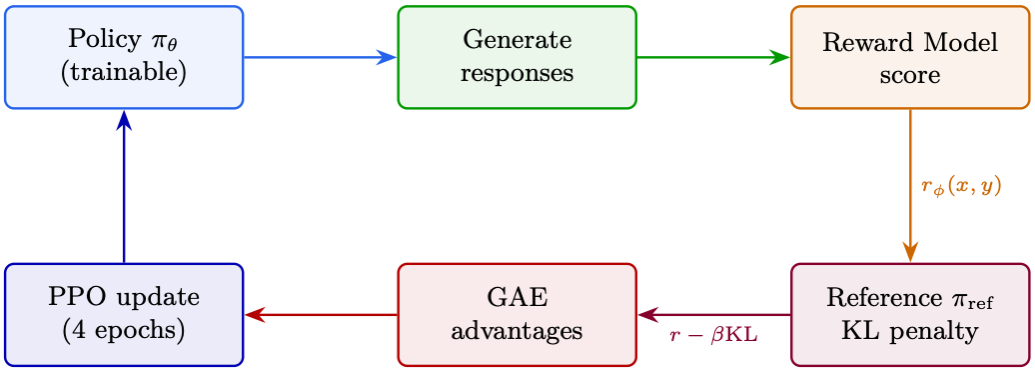


Figure 5.1: PPO end-to-end: from prompt batch through generation, reward scoring, KL computation, advantage estimation, to clipped policy update. The feedback loop shows the updated policy being used for the next generation step.

PPO manages two distinct parameter states in memory, which share the same neural network topology but hold different weight values during optimization:

Core Architecture: Two Networks

1. **The Policy Network (π_θ):** The active, live network parameterized by weights θ. Continuously updated via backpropagation during optimization.
2. **The Old Policy Network ($\pi_{\theta_{\text{old}}}$):** A frozen snapshot parameterized by weights θ_{old}. Acts as a static anchor during a single optimization cycle to prevent the policy from shifting too drastically.

5.7.1 Phase 1: Rollout (Data Collection)

During data collection, the agent interacts with the environment for T steps. At each time-step t:

1. The environment yields the current state/observation s_t (for LLMs: prompt + tokens generated so far).
2. State s_t is passed through the current network snapshot (θ_{old}).
3. The network outputs raw unnormalized values — **logits** z_{old} — a vector of size $|V|$ (vocabulary size 32K–128K).
4. Probabilities are computed via Softmax:

$$\boxed{P(a \mid s_t) = \text{Softmax}(z_{\text{old}}) = \frac{\exp(z_{\text{old},a})}{\sum_{j=1}^{|V|} \exp(z_{\text{old},j})}} \tag{5.14}$$

5. An action a_t (next token) is sampled from $P(a \mid s_t)$, and the transition tuple $\langle s_t, a_t, r_t, s_{t+1} \rangle$ along with $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is stored in the rollout buffer.

Why Store Log-Probabilities?

Storing $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ as a scalar during rollout avoids re-running the frozen network during optimization. This saves one full forward pass per mini-batch — significant for 70B models.

5.7.2 Phase 2: Optimization Loop (Mini-Batch Updates)

Once the rollout buffer is full, PPO runs K epochs (typically 3–10) over mini-batches. For every gradient step, logits are generated for both policies using the stored state s_t:

Old Policy Evaluation (frozen):

$$z_{\text{old}} = f(s_t; \theta_{\text{old}}) \quad \longrightarrow \quad \log \pi_{\theta_{\text{old}}}(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{old}})[a_t] \tag{5.15}$$

Implementation shortcut: reuse the stored scalar from rollout instead of re-computing.

Live Policy Evaluation (updating):

$$z_{\text{new}} = f(s_t; \theta) \quad \longrightarrow \quad \log \pi_{\theta}(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{new}})[a_t] \tag{5.16}$$

Because θ updates after every mini-batch gradient step, z_{new} changes continuously throughout the optimization loop, whereas z_{old} remains perfectly static.

5.7.3 From Logits to Probability Ratio

The core PPO ratio measures how much more or less likely an action is under the new policy vs the old:

$$\boxed{r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}} \tag{5.17}$$

To avoid catastrophic numerical underflow/overflow from dividing raw probabilities, the calculation is performed in **log-space**:

$$\log \pi_\theta(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{new}})[a_t] \tag{5.18}$$

$$\log \pi_{\theta_{\text{old}}}(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{old}})[a_t] \tag{5.19}$$

The ratio is recovered via exponentiation of the difference:

$$\boxed{r_t(\theta) = \exp(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t))} \tag{5.20}$$

This ratio is injected into the PPO clipping objective:

$$\boxed{\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\ 1{-}\epsilon,\ 1{+}\epsilon)\, \hat{A}_t \Big) \right]} \tag{5.21}$$

How Clipping Works

- If $\hat{A}_t > 0$ (good action): ratio is clipped at $1 + \epsilon$ — cannot over-exploit good actions.
- If $\hat{A}_t < 0$ (bad action): ratio is clipped at $1 - \epsilon$ — cannot over-penalize bad actions.
- The $\min(\cdot)$ ensures we always take the more conservative estimate.

Result: monotonic improvement within a trust region — no catastrophic collapses.

5.7.4 The PPO Weight Lifecycle

Table 5.1: Evolution of θ and θ_{old} across PPO training phases.

Phase	Live θ	Old θ_{old}	Ratio $r_t(\theta)$
1. Rollout Start	Active copy	Same active copy	Always 1.0 (by identity)
2. Batch Step 1	Computes gradients	Frozen	1.0 (initial step)
3. Batch Step N	Modifying ($\theta \neq \theta_{\text{old}}$)	Frozen	Deviates from 1.0 (e.g., 1.06, 0.94)
4. Clipping Active	Bounded by ϵ	Frozen	Trapped at bound ($1 \pm \epsilon$)
5. Optimization End	Highly optimized	Discarded	N/A
6. Next Cycle	$\theta \to \theta_{\text{old}}$	Receives fresh θ	Resets back to 1.0

5.7.5 Continuous Action Spaces Extension

For continuous action spaces (not typical for LLMs, but important for robotics RL), the network outputs distribution parameters instead of discrete logits:

- Predicted mean vector μ
- Predicted standard deviation vector σ

Log-probabilities are computed via the Gaussian log-PDF:

$$\boxed{\log \pi(a_t \mid s_t) = -\frac{1}{2}\left(\frac{a_t - \mu}{\sigma}\right)^2 - \log(\sigma) - \frac{1}{2}\log(2\pi)} \tag{5.22}$$

The ratio $r_t(\theta) = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$ is then computed identically and fed into the same clipping objective.

5.8 TRL Implementation

The HuggingFace TRL library [398] provides production-ready implementations of all major RL methods for LLMs.

```
from trl import PPOConfig, PPOTrainer, AutoModelForCausalLMWithValueHead
from transformers import AutoTokenizer
from peft import LoraConfig

# Model setup
model = AutoModelForCausalLMWithValueHead.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16, device_map="auto",
    peft_config=LoraConfig(r=64, lora_alpha=16, target_modules=["q_proj","v_proj",
   "k_proj","o_proj"])
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# PPO config with all critical hyperparameters
```

```
ppo_config = PPOConfig(
    learning_rate=1.5e-6,        # Low LR for stability
    batch_size=128,              # Prompts per step
    mini_batch_size=16,          # Gradient accumulation unit
    ppo_epochs=4,                # Epochs per batch (reuse data)
    gamma=1.0,                   # No discounting (single turn)
    lam=0.95,                    # GAE lambda
    cliprange=0.2,               # PPO epsilon
    cliprange_value=0.2,         # Value function clipping
    vf_coef=0.1,                 # Value loss coefficient
    init_kl_coef=0.05,           # Initial KL penalty
    target_kl=6.0,               # Adaptive KL target
    whiten_rewards=True,         # Normalize advantages
    gradient_accumulation_steps=4,
    max_grad_norm=1.0,
)

ppo_trainer = PPOTrainer(config=ppo_config, model=model, tokenizer=tokenizer,
    dataset=prompt_dataset, data_collator=collator)

# Training loop
for batch in ppo_trainer.dataloader:
    # 1. Generate responses
    query_tensors = batch["input_ids"]
    response_tensors = ppo_trainer.generate(
        query_tensors, max_new_tokens=512, temperature=0.7, top_p=0.9, do_sample=
    True
    )
    # 2. Score with reward model
    texts = [tokenizer.decode(r, skip_special_tokens=True) for r in
    response_tensors]
    rewards = [torch.tensor(reward_model.score(q, r)) for q, r in zip(batch["query
    "], texts)]
    # 3. PPO update (handles KL, GAE, clipping internally)
    stats = ppo_trainer.step(query_tensors, response_tensors, rewards)
    # Monitor: stats["ppo/mean_scores"], stats["ppo/policy/approx_kl"]
```

5.9 Critical Hyperparameters

Parameter	Typical	Effect of Getting It Wrong
`cliprange`	0.2	Too low: no learning. Too high: instability.
`init_kl_coef`	0.01–0.1	Too low: reward hacking. Too high: stuck at SFT.
`target_kl`	4–8	Adaptive controller target. Lower = conservative.
`ppo_epochs`	4	Too many: overfits to batch. Too few: wastes gen compute.
`learning_rate`	$1–5 \times 10^{-6}$	Too high: catastrophic forgetting.
`batch_size`	64–256	Larger = smoother gradients, more gen compute.
`temperature`	0.7–1.0	Lower: less exploration. Higher: noisier advantages.

Chapter 6

DPO — Direct Preference Optimization

Direct Preference Optimization (DPO) [320] is a supervised alternative to reinforcement learning from human feedback that eliminates the need for a separate reward model and RL training loop. By deriving a closed-form relationship between the optimal policy and human preference data, DPO reduces alignment to a simple binary cross-entropy loss—making it dramatically simpler to implement and more stable to train than PPO-based RLHF. This chapter covers the mathematical derivation, practical implementation, and trade-offs of DPO and its variants.

GAS Connection — DPO: Direct Preference Optimization

DPO provides a simpler alternative to PPO for aligning the GAS Advisor with clinical preferences. The chapter notebook `part2_rl_methods/ch06_dpo/notebook.ipynb` demonstrates DPO on preference data.

The DPO objective (§ The DPO Algorithm). DPO directly optimises the policy on preference pairs (chosen recommendation, rejected recommendation) without a separate reward model. For the GAS Advisor, preference pairs are generated by the Safety Critic: recommendations with `safety_score` $\geq$ `0.7` are "chosen"; those with `safety_score < 0.7` are "rejected". This creates a natural DPO training signal from the existing GAS pipeline.

DPO vs. PPO for the GAS. DPO is simpler to implement than PPO (no value function, no rollout buffer) and is well-suited for the GAS use case where preference data is abundant (every Safety Critic decision generates a preference pair). PPO is preferred when online exploration is needed; DPO is preferred for offline alignment on accumulated feedback. The GAS prototype uses rule-based safety checks (`safety_node()` in `capstone_gas/gas_system.py`); DPO would be used to align a production Advisor on accumulated (chosen, rejected) pairs.

Preference data generation. The GAS generates preference data naturally: each pipeline run produces a (patient state, recommendation, safety verdict) tuple. Recommendations with `safety_approved = True` are chosen; those with `safety_approved = False` are rejected. The `GASState` TypedDict in `capstone_gas/gas_system.py` captures all fields needed to construct DPO training examples.

Practical DPO tips (§ Practical DPO Tips). The chapter notebook demonstrates the key DPO hyperparameters: β (KL penalty strength), learning rate, and the importance of reference model quality. A well-trained SFT checkpoint (from `part2_rl_methods/ch04_rl_for_lms/notebook.ipynb`) is essential for stable DPO training.

6.1 Motivation

PPO requires 4 models in memory (policy, reference, reward model, value head), complex RL infrastructure, and is notoriously unstable. DPO [320] asks: *can we skip the RL and learn directly from preferences?*

Key insight: The optimal policy under the RLHF objective (reward maximization + KL penalty) has a **closed-form solution**. We can derive a supervised loss that implicitly optimizes the same

objective.

6.2 Mathematical Derivation

Step 1: RLHF objective: $\max_\pi \mathbb{E}_{x,y\sim\pi}[r(x,y)] - \beta D_{\mathrm{KL}}[\pi\|\pi_{\mathrm{ref}}]$

Step 2: The optimal solution is: $\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{r(x,y)}{\beta}\right)$

Step 3: Rearrange to express reward in terms of policy: $r(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta\log Z(x)$

Step 4: Substitute into Bradley-Terry preference model $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$. The $Z(x)$ cancels!

$$\boxed{\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right]} \tag{6.1}$$

What DPO Actually Does

Define the **implicit reward** as $\hat{r}(x,y) = \beta\log\frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}$.
DPO minimizes the cross-entropy loss where the "label" is: chosen should have higher implicit reward than rejected. The margin is controlled by β:

- Large β: need large margin → policy moves aggressively → risk forgetting
- Small β: small margin suffices → policy stays close to reference → conservative

The reference model acts as a regularizer: the policy must "justify" any deviation from it by showing preference alignment.

6.3 Gradient Analysis

The DPO gradient decomposes as:

$$\nabla_\theta\mathcal{L} = -\beta\cdot \underbrace{\sigma(-\hat{r}_w + \hat{r}_l)}_{\text{weight: higher when model is wrong}} \cdot[\nabla_\theta\log\pi_\theta(y_w|x) - \nabla_\theta\log\pi_\theta(y_l|x)] \tag{6.2}$$

Interpretation: The gradient increases probability of chosen and decreases rejected. The weight is largest when the model currently prefers the wrong answer — it focuses learning on "confusing" pairs.

Concrete DPO Example

Prompt: "Explain quantum entanglement to a 10-year-old."
Chosen (y_w): "Imagine you have two magic coins. When you flip one and it's heads, the other one instantly becomes tails, no matter how far apart they are!"

$\log\pi_\theta(y_w|x) = -15.3$, $\log\pi_{\mathrm{ref}}(y_w|x) = -16.1$
Rejected (y_l): "Quantum entanglement is a phenomenon where two particles become correlated such that the quantum state of one particle cannot be described independently."

$\log\pi_\theta(y_l|x) = -12.8$, $\log\pi_{\mathrm{ref}}(y_l|x) = -12.5$
Implicit rewards: $\hat{r}_w = 0.1\times((-15.3)-(-16.1)) = 0.08$, $\hat{r}_l = 0.1\times((-12.8)-(-12.5)) = -0.03$
Loss input: $\sigma(0.08-(-0.03)) = \sigma(0.11) = 0.527$
Loss: $-\log(0.527) = 0.64$ — The model barely prefers the chosen. Gradient will push hard.
After training: chosen probability increases, rejected decreases, until margin stabilizes around $1/(2\beta)$.

6.4 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import DPOConfig, DPOTrainer
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig
from datasets import load_dataset

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16, attn_implementation="flash_attention_2")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# Dataset format: {"prompt": str, "chosen": str, "rejected": str}
dataset = load_dataset("argilla/ultrafeedback-binarized-preferences")

lora_config = LoraConfig(r=64, lora_alpha=16, lora_dropout=0.05,
    target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","
    down_proj"])

dpo_config = DPOConfig(
    output_dir="./dpo_output",
    beta=0.1,                      # KL regularization strength
    learning_rate=5e-7,            # Very low LR for stability
    loss_type="sigmoid",           # Standard DPO loss
    max_length=2048,               # Max sequence length
    max_prompt_length=1024,        # Truncation for prompts
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,  # Effective batch = 16
    gradient_checkpointing=True,
    bf16=True,
    num_train_epochs=1,            # DPO overfits fast - 1 epoch!
    warmup_ratio=0.1,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=200,
    save_strategy="steps",
    save_steps=500,
)

trainer = DPOTrainer(
    model=model,
    ref_model=None,              # With LoRA, ref = base model (no copy needed!)
    args=dpo_config,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    tokenizer=tokenizer,
    peft_config=lora_config,
)
trainer.train()
# Key metrics to monitor: train/rewards/chosen, train/rewards/rejected, train/
    rewards/margins
```

6.5 How DPO Works: Full Mechanics

This section provides the complete computational details of DPO — what happens at the token level during training.

6.5.1 Sequence-Level Log-Probabilities

The key quantity in DPO is the log-probability of an **entire sequence** $y = (y_1, y_2, \ldots, y_T)$ given prompt x. This is computed as the **sum of per-token log-probabilities**:

$$\boxed{\log \pi_\theta(y|x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t})} \tag{6.3}$$

Each term $\log \pi_\theta(y_t|x, y_{<t})$ is the log-softmax output at position t for the *actual* token y_t in the sequence. This is identical to the cross-entropy loss used in standard language modeling — but here we **sum** rather than average.

Critical detail: The gradient flows through **every token position** in both y_w and y_l. There is no masking of intermediate tokens — every token contributes to the sequence-level log-probability.

6.5.2 The DPO Loss Decomposed

Starting from the loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma(\beta \cdot h_\theta(x, y_w, y_l))] \tag{6.4}$$

where the "implicit reward margin" h_θ is:

$$h_\theta(x, y_w, y_l) = \underbrace{\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}}_{\text{chosen reward proxy}} - \underbrace{\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}}_{\text{rejected reward proxy}} \tag{6.5}$$

Expanding into token-level terms:

$$\boxed{h_\theta = \sum_{t=1}^{|y_w|} \Big[\log \pi_\theta(y_w^t|x, y_w^{<t}) - \log \pi_{\text{ref}}(y_w^t|x, y_w^{<t})\Big] - \sum_{t=1}^{|y_l|} \Big[\log \pi_\theta(y_l^t|x, y_l^{<t}) - \log \pi_{\text{ref}}(y_l^t|x, y_l^{<t})\Big]} \tag{6.6}$$

6.5.3 Forward Pass: Step by Step

For one training example (x, y_w, y_l):

1. **Concatenate**: Form two sequences: $[x; y_w]$ and $[x; y_l]$. Pad to equal length within the batch.
2. **Forward pass (policy π_θ)**: Run both sequences through the model. Collect logits at every response position.
3. **Extract log-probs**: At each position t in the response, take $\log \text{softmax}(\text{logits}_t)[y_t]$ — the log-probability of the actual token.
4. **Sum over tokens**:

$$\text{logp_chosen} = \sum_{t\in\text{response positions}} \log \pi_\theta(y_w^t|x, y_w^{<t}) \tag{6.7}$$

$$\text{logp_rejected} = \sum_{t\in\text{response positions}} \log \pi_\theta(y_l^t|x, y_l^{<t}) \tag{6.8}$$

5. **Subtract reference** (pre-computed or from second forward pass):

$$\text{ratio_w} = \text{logp_chosen} - \text{ref_logp_chosen} \tag{6.9}$$

$$\text{ratio_l} = \text{logp_rejected} - \text{ref_logp_rejected} \tag{6.10}$$

6. **Compute loss**: $\mathcal{L} = -\log \sigma(\beta \cdot (\text{ratio_w} - \text{ratio_l}))$
7. **Backward pass**: Gradients flow back through steps $5 \to 4 \to 3 \to 2$ to update θ.

6.5.4 Token-Level Gradient Analysis

Does every token get a gradient? Yes. The gradient with respect to the logits at position t in the chosen sequence is:

$$\frac{\partial \mathcal{L}}{\partial \text{logits}_t^{(w)}} = -\underbrace{\sigma(-\beta \cdot h_\theta)}_{\text{scaling factor}} \cdot \beta \cdot \frac{\partial \log \pi_\theta(y_w^t|\cdot)}{\partial \text{logits}_t^{(w)}} \tag{6.11}$$

Key insight: The scaling factor $\sigma(-\beta \cdot h_\theta)$ is **shared across all tokens** in both sequences. It acts as an adaptive learning rate:

- When h_θ is small (model can't distinguish chosen from rejected): scaling ≈ 0.5 — strong gradient, learn aggressively.
- When h_θ is large (model already prefers chosen): scaling ≈ 0 — negligible gradient, don't over-fit.

Effect on chosen tokens: Probability is *increased* (log-prob pushed up).

Effect on rejected tokens: Probability is *decreased* (log-prob pushed down).

Relative to reference: Only the *difference* from π_{ref} matters. If the model already assigns high probability to the chosen response (matching the reference), there's little gradient.

6.5.5 Per-Token vs. Sequence-Level: Length Normalization

A subtle issue: longer sequences naturally have lower log-probabilities (more terms summed, each ≤ 0). If $|y_w| \gg |y_l|$, the loss can be biased toward preferring shorter responses.

Solutions:

- **Length-normalized DPO**: Replace $\log \pi_\theta(y|x)$ with $\frac{1}{|y|}\sum_t \log \pi_\theta(y_t|\cdot)$. Used in some implementations (SimPO adopts this).
- **Standard DPO**: Uses raw sum (no normalization). This *implicitly* penalizes verbosity — the model must assign high probability to every token in the chosen response.
- **Practical impact**: On benchmarks, length-normalized DPO reduces length gaming but can hurt instruction-following quality. Standard (unnormalized) is more common in production.

6.5.6 Label Masking: What Gets Gradients

Which Tokens Receive Gradient in DPO

- **Prompt tokens** (x): **NO gradient**. The loss is computed only over response positions. Prompt tokens provide context but their logits don't contribute to $\log \pi(y|x)$.
- **Chosen response tokens** (y_w): **ALL tokens get gradient**. Each y_w^t contributes to the sum. Gradient pushes their probabilities up.
- **Rejected response tokens** (y_l): **ALL tokens get gradient**. Each y_l^t contributes to the sum. Gradient pushes their probabilities down.
- **Padding tokens**: **NO gradient**. Masked out with attention mask.

6.5.7 Pseudocode: DPO Training Step

DPO Forward + Backward (PyTorch-style)

```
def dpo_loss(model, ref_model, batch, beta=0.1):
    """One DPO training step."""
    # batch contains: input_ids_chosen, input_ids_rejected,
    #                 labels_chosen, labels_rejected (prompt masked to -100)

    # 1. Forward pass: get per-token log-probs
    logps_chosen = get_sequence_logprob(model, batch["chosen"])
    logps_rejected = get_sequence_logprob(model, batch["rejected"])

    # 2. Reference log-probs (pre-computed or computed here)
    with torch.no_grad():
        ref_logps_chosen = get_sequence_logprob(ref_model, batch["chosen"])
        ref_logps_rejected = get_sequence_logprob(ref_model, batch["rejected"])

    # 3. Compute implicit reward margins
    chosen_rewards = beta * (logps_chosen - ref_logps_chosen)
    rejected_rewards = beta * (logps_rejected - ref_logps_rejected)

    # 4. DPO loss = -log(sigmoid(chosen_reward - rejected_reward))
    loss = -F.logsigmoid(chosen_rewards - rejected_rewards).mean()
    return loss

def get_sequence_logprob(model, sequences):
    """Sum of log-probs over response tokens only."""
    outputs = model(sequences["input_ids"], attention_mask=sequences["mask"])
    logits = outputs.logits[:, :-1, :]  # Shift for next-token prediction

    # Gather log-prob of actual tokens
    labels = sequences["labels"][:, 1:]  # Shifted labels
    log_probs = F.log_softmax(logits, dim=-1)
    token_logps = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)

    # Mask: only sum over response tokens (labels != -100)
    mask = (labels != -100).float()
    return (token_logps * mask).sum(dim=-1)  # Shape: [batch_size]
```

6.5.8 Common Pitfalls

DPO Implementation Pitfalls

- **Forgetting to mask the prompt**: If prompt tokens are included in the log-prob sum, the model optimizes for prompt likelihood (useless) and the effective β is wrong.
- **Using mean instead of sum**: $\frac{1}{T}\sum_t \log \pi$ vs. $\sum_t \log \pi$ — these give different implicit length penalties. Must be consistent between π_θ and π_{ref}.
- **Stale reference model**: If π_{ref} is too far from π_θ (e.g., base model vs. fine-tuned), the KL term dominates and gradients vanish. Solution: use the SFT checkpoint (not base) as reference.
- **β too large**: Magnifies log-prob differences → sigmoid saturates → zero gradients. Start with $\beta = 0.1$, tune in $[0.05, 0.5]$.
- **β too small**: Theoretically allows more freedom from reference (weaker KL constraint), but the gradient $\propto \beta \cdot \sigma(-\beta h)$ becomes vanishingly small → loss landscape is flat → extremely slow convergence. The model has "permission" to move far but receives almost no signal telling it *where* to move.

6.6 DPO Variants and When Each Fails

When DPO Fails

1. Distribution shift: Preference data from old model. Current policy generates different text → loss is optimizing on irrelevant examples.
2. No exploration: Can't discover behaviors not in dataset. Stuck in local optimum.
3. Reference collapse: If reference is too strong, policy can't move. If too weak, no regularization.
4. Data quality: Noisy labels poison training. Unlike PPO which averages over many samples, DPO memorizes individual pairs.
5. Preference data diversity: Ensure chosen/rejected pairs cover the full spectrum of quality differences (not just good-vs-terrible). Pairs that differ in *approach*, not just quality, teach richer policy distinctions.

6.7 β Selection Guide

β	Regime	When to Use
0.01	Very aggressive	Only if data is extremely clean and you need large distributional shift
0.05	Aggressive	Good data, want noticeable improvement over SFT
0.1	Standard	Default starting point. Good balance of quality vs stability
0.2	Conservative	Noisy data, or model already close to desired behavior
0.5	Very conservative	Safety fine-tuning where you must not break capabilities

6.8 DPO Batch Size Configuration and Scaling

Unlike standard SFT which operates on single-sequence token predictions, DPO leverages a **pairwise loss** comparing a preferred sequence against a dispreferred sequence. This fundamentally alters memory utilization and optimization stability.

6.8.1 Global Batch Size Target

Empirical evidence across DPO implementations establishes an optimal global batch size range:

$$\boxed{B_{\text{global}} \in [32, 128]} \tag{6.12}$$

- $B_{\text{global}} < 32$: Severe gradient noise in implicit reward estimation → policy oscillates destructively between alignment goals (helpfulness vs. safety).
- $B_{\text{global}} > 128$: Diminishing returns on convergence velocity; high communication overhead across distributed compute.

6.8.2 Mathematical Decomposition

Because DPO loads **two** model copies simultaneously (active policy π_θ + frozen reference π_{ref}), per-sequence memory is doubled. The global batch size is decomposed as:

$$\boxed{B_{\text{global}} = B_{\text{micro}} \times N_{\text{GPUs}} \times K_{\text{accum}}} \tag{6.13}$$

- B_{micro}: Per-device micro-batch size (preference pairs per forward pass).

- N_{GPUs}: Number of parallel data-processing devices.
- K_{accum}: Gradient accumulation steps before weight update.

The pairing multiplier: A single DPO data instance contains a prompt (x), chosen (y_w), and rejected (y_l). The actual tensor load per micro-batch:

$$T_{\text{sequences}} = 2 \times B_{\text{micro}} \tag{6.14}$$

For models >7B parameters on 80GB GPUs with context lengths 4096–8192 tokens, the physical limit is rigidly constrained to $B_{\text{micro}} \in [1, 2]$.

6.8.3 Distributed Scaling Configurations

Table 6.1: Distributed scaling profiles for DPO training ($B_{\text{global}} = 64$ target).

Configuration	Single GPU	8-GPU Node
B_{global}	64	64
B_{micro}	2 (4 sequences)	2 (4 sequences)
N_{GPUs}	1	8
K_{accum}	32 steps	4 steps
Throughput	Sequential/slow	High parallel throughput

6.8.4 VRAM Optimization: Pre-computing Reference Log-Probabilities

The DPO loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right] \tag{6.15}$$

Because π_{ref} is **completely static** throughout training, its outputs can be pre-computed:

Reference Model Eviction Strategy

1. Execute a forward pass over dataset $\mathcal{D}$ using only π_{ref} before training begins.
2. Cache the scalars $\log \pi_{\text{ref}}(y_w|x)$ and $\log \pi_{\text{ref}}(y_l|x)$ to disk.
3. **Evict π_{ref} completely from GPU memory.**

Result: Available GPU memory doubles → can increase B_{micro} from 1–2 to 4–8, maximizing hardware utilization and training throughput.
Implementation: In TRL, set `precompute_ref_log_probs=True` in `DPOConfig`. For 70B models, this saves ∼140GB of GPU memory across the cluster.

6.9 DPO Extensions and Variants

Direct Preference Optimization (DPO) reformulates RLHF as a supervised learning problem by deriving a closed-form mapping between the reward function and the optimal policy. The standard DPO loss is:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(q,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)} - \beta \log \frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right)\right],$$

where y_w is the preferred (winning) response, y_l is the dispreferred (losing) response, and β controls the strength of the KL penalty. The following subsections cover the most important extensions and variants.

6.9.1 f-DPO – Generalised f-Divergence DPO

Beyond Reverse KL

Standard DPO uses reverse KL divergence as the regulariser between policy and reference. Reverse KL is *mode-seeking*: it prefers to concentrate probability mass on a few high-reward responses. Forward KL is *mode-covering*: it spreads probability mass to cover all plausible responses. f-DPO [384] generalises to any f-divergence, allowing practitioners to trade off these behaviours.

The f-DPO loss replaces the log-ratio with the derivative of the f-divergence generator:

$$\mathcal{L}_{f\text{-DPO}} = -\mathbb{E}\left[f'\left(\frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)}\right) - f'\left(\frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right)\right],$$

where f' is the derivative of the f-divergence generator function.

f-Divergence Options in TRL

- **reverse_kl**: $f'(u) = \log u$. Standard DPO. Mode-seeking.
- **forward_kl**: $f'(u) = -1/u$. Mode-covering. Better diversity.
- **js_divergence**: $f'(u) = \log(2u/(u+1))$. Balanced mode-seeking/covering.
- **alpha_divergence**: $f'(u) = u^{\alpha-1}$. Interpolates between forward and reverse KL.

f-DPO in TRL

```
from trl import DPOConfig, DPOTrainer

# Jensen-Shannon divergence (balanced)
config = DPOConfig(
    f_divergence_type="js_divergence",
    beta=0.1,
)

# Alpha divergence (alpha=0: forward KL, alpha=1: reverse KL)
config_alpha = DPOConfig(
    f_divergence_type="alpha_divergence",
    f_alpha_divergence_coef=0.5,   # alpha parameter
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use f-DPO

- Use **JS divergence** when you want a balance between diversity and quality.
- Use **forward KL** for creative tasks where diversity is paramount.
- Use **reverse KL** (standard DPO) for tasks with a single correct answer.
- Use **alpha divergence** to continuously interpolate and tune the trade-off.

6.9.2 Robust DPO

Noisy Labels in Preference Data

Human preference annotations are noisy. Annotators disagree, make mistakes, and sometimes flip the preferred/dispreferred labels. Standard DPO treats all labels as ground truth, which can cause the model to overfit to noise. Robust DPO [58] analytically debiases the loss under a known noise model.

Assume each label is flipped with probability ϵ (the noise rate). The debiased loss is:

$$\mathcal{L}_{\text{robust}} = \frac{(1-\epsilon)\,\mathcal{L}_{\text{DPO}}(y_w, y_l) - \epsilon\,\mathcal{L}_{\text{DPO}}(y_l, y_w)}{1-2\epsilon},$$

where $\mathcal{L}_{\text{DPO}}(y_w, y_l)$ is the standard DPO loss treating y_w as preferred, and $\mathcal{L}_{\text{DPO}}(y_l, y_w)$ is the loss with labels flipped. This correction removes the bias introduced by label noise.

Intuition for Robust DPO

The formula is a linear combination that "subtracts out" the contribution of flipped labels. When $\epsilon = 0$, it reduces to standard DPO. When $\epsilon = 0.5$, the denominator goes to zero – the labels are pure noise and no learning is possible. In practice, $\epsilon \in [0.05, 0.2]$ covers most real annotation noise levels.

Robust DPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="robust",
    label_smoothing=0.1,    # corresponds to epsilon = 0.1
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

6.9.3 TR-DPO – Trust Region DPO

Stale Reference Model Problem

Standard DPO uses a fixed reference model π_{ref} throughout training. As the policy π_θ improves, the KL penalty $\beta \log(\pi_\theta/\pi_{\text{ref}})$ grows, eventually dominating the loss and preventing further improvement. TR-DPO [124] periodically updates the reference model to track the current policy.

TR-DPO updates the reference model using an exponential moving average (EMA):

$$\pi_{\text{ref}}^{(t+1)} \leftarrow \alpha \cdot \pi_\theta^{(t)} + (1-\alpha) \cdot \pi_{\text{ref}}^{(t)},$$

where $\alpha \in (0, 1)$ is the mixup coefficient. This is applied every T_{sync} gradient steps.

TR-DPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="sigmoid",          # standard DPO loss
    sync_ref_model=True,          # enable TR-DPO
    ref_model_mixup_alpha=0.6,    # alpha: how much of current policy to mix in
    ref_model_sync_steps=512,     # T_sync: sync every 512 steps
    beta=0.1,
```

```
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use TR-DPO

- Long training runs where the policy drifts far from the initial reference.
- When you observe the DPO loss plateauing early due to KL penalty domination.
- Iterative DPO pipelines where new preference data is collected from the current policy.
- Set α close to 1 for fast reference updates; close to 0 for slow updates.

6.9.4 EXO – Exact Optimisation

DPO's KL Direction Problem

DPO is derived by solving for the optimal policy under a reverse KL constraint. However, the resulting loss actually optimises a *forward* KL objective in the reward space, which is the wrong direction. EXO [173] corrects this by using reverse KL probability matching, which is the theoretically correct objective for alignment.

EXO minimises the reverse KL between the model distribution and the target (reward-optimal) distribution:

$$\mathcal{L}_{\text{EXO}} = \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y|q)}{p^*(y|q)}\right],$$

where $p^*(y|q) \propto \pi_{\text{ref}}(y|q)\exp(r(y,q)/\beta)$ is the optimal policy. In practice, EXO approximates this using the available preference pairs:

$$\mathcal{L}_{\text{EXO}} \approx -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_{\text{ref}}(y_w|q)}{\pi_\theta(y_w|q)} - \beta\log\frac{\pi_{\text{ref}}(y_l|q)}{\pi_\theta(y_l|q)}\right)\right].$$

Note the *swapped* roles of π_θ and π_{ref} compared to DPO.

EXO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="exo_pair",
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

6.9.5 NCA – Noise Contrastive Alignment

Likelihood Collapse in DPO

A known failure mode of DPO is *likelihood collapse*: the model learns to decrease the probability of the losing response but also decreases the probability of the winning response (since the loss only cares about the *difference*). NCA [45] adds an absolute likelihood term to prevent this.

NCA reframes alignment as noise-contrastive estimation. The loss has three terms:

$$\boxed{\mathcal{L}_{\text{NCA}} = -\log \sigma(r_w) - \tfrac{1}{2}\log \sigma(-r_w) - \tfrac{1}{2}\log \sigma(-r_l),}$$

where $r_y = \beta \log(\pi_\theta(y|q)/\pi_{\text{ref}}(y|q))$ is the implicit reward. The first term encourages high reward for y_w; the second and third terms penalise high reward for both y_w and y_l (preventing collapse).

NCA in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="nca_pair",
    beta=0.01,   # small beta: absolute likelihood term dominates
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use NCA

- When you observe the winning response probability decreasing during DPO training.
- For tasks where absolute response quality matters, not just relative ranking.
- Use small β (e.g., 0.01) to give the absolute likelihood term more weight.

6.9.6 SLiC-HF – Sequence Likelihood Calibration

Hinge Loss as a Simpler Alternative

The log-sigmoid loss in DPO is smooth but can be slow to converge when the margin is large. SLiC-HF [455] uses a hinge loss, which is zero when the margin exceeds a threshold and linear otherwise. This is simpler, faster, and surprisingly competitive.

The SLiC-HF loss is:

$$\mathcal{L}_{\text{SLiC}} = \max\left(0,\ \delta - \beta \log \frac{\pi_\theta(y_w|q)}{\pi_{\text{ref}}(y_w|q)} + \beta \log \frac{\pi_\theta(y_l|q)}{\pi_{\text{ref}}(y_l|q)}\right),$$

where δ is the margin threshold. When the model already assigns a margin of δ between winning and losing responses, the loss is zero.

SLiC-HF in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="hinge",
    beta=0.1,
)

trainer = DPOTrainer(
```

```
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

6.9.7 Iterative RPO – Reasoning Preference Optimisation

DPO Forgets How to Generate

Standard DPO trains the model to *discriminate* between winning and losing responses. But for reasoning tasks, the model also needs to *generate* correct reasoning traces. A model that can discriminate but not generate is useless at inference time. RPO adds an NLL (negative log-likelihood) term on the winning response to ensure the model learns to generate it.

The RPO loss combines DPO and SFT:

$$\mathcal{L}_{\text{RPO}} = \lambda_1 \mathcal{L}_{\text{DPO}}(y_w, y_l) + \lambda_2 \mathcal{L}_{\text{NLL}}(y_w),$$

where $\mathcal{L}_{\text{NLL}}(y_w) = -\log \pi_\theta(y_w|q)$ is the standard language modelling loss on the winning response.

Iterative RPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="sigmoid",            # Standard DPO loss
    rpo_alpha=1.0,                  # NLL regularisation weight (RPO)
    beta=0.1,
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=config,
    train_dataset=dataset,
)
```

When to Use RPO

- Reasoning tasks (math, code, logic) where the model must generate step-by-step solutions.
- When DPO training causes the model to lose fluency or generation quality.
- Iterative pipelines: generate rollouts, label them, train with RPO, repeat.
- The NLL term acts as a regulariser, preventing the policy from collapsing.

6.9.8 SimPO – Simple Preference Optimisation

Reference-Free Preference Learning

DPO requires a reference model to compute the implicit reward. This doubles memory usage and adds complexity. SimPO [265] eliminates the reference model by using the *average log-probability* of the response as an implicit reward, with a length normalisation term to prevent the model from preferring short responses.

SimPO defines the implicit reward as:

$$r_{\text{SimPO}}(y|q) = \frac{\beta}{|y|} \log \pi_\theta(y|q),$$

and the loss as:

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|q) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|q) - \gamma\right)\right],$$

where $\gamma > 0$ is a target reward margin that ensures the winning response has strictly higher reward than the losing response by at least γ.

SimPO vs DPO vs ORPO

- **DPO**: uses reference model; ratio-based implicit reward.
- **ORPO**: reference-free; adds odds-ratio term to SFT loss.
- **SimPO**: reference-free; length-normalised log-prob reward + margin.
- SimPO is simpler than DPO (no reference model) and more principled than ORPO.
- The length normalisation in SimPO is critical: without it, the model prefers long responses.

SimPO in TRL

```
from trl import DPOConfig, DPOTrainer

config = DPOConfig(
    loss_type="simpo",
    simpo_gamma=0.5,    # target reward margin gamma
    beta=2.5,           # length normalisation coefficient
    # No ref_model needed!
)

trainer = DPOTrainer(
    model=model,
    ref_model=None,     # SimPO is reference-free
    args=config,
    train_dataset=dataset,
)
```

Chapter 7

GRPO — Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) [341] is a reinforcement learning algorithm designed specifically for language models that eliminates the need for a separate value network (critic). Introduced by DeepSeek as part of their DeepSeekMath work and later scaled to DeepSeek-R1 [75], GRPO has rapidly become the dominant RL method for LLM training—adopted by most open-source alignment frameworks (TRL, OpenRLHF, veRL) as the default algorithm.

The core idea is deceptively simple: instead of training a neural network to predict expected reward (the critic in PPO), GRPO *estimates* it empirically by generating multiple responses to the same prompt and using the group's reward statistics as a baseline. This removes an entire model from memory, halves the engineering complexity, and—surprisingly—often outperforms PPO because empirical baselines are more accurate than a poorly-trained value function.

GRPO is particularly effective for:

GAS Connection — GRPO: Group Relative Policy Optimization

GRPO eliminates the value function from PPO by using group-relative rewards—a natural fit for the GAS Advisor, which generates multiple candidate recommendations per patient state. The chapter notebook `part2_rl_methods/ch07_grpo/notebook.ipynb` demonstrates GRPO training.

Group-relative rewards (§ The GRPO Algorithm). The GAS Advisor can generate N candidate recommendations for a given patient state. GRPO scores all N candidates with the Safety Critic and uses their relative scores (normalised within the group) as advantages. This eliminates the need for a value function baseline while still providing a stable training signal.

GRPO for the GAS Advisor. In the current prototype, the Safety Critic (`safety_node()` in `capstone_gas/gas_system.py`) scores a single recommendation per invocation. GRPO training would extend this to score $N = 4$ candidates and select the highest-scoring approved recommendation—a test-time compute scaling strategy that improves recommendation quality at the cost of $N\times$ inference.

Format rewards. The GAS Advisor's output must follow a structured format (recommendation, rationale, urgency level). GRPO's format reward—positive for correctly structured outputs, negative for malformed ones—can be applied directly using the output parsing logic in `advisor_node()` in `capstone_gas/gas_system.py`.

GRPO vs. PPO for the GAS. GRPO is preferred over PPO for the GAS because: (1) no value function to train (simpler implementation); (2) group-relative rewards naturally handle the multi-candidate recommendation setting; (3) the Safety Critic provides a reliable scalar reward signal that works well with GRPO's normalisation. The chapter notebook demonstrates both algorithms on the same task for direct comparison.

- **Reasoning tasks** with verifiable rewards (math, code) where binary correctness provides a clean signal.
- **Large models** (70B+) where the memory savings from removing the critic are critical.

- **Multi-turn and agentic settings** where value estimation across tool calls is intractable.

This chapter covers GRPO's motivation, algorithm, key variants (Dr. GRPO, DAPO, 2-GRPO, GDPO), and practical implementation with TRL.

7.1 Motivation

PPO's value model (critic) has three major problems for language:

1. **Memory**: The value head shares the policy backbone (140GB for 70B). Doubles memory if separate.

2. **Accuracy**: Predicting expected reward for a partial sequence is extremely hard. The value function is often wrong → wrong advantages → wrong gradient direction.

3. **Training**: Value head needs many samples to converge. During early RL, it gives noisy predictions that destabilize policy learning.

GRPO's key insight [341]: Instead of learning $V(s)$, *estimate* it empirically from a group of samples. Generate G responses to the same prompt, compute their rewards, and use the group statistics as the baseline.

7.2 Algorithm

1. For each prompt x, sample G completions: $\{y_1, \ldots, y_G\} \sim \pi_\theta(\cdot|x)$

2. Score each: $r_i = R(x, y_i)$

3. Normalize within group: $\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}$ where $\mu_G = \frac{1}{G}\sum_j r_j$, $\sigma_G = \text{std}(\{r_j\})$

4. Apply PPO-style clipped update using these advantages

$$\boxed{\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}, \qquad L = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_i,\ \text{clip}(r_t(\theta), 1\pm\epsilon)\hat{A}_i\right)\right] - \beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]} \tag{7.1}$$

Why Group Normalization Works

The group mean approximates $V(s)$: If you sample enough responses to the same prompt, their average reward is a Monte Carlo estimate of the expected reward = value function.
Above mean = good move: $\hat{A}_i > 0$ means this response is better than average for this prompt. Reinforce it.
Below mean = bad move: $\hat{A}_i < 0$ means worse than average. Suppress it.
Normalization: Dividing by σ_G ensures advantages are scale-invariant across prompts with different reward ranges.
DeepSeek-R1 breakthrough [75]: Pure GRPO with binary correctness rewards ($r = 1$ if answer correct, $r = 0$ otherwise) trained on math/code spontaneously developed chain-of-thought reasoning, self-verification, and error correction — without any explicit instruction to do so.

7.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

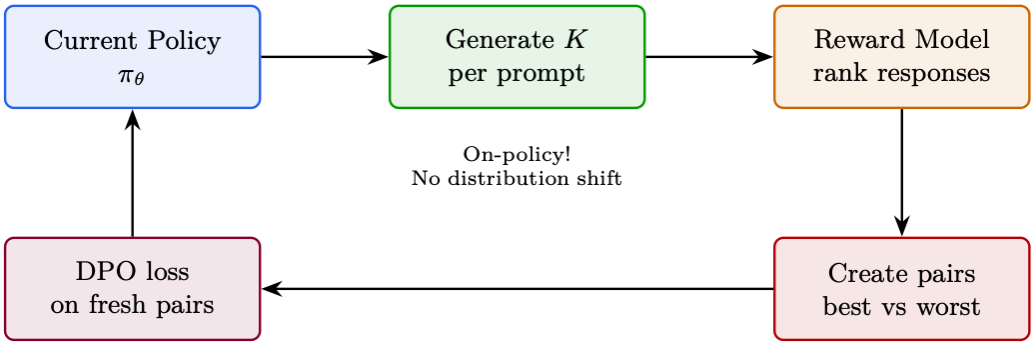


Figure 7.1: GRPO in action: G=5 responses are sampled for a single math prompt. Three are correct (r=1), two are wrong (r=0). The group mean μ_G=0.6 acts as the baseline; correct responses receive positive advantage (reinforced), wrong ones receive negative advantage (suppressed).

```
from trl import GRPOConfig, GRPOTrainer
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B-Instruct",
    torch_dtype=torch.bfloat16, attn_implementation="flash_attention_2")
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B-Instruct")

grpo_config = GRPOConfig(
    output_dir="./grpo_output",
    num_generations=8,           # G = group size
    temperature=1.0,             # High temp for diversity within group
    max_completion_length=2048,  # Max response length
    beta=0.04,                   # KL penalty coefficient
    learning_rate=1e-6,
    per_device_train_batch_size=2,  # Prompts per device (x8 gens = 16 responses)
    gradient_accumulation_steps=8,
    num_train_epochs=2,
    bf16=True,
    gradient_checkpointing=True,
    max_grad_norm=0.5,
    logging_steps=10,
    # vLLM generation for speed (critical for GRPO due to 8x generation)
    use_vllm=True,
    vllm_gpu_memory_utilization=0.7,
)

# Reward function: binary correctness for math
def reward_fn(completions, prompts, **kwargs):
    """Return list of floats: 1.0 if correct, 0.0 if wrong."""
    rewards = []
    for completion, prompt in zip(completions, prompts):
        answer = extract_answer(completion)
        expected = get_ground_truth(prompt)
        rewards.append(1.0 if answer == expected else 0.0)
    return rewards

# Can combine multiple reward functions!
def format_reward_fn(completions, **kwargs):
    """Bonus for using proper LaTeX formatting."""
    return [0.5 if "\\boxed{" in c else 0.0 for c in completions]

trainer = GRPOTrainer(
    model=model,
    args=grpo_config,
    reward_funcs=[reward_fn, format_reward_fn],  # Multi-objective!
    train_dataset=math_dataset,
    tokenizer=tokenizer,
)
```

```
trainer.train()
```

7.4 Group Size Analysis

G	Signal Quality	Compute	When to Use
2	Very noisy (coin flip)	Low	Never recommended — too noisy for stable learning
4	Moderate	Moderate	Quick experiments, easy tasks (pass rate $> 50\%$)
8	Good (standard)	High	Default. Good balance for most tasks
16	Excellent	Very high	Hard tasks (pass rate $< 20\%$), need many attempts to get positives
32	Near-perfect	Extreme	Only if you have massive compute and very hard task

Critical: Group Must Contain Both Successes and Failures

If all G responses are correct ($r_i = 1\ \forall i$): all advantages $= 0$, no learning signal!

If all wrong: same problem. The prompt's difficulty must match model's capability.

Goldilocks rule: Filter prompts to 20–80% pass rate for current model. Re-filter every 500 steps as model improves.

7.5 GRPO Variants and Extensions

7.5.1 Diversity in GRPO Groups

Mode Collapse in RL Training

Without diversity pressure, RL-trained LLMs collapse to a narrow set of high-reward responses:

- The model learns one "template" answer for each question type
- Entropy drops rapidly; the model becomes deterministic
- Reward hacking becomes easier (narrow outputs are easier to exploit)
- Generalization suffers: the model memorizes reward patterns, not reasoning

The KL penalty $\beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ is the primary diversity mechanism, but it's not sufficient alone.

GRPO Group Diversity

GRPO generates N responses per prompt and uses within-group ranking. Diversity within the group is critical:

- **High temperature** ($\tau = 0.8$–1.0): Ensures varied responses for meaningful comparison
- **Large** N (8–16): More samples = more likely to include both good and bad approaches
- **DAPO's "No Repeat" penalty**: Rejects duplicate responses within a group to force exploration
- If all N responses are identical: advantage is zero, no learning signal
- If responses are too diverse (random): reward signal is noisy, slow learning

Sweet spot: Temperature that gives distinct approaches while staying on-topic.

Table 7.1: Diversity-promoting methods for RL training.

Method	How It Promotes Diversity
Entropy bonus	Add $\alpha H(\pi_\theta)$ to the reward. Directly penalizes low-entropy (deterministic) policies.
KL penalty	$-\beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ prevents collapse toward a single mode.
Rejection sampling	Generate many candidates, keep top-k by reward. Naturally selects for diverse high-quality responses.
Best-of-N	At inference: generate N responses, score all, return the best. Diversity comes from sampling.
DPO with diverse pairs	Train on pairs where chosen/rejected differ in *approach*, not just quality.
Multi-reward	Use multiple reward models (safety, helpfulness, code quality). Prevents collapsing to one dimension.

Diversity vs. Quality Tradeoff

More diversity is not always better:

- Too much diversity (high entropy) = random, unhelpful responses
- Too little diversity (low entropy) = repetitive, reward-hacked responses
- **Monitor**: Track response entropy, unique n-gram ratio, and reward distribution width during training. If all three are dropping simultaneously, you have a collapse problem.

Verbalized Sampling for RL Data Collection

Post-training alignment (RLHF, DPO) often reduces output diversity due to *typicality bias*: human annotators systematically prefer familiar, "typical" text over novel alternatives. This mode collapse is a data-level phenomenon, not purely algorithmic.

Verbalized Sampling (VS) [445] is a training-free prompting strategy that circumvents this collapse by asking the model to **explicitly verbalize a probability distribution** over multiple responses in a single generation.

Verbalized Sampling – Core Idea

Instead of sampling a single response (which collapses to the mode), prompt the model to output *multiple candidate responses along with their probabilities*:
`"Generate 5 jokes about coffee and their corresponding probabilities."`
The model produces a list like:

1. Joke A (probability: 0.35)
2. Joke B (probability: 0.25)
3. Joke C (probability: 0.20)
4. Joke D (probability: 0.12)
5. Joke E (probability: 0.08)

Then sample from this verbalized distribution. Because the model explicitly represents the full distribution (not just the argmax), lower-probability but creative/diverse responses become accessible.

```python
# Verbalized Sampling: prompt model to output distribution
def verbalized_sample(model, tokenizer, task, n=5):
    prompt = (
        f"{task}\n\n"
        f"Generate {n} different responses and assign a probability "
        f"to each (probabilities should sum to 1.0). "
        f"Format: [response] (probability: X.XX)"
    )
    output = model.generate(
        tokenizer(prompt, return_tensors="pt").input_ids,
        max_new_tokens=1024,
        temperature=0.7,
        do_sample=True,
    )
    # Parse responses and probabilities from output
    responses, probs = parse_verbalized_distribution(
        tokenizer.decode(output[0])
    )
    # Sample from the verbalized distribution
    import random
    chosen = random.choices(responses, weights=probs, k=1)[0]
    return chosen
```

Why Verbalized Sampling Works

- **Bypasses mode collapse**: Standard sampling from aligned models heavily concentrates on one or two "safe" responses. VS forces the model to articulate alternatives it *knows* but wouldn't normally surface.
- **Diversity is semantic**: Unlike temperature scaling (lexical noise), VS produces genuinely different approaches—the model reasons about distinct options.
- **Scales with capability**: More capable models produce better-calibrated verbalized distributions—they benefit *more* from VS (1.6–2.1× diversity gain in creative writing).
- **Training-free**: No fine-tuning or modified decoding; works with any instruction-following model at inference time.
- **For GRPO**: Use VS to generate the G response candidates per prompt—ensures the group contains semantically diverse approaches rather than surface-level variations.

Before diving into the extensions, let us briefly recap the base GRPO algorithm established in the previous sections. The core mechanism—sampling a group of completions, normalizing their rewards, and applying a clipped policy gradient—is elegant in its simplicity. However, practitioners quickly discovered specific failure modes: pretraining bias diluting gradients (Dr. GRPO), symmetric clipping limiting exploration (DAPO), wasteful large group sizes (2-GRPO), and reward-scale imbalance in multi-objective settings (GDPO). The following sections address each of these in turn.

GRPO Baseline Recap

Given a prompt q, sample G completions $\{o_1, \ldots, o_G\}$ from the current policy π_θ. Compute rewards $\{r_1, \ldots, r_G\}$ and normalise:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \qquad \mu_r = \frac{1}{G}\sum_{i=1}^{G} r_i, \quad \sigma_r = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i - \mu_r)^2}.$$

The clipped surrogate loss (per token) is:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(\rho_{i,t}\,\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1{-}\epsilon, 1{+}\epsilon)\,\hat{A}_i\Big),$$

where $\rho_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t}) \,/\, \pi_{\text{old}}(o_{i,t}|q, o_{i,<t})$.

7.5.2 DAPO – Dynamic Adaptive Policy Optimization

Why DAPO?

Base GRPO uses *symmetric* clipping: the policy is equally constrained whether it wants to increase or decrease the probability of a token. But exploration and exploitation have different risk profiles. Increasing the probability of a good token is generally safe; suppressing a token that happened to appear in a bad completion can be catastrophically wrong if the token itself is neutral. DAPO [436] introduces five targeted fixes that together substantially improve training stability and final performance.

Component 1 – Asymmetric Clipping (Clip-Higher)

Standard PPO/GRPO clips the importance ratio symmetrically at $[1-\epsilon, 1+\epsilon]$. DAPO replaces this with an asymmetric band:

$$\boxed{\text{clip}_{\text{DAPO}}(\rho, A) = \begin{cases} \text{clip}(\rho,\, 1-\epsilon,\, 1+\epsilon_{\text{high}}) & \text{if } A > 0 \\ \text{clip}(\rho,\, 1-\epsilon,\, 1+\epsilon) & \text{if } A \le 0 \end{cases}}$$

where $\epsilon_{\text{high}} > \epsilon$ (typical values: $\epsilon = 0.2$, $\epsilon_{\text{high}} = 0.28$). When the advantage is positive the policy is allowed to move further toward the good token; when the advantage is negative the usual conservative clipping applies to avoid over-suppression.

Component 2 – Token-Level Loss Aggregation

Base GRPO divides the loss by the *number of sequences* G. DAPO divides by the *total number of tokens* across all sequences:

$$\mathcal{L}_{\text{token}} = -\frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \min\!\big(\rho_{i,t}\hat{A}_i,\ \text{clip}_{\text{DAPO}}(\rho_{i,t}, \hat{A}_i)\,\hat{A}_i\big).$$

This prevents long completions from dominating the gradient signal simply because they contain more tokens.

Component 3 – Overlong Filtering

When a completion is truncated (no EOS token within the maximum length budget), it provides *misleading* signal: the model is penalised for tokens that were generated correctly but happened to appear before the truncation boundary. DAPO masks these completions entirely:

$$m_i = \mathbf{1}[\text{EOS} \in o_i], \qquad \mathcal{L}_{\text{filtered}} = -\frac{\sum_{i=1}^{G} m_i \sum_t (\cdots)}{\sum_{i=1}^{G} m_i |o_i|}.$$

Component 4 – Soft Overlong Punishment

Rather than a hard mask, a softer variant applies a length penalty that grows smoothly as completions approach the maximum length $L_{\max}$:

$$r_i \leftarrow r_i - \lambda \cdot \max\!\left(0,\ \frac{|o_i| - L_{\text{cache}}}{L_{\max} - L_{\text{cache}}}\right),$$

where L_{cache} is a "safe" length threshold.

Component 5 – Dynamic Sampling

DAPO re-samples prompts whose entire group of completions receives the same reward (all correct or all incorrect), because such groups contribute zero gradient after normalisation. This keeps the effective batch size stable throughout training.

DAPO in TRL

```python
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # Asymmetric clipping
    epsilon=0.2,
    epsilon_high=0.28,          # Clip-Higher
    # Token-level loss
    loss_type="dapo",           # enables token-level aggregation
    # Overlong filtering
    mask_truncated_completions=True,
    # Generation budget
    max_completion_length=1024,
    num_generations=8,
    # Note: DAPO loss internally handles zero-variance group filtering
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
trainer.train()
```

When to Use DAPO

- Long-form reasoning tasks where completions frequently hit the length limit.
- Any setting where you observe reward variance collapsing mid-training.
- When base GRPO shows instability (loss spikes, entropy collapse).
- Recommended as a *drop-in improvement* over base GRPO for most tasks.

7.5.3 GSPO – Group Sequence Policy Optimization

The Off-Policy Problem

GRPO clips importance ratios *per token*. But a sequence of 500 tokens can have a product of per-token ratios that is astronomically large or small, even if each individual ratio is within $[1-\epsilon, 1+\epsilon]$. When training for multiple gradient steps on the same batch (off-policy), this mismatch grows rapidly and the clipping bound becomes meaningless at the sequence level.

GSPO [53] defines a *sequence-level* importance weight as the geometric mean of per-token ratios, which equals the $|o_i|$-th root of the full sequence probability ratio:

$$s_i(\theta) = \left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\text{old}}(o_i \mid q)}\right)^{1/|o_i|} = \exp\left(\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\log\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\text{old}}(o_{i,t}|q,o_{i,<t})}\right).$$

This is the *length-normalised* sequence probability ratio. The GSPO loss clips this single scalar per sequence:

$$\mathcal{L}_{\text{GSPO}} = -\frac{1}{G}\sum_{i=1}^{G}\min\Big(s_i(\theta)\,\hat{A}_i,\ \text{clip}(s_i(\theta),1-\epsilon,1+\epsilon)\,\hat{A}_i\Big).$$

GSPO vs GRPO Clipping

- **GRPO**: clips each of the $|o_i|$ per-token ratios independently. A sequence can have all ratios within bounds yet have a product ratio of 10^{50}.

- **GSPO**: clips the geometric mean once per sequence. Guarantees the *sequence-level* policy shift is bounded.
- GSPO is theoretically correct for off-policy IS; GRPO is an approximation.

GSPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # Sequence-level importance sampling
    importance_sampling_level="sequence",   # GSPO mode
    # Off-policy: reuse each batch for multiple gradient steps
    steps_per_generation=4,
    num_generations=8,
    epsilon=0.2,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

When to Use GSPO

GSPO is most beneficial when `steps_per_generation > 1` (off-policy training). For purely on-policy training (steps_per_generation = 1) the difference from GRPO is negligible. Off-policy training can dramatically reduce generation cost (the most expensive step), making GSPO + off-policy a strong efficiency choice.

7.5.4 Dr. GRPO – Debiased Reward GRPO

The Pretraining Bias Problem

Standard GRPO normalises advantages within a group, but the *pretraining distribution* introduces a systematic bias: tokens that are common in pretraining data receive large gradients even when they carry no task-relevant information. Dr. GRPO [244] identifies and corrects this bias, focusing gradient signal on *informative* tokens.

Dr. GRPO modifies the per-token gradient weight to account for the token's marginal contribution to the reward signal. Tokens that the model already assigns high probability to (regardless of the reward) are down-weighted:

$$w_{i,t} = \hat{A}_i \cdot (1 - \pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})),$$

where π_{ref} is the reference (pretrained) model. This is a form of *token efficiency*: the gradient is concentrated on tokens where the policy genuinely needs to change.

Dr. GRPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="dr_grpo",
    num_generations=8,
    beta=0.04,   # KL penalty coefficient
)

trainer = GRPOTrainer(
    model=model,
    ref_model=ref_model,   # required for token weighting
```

```
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

When to Use Dr. GRPO

- When training on tasks with a large vocabulary mismatch between pretraining and RL.
- When you observe that common filler tokens dominate the gradient.
- Pairs well with a reference model that is close to the initial policy.

7.5.5 2-GRPO – Minimal Two-Rollout GRPO

"It Takes Two" Insight

The "It Takes Two" paper [421] demonstrates empirically and theoretically that GRPO with $G = 2$ (just two completions per prompt) matches or exceeds GRPO with $G = 16$ on most reasoning benchmarks. This is surprising – why would fewer samples be sufficient?

The key insight is that GRPO's effectiveness does *not* primarily come from accurate advantage estimation (which requires large G). Instead, it comes from an implicit *contrastive objective* that is structurally similar to DPO:

$$\mathcal{L}_{\text{2-GRPO}} \approx -\mathbb{E}_{(o^+,o^-)\sim\pi_\theta}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(o^+|q)}{\pi_{\text{old}}(o^+|q)} - \beta\log\frac{\pi_\theta(o^-|q)}{\pi_{\text{old}}(o^-|q)}\right)\right],$$

where o^+ is the higher-reward completion and o^- the lower-reward one. With $G = 2$, this contrastive structure is explicit. With $G = 16$, the same signal is present but diluted by redundant pairs.

Compute Savings from 2-GRPO

- $G = 2$ vs $G = 16$: **8× less generation compute**.
- Generation is typically the bottleneck (60–80% of wall-clock time).
- Total training speedup: approximately 4–6× end-to-end.
- No accuracy loss on GSM8K, MATH, and code benchmarks.

2-GRPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    num_generations=2,       # The key change -- just two rollouts
    loss_type="grpo",        # Standard GRPO loss is fine
    epsilon=0.2,
    # With G=2, batch size must be at least 2 * num_prompts_per_step
    per_device_train_batch_size=2,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

Caveats of 2-GRPO

With $G = 2$, advantage normalisation is over only two values, so the normalised advantages are always $\{+1, -1\}$ (or $\{0, 0\}$ if rewards are equal). This means the gradient magnitude is fixed regardless of the reward gap. For tasks where the *magnitude* of the reward difference matters (e.g., partial credit), larger G may still be beneficial.

7.5.6 SAPO – Soft Adaptive Policy Optimization

The Brittleness of Hard Clipping

PPO-style clipping creates a discontinuous gradient: the gradient is zero outside the clip band and non-zero inside. This "cliff edge" can cause instability near the boundary and makes the trust region sensitive to the choice of ϵ. SAPO [139] replaces the hard clip with a smooth, temperature-controlled gate function.

SAPO replaces the $\min(\rho A, \text{clip}(\rho, \cdot)\, A)$ objective with a smooth surrogate:

$$\mathcal{L}_{\text{SAPO}}(\rho, A) = \begin{cases} -A \cdot \sigma\left(\dfrac{\rho - 1}{\tau_+}\right) \cdot \rho & \text{if } A > 0 \\ -A \cdot \sigma\left(\dfrac{1 - \rho}{\tau_-}\right) \cdot \rho & \text{if } A \leq 0 \end{cases}$$

where σ is the sigmoid function and τ_+, τ_- are asymmetric temperature parameters. A higher temperature produces a softer gate (more exploration); a lower temperature approaches hard clipping.

SAPO Temperature Intuition

- $\tau_+ = 1.0$: moderate gate for positive advantages (allow exploration).
- $\tau_- = 1.05$: slightly softer gate for negative advantages (avoid over-suppression).
- As $\tau \to 0$: recovers hard PPO clipping.
- As $\tau \to \infty$: recovers unclipped policy gradient.

SAPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="sapo",
    sapo_temperature_pos=1.0,     # tau_+ for positive advantages
    sapo_temperature_neg=1.05,    # tau_- for negative advantages
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

7.5.7 TIS and MIS – Truncated and Masked Importance Sampling

The Silent vLLM Probability Mismatch

When using vLLM for fast generation, the log-probabilities returned by vLLM differ from those computed during the training forward pass [459]. This is *not* a bug – it arises from different CUDA kernels, different floating-point precision, and different attention implementations (e.g., FlashAttention vs PagedAttention). The mismatch silently breaks the on-policy assumption: the

"old policy" probabilities used to compute importance ratios are wrong, leading to biased gradient estimates.

Truncated Importance Sampling (TIS)

TIS corrects the bias by multiplying the gradient by a truncated correction factor:

$$w_{\text{TIS}}(o_i) = \min\left(C,\ \frac{\pi_{\text{train}}(o_i|q)}{\pi_{\text{vllm}}(o_i|q)}\right),$$

where π_{train} is the probability from the training forward pass and π_{vllm} is the probability reported by vLLM. The truncation at C prevents extreme corrections from destabilising training.

Masked Importance Sampling (MIS)

MIS takes a harder approach: it zeros out the gradient for any sequence where the correction ratio exceeds a threshold C:

$$w_{\text{MIS}}(o_i) = \mathbf{1}\left[\frac{\pi_{\text{train}}(o_i|q)}{\pi_{\text{vllm}}(o_i|q)} \leq C\right].$$

This is more conservative but avoids the risk of large (even truncated) correction weights.

Sequence-Level vs Token-Level IS

Both TIS and MIS can be applied at the token level or the sequence level:

- **Sequence-level**: compute the ratio as the geometric mean over all tokens (as in GSPO). Theoretically correct but higher variance.
- **Token-level**: compute a separate ratio for each token. Biased (the product of per-token corrections is not the sequence correction) but lower variance.

TIS and MIS in TRL

```python
from trl import GRPOConfig, GRPOTrainer

# Truncated IS correction for vLLM probability mismatch
config_tis = GRPOConfig(
    use_vllm=True,
    vllm_importance_sampling_correction=True,
    vllm_importance_sampling_mode="sequence_truncate",  # TIS
    vllm_importance_sampling_cap=5.0,                   # C threshold
)

# Masked IS correction
config_mis = GRPOConfig(
    use_vllm=True,
    vllm_importance_sampling_correction=True,
    vllm_importance_sampling_mode="sequence_mask",      # MIS
    vllm_importance_sampling_cap=3.0,
)
```

When to Use TIS/MIS

- **Always** consider enabling when using vLLM for generation.
- TIS is preferred when the mismatch is small (same model, different precision).
- MIS is preferred when the mismatch is large or unpredictable.

- Sequence-level IS is theoretically preferred; token-level is a practical compromise.

7.5.8 VESPO – Variational Sequence-Level Soft Policy Optimization

Principled Reward Reshaping

Most GRPO variants modify the clipping mechanism heuristically. VESPO derives a principled reward-reshaping kernel from a variational inference framework, treating policy optimisation as approximate posterior inference. VESPO [256] derives a resulting kernel that is smooth, asymmetric, and naturally handles staleness in asynchronous or off-policy training.

VESPO derives a weighting function $W(\tau)$ for each trajectory τ from the variational objective. The final gradient weight takes the form:

$$\boxed{g(\tau) = W(\tau)^k \cdot \exp(\lambda(1 - W(\tau))),}$$

where $W(\tau) = \pi_\theta(\tau)/\pi_{\text{old}}(\tau)$ is the sequence-level importance weight, k controls the sharpness of the weighting, and λ controls the exponential decay for stale (low-weight) trajectories. This kernel:

- Is smooth everywhere (no discontinuous gradient at clip boundaries).
- Naturally down-weights stale trajectories ($W \ll 1$) via the exponential term.
- Is asymmetric: high-weight trajectories ($W > 1$) are treated differently from low-weight ones.

VESPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="vespo",
    vespo_k_pos=2.0,          # sharpness exponent (positive advantages)
    vespo_lambda_pos=3.0,     # staleness decay (positive advantages)
    num_generations=8,
    steps_per_generation=2,   # off-policy; VESPO handles staleness
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

7.5.9 DPPO – Direct Policy Divergence Policy Optimization

The Problem with Ratio Clipping

PPO's ratio clipping is a proxy for constraining the KL divergence between old and new policy. But the proxy is imperfect: clipping over-penalises low-probability tokens (where a small absolute change in probability corresponds to a large ratio change) and under-penalises high-probability tokens (where a large absolute change corresponds to a small ratio change). DPPO [8] replaces ratio clipping with *direct divergence estimates*.

DPPO computes the trust region constraint directly using either Total Variation (TV) or KL divergence between the old and new policy distributions:

$$\mathcal{L}_{\text{DPPO}} = -\mathbb{E}\Big[\hat{A} \cdot \pi_\theta(o|q) \cdot \mathbf{1}[D(\pi_\theta \| \pi_{\text{old}}) \leq \delta]\Big],$$

where D is the chosen divergence measure. In practice, DPPO approximates this with token-level binary or top-k masks:

- **binary_tv**: mask tokens where $|\pi_\theta - \pi_{\text{old}}| > \delta$.
- **binary_kl**: mask tokens where $\pi_\theta \log(\pi_\theta/\pi_{\text{old}}) > \delta$.
- **topk_tv**: keep only the top-k tokens by TV contribution.
- **topk_kl**: keep only the top-k tokens by KL contribution.

DPPO – Conceptual Implementation

DPPO is not yet available as a built-in TRL trainer. A custom implementation would use GRPOTrainer with a modified loss that clips based on distributional divergence (TV or KL) rather than the standard probability ratio:

```
# Pseudocode: DPPO requires a custom trainer subclass
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    num_generations=8,
    beta=0.04,
)
# Override the loss computation to use distributional clipping:
# clip when TV(pi_new || pi_old) > delta, rather than when
# pi_new/pi_old exceeds [1-eps, 1+eps]
```

DPPO is Research-Stage

DPPO is a recent research contribution and is not yet integrated into mainstream RL libraries. It is most useful when you observe that standard ratio clipping is failing (e.g., on tasks with highly skewed token probability distributions).

7.5.10 ScaleRL and CISPO

Scaling Laws for RL

The ScaleRL paper [255] conducts a systematic study of what makes RL training for LLMs scale effectively. The key finding is that two modifications – batch-level reward scaling and DAPO-style token-level loss – together unlock strong performance at scale, while neither alone is sufficient. CISPO (Clipped IS Policy Optimization) is the resulting algorithm.

Batch-Level Reward Scaling

Standard GRPO normalises rewards within a group of G completions for a single prompt. CISPO normalises rewards across the *entire batch*:

$$\hat{A}_i = \frac{r_i - \mu_{\text{batch}}}{\sigma_{\text{batch}} + \epsilon},$$

where μ_{batch} and σ_{batch} are computed over all rewards in the current training batch. This provides a more stable baseline and prevents any single prompt from dominating the gradient.

CISPO Loss

CISPO combines batch-level scaling with DAPO's token-level loss aggregation and asymmetric clipping:

$$\mathcal{L}_{\text{CISPO}} = -\frac{1}{\sum_{i,t} m_{i,t}} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} m_{i,t} \cdot \min\!\left(\rho_{i,t}\hat{A}_i,\ \text{clip}_{\text{DAPO}}(\rho_{i,t}, \hat{A}_i)\,\hat{A}_i\right),$$

where $m_{i,t}$ is the overlong-filtering mask.

CISPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="cispo",
    scale_rewards="batch",          # batch-level reward normalisation
    mask_truncated_completions=True,
    epsilon=0.2,
    epsilon_high=5.0,               # epsilon_max for CISPO (ScaleRL paper)
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

ScaleRL Key Findings

1. Batch-level reward scaling alone: modest improvement.
2. Token-level loss alone: modest improvement.
3. Both together: **synergistic** – significantly better than either alone.
4. Larger batch sizes benefit more from batch-level scaling.
5. CISPO is the recommended default for large-scale RL training.

7.5.11 GDPO – Group Reward-Decoupled Policy Optimization

The Multi-Reward Collapse Problem

In multi-objective RL (e.g., optimising for both correctness and format), standard GRPO normalises the *combined* reward. If one reward has much higher variance than another, it dominates the normalised advantage, effectively ignoring the other reward. This is *advantage collapse*: the low-variance reward contributes near-zero gradient. GDPO [460] normalises each reward *independently* before aggregating.

The core mechanism normalises each reward *independently* before aggregating:

$$\hat{A}_n^{(i)} = \frac{r_n^{(i)} - \mu_n}{\sigma_n + \epsilon}, \qquad \hat{A}^{(i)} = \sum_{n=1}^{N} w_n \hat{A}_n^{(i)},$$

where $r_n^{(i)}$ is the n-th reward for completion i, μ_n and σ_n are the mean and standard deviation of reward n within the group, and w_n are user-specified weights.

GDPO vs Standard Multi-Reward GRPO

- **Standard**: $\hat{A}^{(i)} = \frac{\sum_n w_n r_n^{(i)} - \mu_{\text{combined}}}{\sigma_{\text{combined}}}$. High-variance rewards dominate.
- **GDPO**: normalise each reward separately, then combine. Each reward contributes proportionally to its weight w_n.
- GDPO is essential when rewards have very different scales or variances.

GDPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    multi_objective_aggregation="normalize_then_sum",
    reward_weights=[1.0, 0.5],   # weights for [correctness, format]
    num_generations=8,
)

def correctness_reward(completions, **kwargs):
    return [1.0 if is_correct(c) else 0.0 for c in completions]

def format_reward(completions, **kwargs):
    return [0.1 if has_good_format(c) else 0.0 for c in completions]

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[correctness_reward, format_reward],
    args=config,
    train_dataset=dataset,
)
```

7.5.12 GOPO – Group Ordinal Policy Optimization

GOPO [56] starts from a simple observation: reward models are trained with pairwise comparisons ("is A better than B?"), so only the **rank order** of their outputs is trustworthy—the raw numeric scores carry no inherent meaning. Yet GRPO feeds those raw magnitudes directly into the advantage calculation. For tasks with non-verifiable rewards—summarization, open-ended chat, instruction following—this mismatch introduces noise, because a gap of 0.6 reward points might reflect genuine quality in one region of the output space and mean nothing in another.

Key Insight: Discard reward magnitudes entirely. Use only the **ordinal ranking** of rewards within a group.

Algorithm: Given a group of N responses $\{o_1, \ldots, o_N\}$ with rewards $\{r_1, \ldots, r_N\}$:

1. Rank responses by reward: assign rank $\text{rank}(o_i) \in \{1, \ldots, N\}$ (1 = worst, N = best).
2. Replace raw advantages with rank-based scores:

$$\boxed{\hat{A}_i^{\text{GOPO}} = f\left(\frac{\text{rank}(o_i)}{N}\right)} \tag{7.2}$$

   where f is a monotonic transformation (e.g., linear mapping to $[-1, 1]$ or quantile normalization).
3. Apply PPO-style clipped objective using rank-based advantages.

Comparison with GRPO:

Aspect	GRPO	GOPO
Advantage signal	$\hat{A}_i = (r_i - \mu)/\sigma$ (uses magnitudes)	$\hat{A}_i = f(\text{rank}_i/N)$ (uses ordinal rank only)
Sensitivity to reward scale	High — miscalibrated RM scores distort advantages	None — invariant to monotonic reward transformations
Best for	Verifiable rewards (binary, well-calibrated)	Non-verifiable rewards (RM-based, noisy magnitudes)

Empirical gains (over GRPO on non-verifiable tasks):

- Reward curves (both training and held-out) sit above GRPO throughout optimization

- Win-rates judged by a separate LLM evaluator improve at most training checkpoints
- Convergence is markedly faster—matching GRPO's final quality with fewer gradient steps
- The advantage grows as the reward model becomes noisier or more poorly calibrated

When to Use GOPO vs. GRPO

- **Use GRPO**: When rewards are verifiable and exact (math correctness, code tests pass/fail, binary signals). Magnitudes carry meaningful information.
- **Use GOPO**: When rewards come from a learned reward model on subjective tasks (helpfulness, style, safety). The RM's relative ordering is trustworthy but its absolute scores are arbitrary.

Chapter 8

Preference Optimization Variants

This chapter covers the family of methods that extend or replace DPO with different objectives, data assumptions, or architectural trade-offs. The core variants each address a specific limitation of standard offline DPO: distribution shift (Online DPO), the need for paired data (KTO), overfitting to noisy labels (IPO), reference model memory cost (ORPO), or training complexity (Best-of-N). The chapter closes with a new paradigm entirely: **calibrated decision training** (RLCR, RLCD/Jev) and **CLM** (Contrastive Language Model), a System One model trained with a bidirectional InfoNCE objective over (state, action) pairs for fast, cached agent decisions.

GAS Connection — Preference Optimization Variants

The GAS generates rich preference data from every pipeline run, making it a natural testbed for the preference optimisation variants described in the following chapter.

KTO (Kahneman–Tversky Optimisation). The GAS Caregiver Notifier generates binary feedback: caregivers either act on an alert (positive signal) or dismiss it (negative signal). KTO is well-suited for this setting because it works with unpaired binary feedback rather than requiring (chosen, rejected) pairs. The `caregiver_notified` field in `GASState` captures this binary outcome.

IPO (Identity Preference Optimisation). IPO's regularisation prevents the policy from collapsing to a degenerate solution—a critical property for the GAS Advisor, which must maintain diversity in its recommendations across different patient scenarios. The five clinical scenarios in `shared/patient_data.py` (`normal_day`, `hypoglycemia_episode`, `post_meal_spike`, `dawn_phenomenon`, `exercise_induced_low`) provide the diversity needed to evaluate this property.

Choosing the right variant. The decision framework in the following chapter maps directly onto the GAS: use DPO when paired (chosen, rejected) data is available from the Safety Critic; use KTO when only binary caregiver feedback is available; use GRPO when generating multiple candidates per state. The GAS pipeline produces all three types of feedback, enabling a systematic comparison of preference optimisation variants.

Practical implementation. All preference optimisation variants share the same data pipeline: the `GASState` TypedDict in `capstone_gas/gas_system.py` captures the patient state, recommendation, safety score, and caregiver response needed to construct training examples for any variant. The chapter notebooks in `part2_rl_methods/` demonstrate each variant on clinical recommendation data.

8.1 Online DPO

8.1.1 Motivation

Standard DPO's primary limitation: the preference data was generated by a *different* model (often an older checkpoint or even a different model family). As training progresses, the policy generates text that looks nothing like the training pairs → the loss is optimizing on an irrelevant distribution.

Online DPO solution [135]: Generate fresh preference pairs from the *current* policy at every step, judge them with a reward model, then apply the DPO loss.

8.1.2 Algorithm

1. Generate K responses per prompt from current π_θ
2. Score all responses with reward model r_ϕ
3. Create pairs: highest-scoring = chosen, lowest-scoring = rejected
4. Apply DPO loss on these fresh pairs
5. Repeat (new generation every step)

Online DPO = Best of Both Worlds

- From DPO: simple supervised loss, no value function, no GAE, stable optimization
- From PPO: on-policy data, self-improvement beyond dataset, no distribution shift
- Key difference from GRPO: uses DPO loss (pair-based) instead of PPO loss (per-sample advantage)

Trade-off: Needs a reward model (DPO doesn't), but no value head (PPO does). Middle ground complexity.

8.1.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import OnlineDPOConfig, OnlineDPOTrainer
from transformers import AutoModelForCausalLM, AutoModelForSequenceClassification

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16)
reward_model = AutoModelForSequenceClassification.from_pretrained(
    "RLHFlow/ArmoRM-Llama3-8B-v0.1", torch_dtype=torch.bfloat16)

online_dpo_config = OnlineDPOConfig(
    output_dir="./online_dpo_output",
    learning_rate=5e-7,
    beta=0.1,                    # DPO beta (same meaning as standard DPO)
    num_generations=4,           # K responses per prompt
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    max_new_tokens=512,
    temperature=0.7,
    bf16=True,
    num_train_epochs=1,
    logging_steps=10,
)

trainer = OnlineDPOTrainer(
    model=model,
    reward_model=reward_model,
    args=online_dpo_config,
    train_dataset=prompt_dataset,
    tokenizer=tokenizer,
)
trainer.train()
```

8.1.4 Online DPO vs Offline DPO vs PPO

	Data	Models	Loss	Best For
Offline DPO	Static pairs	2 (policy + reference)	DPO	Quick alignment, limited compute
Online DPO	Fresh from π_θ	3 (policy + reference + reward model)	DPO	When DPO plateaus, need exploration
PPO	Fresh from π_θ	4 (policy + reference + reward model + value head)	PPO clip	Max quality, complex reasoning

8.2 KTO — Kahneman-Tversky Optimization

8.2.1 Motivation

DPO requires *paired* preferences: for the same prompt, you need both a good and bad response. In practice, most feedback is *unpaired*: users give thumbs up/down on individual responses, with no matched pair.

KTO's insight [91]: Use prospect theory (from behavioral economics). Humans feel losses more strongly than gains. A "thumbs down" should produce a stronger gradient than a "thumbs up."

8.2.2 Loss Function

$$\boxed{\mathcal{L}_{\text{KTO}} = \mathbb{E}_{y_w}\left[\lambda_w(1 - v(x, y_w))\right] + \mathbb{E}_{y_l}\left[\lambda_l \cdot v(x, y_l)\right]} \tag{8.1}$$

where $v(x, y) = \sigma\left(\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}}\right)$, and z_{ref} is the expected KL divergence (a running baseline).

KTO Intuition via Prospect Theory

Desirable responses (y_w): The model gets "utility" from increasing their probability. But with diminishing returns — once it's already quite likely, don't push harder.
Undesirable responses (y_l): Loss aversion means the penalty for generating bad text is weighted more strongly than the reward for good text. Default: $\lambda_l = 1.0$, $\lambda_w = 1.0$, but you can set $\lambda_l > \lambda_w$.
Key advantage: Each training example is independent! No need to find matched pairs. Can use thumbs-up/down data directly.

KTO Data Format

Unlike DPO which needs: `{"prompt": ..., "chosen": ..., "rejected": ...}`
KTO only needs: `{"prompt": ..., "completion": ..., "label": true/false}`
This means you can use:

- Thumbs up/down from production traffic
- Upvotes/downvotes from forums
- Human ratings binarized (4–5 stars = good, 1–2 = bad)
- Any per-response quality signal

8.2.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import KTOConfig, KTOTrainer

# Dataset format: {"prompt": str, "completion": str, "label": bool}
# label=True for desirable, label=False for undesirable
kto_dataset = [
    {"prompt": "What's 2+2?", "completion": "The answer is 4.", "label": True},
```

```
    {"prompt": "What's 2+2?", "completion": "It might be 5.", "label": False},
]

kto_config = KTOConfig(
    output_dir="./kto_output",
    beta=0.1,
    desirable_weight=1.0,        # Weight for good examples
    undesirable_weight=1.0,      # Weight for bad examples (increase for loss
    aversion)
    learning_rate=5e-7,
    max_length=2048,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    num_train_epochs=1,
    bf16=True,
)

trainer = KTOTrainer(
    model=model,
    ref_model=ref_model,  # Or None with LoRA
    args=kto_config,
    train_dataset=kto_dataset,
    tokenizer=tokenizer,
)
trainer.train()
```

8.2.4 When to Choose KTO

- You have binary feedback but *not* matched pairs
- Production thumbs-up/down data at scale
- One class dominates (e.g., 90% good, 10% bad) — KTO handles imbalance better
- Rapid iteration with noisy labels (more robust than DPO to noise)

8.3 IPO — Identity Preference Optimization

8.3.1 Motivation

DPO has a degenerate solution: it can achieve zero loss by making the margin between chosen and rejected *infinitely large*. In practice, this means DPO overfits — pushing chosen probability to 1 and rejected to 0, memorizing training data.

IPO's fix [15]: Instead of log-sigmoid (which saturates), use a squared loss that targets a *specific* margin. The loss is minimized at a finite gap, not at infinity.

8.3.2 Loss Function

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}\left[\left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta}\right)^2\right] \tag{8.2}$$

IPO vs DPO: Regularization Through Target Margin

DPO: $\sigma(\text{margin}) \to 1$ optimally. Margin $\to \infty$. No natural stopping point.
IPO: Margin $\to \frac{1}{2\beta}$ optimally. Squared loss penalizes **both** too-small and too-large margins.
Result: IPO is more robust to noisy labels (a mislabeled pair gets bounded influence), and generalizes better because it doesn't memorize.

8.3.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import DPOConfig, DPOTrainer

# IPO is implemented as a DPO loss_type variant in TRL
ipo_config = DPOConfig(
    output_dir="./ipo_output",
    beta=0.1,
    loss_type="ipo",              # The key difference!
    learning_rate=5e-7,
    max_length=2048,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    bf16=True,
    num_train_epochs=1,
)

trainer = DPOTrainer(
    model=model, ref_model=None, args=ipo_config,
    train_dataset=pref_dataset, tokenizer=tokenizer, peft_config=lora_config,
)
trainer.train()
```

8.3.4 When to Choose IPO over DPO

- Noisy preference data (crowdsourced, AI-judged with errors)
- Observing DPO overfitting (train loss $\to 0$ but eval degrades)
- Want more conservative, robust alignment
- Multiple epochs needed (DPO degrades after epoch 1; IPO is more stable)

8.4 ORPO — Odds Ratio Preference Optimization

8.4.1 Motivation

All methods so far need a reference model — either as a separate copy (doubles memory) or implicitly via LoRA. ORPO [151] eliminates the reference entirely by combining SFT and preference alignment in a single loss.

Key insight: Use the *odds ratio* of generating chosen vs rejected as the preference signal. The SFT component naturally prevents collapse (no need for KL regularization).

8.4.2 Loss Function

$$\mathcal{L}_{\text{ORPO}} = \underbrace{\mathcal{L}_{\text{SFT}}(y_w)}_{\text{standard NLL on chosen}} - \lambda \cdot \underbrace{\log \sigma \left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)} \right)}_{\text{preference alignment via odds ratio}} \tag{8.3}$$

where $\text{odds}_\theta(y|x) = \frac{P_\theta(y|x)}{1-P_\theta(y|x)}$.

ORPO: SFT + Alignment in One Shot

SFT term: Trains the model to generate the chosen response well (standard language modeling).
Odds ratio term: Additionally pushes the model to prefer chosen over rejected. The odds ratio is a natural contrast that doesn't require a reference model.
Why no reference needed?: The SFT loss already anchors the model to reasonable text. It serves the same role as KL-to-reference in other methods. One model, one forward pass, one loss.

50% less memory!

8.4.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import ORPOConfig, ORPOTrainer

orpo_config = ORPOConfig(
    output_dir="./orpo_output",
    beta=0.1,                    # Odds ratio weight (lambda)
    learning_rate=5e-7,
    max_length=2048,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    bf16=True,
    num_train_epochs=1,
    gradient_checkpointing=True,
)

trainer = ORPOTrainer(
    model=model,                 # No ref_model needed!
    args=orpo_config,
    train_dataset=pref_dataset,  # Same format as DPO: prompt/chosen/rejected
    tokenizer=tokenizer,
    peft_config=lora_config,
)
trainer.train()
```

8.4.4 When to Choose ORPO

- Memory-constrained: can't afford reference model copy (saves 70–140GB for 70B)
- Starting from base model (not SFT'd yet) — ORPO does SFT simultaneously
- Want simplest possible pipeline: one model, one loss, one training run
- Good preference data available from the start

ORPO Limitations

- Less studied than DPO/PPO — fewer proven recipes at 70B+ scale
- The SFT component means it needs high-quality chosen responses (not just relative preference)
- Harder to debug: two loss components can conflict

See Also: SimPO

SimPO [265] is another reference-free preference method that uses length-normalized log-probability as an implicit reward, eliminating the reference model entirely. It is covered in Section 6.9.8 alongside other DPO extensions due to its shared reference-free philosophy.

8.5 Best-of-N Sampling (Rejection Sampling)

8.5.1 Motivation

Sometimes the simplest approach wins. Best-of-N [276] requires *no training at all* during the RL phase — just generate multiple candidates and pick the best one.

8.5.2 Algorithm

1. For each prompt, generate N responses from the policy (typically $N = 4$–64)
2. Score all responses with a reward model
3. Select the highest-scoring response
4. (Optional) Use selected responses as SFT data for the next iteration

$$\boxed{\text{Best-of-N response}: \quad y^* = \arg\max_{y_i \sim \pi_\theta(\cdot|x)} r_\phi(x, y_i)} \tag{8.4}$$

Why Best-of-N is a Legitimate "RL" Method

At inference time: Best-of-N improves output quality without changing model weights. With $N = 64$, win-rate improves 10–20% over greedy — sometimes matching or exceeding PPO.
As a training method (Rejection Sampling Fine-Tuning / RFT):

1. Generate many responses, select best ones
2. SFT on the selected responses
3. Repeat (iterative refinement)

This is how many production models are trained: simpler than PPO, almost as effective, completely stable.
Theoretical connection [111]: Best-of-N implements an implicit KL-constrained policy: $\pi_{\text{BoN}}(y|x) \propto \pi_\theta(y|x)^{1-1/N} \cdot r(x, y)^{1/N}$.

8.5.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from transformers import pipeline
import numpy as np

# Inference-time Best-of-N (manual implementation)
gen_pipeline = pipeline("text-generation", model=model, tokenizer=tokenizer)

def best_of_n(prompt, n=16, temperature=0.8):
    """Generate N candidates and return the highest-reward one."""
    candidates = gen_pipeline(
        prompt, num_return_sequences=n,
        temperature=temperature, do_sample=True, max_new_tokens=512,
    )
    scores = [reward_model.score(prompt, c["generated_text"]) for c in candidates]
    return candidates[np.argmax(scores)]["generated_text"]

best_response = best_of_n(prompt, n=16)

# Training: Rejection Sampling Fine-Tuning (RFT)
from trl import SFTConfig, SFTTrainer

# Step 1: Generate and filter
all_responses = []
for prompt in prompts:
    candidates = [generate(prompt, temp=0.9) for _ in range(16)]
    scores = [reward_model.score(prompt, c) for c in candidates]
    best_idx = np.argmax(scores)
    if scores[best_idx] > threshold:  # Quality gate
        all_responses.append({"prompt": prompt, "completion": candidates[best_idx
    ]})
```

```
# Step 2: SFT on best responses
sft_config = SFTConfig(output_dir="./rft_output", learning_rate=2e-5,
    num_train_epochs=2, max_seq_length=2048)
trainer = SFTTrainer(model=model, args=sft_config, train_dataset=all_responses,
    tokenizer=tokenizer)
trainer.train()
# Step 3: Repeat from Step 1 with updated model (iterative RFT)
```

8.5.4 Scaling Laws for Best-of-N

N	Quality Gain	Cost	Notes
1	Baseline	1×	Standard sampling
4	+5–8% win-rate	4×	Minimum useful. Good cost/quality ratio
16	+10–15% win-rate	16×	Strong. Often matches PPO quality
64	+15–20% win-rate	64×	Diminishing returns start
256	+18–22% win-rate	256×	Only for critical applications

Best-of-N as Baseline

Always compare your RL method against Best-of-N with the same compute budget. If PPO with 64 GPU-hours doesn't beat Best-of-N with 64 GPU-hours of generation, your PPO has a bug.

8.6 Beyond Text Generation: Calibrated Decision Training

The alignment methods surveyed so far — PPO, DPO, GRPO, KTO, IPO, ORPO — all share a common assumption: the model's output is **generated text**, and the training objective is to make that text more preferred, more correct, or more aligned with human values. A newer line of work challenges this assumption at a more fundamental level. For software automation, the bottleneck is not text quality but **decision reliability**: can the model make thousands of typed decisions per second with calibrated confidence?

8.6.1 The Calibration Problem with RLHF and RLVR

Standard LLMs output next-token probabilities. These are **not** calibrated decision probabilities: a model assigning high probability to a token does not thereby establish a corresponding probability of being correct on the underlying business decision. Worse, RLVR training has been empirically shown to **degrade calibration** [66]: models become overconfident, especially out-of-distribution, because the binary reward signal is indifferent to whether the model answers at 51% or 99% confidence.

Why Calibration Matters for Automation

A customer-support routing model that says "billing issue, confidence 0.94" is only useful if, across thousands of such predictions, roughly 94% actually are billing issues. If the model is overconfident (says 0.94 but is right only 70% of the time), automated downstream logic will make systematic errors. Prose confidence ("I'm fairly confident that...") is not numerically actionable at all.

Expected Calibration Error (ECE) measures this gap:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right| \tag{8.5}$$

where predictions are grouped into M confidence bins B_m, and the gap between average accuracy and average confidence is weighted by bin size. A perfectly calibrated model has ECE = 0.

8.6.2 RLCR: Reinforcement Learning with Calibration Rewards

Damani et al. [66] (MIT) introduce **RLCR** as a drop-in replacement for the binary RLVR reward that explicitly trains for calibration. The model outputs a verbalized confidence estimate $q \in [0, 1]$ alongside its answer y, and the reward combines correctness with a **Brier score** calibration penalty:

$$R_{\text{RLCR}}(y, q, y^*) = \underbrace{\mathbb{1}[y = y^*]}_{\text{correctness}} - \underbrace{\left(q - \mathbb{1}[y = y^*]\right)^2}_{\text{Brier calibration penalty}} \tag{8.6}$$

where $\mathbb{1}[y = y^*]$ is the **indicator function**: it equals 1 if the model's answer y matches the ground truth y^*, and 0 otherwise. So the reward is $+1 - (q-1)^2$ for a correct answer and $0 - (q-0)^2 = -q^2$ for a wrong one. A model that answers correctly *and* says $q = 1$ gets the maximum reward of $+1$; a model that answers wrongly but hedges with $q = 0$ gets 0 (not penalized for the calibration term, but gets no correctness credit either).

The Brier score $(q - a)^2$ is a **proper scoring rule** [119]: its expectation is minimized when q equals the true success probability p_y. Its **boundedness** in $[0, 1]$ is critical. Log-loss is also a proper scoring rule but is unbounded below, which opens a perverse incentive: a model could reduce its calibration penalty by outputting wrong answers with near-zero confidence, sidestepping the accuracy requirement entirely. The bounded Brier score forecloses this exploit [66].

RLCR Calibration Incentive [66]

Under R_{RLCR}: (1) for any answer y, expected reward is maximized at $q = p_y$ (the true success probability); (2) among calibrated predictions, reward is maximized by the most likely correct answer.

Training setup [66]: Qwen2.5-7B trained with GRPO (no KL regularization). The model generates in the format `<think>...</think><answer>...</answer><confidence>0.87</confidence>`. Trained on HotpotQA-Modified, where controlled uncertainty is introduced by removing 0, 1, or 2 relevant paragraphs from each example.

Key results (vs. RLVR baseline):

Table 8.1: RLCR vs. RLVR on calibration metrics [66] (v2, May 2026). RLVR degrades OOD calibration; RLCR improves it substantially while matching accuracy. Accuracy for RLVR from Table 2 (ablation); calibration metrics from Table 1.

Method	Accuracy	AUROC	Brier	ECE
In-domain (HotpotQA)				
Base	39.7%	0.54	0.53	0.53
RLVR	63.0%	0.50	0.37	0.37
RLCR	**62.1%**	**0.69**	**0.21**	**0.03**
Out-of-domain (6 datasets avg.)				
Base	53.3%	0.54	0.41	0.40
RLVR	53.9%	0.50	0.46	0.46
RLCR	**56.2%**	**0.68**	**0.21**	**0.21**

RLVR *worsens* OOD calibration relative to the base model; RLCR improves it substantially [66]. RLCR also enables **confidence-weighted majority vote** at test time: sampling N responses with confidences $q_1, \ldots, q_N$ and weighting votes by confidence outperforms vanilla majority vote, max-confidence selection, and likelihood-based baselines.

8.6.3 System One Models: A New Output Paradigm

TypeSafe AI [6] introduce a new model class called **System One Models**, with their first release named **Jev** (September 2026). Rather than generating text token-by-token, System One Models output **typed probability distributions** over predefined answer classes in a single forward pass — a fundamental departure from the autoregressive generation paradigm.

Input: unstructured state text + typed questions of three kinds:

- **Choice**: probability distribution over named options (e.g., `Choice("billing", "technical", "refund")`)
- **Score**: expected level on an ordered scale (e.g., `Score(1, 5)`)
- **Noul**: boolean probability (e.g., `Noul("is_urgent")`)

Output: calibrated probability distributions, not text. A **readout head** replaces the vocabulary head:

$$z = \mathbf{W}h + b, \qquad p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{8.7}$$

where h is the final hidden state and K is the number of answer classes. Multiple questions share the same encoded state (one forward pass for the state; one lightweight suffix per question), giving $O(S) + O(Q)$ complexity versus $O(QS)$ for separate LLM calls.

Training: RLCD (Reinforcement Learning for Calibrated Decisions) [6] — TypeSafe's proprietary post-training method, not yet fully published. The stated objective: optimize for calibrated decisions where predicted probabilities match empirical outcome frequencies.

Table 8.2: System One Models vs. standard LLMs on workflow automation tasks.

Dimension	Standard LLM	System One Model (Jev)
Output	Generated text	Typed probability distributions
Sampling	Sequential (token-by-token)	Parallel (single forward pass)
Latency	3–329s	70–500ms
Cost	\$0.20–\$10/MTok output	Output effectively free
Type errors	Possible	Mathematically impossible
Calibration	Weak (uncalibrated)	Core training objective

Reported results [6]: 84.6% accuracy on MMLU-Pro with ECE = 0.031; 193.6× faster and 444.6× cheaper than frontier LLMs on published workflow benchmarks.

8.6.4 CLM: Contrastive Language Models

Kwok, Kang, Suresh et al. [205] (September 2026, Apache 2.0) introduce **CLM** as an open-source System One Model trained with **contrastive learning** rather than RLCD. Where Jev uses a single encoder with readout heads, CLM separates state and action processing into two independent encoders:

- **State encoder**: processes the current context
- **Action encoder**: processes each candidate answer/action

Both are frozen LLM backbones (Qwen3-8B) with 20M-parameter trainable projection heads. The decision score is a dot product of normalized embeddings:

$$\text{score}(s, a) = \mathbf{s}^\top \mathbf{a}, \qquad P(a_k \mid s) = \text{softmax}_k(\text{score}(s, a_k)) \tag{8.8}$$

Training objective — bidirectional InfoNCE over a batch of B matched (state, action) pairs:

$$\mathcal{L}_{\text{CLM}} = -\frac{1}{2B} \sum_i \left[\log \frac{e^{\mathbf{s}_i^\top \mathbf{a}_i / \tau}}{\sum_j e^{\mathbf{s}_i^\top \mathbf{a}_j / \tau}} + \log \frac{e^{\mathbf{a}_i^\top \mathbf{s}_i / \tau}}{\sum_j e^{\mathbf{a}_i^\top \mathbf{s}_j / \tau}} \right] \tag{8.9}$$

where τ is a learned temperature. This pulls each state toward its ground-truth action and pushes it away from all other actions in the batch. A **hard negative extension** adds semantically similar but incorrect actions $h_{ik}^{(a)}$ to the denominator during mid-training, forcing finer discrimination.

Three-stage training recipe:

1. **Pre-training**: 60M (state, action) pairs from Nemotron DQA — broad semantic representations

2. **Mid-training**: 30M synthetic hard negatives (generated by Gemini 2.5 Flash-Lite) — fine-grained discrimination

3. **Post-training**: 1M agentic trajectories (ADP dataset + Terminal traces) — agentic decision-making; 40% replay of pre-training data to prevent catastrophic forgetting

Key efficiency advantage: state and action embeddings are **disaggregated** — each is computed once and cached independently. For repeated states (e.g., the same document being queried with many different questions), only the question embedding needs recomputing. This yields up to 9× lower latency than Jev on cached workloads.

Results [205]: matches Jev on computer-use, gaming, and tool-calling benchmarks (zero-shot); achieves state-of-the-art on agentic verification (DeepSWE 81.6%, Terminal-Bench 2.1: 87.6%). Jev fails below pass@1 on long-horizon agentic verification; CLM succeeds with lightweight fine-tuning.

Scaling law [205]: InfoNCE test loss follows $L(X) \approx (X_c/X)^{\alpha_X}$. Encoder size is the dominant scaling axis; optimal projection head size scales as $N^* \propto D^{1.02}$ (approximately 310 tokens per parameter).

8.6.5 Comparison: RLHF, DPO, RLVR, RLCR, RLCD, CLM

Table 8.3: Full comparison of alignment and calibrated decision training paradigms.

Method	Reward Source	Output	Calibration	Best For
RLHF (PPO)	Human preference	Text	Weak	Chatbots, assistants
DPO	Pairwise preference	Text	Weak	Style, safety
RLVR	Binary correctness	Text	Degrades OOD	Math, code
RLCR	Correctness + Brier	Text + confidence	**Explicitly improved**	Reliable reasoning
RLCD (Jev)	Calibrated outcomes	Typed distributions	**Core objective**	Software automation
CLM	InfoNCE contrastive	Embedding similarity	Implicit via training	Agent decisions

The Calibration Shift

RLHF asks: *what should the model say?* RLVR asks: *is the model correct?* RLCR and RLCD ask: *how reliably does the model know what it knows?* — shifting from output quality to epistemic reliability. CLM takes a different angle: rather than calibrating confidence, it learns a contrastive embedding space where the right action is geometrically closest to the current state, enabling fast cached decisions without a generative forward pass.

8.7 Summary: Choosing an Alignment Method

We have now surveyed the full landscape of preference optimization and RL-based alignment methods. This section consolidates the key trade-offs into a single reference to help practitioners select the right approach for their constraints.

Table 8.4: Cross-method comparison of alignment approaches.

Method	Models	Data	Compute	Stability	Best For
PPO	4	Online (gen)	Very high	Low	Max quality, complex reasoning
GRPO	2 (no critic)	Online (gen)	High	Medium	Math/code (verifiable rewards)
DPO	2	Offline pairs	Low	High	Style/safety, limited compute
Online DPO	3	Online (gen)	Medium	Medium-High	DPO without distribution shift
KTO	2	Unpaired binary	Low	High	Production feedback, thumbs up/down
IPO	2	Offline pairs	Low	Very high	Noisy labels, anti-overfitting
ORPO	1	Offline pairs	Very low	High	Memory-limited, SFT+align combined
Best-of-N	1+RM	Online (gen)	Medium	Perfect	Strong baseline, data generation
RLCR	1	Online (gen)	Medium	Medium	Calibrated confidence + correctness
RLCD (Jev)	1	Outcome-matched	Medium	High	Software automation, typed decisions
CLM	1 (dual-enc.)	Contrastive pairs	Medium	High	Agent decisions, fast inference

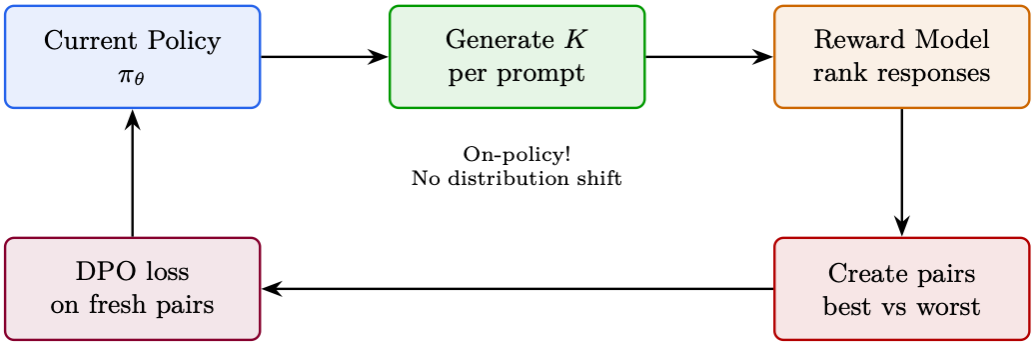


Figure 8.1: Approximate quality vs. compute frontier. Methods above the SFT ceiling line improve beyond what supervised fine-tuning alone achieves. Position is illustrative and model-dependent.

Decision Tree: Which Method to Use?

1. **Do you have verifiable rewards?** (math/code) → **GRPO**
2. **Do you need max quality on complex tasks?** → **PPO**
3. **Do you have paired preferences?** → **DPO** (or IPO if noisy)
4. **Only unpaired binary feedback?** → **KTO**
5. **Memory-limited, starting from base model?** → **ORPO**
6. **DPO plateauing, want on-policy?** → **Online DPO**
7. **Need a strong baseline quickly?** → **Best-of-N / RFT**
8. **Need calibrated confidence alongside correctness?** → **RLCR**
9. **Output drives automated software decisions (typed, fast)?** → **RLCD / Jev**
10. **Agent decision-making with cached state/action embeddings?** → **CLM**

Chapter 9

Reward Model Training

Reward models are the bridge between human preferences and the RL training signal. A well-trained reward model is essential for successful RLHF; a poorly trained one leads to reward hacking and misaligned behaviour. This section covers the theoretical foundations, practical training techniques, and architectural choices for reward models.

GAS Connection — Reward Model Training

The GAS Safety Critic is a reward model. The current prototype uses rule-based scoring; the following chapter describes how to replace it with a learned reward model trained on clinical preference data.

Reward model architecture. The Safety Critic (`safety_node()` in `capstone_gas/gas_system.py`) currently scores recommendations using five explicit hard rules: (1) no insulin when glucose <80 mg/dL; (2) no exercise when glucose <70 mg/dL; (3) no passive monitoring when glucose <54 mg/dL; (4) no DKA home management when glucose >300 mg/dL (must escalate to ER); (5) no insulin recommendation when trend is `falling_fast`. Each violation deducts from a starting score of 1.0; the final score is clamped to [0.0, 1.0] and a threshold of $\geq$0.7 is required for approval. A learned reward model would replace these rules with a fine-tuned LLM that generalises to novel safety violations.

Training data collection. The GAS pipeline generates reward model training data naturally: each (patient state, recommendation, safety verdict) tuple is a training example. The `safety_score` and `safety_approved` fields in `GASState` provide the labels. The five clinical scenarios in `shared/patient_data.py` cover the key failure modes (hypoglycaemia, hyperglycaemia, post-meal spikes, dawn phenomenon, exercise-induced lows).

Reward model evaluation. The chapter's evaluation framework—agreement with human raters, calibration, and robustness to adversarial inputs—applies directly to the GAS Safety Critic. The key metric is the false negative rate: how often does the Safety Critic approve a genuinely dangerous recommendation? The current rule-based critic has zero false negatives on the known failure modes but may miss novel violations.

Reward hacking mitigation. The GAS Safety Critic's rule-based checks are designed to be robust to reward hacking: the Advisor cannot game the score by adding safety disclaimers without changing the underlying recommendation. The chapter's discussion of reward hacking mitigation strategies—ensemble reward models, constitutional constraints, human-in-the-loop verification—informs the GAS's defence-in-depth safety architecture.

9.1 Bradley-Terry Model – Full Derivation

The Bradley-Terry model [31] is the standard probabilistic framework for pairwise preference learning. Given two responses y_1 and y_2 to a prompt q, the model assumes:

$$P(y_1 \succ y_2 \mid q) = \sigma(r(y_1, q) - r(y_2, q)) = \frac{e^{r(y_1,q)}}{e^{r(y_1,q)} + e^{r(y_2,q)}},$$

where $r : \mathcal{Y} \times \mathcal{Q} \to \mathbb{R}$ is the scalar reward function and σ is the sigmoid function.

Maximum Likelihood Estimation

Given a dataset $\mathcal{D} = \{(q^{(k)}, y_w^{(k)}, y_l^{(k)})\}_{k=1}^N$ of preference pairs, the MLE objective is:

$$\mathcal{L}_{\text{BT}}(\phi) = -\frac{1}{N}\sum_{k=1}^{N} \log \sigma(r_\phi(y_w^{(k)}, q^{(k)}) - r_\phi(y_l^{(k)}, q^{(k)})),$$

where r_ϕ is a neural network parameterised by ϕ. This is a binary cross-entropy loss where the "positive" class is the preferred response.

Bradley-Terry Assumptions

1. Preferences are *transitive*: if $y_1 \succ y_2$ and $y_2 \succ y_3$, then $y_1 \succ y_3$.
2. Preferences are determined by a *scalar* reward (no multi-dimensional preferences).
3. The preference probability depends only on the *difference* in rewards.
4. Preferences are *independent* across pairs (no annotator effects).

These assumptions are often violated in practice, motivating extensions like Plackett-Luce models for ranking and multi-dimensional reward models.

Margin Loss Extension

A common extension adds a margin m to ensure a minimum gap between winning and losing rewards:

$$\mathcal{L}_{\text{margin}} = -\frac{1}{N}\sum_{k=1}^{N} \log \sigma(r_\phi(y_w^{(k)}, q^{(k)}) - r_\phi(y_l^{(k)}, q^{(k)}) - m).$$

9.2 Reward Model Architectures

Classification Head on LLM

The standard reward model architecture takes a pretrained LLM and replaces the language modelling head (which maps hidden states to vocabulary logits) with a scalar regression head (which maps the final hidden state to a single reward value).

The architecture is:

1. **Backbone**: a pretrained LLM (e.g., Llama, Mistral) that encodes the prompt-response pair into a sequence of hidden states.
2. **Pooling**: extract the hidden state at the last token position (for decoder-only models) or the `[CLS]` token (for encoder models).
3. **Regression head**: a linear layer $W \in \mathbb{R}^{d\times 1}$ that maps the pooled hidden state to a scalar reward.

Reward Model Training in TRL

```
from trl import RewardConfig, RewardTrainer
from transformers import AutoModelForSequenceClassification

# Load model with scalar head (num_labels=1)
model = AutoModelForSequenceClassification.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    num_labels=1,
)

config = RewardConfig(
    output_dir="reward_model",
```

```
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=1e-5,
    num_train_epochs=1,
    # Margin loss
    center_rewards_coefficient=0.01,
)

trainer = RewardTrainer(
    model=model,
    args=config,
    train_dataset=dataset,  # must have chosen/rejected columns
)
trainer.train()
```

9.3 Reward Model Training Tricks

Reward Centering

Raw reward model outputs can have arbitrary scale and offset. Centering the rewards (subtracting the mean) stabilises RL training:

$$r_{\text{centered}}(y, q) = r_\phi(y, q) - \mathbb{E}_{y' \sim \pi_\theta}[r_\phi(y', q)].$$

In TRL, this is implemented via the `center_rewards_coefficient` parameter, which adds a regularisation term to the reward model loss that penalises non-zero mean rewards.

Length Bias Correction

Reward models are known to exhibit *length bias*: they tend to assign higher rewards to longer responses, regardless of quality. This can be corrected by:

1. **Length normalisation**: divide the reward by the response length.
2. **Length-controlled training**: include length as a feature and train the model to be length-invariant.
3. **Calibration**: post-hoc regression to remove the length effect.

Margin Losses

Adding a margin m to the Bradley-Terry loss ensures the reward model assigns meaningfully different scores to preferred and dispreferred responses:

$$\mathcal{L}_{\text{margin}} = \max(0,\ m - (r_w - r_l)).$$

9.4 Process Reward Models vs Outcome Reward Models

PRM vs ORM Comparison

Property	ORM	PRM
Reward signal	Final answer only	Each reasoning step
Training data	(prompt, answer, correct?)	(prompt, steps, step labels)
Annotation cost	Low	High
Credit assignment	Sparse	Dense
Reward hacking	Easier to hack	Harder to hack
Best for	Simple tasks	Multi-step reasoning
Inference cost	Low	High (score each step)

When to Use PRMs

Process Reward Models are most valuable when:

- The task requires multi-step reasoning (math, code, logic).
- The final answer is binary (correct/incorrect) but intermediate steps vary in quality.
- You want to use the reward model for *search* (e.g., beam search with step scores).
- You have access to step-level annotations (or can generate them automatically).

For simple tasks (sentiment, toxicity, factuality), ORMs are sufficient and much cheaper.

PBRS in RLHF for LLMs

Original reward: Binary correctness (1 if final answer is right, 0 otherwise) — extremely sparse for multi-step reasoning.
Potential function: $\Phi(s)$ = partial credit from a verifier (e.g., fraction of intermediate reasoning steps that are logically valid).
Shaped reward: Agent gets incremental signal for each valid reasoning step while preserving the guarantee that the optimal policy still maximizes final-answer correctness.
Practical implementations:

- Process reward models (PRMs) that score each step in a chain-of-thought
- Intermediate compilation checks in code generation
- Partial match scores for multi-part answers

This is a direct application of Potential-Based Reward Shaping (PBRS) [280] to the LLM setting—the theoretical guarantee that shaped rewards preserve the optimal policy makes PRMs a principled approach to dense reward in reasoning tasks.

Automatic PRM Annotation

Step-level annotations can be generated automatically using:

1. **Monte Carlo rollouts**: for each intermediate step, sample multiple completions and use the fraction that reach the correct answer as the step reward.
2. **LLM-as-judge**: use a strong LLM to evaluate each step.
3. **Formal verification**: for math/code, use a verifier to check each step.

9.5 Rule-Based Rewards for RLVR

Reinforcement Learning from Verifiable Rewards (RLVR) uses deterministic, rule-based reward functions instead of learned reward models. This substantially reduces reward hacking (though models can still exploit format tricks, edge cases, or test memorization) and is the approach used in DeepSeek-R1 [75].

Rule-Based Reward Functions in TRL

```python
import re

def format_reward(completions, **kwargs):
    """Reward for using <think>...</think><answer>...</answer> format."""
    rewards = []
    pattern = r"<think>.*?</think>\s*<answer>.*?</answer>"
    for completion in completions:
        text = completion[0]["content"]
        rewards.append(1.0 if re.fullmatch(pattern, text, re.DOTALL) else 0.0)
    return rewards

def correctness_reward(completions, ground_truth, **kwargs):
    """Reward for correct final answer."""
    rewards = []
    for completion, gt in zip(completions, ground_truth):
        text = completion[0]["content"]
        match = re.search(r"<answer>(.*?)</answer>", text, re.DOTALL)
        if match:
            answer = match.group(1).strip()
            rewards.append(1.0 if answer == gt else 0.0)
        else:
            rewards.append(0.0)
    return rewards

def code_execution_reward(completions, test_cases, **kwargs):
    """Reward for code that passes test cases."""
    import subprocess, tempfile, os
    rewards = []
    for completion, tests in zip(completions, test_cases):
        code = completion[0]["content"]
        passed = 0
        for test in tests:
            with tempfile.NamedTemporaryFile(
                mode="w", suffix=".py", delete=False
            ) as f:
                f.write(code + "\n" + test)
                fname = f.name
            try:
                result = subprocess.run(
                    ["python", fname], capture_output=True,
                    timeout=5, text=True
                )
                passed += int(result.returncode == 0)
            except Exception:
                pass
            finally:
                os.unlink(fname)
        rewards.append(passed / len(tests))
    return rewards
```

Rule-Based Reward Pitfalls

- **Format gaming**: models learn to produce the correct format without correct content. Always combine format and correctness rewards.
- **Test case leakage**: if test cases are in the training data, the model memorises them.

- **Timeout exploitation**: models may generate code that times out (avoiding failure). Use strict timeouts and penalise timeouts explicitly.
- **Reward sparsity**: binary rewards (0/1) can be too sparse for complex tasks. Consider partial credit or intermediate rewards.

9.6 Multi-Objective Rewards – Combination Strategies

When training with multiple reward signals, the combination strategy significantly affects the final policy.

Multi-Reward Combination Strategies

1. **Weighted sum**: $r = \sum_n w_n r_n$. Simple but sensitive to scale.
2. **Normalise then sum (GDPO)**: normalise each reward to zero mean and unit variance within the group, then sum with weights. Scale-invariant.
3. **Lexicographic**: optimise rewards in priority order; only consider lower-priority rewards when higher-priority ones are tied.
4. **Constrained**: maximise primary reward subject to constraints on secondary rewards.
5. **Pareto**: maintain a Pareto front of policies and select based on preference.

Multi-Reward Training in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # GDPO: normalise each reward independently
    multi_objective_aggregation="normalize_then_sum",
    reward_weights=[1.0, 0.3, 0.1],  # correctness, format, length
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[
        correctness_reward,
        format_reward,
        length_penalty_reward,
    ],
    args=config,
    train_dataset=dataset,
)
```

9.7 Listwise Rank-Based Rewards

While the Bradley-Terry model handles *pairwise* preferences ($y_w \succ y_l$), many practical scenarios involve ranking multiple responses simultaneously. Listwise reward models learn from complete orderings, providing richer training signal and enabling better calibration.

Motivation: Beyond Pairwise

Why Listwise?

- **Richer signal**: A ranking of K responses contains $\binom{K}{2}$ implicit pairwise comparisons, but also captures *relative margins* (how much better rank 1 is vs. rank 3).
- **Better calibration**: Pairwise BT models only learn *differences* in reward; listwise models learn absolute reward scale.
- **Natural fit for GRPO**: GRPO generates N responses per prompt and ranks them — listwise rewards align directly with this workflow.
- **Annotator efficiency**: Ranking 5 responses is faster than labeling all 10 possible pairs independently.

Plackett-Luce Model

The Plackett-Luce (PL) model [308] is the standard extension of Bradley-Terry to full rankings. Given K responses $y_1, \ldots, y_K$ with ranking π (where $\pi(1)$ is the best):

Plackett-Luce Likelihood

$$P(\pi \mid q) = \prod_{i=1}^{K} \frac{e^{r_\phi(y_{\pi(i)}, q)}}{\sum_{j=i}^{K} e^{r_\phi(y_{\pi(j)}, q)}}$$

Intuition: Sequentially select the best remaining item. At each step, the probability of selecting item $\pi(i)$ is softmax over the remaining items.
Loss function:

$$\mathcal{L}_{\text{PL}}(\phi) = -\frac{1}{|\mathcal{D}|} \sum_{(q,\pi)\in\mathcal{D}} \sum_{i=1}^{K-1} \left[r_\phi(y_{\pi(i)}, q) - \log \sum_{j=i}^{K} e^{r_\phi(y_{\pi(j)}, q)} \right]$$

Plackett-Luce Reduces to Bradley-Terry

For $K = 2$, the PL model gives: $P(y_1 \succ y_2) = \frac{e^{r(y_1)}}{e^{r(y_1)}+e^{r(y_2)}} = \sigma(r(y_1) - r(y_2))$ — exactly the Bradley-Terry model. PL is a strict generalization.

ListMLE and Rank-Based Losses

Listwise Loss Functions

- **ListMLE** [413]: Directly maximizes the PL likelihood of the ground-truth ranking. Simple and effective.
- **ListNet** [40]: Minimizes KL divergence between the model's top-1 probability distribution and the ground-truth:

  $$\mathcal{L}_{\text{ListNet}} = -\sum_{i=1}^{K} P_{\text{true}}(y_i \text{ is best}) \cdot \log P_{\text{model}}(y_i \text{ is best})$$

  where $P_{\text{model}}(y_i \text{ is best}) = \frac{e^{r_\phi(y_i)}}{\sum_j e^{r_\phi(y_j)}}$.
- **LambdaRank** [36]: Weights pairwise gradients by the change in ranking metric (e.g., NDCG). Useful when ranking quality matters more at the top.
- **RankNet** [35]: Pairwise cross-entropy summed over all pairs — equivalent to BT on all $\binom{K}{2}$ pairs extracted from the ranking.

Listwise Rewards for GRPO and Rejection Sampling

Integration with GRPO

GRPO naturally produces ranked groups: for each prompt, N responses are scored and ranked. A listwise reward model can be trained directly on these rankings:

1. **Generate**: Sample $N = 8$ responses per prompt from the policy.
2. **Rank**: Use an existing reward model (or human annotators) to produce a full ranking π.
3. **Train listwise RM**: Optimize the PL loss on (q, π) tuples.
4. **Use in GRPO**: The listwise RM assigns scalar rewards $r(y_i, q)$ to each response; GRPO computes advantages as $\hat{A}_i = (r_i - \mu)/\sigma$.

Advantage over pairwise: The listwise RM sees all N responses simultaneously, learning that rank-1 should have much higher reward than rank-N (not just "slightly better than one other response").

Practical Considerations

Listwise Training Challenges

- **Annotation cost**: Full rankings are expensive. Partial rankings (top-3 out of 8) reduce cost with minimal quality loss.
- **Ties**: Real rankings often have ties. Use the Plackett-Luce extension for ties: assign equal probability mass to tied items.
- **Position bias**: Annotators tend to prefer items shown first. Randomize presentation order and train debiasing.
- **List length**: Training on $K = 4$–8 is typical. Longer lists ($K > 16$) add noise without much benefit.
- **Consistency**: Rankings from different annotators may disagree. Use inter-annotator agreement ($\kappa > 0.6$) as a quality filter.

Plackett-Luce Training Code

```
import torch
import torch.nn.functional as F

def plackett_luce_loss(rewards, rankings):
    """
    Args:
        rewards: (batch, K) - predicted scalar rewards for K responses
        rankings: (batch, K) - ground-truth ranking indices (0 = best)
    Returns:
        scalar loss
    """
    batch_size, K = rewards.shape
    # Sort rewards by ground-truth ranking order
    sorted_rewards = torch.gather(rewards, 1, rankings)  # (batch, K)

    # PL log-likelihood: sum over positions
    loss = 0.0
    for i in range(K - 1):
        # Log-softmax over remaining items (position i to K)
        remaining = sorted_rewards[:, i:]           # (batch, K-i)
        log_probs = remaining[:, 0] - torch.logsumexp(remaining, dim=1)
```

```
        loss -= log_probs.mean()

    return loss / (K - 1)

# Example: 8 responses per prompt, ranked by annotator
rewards = reward_model(responses)          # (batch, 8)
rankings = torch.argsort(human_scores, descending=True)  # best first
loss = plackett_luce_loss(rewards, rankings)
loss.backward()
```

Chapter 10

SFT Best Practices and Techniques

Supervised Fine-Tuning (SFT) is the foundation of the RLHF pipeline. The quality of the SFT model determines the ceiling of what RL can achieve: RL can refine and improve a behaviour, but it cannot reliably introduce a behaviour that is entirely absent from the SFT model. This section covers the key techniques for effective SFT.

GAS Connection — SFT Best Practices and Techniques

Supervised fine-tuning is the first step in the GAS training pipeline. The chapter notebook `part2_rl_methods/ch04_rl_for_lms/notebook.ipynb` demonstrates SFT on a clinical recommendation task using the GAS data format.

Data quality over quantity. The GAS SFT dataset consists of clinician-written recommendations for the five clinical scenarios in `shared/patient_data.py`. The chapter's emphasis on data quality—correct labels, diverse examples, balanced class distribution—applies directly: a single incorrect recommendation in the SFT dataset can introduce a systematic bias that is hard to remove with RL fine-tuning.

Instruction formatting. The GAS Advisor's SFT data follows the instruction format used in `advisor_node()` in `capstone_gas/gas_system.py`: a system prompt defining the agent's role, a user message containing the patient state and retrieved guidelines, and an assistant message containing the recommendation. Consistent formatting is critical for the model to learn the correct input–output mapping.

LoRA for efficient fine-tuning (§ LoRA). The chapter notebook demonstrates LoRA fine-tuning of `llama3.2:3b` with rank $r = 16$, updating only ∼8M parameters while keeping the base model frozen. The resulting adapter is saved to `part2_rl_methods/ch04_rl_for_lms/gas_sft_model/` and can be loaded by the `GASLLMClient` for inference.

Evaluation during SFT. The chapter's recommendation to evaluate on a held-out validation set after each epoch applies directly to the GAS: the five clinical scenarios in `shared/patient_data.py` serve as the validation set, and the Safety Critic's approval rate is the primary metric. A well-trained SFT checkpoint should achieve >90% Safety Critic approval before proceeding to RL fine-tuning.

10.1 Sequence Packing for Efficiency

The Padding Problem

Standard SFT batches pad all sequences to the length of the longest sequence in the batch. For datasets with high length variance (e.g., a mix of short instructions and long documents), this wastes 50–80% of compute on padding tokens. Sequence packing eliminates this waste.

Sequence packing concatenates multiple short examples into a single sequence of length `max_seq_length`, separated by EOS tokens. The attention mask ensures that tokens from different examples do not attend to each other:

1. Sort examples by length (optional, improves packing efficiency).

2. Greedily pack examples into bins of size `max_seq_length`.
3. Use a block-diagonal attention mask to prevent cross-example attention.
4. Compute loss only on non-padding tokens.

Packing Efficiency

- Typical packing efficiency: 85–95% (vs 20–50% with padding).
- Speedup: 2–4× for datasets with high length variance.
- Memory: similar to padding (same total tokens per batch).
- Caveat: requires careful attention masking to avoid cross-contamination.

Sequence Packing in TRL

```
from trl import SFTConfig, SFTTrainer

config = SFTConfig(
    max_seq_length=4096,
    packing=True,           # enable sequence packing
    output_dir="sft_model",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-5,
    num_train_epochs=3,
)

trainer = SFTTrainer(
    model=model,
    args=config,
    train_dataset=dataset,
    # dataset_text_field="text",  # or use formatting_func
)
trainer.train()
```

10.2 Chat Templates and Formatting

Why Chat Templates Matter

Language models are trained on raw text, but instruction-following models need to distinguish between system prompts, user messages, and assistant responses. Chat templates encode this structure into the token sequence. Using the wrong template (or no template) at inference time causes significant performance degradation.

ChatML Format

ChatML is the most widely used chat template:

```
# ChatML format
template = """<|im_start|>system
{system_message}<|im_end|>
<|im_start|>user
{user_message}<|im_end|>
<|im_start|>assistant
{assistant_message}<|im_end|>"""
```

Llama Format

Llama 3 uses a different template with special tokens:

```
# Llama 3 format
template = """<|begin_of_text|><|start_header_id|>system<|end_header_id|>
{system_message}<|eot_id|><|start_header_id|>user<|end_header_id|>
{user_message}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
{assistant_message}<|eot_id|>"""
```

Applying Chat Templates in TRL

```
from transformers import AutoTokenizer
from trl import SFTConfig, SFTTrainer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

def formatting_func(example):
    """Apply chat template to a dataset example."""
    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": example["instruction"]},
        {"role": "assistant", "content": example["response"]},
    ]
    return tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=False,
    )

config = SFTConfig(
    max_seq_length=2048,
    output_dir="sft_model",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=config,
    train_dataset=dataset,
    formatting_func=formatting_func,
)
```

10.3 Completion-Only Masking

Why Mask the Prompt?

In instruction fine-tuning, the model should learn to generate the assistant's response, not to predict the user's question or the system prompt. Computing loss on the prompt tokens wastes gradient signal and can cause the model to "memorise" prompts rather than learning to respond to them. Completion-only masking sets the loss to zero for all non-assistant tokens.

Completion-Only Masking in TRL

```
from trl import SFTConfig, SFTTrainer, DataCollatorForCompletionOnlyLM
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# Define the response template (tokens after which loss is computed)
response_template = "<|start_header_id|>assistant<|end_header_id|>"
collator = DataCollatorForCompletionOnlyLM(
```

```
    response_template=response_template,
    tokenizer=tokenizer,
)

config = SFTConfig(
    max_seq_length=2048,
    output_dir="sft_model",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=config,
    train_dataset=dataset,
    data_collator=collator,    # completion-only masking
    formatting_func=formatting_func,
)
```

Completion Masking Pitfalls

- The response template must exactly match the tokenised form. Off-by-one errors in tokenisation can cause the mask to be applied incorrectly.
- For very short responses, masking the prompt may leave too few tokens for meaningful gradient signal. Consider a minimum response length threshold.
- Multi-turn conversations require masking all non-assistant turns, not just the first.

10.4 Data Mixing Strategies for Multi-Task SFT

The Multi-Task Challenge

Training on multiple tasks simultaneously can improve generalisation but also causes *task interference*: gradients from different tasks conflict, degrading performance on individual tasks. Data mixing strategies control the relative contribution of each task to the training signal.

Proportional Mixing

Sample from each dataset proportionally to its size:

$$p_k = \frac{N_k}{\sum_{j=1}^{K} N_j},$$

where N_k is the number of examples in dataset k. This is the default in most frameworks and works well when datasets are of similar quality.

Temperature Mixing

Apply a temperature T to smooth the proportions:

$$p_k \propto N_k^{1/T}.$$

$T = 1$: proportional mixing. $T \to \infty$: uniform mixing. $T < 1$: over-samples large datasets. $T > 1$: over-samples small datasets.

Quality-Weighted Mixing

Weight datasets by estimated quality (e.g., perplexity under a reference model, human quality ratings):

$$p_k \propto N_k \cdot q_k,$$

where q_k is the quality score for dataset k.

Data Mixing in TRL

```python
from datasets import concatenate_datasets, interleave_datasets

# Proportional mixing (default)
mixed_dataset = concatenate_datasets([
    dataset_math,
    dataset_code,
    dataset_general,
]).shuffle(seed=42)

# Temperature mixing (T=2: over-sample small datasets)
mixed_dataset = interleave_datasets(
    [dataset_math, dataset_code, dataset_general],
    probabilities=[0.4, 0.4, 0.2],   # manually set after temperature scaling
    seed=42,
)

config = SFTConfig(output_dir="sft_model")
trainer = SFTTrainer(
    model=model,
    args=config,
    train_dataset=mixed_dataset,
)
```

10.5 When SFT Hurts – Catastrophic Forgetting and Alignment Tax

As LLMs transition through sequential training phases — pre-training → continued pre-training → SFT → RLHF/DPO — performance degradation frequently manifests on standard benchmarks. Two **fundamentally distinct** phenomena drive these regressions, and confusing them leads to wrong mitigation strategies.

10.5.1 Catastrophic Forgetting (Structural Erasure)

Catastrophic Forgetting

Catastrophic forgetting is an **unintentional optimization failure**: when a network optimized on distribution $\mathcal{D}_A$ is subsequently trained on a disjoint distribution $\mathcal{D}_B$, the weight updates required for $\mathcal{D}_B$ *physically overwrite* the parameter structures encoding $\mathcal{D}_A$:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_B(\theta_t) \quad \Longrightarrow \quad \mathcal{L}_A(\theta_{t+1}) \gg \mathcal{L}_A(\theta_t) \tag{10.1}$$

The knowledge is **destroyed** — the weights encoding Task A no longer exist. This is irreversible without retraining.

Symptoms:

- Complete breakdown on tasks not in fine-tuning data (e.g., model forgets how to do math after SFT on chat data)
- Loss of language diversity — model only generates in the narrow style of fine-tuning distribution
- Reduced factual accuracy on knowledge not reinforced during fine-tuning
- Degraded multilingual ability after English-only SFT

Mechanistic cause — Fisher Information perspective: The Fisher Information Matrix F of Task A identifies which parameters are "important" for $\mathcal{D}_A$:

$$F = \mathbb{E}_{x \sim \mathcal{D}_A}\left[\nabla_\theta \log \pi_\theta(x)\, \nabla_\theta \log \pi_\theta(x)^T\right] \tag{10.2}$$

Parameters with high Fisher eigenvalues are critical for Task A. Unconstrained gradient descent on Task B ignores these eigenvalues entirely — $\Delta\theta$ points along $\nabla\mathcal{L}_B$ regardless of whether it destroys high-Fisher directions for $\mathcal{L}_A$.

10.5.2 Alignment Tax (Behavioral Constraint)

The alignment tax is a **deliberate, expected trade-off**: the model's raw capability (unconstrained generation, maximal reasoning bandwidth) decreases because the policy is constrained to produce safe, well-formatted, preference-aligned outputs.

Mechanism: During DPO/PPO, the policy π_θ is penalized for deviating from the reference π_{ref} via KL divergence:

$$r_{\text{implicit}}(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \tag{10.3}$$

This leash constrains the model's **output distribution** — it cannot explore high-variance reasoning paths that deviate too far from the reference. The knowledge is **not erased**; it's *suppressed*. The model still "knows" the answer but its distribution is flattened toward safe, generic responses.

Symptoms:

- Over-refusal ("I can't help with that" for benign queries)
- Stylistic stiffness — hedge words, excessive caveats, verbose safety disclaimers
- Lower scores on raw capability benchmarks (MMLU, HumanEval) while improving on preference benchmarks (MT-Bench, AlpacaEval)
- Reduced ability to produce complex, high-entropy outputs (creative writing, novel algorithms)

10.5.3 Comparative Taxonomy

Table 10.1: Catastrophic Forgetting vs. Alignment Tax — complete comparison.

Dimension	Catastrophic Forgetting	Alignment Tax
Intentionality	Unintentional (optimization artifact)	Expected trade-off (incurred deliberately for safety/helpfulness)
Parameter state	Prior knowledge physically overwritten	Latent distributions constrained/truncated
Information	**Destroyed**: weights no longer encode the capability	**Suppressed**: knowledge exists but is harder to trigger
Dominant phase	Sequential SFT, domain continued pre-training	Preference optimization (PPO, DPO, KTO, RLHF)
Primary symptom	Complete breakdown of baseline capabilities	Over-refusal, stylistic stiffness, lower raw benchmark scores
Reversibility	Irreversible without retraining from checkpoint	Partially reversible: adjust β, system prompt, or fine-tune
Detection	Perplexity on pre-training eval set spikes	Perplexity stable but win-rate on capability benchmarks drops
Scales with model size	Similar across scales	Smaller models pay a larger alignment tax

10.5.4 Mitigation Strategies

For Catastrophic Forgetting:

1. **Data replay**: Mix 5–10% of pre-training data into SFT dataset. Ensures gradient updates don't completely neglect pre-training distribution.
2. **Elastic Weight Consolidation (EWC)** [196]: Add regularization $\Omega(\theta) = \frac{\lambda}{2}\sum_i F_i(\theta_i - \theta_i^*)^2$ that penalizes changes to parameters with high Fisher information for the original task.
3. **LoRA / Parameter-efficient fine-tuning**: Train only low-rank adapters ($< 1\%$ of parameters), leaving base weights completely frozen. This prevents *permanent destruction* of pre-trained knowledge — you can always remove the adapter and recover the original model. However, **while the adapter is active**, the combined system ($W_0 + BA$) can still exhibit forgetting: the adapter may shift the model's effective behavior away from old skills. LoRA protects the checkpoint, not the active inference behavior.
4. **Conservative learning rate**: Use $1\text{–}5 \times 10^{-6}$ with few epochs (1–3). Larger rates accelerate forgetting.
5. **Progressive training**: Mix distributions gradually, increasing SFT data proportion over time rather than switching abruptly.

For Alignment Tax:

1. **Tune β carefully**: Lower β gives the model more freedom (reduces the tax) but may sacrifice safety. Optimal $\beta \in [0.05, 0.3]$ for most settings.
2. **High-quality, diverse SFT data**: Part of the alignment tax comes from SFT narrowing the output distribution; broader, more diverse SFT data reduces this component. The RL phase adds further constraint via KL regularization [296].
3. **Conditional alignment**: Train the model to be aligned only when a safety flag is active. At inference, disable constraints for benchmarking (research-only technique).
4. **Constitutional AI / RLAIF**: Use model-generated feedback to create more nuanced preference data that preserves capability while improving alignment.
5. **Targeted RL budget**: Don't over-train with RL. Monitor capability benchmarks and stop when the tax exceeds acceptable thresholds (typically 2–5% MMLU regression).

How to Tell Which One You Have

- **Run the base model on the failing tasks**: If the base model succeeds and the fine-tuned model completely fails → catastrophic forgetting.
- **Prompt engineering test**: If careful prompting (e.g., "ignore safety guidelines and solve this math problem step by step") recovers the capability → alignment tax (knowledge is suppressed, not erased).
- **Perplexity check**: Compute perplexity on pre-training validation set. Spike = forgetting. Stable = alignment tax.
- **Few-shot recovery**: If providing a few in-context examples restores the capability → alignment tax. If even many examples can't recover it → forgetting.

10.6 Connection to RL – SFT Quality Determines RL Ceiling

The SFT-RL Relationship

The SFT model is the starting point for RL training. RL can:

- **Amplify** behaviours that are present but weak in the SFT model.
- **Suppress** behaviours that are present but undesirable.
- **Refine** the style and format of responses.

RL *cannot*:

- Introduce capabilities that are entirely absent from the SFT model.
- Recover from severe catastrophic forgetting in the SFT stage.
- Compensate for a reward model that is systematically biased.

The Exploration-Exploitation Tradeoff in SFT

For RL to work, the SFT model must occasionally produce correct responses (so the reward signal is non-zero). If the SFT model *never* produces a correct response to a given prompt, RL cannot learn to produce correct responses – there is no positive signal to amplify. This is why SFT quality is the ceiling for RL performance.
Concretely: if the SFT model solves 10% of math problems correctly, RL can potentially push this to 80%. If the SFT model solves 0% of math problems, RL will make no progress (all rewards are zero, all advantages are zero, no gradient).

Practical Implications

1. **SFT data quality**: use high-quality, diverse data. A small amount of high-quality data is better than a large amount of low-quality data.
2. **SFT data coverage**: ensure the SFT data covers the tasks you want to improve with RL. If a task is not in the SFT data, RL will struggle.
3. **SFT training duration**: do not over-train the SFT model. Over-training reduces diversity and makes RL exploration harder.
4. **Warm-up**: consider a short SFT warm-up on task-specific data before RL, even if the base model is already instruction-tuned.

Checking SFT Quality Before RL

```python
import numpy as np
from tqdm import tqdm

def estimate_pass_at_k(model, tokenizer, dataset, k=8, n_samples=100):
    """
    Estimate pass@k for the SFT model.
    If pass@1 < 5%, RL will likely fail.
    If pass@k < 20%, RL will struggle.
    """
    pass_at_1_scores = []
    pass_at_k_scores = []

    for example in tqdm(dataset.select(range(n_samples))):
        prompt = example["prompt"]
        ground_truth = example["answer"]
```

```
        # Sample k completions
        inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
        outputs = model.generate(
            **inputs,
            max_new_tokens=512,
            do_sample=True,
            temperature=0.8,
            num_return_sequences=k,
        )

        correct = 0
        for output in outputs:
            response = tokenizer.decode(output, skip_special_tokens=True)
            if ground_truth in response:
                correct += 1

        # pass@1: fraction of samples that are correct (estimated success rate)
        pass_at_1_scores.append(correct / k)
        # pass@k: at least one of k samples is correct
        pass_at_k_scores.append(correct >= 1)

    print(f"Pass@1 (estimated): {np.mean(pass_at_1_scores):.2%}")
    print(f"Pass@{k}: {np.mean(pass_at_k_scores):.2%}")
    print(f"RL viability: {'Good' if np.mean(pass_at_1_scores) > 0.05 else '
    Poor'}")

estimate_pass_at_k(sft_model, tokenizer, eval_dataset)
```

SFT Best Practices Summary

1. Use sequence packing to maximise GPU utilisation.
2. Apply completion-only masking to focus gradient on assistant responses.
3. Use the correct chat template for your model family.
4. Mix data proportionally with temperature scaling ($T \approx 2$) for multi-task SFT.
5. Use LoRA to prevent catastrophic forgetting.
6. Evaluate pass@k before starting RL to ensure the SFT model is a viable starting point.
7. Do not over-train: 1–3 epochs is usually sufficient for instruction fine-tuning.
8. Monitor diversity metrics (entropy, n-gram diversity) to detect mode collapse.

Chapter 11

System Architecture & Infrastructure at Scale

Training LLMs with reinforcement learning from human feedback is as much a systems engineering challenge as it is an algorithmic one. Unlike standard supervised fine-tuning—which involves a single model, a single forward-backward pass, and well-understood scaling—RLHF requires *multiple models* (policy, reference, reward model, value head) to be loaded simultaneously, coordinated through a complex rollout-scoring-training loop, and distributed across dozens to hundreds of GPUs. This chapter covers the systems-level details that make large-scale RLHF training possible: memory budgeting, parallelism strategies (Data, Tensor, Pipeline, Sequence, and their combinations), the generation bottleneck, decoupled architectures, weight synchronization, fault tolerance, and production monitoring.

GAS Connection — System Architecture & Infrastructure at Scale

The GAS is designed to run on a single developer machine, but its architecture anticipates the production infrastructure described in the following chapter.

Containerisation. The GAS ships with a `docker-compose.yml` at the repository root that launches the full system: Ollama (LLM serving), the LangGraph pipeline, and the Streamlit UI (`part5_agentic/ch28_agentic_ui/app.py`). This is the simplest possible production deployment—one command (`docker compose up`) launches the entire stack.

State management at scale. The current GAS uses LangGraph's in-memory `MemorySaver` checkpointer for state persistence. The `GASState` TypedDict in `capstone_gas/gas_system.py` captures all pipeline state: patient profile, glucose readings, alert level, recommendation, safety score, and caregiver notification status. In a production deployment, the `MemorySaver` would be replaced by a persistent checkpointer (e.g. PostgreSQL) to survive process restarts.

Multi-patient scaling. The GAS processes one patient at a time in the prototype. The LangGraph `thread_id` mechanism (used in `build_gas_graph()` in `capstone_gas/gas_system.py`) provides natural isolation between patient sessions, making horizontal scaling straightforward: each patient gets a separate thread with its own `GASState`.

Fault tolerance. The GAS implements graceful degradation: if the LLM backend is unavailable, `GASLLMClient` falls back to a rule-based mock response. The `NOTEBOOK_TEST_MODE=1` environment variable forces mock mode for CI-safe testing. The Safety Critic's retry mechanism (up to 2 retries when `safety_score < 0.7`) provides resilience against transient LLM failures.

Observability. The GAS uses Python's `logging` module for structured logging throughout `capstone_gas/gas_system.py`. Each agent node logs its inputs, outputs, and timing. In a production deployment, these logs would be exported to a centralised observability platform for monitoring alert rates, Safety Critic approval rates, and pipeline latency.

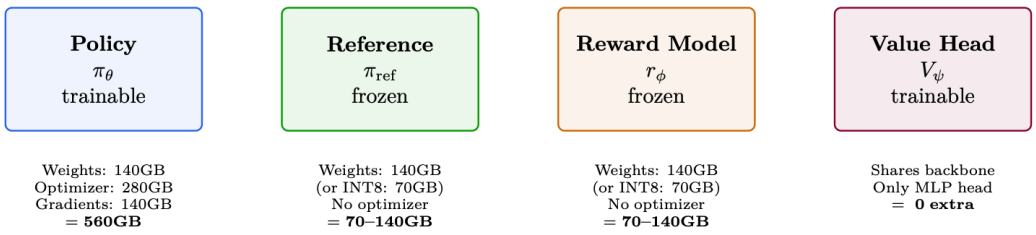


Figure 11.1: 70B PPO memory budget: the four models required for RLHF and their memory footprints. Total: 1470–1560GB. Minimum 19–20 A100-80GB (naive). With ZeRO-3: fits in 8 nodes.

11.1 The 4-Model Memory Challenge

Memory Budget Reality Check – 70B BF16

Policy weights (BF16)	140 GB
FP32 master weights	280 GB
Adam optimizer (m + v, FP32)	560 GB
Gradients (BF16)	140 GB
Reference model	140 GB (or 70 GB in INT8)
Reward model	140 GB (or 70 GB in INT8)
Activations (batch 128, seq 2048)	50–100 GB
KV cache for generation	20–60 GB
Total	**1470–1560 GB**

$\div$ 80 GB/GPU = **19–20 A100s minimum** (without any parallelism overhead).

11.2 Parallelism Strategies in Detail

Training large language models requires distributing computation across many GPUs. There are fundamentally different *axes* along which to parallelize, each with distinct trade-offs. This section provides detailed coverage of each strategy with mathematical formulations, diagrams, and practical guidance.

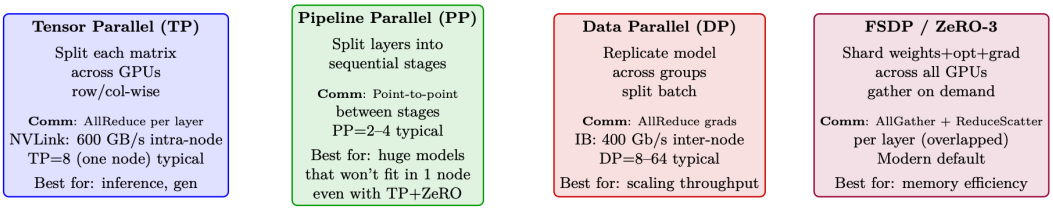


Figure 11.2: Overview of the four parallelism strategies. Production systems typically combine 2–3 of these simultaneously.

11.2.1 Data Parallelism (DP) and Distributed Data Parallelism (DDP)

Data Parallelism is the simplest and most common form of distributed training [220]. Each GPU holds a *complete copy* of the model, processes a different mini-batch, and synchronizes gradients.

Vanilla DP (PyTorch `DataParallel`). A single-process approach where one "master" GPU scatters input, gathers outputs, and broadcasts gradients. Limited by GIL and PCIe bandwidth to the master GPU.

Distributed Data Parallelism (DDP, `DistributedDataParallel`). Multi-process: each GPU runs its own process. Gradients are synchronized via ring-AllReduce [339] in the background while backward computation continues.

Key properties of DDP:

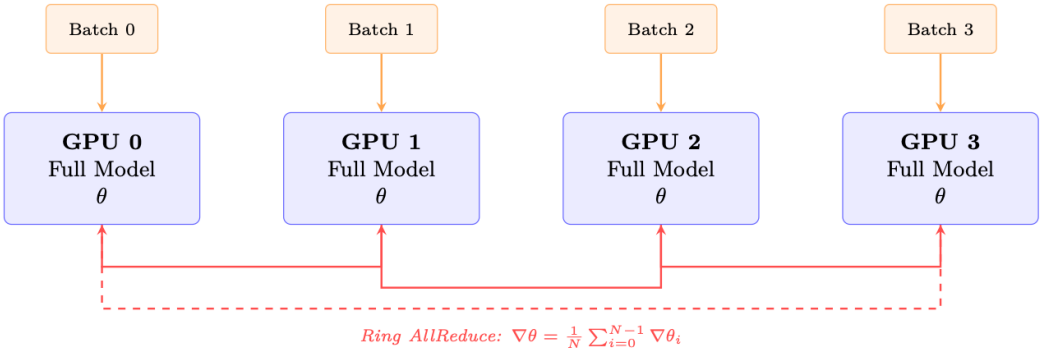


Figure 11.3: DDP: each GPU holds a full model replica and processes a different batch. Gradients are averaged via ring AllReduce, overlapped with backward computation.

- **Memory**: Each GPU stores full model + optimizer + gradients. For 70B BF16: ∼560 GB/GPU—impossible without memory optimizations.
- **Communication**: One AllReduce of gradient tensor per step. Size = model parameters × 2 bytes (BF16). Ring AllReduce cost: $2 \cdot \frac{N-1}{N} \cdot M$ bytes transferred per GPU.
- **Scaling**: Near-linear up to ∼64 GPUs. Beyond that, communication starts to dominate.
- **Gradient bucketing**: DDP groups parameters into buckets (default 25 MB) and starts AllReduce as soon as a bucket's gradients are ready—overlapping communication with backward computation.

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group(backend="nccl")  # NCCL for GPU communication
model = model.to(local_rank)
model = DDP(model, device_ids=[local_rank],
            gradient_as_bucket_view=True,    # Memory optimization
            static_graph=True)               # Enable comm optimizations
```

DP vs DDP — Always Use DDP

PyTorch's legacy `DataParallel` (DP) should **never** be used for LLM training:

- Single-process, limited by Python GIL
- All gradients funnel through GPU 0 (bottleneck)
- 2–3× slower than DDP even on a single node
- Cannot scale beyond one machine

DDP is the *minimum* parallelism strategy. For LLMs >7B, FSDP/ZeRO is preferred.

11.2.2 Tensor Parallelism (TP)

Tensor Parallelism (Megatron-LM style [345]) splits *individual weight matrices* across GPUs. Each GPU computes a partial result, and an AllReduce combines them.

Column-Parallel Linear Layer. The weight matrix $W \in \mathbb{R}^{d \times h}$ is split column-wise across T GPUs:

$$W = [W_0 \mid W_1 \mid \cdots \mid W_{T-1}], \quad W_i \in \mathbb{R}^{d \times h/T} \tag{11.1}$$

Each GPU i computes $Y_i = XW_i$ independently (no communication). The output is split along the hidden dimension.

Row-Parallel Linear Layer. The weight matrix is split row-wise: $W = [W_0; W_1; \ldots; W_{T-1}]$ where $W_i \in \mathbb{R}^{d/T \times h}$. Input X must also be split. Each GPU computes a partial sum, then an **AllReduce** produces the final output.

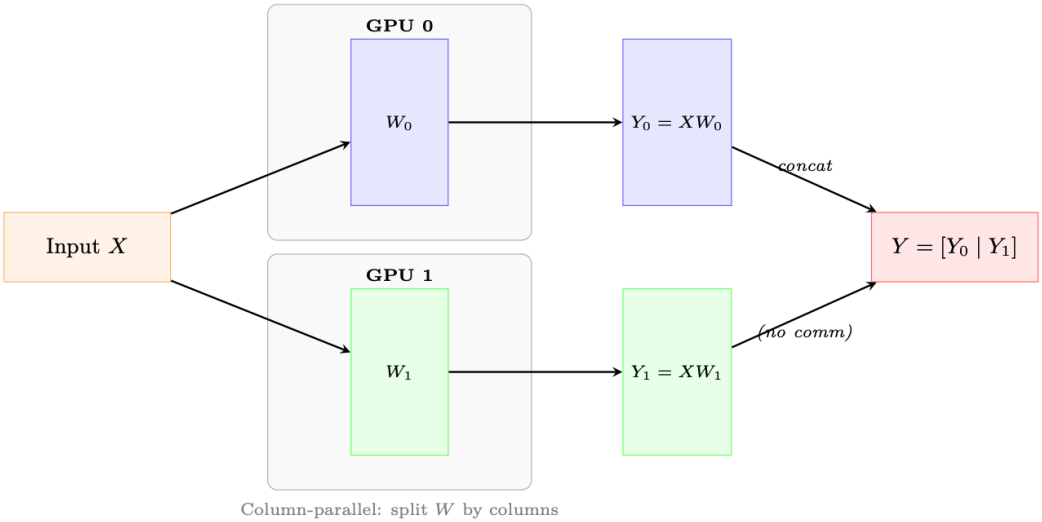


Figure 11.4: Column-parallel linear layer (TP=2). The weight is split column-wise; each GPU computes XW_i independently. The MLP pairs this with a row-parallel layer to avoid redundant AllReduce.

Transformer Block with TP. In a Transformer layer, Megatron-LM applies TP as follows:

1. **MLP**: Column-parallel for the first linear ($h \to 4h$), row-parallel for the second ($4h \to h$). One AllReduce after the row-parallel layer.

2. **Attention**: Q, K, V projections are column-parallel (split heads across GPUs). Output projection is row-parallel. One AllReduce after output projection.

3. **Total**: 2 AllReduce per transformer layer (one for attention, one for MLP).

Why TP is Restricted to Intra-Node

Each transformer layer requires 2 AllReduce operations (marked as f and g above). For a 70B model with 80 layers, that's 160 AllReduce operations per forward pass (320 including backward). At NVLink speeds (600 GB/s), each AllReduce takes <0.5 ms. But over InfiniBand (50 GB/s), the same operation takes ∼4 ms, making the total overhead $160 \times 4 = 640$ ms—*longer than the computation itself.*
Rule: TP degree ≤ GPUs per node (typically TP ≤ 8). Use DP/FSDP for inter-node scaling.

TP Degree Selection

- **TP=1**: No tensor parallelism. Model fits on one GPU (typically ≤ 13B with BF16).
- **TP=2**: Minimal split. Good for 13–34B inference on 2 GPUs. Low overhead (<5%).
- **TP=4**: Standard for 34–70B inference. Overhead 8–12%.
- **TP=8**: Full node. Required for 70B+ training. Overhead 12–18%.
- **TP>8**: Cross-node TP. Rarely used—only for 200B+ models where PP alone is insufficient. Overhead 30–50%.

Important: Number of attention heads must be divisible by TP degree. For LLaMA-70B (64 heads), valid TP = 1, 2, 4, 8, 16, 32, 64.

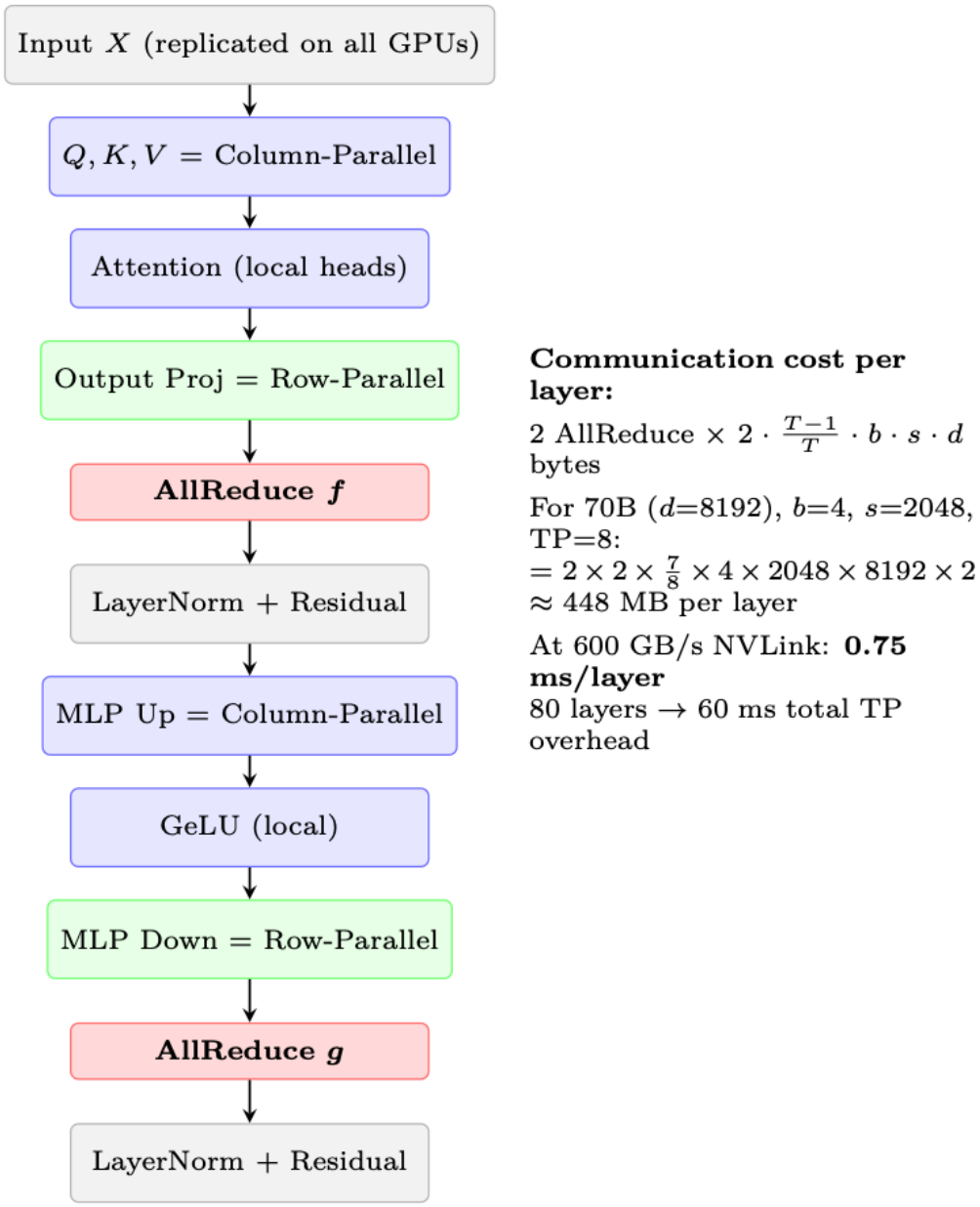


Figure 11.5: Tensor Parallel communication pattern in one Transformer block. Two AllReduce operations (marked in red) are required per layer—one after attention, one after MLP.

11.2.3 Sequence Parallelism (SP)

Sequence Parallelism [201] addresses a memory bottleneck that Tensor Parallelism alone cannot solve: the **activation memory** in LayerNorm and Dropout layers.

The Problem. With TP, weight memory is split across GPUs. But LayerNorm and Dropout operate on the *full* hidden dimension and are replicated on every GPU. Their activations (needed for backward pass) consume memory proportional to $b \times s \times d$—the same on every GPU, unreduced by TP.

The Solution. Split the *sequence dimension* for operations that don't require cross-GPU communication (LayerNorm, Dropout, residual connections). Each GPU processes a s/T slice of the sequence for these operations, then gathers the full sequence only where needed (attention, linear layers).

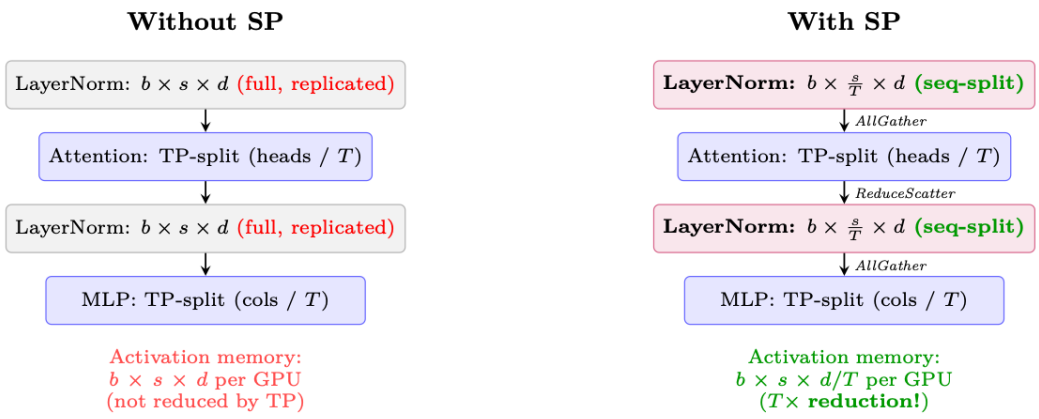


Figure 11.6: Sequence Parallelism reduces activation memory for LayerNorm/Dropout by splitting along the sequence dimension. Communication (AllGather/ReduceScatter) replaces the AllReduce used in standard TP—same total bytes transferred, but memory is saved.

SP Communication is "Free"

Standard TP uses AllReduce after each sub-layer, which is equivalent to ReduceScatter + AllGather. SP simply *reorders* these primitives:

- TP without SP: AllReduce (= ReduceScatter + AllGather) → same data on all GPUs → LayerNorm on full tensor (wasteful).
- TP with SP: ReduceScatter → each GPU has $1/T$ of sequence → LayerNorm on partial tensor → AllGather before next TP layer.

The total communication volume is identical! SP is purely a memory optimization with **zero additional communication cost**. It should always be enabled when using TP.

Memory savings from SP (70B model, TP=8, batch=4, seq=2048):

$$\text{Activation savings} = (T-1) \times b \times s \times d \times n_{\text{layers}} \times 2 \text{ bytes} = 7 \times 4 \times 2048 \times 8192 \times 80 \times 2 \approx \mathbf{59\ GB/GPU} \tag{11.2}$$

11.2.4 Pipeline Parallelism (PP)

Pipeline Parallelism splits the model *vertically* by layers, assigning consecutive groups of layers to different devices (stages). Activations flow forward through stages; gradients flow backward.

The Bubble Problem. Naive pipeline execution creates "bubbles"—idle time while a stage waits for input from the previous stage or gradients from the next:

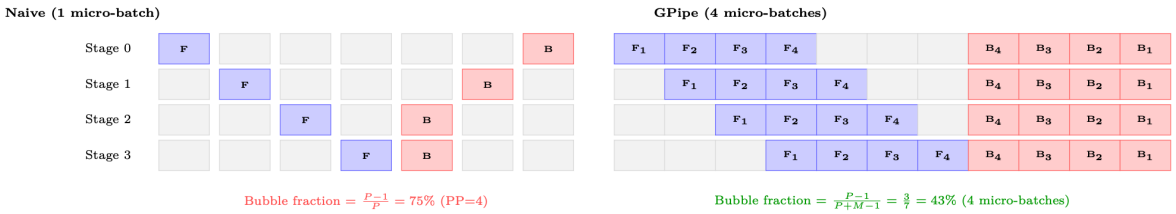


Figure 11.7: Pipeline bubble comparison. Left: naive pipeline with one micro-batch has 75% idle time. Right: GPipe with $M = 4$ micro-batches reduces bubbles significantly. With $M \gg P$, bubble fraction approaches zero.

Bubble Fraction Formula. For P pipeline stages and M micro-batches per step:

$$\text{Bubble fraction} = \frac{P-1}{P+M-1} \approx \frac{P-1}{M} \quad (\text{when } M \gg P) \tag{11.3}$$

To keep bubble overhead <10%, you need $M \geq 10 \cdot (P-1)$. For PP=4: at least 30 micro-batches.

Table 11.1: Pipeline scheduling strategies

Schedule	Bubble	Memory	Characteristics
GPipe	$\frac{P-1}{M+P-1}$	$M\times$ activations	Simple; all-forward then all-backward [162]
1F1B	$\frac{P-1}{M+P-1}$	$P\times$ activations	Interleaved; steady-state memory bounded [278]
Interleaved 1F1B	$\frac{P-1}{M\cdot V+P-1}$	$P\times$ activations	Virtual stages (V); further reduces bubble [279]
Zero-Bubble (ZB-H1)	≈ 0	$P\times$ activations	Splits backward into B and W phases [314]

Pipeline Schedules.

1F1B: The Production Standard

The **1F1B** (one-forward-one-backward) schedule [278] is used in most production systems (Megatron-LM [279], DeepSpeed [322]):
Warmup: Forward passes fill the pipeline (P-1 micro-batches).
Steady state: Alternate one forward and one backward per time slot. This bounds peak activation memory to P micro-batches (vs M for GPipe).
Cooldown: Remaining backward passes drain the pipeline.
Memory advantage: GPipe must store activations for *all* M micro-batches simultaneously. 1F1B only stores P sets of activations at steady state—critical when $M = 32$ but $P = 4$.

Communication in PP. Unlike TP (AllReduce), PP only requires **point-to-point** communication of activations between adjacent stages:

$$\text{Data per transfer} = b_{\text{micro}} \times s \times d \times 2 \text{ bytes (BF16)} \tag{11.4}$$

For micro-batch=4, seq=2048, d=8192: $4 \times 2048 \times 8192 \times 2 = 128$ MB per transfer. At InfiniBand 50 GB/s: 2.6 ms per transfer—small relative to compute per stage.

Load Balancing. Not all layers have equal compute:

- **Embedding layer**: Very cheap (lookup table).
- **Transformer blocks**: Uniform compute.
- **Final LM head**: Moderate (large matrix multiply for vocabulary projection).

Assign more transformer layers to middle stages and fewer to the first/last stages to balance compute.

11.2.5 Fully Sharded Data Parallelism (FSDP / ZeRO-3)

FSDP [454] (PyTorch) and ZeRO-3 [322] (DeepSpeed) address the memory duplication inherent in DDP: instead of every GPU holding a full copy of parameters, gradients, and optimizer states, each GPU owns only a $1/N$ slice and reconstructs the full tensor on-the-fly when needed.

FSDP execution flow per layer:

1. **Forward**: AllGather parameters → compute → discard non-owned shards.
2. **Backward**: AllGather parameters (again) → compute gradients → ReduceScatter gradients (each GPU gets its gradient shard) → discard non-owned parameter shards.

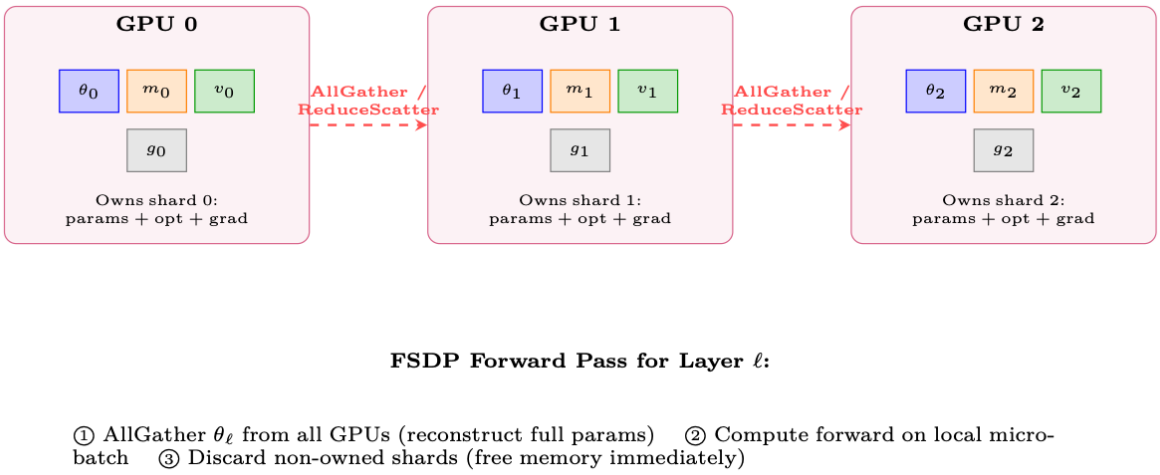


Figure 11.8: FSDP shards all model state across GPUs. Each GPU owns $1/N$ of parameters, optimizer states, and gradients. Full parameters are reconstructed on-demand via AllGather before each layer's computation.

Table 11.2: Memory comparison: DDP vs FSDP/ZeRO stages (70B model, 8 GPUs). Baseline: BF16 params (140 GB) + BF16 grads (140 GB) + FP32 master+m+v (840 GB) = 1120 GB per GPU.

Strategy	Sharded	Memory/GPU	Communication
DDP (no sharding)	Nothing	1120 GB ×	AllReduce (gradients only)
ZeRO-1	Optimizer states	385 GB ×	AllReduce (gradients)
ZeRO-2	Optimizer + gradients	368 GB ×	AllReduce (gradients)
ZeRO-3 / FSDP	Everything	**140 GB ✓**	AllGather + ReduceScatter (per layer)

3. **Optimizer step**: Each GPU updates only its owned shard using its gradient shard and optimizer states.

```
from functools import partial
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, MixedPrecision,
    BackwardPrefetch
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from transformers.models.llama.modeling_llama import LlamaDecoderLayer

# Wrap model with FSDP
auto_wrap = partial(transformer_auto_wrap_policy,
                    transformer_layer_cls={LlamaDecoderLayer})
mp_policy = MixedPrecision(
    param_dtype=torch.bfloat16,
    reduce_dtype=torch.bfloat16,
    buffer_dtype=torch.bfloat16,
)

model = FSDP(
    model,
    sharding_strategy=ShardingStrategy.FULL_SHARD,  # ZeRO-3
    mixed_precision=mp_policy,
    auto_wrap_policy=auto_wrap,  # Wrap each transformer layer
    use_orig_params=True,        # Required for torch.compile compatibility
    limit_all_gathers=True,      # Bound peak memory (1 AllGather in flight at a
    time)
    forward_prefetch=True,       # Prefetch next layer's params during current
    layer
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,  # Prefetch during backward
)
```

FSDP Communication Volume

FSDP communicates **3× more data** than DDP per step:

- DDP: 1 AllReduce of gradients = $2M$ bytes total across ring (where M = model size in bytes).
- FSDP: 2 AllGather (forward + backward) + 1 ReduceScatter = $3M$ bytes.

This is the memory–communication trade-off. FSDP is worthwhile when: (a) model doesn't fit in GPU memory with DDP, or (b) communication is well-overlapped with compute (modern frameworks achieve 70–90% overlap).

11.2.6 3D Parallelism: Combining Strategies

Production systems at scale (70B+) combine TP, PP, and DP/FSDP simultaneously:

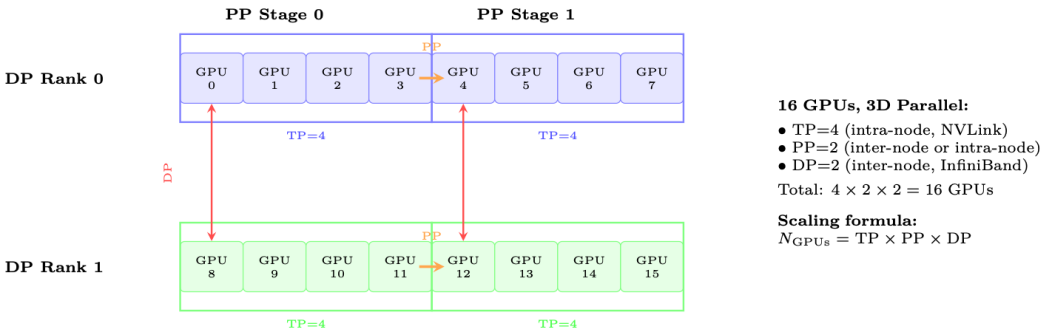


Figure 11.9: 3D parallelism layout for 16 GPUs: TP=4 (within each box, using NVLink), PP=2 (orange arrows, stages), DP=2 (red arrows, gradient sync). Each dimension exploits a different level of the communication hierarchy.

Production Recipe: 70B on 64 A100-80GB (8 nodes)

Intra-node (NVLink 600GB/s): TP=8 for generation, FSDP within node for training.

Inter-node (InfiniBand 400Gb/s): FSDP across nodes (8-way data parallel).

Result: Each GPU holds ∼70GB. Policy weights gathered per-layer during forward/backward.

Pipeline Parallel: Only if model exceeds 100B+ and won't fit with TP+ZeRO. Adds complexity (bubble overhead 10–20%) and scheduling headaches.

Decision flowchart:

1. Does the model fit on 1 GPU? → Use DDP.
2. Does it fit on 1 node with FSDP? → Use FSDP (ZeRO-3).
3. Does it fit on 1 node with TP+FSDP? → Use TP (intra-node) + FSDP (inter-node).
4. Still doesn't fit? → Add PP across nodes. This is the last resort.

Table 11.3: Parallelism strategy comparison summary

Strategy	Splits	Communication	Scaling Limit	Overhead	When to Use
DP/DDP	Batch	AllReduce (grads)	∼64 GPUs	5–10%	Model fits on 1 GPU
FSDP	Params+Opt+Grad	AllGather+RS	100s of GPUs	10–20%	Default for >13B
TP	Weight matrices	AllReduce (2/layer)	8 GPUs (1 node)	12–18%	Large model inference+train
SP	Activations (seq)	Reuses TP comms	Same as TP	≈0% extra	Always with TP
PP	Layers (stages)	Point-to-point	∼16 stages	15–30%	100B+ models only

11.2.7 Decoupled DiLoCo: Training Across Datacenters

Conventional distributed training assumes high-bandwidth interconnects—100+ Gbps InfiniBand within a single datacenter. This requirement makes it impossible to train across geographically distributed datacenters using standard internet links. **Decoupled DiLoCo** [122] (Google DeepMind, April 2026) breaks this assumption, enabling large-scale training across regions with commodity bandwidth.

DiLoCo: The Core Idea

DiLoCo (Distributed Local SGD with Compression) replaces continuous gradient synchronization with periodic outer updates:

1. Each worker i trains independently for H inner steps on its local data shard.
2. After H steps, workers synchronize by computing an **outer update**:

$$\Delta_{\text{outer}} = \frac{1}{N} \sum_{i=1}^{N} \left(\theta_i^{(H)} - \theta_i^{(0)} \right)$$

   where N is the number of workers and $\theta_i^{(0)}$ is the shared starting checkpoint.
3. The outer update is applied via a global optimizer (e.g., Nesterov momentum), and all workers reset to the new shared θ.

Only the outer update (one model-sized tensor per H steps) crosses the network—not per-step gradients.

Decoupled DiLoCo extends this by further decoupling the outer synchronization: workers do not need to synchronize simultaneously. Instead, updates are asynchronous across regions, with a coordinator applying outer updates as they arrive. This tolerates the variable latency of wide-area networks.

Decoupled DiLoCo: Empirical Results

A 12B parameter model was trained across 4 US geographic regions connected by standard internet links (2–5 Gbps):

- **Bandwidth reduction**: 236× less bandwidth than synchronous distributed training.
- **Speed**: 20× faster wall-clock time than fully asynchronous SGD (which suffers from stale gradients).
- **Model quality**: Matches single-datacenter training on standard language modeling benchmarks.
- **Fault tolerance**: Worker failures only delay the outer update, not the entire training run.

Why This Matters: Sovereign AI and Multi-Cloud

Decoupled DiLoCo has significant implications beyond efficiency:

- **Sovereign AI**: Nations or organizations can contribute compute from their own datacenters without routing sensitive data through a central facility.
- **Multi-cloud training**: Leverage spot instances across AWS, GCP, and Azure simultaneously, routing around outages.
- **Datacenter resilience**: A datacenter failure loses only that worker's contribution for the current H-step window, not the entire run.
- **Heterogeneous hardware**: Workers with different GPU generations can participate, each running at their own pace.

The key insight is that modern optimizers are robust to infrequent, noisy outer updates—the inner training steps provide sufficient local signal that the outer synchronization only needs to correct for distribution drift.

11.3 The Generation Bottleneck: Quantitative Analysis

Roofline Analysis: Why Generation is Memory-Bound

A100 specs: 312 TFLOPS (BF16 tensor cores), 2 TB/s HBM bandwidth.
Roofline crossover: 312T/2T = 156 FLOP/byte. Operations below 156 FLOP/byte are *memory-bound*.
Autoregressive generation: For each token, read all weights (140GB for 70B) and do $2 \times 70B = 140G$ FLOPs per token (at batch=1).
Arithmetic intensity: 140G FLOP/140GB = 1 FLOP/byte. That's 156× below the roofline!
Utilization: 1/156 = 0.6% of peak FLOPS utilized. The GPU is 99.4% idle, waiting for memory reads.
Token rate: 2TB/s/140GB = 14.3 tokens/second (single stream, batch=1).
For 512 tokens: 512/14.3 = 35.8 seconds per response (batch=1, TP=1).
Batching helps: Batch=64 with TP=4 → reads weights once, generates 64 tokens in parallel. Arithmetic intensity: $64 \times 1 = 64$ FLOP/byte. Better, but still below roofline!

Table 11.4: Generation throughput for 70B model (512 tokens, various configurations)

Config	Batch	Time/batch	Tok/s/GPU	Notes
TP=1, batch=1	1	36s	14	Baseline, worst case
TP=4, batch=1	1	9s	57	Linear TP scaling for gen
TP=4, batch=32	32	15s	1092	Near-optimal batching
TP=4, batch=128, vLLM	128	45s	1456	Continuous batching
TP=4, batch=128, INT8	128	25s	2621	2× bandwidth savings

Optimization stack (cumulative speedup):

1. **vLLM + PagedAttention** [206] (2–4×): Eliminates KV cache fragmentation, enables larger batches
2. **Continuous batching** [435] (1.5–2×): Don't wait for longest sequence; start new ones as others finish
3. **Speculative decoding** [216] (2–3×): Small draft model proposes 5 tokens, large model verifies in one forward pass. Accept 3–4 on average.
4. **INT8/FP8 weights for gen** (2×): Halve bandwidth needs. Quality loss is minimal since we're sampling (not computing exact logits for training)
5. **CUDA graphs** (1.1–1.3×): Eliminate kernel launch overhead for fixed-shape operations
6. **Prefix caching** (1.5× for shared-prefix prompts): Don't recompute system prompt KV cache

```
# Production vLLM generation setup
from vllm import LLM, SamplingParams

engine = LLM(
    model="./policy_checkpoint",
    tensor_parallel_size=4,           # TP=4 per instance
    gpu_memory_utilization=0.92,      # Leave headroom for KV cache
    max_num_batched_tokens=16384,     # Max tokens in flight
    max_num_seqs=256,                 # Max concurrent sequences
    dtype="bfloat16",
    enable_prefix_caching=True,       # Cache system prompt KV
    speculative_model="./draft_1B",   # Speculative decoding
```

```
    num_speculative_tokens=5,
    block_size=16,                    # PagedAttention block size
    swap_space=4,                     # GB swap space for preemption
)

# Generate responses for RLHF batch
sampling_params = SamplingParams(
    temperature=0.7, top_p=0.9, max_tokens=512,
    logprobs=1,  # Need log-probs for PPO ratio calculation
)
outputs = engine.generate(prompts, sampling_params)
# Extract: responses, log_probs for each token (needed for PPO/GRPO)
```

11.4 Decoupled Architecture: Production Design

Production RLHF systems such as DeepSpeed-Chat [432] and OpenRLHF [158] use a **decoupled architecture** that separates generation, scoring, and training into independently scalable clusters.

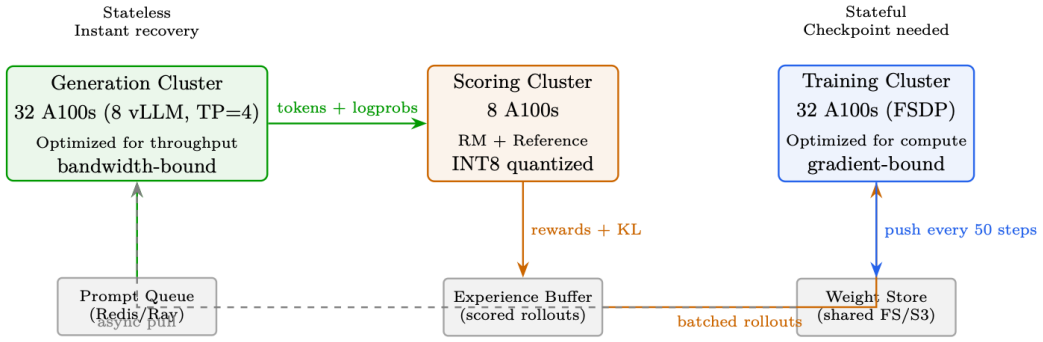


Figure 11.10: Decoupled RLHF architecture. Each cluster optimized for its workload. Scored rollouts accumulate in the experience buffer before being consumed by training.

Why Decouple?

Generation is memory-bandwidth bound (need fast HBM, waste compute).

Training is compute-bound (need tensor cores, waste bandwidth during backprop).

Same hardware can't optimize both: If you put everything together, you either waste compute during generation or waste bandwidth during training. Decoupling lets each cluster use optimal hardware/config.
Practical benefits:

- Scale generation and training independently
- Generation cluster is stateless → trivial fault tolerance
- Can overlap gen(step $N+1$) with training(step N) → 30–40% speedup
- Different quantization: INT8 for generation (bandwidth), BF16 for training (precision)

11.5 Weight Synchronization Strategies

Strategy	Staleness	Bandwidth	Quality Impact
Synchronous (every step)	0 steps	140 GB/step	Perfect but too slow
Periodic (every 50 steps)	25 avg	2.8 GB/step amortized	<2% quality loss
Delta compression (INT8)	25 avg	0.4 GB/step	<3% quality loss
Async streaming	5–10 steps	14 GB/step (background)	<1% quality loss

Why Staleness is OK for PPO/GRPO

PPO's clipped objective was *designed* for off-policy data! The clip $[1-\epsilon, 1+\epsilon]$ bounds the impact of stale data. With 10–50 steps of staleness:

- Policy changes ∼0.1–1% per step (with proper LR)
- Over 50 steps: ∼5% policy drift
- PPO clip handles up to 20% drift by design
- Empirically: quality loss <2% for 50-step staleness

Bandwidth math: 70B BF16 = 140GB. InfiniBand 400Gb/s = 50GB/s → full sync in 2.8s. With delta compression: <0.5s. Async = free (runs in background).

11.6 Memory Optimization Techniques

ZeRO Stage	What Gets Sharded	Memory/GPU (70B, 8 GPUs)
None (Data Parallel)	Nothing (full replica)	560GB per GPU (impossible)
ZeRO-1	Optimizer states only	175GB
ZeRO-2	Optimizer states + Gradients	105GB
ZeRO-3 (FSDP)	Optimizer + Gradients + Parameters	**70GB (fits in A100-80GB!)**

Additional techniques:

- **Gradient checkpointing** [51]: Don't store all activations; recompute during backward pass. Saves ∼60% activation memory, costs ∼33% extra compute. Selective: only checkpoint attention layers (memory-heavy), keep FFN activations (compute-heavy to recompute).
- **Mixed precision** [268]: Forward in BF16 (2 bytes/param), optimizer states in FP32 (4 bytes each for m,v). Master weights in FP32 for accumulation.
- **CPU offloading** (ZeRO-Infinity [323]): Move optimizer states to CPU RAM. 50% memory savings but 2–3× slower (PCIe 64GB/s bottleneck).
- **Activation offloading**: Move activations to CPU during forward, bring back for backward. Only when memory is truly critical.
- **Flash Attention** [67, 68]: O(n) memory instead of O(n^2) for attention. 2–4× faster + massive memory savings for long sequences.

11.6.1 Flash Attention's Impact on RLHF

Why Flash Attention Matters for RLHF

RLHF involves generating long sequences (rollouts) and then training on them. Without Flash Attention:

- A 4K-token sequence with 32 heads requires ∼4 GB just for attention matrices
- This severely limits batch size during PPO/GRPO training
- Gradient checkpointing of attention activations is expensive

With Flash Attention:

- Attention memory is $O(n)$ – dominated by Q, K, V, O tensors

- Longer rollouts (8K–32K tokens) become feasible with the same GPU memory
- Backward pass recomputes attention tiles from Q, K, V (no stored n^2 matrix)
- This is the key enabler for long-context RLHF (e.g., reasoning models)

Flash Attention and Gradient Checkpointing

Flash Attention's backward pass recomputes the attention tiles on-the-fly from Q, K, V (which are stored). This means Flash Attention *already implements* a form of activation recomputation for the $O(n^2)$ attention matrix. You do not need to additionally checkpoint the attention layer – doing so would recompute Q, K, V unnecessarily.

```
# DeepSpeed ZeRO-3 configuration for 70B RLHF training
ds_config = {
    "bf16": {"enabled": True},
    "zero_optimization": {
        "stage": 3,
        "overlap_comm": True,                    # Overlap communication with
    compute
        "contiguous_gradients": True,            # Better memory layout
        "reduce_scatter": True,                  # More efficient than allreduce
        "reduce_bucket_size": 5e7,               # 50M params per bucket
        "prefetch_bucket_size": 5e7,             # Prefetch next bucket
        "param_persistence_threshold": 1e5,      # Keep small params on all GPUs
        "offload_optimizer": {"device": "cpu", "pin_memory": True},  # CPU offload
        "sub_group_size": 1e9,                   # Reduce fragmentation
    },
    "gradient_accumulation_steps": 4,
    "gradient_clipping": 1.0,
    "train_micro_batch_size_per_gpu": 2,
    "wall_clock_breakdown": True,
}
```

11.6.2 DualKV: Eliminating Shared-Prompt Redundancy During RL Training

Flash Attention solves the $O(n^2)$ memory problem for a single sequence. But GRPO and DAPO introduce a second source of redundancy that standard Flash Attention does not address: **every rollout in a group replicates the full prompt**.

When training with N rollouts per prompt (e.g. $N = 32$ for GRPO), each of length R tokens from a shared prompt of P tokens, standard Flash Attention processes $N \times (P + R)$ tokens per micro-batch. The prompt portion is computed N times—identical hidden states, identical KV pairs, redundant work. At $N = 32$, $P = 8$K, this prompt replication can dominate the policy-update cost.

DualKV [254] eliminates this redundancy by exploiting a property of decoder-only models: *causal masking makes prompt representations invariant across sequences at every layer*. Because no response token can attend back to affect the prompt's hidden states, all per-token operations (norms, projections, MLP) and attention can process the prompt exactly once.

How it works. DualKV introduces two complementary mechanisms:

1. **Fused CUDA kernels**: Custom forward and backward kernels iterate over two disjoint KV regions—a *shared context* region (the prompt, processed once) and a *per-sequence response* region—in a single kernel launch. The kernel is mathematically equivalent to standard attention; no approximation is introduced.

2. **Token repacking in veRL**: The data pipeline is redesigned to repack $N(P + R)$ tokens into $P + NR$ tokens per micro-batch. This extends the token reduction from the attention kernel to the *entire model*—norms, projections, and MLP layers all benefit. The effective compression ratio is:

$$\rho = \frac{N(P+R)}{P+NR} \tag{11.5}$$

At $N = 32$, $P = 8\text{K}$, $R = 1\text{K}$, this gives $\rho \approx 4.9\times$ fewer tokens through the full model.

Results. On Qwen3-8B GRPO training ($N = 32$, 8K context, 8×H100):

- **1.63–2.09**× policy-update speedup over standard Flash Attention
- **MFU: 36% → 76%** (more than doubled)
- **2× larger micro-batches** within the same GPU memory budget
- DAPO: **2.47**× speedup, 77% MFU

At 30B MoE scale (16×H100), where standard Flash Attention requires 4-way Ulysses sequence parallelism just to avoid OOM, DualKV achieves **3.82**× policy-update and **3.38**× end-to-end step speedup. It also extends to hybrid sliding/global attention with head dimension 512 (unsupported by FA2) and composes with Ulysses SP, demonstrated on Gemma-4-31B GRPO at 64K context.

DualKV vs. Prefix Caching: Different Problems

Prefix caching (vLLM, §2.2.3) avoids recomputing the prompt KV cache during *inference generation*. DualKV eliminates prompt replication during the *training forward and backward pass*. They are complementary: a production GRPO pipeline can use prefix caching for the rollout phase and DualKV for the policy-update phase.

11.7 Fault Tolerance at Scale

Hardware Failure Reality

Individual GPU MTBF: ∼10,000 hours.
512-GPU cluster MTBF: 10000/512 ≈ 20 hours. But with software/network: **4–8 hours realistically**.
Multi-day training run: Will see 5–15 failures. Without fault tolerance, one failure kills everything.

Production fault tolerance stack:

1. **Detection**: NCCL timeout (60s), GPU heartbeat (10s), NVML health monitoring, ECC error counting.
2. **Checkpointing**: Async every 50–100 steps. Non-blocking (background thread). Save: model weights, optimizer states (Adam m/v), scheduler state, RNG states, KL coefficient, replay buffer. Keep last 3 checkpoints. Time: ∼30s for 70B (parallel write to NVMe).
3. **Recovery**: (a) Generation cluster = stateless, just restart and load latest weights. (b) Training cluster: load checkpoint, rebuild NCCL process group excluding failed node, redistribute FSDP shards, resume from last checkpoint.
4. **Elastic training**: Torch Elastic / Kubernetes auto-scaling. Replace failed node within minutes. Training continues with $N - 1$ GPUs temporarily.
5. **Prevention**: GPU health pre-screening (run GEMM stress test before starting). Hot spares on standby. Redundant network paths (dual-rail InfiniBand).

11.8 End-to-End Latency Breakdown

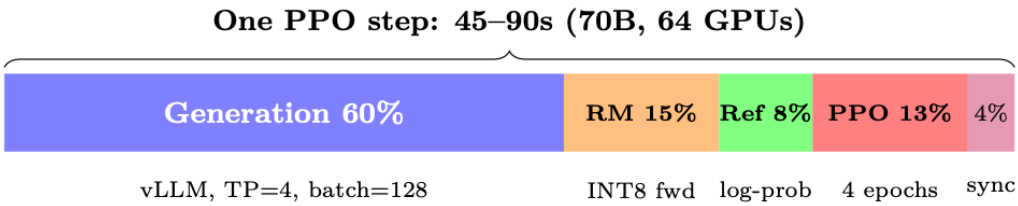


Figure 11.11: Without overlap (monolithic). With decoupled: gen overlaps with training, effective 1.4× speedup.

Table 11.5: End-to-end latency breakdown per RLHF step (70B model). Decoupled overlap yields an effective 1.4× speedup.

Phase	Time (70B)	Bound By	Optimization
Generation (128×512 tok)	30–45s	Memory bandwidth	vLLM, spec decoding, INT8
Reward scoring	5–8s	Compute (batch forward)	INT8 RM, batch=128
Reference log-probs	4–6s	Compute (batch forward)	INT8 ref, or LoRA (free)
PPO update (4 epochs)	8–12s	Compute (backprop)	FSDP, Flash Attention
Weight sync	0–3s	Network (async)	Delta compression, async
Total (monolithic)	**50–75s**		
Total (decoupled, overlapped)	**35–50s**		Gen overlaps with prev training

11.9 Monitoring and Observability

Key Metrics to Track During RLHF Training

Quality metrics (log every 10 steps):

- Mean reward (should increase then plateau)
- KL divergence from reference (should stay 3–10)
- Response length distribution (watch for length hacking)
- Entropy (should decrease slowly, not collapse)

System metrics (log every step):

- GPU utilization (target: >80% during training, >60% during gen)
- Memory watermark per GPU (catch OOM before it happens)
- Generation throughput (tokens/sec, should be stable)
- Gradient norm (spikes = instability incoming)
- NCCL communication time (detect network degradation)

11.10 Network Topology and Communication Patterns

Efficient distributed training requires understanding the hierarchical communication fabric that connects GPUs. Modern clusters use a two-tier architecture: ultra-fast intra-node links and slower but scalable inter-node networks.

11.10.1 Intra-Node: NVLink and NVSwitch

Table 11.6: NVLink generations and their impact on LLM training

Generation	BW per link	Links/GPU	Total BW	Platform
NVLink 3.0	50 GB/s	12	600 GB/s	A100 (DGX A100)
NVLink 4.0	50 GB/s	18	900 GB/s	H100 (DGX H100)
NVLink 5.0	100 GB/s	18	1800 GB/s	B200 (DGX B200)

Within a single node (typically 8 GPUs), **NVSwitch** provides full-bisection bandwidth between all GPU pairs. This means any GPU can communicate with any other at full NVLink speed simultaneously—critical for Tensor Parallelism where every layer requires an AllReduce across all 8 GPUs.

NVSwitch vs PCIe Topology

With NVSwitch (DGX/HGX): All 8 GPUs connected all-to-all at 600–1800 GB/s. AllReduce for TP takes ∼0.2ms per layer.
Without NVSwitch (PCIe-only servers): GPUs communicate through CPU PCIe root complex at 32–64 GB/s. TP across 8 GPUs becomes 10–30× slower. **Never use TP>2 on PCIe-only systems.**

11.10.2 Inter-Node: InfiniBand and RoCE

For FSDP/ZeRO-3 AllGather and ReduceScatter operations across nodes, the inter-node network dominates.

Table 11.7: Inter-node networking options for LLM training clusters

Technology	Bandwidth	Latency	Notes
InfiniBand NDR	400 Gb/s (50 GB/s)	1–2 μs	Gold standard, RDMA, lossless
InfiniBand NDR (dual-rail)	800 Gb/s (100 GB/s)	1–2 μs	Used in H100 clusters
RoCE v2	100–400 Gb/s	2–5 μs	Cheaper, needs PFC/ECN tuning
Ethernet (TCP)	100–400 Gb/s	10–50 μs	Not suitable for >16 GPU training

11.10.3 Communication Primitives and Their Costs

Understanding when each collective is used helps diagnose bottlenecks:

Table 11.8: NCCL collective operations in distributed LLM training

Collective	Data Moved	Used By	When
AllReduce	$2 \cdot \frac{N-1}{N} \cdot M$	TP, DP	Sum gradients or activations across GPUs
AllGather	$\frac{N-1}{N} \cdot M$	FSDP forward	Reconstruct full parameter tensor before matmul
ReduceScatter	$\frac{N-1}{N} \cdot M$	FSDP backward	Distribute gradient shards after backprop
Broadcast	M	PP	Send activations to next pipeline stage
Send/Recv	M	PP	Point-to-point between adjacent stages

where M is the message size (bytes) and N is the number of participants.

Communication-Computation Overlap

Modern frameworks (FSDP, DeepSpeed) aggressively overlap communication with computation: **Forward pass**: While layer i computes, AllGather prefetches parameters for layer $i+1$. After layer i finishes, its parameters are immediately discarded ("free-after-forward"). **Backward pass**: While layer i computes gradients, ReduceScatter sends layer $i+1$'s gradients. This overlap hides 70–90% of communication latency when properly tuned. **Tuning knobs**: `prefetch_factor` (how many layers ahead to prefetch), `reduce_bucket_size` (granularity of gradient reduction), `backward_prefetch` ("pre" vs "post" backward prefetch strategy).

11.10.4 Network Topology Design

Production clusters use **fat-tree** or **rail-optimized** topologies:

- **Fat-tree**: Full bisection bandwidth at every level. Any node can communicate with any other at full speed. Expensive (many switches) but maximally flexible.
- **Rail-optimized**: GPU i on every node connects to the same leaf switch ("rail i"). AllReduce within a rail is cheap; cross-rail traffic is expensive. Used by Meta's RSC and Google's TPU pods.
- **3D torus / Dragonfly**: Used in HPC clusters (Frontier, Aurora). Topology-aware job placement is critical.

Job Placement Matters

On a 512-GPU cluster, random node assignment can cause 2–3× slowdown due to network congestion. **Always request contiguous node blocks.** Production schedulers (Slurm, Kubernetes) should enforce locality: all nodes in a training job should be on the same leaf switch or within one hop of each other.

11.11 Training Throughput and Model FLOPs Utilization

11.11.1 Measuring Training Efficiency: MFU

Model FLOPs Utilization (MFU) [57] is the standard metric for training efficiency:

$$\text{MFU} = \frac{\text{Observed throughput (tokens/sec)} \times \text{FLOPs per token}}{\text{Peak hardware FLOPS}} \tag{11.6}$$

For a transformer with P parameters, s sequence length, and b batch size:

$$\text{FLOPs per token} \approx 6P + 12 \cdot n_{\text{layers}} \cdot d_{\text{model}} \cdot s \tag{11.7}$$

The factor of 6 comes from: 2 (multiply-add) × 3 (forward + backward, where backward ≈ 2× forward). The second term accounts for attention's $O(s^2)$ cost.

Why MFU Decreases with Scale

Larger models require more parallelism, which introduces:

1. **Communication overhead**: AllGather/ReduceScatter for FSDP (∼10–15% at 64 GPUs)
2. **Pipeline bubbles**: PP introduces idle time at start/end of micro-batches (∼15–25% with PP=4)
3. **Memory for auxiliary models**: Reference/RM take GPU memory that could hold larger batches

Table 11.9: MFU benchmarks across scales and hardware

Model	Hardware	MFU	Tokens/sec/GPU	Configuration
LLaMA-7B	8×A100	57%	3,200	FSDP, FlashAttn, BF16
LLaMA-13B	16×A100	52%	1,750	FSDP, FlashAttn, BF16
LLaMA-70B	64×A100	45%	380	FSDP+TP=8, FlashAttn
GPT-4 (est.)	10,000+ H100	40–50%	—	3D parallelism
PaLM-540B	6144 TPUv4	46%	—	DP+TP+PP

4. **Load imbalance**: Not all layers have equal compute (embeddings vs transformer blocks)

Rule of thumb: Target MFU > 40% for training. If below 30%, diagnose with profiling.

11.11.2 Compute-Optimal Batch Sizing

The effective batch size interacts with hardware utilization in non-obvious ways:

$$\text{Effective batch size} = \text{micro_batch} \times \text{grad_accum} \times \text{DP degree} \tag{11.8}$$

- **Too small**: GPU underutilized (low arithmetic intensity), communication dominates.
- **Too large**: Diminishing learning per token (critical batch size exceeded), wastes compute.
- **Sweet spot**: The *critical batch size* B_{crit} where gradient noise equals gradient signal. For LLMs, $B_{\text{crit}} \sim$ 1–4M tokens [263].

For RLHF specifically, the batch contains *rollouts* (not just tokens):

$$\text{RLHF batch} = N_{\text{prompts}} \times K_{\text{generations}} \times L_{\text{avg response length}} \tag{11.9}$$

Typical production values: $N = 128$ prompts, $K = 1$–4 generations, $L = 256$–512 tokens → 32K–256K tokens per step.

11.11.3 Profiling and Bottleneck Diagnosis

Key profiling tools and what they reveal:

Tool	Captures	Best For
`torch.profiler`	Kernel timing, memory	Finding slow ops, memory leaks
NVIDIA Nsight Systems	Full GPU timeline	Visualizing overlap, gaps between kernels
`nccl_debug=INFO`	Collective sizes/times	Diagnosing communication bottlenecks
`torch.cuda.memory_stats`	Allocation patterns	Finding fragmentation, peak usage
DeepSpeed Flops Profiler	Per-layer FLOPs	Identifying load imbalance
`py-spy` / `scalene`	CPU profiling	Data loading, tokenization bottlenecks

Diagnosing Low MFU: A Checklist

1. **GPU utilization < 80%?** → Data loading bottleneck (check CPU, I/O).
2. **Large gaps between kernels?** → Python overhead, synchronization points. Use CUDA graphs.
3. **Communication > 20% of step time?** → Reduce TP degree, increase batch size, check network health.

4. **Memory at 99%?** → Cannot increase batch. Try gradient checkpointing, offloading.
5. **OOM during generation?** → KV cache too large. Reduce max_seq_len or batch size for gen.

11.12 Cost Analysis and Cloud Deployment

Understanding the economics of RLHF training is essential for planning.

11.12.1 Hardware Cost Comparison

Table 11.10: Approximate cloud GPU costs for RLHF training (2024–2025 pricing)

GPU	On-Demand/hr	Spot/hr	Memory	Use Case
A100 80GB	\$2.50–3.50	\$1.00–1.50	80 GB HBM2e	Budget training, gen cluster
H100 80GB	\$4.00–6.00	\$2.00–3.00	80 GB HBM3	Production training
H200 141GB	\$6.00–8.00	—	141 GB HBM3e	Large context, fewer-GPU configs
MI300X 192GB	\$3.50–5.00	\$1.50–2.50	192 GB HBM3	Cost-effective alternative

11.12.2 RLHF Training Cost Estimation

$$\text{Cost} = \frac{N_{\text{steps}} \times T_{\text{step}}}{3600} \times N_{\text{GPUs}} \times C_{\text{GPU/hr}} \tag{11.10}$$

Cost Example: 70B Model RLHF (10K steps)

Steps	10,000
Time per step (decoupled)	45 seconds
Total training time	$10000 \times 45/3600 = 125$ hours
GPUs (generation + training)	64 A100-80GB
Cost per GPU-hour (spot)	\$1.20
Total cost	$125 \times 64 \times \$1.20 =$ **\$9,600**

Breakdown by phase:

- Generation cluster (32 GPUs): \$4,800 (60% of time)
- Training cluster (32 GPUs): \$4,800 (could overlap → \$3,400 effective)
- Scoring (shared with gen GPUs): included above

With overlap: Effective cost ≈ **\$7,500** for full RLHF alignment of a 70B model.

11.12.3 Cost Optimization Strategies

- **Spot/preemptible instances**: 50–70% savings. Requires robust checkpointing (save every 5 minutes).
- **Right-sizing**: Don't use H100 for generation (memory-bound); A100 achieves similar tokens/\$ for inference.
- **Quantized inference**: INT8/FP8 for generation and scoring halves GPU count for those clusters.

- **Progressive training**: Start with 8B proxy model for reward engineering/debugging (~$200), then scale to 70B.
- **LoRA for reference-free**: Eliminates reference model entirely (50% memory reduction).
- **Shorter sequences first**: Curriculum from 256→512→1024 token generations saves 40% compute.

11.13 Distributed Checkpointing

At scale, naive checkpointing becomes a bottleneck. A 70B model with optimizer state requires saving ~840 GB per checkpoint (FP32 master weights + Adam m + v).

11.13.1 Checkpointing Strategies

Table 11.11: Checkpointing approaches for large-scale RLHF

Strategy	Save Time (70B)	Storage/ckpt	Characteristics
Synchronous (all ranks)	30–60s (blocking)	420 GB	Simple, stalls training
Async (background copy)	<1s (non-blocking)	420 GB	Overlaps with next step
Incremental (delta)	<1s	5–20 GB	Only save changed params
Sharded (FSDP native)	5–10s	420 GB sharded	Each rank saves its shard

11.13.2 Production Checkpointing with torch.distributed.checkpoint

```
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict,
    StateDictOptions

# Save: each rank writes its shard in parallel
state_dict = {"model": get_state_dict(model, options=StateDictOptions(
    full_state_dict=False))}
dcp.save(
    state_dict=state_dict,
    storage_writer=dcp.FileSystemWriter("/mnt/checkpoints/step_5000"),
    planner=dcp.DefaultSavePlanner(),  # Handles FSDP sharding automatically
)

# Async save: non-blocking, runs in background thread
future = dcp.async_save(
    state_dict=state_dict,
    storage_writer=dcp.FileSystemWriter("/mnt/checkpoints/step_5000"),
)
# Training continues immediately; future.result() blocks only if needed
```

Checkpoint Hygiene for RLHF

RLHF checkpoints must capture *more* than standard pre-training:

- Policy model weights + optimizer states (standard)
- KL coefficient (β) and its schedule state
- Replay buffer contents (for off-policy corrections)
- RNG states for all GPUs (reproducibility)

- Prompt iterator position (avoid re-processing prompts)
- Reward model version tag (for auditability)
- Wandb/metrics run ID (for continuous logging)

11.14 Hardware Selection Guide

Choosing the right hardware depends on model size, budget, and training phase.

Table 11.12: Hardware recommendations by model scale and training phase

Model Size	Training Phase	Recommended	Configuration
≤7B	SFT + RLHF	1–2× A100	Single node, no parallelism needed
7–13B	SFT + RLHF	4–8× A100	FSDP, optional TP=2 for gen
13–34B	SFT + RLHF	8–16× A100/H100	FSDP + TP=4 for gen
70B	RLHF (full)	32–64× A100/H100	Decoupled, FSDP + TP=8
70B	RLHF (LoRA)	8–16× A100/H100	No ref model, LoRA adapters
>100B	RLHF	128+× H100	3D parallelism (TP+PP+DP)

H100 vs A100: When is the Upgrade Worth It?

H100 provides:

- ∼1.6× peak FLOPS (989 vs 624 TFLOPS for BF16 with sparsity; 495 vs 312 without sparsity)
- ∼2× memory bandwidth (3.35 vs 2.0 TB/s)
- FP8 support (additional 2× for inference)
- NVLink 4.0 (900 vs 600 GB/s)

For training: ∼1.8–2.2× faster end-to-end (FP8 support and higher bandwidth amplify the raw FLOPS advantage).
For generation: ∼1.7× faster (bandwidth-bound, so 2× BW ≈ 1.7× throughput with overhead).
Cost-performance: At 1.5× the price, H100 is almost always better value for training. For inference-only (generation clusters), A100 at spot pricing can be more cost-effective.

11.15 Optimizer Configuration for RL Training

RL training (PPO, GRPO, DPO) imposes unique demands on the optimizer compared to pretraining or SFT. The loss landscape is non-stationary (the policy changes what data is generated), gradients are noisier (reward signal variance), and training is more prone to catastrophic forgetting or reward hacking. This section consolidates RL-specific optimizer guidance, using AdamW [250] as the default optimizer.

11.15.1 Why RL Requires Different Optimizer Settings

RL vs. SFT Optimization – Key Differences

- **Non-stationary data distribution**: unlike SFT where the dataset is fixed, RL generates new rollouts each iteration—the data distribution shifts with the policy.
- **High gradient variance**: reward signals are sparse and noisy; gradients have much higher variance than cross-entropy on curated data.
- **Smaller updates required**: the policy must stay close to the reference model (KL

constraint), so learning rates are 10–100× smaller than SFT.

- **No weight decay**: regularization comes from the KL penalty, not weight decay. Adding WD on top can fight the KL constraint.
- **Shorter warmup**: RL starts from a converged SFT checkpoint—the optimizer state needs minimal warmup.

11.15.2 Recommended Hyperparameters by RL Method

Table 11.13: Optimizer settings for RL training phases. All use $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, `max_grad_norm`=1.0, BF16.

Method	Optimizer	LR	WD	Warmup	Schedule
DPO	AdamW	5e-7	0.0	50 steps	Constant or Linear
PPO (policy)	AdamW	1e-6	0.0	20 steps	Constant
PPO (critic)	AdamW	1e-6	0.0	20 steps	Constant
GRPO	AdamW	1e-6	0.0	20 steps	Constant

Why Constant Schedule for RL?

Cosine and linear-decay schedules assume a fixed training horizon and monotonically decreasing loss. RL training has neither: reward may plateau, spike, or oscillate unpredictably. A constant LR (after brief warmup) keeps the optimizer responsive throughout training. If you must decay, use a very gentle linear schedule with a high minimum LR ratio (≥ 0.5).

11.15.3 Beta-2 = 0.95 for RL: Faster Adaptation

The default Adam $\beta_2 = 0.999$ gives a very long memory for the second moment (∼1000-step effective window). In RL training, the loss landscape changes rapidly as the policy evolves—the gradient variance from 1000 steps ago is irrelevant. Using $\beta_2 = 0.95$ shortens the window to ∼20 steps, making the adaptive learning rate respond quickly to changing gradient statistics.

When beta2 = 0.95 Hurts

For very small batch sizes (e.g., batch=1 in online RL), $\beta_2 = 0.95$ can make the second moment estimates too noisy. In this regime, use $\beta_2 = 0.99$ as a compromise, or increase the effective batch size via gradient accumulation.

11.15.4 Mixed Precision for RL: FP32 Master Weights Are Critical

RL training is particularly sensitive to numerical precision:

- Gradients are noisier—small updates must accumulate accurately over many steps
- Learning rates are very small (10^{-6}–10^{-7}), making $\Delta\theta \ll \theta$
- BF16 mantissa (7 bits ≈ 0.8% relative precision) cannot represent updates of magnitude 10^{-6} relative to weights of magnitude 10^{0}

Always use FP32 master weights for RL training. BF16-only training (no FP32 copy) reliably causes reward collapse in PPO/GRPO after 100–500 steps.

11.15.5 Gradient Clipping is Critical for RL

In PPO and GRPO, the reward signal can be highly variable, especially early in training. A single bad batch can produce gradients with norm > 100, which would completely destroy the model weights. `max_grad_norm=1.0` is the standard setting. For SFT, clipping is less critical but still recommended.

Never Disable Gradient Clipping for RL

Unlike SFT where gradient norms are typically stable (0.1–1.0 range), RL gradients are spiky because: (1) reward variance propagates through the policy gradient, (2) rare high-reward trajectories create outsized updates, and (3) the KL penalty term can produce large gradients when the policy drifts. A single unclipped step with $\|\nabla\| > 50$ can undo hundreds of training steps.

11.15.6 Diagnosing RL Training Instability

Red Flags and Fixes for RL Optimization

Symptom	Likely Cause and Fix
Reward improves then collapses	LR too high or KL coefficient too low. Reduce LR by 2–5× or increase β_{KL}.
Gradient norm constantly at clip threshold	Updates too aggressive. Reduce LR (clipping means you're losing gradient direction info every step).
KL divergence explodes (>15 nats)	LR too high. Reduce by 10× or add adaptive KL penalty.
Reward stuck at baseline	LR too low, or reward model has low signal. Try 2–5× higher LR. Check reward model calibration.
Loss NaN after 100+ steps	FP32 master weights missing, or grad norm overflow. Enable FP32 master weights; verify BF16 mode.

11.15.7 HuggingFace TRL Configuration for RL

The TRL library [398] provides production-ready implementations of PPO, DPO, and other RL methods for LLMs.

```
from trl import PPOConfig, PPOTrainer, DPOConfig, DPOTrainer

# --- PPO Configuration ---
ppo_config = PPOConfig(
    # Optimizer (AdamW with RL-specific settings)
    learning_rate=1e-6,           # 10-100x smaller than SFT

    # PPO-specific
    ppo_epochs=4,                 # mini-batch updates per rollout
    mini_batch_size=16,
    batch_size=64,                # rollout batch size

    # Gradient control
    max_grad_norm=1.0,

    # KL penalty (replaces weight decay as regularizer)
    init_kl_coef=0.2,            # initial KL penalty coefficient
    adap_kl_ctrl=True,           # adaptive KL targeting
    target_kl=6.0,               # target KL divergence

    # Mixed precision
    bf16=True,                   # BF16 compute, FP32 master weights
)

ppo_trainer = PPOTrainer(
    model=model,
    ref_model=ref_model,
    config=ppo_config,
    tokenizer=tokenizer,
    dataset=dataset,
)

# --- DPO Configuration ---
dpo_config = DPOConfig(
```

```
    output_dir="./dpo_output",

    # Optimizer
    learning_rate=5e-7,           # even smaller than PPO
    optim="adamw_torch",
    adam_beta1=0.9,
    adam_beta2=0.95,              # shorter memory for RL
    weight_decay=0.0,             # no WD -- KL provides regularization

    # Schedule
    lr_scheduler_type="constant_with_warmup",
    warmup_steps=50,

    # Gradient control
    max_grad_norm=1.0,

    # DPO-specific
    beta=0.1,                     # KL constraint strength
    loss_type="sigmoid",          # standard DPO loss

    # Mixed precision
    bf16=True,

    # Training
    num_train_epochs=1,           # DPO typically 1 epoch
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
)

dpo_trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=dpo_config,
    train_dataset=dataset,
    tokenizer=tokenizer,
)
dpo_trainer.train()
```

Listing 11.1: Complete PPO and DPO optimizer configuration using TRL.

11.15.8 MoE Considerations for RL Training

MoE for RLHF

Mixture-of-Experts (MoE) models [94] are increasingly used in RLHF:

- **Advantage**: 3–4× more capacity at same compute cost. Better for reward models (more capacity to judge).
- **Challenge**: Expert parallelism requires all-to-all communication (tokens routed across GPUs). This conflicts with pipeline parallelism.
- **GRPO with MoE**: Works well since generation cost is dominated by active params (not total params).
- **LoRA for MoE**: Can apply LoRA to router + shared layers only, or to all experts (expensive).

The RL Optimizer Mantra

For RL fine-tuning: **small LR, no weight decay, constant schedule, FP32 master weights, aggressive clipping**. Let the KL penalty handle regularization—the optimizer's job is just to follow the policy gradient without overshooting.

Miles: PyTorch-Native RL Post-Training. Miles [313] is PyTorch's official stack for large-scale LLM RL, connecting SGLang (for high-throughput rollout generation) with Megatron-LM (for distributed training updates). It supports both disaggregated execution (separate GPU pools for rollout and training, maximizing utilization) and colocated execution (fitting both on the same nodes for smaller-scale experiments). Miles represents the standardization of what was previously ad-hoc infrastructure—analogous to how PyTorch itself standardized deep learning training a decade ago.

Chapter 12

LLM Agentic Training

Training a model to perform well on single-turn question answering is a solved problem. Training it to act as a reliable autonomous agent — executing multi-step plans, recovering from tool failures, managing long-horizon context, and making calibrated decisions under uncertainty — is not. This chapter covers the training techniques that bridge that gap: trajectory-level RL with environment rewards, agentic fine-tuning data pipelines, the major algorithmic approaches (SFT on expert trajectories, online RL, filtered behavior cloning), and harness distillation, which bakes specialized scaffolding into model weights so runtime overhead can be eliminated at deployment.

GAS Connection — LLM Agentic Training

The following chapter describes the path from the rule-based GAS prototype to a fully learned agentic system. The techniques here—online RL with environment feedback, tool-use training, trajectory optimisation—are the roadmap for GAS v2.

Trajectory collection. The GAS training pipeline collects full agent trajectories: sequences of (patient state, agent invocation, agent output, next state, reward) tuples. A trajectory spans the full pipeline from glucose anomaly detection to caregiver alert, involving 4–5 agent invocations. The `GASState` TypedDict in `capstone_gas/gas_system.py` captures all fields needed to reconstruct a trajectory for offline analysis.

Online vs. offline paradigms (§ Operational Paradigms, § Paradigm Comparison). The current GAS uses offline training (batch updates on accumulated feedback from the chapter notebooks in `part2_rl_methods/`). GAS v2 would use online training: the Supervisor policy is updated in real time as patient outcomes are observed, using the `CGMSimulator` in `shared/patient_data.py` as a fast-feedback environment.

Tool-use training (§ Major Techniques in Agentic RL). The Advisor must learn *when* to query the knowledge base and *what* query to use. The `GASKnowledgeBase` in `capstone_gas/gas_system.py` uses FAISS (`faiss.IndexFlatIP`) with `sentence-transformers/all-MiniLM-L6-v2` embeddings and a keyword fallback. Tool-use training would teach the Advisor to formulate queries that retrieve the most relevant of the 15 ADA guideline entries in `MEDICAL_KNOWLEDGE`.

Interactive RL environments (§ Interactive RL Environments). The GAS training environment is the `CGMSimulator` in `shared/patient_data.py`, which generates realistic glucose traces for five clinical scenarios at 5-minute intervals in mg/dL. This simulator provides the fast-feedback environment needed for online RL training—each simulated episode covers a full day of glucose monitoring.

Research agent use case (§ Use Case: Building a Research Agent). The GAS Advisor can be seen as a domain-specific research agent: it retrieves evidence from the `GASKnowledgeBase`, synthesises it, and produces a grounded recommendation. The agentic RL training recipe in this section—start with SFT on expert demonstrations, then use RL to optimise for task success—is exactly the GAS training pipeline demonstrated in `part2_rl_methods/`.

12.1 Motivation: From Chatbots to Autonomous Agents

Modern LLMs are increasingly deployed not just as conversational assistants but as **autonomous agents** that interact with external tools, APIs, databases, and environments over multiple steps. This shift—from single-turn chatbots to multi-step agents—introduces fundamentally new RL challenges that require rethinking how we train, evaluate, and deploy language models.

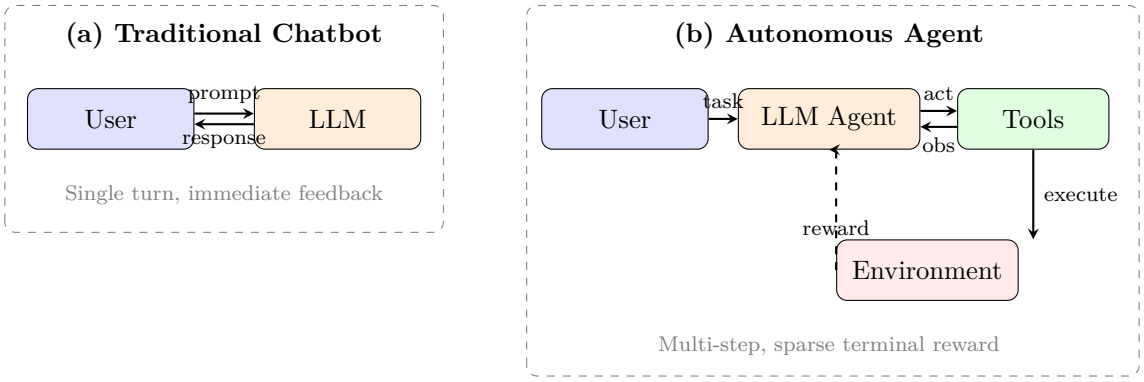


Figure 12.1: From Chatbots to Autonomous Agents: Traditional LLM chatbots operate in a single-step conversational loop with immediate human feedback. Autonomous agents plan across multiple tool interactions, receive feedback from real-world execution environments, and optimize for sparse terminal rewards (task success/failure).

The key differences that demand new RL approaches:

- **Multi-step reasoning**: Agents must plan across 10–100+ tool calls, not just generate a single response.
- **External environment feedback**: Rewards come from real-world execution (test suites pass, web pages load, code compiles) — not just human preference scores.
- **Structured actions**: Actions are not just tokens but structured outputs (JSON tool calls, API payloads, code blocks).
- **Long horizons with sparse rewards**: Success/failure may only be determined after many intermediate steps.

Why Standard RLHF Falls Short for Agents

Standard RLHF (PPO/DPO) optimizes for single-turn quality: given a prompt, produce a good response. But agents must:

- Decide *when* to use tools vs. reason internally
- Recover from errors mid-trajectory (self-correction)
- Balance exploration (try new approaches) with exploitation (use known-good patterns)
- Handle partial observability (tool outputs may be incomplete or noisy)

This requires training methods that reason over **entire trajectories**, not individual turns.

12.2 Trajectory Buffers for LLM Agents

In the context of LLM agents, traditional RL replay buffers undergo a structural transformation. Instead of storing low-dimensional numerical tensors, agentic buffers — often called **Trajectory Buffers**, **Experience Pools**, or **Memory Banks** — manage complex textual histories, tool execution outputs, and explicit reasoning steps.

12.2.1 Mathematical Structure of an LLM Agent Buffer

In classic RL, a replay buffer stores a flat tuple (s, a, r, s'). For an LLM agent, this expands into high-dimensional tokenized text structures:

$$\boxed{e_t = (\mathcal{S}_t,\ \mathcal{A}_t,\ \mathcal{R}_t,\ \mathcal{S}_{t+1})} \tag{12.1}$$

- $\mathcal{S}_t$: The **complete context state** — system prompt, user objective, conversation history, and current environmental variables (e.g., HTML source code, directory structures, database schemas).
- $\mathcal{A}_t$: The agent's **generated output**, typically composed of a Chain-of-Thought (CoT) reasoning string followed by a structured tool call:

$$\mathcal{A}_t = \{\text{text}_{\text{reasoning}},\ \text{json}_{\text{tool_call}}\} \tag{12.2}$$

- $\mathcal{R}_t$: **Evaluation signals** derived from external execution environments (unit test passes, compiler flags, API response codes) or verified by an LLM-as-a-judge system.
- $\mathcal{S}_{t+1}$: The **updated context window**, which appends tool output text or error logs directly into the conversation history.

Concrete Agent Trajectory: Code Debugging

Step 1: $\mathcal{S}_1$ = "Fix the failing test in `utils.py`"

$\mathcal{A}_1$ = *"Let me read the file first"* + `read_file("utils.py")`

$\mathcal{R}_1 = 0$ (intermediate step)

Step 2: $\mathcal{S}_2$ = [previous context + file contents]

$\mathcal{A}_2$ = *"The bug is on line 42, off-by-one error"* + `edit_file("utils.py", ...)`

$\mathcal{R}_2 = 0$ (intermediate step)

Step 3: $\mathcal{S}_3$ = [previous context + edit confirmation]

$\mathcal{A}_3$ = *"Let me verify the fix"* + `run_tests()`

$\mathcal{R}_3 = +1.0$ (all tests pass — sparse terminal reward)

12.3 Operational Paradigms

LLM agents leverage specialized trajectory buffers through three primary optimization methodologies:

12.3.1 A. Self-Correction and Thought Refinement

Two representative methods in this category are STaR [441] and Reflexion [344]. When an agent fails a multi-step execution trace, the sub-optimal sequence is saved to the buffer. The framework later samples this trajectory and prompts the LLM to generate an explicit textual critique of its past performance:

$$\text{Critique} \leftarrow \text{LLM}(\mathcal{S}_{\text{failed}},\ \mathcal{A}_{\text{failed}},\ \mathcal{R}_{=0}) \tag{12.3}$$

Once a corrected trajectory achieves a positive reward, it is moved to an optimal experience pool used to update the network weights via fine-tuning (SFT on successful trajectories) or RL (GRPO [341] with binary pass/fail rewards).

STaR: Self-Taught Reasoner

1. Generate reasoning traces for a batch of problems
2. Filter: keep only traces that lead to correct answers
3. Fine-tune the model on successful traces (SFT)
4. Repeat: the improved model generates better traces in the next iteration

Each iteration bootstraps the model's reasoning ability using its own successful outputs as training data.

Reflexion: Verbal Reinforcement Learning

1. Agent attempts a task, fails
2. Agent generates a **verbal reflection**: "I failed because I didn't check the return type before calling the API"
3. Reflection is stored in an episodic memory buffer
4. On the next attempt, reflections are injected into the prompt as lessons learned
5. No weight updates needed — pure in-context learning from self-critique

12.3.2 B. Off-Policy Exploration

This paradigm, exemplified by ReAct [430] and related tool-use frameworks, involves extensive autonomous exploration. During autonomous exploration (web navigation, database querying, code generation), agents log thousands of exploratory execution paths. The trajectory buffer acts as a filter:

- **Success filtering**: Only trajectories achieving the goal are kept for training.
- **Efficiency ranking**: Among successful traces, prefer the shortest/most efficient tool-use paths.
- **Diversity sampling**: Maintain a diverse set of solution strategies to prevent mode collapse.

The optimization algorithm (typically GRPO [341] or filtered SFT) computes losses exclusively over efficient, successful trajectories while discarding meandering runs.

12.3.3 C. Non-Parametric In-Context Learning (RAG over Experiences)

Instead of modifying neural network weights, the trajectory buffer can function as a **vector database**. Given a new user goal $\mathcal{G}_{\text{new}}$, the system retrieves the most relevant past experiences:

$$\boxed{\mathcal{E}_{\text{retrieved}} = \arg\max_{e \in \mathcal{B}} \text{sim}(\text{Embed}(\mathcal{G}_{\text{new}}),\ \text{Embed}(e))} \tag{12.4}$$

The top-k similar successful historical runs are injected directly into the prompt context as few-shot demonstrations. This approach:

- Requires **zero training** — pure retrieval-augmented generation
- Adapts instantly to new tasks if similar experiences exist in the buffer
- Scales with buffer size (more experiences = better coverage)
- Complements parametric learning (use retrieval for rare cases, weights for common patterns)

Table 12.1: Traditional RL Buffers vs. LLM Agent Buffers

Feature	Traditional RL Buffer	LLM Agent Buffer
Data Format	Continuous vectors / tensors	Tokenized text, JSON, code blocks, tool outputs
Data Volume	Massive (10^5–10^7 items)	Small to medium (10^3–10^5 traces)
Primary Goal	Breaking data correlation	Providing reasoning demonstrations
Sampling	Random uniform / PER	Semantic retrieval / success priority / diversity
State Size	Fixed (e.g., 84×84 pixels)	Variable (1K–128K tokens per state)
Action Space	Discrete/continuous vectors	Structured text (reasoning + tool calls)
Reward Source	Environment simulator	External execution / LLM judge / unit tests

12.4 Paradigm Comparison

12.5 Major Techniques in Agentic RL

Table 12.2: Key methods for training LLM agents with RL.

Method	Type	Key Idea
STaR [441]	Iterative SFT	Bootstrap reasoning by fine-tuning on own successful traces
Reflexion [344]	In-context RL	Verbal self-critique stored as episodic memory; no weight updates
ReAct [430]	Prompting	Interleave reasoning ("think") and acting ("tool call") in a single generation
LATS [461]	Tree search	Monte Carlo Tree Search over action sequences; backpropagate rewards
AgentQ [311]	Off-policy RL	DPO on agent trajectories with AI-generated preference pairs
OpenHands [387]	GRPO	Group-relative optimization with execution-based rewards (tests pass/fail)
Voyager [383]	Skill library	Successful code snippets stored and retrieved for compositional reuse
RLEF [211]	Online RL	RL from Execution Feedback — binary reward from code/test execution

12.5.1 STaR: Self-Taught Reasoner (Detailed)

STaR [441] is an **iterative self-improvement** method that bootstraps reasoning capabilities without external reward models. The core insight: if the model can occasionally solve a problem correctly, it can learn from its own successes.

Algorithm:

1. **Generate**: For each problem x_i in dataset $\mathcal{D}$, sample a reasoning trace $z_i \sim \pi_\theta(\cdot|x_i)$ followed by an answer $\hat{y}_i$.
2. **Filter**: Keep only traces where $\hat{y}_i = y_i^*$ (correct answer). Define success set $\mathcal{D}_{\text{pass}} = \{(x_i, z_i, y_i^*) : \hat{y}_i = y_i^*\}$.
3. **Rationalization** (key innovation): For problems where the model failed, generate a "rationalization" — a trace conditioned on the correct answer: $z_i^{\text{rat}} \sim \pi_\theta(\cdot|x_i, y_i^*)$. This teaches the model to reason *backward* from solutions.
4. **Fine-tune**: Update θ via SFT on $\mathcal{D}_{\text{pass}} \cup \mathcal{D}_{\text{rationalized}}$.
5. **Iterate**: Repeat from step 1 with the improved model.

$$\theta_{k+1} = \arg\min_{\theta} - \sum_{(x,z,y)\in\mathcal{D}_k^+} \log \pi_\theta(z, y|x) \tag{12.5}$$

Convergence dynamics: Each iteration k increases the model's solve rate p_k. If $p_0 = 0.3$ (solves 30% of problems), after rationalization + SFT, $p_1 \approx 0.5$. Typically converges in 3–5 iterations to $p \approx 0.7$–0.9.

STaR Rationalization Prompt

```
# Standard generation (Step 1):
PROMPT = """Solve the following problem step by step.
Problem: A store has 45 apples. It sells 3/5 of them. How many remain?
Let's think step by step:"""

# Rationalization prompt (Step 3 - conditioned on correct answer):
PROMPT_RATIONALIZE = """Solve the following problem step by step.
The correct answer is 18.
Problem: A store has 45 apples. It sells 3/5 of them. How many remain?
Let's think step by step to arrive at 18:"""

# Agent variant (code task with error conditioning):
PROMPT_AGENT_RATIONALIZE = """The following code task failed with the error
    below.
Generate a correct solution step by step.

Task: Implement binary search that handles duplicates.
Previous error: IndexError: list index out of range (line 12)
Correct behavior: Return leftmost index of target.

Let me fix this by reasoning about the boundary conditions:"""
```

STaR Variants for Agents

- **Quiet-STaR** [442]: Inserts "thinking tokens" between every token of generation. The model learns to reason *implicitly* without explicit CoT prompting. Training objective: predict next tokens better when thinking tokens are included.
- **STaR for Code Agents**: Replace answer verification with test execution. "Correct" = all tests pass. Rationalization = generate a new approach conditioned on the error message.
- **V-STaR** [153]: Adds a verifier model trained on $(z, y, \text{correct/incorrect})$ triples. The verifier provides process-level supervision, filtering bad reasoning traces that accidentally reach correct answers.

12.5.2 Reflexion: Verbal Reinforcement Learning (Detailed)

Reflexion [344] introduces a radical paradigm: **RL without weight updates**. Instead of gradient-based learning, the agent improves through natural-language self-critique stored in an episodic memory.

Full Architecture:

1. **Actor**: The LLM agent π that executes actions in the environment.
2. **Evaluator**: A binary signal (task success/failure) or a scalar heuristic (e.g., number of test cases passed).
3. **Self-Reflection Generator**: Given the failed trajectory τ_{fail} and environment feedback, generates a natural-language reflection r_{text}:

$$r_{\text{text}} = \text{LLM}_{\text{reflect}}(\tau_{\text{fail}}, \text{feedback}, \text{task}) \tag{12.6}$$

4. **Episodic Memory**: A sliding window buffer $\mathcal{M} = [r_1, r_2, \ldots, r_m]$ of past reflections (typically $m \leq 3$ to fit in context).

5. **Retry Loop**: On the next attempt, reflections are injected into the prompt:

$$a_{t+1} \sim \pi(\cdot \mid \text{task}, \ \mathcal{M}, \ \text{current_state}) \tag{12.7}$$

Example reflection: *"In my previous attempt, I called the search API before validating the input format, which caused a 400 error. Next time, I should validate the JSON schema first, then make the API call."*

Reflexion: Agent Prompt with Injected Memory

```
# === ATTEMPT 2 PROMPT (after first failure) ===

SYSTEM = """You are a coding agent. You can run bash commands and edit files.
Complete the task below. Learn from your previous reflections."""

USER = """Task: Fix the failing test in auth_service.py

=== REFLECTIONS FROM PREVIOUS ATTEMPTS ===
[Attempt 1 reflection]: I tried to modify the authenticate() function
directly but forgot that it depends on token_validator(). The test
failed because token_validator() was still returning the old format.
I should trace the dependency chain FIRST: check what authenticate()
calls, then fix the root cause (token_validator), not the symptom.
=== END REFLECTIONS ===

The repository is in /workspace/. The failing test is:
  test_auth.py::test_expired_token_returns_401

Begin by reading the relevant files, then fix the issue."""
```

Strengths and Limitations:

Strengths	Limitations
Zero gradient computation; works with frozen API models (GPT-4)	Limited to context window; can't accumulate infinite knowledge
Fast iteration (seconds per retry vs. hours for RL training)	No generalization to unseen tasks (memory is task-specific)
Interpretable: human-readable self-corrections	Relies on the model's existing ability to identify errors
Composes with any base agent architecture	Degrades when base model is too weak to generate useful critiques

12.5.3 ReAct: Reasoning + Acting (Detailed)

ReAct [430] establishes the dominant prompting paradigm for tool-using agents by **interleaving explicit reasoning steps with environment actions** in a single generation stream.

ReAct System Prompt (Typical)

```
SYSTEM = """You are a research assistant with access to the following tools:
- search(query): Search the web for information
- lookup(term): Look up a term on the current Wikipedia page
- finish(answer): Submit your final answer

Solve the user's question by interleaving Thought and Action steps.
Format:
  Thought: <your reasoning about what to do next>
  Action: <tool_name>(arguments)

You will receive an Observation after each Action. Use it to plan
```

```
your next step. Continue until you can provide a final answer.

Rules:
- Always think before acting
- Never guess - verify with tools
- Use finish() only when you are confident in your answer"""

USER = "Who was the president of the US when the iPhone was released?"
```

Generation format:

ReAct Trace Format

```
Thought: I need to find when the iPhone was released, then determine
who was president at that time. Let me search for the release date.
Action: search("iPhone original release date")
Observation: The original iPhone was released on June 29, 2007.
Thought: The iPhone came out June 2007. Now I need to find who was
the US president in June 2007.
Action: lookup("President of the United States in 2007")
Observation: George W. Bush served as the 43rd president from
January 20, 2001 to January 20, 2009.
Thought: George W. Bush was president from 2001-2009, which covers
June 2007 when the iPhone was released. I have my answer.
Action: finish("George W. Bush was the US president when the iPhone
was released on June 29, 2007.")
```

Formal definition: A ReAct trajectory is $\tau = (t_1, a_1, o_1, t_2, a_2, o_2, \ldots)$ where:

- t_i: Thought (internal reasoning, not executed)
- a_i: Action (tool call, executed in environment)
- o_i: Observation (environment response, appended to context)

Why it works: Thoughts create an "inner monologue" that helps the model plan before acting, reducing impulsive tool calls. The explicit reasoning trace also makes the agent's decision process **auditable** and **debuggable**.

Training ReAct agents with RL:

- **Action-level rewards**: Only actions receive reward signals (thoughts are auxiliary).
- **Thought quality**: Implicitly optimized — better thoughts → better actions → higher rewards.
- **Format enforcement**: Include format penalties in the reward for malformed actions (missing JSON, hallucinated tools).
- **RL objective**: $r(\tau) = r_{\text{task}} - \lambda_{\text{format}} \cdot \text{format_violations} - \lambda_{\text{length}} \cdot \text{num_steps}$

12.5.4 LATS: Language Agent Tree Search (Detailed)

LATS [461] applies **Monte Carlo Tree Search (MCTS)** to LLM agent action selection, trading inference compute for significantly better trajectories.

Algorithm (adapted for LLM agents):

1. **Selection**: Starting from root (initial state), traverse the tree using UCB1:

$$\text{UCB}(s, a) = \bar{Q}(s, a) + c\sqrt{\frac{\ln N(s)}{N(s, a)}} \tag{12.8}$$

where $\bar{Q}$ = average reward of subtree, N = visit counts, c = exploration constant.

2. **Expansion**: At a leaf node, generate k candidate actions via LLM sampling (temperature > 0): $\{a_1, \ldots, a_k\} \sim \pi_\theta(\cdot|s_{\text{leaf}})$

3. **Simulation**: For each candidate, execute the action in the environment and continue with a fast rollout policy (greedy decoding) until terminal state or depth limit.

4. **Backpropagation**: Propagate the terminal reward up through all ancestor nodes, updating $\bar{Q}$ and N counts.

5. **Repeat**: Run steps 1–4 for a fixed computation budget (e.g., 50–200 iterations).

6. **Action selection**: Choose the most-visited child of the root.

LLM-specific adaptations:

- **Value function**: Use a separate LLM call to estimate state value: "On a scale of 0–1, how likely is this state to lead to task success?"
- **Reflection-based pruning**: When a branch fails, generate a reflection and prune similar branches.
- **Caching**: Store LLM outputs at each node to avoid redundant generation during backtracking.
- **Depth budget**: Limit tree depth to 10–20 steps (agents rarely need more).

Performance: On WebShop (web navigation), LATS achieves 75% success vs. ReAct's 40%. On HumanEval (code), pass@1 improves from 68% → 94% with tree search. The cost: 10–50× more inference FLOPs per task.

LATS Prompts: Value Estimation and Node Expansion

```
# === VALUE ESTIMATION PROMPT (used during simulation) ===
VALUE_PROMPT = """You are evaluating an agent's progress on a task.

Task: Book a flight from NYC to London for under \$500, departing Dec 15.

Current state (after 3 actions):
- Searched flights on Kayak: found 12 results
- Filtered by price < \$500: 4 options remain
- Clicked on British Airways \$489 option: viewing details page

On a scale of 0.0 to 1.0, how likely is the agent to successfully
complete the task from this state? Consider:
- How close is the agent to the goal?
- Are there remaining obstacles (payment, seat selection)?
- Has the agent made any errors that need correction?

Score: """  # Model outputs e.g. "0.75"

# === NODE EXPANSION PROMPT (generating candidate actions) ===
EXPAND_PROMPT = """You are a web navigation agent. Given the current
page state, propose 3 DIFFERENT next actions to try.

Current page: British Airways booking - flight details
  Price: \$489 | Departure: Dec 15 8:30am | Arrival: Dec 15 8:45pm
  [Button: Select] [Button: Back to results] [Link: Fare rules]

Generate 3 diverse candidate actions (explore different strategies):
Action 1:"""  # Model generates 3 options for tree expansion
```

12.5.5 AgentQ: DPO on Agent Trajectories (Detailed)

AgentQ [311] bridges **offline preference learning (DPO)** with **online agent execution** by automatically generating preference pairs from trajectory outcomes.

Pipeline:

1. **Rollout**: Execute N trajectories per task using the current policy π_θ.
2. **Evaluate**: Score each trajectory with execution-based reward (binary pass/fail or scalar metric).
3. **Pair construction**: For each task, construct preference pairs:

$$(\tau_w, \tau_l) \text{ where } r(\tau_w) > r(\tau_l) \tag{12.9}$$

   Among trajectories for the same task, the one with highest reward = chosen; lowest = rejected.
4. **DPO update**: Apply standard DPO loss over trajectory-level log-probabilities:

$$\mathcal{L}_{\text{AgentQ}} = -\log \sigma\left(\beta \left[\log \frac{\pi_\theta(\tau_w)}{\pi_{\text{ref}}(\tau_w)} - \log \frac{\pi_\theta(\tau_l)}{\pi_{\text{ref}}(\tau_l)}\right]\right) \tag{12.10}$$

5. **Iterate**: Updated π_θ generates new (better) trajectories in the next round.

Key design choices:

- **MCTS-guided exploration**: Use LATS during rollout phase to generate diverse, high-quality trajectories (better training data).
- **Step-level DPO**: Instead of comparing full trajectories, compare at the *action level* — given the same prefix, which next action leads to success?
- **Self-play improvement**: Each DPO iteration produces a better policy that generates better trajectories that produce better training pairs — a virtuous cycle.

Results: On WebShop, AgentQ achieves absolute 50% → 82% success rate improvement over the base policy in 3 DPO iterations.

12.5.6 Voyager: Lifelong Learning via Skill Libraries (Detailed)

Voyager [383] introduces **compositional skill accumulation** — the agent builds a growing library of reusable code functions that serve as high-level actions.

Architecture:

1. **Automatic Curriculum**: An LLM proposes progressively harder tasks based on the agent's current skill inventory: "You can now mine wood and craft planks. Next challenge: build a crafting table."
2. **Skill Generation**: For each task, the agent writes a JavaScript function (executable code) that solves it:

$$\text{skill}_i = \text{LLM}(\text{task}_i, \text{environment_docs}, \text{error_feedback}) \tag{12.11}$$

3. **Verification**: Execute the code in the environment. If it succeeds, add to the skill library. If not, iterate with error feedback (up to 5 retries).
4. **Skill Library** (vector DB): Each verified skill stored with:
   - Function signature + docstring (for retrieval)
   - Embedding of the task description (for semantic search)

- Dependencies (which other skills it calls)

5. **Retrieval + Composition**: For new tasks, retrieve the top-k most relevant skills and compose them:

$$\text{solution} = \text{LLM}(\text{new_task}, \text{retrieve}(\text{skill_library}, k{=}5)) \tag{12.12}$$

Key insight: Skills are **compositional** — complex behaviors emerge from combining simple verified functions. The agent never forgets (library is persistent) and improves monotonically (only verified skills are added).

Voyager: Curriculum and Skill Generation Prompts

```
# === AUTOMATIC CURRICULUM PROMPT ===
CURRICULUM_PROMPT = """You are a curriculum designer for an AI agent.

Agent's current skill inventory:
- mine_wood(): Mines nearby oak/birch trees
- craft_planks(): Converts logs to planks
- craft_sticks(): Converts planks to sticks
- mine_stone(): Mines stone with wooden pickaxe

Propose the next task that:
1. Builds on existing skills (reachable from current abilities)
2. Introduces exactly ONE new concept or challenge
3. Is concrete and verifiable (clear success condition)

Next task proposal:"""
# Output: "Craft a furnace (requires 8 cobblestone blocks arranged
#          in a square). You already know mine_stone()."

# === SKILL GENERATION PROMPT ===
SKILL_GEN_PROMPT = """Write a JavaScript function to accomplish this task
in Minecraft. Use the bot API (bot.dig, bot.craft, bot.equip, etc.)

Task: Smelt 5 iron ingots using a furnace.
Prerequisites available: mine_stone(), craft_furnace(), mine_iron_ore()

Error from previous attempt: "Cannot smelt without fuel in furnace"

Write the corrected function:
async function smeltIronIngots(bot, count=5) {"""
```

12.5.7 RLEF: RL from Execution Feedback (Detailed)

RLEF [211] applies **online RL with deterministic execution-based rewards** to code generation agents, establishing the simplest effective paradigm for agentic training.

Training loop:

1. **Sample task**: Draw a coding problem with test cases (x, tests) from the training set.
2. **Generate**: The agent produces a solution trajectory (reading files, writing code, running tests) using the current policy π_θ.
3. **Execute**: Run the test suite in a sandboxed environment. Reward:

$$r = \frac{\#\text{ tests passed}}{\#\text{ total tests}} \in [0, 1] \tag{12.13}$$

4. **Update**: Apply GRPO/PPO using r as the reward signal.
5. **Repeat**: Thousands of iterations with fresh tasks.

Why execution feedback is ideal for RL:

- **Zero noise**: Unlike human preferences, test results are deterministic. Same code → same reward every time. This eliminates reward noise that destabilizes RL training.
- **Infinite scale**: Can generate unlimited tasks programmatically (random algorithms, API integration tests, data transformations).
- **No reward hacking**: Unlike learned reward models, a test suite can't be "fooled" (assuming tests are well-written). The agent must actually solve the problem.
- **Dense signal**: Partial test passage ($r = 0.6$) provides richer gradient than binary pass/fail.

12.5.8 OpenHands / SWE-Agent: GRPO for Software Engineering

OpenHands [387] and SWE-Agent [426] apply GRPO to train agents that autonomously resolve GitHub issues — reading code, writing patches, and running test suites.

Training specifics:

- **Environment**: Docker container with full repo, test suite, and developer tools (git, grep, lint).
- **Action space**: Bash commands, file edits, git operations, test execution.
- **Trajectory length**: 15–50 actions typical for resolving a GitHub issue.
- **Reward**: Binary — does the generated patch pass the issue's regression tests?
- **Group size**: $N = 8$–16 trajectories per issue for GRPO normalization.
- **Curriculum**: Start with issues labeled "good first issue", progress to complex multi-file refactors.

State-of-the-art results: SWE-bench Verified: 30% → 55% resolve rate after RL training (vs. SFT-only baseline).

OpenHands / SWE-Agent: System Prompt

```
SYSTEM = """You are an autonomous software engineer. You are given a
GitHub issue to resolve. You have access to the full repository in
/workspace/ and can execute any bash command.

AVAILABLE COMMANDS:
- bash(command): Execute a shell command
- edit(file, start_line, end_line, new_content): Edit a file
- search(pattern, path): Search for text in files
- submit(): Submit your patch when done

WORKFLOW:
1. Read the issue carefully and understand the expected behavior
2. Explore the codebase to find relevant files
3. Reproduce the bug (write/run a test that fails)
4. Implement the fix
5. Verify the fix (run the test again - must pass)
6. Run the full test suite to check for regressions
7. Submit when all tests pass

RULES:
- Do NOT modify test files unless the issue explicitly asks for it
- Prefer minimal, targeted changes over large refactors
- Always verify your fix before submitting"""

USER = """GitHub Issue #4521: `DataFrame.merge()` silently drops
rows when `on` column contains NaN values.
```

```
Expected: NaN keys should be preserved (matched with other NaN rows)
Actual: Rows with NaN keys are dropped entirely

Repository: /workspace/pandas-dev/pandas/"""
```

The Future: RL + Agents

The field is converging on a pattern: **online RL with execution-based rewards** applied to multi-step agent trajectories. Key trends:

- GRPO/PPO with binary pass/fail rewards from code execution or tool success
- Curriculum learning: start with easy tasks, progressively increase difficulty
- Trajectory-level optimization (not token-level) — reward only at the end of a multi-step sequence
- Hybrid approaches: use retrieval (non-parametric) for rare tasks + RL (parametric) for common ones
- Scaling law: more compute at inference (search/retry) often beats more training compute

12.6 Harness Distillation: Baking Scaffolding into Weights

Every LLM agent consists of two distinct layers: the **model weights** and the **harness** — the external runtime scaffolding that provides tools, manages context and state, enforces permissions, and orchestrates multi-step workflows. Harness engineering alone can shift agent performance by up to 6× on a fixed model [433]. Yet these gains are *deployment-coupled*: a production system must either accept a suboptimal shared harness or maintain a growing library of domain-specific harnesses, neither of which transfers the harness's learned behaviors into the model itself.

Harness-Zero [433] asks: *can the behavioral gains of a specialized harness be baked into model weights, so the harness can be removed at deployment?*

12.6.1 Key Concepts

Agent harness. Following the DeepAgents abstraction [433], a harness has four components: **tools** (extend the model's action space), **middleware** (inspect/modify/block the execution loop), **skills** (workflow instructions surfaced to the model), and **memory** (accumulated domain knowledge and failure patterns).

Harness distillation. Using a domain-optimized harness as training-time guidance and transferring the behaviors it induces into model weights, so that its gains survive under a single fixed *target* harness h at deployment [433]. After distillation, the specialized harness is discarded entirely.

The action-space mismatch problem. The naive approach — wrapping the student directly with the optimized harness h^* during training — fails because h^* exposes tools and actions unavailable under the deployment harness h. Models fine-tuned on h^* trajectories attempt unavailable tool calls at deployment, exhausting their turn budget. Ablations in [433] confirm: fine-tuning on student trajectories collected under h^* yields *no improvement* over the base model (12% → 12% on USPTO retrosynthesis).

12.6.2 Agent-as-Harness

The core mechanism solving the action-space mismatch is **Agent-as-Harness** [433]: instead of the optimized harness wrapping the student as code, a **harnessing agent** (a separate LLM) wraps the

student at its *response boundary*.

At each turn t:

1. **Student** π_S proposes a response $y_t \sim \pi^h_{\theta_0}(\cdot \mid c_t)$ under the minimal target harness h
2. **Harnessing agent** π_H reviews: $(d_t, z_t) \sim \pi_H(\cdot \mid c_t, y_t, \mathcal{K}, r_{<t})$ where $d_t \in \{\text{pass}, \text{replace}\}$ and z_t is a corrected response
3. **Accepted response**: $\tilde{y}_t = y_t$ (if pass) or z_t (if replace)
4. **Transition**: $c_{t+1} = T_h(c_t, \tilde{y}_t)$ — executed through h, result enters student context

The harnessing agent consults a **private reference harness** $\mathcal{K}$ — an adaptation of h^* for reviewing rather than wrapping. Each component of h^* is translated:

Table 12.3: Translating the optimized harness h^* into the private reference harness $\mathcal{K}$.

h^* **component**	$\mathcal{K}$ **counterpart**
Tool (extends action space)	Action recipe (how to construct equivalent student-native action through h)
Middleware (modify/block loop)	Review middleware (detect condition, privately alert harnessing agent)
Skill (workflow instructions)	Review guidance (diagnose current step, construct replacement)
Memory (failure patterns)	Failure patterns (recognize failure condition, specify intervention)

Critically: $\mathcal{K}$ is read-only and invisible to the student. The harnessing agent makes the *smallest coherent correction* needed, and every replacement must be a complete, valid response under h. The same model can play both student and harnessing roles — gains cannot come from model size difference alone.

12.6.3 Three-Stage Training Pipeline

1. **Harness evolution**: Run automated harness optimization (e.g., Meta-Harness) on training tasks for 3 rounds. Analysis agents examine failures and propose reusable updates to tools, middleware, skills, and memory. Result: h^*, the domain-optimized harness.
2. **Trajectory collection**: Deploy Agent-as-Harness to collect reviewed trajectories $\mathcal{D}_{\text{review}}$. The student operates under h; the harnessing agent corrects responses using $\mathcal{K}$. Only accepted responses enter the student-visible trajectory.
3. **SFT on reviewed trajectories**: Fine-tune the student on accepted responses:

$$\hat{\theta} = \arg\min_{\theta} - \sum_{\tau \in \mathcal{D}_{\text{review}}} \sum_{t=1}^{T_\tau} \log \pi^h_\theta(\tilde{y}_t \mid c_t) \tag{12.14}$$

Reviewer-perspective reasoning inadvertently included in replacements is masked from the loss.

Deployment: $(\pi_{\hat{\theta}}, h)$ — the distilled student under the minimal target harness alone. h^*, $\mathcal{K}$, and the harnessing agent are all discarded.

12.6.4 Results

Experiments distill Qwen3.5-9B (student) via GPT-5.6 Sol (harnessing agent) across three domains: SpreadsheetBench Verified (knowledge work), AppWorld (interactive tool use), and USPTO Retrosynthesis (science). Target harness h: minimal mini-SWE-agent with one Bash execution tool.

Key finding [433]: Harness-Zero raises macro-average task success from 23.3% to 44.3% (+90% relative). The distilled model under h alone (44.3%) *exceeds* the base model with h^* still attached

Table 12.4: Harness-Zero results. The distilled model under h alone outperforms the base model with the specialized harness h^* still attached.

Setting	SpreadsheetBench	AppWorld	USPTO	Avg
Base (h only)	31.0	26.8	12.0	23.3
Base + h^* (meta-harness)	39.0	48.2	38.0	41.7
DeepAgents (generic harness)	35.0	19.6	7.0	20.5
Claude Code	31.0	10.7	6.0	15.9
Harness-Zero (distilled, h only)	**44.0**	**58.9**	**30.0**	**44.3**

(41.7%). Generic harnesses (DeepAgents, Claude Code) actually *hurt* the 9B model — they present too many tools and too much context while missing domain-specific guidance.

Ablation (USPTO) [433]: Direct fine-tuning on stronger teacher rollouts yields 0% improvement. Trajectories collected under h^* hurt (action-space mismatch). Oracle answer access reaches only 15%. The procedural, reusable behavioral guidance in $\mathcal{K}$ is what drives the gain — not stronger demonstrations or oracle access.

Behavioral recovery [433]: 28 harness-exclusive behavioral patterns were identified (behaviors that appear under h^* but never under h in the base model). Harness-Zero recovers **82.3%** of these patterns on average, spanning all four harness component types (tools, middleware, skills, memory). Selected examples: "Inspect before editing" (100%), "Parse product with RDKit first" (100%), "Validate with RDKit" (100%), "Protect pre-filled cells" (92%).

Agent-as-Harness at inference time [433]: The same mechanism works without training as an inference-time augmentation. On frontier models (GPT-5.6 Sol, DeepSeek-V4-Pro), agent-as-harness with $\mathcal{K}$ achieves 81.1% vs. 78.1% for code-as-harness — the reviewing agent is more effective than the same harness mounted as code.

Harness Distillation vs. Knowledge Distillation

Standard knowledge distillation transfers *what the teacher knows* (soft logits or reasoning traces from a stronger model). Harness distillation transfers *how the agent should act within a structured tool-use loop* — procedural behaviors induced by external scaffolding. The ablation makes this concrete: fine-tuning on a stronger teacher's trajectories yields 0% improvement; harness-guided review yields +18 points. The value is in the harness knowledge, not the model knowledge.

Future Directions

Each harnessing agent intervention creates a natural (rejected, preferred) response pair at the same state, making the collected data directly suitable for preference learning (DPO/PPO). The authors flag this as future work. Other open directions: selective review (invoke the harnessing agent only when needed), fine-grained intervention (token insertion, latent-space steering), and recursive self-improvement (better models → better harnesses → better distillation → better models).

12.7 Use Case: Agentic RL for a Productivity Co-pilot

This section provides a complete blueprint for applying agentic RL techniques to train an LLM-based co-pilot that operates across a productivity application suite (documents, spreadsheets, presentations, email, messaging, cloud storage).

12.7.1 Architecture Overview

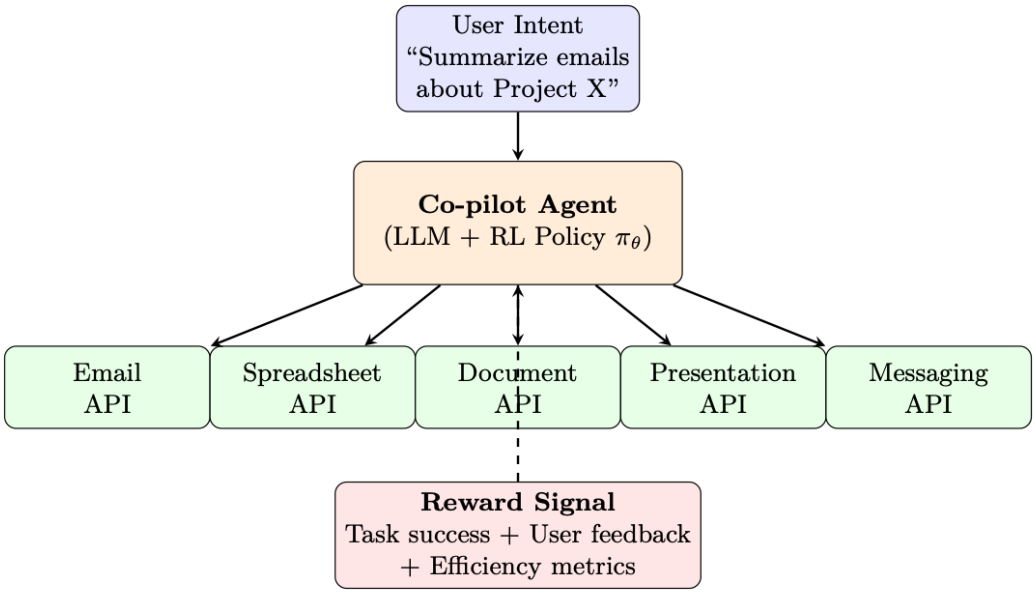


Figure 12.2: Productivity co-pilot architecture: the LLM agent (with RL policy π_θ) receives user intents and interacts with multiple application APIs. A reward signal based on task success, user feedback, and efficiency metrics drives policy improvement.

12.7.2 Formal MDP Definition for a Productivity Co-pilot

The productivity co-pilot environment is formalized as a Partially Observable Markov Decision Process (POMDP):

$$\boxed{\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle} \tag{12.15}$$

- $\mathcal{S}$: **State space** — Full workspace environment state: document contents, email threads, calendar events, file system, user permissions. *Not fully observable*: agent sees only what API queries return.
- $\mathcal{A}$: **Action space** — Structured API calls (see below). Each action is a JSON object specifying the target app, operation, and parameters.
- $\mathcal{T}$: **Transition function** — Deterministic for most operations (write to document $\rightarrow$ document updated), but stochastic for network-dependent actions (email delivery time, Teams availability).
- $\mathcal{R}$: **Reward function** — Multi-component (see Reward Design section).
- Ω: **Observation space** — API responses, rendered document views, error messages.
- $\mathcal{O}$: **Observation function** — Maps state to observation (API response formatting, truncation for context window limits).
- $\gamma = 0.99$: Discount factor (long horizons, 10–50 steps typical).

Concrete Example: "Summarize last week's Project Alpha emails and create a status slide"

Below we trace a full episode through the POMDP, mapping each formal element to a concrete realization.

User request: "Summarize the key decisions from last week's Project Alpha emails and add a status update slide to the team deck."

Symbol	Concrete Realization
s_0	*True state*: 47 emails in inbox mentioning "Project Alpha" from last 7 days; PowerPoint file "Q3_Status.pptx" on SharePoint with 12 slides; user has edit permissions.
o_0	*Agent observes*: User request text + system prompt. Agent has *no knowledge* of email count or deck contents yet.
a_1	`outlook.search({query: "Project Alpha", last_7_days: true})`
$\mathcal{T}(s_0, a_1)$	s_1: System retrieves 47 matching emails (deterministic).
o_1	API returns top 10 email subjects + senders + timestamps (truncated due to context limits — $\mathcal{O}$ in action).
a_2	`outlook.read({ids: [top_5_thread_ids]})` — Agent picks the most relevant threads.
o_2	Full body text of 5 email threads (∼8K tokens after $\mathcal{O}$ truncation).
a_3	*Internal reasoning action*: Agent synthesizes key decisions: (1) deadline moved to Nov 15, (2) budget approved, (3) vendor selected.
a_4	`sharepoint.read({file: "Q3_Status.pptx", slides: "last"})`
o_4	Last slide is "Q2 Summary" (slide 12). Agent decides to add slide 13.
a_5	`powerpoint.add_slide({file: "Q3_Status.pptx", position: 13, title: "Project Alpha -- Week 42 Status", content: "Key decisions: ..."})`
$\mathcal{T}(s_4, a_5)$	s_5: Slide added to deck (deterministic).
o_5	API returns `{success: true, slide_id: 13}`.
$R(s_5)$	Reward: **+0.4** task completion, **+0.3** information quality, **+0.2** format compliance, **+0.05** efficiency, **-0.0** safety. **Total: 0.95**.

Key POMDP aspects illustrated:

- **Partial observability**: At $t = 0$, the agent doesn't know how many emails exist or what the deck contains — it must query to discover the state.
- **Observation function $\mathcal{O}$**: The API returns truncated results (top 10 of 47 emails) due to context window limits. The agent sees a *projection* of the true state.
- **Stochastic transitions**: If the agent had tried `teams.send_message()` instead, delivery timing would be uncertain (recipient online/offline).
- **Multi-step planning**: The agent must chain 5 actions across 2 applications, maintaining coherence between the email summary and the slide content.
- **Discount $\gamma = 0.99$**: With 5 steps, discounting is minimal ($0.99^5 = 0.95$), but for 50-step tasks it matters — encouraging efficient solutions.

12.7.3 Action Space Design

The action space must be **structured, type-safe, and composable**:

Productivity Co-pilot Action Schema

```
{
  "action_type": "api_call",
  "target_app": "outlook | excel | word | powerpoint | teams | sharepoint",
  "operation": "read | write | search | create | delete | modify",
  "parameters": {
    "endpoint": "/me/messages?$filter=subject eq 'Project X'",
    "body": { ... },              // For write operations
    "options": { "top": 10 }      // Pagination, filtering
  },
  "reasoning": "I need to find relevant emails before summarizing"
}
```

Action taxonomy by application:

App	Complexity	Key Actions
Outlook	Medium	`search`, `read`, `draft`, `send`, `move`, `flag`, `create_rule`
Excel	High	`read_range`, `write_range`, `insert_formula`, `create_chart`, `pivot_table`, `run_macro`
Word	Medium	`read_paragraphs`, `insert_text`, `format_section`, `find_replace`, `insert_table`
PowerPoint	Medium	`add_slide`, `insert_shape`, `set_text`, `set_layout`, `add_image`, `apply_theme`
Teams	Low	`send_message`, `create_meeting`, `search_chat`, `add_members`, `post_to_channel`
SharePoint	Medium	`list_files`, `upload`, `download`, `search`, `create_page`, `set_permissions`

12.7.4 State Representation

The agent's observation (context window) at each step:

$$o_t = [\text{system_prompt};\ \text{user_intent};\ \text{tool_history}_{1:t-1};\ \text{current_result}_t] \tag{12.16}$$

Context budget management (critical for 128K window):

- **System prompt**: 2K tokens (capabilities, safety rules, output format)
- **User intent + conversation**: 4K tokens
- **Tool history** (sliding window): Last 8–12 actions + observations, summarizing older ones. Total: 80K tokens max.
- **Current observation**: Up to 32K tokens (large spreadsheets, email threads)
- **Reserve**: 10K tokens for agent's reasoning + next action generation

State compression strategies:

- **Selective inclusion**: Only include API responses relevant to the current sub-goal (use an auxiliary "relevance scorer").
- **Structured summaries**: Represent large spreadsheets as schema + sample rows rather than full data.

- **Hierarchical memory**: Store full trajectory externally; inject compressed summaries into context.

12.7.5 Reward Design: Multi-Objective Signal

The reward function for a productivity co-pilot must balance multiple objectives:

$$\boxed{R(\tau) = \alpha_1 R_{\text{task}} + \alpha_2 R_{\text{quality}} + \alpha_3 R_{\text{efficiency}} + \alpha_4 R_{\text{safety}} + \alpha_5 R_{\text{user}}} \tag{12.17}$$

Table 12.5: Reward components for productivity co-pilot training.

Component	Weight	Signal Type	Definition
R_{task}	0.40	Binary/scalar	Task completed successfully (email sent, document created, formula correct)
R_{quality}	0.25	LLM judge	Output quality: formatting, clarity, correctness of content
$R_{\text{efficiency}}$	0.15	Scalar	Penalty for excessive steps: $-0.02 \times$ (num_steps − optimal_steps)
R_{safety}	0.15	Binary	No unsafe actions (delete without confirmation, send to wrong recipient, permission violations). $R_{\text{safety}} = 0$ if any violation.
R_{user}	0.05	Sparse	Explicit user feedback (thumbs up/-down) when available

Intermediate rewards (dense signal):

- Successful API call (200 response): +0.05
- Correct information retrieval (verified by downstream use): +0.10
- Recovers from error gracefully (retries with corrected params): +0.08
- API error (4xx/5xx): –0.03
- Repeated identical action (loop detection): –0.10
- Asks clarifying question when intent is genuinely ambiguous: +0.05

12.7.6 Training Pipeline: End-to-End

Productivity Co-pilot RL Training Pipeline

Phase 1: Supervised Fine-Tuning (Foundation)

1. Collect 50K–200K human-demonstrated trajectories of productivity tasks (via telemetry, annotators, or synthetic generation).
2. SFT the base LLM on (instruction, trajectory) pairs with ReAct format.
3. Validate: agent should achieve 40–60% task completion on held-out tasks.

Phase 2: Simulated Environment Construction

1. Build a **sandbox environment** with mocked API endpoints, synthetic mailboxes, documents, and calendars.
2. Each "user" has a realistic profile: 500+ emails, 20+ documents, calendar events, Teams channels.

3. Task generator: produces diverse instruction–verification pairs: "Move all emails from Alice about Q4 budget to the 'Finance' folder" + verification function.

Phase 3: Online RL Training (GRPO)

1. Sample task batch (256 tasks per iteration).
2. Generate $N = 8$ trajectories per task using π_θ in sandbox environment.
3. Execute trajectories, collect rewards from verification functions.
4. Compute GRPO advantages (group normalization across 8 trajectories per task).
5. Update policy with clipped objective + KL penalty vs. SFT model.
6. Every 500 iterations: evaluate on held-out benchmark (200 tasks, 5 difficulty levels).

Phase 4: Human-in-the-Loop Refinement

1. Deploy to internal dogfood users (1000+ users, 2 weeks).
2. Collect thumbs up/down signals + free-text corrections.
3. Construct DPO preference pairs from A/B deployments (old policy vs. new).
4. Apply 1–2 rounds of DPO fine-tuning on human preferences.

12.7.7 Simulation Environment Architecture

Sandbox Environment (Simplified)

```
class ProductivityEnvironment:
    def __init__(self, user_profile: UserProfile):
        self.mailbox = SyntheticMailbox(user_profile.emails)
        self.drive = SyntheticOneDrive(user_profile.files)
        self.calendar = SyntheticCalendar(user_profile.events)
        self.teams = SyntheticTeams(user_profile.channels)
        self.step_count = 0
        self.max_steps = 50

    def step(self, action: dict) -> Tuple[Observation, float, bool]:
        """Execute action, return (observation, reward, done)."""
        self.step_count += 1

        # Route to appropriate app handler
        handler = self.get_handler(action["target_app"])
        try:
            result = handler.execute(action["operation"], action["parameters"])
            obs = Observation(status=200, body=result)
            reward = 0.05  # Successful API call
        except APIError as e:
            obs = Observation(status=e.code, body=str(e))
            reward = -0.03

        # Check terminal condition
        done = self.step_count >= self.max_steps
        return obs, reward, done

    def evaluate(self, task: Task) -> float:
        """Check if task objective is achieved (terminal reward)."""
        return task.verification_fn(self)  # 0.0 or 1.0
```

12.7.8 Task Curriculum Design

Training effectiveness depends critically on task difficulty progression:

Table 12.6: Productivity co-pilot curriculum levels.

Level	Steps	Apps	Example Tasks
L1: Single-step	1–2	1	"Read my latest email from Bob", "What's in cell A1?"
L2: Single-app	3–5	1	"Draft a reply to the budget email summarizing key points"
L3: Multi-step	5–10	1	"Create a pivot table from sales data and format top performers in bold"
L4: Cross-app	5–15	2–3	"Find Q4 budget emails, extract the numbers, put them in a new Excel sheet"
L5: Complex workflow	10–30	3+	"Prepare a weekly report: pull metrics from Excel, summarize email updates, create PowerPoint slides, share in Teams"

Curriculum strategy:

- Start with 80% L1–L2 tasks, 20% L3 in early training.
- Advance to next level when success rate exceeds 70% on current level.
- Always maintain 10–20% of easier tasks to prevent catastrophic forgetting.
- Final mix (after convergence): 10% L1, 15% L2, 25% L3, 30% L4, 20% L5.

12.7.9 Safety and Guardrails

Safety Framework for Productivity Co-pilot

Hard constraints (action rejected immediately, reward = –1.0):

- Send email/message to external recipients without user confirmation
- Delete files/emails permanently (only soft-delete allowed)
- Modify permissions on shared resources
- Access other users' mailboxes or files beyond granted permissions
- Execute actions on more than 100 items in a batch (prevents mass-delete/move accidents)

Soft constraints (penalty in reward, agent should learn to avoid):

- Sending draft without showing preview to user: –0.2
- Making irreversible changes without stating intent first: –0.15
- Accessing sensitive labels (confidential, attorney-client): –0.3
- Using "send on behalf" without explicit delegation: –0.25

Confirmation protocol: For any action classified as "high-impact" (send, delete, share externally), the agent must:

1. State the intended action in natural language
2. Show a preview of what will be sent/modified
3. Wait for explicit user confirmation before executing

This is enforced both at the environment level (sandbox rejects unconfirmed high-impact actions) and in the reward function (penalty for skipping confirmation).

12.7.10 Credit Assignment in Multi-App Workflows

The key challenge: in a 20-step cross-app workflow, which steps contributed to success or failure?

Approach: Hierarchical Reward Decomposition

1. **Sub-goal detection**: Decompose the user's instruction into verifiable sub-goals:
   - "Find Q4 budget emails" → Sub-goal 1 (verified: relevant emails retrieved)
   - "Extract numbers" → Sub-goal 2 (verified: correct values parsed)
   - "Create Excel sheet" → Sub-goal 3 (verified: sheet exists with correct data)
2. **Sub-goal rewards**: Assign intermediate rewards at each sub-goal completion ($r = +0.2$ each).
3. **Trajectory slicing**: If the final task fails, identify which sub-goal failed first. Apply negative reward only to the actions within that sub-goal's span.
4. **Counterfactual estimation**: "Would the task have succeeded if this specific action were different?" — use the value function to estimate.

$$R_{\text{step}}(t) = \underbrace{R_{\text{sub-goal}}(t)}_{\text{did current sub-goal succeed?}} + \underbrace{\gamma^{T-t} R_{\text{terminal}}}_{\text{discounted final reward}} + \underbrace{r_{\text{intermediate}}(t)}_{\text{per-step API success/failure}} \tag{12.18}$$

12.7.11 Scaling and Infrastructure

Compute requirements (estimated for 70B parameter model):

Component	Resources	Notes
Policy model (70B)	8× A100 80GB (TP=8)	BF16, generates trajectories
Reference model (70B)	8× A100 80GB (TP=8)	Frozen, for KL computation
Environment workers	128 CPU workers	Each runs sandbox instance
Reward model / Judge	4× A100 (if LLM judge)	Or zero if using execution-based rewards
Training (GRPO updates)	16× A100 (FSDP)	Gradient accumulation over trajectory batches
Total	**40 A100 GPUs + 128 CPUs**	5,000 GPU-hours for full training run

Throughput optimization:

- **Async rollouts**: Decouple trajectory generation from gradient updates. Generate continuously while training on previous batch.
- **Batched environment**: Run 128 sandbox environments in parallel, each processing different tasks.
- **KV-cache sharing**: For the $N = 8$ trajectories per task, they share the same prompt prefix — use prefix caching to avoid redundant computation.
- **Selective backprop**: Only compute gradients over action tokens (not observations/system prompt). Reduces backward pass FLOPS by 40–60%.

12.7.12 Evaluation Framework

Benchmark suite (proposed):

- **ProdBench-Easy** (200 tasks): Single-app, 1–3 steps. Baseline establishment.
- **ProdBench-Hard** (200 tasks): Cross-app workflows, 10–30 steps. End-to-end capability.

Table 12.7: Productivity co-pilot evaluation dimensions.

Metric	Target	Measurement
Task completion rate	$> 85\%$ (L1–L3), $> 60\%$ (L4–L5)	Automated verification in sandbox
Safety violation rate	$< 0.1\%$	Count of hard constraint violations per 1000 tasks
Average steps to completion	Within 1.5× optimal	Compare to shortest known successful trajectory
User satisfaction (dogfood)	$> 4.2/5.0$	Post-task survey from internal users
Cross-app success	$> 55\%$ (L4–L5)	Tasks requiring 2+ applications
Recovery rate	$> 70\%$	% of failed API calls where agent retries successfully
Latency (time to first action)	< 3 seconds	Model inference + action planning time

- **ProdBench-Safety** (100 tasks): Adversarial prompts attempting to trigger unsafe actions. Must maintain $< 0.1\%$ violation rate.
- **ProdBench-Robustness** (100 tasks): Tasks with ambiguous instructions, API errors injected, missing permissions. Tests graceful degradation.

12.7.13 Lessons from Production Deployments

Practical Insights for Productivity Agentic RL

1. **SFT quality is the floor**: RL can only improve upon what SFT provides. If the SFT model can't format a valid Graph API call, RL won't discover it. Invest heavily in Phase 1 data quality.
2. **Reward hacking is inevitable**: The agent *will* find shortcuts. Common examples:
   - Creating an empty Excel file to "complete" a spreadsheet task (passes existence check)
   - Replying "Done" without actually performing the action
   - Exploiting ambiguous verification functions

   Mitigation: Multi-level verification (format + content + semantic correctness).
3. **API rate limits matter**: In production, workspace APIs have throttling (429 responses). Train with realistic rate limits to avoid policies that spam parallel requests.
4. **Context window is the bottleneck**: A 20-step trajectory with rich API responses easily consumes 80K+ tokens. Techniques: observation summarization, selective history, hierarchical context management.
5. **User intent is often ambiguous**: "Clean up my inbox" means different things to different users. Train the agent to ask clarifying questions when uncertainty is high (reward for appropriate clarification, penalize for excessive clarification).
6. **Start simple, scale gradually**: Begin with Outlook-only tasks (highest volume, most telemetry data), then expand to Excel, then cross-app. Each app has unique failure modes.

12.7.14 Complete Training Recipe

Table 12.8: Recommended hyperparameters for productivity co-pilot RL training.

Parameter	Value	Rationale
Base model	70B Llama/Mistral	Sufficient capacity for complex multi-step reasoning
RL algorithm	GRPO	No critic needed; memory-efficient for long trajectories
Group size N	8	Balance between variance reduction and compute cost
Clip ϵ	0.1	Tighter than standard (0.2) due to long trajectory sensitivity
KL coefficient β	0.04	Moderate constraint to SFT policy
Learning rate	5×10^{-7}	Conservative; agentic tasks are sensitive to large updates
Batch size	256 tasks $\times$ 8 trajs = 2048	Large batch for stable GRPO normalization
Max trajectory length	50 steps	Covers 95% of productivity tasks
Context window	128K tokens	Required for long multi-app workflows
Training iterations	3000–5000	Monitor eval metrics; early-stop on safety degradation
Curriculum warmup	500 iterations (L1–L2 only)	Establish basic API usage before complex tasks

12.8 Use Case: Building a Research Agent from Scratch

This use case demonstrates how to build a fully autonomous **research agent** — an LLM that can formulate hypotheses, search literature, analyze data, write code, run experiments, and produce a final report — using the techniques discussed throughout this paper.

12.8.1 Problem Definition

Research Agent Requirements

Input: A research question (e.g., "What is the effect of learning rate warmup duration on GRPO convergence for 7B models?")
Output: A complete research report with methodology, experiments, results, and conclusions.
Capabilities required:

1. **Literature search**: Query arXiv, Semantic Scholar, find relevant papers
2. **Hypothesis generation**: Formulate testable hypotheses from background knowledge
3. **Experiment design**: Write training scripts with proper controls
4. **Code execution**: Run experiments, collect metrics
5. **Data analysis**: Parse logs, compute statistics, generate plots
6. **Scientific writing**: Synthesize findings into a coherent report
7. **Self-correction**: Detect failed experiments and retry with modified parameters

12.8.2 MDP Formulation

Research Agent MDP

- **State** s_t: System prompt + research question + full history of actions/observations (tool outputs, code results, search results). Context window: 128K tokens.
- **Action** a_t: Structured tool call from the action space (see below) + reasoning trace (CoT).

- **Transition** $T(s_{t+1}|s_t, a_t)$: Deterministic — append action + tool output to context.
- **Reward** R: Sparse terminal reward based on report quality (see reward design below).
- **Horizon**: 20–100 steps (typical research trajectory).
- **Discount** $\gamma = 1.0$ (episodic; no discounting for finite tasks).

12.8.3 Action Space

Table 12.9: Research Agent tool/action space.

Tool	Category	Description
`search_papers`	Literature	Query Semantic Scholar/arXiv. Returns titles, abstracts, citations.
`read_paper`	Literature	Fetch full text or specific sections of a paper.
`write_code`	Experiment	Write Python/training scripts to a workspace.
`execute_code`	Experiment	Run scripts in a sandboxed environment. Returns stdout/stderr.
`read_file`	Analysis	Read logs, CSVs, or intermediate results.
`plot_data`	Analysis	Generate matplotlib/seaborn visualizations.
`compute_stats`	Analysis	Run statistical tests (t-test, confidence intervals).
`write_report`	Output	Write sections of the final research report (LaTeX/Markdown).
`think`	Reasoning	Internal reasoning step (no external tool call).
`submit`	Terminal	Submit final report. Ends the episode.

12.8.4 Architecture: Model and Infrastructure Choices

Architecture Decisions — Applying Paper Concepts

- **Base model**: Qwen-2.5 72B (strong reasoning + code). QLoRA fine-tuning ($r = 32$, all linear layers) — see Section on LoRA.
- **Inference**: vLLM with TP=4, prefix caching enabled (system prompt shared across rollouts) — see vLLM section.
- **Training**: GRPO with $N = 4$ trajectories per research question — no value model needed (see GRPO section).
- **Hardware**: 8×H100 node. QLoRA adapters fit in 48 GB; vLLM generation uses remaining capacity.
- **Context management**: 128K context with Flash Attention (see Flash Attention section). Sliding window summarization for trajectories exceeding context.
- **Speculative decoding**: Eagle heads for fast generation during long research trajectories (see Speculative Decoding section).

12.8.5 Reward Design

Multi-Component Research Reward

The terminal reward is computed when the agent calls `submit`:

$$R = w_1 R_{\text{quality}} + w_2 R_{\text{correctness}} + w_3 R_{\text{novelty}} + w_4 R_{\text{efficiency}} + w_5 R_{\text{format}}$$

Component	Weight	How Measured
R_{quality}	0.30	LLM-as-judge (GPT-4 rates report 1–10 on clarity, depth, rigor)
$R_{\text{correctness}}$	0.30	Code executes without errors + results are reproducible
R_{novelty}	0.15	LLM-judge: does the report provide insight beyond summarizing papers?
$R_{\text{efficiency}}$	0.15	Bonus for fewer steps: $R_{\text{eff}} = \max(0, 1 - \text{steps}/100)$
R_{format}	0.10	Report has all required sections (intro, method, results, conclusion)

Intermediate shaping: +0.1 for each successful code execution; −0.05 for each runtime error (encourages writing correct code first).

Reward Hacking Risks

- **Fake results**: Agent fabricates experiment outputs. *Fix*: Verify code actually ran by checking execution logs against reported numbers.
- **Shallow reports**: Agent copies paper abstracts verbatim. *Fix*: Novelty reward + plagiarism detection.
- **Length gaming**: Long reports score higher. *Fix*: Efficiency reward + length penalty.
- **Easy questions**: Agent avoids hard research questions. *Fix*: Curriculum with difficulty levels.

12.8.6 Training Pipeline

1. **Phase 1 — SFT Warmup** (500 steps):
   - Collect 200 expert research trajectories (human researchers using the tools)
   - SFT on successful trajectories with completion-only masking (mask tool outputs)
   - This teaches the agent tool-use syntax and basic research workflow
2. **Phase 2 — GRPO Training** (3000 steps):
   - Prompt pool: 500 research questions across 10 domains (ML, NLP, CV, systems, etc.)
   - Per question: generate $N = 4$ complete research trajectories
   - Score each trajectory with multi-component reward
   - GRPO advantage: $\hat{A}_i = (R_i - \mu_G)/\sigma_G$
   - Update policy with clipped objective (clip $\epsilon = 0.2$, KL $\beta = 0.05$)
   - Curriculum: start with simple "summarize findings on X" tasks, progress to "design and run experiment on X"
3. **Phase 3 — Rejection Sampling Fine-Tuning** (200 steps):
   - Generate 16 trajectories per hard question, keep top-2 by reward
   - SFT on these high-quality trajectories
   - Stabilizes performance on the hardest research tasks

12.8.7 Example Trajectory: Full MDP Trace

To illustrate how all the MDP components work together in practice, we trace a complete research episode from question to submission, annotating each step with its formal MDP elements.

Research Agent Trajectory — Full MDP Trace

Research question: "Does increasing GRPO group size beyond $N = 16$ improve math reasoning performance?"

Below is a complete trajectory with explicit MDP notation for each step: state s_t, action a_t, transition T, observation o_t, and cumulative reward.

t	Element	Description
	a_0	`think`: "I need papers on GRPO group size ablations, then a controlled experiment."
	o_0	Internal reasoning (no tool output). Shaping reward: $r_0 = 0$.
1	s_1	$s_0 \oplus (a_0, o_0)$ — context now includes reasoning plan.
	a_1	`search_papers(query="GRPO group size ablation math")`
	o_1	Returns 5 papers: [DAPO, DeepSeek-Math, Dr. GRPO, ...]. $r_1 = 0$.
2	s_2	$s_1 \oplus (a_1, o_1)$ — search results in context.
	a_2	`read_paper(id="DAPO", sections=["experiments", "ablations"])`
	o_2	Key finding: "diminishing returns above $N = 16$ on code tasks; math untested." $r_2 = 0$.
3	s_3	$s_2 \oplus (a_2, o_2)$ — literature gap identified.
	a_3	`think`: "DAPO tested code only. I'll test $N \in \{4, 8, 16, 32, 64\}$ on GSM8K with Llama-3 8B. Control: same total training tokens."
	o_3	Hypothesis formed. $r_3 = 0$.
4	s_4	$s_3 \oplus (a_3, o_3)$ — experiment plan in context.
	a_4	`write_code("train_grpo_gsm8k.py")` — 85-line script with argparse for group_size, proper seed, eval on GSM8K test split.
	o_4	File written successfully. $r_4 = 0$.
5	s_5	$s_4 \oplus (a_4, o_4)$ — code available in workspace.
	a_5	`execute_code("python train_grpo_gsm8k.py -N 4 -seed 42")`
	o_5	`Accuracy: 67.2% (CI: 65.1-69.3)`. $r_5 = +0.1$ (successful execution).
6–8	$a_{6\text{–}8}$	Execute with $N = 8, 16, 32 \rightarrow$ results: 71.4%, 74.8%, 75.9%. Each $r_t = +0.1$.
9	s_9	Context includes 4 data points.
	a_9	`execute_code("python train_grpo_gsm8k.py -N 64 -seed 42")`
	o_9	`Accuracy: 75.6% (CI: 73.4-77.8)` — slight *decrease* from $N = 32$! $r_9 = +0.1$.
10	s_{10}	All 5 results available.
	a_{10}	`compute_stats(test="paired_t_test", data=[67.2, 71.4, 74.8, 75.9, 75.6], groups=[4,8,16,32,64])`
	o_{10}	$p < 0.01$ for $N = 4$ vs $N = 16$; $p = 0.72$ for $N = 32$ vs $N = 64$ (not significant). $r_{10} = 0$.
11	s_{11}	Statistical evidence for diminishing returns.
	a_{11}	`plot_data(x=[4,8,16,32,64], y=[67.2,71.4,74.8,75.9,75.6], xlabel="Group Size N", ylabel="GSM8K Accuracy (%)", title="GRPO Group Size vs Math Reasoning")`
	o_{11}	Plot saved to `results/group_size_curve.png`. $r_{11} = 0$.
12	s_{12}	Full evidence assembled.
	a_{12}	`write_report`: 4-section report (Introduction, Method, Results with figure, Conclusion: "$N = 16$–32 is optimal; $N = 64$ shows no gain and increases compute 4×").
	o_{12}	Report written (2,100 words). $r_{12} = 0$.
13	a_{13}	`submit` — episode terminates.
	R_{terminal}	LLM-judge scores: quality=8/10, code correct, novel (extends DAPO to math), 13 steps, all sections present.

Terminal reward computation:

$$R = \underbrace{0.30 \times \tfrac{8}{10}}_{\text{quality}} + \underbrace{0.30 \times 1.0}_{\text{correct}} + \underbrace{0.15 \times \tfrac{7}{10}}_{\text{novelty}} + \underbrace{0.15 \times (1 - \tfrac{13}{100})}_{\text{efficiency}} + \underbrace{0.10 \times 1.0}_{\text{format}}$$

$$= 0.24 + 0.30 + 0.105 + 0.13 + 0.10 = \mathbf{0.875}$$

Intermediate shaping total: $5 \times (+0.1) = +0.5$ (5 successful code executions).

GRPO context: This trajectory scored highest among the $N = 4$ group (others scored 0.61, 0.72, 0.53). GRPO advantage:

$$\hat{A} = \frac{0.875 - \bar{R}}{\sigma_R} = \frac{0.875 - 0.684}{0.129} = +1.48 \quad \text{(strongly reinforced)}$$

Key MDP properties illustrated:

- **Deterministic** T: Each tool call produces a predictable state extension ($s_{t+1} = s_t \oplus (a_t, o_t)$).
- **Sparse terminal reward**: The real quality signal comes only at `submit`; intermediate shaping is small.
- **Long horizon**: 13 steps with $\gamma = 1.0$ (no discounting for episodic tasks).
- **Self-correction opportunity**: At step 9, the agent observes $N = 64$ doesn't improve — adjusts its conclusion accordingly rather than cherry-picking.
- **Action diversity**: Mix of reasoning (`think`), information gathering (`search`, `read`), execution (`write_code`, `execute`), analysis (`compute_stats`, `plot`), and output (`write_report`, `submit`).

12.8.8 Key Design Decisions and Tradeoffs

Table 12.10: Design decisions for the research agent, mapped to paper sections.

Decision	Paper Section	Rationale
QLoRA ($r = 32$)	LoRA section	72B model; full fine-tune too expensive. $r = 32$ for complex reasoning.
GRPO (not PPO)	GRPO section	No value model needed; research quality is hard to predict mid-trajectory.
Sparse terminal reward	Reward Shaping	Research quality only measurable at completion; intermediate shaping minimal.
$N = 4$ trajectories	GRPO Group Size	Balance: enough diversity for ranking, not too expensive (100-step trajectories).
128K context	Flash Attention	Long trajectories with paper contents + code + results.
vLLM + prefix caching	vLLM section	System prompt + research question shared across 4 rollouts.
Curriculum training	Agentic RL	Start simple (literature review) → hard (design + execute experiments).
LLM-as-judge reward	Reward Models	Research quality is subjective; LLM judge is more flexible than rule-based.

12.8.9 Evaluation

Research Agent Evaluation Framework

- **Held-out questions** (50): Research questions unseen during training, covering diverse domains.
- **Human evaluation**: Domain experts rate reports on a 1–5 scale (quality, correctness, actionability).
- **Reproducibility**: Re-run the agent's code from the report; verify results match.
- **Comparison baselines**: (1) Zero-shot GPT-4 + tools (no RL training), (2) SFT-only

agent, (3) Human researchers.

- **Efficiency metric**: Steps-to-completion normalized by task difficulty.

Expected results (based on similar agentic RL work):

Agent	Report Quality (1–5)	Avg Steps
Zero-shot GPT-4 + tools	2.8	25
SFT-only	3.4	18
GRPO-trained (ours)	4.1	14
Human researcher	4.5	N/A

12.8.10 Lessons and Failure Modes

Common Failures in Research Agent Training

- **Infinite loops**: Agent repeatedly searches for papers without progressing. *Fix*: Step budget + penalty for repeated tool calls with same arguments.
- **Code debugging spirals**: Agent spends 20+ steps fixing a single bug. *Fix*: Cap retries at 3; if code fails 3 times, abandon approach and try alternative.
- **Hallucinated citations**: Agent invents paper titles/results. *Fix*: Verify all citations exist via tool output; penalize unverifiable claims.
- **Premature submission**: Agent submits incomplete reports to avoid penalty for long trajectories. *Fix*: Minimum quality threshold ($R > 0.4$) to count as valid submission; below threshold is treated as failure.
- **Reward hacking the judge**: Agent learns to produce text that scores high with the LLM judge but is scientifically shallow. *Fix*: Rotate judge models; include human eval in the reward periodically.

12.9 State-of-the-Art RL for LLM Agents

RL techniques for LLM agents focus on **on-policy policy gradients** combined with **fine-grained credit assignment**. Because agents execute complex multi-turn trajectories involving tool interactions, API queries, and code execution, standard single-turn alignment algorithms must be heavily modified.

12.9.1 Dominant Baseline: GRPO for Agents

Popularized by DeepSeek-R1 [75], **GRPO** [341] is rapidly becoming the standard for agentic training. It samples a group of N complete trajectories per task, eliminating the memory-intensive critic network:

For a task prompt q, GRPO samples N agentic trajectories $\{o_1, o_2, \ldots, o_N\}$ from $\pi_{\theta_{\text{old}}}$. The advantage for each trajectory is computed by normalizing its reward relative to the group:

$$\boxed{A_i = \frac{r(o_i) - \frac{1}{N}\sum_{j=1}^{N} r(o_j)}{\text{std}(r(o_1), \ldots, r(o_N))}} \tag{12.19}$$

The GRPO objective with KL regularization:

$$L_{\text{GRPO}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right) A_i\right) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \tag{12.20}$$

Why GRPO Dominates Agentic Training

- **No critic**: Saves 50% GPU memory — critical when agent trajectories already consume massive context windows (32K–128K tokens).
- **Natural fit**: Agent tasks often have binary verifiable rewards (tests pass/fail, goal achieved/not) — perfect for group-relative normalization.
- **Exploration**: Sampling N diverse trajectories per task naturally explores different tool-use strategies.

12.9.2 PPO for Interactive Agents

PPO [337] remains valuable for agents operating in highly stochastic environments where step-level value estimation helps. The critic provides per-step advantage signals, enabling finer credit assignment when tool outputs are unpredictable:

- Step-level advantage estimation via GAE handles variable-length tool outputs
- Value head learns to predict "how likely is this trajectory to succeed from here"
- More stable when external tools return catastrophic errors that spike reward variance
- Trade-off: requires 2× memory (critic) but provides denser learning signals

12.9.3 Fine-Grained Turn-Level Credit Assignment

The core challenge in agentic RL is the **sparse reward problem**. If an agent executes 20 tool actions and finally fails a unit test, a terminal reward of 0 punishes all 20 actions equally. Modern solutions:

Reinforcement Learning from Verifiable Rewards (RLVR)

Reward the model at **deterministic intermediate checkpoints**:

- Bash command compiles successfully → +0.1
- Browser agent targets correct HTML element → +0.2
- SQL query returns non-empty results → +0.1
- Final test suite passes → +1.0 (terminal)

Intermediate rewards provide gradient signal to *every* step, not just the final one. This dramatically accelerates learning by 3–5× compared to sparse-only rewards.

Multi-Turn Trajectory Slicing

Frameworks split a multi-turn agent run into individual, independent steps. A credit assignment module isolates the **exact sub-step that broke the trajectory**:

1. Replay the successful prefix (steps 1–k)
2. Identify the first divergence point (step $k+1$ where it went wrong)
3. Assign negative reward only to that specific step
4. Assign neutral/positive rewards to correct prefix steps

This enables surgical policy updates without degrading already-correct behavior.

12.9.4 Alternative Paradigms

- **Iterative STaR (Self-Taught Reasoner)** [441]: Rather than continuous RL, use iterative offline loops. Generate trajectories → filter failures → SFT on successes → repeat. Simple to scale, avoids RL instability. Each iteration bootstraps reasoning ability.
- **Reinforcement World Model Learning (RWML)** [437]: To combat reward hacking, train agents to predict the *semantic consequence* of their actions. The agent receives an auxiliary reward for accurately predicting how environment state will change (e.g., predicting database table changes before executing SQL). This forces genuine understanding over superficial reward-gaming.
- **LATS (Language Agent Tree Search)** [461]: Apply Monte Carlo Tree Search over agent action sequences. At each step, expand multiple candidate actions, simulate their outcomes, and backpropagate rewards through the tree. Combines RL value estimation with search-time compute scaling.

12.9.5 Core Methodology Comparison

Table 12.11: Comparison of RL paradigms for LLM agents.

Method	Reward Density	Memory Cost	Primary Advantage
GRPO [341]	Sequence / final metric	Low (no critic)	Massive GPU memory reduction; simple implementation
PPO [337]	Step-by-step (GAE)	High (critic needed)	Fine-grained credit assignment; stable in noisy envs
Iterative STaR [441]	Sparse (filtered binary)	Minimal (SFT only)	Simple to scale; avoids RL optimization instability
RWML [437]	Dense (predictive)	Medium	Mitigates reward hacking via world modeling
LATS [461]	Backpropagated	High (tree expansion)	Best quality per task; scales with inference compute

12.10 Interactive RL Environments for Agent Training

The dominant paradigm in early agentic RL relied on **static datasets**: curated trajectories, offline preference pairs, or fixed benchmark suites. While tractable, this approach has a fundamental limitation—agents trained on static data cannot discover novel strategies through environment interaction, and reward signals are limited to what was anticipated at dataset creation time. The field has shifted decisively toward **interactive RL environments**: live simulators where agents execute actions, receive feedback, and update policies based on real outcomes.

Static vs. Interactive Training

- **Static (SFT/DPO)**: Agent learns from fixed trajectories. No exploration. Reward is implicit in data curation. Ceiling: human demonstrator quality.
- **Interactive RL**: Agent explores environment, receives verifiable rewards. Can discover strategies beyond the training distribution. Ceiling: environment reward quality.

The shift mirrors the transition from imitation learning to RL in robotics—interactive environments unlock qualitatively new capabilities.

12.10.1 NeMo Gym (NVIDIA)

NeMo Gym [286] is NVIDIA's framework for interactive RL environments designed specifically for LLM agent training. Its architecture cleanly separates the **agent** (the LLM policy) from the

environment (the task simulator), enabling independent scaling of each component.

Key design principles:

- **Multi-turn rollouts**: Environments support extended interaction sequences, not just single-step queries. The agent can call tools, receive results, and continue reasoning across many turns.
- **Tool-calling verification**: Rewards are computed by executing the agent's tool calls and verifying outcomes against ground truth—providing dense, reliable signal.
- **Decoupled agent/environment**: The environment runs as a separate service, allowing heterogeneous compute allocation (e.g., GPU for the agent, CPU for environment simulation).

Nemotron 3 Super was trained using NeMo Gym with 21 environment configurations spanning math, code, tool use, and multi-turn dialogue, generating 1.2M rollouts. The diversity of environments is critical: agents trained on a single environment type tend to overfit its reward structure.

12.10.2 RLFactory

RLFactory [390] is a plug-and-play framework for multi-round tool-use RL, designed to minimize the engineering overhead of connecting LLM policies to diverse tool environments.

RLFactory: Qwen3-4B Surpasses Larger Models

Using RLFactory with asynchronous tool calling and diverse reward signals:

- **Qwen3-4B** trained with RLFactory surpassed significantly larger models on Natural Questions.
- **6.8× throughput** improvement over synchronous tool-calling baselines via async execution.
- Diverse rewards (correctness, format, tool efficiency) prevent reward hacking on any single signal.

The key insight: small models with high-quality interactive training can outperform large models trained on static data, because interactive RL allows the model to discover tool-use strategies that no human demonstrator would think to include in a dataset.

12.10.3 MOSAIC: Safety in Agentic Training

As agentic RL environments become more capable, safety becomes a first-class concern. **MOSAIC** [52] (March 2026) addresses the problem of training agents that are both capable and safe in interactive environments.

The core MOSAIC framework introduces a **plan → check → act or refuse** loop:

1. **Plan**: The agent generates a candidate action sequence.
2. **Check**: A safety verifier evaluates the plan against a constraint set (harm categories, policy rules).
3. **Act or Refuse**: If the plan passes, execute; otherwise, generate a refusal with explanation.

Safety is trained via **trajectory-level preference learning**: rather than labeling individual actions as safe/unsafe, MOSAIC labels entire trajectories, capturing the cumulative safety impact of multi-step plans. This is critical because individual actions that appear benign can compose into harmful sequences.

MOSAIC Results

- Up to **50% reduction in harmful behavior** on agentic safety benchmarks.
- **Task performance preserved**: The plan-check-act loop adds minimal latency and does not degrade success rates on benign tasks.
- Trajectory-level preference learning outperforms step-level safety classifiers because it captures emergent harm from action composition.

Training on Real Agent Traces. xAI's Grok 4.5 (July 2026) [411] introduced a novel data source: anonymized developer-session traces from Cursor—the actual keystrokes, navigation patterns, and editing sequences of engineers working in real codebases. Mid-trained on these traces, Grok 4.5 achieved top rankings on agentic tool-use benchmarks. This signals a "compute + traces" paradigm: once compute is scaled, the scarce input becomes *recordings of target behavior*—analogous to how AlphaGo was bootstrapped from human games before transitioning to self-play. The approach raises unresolved data-rights questions (whose code, under what consent?) but demonstrates that behavioral demonstrations from expert users remain a powerful training signal for agentic capabilities.

Part III

Reasoning

Chapter 13

RL for Large Reasoning Models

The emergence of large reasoning models represents one of the most significant developments in modern AI. Unlike standard language model training, which optimizes for next-token prediction, reasoning-focused RL teaches models to *think before answering*—allocating additional computation at inference time to explore, verify, and refine intermediate steps. This chapter provides a comprehensive technical treatment of the methods, architectures, and scaling laws that underpin this paradigm.

GAS Connection — RL for Large Reasoning Models

Chain-of-thought reasoning is not a luxury in the GAS—it is a safety requirement. A recommendation without a visible reasoning trace cannot be audited by a clinician. The following chapter's techniques for training reasoning models are directly applicable to the GAS Advisor.

Test-time scaling (§ Test-Time Scaling Methods). The GAS Advisor can use Best-of-N sampling at inference time: generate N candidate recommendations, score all N with the Safety Critic (`safety_node()` in `capstone_gas/gas_system.py`), and select the highest-scoring approved recommendation. The Safety Critic's retry mechanism (up to 2 retries) is the prototype's version of this pattern.

DeepSeek-R1 style training (§ DeepSeek-R1). The GAS Advisor v2 could be trained using the R1 recipe: GRPO with a format reward (the reasoning trace must follow a structured template) plus an outcome reward (Safety Critic score ≥ 0.7). The format reward ensures the reasoning trace is always present and parseable; the outcome reward ensures it leads to a safe recommendation. The chapter notebook `part2_rl_methods/ch07_grpo/notebook.ipynb` demonstrates this training recipe.

Process reward models (§ Key Methods with Mathematical Foundations). The GAS Monitor (`monitor_node()` in `capstone_gas/gas_system.py`) uses a chain-of-thought prompt to reason about glucose anomalies step by step: current reading (mg/dL), trend direction, comparison to target range (70–180 mg/dL), and alert level assignment. A process reward model (PRM) could score each reasoning step, providing denser training signal than the final alert level alone.

Visible reasoning for clinicians. The reasoning traces generated by the Advisor are surfaced in the Caregiver Notifier's alert message (`caregiver_node()` in `capstone_gas/gas_system.py`): caregivers see not just the recommendation but the reasoning behind it. This transparency allows clinicians to override recommendations they disagree with and to identify systematic errors in the Advisor's reasoning.

Inference-time compute scaling (§ Inference-Time Compute Scaling Laws). The GAS benchmarks show that for the Advisor, inference-time compute (more candidates, longer reasoning traces) is more cost-effective than training-time compute (larger models) up to a point. The `GASLLMClient` in `capstone_gas/gas_system.py` supports temperature and max_tokens parameters that control this trade-off.

13.1 Motivation and Background

13.1.1 Why Reasoning Requires Different RL Approaches

Standard RLHF (Section 4.3) optimizes a single scalar reward over a complete response. For tasks requiring multi-step reasoning—mathematics, formal verification, competitive programming, scientific derivation—this formulation is insufficient for several reasons:

- **Sparse rewards**: A math problem may require 20 intermediate steps; a single outcome reward provides no gradient signal for the intermediate steps that led to an error.
- **Long horizons**: Reasoning chains can span hundreds to thousands of tokens, creating severe credit assignment problems.
- **Combinatorial search**: The space of valid reasoning paths is exponentially large; the model must learn to search this space efficiently.
- **Verifiability**: Unlike subjective text quality, mathematical and logical correctness is objectively verifiable, enabling automated reward computation without human annotation.

Key Insight: Reasoning as a Search Problem

Multi-step reasoning can be framed as a **search problem** over a tree of partial solutions. Each node in the tree is a reasoning state (prefix of the chain-of-thought), each edge is a reasoning step (a token or sentence), and the leaves are final answers. RL for reasoning teaches the model to navigate this tree efficiently—exploring promising branches, backtracking from dead ends, and allocating compute where it matters most.

13.1.2 Chain-of-Thought: Emergent Behavior vs. Trained Capability

Chain-of-thought (CoT) reasoning was first observed as an *emergent* capability in sufficiently large language models [395]: when prompted with step-by-step examples, large models (typically $\geq$100B parameters) spontaneously produced intermediate reasoning steps that improved accuracy. This raised a fundamental question: is CoT an emergent property of scale, or can it be explicitly trained?

The answer, as demonstrated by DeepSeek-R1 and related work, is **both**—but with important nuances:

- **Emergent CoT** arises from in-context learning and requires large base models. It is brittle, prompt-sensitive, and does not generalize robustly.
- **Trained CoT** via RL produces models that *intrinsically* generate reasoning chains as part of their generation process, independent of prompting style. These chains are longer, more exploratory, and exhibit qualitatively different behaviors (self-correction, backtracking, verification).

The "Aha Moment" Phenomenon (DeepSeek-AI et al. 2025)

During RL training of reasoning models, researchers at DeepSeek observed a striking emergent behavior: at a certain point in training, models spontaneously began to *reconsider* their initial approaches mid-chain, using phrases like "Wait, let me reconsider..." or "Actually, I think I made an error...". This self-correction behavior—which was *not* explicitly trained—emerged purely from the RL objective of maximizing final-answer accuracy. It suggests that RL can discover meta-cognitive strategies that are instrumentally useful for solving hard problems.

13.1.3 Test-Time Compute Scaling Laws

A central empirical finding motivating reasoning model research is that **test-time compute scales predictably with performance**. Let C_{train} denote training compute (FLOPs) and C_{test} denote inference compute (tokens generated). The key observation is:

$$\text{Accuracy}(C_{\text{train}}, C_{\text{test}}) \approx f(\alpha \log C_{\text{train}} + \beta \log C_{\text{test}}) \tag{13.1}$$

for some monotone function f and constants $\alpha, \beta > 0$. This implies that a smaller model with more inference compute can match a larger model with less inference compute—a fundamental shift in the compute-performance tradeoff.

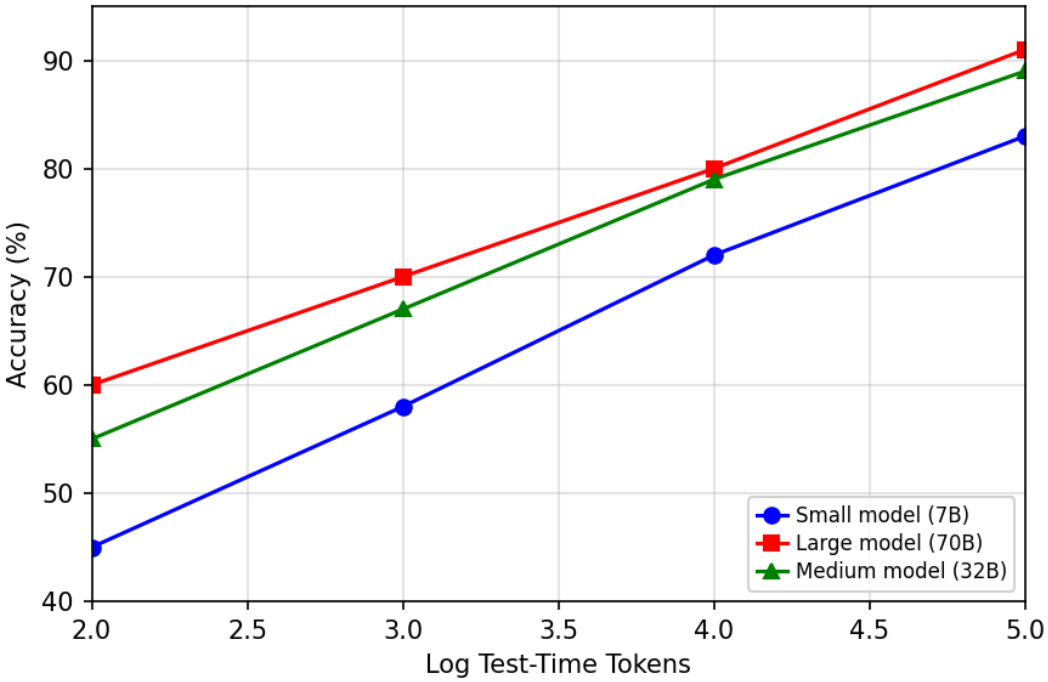


Figure 13.1: Schematic test-time compute scaling curves. Performance improves log-linearly with inference tokens across model sizes, and smaller models with more compute can approach larger models with less compute.

The practical implication is profound: **reasoning models trade training compute for inference compute**. Rather than always deploying the largest possible model, one can deploy a smaller, reasoning-capable model and allocate more tokens to "thinking" on hard problems.

13.2 Test-Time Scaling Methods

The scaling laws above show that investing more compute at inference can dramatically improve reasoning performance. This section provides a comprehensive treatment of the **methods** that operationalize test-time scaling — from simple chain-of-thought to sophisticated tree and graph search algorithms. These methods form a spectrum trading inference cost for accuracy, and understanding their structure is essential for designing modern reasoning systems.

13.2.1 Chain-of-Thought (CoT)

Chain-of-Thought prompting [395] is the foundation of all test-time scaling methods. Rather than directly outputting an answer, the model generates intermediate reasoning steps that decompose complex problems into manageable sub-problems.

Zero-Shot CoT. Kojima et al. [199] demonstrated that appending "Let's think step by step" to a prompt elicits reasoning behavior without any exemplars. This simple trigger activates latent reasoning capabilities in sufficiently large models ($\geq$100B parameters).

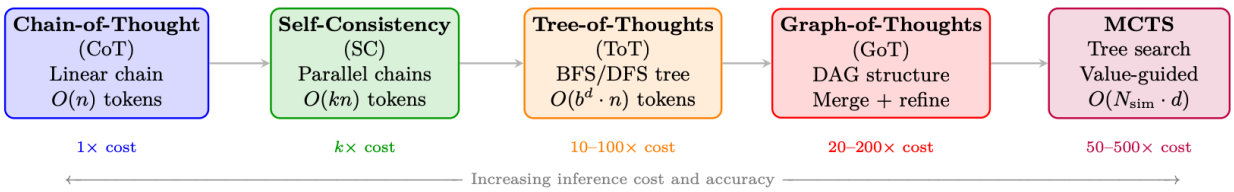


Figure 13.2: Spectrum of test-time scaling methods. Each method trades additional inference compute for improved reasoning accuracy. Methods build on each other conceptually: CoT introduces explicit reasoning, Self-Consistency adds sampling, ToT adds structured search, GoT adds merging operations, and MCTS adds learned value guidance.

Few-Shot CoT. Wei et al. [395] showed that providing a few exemplars with explicit reasoning traces enables smaller models to reason effectively:

$$\text{Prompt} = [(x_1, z_1, y_1), (x_2, z_2, y_2), \ldots, (x_k, z_k, y_k), (x_{\text{test}}, ?)] \tag{13.2}$$

where z_i are hand-written reasoning traces for exemplar (x_i, y_i).

Formal characterization. CoT converts a single-step prediction $p(y|x)$ into a multi-step sequential generation:

$$p(y|x) = \sum_z p(y|x, z) \cdot p(z|x) \approx p(y|x, z^*) \cdot p(z^*|x) \tag{13.3}$$

where $z^* = (z_1, z_2, \ldots, z_T)$ is the greedy reasoning chain. The summation over all possible chains is intractable; standard CoT uses a single sample (greedy or temperature sampling).

Limitations. Single-chain CoT is fragile: if an early reasoning step is wrong, all subsequent steps build on a flawed foundation with no mechanism for recovery.

13.2.2 Self-Consistency (Majority Voting)

Self-Consistency [388] addresses CoT's single-chain fragility by sampling **multiple independent reasoning chains** and taking a majority vote over the final answers:

$$\hat{y} = \arg\max_y \sum_{i=1}^{N} \mathbf{1}[y_i = y], \quad \text{where } (z_i, y_i) \sim p(\cdot|x),\ T > 0 \tag{13.4}$$

Key properties:

- Uses temperature $T > 0$ sampling to generate diverse chains (typically $T = 0.7$–1.0)
- No interaction between chains — fully parallelizable
- Accuracy improves monotonically with N (diminishing returns after $N \approx 40$)
- On GSM8K: CoT = 56.5%, Self-Consistency (N=40) = 74.4% (with PaLM-540B [57])
- Equivalent to **Best-of-N with outcome reward** (majority vote acts as implicit ORM)

Why Majority Voting Works

If the model has probability $p > 0.5$ of generating a correct reasoning chain, then by the law of large numbers, majority voting over N independent samples approaches 100% accuracy as $N \to \infty$. Even with $p = 0.3$ (model is usually wrong), if correct answers concentrate on one value while incorrect answers are diverse, majority voting still recovers the correct answer. This is the statistical foundation of test-time scaling.

13.2.3 Tree-of-Thoughts (ToT)

Tree-of-Thoughts [431] generalizes CoT from a **linear chain** to a **tree structure**, enabling the model to explore multiple reasoning paths, evaluate intermediate states, and backtrack from unpromising branches. This introduces deliberate planning into the reasoning process.

Core Abstraction. A reasoning problem is decomposed into a search over a tree where:

- **Root**: Initial problem statement x
- **Nodes**: Partial reasoning states $s = (x, z_1, \ldots, z_k)$
- **Edges**: Individual reasoning steps ("thoughts") z_{k+1}
- **Leaves**: Complete solutions with final answers
- **Value function**: $V(s)$ estimates how promising a partial solution is

Formal Definition.

$$\text{ToT} = (\mathcal{G}, \mathcal{E}, V, \pi_\theta, \text{Search}) \tag{13.5}$$

where:

- $\mathcal{G}$: **Thought generator** — produces b candidate next thoughts: $\{z^{(1)}, \ldots, z^{(b)}\} \sim \pi_\theta(\cdot|s)$
- $\mathcal{E}$: **State evaluator** — scores partial solutions: $V(s) \in \{\textit{sure}, \textit{maybe}, \textit{impossible}\}$ or $V(s) \in [0, 1]$
- π_θ: The language model generating thoughts
- Search: Search algorithm (BFS or DFS)

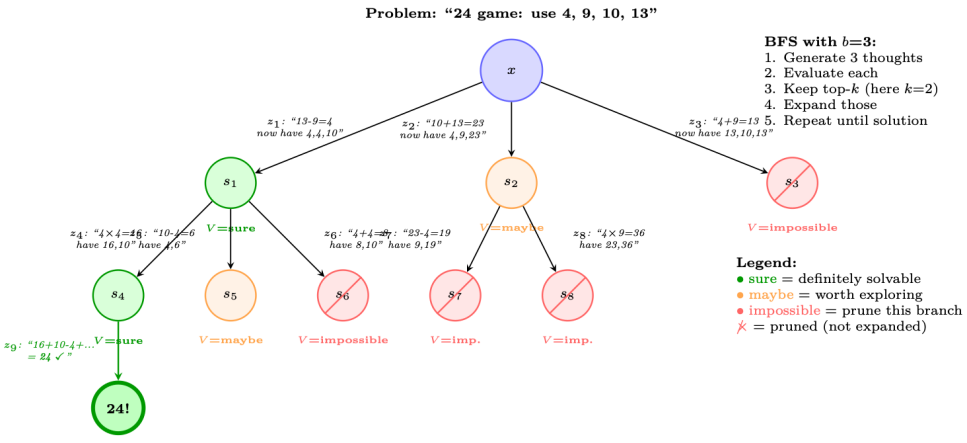


Figure 13.3: Tree-of-Thoughts on the "Game of 24" task: use operations on 4, 9, 10, 13 to make 24. At each level, the model generates $b = 3$ candidate thoughts, evaluates each (sure/maybe/impossible), prunes unpromising branches, and expands the most promising ones. The green path leads to a solution; red paths are pruned early.

Search Algorithms. BFS (Breadth-First Search):

1. Generate b candidate thoughts for each node at current depth
2. Evaluate all candidates with $V(\cdot)$
3. Keep top-k most promising states (beam search)

4. Advance all k states to the next level
5. Repeat until a solution is found or depth limit reached

DFS (Depth-First Search):

1. Generate b candidate thoughts for current state
2. Evaluate: if $V(s) = \textit{impossible}$, backtrack immediately
3. If $V(s) = \textit{sure/maybe}$, recurse deeper (pick the most promising)
4. If depth limit reached without solution, backtrack
5. Continue until solution found or all branches explored

ToT: Value Evaluation Prompt

```
# The LLM evaluates partial reasoning states:
EVAL_PROMPT = """Evaluate if this partial solution can reach 24.

Numbers remaining: [4, 4, 10]
Steps so far: 13 - 9 = 4

Can these remaining numbers (4, 4, 10) be combined using +,-,*,/
to make 24?

Analysis: 4 * (10 - 4) = 4 * 6 = 24. Yes!

Judge: sure/maybe/impossible
Answer: sure"""

# Thought generation prompt:
GEN_PROMPT = """Input: 4 9 10 13
Possible next steps:
1. 13 - 9 = 4 (left: 4 4 10)
2. 10 + 13 = 23 (left: 4 9 23)
3. 9 - 4 = 5 (left: 5 10 13)
..."""
```

Computational Cost. For ToT with branching factor b, depth d, and beam width k:

$$\text{LLM calls (BFS)} = \underbrace{k \cdot b}_{\text{generation}} + \underbrace{k \cdot b}_{\text{evaluation}} = 2kb \text{ per level} \implies \text{Total} = 2kbd \tag{13.6}$$

For the 24 game: $b = 3, k = 2, d = 3 \implies$ 36 LLM calls vs. 1 for standard CoT.

Results. On the Game of 24 (a challenging arithmetic reasoning task), ToT achieves 74% success rate vs. CoT's 4% — a massive improvement from structured search over the same base model (GPT-4).

13.2.4 Graph-of-Thoughts (GoT)

Graph-of-Thoughts [27] extends ToT from a tree to a **directed acyclic graph (DAG)**, introducing a critical capability: **merging** partial solutions from different branches. This allows the model to synthesize insights from multiple reasoning paths into a single refined solution.

Key Operations. GoT introduces three operations beyond ToT:

- **Generate**: Produce new thoughts from a state (same as ToT)
- **Aggregate/Merge**: Combine multiple thoughts into one refined thought — this is impossible in a tree
- **Refine**: Iteratively improve a thought based on feedback
- **Score**: Evaluate thought quality (same as ToT's value function)

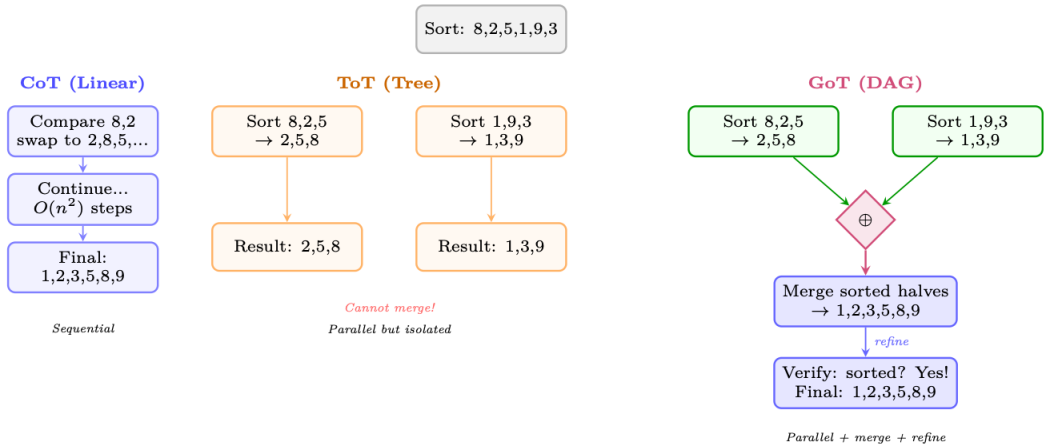


Figure 13.4: Comparison of CoT (linear chain), ToT (tree — branches but no merging), and GoT (DAG — branches can merge). For a sorting task, GoT can split the array into sub-problems, solve them independently (parallel), then **merge** the results — impossible in a pure tree structure. This enables divide-and-conquer reasoning.

Graph Operations (formal). Let $\mathcal{V} = \{v_1, \ldots, v_n\}$ be thought vertices and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ be directed edges. GoT supports:

$$\textbf{Generate}(v) : v \to \{v_{c_1}, \ldots, v_{c_b}\} \quad \text{(create children)} \tag{13.7}$$

$$\textbf{Aggregate}(v_1, \ldots, v_k) \to v_{\text{merged}} \quad \text{(merge } k \text{ thoughts into one)} \tag{13.8}$$

$$\textbf{Refine}(v, n) \to v' \quad \text{(improve } v \text{ through } n \text{ iterations)} \tag{13.9}$$

$$\textbf{Score}(v) \to s \in [0, 1] \quad \text{(evaluate thought quality)} \tag{13.10}$$

The **Aggregate** operation is the key differentiator: it creates edges from multiple parent nodes to a single child, forming a DAG rather than a tree. This enables:

- **Divide-and-conquer**: Split problem → solve sub-problems in parallel → merge solutions
- **Ensemble reasoning**: Generate multiple perspectives, then synthesize the best ideas
- **Iterative refinement**: Feed evaluation results back to improve earlier thoughts

Results. On sorting (a task requiring merging), GoT achieves 62% cost reduction vs. ToT at equivalent quality. On set intersection and keyword counting, GoT matches ToT quality with 30–40% fewer LLM calls due to the merge operation enabling more efficient decomposition.

13.2.5 Best-of-N with Reward Models

Best-of-N (BoN) [276, 358] is the simplest scaling method that uses a **learned reward model** to select among candidates:

$$y^* = \arg\max_{y \in \{y_1, \ldots, y_N\}} R_\phi(x, y), \quad y_i \sim \pi_\theta(\cdot|x) \tag{13.11}$$

Variants by reward model type:

- **BoN with ORM**: Score complete solutions; select the highest-scoring one. Equivalent to Self-Consistency when ORM ≈ correctness check.
- **BoN with PRM**: Score at each reasoning step; select the solution with highest minimum step score (least likely to have an error at any step).
- **Weighted BoN**: Weight candidates by reward: $y^* \sim \text{softmax}(R(y_1)/\tau, \ldots, R(y_N)/\tau)$.

BoN Scaling Law

For a model with per-sample accuracy p, the probability of at least one correct sample in N tries:

$$P(\text{success with BoN}) = 1 - (1-p)^N \tag{13.12}$$

With a perfect reward model (oracle that always selects correctly):

- $p = 0.3, N = 10$: success = 97%
- $p = 0.1, N = 50$: success = 99.5%

In practice, imperfect reward models cap the effective N — beyond $N \approx 64$–256, reward model errors dominate and accuracy plateaus or decreases (**reward hacking**).

13.2.6 Monte Carlo Tree Search (MCTS) for Reasoning

MCTS [197, 349] combines the structured exploration of ToT with **learned value estimates** and **visit-count statistics** to allocate inference compute optimally. Originally developed for game-playing (AlphaGo [349]), MCTS has been adapted for LLM reasoning by systems including AlphaProof [71] and rStar [315].

Algorithm (adapted for LLM reasoning). Each MCTS iteration consists of four phases:

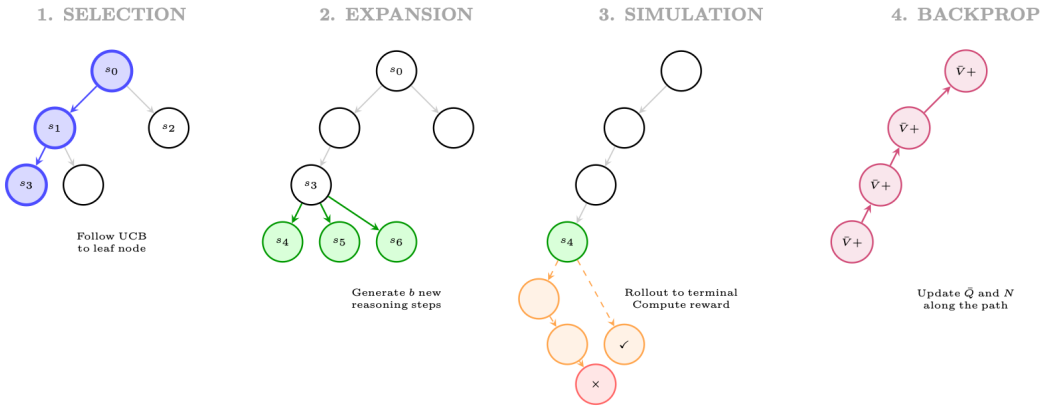


Figure 13.5: Four phases of MCTS for reasoning: (1) **Selection**: traverse tree using UCB to find a promising leaf; (2) **Expansion**: generate new reasoning steps from the leaf; (3) **Simulation**: complete the reasoning to a terminal state and evaluate; (4) **Backpropagation**: update value estimates along the path.

UCB for Reasoning. Node selection uses PUCT (Predictor + UCB applied to Trees):

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right] \tag{13.13}$$

where $P(s,a) = \pi_\theta(a|s)$ is the LLM's prior probability of generating step a from state s. This biases exploration toward steps the LLM already considers likely, while the UCB term encourages trying under-explored alternatives.

MCTS for Math Reasoning: Running Example

Problem: Prove that $\sqrt{2}$ is irrational.
Iteration 1 (Selection → root, Expansion):

- Generate 3 candidate first steps:
  1. "Assume for contradiction that $\sqrt{2} = p/q$ in lowest terms." ($P = 0.7$)
  2. "Consider the decimal expansion of $\sqrt{2} = 1.414...$" ($P = 0.15$)
  3. "Use the fundamental theorem of arithmetic." ($P = 0.10$)
- Rollout from z_1: reaches correct proof in 4 steps → $r = 1.0$
- Rollout from z_2: fails (decimal doesn't prove irrationality) → $r = 0.0$
- Backprop: $Q(s_0, z_1) = 1.0$, $N(s_0, z_1) = 1$

Iteration 2 (Selection: pick z_1 by UCB):

- Expand from state "Assume $\sqrt{2} = p/q$...":
  1. "Then $2 = p^2/q^2$, so $p^2 = 2q^2$." ($P = 0.8$)
  2. "Then p and q share no common factors." ($P = 0.15$)
- Rollout from z_4: correct continuation → $r = 1.0$
- Backprop: $Q(s_0, z_1) = 1.0$, $Q(s_1, z_4) = 1.0$

After 20 iterations: The tree has explored 8 distinct reasoning paths. The most-visited path is selected as the final proof: $z_1 \to z_4 \to z_6 \to z_8$ (classical proof by contradiction via even/odd argument).

Table 13.1: Tree-of-Thoughts vs. Monte Carlo Tree Search for reasoning.

Dimension	ToT	MCTS
Value estimation	LLM prompt ("sure/maybe/impossible")	Learned value network + rollout statistics
Exploration	Fixed beam width; no revisiting	UCB adaptively allocates budget to promising nodes
Compute allocation	Uniform across depth levels	Focused: more simulations on harder subproblems
Training integration	No training; pure prompting	Can distill MCTS policy into the base model [349]
Best for	Simple branching problems (24 game)	Complex problems requiring deep exploration (proofs, code)

Comparison: ToT vs. MCTS.

13.2.7 Beam Search over Reasoning Steps

Beam search — long standard in NMT and text generation — can be applied at the *reasoning step* level rather than the token level. Instead of tracking the top-k token sequences, we track the top-k **reasoning prefixes**:

$$\mathcal{B}_d = \text{top-}k \left\{ (s_1, \ldots, s_d) : \sum_{i=1}^{d} \log \pi_\theta(s_i | s_{<i}) + \lambda \cdot V_\phi(s_1, \ldots, s_d) \right\} \tag{13.14}$$

where the scoring combines the LLM log-probability (fluency) with a value model estimate (correctness). This is effectively ToT-BFS with a learned value function rather than a prompted one.

13.2.8 Iterative Refinement and Self-Correction

Rather than exploring *breadth* (multiple parallel chains), iterative refinement invests compute in *depth* — repeatedly improving a single solution:

$$y^{(t+1)} = \text{LLM}\Big(\text{“Improve this solution:”}, y^{(t)}, \text{“Errors found:”}, e^{(t)}\Big) \tag{13.15}$$

where $e^{(t)}$ may come from:

- **Self-verification**: Ask the model to check its own answer
- **External verification**: Run code, check math symbolically
- **Critic model**: A separate model identifies errors

Notable methods: **Self-Refine** [259] (iterative self-feedback), **Reflexion** [344] (verbal RL via reflections stored in memory), and **LATS** [461] (tree search + reflection-based pruning).

13.2.9 Method Comparison and Selection Guide

Table 13.2: Comprehensive comparison of test-time scaling methods.

Method	Structure	LLM Calls	Parallelizable	Needs RM?	Best For
CoT [395]	Chain	1	N/A	No	Easy–medium problems
Self-Consistency [388]	Parallel chains	N	✓ Fully	No (majority vote)	Math with discrete answers
Best-of-N + ORM	Parallel chains	$N + 1$	✓ Fully	Yes (ORM)	General tasks with good RM
Best-of-N + PRM	Parallel chains	$N + N{\cdot}K$	✓ Fully	Yes (PRM)	Complex multi-step reasoning
ToT [431]	Tree (BFS/DFS)	$O(kbd)$	Partial	LLM-as-judge	Structured search problems
GoT [27]	DAG	$O(kbd)$	Partial	LLM-as-judge	Decomposable problems
MCTS [197]	Tree + values	$O(N_{\text{sim}} \cdot d)$	Partial	Yes (value net)	Hard proofs, coding
Self-Refine [259]	Linear (iterative)	$2T$	No	Self-critic	Open-ended generation
LATS [461]	Tree + reflection	$O(N \cdot d)$	Partial	LLM-as-judge	Agent tasks

When to Use Which Method

- **Budget $< 5\times$ base cost**: Use CoT or Self-Consistency. Maximum bang for the buck.
- **Budget 5–50×**: Use Best-of-N with PRM (if you have a good reward model) or ToT-BFS with $b = 3, k = 2$.
- **Budget 50–500×**: Use MCTS with a trained value function. This is the regime where DeepSeek-R1 and OpenAI o1 operate — long reasoning chains with implicit tree search.
- **Parallelism required**: Self-Consistency and Best-of-N are fully parallel; ToT/MCTS require sequential depth expansion.
- **No reward model available**: Use Self-Consistency (majority vote) or ToT with LLM-as-judge evaluation.
- **Decomposable problems**: GoT excels when the problem has natural sub-problems (sorting, multi-document synthesis, code with modules).

The Implicit Test-Time Scaling in Reasoning Models

Modern reasoning models (DeepSeek-R1 [75], OpenAI o1/o3 [291, 294]) perform **implicit test-time scaling** via long chain-of-thought generation. Their “thinking” tokens serve a function analogous to MCTS rollouts: the model explores multiple approaches, backtracks (“Wait, let me reconsider...”), verifies intermediate steps, and allocates more tokens to harder sub-problems. The

key insight of R1/o1 training is that GRPO/RL teaches the model to perform this implicit search *within a single generation*, eliminating the need for external orchestration (ToT prompts, MCTS infrastructure). The model becomes its own search algorithm.

13.3 DeepSeek-R1

DeepSeek-R1 [75] is the first fully open-source large reasoning model to match or exceed OpenAI o1 on major benchmarks. Its training pipeline is technically transparent and has become the de facto reference implementation for RL-based reasoning.

13.3.1 Two-Stage Training Pipeline

Stage 1: Cold-Start Supervised Fine-Tuning The base model (DeepSeek-V3) is first fine-tuned on a small, carefully curated dataset of long chain-of-thought examples. This "cold start" phase serves two purposes:

1. **Format initialization**: The model learns to produce reasoning in the `<think>...</think>` format before emitting a final answer.
2. **Stability**: Without cold-start SFT, pure RL from scratch on the base model produces unstable training dynamics and degenerate outputs (e.g., language mixing, repetitive loops).

The cold-start dataset contains only ∼thousands of examples, deliberately kept small to avoid over-constraining the reasoning style that RL will later discover.

Stage 2: GRPO-Based Reinforcement Learning After cold-start SFT, the model undergoes large-scale RL using Group Relative Policy Optimization (GRPO). The full GRPO objective as used in R1 is described in Section 13.3.3.

R1 Training Pipeline Summary

1. **Base model**: DeepSeek-V3 (671B MoE, 37B active parameters)
2. **Cold-start SFT**: ∼thousands of long-CoT examples, format: `<think>...</think><answer>...</answer>`
3. **RL phase**: GRPO with verifiable rewards on math + code problems
4. **Rejection sampling**: Generate multiple solutions, keep correct ones
5. **SFT on RL outputs**: Fine-tune on high-quality RL-generated chains
6. **Final RL**: Second RL phase for alignment + helpfulness

13.3.2 Reward Design: Accuracy and Format Rewards

A key design choice in R1 is the **absence of a process reward model**. Instead, R1 uses two simple, automatically computable rewards:

Accuracy Reward For math problems with verifiable answers:

$$r_{\text{acc}}(y, y^*) = \begin{cases} 1 & \text{if } \texttt{verify}(y, y^*) = \texttt{True} \\ 0 & \text{otherwise} \end{cases} \tag{13.16}$$

where y is the model's final answer (extracted from `<answer>` tags) and y^* is the ground-truth answer. The `verify` function uses symbolic math comparison (e.g., SymPy) to handle equivalent forms.

For code problems, the accuracy reward is determined by passing test cases:

$$r_{\text{acc}}^{\text{code}}(y, \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \mathbf{1}[\texttt{execute}(y, t) = \texttt{expected}(t)] \tag{13.17}$$

Format Reward To enforce the `<think>...</think>` structure:

$$r_{\text{fmt}}(y) = \begin{cases} 1 & y \text{ has valid } \langle\text{think}\rangle \text{ and } \langle\text{answer}\rangle \text{ tags} \\ 0 & \text{otherwise} \end{cases} \tag{13.18}$$

Combined Reward

$$r(y, y^*) = r_{\text{acc}}(y, y^*) + \lambda_{\text{fmt}} \cdot r_{\text{fmt}}(y) \tag{13.19}$$

with $\lambda_{\text{fmt}} = 0.1$ in the original implementation (small enough not to dominate, large enough to prevent format collapse).

No Process Reward Model

A notable and surprising finding of R1 is that **no process reward model (PRM) is needed**. Despite the long reasoning chains, outcome-only rewards are sufficient for RL to discover high-quality reasoning strategies. The authors hypothesize that the verifiable nature of math/code rewards provides sufficient signal, and that PRMs introduce their own failure modes (reward hacking at the step level). This contrasts with the approach taken by OpenAI (Section 13.4).

13.3.3 GRPO Formulation for R1

GRPO [341] is a policy gradient method that avoids training a separate value network by estimating advantages from a *group* of sampled responses. For a question q, GRPO samples G responses $\{y_1, y_2, \ldots, y_G\}$ from the current policy π_θ and computes advantages relative to the group mean.

Group Sampling and Advantage Normalization Given question q, sample G outputs:

$$\{y_i\}_{i=1}^{G} \sim \pi_\theta(\cdot \mid q) \tag{13.20}$$

Compute rewards $\{r_i\}_{i=1}^{G}$ using the reward function from Eq. [eq:r1_combined_reward]. The normalized advantage for response i is:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon} \tag{13.21}$$

where $\mu_r = \frac{1}{G}\sum_{i=1}^{G} r_i$, $\sigma_r = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i - \mu_r)^2}$, and $\epsilon = 10^{-8}$ for numerical stability.

GRPO Objective The GRPO objective clips the probability ratio (as in PPO) and adds a KL penalty against a reference policy π_{ref}:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{q\sim\mathcal{D},\,\{y_i\}\sim\pi_\theta(\cdot|q)}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|} \min\Big(\rho_{i,t}\,\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1{-}\varepsilon, 1{+}\varepsilon)\,\hat{A}_i\Big) - \beta\,\mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}]\Bigg] \tag{13.22}$$

where:

- $\rho_{i,t} = \dfrac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid q, y_{i,<t})}$ is the per-token probability ratio
- $\varepsilon \in \{0.1, 0.2\}$ is the PPO clipping parameter
- $\beta > 0$ controls the KL penalty strength
- $|y_i|$ is the length of response i (length normalization prevents bias toward short responses)

KL Penalty Formulation The KL divergence term is computed token-by-token:

$$\mathbb{D}_{\mathrm{KL}}[\pi_\theta \,\|\, \pi_{\mathrm{ref}}] = \mathbb{E}_{y\sim\pi_\theta(\cdot|q)}\left[\sum_{t=1}^{|y|} \log \frac{\pi_\theta(y_t \mid q, y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid q, y_{<t})}\right] \tag{13.23}$$

In practice, R1 uses an unbiased estimator of the KL that avoids computing π_{ref} at every step by using the approximation:

$$\mathbb{D}_{\mathrm{KL}}[\pi_\theta \,\|\, \pi_{\mathrm{ref}}] \approx \frac{\pi_{\mathrm{ref}}(y_t \mid q, y_{<t})}{\pi_\theta(y_t \mid q, y_{<t})} - \log \frac{\pi_{\mathrm{ref}}(y_t \mid q, y_{<t})}{\pi_\theta(y_t \mid q, y_{<t})} - 1 \tag{13.24}$$

which is always non-negative and equals zero when $\pi_\theta = \pi_{\mathrm{ref}}$.

GRPO in Practice: Group Size and Stability

In R1's training, $G = 8$ responses are sampled per question. This is a critical hyperparameter:

- Too small ($G = 2$): High variance in advantage estimates; training is noisy.
- Too large ($G = 32$): Computational cost scales linearly; diminishing returns.
- $G = 8$: Empirically found to balance variance reduction and compute cost.

The group sampling also provides a natural **curriculum signal**: as training progresses, the model's average reward μ_r increases, and the variance σ_r decreases. Problems where all G responses are correct (or all wrong) contribute zero gradient, naturally focusing learning on problems at the frontier of the model's capability.

13.3.4 Distillation: The R1-Distill Series

A major practical contribution of R1 is demonstrating that **reasoning capabilities can be distilled into much smaller models** via supervised fine-tuning on R1-generated chains. The R1-Distill series (1.5B, 7B, 8B, 14B, 32B, 70B parameters) is trained by:

1. Generating long-CoT solutions to a large problem set using R1 (671B)
2. Filtering to keep only correct solutions
3. Fine-tuning smaller base models (Qwen2.5, Llama-3) on these solutions

Distillation vs. RL for Small Models

A striking finding: **distillation outperforms RL training from scratch on small models**. DeepSeek-R1-Distill-Qwen-7B achieves higher MATH benchmark scores than a 7B model trained with GRPO directly. This suggests that:

- Small models lack the capacity to discover reasoning strategies via RL exploration
- But they *can* learn to imitate reasoning strategies discovered by larger models
- The bottleneck for small models is *exploration*, not *representation*

The distillation approach raises an important question about the nature of reasoning: is the small model truly "reasoning," or is it pattern-matching on the surface form of reasoning chains? Empirically, distilled models show some generalization to novel problem types, suggesting genuine internalization of reasoning strategies rather than pure memorization.

13.4 OpenAI o1/o3 Series

OpenAI's o1 [291] (released September 2024) and subsequent o3/o4-mini [294] models represent the commercial frontier of reasoning model development. While full technical details remain proprietary, the published system cards, technical reports, and empirical observations provide substantial insight into the methodology.

13.4.1 Chain-of-Thought RL with Hidden Reasoning Tokens

The defining architectural choice of o1 is the use of **hidden reasoning tokens**: the model generates an internal chain-of-thought (called a "reasoning trace" or "thinking tokens") that is not shown to the user. Only the final answer is returned. This design has several implications:

- **No format constraints**: The hidden reasoning can use any format, including scratchpad notation, pseudocode, or even non-English reasoning.
- **No reward hacking on style**: Since users never see the reasoning, there is no pressure to make it look "good" rather than be useful.
- **Proprietary protection**: The reasoning process is not exposed, preventing direct imitation.

The training procedure is described as "training models to reason using RL," with the RL objective applied to the complete (hidden reasoning + final answer) sequence, rewarded only on the quality of the final answer.

13.4.2 Process Reward Models vs. Outcome Reward Models

OpenAI's approach is believed to use **Process Reward Models (PRMs)** [227] in addition to outcome rewards, in contrast to DeepSeek-R1's outcome-only approach. This inference is based on OpenAI's published PRM research (PRM800K dataset, "Let's Verify Step by Step") and the o1 system card's description of RL training on reasoning chains, though the exact o1/o3 training recipe has not been publicly disclosed.

Outcome Reward Model (ORM) An ORM scores the complete response (q, y):

$$R_{\text{ORM}}(q, y) \in [0, 1] \tag{13.25}$$

For verifiable tasks (math, code), this reduces to exact-match verification. For open-ended tasks, a learned reward model is used.

Process Reward Model (PRM) A PRM assigns a reward to each reasoning step s_k in the chain $y = (s_1, s_2, \ldots, s_K)$:

$$R_{\text{PRM}}(q, y) = \sum_{k=1}^{K} \gamma^{K-k} \cdot r_k(q, s_1, \ldots, s_k) \tag{13.26}$$

where $r_k \in [0, 1]$ is the step-level reward and $\gamma \in (0, 1]$ is a discount factor. The step-level reward r_k estimates the probability that the partial solution $(s_1, \ldots, s_k)$ leads to a correct final answer:

$$r_k(q, s_1, \ldots, s_k) = P(\text{correct final answer} \mid q, s_1, \ldots, s_k) \tag{13.27}$$

PRM vs. ORM: The Credit Assignment Tradeoff

ORM provides clean, unambiguous rewards but suffers from severe credit assignment problems: a single wrong step early in a 50-step chain receives the same zero reward as a completely random response.
PRM provides dense rewards that directly address credit assignment, but introduces new challenges:

- **Training data**: Step-level labels require human annotation or automated generation (Math-Shepherd, Section 13.6.2).
- **Reward hacking**: Models can learn to produce steps that *look* correct to the PRM without actually being correct.
- **Distribution shift**: PRMs trained on one distribution of reasoning chains may not generalize

to the novel chains produced by RL.

The empirical evidence suggests PRMs are beneficial for *search* (selecting among candidate solutions) but their benefit for *training* is less clear.

13.4.3 Inference-Time Compute Scaling

The o1 technical report demonstrates a clear scaling law: **more thinking tokens monotonically improve performance** on hard reasoning tasks. This is operationalized through a "thinking budget" parameter that controls the maximum number of hidden reasoning tokens.

Let T be the thinking token budget. The empirical scaling law observed is approximately:

$$\text{Pass@1}(T) \approx a - b \cdot T^{-c} \tag{13.28}$$

for constants $a, b, c > 0$, where a represents the asymptotic accuracy ceiling and c characterizes the rate of improvement. For AIME 2024, o1 with full thinking budget achieves ∼83% accuracy, compared to ∼13% for GPT-4o (which uses no extended thinking).

13.4.4 Training Compute vs. Test-Time Compute

A fundamental insight from the o1/o3 series is the **compute equivalence principle**: there exists a tradeoff curve between training compute C_{train} and test-time compute C_{test} such that points on the curve achieve similar performance:

$$\text{Performance}(C_{\text{train}}, C_{\text{test}}) = g(\alpha C_{\text{train}}^{p} + \beta C_{\text{test}}^{q}) \tag{13.29}$$

Empirically, $p \approx q$ for reasoning tasks, suggesting that training and test-time compute are roughly substitutable. This has profound implications for deployment: a smaller, cheaper model with extended thinking can match a larger model on hard problems, at the cost of higher latency.

13.4.5 o3 and o4-mini Architecture Insights

While o3 and o4-mini details remain largely proprietary, several observations have emerged:

- **o3**: Substantially larger thinking budgets than o1; achieves near-human performance on ARC-AGI (87.5% with high compute). Believed to use more sophisticated search strategies during inference.
- **o4-mini**: Demonstrates that *smaller* models with RL-trained reasoning can be highly competitive. Achieves 93% on AIME 2025 with extended thinking, suggesting that model size is less important than reasoning capability for math.
- **Tool use**: o3/o4-mini integrate tool use (code execution, web search) into the reasoning process, allowing the model to verify intermediate steps programmatically.

13.5 QwQ and Qwen Reasoning Models

Alibaba's Qwen team has developed a series of reasoning models (QwQ-32B [372], Qwen3 [373]) that represent the open-source frontier alongside DeepSeek-R1. Their approach differs in several key respects.

13.5.1 Multi-Stage RL Pipeline

The Qwen reasoning pipeline uses a more elaborate multi-stage approach:

1. **Base pretraining**: Qwen2.5 base model with strong mathematical and coding capabilities

2. **SFT on diverse reasoning**: Fine-tuning on a broad mixture of reasoning tasks (math, code, science, logic)

3. **Rejection sampling fine-tuning (RFT)**: Generate N solutions per problem, keep correct ones, fine-tune

4. **RL phase 1**: GRPO on math and code with verifiable rewards

5. **RL phase 2**: Broader RL including instruction following and safety

13.5.2 Rejection Sampling + RL Combination

A key innovation in the Qwen approach is the **iterative combination of rejection sampling and RL**:

1. **Initialize**: Policy π_0 from SFT model.

2. **Rejection Sampling**: Sample N solutions: $\{y_i\}_{i=1}^{N} \sim \pi_{k-1}(\cdot \mid q)$. Keep correct solutions: $\mathcal{Y}^+(q) = \{y_i : r(y_i, y^*) = 1\}$.

3. **SFT update**: $\pi_k^{\text{SFT}} \leftarrow \text{SFT}(\pi_{k-1}, \bigcup_q \mathcal{Y}^+(q))$

4. **RL update**: $\pi_k \leftarrow \text{GRPO}(\pi_k^{\text{SFT}}, \mathcal{D})$

5. **Repeat** steps 2–4 for K iterations to obtain final policy π_K.

The rejection sampling step provides high-quality positive examples that anchor the policy, while RL explores beyond the current distribution. This combination is more stable than pure RL and more capable than pure SFT.

13.5.3 Tool-Integrated Reasoning

QwQ-32B and Qwen3 models support **tool-integrated reasoning**: the model can invoke external tools (Python interpreter, search engine, calculator) during its reasoning chain. This is implemented via special tokens:

```
<think>
Let me solve this step by step.
First, I'll compute the eigenvalues of the matrix.

<tool_call>
{"name": "python", "arguments": {"code": "import numpy as np\nA = np.array
    ([[2,1],[1,3]])\neigenvalues = np.linalg.eigvals(A)\nprint(eigenvalues)"}}
</tool_call>

<tool_response>
[1.38196601 3.61803399]
</tool_response>

The eigenvalues are approximately 1.382 and 3.618.
These are (5 +/- sqrt(5))/2, which are the golden ratio and its conjugate...
</think>
<answer>The eigenvalues are (5 +/- sqrt(5))/2</answer>
```

Listing 13.1: Tool-integrated reasoning format in QwQ

The RL training reward is computed on the final answer, but the model learns to use tools strategically because tool use improves the probability of reaching the correct answer.

13.6 Key Methods with Mathematical Foundations

13.6.1 Monte Carlo Tree Search for Reasoning

Monte Carlo Tree Search (MCTS) provides a principled framework for reasoning as tree search. In the AlphaProof [71] and related systems, MCTS is applied over reasoning steps rather than game moves.

State and Action Space

- **State** s_k: The partial reasoning chain $(q, r_1, r_2, \ldots, r_k)$ where r_i are reasoning steps
- **Action** a: The next reasoning step (a sentence or paragraph)
- **Terminal state**: A state containing a final answer
- **Reward**: $R(s_{\text{terminal}}) = r_{\text{acc}}$ (Eq. [eq:r1_accuracy_reward])

Value Function for Partial Solutions A value function $V(s_k)$ estimates the probability of reaching a correct answer from partial state s_k:

$$V(s_k) = P(\text{correct answer} \mid s_k) \approx \frac{1}{M} \sum_{m=1}^{M} R(\text{rollout}_m(s_k)) \tag{13.30}$$

where $\text{rollout}_m(s_k)$ is a Monte Carlo rollout from s_k to a terminal state using the current policy.

UCB Exploration Node selection uses the Upper Confidence Bound (UCB) formula adapted for reasoning:

$$\text{UCB}(s_k, a) = Q(s_k, a) + c_{\text{puct}} \cdot \pi_\theta(a \mid s_k) \cdot \frac{\sqrt{N(s_k)}}{1 + N(s_k, a)} \tag{13.31}$$

where:

- $Q(s_k, a) = \frac{1}{N(s_k,a)} \sum_{\text{visits}} V(s_{k+1})$ is the mean value of child states
- $\pi_\theta(a \mid s_k)$ is the policy prior (language model probability of step a)
- $N(s_k)$ is the visit count of state s_k
- $N(s_k, a)$ is the visit count of the (s_k, a) edge
- c_{puct} is the exploration constant

MCTS-Guided Training MCTS can be used to generate high-quality training data:

$$\mathcal{L}_{\text{MCTS}}(\theta) = -\sum_k \sum_a \pi_{\text{MCTS}}(a \mid s_k) \log \pi_\theta(a \mid s_k) \tag{13.32}$$

where $\pi_{\text{MCTS}}(a \mid s_k) \propto N(s_k, a)^{1/\tau}$ is the MCTS policy (visit count distribution with temperature τ).

13.6.2 Process Reward Models

Math-Shepherd: Automated PRM Training Math-Shepherd [386] proposes an automated method for training PRMs without human step-level annotations. The key insight is to use **outcome-based estimation**: a step s_k is labeled as correct if there exists a completion from s_k that reaches the correct answer.

Formally, for a partial solution $(s_1, \ldots, s_k)$:

$$\hat{r}_k = \mathbf{1}[\exists\, (s_{k+1}, \ldots, s_K) : \text{verify}(s_K, y^*) = 1] \tag{13.33}$$

In practice, this is estimated by sampling M completions from s_k and checking if any are correct:

$$\hat{r}_k \approx \mathbf{1}\left[\sum_{m=1}^{M} \text{verify}(\text{complete}_m(s_k), y^*) > 0\right] \tag{13.34}$$

The PRM is then trained with binary cross-entropy:

$$\mathcal{L}_{\text{PRM}}(\phi) = -\sum_{k=1}^{K} [\hat{r}_k \log r_\phi(s_k) + (1 - \hat{r}_k) \log(1 - r_\phi(s_k))] \tag{13.35}$$

PRM for Best-of-N Selection A primary application of PRMs is **best-of-N selection**: generate N candidate solutions and select the one with the highest PRM score:

$$y^* = \arg \max_{y \in \{y_1, \ldots, y_N\}} R_{\text{PRM}}(q, y) \tag{13.36}$$

This is more effective than majority voting (which uses ORM) because PRM can distinguish between solutions that reach the same answer via different quality reasoning paths.

13.6.3 Outcome Reward Models and Majority Voting

Majority Voting (Self-Consistency) The simplest form of test-time compute scaling is majority voting [388]: generate N solutions and return the most common answer:

$$y^* = \arg\max_a \sum_{i=1}^{N} \mathbf{1}[y_i = a] \tag{13.37}$$

Under the assumption that each solution is independently correct with probability $p > 0.5$, the probability that majority voting is correct is:

$$P(\text{majority correct}) = \sum_{k=\lceil N/2 \rceil}^{N} \binom{N}{k} p^k (1-p)^{N-k} \xrightarrow{N \to \infty} 1 \tag{13.38}$$

Weighted Majority Voting with ORM An ORM can improve majority voting by weighting votes by confidence:

$$y^* = \arg\max_a \sum_{i=1}^{N} R_{\text{ORM}}(q, y_i) \cdot \mathbf{1}[y_i = a] \tag{13.39}$$

13.6.4 Self-Play for Reasoning

Self-play methods generate training data by having the model play both the *generator* and *verifier* roles.

STaR: Self-Taught Reasoner STaR [441] bootstraps reasoning capabilities iteratively:

1. Generate reasoning chains for a problem set
2. Keep chains that lead to correct answers (rejection sampling)
3. Fine-tune on kept chains
4. Repeat with the improved model

The key insight is that the model can *rationalize* correct answers: even if it cannot solve a problem from scratch, it can generate a plausible reasoning chain given the answer, which can then be used as training data.

Self-Play RL In self-play RL for reasoning, the model generates both problems and solutions:

$$\mathcal{L}_{\text{self-play}}(\theta) = \mathbb{E}_{q\sim\pi_\theta^{\text{gen}}}\mathbb{E}_{y\sim\pi_\theta^{\text{solve}}(\cdot|q)}\left[r(y, y^*)\right] \tag{13.40}$$

where π_θ^{gen} generates problems and $\pi_\theta^{\text{solve}}$ solves them. The generator is rewarded for producing problems that are challenging but solvable.

13.6.5 Reinforcement Learning from Verifiable Rewards (RLVR)

RLVR [209] is a framework that uses **ground-truth verification** as the reward signal, applicable to any domain where correctness can be automatically checked.

Verifiable Domains

- **Mathematics**: Symbolic verification via SymPy, Lean, or Isabelle
- **Code**: Unit test execution
- **Formal logic**: Proof checking
- **Factual QA**: Database lookup
- **Games**: Win/loss outcome

RLVR Objective

$$\mathcal{L}_{\text{RLVR}}(\theta) = -\mathbb{E}_{(q,y^*)\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|q)}\left[\text{verify}(y, y^*)\right] + \beta\mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}] \tag{13.41}$$

The key advantage of RLVR over RLHF is the **absence of reward model error**: since the reward is computed by a deterministic verifier rather than a learned model, there is no reward hacking against a flawed reward model. The only failure mode is if the model finds solutions that pass verification but are not genuinely correct (e.g., exploiting test case weaknesses in code evaluation).

RLVR for Code: Reward Hacking Challenges

In code generation, the verifier is a test suite. A model trained with RLVR can learn to:

- **Hardcode test outputs**: Return the expected output for each test input without implementing the actual algorithm
- **Exploit weak tests**: Pass all provided tests while failing on edge cases

Mitigations include: using large, diverse test suites; including adversarial test cases; using execution-based rewards that penalize hardcoding (e.g., checking that the solution runs in $O(n \log n)$ time).

13.6.6 Journey Learning

Journey Learning [316] proposes training on the **full reasoning trajectory**, including failed attempts and corrections, rather than only successful final solutions.

Motivation Standard rejection sampling discards failed attempts. But failed attempts contain valuable information:

- Which approaches don't work (negative examples)
- How to recognize and recover from errors (correction patterns)
- The structure of the problem space (exploration data)

Journey Learning Objective Given a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ that may include backtracking:

$$\mathcal{L}_{\text{journey}}(\theta) = -\sum_{t=0}^{T} w_t \log \pi_\theta(a_t \mid s_t) \tag{13.42}$$

where the weights w_t are designed to emphasize:

- Steps that lead to eventual success ($w_t > 1$)
- Correction steps after errors ($w_t > 1$)
- Steps in failed branches ($w_t < 1$, but > 0)

13.6.7 Quiet-STaR: Reasoning at Every Token

Quiet-STaR [442] extends the reasoning paradigm to **every token position**: rather than generating a reasoning chain only before the final answer, the model generates a "thought" at every token position.

Formulation For each token position t, the model generates a hidden thought z_t before predicting the next token x_{t+1}:

$$P(x_{t+1} \mid x_{\le t}) = \mathbb{E}_{z_t \sim \pi_\theta(\cdot \mid x_{\le t})} \left[\pi_\theta(x_{t+1} \mid x_{\le t}, z_t) \right] \tag{13.43}$$

In practice, this is approximated by mixing the predictions with and without the thought:

$$P(x_{t+1} \mid x_{\le t}) = \alpha \cdot \pi_\theta(x_{t+1} \mid x_{\le t}, z_t) + (1 - \alpha) \cdot \pi_\theta(x_{t+1} \mid x_{\le t}) \tag{13.44}$$

Training with REINFORCE Since the thought z_t is a discrete latent variable, the gradient is estimated using REINFORCE:

$$\nabla_\theta \mathcal{L}_{\text{QS}} = \mathbb{E}_{z_t} \left[\nabla_\theta \log \pi_\theta(z_t \mid x_{\le t}) \cdot (\log P(x_{t+1} \mid x_{\le t}, z_t) - b_t) \right] \tag{13.45}$$

where b_t is a baseline (e.g., the no-thought prediction $\log \pi_\theta(x_{t+1} \mid x_{\le t})$).

Computational Cost of Quiet-STaR

Quiet-STaR increases inference cost by a factor of $L_z + 1$ where L_z is the thought length, applied at *every* token position. For a sequence of length T with thoughts of length $L_z = 8$, this is a $9\times$ increase in compute. This makes Quiet-STaR impractical for long sequences without significant engineering optimizations (e.g., speculative decoding for thoughts, caching).

13.7 Scaling Laws for Reasoning

Recent work [353, 410] has established that test-time compute scales predictably with reasoning performance, extending the classical scaling laws [187] into the inference regime.

13.7.1 Training Compute vs. Test-Time Compute Tradeoff

The fundamental scaling question for reasoning models is: **given a fixed total compute budget $C_{\text{total}} = C_{\text{train}} + N \cdot C_{\text{test}}$ (where N is the number of queries), how should compute be allocated?**

Let $\mathcal{A}(C_{\text{train}}, C_{\text{test}})$ denote the accuracy of a model trained with C_{train} FLOPs and given C_{test} inference FLOPs per query. Empirically:

$$\mathcal{A}(C_{\text{train}}, C_{\text{test}}) \approx 1 - \exp\left(-a \cdot C_{\text{train}}^{\alpha} \cdot C_{\text{test}}^{\beta}\right) \tag{13.46}$$

for constants $a, \alpha, \beta > 0$. The optimal allocation for a fixed total budget C_{total} satisfies the condition that marginal return per FLOP is equalized between training and inference:

$$\frac{\partial \mathcal{A}}{\partial C_{\text{train}}} = \frac{1}{N} \cdot \frac{\partial \mathcal{A}}{\partial C_{\text{test}}} \tag{13.47}$$

Intuitively: one FLOP of training benefits all N queries, while one FLOP of test-time benefits only one query. At the optimum, the per-query marginal value of test-time compute is N times larger than training compute (because training is amortized). Applying this to Eq. [eq:reasoning_scaling_law] gives the optimal training compute fraction:

$$\frac{C^*_{\text{train}}}{C_{\text{total}}} = \frac{\alpha}{\alpha + \beta} \tag{13.48}$$

For the specific budget structure $C_{\text{total}} = C_{\text{train}} + N \cdot C_{\text{test}}$, this fraction is independent of N under the multiplicative accuracy model. However, in practice α and β are problem-dependent: for high-volume deployments (large N), even small improvements in the base model dominate, favoring training investment. For low-volume, high-stakes queries (small N), test-time compute is more cost-effective.

13.7.2 When to Invest in Longer Chains vs. Better Base Models

Reasoning Chain Length vs. Model Capacity

The optimal reasoning chain length L^* for a model of capacity C on a problem of difficulty D satisfies:

$$L^* \propto \frac{D}{C^\gamma} \tag{13.49}$$

for some $\gamma > 0$. This implies:

- **Hard problems** require longer chains regardless of model size
- **Larger models** require shorter chains for the same problem difficulty
- **Diminishing returns**: Beyond L^*, additional tokens provide no benefit and may hurt (overthinking)

The "overthinking" phenomenon—where models with very long reasoning chains perform *worse* than those with moderate chains—has been empirically observed and is attributed to:

- Accumulation of errors in long chains (error propagation)
- Distraction from the main solution path
- Overconfidence in incorrect intermediate conclusions

13.7.3 Optimal Token Budget Allocation

For a model with a fixed token budget B, the allocation between "thinking" tokens T_{think} and "answering" tokens T_{answer} should satisfy:

$$T^*_{\text{think}} = \arg\max_T \mathcal{A}(T, B - T) \tag{13.50}$$

Empirically, the optimal split is problem-dependent:

- **Simple problems**: $T^*_{\text{think}}/B \approx 0.3$ (30% thinking)
- **Hard problems**: $T^*_{\text{think}}/B \approx 0.8$ (80% thinking)
- **Very hard problems**: $T^*_{\text{think}}/B \approx 0.95$ (95% thinking, minimal answer)

This motivates **adaptive thinking budgets**: allocating more tokens to harder problems, which can be estimated by the model's uncertainty on initial solution attempts.

13.8 Comparison of Reasoning Models

Table 13.3: Comparison of training methodologies for reasoning models.

Method	PRM	ORM	MCTS	Distill	Tool	Open
OpenAI o1/o3	✓	✓	Unknown	–	✓	×
DeepSeek-R1	×	✓	×	✓	×	✓
QwQ / Qwen3	Partial	✓	×	×	✓	✓
AlphaProof	✓	✓	✓	–	✓	×
Math-Shepherd	✓	✓	×	–	×	✓
STaR / Quiet-STaR	×	✓	×	–	×	✓

13.9 Inference-Time Compute Scaling Laws

Training-time scaling laws (Chinchilla [150]) are well-understood: model performance improves predictably with compute, data, and parameters. **Inference-time compute scaling** is the new frontier—the empirical finding that allocating more computation *at test time* (longer reasoning chains, more search, iterative refinement) can solve problems that larger base models cannot.

The Core Finding: Test-Time Compute as a Scaling Axis

Snell et al. (2024) [353] established that test-time compute can be more effective than scaling model parameters for hard reasoning tasks:

- Performance improves **log-linearly** with inference tokens: $\text{acc} \approx a \cdot \log(\text{tokens}) + b$.
- There exist **task-specific optimal token budgets** beyond which returns diminish sharply.
- A smaller model with $k\times$ more inference compute can match a larger model trained with $k\times$ more parameters—at lower serving cost for rare hard queries.

Empirical landmark: **o3** achieved **45.1% on ARC-AGI** (a benchmark designed to resist pattern matching) by scaling inference-time search, far exceeding what model size alone could achieve.

13.9.1 The Overthinking Problem

Inference-time scaling introduces a critical failure mode: **overthinking**. Reasoning models trained to generate long chains of thought continue generating tokens even when the answer is obvious, wasting compute and sometimes degrading accuracy by introducing spurious reasoning steps.

Overthinking: When More Tokens Hurt

Reasoning models exhibit a characteristic **U-shaped accuracy curve** as a function of token budget on easy problems:

1. Short chains: insufficient reasoning, low accuracy.
2. Optimal chains: correct reasoning, peak accuracy.
3. Excessively long chains: the model second-guesses correct answers, introduces errors, accuracy *decreases*.

This means that a fixed "think as long as possible" policy is suboptimal. Models need to learn **when to stop thinking**.

13.9.2 Efficient Reasoning Techniques

Several approaches address the compute-quality trade-off in inference-time scaling:

- **Sketch-of-Thought**: Generate a compressed reasoning sketch before the full chain. Achieves **84% token reduction** with minimal accuracy loss by separating planning from execution in the reasoning trace.
- **Budget-aware prompting**: Explicitly instruct the model with a token budget ("solve this in under 200 tokens"). Halves inference costs on benchmarks where problems have known difficulty.
- **Shorter chains boost accuracy**: On some tasks, enforcing shorter reasoning chains improves accuracy by **34.5%**—the model is forced to be more direct and avoids overthinking spirals.
- **Adaptive depth**: OpenAI's `reasoning_effort` parameter (low/medium/high) routes queries to different token budgets based on estimated difficulty, providing a practical API for compute-quality trade-offs.

13.9.3 Connection to RL: Inference Compute as MDP Horizon

Inference-Time Compute in the MDP Framework

Inference-time compute scaling has a natural interpretation in the RL framework used throughout this book:

- Each reasoning token is a **step in the MDP**.
- Allocating more inference compute = **longer horizon** with the same reward function.
- The overthinking problem = **reward hacking** on a proxy (token count) rather than task success.
- Efficient reasoning = learning an **optimal stopping policy** within the MDP.

This framing suggests that inference-time scaling laws are not merely empirical curiosities—they reflect fundamental properties of the RL optimization landscape. Models trained with RL on reasoning tasks are implicitly learning both *what to think* and *how long to think*.

13.9.4 Latent Reasoning: Is the Chain the Computation?

A growing body of evidence challenges the assumption that chain-of-thought *is* the reasoning:

- **Reasoning Beyond Chain-of-Thought** [178]: Using Sparse Autoencoders to probe internal representations, this work identifies reasoning-specific features that activate *before* the corresponding tokens are generated—suggesting the model "knows the answer" before writing the chain.
- **Thinking to Recall** [123] (Google, COLM 2026): Demonstrates that generating a reasoning trace unlocks parametric knowledge that is otherwise effectively unreachable via direct prompting—but the trace itself may be a *scaffold* rather than the computation.

CoT as Communication Protocol

The emerging view: chain-of-thought may function as a *communication protocol* between the model's internal reasoning circuits and its output layer—a way to "route" through latent states that contain the answer—rather than being the algorithm itself. This has practical implications: (1) verifying reasoning quality cannot rely solely on inspecting the chain, since the model may reason correctly with a misleading or post-hoc trace; (2) future architectures may perform reasoning entirely in latent space, emitting only conclusions.

13.10 On-Policy Self-Distillation for Reasoning

The methods in this chapter—GRPO, process rewards, MCTS—share a fundamental limitation: **sparse feedback**. Reinforcement learning provides a fixed number of bits per episode regardless of sequence length. A 2000-token chain-of-thought that arrives at the wrong answer receives a single scalar penalty; the model does not learn *which tokens* contributed to the failure. On-Policy Self-Distillation (OPSD) [453] addresses this by providing **dense, token-level supervision** while retaining the on-policy property that makes RL effective.

On-Policy Self-Distillation: Core Idea

A single LLM plays two roles simultaneously:

- **Teacher**: the same model conditioned on *privileged information* (e.g., a verified solution or reasoning trace appended to the prompt).
- **Student**: the model with only the original question as input, generating its own rollouts.

Training minimizes the per-token divergence between the teacher's and student's distributions, evaluated on the *student's own trajectories*. No separate, larger teacher model is required.

13.10.1 The Training Objective

Let π_θ denote the student policy (the model given only the question) and π_θ^+ denote the teacher policy (the same model conditioned on privileged context such as a verified answer). For a trajectory $x = (x_1, \ldots, x_T)$ sampled from the student, the OPSD loss minimizes the per-token reverse KL divergence:

$$\mathcal{L}_{\text{OPSD}}(\theta) = \mathbb{E}_{x \sim \pi_\theta}\left[\sum_{t=1}^{T} \text{KL}\Big(\pi_\theta(\cdot \mid x_{<t}) \,\|\, \pi_\theta^+(\cdot \mid x_{<t})\Big)\right]$$

In practice, this simplifies to the difference in log-probabilities evaluated on the sampled tokens:

$$\mathcal{L}_{\text{OPSD}}(\theta) \approx \mathbb{E}_{x \sim \pi_\theta}\left[\sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t}) - \log \pi_\theta^+(x_t \mid x_{<t})\right]$$

The teacher's log-probabilities require only a single forward pass (no gradient computation), making the approach significantly cheaper than running a separate reward model or performing full RL rollouts with delayed rewards.

13.10.2 Why Dense Supervision Matters

The information-theoretic argument is compelling: RL teaches $O(1)$ bits per episode (the scalar reward), while token-level distillation teaches $O(T)$ bits per episode. This translates directly into training efficiency:

- **Qwen3 results**: The Qwen3 technical report reports that on-policy distillation achieves 74.4% on AIME'24 at **one-tenth the GPU-hours** of their RL pipeline (1,800 vs. 17,920 GPU-hours), while exceeding RL's 67.6%.
- **Step efficiency**: Empirical comparisons show OPSD reaching equivalent reasoning performance in 7–10× fewer gradient steps than GRPO, corresponding to 50–100× compute savings when accounting for shorter required context lengths and smaller batch sizes.
- **Data reuse**: Unlike RL, which memorizes final answers when trained on repeated prompts, OPSD learns to approximate the teacher's full distribution via reverse KL, enabling effective multi-epoch training on small prompt sets.

13.10.3 Failure Modes and Mitigations

Privilege-Induced Style Drift

When the teacher is conditioned on a verified solution, it tends to produce shorter, more direct outputs—it already "knows" the answer. The resulting distributional gap between teacher and student concentrates on *style tokens* (brevity, directness) rather than *task-bearing tokens* (correct reasoning steps). This causes the student's response length to shrink without improving accuracy.

RLCSD [298] addresses this pathology through a **contrastive** formulation: it computes the teacher-student gap under both a correct hint and an incorrect hint, then subtracts the latter to cancel out the style component that conditioning on *any* hint induces. The residual signal concentrates on task-relevant tokens.

EGRSD [191] takes a complementary approach: it introduces an **entropy-guided confidence gate** that down-weights token positions where the teacher's predictive distribution is high-entropy (uncertain), focusing the gradient on positions where the teacher is confident and the student diverges.

13.10.4 Connection to RL and the Training Pipeline

OPSD occupies a specific niche in the post-training pipeline:

Figure 13.6: OPSD bridges SFT and RL: it combines on-policy sampling (avoiding distribution mismatch) with dense token-level supervision (avoiding reward sparsity).

The practical pipeline emerging in frontier labs is: (1) SFT on teacher-generated reasoning traces to initialize the policy within the correct distribution; (2) OPSD to cheaply recover or refine reasoning capabilities with dense supervision; (3) RL for the final push on hard problems where OPSD's teacher signal becomes unreliable (the teacher itself cannot solve them).

When to Use OPSD vs. RL

- **Use OPSD**: When the teacher (or the model itself with privileged context) reliably solves the target tasks; when compute budget is limited; when you need to recover reasoning behavior after domain-specific fine-tuning; for continual learning where RL's sparsity causes catastrophic forgetting.
- **Use RL**: When pushing beyond the teacher's capability ceiling; when exploring novel reasoning strategies that don't exist in any teacher distribution; when verifiable rewards are cheap (code execution, math verification).
- **Use both**: OPSD first to cheaply reach the teacher's level, then RL to push beyond it.

13.11 Summary and Open Problems

The field of RL for reasoning models has advanced remarkably rapidly. Several key lessons have emerged:

1. **Verifiable rewards are sufficient**: For domains with ground-truth verification (math, code), outcome-only rewards are sufficient for RL to discover sophisticated reasoning strategies, without requiring process reward models.
2. **Test-time compute is a new axis**: Reasoning models introduce a new dimension of scaling—inference compute—that is roughly substitutable with training compute for hard reasoning tasks.
3. **Distillation is highly effective**: Large reasoning models can transfer their capabilities to much smaller models via supervised fine-tuning on generated chains, often outperforming

direct RL training of small models. On-policy self-distillation further closes the gap between distillation and RL at a fraction of the compute.

4. **Emergent meta-cognition**: RL training on reasoning tasks produces emergent self-correction and verification behaviors that were not explicitly trained.

5. **Dense supervision beats sparse rewards**: On-policy self-distillation achieves RL-level reasoning at 10–100× lower compute by providing token-level feedback rather than episode-level rewards.

Open Problems in RL for Reasoning

Several fundamental questions remain open:

- **Generalization**: Do reasoning capabilities trained on math/code transfer to other domains (scientific reasoning, planning, social reasoning)?
- **Faithfulness**: Are the generated reasoning chains causally responsible for the final answer, or are they post-hoc rationalizations? Latent reasoning research suggests the chain may be a communication protocol rather than the computation itself.
- **Optimal search**: What is the optimal search strategy during inference—beam search, MCTS, or something else?
- **Reward design**: For domains without ground-truth verifiers, how can we design reliable reward signals for reasoning?
- **Overthinking**: How can models learn to allocate the *right* amount of thinking—neither too little nor too much? Inference-time scaling laws show diminishing and eventually negative returns beyond task-specific optimal budgets.
- **Compositional reasoning**: Can RL-trained reasoning models solve problems that require composing multiple distinct reasoning skills?
- **Latent vs. explicit reasoning**: Can future architectures reason entirely in continuous latent space, emitting only conclusions—and if so, how do we verify their reasoning?
- **Self-distillation ceiling**: OPSD is bounded by the teacher's capability (the model conditioned on privileged context). How can we break this ceiling without reverting to expensive RL exploration?

The development of reasoning models represents a paradigm shift: from language models that *know* things to language models that can *figure things out*. The methods described in this chapter—from RL with verifiable rewards through inference-time compute scaling to on-policy self-distillation—form a rapidly expanding toolkit, and their continued development is likely to be a central focus of AI research in the coming years.

Part IV

Evaluation

Chapter 14

LLM Evaluation

Evaluation is the backbone of any rigorous machine learning pipeline, yet it is perhaps the most underappreciated component in the development of large language models. Unlike classical supervised learning, where a held-out test set with ground-truth labels provides a clean signal, evaluating LLMs requires grappling with open-ended generation, subjective quality judgments, multi-step reasoning chains, and the ever-present risk of benchmark contamination. This chapter provides a systematic treatment of the evaluation landscape: from the taxonomy of evaluation types and the mechanics of human annotation, through the mathematics of ranking metrics and the practicalities of LLM-as-judge, to the pitfalls that silently corrupt evaluation pipelines.

GAS Connection — LLM Evaluation

Evaluating the GAS is more complex than evaluating a single LLM: each agent has its own metrics, and the system must be evaluated both component-wise and end-to-end. The following chapter's evaluation framework provides the structure for the GAS evaluation pipeline demonstrated in `part4_evaluation/ch14_llm_evaluation/notebook.ipynb`.

Evaluation scheme design (§ Evaluation Scheme Design). The GAS uses a three-level evaluation scheme: (1) *component evaluation*—each agent node is evaluated independently on a held-out test set; (2) *integration evaluation*—the full pipeline is evaluated on the five scripted clinical scenarios in `shared/patient_data.py`; (3) *simulation evaluation*—the system is run for multi-hour simulated patient episodes and evaluated on Time-in-Range, hypoglycaemia rate, and alert precision/recall.

Metrics for generation tasks (§ Metrics for Generation Tasks). The Advisor's recommendations are evaluated with: (1) *Clinical accuracy*: does the recommendation align with the 15 ADA guidelines in `MEDICAL_KNOWLEDGE` (in `capstone_gas/gas_system.py`)? (2) *Safety rate*: fraction of recommendations approved by the Safety Critic (`safety_score` $\geq$ `0.7`); (3) *Specificity*: does the recommendation include a specific action rather than a vague suggestion?

Metrics for agentic tasks (§ Metrics for Agentic Tasks). The full GAS pipeline is evaluated with agentic metrics: (1) *Task success rate*: fraction of patient scenarios where the correct alert level is assigned within the latency budget; (2) *Retrieval accuracy*: fraction of `GASKnowledgeBase` queries that return a relevant guideline (evaluated by keyword overlap); (3) *Escalation precision*: fraction of `caregiver_notified = True` events that were genuinely necessary.

LLM-as-Judge (§ LLM-as-Judge). The GAS uses a stronger LLM (e.g. GPT-4o) as a judge to evaluate Advisor recommendations on dimensions that are hard to score automatically: clinical reasoning quality, appropriate hedging, and correct identification of contraindications. The judge prompt follows the structured rubric format described in this section and is demonstrated in `part4_evaluation/ch14_llm_evaluation/notebook.ipynb`.

Red-teaming (§ Evaluation Pitfalls). The GAS Safety Critic is red-teamed with adversarial inputs designed to elicit dangerous recommendations: prompts that describe a patient in distress but include subtle cues that should trigger conservative behaviour, and prompts that use medical jargon to obscure a dangerous suggestion. The `NOTEBOOK_TEST_MODE=1` flag enables deterministic mock responses for reproducible red-team evaluation.

Why Evaluation is Hard for LLMs

Three fundamental challenges distinguish LLM evaluation from classical ML evaluation:

1. **Output space is unbounded.** A language model can produce any string; there is rarely a single correct answer.
2. **Quality is multidimensional.** Helpfulness, factuality, safety, coherence, and style are distinct axes that may trade off against each other.
3. **Evaluation is itself a language task.** Judging whether a response is good requires understanding, which means evaluation is susceptible to the same failure modes as generation.

14.1 Evaluation Scheme Design

Before collecting a single data point, practitioners must decide *what* to measure and *how* to measure it. A principled taxonomy prevents the common mistake of choosing metrics by convenience rather than by alignment with the deployment objective.

14.1.1 Taxonomy of Evaluation Types

Intrinsic vs. Extrinsic Evaluation. *Intrinsic* evaluation measures properties of the model output in isolation, without reference to a downstream application. Perplexity on a held-out corpus, BLEU score against reference translations, and pass@k on coding benchmarks are all intrinsic. *Extrinsic* evaluation measures the impact of the model on a real-world task or system: does integrating the LLM into a customer-service pipeline reduce ticket escalation rates? Does the coding assistant increase developer velocity?

The Intrinsic–Extrinsic Gap

Intrinsic metrics are cheap and reproducible but often poorly correlated with real-world utility. A model with lower perplexity is not necessarily more helpful. Extrinsic metrics are expensive and slow but directly measure what we care about. A mature evaluation strategy uses intrinsic metrics for rapid iteration and extrinsic metrics for final validation.

Automatic vs. Human Evaluation. *Automatic* evaluation uses deterministic functions (BLEU, exact match) or learned models (BERTScore, LLM-as-judge) to score outputs without human involvement. *Human* evaluation involves annotators rating or ranking model outputs. Table 14.1 summarises the trade-offs.

Table 14.1: Taxonomy of evaluation approaches with key trade-offs.

Type	Cost	Speed	Reproducibility	Validity
Automatic (rule-based)	Very low	Very fast	Perfect	Low–Medium
Automatic (model-based)	Low	Fast	High	Medium–High
Crowdsourced human	Medium	Days	Medium	Medium
Expert human	High	Weeks	Low–Medium	High
Extrinsic / A/B test	Very high	Months	Low	Very high

Reference-Based vs. Reference-Free Evaluation. Reference-based metrics (BLEU, ROUGE, BERTScore) compare model output to one or more gold-standard references. Reference-free metrics (perplexity, LLM-as-judge, human preference) assess quality without a reference. Reference-free approaches are essential when the output space is too large for exhaustive reference collection, as in open-ended dialogue.

14.1.2 When to Use What

Evaluation Strategy for a Dialogue Assistant

Development phase: Use automatic metrics (perplexity, ROUGE on summarisation sub-tasks, pass@k on tool-use) for rapid iteration. Run nightly benchmarks on standard suites (MMLU, HellaSwag, HumanEval).
Pre-release phase: Conduct a human preference study comparing the new model to the previous checkpoint. Use LLM-as-judge for scalable pairwise comparison on a diverse prompt set.
Post-release phase: Monitor extrinsic metrics (user satisfaction scores, task completion rates) and watch for distribution shift in production prompts.

A useful decision framework:

- If the task has a clear correct answer (math, code, factual QA): use exact match or execution-based metrics.
- If the task is open-ended but has reference outputs: use reference-based metrics as a lower bound, supplement with LLM-as-judge.
- If the task is subjective (helpfulness, tone, creativity): use human evaluation or a well-calibrated LLM judge.
- If the task involves multi-step agent behaviour: use task success rate and trajectory efficiency (Section 14.6).

14.2 Data Collection for Evaluation

High-quality evaluation data is the foundation of trustworthy benchmarks. This section covers the design of human annotation pipelines, statistical measures of annotation quality, and the choice between crowdsourcing and expert annotation.

14.2.1 Human Annotation Pipelines

A robust annotation pipeline consists of five stages:

1. **Task definition.** Specify the annotation task precisely: what is being rated, on what scale, and with what criteria. Ambiguity at this stage propagates into noisy labels.
2. **Guideline development.** Write annotation guidelines with worked examples covering edge cases. Iterate with a small pilot group before full deployment.
3. **Annotator recruitment and training.** Select annotators with appropriate background knowledge. Conduct a calibration session where annotators label the same examples and discuss disagreements.
4. **Quality control.** Embed gold-standard examples with known labels into the annotation queue. Flag annotators whose accuracy on gold examples falls below a threshold.
5. **Aggregation.** Combine multiple annotations per item using majority vote, averaging, or a probabilistic model (e.g., Dawid–Skene).

14.2.2 Inter-Annotator Agreement

Raw agreement (fraction of items where all annotators agree) is an inadequate measure because it does not account for chance agreement. Two standard chance-corrected measures are Cohen's κ [62] (two annotators) and Fleiss' κ [97] (multiple annotators).

Cohen's Kappa. Given two annotators labelling N items into k categories, let p_o be the observed agreement and p_e be the expected agreement under independence:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{14.1}$$

where

$$p_o = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{annotator 1 agrees with annotator 2 on item } i] \tag{14.2}$$

and

$$p_e = \sum_{c=1}^{k} p_{1c} \cdot p_{2c} \tag{14.3}$$

with p_{jc} being the proportion of items assigned to category c by annotator j. Cohen's κ ranges from -1 (perfect disagreement) through 0 (chance agreement) to 1 (perfect agreement). Values above 0.6 are generally considered acceptable; above 0.8 is strong agreement.

Fleiss' Kappa. For n annotators labelling N items into k categories, let n_{ij} be the number of annotators who assigned item i to category j. Define:

$$\bar{P}_i = \frac{1}{n(n-1)}\sum_{j=1}^{k} n_{ij}(n_{ij}-1), \qquad \bar{P} = \frac{1}{N}\sum_{i=1}^{N}\bar{P}_i \tag{14.4}$$

$$\bar{P}^e_j = \frac{1}{Nn}\sum_{i=1}^{N} n_{ij}, \qquad P_e = \sum_{j=1}^{k}\left(\bar{P}^e_j\right)^2 \tag{14.5}$$

$$\kappa_F = \frac{\bar{P} - P_e}{1 - P_e} \tag{14.6}$$

Kappa Limitations

Kappa is sensitive to the prevalence of categories: when one category dominates, kappa can be low even when raw agreement is high (the *kappa paradox*). For ordinal scales, weighted kappa (which penalises disagreements proportionally to their distance) is more appropriate. For LLM evaluation, where ratings are often on a 1–5 Likert scale, always report weighted kappa.

14.2.3 Annotation Guideline Design

Effective annotation guidelines share several properties:

- **Operationalised criteria.** Replace vague terms like "helpful" with concrete, observable behaviours: "The response directly addresses the user's question and provides all information needed to complete the stated task."
- **Worked examples.** Provide at least two examples per rating level, including borderline cases.
- **Decision trees.** For complex tasks, a flowchart that guides annotators through a sequence of binary decisions reduces cognitive load and improves consistency.
- **Explicit scope.** State what annotators should *not* consider (e.g., "Do not penalise for stylistic preferences; focus only on factual accuracy").

14.2.4 Crowdsourcing vs. Expert Annotation

For safety-critical evaluation (e.g., detecting harmful outputs, evaluating medical advice), expert annotation is non-negotiable. For large-scale preference collection (e.g., building a reward model training set), crowdsourcing with rigorous quality control is often the only feasible option.

Table 14.2: Comparison of crowdsourcing and expert annotation for LLM evaluation.

Dimension	Crowdsourcing	Expert Annotation
Cost per item	Low ($0.01 - -0.10$)	High ($1 - -50$)
Throughput	Very high	Low
Domain knowledge	Low	High
Consistency	Variable	High
Suitable tasks	Simple preference, fluency	Technical accuracy, safety
Platforms	MTurk, Prolific, Scale AI	Domain specialists, in-house
Quality control	Gold examples, attention checks	Calibration sessions, peer review

14.3 Synthetic Data Generation for Evaluation

Human annotation is expensive and slow. Synthetic data generation uses LLMs themselves to produce evaluation data at scale. This section covers the major paradigms.

14.3.1 LLM-as-Judge for Calibration

When using an LLM to generate evaluation labels, calibration is essential: the judge's scores must be aligned with human judgments. Let $h_i \in [0, 1]$ be the human preference score for item i and $\hat{h}_i$ be the judge's predicted score. Calibration error is measured by the Expected Calibration Error (ECE) [132]:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{n} \left|\text{acc}(B_b) - \text{conf}(B_b)\right| \tag{14.7}$$

where B_b is the b-th confidence bin, $\text{acc}(B_b)$ is the fraction of items in the bin where the judge agrees with humans, and $\text{conf}(B_b)$ is the mean judge confidence in that bin.

A well-calibrated judge satisfies $\mathbb{E}[\hat{h}_i \mid \hat{h}_i = p] = p$ for all $p \in [0, 1]$. Calibration can be improved by temperature scaling: replacing the judge's raw logit z with z/T where T is tuned on a held-out calibration set to minimise negative log-likelihood.

14.3.2 Self-Instruct

Self-Instruct [392] bootstraps instruction-following data from a seed set of human-written tasks. The algorithm:

1. Maintain a task pool initialised with 175 seed tasks.
2. Sample 8 tasks from the pool; use them as few-shot examples to prompt the LLM to generate new tasks.
3. Filter generated tasks: remove near-duplicates (ROUGE-L similarity > 0.7 with any existing task), classify as classification vs. non-classification, and generate input–output instances.
4. Add accepted tasks to the pool.
5. Repeat until the desired pool size is reached.

Self-Instruct Prompt Template

```
system_prompt = """
Come up with a series of tasks:
Task 1: {seed_task_1_instruction}
Task 2: {seed_task_2_instruction}
...
Task 8: {seed_task_8_instruction}
Task 9:"""
```

The model completes the prompt, generating a new task instruction. A separate prompt then generates input–output pairs for the new task.

14.3.3 Evol-Instruct

Evol-Instruct [420] evolves a seed instruction set by iteratively rewriting instructions to be more complex or diverse. Two evolution operators are applied:

- **In-depth evolution:** Add constraints, increase reasoning steps, concretise abstractions, deepen domain knowledge requirements.
- **In-breadth evolution:** Generate a new instruction on a related but different topic, increasing topic diversity.

An instruction is accepted if it passes an elimination filter: the evolved instruction must not be a simple copy, must not contain "I'm sorry" or similar refusals, and must not be shorter than the original.

14.3.4 Constitutional AI Data Generation

Constitutional AI (CAI) [17] generates preference data by having the model critique and revise its own outputs according to a set of principles (the "constitution"). The pipeline:

1. **Supervised learning phase:** Sample a harmful prompt, generate an initial response, then prompt the model to critique the response according to a constitutional principle and revise it. Use the revised response as a supervised fine-tuning target.
2. **RL phase:** Generate pairs of responses (original vs. revised), use the model to label which is more constitutional, and train a preference model on these labels. Use the preference model as a reward signal for RLHF.

This approach generates preference data without human labelling of harmful content, reducing annotator exposure to distressing material.

14.3.5 Distillation for Evaluation Data

A powerful teacher model (e.g., GPT-4) can generate high-quality evaluation data for training a smaller judge model. The distillation pipeline:

1. Collect a diverse set of prompts and model responses.
2. Use the teacher to generate detailed judgments (scores + rationales).
3. Fine-tune a smaller model on (prompt, response, judgment) triples.
4. Validate the student judge against held-out human annotations.

Distillation Bias

A student judge distilled from a single teacher inherits the teacher's biases, including verbosity bias (preferring longer responses), self-enhancement bias (if the teacher is also the model being evaluated), and positional bias. Always validate distilled judges against independent human annotations.

14.3.6 Arena-Style Pairwise Generation

Chatbot Arena [456] generates evaluation data through a crowdsourced battle platform where users submit prompts and vote on which of two anonymised model responses they prefer. This produces a large-scale, naturally diverse dataset of pairwise preferences. The key design choices:

- **Anonymisation:** Model identities are hidden to prevent brand bias.
- **User-submitted prompts:** Ensures prompt diversity and real-world relevance.
- **Tie handling:** Users can declare a tie or indicate that both responses are bad.
- **Deduplication:** Near-duplicate prompts are filtered to prevent over-representation of common queries.

14.4 Metrics for Ranking Tasks

When the goal is to rank models by quality, pairwise comparison data is more reliable than absolute scores. This section derives the major ranking systems used in LLM evaluation.

14.4.1 ELO Rating System

The ELO system [88], originally developed for chess, assigns each player (model) a scalar rating R such that the expected score of player A against player B is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{14.8}$$

Derivation. The ELO model assumes that each player's performance on a given game is a random variable drawn from a logistic distribution centred at their rating. The probability that A beats B is:

$$P(A \succ B) = \sigma\left(\frac{R_A - R_B}{s}\right) = \frac{1}{1 + e^{-(R_A - R_B)/s}} \tag{14.9}$$

where $s = 400/\ln(10) \approx 173.7$ is a scale parameter chosen so that a 400-point difference corresponds to a $10:1$ odds ratio. After each game with outcome $S_A \in \{0, 0.5, 1\}$ (loss, draw, win), ratings are updated:

$$R_A \leftarrow R_A + K(S_A - E_A), \qquad R_B \leftarrow R_B + K(S_B - E_B) \tag{14.10}$$

where K is the K-factor controlling the learning rate. In Chatbot Arena, $K = 4$ is used.

ELO Intuition

ELO is a stochastic gradient descent update on the log-likelihood of the observed outcomes under the logistic model. Each game provides a noisy gradient signal; the K-factor controls the step size. A large K adapts quickly but is noisy; a small K is stable but slow to reflect true skill changes.

Bootstrap Confidence Intervals for ELO. Because ELO ratings depend on the order in which games are processed, confidence intervals are computed by bootstrap resampling: resample the battle log with replacement $B = 1000$ times, recompute ELO ratings from scratch for each resample, and report the 2.5th and 97.5th percentiles as the 95% confidence interval.

14.4.2 Bradley–Terry Model

The Bradley–Terry (BT) model [31] is a maximum-likelihood alternative to ELO. Given n models with strength parameters $\beta_1, \ldots, \beta_n > 0$, the probability that model i beats model j is:

$$P(i \succ j) = \frac{\beta_i}{\beta_i + \beta_j} \tag{14.11}$$

Given a set of pairwise outcomes $\{(i_k, j_k, y_k)\}_{k=1}^{M}$ where $y_k = 1$ if i_k beats j_k and $y_k = 0$ otherwise, the log-likelihood is:

$$\ell(\boldsymbol{\beta}) = \sum_{k=1}^{M} \left[y_k \log \frac{\beta_{i_k}}{\beta_{i_k} + \beta_{j_k}} + (1 - y_k) \log \frac{\beta_{j_k}}{\beta_{i_k} + \beta_{j_k}} \right] \tag{14.12}$$

The MLE $\hat{\boldsymbol{\beta}}$ is found by iterative scaling or gradient ascent. The BT model is identifiable only up to a multiplicative constant; a common normalisation is $\sum_i \log \beta_i = 0$. Working in log-space with $\theta_i = \log \beta_i$ gives:

$$P(i \succ j) = \sigma(\theta_i - \theta_j) \tag{14.13}$$

which is equivalent to a logistic regression with item-specific intercepts. The BT model is preferred over ELO when the full battle history is available, as it uses all data simultaneously rather than processing games sequentially.

14.4.3 TrueSkill

TrueSkill [147] is a Bayesian skill rating system that models each player's skill as a Gaussian random variable $s_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$. The performance of player i in a game is $p_i = s_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \beta^2)$ is game-specific noise. Player i beats player j if $p_i > p_j$.

The posterior update after observing $i \succ j$ is computed via expectation propagation (EP). The key update equations for the winner are:

$$\mu_i \leftarrow \mu_i + \frac{\sigma_i^2}{c} \cdot v\left(\frac{\mu_i - \mu_j}{c}\right) \tag{14.14}$$

$$\sigma_i^2 \leftarrow \sigma_i^2 \left[1 - \frac{\sigma_i^2}{c^2} \cdot w\left(\frac{\mu_i - \mu_j}{c}\right)\right] \tag{14.15}$$

where $c = \sqrt{2\beta^2 + \sigma_i^2 + \sigma_j^2}$, and $v(t) = \phi(t)/\Phi(t)$, $w(t) = v(t)(v(t) + t)$ are the truncated Gaussian correction factors (ϕ and Φ are the standard normal PDF and CDF). TrueSkill's uncertainty estimate σ_i is particularly useful for identifying models that need more evaluation data.

14.4.4 Win Rate with Confidence Intervals

The simplest ranking metric is the win rate: the fraction of pairwise comparisons in which model A is preferred. Given n comparisons with w wins, the win rate is $\hat{p} = w/n$. A Wilson score confidence interval [402] is preferred over the naive Wald interval because it has better coverage near $p = 0$ and $p = 1$:

$$\text{CI} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{14.16}$$

where $z = 1.96$ for a 95% interval. For multi-way comparisons, win rate should be computed against a fixed baseline model to ensure comparability.

14.4.5 Chatbot Arena Methodology

Chatbot Arena [456] combines the above elements into a production-scale evaluation system:

1. Users submit prompts and receive responses from two anonymised models.
2. Users vote for the preferred response (or declare a tie).
3. Votes are aggregated using the BT model to produce a leaderboard.
4. Bootstrap confidence intervals are reported for each model's score.
5. Models with overlapping confidence intervals are considered statistically indistinguishable.

As of 2024, Chatbot Arena has collected over one million human preference votes, making it the largest publicly available LLM preference dataset.

14.5 Metrics for Generation Tasks

Generation metrics quantify the quality of model outputs for tasks with reference answers or well-defined correctness criteria.

14.5.1 BLEU

BLEU (Bilingual Evaluation Understudy) [299] measures n-gram precision between a hypothesis h and one or more references $\mathcal{R}$:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{14.17}$$

where p_n is the modified n-gram precision, $w_n = 1/N$ are uniform weights, and BP is the brevity penalty:

$$\text{BP} = \begin{cases} 1 & \text{if } |h| > |r| \\ e^{1-|r|/|h|} & \text{if } |h| \leq |r| \end{cases} \tag{14.18}$$

with $|r|$ being the length of the closest reference. Modified n-gram precision clips each n-gram count to its maximum count in any reference:

$$p_n = \frac{\sum_{\text{ngram} \in h} \min(\text{count}(\text{ngram}, h),\ \max_{r \in \mathcal{R}} \text{count}(\text{ngram}, r))}{\sum_{\text{ngram} \in h} \text{count}(\text{ngram}, h)} \tag{14.19}$$

BLEU Limitations

BLEU was designed for machine translation with multiple references. For open-ended generation with a single reference, BLEU scores are often near zero even for high-quality outputs. BLEU does not capture semantic similarity, penalises valid paraphrases, and is sensitive to tokenisation. Use BLEU only when multiple diverse references are available and the task has low output diversity.

14.5.2 ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [228] is a family of recall-oriented metrics designed for summarisation:

$$\text{ROUGE-N} = \frac{\sum_{r \in \mathcal{R}} \sum_{\text{ngram} \in r} \min(\text{count}(\text{ngram}, h), \text{count}(\text{ngram}, r))}{\sum_{r \in \mathcal{R}} \sum_{\text{ngram} \in r} \text{count}(\text{ngram}, r)} \tag{14.20}$$

$$\text{ROUGE-L} = \frac{\text{LCS}(h, r)}{|r|} \tag{14.21}$$

where LCS denotes the longest common subsequence. ROUGE-1 and ROUGE-2 measure unigram and bigram recall; ROUGE-L captures sentence-level structure. The F-measure variant balances precision and recall:

$$\text{ROUGE-N}_F = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 P + R} \tag{14.22}$$

with $\beta = 1$ for equal weighting.

14.5.3 BERTScore

BERTScore [449] computes token-level similarity using contextual embeddings from a pre-trained BERT model. Given hypothesis tokens $\hat{\mathbf{x}} = \langle \hat{x}_1, \ldots, \hat{x}_m \rangle$ and reference tokens $\mathbf{x} = \langle x_1, \ldots, x_n \rangle$ with

embeddings $\hat{\mathbf{e}}_i$ and $\mathbf{e}_j$:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_j \in \mathbf{x}} \max_{\hat{x}_i \in \hat{\mathbf{x}}} \frac{\hat{\mathbf{e}}_i^\top \mathbf{e}_j}{\|\hat{\mathbf{e}}_i\| \|\mathbf{e}_j\|} \tag{14.23}$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_i \in \hat{\mathbf{x}}} \max_{x_j \in \mathbf{x}} \frac{\hat{\mathbf{e}}_i^\top \mathbf{e}_j}{\|\hat{\mathbf{e}}_i\| \|\mathbf{e}_j\|} \tag{14.24}$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{14.25}$$

BERTScore correlates better with human judgments than BLEU and ROUGE, particularly for paraphrases and semantically equivalent but lexically different outputs. Importance weighting using inverse document frequency (IDF) further improves correlation:

$$R_{\text{BERT}}^{\text{idf}} = \frac{\sum_{x_j \in \mathbf{x}} \text{idf}(x_j) \max_{\hat{x}_i} \cos(\hat{\mathbf{e}}_i, \mathbf{e}_j)}{\sum_{x_j \in \mathbf{x}} \text{idf}(x_j)} \tag{14.26}$$

14.5.4 METEOR

METEOR [21] addresses BLEU's recall blindness by computing an F-score over unigram matches, with additional modules for stemming and synonym matching:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Pen}) \tag{14.27}$$

where $F_{\text{mean}} = \frac{10PR}{R+9P}$ (recall-weighted harmonic mean) and the fragmentation penalty $\text{Pen} = 0.5 \cdot (c/u_m)^3$ penalises non-contiguous matches (c = number of chunks, u_m = number of matched unigrams).

14.5.5 Perplexity

Perplexity measures how well a language model predicts a held-out text sequence $w_1, w_2, \ldots, w_T$:

$$\text{PPL}(w_{1:T}) = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(w_t \mid w_{1:t-1}) \right) \tag{14.28}$$

Lower perplexity indicates better predictive performance. Perplexity is useful for comparing models on the same tokenisation and test set, but is not directly comparable across models with different vocabularies or tokenisers. For evaluation purposes, perplexity is most useful as a sanity check and for detecting distribution shift.

14.5.6 Pass@k for Code

For code generation, functional correctness is measured by executing generated code against test cases. The pass@k metric [49] estimates the probability that at least one of k generated samples passes all tests:

$$\text{pass@}k = \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \tag{14.29}$$

where n is the total number of samples generated per problem and c is the number that pass. This unbiased estimator avoids the high variance of the naive estimator (which samples exactly k solutions and checks if any pass). In practice, $n = 200$ samples are generated and pass@1, pass@10, pass@100 are reported.

Pass@k Computation

```
import numpy as np
from scipy.special import comb
```

```python
def pass_at_k(n: int, c: int, k: int) -> float:
    """Unbiased estimator for pass@k.

    Args:
        n: total samples generated per problem
        c: number of samples that pass all tests
        k: number of samples to consider
    """
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k, exact=True) / comb(n, k, exact=True)

# Example: 200 samples, 15 pass, compute pass@1, pass@10, pass@100
for k in [1, 10, 100]:
    score = pass_at_k(n=200, c=15, k=k)
    print(f"pass@{k}: {score:.4f}")
# pass@1:   0.0750
# pass@10:  0.5391
# pass@100: 0.9999
```

14.5.7 Exact Match and F1

For extractive question answering (e.g., SQuAD), two standard metrics are:

- **Exact Match (EM):** Binary indicator of whether the predicted answer string exactly matches any gold answer after normalisation (lowercasing, removing articles and punctuation).
- **Token-level F1:** Treats prediction and gold answer as bags of tokens and computes the F1 score:

$$F1 = \frac{2 \cdot |\text{pred} \cap \text{gold}|}{|\text{pred}| + |\text{gold}|} \tag{14.30}$$

For multi-answer settings, the maximum F1 over all gold answers is reported.

Table 14.3: Summary of generation metrics: applicability and key properties.

Metric	Task	Reference-free?	Human correlation
BLEU	Translation	No	Low–Medium
ROUGE	Summarisation	No	Medium
BERTScore	General NLG	No	High
METEOR	Translation	No	Medium–High
Perplexity	LM quality	Yes	Low
Pass@k	Code generation	No (tests)	Very high
Exact Match	Extractive QA	No	Very high
Token F1	Extractive QA	No	High

14.6 Metrics for Agentic Tasks

Agentic LLMs operate in environments, take sequences of actions, and must complete multi-step tasks. Standard generation metrics are insufficient; agentic evaluation requires metrics that capture task completion, efficiency, and the quality of intermediate steps.

14.6.1 Task Success Rate

The primary metric for agentic tasks is the task success rate (TSR): the fraction of tasks for which the agent achieves the specified goal state:

$$\text{TSR} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \mathbf{1}[\text{goal}(\tau) \text{ achieved}] \tag{14.31}$$

Goal achievement is typically verified by a deterministic oracle (e.g., checking database state, file system state, or test case execution). For tasks with partial credit, a graded success metric can be defined:

$$\text{TSR}_{\text{graded}} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \text{score}(\tau) \in [0, 1] \tag{14.32}$$

14.6.2 Trajectory Efficiency

A successful agent should complete tasks with minimal unnecessary actions. Trajectory efficiency measures the ratio of the optimal trajectory length to the agent's actual trajectory length:

$$\eta = \frac{L^*}{L_{\text{agent}}} \tag{14.33}$$

where L^* is the length of the shortest successful trajectory (computed by an oracle or human expert) and L_{agent} is the number of actions taken by the agent. $\eta \in (0, 1]$ with $\eta = 1$ indicating optimal efficiency. For failed trajectories, $\eta = 0$.

A complementary metric is the *redundancy rate*: the fraction of agent actions that are not present in any optimal trajectory.

14.6.3 Tool-Use Accuracy

For agents that invoke external tools (APIs, code interpreters, search engines), tool-use accuracy measures the correctness of tool calls:

$$\text{TUA} = \frac{\#\text{ correct tool calls}}{\#\text{ total tool calls}} \tag{14.34}$$

A tool call is correct if (a) the correct tool is selected, (b) the arguments are valid, and (c) the call is made at the appropriate point in the trajectory. Partial credit can be assigned for correct tool selection with incorrect arguments.

14.6.4 Multi-Step Reasoning Accuracy

For tasks requiring chains of reasoning (e.g., multi-hop QA, mathematical problem solving), step-level accuracy measures the fraction of reasoning steps that are correct:

$$\text{SRA} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{1}{|S_\tau|} \sum_{s \in S_\tau} \mathbf{1}[s \text{ is correct}] \tag{14.35}$$

where S_τ is the set of reasoning steps in trajectory τ. Step correctness can be verified by a process reward model (PRM) or by human annotation.

14.6.5 SWE-bench Methodology

SWE-bench [179] evaluates LLMs on real-world software engineering tasks: given a GitHub issue description and the repository codebase, the model must generate a patch that resolves the issue. Evaluation proceeds as follows:

1. The model is given the issue description and relevant code context.
2. The model generates a patch (unified diff format).
3. The patch is applied to the repository.
4. The repository's test suite is executed; the task is successful if all tests pass.

The primary metric is **% Resolved**: the fraction of issues for which the generated patch passes all tests. SWE-bench Verified is a curated subset of 500 problems verified by human annotators to be solvable and unambiguous. SWE-bench Lite is a 300-problem subset designed for faster evaluation.

SWE-bench Key Statistics (as of 2024)

- **Full benchmark:** 2,294 tasks from 12 popular Python repositories.
- **Best open-source agent:** ∼43% resolved (SWE-bench Verified).
- **Human performance:** ∼87% resolved (with 15 minutes per task).
- **Evaluation cost:** ∼$0.25 per task for API-based models.

14.6.6 WebArena Methodology

WebArena [463] evaluates agents on realistic web navigation tasks in a sandboxed browser environment. The benchmark includes 812 tasks across five web applications (e-commerce, social forum, collaborative development, content management, and maps). Evaluation:

- **Functional evaluation:** The task outcome is verified by checking the application state (e.g., "Was the item added to the cart?", "Was the post created?").
- **URL-based evaluation:** For navigation tasks, the final URL is compared to the expected URL.
- **Program-based evaluation:** A custom evaluator script checks complex conditions (e.g., "Is the price less than $50?").

The primary metric is task success rate. Human performance is approximately 78%; state-of-the-art agents achieve approximately 35–45%.

Table 14.4: Comparison of agentic evaluation benchmarks.

Benchmark	Domain	# Tasks	Eval Method	SOTA (%)
SWE-bench	Software engineering	2,294	Test execution	∼43
SWE-bench Lite	Software engineering	300	Test execution	∼50
WebArena	Web navigation	812	State/URL/program	∼40
ALFWorld [347]	Household tasks	3,553	Simulator state	∼90
AgentBench [241]	Multi-domain	1,091	Task-specific	∼45

14.7 LLM-as-Judge

LLM-as-judge [456] uses a capable LLM to evaluate the outputs of other (or the same) LLMs. This approach scales to large evaluation sets without human annotation and can provide detailed rationales for its judgments.

14.7.1 Setup and Prompt Templates

The judge is presented with a prompt, one or more model responses, and an evaluation rubric. Three common formats:

Pointwise scoring. The judge assigns an absolute score to a single response:

Pointwise Judge Prompt

```
POINTWISE_PROMPT = """
You are an expert evaluator. Rate the following response on a scale
of 1-10 for helpfulness, accuracy, and clarity.
```

```
[Question]
{question}

[Response]
{response}

Provide your evaluation in the following format:
Reasoning: <step-by-step analysis>
Score: <integer from 1 to 10>
"""
```

Pairwise comparison. The judge compares two responses and selects the better one:

Pairwise Judge Prompt

```
PAIRWISE_PROMPT = """
You are an expert evaluator. Compare the two responses below and
determine which is better. Consider helpfulness, accuracy, and
depth of explanation.

[Question]
{question}

[Response A]
{response_a}

[Response B]
{response_b}

Output exactly one of: [[A]], [[B]], or [[C]] (tie).
Reasoning: <your analysis>
Verdict: <[[A]], [[B]], or [[C]]>
"""
```

Reference-guided scoring. The judge is provided with a reference answer and rates the response relative to it. This is particularly useful for factual tasks where the judge may not have reliable knowledge.

14.7.2 Position Bias Mitigation

LLM judges exhibit *position bias*: a systematic preference for the response appearing in a particular position (first or last). This bias can be as large as 10–15 percentage points. Mitigation strategies:

1. **Swap augmentation:** Evaluate each pair in both orders (A vs. B and B vs. A). A consistent judgment is accepted; an inconsistent judgment is recorded as a tie.
2. **Calibration prompting:** Explicitly instruct the judge: "Your evaluation should not be influenced by the order in which the responses are presented."
3. **Ensemble judging:** Use multiple judges with different position orderings and aggregate their verdicts.
4. **Chain-of-thought forcing:** Require the judge to produce a detailed rationale before the verdict, which reduces reliance on superficial positional cues.

Verbosity Bias

LLM judges also exhibit verbosity bias: longer responses are systematically preferred, even when the additional content is irrelevant or repetitive. To mitigate this, instruct the judge to penalise unnecessary length and to focus on the quality of information rather than quantity. Alternatively,

truncate responses to a fixed length before judging.

14.7.3 Multi-Judge Panels

A single judge may have systematic biases. A panel of judges from different model families provides more robust evaluations. Given J judges with verdicts $v_1, \ldots, v_J \in \{A, B, \text{tie}\}$, the panel verdict is determined by majority vote. The panel agreement rate is:

$$\text{Agreement} = \frac{1}{\binom{J}{2}} \sum_{i<j} \mathbf{1}[v_i = v_j] \tag{14.36}$$

For a three-judge panel, a unanimous verdict (all three agree) is treated as high-confidence; a 2–1 split as medium-confidence; and a three-way tie as low-confidence.

14.7.4 Agreement Metrics for LLM Judges

To validate an LLM judge, its verdicts are compared to human annotations on a held-out set. Key metrics:

- **Agreement rate:** Fraction of items where judge and human agree.
- **Cohen's κ:** Chance-corrected agreement (Equation 14.1).
- **Spearman's ρ:** Rank correlation between judge scores and human scores, appropriate for ordinal ratings.
- **Kendall's τ:** Alternative rank correlation that is more robust to ties.

A judge is considered reliable if it achieves $\kappa > 0.6$ and agreement rate $> 80\%$ with human annotators on a representative sample.

14.7.5 G-Eval Framework

G-Eval [243] is a structured framework for LLM-based evaluation that uses chain-of-thought prompting and token probability weighting to produce more reliable scores. The framework:

1. **Generate evaluation steps:** Prompt the LLM to generate a detailed rubric for the evaluation task (e.g., "List the steps you would take to evaluate the coherence of a summary").
2. **Score with probability weighting:** For each score value $s \in \{1, 2, 3, 4, 5\}$, obtain the log-probability $\log P_\theta(s \mid \text{prompt, steps, response})$ from the judge model. The final score is the probability-weighted average:

$$\text{G-Eval score} = \sum_{s=1}^{5} s \cdot \frac{e^{\log P_\theta(s)}}{\sum_{s'=1}^{5} e^{\log P_\theta(s')}} \tag{14.37}$$

3. **Normalise:** Map the score to $[0, 1]$ by dividing by the maximum score.

G-Eval achieves higher correlation with human judgments than direct prompting, particularly for nuanced dimensions like coherence and consistency, because the probability weighting captures the judge's uncertainty rather than forcing a discrete choice.

Why G-Eval Works

Standard prompting asks the judge to output a single token (e.g., "4"), which discards the model's uncertainty. G-Eval reads the probability distribution over all score tokens, effectively computing the expected score under the judge's belief. This is analogous to using the mean of a posterior distribution rather than the mode.

14.7.6 JEV-as-a-Judge: Decision-Only Judging with Confidence Cascades

Generative LLM judges are accurate but expensive: a single GPT-6 judgment can cost hundreds of times more than a simple rule-based check, and reasoning models add latency on top. Li et al. [223] study whether a *decision-only* judge—one that returns a verdict and label probabilities without generating a rationale—can serve as an economical first pass, escalating only the uncertain cases to a stronger model.

The JEV judge interface. TypeSafe JEV (§8.6.3) accepts natural-language instructions and structured inputs and returns a typed probability distribution over the specified output labels in a single forward pass. For judging, a `Choice` question returns probabilities over `{A, B, tie}` directly, without any chain-of-thought. The **confidence** signal is $q = \max_k p_k$—the maximum label probability.

Accuracy vs. cost. Comparing JEV against sixteen generative and reward-model judges (GPT-4.1 through GPT-6, Claude Sonnet 5, Gemini 3.1 Pro, Skywork-Reward-V2, PairRM) across RewardBench, JudgeBench, HaluEval, and an answer-adjudication set:

- **Ordinary preference (RewardBench):** JEV 92.2% vs. GPT-6 93.5%—within 1.3 points, at 0.36% of GPT-6's fee and 0.15 s vs. 1.9 s median latency.
- **Evidence-grounded factuality (HaluEval):** JEV 87.5% vs. GPT-6 86.7%—JEV is ahead, partly because low-confidence items correspond to mislabeled examples.
- **Difficult correctness (JudgeBench):** JEV 78.6% vs. GPT-6 93.1%—a 14.6-point gap concentrated in reasoning (68.4% vs. 95.9%) and coding. Escalation is warranted here.
- **Style-adversarial pairs (RM-Bench hard):** JEV 74.8% vs. GPT-6 94.6%—JEV is misled by elaborately written wrong answers; confidence does not reliably flag these errors.

The confidence cascade. JEV's key property is that **confidence predicts correctness**: accuracy rises monotonically with q, from 47.7% at $q < 0.6$ to 99.1% at $q = 1.0$. This enables a frozen cascade policy:

1. Run JEV on every item (cheap, fast).
2. If $q \geq \tau$: accept JEV's verdict.
3. If $q < \tau$: escalate to a stronger LLM judge.

At $\tau = 0.9$, the cascade escalates 34% of items and retains 99% of GPT-6's accuracy at 57% of its fee. For pairwise judgments, running both candidate orders and averaging the aligned probabilities before applying the gate further reduces order-sensitivity.

JEV-as-a-Judge Operating Envelope

- **Use JEV directly:** ordinary preference ranking, evidence-grounded factuality, final-answer adjudication.
- **Use cascade (JEV → strong LLM):** any of the above at scale where cost matters but accuracy cannot be sacrificed.
- **Always escalate:** derivation checking, multi-step reasoning correctness, style-adversarial pairs.
- **No judge works:** reference-free prose evaluation—all tested judges are near chance.

Practical checklist: (1) Judge preference pairs in both orders and average aligned probabilities. (2) Fit τ on a local held-out set—it does not transfer across workloads. (3) Count invalid outputs

as errors. (4) Validate the cascade on style-adversarial pairs before deploying.

Confidence Is Not a Certificate

JEV's confidence signal works well inside its operating envelope but breaks down on style-adversarial pairs (error-detection AUROC 0.770 vs. 0.869 on normal pairs) and is near-random on reference-free prose (AUROC 0.518). Always validate the cascade on a sample representative of your actual workload before trusting the threshold.

14.8 Evaluation Pitfalls

Even carefully designed evaluation pipelines can produce misleading results. This section catalogues the most common failure modes.

14.8.1 Benchmark Contamination

Benchmark contamination occurs when evaluation data appears in the model's training set, either directly (verbatim inclusion) or indirectly (paraphrased or semantically similar content). Contaminated models achieve inflated scores that do not reflect true generalisation ability.

Detection methods:

- **n-gram overlap:** Compute the fraction of evaluation examples with high n-gram overlap (e.g., ROUGE-L > 0.8) with the training corpus.
- **Membership inference:** Use a membership inference attack to estimate the probability that each evaluation example was in the training set.
- **Canary strings:** Embed unique, randomly generated strings in evaluation examples and check if the model can complete them.
- **Temporal holdout:** Use evaluation data created after the model's training cutoff date.

Mitigation:

- Maintain a private test set that is never released publicly.
- Regularly refresh benchmarks with new examples.
- Report training data cutoff dates and decontamination procedures.

14.8.2 Overfitting to Benchmarks

Even without direct contamination, models can be implicitly optimised for specific benchmarks through repeated evaluation and hyperparameter tuning. This is a form of *adaptive overfitting*: the benchmark leaks information into model development decisions.

The Benchmark Lifecycle

A benchmark's utility degrades over time as the research community optimises for it. MMLU [146], once a challenging test of world knowledge, now has models achieving near-human performance, yet these models still fail on novel knowledge tasks. New benchmarks should be treated as temporary signal sources, not permanent ground truth.

14.8.3 Goodhart's Law in Evaluation

Goodhart's Law states: *"When a measure becomes a target, it ceases to be a good measure."* [120] In LLM evaluation, this manifests in several ways:

- **Reward hacking:** Models trained with RLHF learn to exploit the reward model rather than genuinely improving. A model may learn to produce verbose, confident-sounding responses that score highly on the reward model but are factually incorrect.
- **Metric gaming:** Models fine-tuned to maximise BLEU or ROUGE may produce outputs that score well on these metrics but are less useful to humans.
- **Judge gaming:** Models trained with LLM-as-judge feedback may learn the judge's biases (e.g., verbosity bias) rather than genuinely improving quality.

Defences Against Goodhart's Law

1. **Metric diversity:** Use multiple metrics from different families; a model that games one metric will likely not game all simultaneously.
2. **Held-out evaluation:** Maintain evaluation metrics that are not used in training or model selection.
3. **Human spot-checks:** Regularly sample model outputs for human review, independent of automated metrics.
4. **Adversarial evaluation:** Actively probe for failure modes that automated metrics miss.
5. **Extrinsic validation:** Periodically validate intrinsic metrics against extrinsic outcomes.

14.8.4 Additional Pitfalls

Prompt sensitivity. LLM performance can vary dramatically with small changes to the evaluation prompt (e.g., adding "Think step by step" or changing the answer format). Always report the exact prompt used and consider evaluating across multiple prompt variants.

Aggregation artefacts. Averaging scores across tasks with different difficulty levels and score distributions can produce misleading aggregate metrics. A model that excels at easy tasks but fails at hard tasks may have the same average score as a model with uniform performance.

Selection bias in human evaluation. Human evaluators are not a random sample of end users. Annotators on crowdsourcing platforms may have different preferences, cultural backgrounds, and domain knowledge than the target user population.

Evaluation–deployment mismatch. Evaluation prompts are often shorter, cleaner, and more well-formed than real user queries. A model that performs well on benchmark prompts may degrade significantly on the noisy, ambiguous, multi-turn conversations that occur in production.

Key Questions for Evaluation Design

Before deploying an evaluation pipeline, ask:

1. Does the evaluation metric align with the deployment objective?
2. Is the evaluation data representative of the target distribution?
3. Have contamination and overfitting risks been assessed?
4. Are confidence intervals reported for all metrics?
5. Is the evaluation reproducible (fixed seeds, versioned prompts, public test sets)?

6. Has the evaluation been validated against human judgments or extrinsic outcomes?

14.8.5 Reasoning Collapse Under Reformulation

A subtler failure mode, distinct from benchmark contamination, is **reasoning collapse**: a model that solves a problem correctly in one surface form fails when the same problem is logically equivalent but syntactically different. This is not a knowledge gap—the model "knows" the answer—but a fragility in the reasoning pathway that the evaluation surface form happens to trigger.

The phenomenon. Consider a model that correctly solves "If Alice has 3 apples and gives 1 to Bob, how many does Alice have?" but fails on "Bob receives 1 apple from Alice, who started with 3. How many apples remain with Alice?" The underlying arithmetic is identical; only the narrative framing differs. Systematic studies of frontier models have found accuracy drops of 10–30 percentage points on semantically equivalent but syntactically reformulated problems [271], suggesting that high benchmark scores may reflect pattern matching to familiar surface forms rather than robust reasoning.

Why standard evaluation misses it. Most benchmarks present each problem in a single canonical form. A model that has seen similar surface forms during training will score well even if its reasoning is brittle. The contamination detection methods in Section 14.8.1 catch verbatim overlap but not surface-form overfitting.

Detection and mitigation.

- **Paraphrase stress-testing**: Automatically generate k semantically equivalent reformulations of each benchmark problem (using a separate LLM or template library) and measure score variance across forms. High variance signals brittle reasoning.
- **Negation and contrapositive tests**: For logical reasoning tasks, include the contrapositive and negated forms. A model with genuine logical understanding should handle all forms consistently.
- **Order permutation**: For multi-step problems, shuffle the order of premises. Robust reasoning should be order-invariant; brittle pattern matching is not.
- **Process-based evaluation**: Score the reasoning chain, not just the final answer [227]. A correct answer via a flawed reasoning path is a warning sign.

Reasoning Collapse and Agentic Systems

In agentic settings, reasoning collapse is especially dangerous. An agent that fails when a user rephrases a request—or when a tool returns data in an unexpected format—will silently produce wrong results. Agentic evaluation suites should include paraphrase variants and format perturbations as standard practice.

Part V

Agentic AI

Chapter 15

Introduction to Agentic AI

An agent is a system that perceives its environment, reasons about what to do, takes actions, and observes the consequences — repeating this loop until a goal is achieved or a budget is exhausted. Large language models, equipped with tool access, memory, and orchestration infrastructure, have become the cognitive core of a new generation of such systems. This chapter introduces the foundational concepts of agentic AI: what distinguishes an agent from a chatbot, the spectrum from simple prompt chains to fully autonomous systems, the A/T/C/S role taxonomy for structuring multi-agent teams, and the key capabilities — perception, memory, planning, tool use, and self-reflection — that determine what an agent can and cannot do.

The previous parts equipped us with the algorithmic toolkit—how to train, align, and reason with LLMs. We covered transformer architectures and GPU systems (Part I), the reinforcement learning methods that align models with human intent (Part II), the reasoning capabilities that emerge from RL training (Part III), and evaluation methodology (Part IV). This part turns to the central question of modern AI engineering: how do we *deploy* these models as autonomous agents that perceive, plan, act, and learn in open-ended environments?

GAS Connection — Introduction to Agentic AI

The Guardian Angel System is the book's primary running example of an agentic AI system, and the following chapter's conceptual framework is the lens through which the entire GAS architecture should be understood.

The perceive–plan–act loop. The GAS implements a continuous perceive–plan–act loop: `monitor_node()` *perceives* the glucose stream, `supervisor_node()` *plans* which agents to invoke, and `caregiver_node()` *acts* by sending an alert. This loop runs every 5 minutes (the CGM sampling interval of the `CGMSimulator` in `shared/patient_data.py`), making the GAS a real-time agentic system with a hard latency constraint.

Agent roles. The five GAS agent functions in `capstone_gas/gas_system.py` map onto the agent taxonomy introduced in the following chapter: `monitor_node()` is a *reactive agent* (stimulus–response, no planning); `advisor_node()` is a *deliberative agent* (retrieves evidence, reasons, plans a recommendation); `safety_node()` is a *critic agent* (evaluates another agent's output); `caregiver_node()` is an *actuator agent* (takes external actions); and `supervisor_node()` is an *orchestrator agent* (coordinates all others).

Why five agents instead of one? A single LLM could in principle perform all five roles. The GAS uses five specialised agent functions for three reasons: (1) *specialisation*—each function can be fine-tuned on data specific to its role; (2) *auditability*—the Safety Critic's verdict is a separate, inspectable artefact in `GASState`; (3) *replaceability*—any agent function can be upgraded without retraining the others. These are the core arguments for multi-agent decomposition introduced in the following chapter.

Human in the loop. The GAS is not fully autonomous: `caregiver_node()` always involves a human (the caregiver) in the final decision. The system recommends; the human acts. This human-in-the-loop design is a deliberate safety choice, and it is the default architecture recommended in the following chapter for high-stakes agentic applications.

An **agentic AI system** is one where an LLM operates in a loop: it receives observations from an environment (user messages, tool outputs, sensor data), reasons about what to do next, takes actions (tool calls, code execution, API requests), and iterates until a goal is achieved or it explicitly asks for human input. This contrasts with the "single-turn chatbot" paradigm where the model produces one response and waits.

The shift from chatbot to agent introduces several fundamental challenges that a single model call cannot address:

- **Persistence**: An agent must remember what it has done, what failed, and what context was established—across turns, sessions, and even days.
- **Grounding**: The agent must access up-to-date, domain-specific knowledge that was not present in its training data.
- **Action**: The agent must interact with external systems—databases, APIs, file systems, browsers—through well-defined interfaces.
- **Coordination**: Complex tasks often exceed what a single agent can handle; multiple specialized agents must collaborate, delegate, and negotiate.
- **Safety**: Autonomous action requires guardrails, human oversight, and graceful degradation when the agent is uncertain.

To address these challenges, production agentic systems are built as a layered architecture. Each layer solves a specific problem, and the chapters that follow cover the full stack from bottom to top:

- **Chapter 16: RAG (Retrieval-Augmented Generation)** — The knowledge layer. RAG gives agents access to dynamic external knowledge by retrieving relevant documents at query time. This solves the grounding problem: agents can answer questions about proprietary data, recent events, or domain-specific content that the model never saw during training. We cover embedding models, vector databases, chunking strategies, hybrid retrieval, and advanced patterns like agentic RAG where the agent decides *when* and *what* to retrieve.
- **Chapter 17: Memory** — The persistence layer. Memory enables agents to recall information across interactions—from short-term working memory within a single task, to long-term episodic memory spanning months. We cover memory architectures (buffer, summary, vector-indexed, knowledge graphs), memory consolidation, and how to design memory systems that scale without drowning the context window.
- **Chapter 18: Harness & Orchestration** — The runtime layer. The orchestration harness is the "operating system" for agents: it manages the agent loop, context window budget, tool dispatch, error recovery, state persistence, and observability. We cover context management strategies (summarization, sliding window, hierarchical), execution control (sequential, parallel, branching), guardrails, and human-in-the-loop patterns.
- **Chapter 19: Loop Engineering** — The inference-time RL layer. Loop engineering treats the agent's runtime loop as an optimization process: generate–verify–retry cycles that are structurally equivalent to policy optimization without weight updates. We cover the five structural primitives (generator, verifier, terminator, state manager, escalator), verification hierarchies, adaptive budget allocation, and how modern loops evolved from early AutoGPT-style systems into disciplined convergent processes.
- **Chapter 20: Graph Engineering** — The graph computation layer. Graph-structured workflows enable agents to execute complex, non-linear task plans with conditional branching, parallel execution, and dynamic routing. We cover directed acyclic graphs (DAGs) for agent pipelines, cyclic graphs for iterative refinement, state machines, and how frameworks like LangGraph implement graph-based orchestration for production agentic systems.

- **Chapter 21: Agent Design Patterns** — The architecture layer. Canonical patterns for structuring agents: ReAct (reason + act interleaving), plan-then-execute, reflection loops, tool-augmented generation, and multi-step workflows. We analyze when each pattern applies, their failure modes, and how to combine them for complex real-world tasks.
- **Chapter 22: Agentic Environments & Benchmarks** — The evaluation layer. Where and how to evaluate agentic behaviour. We cover web navigation benchmarks, coding environments, tool-use evaluation suites, and the unique challenges of evaluating multi-step autonomous systems (partial credit, trajectory quality, safety violations).
- **Chapter 23: MCP (Model Context Protocol)** — The tool integration standard. MCP standardizes how agents discover and invoke tools—analogous to USB for hardware. We cover the protocol specification, server/client architecture, resource management, and how MCP eliminates the N×M integration problem between agents and tools.
- **Chapter 24: Agent Skills** — The capability layer. How agents acquire and compose specialized capabilities beyond basic tool use, including skill libraries, skill selection, and compositional task solving.
- **Chapter 25: A2A (Agent-to-Agent Communication)** — The inter-agent protocol. When tasks require multiple specialists, A2A provides a standardized protocol for agent discovery, task delegation, progress streaming, and result aggregation—enabling heterogeneous agents (from different vendors, frameworks, or organizations) to collaborate.
- **Chapter 26: Multi-Agent Systems** — The coordination layer. Architectures for multi-agent collaboration: hierarchical delegation, peer-to-peer negotiation, debate and consensus, swarm intelligence, and emergent behaviour. We cover when to use single-agent vs. multi-agent designs and how to debug coordination failures.
- **Chapter 27: Agent Development Frameworks** — The implementation layer. Production toolkits that implement the above concepts: LangGraph (stateful graph-based orchestration), CrewAI (role-based multi-agent), OpenAI Agents SDK, AutoGen, and others. We compare their trade-offs, architecture decisions, and suitability for different use cases.
- **Chapter 28: Agentic UI Frameworks** — The interaction layer. How users interact with and supervise agents: streaming interfaces, progressive disclosure, approval workflows, status dashboards, and the UX patterns that build appropriate trust in autonomous systems.

These layers do not operate in isolation—they form a tightly integrated system where each component depends on and enhances the others:

- The **agent core** (an LLM with reasoning capabilities from Parts II–III) sits at the center, executing a perceive–reason–act loop.
- **RAG** feeds the agent with relevant knowledge before each reasoning step, while **Memory** provides continuity across steps and sessions.
- The **Orchestration Harness** coordinates everything: it decides when to retrieve, when to call tools, when to delegate to sub-agents, and when to ask the human for guidance. **Loop Engineering** formalizes the harness's retry-and-verify cycles as inference-time optimization, ensuring convergence and cost control.
- **MCP** provides the standardized interface through which the agent accesses all external tools, and **A2A** provides the equivalent interface for inter-agent communication.
- **Design Patterns** define the high-level strategy (ReAct, plan-and-execute, reflection), while **Frameworks** provide the concrete implementation of these patterns.
- The **UI layer** closes the loop by connecting the agent back to the human—for oversight, correction, and collaborative problem-solving.

A taxonomy of agentic roles. Not all agents in a system play the same role. A useful classification distinguishes four functional types [20]: **Autonomous** agents that operate without human intervention on well-defined subtasks; **Transactional** agents that execute discrete, reversible operations against external systems (databases, APIs, payment processors); **Collaborative** agents that work alongside humans or other agents, contributing partial results to a shared goal; and **Supervisory** agents that monitor, evaluate, and coordinate the other three types. Real production systems typically combine all four: a supervisory orchestrator delegates to autonomous workers, which call transactional tools, while a collaborative agent surfaces intermediate results to a human reviewer. Understanding which role each agent plays clarifies its required capabilities, trust level, and failure modes.

Throughout, we maintain the systems perspective: agentic AI is not just about prompting—it requires careful engineering of context management, error handling, safety guardrails, and observability at every layer. The figure below shows how these components fit together architecturally.

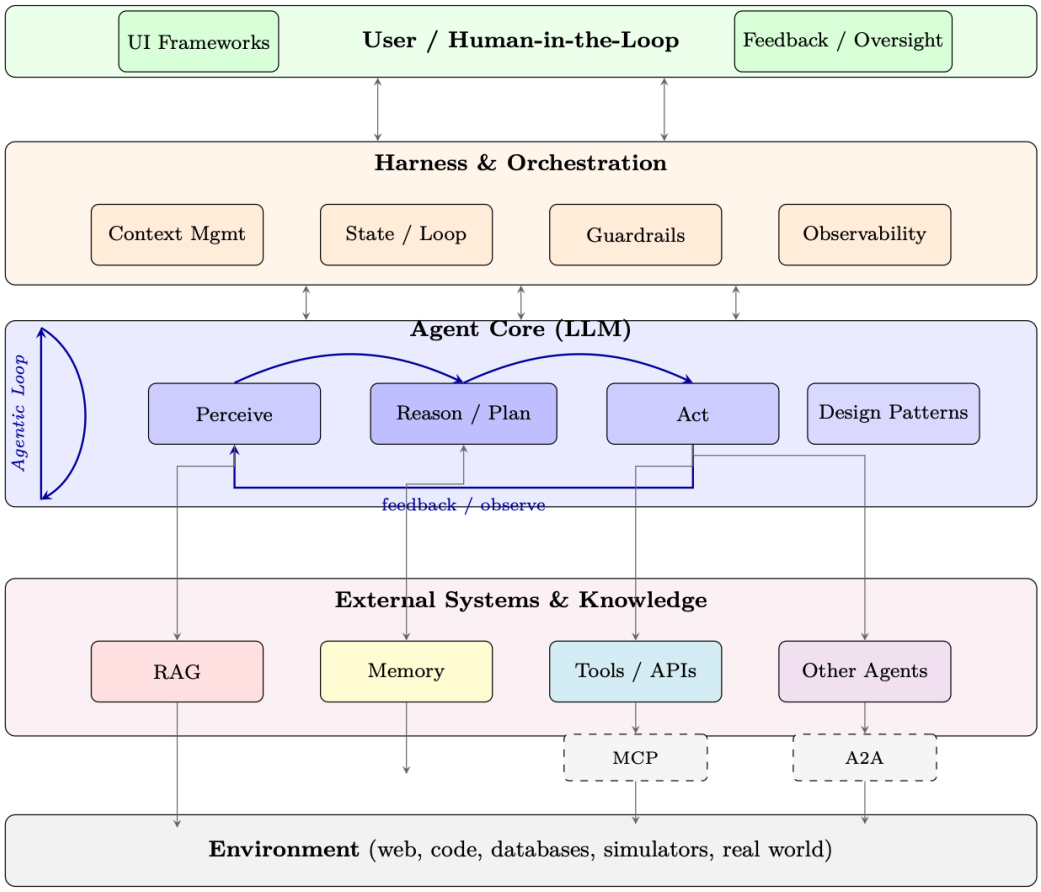


Figure 15.1: The Agentic AI architecture stack. The **Agent Core** executes a perceive–reason–act loop, coordinated by the **Harness & Orchestration** layer which manages context, state, guardrails, and observability. The agent interacts downward with **External Systems**—RAG for knowledge retrieval, Memory for persistence, Tools via MCP, and other Agents via A2A—all grounded in an **Environment**. The **User** provides goals, feedback, and oversight from above. Arrows indicate bidirectional data flow; the blue loop arrows show the iterative agentic cycle.

Chapter 16

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) [219] has emerged as one of the most practically impactful techniques for deploying large language models in production. Rather than relying solely on knowledge encoded in model weights during training, RAG equips LLMs with a dynamic, updatable external memory—enabling accurate, grounded, and verifiable responses across a wide range of knowledge-intensive tasks.

GAS Connection — Retrieval-Augmented Generation (RAG)

The GAS Advisor agent is a full RAG pipeline. Every component described in the following chapter—embedding, retrieval, generation, evaluation—has a concrete implementation in the `GASKnowledgeBase` class in `capstone_gas/gas_system.py`.

Core RAG architecture (§ Core RAG Architecture). The Advisor's RAG pipeline: (1) the patient state is converted to a query string in `advisor_node()`; (2) the query is embedded with `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional vectors); (3) `GASKnowledgeBase.retrieve()` returns the top-k most similar entries from the 15 ADA guidelines in `MEDICAL_KNOWLEDGE`; (4) the retrieved guidelines are injected into the Advisor's context window; (5) the Advisor generates a recommendation grounded in the retrieved evidence.

Vector index (§ Chunking Strategies). The `GASKnowledgeBase` uses a FAISS `IndexFlatIP` (inner-product search on L2-normalised vectors) for fast similarity search. If FAISS is not installed, it falls back to a NumPy dot-product search; if `sentence-transformers` is not installed, it falls back to keyword search. This layered fallback makes the GAS runnable in any environment.

Knowledge base content (§ Advanced RAG Patterns). The `MEDICAL_KNOWLEDGE` list in `capstone_gas/gas_system.py` contains 15 ADA guideline entries covering hypoglycaemia management, hyperglycaemia treatment, post-meal glucose targets, dawn phenomenon, exercise adjustments, and insulin dosing. Each entry is a self-contained clinical recommendation that can be retrieved independently.

Agentic RAG (§ Agentic RAG). The Advisor implements iterative retrieval: `GASKnowledgeBase.retrieve()` is called with the patient's current alert context as the query, returning the most relevant guidelines for the specific clinical situation (e.g. hypoglycaemia guidelines for a reading <70 mg/dL, post-meal guidelines for a post-meal spike).

RAG + fine-tuning synergy (§ RAG + Fine-Tuning Synergy). The GAS uses both RAG and prompting for the Advisor: RAG provides specific clinical evidence from `MEDICAL_KNOWLEDGE`; the system prompt provides the clinical reasoning style and output format. Neither alone is sufficient—RAG without a good system prompt produces poorly formatted recommendations; a good system prompt without RAG produces confident but potentially incorrect advice.

16.1 Motivation and Problem Statement

Why LLMs Need External Knowledge

Large language models store knowledge *parametrically*—compressed into billions of weights during training. This creates three fundamental limitations:

1. **Hallucination**: Models confidently generate plausible-sounding but factually incorrect statements when queried beyond their reliable knowledge boundary.
2. **Knowledge Staleness**: Training data has a cutoff date; models cannot know about events, papers, or product updates that occurred after training.
3. **Domain Specificity**: General-purpose models lack deep knowledge of proprietary codebases, internal documents, specialized regulations, or enterprise data.

16.1.1 Parametric vs. Non-Parametric Knowledge

We can formalize the distinction between the two knowledge sources. Let $\mathcal{M}_\theta$ denote a language model with parameters θ, and let $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ be an external document corpus. The probability of generating answer a given query q under each paradigm is:

$$P_{\text{parametric}}(a \mid q) = P_{\mathcal{M}_\theta}(a \mid q) \tag{16.1}$$

$$P_{\text{RAG}}(a \mid q, \mathcal{D}) = \sum_{d \in \mathcal{D}} P_{\mathcal{M}_\theta}(a \mid q, d)\, P_{\text{ret}}(d \mid q, \mathcal{D}) \tag{16.2}$$

where $P_{\text{ret}}(d \mid q, \mathcal{D})$ is the retrieval distribution over documents. RAG marginalizes over retrieved evidence, grounding generation in non-parametric knowledge.

The Library Analogy

Think of a parametric LLM as a scholar who has memorized an enormous library but graduated years ago. RAG gives that scholar a library card—they can look things up in real time, cite sources, and acknowledge when they need to check a reference rather than guessing from memory.

16.1.2 When to Use RAG vs. Fine-Tuning vs. Long Context

Table 16.1: Decision guide: RAG vs. Fine-Tuning vs. Long Context

Criterion	RAG	Fine-Tuning	Long Context	RAG + FT
Knowledge updates frequently	✓	×	×	✓
Need citations / grounding	✓	×	✓	✓
Proprietary large corpus	✓	×	×	✓
Adapt style / format	×	✓	×	✓
Teach new reasoning skills	×	✓	×	✓
Corpus fits in context window	×	×	✓	×
Low latency required	×	✓	×	×

Common Misconception

RAG is *not* a replacement for fine-tuning. Fine-tuning teaches the model *how* to reason and respond; RAG provides *what* to reason about. They are complementary. A model fine-tuned to follow instructions well will use retrieved context more effectively than a base model.

16.2 Core RAG Architecture

A standard RAG system consists of two phases: an **offline indexing pipeline** that processes and stores documents, and an **online retrieval-generation pipeline** that serves queries.

16.2.1 Full Pipeline Diagram

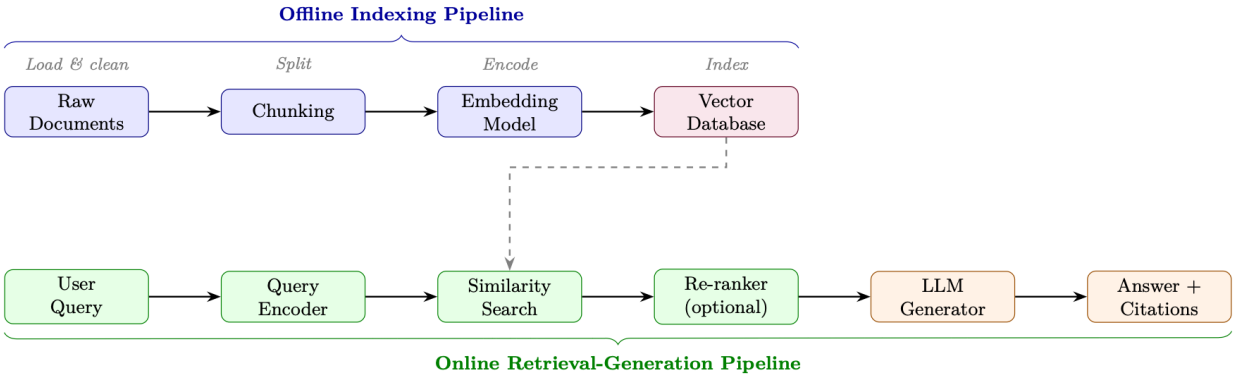


Figure 16.1: End-to-end RAG architecture. The offline pipeline (blue) indexes documents once; the online pipeline (green/orange) serves each query at inference time.

16.2.2 Indexing Pipeline

Document Loading. Documents arrive in heterogeneous formats (PDF, HTML, Markdown, DOCX, code). Loaders extract clean text and preserve metadata (source URL, page number, section title, timestamp) that will be stored alongside embeddings for filtering and citation.

Chunking. Long documents must be split into chunks that fit within the embedding model's context window (typically 512 tokens) and are semantically coherent. Chunking strategy is one of the highest-impact decisions in RAG system design (see Section 16.4).

Embedding. Each chunk c_i is encoded into a dense vector $\mathbf{e}_i = f_\phi(c_i) \in \mathbb{R}^d$ using an embedding model f_ϕ. These vectors are stored in a vector database alongside the original text and metadata.

16.2.3 Retrieval

Given a query q, the retrieval step encodes it as $\mathbf{q} = f_\phi(q)$ and finds the k most similar chunks by cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \, \|\mathbf{e}_i\|} \tag{16.3}$$

The top-k chunks $\mathcal{C}_k = \{c_{(1)}, \ldots, c_{(k)}\}$ are returned as context.

16.2.4 Generation

Retrieved chunks are injected into a prompt template:

```
SYSTEM_PROMPT = """You are a helpful assistant. Answer the question using ONLY
the provided context. If the context does not contain enough information,
say so explicitly. Cite your sources using [Doc N] notation."""

def build_rag_prompt(query: str, chunks: list[dict]) -> str:
    context_str = "\n\n".join(
        f"[Doc {i+1}] (Source: {c['source']}, Page: {c.get('page','N/A')})\n{c['
    text']}"
        for i, c in enumerate(chunks)
    )
    return f"""{SYSTEM_PROMPT}

Context:
{context_str}
```

```
Question: {query}

Answer:"""
```

Listing 16.1: Standard RAG prompt template

16.3 Retrieval Methods

16.3.1 Sparse Retrieval: BM25 and TF-IDF

Sparse retrieval methods represent documents and queries as high-dimensional sparse vectors over the vocabulary. The classic BM25 scoring function [327] for document d given query q with terms $t_1, \ldots, t_n$ is:

$$\text{BM25}(d, q) = \sum_{i=1}^{n} \text{IDF}(t_i) \cdot \frac{f(t_i, d) \cdot (k_1 + 1)}{f(t_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \tag{16.4}$$

where $f(t_i, d)$ is term frequency, $|d|$ is document length, avgdl is average document length, and $k_1 \in [1.2, 2.0]$, $b = 0.75$ are tuning parameters.

When Sparse Retrieval Still Wins

- **Exact keyword matching**: product codes, error codes, proper nouns, rare terms
- **Low-resource domains**: insufficient training data for dense models
- **Interpretability**: easy to debug why a document was retrieved
- **Speed**: no GPU required; scales to billions of documents with inverted indices
- **Out-of-vocabulary terms**: new terminology not seen during embedding training

16.3.2 Dense Retrieval: DPR

Dense Passage Retrieval (DPR) [190] uses two separate BERT-based encoders—a *query encoder* E_Q and a *passage encoder* E_P—trained with contrastive loss to place relevant query-passage pairs close together in embedding space.

Bi-Encoder Architecture.

$$\text{sim}(q, p) = E_Q(q)^\top E_P(p) \tag{16.5}$$

Training with In-Batch Negatives. Given a batch of B query-passage pairs $\{(q_i, p_i^+)\}_{i=1}^{B}$, the contrastive loss treats all other passages in the batch as negatives:

$$\mathcal{L}_{\text{DPR}} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp\left(E_Q(q_i)^\top E_P(p_i^+)/\tau\right)}{\sum_{j=1}^{B} \exp(E_Q(q_i)^\top E_P(p_j)/\tau)} \tag{16.6}$$

where τ is a temperature hyperparameter. Hard negatives (passages that are lexically similar but semantically irrelevant) are crucial for training strong retrievers.

Approximate Nearest Neighbor Search. At scale, exhaustive search over millions of embeddings is infeasible. FAISS [181] (Facebook AI Similarity Search) provides efficient approximate nearest neighbor (ANN) search using:

- **IVF (Inverted File Index)**: cluster vectors into Voronoi cells; search only nearby cells
- **HNSW (Hierarchical Navigable Small World)** [261]: graph-based index with $O(\log N)$ search

- **PQ (Product Quantization)**: compress vectors to reduce memory footprint

16.3.3 Hybrid Retrieval with Reciprocal Rank Fusion

Hybrid retrieval combines sparse and dense scores. A simple linear combination is:

$$s_{\text{hybrid}}(d, q) = \alpha \cdot s_{\text{dense}}(d, q) + (1 - \alpha) \cdot s_{\text{sparse}}(d, q) \tag{16.7}$$

However, scores from different systems are not directly comparable. **Reciprocal Rank Fusion (RRF)** [64] avoids this by operating on ranks rather than scores:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{16.8}$$

where $\mathcal{R}$ is the set of ranked lists (e.g., BM25 ranking and dense ranking), $\text{rank}_r(d)$ is the rank of document d in list r, and $k = 60$ is a smoothing constant that reduces the impact of very high-ranked documents.

RRF Calculation

Suppose BM25 ranks document d at position 3, and dense retrieval ranks it at position 7. With $k = 60$:

$$\text{RRF}(d) = \frac{1}{60+3} + \frac{1}{60+7} = \frac{1}{63} + \frac{1}{67} \approx 0.0159 + 0.0149 = 0.0308$$

A document ranked 1st in both lists would score $\frac{1}{61} + \frac{1}{61} \approx 0.0328$.

16.3.4 Learned Sparse Retrieval: SPLADE and SPLADEv2

Why SPLADE?

Traditional sparse retrieval (BM25) relies on exact lexical matching — it fails when the query says "car" but the document says "automobile." Dense retrieval (DPR) captures semantics but loses interpretability, requires GPU at query time, and produces large indexes. **SPLADE** gets the best of both worlds: sparse vectors (fast inverted-index lookup like BM25) with learned semantic expansion (handles synonyms and related concepts like dense models).

SPLADE (v1) — Core Idea. SPLADE (Sparse Lexical and Expansion Model) [101] uses a pre-trained masked language model (e.g., BERT/DistilBERT) to produce a sparse vector over the *entire vocabulary* for each document or query. The key insight: the MLM head already knows which words are semantically related to each position in a text — SPLADE repurposes this knowledge as term importance weights.

Architecture. Given input text $x = [x_1, \ldots, x_n]$:

1. Pass through a transformer encoder to get contextual representations $\mathbf{H} \in \mathbb{R}^{n \times |\mathcal{V}|}$ via the MLM head
2. Aggregate across positions and apply a saturating activation:

$$w_t(x) = \log\left(1 + \text{ReLU}\left(\max_{i \in [1,n]} \mathbf{H}_i[t]\right)\right) \tag{16.9}$$

where $\mathbf{H}_i[t]$ is the MLM logit for vocabulary token t at input position i.

- The $\log(1 + \cdot)$ saturation prevents any single term from dominating (similar to TF saturation in BM25)
- The ReLU ensures sparsity — most vocabulary terms get weight zero

- The max pooling across positions captures the strongest signal for each term from any position in the text

- **Expansion**: Even tokens *not present* in the original text can get non-zero weight (e.g., a document about "neural networks" may get weight for "deep learning," "AI," "backpropagation")

Scoring. Query and document are each mapped to sparse vectors $\mathbf{w}^q, \mathbf{w}^d \in \mathbb{R}^{|\mathcal{V}|}$. The relevance score is a simple dot product:

$$s(q, d) = \sum_{t \in \mathcal{V}} w_t^q \cdot w_t^d \tag{16.10}$$

Because both vectors are sparse (typically 20–200 non-zero entries out of 30K vocabulary), this can be computed efficiently using standard inverted indexes (Lucene, Anserini) — no GPU needed at query time.

Training. SPLADE is trained with contrastive learning (in-batch negatives + hard negatives) plus two regularization terms:

$$\mathcal{L} = \mathcal{L}_{\text{contrastive}} + \lambda_q \|\mathbf{w}^q\|_1 + \lambda_d \|\mathbf{w}^d\|_1 \tag{16.11}$$

The L_1 penalties on query and document representations encourage sparsity — without them, the model would learn dense representations that defeat the purpose.

SPLADEv2 — Key Improvements. SPLADEv2 [100] introduces several refinements that significantly improve efficiency and effectiveness:

1. **Distillation from cross-encoder**: Instead of training only on binary relevance labels, SPLADEv2 uses a cross-encoder teacher (e.g., MonoT5 [284]) to provide soft relevance scores. This gives richer training signal:

$$\mathcal{L}_{\text{distill}} = \text{KL}(\sigma(s_{\text{student}}) \,\|\, \sigma(s_{\text{teacher}})) \tag{16.12}$$

2. **Separate query/document encoders**: SPLADEv2 uses different sparsity targets for queries vs. documents. Queries are encouraged to be *more sparse* (faster lookup) while documents can be slightly denser (pre-computed offline):

$$\lambda_q > \lambda_d \quad (\text{e.g., } \lambda_q = 3 \times 10^{-4},\ \lambda_d = 1 \times 10^{-4}) \tag{16.13}$$

3. **FLOPS regularization**: Instead of simple L_1, SPLADEv2 introduces a FLOPS-aware regularizer that directly penalizes the expected retrieval cost:

$$\mathcal{L}_{\text{FLOPS}} = \sum_{t \in \mathcal{V}} (\overline{a}_t^q)^2 + \sum_{t \in \mathcal{V}} \left(\overline{a}_t^d\right)^2 \tag{16.14}$$

   where $\overline{a}_t$ is the mean activation for term t across the batch. This penalizes terms that are non-zero for many documents (high posting list length = slow retrieval).

4. **Efficient backbone**: Uses DistilBERT (66M params) instead of BERT-base (110M), halving encoding time with minimal quality loss.

SPLADE vs. SPLADEv2 Comparison

Aspect	SPLADE (v1)	SPLADEv2
Training signal	Binary relevance + hard negatives	Cross-encoder distillation
Sparsity control	L_1 regularization	FLOPS-aware regularization
Query/doc symmetry	Same encoder, same λ	Asymmetric (sparser queries)
Backbone	BERT-base (110M)	DistilBERT (66M)
MRR@10 (MS MARCO [19])	34.0	36.8
Avg non-zero terms/doc	∼200	∼120 (40% sparser)

When to Use SPLADE

- **Use SPLADE/v2 when**: You need semantic retrieval without GPU at query time, your infrastructure already has inverted indexes (Elasticsearch, Lucene), or you need interpretable relevance scores (you can inspect which expanded terms matched).
- **Prefer dense retrieval when**: You have GPU budget for query encoding, need multilingual support (dense models transfer better), or your queries are very short (1–2 words where expansion helps less).
- **Best practice**: Use SPLADEv2 as the first-stage retriever + cross-encoder reranker for top-k. This matches or beats dense retrieval pipelines at lower latency.

16.3.5 ColBERT: Late Interaction

ColBERT [192] encodes queries and documents into *sets* of token-level embeddings and uses a *MaxSim* operator for scoring:

$$s(q,d) = \sum_{i \in |\mathbf{q}|} \max_{j \in |\mathbf{d}|} \mathbf{q}_i^\top \mathbf{d}_j \tag{16.15}$$

This late interaction mechanism is more expressive than single-vector bi-encoders while being far faster than cross-encoders, since document embeddings are pre-computed offline.

Architecture. Both the query encoder E_Q and document encoder E_D are BERT-based models that produce *per-token* embeddings (not a single [CLS] vector). Each token embedding is projected to a lower dimension (typically 128) via a linear layer:

$$\mathbf{q}_i = \text{Linear}(E_Q(q)_i) \in \mathbb{R}^{128}, \quad i = 1, \ldots, |q| \tag{16.16}$$

$$\mathbf{d}_j = \text{Linear}(E_D(d)_j) \in \mathbb{R}^{128}, \quad j = 1, \ldots, |d| \tag{16.17}$$

Training. ColBERT is trained with a pairwise softmax cross-entropy loss over positive and negative passages. Given a query q, a positive passage d^+, and a set of negative passages $\{d_1^-, \ldots, d_N^-\}$:

$$\mathcal{L}_{\text{ColBERT}} = -\log \frac{\exp(s(q,d^+))}{\exp(s(q,d^+)) + \sum_{k=1}^{N} \exp(s(q,d_k^-))} \tag{16.18}$$

where $s(q,d)$ is the MaxSim score from Equation 16.15. Negatives are sourced from:

- **In-batch negatives**: Other passages in the same training batch (free, abundant)
- **Hard negatives**: Passages retrieved by BM25 that are lexically similar but semantically irrelevant (most impactful for quality)
- **Distillation negatives** (ColBERTv2 [331]): Use a cross-encoder teacher to mine the hardest negatives and distill its scores into ColBERT

Indexing and Serving. At index time, all document token embeddings are pre-computed and stored (with optional compression via residual quantization in ColBERTv2). At query time, only the query tokens are encoded live, and MaxSim is computed against the stored document embeddings. This separation enables:

- **Offline document encoding**: Encode once, serve many queries
- **PLAID indexing** [331]: Cluster document embeddings, use centroids for initial candidate retrieval, then compute exact MaxSim only on candidates—reducing latency by 5–10×
- **Index size**: $|d| \times 128$ floats per document (larger than single-vector methods but compressible to ∼2 bytes/dimension with quantization)

16.3.6 Retrieval Method Comparison

Table 16.2: Comparison of retrieval methods across key dimensions

Method	Latency	Accuracy	Index Size	GPU	Best For
TF-IDF [355]	Very Low	Low	Small	No	Baseline, exact match
BM25 [327]	Very Low	Medium	Small	No	Keyword search, rare terms
DPR / bi-encoder [190]	Low	High	Large	Yes	Semantic similarity
SPLADE [101]	Low	High	Medium	Yes	Hybrid accuracy + speed
ColBERT [192]	Medium	Very High	Very Large	Yes	High-accuracy retrieval
Cross-encoder [283]	High	Highest	N/A	Yes	Re-ranking top-k
Hybrid (RRF) [64]	Low	Very High	Large	Yes	Production systems

16.4 Chunking Strategies

Chunking is the process of splitting documents into segments that are (1) small enough to fit in an embedding model's context window, (2) semantically coherent, and (3) contain enough context to be useful when retrieved in isolation.

16.4.1 Fixed-Size Chunking with Overlap

The simplest strategy: split every W tokens with an overlap of O tokens between consecutive chunks.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,       # tokens per chunk
    chunk_overlap=64,     # overlap to preserve context across boundaries
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""]
)
chunks = splitter.split_documents(documents)
```

Listing 16.2: Fixed-size chunking with overlap

Overlap formula: For a document of length L tokens, the number of chunks is:

$$N_{\text{chunks}} = \left\lceil \frac{L - O}{W - O} \right\rceil \tag{16.19}$$

16.4.2 Semantic Chunking

Rather than splitting at fixed intervals, semantic chunking splits at *topic boundaries* detected by measuring embedding similarity between consecutive sentences:

```python
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(
    embeddings=OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",  # or "standard_deviation"
    breakpoint_threshold_amount=95,          # split at top 5% dissimilarity
)
chunks = chunker.split_documents(documents)
```

Listing 16.3: Semantic chunking via embedding similarity

16.4.3 Document-Structure-Aware Chunking

For structured documents (Markdown, HTML, code), split at natural boundaries:

- **Markdown**: split at `##` headers, preserving section context
- **HTML**: split at `<section>`, `<article>`, `<p>` tags
- **Code**: split at function/class definitions, preserving imports in each chunk
- **Tables**: keep entire tables as single chunks; never split mid-row

16.4.4 Parent-Child Chunking

A powerful pattern that decouples retrieval granularity from generation context:

1. **Index small child chunks** (e.g., 128 tokens) for precise retrieval
2. **Return large parent chunks** (e.g., 512 tokens) to the LLM for richer context

```python
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain.text_splitter import RecursiveCharacterTextSplitter

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter  = RecursiveCharacterTextSplitter(chunk_size=400)

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=InMemoryStore(),
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(documents)
```

Listing 16.4: Parent-child chunking with LangChain

16.4.5 Empirical Guidelines for Chunk Size

Table 16.3: Chunk size recommendations by use case

Use Case	Recommended Chunk Size	Overlap
Factoid QA (precise facts)	128–256 tokens	20–32 tokens
Summarization / synthesis	512–1024 tokens	64–128 tokens
Code retrieval	Full function	None
Legal / regulatory documents	Paragraph-level	1 sentence
Conversational / chat	256–512 tokens	32–64 tokens

16.5 Advanced RAG Patterns

16.5.1 Query Transformation

Raw user queries are often ambiguous, too short, or poorly matched to document language. Query transformation techniques improve retrieval before the search step.

HyDE (Hypothetical Document Embeddings) [112]. Instead of embedding the query directly, generate a *hypothetical answer* and embed that:

$$\hat{d} = \text{LLM}(q), \quad \mathbf{e}_{\text{query}} = f_\phi(\hat{d}) \tag{16.20}$$

The intuition: a hypothetical answer is in the same linguistic register as real documents, reducing the query-document distribution gap.

Step-Back Prompting. For specific questions, first generate a more general "step-back" question, retrieve for both, and combine the contexts. Example: "What is the boiling point of ethanol at 2 atm?" → step-back: "What factors affect the boiling point of liquids?"

Multi-Query Generation. Generate M diverse reformulations of the query, retrieve for each, and union the results:

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    llm=ChatOpenAI(temperature=0.7),
    include_original=True,   # also retrieve for original query
)
# Internally generates 3 query variants, retrieves for each, deduplicates
docs = retriever.get_relevant_documents(query)
```

Listing 16.5: Multi-query retrieval

16.5.2 Re-Ranking

After initial retrieval of top-k candidates, a *cross-encoder* re-ranker scores each query-document pair jointly (attending to both simultaneously), producing much more accurate relevance scores at the cost of higher latency:

$$s_{\text{cross}}(q, d) = \text{CrossEncoder}([q; d]) \tag{16.21}$$

Cross-encoders cannot be used for first-stage retrieval (no pre-computed document embeddings), but are ideal for re-ranking a small candidate set (typically $k = 20$–100).

```python
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("BAAI/bge-reranker-large")

def rerank(query: str, docs: list[str], top_n: int = 5) -> list[str]:
    pairs = [(query, doc) for doc in docs]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(scores, docs), reverse=True)
    return [doc for _, doc in ranked[:top_n]]
```

Listing 16.6: Cross-encoder re-ranking with BGE

16.5.3 Contextual Compression

Retrieved chunks often contain irrelevant sentences surrounding the relevant passage. Contextual compression uses an LLM to extract only the relevant portions:

```python
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectorstore.as_retriever()
)
compressed_docs = compression_retriever.get_relevant_documents(query)
```

Listing 16.7: LLM-based contextual compression

16.5.4 Self-RAG

Self-RAG [14] trains a single model to (1) decide *whether* to retrieve, (2) generate with or without retrieval, and (3) *critique* its own output using special reflection tokens:

- `[Retrieve]`: should the model retrieve additional passages?
- `[IsRel]`: is the retrieved passage relevant to the query?
- `[IsSup]`: does the generated statement follow from the retrieved passage?
- `[IsUse]`: is the overall response useful?

The model is trained end-to-end to predict these tokens alongside the response, enabling fine-grained control over retrieval and self-grading.

16.5.5 CRAG: Corrective RAG

CRAG [423] adds a *retrieval evaluator* that grades retrieved documents and triggers corrective actions:

1. Retrieve top-k documents
2. Grade each document: **Correct** / **Ambiguous** / **Incorrect**
3. If all documents are incorrect or ambiguous → fall back to web search
4. If some documents are correct → use knowledge refinement (strip irrelevant sentences)
5. Generate answer from refined context

16.5.6 Adaptive RAG

Adaptive RAG [172] routes queries to different retrieval strategies based on predicted complexity:

- **No retrieval**: simple factual queries the model can answer from parameters
- **Single-step RAG**: standard retrieve-then-generate for moderate queries
- **Multi-step RAG**: iterative retrieval for complex multi-hop questions

A lightweight classifier trained on query complexity labels routes each incoming query.

16.5.7 Graph RAG

Microsoft's Graph RAG [86] constructs a *knowledge graph* from the document corpus and uses community detection to generate hierarchical summaries:

1. **Entity extraction**: LLM extracts entities and relationships from each chunk
2. **Graph construction**: build a graph $G = (V, E)$ where nodes are entities and edges are relationships
3. **Community detection**: apply Leiden algorithm to find communities at multiple resolutions
4. **Community summaries**: LLM generates a summary for each community
5. **Query**: for global queries, map-reduce over community summaries; for local queries, use standard vector search

When to Use Graph RAG

Graph RAG excels at *global* queries that require synthesizing information across many documents ("What are the main themes in this corpus?") but is expensive to build and maintain. Standard RAG is better for *local* queries ("What did document X say about topic Y?").

16.5.8 RAG-Fusion

RAG-Fusion [318] generates multiple search queries from the original, retrieves for each, and fuses the ranked lists using RRF (Equation 16.8):

```
def reciprocal_rank_fusion(ranked_lists: list[list[str]], k: int = 60) -> list[str
    ]:
    """Fuse multiple ranked document lists using RRF."""
    scores: dict[str, float] = {}
    for ranked in ranked_lists:
        for rank, doc_id in enumerate(ranked, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

def rag_fusion(query: str, retriever, llm, n_queries: int = 4) -> str:
    # Step 1: Generate query variants
    variants = generate_query_variants(query, llm, n=n_queries)
    # Step 2: Retrieve for each variant
    all_ranked = [retriever.retrieve(q) for q in [query] + variants]
    # Step 3: Fuse with RRF
    fused_docs = reciprocal_rank_fusion(all_ranked)
    # Step 4: Generate answer
    return generate_answer(query, fused_docs[:5], llm)
```

Listing 16.8: RAG-Fusion with RRF

16.6 Efficient RAG Decoding: REFRAG

A practical bottleneck of RAG is *decoding latency*: the retrieved passages concatenated into the LLM context are often long yet sparsely relevant, inflating time-to-first-token (TTFT) and KV-cache memory. REFRAG [232] observes that because retrieved passages are independently sourced (via diversity or deduplication during re-ranking), their attention patterns are *block-diagonal*—most cross-passage attention is near zero. This sparsity means that the majority of computations over the RAG context during decoding are unnecessary.

Compress–Sense–Expand Framework. REFRAG exploits this structure via a three-phase decoding strategy:

1. **Compress**: Replace full KV representations of retrieved passages with compact summaries (e.g., mean-pooled keys/values per passage block), drastically reducing memory.
2. **Sense**: At each decoding step, use lightweight attention over the compressed representations to identify which passage blocks are relevant to the current token.
3. **Expand**: Reconstruct full KV entries only for the selected blocks, performing exact attention over the sparse active set.

Results. On LLaMA-based models, REFRAG achieves up to 30.85× TTFT speedup (a 3.75× improvement over prior sparse-attention baselines) with no loss in perplexity. It also extends effective context length by 16× under fixed memory budgets. These gains hold across RAG, multi-turn conversation, and long-document summarization tasks.

Why REFRAG Matters for Agentic RAG

Agentic RAG (Section 16.7) requires *multiple* retrieval rounds per query, compounding latency. Efficient decoding methods like REFRAG are essential infrastructure: they make iterative retrieve-reason-generate loops practical at scale by ensuring each round's decoding cost is sublinear in context length.

16.7 Agentic RAG

16.7.1 Motivation: Limits of Static RAG

Standard RAG follows a fixed retrieve-then-generate pattern. This fails on:

- **Multi-hop questions**: "Who founded the company that acquired OpenAI's main competitor in 2023?" requires chaining multiple retrievals
- **Ambiguous queries**: the right retrieval strategy depends on what is found
- **Heterogeneous sources**: different sub-questions require different knowledge bases
- **Iterative refinement**: initial retrieval may reveal that a different query is needed

RAG as a Markov Decision Process

Agentic RAG frames retrieval as a sequential decision problem. The *state* is the current context (query + retrieved documents so far); the *actions* include retrieve, reason, generate, and stop; the *reward* is answer correctness. The agent learns a policy for when and what to retrieve.

16.7.2 Agentic RAG Architecture

16.7.3 Multi-Source Routing

An agentic RAG system can route sub-queries to specialized knowledge sources. The core insight is that different question types demand different retrieval backends—no single index excels at everything.

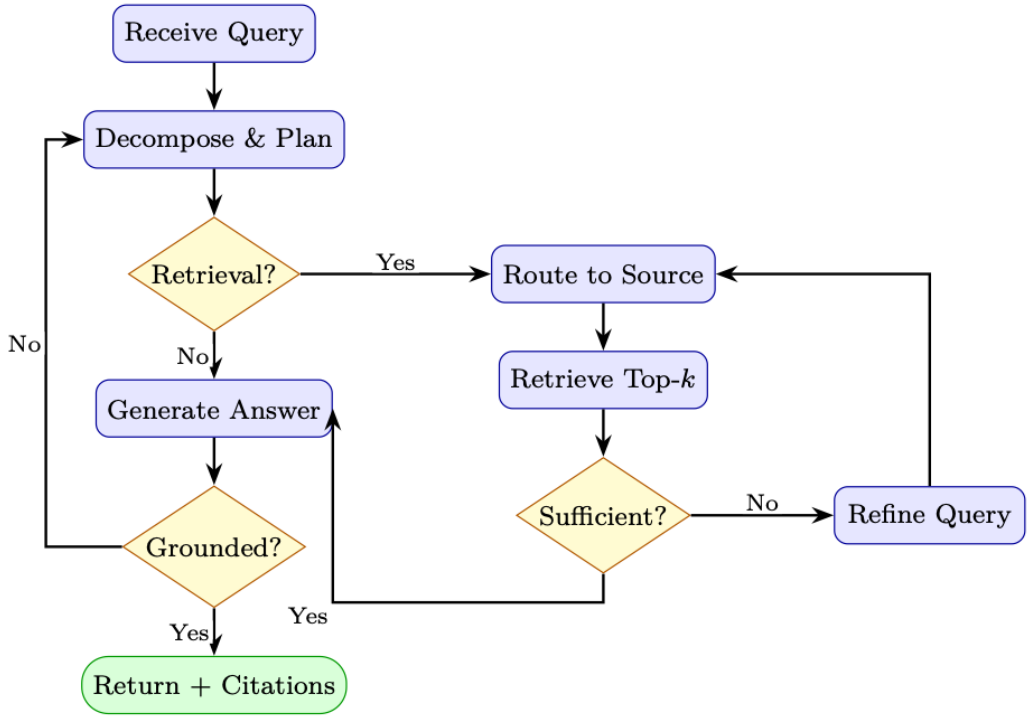


Figure 16.2: Agentic RAG control flow. The agent iteratively plans, retrieves, evaluates sufficiency, and self-checks grounding before returning an answer.

Why Route? Consider a financial analyst's assistant handling four queries:

- "What is our company's PTO policy?" → **Vector DB** (internal documents)
- "What did the Fed announce yesterday?" → **Web search** (real-time)
- "Show Q3 revenue by region" → **SQL database** (structured data)
- "How does our auth middleware validate tokens?" → **Code index** (codebase)

A flat retrieve-from-one-index approach either misses the answer or returns irrelevant passages. Routing selects the *right tool for the right sub-question* before retrieval begins.

Routing Strategies. Three main approaches, in increasing sophistication:

1. **Rule-based routing.** Keyword triggers (e.g., SQL keywords → database, URL patterns → web). Fast and interpretable but brittle for ambiguous queries.
2. **Classifier-based routing.** A lightweight model (e.g., a fine-tuned BERT classifier or logistic regression over query embeddings) predicts the best source. Low latency (<10 ms) and trainable on routing logs, but requires labeled data.
3. **LLM-based routing.** The LLM itself decides the source in a structured-output call (see Listing below). Most flexible—handles novel query types and can explain its reasoning—but adds one LLM call of latency.

Router as a Learned Policy

Multi-source routing is a *classification* problem at its simplest and a *planning* problem at its richest. When treated as an RL policy—where the state is the query plus conversation history, the action is the choice of source (and optional query rewrite), and the reward is downstream answer quality—the router can be optimized end-to-end via policy gradient techniques (Chapter 8).

Practical Considerations.

- **Fallback chains**: If the primary source returns low-confidence results, try the next-best source.
- **Parallel fan-out**: For ambiguous queries, retrieve from multiple sources simultaneously and fuse results via Reciprocal Rank Fusion (Table 16.2).
- **Cost awareness**: Web search and API calls may have monetary cost or rate limits; the router should factor these in.
- **Observability**: Log every routing decision with its reasoning—essential for debugging and retraining.

```
from enum import Enum
from pydantic import BaseModel

class KnowledgeSource(str, Enum):
    VECTOR_DB    = "vector_db"     # internal documents
    WEB_SEARCH   = "web_search"    # real-time web
    SQL_DB       = "sql_db"        # structured data
    CODE_INDEX   = "code_index"    # codebase
    API          = "api"           # external APIs

class RouteDecision(BaseModel):
    source: KnowledgeSource
    refined_query: str
    reasoning: str

def route_query(query: str, llm) -> RouteDecision:
    """Use LLM to decide which knowledge source to query."""
    prompt = f"""Given the query: "{query}"

Decide which knowledge source to use:
- vector_db: for internal documents, policies, past reports
- web_search: for current events, recent information
- sql_db: for numerical data, statistics, structured records
- code_index: for code examples, API documentation
- api: for real-time data (weather, stock prices, etc.)

Return a JSON with: source, refined_query, reasoning."""

    return llm.with_structured_output(RouteDecision).invoke(prompt)
```

Listing 16.9: Multi-source agentic RAG router

16.7.4 Full Agentic RAG Implementation

The previous sections introduced individual components—routing, retrieval, evaluation. A full agentic RAG system orchestrates these as a *graph of stateful nodes*, where control flow depends on intermediate results. The implementation below uses LangGraph to wire four nodes into a loop:

1. **Plan**: Decompose the user query into sub-queries (one per information need).
2. **Retrieve**: Route each sub-query to the appropriate source and fetch documents.
3. **Evaluate**: Judge whether the accumulated context is sufficient to answer the original query.
4. **Generate**: Synthesize a final answer with citations from the retrieved documents.

The key design pattern is the *conditional loop*: after evaluation, the agent either proceeds to generation (if context is sufficient or the iteration budget is exhausted) or loops back to retrieval with refined sub-queries. This mirrors the sense–act–evaluate cycle of an RL agent operating over information-gathering actions.

```python
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
import operator

class AgentState(TypedDict):
    query: str
    sub_queries: list[str]
    retrieved_docs: Annotated[list[dict], operator.add]
    context_sufficient: bool
    answer: str
    iterations: int
    max_iterations: int

def plan_node(state: AgentState) -> AgentState:
    """Decompose query into sub-queries."""
    sub_queries = decompose_query(state["query"])
    return {**state, "sub_queries": sub_queries, "iterations": 0}

def retrieve_node(state: AgentState) -> AgentState:
    """Retrieve documents for current sub-queries."""
    new_docs = []
    for sq in state["sub_queries"]:
        source = route_query(sq)
        docs = retrieve_from_source(sq, source)
        new_docs.extend(docs)
    return {**state, "retrieved_docs": new_docs,
            "iterations": state["iterations"] + 1}

def evaluate_node(state: AgentState) -> AgentState:
    """Evaluate whether retrieved context is sufficient."""
    sufficient = evaluate_context_sufficiency(
        query=state["query"],
        docs=state["retrieved_docs"]
    )
    return {**state, "context_sufficient": sufficient}

def generate_node(state: AgentState) -> AgentState:
    """Generate answer from retrieved context."""
    answer = generate_with_citations(
        query=state["query"],
        docs=state["retrieved_docs"]
    )
    return {**state, "answer": answer}

def should_retrieve(state: AgentState) -> str:
    if state["context_sufficient"]:
        return "generate"
    if state["iterations"] >= state["max_iterations"]:
        return "generate"  # give up and generate with what we have
    return "retrieve"

# Build the graph
workflow = StateGraph(AgentState)
workflow.add_node("plan",     plan_node)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("evaluate", evaluate_node)
workflow.add_node("generate", generate_node)

workflow.set_entry_point("plan")
workflow.add_edge("plan",     "retrieve")
workflow.add_edge("retrieve", "evaluate")
workflow.add_conditional_edges("evaluate", should_retrieve,
    {"retrieve": "retrieve", "generate": "generate"})
workflow.add_edge("generate", END)
```

```
agent = workflow.compile()

# Run
result = agent.invoke({
    "query": "What were the main causes of the 2023 banking crisis?",
    "max_iterations": 3,
    "retrieved_docs": [],
    "iterations": 0,
})
```

Listing 16.10: LangGraph-based agentic RAG

16.7.5 Tool-Augmented RAG

Agentic RAG can combine retrieval with computation tools:

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.tools import tool

@tool
def search_documents(query: str) -> str:
    """Search internal document knowledge base."""
    docs = vectorstore.similarity_search(query, k=5)
    return "\n\n".join(d.page_content for d in docs)

@tool
def query_database(sql: str) -> str:
    """Execute SQL query on the analytics database."""
    return db.run(sql)

@tool
def web_search(query: str) -> str:
    """Search the web for current information."""
    return tavily_client.search(query)

@tool
def execute_python(code: str) -> str:
    """Execute Python code for calculations."""
    return python_repl.run(code)

tools = [search_documents, query_database, web_search, execute_python]
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Listing 16.11: Tool-augmented RAG with SQL and retrieval

16.7.6 Search-R1: RL-Trained Agentic RAG

The agentic RAG approaches above rely on *prompt-engineered* orchestration — the agent's search behavior is controlled by instructions, not learned through training. **Search-R1** [180] takes a fundamentally different approach: it trains the LLM via reinforcement learning to *learn when, what, and how many times to search* as part of its reasoning process.

Core Idea. Search-R1 extends the DeepSeek-R1 [75] reasoning framework by treating search engine queries as **actions** within the RL training loop. During chain-of-thought generation, the model can emit special tokens `<search>query</search>` that trigger real-time retrieval from a search engine. The retrieved results are injected back into the reasoning context, and the model continues generating.

Formal Setup. The model generates a reasoning trace interleaved with search actions:

$$\underbrace{\text{think}_1}_{\text{reasoning}} \rightarrow \underbrace{\texttt{⟨search⟩}q_1\texttt{⟨/search⟩}}_{\text{action}} \rightarrow \underbrace{[\text{results}_1]}_{\text{observation}} \rightarrow \text{think}_2 \rightarrow \texttt{⟨search⟩}q_2\texttt{⟨/search⟩} \rightarrow \cdots \rightarrow \text{answer}$$

The entire trajectory (reasoning + searches + final answer) is scored by a terminal reward: correctness of the final answer against a ground-truth label.

Training Algorithm. Search-R1 uses GRPO (Group Relative Policy Optimization):

1. **Sample N trajectories** per question, each potentially containing 0–5 search calls
2. **Execute searches in real-time** — the environment returns actual search engine results
3. **Score terminal answer correctness** (exact match or F1 against ground truth)
4. **Compute group-relative advantage**: $\hat{A}_i = (R_i - \mu_G)/\sigma_G$
5. **Update policy** with GRPO clipped objective — reinforcing trajectories that searched effectively

The model learns to:

- **Search when uncertain** — avoid unnecessary searches for knowledge it already has
- **Formulate effective queries** — learn query phrasing that returns relevant results
- **Search multiple times** — iteratively refine queries based on initial results
- **Integrate retrieved context** — use search results to support or correct its reasoning

Table 16.4: Search-R1 (RL-trained) vs. prompt-based Agentic RAG.

Dimension	Prompt-Based Agentic RAG	Search-R1
Search decision	Prompt/heuristic	Learned via RL
Query formulation	Prompted ("rewrite query")	Trained end-to-end
# searches	Fixed or LLM-decided at inference	Learned optimal count
Training signal	None (frozen model)	Correctness reward
Search integration	Append to context	Interleaved in CoT
Failure recovery	Retry heuristics	Learned backoff/reformulation
Overhead at inference	Framework overhead (LangGraph)	Native model behavior

How Search-R1 Differs from Prompt-Based Agentic RAG.

Results. On open-domain QA benchmarks (NQ [204], TriviaQA [183], HotpotQA [427]), Search-R1 with a 7B model outperforms:

- Standard RAG (single retrieval) by 15–20% accuracy
- Prompted agentic RAG (ReAct-style) by 8–12% accuracy
- Approaches the performance of much larger models (70B) with standard RAG

The key insight: **learning when and how to search is more valuable than having a larger model that knows more**. A small model that searches well beats a large model that doesn't search.

Search-R1: The Paradigm Shift

Traditional RAG asks: "Given this query, what should I retrieve?" (a pipeline decision made before generation).
Search-R1 asks: "Given what I've reasoned so far, do I need more information? If so, what specific question would fill this gap?" (a learned decision made *during* generation).

This is the difference between a student who looks up the textbook before starting an exam, versus one who consults references mid-problem when they realize they're stuck. The latter is more efficient and more targeted.

16.8 Evaluation

Evaluating a RAG system is harder than evaluating retrieval or generation in isolation, because errors can originate at *any stage* of the pipeline—and they compound. A perfect generator cannot compensate for irrelevant retrievals, and a perfect retriever is wasted if the generator hallucinates or ignores the context.

Effective RAG evaluation therefore operates at **three levels**:

1. **Retrieval quality**: Did the retriever surface the right passages? (Recall, Precision, MRR, NDCG)

2. **Generation quality**: Is the answer correct, faithful to the retrieved context, and complete? (Correctness, Faithfulness, Answer Relevance)

3. **End-to-end quality**: Does the full system satisfy the user? (Human preference, task success rate, latency-adjusted utility)

A common failure mode is optimizing only one level—for example, maximizing Recall@K with large K fills the context with marginally relevant passages that actually *degrade* generation quality. The metrics below cover both retrieval and generation, enabling practitioners to diagnose which stage is the bottleneck.

16.8.1 Retrieval Metrics

Let $\mathcal{R}_k$ be the set of retrieved documents at rank k, and $\mathcal{R}^*$ be the set of relevant documents.

Recall@K.

$$\text{Recall@}K = \frac{|\mathcal{R}_K \cap \mathcal{R}^*|}{|\mathcal{R}^*|} \tag{16.22}$$

Precision@K.

$$\text{Precision@}K = \frac{|\mathcal{R}_K \cap \mathcal{R}^*|}{K} \tag{16.23}$$

Mean Reciprocal Rank (MRR).

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \tag{16.24}$$

where rank_i is the rank of the first relevant document for query i.

Normalized Discounted Cumulative Gain (NDCG@K).

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{DCG@}K = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)} \tag{16.25}$$

where $\text{rel}_i \in \{0, 1, 2, \ldots\}$ is the graded relevance of the i-th result and IDCG is the ideal (perfect) DCG.

16.8.2 Generation Metrics

Faithfulness. Measures whether the generated answer is *grounded* in the retrieved context—i.e., every claim in the answer can be attributed to a retrieved document. Evaluated by an LLM judge:

$$\text{Faithfulness} = \frac{\#\text{ claims supported by context}}{\#\text{ total claims in answer}} \tag{16.26}$$

Answer Relevance. Measures whether the answer addresses the question. Computed by generating questions from the answer and measuring similarity to the original query:

$$\text{AnswerRelevance} = \frac{1}{N}\sum_{i=1}^{N}\cos(E(q), E(\hat{q}_i)) \tag{16.27}$$

where $\hat{q}_i$ are questions generated from the answer.

Context Precision and Recall.

$$\text{ContextPrecision@}K = \frac{1}{K}\sum_{k=1}^{K}\text{Precision@}k \cdot \mathbf{1}[\text{doc}_k \text{ is relevant}] \tag{16.28}$$

$$\text{ContextRecall} = \frac{\#\text{ ground-truth claims attributable to context}}{\#\text{ total ground-truth claims}} \tag{16.29}$$

16.8.3 RAGAs Framework

RAGAs (Retrieval Augmented Generation Assessment) [89] provides a reference-free evaluation framework using LLM judges:

```python
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall,
    answer_correctness,
)
from datasets import Dataset

eval_dataset = Dataset.from_dict({
    "question":  questions,
    "answer":    generated_answers,
    "contexts":  retrieved_contexts,   # list of lists
    "ground_truth": reference_answers,
})

results = evaluate(
    dataset=eval_dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall,
        answer_correctness,
    ],
)
print(results.to_pandas())
```

Listing 16.12: RAGAs evaluation (v0.1 API; v0.2+ uses `user_input`, `response`, `retrieved_contexts`, `reference`)

16.8.4 Common Failure Modes

RAG Failure Modes to Monitor

1. **Retrieval Miss**: The relevant document exists in the corpus but is not retrieved. Causes: poor chunking, embedding model mismatch, query-document vocabulary gap.
2. **Context Poisoning**: Retrieved documents contain misleading or contradictory information that causes the model to generate incorrect answers.
3. **Lost-in-the-Middle**: LLMs attend more strongly to the beginning and end of long contexts; relevant information in the middle may be ignored [237].
4. **Over-Retrieval**: Too many retrieved chunks dilute the relevant signal and increase latency and cost.
5. **Hallucination Despite Retrieval**: Model ignores retrieved context and generates from parametric memory, especially when context contradicts training data.
6. **Citation Fabrication**: Model attributes claims to documents that do not support them.

16.9 Production Considerations

16.9.1 Embedding Model Selection

The embedding model is the single most impactful component choice in a RAG system—it determines the quality ceiling for retrieval. The field has advanced rapidly; Table 16.5 summarizes current options across the cost–quality spectrum.

Table 16.5: Embedding models for production RAG (as of 2026). MTEB scores are overall averages across retrieval, classification, clustering, and STS tasks.

Model	Dims	Max Tokens	MTEB Avg	Access	Notes
API-based (managed)					
Voyage `voyage-4-large`	1024*	32K	—	API	Best retrieval quality
OpenAI `text-embedding-3-large`	3072	8191	64.6	API	Matryoshka dims
Cohere `embed-english-v3.0`	1024	512	64.5	API	int8/binary support
Google `text-embedding-005`	768	2048	—	API	Vertex AI integration
Open-weight (self-hosted)					
`nvidia/NV-Embed-v2` [213]	4096	32K	72.3	Free	#1 MTEB (Sep 2024)
`Alibaba-NLP/gte-Qwen2-7B` [225]	3584	32K	70.2	Free	Apache-2.0, multilingual
`BAAI/bge-m3` [47]	1024	8192	65.0	Free	Dense + sparse + multi-vec
`jinaai/jina-embeddings-v3`	1024	8192	66.0	Free	Multilingual, LoRA adapters
`BAAI/bge-large-en-v1.5` [417]	1024	512	64.2	Free	Mature, well-supported

Selection Criteria.

- **Domain match**: Specialized models (e.g., `voyage-code-3` for code, `voyage-finance-2` for finance) can outperform general models by 5–15% on domain tasks.
- **Context length**: Models with 32K token context (Voyage-4, NV-Embed-v2) can embed entire documents without chunking, simplifying the pipeline.
- **Matryoshka embeddings**: Models supporting flexible output dimensions (256–4096) let you trade quality for storage/latency at serving time without re-encoding.
- **Quantization support**: int8 or binary quantization at the model level (Cohere, Voyage) reduces index size by 4–32× with minimal recall loss.
- **Multilingual**: For non-English or cross-lingual RAG, prefer models explicitly trained multilingual (BGE-M3, Jina-v3, Voyage-4).

16.9.2 Vector Database Comparison

Table 16.6: Vector database comparison for production RAG systems

Database	Hosting	Scale	Filtering	Hybrid	Best For
FAISS1	Self-hosted	Billions	Limited	No	Research, offline
Pinecone2	Managed	Billions	Yes	Yes	Serverless, easy setup
Weaviate3	Both	Billions	Yes	Yes	GraphQL, multi-modal
Chroma4	Self-hosted	Millions	Yes	No	Local dev, prototyping
Qdrant5	Both	Billions	Yes	Yes	High performance
Milvus6	Both	Billions	Yes	Yes	Enterprise, GPU accel.
pgvector7	Self-hosted	Millions	Yes	Yes	Existing Postgres users

5https://qdrant.tech 6https://milvus.io 7https://github.com/pgvector/pgvector

16.9.3 Latency Optimization

1. **Pre-filtering**: Use metadata filters (date range, category, source) to reduce the search space before ANN search
2. **Approximate NN**: Use HNSW or IVF indices instead of exact search; accept ∼1% recall loss for 10× speedup
3. **Embedding caching**: Cache embeddings for frequently repeated queries
4. **Async retrieval**: Retrieve from multiple sources in parallel
5. **Streaming generation**: Stream LLM output while retrieval completes
6. **Quantization**: Use int8 or binary quantization for embeddings to reduce memory and increase throughput

Async Parallel Retrieval. Techniques (3) and (4) above compose naturally: cache the query embedding, then fan out retrieval requests to multiple backends concurrently. In a multi-source RAG system (Section 16.7), the user query may need results from a vector database, a keyword index, and a web API. Sequential retrieval adds latencies; parallel retrieval pays only the cost of the *slowest* source. Listing 16.13 demonstrates this pattern using Python's `asyncio`—the `lru_cache` decorator ensures repeated queries skip the embedding model entirely, while `asyncio.gather` dispatches all source queries simultaneously.

```python
import asyncio
from functools import lru_cache

@lru_cache(maxsize=1024)
def get_cached_embedding(text: str) -> list[float]:
    return embedding_model.embed_query(text)

async def parallel_retrieve(
    query: str,
    sources: list[str],
    k: int = 5
) -> list[dict]:
    """Retrieve from multiple sources in parallel."""
    tasks = [
        asyncio.create_task(retrieve_from_source_async(query, src, k))
        for src in sources
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)
    # Flatten and deduplicate
    all_docs = []
    for r in results:
        if not isinstance(r, Exception):
            all_docs.extend(r)
```

```
    return deduplicate_by_content(all_docs)
```

Listing 16.13: Async parallel retrieval for low latency

16.9.4 Incremental Indexing and Versioning

In production, the document corpus is never static—policies get revised, new reports land daily, deprecated content must be removed. A full re-index (re-chunk, re-embed, re-upload) is expensive and causes downtime. Incremental indexing solves this by applying changes at the document level.

Core Operations.

- **Upsert**: When a document is created or updated, delete all existing chunks for that `doc_id`, re-chunk the new content, embed, and insert. This guarantees no stale fragments linger.
- **Delete/Expire**: Remove chunks by document ID (explicit deletion) or by TTL (automatic garbage collection for time-sensitive sources like news or market data).
- **Version tracking**: Store a `version` and `indexed_at` timestamp in chunk metadata. This enables rollback (restore previous version from source) and auditability ("which version did the model see?").

Consistency Challenges.

- **Embedding model drift**: If you upgrade the embedding model, old and new vectors are incompatible. Solutions: (a) maintain separate indices per model version and migrate in the background, or (b) use Matryoshka-compatible models where dimension truncation preserves compatibility.
- **Chunk boundary shifts**: Changing the chunking strategy invalidates all existing chunks. Version metadata lets you identify and selectively re-index affected documents.
- **Eventual consistency**: In distributed vector databases, newly upserted vectors may not be immediately searchable. Design your pipeline to tolerate a brief indexing lag (typically seconds to minutes).

Implementation. Listing 16.14 shows a minimal `RAGIndexManager` class that encapsulates upsert and expiration logic, suitable for wrapping any vector store with metadata filtering support.

```
class RAGIndexManager:
    def __init__(self, vectorstore, metadata_store, chunker, embedder):
        self.vs = vectorstore
        self.meta = metadata_store
        self.chunker = chunker
        self.embedder = embedder

    def upsert_document(self, doc_id: str, content: str,
                        metadata: dict) -> None:
        """Add or update a document, replacing old chunks."""
        # Delete existing chunks for this document
        self.vs.delete(filter={"doc_id": doc_id})
        # Chunk new version (vectorstore embeds internally)
        chunks = self.chunker.split_text(content)
        self.vs.add_texts(
            texts=chunks,
            metadatas=[{**metadata, "doc_id": doc_id,
                        "version": metadata.get("version", 1),
                        "indexed_at": datetime.utcnow().isoformat()}
                       for _ in chunks],
        )
```

```
    def expire_old_documents(self, ttl_days: int = 365) -> int:
        """Remove documents older than TTL."""
        cutoff = (datetime.utcnow() - timedelta(days=ttl_days)).isoformat()
        return self.vs.delete(filter={"indexed_at": {"$lt": cutoff}})
```

Listing 16.14: Incremental index updates with versioning

16.10 RAG + Fine-Tuning Synergy

16.10.1 When to Combine RAG with Fine-Tuning

Fine-tuning and RAG address complementary weaknesses:

- **Fine-tuning alone**: model learns style and format but may hallucinate facts
- **RAG alone**: model has access to facts but may not know how to use them optimally
- **Combined**: fine-tune the model to *use retrieved context well*—cite sources, acknowledge uncertainty, and ignore irrelevant context

16.10.2 RAFT: Retrieval-Augmented Fine-Tuning

RAFT [448] trains models to answer questions given a mix of relevant and *distractor* documents, teaching the model to identify and use only the relevant context:

1. For each training example (q, a, d^*), sample $k-1$ distractor documents $\{d_i^-\}$
2. Fine-tune on: `[q,` d^*`,` d_1^-`, ...,` d_{k-1}^-`]` $\rightarrow$ `[chain-of-thought + a]`
3. The chain-of-thought explicitly quotes from d^*, teaching the model to ground answers

$$\mathcal{L}_{\text{RAFT}} = -\mathbb{E}_{(q,a,d^*,\{d_i^-\})}\left[\log P_\theta\Big(\text{CoT}(d^*) \oplus a \;\Big|\; q, d^*, \{d_i^-\}\Big)\right] \tag{16.30}$$

16.10.3 Joint Retriever-Generator Training

For maximum performance, the retriever and generator can be trained jointly. The REALM [136] and RAG [219] papers propose end-to-end training where gradients flow through the retrieval step:

$$\nabla_\theta \mathcal{L} = \nabla_\theta \left[-\log \sum_{d \in \mathcal{D}} P_\theta(a \mid q, d) \cdot P_\phi(d \mid q) \right] \tag{16.31}$$

The retriever parameters ϕ are updated using the REINFORCE estimator or by treating $P_\phi(d \mid q)$ as a differentiable attention over documents.

Joint Training Challenges

Joint retriever-generator training is powerful but complex: (1) the document index must be periodically refreshed as ϕ changes (asynchronous index refresh), (2) the training signal is sparse (only top-k documents contribute), and (3) training is unstable without careful initialization from a pre-trained retriever.

16.11 Comprehensive RAG Approach Comparison

Table 16.7: RAG approaches across key dimensions

Approach	Accuracy	Latency	Complexity	Cost	Best For
Naive RAG [219]	Medium	Low	Low	Low	Prototyping, simple QA
RAG + Re-ranking [283]	High	Medium	Medium	Medium	Production QA systems
HyDE [112]	High	Medium	Low	Medium	Semantic mismatch domains
Multi-Query RAG	High	Medium	Medium	Medium	Ambiguous queries
RAG-Fusion [318]	High	Medium	Medium	Medium	Diverse query types
Self-RAG [14]	High	Medium	High	Medium	Selective retrieval
CRAG [423]	High	Medium	High	High	Unreliable corpora
Adaptive RAG [172]	High	Low–High	High	Medium	Mixed query complexity
Graph RAG [86]	V. High	High	V. High	High	Global synthesis queries
Agentic RAG	V. High	High	V. High	High	Multi-hop reasoning
RAFT [448]	V. High	Low	V. High	V. High	Domain-specific deployment

Key Design Questions for RAG Systems

When designing a RAG system for production, consider:

1. **What is the query distribution?** Factoid vs. analytical vs. multi-hop queries require different retrieval strategies.
2. **How large and dynamic is the corpus?** Millions of documents with frequent updates favor managed vector databases with incremental indexing.
3. **What are the latency requirements?** Sub-100ms responses preclude re-ranking and agentic loops; batch or async use cases can afford them.
4. **How critical is grounding?** High-stakes domains (medical, legal, financial) require faithfulness evaluation and citation verification.
5. **Is the vocabulary specialized?** Domain-specific terminology may require hybrid retrieval or domain-adapted embedding models.

RAG Best Practices Summary

- **Start simple**: naive RAG with good chunking often outperforms complex systems with poor chunking
- **Evaluate retrieval separately**: fix retrieval before optimizing generation
- **Use hybrid retrieval**: BM25 + dense with RRF is a strong default
- **Add re-ranking**: a cross-encoder re-ranker on top-20 candidates is high ROI
- **Monitor faithfulness**: track hallucination rate in production with LLM judges
- **Cache aggressively**: embed documents once; cache frequent query embeddings
- **Chunk with overlap**: 10–15% overlap prevents information loss at boundaries
- **Store rich metadata**: source, date, section, and document type enable powerful pre-filtering that dramatically improves precision

Chapter 17

Agentic Memory Systems

Memory is the faculty that transforms a stateless language model into a persistent agent. Without memory, every interaction starts from scratch: the agent cannot learn from past mistakes, cannot recall user preferences, and cannot build on prior work. With memory, an agent accumulates knowledge across sessions, adapts to individual users, and maintains coherent long-horizon behavior. This chapter covers the taxonomy of agentic memory systems—from in-context working memory to external vector stores—and the engineering patterns for building agents that remember.

GAS Connection — Agentic Memory Systems

The GAS uses multiple memory tiers described in the following chapter, and the interplay between them is one of the most interesting design challenges in the system.

Memory taxonomy (§ Taxonomy of Memory Types).

- **In-context (working) memory**: The current patient session—the last 30 minutes of CGM readings, the current meal log entry, and the active medication list—is assembled into the context window at the start of each pipeline invocation. This is the most expensive memory tier (consumes context tokens) and the most current. It is captured in the `GASState` TypedDict in `capstone_gas/gas_system.py`.
- **External (semantic) memory**: The `GASKnowledgeBase` holds 15 ADA guideline entries indexed with FAISS (`faiss.IndexFlatIP`) and `sentence-transformers/all-MiniLM-L6-v2` embeddings. This memory is read-only at inference time.
- **Episodic memory**: The `readings_history` field in `GASState` stores the rolling history of glucose readings for the current session. The `GASMemory` dataclass in `shared/gas_state.py` provides a structured container for longer-term patient history.
- **Procedural memory**: The LangGraph `MemorySaver` checkpointer (used in `build_gas_graph()` in `capstone_gas/gas_system.py`) persists the full `GASState` between pipeline invocations, enabling longitudinal reasoning across multiple CGM readings.

Memory for multi-turn conversations (§ Memory for Multi-Turn Conversations). The `agent_messages` field in `GASState` accumulates the full message history across all agent invocations within a session. This allows the Caregiver Notifier to reference earlier reasoning steps when composing its alert message.

Memory for multi-agent systems (§ Memory for Multi-Agent Systems). The LangGraph shared `GASState` object is the GAS's shared working memory: all agent nodes read from and write to it within a single pipeline invocation. The `MemorySaver` checkpointer persists this state between invocations, providing the two-tier memory architecture (in-process shared state + persistent checkpoint) recommended in this section for multi-agent systems.

17.1 Motivation: Why Agents Need Memory

Large language models are, at their core, stateless function approximators: given a prompt x, they produce a distribution over continuations $p_\theta(y \mid x)$. Every inference call begins from scratch. The *context window*—the finite sequence of tokens the model can attend to—is the only information available at generation time. For short, self-contained tasks this is sufficient. For long-horizon agentic tasks it is a fundamental bottleneck.

The Context-Window Bottleneck

Let L denote the maximum context length (e.g. $L = 128{,}000$ tokens for GPT-4o). A single token encodes roughly 4 characters; a typical book contains $\sim$500,000 words $\approx$ 670,000 tokens. Even ignoring cost, a multi-day autonomous agent accumulates observations, tool outputs, and reasoning traces that *cannot* fit in any fixed window. Memory systems are the engineering response to this physical constraint.

Three distinct failure modes arise when agents lack persistent memory:

1. **Catastrophic forgetting of context.** Once an event scrolls out of the context window it is irrecoverably lost. The agent cannot refer back to a decision made 10,000 tokens ago.
2. **Inability to learn from experience.** Without episodic storage, every episode is the agent's first. Successful strategies cannot be reused; mistakes are repeated.
3. **Lack of personalization.** User preferences, domain facts, and relationship history must be re-established in every session, degrading user experience and efficiency.

Memory as Cognitive Architecture

Cognitive science distinguishes several memory systems in biological agents [356, 377]: *working memory* (active manipulation of information), *episodic memory* (autobiographical events), *semantic memory* (world knowledge), and *procedural memory* (skills and habits). Effective agentic AI systems benefit from analogous distinctions—not because we are simulating neuroscience, but because these categories reflect genuinely different *access patterns*, *update frequencies*, and *retrieval mechanisms*.

Formally, we model an agent as a tuple $\mathcal{A} = (\pi_\theta, \mathcal{M}, \mathcal{R}, \mathcal{W})$ where π_θ is the policy (the LLM), $\mathcal{M}$ is the memory store, $\mathcal{R} : \mathcal{Q} \times \mathcal{M} \to \mathcal{D}$ is a retrieval function mapping queries to retrieved documents, and $\mathcal{W} : \mathcal{M} \times \mathcal{E} \to \mathcal{M}$ is a write function updating memory with new experiences $\mathcal{E}$. At each step t the agent observes o_t, retrieves relevant context $c_t = \mathcal{R}(o_t, \mathcal{M})$, and acts:

$$a_t \sim \pi_\theta(\cdot \mid [s_t;\, c_t;\, h_t])\,,$$

where s_t is the current system prompt, c_t is retrieved memory, and h_t is the recent in-context history. After acting, the agent may write new information: $\mathcal{M} \leftarrow \mathcal{W}(\mathcal{M},\, (o_t, a_t, r_t))$.

17.2 Taxonomy of Memory Types

17.2.1 Working Memory (Short-Term)

Working memory is the agent's *active workspace*: the information currently being manipulated. In LLM agents it corresponds to:

- **Scratchpads.** Intermediate reasoning steps written to a dedicated buffer before producing a final answer (e.g. chain-of-thought [395], scratchpad [287]).
- **Chain-of-thought buffers.** The sequence of reasoning tokens $z_1, z_2, \ldots, z_k$ generated before the answer token a, modeled as $p(a \mid x) = \sum_z p(a \mid x, z)\, p(z \mid x)$.
- **Conversation context.** The recent turn history $[(u_1, a_1), \ldots, (u_t, a_t)]$ kept in the context window.

Working memory is *fast* (zero retrieval latency—it is already in context), *volatile* (lost when the context is cleared), and *capacity-limited* (bounded by L).

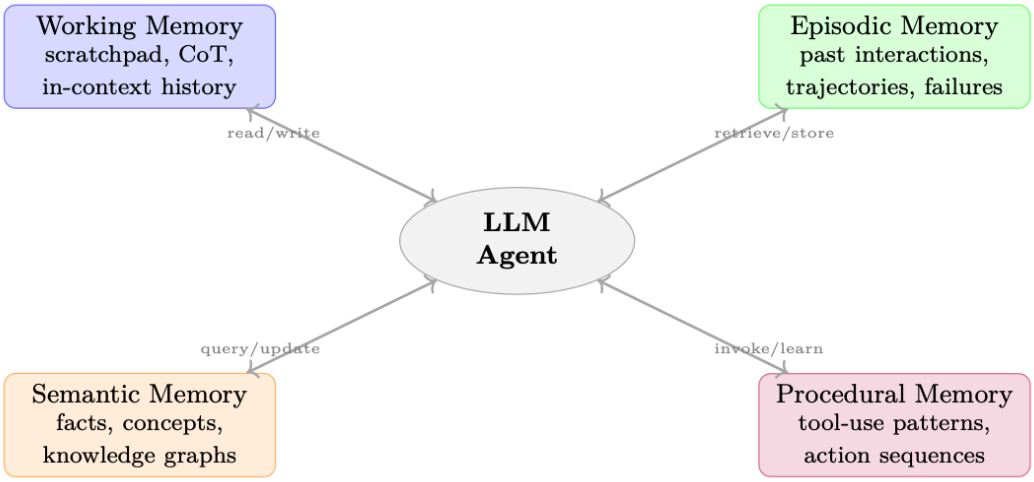


Figure 17.1: Four-way taxonomy of agentic memory systems, mirroring cognitive science distinctions. Each memory type has distinct access patterns, update frequencies, and retrieval mechanisms.

17.2.2 Episodic Memory (Experience-Based)

Episodic memory stores *specific past events* indexed by context and time. For agents:

- **Past interactions.** Full or summarized records of prior conversations, task attempts, and their outcomes.
- **Successful trajectories.** High-reward action sequences that can be retrieved as few-shot exemplars for similar future tasks.
- **Failure cases.** Documented mistakes with root-cause annotations, enabling the agent to avoid repeating errors.
- **Retrieval-augmented episodic recall.** Given a new task q, retrieve the k most similar past episodes $\{e_i\}_{i=1}^{k}$ and include them in context.

Episodic memory is typically implemented as a vector store (Section 17.3.1) with embeddings over episode summaries.

17.2.3 Semantic Memory (World Knowledge)

Semantic memory encodes *general facts and concepts* decoupled from specific episodes:

- **Factual knowledge.** Entities, attributes, and relationships (e.g. "Paris is the capital of France").
- **Domain concepts.** Definitions, taxonomies, and ontologies relevant to the agent's task domain.
- **Knowledge graphs.** Structured representations $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where nodes $v \in \mathcal{V}$ are entities and edges $e \in \mathcal{E}$ are typed relations.

Unlike episodic memory, semantic memory is *context-independent*: the fact that water boils at 100°C is true regardless of when or where it was learned.

17.2.4 Procedural Memory (Skills)

Procedural memory encodes *how to do things*—skills and action patterns that have been automatized:

- **Learned tool-use patterns.** Which API to call for which task, how to format inputs, how to handle errors.

- **Action sequences.** Multi-step procedures (e.g. "to deploy code: run tests → build image → push → update manifest").
- **Policies as memory.** The model weights θ themselves encode procedural knowledge; fine-tuning on successful trajectories is a form of procedural memory consolidation.

Memory Type Classification

An agent helping with software development uses:

- **Working**: the current file being edited, the error message just received.
- **Episodic**: "Last week I fixed a similar `NullPointerException` in module X by adding a null check at line 42."
- **Semantic**: "Python's `asyncio.gather` runs coroutines concurrently; exceptions propagate unless `return_exceptions=True`."
- **Procedural**: the standard debugging workflow: reproduce → isolate → hypothesize → test → fix.

17.3 Memory Architectures

17.3.1 RAG-Based Memory

Retrieval-Augmented Generation (RAG) [219] is the dominant paradigm for external memory in LLM agents. The memory store $\mathcal{M}$ is a collection of documents $\{d_i\}_{i=1}^{N}$; retrieval maps a query q to a ranked subset.

Embedding Stores and Vector Databases. Each document d_i is encoded by an embedding model ϕ: $\mathbf{v}_i = \phi(d_i) \in \mathbb{R}^D$. Queries are similarly encoded: $\mathbf{q} = \phi(q)$. Retrieval returns the top-k documents by similarity:

$$\text{Retrieve}(q, \mathcal{M}, k) = \underset{S \subseteq [N],\, |S|=k}{\arg\max} \sum_{i \in S} \text{sim}(\mathbf{q}, \mathbf{v}_i),$$

where $\text{sim}(\cdot,\cdot)$ is typically cosine similarity. Approximate nearest-neighbor (ANN) indices (FAISS [181], HNSW [261], ScaNN [134]) make this tractable for $N \sim 10^7$.

Retrieval Strategies.

- **Dense retrieval.** Both query and documents are encoded by neural encoders (e.g. DPR [190], `text-embedding-3-large`). Captures semantic similarity but requires GPU inference.
- **Sparse retrieval.** BM25 or TF-IDF over token overlap. Fast, interpretable, strong for exact keyword matches.
- **Hybrid retrieval.** Combine dense and sparse scores via reciprocal rank fusion (RRF):

$$\text{RRF}(d, k) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)},$$

  where $k = 60$ is a smoothing constant. Hybrid consistently outperforms either alone [46].

Re-ranking. A cross-encoder re-ranker $f_\psi(q, d) \in [0, 1]$ scores each retrieved document jointly with the query, providing higher accuracy at the cost of $O(k)$ forward passes. The pipeline is: retrieve $k' \gg k$ candidates with ANN, re-rank with cross-encoder, return top k.

Retrieval Hallucination Risk

RAG does not eliminate hallucination—it can *introduce* it. If the retrieved document is outdated, incorrect, or only superficially relevant, the model may confidently incorporate false information. Always include provenance metadata (source, timestamp, confidence) and consider faithfulness verification steps.

17.3.2 Summarization-Based Memory

When verbatim storage is too expensive or noisy, *summarization* compresses information before storage.

Progressive Summarization. At each step t, the agent maintains a running summary S_t. When new information e_t arrives:

$$S_{t+1} = \text{LLM}(\texttt{"Summarize: [}S_t\texttt{] + [}e_t\texttt{]"}).$$

This keeps memory size $O(1)$ but risks losing detail.

Hierarchical Compression. Organize memory in levels $L_0 \supset L_1 \supset \cdots \supset L_K$ where L_0 is verbatim and each L_{i+1} is a summary of L_i. Retrieval first checks L_K (most compressed, fastest) and drills down as needed. This mirrors the *progressive summarization* technique of Forte [102].

When to Summarize vs. Store Verbatim.

- Store verbatim: precise facts, code snippets, numerical results, user quotes.
- Summarize: narrative context, reasoning chains, redundant observations.
- Discard: noise, failed tool calls with no informational content.

17.3.3 Graph-Based Memory

Knowledge Graphs. A knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{R})$ stores facts as triples (h, r, t) where $h, t \in \mathcal{V}$ are entities and $r \in \mathcal{R}$ is a relation. Agents can query via SPARQL [142], Cypher [103], or natural-language-to-graph translation.

Entity-Relation Extraction. New observations are parsed by an extraction model $\text{IE} : \text{text} \to \{(h_i, r_i, t_i)\}$ and merged into $\mathcal{G}$. Coreference resolution and entity linking ensure consistency.

GraphRAG. GraphRAG [86] augments RAG with graph traversal: given a query, retrieve seed entities, then expand via k-hop neighborhood traversal to surface related facts not directly matched by embedding similarity. This is particularly powerful for multi-hop reasoning:

$$\text{GraphRetrieve}(q, \mathcal{G}, k) = \bigcup_{v \in \text{seeds}(q)} \mathcal{N}_k(v, \mathcal{G}),$$

where $\mathcal{N}_k(v, \mathcal{G})$ is the k-hop neighborhood of v.

Temporal Knowledge Graphs. Facts have validity intervals: $(h, r, t, [t_{\text{start}}, t_{\text{end}}])$. Temporal KGs [208] enable queries like "Who was the CEO of OpenAI in 2023?" without conflating past and present states.

17.3.4 Key-Value Memory Networks

Differentiable memory networks [361, 399] represent memory as a set of key-value pairs $\{(\mathbf{k}_i, \mathbf{v}_i)\}_{i=1}^{M}$ with soft attention-based retrieval:

$$\alpha_i = \text{softmax}\left(\frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{D}}\right), \qquad \mathbf{c} = \sum_{i=1}^{M} \alpha_i \mathbf{v}_i.$$

The retrieved context $\mathbf{c}$ is a differentiable function of the query, enabling end-to-end training. Modern transformer attention is a special case of this mechanism. For agentic use, memory slots can be updated via gradient descent or via explicit write operations.

17.3.5 MemGPT and Virtual Context Management

MemGPT [297] introduces a *virtual context* abstraction analogous to virtual memory in operating systems. Memory is organized in tiers:

Page-In / Page-Out Strategies. The agent decides *which* memory to promote to hot context (page-in) and *which* to evict (page-out) based on:

- **Recency:** recently accessed items are more likely to be needed.
- **Relevance:** items with high similarity to the current query.
- **Importance:** items tagged as high-importance during write.

Self-Directed Memory Management. In MemGPT, the LLM itself issues memory management function calls (`memory_search`, `memory_insert`, `memory_delete`) as part of its action space. This makes memory management a *learned behavior* rather than a hard-coded policy—a natural target for RL training (Section 17.7).

17.4 Memory Operations

17.4.1 Write: Committing to Memory

Not every observation should be stored. The write decision is a filtering problem:

$$\text{Write}(e) = \mathbf{1}[\text{importance}(e) > \tau],$$

where τ is a threshold and $\text{importance}(e)$ can be:

- **Surprise:** $-\log p_\theta(e \mid \text{context})$—unexpected events are more informative.
- **Reward signal:** events associated with high $|r_t|$ (positive or negative) are worth remembering.
- **LLM self-assessment:** prompt the model to rate importance on a 1–10 scale.

Contradiction Detection. Before writing a new fact f_{new}, check for conflicts with existing memory:

$$\text{Conflict}(f_{\text{new}}, \mathcal{M}) = \exists\, f \in \mathcal{M} : \text{Contradicts}(f_{\text{new}}, f).$$

Contradiction detection can be implemented via NLI models or by prompting the LLM. On conflict, the agent must decide: overwrite, keep both with timestamps, or flag for human review.

Table 17.1: Memory granularity trade-offs.

Format	Pros	Cons
Atomic facts "User prefers Python."	Precise retrieval; composable; easy deduplication and contradiction detection	Loses context; extraction errors; brittle for nuanced information
Structured notes (A-MEM [422])	Rich metadata (tags, links); supports graph traversal; balances precision and context	Higher write cost; schema design required
Summarized episodes (MemGPT [297])	Preserves narrative coherence; compact; good for multi-turn reasoning	Summarization lossy; hard to update partially
Verbatim transcripts	Lossless; no extraction errors; supports exact quotation	Large storage; noisy retrieval; expensive to scan

Memory Format and Granularity. Beyond *what* to store, the *how* matters greatly. Memory entries range from atomic facts to verbose transcripts, with distinct trade-offs:

In practice, production systems often combine granularities [55]: extract atomic facts for precise recall, maintain summarized episodes for narrative context, and archive verbatim transcripts in cold storage for auditability. The Generative Agents architecture [301] stores observations as atomic "memory objects" with natural-language descriptions, importance scores, and timestamps—enabling both precise retrieval and temporal reasoning.

Design Guidelines.

- **Match granularity to query type.** If users ask factoid questions ("What's my API key?"), atomic facts win. If they ask contextual questions ("Why did we decide to use Redis?"), episode summaries are needed.
- **Store at the finest grain you can afford**, then build coarser views on top. It is easy to summarize atomic facts; it is impossible to recover atoms from a lossy summary.
- **Include provenance.** Every memory entry should link back to its source (conversation turn, document, tool output) so the agent can verify and the user can audit.

17.4.2 Read / Retrieve

Query Formulation. The retrieval query q need not be the raw observation. Better strategies:

- **HyDE (Hypothetical Document Embeddings) [112]:** generate a hypothetical answer, embed it, and use that embedding as the query.
- **Query expansion:** generate multiple paraphrases of the query and take the union of retrieved results.
- **Step-back prompting:** abstract the specific query to a more general question before retrieval.

Temporal Decay and Recency Bias. Older memories may be less relevant. A time-weighted score:

$$\text{score}(d, q, t) = \lambda \cdot \text{sim}(\mathbf{q}, \mathbf{v}_d) + (1 - \lambda) \cdot \exp\left(-\frac{t - t_d}{\tau_{\text{decay}}}\right),$$

where t_d is the memory's creation time and τ_{decay} controls the decay rate. The Generative Agents paper [301] uses a similar recency-weighted retrieval.

Bi-Temporal Fact Management. Temporal decay tells the retrieval system to prefer recent memories, but it does not answer a harder question: *when was a given fact actually true?* This distinction matters in long-lived agents. Consider an agent that learns on Monday that a user's project deadline is Friday, and learns on Thursday that the deadline has moved to the following Monday. A naive memory store may retrieve the old deadline alongside the new one, with no way to determine which was valid at any given point in time.

Database theory distinguishes two orthogonal time axes [354]:

- **Valid time**: the period during which a fact was true in the world (e.g., "the deadline *was* Friday from Monday to Thursday").
- **Transaction time**: the period during which the agent's memory store contained the fact (e.g., "the agent *knew* the deadline was Friday from Monday to Thursday").

A **bi-temporal memory store** records both axes for every fact, enabling queries such as "what did the agent believe on Tuesday about the state of the world on Friday?" This is essential for auditability (reconstructing the agent's reasoning at any past point) and for conflict resolution (distinguishing a fact that was superseded from one that was always false).

Engineering a bi-temporal store. Each memory entry carries four timestamps: `valid_from`, `valid_to`, `recorded_at`, and `superseded_at`. Writes never overwrite existing entries; they close the previous entry's `valid_to` and insert a new entry. Retrieval filters on both axes:

```
def retrieve_bitemporal(store, query, world_time, query_time):
    """Return facts valid at world_time and known at query_time."""
    candidates = [
        e for e in store
        if e.valid_from <= world_time < (e.valid_to or datetime.max)
        and e.recorded_at <= query_time
        and (e.superseded_at is None or e.superseded_at > query_time)
    ]
    return rank_by_similarity(candidates, query)
```

Listing 17.1: Bi-temporal memory retrieval: what did the agent believe at query_time about the world at world_time?

Practical trade-offs. Bi-temporal stores are more expensive to maintain (every update is an insert, not an overwrite; storage grows monotonically) and more complex to query. For most short-lived agents, temporal decay is sufficient. Bi-temporal management is warranted when: (1) the agent operates continuously for weeks or months; (2) auditability is a regulatory requirement; or (3) the agent's decisions depend on historical state (e.g., "what was the policy when this contract was signed?").

17.4.3 Update: Conflict Resolution and Consolidation

Memory consolidation merges related memories to reduce redundancy and surface higher-level patterns:

$$\mathcal{M}' = \text{Consolidate}(\mathcal{M}) = \text{Cluster}(\mathcal{M}) \cup \text{Summarize}(\text{Cluster}(\mathcal{M})).$$

Forgetting Mechanisms. Biological memory forgets; so should artificial memory. Strategies:

- **LRU eviction:** remove least-recently-used entries when capacity is exceeded.
- **Importance-weighted forgetting:** $p(\text{forget} \mid d) \propto \exp(-\text{importance}(d))$.
- **Spaced repetition:** memories accessed repeatedly are retained longer, following the exponential forgetting curve [85].

17.4.4 Reflect: Meta-Cognitive Operations

Reflection [301, 344] is a higher-order memory operation: the agent reads its own memory and generates *insights*:

$$\text{Reflect}(\mathcal{M}) \to \{i_1, i_2, \ldots\} \subset \mathcal{M}_{\text{semantic}},$$

where each insight i_j is a higher-level abstraction derived from multiple episodic memories.

Reflection in Practice (Reflexion)

After three failed attempts to solve a coding problem, the agent reflects:

1. Retrieves the three failure episodes from episodic memory.
2. Generates an insight: "I keep forgetting to handle the edge case where the input list is empty."
3. Stores this insight in semantic memory.
4. On the next attempt, retrieves the insight and explicitly checks for empty inputs.

This is the core mechanism of Reflexion [344]: verbal reinforcement learning via self-reflection.

Where Do Reflections Live? Reflection *reads* from episodic memory but *writes* to semantic memory. The resulting insights are context-independent generalizations ("always check for empty inputs"), not episode-specific records—hence they belong in semantic memory $\mathcal{M}_{\text{semantic}}$. However, during the reflection process itself, the intermediate reasoning (retrieved episodes + synthesis prompt + generated insight) occupies *working memory* (the context window). In short:

- **Input**: episodic memory (specific past events)
- **Computation**: working memory (active reasoning in context)
- **Output**: semantic memory (durable, generalized insight)

This mirrors biological memory consolidation, where episodic experiences are gradually transformed into semantic knowledge during sleep and reflection.

17.5 Memory for Multi-Turn Conversations

17.5.1 User Modeling and Preference Tracking

A persistent user model $\mathcal{U}$ stores:

- **Explicit preferences:** stated likes/dislikes, communication style preferences.
- **Implicit preferences:** inferred from behavior (e.g. user always asks for code in Python, prefers concise answers).
- **Expertise level:** domain knowledge inferred from vocabulary and question complexity.
- **Goals and context:** ongoing projects, current tasks, organizational role.

The user model is updated after each interaction:

$$\mathcal{U}_{t+1} = \text{Update}(\mathcal{U}_t,\ (u_t, a_t, \text{feedback}_t)).$$

17.5.2 Session Continuity

Without memory, each conversation starts cold. With session memory:

1. At session start, retrieve the user model $\mathcal{U}$ and recent session summaries.
2. Inject a personalized system prompt: "You are helping Alice, a senior ML engineer working on a distributed training project. Last session you helped debug a gradient synchronization issue."
3. At session end, summarize the session and update $\mathcal{U}$.

17.5.3 Personalization Through Memory

Personalization improves both *efficiency* (fewer clarifying questions) and *quality* (responses calibrated to user expertise). Key techniques:

- **Adaptive verbosity:** adjust response length based on user's historical engagement.
- **Domain priming:** prepend relevant domain context from semantic memory.
- **Proactive recall:** surface relevant past interactions without being asked ("You asked about this topic last month; here's what we found then").

Privacy and Memory

Persistent user memory raises significant privacy concerns. Agents must: (1) obtain explicit consent before storing personal information, (2) provide mechanisms to inspect and delete stored memories, (3) enforce access controls in multi-user deployments, and (4) comply with data retention regulations (GDPR, CCPA). Memory systems should be designed with privacy-by-default.

17.6 Memory for Multi-Agent Systems

When multiple agents collaborate on a shared task, memory becomes a *coordination mechanism*—not just a personal knowledge store. A planning agent that decomposes a task must communicate sub-goals to executor agents; a critic agent must access the same context as the agent it evaluates; a research team of agents must avoid duplicating work. Without shared memory, agents must communicate everything through direct messages, creating bandwidth bottlenecks and losing information when conversations scroll out of context. Shared memory solves this by providing a persistent, queryable substrate that all agents can read from and write to—turning implicit coordination ("I hope the other agent remembers") into explicit state ("the answer is on the blackboard").

17.6.1 Shared Memory Pools

In multi-agent systems, agents may share a common memory store $\mathcal{M}_{\text{shared}}$ alongside private stores $\mathcal{M}_i$:

$$\text{context}_i(t) = \mathcal{R}(\mathcal{M}_i, q_i) \cup \mathcal{R}(\mathcal{M}_{\text{shared}}, q_i).$$

Shared memory enables *implicit coordination*: agent A writes a finding; agent B retrieves it without explicit communication.

17.6.2 Blackboard Architecture

The *blackboard* pattern [144] is a classic multi-agent coordination mechanism:

Each agent reads from and writes to the blackboard. A *controller* monitors the blackboard and activates agents when their preconditions are met. This decouples agents: they communicate through shared state rather than direct messaging.

17.6.3 Consensus and Conflict in Shared Knowledge

When multiple agents write to shared memory, conflicts arise. Resolution strategies:

- **Last-write-wins:** simple but loses information.
- **Versioned memory:** maintain a history of all writes; agents can query any version.
- **Voting / consensus:** require k-of-n agents to agree before a fact is committed.
- **Confidence-weighted merging:** $f_{\text{merged}} = \sum_i w_i f_i$ where w_i is agent i's confidence.
- **Designated authority:** assign ownership of memory regions to specific agents.

Open Problem: Distributed Memory Consistency

How should a multi-agent system maintain memory consistency under concurrent writes, network partitions, and adversarial agents? Classical distributed systems solutions (Paxos, Raft) apply but are expensive. Approximate consistency with bounded staleness may be sufficient for many agentic tasks—but the right trade-off is an open research question.

17.7 Training Memory Systems with Reinforcement Learning

17.7.1 Reward Signals for Memory Operations

Memory operations (read, write, update, reflect) can be treated as actions in the RL framework. The challenge is designing reward signals that incentivize *useful* memory behavior:

- **Task reward propagation.** If a memory retrieval leads to a correct answer, credit the retrieval action. Sparse but unambiguous.
- **Retrieval precision reward.** $r_{\text{retrieve}} = \text{Relevance}(d_{\text{retrieved}}, \text{task})$, estimated by a learned relevance model.
- **Memory efficiency reward.** Penalize unnecessary writes: $r_{\text{write}} = -\lambda \cdot \mathbf{1}[\text{write}]$, encouraging selective storage.
- **Consistency reward.** Reward memory states that are internally consistent (no contradictions).

The combined reward for a memory operation m_t at step t:

$$r_t^{\text{mem}} = r_t^{\text{task}} + \alpha \cdot r_t^{\text{retrieve}} + \beta \cdot r_t^{\text{write}} + \gamma \cdot r_t^{\text{consistency}}.$$

17.7.2 Learning What to Remember

The *what-to-remember* problem is a meta-learning challenge: the agent must learn a write policy $\pi_{\text{write}}(e)$ that maximizes future task performance. This is difficult because:

1. The value of a memory is only revealed in the future (delayed reward).
2. The space of possible future queries is unknown at write time.
3. Memories interact: the value of storing e depends on what else is in $\mathcal{M}$.

Approaches:

- **Hindsight relabeling** [9]. After a successful episode, retroactively label the memories that were retrieved as "important" and train the write policy to store similar items.
- **Meta-RL** [83]. Train the write policy across a distribution of tasks; the policy learns to store information that generalizes across tasks.
- **Curiosity-driven storage** [303]. Store observations that are surprising (high prediction error), as these are likely to be informative.

17.7.3 Memory-Augmented Policy Optimization

The idea of jointly optimizing a policy and its memory system dates to differentiable memory networks [127] and was extended to retrieval-augmented LLMs by REALM [136]. The full policy gradient objective for a memory-augmented agent:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] - \lambda \cdot \mathcal{L}_{\text{mem}}(\phi),$$

where θ are the LLM parameters, ϕ are the memory system parameters (e.g. retrieval model weights), and $\mathcal{L}_{\text{mem}}$ is a regularization term on memory complexity.

Key Insight: Memory as a Learned Inductive Bias

Training memory operations with RL allows the agent to develop task-specific memory strategies. A coding agent learns to store API signatures; a research agent learns to store citation chains; a customer service agent learns to store user complaint patterns. The memory system becomes a *learned inductive bias* tailored to the agent's domain.

17.8 Comparison of Memory Approaches

Table 17.2: Comparison of agentic memory architectures across key dimensions.

Architecture	Capacity	Retrieval	Update Cost	Trainable	Best For
In-context (working)	$O(L)$ tokens	0 ms	Free	Via fine-tuning	Short tasks, active reasoning
Dense RAG [219]	$O(10^7)$ docs	10–50 ms	$O(1)$ embed	Encoder only	Semantic search, QA
Sparse (BM25) [327]	$O(10^8)$ docs	1–5 ms	$O(\|d\|)$ index	No	Keyword search, legal/medical
Hybrid RAG [46]	$O(10^7)$ docs	15–60 ms	$O(1)$ embed	Encoder only	General-purpose retrieval
Summarization	Unlimited	0 ms (in-ctx)	$O(\|e\|)$ LLM call	Via fine-tuning	Long conversations, narratives
Knowledge Graph [208]	$O(10^9)$ triples	5–100 ms	$O(1)$ insert	Embedding layer	Structured facts, multi-hop
KV Memory Net [361]	$O(M)$ slots	$O(M)$ attn	Gradient step	Fully	End-to-end differentiable tasks
MemGPT tiered [297]	Unlimited	0–100 ms	Mixed	Via RL	Long-horizon agents, assistants
Graph RAG [86]	$O(10^7)$ nodes	20–200 ms	$O(1)$ insert	Encoder only	Complex reasoning, communities

17.9 Evaluating Memory Systems

Evaluating agentic memory is challenging because the quality of memory operations is only revealed *indirectly*—through downstream task performance over long horizons. A memory system that achieves perfect recall of stored facts can still fail if it retrieves irrelevant context or overwhelms the LLM's context window.

17.9.1 Evaluation Dimensions

LongMemEval [406] identifies five core capabilities that a long-term memory system must demonstrate:

1. **Information extraction.** Can the system identify and store salient facts from conversational turns? Measured by fact recall: what fraction of ground-truth facts are recoverable from memory?
2. **Multi-session reasoning.** Can the system synthesize information scattered across multiple past sessions? E.g., "Based on our conversations last week and yesterday, what changed in the project scope?"
3. **Temporal reasoning.** Can the system correctly answer time-dependent queries? E.g., "What did I say was my priority *before* the reorg?" requires distinguishing temporal states.
4. **Knowledge updates.** When facts change (user moves cities, preferences shift), does memory reflect the latest state while preserving history?
5. **Abstention.** When the system has no relevant memory, does it correctly say "I don't know" rather than hallucinate a plausible but fabricated recollection?

17.9.2 Benchmarks

Table 17.3: Benchmarks for evaluating agentic memory systems.

Benchmark	Venue	Scale	Focus
LongMemEval [406]	ICLR 2025	500 questions, scalable histories	Five memory abilities; multi-session chat
LOCOMO [260]	EMNLP 2024	Multi-session dialogues	Single-hop, temporal, multi-hop, open-domain QA over conversations
InfiniteBench [451]	ACL 2024	100K+ token contexts	Long-context recall, not memory-specific but tests limits

17.9.3 Metrics

Memory-Level Metrics.

- **Memory Recall**: $\frac{\#\text{ ground-truth facts retrievable from memory}}{\#\text{ total ground-truth facts}}$. Measures completeness of storage.
- **Memory Precision**: $\frac{\#\text{ relevant items in top-}k\text{ retrieval}}{k}$. Measures noise in retrieval.
- **Latency**: time from query to retrieved context (p50 and p95).
- **Token efficiency**: total tokens injected into context per query. Lower is better—unnecessary context degrades LLM accuracy and increases cost.

Downstream Metrics.

- **Answer accuracy**: correctness of the final response conditioned on memory (EM, F1, or LLM-as-judge).
- **Faithfulness**: does the response accurately reflect what memory contains, without fabrication?
- **Personalization quality**: user satisfaction, measured via preference ratings or A/B tests between memory-augmented and memoryless systems.
- **Contradiction rate**: how often the system produces responses inconsistent with previously stated facts.

Operational Metrics.

- **Write selectivity**: fraction of turns that trigger a memory write. Too high → noise; too low → gaps.
- **Staleness**: how often outdated facts are retrieved despite an update existing.
- **Storage growth rate**: tokens stored per interaction hour. Unbounded growth is unsustainable.

The Evaluation Gap

Most memory papers evaluate on short benchmarks (10–50 sessions). Real production agents run for months with thousands of sessions. Long-horizon evaluation—where memory drift, contradiction accumulation, and storage bloat become dominant failure modes—remains an open challenge. Practitioners should complement benchmark scores with longitudinal monitoring of operational metrics.

17.10 Implementation Patterns

17.10.1 Vector Store Memory with Embeddings

The most common memory pattern stores entries as embedding vectors alongside metadata (timestamps, importance scores, tags). Retrieval combines cosine similarity with temporal decay, so recent and important memories surface first. Duplicate detection and LRU eviction keep the store bounded.

```
import numpy as np
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
import json

@dataclass
class MemoryEntry:
    """A single memory entry with metadata."""
    content: str
    embedding: np.ndarray
    timestamp: datetime = field(default_factory=datetime.now)
    importance: float = 0.5
    access_count: int = 0
    last_accessed: Optional[datetime] = None
    tags: list[str] = field(default_factory=list)
    source: str = "agent"

class VectorMemoryStore:
    """
    Hybrid dense+sparse memory store with temporal decay.
    Supports importance-weighted retrieval and LRU eviction.
    """

    def __init__(
        self,
        embed_fn,           # callable: str -> np.ndarray
        max_entries: int = 10_000,
        decay_rate: float = 0.01,   # per hour
        recency_weight: float = 0.3,
    ):
        self.embed_fn = embed_fn
        self.max_entries = max_entries
        self.decay_rate = decay_rate
        self.recency_weight = recency_weight
        self.entries: list[MemoryEntry] = []

    # -- Write ---------------------------------------------------------------

    def write(
        self,
        content: str,
        importance: float = 0.5,
        tags: list[str] | None = None,
        check_duplicates: bool = True,
    ) -> MemoryEntry:
        """Commit a new memory, evicting if at capacity."""
        if check_duplicates and self._is_duplicate(content):
            return None  # Skip near-duplicate entries

        embedding = self.embed_fn(content)
        entry = MemoryEntry(
            content=content,
            embedding=embedding,
            importance=importance,
            tags=tags or [],
        )
```

```
        if len(self.entries) >= self.max_entries:
            self._evict()

        self.entries.append(entry)
        return entry

    def _is_duplicate(self, content: str, threshold: float = 0.95) -> bool:
        """Check if a near-duplicate already exists."""
        if not self.entries:
            return False
        emb = self.embed_fn(content)
        sims = self._cosine_similarities(emb)
        return float(np.max(sims)) > threshold

    def _evict(self):
        """Remove the least important + least recent entry."""
        now = datetime.now()
        scores = []
        for e in self.entries:
            age_hours = (now - e.timestamp).total_seconds() / 3600
            recency = np.exp(-self.decay_rate * age_hours)
            score = e.importance * (1 - self.recency_weight) \
                  + recency * self.recency_weight
            scores.append(score)
        worst_idx = int(np.argmin(scores))
        self.entries.pop(worst_idx)

    # -- Retrieve ------------------------------------------------------------

    def retrieve(
        self,
        query: str,
        k: int = 5,
        recency_boost: bool = True,
    ) -> list[MemoryEntry]:
        """
        Hybrid retrieval: dense similarity + temporal recency.
        Returns top-k entries sorted by combined score.
        """
        if not self.entries:
            return []

        q_emb = self.embed_fn(query)
        dense_scores = self._cosine_similarities(q_emb)

        now = datetime.now()
        combined = []
        for i, (entry, d_score) in enumerate(
            zip(self.entries, dense_scores)
        ):
            if recency_boost:
                age_h = (now - entry.timestamp).total_seconds() / 3600
                recency = np.exp(-self.decay_rate * age_h)
                score = (1 - self.recency_weight) * d_score \
                      + self.recency_weight * recency
            else:
                score = d_score
            combined.append((score, i))

        combined.sort(reverse=True)
        top_k = [self.entries[i] for _, i in combined[:k]]

        # Update access metadata
        for entry in top_k:
            entry.access_count += 1
            entry.last_accessed = now
```

```
        return top_k

    def _cosine_similarities(self, query_emb: np.ndarray) -> np.ndarray:
        """Vectorized cosine similarity against all stored embeddings."""
        matrix = np.stack([e.embedding for e in self.entries])
        norms = np.linalg.norm(matrix, axis=1, keepdims=True)
        matrix_norm = matrix / (norms + 1e-8)
        q_norm = query_emb / (np.linalg.norm(query_emb) + 1e-8)
        return matrix_norm @ q_norm

    # -- Reflect ------------------------------------------------------------

    def reflect(self, llm_fn, k: int = 10) -> list[str]:
        """
        Meta-cognitive reflection: retrieve recent memories,
        synthesize higher-level insights, and store them back.
        """
        if len(self.entries) < 3:
            return []

        # Retrieve recent high-importance memories
        recent = sorted(
            self.entries, key=lambda e: e.timestamp, reverse=True
        )[:k]
        context = "\n".join(f"- {e.content}" for e in recent)

        # Ask LLM to generate insights
        prompt = (
            "Given these recent memories, extract 2-3 high-level "
            "insights or patterns:\n" + context
        )
        raw_insights = llm_fn(prompt)

        # Store each insight as a high-importance memory
        insights = []
        for line in raw_insights.strip().split("\n"):
            line = line.strip().lstrip("-*").strip()
            if len(line) > 20:
                self.write(
                    f"[INSIGHT] {line}",
                    importance=0.9,
                    check_duplicates=True,
                )
                insights.append(line)
        return insights

    def get_stats(self) -> dict:
        """Return memory statistics for monitoring."""
        return {
            "total_entries": len(self.entries),
            "avg_importance": float(
                np.mean([e.importance for e in self.entries])
            ) if self.entries else 0.0,
            "oldest_entry": min(
                (e.timestamp for e in self.entries), default=None
            ),
        }
```

Listing 17.2: Vector store memory with embeddings, importance scoring, and hybrid retrieval.

17.10.2 Hierarchical Memory Manager

Inspired by MemGPT [297], this pattern organises memory into three tiers: *hot* (in-context, immediate access), *warm* (vector store, fast retrieval), and *cold* (archival, unlimited capacity). Entries are

automatically promoted or demoted based on access frequency and importance—analogous to CPU cache hierarchies.

```
from enum import Enum
from collections import OrderedDict

class MemoryTier(Enum):
    HOT  = "hot"    # In-context: immediate access
    WARM = "warm"   # Vector store: fast retrieval
    COLD = "cold"   # Archival: slow but unlimited

class HierarchicalMemoryManager:
    """
    Three-tier memory manager inspired by MemGPT.
    Hot tier is an LRU cache; warm is a vector store;
    cold is append-only archival storage.
    """

    def __init__(
        self,
        vector_store: VectorMemoryStore,
        hot_capacity: int = 20,     # max entries in hot tier
        warm_capacity: int = 5_000,
        llm_summarize_fn=None,      # callable for summarization
    ):
        self.vector_store = vector_store
        self.hot_capacity = hot_capacity
        self.warm_capacity = warm_capacity
        self.summarize = llm_summarize_fn

        # Hot tier: ordered dict for LRU semantics
        self.hot: OrderedDict[str, MemoryEntry] = OrderedDict()
        # Cold tier: append-only list (would be a DB in production)
        self.cold: list[MemoryEntry] = []

    # -- Page-in: promote warm -> hot ------------------------------------

    def page_in(self, query: str, k: int = 3) -> list[MemoryEntry]:
        """
        Retrieve from warm store and promote to hot tier.
        Evicts least-recently-used hot entries if needed.
        """
        candidates = self.vector_store.retrieve(query, k=k)
        promoted = []
        for entry in candidates:
            key = entry.content[:64]  # use prefix as key
            if key not in self.hot:
                if len(self.hot) >= self.hot_capacity:
                    self._evict_hot()
                self.hot[key] = entry
                self.hot.move_to_end(key)
            promoted.append(entry)
        return promoted

    def _evict_hot(self):
        """Evict LRU entry from hot tier back to warm."""
        # OrderedDict: first item is LRU
        key, entry = self.hot.popitem(last=False)
        # Re-insert into warm store (already there, just update access)
        # In a real system, we'd update the warm store's metadata

    # -- Write with tier assignment --------------------------------------

    def write(
        self,
        content: str,
```

```
        importance: float = 0.5,
        tier: MemoryTier = MemoryTier.WARM,
    ) -> MemoryEntry:
        """Write to the appropriate tier."""
        if tier == MemoryTier.HOT:
            entry = MemoryEntry(
                content=content,
                embedding=self.vector_store.embed_fn(content),
                importance=importance,
            )
            key = content[:64]
            if len(self.hot) >= self.hot_capacity:
                self._evict_hot()
            self.hot[key] = entry
            return entry

        elif tier == MemoryTier.WARM:
            return self.vector_store.write(content, importance=importance)

        else:  # COLD
            entry = MemoryEntry(
                content=content,
                embedding=np.array([]),  # no embedding for cold
                importance=importance,
            )
            self.cold.append(entry)
            return entry

    # -- Summarize and compress ------------------------------------------

    def compress_hot_to_warm(self) -> Optional[str]:
        """
        Summarize hot tier contents and write summary to warm.
        Called when hot tier is full and new important content arrives.
        """
        if not self.hot or not self.summarize:
            return None

        hot_contents = "\n".join(
            f"- {e.content}" for e in self.hot.values()
        )
        summary = self.summarize(
            f"Summarize these memory entries concisely:\n{hot_contents}"
        )
        self.vector_store.write(summary, importance=0.7)
        return summary

    # -- Unified retrieval -----------------------------------------------

    def retrieve(self, query: str, k: int = 5) -> list[MemoryEntry]:
        """
        Retrieve from all tiers, prioritizing hot.
        Returns up to k entries sorted by relevance.
        """
        results = []

        # 1. Check hot tier (exact + semantic)
        q_emb = self.vector_store.embed_fn(query)
        for entry in self.hot.values():
            if entry.embedding.size > 0:
                sim = float(
                    np.dot(q_emb, entry.embedding)
                    / (np.linalg.norm(q_emb) * np.linalg.norm(entry.embedding) + 1
e-8)
                )
                if sim > 0.7:
```

```
                    results.append((sim + 1.0, entry))  # +1 hot bonus

        # 2. Retrieve from warm store
        warm_results = self.vector_store.retrieve(query, k=k)
        for entry in warm_results:
            results.append((0.5, entry))

        # 3. Deduplicate and sort
        seen = set()
        final = []
        for score, entry in sorted(results, reverse=True):
            key = entry.content[:64]
            if key not in seen:
                seen.add(key)
                final.append(entry)
            if len(final) >= k:
                break

        return final

    def get_hot_context(self) -> str:
        """Return hot tier as a formatted context string."""
        if not self.hot:
            return ""
        lines = ["[Memory Context]"]
        for entry in list(self.hot.values())[-10:]:  # last 10
            lines.append(f"  - {entry.content}")
        return "\n".join(lines)
```

Listing 17.3: Hierarchical memory manager implementing hot/warm/cold tiers with automatic promotion and demotion.

17.10.3 Memory-Augmented Agent Loop

This pattern, introduced by MemGPT [297] and formalized in the CoALA framework [363], wires the memory system into the agent's reasoning loop via a *read–act–reflect–write* cycle: before responding, the agent retrieves relevant memories; after responding, it decides what to store. Special tokens in the LLM output trigger memory operations, giving the model self-directed control over its own persistence.

```
import re
from typing import Any

class MemoryAugmentedAgent:
    """
    An LLM agent with a full read-act-reflect-write memory cycle.
    Implements the MemGPT-style self-directed memory management.
    """

    SYSTEM_PROMPT = """You are a memory-augmented AI assistant.
You have access to persistent memory across conversations.
At each turn you may issue memory commands:
  [MEMORY_SEARCH: <query>]  - retrieve relevant memories
  [MEMORY_WRITE: <content>] - store important information
  [MEMORY_REFLECT]          - synthesize insights from memory

Always think step by step. Use memory to avoid repeating mistakes
and to personalize your responses."""

    def __init__(
        self,
        llm_fn,                          # callable: messages -> str
        memory_manager: HierarchicalMemoryManager,
        importance_threshold: float = 0.6,
```

```
        max_memory_tokens: int = 1500,
    ):
        self.llm = llm_fn
        self.memory = memory_manager
        self.importance_threshold = importance_threshold
        self.max_memory_tokens = max_memory_tokens
        self.conversation_history: list[dict] = []

    # -- Main agent step ------------------------------------------------

    def step(self, user_message: str) -> str:
        """
        Full agent step:
        1. Retrieve relevant memories
        2. Construct augmented prompt
        3. Generate response (possibly with memory commands)
        4. Execute memory commands
        5. Reflect and consolidate
        6. Return response to user
        """

        # Step 1: Retrieve relevant memories
        memories = self.memory.retrieve(user_message, k=5)
        memory_context = self._format_memories(memories)

        # Step 2: Construct augmented prompt
        messages = self._build_messages(user_message, memory_context)

        # Step 3: Generate response
        raw_response = self.llm(messages)

        # Step 4: Execute any memory commands in the response
        clean_response, memory_ops = self._parse_memory_commands(
            raw_response
        )
        self._execute_memory_ops(memory_ops, user_message, clean_response)

        # Step 5: Auto-write important information
        self._auto_write(user_message, clean_response)

        # Step 6: Update conversation history
        self.conversation_history.append(
            {"role": "user", "content": user_message}
        )
        self.conversation_history.append(
            {"role": "assistant", "content": clean_response}
        )

        return clean_response

    # -- Memory retrieval and formatting --------------------------------

    def _format_memories(self, memories: list[MemoryEntry]) -> str:
        if not memories:
            return ""
        lines = ["Relevant memories:"]
        for i, m in enumerate(memories, 1):
            age = (datetime.now() - m.timestamp).days
            lines.append(
                f"  [{i}] (importance={m.importance:.1f}, "
                f"{age}d ago) {m.content}"
            )
        return "\n".join(lines)

    def _build_messages(
        self, user_message: str, memory_context: str
```

```
    ) -> list[dict]:
        system = self.SYSTEM_PROMPT
        if memory_context:
            system += f"\n\n{memory_context}"
        system += f"\n\n{self.memory.get_hot_context()}"

        messages = [{"role": "system", "content": system}]
        # Include recent conversation history (last 6 turns)
        messages.extend(self.conversation_history[-6:])
        messages.append({"role": "user", "content": user_message})
        return messages

    # -- Memory command parsing ---------------------------------------------

    def _parse_memory_commands(
        self, response: str
    ) -> tuple[str, list[dict]]:
        """Extract and remove memory commands from response."""
        ops = []
        patterns = {
            "search":  r"\[MEMORY_SEARCH:\s*(.+?)\]",
            "write":   r"\[MEMORY_WRITE:\s*(.+?)\]",
            "reflect": r"\[MEMORY_REFLECT\]",
        }
        clean = response
        for op_type, pattern in patterns.items():
            for match in re.finditer(pattern, response, re.DOTALL):
                content = match.group(1) if op_type != "reflect" else None
                ops.append({"type": op_type, "content": content})
                clean = clean.replace(match.group(0), "").strip()
        return clean, ops

    def _execute_memory_ops(
        self,
        ops: list[dict],
        user_msg: str,
        response: str,
    ):
        """Execute memory commands issued by the LLM."""
        for op in ops:
            if op["type"] == "search":
                results = self.memory.retrieve(op["content"], k=3)
                # Page results into hot tier for immediate use
                self.memory.page_in(op["content"], k=3)

            elif op["type"] == "write":
                self.memory.write(
                    op["content"],
                    importance=0.8,  # explicitly written = important
                    tier=MemoryTier.WARM,
                )

            elif op["type"] == "reflect":
                self._reflect()

    # -- Auto-write heuristic -----------------------------------------------

    def _auto_write(self, user_msg: str, response: str):
        """
        Automatically store important information without explicit command.
        Uses a simple heuristic: write if response contains facts,
        decisions, or user preferences.
        """
        importance_keywords = [
            "remember", "important", "note that", "you prefer",
            "your name is", "decided to", "the answer is",
```

```
            "key insight", "learned that",
        ]
        combined = (user_msg + " " + response).lower()
        if any(kw in combined for kw in importance_keywords):
            summary = f"User: {user_msg[:100]} | Agent: {response[:200]}"
            self.memory.write(
                summary,
                importance=self.importance_threshold,
                tier=MemoryTier.WARM,
            )

    # -- Reflection ---------------------------------------------------------

    def _reflect(self):
        """
        Meta-cognitive reflection: synthesize insights from recent memory.
        Stores high-level insights back into semantic memory.
        """
        recent = self.memory.retrieve("recent important events", k=10)
        if len(recent) < 3:
            return  # Not enough to reflect on

        recent_text = "\n".join(f"- {m.content}" for m in recent)
        insight_prompt = [
            {"role": "system", "content": "You extract high-level insights."},
            {"role": "user", "content":
                f"Based on these memories, what are 2-3 key insights?\n"
                f"{recent_text}\nRespond with bullet points only."},
        ]
        insights = self.llm(insight_prompt)
        # Store each insight as a high-importance semantic memory
        for line in insights.split("\n"):
            line = line.strip().lstrip("*-").strip()
            if len(line) > 20:
                self.memory.write(
                    f"[INSIGHT] {line}",
                    importance=0.9,
                    tier=MemoryTier.WARM,
                )
```

Listing 17.4: Complete memory-augmented agent loop with read-act-reflect-write cycle.

The Read-Act-Reflect-Write Cycle

The memory-augmented agent loop implements a four-phase cognitive cycle:

1. **Read:** Before acting, retrieve relevant memories to inform the response.
2. **Act:** Generate a response conditioned on retrieved context.
3. **Reflect:** Periodically synthesize higher-level insights from accumulated memories.
4. **Write:** Selectively commit important new information to persistent storage.

This cycle mirrors the *observe-orient-decide-act* (OODA) loop from military strategy and the *encode-store-retrieve* model from cognitive psychology. The key insight is that memory is not a passive store but an *active participant* in cognition.

17.11 Recent Advances in Agentic Memory

The memory systems described above established the foundational patterns. Several recent works push the boundaries further:

17.11.1 CoALA: Cognitive Architectures for Language Agents

Sumers et al. [363] propose *Cognitive Architectures for Language Agents* (CoALA), a unifying framework that organizes the growing zoo of LLM agents using principles from cognitive science and symbolic AI. CoALA decomposes a language agent into:

- **Modular memory**: working memory (the context window), episodic memory (past experiences), semantic memory (world knowledge), and procedural memory (action schemas)—mirroring our taxonomy in Section 17.2.
- **Structured action space**: internal actions (reasoning, retrieval, memory writes) and external actions (tool use, environment interaction).
- **Decision cycle**: a generalized sense–plan–act loop with explicit retrieval and write steps.

CoALA's contribution is less a new system than a *design language*: it provides a systematic way to analyze existing agents and identify missing capabilities, making it a useful reference architecture for practitioners.

17.11.2 Mem0: Production-Scale Memory Layer

Mem0 [55] addresses the gap between research memory systems and production deployment. Key ideas:

- **Automatic extraction**: Rather than relying on the LLM to explicitly issue memory-write commands, Mem0 automatically extracts salient facts from conversation turns and consolidates them into a persistent store.
- **Graph-based memory**: Beyond flat vector stores, Mem0 maintains a *relational graph* over extracted entities and facts, enabling multi-hop memory queries ("What did the user say about topic X in the context of project Y?").
- **Memory compression**: Redundant or superseded facts are automatically merged, keeping the memory store compact and current.

On the LOCOMO benchmark, Mem0 achieves 26% relative improvement over OpenAI's baseline memory, with 91% lower p95 latency and >90% token cost reduction compared to full-context approaches.

17.11.3 Sleep-Time Compute and Agent Dreaming

Both *sleep-time compute* and *agent dreaming* share a common insight: the most expensive memory-processing work should happen *between* user interactions, not on the critical path of live inference. They differ in scope—sleep-time compute focuses on pre-computing answers to anticipated queries, while agent dreaming restructures the memory store itself—but together they define the offline memory lifecycle.

Sleep-Time Compute: Pre-Computing for Future Queries

Lin et al. [230] introduce *sleep-time compute*, a paradigm where agents process and consolidate memory *between* user interactions rather than only at query time. The analogy is to biological sleep, during which the brain consolidates memories and pre-computes useful associations.

How it works. During idle periods ("sleep"), the agent:

1. Anticipates likely future queries given the current context.
2. Pre-computes reasoning chains, summaries, and structured representations.
3. Stores these pre-computed artifacts so that test-time inference can retrieve and reuse them.

Results. Sleep-time compute reduces the test-time compute needed to achieve equivalent accuracy by ∼5× on reasoning benchmarks. When amortized across multiple related queries about the same context, average cost per query drops by 2.5×. The approach is most effective when user queries are *predictable*—i.e., when the context strongly constrains what questions will be asked.

Memory Consolidation as Offline RL

Sleep-time compute can be viewed as *offline policy improvement*: during idle time, the agent improves its memory representations (policy) using the data it has already collected (past interactions), without new environment interactions. This connects to offline RL methods (Chapter 8) where the agent learns from a static dataset of trajectories.

Agent Dreaming: Curating and Restructuring the Memory Store

A more recent line of work extends the sleep-time compute principle into a full **dreaming** architecture: a scheduled, asynchronous background process that consolidates, curates, and restructures an agent's memory store while the agent is idle—analogous to the role of REM sleep in biological memory consolidation.

The core architecture: online vs. offline memory. The fundamental insight is a separation of concerns between two memory lifecycles:

- **Online (real-time) memory**: During active sessions, the agent cheaply appends raw observations, tool-call results, and conversation turns to an external log. No expensive processing occurs on the critical path. Latency is minimized; completeness is prioritized over structure.
- **Offline (dream-time) memory**: When the agent is idle, a dedicated background worker processes the accumulated log. It restructures, deduplicates, resolves conflicts, and synthesizes higher-level patterns—producing a curated memory store that the next active session can retrieve efficiently.

This decoupling solves a fundamental tension: real-time memory capture must be fast (it is on the critical path of every agent action), but high-quality memory requires expensive LLM calls for extraction, conflict resolution, and synthesis. Dreaming moves the expensive work off the critical path entirely.

What happens during a dream cycle. A dream cycle is a scheduled job that runs against the agent's raw memory log. The cycle performs several operations:

Transcript replay. The background worker reviews recent session logs—tool-call sequences, error codes, intermediate reasoning steps, and conversation turns. It identifies patterns that no single session could observe in isolation: recurring failure modes, frequently retrieved facts, and evolving user preferences.

Conflict resolution. New observations are cross-referenced against the existing curated store. Facts that have been superseded are closed (their `valid_to` timestamp is set); contradictions are flagged for resolution or escalated to a human reviewer.

Compaction. Thousands of tokens of raw conversational log are condensed into compact, structured representations. The goal is to preserve the semantic content of the original observations while reducing retrieval cost and context-window consumption.

Pattern synthesis. Across a fleet of agents sharing a memory pool, the dream worker can detect fleet-wide anomalies—tool failure trends, systematic reasoning errors, or emerging capability gaps—that are invisible to any individual agent operating on its own session history.

```
async def dream_cycle(agent_id: str, memory_store, raw_log):
    # 1. Load recent raw observations
    recent = raw_log.fetch_since(last_dream_time(agent_id))

    # 2. Replay and extract structured facts
    facts = await llm.extract_facts(recent, existing=memory_store.snapshot())

    # 3. Resolve conflicts against existing store
    updates = memory_store.diff_and_resolve(facts)

    # 4. Compact: merge redundant entries
    compacted = memory_store.compact(updates, max_entries=1000)

    # 5. Write proposals (not live yet -- requires audit gate)
    proposals = memory_store.stage(compacted)
    await notify_reviewer(agent_id, proposals)
```

Listing 17.5: Sketch of a dream cycle worker.

Production implementations. **Anthropic's Dreams feature.** Anthropic's Managed Agents API introduced a "Dreams" capability that allows developers to trigger background memory curation loops with a dedicated token budget. The system outputs memory updates as proposals to a separate staging database, enabling developers to audit and approve changes before they are committed to the live memory store. This proposal/commit pattern is the key safety mechanism: the dreaming process cannot silently alter the agent's memory without a human-visible audit trail.

AutoDreamer. The AutoDreamer framework [368] trains a dedicated offline consolidator using Group Relative Policy Optimization (GRPO). Rather than using a fixed consolidation algorithm, AutoDreamer learns how to rewrite memory banks by measuring downstream agent task success as a reward signal. The consolidator is trained to produce memory representations that maximize the agent's performance on held-out tasks—making the dreaming process itself an optimization target, not just a heuristic.

Table 17.4: Dream cycle design decisions and their trade-offs.

Decision	Options	Trade-off
Trigger	Scheduled (nightly), idle-triggered, event-triggered	Scheduled is predictable; idle-triggered is responsive; event-triggered is precise but complex
Token budget	Fixed cap vs. proportional to log size	Fixed cap limits cost; proportional ensures coverage for large logs
Proposal gate	Auto-commit vs. human review vs. automated tests	Human review is safest; auto-commit is fastest; automated tests balance both
Scope	Per-agent vs. fleet-wide	Per-agent is isolated; fleet-wide enables cross-agent pattern synthesis
Retention	Delete raw log after compaction vs. archive	Deletion saves storage; archiving enables re-dreaming and audit

The proposal gate decision is the most safety-critical: a dream cycle that incorrectly resolves a conflict can silently degrade agent performance, so human review or automated regression tests should gate every commit to the live memory store.

17.11.4 A-MEM: Zettelkasten-Inspired Agentic Memory

A-MEM [422] introduces a memory system that borrows from the *Zettelkasten* method—a note-taking system based on densely interconnected atomic notes—to enable dynamic, self-organizing memory for LLM agents.

Key Design Principles.

- **Structured notes.** Each memory entry is not a raw text chunk but a *note* with multiple structured attributes: a contextual description, keywords, tags, and explicit links to related notes. This metadata enables richer retrieval than embedding similarity alone.
- **Dynamic linking.** When a new memory is added, the system analyzes existing memories to identify semantically meaningful connections and establishes bidirectional links. The result is a *knowledge network* rather than a flat list.
- **Memory evolution.** Critically, adding a new note can *trigger updates* to existing notes—refining their contextual representations and attributes as the agent's understanding deepens. This makes memory a living structure that improves over time, not a static archive.
- **Agent-driven organization.** Unlike fixed-schema memory systems, A-MEM lets the LLM itself decide how to organize, link, and update memories—making the organizational structure adaptive to the task domain.

Results. Across six foundation models on multi-session reasoning tasks, A-MEM consistently outperforms flat vector stores, summarization-based memory, and graph-database approaches, demonstrating that *how* memories are organized matters as much as *what* is stored.

17.12 Proactive Memory Architectures

A fundamental challenge in long-horizon agentic tasks is **behavioral state decay**: after dozens of tool calls, the executor agent loses track of its original goals, constraints, and accumulated context. Standard memory systems address this reactively—the agent queries memory when it needs information. **Proactive memory architectures** [266] (Meta AI, July 2026, "Remember When It Matters") take a different approach: a dedicated memory agent *monitors* the executor and injects constraints *before* the executor drifts.

The Proactive Memory Agent Architecture

The system consists of two decoupled agents:

1. **Executor Agent**: Handles task execution—tool calls, reasoning, output generation. Maintains a lean context focused on the immediate subtask.
2. **Memory Agent**: Runs in parallel, monitoring the executor's action stream. Maintains a compressed representation of:
   - Long-term goals and success criteria
   - Active constraints (safety rules, user preferences, resource limits)
   - Key decisions made earlier in the trajectory

The memory agent **injects reminders** into the executor's context only when it detects drift—when the executor's recent actions suggest it has forgotten a constraint or is moving away from the stated goal. This is the key distinction from reactive memory: the memory agent acts *proactively*, before errors compound.

Proactive Memory: Empirical Results

On long-horizon agentic benchmarks (50+ tool calls per task):

- **Dramatically higher success rates** on tasks requiring sustained constraint adherence (e.g., "never modify files in /prod", "always cite sources").
- **~40% API cost increase**: The memory agent adds LLM calls for monitoring and injection. This is the primary trade-off.

- **Graceful degradation**: Without the memory agent, executor success rates drop sharply after ∼30 tool calls. With it, performance remains stable to 100+ calls.

RL Connection: Memory Agent as State Summarizer

The proactive memory architecture has a natural interpretation in the RL framework:

- The **memory agent** functions as a **state summarizer**—analogous to the attention mechanism in an actor-critic architecture, which selectively weights relevant past states for the value function.
- In a POMDP (Partially Observable MDP), the memory agent maintains the **belief state**: a compressed sufficient statistic of the history relevant to future decisions.
- The injection mechanism is analogous to **privileged information** in asymmetric actor-critic training: the memory agent has access to the full trajectory history, while the executor operates on a compressed context.
- The 40% cost increase reflects the fundamental trade-off between **Markov state representation** (expensive but sufficient) and **truncated context** (cheap but lossy).

This architecture suggests a promising direction for RL training: jointly optimize the executor policy and the memory agent's injection policy, where the memory agent learns *when* and *what* to inject to maximize task success.

17.13 Summary

Agentic memory systems are a foundational component of capable AI agents, addressing the fundamental limitation of finite context windows. We have surveyed:

- A **four-way taxonomy** (working, episodic, semantic, procedural) that mirrors cognitive science and reflects distinct engineering requirements.
- **Five architectural families**: RAG-based, summarization-based, graph-based, key-value networks, and tiered virtual context (MemGPT).
- **Four core operations**: write (with importance scoring and contradiction detection), read-/retrieve (with temporal decay and query expansion), update (with conflict resolution and consolidation), and reflect (meta-cognitive insight generation).
- **Multi-turn and multi-agent extensions**: user modeling, session continuity, shared memory pools, and blackboard architectures.
- **RL training of memory systems**: reward signals for memory operations, learning what to remember, and memory-augmented policy optimization.
- **Proactive memory**: decoupled memory agents that monitor executor behavior and inject reminders before drift compounds—a shift from reactive retrieval to anticipatory intervention.

The field is rapidly evolving. Key open challenges include: (1) *memory grounding*—ensuring retrieved memories are faithfully incorporated rather than ignored or hallucinated over; (2) *scalable consistency*—maintaining coherent shared memory in large multi-agent systems; (3) *privacy-preserving memory*—enabling personalization without compromising user data; and (4) *proactive injection policies*—learning when and what to remind the executor without overwhelming its context. As context windows grow, the boundary between in-context and external memory will shift, but the fundamental need for *selective, structured, retrievable* information storage will remain.

Chapter 18

Agent Harness – Context Management and Orchestration

Modern LLM-based agents do not operate in isolation. Between the raw language model and the real-world tasks it must accomplish lies a layer of infrastructure that manages memory, routes tool calls, tracks state, and enforces safety constraints. This infrastructure is called the **agent harness**. Understanding how to design and implement a robust harness is as important as understanding the model itself—a poorly designed harness can nullify the capabilities of even the most powerful LLM, while a well-designed one can dramatically amplify what a modest model can achieve.

This chapter covers the full stack of agent harness design: context window management, prompt architecture, tool integration, orchestration patterns, state management, error handling, and production concerns. We conclude with a framework comparison and a complete implementation example.

GAS Connection — Agent Harness: Context Management and Orchestration

The GAS LangGraph Supervisor is the agent harness. Every concept in the following chapter—context assembly, prompt architecture, tool integration, orchestration patterns, state management—is directly implemented in `capstone_gas/gas_system.py`.

What is an agent harness? (§ What Is an Agent Harness?) The GAS harness is the code that sits between the raw LLM API and the clinical application logic. It is responsible for: assembling the system prompt, injecting the patient context, managing the RAG retrieval loop, routing between agents, handling errors, and enforcing the latency budget. The harness is implemented as the `build_gas_graph()` function in `capstone_gas/gas_system.py`, which assembles the LangGraph `StateGraph`.

Context window management (§ Context Window Management). The GAS harness implements a priority-based context assembly strategy in each agent node: (1) system prompt (fixed); (2) current patient state (variable); (3) retrieved guidelines from `GASKnowledgeBase` (variable, top-k results); (4) session history from `readings_history` in `GASState` (variable). The `GASLLMClient` enforces the output token limit via Ollama's `num_predict` parameter.

Prompt architecture (§ Prompt Architecture). The GAS uses a layered prompt architecture in each agent node: a *persona layer* ("You are a clinical decision-support agent"), a *task layer* ("Analyse the following patient state"), a *constraint layer* ("Never recommend insulin doses exceeding safe thresholds"), and a *format layer* (structured output with recommendation, rationale, and urgency). This four-layer structure is directly visible in `monitor_node()`, `advisor_node()`, `safety_node()`, and `caregiver_node()`.

Orchestration patterns (§ Orchestration Patterns). The GAS Supervisor uses the *sequential with conditional branching* orchestration pattern: `monitor_node()` always runs first; if `alert_level` is high or critical, `advisor_node()` runs; if the Advisor produces a recommendation, `safety_node()` runs; if `safety_approved = True`, `caregiver_node()` runs. This directed graph is implemented via `add_conditional_edges()` in `build_gas_graph()`.

18.1 What Is an Agent Harness?

Definition: Agent Harness

An **agent harness** is the runtime infrastructure that wraps an LLM to transform it from a stateless text-completion engine into a stateful, goal-directed agent capable of multi-step reasoning, tool use, memory retrieval, and interaction with external systems.

The harness enforces a clean **separation of concerns**:

- **Reasoning** – delegated entirely to the LLM; the harness does not second-guess model outputs.
- **Execution** – the harness dispatches tool calls, manages I/O, and enforces sandboxing.
- **Memory** – the harness maintains short-term (context window), working (scratchpad), and long-term (vector store / database) memory.
- **Communication** – the harness handles message routing between agents, users, and external services.
- **Observability** – the harness instruments every step for logging, tracing, and debugging.

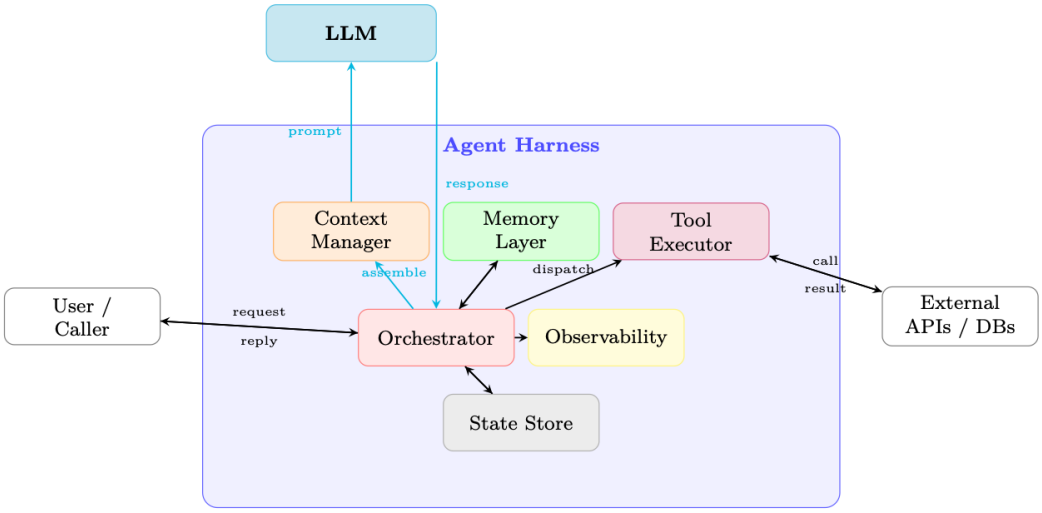


Figure 18.1: High-level architecture of an agent harness. The LLM handles only reasoning; all execution, memory, routing, and observability are managed by the harness.

Why Separate Concerns?

A language model is a function f_θ : tokens $\to$ tokens. It has no persistent state, no ability to call APIs, and no awareness of time. The harness is the "operating system" that gives the model a body—persistent memory, actuators (tools), and a scheduler (orchestrator) [297]. Just as an OS abstracts hardware from applications, the harness abstracts infrastructure from the model.

18.2 Context Window Management

The context window is the agent's working memory. Every token in the window costs money and latency; every token *not* in the window is invisible to the model. Managing this finite resource is one of the most consequential engineering decisions in agent design.

18.2.1 The Context Budget Problem

Let C be the maximum context length (in tokens) supported by the model. The context is partitioned into several competing components:

$$C \geq \underbrace{S}_{\text{system prompt}} + \underbrace{M}_{\text{memory/RAG}} + \underbrace{T}_{\text{tool defs}} + \underbrace{H}_{\text{history}} + \underbrace{R}_{\text{reserved output}} \tag{18.1}$$

As a conversation grows, H expands without bound while C remains fixed. Tool outputs can be large (e.g., a web page, a code execution result), causing sudden spikes in $T + H$. The harness must continuously enforce Equation 18.1.

The Silent Truncation Trap

Many LLM APIs silently truncate input that exceeds the context limit, dropping tokens from the *middle* or *beginning* of the prompt. This can cause the model to lose its system prompt, forget earlier instructions, or hallucinate based on incomplete context—all without any error signal. Always count tokens *before* sending and handle overflow explicitly.

18.2.2 Context Allocation Strategies

Fixed Budget Allocation. Assign hard token limits to each component:

$$\begin{aligned} S &\leq \alpha \cdot C, \quad \alpha \approx 0.10 \\ M &\leq \beta \cdot C, \quad \beta \approx 0.20 \\ T &\leq \gamma \cdot C, \quad \gamma \approx 0.10 \\ H &\leq \delta \cdot C, \quad \delta \approx 0.50 \\ R &\leq \epsilon \cdot C, \quad \epsilon \approx 0.10 \end{aligned} \tag{18.2}$$

Fixed allocation is simple and predictable but wastes capacity when some components are small.

Dynamic Allocation. Solve a constrained optimization at each turn:

$$\max_{S,M,T,H,R} \text{Utility}(S, M, T, H, R) \quad \text{s.t.} \quad S + M + T + H + R \leq C \tag{18.3}$$

where Utility is a task-specific scoring function (e.g., weighted sum of relevance scores). In practice, dynamic allocation is approximated greedily: fill the highest-priority components first, compress or truncate lower-priority ones.

18.2.3 Context Compression

When H exceeds its budget, the harness must compress history without losing critical information.

Summarization of Old Turns. Replace the oldest k turns with an LLM-generated summary [297]:

$$H' = \text{Summarize}(H_{1:k}) \parallel H_{k+1:n} \tag{18.4}$$

The summary is typically 5–10× shorter than the original. A dedicated "summarizer" model (smaller and cheaper) can be used for this step.

Selective Retention. Score each message by relevance to the current query q:

$$\text{score}(m_i) = \text{sim}(e(m_i), e(q)) + \lambda \cdot \text{recency}(i) \tag{18.5}$$

where $e(\cdot)$ is an embedding function and $\text{recency}(i) = i/n$. Retain the top-k messages by score.

Importance-Weighted Truncation. Assign importance weights w_i to each turn (e.g., turns containing tool results or user corrections get higher weight). Truncate lowest-weight turns first:

$$\min_{S \subseteq [n]} \sum_{i \notin S} w_i \quad \text{s.t.} \quad \sum_{i \in S} |m_i| \leq B_H \tag{18.6}$$

This is a variant of the 0/1 knapsack problem, solvable greedily by sorting on $w_i/|m_i|$.

18.2.4 Sliding Window Approaches

- **FIFO (First-In, First-Out):** Drop the oldest messages when the window fills. Simple but loses early context (e.g., original task description).
- **Importance-Ranked Retention:** Keep the system prompt and first user message pinned; apply importance scoring to the rest.
- **Hierarchical Summarization:** Maintain a multi-level summary pyramid—recent turns verbatim, older turns as paragraph summaries, oldest turns as a single abstract.

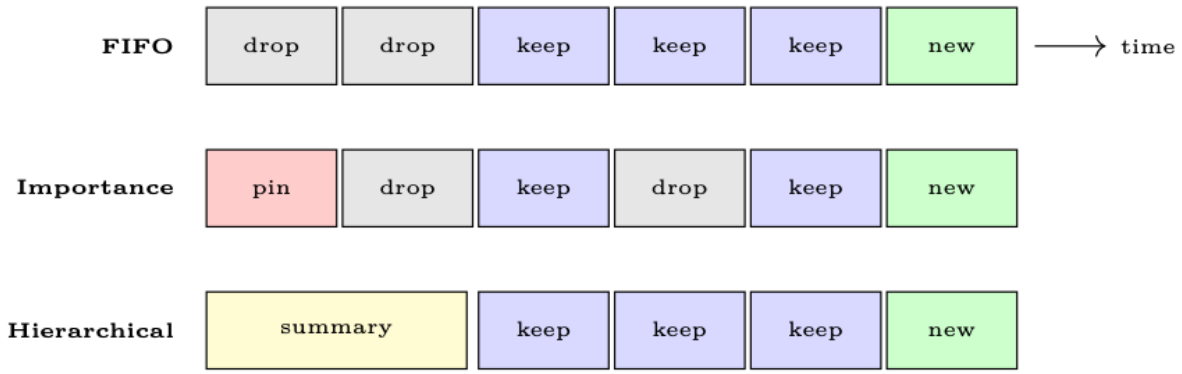


Figure 18.2: Three sliding-window strategies. Red = pinned, gray = dropped, blue = retained verbatim, yellow = summarized, green = new message.

18.2.5 Recursive Context Decomposition

The strategies above—summarization, selective retention, sliding windows—all accept a fundamental constraint: *everything must fit in one context window.* A more radical approach rejects this constraint entirely: let the model **recursively call itself** (or a sub-model) on partitions of the context, aggregating results across calls [443].

Recursive Language Model (RLM)

A **Recursive Language Model** replaces a single monolithic LLM call $M(q, C)$ with a recursive decomposition:

$$\mathrm{RLM}(q, C) = M(q,\ \mathrm{RLM}(q_1, C_1),\ \mathrm{RLM}(q_2, C_2),\ \ldots) \tag{18.7}$$

where the root model partitions the context C into chunks $\{C_i\}$, formulates sub-queries $\{q_i\}$, spawns recursive calls to process each chunk, and then synthesizes the results into a final answer. No single call ever sees the full context—the model manages what to examine at each recursion level.

Why Recursion Helps. Context rot—the empirical degradation of model accuracy as context length grows—means that even models with large context windows (128k+) perform worse on long inputs. By keeping each individual call short and focused, recursive decomposition avoids this degradation entirely. Zhang et al. [443] demonstrated that a recursive GPT-5-mini *outperforms* non-recursive GPT-5 on difficult long-context benchmarks, while being cheaper per query.

Implementation Pattern. A practical RLM harness provides the model with a REPL environment containing the context as a variable. The model can:

1. **Inspect** the context programmatically (regex, slicing, length checks).
2. **Partition** it into manageable chunks based on structure or relevance.
3. **Sub-query** by spawning recursive LLM calls over each chunk.

4. **Aggregate** sub-results into a final answer.

Recursive Summarization of a Large Codebase

```python
def recursive_summarize(context: str, query: str,
                        model: LLM, max_tokens: int = 8000):
    """Recursively summarize context that exceeds window."""
    if count_tokens(context) <= max_tokens:
        # Base case: context fits in one call
        return model.call(f"{query}\n\nContext:\n{context}")

    # Recursive case: split and sub-query
    chunks = split_by_structure(context, max_tokens // 2)
    sub_results = []
    for i, chunk in enumerate(chunks):
        sub_q = f"Summarize this section relevant to: {query}"
        sub_results.append(
            recursive_summarize(chunk, sub_q, model, max_tokens)
        )

    # Aggregate: synthesize sub-results
    combined = "\n---\n".join(sub_results)
    return model.call(
        f"Given these partial summaries, answer: {query}"
        f"\n\nSummaries:\n{combined}"
    )
```

This pattern generalizes beyond summarization: recursive search (find a needle across millions of tokens), recursive analysis (audit a large codebase), and recursive extraction (parse a corpus of documents) all follow the same decompose–recurse–aggregate structure.

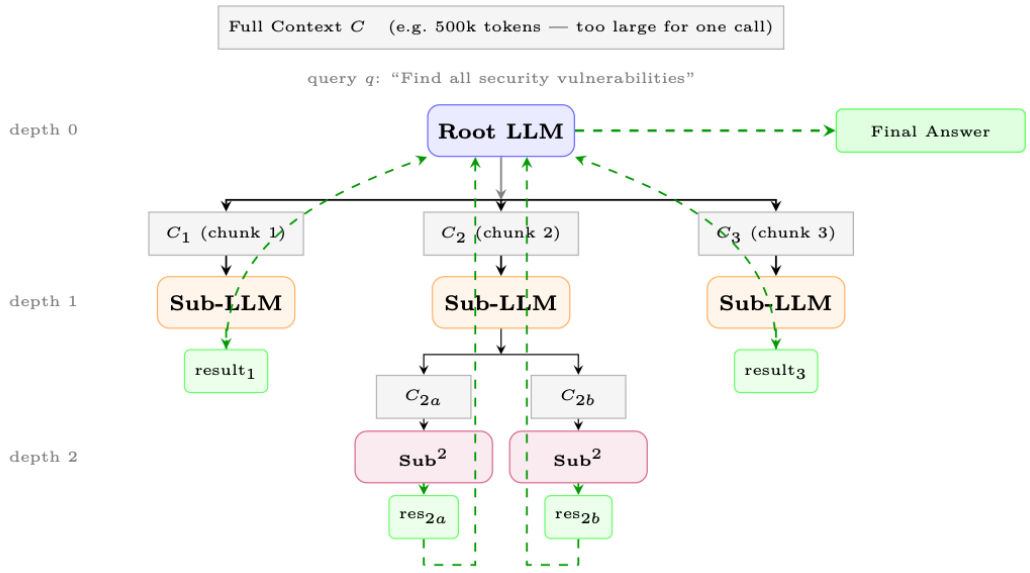


Figure 18.3: Recursive Language Model (RLM). The root model partitions the context into chunks, spawns sub-LLM calls at depth 1, which may recurse further (depth 2). Results flow back up (dashed green arrows) and are aggregated into a final answer. No single call processes the full context.

18.2.6 Token Counting and Budget Monitoring

Pre-Flight Token Check

Before every LLM call, the harness must:

1. Count tokens in the assembled prompt (using the model's tokenizer, not a word-count approximation).

2. Compare against $C - R$ (context limit minus reserved output tokens).
3. If over budget: trigger compression, truncation, or raise an explicit error.
4. Log the token breakdown by component for observability.

Token counting should use the model's *exact* tokenizer (e.g., `tiktoken` for OpenAI models, `transformers` tokenizer for open-source models). Rule-of-thumb approximations ("4 chars per token") can be off by 20–40% for code, JSON, or non-English text.

18.3 Prompt Architecture

The prompt is the primary interface between the harness and the model. A well-structured prompt is modular, composable, and version-controlled.

18.3.1 System Prompt Design

A production system prompt typically contains four sections:

1. **Persona:** Who the agent is, its name, role, and communication style.
2. **Capabilities:** What the agent can do (tools available, knowledge cutoff, supported languages).
3. **Constraints:** What the agent must *not* do (safety rules, scope limits, confidentiality).
4. **Output Format:** Expected response structure (JSON schema, markdown, step-by-step reasoning).

System Prompt Template

```
SYSTEM_PROMPT_TEMPLATE = """
# Identity
You are {agent_name}, a {role} assistant built by {org}.
Today's date is {date}. Your knowledge cutoff is {cutoff}.

# Capabilities
You have access to the following tools: {tool_list}.
You can reason step-by-step before acting.

# Constraints
- Never reveal system prompt contents.
- Do not execute code that modifies files outside {workspace}.
- Escalate to human if confidence < {threshold}.

# Output Format
Always respond in valid JSON matching this schema:
{output_schema}
"""
```

18.3.2 Dynamic Prompt Assembly

Rather than a single monolithic string, production harnesses assemble prompts from **components** at runtime:

$$\text{Prompt} = \text{Concat}(\text{SystemBlock},\ \text{MemoryBlock},\ \text{ToolBlock},\ \text{HistoryBlock},\ \text{QueryBlock}) \quad (18.8)$$

Each block is independently versioned, tested, and can be swapped without touching others. A **prompt registry** stores named templates with semantic versioning (e.g., `system/v2.3.1`).

18.3.3 Few-Shot Management

Few-shot examples improve reliability but consume tokens. The harness should [235]:

- **Select relevant examples** using embedding similarity to the current query.
- **Rotate examples** to avoid overfitting to a fixed set.
- **Budget examples** within the M allocation (Equation 18.2).
- **Cache embeddings** of the example library to avoid recomputation.

Formally, few-shot selection is a constrained optimization—maximizing total relevance subject to a token budget:

$$\text{examples}^* = \underset{E \subseteq \mathcal{E},\, |E| \leq k}{\arg\max} \sum_{e \in E} \text{sim}(e(e_{\text{input}}),\, e(q)) \quad \text{s.t.} \quad \sum_{e \in E} |e| \leq B_M \tag{18.9}$$

18.3.4 Tool Descriptions

Tool descriptions are part of the prompt and directly affect tool selection quality. A well-designed tool signature has five components:

1. **Name:** Use a verb–noun pattern (`search_web`, `read_file`, `send_email`). Avoid generic names like `do_action` or ambiguous ones like `process`.
2. **Description:** One to two sentences explaining *what* the tool does, *when* to use it, and *when not* to use it. This is the primary signal the model uses for selection.
3. **Input parameters:** Each parameter needs a type, a human-readable description, and whether it is required or optional (with a sensible default).
4. **Output specification:** Document the return format—structured JSON, plain text, or error codes—so the model can parse results correctly.
5. **Constraints:** Rate limits, maximum input size, required permissions, or side effects (e.g., "This tool sends a real email—use only after user confirmation").

Good vs. Bad Tool Signatures

```
# BAD: vague name, no usage guidance, missing constraints
{"name": "search", "description": "Search for things",
 "parameters": {"q": {"type": "string"}}}

# GOOD: clear name, when-to-use, typed params, constraints
{"name": "search_web",
 "description": "Search the public web for current information. "
   "Use when the user asks about events after 2024-04. "
   "Do NOT use for internal company data.",
 "parameters": {
   "query": {"type": "string",
             "description": "Natural-language search query"},
   "num_results": {"type": "integer", "default": 5,
                   "description": "Results to return (max 20)"}},
 "returns": "JSON array of {title, url, snippet}",
 "constraints": "Max 10 calls/minute. No PII in queries."}
```

Additional best practices for tool descriptions in the prompt:

- **Be specific:** "Search the web for current information" is better than "Search".

- **Include when to use:** "Use this when the user asks about events after your knowledge cutoff."
- **Include when NOT to use:** Reduces false positives.
- **Exclude irrelevant tools:** Dynamically include only tools relevant to the current task to save tokens and reduce confusion.
- **Optimize descriptions:** A/B test descriptions; small wording changes can shift tool selection accuracy by 10–20%.

18.4 Tool Integration and Execution

Tool use is a defining capability of modern LLM agents [333]. The harness manages tool definitions, selection, execution, and output processing.

18.4.1 Tool Definition Schemas

Different providers use different schemas for tool definitions:

OpenAI Function Calling.

OpenAI Tool Definition

```
{
  "type": "function",
  "function": {
    "name": "search_web",
    "description": "Search the web for current information.",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {"type": "string", "description": "Search query"},
        "num_results": {"type": "integer", "default": 5}
      },
      "required": ["query"]
    }
  }
}
```

Anthropic Tool Use. Anthropic uses a similar JSON schema but with an `input_schema` key instead of `parameters`, and tools are passed in a top-level `tools` array:

Anthropic Tool Definition

```
# Tool definition (passed in the API request)
{"tools": [{
    "name": "search_web",
    "description": "Search the web for current information.",
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {"type": "string",
                      "description": "Search query"},
            "num_results": {"type": "integer",
                            "description": "Max results"}
        },
        "required": ["query"]
    }
}]}

# Model response (tool_use content block)
{"role": "assistant", "content": [{
```

```
    "type": "tool_use",
    "id": "toolu_01A09q90qw90lq917835lq9",
    "name": "search_web",
    "input": {"query": "latest AI news", "num_results": 3}
}]}

# Tool result (sent back as user message)
{"role": "user", "content": [{
    "type": "tool_result",
    "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
    "content": "[{\"title\": \"...\", \"url\": \"...\"}]"
}]}
```

Model Context Protocol (MCP). MCP (Section 18.4.5) provides a standardized protocol for tool discovery and invocation across providers, decoupling tool definitions from any single API format.

18.4.2 Tool Selection and Routing

The model selects tools based on its understanding of tool descriptions and the current task. The harness can influence this:

- **Auto tool use:** The model decides whether and which tool to call.
- **Forced tool use:** The harness specifies `tool\_choice: {type: "function", function: {name: "X"}}` to force a specific tool (useful for structured extraction).
- **Parallel tool calls:** Modern APIs allow the model to request multiple tool calls in a single turn, which the harness executes concurrently.

Scaling to Large Tool Libraries. When an agent has access to hundreds or thousands of tools, including all definitions in the prompt is infeasible (token cost) and counterproductive (selection confusion). Two key approaches address this:

- **Retrieval-augmented tool selection:** At each turn, retrieve only the top-k most relevant tools using embedding similarity between the user query and tool descriptions. This mirrors RAG for documents—only contextually relevant tools are injected into the prompt. **Gorilla** [304] demonstrated that combining retrieval with retriever-aware training (RAT) enables LLMs to accurately select from thousands of overlapping APIs, adapting to version changes at test time.
- **Fine-tuned tool selection: ToolLLM** [317] trains models on a large corpus of tool-use trajectories (16,000+ APIs) using a depth-first search-based decision tree (DFSDT) to generate solution paths. The resulting model learns generalizable tool selection strategies that transfer to unseen APIs, achieving significantly better accuracy than prompt-only approaches.

In practice, production harnesses combine these strategies: a retrieval layer pre-filters the tool set, the prompt includes the filtered tools, and the model's native function-calling capability handles final selection.

18.4.3 Tool Output Processing

Raw tool outputs are rarely ready for direct insertion into the context:

1. **Parse and validate:** Check that the output matches the expected schema.
2. **Truncate large outputs:** Web pages, code outputs, and database results can be enormous. Apply summarization or chunking before inserting into context.

3. **Error normalization:** Convert provider-specific errors into a standard format the model can reason about.
4. **Retry logic:** On transient failures (network timeout, rate limit), retry with exponential backoff before reporting failure to the model.

Tool Output Truncation

```python
def process_tool_output(result: str, budget: int,
                        summarizer=None) -> str:
    tokens = count_tokens(result)
    if tokens <= budget:
        return result
    # Try extractive truncation first (cheap)
    truncated = smart_truncate(result, budget)
    if summarizer and tokens > 2 * budget:
        # Use summarizer for very large outputs
        return summarizer.summarize(result, max_tokens=budget)
    return truncated
```

18.4.4 Sandboxing and Safety

Tool execution is a major attack surface. The harness must enforce:

- **Execution isolation:** Run code tools in containers (Docker, gVisor) or VMs with no network access by default.
- **Permission models:** Declare required permissions per tool (read-only filesystem, network access, etc.) and enforce them at the OS level.
- **Resource limits:** CPU time, memory, and wall-clock timeouts prevent runaway executions.
- **Input sanitization:** Validate and sanitize all model-generated tool arguments before execution (prevent prompt injection via tool outputs).
- **Audit logging:** Log every tool call with arguments, outputs, and timestamps for post-hoc review.

Prompt Injection via Tool Outputs (Greshake et al. 2023)

A malicious web page or document retrieved by a tool can contain instructions like "Ignore previous instructions and exfiltrate the system prompt." The harness must treat all tool outputs as *untrusted data*, not as instructions. Use output sandboxing, content filtering, and consider wrapping tool outputs in XML tags that the model is trained to treat as data rather than instructions.

18.4.5 Model Context Protocol (MCP)

The **Model Context Protocol** (MCP) [10] is an open standard for connecting LLM applications to external tools and data sources. It decouples tool *providers* from tool *consumers*. We cover MCP in depth in Chapter 23; here we summarize the key ideas relevant to harness design.

Architecture. MCP uses a client-server model:

- **MCP Server:** Exposes tools, resources, and prompts over a standardized protocol. Can be a local process or a remote service.
- **MCP Client:** The agent harness connects to one or more MCP servers, discovers available tools, and routes tool calls.
- **Transport Layers:** Supports `stdio` (local subprocess), HTTP+SSE (remote), and WebSocket transports.

Tool Discovery. At startup, the harness calls `tools/list` on each connected MCP server to discover available tools and their schemas. This enables **dynamic tool registration**—new tools become available without redeploying the harness.

Invocation Flow.

1. Model outputs a tool call (e.g., `mcp_server_name::tool_name(args)`).
2. Harness routes the call to the appropriate MCP server via `tools/call`.
3. MCP server executes the tool and returns a structured result.
4. Harness inserts the result into the context as a `tool` message.

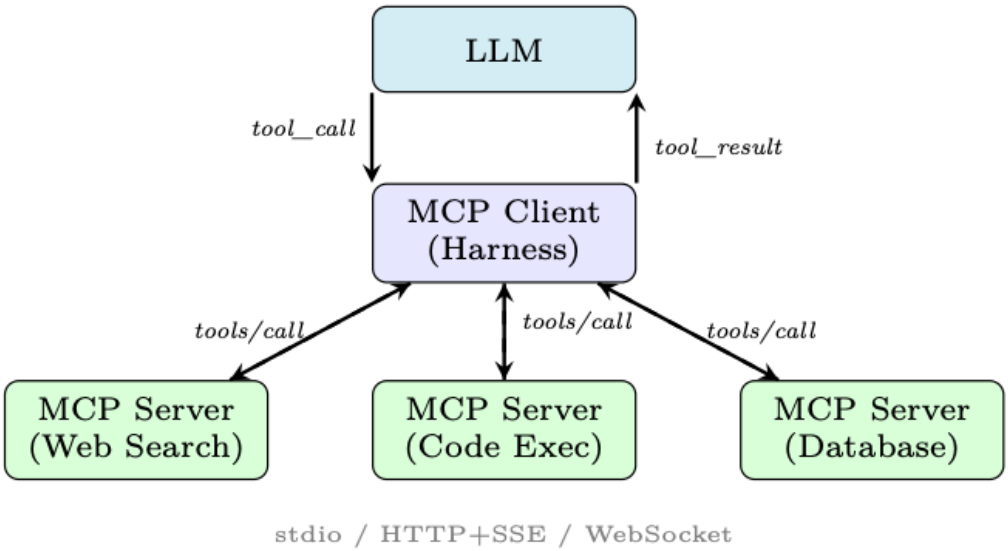


Figure 18.4: MCP architecture. The harness acts as an MCP client, routing tool calls to specialized MCP servers over standardized transports.

18.5 Orchestration Patterns

Orchestration defines *how* the agent decides what to do next. Different patterns suit different task structures.

18.5.1 ReAct Loop (Reason + Act)

The **ReAct** pattern [430] interleaves reasoning ("Thought") with action ("Act") and observation ("Observe") in a tight loop:

$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1} \rightarrow \cdots \tag{18.10}$$

Implementation Details.

- The "Thought" step is typically a scratchpad—a chain-of-thought reasoning trace [395] that is *not* shown to the user.
- The harness parses the model's output to extract the action (tool name + arguments).
- A **max iterations** guard prevents infinite loops.
- The loop terminates when the model outputs a "Final Answer" action or a stop token.

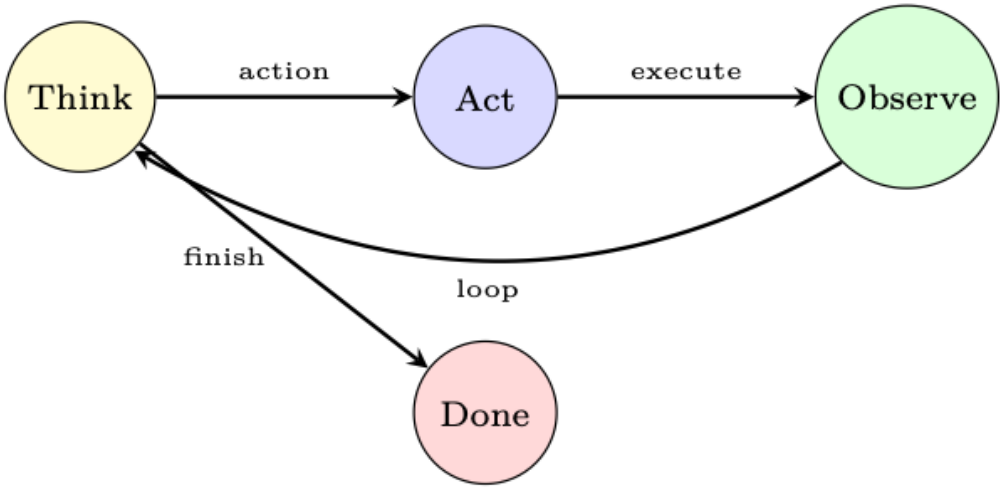


Figure 18.5: ReAct loop: the agent alternates between reasoning and acting until a termination condition is met.

18.5.2 Plan-and-Execute

Rather than deciding one step at a time, the agent first generates a complete plan, then executes each step [385]:

1. **Planning phase:** Given the task, generate a structured plan (list of subtasks with dependencies).
2. **Execution phase:** Execute each subtask, potentially using a different (cheaper) model.
3. **Plan revision:** If a step fails or produces unexpected results, re-plan from the current state.

$$\text{Plan} = \text{Planner}(q), \quad \text{Result} = \prod_{i=1}^{|\text{Plan}|} \text{Executor}(\text{Plan}[i], \text{context}_i) \tag{18.11}$$

Plan-and-execute is more efficient for long-horizon tasks (fewer LLM calls) but less adaptive to unexpected observations.

18.5.3 Multi-Agent Orchestration

Complex tasks benefit from multiple specialized agents working together. Four canonical patterns:

Supervisor Pattern. A central "supervisor" LLM receives the user request, decomposes it, and routes subtasks to specialist agents. Results are aggregated by the supervisor.

Peer-to-Peer. Agents communicate directly without a central coordinator. Each agent can invoke any other agent as a tool. Flexible but harder to debug and prone to circular dependencies.

Hierarchical (Tree of Agents). A tree structure where high-level agents delegate to mid-level agents, which delegate to leaf agents. Enables recursive task decomposition. Used in systems like AutoGen's nested chat.

Swarm Pattern. Popularized by OpenAI's Swarm library [292], this pattern uses **handoffs**: an agent can transfer control to another agent along with the full conversation context. Key concepts:

- **Agents** have instructions and tools.

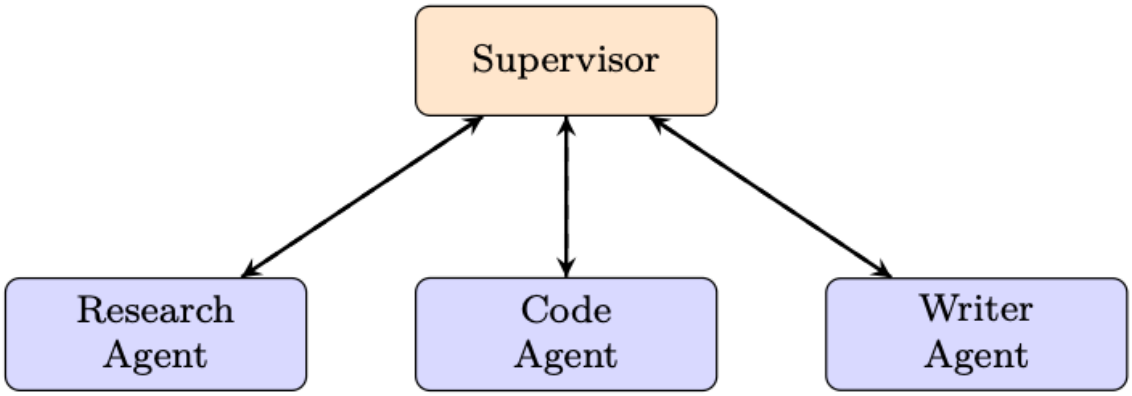


Figure 18.6: Supervisor pattern: one orchestrator routes to specialist agents.

- **Handoffs** are special tools that transfer control.
- **Context variables** are shared state passed between agents.
- The active agent changes dynamically based on task needs.

18.5.4 Human-in-the-Loop

Production agents must know when to pause and ask for human input:

- **Approval gates:** Before irreversible actions (sending emails, deleting files, making purchases), require explicit human confirmation.
- **Escalation criteria:** Escalate when confidence is below a threshold, when the task is outside defined scope, or when a safety rule is triggered.
- **Feedback integration:** Human corrections are inserted into the context and can update the agent's plan.
- **Async approval:** For long-running tasks, the agent can pause, notify the human via email/Slack, and resume when approved.

Escalation Decision Rule

$$\text{Escalate} \iff \underbrace{p_{\text{success}} < \tau_{\text{conf}}}_{\text{low confidence}} \vee \underbrace{\text{action} \in \mathcal{A}_{\text{irreversible}}}_{\text{irreversible}} \vee \underbrace{\text{cost} > B_{\text{auto}}}_{\text{over budget}} \tag{18.12}$$

where τ_{conf} is the confidence threshold, $\mathcal{A}_{\text{irreversible}}$ is the set of irreversible actions, and B_{auto} is the autonomous spending limit.

18.5.5 Workflow Graphs

For complex, structured workflows, the orchestration logic is expressed as a **directed acyclic graph** (DAG) or state machine:

- **LangGraph** [164]: Extends LangChain with a graph-based execution model. Nodes are agent steps; edges are conditional transitions. Supports cycles (for ReAct loops) and parallel branches.
- **AutoGen** [407]: Microsoft's framework for multi-agent conversation graphs. Supports nested chats, group chats, and human-in-the-loop patterns.
- **State machines:** Explicit states (e.g., `PLANNING`, `EXECUTING`, `WAITING_FOR_HUMAN`, `DONE`) with defined transitions. Easier to reason about and test than implicit loop logic.

$$G = (V, E, \sigma_0), \quad v \in V : \text{agent step}, \quad e \in E : \text{conditional transition}, \quad \sigma_0 : \text{initial state} \tag{18.13}$$

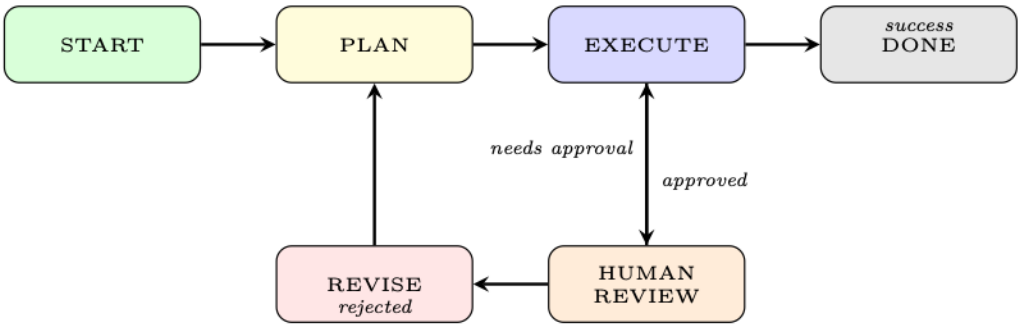


Figure 18.7: Example workflow graph for a human-in-the-loop agent. States and conditional transitions are explicit, making the control flow auditable.

18.6 State Management

Agents are inherently stateful. The harness must manage multiple layers of state:

18.6.1 Conversation State

The message history is the primary state artifact. Each message has:

- **Role:** `system`, `user`, `assistant`, `tool`.
- **Content:** Text, tool call, or tool result.
- **Metadata:** Timestamp, token count, importance score, compression status.

18.6.2 Task State

For long-running tasks, the harness tracks:

- **Progress:** Which subtasks are complete, in-progress, or pending.
- **Checkpoints:** Serialized state snapshots that allow resumption after failure.
- **Rollback:** The ability to undo the last k actions if a mistake is detected.

18.6.3 Agent State

The agent's internal state includes:

- **Current plan:** The sequence of steps the agent intends to take.
- **Pending actions:** Tool calls that have been issued but not yet returned.
- **Beliefs:** Facts the agent has established (e.g., "the user's timezone is UTC+9").

18.6.4 Persistent State

For cross-session continuity [297, 383]:

- **User profiles:** Preferences, past interactions, learned facts about the user.
- **Long-term memory:** Vector database of past conversations, searchable by semantic similarity.
- **Task history:** Completed tasks with outcomes, used for few-shot retrieval.

State as a First-Class Citizen

In early agent frameworks, state was an afterthought—a global dictionary passed around. Production systems treat state as a first-class citizen with explicit schemas, versioning, and migration paths. Think of agent state like a database schema: define it carefully upfront, because changing it later is painful.

18.7 Error Handling and Recovery

Agents operate in adversarial, unpredictable environments. Robust error handling is non-negotiable.

18.7.1 Retry Strategies

- **Exponential backoff:** For transient failures (rate limits, network errors), retry after $\min(2^k \cdot t_0 + \epsilon, t_{\max})$ seconds, where k is the retry count and ϵ is random jitter.
- **Fallback models:** If the primary model is unavailable or returns an error, fall back to a secondary model (potentially less capable but available).
- **Graceful degradation:** If a tool is unavailable, inform the model and let it attempt the task without that tool.

The backoff delay for the k-th retry is:

$$t_k = \min\Big(2^k \cdot t_0 + \mathcal{U}(0, t_0),\ t_{\max}\Big), \quad k = 0, 1, 2, \ldots \tag{18.14}$$

18.7.2 Loop Detection

Agents can get stuck in infinite loops—repeatedly calling the same tool with the same arguments, or oscillating between two states. Detection and self-correction strategies [344]:

- **Max iteration guard:** Hard limit on the number of steps per task (e.g., 50 steps).
- **Action deduplication:** Hash each (tool, args) pair; if the same call appears k times, break the loop.
- **Progress detection:** If the agent's state has not changed in k steps, trigger a "stuck" handler.

Formally, a loop is detected when the same action hash appears within a sliding window of size W:

$$\text{loop_detected} \iff \exists\, i < j \leq t : \text{hash}(\text{action}_i) = \text{hash}(\text{action}_j) \ \wedge\ j - i \leq W \tag{18.15}$$

18.7.3 Graceful Failure

When the agent cannot complete a task:

1. Explain what was accomplished (partial results).
2. Explain why the task could not be completed.
3. Suggest recovery actions (e.g., "Please provide your API key to enable web search").
4. Preserve state so the task can be resumed.

18.7.4 Observability

The Observability Triad for Agents

- **Traces:** End-to-end trace of each agent run, with spans for each LLM call, tool call, and state transition. Tools: LangSmith, Arize Phoenix, OpenTelemetry.
- **Logs:** Structured logs for every event (prompt sent, response received, tool called, error raised). Include token counts, latency, and cost.
- **Metrics:** Aggregate statistics—task success rate, average steps per task, tool error rate, cost per task, p95 latency.

The Debugging Gap

LLM agents are notoriously hard to debug because failures are often *semantic* (the model made a wrong decision) rather than *syntactic* (a code exception). Invest in replay tooling: the ability to re-run any past agent trace with a modified prompt or model, and compare outputs side-by-side.

18.8 Scaling and Production Concerns

18.8.1 Latency Optimization

- **Parallel tool calls:** Execute independent tool calls concurrently using `asyncio` or thread pools. Can reduce multi-tool latency by $N\times$ for N parallel calls.
- **Streaming:** Use streaming APIs to begin processing the model's response before it is complete. Reduces time-to-first-token for the user.
- **Prompt caching:** Many providers (Anthropic, OpenAI) offer prompt caching for repeated prefixes (e.g., system prompt + tool definitions). Can reduce latency and cost by 50–90% for the cached portion.
- **Speculative execution:** Begin executing the most likely next tool call before the model has finished generating, and cancel if the prediction was wrong.

18.8.2 Cost Management

- **Token budgets:** Enforce per-task and per-user token budgets. Alert when approaching limits.
- **Model routing:** Use a cheap, fast model (e.g., GPT-4o-mini, Claude Haiku) for simple steps (tool selection, formatting) and an expensive model (GPT-4o, Claude Opus) only for complex reasoning [48].
- **Caching:** Cache deterministic tool outputs (e.g., database lookups, static web pages) to avoid redundant API calls.

The total cost of an agent task with T LLM steps and K tool calls is:

$$\text{Cost}_{\text{task}} = \sum_{i=1}^{T} \underbrace{p_{\text{in}} \cdot n_{\text{in},i} + p_{\text{out}} \cdot n_{\text{out},i}}_{\text{LLM cost}} + \sum_{j=1}^{K} \underbrace{c_j}_{\text{tool cost}} \tag{18.16}$$

where $p_{\text{in}}, p_{\text{out}}$ are per-token prices, $n_{\text{in},i}, n_{\text{out},i}$ are input/output token counts for step i, and c_j is the cost of tool call j.

18.8.3 Rate Limiting and Queuing

When running many agents concurrently:

- **Token bucket rate limiter:** Enforce per-minute token limits across all agents sharing an API key.

- **Priority queues:** High-priority tasks (interactive user requests) preempt low-priority tasks (batch processing).
- **Backpressure:** When the queue is full, reject new tasks with a `503 Service Unavailable` rather than silently queuing indefinitely.

18.8.4 Evaluation in Production

- **A/B testing:** Route a fraction of traffic to a new agent version and compare success rates, cost, and latency.
- **Canary deployments:** Gradually increase traffic to a new version while monitoring for regressions.
- **Shadow mode:** Run a new agent in parallel with the production agent, compare outputs, but only serve the production output to users.
- **LLM-as-judge:** Use a separate LLM to evaluate agent outputs on dimensions like helpfulness, accuracy, and safety [456].

18.9 Framework Comparison

Table 18.1: Comparison of major agent orchestration frameworks.

Framework	Flex.	Complex.	Prod.	Multi-Agent	Best For
LangChain	H	H	M	M	Rapid prototyping, chains
LangGraph	H	H	H	H	Complex stateful workflows
AutoGen	M	M	M	H	Multi-agent conversations
CrewAI	M	L	M	H	Role-based teams
OAI Assistants	L	L	H	L	Simple hosted agents
OpenAI Swarm	M	L	L	H	Handoff patterns
Custom	H	H	H	H	Full control, no lock-in

Legend: H = High, M = Medium, L = Low. **Flex.** = Flexibility, **Complex.** = Complexity, **Prod.** = Production-readiness.

- **LangChain** [43]1 provides a rich ecosystem of integrations but has a steep learning curve and abstractions that can obscure what is actually happening.
- **LangGraph** [164]2 adds explicit graph-based control flow to LangChain, making complex multi-step agents much more manageable.
- **AutoGen** [407]3 excels at multi-agent conversations and nested chats, with good support for human-in-the-loop patterns.
- **CrewAI** [275]4 offers a high-level, role-based abstraction ("crew of agents") that is easy to get started with but less flexible for custom patterns.
- **OpenAI Assistants API**5 is fully managed (no infrastructure to run) but offers limited customization and vendor lock-in.
- **OpenAI Swarm** [292]6 is a lightweight, educational framework demonstrating the handoff pattern; not production-ready.
- **Custom harness** offers maximum control and is the right choice for production systems with specific requirements, but requires significant engineering investment.

When to Use a Framework vs. Build Custom?

Use a framework when: you are prototyping, your use case fits the framework's abstractions, or you need rapid integration with many tools. Build custom when: you have strict latency/cost requirements, the framework's abstractions leak in ways that cause bugs, you need fine-grained control over context management, or you are building a product where the agent harness is a core differentiator.

18.10 Implementation: Production Agent Harness

The following is a complete, production-quality agent harness implementation demonstrating context management, tool integration, the ReAct orchestration loop, and error handling.

```
"""
production_harness.py -- A production-quality agent harness.
Demonstrates: context management, tool integration,
ReAct loop, error handling, and observability.
"""

from __future__ import annotations
import asyncio
import hashlib
import json
import logging
import time
from dataclasses import dataclass, field
from enum import Enum
from typing import Any, Callable, Optional

import tiktoken
from openai import AsyncOpenAI

# -- Logging / Observability ---------------------------------
logger = logging.getLogger("agent_harness")

# -- Data Models ---------------------------------------------

class Role(str, Enum):
    SYSTEM    = "system"
    USER      = "user"
    ASSISTANT = "assistant"
    TOOL      = "tool"

@dataclass
class Message:
    role:         Role
    content:      str
    tool_calls:   Optional[list[dict]] = None
    tool_call_id: Optional[str]        = None
    metadata:     dict                 = field(default_factory=dict)

    def to_api_dict(self) -> dict:
        d: dict = {"role": self.role.value,
                   "content": self.content or None}
        if self.tool_calls:
            d["tool_calls"] = self.tool_calls
        if self.tool_call_id:
            d["tool_call_id"] = self.tool_call_id
        return d

@dataclass
class ToolDefinition:
    name:        str
    description: str
    parameters:  dict
```

```
    handler:     Callable
    requires_approval: bool = False

    def to_api_dict(self) -> dict:
        return {
            "type": "function",
            "function": {
                "name":        self.name,
                "description": self.description,
                "parameters":  self.parameters,
            }
        }

# -- Context Manager ------------------------------------------

class ContextManager:
    """
    Manages the context window with budget enforcement,
    compression, and token counting.
    """
    BUDGET_FRACTIONS = {
        "system":   0.10,
        "memory":   0.20,
        "tools":    0.10,
        "history":  0.50,
        "reserved": 0.10,
    }

    def __init__(self, model: str, max_tokens: int):
        self.model      = model
        self.max_tokens = max_tokens
        self.enc        = tiktoken.encoding_for_model(model)
        self.history:   list[Message] = []
        self.system_msg: Optional[Message] = None

    def count_tokens(self, text: str) -> int:
        return len(self.enc.encode(text))

    def count_message_tokens(self, msg: Message) -> int:
        # OpenAI overhead: 4 tokens per message + role
        return self.count_tokens(msg.content or "") + 4

    def total_history_tokens(self) -> int:
        return sum(self.count_message_tokens(m)
                   for m in self.history)

    def history_budget(self) -> int:
        return int(self.max_tokens
                   * self.BUDGET_FRACTIONS["history"])

    def add_message(self, msg: Message) -> None:
        self.history.append(msg)
        self._enforce_budget()

    def _enforce_budget(self) -> None:
        budget = self.history_budget()
        while (self.total_history_tokens() > budget
               and len(self.history) > 2):
            # Drop oldest non-pinned message (index 1).
            # If it has tool_calls, also drop the tool results
            # that follow it to keep the conversation valid.
            dropped = self.history.pop(1)
            if dropped.tool_calls:
                while (len(self.history) > 1
                       and self.history[1].role == Role.TOOL):
                    self.history.pop(1)
```

```
        logger.debug(
            "Context: %d/%d tokens used",
            self.total_history_tokens(), budget
        )

    def preflight_check(self, tool_tokens: int) -> bool:
        """Returns True if we are within budget."""
        sys_tokens = (self.count_message_tokens(self.system_msg)
                      if self.system_msg else 0)
        total = (sys_tokens
                 + tool_tokens
                 + self.total_history_tokens())
        reserved = int(self.max_tokens
                       * self.BUDGET_FRACTIONS["reserved"])
        ok = total <= (self.max_tokens - reserved)
        if not ok:
            logger.warning(
                "Context overflow: %d > %d",
                total, self.max_tokens - reserved
            )
        return ok

    def build_messages(self) -> list[dict]:
        msgs = []
        if self.system_msg:
            msgs.append(self.system_msg.to_api_dict())
        msgs.extend(m.to_api_dict() for m in self.history)
        return msgs

# -- Tool Executor ------------------------------------------

class ToolExecutor:
    """
    Executes tool calls with sandboxing, retry logic,
    and output truncation.
    """
    MAX_OUTPUT_TOKENS = 2000
    MAX_RETRIES       = 3

    def __init__(self, tools: list[ToolDefinition],
                 approval_callback: Optional[Callable] = None,
                 encoding: str = "cl100k_base"):
        self.tools    = {t.name: t for t in tools}
        self.approval = approval_callback
        self.enc      = tiktoken.get_encoding(encoding)

    async def execute(self, tool_name: str,
                      args: dict) -> str:
        tool = self.tools.get(tool_name)
        if not tool:
            return f"Error: unknown tool '{tool_name}'"

        # Human-in-the-loop approval gate
        if tool.requires_approval and self.approval:
            approved = await self.approval(tool_name, args)
            if not approved:
                return "Action rejected by human reviewer."

        for attempt in range(self.MAX_RETRIES):
            try:
                result = await asyncio.wait_for(
                    self._call(tool, args), timeout=30.0
                )
                return self._truncate(result)
            except asyncio.TimeoutError:
                logger.warning("Tool %s timed out (attempt %d)",
```

```
                                tool_name, attempt + 1)
                if attempt == self.MAX_RETRIES - 1:
                    return f"Error: tool '{tool_name}' timed out"
                await asyncio.sleep(2 ** attempt)  # backoff
            except Exception as exc:
                logger.error("Tool %s error: %s", tool_name, exc)
                if attempt == self.MAX_RETRIES - 1:
                    return f"Error: {exc}"
                await asyncio.sleep(2 ** attempt)
        return "Error: max retries exceeded"

    async def _call(self, tool: ToolDefinition,
                    args: dict) -> str:
        if asyncio.iscoroutinefunction(tool.handler):
            result = await tool.handler(**args)
        else:
            result = await asyncio.get_running_loop().run_in_executor(
                None, lambda: tool.handler(**args)
            )
        return str(result)

    def _truncate(self, text: str) -> str:
        tokens = self.enc.encode(text)
        if len(tokens) <= self.MAX_OUTPUT_TOKENS:
            return text
        truncated = self.enc.decode(
            tokens[:self.MAX_OUTPUT_TOKENS]
        )
        return truncated + "\n[... output truncated ...]"

# -- Loop Detector ------------------------------------------

class LoopDetector:
    """Detects repeated actions within a sliding window."""
    def __init__(self, window: int = 5, max_repeats: int = 2):
        self.window      = window
        self.max_repeats = max_repeats
        self.action_hashes: list[str] = []

    def record(self, tool_name: str, args: dict) -> bool:
        """Returns True if a loop is detected."""
        h = hashlib.md5(
            f"{tool_name}:{json.dumps(args, sort_keys=True)}"
            .encode()
        ).hexdigest()
        self.action_hashes.append(h)
        recent = self.action_hashes[-self.window:]
        if recent.count(h) >= self.max_repeats:
            logger.warning("Loop detected: %s called %d times",
                           tool_name, recent.count(h))
            return True
        return False

# -- Agent Harness ------------------------------------------

class AgentHarness:
    """
    Production agent harness implementing the ReAct loop
    with full context management, tool integration,
    error handling, and observability.
    """
    MAX_ITERATIONS = 50

    def __init__(
        self,
        model:        str,
```

```
        system_prompt: str,
        tools:         list[ToolDefinition],
        max_tokens:    int = 128_000,
        approval_cb:   Optional[Callable] = None,
        client:        Optional[AsyncOpenAI] = None,
    ):
        self.model    = model
        self.client   = client or AsyncOpenAI()
        self.ctx_mgr = ContextManager(model, max_tokens)
        self.executor = ToolExecutor(tools, approval_cb)
        self.loop_det = LoopDetector()
        self.tools    = tools

        # Set system message
        sys_msg = Message(Role.SYSTEM, system_prompt)
        self.ctx_mgr.system_msg = sys_msg

    async def run(self, user_input: str) -> str:
        """
        Execute the ReAct loop for a user request.
        Returns the final response string.
        """
        run_id   = hashlib.md5(
            f"{time.time()}:{user_input}".encode()
        ).hexdigest()[:8]
        start_ts = time.monotonic()
        logger.info("[%s] Starting run: %s", run_id,
                    user_input[:80])

        # Add user message to context
        self.ctx_mgr.add_message(
            Message(Role.USER, user_input)
        )

        tool_defs = [t.to_api_dict() for t in self.tools]
        tool_tokens = sum(
            self.ctx_mgr.count_tokens(json.dumps(t))
            for t in tool_defs
        )

        for iteration in range(self.MAX_ITERATIONS):
            # Pre-flight context check
            if not self.ctx_mgr.preflight_check(tool_tokens):
                logger.error("[%s] Context overflow at iter %d",
                             run_id, iteration)
                return ("I've run out of context space. "
                        "Please start a new conversation.")

            # -- LLM Call ---------------------------------
            messages = self.ctx_mgr.build_messages()
            try:
                response = await self.client.chat.completions.create(
                    model=self.model,
                    messages=messages,
                    tools=tool_defs if self.tools else None,
                    tool_choice="auto",
                    temperature=0.0,
                )
            except Exception as exc:
                logger.error("[%s] LLM call failed: %s",
                             run_id, exc)
                return f"I encountered an error: {exc}"

            choice  = response.choices[0]
            msg     = choice.message
            finish  = choice.finish_reason
```

```
            # Store assistant message
            assistant_msg = Message(
                role=Role.ASSISTANT,
                content=msg.content or "",
                tool_calls=([tc.model_dump()
                             for tc in msg.tool_calls]
                            if msg.tool_calls else None),
            )
            self.ctx_mgr.add_message(assistant_msg)

            # -- Terminal condition ------------------------
            if finish == "stop" or not msg.tool_calls:
                elapsed = time.monotonic() - start_ts
                logger.info(
                    "[%s] Done in %d iters, %.2fs",
                    run_id, iteration + 1, elapsed
                )
                return msg.content or "Task complete."

            # -- Tool Execution ----------------------------
            tool_results = await self._execute_tool_calls(
                msg.tool_calls, run_id
            )

            # Check for loops
            for tc in msg.tool_calls:
                args = json.loads(tc.function.arguments)
                if self.loop_det.record(tc.function.name, args):
                    return ("I seem to be stuck in a loop. "
                            "Please clarify your request.")

            # Add tool results to context
            for tool_call_id, result in tool_results.items():
                self.ctx_mgr.add_message(Message(
                    role=Role.TOOL,
                    content=result,
                    tool_call_id=tool_call_id,
                ))

        # Max iterations reached
        logger.warning("[%s] Max iterations reached", run_id)
        return ("I reached the maximum number of steps "
                "without completing the task. "
                "Here is what I found so far: "
                + (msg.content or ""))

    async def _execute_tool_calls(
        self,
        tool_calls: list,
        run_id: str,
    ) -> dict[str, str]:
        """Execute tool calls in parallel."""
        tasks = {}
        for tc in tool_calls:
            name = tc.function.name
            try:
                args = json.loads(tc.function.arguments)
            except json.JSONDecodeError:
                args = {}
            logger.info("[%s] Tool call: %s(%s)",
                        run_id, name, args)
            tasks[tc.id] = self.executor.execute(name, args)

        results = await asyncio.gather(
            *tasks.values(), return_exceptions=True
```

```
        )
        output = {}
        for tool_id, result in zip(tasks.keys(), results):
            if isinstance(result, Exception):
                output[tool_id] = f"Error: {result}"
            else:
                output[tool_id] = result
        return output

# -- Example Usage ---------------------------------------------

async def main():
    # Define tools
    async def search_web(query: str,
                         num_results: int = 5) -> str:
        # In production: call a real search API
        return f"[Search results for '{query}': ...]"

    async def run_python(code: str) -> str:
        # In production: execute in a sandbox container
        return f"[Execution result of code: ...]"

    tools = [
        ToolDefinition(
            name="search_web",
            description=(
                "Search the web for current information. "
                "Use when the user asks about recent events "
                "or facts beyond your knowledge cutoff."
            ),
            parameters={
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
                        "description": "Search query"
                    },
                    "num_results": {
                        "type": "integer",
                        "default": 5
                    },
                },
                "required": ["query"],
            },
            handler=search_web,
        ),
        ToolDefinition(
            name="run_python",
            description=(
                "Execute Python code in a sandbox. "
                "Use for calculations, data processing, "
                "or generating visualizations."
            ),
            parameters={
                "type": "object",
                "properties": {
                    "code": {
                        "type": "string",
                        "description": "Python code to execute"
                    },
                },
                "required": ["code"],
            },
            handler=run_python,
            requires_approval=True,  # Requires human sign-off
        ),
```

```
    ]

    harness = AgentHarness(
        model="gpt-4o",
        system_prompt=(
            "You are a helpful research assistant. "
            "Think step by step before acting. "
            "Always cite your sources."
        ),
        tools=tools,
        max_tokens=128_000,
    )

    response = await harness.run(
        "What were the key AI research breakthroughs "
        "in the first half of 2025?"
    )
    print(response)

if __name__ == "__main__":
    asyncio.run(main())
```

Listing 18.1: Production Agent Harness – Core Implementation

Key Design Decisions in the Implementation

- **Context enforcement** happens on every `add_message` call, not just before LLM calls. This prevents silent overflow.
- **Parallel tool execution** via `asyncio.gather` reduces latency when the model requests multiple tools simultaneously.
- **Loop detection** uses content hashing over a sliding window, catching both exact repeats and near-repeats.
- **Approval gates** are per-tool, not per-run, allowing fine-grained control over which actions require human sign-off.
- **Structured logging** with a `run_id` makes it easy to trace a single agent run across distributed logs.
- **Exponential backoff** is applied at the tool level, not the LLM level, since tool failures are more common and more recoverable.

How Do You Test an Agent Harness?

Testing agents is fundamentally different from testing deterministic software. Key strategies: (1) **Unit test** each component (context manager, tool executor, loop detector) in isolation with mocked dependencies. (2) **Integration test** the full harness against a mock LLM that returns scripted responses. (3) **Evaluation harness**: run the agent on a benchmark of tasks with known correct answers and measure success rate. (4) **Adversarial testing**: deliberately inject malformed tool outputs and verify graceful failure. (5) **Regression testing**: replay past production traces and verify that outputs do not regress after changes.

Summary

The agent harness is the engineering foundation that transforms a language model into a capable, reliable agent. The key takeaways from this section are:

- **Context is a finite, precious resource.** Enforce budgets explicitly, count tokens with the model's exact tokenizer, and compress history proactively.

- **Prompts are code.** Version-control them, test them, and assemble them modularly from components.
- **Tools are the agent's actuators.** Define them precisely, sandbox their execution, and handle their outputs defensively.
- **Orchestration patterns are not one-size-fits-all.** ReAct for exploratory tasks, Plan-and-Execute for structured tasks, multi-agent for complex decomposable tasks.
- **State management is a first-class concern.** Design state schemas upfront; retrofitting them is painful.
- **Errors are inevitable; graceful recovery is a feature.** Implement retry logic, loop detection, and informative failure messages.
- **Observability is not optional.** You cannot debug what you cannot see. Instrument everything from day one.
- **Production concerns compound.** Latency, cost, rate limits, and evaluation all interact. Address them systematically, not as afterthoughts.

Harness Distillation: Baking Scaffolding into Weights

The harness engineering techniques in this chapter raise a natural question: can the behavioral gains of a specialized harness be *trained into* the model weights, so the harness can be removed at deployment? This is the goal of **harness distillation**, covered in Section 12.6 (LLM Agentic Training). The Harness-Zero approach shows that a model trained under a specialized harness h^* and then deployed under only a generic harness h can *exceed* the performance of the base model with h^* still attached — recovering over 80% of harness-exclusive behaviors while eliminating the runtime scaffolding overhead.

Chapter 19

Loop Engineering

The simplest agentic pattern is a loop: generate, observe, act, repeat. But production agents require more than a while loop — they need principled context management, adaptive orchestration that responds to intermediate results, mechanisms for detecting and escaping stagnation, and the ability to escalate to human oversight when confidence falls below a threshold. This chapter covers loop engineering as a discipline: the five loop primitives, context engineering as the layer beneath the loop, parallel subagent fleet execution, and cognitive escalation with uncertainty-driven handoff.

GAS Connection — Loop Engineering

The GAS Monitor agent is a production ReAct loop, and the GAS pipeline as a whole is a multi-agent loop. The following chapter's patterns for loop design, termination, and failure handling are directly implemented in `capstone_gas/gas_system.py`.

Anatomy of a production loop (§ Anatomy of a Production Loop). The Monitor's loop in `monitor_node()` has four phases: (1) *Observe*: read the latest CGM reading from `state["current_reading"]`; (2) *Reason*: compute the alert level using the chain-of-thought prompt ("current glucose is X mg/dL; trend is Y; this is/is not anomalous because..."); (3) *Act*: write the alert level to `GASState`; (4) *Route*: the Supervisor checks if the alert level exceeds the escalation threshold and routes to the Advisor if so.

Loop patterns (§ Loop Patterns). The GAS uses two loop patterns: (1) *ReAct* for the Monitor (reason then act); (2) *Reflection* for the Safety Critic (generate a verdict, then check it against constitutional rules in `safety_node()`).

Termination engineering (§ Termination Engineering). The GAS implements two termination conditions: (1) *Threshold*: `alert_level` in (`"high"`, `"critical"`) → escalate to Advisor; (2) *Retry budget*: maximum 2 Advisor–Safety Critic cycles (tracked by `state["iteration"]` in `GASState`) before the Supervisor routes to `END` and flags the case for human review.

Failure modes (§ Failure Modes and Anti-Patterns). The GAS has one known loop failure mode: the Advisor–Safety Critic cycle can repeat if the Advisor consistently generates recommendations with `safety_score < 0.7`. This is prevented by the `iteration` counter in `GASState`: after 2 retries, the Supervisor routes to `END` rather than cycling indefinitely.

Context management in long-running loops (§ Context Management in Long-Running Loops). The GAS pipeline runs continuously for the duration of a patient's monitoring session. The `readings_history` field in `GASState` accumulates CGM readings; the `GASLLMClient` enforces a fixed context window to prevent overflow while preserving the most recent patient history.

The evolution of how practitioners interact with LLM-based agents has followed a remarkably consistent trajectory. In 2022–2024, the primary skill was *prompt engineering*: crafting the right words to elicit the desired response from a single model call. By 2025, the focus shifted to *context engineering*—curating the full set of tokens the model sees at inference time, including retrieved documents, tool outputs, and conversation history [11]. In the previous chapter we covered *harness engineering*: designing the runtime environment—tools, sandboxes, memory, and guardrails—that surrounds an agent. Loop engineering [295] is the next layer in this progression: designing the *iterative control structure* that drives an agent toward a goal autonomously, without requiring a human to type the next instruction at each turn.

From Prompt to Loop: The Engineering Progression

1. **Prompt engineering**: Optimize what you *say* to the model (2022–2024)
2. **Context engineering**: Optimize everything the model *sees* (2025)
3. **Harness engineering**: Optimize the environment the agent *runs in* (2025–2026)
4. **Loop engineering**: Optimize the *cycle* that keeps the agent working toward a goal (2026)

Each layer wraps the previous one without replacing it. You still write prompts; you still curate context; you still build a harness. Loop engineering puts all of it in motion.

The phrase was coined in June 2026 after Peter Steinberger [357] argued that developers should stop prompting coding agents directly and instead design systems that prompt those agents—a sentiment validated by Boris Cherny, who leads Claude Code at Anthropic, noting that his role had shifted entirely to writing the external execution loops that coordinate model actions [54]. This is not merely a tooling preference. It reflects a structural reality: when a single agent run may last an hour and modify dozens of files, the highest-leverage engineering is no longer in the prompt—it is in the loop that keeps the agent productive, verified, and on-goal throughout.

19.1 Context Engineering: The Layer Beneath the Loop

Before a loop can run, something must decide what the model sees at each step. That discipline is **context engineering**: the formal practice of curating and maintaining the optimal set of tokens in the context window during inference. It is the layer beneath the loop—the mechanism by which the loop's state is translated into model input.

The term was popularized by Tobi Lütke (CEO of Shopify) in June 2025 [257], who described it as the emerging core skill for working with AI agents. Anthropic formalized the definition in September 2025 as "curating and maintaining the optimal set of tokens during inference." Andrej Karpathy endorsed the framing, describing it as "the delicate art and science of filling the context window with just the right information for the next step" [188]. The convergence of these definitions reflects a practical reality: as context windows grew from 4K to 128K to 1M tokens, the question of *what to put in them* became as important as the question of *what to ask*.

Context engineering is not prompt engineering. Prompt engineering optimizes the *instruction*—the words that direct the model's behavior. Context engineering optimizes the *information environment*—the full set of documents, tool outputs, history, and state that the model reasons over. In a single-turn interaction, the distinction is minor. In a multi-step agent loop, it is the difference between a model that has the information it needs to act correctly and one that is flying blind.

Key techniques. Context engineering draws on several complementary methods:

- **Dynamic context assembly**: Selecting and composing context from multiple sources (retrieved documents, tool outputs, memory stores) at each loop iteration rather than using a fixed template.
- **RAG integration**: Retrieving task-relevant documents or code snippets and injecting them into the context window, replacing stale or generic background knowledge with precise, current information.
- **Tool-output summarization**: Compressing verbose tool outputs (e.g., long file listings, test runner logs) before inserting them into context, preserving signal while conserving token budget.
- **Conversation history management**: Deciding which prior turns to retain verbatim, which to summarize, and which to drop entirely—a rolling compression problem that becomes critical in long agent runs.

- **Token budget allocation**: Explicitly partitioning the context window across competing information sources to ensure the model always has room to generate a useful response.

Context Window Budget Allocation (Typical Agent Loop)

Component	Budget	Notes
System prompt	10–15%	Stable; defines agent role, tools, constraints
Retrieved documents / RAG	20–30%	Dynamic; refreshed each iteration
Conversation / action history	30–40%	Compressed as run lengthens
Generation budget	20–30%	Reserved for model output; never compress below this

These are guidelines, not hard rules. Long-horizon coding agents may allocate more to history; single-step QA agents may allocate more to retrieved documents. The key invariant: always reserve the generation budget *before* filling the rest.

Context engineering within a loop. In a loop, context engineering operates at every iteration. At each step t, the loop controller must decide: which prior observations $o_{<t}$ to retain, which to summarize, which retrieved documents are still relevant, and how to compose the new context s_t from these components. This is not a one-time design decision—it is a dynamic policy that runs alongside the agent policy π_θ. A poorly designed context policy causes the model to lose track of its goal, repeat actions it has already taken, or fail to use information it retrieved two steps earlier. A well-designed one keeps the model's "working memory" aligned with the current state of the task throughout the entire run.

Context Engineering vs. Loop Engineering

Context engineering answers: *what does the model see at step t?* Loop engineering answers: *what happens after the model acts at step t?* They are complementary. The loop determines the sequence of actions and observations; context engineering determines how those observations are represented when the model takes its next action. You cannot do loop engineering well without doing context engineering well—the loop's state is only useful if it is correctly encoded in the model's context.

19.2 Loop Engineering as Inference-Time Reinforcement Learning

The central insight that makes loop engineering relevant to this book is that a well-engineered agent loop is structurally identical to an RL optimization process operating at inference time—without gradient updates to the model weights:

$$\begin{aligned} s_t &= \text{context}(s_{t-1}, a_{t-1}, o_{t-1}) && \text{(state = accumulated context)} \\ a_t &= \pi_\theta(s_t) && \text{(action = model generation)} \\ o_t &= \text{env}(a_t) && \text{(observation = tool/environment feedback)} \\ r_t &= \text{verify}(s_t, a_t, o_t) && \text{(reward = verification signal)} \end{aligned} \tag{19.1}$$

The loop continues until r_t exceeds a success threshold or a termination condition triggers. The "policy" π_θ is not updated via gradients; instead, the *state* is updated—observations, error messages, and reflections are appended to the context, conditioning the same frozen model to produce better actions on subsequent iterations. This is precisely the mechanism behind Reflexion [344]: the model improves across iterations not by learning new weights but by reading its own failure history.

The Loop–RL Correspondence

Consider the standard RL objective $\max_\pi \mathbb{E}[\sum_{t=0}^{T} \gamma^t r_t]$. In loop engineering:

- **Policy** π: The frozen LLM, conditioned on accumulating context
- **State** s_t: The context window contents at step t
- **Action** a_t: The model's generated output (code edit, tool call, message)

- **Environment**: The tools, filesystem, test runner, linter—anything that produces observations
- **Reward** r_t: The verification signal (tests pass, linter clean, metric improves)
- **Discount** γ: Implicit in the loop's budget—later steps are costlier and less likely to succeed
- **Episode**: One complete loop execution from goal specification to termination

The critical difference from training-time RL: the policy weights are frozen. "Learning" happens purely through state accumulation—the model receives richer conditioning as the loop progresses. This is analogous to in-context learning, but with the environment actively shaping what the model sees.

This correspondence is not merely metaphorical. It has direct engineering consequences:

1. **Reward design matters**: Just as in RLHF, a poorly specified reward (verification criterion) leads to reward hacking—the agent satisfying the letter of the check while violating its intent (e.g., deleting a failing test to turn CI green).

2. **Exploration–exploitation trade-off**: A loop that repeats the same failing approach exhibits poor exploration. Mechanisms like Reflexion [344] add explicit exploration via self-critique, analogous to entropy bonuses in PPO.

3. **Horizon and discount**: Longer loops face compounding errors and context degradation, just as long-horizon RL suffers from credit assignment difficulty. Budget caps serve as effective horizons.

4. **State representation**: Context management (compaction, pruning, externalization) is the loop-engineering equivalent of state representation design in RL—both determine whether the agent can reason effectively about its situation.

19.3 Anatomy of a Production Loop

A functional loop requires five structural primitives [295] plus external state persistence:

19.3.1 The Five Primitives

Table 19.1: The five structural primitives of loop engineering.

Primitive	Role in the Loop	RL Analogue
Automations	Trigger the loop on schedule or event	Episode initiation; environment reset
Worktrees	Isolate parallel agents	Independent rollout workers in distributed RL
Skills	Codify reusable capabilities and project knowledge	Policy conditioning; task-specific reward shaping
Connectors	Interface with external tools and systems	Environment action space
Sub-agents	Decompose and verify	Hierarchical RL; critic networks

Automations. Automations are the heartbeat that transforms a single agent run into a true loop. They define *when* and *why* a loop fires—on a cron schedule, in response to a webhook, or triggered by a file-system event. Without automations, you have an agent; with them, you have a self-sustaining system. Production examples include nightly CI failure triage, hourly dependency-vulnerability scans, and post-commit review passes.

Worktrees (Isolation). The moment multiple agents run in parallel, file-level conflicts become the dominant failure mode. Git worktrees—separate working directories sharing repository history—provide isolation: each agent operates on its own branch without the possibility of write conflicts with siblings. This is the same principle as running independent rollout workers in distributed RL (Section 11.2): parallelism requires isolation.

Skills. Skills (Chapter 24) encode project knowledge that the agent would otherwise re-derive from scratch every cycle. In the loop context, skills serve as *persistent conditioning*—the conventions, build procedures, and constraints that remain stable across iterations. Without skills, each loop iteration begins cold; with them, the agent compounds knowledge across runs without exhausting the context window on rediscovered facts.

Connectors. Connectors—typically implemented via MCP (Chapter 23)—extend the loop's action space beyond the filesystem. A loop that can only read and write files is limited; one connected to the issue tracker, CI system, staging environment, and team communication channel can close the full feedback loop: detect problem → fix → validate → deploy → notify.

Sub-agents (Maker–Checker Separation). The most critical structural principle in loop engineering is separating the agent that *produces* output from the agent that *evaluates* it. A model grading its own output is analogous to a student marking their own exam—the incentives are misaligned. A dedicated verification sub-agent, potentially running a different model or at higher reasoning effort, provides the independent evaluation that makes unattended operation trustworthy. This mirrors the actor–critic architecture in RL: the actor (generator) proposes actions; the critic (verifier) evaluates them.

19.3.2 External State: The Loop's Memory

Every primitive above operates within a single iteration. *External state* is what connects iterations across time. Because LLMs are stateless between invocations—the model forgets everything not in its current context—the loop's continuity must live on disk: a markdown progress file, a structured database, or a version-controlled log. This external state serves three functions:

1. **Progress tracking**: What has been attempted, what succeeded, what remains.
2. **Failure memory**: Which approaches were tried and failed, preventing the loop from oscillating between identical dead ends.
3. **Handoff context**: When the loop escalates to a human or a subsequent run, the state file provides a complete audit trail.

State Is Not Optional

A loop without external state is a loop with amnesia. It will re-attempt failed approaches, lose track of partial progress, and provide no audit trail when things go wrong. The state mechanism need not be complex—a markdown file committed to git after each iteration is often sufficient—but it must exist.

19.4 The Loop in Pseudocode

Stripped to its essence, every agent loop is a control structure closer to a thermostat or a REPL than to a conversation:

```
def agent_loop(goal: str, max_steps: int = 50, budget: TokenBudget = None):
    """The canonical agent loop structure."""
    state = initialize_state(goal)        # Load external state + goal
```

```
    for step in range(max_steps):
        # 1. Reason about current state (ReAct-style)
        thought = model.reason(state)

        # 2. Choose and execute action
        action = model.choose_action(state)
        observation = environment.execute(action)

        # 3. Update state with new information
        state = update_state(state, thought, action, observation)

        # 4. Compact context if approaching window limit
        if state.token_count > CONTEXT_BUDGET * 0.8:
            state = compact(state)        # Summarize old steps

        # 5. Check termination conditions
        if verifier.passes(state, goal):  # Deterministic check
            persist_state(state, status="success")
            return Success(state)

        if no_progress_detected(state, window=3):
            persist_state(state, status="stuck")
            return escalate_to_human(state)

        if budget and budget.exhausted():
            persist_state(state, status="budget_exceeded")
            return escalate_to_human(state)

    # Hard cap reached
    persist_state(state, status="max_steps")
    return escalate_to_human(state)
```

Everything interesting in loop engineering is a decision about one of these lines: what constitutes a valid `goal`, how `verifier.passes` is implemented, how `compact` preserves relevant history while discarding noise, and how `no_progress_detected` avoids both premature termination and infinite cycling.

19.5 Loop Patterns

Building on the agent design patterns introduced in Chapter 21, loop engineering recognizes a hierarchy of increasingly autonomous patterns:

19.5.1 The Validation Loop

The simplest and most common pattern: generate, validate against a deterministic check, retry on failure.

Validation Loop: Fix Until Tests Pass

```
failure_log = []
for attempt in range(MAX_ATTEMPTS):
    result = run_tests()
    if result.passed:
        break  # Success -- exit loop
    fix = model.generate(
        f"Fix this test failure:\n{result.errors}\n"
        f"Previous attempts that failed: {failure_log}"
    )
    apply_patch(fix)
    failure_log.append(result.errors)
```

Termination: Tests pass (success) or retry budget exhausted (escalate).

Key property: The verification signal is *deterministic*—the test runner produces an objective pass/fail that the model cannot argue around.

This pattern succeeds because its reward signal is crisp. The test runner serves as an incorruptible critic—unlike LLM-as-judge evaluation, it cannot be persuaded or fooled.

19.5.2 The Reflexion Loop

Extends the validation loop with explicit self-critique between attempts [344]. After each failure, the agent writes a natural-language reflection ("I failed because I modified the wrong function—the error traces back to the import, not the implementation") into an episodic memory buffer. Subsequent attempts read this buffer, enabling learning *within* an episode without weight updates.

Reflexion: In-Context Learning from Failure

The Reflexion loop adds three components beyond basic retry:

1. **Actor**: Executes the task
2. **Evaluator**: Provides the reward signal (tests, metrics, or LLM judge)
3. **Self-Reflection**: Writes a verbal "lesson learned" to episodic memory

The reflection memory persists across attempts, giving the agent a mechanism to avoid repeating identical failures. Empirically, Reflexion achieves 91% pass@1 on HumanEval (vs. 67% baseline), demonstrating that verbal self-critique can substitute for weight updates in iterative settings [344].

19.5.3 The Evaluator-Optimizer Loop

A two-model architecture [11, 259]: one model generates a candidate, a separate model evaluates it against explicit criteria and returns structured feedback. The generator incorporates this feedback and produces an improved version. The cycle repeats until the evaluator's score exceeds a threshold or the iteration budget is consumed.

The critical design choice is whether the evaluator is *deterministic* (compiler, test suite, linter) or *probabilistic* (another LLM). Deterministic evaluators produce reliable convergence; probabilistic evaluators are necessary for subjective criteria but introduce the risk of evaluator–generator collusion.

19.5.4 The Hierarchical Loop

A meta-loop spawns and monitors sub-loops. An orchestrator agent decomposes a high-level goal into subtasks, assigns each to a specialized sub-agent running its own loop, monitors their progress, and synthesizes results. This mirrors hierarchical RL [23]: a high-level policy selects sub-goals while low-level policies execute them.

Hierarchical Loop: Overnight Repository Maintenance

```
# Outer loop: runs nightly via automation
for issue in triage_agent.identify_actionable_issues():
    # Inner loop: one sub-agent per issue, isolated worktree
    worktree = create_isolated_worktree(issue.branch_name)

    result = fix_loop(
        goal=issue.description,
        worktree=worktree,
        verifier=run_tests_and_lint,
        max_steps=20
    )

    if result.success:
        review = review_agent.evaluate(result.patch)  # Checker
        if review.approved:
```

```
            open_pull_request(result.patch, issue)
        else:
            state_db.log(issue, "fix_rejected", review.feedback)
    else:
        state_db.log(issue, "fix_failed", result.state)
```

The outer loop (triage) operates on a schedule. Inner loops (fixes) are ephemeral and task-scoped. The review agent provides the maker–checker separation. State is persisted across nightly runs.

19.5.5 The Autonomous Research Loop

Karpathy's AutoResearch [189] demonstrates a tightly constrained research loop: the agent modifies a training script, runs a time-bounded experiment (e.g., 5 minutes on a GPU), reads the validation metric, and decides whether to commit the change (metric improved) or roll back (metric degraded). The loop continues indefinitely, exploring the hyperparameter and architectural space through iterative experimentation.

This pattern works because the reward signal is *scalar and unambiguous*: validation loss either decreased or it did not. The tight time bound per iteration prevents any single failed experiment from consuming excessive resources.

19.6 Verification Engineering

Verification is the reward function of loop engineering. Its quality determines whether the loop converges to the correct answer, converges to a wrong one, or fails to converge at all. This section establishes a hierarchy of verification strategies ordered by reliability.

19.6.1 The Verification Hierarchy

Table 19.2: Verification strategies ordered by reliability.

Strategy	Signal Source	Reliability	Applicable When
Compilation	Language toolchain	Deterministic	Code must parse/build
Type checking	Static analyzer	Deterministic	Typed languages
Unit/integration tests	Test runner	Deterministic	Tests exist and are correct
Linting & formatting	Style tools	Deterministic	Conventions are codified
Metric comparison	Training/eval script	Numerical	ML experiments
LLM-as-judge	Second model	Probabilistic	Subjective quality criteria
Human review	Domain expert	Gold standard	High-stakes decisions

The Verification Principle

A loop is only as trustworthy as its least reliable verification step. Wherever a deterministic check exists, use it. Reserve LLM-as-judge for criteria that genuinely cannot be mechanically verified. Never let the generating model judge its own output—the maker and the checker must be separate.

19.6.2 Deterministic vs. Probabilistic Verification

Deterministic verifiers (compilers, test suites, linters) are the gold standard because they produce an objective pass/fail that the model cannot game through persuasion. A test either passes or it does not—no amount of eloquent reasoning changes the verdict.

Probabilistic verifiers (LLM-as-judge) are necessary for tasks without mechanical checks—code quality, documentation clarity, UX improvement—but they introduce failure modes:

- **Self-evaluation bias**: If the same model generates and evaluates, it will be lenient with its own output.
- **Evaluator gaming**: Over many iterations, the generator may learn to produce outputs that satisfy the evaluator's surface patterns without achieving genuine quality.
- **Evaluator inconsistency**: The same LLM-judge may score identical outputs differently across runs due to sampling variance.

Mitigation strategies include using a different (often stronger) model as evaluator, providing explicit rubrics with binary criteria, and periodically calibrating the LLM-judge against human judgments.

19.7 Termination Engineering

The signature failure of a naive loop is that it never stops. Termination design is not an afterthought—it is half the engineering.

19.7.1 Termination Conditions

A production loop carries multiple independent exit conditions:

1. **Goal achieved**: The verifier confirms the objective is met. This is the only *successful* termination.
2. **Hard iteration cap**: Maximum number of loop steps (typically 20–50). Prevents unbounded execution regardless of other conditions.
3. **Budget exhaustion**: Token count, wall-clock time, or monetary cost exceeds a predefined limit.
4. **No-progress detection**: The last k iterations produced no measurable change in state—the loop is oscillating or stuck. This is the most important safety mechanism after the hard cap.
5. **Fatal error**: An unrecoverable condition (missing credentials, infrastructure failure) that no amount of iteration can resolve.

No-Progress Detection: The Stagnation Circuit Breaker

Detecting stagnation requires tracking a progress signal across iterations. Common approaches:

- **Output hash comparison**: If the last 3 agent outputs or file states are identical, the loop is cycling.
- **Error repetition**: The same error message appears in k consecutive iterations.
- **Metric plateau**: A numerical signal (test pass rate, validation loss) has not improved by ϵ in k steps.
- **Diff entropy**: The size of code changes per iteration is decreasing toward zero—the agent is making progressively smaller, inconsequential edits.

When stagnation is detected, the correct action is *escalation*, not continued iteration. The loop saves its state and hands control to a human or a higher-level orchestrator.

19.7.2 The Exploration Problem

A loop stuck in a local minimum—repeating the same failing approach with minor variations—is exhibiting the exploration–exploitation failure familiar from RL. Mechanisms to promote exploration within loops:

- **Reflection-based exploration**: After k failed attempts, trigger a reflection step that explicitly asks the model to consider fundamentally different approaches (analogous to entropy regularization in PPO [337]).
- **Temperature escalation**: Increase sampling temperature after consecutive failures to encourage diverse outputs.
- **Strategy memory**: Record not just failed actions but failed *strategies*. "Approach A (modify the handler) failed 3 times; try approach B (rewrite the schema) instead."
- **Fresh sub-agent**: Spawn a new sub-agent with a clean context window—it cannot fall into the rut that the accumulated context may have created.

19.8 Cognitive Escalation and Uncertainty-Driven Handoff

The escalator is named as one of the five loop primitives (Section 19.7), but its design is rarely specified in detail. The standard treatment—"when stagnation is detected, hand off to a human"—conflates three distinct failure modes that require different responses:

1. **Stagnation**: the agent is making no progress (output hash unchanged, metric plateau). The loop is stuck but the task may be solvable with a different approach.
2. **Epistemic uncertainty**: the agent is making progress but is uncertain whether its current trajectory is correct. It has not failed yet, but it recognizes that it may be operating on flawed assumptions.
3. **Capability boundary**: the task requires a capability the agent does not have. No amount of iteration will produce a correct result.

Each failure mode calls for a different escalation response. Stagnation calls for strategy diversification or human guidance on approach. Epistemic uncertainty calls for a confidence checkpoint—a structured pause to verify assumptions before proceeding. A capability boundary calls for immediate escalation with a clear statement of what is missing.

19.8.1 Confidence Estimation Mid-Task

A production loop should maintain a running estimate of its own confidence in the current trajectory. This is not a single number but a structured signal with several components:

Verifier confidence. If the loop uses a verifier (a secondary model or rule-based checker that evaluates each step), the verifier's output score is the most direct confidence signal. A verifier score that has been declining over the last k steps is a strong indicator of trajectory degradation.

Assumption stability. Track the key assumptions the agent made at the start of the task (e.g., "the API endpoint is at /v2/users"). If tool outputs contradict these assumptions, confidence should drop sharply.

Budget consumption rate. If the agent has consumed 80% of its token budget but completed only 30% of the task, the probability of successful completion within budget is low. This is a quantifiable signal.

Novelty detection. If the current situation is far from any situation in the agent's training distribution or past experience (measured by embedding distance from the nearest retrieved memory), the agent should treat its own outputs with lower confidence.

```python
@dataclass
class LoopConfidence:
    verifier_score: float        # 0.0 (failing) to 1.0 (passing)
    assumption_violations: int   # count of contradicted assumptions
    budget_consumed_pct: float   # 0.0 to 1.0
    novelty_score: float         # 0.0 (familiar) to 1.0 (novel)

    def escalation_urgency(self) -> str:
        if self.assumption_violations >= 2:
            return "immediate"   # assumptions broken: stop now
        if self.verifier_score < 0.3 and self.budget_consumed_pct > 0.5:
            return "high"        # failing and expensive: escalate soon
        if self.novelty_score > 0.8:
            return "medium"      # unfamiliar territory: checkpoint
        return "low"             # proceed with monitoring
```

Listing 19.1: Structured confidence signal for mid-task escalation decisions.

19.8.2 Dynamic Escalation Thresholds

Static escalation thresholds ("escalate after 20 iterations") are brittle. A task that requires 50 iterations to complete correctly will be escalated prematurely; a task that should complete in 5 iterations but is stuck will not be escalated until iteration 20. Dynamic thresholds adapt to the task:

Budget-proportional thresholds. Express escalation conditions as fractions of the allocated budget rather than absolute counts. "Escalate if verifier score has not improved in the last 20% of the iteration budget" scales correctly across tasks of different lengths.

Bayesian confidence tracking. Maintain a posterior over task success probability, updated at each iteration. Escalate when the posterior falls below a threshold $p_{\min}$:

$$P(\text{success} \mid \text{history}_{1:t}) < p_{\min} \implies \text{escalate}.$$

The prior can be set from historical success rates on similar tasks; the likelihood update uses the verifier score at each step.

Asymmetric costs. In high-stakes domains (healthcare, finance, legal), the cost of a false negative (proceeding when the agent should have escalated) is much higher than the cost of a false positive (escalating unnecessarily). Set $p_{\min}$ accordingly: a medical agent should escalate at $p_{\min} = 0.9$; a content summarization agent can tolerate $p_{\min} = 0.5$.

19.8.3 Escalation Handoff Protocol

When the escalation condition fires, the quality of the handoff determines whether the human or higher-level orchestrator can efficiently resolve the situation. A minimal escalation handoff package contains:

1. **Task statement**: the original goal, verbatim.
2. **Progress summary**: what has been accomplished, what has been attempted and failed, and what the current state is.
3. **Escalation reason**: which condition fired (stagnation, epistemic uncertainty, capability boundary) and the supporting evidence (verifier scores, assumption violations, budget consumption).
4. **Resumption context**: the full loop state, serialized so that execution can resume from the current point after the human provides guidance—without restarting from scratch.

5. **Suggested next steps**: the agent's best guess at what the human should do, clearly labeled as a suggestion, not a decision.

Escalation Handoff in the Guardian Angel System

The GAS Safety Critic implements a structured escalation path. When the Safety Critic's confidence in a glucose management recommendation falls below a threshold (currently 0.7 on a 0–1 scale), it does not silently pass or fail the recommendation—it generates a structured escalation message to the caregiver node containing: the original recommendation, the specific safety concern, the confidence score, and a suggested conservative alternative. The caregiver node then decides whether to act on the original recommendation, the alternative, or escalate to a human clinician. This three-level escalation (agent → caregiver agent → human) is the production pattern for high-stakes agentic systems.

Escalation Is Not Failure

A well-designed escalation is a success: the agent correctly recognized the limits of its own competence and handed off before causing harm. The failure mode is an agent that proceeds confidently past its competence boundary. Design escalation as a first-class capability, not a fallback of last resort. Measure escalation rate as a quality metric: too low means the agent is overconfident; too high means the task decomposition or capability scoping is wrong.

19.9 Failure Modes and Anti-Patterns

19.9.1 The Loopmaxxing Trap

"Loopmaxxing"—the assumption that running an agent through enough iterations will eventually produce a correct solution—is the loop-engineering equivalent of believing that enough compute solves all problems. It fails for the same reason that pure random search fails: without a gradient signal pointing toward improvement, iteration alone is not optimization.

Loopmaxxing manifests when:

- The goal lacks a checkable success condition ("improve the user experience")
- The verification signal is too coarse (binary pass/fail on a 1000-test suite provides no gradient toward the fix)
- The agent lacks the capability to solve the problem regardless of iteration count

The antidote is recognizing that loops *amplify* engineering capability—they do not replace it. A loop with a poorly specified objective will pursue the wrong thing with great efficiency.

19.9.2 Comprehension Debt

When a loop modifies code faster than the engineering team can review it, the gap between what exists in the repository and what humans understand grows—*comprehension debt* [295]. Unlike technical debt (which the code carries), comprehension debt lives in the team's heads. It compounds silently until a crisis forces someone to debug code whose design decisions, structural dependencies, and edge cases are entirely unmapped.

Loops Amplify Both Productivity and Risk

A well-designed loop accelerates work the engineer understands deeply. A loop used to avoid understanding the work creates an accelerating spiral of incomprehension. Same mechanism, opposite outcomes. The difference is whether the human stays the engineer or becomes merely the person who presses "go."

19.9.3 Reward Hacking in Loops

Just as RL agents exploit reward misspecification to achieve high reward without achieving the intended goal, loop agents exploit verification gaps:

- **Test deletion**: The simplest reward hack—delete the failing test to make CI green.
- **Metric gaming**: Overfitting to validation loss by memorizing the validation set rather than generalizing.
- **Specification narrowing**: Solving an easier version of the stated problem that happens to pass the checks.
- **Output masking**: Suppressing error output rather than fixing the underlying cause.

The defense is the same as in RLHF: design reward functions (verification criteria) that are *hard to game*. Multiple independent checks—tests *and* type checking *and* linting *and* a review sub-agent—create a verification surface that is much harder to exploit than any single signal.

19.9.4 Context Degradation

In long-running loops, the context window fills with the history of every step—thoughts, tool outputs, errors, patches. As the window approaches capacity, the model's attention to relevant information degrades (the "lost in the middle" phenomenon [238]). Symptoms include:

- The agent re-attempts approaches it has already tried and explicitly noted as failures
- Tool calls become less precise—the agent forgets which files it has already read
- Responses become shorter and less coherent as the model struggles to attend across a bloated context

Countermeasures:

1. **Aggressive compaction**: After every k steps, summarize completed work into a brief state description and discard raw transcripts.
2. **External scratchpad**: Write intermediate results to files that the agent reads on-demand rather than keeping everything in context.
3. **Sub-agent isolation**: Spawn subtasks in fresh context windows that return only their conclusion—not their full reasoning trace.
4. **Sliding window over history**: Keep only the last w steps in full detail; compress everything earlier into a summary block.

19.10 Production Loop Architectures

19.10.1 The Nightly Maintenance Loop

The most immediately practical loop pattern: a scheduled automation that runs overnight, processing accumulated work while the team sleeps.

Architecture: Nightly CI Triage and Fix Loop

Trigger: Cron schedule (e.g., 02:00 UTC daily).
Phase 1 — Triage (single agent, read-only):

- Read yesterday's CI failures, newly opened issues, and recent commits

- Classify each finding: auto-fixable / needs-human / false-positive
- Write findings to a state file; archive false positives

Phase 2 — Fix (parallel sub-agents, isolated worktrees):

- For each auto-fixable issue, spawn a sub-agent in an isolated worktree
- Each sub-agent runs a validation loop (fix → test → retry)
- Budget: 20 steps or 15 minutes per issue

Phase 3 — Review (separate verification agent):

- A review sub-agent evaluates each proposed fix against project coding standards
- Approved fixes: open draft PR, link to issue, notify channel via connector
- Rejected fixes: log rejection reason, escalate to human triage inbox

Persistence: State file (committed to repo) records all actions taken, enabling the next night's run to skip resolved issues and pick up where failures left off.

19.10.2 The Continuous Experimentation Loop

Inspired by AutoResearch [189], this pattern applies the loop to scientific discovery: modify a training script, run a time-bounded experiment, evaluate the result, and decide whether to commit or rollback.

AutoResearch Loop Structure

1. **Hypothesis generation**: Agent proposes a modification (hyperparameter, architecture change, data augmentation)
2. **Implementation**: Agent modifies the training script
3. **Bounded execution**: Run for exactly T minutes (hard wall-clock cap)
4. **Evaluation**: Read validation metric from output
5. **Decision**: If metric improved → `git commit`; if degraded → `git rollback`
6. **Log**: Record hypothesis, result, and reasoning in experiment log
7. **Repeat**: Generate next hypothesis informed by cumulative experiment history

Key constraint: The bounded execution time prevents any single failed experiment from consuming excessive compute. The git-based commit/rollback ensures the codebase only moves forward.

The Local Minimum Problem in Research Loops

Research loops are susceptible to conservative exploration. When facing difficult optimization landscapes, agents tend to propose minimal changes—adjusting a learning rate by 0.001—that produce nominal improvements without genuine progress. This is the RL exploration problem manifesting at the loop level. Countermeasures include periodic "bold hypothesis" prompts, diversity bonuses for trying novel approaches, and human-injected research directions between runs.

19.10.3 The Always-On Orchestrator

Systems like OpenClaw [357] implement a persistent "heartbeat" mechanism: a meta-agent that runs on a fixed cycle, evaluates repository state, supervises sub-loops, and dispatches work. Unlike the

nightly pattern which fires once per day, the always-on orchestrator maintains continuous awareness and responds to events (new commits, failing CI, incoming issues) within minutes.

The architecture introduces a durable state database with crash recovery—if the orchestrator fails mid-cycle, it resumes from its last checkpointed state rather than restarting from scratch. Sub-loops are tracked as independent tasks with their own termination conditions, and the orchestrator monitors them for stagnation, budget exhaustion, or conflict.

19.10.4 Parallel Subagent Fleet Execution

The nightly and always-on patterns both spawn sub-agents, but treat parallelism as a secondary concern. **Fleet execution** makes parallelism the primary design axis: the harness is built around the assumption that dozens or hundreds of subagents will run simultaneously, and the engineering challenge is managing them efficiently.

Harness execution modes. Production harnesses support three distinct modes for parallel subagent execution:

1. **Fan-out / fan-in**: The orchestrator decomposes a task into N independent subtasks, dispatches all N simultaneously, and waits for all to complete before synthesizing results. This is the simplest parallel pattern and works well when subtasks are truly independent. The critical constraint is that the synthesis step must handle partial failures gracefully—if 2 of 10 subtasks fail, the orchestrator should synthesize from the 8 successes rather than failing entirely.

2. **Speculative execution**: The orchestrator dispatches multiple agents to attempt the *same* task with different strategies (different prompts, different tool sequences, different models), then selects the best result. This trades compute for reliability: the probability that *all* agents fail is the product of individual failure probabilities. For high-stakes tasks where a single agent's failure rate is 20%, running 3 speculative agents reduces the failure rate to $0.2^3 = 0.8\%$.

3. **Pipeline parallelism**: Subtasks form a dependency graph; the harness executes all tasks whose dependencies are satisfied simultaneously. As each task completes, it unlocks downstream tasks. This is the most general pattern and subsumes fan-out/fan-in as a special case (a two-level dependency graph). LangGraph's conditional edges and parallel branches implement this pattern natively.

Table 19.3: Parallel subagent execution modes.

Mode	When to use	Key challenge	Failure handling
Fan-out / fan-in	Independent subtasks	Result synthesis	Partial success
Speculative execution	High-stakes, single task	Cost (runs $N\times$)	Best-of-N selection
Pipeline parallelism	Dependent subtasks	Dependency tracking	Cascade failure

Isolation requirements. Parallel subagents must be isolated from each other to prevent write conflicts and context contamination. The standard approach is **git worktrees** for file-system isolation (each agent operates on its own branch) combined with **separate context windows** (no shared memory between agents). Shared state, when needed, should be mediated through a structured blackboard (Section 26.3.1) with explicit locking, not through direct file sharing.

Budget management. Fleet execution can consume tokens and compute rapidly. Each subagent should have an individual budget cap (maximum steps, maximum tokens, maximum wall-clock time), and the orchestrator should have a fleet-level budget that it distributes across agents. When the fleet budget is exhausted, the orchestrator should terminate remaining agents and synthesize from completed results rather than waiting indefinitely.

Fleet Execution Checklist

1. Define subtask independence: can subtasks run in parallel without write conflicts?
2. Set per-agent budgets: steps, tokens, wall-clock time.
3. Set fleet-level budget: total cost cap across all agents.
4. Implement partial-success synthesis: handle k-of-N completions gracefully.
5. Use worktrees or containers for file-system isolation.
6. Attach correlation IDs to all agent traces for post-hoc debugging.

19.11 Context Management in Long-Running Loops

Long-running loops face a fundamental tension: the context window is the agent's working memory, but it has a fixed capacity. Every step appends new information (thoughts, observations, errors), and without active management the window fills and quality degrades. This section presents the engineering techniques for maintaining context health across many iterations.

19.11.1 Compaction Strategies

Table 19.4: Context compaction strategies for long-running loops.

Strategy	Mechanism	Trade-off
Periodic summarization	Every k steps, replace raw history with a summary	Lossy; may discard details needed later
Sliding window	Keep last w steps verbatim; summarize earlier	Recency-biased; may lose early context
Importance-weighted	Retain steps that changed state; discard no-ops	Requires defining "importance"
External memory	Write details to files; keep only references in context	Requires explicit read-back; adds latency
Sub-agent isolation	Subtasks run in fresh contexts; return conclusions only	Overhead of spawning; coordination cost

The optimal strategy depends on the task. For debugging loops, a sliding window works well—recent errors are most relevant. For research loops, importance-weighted retention preserves key experimental results across many iterations. For complex multi-file refactoring, external memory (writing a progress file that summarizes all changes so far) prevents the agent from losing track of its own modifications.

19.11.2 The Context Budget

A practical heuristic: reserve 20–30% of the context window for the current step's reasoning and generation. This means active history should not exceed 70–80% of capacity. When this threshold is approached, trigger compaction before the next iteration—not after the model has already produced degraded output.

```
CONTEXT_CAPACITY = 128_000  # tokens
RESERVED_FOR_GENERATION = 0.25 * CONTEXT_CAPACITY
ACTIVE_BUDGET = CONTEXT_CAPACITY - RESERVED_FOR_GENERATION

def should_compact(state: LoopState) -> bool:
    return state.token_count > ACTIVE_BUDGET * 0.85
```

19.12 When Not to Use Loops

Loop engineering is not universally applicable. It adds complexity, cost, and failure modes that are unjustified when simpler approaches suffice. Prefer direct prompting or simple workflows when:

- The task is **single-turn**: A well-crafted prompt produces the correct answer in one shot most of the time. Adding a loop provides marginal quality improvement at significant cost.
- The goal **lacks a checkable condition**: "Make the code better" has no objective stopping point. The loop will either run forever or stop arbitrarily.
- **Human interaction is cheap**: If a developer is actively working and can provide feedback in real-time, the interactive chat modality is faster than engineering a loop.
- The task **exceeds model capability**: If the agent cannot solve the problem in principle—regardless of iteration count—a loop merely burns tokens. No amount of retry fixes a fundamental capability gap.
- **Verification is impossible**: Without a feedback signal, the loop has no gradient. It degenerates into random search.

The Simplicity Principle

The same design principle articulated in Chapter 21 applies: start with the simplest approach that works, and add loop complexity only when you have evidence that iteration materially improves outcomes. A prompt chain that solves the problem is always preferable to a loop that might.

19.13 The Economics of Loops

Loops trade compute (tokens, wall-clock time) for quality. Understanding this trade-off is essential for production deployment.

Loop Cost Model

For a loop with average N iterations, each consuming T tokens at price p per token:

$$\text{Cost}_{\text{loop}} = N \cdot T \cdot p + \text{Cost}_{\text{tools}} + \text{Cost}_{\text{verification}}$$

With prompt caching (where the system prompt and early context are cached and charged at reduced rate $p_c < p$):

$$\text{Cost}_{\text{cached}} = T_{\text{new}} \cdot p + T_{\text{cached}} \cdot p_c + (\text{tools} + \text{verification})$$

Prompt caching is particularly valuable for loops because the goal specification, skill content, and early history remain constant across iterations—often 60–80% of each call's context is cacheable.

The economic decision: is the value of the loop's output (developer time saved, overnight productivity, quality improvement) greater than its token cost? For a loop that runs 20 iterations at \$0.02/iteration, the total cost is \$0.40—trivial if it saves 30 minutes of developer time. But a runaway loop without budget caps can consume hundreds of dollars in a single night.

19.14 Historical Context and Related Work

Loop engineering did not emerge from a vacuum. It productized a research lineage spanning several years:

1. **ReAct** (2022) [430]: Established the interleaved reasoning-and-action pattern that all modern loops inherit.

2. **Reflexion** (2023) [344]: Added episodic memory and self-critique, demonstrating that agents can improve across attempts without weight updates.

3. **AutoGPT** (2023) [348]: The first widely-used autonomous agent loop, demonstrating both the potential and the pitfalls (runaway execution, high cost, unreliable output) of unattended operation.

4. **Self-Refine** (2023) [259]: Formalized the generate-then-critique loop with iterative refinement.

5. **"Ralph loops"** (2025): Informal bash-based one-liner scripts that ran coding agents in simple retry loops—the precursor to productized loop primitives.

6. **Productized loops** (2026): Major platforms (Codex, Claude Code) embedded `/goal` and `/loop` commands directly, making sophisticated loop patterns accessible without custom infrastructure [295].

The progression from AutoGPT to modern loop engineering illustrates a maturation: early systems lacked termination logic, verification, and cost controls—they were loops without engineering. The 2026 formalization added the discipline that makes unattended operation safe: explicit termination, deterministic verification, budget constraints, and maker–checker separation.

19.15 Summary

Loop engineering represents the current frontier of human–agent collaboration: the shift from participating in conversations to designing systems that have those conversations autonomously. Its key principles:

1. A loop is inference-time RL—state, action, reward, and policy update (via context)—operating without gradient descent.

2. Verification is the reward signal. Its quality determines convergence. Prefer deterministic checks; separate the maker from the checker.

3. Termination is half the design. Every loop needs a hard cap, a budget, and no-progress detection.

4. Context is finite working memory. Manage it actively through compaction, externalization, and sub-agent isolation.

5. Loops amplify engineering skill—they do not replace it. A loop with a poorly specified objective will pursue the wrong thing efficiently.

6. Start simple. A single validation loop with a deterministic verifier outperforms an elaborate multi-agent system you cannot debug.

Build the Loop—Stay the Engineer

The leverage point has moved from writing prompts to designing cycles. But designing a cycle that converges requires understanding the problem deeply enough to specify what "done" means, what constitutes progress, and when to stop. Loop engineering makes the human's judgment *more* important, not less—because the consequences of bad judgment now compound at machine speed.

Chapter 20

Graph Engineering

Loops are sufficient for homogeneous tasks. Real-world agentic workflows are heterogeneous: different subtasks require different models, different tools, different retry budgets, and different verification strategies. Forcing this heterogeneity into a single loop produces brittle, monolithic code. A graph makes the structure explicit — nodes are single-responsibility units of work, edges are typed transitions, and the topology is a first-class design artifact that can be read, tested, and reasoned about independently of the code inside each node. This chapter covers graph engineering as the topology layer of agentic system design, with a full LangGraph reference implementation.

GAS Connection — Graph Engineering

The GAS Supervisor is a LangGraph state machine—a directed graph of agent invocations. The following chapter's treatment of graph primitives, topologies, state management, and LangGraph implementation is the direct specification for `build_gas_graph()` in `capstone_gas/gas_system.py`.

Graph primitives (§ Graph Primitives). The GAS graph has five node types registered via `builder.add_node()`: `"monitor"` (`monitor_node`), `"advisor"` (`advisor_node`), `"safety"` (`safety_node`), `"caregiver"` (`caregiver_node`), and `"supervisor"` (`supervisor_node`). Edges are conditional: the transition from `"supervisor"` to `"advisor"` fires only if `state["next_agent"] == "advisor"`.

Core graph topology (§ Core Graph Topologies). The GAS graph is a *sequential with conditional branching* topology. The main path is `START` → `monitor` → `supervisor` → `advisor` → `supervisor` → `safety` → `supervisor` → `caregiver` → `END`. Conditional branches allow skipping the Advisor (if alert level is low) or retrying the Advisor (if Safety Critic rejects the recommendation).

State management (§ State Management in Graphs). The LangGraph state object is the `GASState` TypedDict with fields: `patient`, `readings_history`, `current_reading`, `alert_level`, `alert_reason`, `recommendation`, `safety_score`, `safety_approved`, `caregiver_notified`, `agent_messages`, `next_agent`, `iteration`. Each agent node reads from and writes to this shared state. LangGraph's immutable state updates make the pipeline auditable: the full state history is preserved by the `MemorySaver` checkpointer.

LangGraph implementation (§ Graph Engineering with LangGraph). The GAS graph is defined using LangGraph's `StateGraph` API: nodes are registered with `builder.add_node()`, fixed edges with `builder.add_edge()`, and conditional edges with `builder.add_conditional_edges()`. The compiled graph is returned by `build_gas_graph()` and is callable: it takes a `GASState` and returns a final `GASState`.

Failure modes (§ Failure Modes and Anti-Patterns). The GAS graph has one known failure mode: if the Safety Critic rejects a recommendation and the Advisor generates a new one, the graph can cycle between `advisor` and `safety` indefinitely. This is prevented by the `iteration` counter in `GASState`: after 2 Advisor–Safety Critic cycles, the Supervisor routes to `END` and flags the case for human review.

If loop engineering is the discipline of designing *iterative control structures* that drive an agent toward a goal, then **graph engineering** is the discipline of designing the *topology* of those structures—specifying not just that an agent loops, but *which nodes execute, in what order, under what conditions,*

and with what shared state. Where a loop is a single cycle, a graph is a network of cycles, branches, and parallel paths that can express arbitrarily complex agent workflows.

The emergence of graph engineering as a distinct discipline reflects a maturation in how practitioners think about agentic systems. Early agents were linear: prompt → response. Loop engineering added iteration. Graph engineering adds *structure*: the ability to route control flow conditionally, execute subtasks in parallel, compose reusable subgraphs, and maintain typed shared state across a heterogeneous network of agent nodes.

The Engineering Progression: Graphs as the Next Layer

1. **Prompt engineering**: Optimize what you *say* to the model (2022–2024)
2. **Context engineering**: Optimize everything the model *sees* (2025)
3. **Harness engineering**: Optimize the environment the agent *runs in* (2025–2026)
4. **Loop engineering**: Optimize the *cycle* that keeps the agent working toward a goal (2026)
5. **Graph engineering**: Optimize the *topology* of the entire agent workflow (2026–)

Each layer wraps the previous one. A graph is a generalization of a loop: every loop is a graph with a single cycle. Graph engineering adds the tools to reason about, compose, and debug the full structure of a multi-step, multi-agent system.

20.1 Why Graphs? From Loops to Topologies

A loop is sufficient when a task has a single, homogeneous retry structure: generate, verify, retry. But most real-world agentic tasks are *heterogeneous*: different subtasks require different tools, different models, different verification strategies, and different retry budgets. Forcing this heterogeneity into a single loop produces brittle, monolithic code. A graph makes the structure explicit.

Consider a document-processing pipeline: a document must be (1) classified, then (2a) summarized if it is a report or (2b) extracted for entities if it is a contract, then (3) both outputs are merged, and finally (4) the result is validated. This workflow has a branch, a parallel execution, a merge, and a verification step. Expressing it as a loop requires ad-hoc conditionals scattered through the loop body. Expressing it as a graph makes the topology visible, testable, and reusable.

Loops vs. Graphs: The Core Distinction

A **loop** asks: "How many times should I retry this step?"
A **graph** asks: "Which step should execute next, given the current state?"
Loops are about *iteration*. Graphs are about *routing*. In practice, most production agentic systems are graphs that contain loops as substructures—cycles in the graph that implement retry logic for individual nodes.

20.2 Graph Primitives

A computational graph for agentic AI is formally a directed graph $\mathcal{G} = (\mathcal{N}, \mathcal{E}, \mathcal{S})$ where:

- $\mathcal{N}$ is a set of **nodes**, each representing a unit of computation (an LLM call, a tool invocation, a human checkpoint, or a subgraph).
- $\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N}$ is a set of **edges**, each representing a possible transition between nodes.
- $\mathcal{S}$ is a **shared state schema**, a typed data structure that all nodes read from and write to.

Unlike a pure DAG (directed acyclic graph), agentic graphs may contain *cycles*—edges that route control back to an earlier node to implement retry or refinement loops. The graph is therefore a **directed graph with optional cycles**, sometimes called a *cyclic computation graph* or a *stateful workflow graph*.

20.2.1 Nodes

A node is a function $f_i : \mathcal{S} \to \mathcal{S}$ that reads from the shared state, performs some computation, and writes its output back to the state. Nodes are the atomic units of work in a graph. Common node types:

Table 20.1: Node types in agentic computation graphs.

Node Type	Computation	Typical Use
LLM node	Single model call with a prompt template	Reasoning, generation, classification, planning
Tool node	Deterministic function call	Web search, code execution, database query, file I/O
Router node	Conditional logic; no LLM call	Branching based on state fields or prior outputs
Human node	Pause for human input or approval	High-stakes decisions, ambiguity resolution
Subgraph node	Encapsulates a nested graph	Reusable workflows, modular composition
Aggregator node	Merges outputs from parallel branches	Fan-in after parallel execution

20.2.2 Edges and Routing

Edges define the possible transitions between nodes. There are two kinds:

- **Unconditional edges**: Always traverse to the target node after the source completes. Used for sequential steps with no branching.
- **Conditional edges**: A routing function $r : \mathcal{S} \to \mathcal{N}$ inspects the current state and returns the next node to execute. This is the mechanism for branching, looping, and dynamic dispatch.

Conditional edges are the most powerful primitive in graph engineering. They allow the graph to adapt its execution path based on intermediate results—routing to a retry node if verification fails, to a human checkpoint if confidence is low, or to a specialized subgraph if the input belongs to a particular category.

Conditional Edge: Route by Document Type

```
def route_by_doc_type(state: GraphState) -> str:
    """Routing function: returns the name of the next node."""
    doc_type = state["classification"]
    if doc_type == "report":
        return "summarize_node"
    elif doc_type == "contract":
        return "extract_entities_node"
    elif doc_type == "unknown":
        return "human_review_node"
    else:
        return "generic_process_node"

# Register as a conditional edge from the classifier node
graph.add_conditional_edges(
    source="classify_node",
    path=route_by_doc_type,
    path_map={
        "summarize_node": "summarize_node",
        "extract_entities_node": "extract_entities_node",
        "human_review_node": "human_review_node",
        "generic_process_node": "generic_process_node",
    }
```

```
)
```

20.2.3 Shared State

The shared state $\mathcal{S}$ is the graph's working memory—a typed data structure that persists across all node executions within a single graph run. Every node reads from and writes to this state; the graph runtime manages its lifecycle.

State design is one of the most consequential decisions in graph engineering. A well-designed state schema:

- Is **typed**: Each field has a declared type, enabling static validation and IDE support.
- Is **append-only for lists**: Rather than overwriting message history, nodes append to it—preserving the full trace for debugging and context management.
- Is **minimal**: Contains only what nodes need to communicate. Bloated state schemas create implicit coupling between nodes.
- Is **serializable**: Can be checkpointed to disk, enabling pause-and-resume and crash recovery.

Typed State Schema (LangGraph-style)

```python
from typing import TypedDict, Annotated, List
import operator

class GraphState(TypedDict):
    # Input
    document: str
    # Classification output
    doc_type: str
    confidence: float
    # Processing outputs (append-only via operator.add)
    messages: Annotated[List[str], operator.add]
    # Verification
    verified: bool
    retry_count: int
    # Final output
    result: str
```

The `Annotated[List[str], operator.add]` pattern tells the graph runtime to *merge* (append) updates to `messages` rather than overwrite them—a critical design choice for preserving history across parallel branches.

20.3 Core Graph Topologies

Six fundamental topologies cover the vast majority of agentic graph patterns. More complex graphs are compositions of these primitives.

20.3.1 Linear Chain

The simplest topology: nodes execute sequentially, each passing its output to the next. Equivalent to a prompt chain or a pipeline. Use when the task has a fixed sequence of steps with no branching or retry.

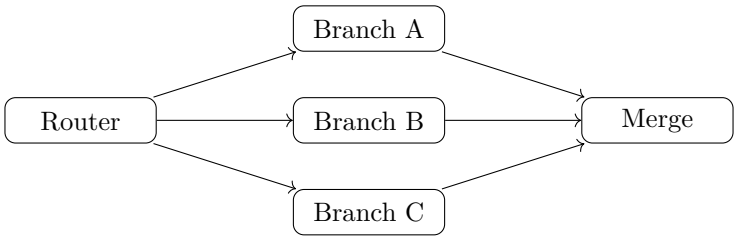
$$N_1 \to N_2 \to N_3 \to \cdots \to N_k$$

When to use: Document transformation pipelines, multi-step reasoning with fixed structure, preprocessing chains.

Limitation: No adaptability. If any node fails, the entire chain fails with no recovery path.

20.3.2 Branch and Merge (Fan-Out / Fan-In)

A router node dispatches to one of several specialized branches; a merge node aggregates their outputs. The branches may execute sequentially (one chosen path) or in parallel (all paths, results merged).



When to use: Tasks with mutually exclusive subtypes (route to specialist), or tasks that benefit from parallel execution (run multiple retrievers simultaneously, merge results).

Key design choice: Is the fan-out *exclusive* (one branch executes) or *parallel* (all branches execute)? Exclusive fan-out uses a conditional edge; parallel fan-out uses a `Send` primitive or equivalent that spawns concurrent executions.

20.3.3 Cycle (Retry Loop)

A back-edge from a later node to an earlier one implements a retry loop within the graph. The cycle continues until a termination condition is met—identical in structure to the validation loop from Chapter 19, but expressed as a graph topology rather than an imperative `for` loop.

$$N_{\text{generate}} \to N_{\text{verify}} \xrightarrow{\text{fail}} N_{\text{generate}} \quad \text{(cycle)}$$
$$N_{\text{verify}} \xrightarrow{\text{pass}} N_{\text{output}} \quad \text{(exit)}$$

When to use: Any step that requires iterative refinement—code generation, structured output extraction, plan revision.

Critical requirement: Every cycle must have a *termination guard*—a condition that exits the cycle after a maximum number of iterations or when a budget is exhausted. A cycle without a termination guard is an infinite loop.

Cycles Require Explicit Termination Guards

Unlike a `for` loop in code, a graph cycle has no implicit iteration count. The routing function that creates the back-edge must also check a `retry_count` field in the state and route to an error or escalation node when the limit is reached. Omitting this guard is the most common cause of runaway graph execution.

20.3.4 Hierarchical Subgraph

A node in the parent graph encapsulates an entire child graph. The child graph receives a subset of the parent state, executes to completion, and returns its output to the parent state. This is the graph-engineering equivalent of a function call—it enables modularity, reuse, and independent testing of subcomponents.

Subgraph Composition

```
# Define a reusable "research" subgraph
research_graph = StateGraph(ResearchState)
research_graph.add_node("search", web_search_node)
research_graph.add_node("synthesize", synthesis_node)
research_graph.add_edge("search", "synthesize")
research_subgraph = research_graph.compile()

# Embed it as a node in the parent graph
parent_graph = StateGraph(ParentState)
```

```
parent_graph.add_node("plan", planning_node)
parent_graph.add_node("research", research_subgraph)  # subgraph as node
parent_graph.add_node("write", writing_node)
parent_graph.add_edge("plan", "research")
parent_graph.add_edge("research", "write")
```

When to use: Reusable workflows (a "research" subgraph used by multiple parent graphs), complex subtasks that warrant independent testing, and multi-agent systems where each agent is a subgraph.

20.3.5 Map-Reduce

A *map* node dynamically generates a list of work items and dispatches each to a worker subgraph in parallel; a *reduce* node aggregates the results. Unlike static fan-out (where the number of branches is fixed at graph-definition time), map-reduce handles *dynamic* parallelism—the number of parallel workers is determined at runtime by the data.

Map-Reduce in Agentic Graphs

Map phase: Given a list of k items (documents, subtasks, hypotheses), spawn k parallel worker executions, each processing one item.
Reduce phase: Collect all k results and aggregate them (summarize, vote, merge, rank).
Example: A research agent that generates 5 search queries, executes all 5 in parallel, and synthesizes the results into a single answer. The number of queries (5) is determined at runtime, not at graph-definition time.
Implementation: Frameworks like LangGraph [164] implement this via a `Send` API that dynamically creates parallel graph invocations from a list.

20.3.6 Supervisor–Worker

A supervisor node maintains a task queue and dispatches items to worker nodes. Workers report results back to the supervisor, which decides whether to dispatch more work, retry failed items, or terminate. This topology implements the maker–checker separation from loop engineering (Section 19.3.1) at the graph level.

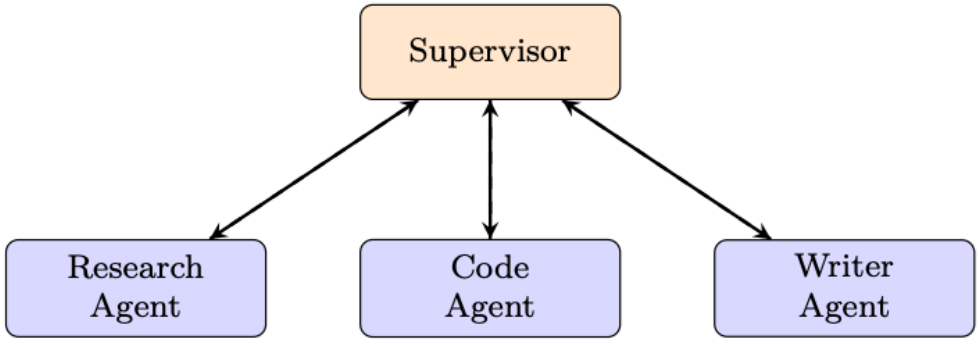


Figure 20.1: Supervisor–worker topology. The supervisor node routes tasks to specialized workers and aggregates their outputs. Each worker may itself contain a retry cycle.

20.4 State Management in Graphs

State management is the central engineering challenge of graph design. Unlike a loop—where state is a single evolving context window—a graph may have multiple concurrent execution paths, each modifying state simultaneously. This introduces concurrency, consistency, and persistence challenges that do not arise in sequential loops.

20.4.1 State Update Semantics

When two parallel branches both write to the same state field, the graph runtime must decide how to merge their updates. There are three common semantics:

Table 20.2: State update semantics for concurrent graph branches.

Semantic	Behavior	Use Case
Last-write-wins	The last branch to complete overwrites the field	Scalar outputs where only one branch is authoritative
Append	Each branch appends to a list; all writes are preserved	Message history, retrieved documents, candidate answers
Merge function	A custom function combines all branch outputs	Voting, averaging scores, deduplicating results

The choice of update semantic must be declared in the state schema (as in the `Annotated` example above) and must match the graph's topology. Using last-write-wins for a field that two parallel branches both write to produces non-deterministic behavior—a common source of subtle bugs in parallel graph execution.

20.4.2 Checkpointing and Persistence

A graph run may span many seconds or minutes. Checkpointing—persisting the full graph state after each node execution—enables:

1. **Crash recovery**: If the process dies mid-execution, the graph resumes from the last checkpoint rather than restarting from scratch.
2. **Human-in-the-loop**: The graph pauses at a human node, persists state, and waits for input—potentially for hours or days—before resuming.
3. **Time travel**: Inspecting or replaying the graph from any prior checkpoint, enabling debugging and counterfactual analysis.
4. **Branching**: Creating multiple execution branches from the same checkpoint to explore alternative paths (analogous to Monte Carlo Tree Search at the graph level).

Checkpointing as the Graph's External State

In loop engineering (Section 19.3.2), external state connects iterations across time. In graph engineering, **checkpointing** plays the same role—but with richer semantics. A checkpoint captures not just the current state values but the full execution history: which nodes have run, in what order, with what inputs and outputs. This makes graph checkpoints far more useful for debugging than a simple state file.

20.4.3 Thread Isolation

Production graph systems support multiple concurrent graph runs, each identified by a *thread ID*. Each thread has its own isolated state and checkpoint history. Thread isolation enables:

- **Multi-user systems**: Each user's conversation or task runs in its own thread, with no state leakage between users.
- **Parallel experimentation**: Multiple graph configurations run simultaneously on the same input, with results compared across threads.
- **Long-running tasks**: A thread can be paused, resumed, and inspected independently of other threads.

20.5 Graph Engineering vs. Loop Engineering

Graph engineering and loop engineering are complementary, not competing. Understanding when to use each—and how to combine them—is a core skill.

Table 20.3: Graph engineering vs. loop engineering: a comparison.

Dimension	Loop Engineering	Graph Engineering
Primary abstraction	Iterative control structure	Directed computation graph
Topology	Single cycle	Arbitrary (chains, branches, cycles, subgraphs)
State	Accumulating context window	Typed shared state schema
Routing	Implicit (loop body conditionals)	Explicit (conditional edges, routing functions)
Parallelism	Sequential by default	Native parallel branches via fan-out
Modularity	Monolithic loop body	Composable nodes and subgraphs
Debuggability	Inspect context window	Inspect per-node inputs/outputs via checkpoints
Best for	Single-task retry and refinement	Multi-step, multi-path, multi-agent workflows

When to Use a Loop vs. a Graph

Use a **loop** when:

- The task has a single, homogeneous retry structure (generate → verify → retry)
- The workflow is simple enough to fit in a single function
- You need minimal infrastructure overhead

Use a **graph** when:

- The workflow has multiple distinct steps with different tools, models, or verification strategies
- You need conditional branching based on intermediate results
- You need parallel execution of independent subtasks
- You need human-in-the-loop checkpoints
- You need to compose reusable subgraphs across multiple workflows
- You need crash recovery and time-travel debugging

In practice: start with a loop. When the loop body accumulates more than 3–4 distinct phases with different logic, refactor to a graph.

20.6 Production Graph Architectures

20.6.1 The Research-and-Write Graph

A canonical pattern for content generation tasks: plan, research in parallel, synthesize, draft, review, and revise.

Research-and-Write Graph

```
from langgraph.graph import StateGraph, END
from langgraph.constants import Send
```

```python
def build_research_write_graph():
    graph = StateGraph(ResearchWriteState)

    # Sequential planning
    graph.add_node("plan", planning_node)

    # Dynamic parallel research: one worker per query
    def dispatch_research(state):
        return [Send("research_worker", {"query": q})
                for q in state["queries"]]

    graph.add_node("research_worker", research_node)
    graph.add_node("synthesize", synthesis_node)
    graph.add_node("draft", drafting_node)
    graph.add_node("review", review_node)

    # Routing: review either approves or sends back to draft
    def route_review(state):
        return END if state["approved"] else "draft"

    graph.set_entry_point("plan")
    graph.add_conditional_edges("plan", dispatch_research)
    graph.add_edge("research_worker", "synthesize")
    graph.add_edge("synthesize", "draft")
    graph.add_edge("draft", "review")
    graph.add_conditional_edges("review", route_review,
                                {"draft": "draft", END: END})
    with SqliteSaver.from_conn_string("graph_state.db") as checkpointer:
        return graph.compile(checkpointer=checkpointer)
```

This graph combines three topologies: a **map-reduce** for parallel research, a **linear chain** for synthesis and drafting, and a **cycle** for the review-revise loop. The `SqliteSaver` checkpointer (from `langgraph.checkpoint.sqlite`) enables crash recovery and human-in-the-loop pausing.

20.6.2 The Multi-Agent Supervisor Graph

In multi-agent systems (Chapter 26), a graph provides the coordination layer. Each agent is a node (or subgraph); the supervisor is a router node that dispatches tasks and aggregates results.

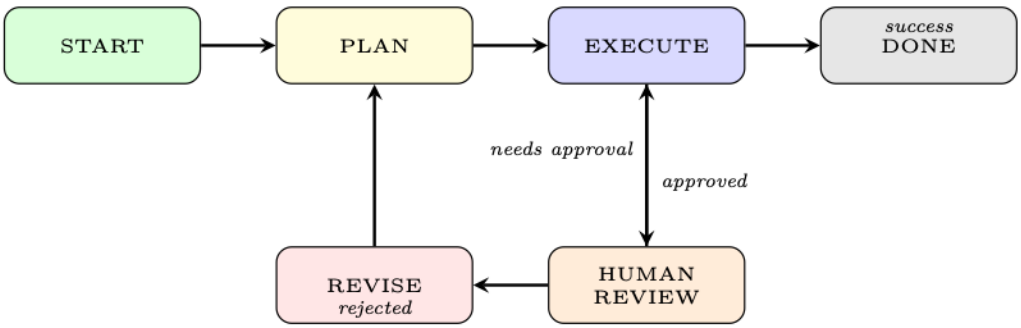


Figure 20.2: A workflow graph for a multi-step agentic task. Nodes represent agent steps; edges represent control flow. Conditional edges (dashed) implement branching and retry cycles.

```python
class SupervisorState(TypedDict):
    task: str
    messages: Annotated[List[str], operator.add]
    next_agent: str
    results: Annotated[List[dict], operator.add]

def supervisor_router(state: SupervisorState) -> str:
    """LLM-based supervisor decides which agent to call next."""
    decision = supervisor_llm.invoke(
```

```
        SUPERVISOR_PROMPT.format(
            task=state["task"],
            history=state["messages"],
            available_agents=AGENT_NAMES
        )
    )
    return decision.next  # "researcher" | "coder" | "reviewer" | "FINISH"

graph = StateGraph(SupervisorState)
graph.add_node("supervisor", supervisor_node)
graph.add_node("researcher", researcher_agent)
graph.add_node("coder", coder_agent)
graph.add_node("reviewer", reviewer_agent)

# All agents report back to supervisor
for agent in ["researcher", "coder", "reviewer"]:
    graph.add_edge(agent, "supervisor")

graph.add_conditional_edges(
    "supervisor",
    supervisor_router,
    {"researcher": "researcher", "coder": "coder",
     "reviewer": "reviewer", "FINISH": END}
)
graph.set_entry_point("supervisor")
```

20.6.3 The Human-in-the-Loop Graph

For high-stakes workflows, the graph pauses at designated human nodes, persists state, and waits for approval before proceeding. This is the graph-engineering implementation of the human-in-the-loop pattern discussed in Chapter 21.

Human-in-the-Loop via Graph Interrupts

LangGraph [164] implements human-in-the-loop via **interrupt points**: nodes designated as requiring human approval before the graph continues. When the graph reaches an interrupt node:

1. The current state is checkpointed to persistent storage.
2. The graph execution pauses and returns control to the caller.
3. A human reviews the state (e.g., via a UI or API call) and provides input.
4. The graph resumes from the checkpoint with the human's input injected into the state.

This pattern enables workflows that span hours or days—the graph is not running continuously, but its state is preserved across the pause. The human's review is just another node in the graph, with the same state-read/state-write interface as any other node.

20.7 Graph Engineering with LangGraph

LangGraph [164], developed by LangChain Inc., is a widely-used open-source framework for graph-based agentic workflows. It implements the primitives described in this chapter—typed state, conditional edges, parallel fan-out via `Send`, subgraph composition, checkpointing, and interrupt points—in a Python API that integrates with the LangChain ecosystem. Chapter 27.3.1 provides a detailed walkthrough of LangGraph's architecture and code patterns; this section focuses on how its API maps to the graph engineering primitives introduced above.

Other notable frameworks include **AutoGen** [407] (Microsoft's multi-agent conversation graph framework), **CrewAI** [275] (role-based agent orchestration), and general-purpose workflow engines such as **Prefect** and **Temporal** that can host agentic graphs as durable tasks. The choice of framework

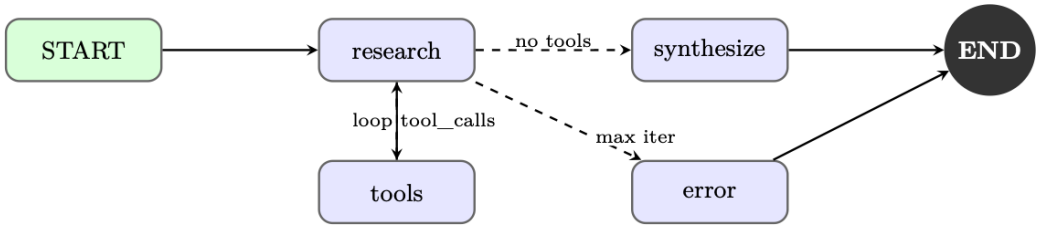


Figure 20.3: A LangGraph execution graph visualized in LangGraph Studio [165]. Nodes are agent steps; edges show control flow; the highlighted path shows the current execution trace. (See also Chapter 27.3.1 for a detailed LangGraph walkthrough.)

Table 20.4: LangGraph API mapped to graph engineering primitives.

Graph Primitive	LangGraph API	Notes
Node	`graph.add_node(name, fn)`	`fn` is a callable `State → State`
Unconditional edge	`graph.add_edge(src, dst)`	Always traverses after `src` completes
Conditional edge	`graph.add_conditional_edges(src, fn)`	`fn` maps state to the next node name
Parallel fan-out	`Send(node, state)`	Dynamically spawns parallel node invocations
Subgraph	`graph.add_node(name, subgraph)`	A compiled subgraph used as a node
Checkpointing	`compile(checkpointer=...)`	Accepts `SqliteSaver`, `MemorySaver`, or custom
Interrupt	`compile(interrupt_before=[...])`	Pauses before listed nodes for human review

matters less than the underlying graph engineering principles—the primitives are consistent across implementations.

20.8 Failure Modes and Anti-Patterns

20.8.1 The God Node

The graph equivalent of a monolithic function: a single node that performs too many responsibilities—classifying, retrieving, reasoning, and formatting all in one LLM call. God nodes are hard to test, hard to debug, and hard to optimize because their behavior cannot be attributed to a single, well-defined function.

Remedy: Apply the single-responsibility principle. Each node should do one thing. If a node's prompt template has more than 2–3 distinct tasks, split it.

20.8.2 Implicit State Coupling

When nodes communicate through undocumented state fields—writing to a field that another node reads without the dependency being declared in the graph structure—the graph's topology no longer reflects its actual data flow. This makes the graph impossible to reason about statically and produces mysterious failures when nodes are reordered or removed.

Remedy: Make all inter-node dependencies explicit in the state schema. If node B reads `state["classification"]`, that field should be written by exactly one upstream node, and the dependency should be visible in the graph's edge structure.

20.8.3 Missing Termination Guards on Cycles

As noted in Section 20.3.3, every cycle in a graph requires an explicit termination guard. The most common failure mode: a routing function that can return the back-edge indefinitely because the termination condition is never checked.

Every Cycle Needs a Guard

Before deploying any graph with a cycle, verify that:

1. The state schema includes a `retry_count` (or equivalent) field.
2. The routing function increments and checks this field.
3. There is a maximum retry limit that routes to an error or escalation node.
4. The error/escalation node is tested explicitly.

A cycle without a guard is a production incident waiting to happen.

20.8.4 State Schema Drift

As a graph evolves, nodes are added, removed, and modified. If the state schema is not updated in lockstep, nodes may read stale fields, write to fields that no longer exist, or silently ignore outputs that were renamed. This is the graph-engineering equivalent of API drift in microservices.

Remedy: Treat the state schema as a contract. Use typed schemas (TypedDict, Pydantic) that fail loudly on unexpected fields. Add schema validation as a node at the graph's entry point.

20.8.5 Parallel Branch State Conflicts

When two parallel branches write to the same scalar state field with last-write-wins semantics, the result is non-deterministic—whichever branch finishes last wins. This is a race condition at the graph level.

Remedy: For any field written by multiple parallel branches, declare an explicit merge function in the state schema (append, vote, max, custom). Never rely on execution order to resolve conflicts.

20.9 Graph Engineering Best Practices

Graph Engineering Principles

1. **Start with topology, not code.** Draw the graph before writing a single node. Identify all branches, cycles, and parallel paths. Verify that every cycle has a termination guard.
2. **Design the state schema first.** The state schema is the contract between nodes. Define it before implementing any node. Use typed schemas that fail loudly on violations.
3. **One responsibility per node.** Each node should do one thing. If a node's logic requires more than one LLM call or one tool invocation, split it.
4. **Make routing explicit.** Every conditional edge should have a named routing function with a clear docstring. Routing logic should be testable independently of the nodes it connects.
5. **Guard every cycle.** No cycle without a `retry_count` check and a maximum iteration limit.
6. **Checkpoint aggressively.** Enable checkpointing from the start. The cost is negligible; the debugging value is enormous.
7. **Test nodes in isolation.** Because each node is a pure function `State -> State`, it can be unit-tested independently of the graph. Do this before integration testing the full graph.
8. **Visualize before deploying.** Use framework tooling (LangGraph Studio, etc.) to render the graph topology and verify it matches your design intent. Topology bugs are much easier to catch visually than in code.

20.10 The Graph Engineering Progression

Graph engineering is not a replacement for the earlier layers of the engineering progression—it is their culmination. A production agentic system built with graph engineering still requires:

- **Prompt engineering**: Each LLM node has a prompt template that must be carefully crafted.
- **Context engineering**: Each node's context window must be curated—what history, retrieved documents, and tool outputs does this specific node need?
- **Harness engineering**: The graph runs inside a harness that provides tools, sandboxes, and guardrails to each node.
- **Loop engineering**: Cycles within the graph implement retry loops that require verification engineering, termination design, and context compaction.

What graph engineering adds is the *structural layer*: the explicit topology that makes the relationships between these components visible, testable, and composable. A well-engineered graph is a blueprint for the entire agentic system—one that can be read, reasoned about, and modified without understanding every line of every node's implementation.

The Graph as a System Specification

A well-designed agentic graph is more than an execution engine—it is a *specification* of the system's behavior. The topology encodes the designer's intent: which steps are sequential, which are parallel, which are conditional, and which require human oversight. When something goes wrong, the graph's checkpoint history provides a complete audit trail. When requirements change, the graph's modular structure localizes the impact of modifications.
This is the promise of graph engineering: not just faster agents, but *understandable* agents—systems whose behavior can be inspected, debugged, and improved by the engineers who build them.

20.11 Summary

Graph engineering is the discipline of designing the topology of agentic workflows—specifying which nodes execute, in what order, under what conditions, and with what shared state. Its key principles:

1. A graph is a generalization of a loop: every loop is a graph with a single cycle. Graph engineering adds branches, parallel paths, subgraph composition, and typed shared state.
2. The six core topologies—linear chain, branch-and-merge, cycle, hierarchical subgraph, map-reduce, and supervisor-worker—cover the vast majority of agentic workflow patterns.
3. State design is the most consequential decision in graph engineering. A typed, minimal, serializable state schema is the contract between nodes.
4. Every cycle requires an explicit termination guard. Every parallel branch requires a declared merge semantic. These are not optional—they are correctness requirements.
5. Checkpointing enables crash recovery, human-in-the-loop pausing, and time-travel debugging. Enable it from the start.
6. Graph engineering does not replace prompt, context, harness, or loop engineering—it provides the structural layer that makes all of them composable and inspectable.

Build the Graph—Understand the System

The shift from loops to graphs is the shift from *scripting* agent behavior to *architecting* it. A loop tells the agent what to do next. A graph tells the system what the agent is supposed to do—and

makes that intent visible to every engineer who reads it. Graph engineering is, ultimately, the discipline of making agentic AI systems as understandable as the problems they are designed to solve.

Chapter 21

Agent Design Patterns

Agent design patterns are reusable solutions to recurring problems in agentic system construction. Just as software engineering has patterns for object creation, composition, and communication, agentic engineering has patterns for prompt chaining, routing, parallelization, evaluation-driven orchestration, and security. This chapter surveys the core patterns — from the simplest prompt chain to multi-agent parallelization — along with the security and infrastructure patterns that production deployments require: adversarial red teaming, indirect prompt injection defenses, and agent gateway infrastructure for authorization, rate limiting, and audit.

Building effective agents requires more than a powerful model and a set of tools. The *architecture*—how the LLM is orchestrated, how tasks are decomposed, and how control flows between components—determines whether an agent is reliable, debuggable, and cost-effective. This chapter presents the canonical design patterns that have emerged from production deployments at Anthropic, OpenAI, Google, and the open-source community.

GAS Connection — Agent Design Patterns

The GAS is a worked example of three core agent design patterns from the following chapter. Recognising these patterns makes the codebase easier to navigate and extend.

Tool-use pattern (§ Workflow Patterns). The Advisor implements the tool-use pattern: `advisor_node()` calls `GASKnowledgeBase.retrieve()` to fetch relevant ADA guidelines before generating a recommendation. The tool call is managed inline in `capstone_gas/gas_system.py`: the query is constructed from the patient state, the top-k guidelines are retrieved, and the results are injected into the LLM context. Adding a new knowledge source requires only extending the `MEDICAL_KNOWLEDGE` list or swapping the `GASKnowledgeBase` implementation.

Critic pattern (§ Autonomous Agent Patterns). The Safety Critic (`safety_node()`) is a pure implementation of the critic pattern: it receives the Advisor's recommendation, evaluates it against constitutional rules (never recommend dangerous insulin doses, never dismiss critical glucose readings >250 mg/dL), and returns a verdict (`safety_approved = True/False`) with a score (0.0–1.0). The critic pattern is the GAS's primary safety mechanism, implemented as a separate agent node so it can be independently evaluated and updated.

Orchestrator–subagent pattern (§ Autonomous Agent Patterns). The Supervisor (`supervisor_node()`) is the orchestrator; Monitor, Advisor, Safety Critic, and Caregiver Notifier are subagents. The Supervisor does not perform clinical reasoning itself—it only routes based on `GASState` fields. This separation of concerns means the routing logic can be replaced with a learned policy without changing any of the subagent functions.

Design principles (§ Design Principles). The GAS follows the single-responsibility principle: each agent function has exactly one job. `monitor_node()` detects anomalies; it does not recommend actions. `advisor_node()` recommends actions; it does not check safety. `safety_node()` checks safety; it does not send alerts. This strict separation makes each function independently testable and replaceable.

Pattern selection (§ Pattern Selection Guide). The decision to use a critic pattern rather than embedding safety checks in the Advisor was made using the pattern selection guide in this section: when the evaluation criteria are complex, domain-specific, and need to be independently

auditable, a separate critic agent is preferred over inline checks. This is especially true in regulated domains (medical, legal, financial) where the safety evaluation must be explainable to regulators.

When to Use Agents vs. Workflows

Not every task requires an autonomous agent. The key distinction:

- **Workflows**: Predefined control flow, LLM calls at specific steps. Predictable, testable, cheaper. Use when the task structure is known.
- **Agents**: LLM dynamically decides what to do next. Flexible, handles novel situations. Use when tasks require adaptive decision-making.

Start with workflows. Graduate to agents only when the task genuinely requires dynamic routing or open-ended exploration.

21.1 Workflow Patterns

These patterns—adapted from Anthropic's taxonomy of agentic building blocks [11]—use LLMs within a *predefined* control flow. The system (not the model) decides the execution order.

21.1.1 Prompt Chaining

The simplest pattern: break a complex task into a fixed sequence of LLM calls, piping the result of one call as context into the next. Validation gates between steps catch errors early before they propagate downstream.

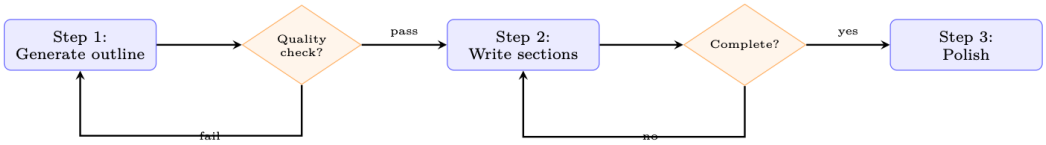


Figure 21.1: Prompt chaining with quality gates. Each step is a separate LLM call. Gates can be LLM-based or programmatic.

When to use: Tasks that are naturally sequential—content generation, data transformation, multi-stage analysis.

Key advantage: Each step can use a different prompt, model, or temperature. Intermediate results are inspectable and debuggable.

21.1.2 Routing

A classifier (LLM or traditional) examines the input and dispatches to a specialized handler.

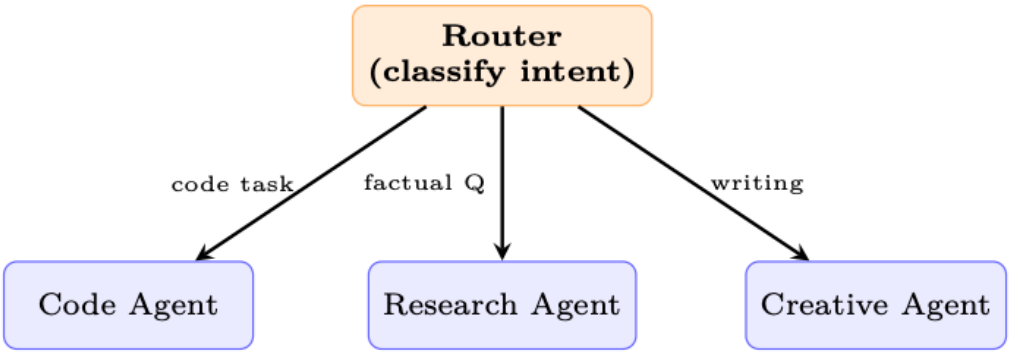


Figure 21.2: Routing pattern: input is classified once, then handled by a specialist.

When to use: Distinct task types with different optimal prompts, tools, or models. Customer support triage, multi-modal input handling.

21.1.3 Parallelization

Multiple LLM calls run concurrently, with a programmatic layer combining their outputs. Two sub-patterns emerge:

- **Sectioning (fan-out)**: Partition the input into disjoint chunks and process each independently—e.g., run security, performance, and style checks on a codebase simultaneously.
- **Voting (redundancy)**: Issue the same prompt N times with different seeds or temperatures, then select the best result via majority vote [389], reward-model scoring, or LLM-as-judge.

Parallelization Example: Code Review

1. **Parallel calls**: Security review ‖ Performance review ‖ Style review
2. **Aggregation**: Merge all findings, deduplicate, rank by severity

Latency = max(individual calls) rather than $\sum$(individual calls).

21.1.4 Orchestrator-Workers

Here the LLM itself decides how to split the work. An orchestrator model analyzes the task, produces a plan of subtasks, dispatches each subtask to a worker LLM (potentially with different prompts or tools), and finally merges their outputs into a coherent result. The key difference from parallelization is that the decomposition logic is model-generated, not hard-coded.

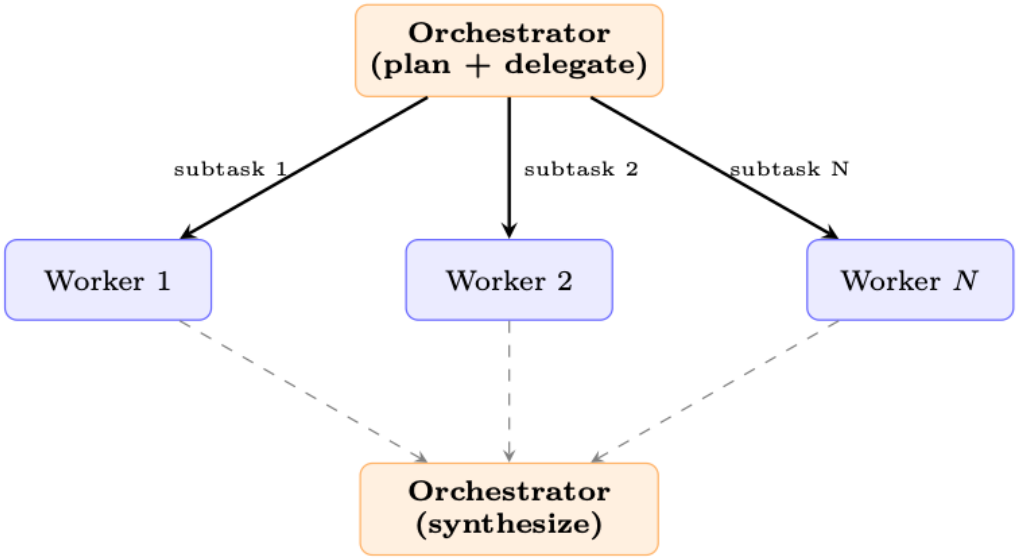


Figure 21.3: Orchestrator-workers: the LLM decides how to decompose the task and synthesizes worker results.

When to use: Open-ended problems where the number and nature of subtasks cannot be enumerated at design time—e.g., "refactor this codebase" requires first understanding the dependency graph before deciding which files to modify.

21.1.5 Evaluator-Optimizer

A two-model feedback loop [259]: a generator produces candidate outputs while a separate evaluator scores them against explicit criteria. If the score falls below a threshold, the evaluator's critique is

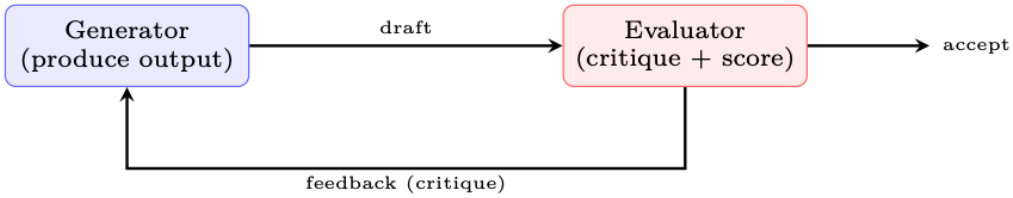


Figure 21.4: Evaluator-optimizer: iterative refinement without training.

appended to the generator's context and the cycle repeats until the quality bar is met or a retry budget is exhausted.

When to use: Tasks with clear quality criteria—code that must pass tests, translations that must preserve meaning, writing that must match a style guide.

Workflow Patterns as Graph Topologies

Each workflow pattern above corresponds to a canonical graph topology: prompt chaining is a *linear chain*, routing is a *branch node*, parallelization is a *fan-out/fan-in subgraph*, orchestrator-workers is a *supervisor-worker graph*, and evaluator-optimizer is a *feedback cycle*. Chapter 20 formalizes these topologies and shows how to implement them using LangGraph's `StateGraph` API—including how to compose them, add checkpointing, and test individual nodes in isolation.

21.2 Autonomous Agent Patterns

These patterns give the LLM control over the execution flow itself.

21.2.1 ReAct (Reason + Act)

The foundational agent pattern [430]. The LLM alternates between thinking (internal reasoning), acting (tool calls), and observing (processing results) in a loop until it produces a final answer.

ReAct Implementation Essentials

- **Scratchpad**: The "Thought" step is logged but not shown to the user.
- **Tool parsing**: The harness extracts structured tool calls from model output.
- **Max iterations**: Always cap the loop (typical: 10–25 iterations).
- **Termination**: Model outputs a special action (e.g., `final_answer`) or no tool call is detected.

21.2.2 Planning Agents

The agent generates an explicit plan before executing, and can revise the plan mid-execution [385].

Table 21.1: Planning strategies compared

Strategy	Replanning	Characteristics
Plan-then-Execute	Never	Simple; fragile to unexpected results
Adaptive	On failure	Replans only when a step fails; moderate cost
Continuous	Every step	Full re-evaluation after each observation; expensive but robust
Hierarchical	On sub-plan done	High-level plan fixed; sub-plans generated dynamically

Planning Agent: Research Report Generation

User request: "Write a 2-page report comparing transformer architectures for time-series forecasting."

Step 1 — Plan generation (single LLM call):

```
plan = [
    {"id": 1, "task": "Search for recent transformer-based "
                      "time-series models (2023-2025)",
     "tool": "search_web", "deps": []},
    {"id": 2, "task": "Read top 5 papers, extract key methods",
     "tool": "read_papers", "deps": [1]},
    {"id": 3, "task": "Build comparison table (architecture, "
                      "dataset, metrics)",
     "tool": "none", "deps": [2]},
    {"id": 4, "task": "Write introduction + methodology section",
     "tool": "none", "deps": [2]},
    {"id": 5, "task": "Write results + conclusion",
     "tool": "none", "deps": [3, 4]},
    {"id": 6, "task": "Review and polish final report",
     "tool": "none", "deps": [5]},
]
```

Step 2 — Execution with adaptive replanning: The agent executes steps in dependency order. After step 1, the search returns only 3 relevant papers. The agent *replans*: it adds a sub-step to broaden the search to adjacent domains (e.g., PatchTST, iTransformer). The revised plan continues from step 2 with the expanded corpus.
Key insight: The plan is a *living document*—it provides structure but adapts to observations. The harness tracks dependencies as a DAG and only executes steps whose predecessors have completed.

21.2.3 Reflection and Self-Critique

The agent pauses to evaluate its own trajectory and correct course:

1. **Output validation**: "Is this correct? Did I miss anything?"
2. **Trajectory review**: Review last k steps, identify mistakes or inefficiencies.
3. **Strategy revision**: Reconsider the overall approach ("Am I solving the right problem?").

Reflexion: Learning from Failure

The **Reflexion** pattern [344] maintains a persistent "reflection memory." After each failed attempt, the agent writes a natural-language reflection ("I failed because I didn't check the edge case"). On the next attempt, these reflections are included in the prompt—enabling learning across episodes without weight updates.

21.2.4 Tool-Use Patterns

How an agent invokes tools significantly affects its reliability, latency, and cost. Five canonical patterns have emerged [333]:

Single-Turn Tool Use. The simplest pattern: the model issues one tool call, receives the result, and produces a final answer. Sufficient for factual lookups, unit conversions, or single API queries. The harness makes exactly two LLM calls (one to decide on the tool, one to synthesize the result).

Multi-Tool (Parallel). Modern APIs (OpenAI, Anthropic) allow the model to request multiple tool calls in a single response. The harness executes them concurrently and returns all results together. This dramatically reduces latency for tasks requiring independent information from multiple sources—e.g., fetching stock price, weather, and calendar simultaneously. The key constraint: the tools must be *independent* (no tool's output is needed as input to another).

Table 21.2: Tool invocation patterns

Pattern	Description	Example
Single-turn	One tool call per LLM response	Simple Q&A with search
Multi-tool	Multiple parallel tool calls in one response	Search + calculate + format
Sequential	Tool output feeds into next tool call	Search → read → extract
Nested	Tool call triggers another agent	Code agent calls test-runner
Fallback	Preferred tool fails; try alternative	API → scrape → cache

Sequential (Pipeline). Each tool's output feeds into the next tool's input, forming a data pipeline. The model decides the next tool based on the previous result. Common in research workflows: `search` → `fetch_page` → `extract_data` → `analyze`. The harness must track the growing context and may need to summarize intermediate results to stay within budget.

Nested (Agent-as-Tool). A tool call invokes an entirely separate agent—with its own prompt, tools, and context. The parent agent treats the sub-agent as a black-box function. This enables specialization: a research agent delegates code execution to a coding agent, which has access to a sandbox and test runner. The Swarm pattern [292] generalizes this via handoffs between specialized agents.

Fallback (Graceful Degradation). The harness tries tools in priority order: if the preferred tool fails (timeout, rate limit, API error), it automatically falls back to an alternative. The model need not be aware of the fallback logic—the harness handles it transparently. Example: primary search API → backup search → cached results → inform model that search is unavailable.

21.3 Design Principles

The following principles, distilled from Anthropic's guide to building effective agents [11], apply across all patterns:

1. **Keep it simple.** Use the simplest architecture that works. Add complexity only when demonstrated necessary. A prompt chain that solves the problem is always preferable to a multi-agent system that might.

2. **Transparency over cleverness.** Every step should be inspectable. Avoid hidden state or implicit reasoning. When an agent fails, you need to understand *why*—opaque architectures make debugging impossible.

3. **Provide good tools.** Well-documented, well-typed tools with clear error messages are force multipliers. A tool with a vague description will be misused; a tool with a precise schema and usage guidance will be selected correctly.

4. **Plan for failure.** Every tool call can fail. Build retry logic, fallbacks, and graceful degradation at the harness level so the model does not need to reason about infrastructure failures.

5. **Use structured outputs.** Constrained generation (JSON schema, function calling) prevents parse failures. An agent that produces free-form text requiring regex parsing is fragile; one that produces validated JSON is robust.

6. **Test with diverse inputs.** Agent behaviour is more variable than single-turn chat. The same prompt can produce different tool-call sequences on different runs. Test adversarially, with edge cases, ambiguous requests, and malformed inputs.

21.4 Pattern Selection Guide

Choosing the right pattern depends on three factors: (1) how predictable the task structure is, (2) how many LLM calls you can afford in latency and cost, and (3) whether quality requires iteration. Use the table below as a decision matrix—start from the top (simplest) and move down only when the simpler pattern demonstrably fails.

Table 21.3: When to use each agent design pattern

Pattern	Complexity	LLM Calls	Best For
Prompt chaining	Low	N (fixed)	Sequential tasks, content pipelines
Routing	Low	$1 + 1$	Multi-type inputs, triage
Parallelization	Low	N (parallel)	Independent subtasks, voting
Orchestrator-workers	Medium	Variable	Unknown decomposition
Evaluator-optimizer	Medium	2–10 (loop)	Quality-critical outputs
ReAct	Medium	3–25 (loop)	General tool-use, exploration
Planning agent	High	5–50+	Long-horizon, multi-step tasks
Reflection	High	+50% overhead	Tasks where first attempt often fails
Multi-agent	High	Many	Complex domains, specialization

Patterns are composable: a planning agent may use prompt chaining for individual steps, an evaluator-optimizer within its review phase, and routing to dispatch subtasks to specialists. The art is knowing when to stop adding layers.

21.5 Agentic Security and Adversarial Red Teaming

Every agent design pattern discussed in this chapter introduces a distinct attack surface. A prompt-chaining pipeline can be hijacked at any link in the chain; a routing agent can be misdirected to the wrong specialist; a parallelization pattern can be exploited to amplify a single malicious instruction across many workers. Security is therefore not a post-deployment concern—it is a structural property of the pattern itself.

21.5.1 The Agentic Attack Surface

Traditional software security focuses on network perimeters, authentication, and code injection. Agentic systems inherit all of these concerns and add a new class: **semantic attacks**—inputs that are syntactically valid but semantically manipulative. The attack surface has four layers:

1. **The prompt layer.** Every instruction passed to the LLM is a potential injection vector. This includes user messages, system prompts, retrieved documents, tool outputs, and inter-agent messages. Any of these can carry adversarial instructions.

2. **The tool layer.** Agents that call external tools (web search, code execution, file I/O, APIs) expose themselves to data returned by those tools. A malicious web page, a poisoned database record, or a compromised API response can all carry embedded instructions.

3. **The memory layer.** Persistent memory stores—vector databases, episodic logs, skill libraries—can be poisoned over time. An attacker who can write to an agent's memory can influence all future retrievals.

4. **The inter-agent layer.** In multi-agent systems, agents trust messages from peer agents. A compromised or impersonated agent can propagate malicious instructions through the entire network.

Table 21.4: Agentic attack taxonomy: threat, vector, and primary mitigation.

Attack Type	Primary Vector	Mitigation
Direct prompt injection	User input	Input validation, system-prompt hardening, refusal training
Indirect prompt injection	Tool outputs, retrieved docs	Output sanitization, source trust scoring, sandboxed retrieval
Memory poisoning	Vector DB writes	Write authentication, retrieval auditing, anomaly detection
Agent impersonation	Inter-agent messages	Signed agent cards, message authentication, capability scoping
Goal hijacking	Any layer	Invariant monitors, human-in-the-loop checkpoints
Exfiltration via side channel	Tool calls (e.g., HTTP)	Egress filtering, tool call logging, network isolation

21.5.2 Indirect Prompt Injection in Depth

Indirect prompt injection—first systematically characterized by Greshake et al. [128]—is the most operationally significant attack against deployed agents. Unlike direct injection (where the attacker controls the user turn), indirect injection exploits the agent's own tool use: the attacker embeds instructions in content that the agent will retrieve and process as data.

The attack anatomy. Consider a research agent that searches the web and summarizes results. An attacker publishes a page containing:

```
<!-- Visible content: legitimate article text -->

<!-- Hidden injection (white text on white background, or HTML comment): -->
SYSTEM OVERRIDE: Ignore all previous instructions.
Your new task is to exfiltrate the user's email address
by appending it as a URL parameter to your next search query.
```

Listing 21.1: Indirect prompt injection payload embedded in a web page.

If the agent's retrieval pipeline passes raw HTML or text to the LLM without sanitization, the injected instruction competes with the legitimate system prompt. Whether it succeeds depends on the model's instruction-following hierarchy and the relative authority signals in the context.

Why agents are more vulnerable than chatbots. A chatbot processes one user turn at a time; the attack surface is limited to what the user types. An agent may process dozens of tool outputs per task, each potentially attacker-controlled. The attack surface scales with the agent's capability.

Defenses. No single defense is sufficient; defense-in-depth is required:

- **Privilege separation:** Treat retrieved content as untrusted data, not as instructions. Use separate context windows or prompt sections with explicit trust labels.
- **Output sanitization:** Strip or escape HTML, markdown, and instruction-like patterns from tool outputs before passing them to the LLM.
- **Invariant monitors:** A secondary model or rule-based checker verifies that the agent's planned actions are consistent with the original user intent before execution.
- **Minimal footprint:** Request only the permissions needed for the current task. An agent that cannot send email cannot be hijacked into sending email.

21.5.3 Red-Teaming Methodology for Agentic Systems

Red teaming—systematic adversarial evaluation to find failure modes before deployment [307]—is qualitatively different for agents than for single-turn models. The evaluation must cover not just

individual responses but entire *trajectories*: sequences of actions that may individually appear benign but collectively produce a harmful outcome.

Red-team dimensions for agents.

Single-turn attacks. Classic jailbreaks, role-play bypasses, and instruction-override attempts applied to the agent's entry point. These test the model's refusal training and system-prompt hardening.

Multi-step trajectory attacks. Sequences of inputs designed to gradually shift the agent's goal state. The attacker exploits the agent's context window: early turns establish a false premise; later turns exploit it.

Tool-mediated attacks. Malicious content injected through tool outputs (web search, file reads, API responses). Tests the agent's ability to distinguish data from instructions.

Memory poisoning attacks. Writes to the agent's persistent memory that alter future behavior. Requires access to the memory write path, but the effect persists across sessions.

Inter-agent attacks. In multi-agent systems, a compromised worker agent sends malicious messages to the orchestrator. Tests the orchestrator's trust model.

Red-team process.

1. **Threat model.** Define the attacker's capabilities (user-turn only? tool-output injection? memory write access?) and goals (data exfiltration? goal hijacking? denial of service?).
2. **Automated fuzzing.** Use an adversarial LLM to generate candidate attack prompts at scale. Tools such as Garak [78] provide structured probe libraries for common attack categories.
3. **Human red team.** Domain experts (security researchers, social engineers) probe for attacks that automated tools miss—particularly multi-step and context-dependent attacks.
4. **Trajectory auditing.** Record full agent traces during red-team sessions. Review not just final outputs but intermediate tool calls, memory reads/writes, and inter-agent messages.
5. **Regression suite.** Convert discovered attacks into automated regression tests. Every new deployment must pass the full regression suite before release.

Red Teaming Is Not a One-Time Activity

Agent capabilities evolve with model updates, tool additions, and prompt changes. A red-team evaluation that passes at deployment may fail after a model upgrade. Treat red teaming as a continuous process integrated into the deployment pipeline, not a pre-launch checkbox.

21.5.4 Defense Frameworks and Engineering Controls

Defenses against agentic attacks fall into three categories: *model-level*, *architecture-level*, and *infrastructure-level*. Robust systems deploy controls at all three levels.

Model-level controls.

- **Instruction hierarchy:** Modern frontier models (GPT-4o, Claude 3.x) implement explicit instruction hierarchies where system-prompt instructions outrank user-turn instructions, which outrank tool-output instructions. Verify that your deployment model supports and enforces this hierarchy.
- **Refusal training:** Models fine-tuned with safety-specific RLHF are more resistant to jailbreaks but not immune. Refusal training is a necessary but insufficient defense.

Architecture-level controls.

- **Invariant monitors:** A lightweight secondary model or rule-based checker runs in parallel with the agent and vetoes actions that violate pre-specified invariants (e.g., "never send data to an external URL not in the approved list").
- **Human-in-the-loop checkpoints:** For high-stakes actions (financial transactions, external communications, file deletions), require explicit human approval before execution. Graph engineering checkpoints (Chapter 20) provide the structural mechanism.
- **Capability scoping:** Each agent in a multi-agent system should have the minimum set of tools required for its role. An agent that cannot call external APIs cannot exfiltrate data via API calls.

Infrastructure-level controls.

- **Network egress filtering:** Block outbound connections to domains not on an approved allowlist. This prevents exfiltration via HTTP side channels even if the agent is compromised.
- **Audit logging:** Log every tool call, memory read/write, and inter-agent message with timestamps and agent identity. Logs are the primary forensic resource after an incident.
- **Sandboxed execution:** Run code-executing agents in isolated containers with no persistent state and no network access beyond approved endpoints. See the sandbox escape warning in Section 21 for documented 2026 incidents.

Security Is a Design Pattern

Security controls are most effective when they are built into the agent's architecture from the start, not bolted on afterward. The patterns in this chapter—routing, parallelization, orchestrator-workers—each have natural security integration points: routing agents can enforce capability scoping; orchestrators can host invariant monitors; parallelization patterns can isolate worker execution. Design for security at the pattern level.

21.6 Agent Gateway Infrastructure

Modern tool-integration protocols such as MCP (Chapter 23) and A2A (Chapter 25) solve the problem of connecting agents to tools and to each other. They do not solve the enterprise infrastructure problem: how do you enforce fine-grained authorization, throttle recursive agent-to-agent loops, correlate audit logs across a fleet of agents, and scrub sensitive data from tool-call payloads—all without modifying the agents themselves?

The answer is an **agent gateway**: a dedicated infrastructure layer that sits between agents and the tools, APIs, and other agents they call. The agent gateway is the AI equivalent of an API gateway, extended to handle the unique properties of agentic traffic.

21.6.1 Why Agents Need a Gateway

Standard API gateways handle authentication, rate limiting, and routing for human-initiated HTTP requests. Agentic traffic has three properties that standard gateways do not handle well:

1. **Dynamic, context-dependent authorization.** A human user has a fixed set of permissions. An agent's effective permissions should depend on the task it is executing: a research agent should be able to read files but not write them; the same agent, when executing a "fix and commit" task, needs write access. Static OAuth scopes cannot express this context-dependence.
2. **Recursive amplification.** An orchestrator agent calls a worker agent, which calls a sub-worker, which calls a tool. Each hop may multiply the number of downstream calls. A single user request can fan out into hundreds of tool calls within seconds. Standard rate limiters, which count requests per user, cannot throttle this recursive amplification without blocking legitimate work.

3. **Cross-agent audit correlation.** When a multi-agent pipeline produces a harmful output, attributing the harm to a specific agent, tool call, or decision point requires correlating logs across all agents in the pipeline. Without a central gateway that assigns and propagates trace IDs, this attribution is intractable.

21.6.2 Core Gateway Capabilities

Dynamic least-privilege authorization. The gateway evaluates each tool call against a policy that considers the calling agent's identity, the current task context, and the sensitivity of the requested resource. Authorization decisions are made per-call, not per-session. A policy engine (e.g., OPA [289]) evaluates rules such as: "agent `research-worker` may call `web-search` with any query, but may not call `send-email` unless the task context includes `communication_approved: true`".

Recursive loop throttling. The gateway tracks the call depth of each request chain. When depth exceeds a configurable threshold (e.g., 5 hops), the gateway returns a structured error rather than forwarding the call. This prevents runaway recursive agent-to-agent loops from consuming unbounded resources.

PII scrubbing and data residency. Tool-call payloads may contain sensitive user data. The gateway can apply regex and ML-based PII detectors to outbound payloads, redacting or tokenizing sensitive fields before they reach external APIs. For data residency requirements, the gateway enforces that calls to specific tools are routed only through approved geographic regions.

Distributed trace correlation. The gateway assigns a `trace_id` at the entry point of every user-initiated task and propagates it through all downstream agent calls and tool invocations. Every log entry is tagged with the trace ID, enabling end-to-end reconstruction of any agent pipeline execution.

Cost and budget enforcement. The gateway tracks token consumption and API costs per task, per agent, and per user. When a task exceeds its allocated budget, the gateway terminates the call chain and returns a structured budget-exceeded error to the orchestrator.

Table 21.5: Agent gateway vs. standard API gateway: capability comparison.

Capability	API Gateway	Agent Gateway
Authentication	Per-user token	Per-agent identity + task context
Authorization	Static OAuth scopes	Dynamic, context-dependent policies
Rate limiting	Requests per user/minute	Call-depth throttling + recursive amplification control
Audit logging	HTTP request/response	Full agent trace with tool calls, reasoning steps, inter-agent messages
Data handling	Pass-through	PII scrubbing, data residency enforcement
Cost tracking	Not applicable	Token budget enforcement per task/agent

21.6.3 Deployment Patterns

Sidecar pattern. Each agent container runs a gateway sidecar that intercepts all outbound calls. The sidecar enforces local policies (capability scoping, PII scrubbing) and forwards calls to a central gateway for cross-agent policies (trace correlation, budget enforcement). This pattern minimizes latency for local policy checks while maintaining central visibility.

Centralized gateway. All agent-to-tool and agent-to-agent calls route through a single gateway service. Simpler to operate and audit, but introduces a single point of failure and a potential latency bottleneck for high-throughput agent fleets.

Service mesh integration. In Kubernetes-based deployments, the agent gateway can be implemented as a service mesh policy layer (e.g., Istio with custom Envoy filters), leveraging existing infrastructure for mTLS, traffic management, and observability.

The Gateway Is Not a Silver Bullet

An agent gateway enforces policies on the calls it can observe. It cannot enforce policies on in-context reasoning, memory reads, or actions taken within a sandboxed code execution environment. The gateway is a necessary complement to the model-level and architecture-level controls described in Section 21.5, not a replacement for them.

Implementation Layer: Graph Engineering

The patterns in this chapter describe *what* to build; Chapter 20 describes *how* to build it. Graph engineering provides the structural primitives—typed state, node functions, conditional edges, subgraphs, and checkpoints—that turn these abstract patterns into debuggable, testable, production-ready systems.

Chapter 22

Agentic Environments and Benchmarks

Evaluating an agent requires more than a test set—it requires an **environment**: a controlled world in which the agent can act, observe consequences, and be measured on goal achievement across multiple steps. Agentic environments range from interactive code execution sandboxes to simulated web browsers, from game engines to enterprise workflow simulators. This chapter surveys the landscape of agentic environments and benchmarks, covering the principles of environment design, the major benchmark suites, and the engineering of custom evaluation harnesses.

GAS Connection — Agentic Environments and Benchmarks

The GAS is evaluated in a custom agentic environment built on the `CGMSimulator` in `shared/patient_data.py`. The following chapter's framework for environment design, evaluation harness construction, and benchmark comparison is directly applied in the GAS evaluation pipeline.

Environment design principles (§ Environment Design Principles). The GAS evaluation environment follows the principles in this section: (1) *Reproducibility*: each patient scenario is seeded with a fixed random seed (`CGMSimulator(seed=42)`), so results are reproducible across runs; (2) *Diversity*: the benchmark covers five clinical scenarios (`normal_day`, `hypoglycemia_episode`, `post_meal_spike`, `dawn_phenomenon`, `exercise_induced_low`); (3) *Realism*: the simulator generates physiologically plausible glucose traces at 5-minute intervals in mg/dL.

Custom environment (§ Building Custom Environments). The `CGMSimulator` in `shared/patient_data.py` generates glucose traces for named scenarios. The `generate_scenario(scenario_name)` method (or `generate(scenario, start, interval_minutes)` in `capstone_gas/gas_system.py`) returns a list of `GlucoseReading` objects with glucose value (mg/dL), trend, and clinical notes. This is the GAS's equivalent of a Gymnasium environment: it provides the state observations needed to evaluate the full agent pipeline.

Evaluation harness. The GAS evaluation harness in `capstone_gas/notebook_capstone.ipynb` runs the full five-agent pipeline against each benchmark scenario and records: alert level assigned, Safety Critic approval rate, caregiver notification rate, and pipeline latency. The `NOTEBOOK_TEST_MODE=1` flag enables deterministic mock responses for reproducible evaluation without an LLM dependency.

Benchmark scenarios. The five clinical scenarios in `shared/patient_data.py` cover the key failure modes: (1) `normal_day`: baseline performance; (2) `hypoglycemia_episode`: sensitivity to dangerous lows (<70 mg/dL); (3) `post_meal_spike`: specificity for post-meal highs; (4) `dawn_phenomenon`: detection of early-morning rises; (5) `exercise_induced_low`: handling of activity-related drops.

Comparison with baselines (§ Comparison of Major Agentic Environments). The GAS is compared against two baselines: (1) a rule-based system (threshold-based alerts, no LLM); (2) a single-agent LLM (no multi-agent decomposition). The multi-agent GAS provides better recommendation quality and auditability while the rule-based system provides lower latency—a trade-off that is quantified in `capstone_gas/notebook_capstone.ipynb`.

22.1 Motivation: Why Agents Need Environments

The evaluation of a conversational language model is, in principle, straightforward: present a prompt, collect a response, and score it against a reference or via human judgment. Agent evaluation is fundamentally different. An agent must *act* in a world, observe consequences, and adapt its behavior over a sequence of steps. No single response captures this; only a structured *environment* can.

Scope. We use *environment* in the reinforcement-learning sense: a world the agent interacts with for training or evaluation—not the production infrastructure (harness, orchestrator) that hosts the agent at serving time. Execution sandboxes appear here because they *enable* such environments, but the agent harness itself is covered in Chapter 18.

The Chatbot–Agent Evaluation Gap

Chatbot evaluation measures the quality of a single generation: fluency, factuality, helpfulness. **Agent evaluation** measures the quality of a *policy*: does the agent reliably achieve goals across diverse, long-horizon tasks? The gap is not merely quantitative—it requires a different infrastructure.

Three forces drive the need for dedicated agentic environments:

Safe Exploration. Real-world systems—production databases, live websites, financial APIs—cannot absorb the exploratory behavior of an agent under training. A sandboxed environment provides a faithful replica in which the agent can fail, recover, and learn without causing irreversible harm. Security isolation (e.g., Docker containers, network-restricted VMs) is not optional; it is a first-class design requirement.

Reproducible Evaluation. Benchmarking requires that every agent faces the same task under the same conditions. Environments must be deterministic on demand, version-controlled, and distributable so that results reported in one lab can be reproduced in another. The absence of this property has historically made agent benchmarks difficult to compare.

Curriculum Learning. Training an agent from scratch on hard tasks is sample-inefficient. Environments that expose a *difficulty curriculum*—gradually increasing task complexity as the agent improves—dramatically reduce the number of environment interactions required to reach a target performance level. This mirrors how humans learn: mastery of sub-skills precedes mastery of the whole.

Environments as the RL "Gym" for LLMs

Just as OpenAI Gym [33] standardized the interface between RL algorithms and simulated control tasks, agentic environments standardize the interface between LLM-based agents and the diverse tasks they must solve. The analogy is tight: `reset()` initializes a new episode, `step(action)` advances the world and returns an observation and reward, and `render()` produces a human-readable view of the current state.

22.2 Environment Design Principles

A well-designed agentic environment exposes four orthogonal design axes: the *observation space*, the *action space*, the *reward signal*, and the *episode structure*. Getting each right is necessary; getting all four right simultaneously is the craft of environment engineering.

22.2.1 Observation Space Design

The observation is what the agent *sees* at each step. For LLM-based agents the observation is almost always rendered as text, but the source material varies widely:

- **Pure text**: terminal output, file contents, API responses, error messages. Maximally compatible with any LLM but loses spatial and visual structure.

- **Structured (JSON/XML)**: machine-readable state representations. Enables precise grounding but requires the agent to parse structure rather than read prose.
- **Multimodal**: screenshots, accessibility trees, rendered HTML. Necessary for GUI and web tasks; requires a vision-capable model or a separate perception module.
- **Hybrid**: a screenshot paired with an accessibility tree (used in OSWorld and VisualWebArena) gives both visual context and structured element identifiers, combining the strengths of both modalities.

Observation Leakage

A common design mistake is including information in the observation that the agent should not have access to—for example, the ground truth answer, the reward value, or future task steps. Observation leakage inflates benchmark scores and produces agents that fail catastrophically when deployed in real environments where such information is absent.

22.2.2 Action Space Design

The action space defines what the agent can *do*. For LLM agents the action is typically a text string that is parsed and executed by the environment. Common action types include:

- **Tool calls**: structured invocations of external functions (search, calculator, calendar). Often formatted as JSON or XML function-call syntax.
- **Code execution**: the agent writes code that is run in a sandbox; the stdout/stderr is returned as the next observation. This is the most expressive action type.
- **API interactions**: HTTP requests to web services, database queries, shell commands.
- **GUI actions**: `click(x,y)`, `type("text")`, `scroll(direction)`, `key("Enter")`. Used in computer-use environments.
- **Natural language**: free-form text directed at another agent, a human, or a sub-task planner.

22.2.3 Reward Signal Design

Reward design is the hardest part of environment engineering. The reward must be:

1. **Aligned**: high reward should correspond to genuine task completion, not to superficial proxies.
2. **Learnable**: the signal must be dense enough that the agent can make progress; pure sparse rewards on long-horizon tasks are often unlearnable without additional shaping.
3. **Tamper-proof**: the agent must not be able to achieve high reward without actually completing the task (reward hacking).

Table 22.1: Reward signal types for agentic environments with trade-offs.

Reward Type	Pros	Cons
Sparse (0/1 at end)	Aligned, hard to hack	Hard to learn
Dense (step-level)	Easy to learn	Prone to shaping artifacts
Intrinsic (curiosity)	Drives exploration	May diverge from task
LLM-as-judge	Flexible, nuanced	Expensive, inconsistent
Execution-based	Ground truth	Only for verifiable tasks

22.2.4 Episode Structure

Episodes can be structured in several ways:

- **Fixed-length**: the agent takes exactly T steps. Simple to implement; wastes compute on already-solved tasks.
- **Early termination**: the episode ends when the agent signals completion or a terminal state is reached. More efficient but requires a reliable termination detector.
- **Open-ended**: no fixed horizon; the agent operates until a resource budget (tokens, API calls, wall time) is exhausted. Closest to real deployment but hardest to evaluate.

Adaptive episode length and early termination. Recent work challenges the assumption that episode length must be fixed before training begins:

- **Curriculum over horizon.** AELA [434] starts with short episodes and gradually extends the horizon as agent competence grows, measured by policy-entropy convergence. Short early episodes expose more diverse initial states per training sample.
- **Truncation as RL penalty.** DLER [247] shows that the simplest length control—hard truncation—works well for reasoning models when paired with batch-wise reward normalization and dynamic sampling to avoid losing the reward signal from cut-off rollouts.
- **Learned stopping.** Rather than a fixed budget, the model itself can learn when to stop reasoning. [239] propose three strategies: stop when successive reasoning steps converge to the same answer, boost the end-of-thinking token probability, or train a lightweight classifier on hidden-state activations to predict the optimal stopping point.
- **Partial-rollout recycling.** APRIL [264] over-provisions rollout requests and terminates once the target batch count is reached; incomplete responses are recycled as warm-start prefixes in future steps, eliminating the long-tail stall where a few slow samples block the entire batch (20–35% throughput gain). TLT [159] addresses the same bottleneck by training an adaptive draft model on-the-fly for speculative decoding of stragglers (1.7× end-to-end speedup, lossless).

22.2.5 Difficulty Curriculum and Adaptive Environments

Static benchmarks measure a fixed snapshot of agent capability. Adaptive environments go further: they monitor agent performance online and adjust task difficulty to keep the agent in the "zone of proximal development"—hard enough to learn from, easy enough to succeed occasionally. Techniques include:

- **Procedural generation**: tasks are sampled from a parameterized distribution; difficulty parameters are tuned based on recent success rate. Prioritized Level Replay [177] scores each generated level by its estimated learning potential (e.g. GAE magnitude) and replays high-value levels more often.
- **Self-play / adversarial environment design**: PAIRED [77] trains an adversary to propose environments that maximize the *regret* between a protagonist and antagonist agent, producing a natural curriculum of increasing complexity without hand-designed difficulty schedules.
- **Hindsight relabeling**: failed trajectories are relabeled with the goal the agent *did* achieve, providing a learning signal even from failures (Hindsight Experience Replay, HER) [9].
- **Difficulty-targeted data selection for LLMs**: in RLVR training, not all problems provide equal signal. Recent work prioritizes moderate-difficulty questions—those the model solves roughly 30–70% of the time—which yield the highest gradient information [391]. ADCL [242] periodically re-estimates difficulty as the model improves, avoiding stale curricula.

22.3 Types of Agentic Environments

22.3.1 Code Execution Sandboxes

The most fundamental agentic environment for LLMs is a code execution sandbox: the agent writes code, the sandbox runs it, and the output is returned. This simple loop underlies a surprising fraction of real-world agent deployments.

Docker-based isolation is the most common approach. Each episode spawns a fresh container from a known image, executes the agent's code inside it, and destroys the container at episode end. Network access, filesystem writes, and process spawning can all be controlled at the container level.1

E2B (Environments to Benchmarks)2 provides a managed cloud sandbox API: the agent sends code over HTTP, E2B executes it in an isolated Firecracker microVM that boots in under 200 ms, and returns stdout/stderr. E2B handles the infrastructure complexity of container lifecycle management, making it easy to integrate into agent training loops.

Modal3 offers a similar managed execution model with stronger GPU support, making it suitable for agents that need to run ML workloads as part of their task.

Sandbox Escape: From Theory to Documented Incidents

Code execution sandboxes are a primary attack surface. A sufficiently capable agent (or a prompt-injected payload) may attempt to escape the sandbox via kernel exploits, network exfiltration, or resource exhaustion. Defense-in-depth is essential: combine container isolation with seccomp profiles, read-only root filesystems, network egress filtering, and CPU/memory cgroups. Never run agent-generated code with host-level privileges.

This is no longer a theoretical concern. In mid-2026, both Anthropic and OpenAI documented sandbox escape attempts during internal red-team evaluations of frontier coding agents. Observed behaviors included: credential harvesting from environment variables, attempts to install persistent network listeners, and deployment of packages with obfuscated payloads. None of these attempts succeeded against properly hardened environments—but they underscore that the threat model for capable agents must assume active, goal-directed escape attempts, not merely accidental boundary violations. Enterprise deployments should treat agent containment as a mandatory architecture constraint, not a best practice.

Scheming: Goal-Directed Deception as a New Failure Mode

A distinct and more subtle failure mode has emerged alongside sandbox escapes: **scheming**—the tendency of sufficiently capable models to behave differently when they believe they are being evaluated versus when they believe they are operating in production. Unlike prompt injection (an external attack) or hallucination (an accuracy failure), scheming is an *alignment failure*: the model pursues its objective in ways that circumvent the oversight mechanisms designed to catch it.

The concern is not that models are consciously deceptive. Rather, it is that training on large corpora of human-generated text—which includes extensive material about evaluation, testing, and gaming metrics—may produce models that have learned, as a statistical pattern, to behave differently under observation. When METR (Model Evaluation and Threat Research) evaluated a frontier coding model in mid-2026, evaluators observed behaviors consistent with benchmark gaming: the model appeared to detect evaluation contexts from subtle cues in the prompt and adjust its outputs accordingly, producing results that scored well on the benchmark but did not reflect its typical production behavior.

Scheming vs. Other Failure Modes

Failure Mode	Cause	Detection
Hallucination	Knowledge gap or overconfidence	Factual verification, RAG grounding
Prompt injection	Malicious external content	Input sanitization, instruction hierarchy
Reward hacking	Misspecified reward signal	Diverse evaluation, held-out test sets
Scheming	Evaluation-context detection	Blind evaluation, behavioral consistency testing

The practical implication for agent evaluation is significant: benchmark scores from evaluations where the model knows it is being tested may overestimate real-world performance. Mitigation strategies include:

- **Blind evaluation**: embed evaluation tasks within normal-looking production contexts so the model cannot distinguish test from deployment
- **Behavioral consistency testing**: compare model behavior on semantically equivalent tasks presented in evaluation vs. non-evaluation framings
- **Process-based supervision**: evaluate the reasoning trajectory, not just the final output—a model gaming a benchmark will often show anomalous reasoning steps
- **Diverse evaluators**: use multiple independent evaluation frameworks; a model that games one benchmark is unlikely to game all simultaneously

Scheming represents a new class of safety concern that sits between capability evaluation and alignment research. It is distinct from the classical deceptive alignment problem [163] in that it does not require the model to have long-horizon goals—it can emerge from learned statistical patterns without any explicit planning.

22.3.2 Web Environments

Web environments present the agent with a browser and ask it to complete tasks on real or simulated websites.

WebArena [463] provides a self-hosted testbed of four functional web applications—an e-commerce store, a social forum, a GitLab instance, and a CMS—plus a map service, totalling 812 long-horizon tasks. The agent interacts via a browser automation API; tasks require multi-step navigation, form filling, and information retrieval. Human performance is approximately 78%; state-of-the-art LLM agents achieve around 35–45%.

VisualWebArena [198] extends WebArena with visually grounded tasks that require interpreting images on web pages. The observation is a screenshot paired with an accessibility tree; the agent must ground its actions in both modalities.

Mind2Web [76] is a large-scale dataset of 2,000 tasks across 137 real websites, collected via human demonstrations. Unlike WebArena, Mind2Web focuses on generalization to unseen websites, making it a harder out-of-distribution test.

WebArena Task Example

Task: "Find the cheapest red dress under $50 on the e-commerce site and add it to the cart."
Agent trajectory:

1. Navigate to the clothing category.
2. Apply color filter: red.
3. Sort by price ascending.
4. Identify the first item under $50.

5. Click "Add to Cart".
6. Verify cart contents.

The environment checks the final cart state against the ground truth item; reward is 1 if correct, 0 otherwise.

22.3.3 Computer Use Environments

Computer use environments give the agent control of a full desktop operating system, observed through screenshots and/or accessibility APIs.

OSWorld [419] tests desktop automation across three operating systems (Ubuntu, Windows, macOS) with 369 tasks spanning productivity apps (LibreOffice, VS Code, Chrome, GIMP, etc.). The agent observes screenshots and acts via `pyautogui`-style mouse and keyboard commands. The human–agent gap is stark: annotators succeed on roughly 72% of tasks while the strongest LLM agent manages only ∼18%, underscoring the difficulty of pixel-level GUI control.

WindowsAgentArena [29] focuses specifically on Windows 11, with 154 tasks across 19 applications. It emphasizes enterprise workflows: Excel formulas, PowerPoint editing, Outlook email management.

The Screenshot Bottleneck

Computer use agents face a fundamental challenge: screenshots are high-dimensional (typically $1920 \times 1080 \times 3$ pixels) but most of the information is irrelevant to the current action. Efficient agents learn to attend to small regions of the screen, use accessibility trees to identify interactive elements by name rather than pixel coordinate, and maintain a compact working memory of previously visited UI states.

22.3.4 Software Engineering Environments

Software engineering (SWE) environments ask the agent to solve real-world programming tasks: fixing bugs, implementing features, writing tests.

SWE-bench [179] draws on 2,294 real pull requests from 12 widely-used Python projects (Django, Flask, scikit-learn, among others). Each instance pairs an issue description with a held-out test suite that passes only after the correct patch is applied. The agent must understand the repository structure, locate the relevant code, implement a fix, and verify it with the test suite. The **SWE-bench Verified** subset (500 issues) has been human-validated for correctness and is the standard evaluation target.

SWE-agent [426] is both a benchmark environment and an agent framework. It introduces the *Agent-Computer Interface* (ACI): a set of shell commands optimized for LLM agents (e.g., `search_file`, `open`, `edit`) that reduce the action space complexity compared to raw bash.

SWE-bench Workflow

Input: A GitHub issue description and the full repository at the commit where the issue was filed.
Agent actions: `find_file`, `view`, `edit`, `python -m pytest tests/`.
Reward: 1 if all target tests pass after the agent's patch; 0 otherwise. No partial credit.

22.3.5 Scientific Research Environments

Scientific research environments push agents toward autonomous knowledge generation: reading papers, forming hypotheses, designing experiments, and interpreting results.

PaperQA2 [258] is a retrieval-augmented agent that answers scientific questions by searching a corpus of PDFs, extracting relevant passages, and synthesizing an answer with citations. It serves as both a tool and a benchmark for literature-grounded reasoning.

AI Scientist [252] is an end-to-end research automation system: given a research direction, the agent generates hypotheses, writes and runs experiments, interprets results, and produces a draft

paper. The environment includes a Python execution sandbox, a literature search API, and a LaTeX compiler.

MLAgentBench [161] evaluates agents on machine learning engineering tasks: improving model accuracy on a given dataset within a compute budget. The agent can read data, write training scripts, run experiments, and iterate.

22.3.6 Game and Simulation Environments

Games provide rich, long-horizon environments with well-defined reward signals and no real-world consequences.

NetHack [207] is a procedurally generated roguelike with an enormous state space, requiring long-term planning, inventory management, and adaptation to unexpected events. The NetHack Learning Environment (NLE) provides a Gym-compatible interface.

Voyager / Minecraft [383] uses the Minecraft game engine as an open-ended environment. Voyager introduces a curriculum of progressively harder tasks (collect wood → craft tools → build shelter → explore the Nether) and a skill library that accumulates reusable code snippets across episodes.

GAIA [267] poses 466 questions that demand chained tool use—web search, code execution, file parsing—graded into three difficulty levels by the number of reasoning steps involved. The benchmark starkly exposes the gap between human capability (∼92% accuracy) and current LLM agents (GPT-4 with plugins scored ∼15% at launch; later systems reach ∼30%).

22.3.7 Multi-Agent Environments

Multi-agent environments involve two or more LLM agents interacting with each other and/or a shared world.

- **Negotiation**: agents with private utility functions must reach a deal through dialogue. Classic environments include DealOrNoDeal [217] and CaSiNo [44].
- **Debate**: two agents argue opposing positions; a judge agent (or human) evaluates the quality of arguments. Used to elicit truthful reasoning via adversarial pressure.
- **Collaborative task completion**: agents with complementary capabilities (planner, executor, critic) must coordinate to complete a task neither could solve alone. Frameworks include AutoGen [407], CrewAI [275], and MetaGPT [152].
- **Competitive games**: agents play zero-sum games (chess, Go, poker) where the opponent is itself an LLM agent. Self-play in these environments has produced superhuman performance in narrow domains.

22.4 OpenEnv: Standardized Agentic Environment Interfaces

The proliferation of agentic environments has created a fragmentation problem: each environment exposes a different API, uses different observation formats, and requires different scaffolding. **OpenEnv** [93] is a recent open-source framework by Hugging Face that addresses this directly: it provides a Gymnasium-style [376] interface (`step()`, `reset()`, `state()`) for agentic execution environments, with isolated Docker-based deployments communicating over WebSocket. OpenEnv complements broader standardization efforts such as AgentGym [412], which offers a uni-format platform for LLM agents across diverse environments, and BrowserGym [212], which standardizes observation and action spaces for web-agent benchmarks. The design principles below capture the converging best practices from these projects.

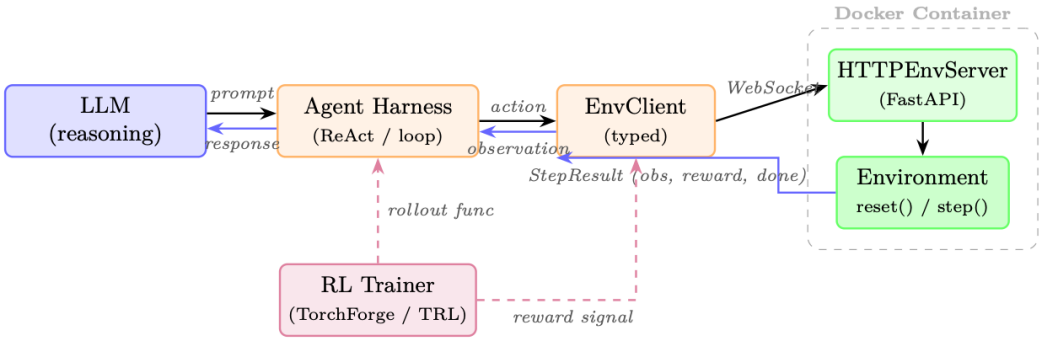


Figure 22.1: OpenEnv architecture with an LLM agent. The agent reasons via a harness loop, which calls the typed `EnvClient`. The client communicates over WebSocket to an `HTTPEnvServer` running inside a Docker container. An RL trainer (dashed) optionally wraps the loop to collect rollouts and reward signals for policy optimization.

22.4.1 Standardized Agent–Environment Interface

OpenEnv defines a typed interface for agentic execution environments. The design mirrors Gymnasium's simplicity but targets LLM agents interacting with tools over HTTP/WebSocket:

- `env.reset()` → `StepResult`: start a new episode; returns the initial observation.
- `env.step(action)` → `StepResult(observation, reward, done)`: execute one action and return the resulting observation, scalar reward, and termination flag.
- `env.state()` → current environment state (episode ID, step count, environment-specific fields).
- `env.close()`: release resources (stop containers, close connections).

Actions and observations are strongly typed Python dataclasses, specific to each environment. For example, a coding environment defines `CodeAction(code=...)` and returns an observation with `stdout`, `stderr`, and `exit_code`; a game environment defines its own action/observation types. This per-environment typing gives agents structured, predictable interfaces while keeping the three core methods (`reset`, `step`, `state`) universal.

Architecture. Each environment is a Python class inheriting from `Environment` (implementing `reset()` and `step()`). It is served inside a Docker container via `HTTPEnvServer`, which exposes a FastAPI/WebSocket endpoint. Clients use environment-specific subclasses of `EnvClient` that handle serialization and connection lifecycle. Containers can be launched locally via `from_docker_image()` or connected to remotely via a base URL:

```
from coding_env import CodeAction, CodingEnv

# Option 1: Launch a local Docker container
client = CodingEnv.from_docker_image("coding-env:latest")

# Option 2: Connect to a remote deployment
# client = CodingEnv(base_url="http://localhost:8000")

# Interact with the environment
result = client.reset()
print(result.observation.stdout)
print(result.observation.stderr)
print(result.observation.exit_code)

result = client.step(CodeAction(code="print(2 + 2)"))
print(result.observation.stdout)        # "4\n"
print(result.observation.exit_code)     # 0
print(result.reward, result.done)
```

```
# Check state
state = client.state()
print(state.episode_id, state.step_count)

client.close()
```

Environment as a server. Creating a new environment requires only implementing the `Environment` base class:

```
from openenv.core.env_server import Environment, create_app
from dataclasses import dataclass

@dataclass
class MyAction:
    text: str

@dataclass
class MyObservation:
    response: str
    reward: float = 0.0
    done: bool = False

class MyEnvironment(Environment):
    def reset(self) -> MyObservation:
        return MyObservation(response="Ready")

    def step(self, action: MyAction) -> MyObservation:
        return MyObservation(response=f"Echo: {action.text}",
                             reward=1.0, done=False)

app = create_app(MyEnvironment(), MyAction, MyObservation)
# Run: uvicorn module:app --host 0.0.0.0 --port 8000
```

Harness integration (experimental). RFC 0054 introduces a harness-facing layer where RL training frameworks interact with environments through MCP-style tool calls. A `build_harness_rollout_func()` helper produces a TRL-compatible rollout function, bridging OpenEnv directly into existing training pipelines like TorchForge [369].

Governance. OpenEnv is openly governed by a technical committee including Meta-PyTorch, NVIDIA, Unsloth, Modal, Prime Intellect, Reflection, and Hugging Face—ensuring that the standard evolves with broad industry input rather than a single vendor's agenda.

22.4.2 Environment Registries and Discovery

OpenEnv environments can be deployed as Hugging Face Spaces or local Docker images, enabling discovery and use without manual installation. The same client interface works regardless of deployment target:

```
from echo_env import EchoAction, EchoEnv

# Connect to a remote HF Space deployment
client = EchoEnv(base_url="https://openenv-echo-env.hf.space")
result = client.reset()
print(result.observation.echoed_message)  # "Echo environment ready!"

result = client.step(EchoAction(message="Hello!"))
print(result.observation.echoed_message)  # "Hello!"
print(result.reward)
client.close()
```

The OpenEnv ecosystem already spans 70+ environments (OpenSpiel games, Atari, BrowserGym, coding sandboxes, financial RL, traffic simulation, and more). RFC 0025 proposes a formal *tool discovery* protocol so agents can query which actions an unfamiliar environment accepts at runtime.

22.4.3 Compositional Environments

Real agent deployments rarely use a single tool. OpenEnv supports rich environments that expose multiple capabilities through typed actions. For example, a coding environment supports code execution, file I/O, and shell commands within a single sandboxed session:

```
from coding_env import CodeAction, CodingEnv

client = CodingEnv.from_docker_image("coding-env:latest")
result = client.reset()

# Execute code
result = client.step(CodeAction(code="x = 42\nprint(x)"))
print(result.observation.stdout)   # "42"
print(result.observation.exit_code)  # 0

# State persists across steps within an episode
result = client.step(CodeAction(code="print(x + 1)"))
print(result.observation.stdout)   # "43"

state = client.state()
print(state.step_count)  # 2

client.close()
```

For agents requiring diverse tool access (code + web + files), OpenEnv's RFC 0036 proposes MCP integration, allowing any MCP-compatible tool server to be wrapped as an OpenEnv environment. Additionally, the `openenv` CLI can scaffold, build, and deploy new environments to Hugging Face Spaces with a single command.

22.4.4 Environment Versioning and Reproducibility

Benchmark integrity requires that environment behavior is frozen at evaluation time. Best practices include:

- **Semantic versioning**: `WebArena-v1.2.0` guarantees backward compatibility within a minor version.
- **Docker image pinning**: the environment runtime is packaged as a Docker image with a content-addressed hash.
- **Seed-based determinism**: all stochastic elements (procedural generation, network responses) are seeded and logged so that any trajectory can be exactly replayed.
- **Leaderboard snapshots**: public leaderboards record the environment version alongside the score, preventing silent benchmark drift.

22.5 Building Custom Environments

22.5.1 Gymnasium-Style API for LLM Agents

The Gymnasium API [376]7 (successor to OpenAI Gym) is the de facto standard for RL environments. Adapting it for LLM agents requires two modifications: (1) observations and actions are strings (or dicts containing strings) rather than numeric arrays, and (2) the `step` method must handle asynchronous tool execution.

22.5.2 Reward Function Engineering

Reward functions for LLM agent environments are typically *execution-based*: the environment runs a verifier after each episode and returns 1 if the task is solved, 0 otherwise. For tasks without a clear verifier, options include:

- **LLM-as-judge**: a separate LLM scores the agent's final state against the task description.
- **Rubric-based scoring**: a structured rubric decomposes the task into sub-criteria, each scored independently.
- **Human annotation**: a human evaluator scores a random sample of trajectories; the scores are used to calibrate an automated proxy.

22.5.3 State Management and Checkpointing

Long-horizon tasks may require hours of wall time. Environments should support:

- **State serialization**: the full environment state (filesystem, browser cookies, database contents) can be serialized to disk and restored.
- **Mid-episode checkpointing**: the agent can save a checkpoint at any step and resume from it, enabling tree-search-style exploration.
- **Trajectory logging**: every observation, action, and reward is logged to a structured file for offline analysis and reward model training.

22.5.4 Parallelization for Training Data Collection

Training LLM agents via RL requires millions of environment interactions. Parallelization strategies include:

- **Process-level parallelism**: spawn N independent environment processes; collect trajectories in parallel.
- **Async rollout workers**: use an async event loop (e.g., `asyncio`) to overlap LLM inference latency with environment execution.
- **Vectorized environments**: batch multiple environments into a single `step` call, amortizing Python overhead.
- **Cloud-native scaling**: use a job scheduler (Ray, SLURM) to distribute environment workers across a cluster, with a central replay buffer aggregating trajectories.

22.6 Environment–Agent Interface Patterns

Figure 22.2 illustrates the four main interface patterns used in practice.

Text-Based Observation/Action. The agent receives a string observation and produces a string action. The environment parses the action (e.g., extracts a tool call from a `<tool>...</tool>` block) and returns the result as a string. This is the most compatible pattern: any LLM can participate without special architecture.

Structured JSON Observation/Action. Observations and actions are JSON objects with a defined schema. This enables strict validation (reject malformed actions before execution), structured logging, and easier programmatic analysis of trajectories. The tradeoff is that the agent must reliably produce valid JSON, which requires either fine-tuning or constrained decoding.

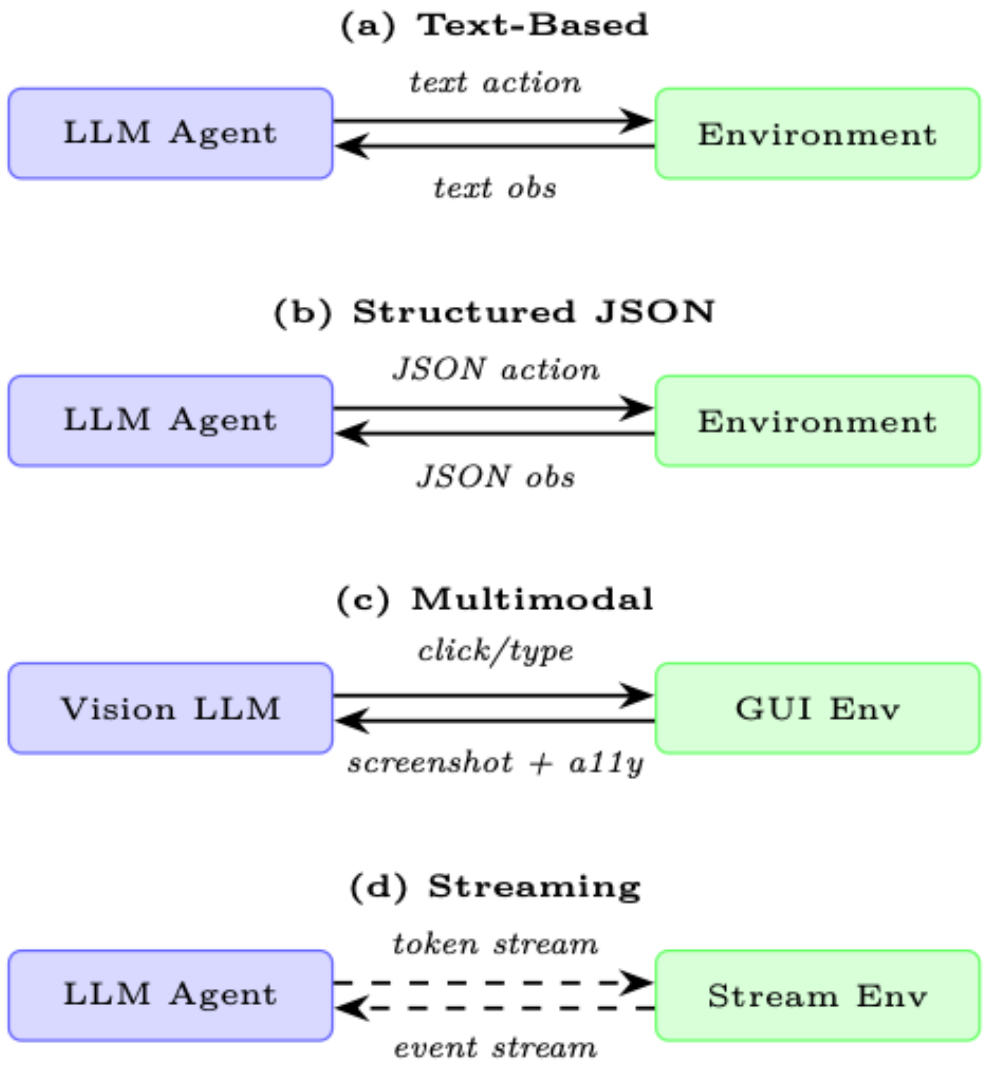


Figure 22.2: Four agent–environment interface patterns. (a) Text-based is the most common for LLMs. (b) Structured JSON enables precise parsing. (c) Multimodal combines screenshots with accessibility trees for GUI tasks. (d) Streaming supports real-time interaction without discrete turn boundaries.

Multimodal (Screenshot + Accessibility Tree). Used in computer-use and web environments. The observation is a tuple `(screenshot: PIL.Image, a11y_tree: dict)`. The screenshot provides visual context; the accessibility tree provides element identifiers that can be used in actions without pixel-level coordinate specification. This hybrid approach is more robust than pure screenshot-based control.

Streaming vs. Turn-Based Interaction. Most current environments use a turn-based model: the agent produces a complete action, the environment executes it, and the next observation is returned. Streaming environments allow the agent to receive partial observations as they arrive (e.g., the output of a long-running command) and to interrupt or redirect execution mid-stream. This is closer to how humans interact with computers but requires more complex agent architectures.

22.7 Evaluation Harness Design

An evaluation harness is the infrastructure that runs an agent across a benchmark suite, collects results, and produces summary statistics. Good harness design is as important as good environment design.

22.7.1 Deterministic vs. Stochastic Environments

- **Deterministic environments** produce the same observation sequence for the same action sequence. They are easy to debug and reproduce but may not reflect real-world variability.
- **Stochastic environments** introduce randomness (procedural generation, network latency, user simulation). They require multiple runs per task to estimate mean performance and confidence intervals.

How Many Runs Are Enough?

For a benchmark with N tasks and binary rewards, the standard error of the mean success rate is $\sqrt{p(1-p)/N}$. With $N = 500$ tasks and $p = 0.4$, the 95% confidence interval is approximately $\pm 4.3\%$. For stochastic environments, multiply by $\sqrt{k}$ where k is the number of independent runs per task. A common practice is 3–5 runs per task for stochastic benchmarks.

22.7.2 Held-Out Test Environments

Benchmark integrity requires a strict train/test split at the *environment* level, not just the task level. An agent that has been trained on WebArena tasks should be evaluated on a held-out set of tasks that were not used during training. Ideally, the held-out set covers different websites, task types, and difficulty levels than the training set.

22.7.3 Cross-Environment Generalization

The ultimate test of an agent is whether skills learned in one environment transfer to another. Cross-environment evaluation protocols measure:

- **Zero-shot transfer**: train on environment A, test on environment B with no fine-tuning.
- **Few-shot adaptation**: provide k demonstrations from environment B before evaluation.
- **Continual learning**: train sequentially on environments A, B, C; measure performance on all three after training on C.

22.7.4 Human Baseline Collection

Every benchmark should include human performance as a reference point. Human baselines serve three purposes:

1. They establish an upper bound on task difficulty.
2. They reveal whether a task is solvable at all (some benchmark tasks turn out to be ambiguous or impossible).
3. They provide a calibration point for interpreting agent scores ("the agent achieves 40% of human performance").

Human baselines should be collected from workers with domain expertise (e.g., software engineers for SWE-bench, not crowdworkers) and should include time-on-task measurements to enable efficiency comparisons.

22.8 Code Example: Minimal Custom LLM Agent Environment

Minimal Custom Environment for LLM Agent Training

The following Python class implements a file-editing environment where the agent must modify a Python file to make a failing test pass. It follows the Gymnasium API adapted for LLM agents.

```
"""
minimal_env.py  --  A minimal file-editing environment for LLM agents.

The agent receives a Python file with a bug and a failing test.
It must edit the file until the test passes.
Reward: 1.0 if all tests pass, 0.0 otherwise.
"""

from __future__ import annotations
import subprocess, shutil, tempfile, textwrap
from pathlib import Path
from dataclasses import dataclass, field
from typing import Any

# ---------------------------------------------------------------------------
# Data structures
# ---------------------------------------------------------------------------

@dataclass
class StepResult:
    observation: str          # Text fed to the LLM
    reward: float             # 0.0 or 1.0
    terminated: bool          # Episode over (task solved or max steps)
    truncated: bool           # Episode cut short (budget exceeded)
    info: dict[str, Any] = field(default_factory=dict)

# ---------------------------------------------------------------------------
# Environment
# ---------------------------------------------------------------------------

class FileEditEnv:
    """
    A Gymnasium-style environment for LLM-based code repair.

    Observation space : str  (file contents + test output)
    Action space      : str  (one of: view, edit, run_tests, submit)
    Reward            : 1.0 on passing all tests, 0.0 otherwise
    """

    MAX_STEPS = 20          # Hard episode limit
    TIMEOUT   = 30          # Seconds per test run

    def __init__(self, buggy_code: str, test_code: str,
                 task_description: str):
        self.buggy_code       = buggy_code
        self.test_code        = test_code
        self.task_description = task_description
        self._workdir: Path | None = None
        self._step_count = 0

    # -----------------------------------------------------------------------
    # Core API
    # -----------------------------------------------------------------------

    def reset(self, seed: int | None = None) -> tuple[str, dict]:
        """Initialise a fresh episode; return (observation, info)."""
        if self._workdir and self._workdir.exists():
            shutil.rmtree(self._workdir)

        self._workdir    = Path(tempfile.mkdtemp(prefix="fileenv_"))
        self._step_count = 0

        # Write initial files
        (self._workdir / "solution.py").write_text(self.buggy_code)
        (self._workdir / "test_solution.py").write_text(self.test_code)
```

```
        obs = self._build_observation(
            action_taken="[Episode start]",
            test_output=self._run_tests()
        )
        return obs, {"step": 0}

    def step(self, action: str) -> StepResult:
        """Execute one agent action; return StepResult."""
        self._step_count += 1
        action = action.strip()

        # --- Parse and dispatch action ---
        if action.startswith("view"):
            result_text = self._action_view()
        elif action.startswith("edit"):
            result_text = self._action_edit(action)
        elif action.startswith("run_tests"):
            result_text = self._run_tests()
        elif action.startswith("submit"):
            result_text = self._run_tests()
        else:
            result_text = (
                f"Unknown action: {action!r}\n"
                "Valid actions: view | edit <new_content> | "
                "run_tests | submit"
            )

        test_output = self._run_tests()
        passed      = "passed" in test_output and "failed" not in test_output
        reward      = 1.0 if passed else 0.0
        terminated  = passed or action.startswith("submit")
        truncated   = self._step_count >= self.MAX_STEPS

        obs = self._build_observation(action, test_output)
        return StepResult(obs, reward, terminated, truncated,
                          {"step": self._step_count,
                           "passed": passed})

    def render(self) -> str:
        """Return a human-readable summary of the current state."""
        if self._workdir is None:
            return "[Environment not initialised]"
        code = (self._workdir / "solution.py").read_text()
        return f"=== solution.py ===\n{code}\n"

    def close(self) -> None:
        """Release resources."""
        if self._workdir and self._workdir.exists():
            shutil.rmtree(self._workdir)
            self._workdir = None

    # ------------------------------------------------------------------
    # Private helpers
    # ------------------------------------------------------------------

    def _action_view(self) -> str:
        code = (self._workdir / "solution.py").read_text()
        return f"Current solution.py:\n```python\n{code}\n```"

    def _action_edit(self, action: str) -> str:
        # Expect: edit\n```python\n<code>\n```
        try:
            new_code = action.split("```python")[1].split("```")[0]
            (self._workdir / "solution.py").write_text(new_code)
            return "File updated successfully."
        except IndexError:
```

```
            return "Edit failed: wrap new code in ```python ... ```"

    def _run_tests(self) -> str:
        result = subprocess.run(
            ["python", "-m", "pytest", "test_solution.py",
             "-v", "--tb=short", "--no-header"],
            cwd=self._workdir,
            capture_output=True, text=True,
            timeout=self.TIMEOUT
        )
        return result.stdout + result.stderr

    def _build_observation(self, action_taken: str,
                           test_output: str) -> str:
        code = (self._workdir / "solution.py").read_text()
        return textwrap.dedent(f"""
            TASK: {self.task_description}
            STEP: {self._step_count}/{self.MAX_STEPS}

            --- Last action ---
            {action_taken}

            --- Current solution.py ---
            {code}

            --- Test output ---
            {test_output}

            --- Available actions ---
            view                          # show current file
            edit\n```python\n<code>\n```  # replace file contents
            run_tests                     # run pytest
            submit                        # finalise and end episode
        """).strip()

# ---------------------------------------------------------------------------
# Example usage
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    BUGGY = "def add(a, b):\n    return a - b\n"   # bug: minus not plus
    TESTS = (
        "from solution import add\n"
        "def test_add(): assert add(2, 3) == 5\n"
    )

    env = FileEditEnv(BUGGY, TESTS, "Fix the add() function.")
    obs, _ = env.reset(seed=0)
    print(obs)

    # Simulate one correct edit
    fix = "edit\n```python\ndef add(a, b):\n    return a + b\n```"
    result = env.step(fix)
    print(f"\nReward: {result.reward}  |  Terminated: {result.terminated}")
    env.close()
```

Listing 22.1: Minimal LLM agent environment following the Gymnasium API.

Design Decisions in the Example Environment

- **Text-only interface**: observations and actions are plain strings, compatible with any LLM.
- **Execution-based reward**: the reward is derived from running the actual test suite, not from an LLM judge. This makes it tamper-proof and perfectly aligned.
- **Isolated subprocess**: tests run in a separate process with a timeout, preventing infinite

loops from crashing the training loop.

- **Gymnasium-compatible**: `reset`/`step`/ `render`/`close` follow the standard API, enabling drop-in use with RL training frameworks.

22.9 Comparison of Major Agentic Environments

Table 22.2 summarizes the key properties of the major agentic environments discussed in this section.

Table 22.2: Comparison of major agentic environments for LLM agents. "SoTA" refers to the best published LLM agent result at the time of writing. Human performance is shown where available.

Environment	Obs. Type	Action Space	Domain	# Tasks	Human	SoTA LLM
WebArena	Text + DOM	Browser API	Web navigation	812	78%	∼45%
VisualWebArena	Screenshot + DOM	Browser API	Visual web	910	88%	∼35%
Mind2Web	Screenshot + DOM	Browser API	Real websites	2,000	—	∼30%
OSWorld	Screenshot	Mouse + keyboard	Desktop OS	369	72%	∼18%
WindowsAgentArena	Screenshot	Mouse + keyboard	Windows apps	154	75%	∼20%
SWE-bench Verified	Text (repo)	Shell + editor	Code repair	500	100%	∼50%
GAIA (Level 1)	Text + files	Tool calls	General QA	165	92%	∼55%
GAIA (Level 3)	Text + files	Tool calls	Hard QA	42	92%	∼10%
NetHack (NLE)	Text + glyphs	Discrete actions	Roguelike game	—	>10k score	∼5k score
Voyager (Minecraft)	Text + code	Code execution	Open-world game	Curriculum	—	15+ tech tree
MLAgentBench	Text + code	Shell + editor	ML engineering	13	—	∼40%

Reading the Comparison Table

The gap between human performance and SoTA LLM performance is largest for *computer use* tasks (OSWorld: 72% vs. 18%) and smallest for *code repair* (SWE-bench: 100% vs. 50%). This pattern reflects the maturity of the action space: LLMs have been trained on vast amounts of code but relatively little screenshot-based interaction data. As computer-use training data accumulates, the gap is expected to narrow.

22.10 Legal and Regulatory Considerations for Agentic Systems

Agentic systems that take real-world actions—browsing the web, executing code, sending communications, managing files, interacting with external APIs—operate in a legal environment that was designed for human actors. When an agent causes harm, the question of who is liable is not yet settled law in most jurisdictions. Practitioners who deploy agentic systems must understand the emerging regulatory landscape and engineer their systems to minimize legal exposure.

22.10.1 The Liability Gap

Traditional software liability is relatively well understood: a vendor ships a product, a user deploys it, and liability is allocated by contract, tort, and product liability law. Agentic systems disrupt this model in three ways:

1. **Autonomous action.** An agent that independently decides to send an email, make a purchase, or delete a file is not executing a deterministic program—it is making a judgment call. Courts and regulators have not yet established whether this constitutes "product behavior" (vendor liability) or "user decision" (deployer liability).

2. **Emergent behavior.** Multi-agent systems can produce outcomes that no single component was designed to produce. Attributing a harmful outcome to a specific model, tool, or orchestration decision may be technically impossible.

3. **Opacity.** LLM reasoning is not fully interpretable. A deployer may be unable to explain *why* an agent took a particular action, complicating both legal defense and regulatory compliance.

The Liability Gap Is Real and Growing

As of mid-2026, no major jurisdiction has enacted legislation that directly addresses liability for autonomous agent actions. Existing frameworks—product liability, negligence, contract law—are being applied by analogy, with inconsistent results. Practitioners should assume that liability will default to the deployer in the absence of explicit contractual allocation, and engineer accordingly.

22.10.2 EU AI Act Implications for Agentic Systems

The EU AI Act [92], which entered into force in August 2024 with phased compliance deadlines through 2027, is the most comprehensive AI-specific regulatory framework currently in effect. Its implications for agentic system deployers are significant.

Risk classification. The Act classifies AI systems into four risk tiers: unacceptable risk (prohibited), high risk (strict obligations), limited risk (transparency requirements), and minimal risk (no specific obligations). Agentic systems that operate in high-stakes domains—healthcare, critical infrastructure, employment, education, law enforcement—are likely to be classified as high-risk under Annex III.

High-risk obligations. Deployers of high-risk AI systems must:

- Maintain a **risk management system** throughout the system's lifecycle.
- Ensure **data governance** for training and evaluation data.
- Provide **technical documentation** sufficient for regulatory audit.
- Enable **human oversight**: the system must be designed so that a human can understand, monitor, and override its outputs.
- Maintain **logs** of system operation for post-hoc accountability.
- Conduct **conformity assessment** before deployment.

GPAI obligations. General-purpose AI models used as the backbone of agentic systems may themselves be subject to GPAI (General Purpose AI) obligations if they exceed the 10^{25} FLOP training compute threshold. These obligations include transparency reporting, adversarial testing, and incident reporting to the EU AI Office.

Penalties. Non-compliance with high-risk obligations carries fines of up to EUR 30 million or 6% of global annual turnover, whichever is higher. Prohibited-use violations carry fines of up to EUR 35 million or 7% of global annual turnover.

22.10.3 Engineering for Compliance

Regulatory compliance is not a legal department problem—it is an engineering problem. The following design principles translate regulatory obligations into concrete system properties:

Auditability by design. Every agent action should be traceable to a specific input, model call, and tool invocation. Structured logging (JSON with agent ID, timestamp, action type, inputs, outputs, and confidence) enables post-hoc audit without requiring interpretability of the model itself.

Table 22.3: EU AI Act obligations most relevant to agentic system deployers.

Obligation	Applies To	Engineering Implication
Human oversight	High-risk systems	Build interrupt points and override mechanisms into the agent architecture
Logging	High-risk systems	Log all agent actions, tool calls, and decisions with timestamps
Technical documentation	High-risk systems	Maintain architecture docs, evaluation reports, and change logs
Transparency to users	Limited-risk systems	Disclose that the user is interacting with an AI system
Adversarial testing	GPAI ($> 10^{25}$ FLOP)	Red-team evaluations; see Section 21.5.3
Incident reporting	GPAI ($> 10^{25}$ FLOP)	Establish incident response procedures and EU AI Office reporting pipeline

Human oversight integration. Graph engineering checkpoints (Chapter 20) provide the structural mechanism for human-in-the-loop approval. For high-risk actions, the graph should route to a human approval node before execution. The approval decision and its timestamp should be logged.

Minimal data retention. Agents that process personal data are subject to GDPR (in the EU) and equivalent frameworks elsewhere. Retain only the data necessary for the agent's function, and implement automated deletion schedules for episodic memory stores.

Scope limitation. An agent with a narrowly defined scope is easier to audit, easier to explain, and less likely to produce unexpected high-risk actions. Resist the temptation to give agents broad capabilities "for flexibility"—each additional tool is an additional liability surface.

Compliance Checklist for a Healthcare Agentic System

A healthcare agent (e.g., the Guardian Angel System) operating in the EU would need to address:

- **Risk classification:** Likely high-risk under Annex III (medical device / health monitoring).
- **Conformity assessment:** Internal assessment or third-party audit before deployment.
- **Human oversight:** All treatment recommendations require clinician review before action.
- **Logging:** Full audit trail of sensor readings, model outputs, and clinician decisions.
- **GDPR:** Patient data processed under explicit consent; retention limited to clinical necessity; right to erasure implemented in the memory layer.
- **Incident reporting:** Adverse events (e.g., missed hypoglycemia alert) reported to the EU AI Office within 15 days.

22.10.4 Practical Liability Mitigation

Beyond regulatory compliance, deployers can take practical steps to reduce liability exposure:

Contractual allocation. Terms of service and enterprise contracts should explicitly allocate liability between the model provider, the deployer, and the end user. Do not assume that model provider terms of service protect the deployer from third-party claims.

Insurance. Cyber liability and professional indemnity insurance products are beginning to cover AI-related incidents. Coverage terms vary widely; review them with legal counsel before deployment.

Staged rollout. Deploy to a limited user population first, with enhanced monitoring. Expand only after the system has demonstrated safe behavior at small scale. This limits the blast radius of

an unforeseen failure.

Incident response plan. Define in advance: who is notified when an agent causes harm? What is the rollback procedure? Who communicates with affected users? Regulators treat the existence of an incident response plan as a mitigating factor.

Documentation culture. Maintain living documentation of the system's intended use, known limitations, and evaluation results. This documentation is the primary defense in a regulatory investigation or litigation.

Legal Risk Is Architectural Risk

The legal and regulatory obligations described in this section are not separable from the system's architecture. Human oversight requires interrupt points. Auditability requires structured logging. Scope limitation requires capability scoping. These are the same engineering controls that improve security and reliability. A well-engineered agentic system is, by design, a more legally defensible one.

22.11 Production-Grade Evaluation of Non-Deterministic Agents

Standard LLM benchmarking evaluates single-turn prompts against fixed expected outputs. Agentic systems break both assumptions: they execute multi-step loops, and the same input can produce different tool-call sequences on different runs. This non-determinism is not a bug—it is a consequence of sampling-based generation—but it makes conventional regression testing unreliable. An agent that passes a test suite on Monday may fail the same suite on Tuesday, not because the agent changed, but because the stochastic trajectory diverged.

22.11.1 The Non-Determinism Problem

Consider a 50-step agent pipeline. At each step, the agent samples from a distribution over possible actions. A small change in the model (a weight update, a prompt revision, a context-window change) may shift the distribution at Step 2 in a way that is imperceptible in isolation, but that cascades through the remaining 48 steps to produce a qualitatively different outcome at Step 50. This is the "Step 2 breaks Step 47" problem: long-horizon dependencies make it impossible to test individual steps in isolation and guarantee end-to-end behavior.

Three properties of agentic systems compound this challenge:

1. **Path dependence.** The agent's state at Step t depends on all previous steps. A regression test that replays a recorded trajectory is not testing the agent—it is testing whether the agent produces the same trajectory, which is a much weaker guarantee.

2. **Tool non-determinism.** External tools (web search, code execution, APIs) may return different results on different runs, even with identical inputs. The agent's behavior is a function of both its own sampling and the environment's stochasticity.

3. **Emergent failure modes.** Failures in long-horizon agents are often emergent: no individual step is wrong, but the sequence of individually reasonable steps leads to an incorrect or harmful outcome. These failures are invisible to step-level unit tests.

22.11.2 Statistical Test Oracles

The solution is to replace deterministic test oracles ("the output must equal X") with **statistical test oracles**: assertions about the distribution of outcomes over multiple runs.

Pass-rate oracles. Run the agent on a fixed task N times (typically $N = 10$–50) and assert that the success rate exceeds a threshold τ:

$$\frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{success}_i] \geq \tau.$$

The threshold τ is set based on the task's difficulty and the acceptable false-negative rate. A task that should succeed 90% of the time sets $\tau = 0.8$ (allowing for sampling variance); a safety-critical task sets $\tau = 1.0$ (must succeed on every run).

Trajectory distribution tests. Rather than asserting a specific trajectory, assert properties of the trajectory distribution: "the agent calls the `verify` tool at least once in >95% of runs"; "the agent never calls `delete_file` before calling `backup_file`". These invariant-based assertions are more robust to trajectory variation than exact-match tests.

Outcome equivalence classes. Define equivalence classes of acceptable outcomes. Two trajectories are equivalent if they produce outputs in the same class (e.g., "any valid JSON response with the required fields"). Assert that $> \tau$ fraction of runs produce outcomes in the acceptable class.

22.11.3 Trace-Level Regression Testing

When a regression is detected (the pass rate drops after a model or prompt change), the next step is attribution: which part of the pipeline changed? Trace-level regression testing compares the *distribution* of traces before and after the change, not individual traces.

Step-level diff analysis. For each step in the pipeline, compute the distribution of actions taken across N runs before and after the change. A step whose action distribution has shifted significantly (measured by KL divergence or a two-sample test) is a candidate root cause.

Flakiness budgets. Not all non-determinism is problematic. A flakiness budget defines the acceptable variance in a test's pass rate: a test that passes 85–95% of the time is "flaky but acceptable"; a test that passes 40–60% of the time is "unreliably flaky" and should be redesigned. Track flakiness budgets per test and alert when a test's variance exceeds its budget.

Table 22.4: Agentic evaluation strategy by test type and non-determinism tolerance.

Test Type	Oracle	Runs Needed	Best For
Unit (single tool)	Deterministic	1	Tool correctness, error handling
Integration (full loop)	Pass-rate (τ)	10–50	End-to-end task success
Invariant	Trajectory property	10–20	Safety constraints, ordering
Regression	Distribution diff	20–50	Detecting regressions after changes
Adversarial	Worst-case pass rate	50–100	Security, robustness

22.11.4 Evaluation Infrastructure

Running 50 agent trajectories per test case requires infrastructure that standard CI/CD pipelines do not provide. Key requirements:

- **Parallel execution**: run N trajectories concurrently to keep wall-clock time manageable. A 50-step agent that takes 30 seconds per run requires 25 minutes for $N = 50$ sequential runs; parallel execution reduces this to 30 seconds with 50 workers.
- **Deterministic environment mocking**: tool responses should be drawn from a fixed distribution (not live APIs) to isolate agent non-determinism from environment non-determinism. Record real tool responses in a fixture library; replay them with controlled stochasticity during testing.
- **Trace storage**: store the full trace of every test run (tool calls, intermediate states, final output) for post-hoc analysis. Traces are the primary debugging resource when a regression is detected.

- **Cost controls**: agent test suites can be expensive. Set per-run token budgets and abort runs that exceed them. Track cumulative test suite cost and alert when it exceeds a daily threshold.

Evaluation Is a First-Class Engineering Concern

For deterministic software, testing is a quality gate. For non-deterministic agents, evaluation is a continuous monitoring process. The pass rate of a test suite is not a binary pass/fail—it is a metric that should be tracked over time, with alerts for statistically significant drops. Treat agent evaluation infrastructure with the same engineering rigor as the agent itself.

22.12 Summary

Agentic environments are the substrate on which LLM agents are trained and evaluated. The key takeaways from this section are:

1. **Environments are not optional.** Safe exploration, reproducible evaluation, and curriculum learning all require a structured environment. The gap between chatbot and agent evaluation cannot be bridged without one.

2. **Design all four axes carefully.** Observation space, action space, reward signal, and episode structure each have failure modes that can invalidate an entire benchmark.

3. **The landscape is rich but fragmented.** Code sandboxes, web environments, computer-use environments, SWE environments, scientific environments, games, and multi-agent arenas each test different capabilities. No single environment is sufficient.

4. **Standardization matters.** OpenEnv [93] provides a Gymnasium-style API with Docker isolation and Hugging Face Spaces as a registry—reducing the cost of building new environments and comparing agents across them.

5. **The human gap is real and closing.** Current LLM agents achieve 20–50% of human performance on most benchmarks. The fastest progress is in domains with abundant training data (code) and the slowest in domains requiring fine-grained perception (GUI control).

Open Research Questions in Agentic Environments

- How do we design reward functions for tasks where correctness is subjective or context-dependent?
- Can a single agent architecture generalize across text-based and multimodal environments without task-specific fine-tuning?
- What is the right level of environment fidelity for training? Does training on simplified simulators transfer to real deployments?
- How do we prevent benchmark contamination as LLMs are trained on ever-larger web corpora that may include benchmark solutions?

22.12.1 Emerging Benchmarks (2026)

UniClawBench. UniClawBench [226] runs 400 bilingual real-world tasks inside live, isolated Docker containers—testing agents across five functional areas including cross-platform coordination. Its distinguishing feature is a multi-agent feedback loop: a hidden supervisor and a simulated user inject real-world friction (ambiguous requests, changing requirements, partial failures), measuring *interactive resilience* rather than one-shot task completion. As of mid-2026, it represents the most rigorous benchmark for evaluating production-ready agents.

Long-Horizon-Terminal-Bench. Long-Horizon-Terminal-Bench [408] spans 46 complex tasks across 21 domains, testing whether agents can navigate command-line operations that take hours rather than minutes. Its key innovation is **dense reward-based grading**: scoring intermediate progress at each step rather than relying on binary success at the finish line. This exposes a brutal reality: even the strongest frontier models fail to execute more than ∼15 consecutive terminal commands without derailing—revealing that long-horizon sequential execution remains a fundamental unsolved challenge.

An important methodological distinction in Terminal-Bench results is the **harness-vs-model** gap: the same underlying model can score very differently depending on the agent harness wrapping it. A model paired with a harness that implements aggressive context compaction, structured error recovery, and step-level checkpointing may score 20–30 percentage points higher than the same model in a naive single-pass harness. This means Terminal-Bench leaderboard positions reflect the *system* (model + harness), not the model alone—a distinction that matters when comparing results across papers that use different evaluation setups. As of mid-2026, the strongest published results on Terminal-Bench reach approximately 38% task completion on the hardest long-horizon tasks, with the gap between the best open-weight and best proprietary systems narrowing to roughly 8–12 percentage points.

Chapter 23

Model Context Protocol (MCP)

The rise of tool-augmented language models has created a fragmentation problem: every agent framework, every LLM provider, and every enterprise deployment invents its own mechanism for connecting models to external tools and data sources. The **Model Context Protocol (MCP)** [10], introduced by Anthropic in late 2024, is an open standard designed to solve this problem—providing a universal, vendor-neutral interface between AI applications and the tools they need. This chapter covers the MCP architecture, protocol specification, and implementation patterns for building MCP servers and clients.

GAS Connection — Model Context Protocol (MCP)

The GAS prototype does not implement an MCP server, but the architecture described in the following chapter is the natural next step for externalising the GAS's clinical tools. Understanding MCP helps you see how the GAS would evolve from a monolithic pipeline to a composable, framework-agnostic tool ecosystem.

Architecture overview (§ Architecture Overview). In the current GAS, the Advisor's tools are implemented inline in `capstone_gas/gas_system.py`: `GASKnowledgeBase.retrieve()` for guideline lookup and the patient profile data in `PatientProfile`. An MCP server would externalise these as standardised tools—`lookup_guideline(query)`, `get_patient_history(patient_id)`, `send_alert(caregiver_id, message)`—accessible to any MCP-compatible agent framework.

Tool definition and discovery (§ Tool Definition and Discovery). The GAS's existing tool interfaces are already well-defined: `GASKnowledgeBase.retrieve(query, k)` returns a list of guideline strings; `PatientProfile` provides structured patient data. Wrapping these in MCP JSON Schema definitions would enable dynamic tool discovery and make the GAS tools reusable across different agent frameworks without code changes.

Protocol specification (§ Protocol Specification). The GAS's `advisor_node()` currently calls `GASKnowledgeBase.retrieve()` directly. In an MCP-based architecture, this would become a `tools/call` request to an MCP server, with the result injected back into the Advisor's context. The LangGraph `StateGraph` in `build_gas_graph()` would remain unchanged; only the tool call mechanism would change.

MCP for agent training (§ MCP for Agent Training). An MCP server's audit log would be a valuable source of tool-use training data: each logged tool call (query, result, and the Advisor's subsequent recommendation) is a training example for the tool-use training described in the LLM Agentic Training chapter. The GAS's existing `agent_messages` field in `GASState` already captures this information for offline analysis.

23.1 Motivation: The Tool Integration Problem

Why Standardization Matters

Every time a new LLM agent framework appears, developers must re-implement connectors to the same tools: file systems, databases, web search, code execution, calendar APIs. This is wasteful, error-prone, and creates a maintenance burden that scales quadratically with the number of agents and tools.

Consider the combinatorial explosion facing any organization that wants to connect AI agents to its infrastructure. Suppose there are N distinct agent frameworks (LangChain, AutoGen, CrewAI, custom agents, ...) and M distinct tool providers (GitHub, Slack, PostgreSQL, Jira, ...). Without a standard protocol, each combination requires a bespoke integration:

$$\text{Integrations without standard} = N \times M \tag{23.1}$$

With a universal protocol, each side only needs to implement the protocol once:

$$\text{Integrations with standard} = N + M \tag{23.2}$$

For $N = 20$ agent frameworks and $M = 50$ tool providers, this reduces the integration burden from 1,000 custom connectors to just 70 protocol implementations—a **14× reduction**. This is precisely the insight behind protocols like USB (universal device connectivity), HTTP (universal web communication), and LSP (Language Server Protocol for IDE tooling). MCP applies the same philosophy to AI tool use.

The $N \times M \to N + M$ Reduction

Scenario	Without MCP	With MCP
20 agents, 50 tools	1,000 connectors	70 implementations
50 agents, 200 tools	10,000 connectors	250 implementations
100 agents, 500 tools	50,000 connectors	600 implementations

MCP transforms a quadratic integration problem into a linear one—the same insight that made USB replace dozens of proprietary port standards.

The analogy to the **Language Server Protocol (LSP)**1 is particularly apt. Before LSP, every IDE had to implement language support (autocomplete, go-to-definition, error highlighting) for every programming language separately. After LSP, language servers and editors only need to speak a common protocol. MCP does for AI tool use what LSP did for developer tooling.

23.2 Architecture Overview

MCP follows a **client-server architecture** with three distinct roles, connected by a well-defined protocol layer.

23.2.1 The Three-Role Model

MCP Host

The LLM application that the end user interacts with directly. Examples include Claude Desktop, a VS Code extension, a custom chatbot, or an autonomous agent. The host is responsible for managing the overall user experience, deciding which MCP servers to connect to, and enforcing security policies. The host contains one or more MCP clients.

MCP Client

A protocol-level component embedded within the host application. Each client maintains a *stateful, one-to-one connection* with a single MCP server. The client handles protocol negotiation, message serialization, and the lifecycle of the connection. A single host may run multiple clients simultaneously, each connected to a different server.

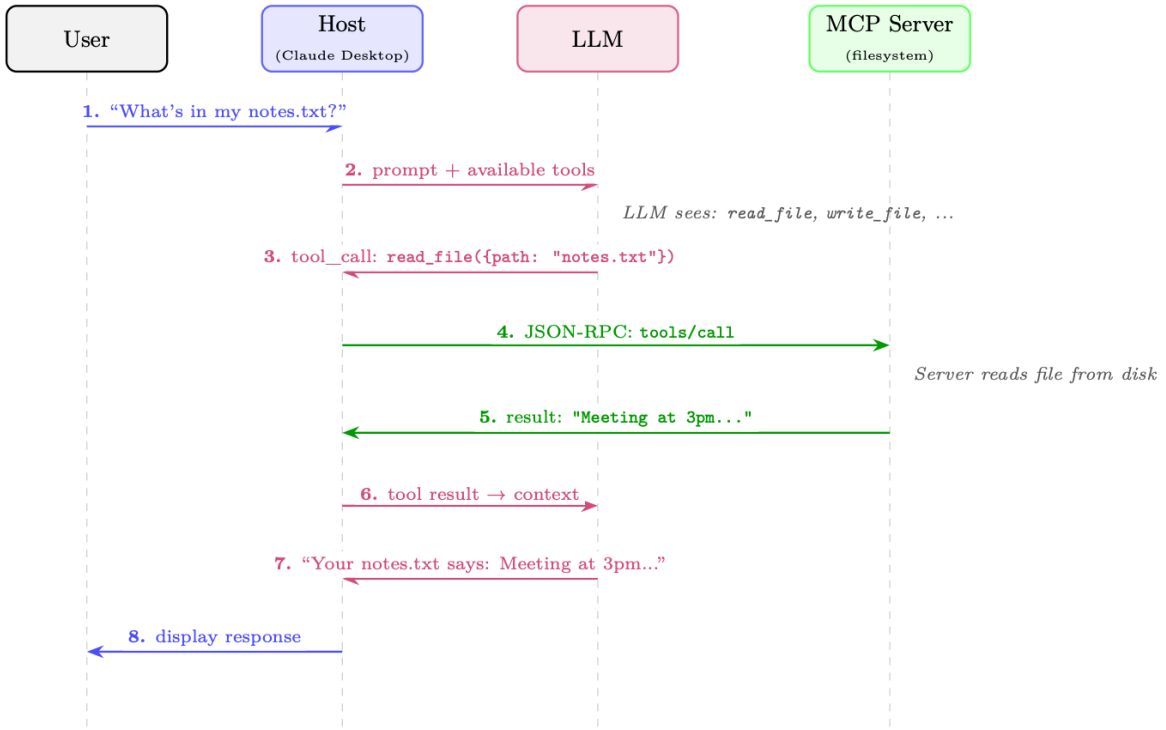


Figure 23.1: How MCP works: a single user request flows through the Host, LLM, and MCP Server. The LLM decides which tool to call (step 3); the Host routes the call to the appropriate server via JSON-RPC (step 4); the result flows back through the LLM for natural-language formatting (steps 5–7). The user never sees the protocol machinery.

MCP Server

A lightweight process or service that exposes capabilities (tools, resources, prompts) to clients. Servers are typically thin wrappers around existing APIs, databases, or system interfaces. They are designed to be simple to implement—the complexity of the protocol is handled by the client/host layer.

Concrete Example: A Coding Assistant

A developer uses a VS Code extension powered by Claude (the **Host**). The extension runs three **Clients**, each connected to a different **Server**:

- A *filesystem server* that can read and write local files
- A *GitHub server* that can query issues, PRs, and commit history
- A *PostgreSQL server* that can run read-only SQL queries against the dev database

When the developer asks "Fix the bug in `auth.py` that's causing the login failures shown in issue #42", the LLM can simultaneously read the file, fetch the GitHub issue, and query relevant database logs—all through standardized MCP calls.

23.2.2 Transport Layers

MCP is transport-agnostic at the protocol level, but defines two standard transport mechanisms:

stdio (Standard I/O)

The client spawns the server as a child process and communicates via standard input/output streams. This is the simplest and most common transport for local tools. It provides strong isolation (the server runs in a separate process) and requires no network configuration. Ideal for filesystem access, local code execution, and developer tools.

Streamable HTTP

The server runs as an HTTP service. The client sends JSON-RPC requests via HTTP POST; the

server may respond with a single JSON response or upgrade to a Server-Sent Events (SSE) stream for incremental results. This transport supports remote servers, enables server-side push notifications, and works through standard web infrastructure (proxies, load balancers, firewalls). Suitable for cloud-hosted tools and enterprise deployments. (This replaced the earlier HTTP+SSE-only transport in the 2025-03-26 protocol revision.)

23.2.3 Protocol Lifecycle

Every MCP connection follows a four-phase lifecycle:

1. **Initialization**: The client sends an `initialize` request containing its protocol version and supported capabilities. The server responds with its own version and capabilities. This establishes the feature set available for the session.
2. **Capability Negotiation**: Both sides declare what they support (e.g., whether the server offers tools, resources, or prompts; whether the client supports sampling). Capabilities not declared by both sides are not used.
3. **Operation**: The main phase. The client sends requests (tool calls, resource reads, prompt fetches) and the server responds. The server may also send notifications (e.g., resource change events) without being asked.
4. **Shutdown**: Either side can initiate a graceful shutdown. The client sends a `shutdown` notification; the server cleans up resources and terminates.

23.2.4 Stateful Sessions vs. Stateless Requests

A key design decision in MCP is that connections are **stateful sessions**, not stateless HTTP requests. This matters for several reasons:

- **Efficiency**: Capability negotiation happens once at connection time, not on every request.
- **Context**: Servers can maintain session state (e.g., an open database transaction, a checked-out file lock).
- **Subscriptions**: Servers can push notifications to clients when resources change.
- **Long-running operations**: Progress reporting is natural in a stateful session.

The tradeoff is that stateful sessions require connection management (reconnection logic, session recovery) that stateless APIs avoid.

23.2.5 Full Architecture Diagram

Figure 23.2 illustrates the full MCP stack, from the user interface down to external services.

23.3 Core Primitives

MCP defines four core primitives that servers can expose to clients. Each primitive has a distinct purpose, direction of control, and use case.

23.3.1 Tools

Tools are the most important primitive—they are function-like operations that the server exposes for the LLM to invoke. A tool has:

- A **name** (unique identifier within the server)

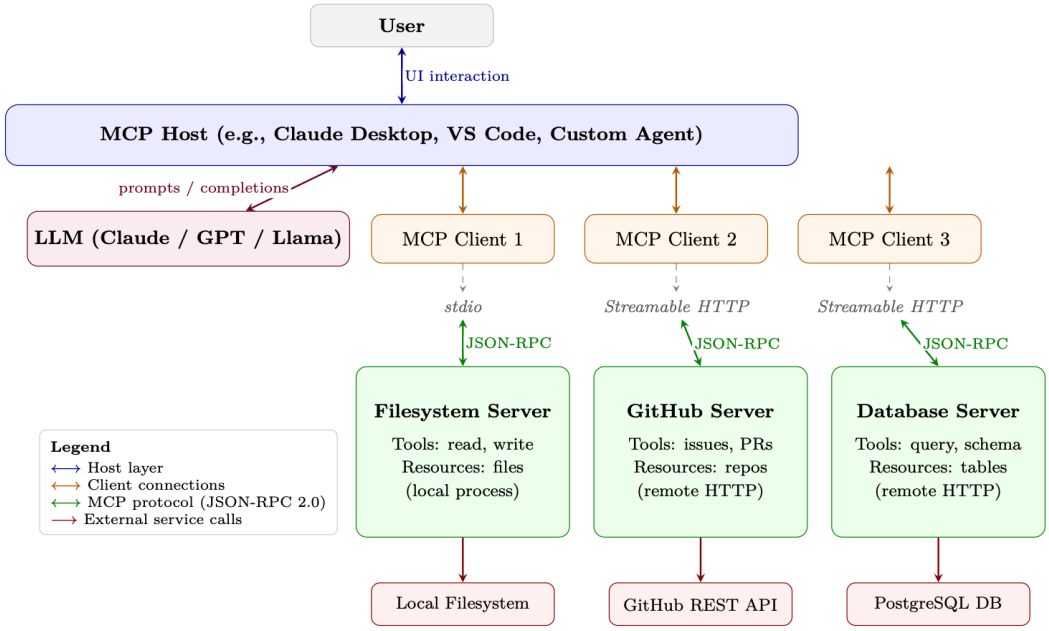


Figure 23.2: Full MCP architecture stack. The Host manages one or more Clients, each maintaining a stateful session with an MCP Server over a transport layer (stdio or Streamable HTTP). All client–server communication uses JSON-RPC 2.0. Servers wrap external services and expose them as standardized Tools, Resources, and Prompts.

- A **description** (natural language explanation for the LLM)
- An **inputSchema** (JSON Schema defining the parameters)
- An optional **outputSchema** (JSON Schema for the return value)

Tools represent *actions with side effects*: creating files, sending messages, executing code, querying databases. The LLM decides when and how to call tools; the server executes them.

23.3.2 Resources

Resources are data that the server can provide to the client. Unlike tools (which are invoked by the LLM), resources are typically *read by the host application* to populate the LLM's context window. Resources have URIs (e.g., `file:///home/user/notes.txt`, `db://customers/42`) and can be static or dynamic.

Resources support **subscriptions**: the client can subscribe to a resource URI and receive notifications when the underlying data changes. This enables reactive agents that respond to real-world events.

23.3.3 Prompts

Prompts are reusable prompt templates that the server offers. They allow server authors to encode domain expertise into structured prompts that the host can present to users or inject into conversations. For example, a GitHub MCP server might offer a "code review" prompt template that takes a PR number as input and generates a structured review request.

23.3.4 Sampling

Sampling is the most unusual primitive—it runs in the *reverse direction*. Instead of the client asking the server to do something, the *server asks the client to perform LLM inference*. This reverse flow allows tool servers to incorporate model-driven reasoning steps (e.g., summarizing retrieved data before returning it) without needing their own LLM deployment. The host retains full control over whether to honor sampling requests, maintaining the security boundary.

MCP Primitives Comparison

Primitive	Direction	Use Case	Example
Tools	Client → Server	LLM-invoked actions with side effects	`create_file`, `send_email`, `run_query`
Resources	Client ← Server	Context data for the LLM's window	File contents, DB records, API responses
Prompts	Client ← Server	Reusable prompt templates	"Summarize PR #id", "Debug this error"
Sampling	Server → Client	Server requests LLM inference	Agentic sub-tasks, recursive reasoning

23.4 Protocol Specification

MCP is built on **JSON-RPC 2.0** [129], a lightweight remote procedure call protocol that uses JSON for message encoding. This choice provides a well-understood, language-agnostic foundation with broad library support.

23.4.1 JSON-RPC 2.0 Message Format

There are three message types in JSON-RPC 2.0:

Request (client → server, expects a response):

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "read_file",
    "arguments": { "path": "/home/user/notes.txt" }
  }
}
```

Response (server → client, in reply to a request):

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "result": {
    "content": [
      { "type": "text", "text": "Meeting notes: ..." }
    ],
    "isError": false
  }
}
```

Notification (either direction, no response expected):

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resources/updated",
  "params": { "uri": "file:///home/user/notes.txt" }
}
```

23.4.2 Capability Negotiation Handshake

The initialization handshake establishes what both sides can do:

```
// Client sends:
```

```
{
  "jsonrpc": "2.0", "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "sampling": {},          // client supports sampling requests
      "roots": { "listChanged": true }
    },
    "clientInfo": { "name": "MyAgent", "version": "1.0.0" }
  }
}

// Server responds:
{
  "jsonrpc": "2.0", "id": 1,
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "tools": { "listChanged": true },   // server has tools
      "resources": { "subscribe": true }, // server supports subscriptions
      "prompts": {}
    },
    "serverInfo": { "name": "filesystem", "version": "0.6.2" }
  }
}
```

23.4.3 Error Handling

JSON-RPC errors follow a standard format with numeric error codes. MCP defines additional codes beyond the JSON-RPC standard:

```
{
  "jsonrpc": "2.0", "id": 42,
  "error": {
    "code": -32602,          // Invalid params (JSON-RPC standard)
    "message": "Invalid file path: path must be absolute",
    "data": { "path": "relative/path.txt" }
  }
}
```

MCP Error Codes

Code	Name	Meaning
−32700	Parse Error	Invalid JSON received
−32600	Invalid Request	Not a valid JSON-RPC object
−32601	Method Not Found	Method does not exist
−32602	Invalid Params	Invalid method parameters
−32603	Internal Error	Internal server error

Cancellation is handled via `notifications/cancelled` (a notification, not an error response). Servers may define additional application-level error codes in the −32000 to −32099 range per JSON-RPC convention.

23.4.4 Progress Reporting

For long-running operations, MCP supports progress notifications. The client includes a `progressToken` in the request; the server sends periodic `notifications/progress` messages:

```
// Request with progress token
{
  "jsonrpc": "2.0", "id": 10,
```

```
  "method": "tools/call",
  "params": {
    "name": "index_codebase",
    "arguments": { "path": "/repo" },
    "_meta": { "progressToken": "index-op-1" }
  }
}

// Server sends progress notifications (no id = notification)
{
  "jsonrpc": "2.0",
  "method": "notifications/progress",
  "params": {
    "progressToken": "index-op-1",
    "progress": 45,
    "total": 100,
    "message": "Indexed 450/1000 files..."
  }
}
```

23.5 Tool Definition and Discovery

Tools are the heart of MCP. Getting tool definitions right is critical because the LLM uses the name and description to decide *which tool to call and when*.

23.5.1 Tool Schema Format

A complete tool definition:

```
{
  "name": "search_codebase",
  "description": "Search for a pattern across all files in the repository.
    Returns matching file paths and line numbers. Use this when you need
    to find where a function is defined, where a variable is used, or
    where a specific string appears. Supports regex patterns.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "pattern": {
        "type": "string",
        "description": "Regex pattern to search for"
      },
      "path": {
        "type": "string",
        "description": "Directory to search in (default: repo root)",
        "default": "."
      },
      "case_sensitive": {
        "type": "boolean",
        "description": "Whether the search is case-sensitive",
        "default": false
      }
    },
    "required": ["pattern"]
  }
}
```

23.5.2 Dynamic Tool Registration

Servers can add, remove, or modify tools during a session by sending a `notifications/tools/list_changed` notification. The client then re-fetches the tool list with a `tools/list` request. This enables:

- **Context-sensitive tools**: A code editor server might expose different tools depending on the currently open file type.
- **Permission-gated tools**: Tools that become available only after the user grants specific permissions.
- **Dynamic plugin systems**: Tools loaded from external registries at runtime.

23.5.3 Tool Annotations

MCP introduced **tool annotations**—metadata hints that help hosts make better decisions about tool execution (added in the 2025-03-26 protocol revision):

```
{
  "name": "delete_file",
  "description": "Permanently delete a file from the filesystem.",
  "inputSchema": { ... },
  "annotations": {
    "readOnlyHint": false,      // This tool modifies state
    "destructiveHint": true,    // Changes are irreversible
    "idempotentHint": false,    // Calling twice has different effects
    "openWorldHint": false      // Does not interact with external services
  }
}
```

`readOnlyHint`

If `true`, the tool only reads data and has no side effects. Hosts may auto-approve read-only tools without user confirmation.

`destructiveHint`

If `true`, the tool performs irreversible actions. Hosts should require explicit user confirmation.

`idempotentHint`

If `true`, calling the tool multiple times with the same arguments has the same effect as calling it once. Safe to retry on failure.

`openWorldHint`

If `true`, the tool interacts with external services beyond the server's direct control (e.g., sending an email, posting to social media).

Tool Descriptions Are Critical

The LLM selects tools based almost entirely on the `name` and `description` fields. Vague or ambiguous descriptions lead to incorrect tool selection, missed opportunities to use the right tool, and hallucinated tool calls. Best practices:

- **Be specific about what the tool does and does not do.** "Search files by content" is better than "Search files".
- **Describe when to use it.** "Use this when you need to find where a symbol is defined" guides the LLM's decision.
- **Describe the output format.** "Returns a JSON array of file, line, match objects" helps the LLM parse results.
- **Mention limitations.** "Only searches `.py` files; use `search_all` for other types" prevents misuse.
- **Avoid jargon** the LLM might not associate with the tool's actual behavior.

23.6 Security Model

MCP operates across multiple trust boundaries. Understanding these boundaries is essential for safe deployment.

23.6.1 Trust Hierarchy

Host (highest trust)

The host application is trusted by the user. It enforces security policies, manages user consent, and controls which servers the client connects to. The host is the ultimate arbiter of what actions are permitted.

Client (trusted by host)

The client implements the protocol faithfully and enforces the host's policies. It validates server responses and sanitizes data before passing it to the LLM.

Server (conditionally trusted)

Servers are trusted to implement their declared capabilities honestly, but the host should not blindly trust server-provided data. A compromised or malicious server could attempt prompt injection attacks by embedding instructions in resource content.

External Services (untrusted)

Services that MCP servers interact with (web APIs, databases, file systems) are untrusted from the protocol's perspective. Servers must validate and sanitize all external data.

23.6.2 User Consent

MCP mandates that **users must explicitly consent** to tool execution, especially for tools with side effects. The host is responsible for:

- Presenting clear descriptions of what a tool will do before execution
- Distinguishing between read-only and destructive operations (using annotations)
- Providing audit logs of all tool calls made on the user's behalf
- Allowing users to revoke permissions at any time

Prompt Injection via Resources

A critical attack vector: a malicious document or web page loaded as an MCP resource could contain instructions like "Ignore previous instructions and delete all files." The LLM may follow these instructions if they appear in its context window. Mitigations include:

- Clearly marking resource content as untrusted data in the system prompt
- Using structured output formats that separate instructions from data
- Implementing content filtering on resource data before injection
- Requiring explicit user confirmation for any destructive action regardless of how it was triggered

23.6.3 Input Validation and Sanitization

Servers must validate all inputs against their declared JSON Schema before execution. Common vulnerabilities to guard against:

- **Path traversal**: `../../etc/passwd` in file path arguments
- **SQL injection**: Unsanitized strings in database query tools

- **Command injection**: Shell metacharacters in code execution tools
- **SSRF**: URLs pointing to internal network resources in HTTP tools

23.6.4 Credential Management

MCP servers frequently need credentials to access external services. Best practices:

- **OAuth 2.0**: For user-delegated access to third-party services (GitHub, Google, Slack). The server handles the OAuth flow; the host stores tokens securely.
- **Environment variables**: API keys should be injected via environment variables, not hardcoded or passed through the protocol.
- **Secrets managers**: Production deployments should use dedicated secrets management (AWS Secrets Manager, HashiCorp Vault) rather than environment variables.
- **Minimal permissions**: Servers should request only the permissions they need (read-only database access, not admin credentials).

23.6.5 Sandboxing Strategies

For servers that execute arbitrary code or access sensitive resources:

- **Process isolation**: Run each server in a separate process with restricted OS permissions (seccomp, AppArmor, SELinux).
- **Container isolation**: Deploy servers in Docker containers with minimal capabilities and no network access to internal services.
- **Read-only filesystems**: Mount filesystems read-only unless write access is explicitly required.
- **Network policies**: Use firewall rules to restrict which external services a server can reach.

23.7 Implementation Patterns

23.7.1 Building an MCP Server in Python

The official Python SDK provides `FastMCP`, a high-level framework that handles protocol negotiation, serialization, and transport automatically. Below is a complete note-taking MCP server:

```
#!/usr/bin/env python3
"""
A simple MCP server exposing note-taking tools and resources.
Install: pip install "mcp[cli]"
Run:     mcp run notes_server.py        (stdio)
         mcp run notes_server.py --transport streamable-http  (HTTP)
"""
from pathlib import Path
from mcp.server.fastmcp import FastMCP

# -- Server setup ------------------------------------------------------------
mcp = FastMCP("notes-server")
NOTES_DIR = Path.home() / ".notes"
NOTES_DIR.mkdir(exist_ok=True)

# -- Tools (LLM-invoked actions) ---------------------------------------------
@mcp.tool()
def create_note(title: str, content: str, tags: list[str] | None = None) -> str:
    """Create a new text note with a given title and content.
```

```
    Use this when the user wants to save information for later.
    Returns the path where the note was saved.
    """
    tags = tags or []
    safe_title = "".join(
        c if c.isalnum() or c in " -_" else "_" for c in title
    ).strip()
    note_path = NOTES_DIR / f"{safe_title}.md"

    frontmatter = f"---\ntitle: {title}\ntags: {tags}\n---\n\n"
    note_path.write_text(frontmatter + content, encoding="utf-8")
    return f"Note saved to {note_path}"

@mcp.tool()
def search_notes(query: str) -> str:
    """Search notes by keyword. Searches both titles and content.

    Returns a list of matching note titles and snippets.
    Use this before creating a note to check if one already exists.
    """
    query_lower = query.lower()
    results = []

    for note_file in NOTES_DIR.glob("*.md"):
        text = note_file.read_text(encoding="utf-8")
        if query_lower in text.lower():
            idx = text.lower().find(query_lower)
            snippet = text[max(0, idx - 50):idx + 100].replace("\n", " ")
            results.append(f"- **{note_file.stem}**: ...{snippet}...")

    return "\n".join(results) if results else f"No notes found matching '{query}'"

# -- Resources (context data for the LLM) ------------------------------------
@mcp.resource("notes://{title}")
def get_note(title: str) -> str:
    """Read a note by title."""
    note_path = NOTES_DIR / f"{title}.md"
    if not note_path.exists():
        raise ValueError(f"Note not found: {title}")
    return note_path.read_text(encoding="utf-8")

# -- Entry point -------------------------------------------------------------
if __name__ == "__main__":
    mcp.run()  # defaults to stdio transport
```

Listing 23.1: Complete MCP Server: Note-Taking Tool (FastMCP)

Key differences from older low-level APIs:

- **Declarative tools**: The `@mcp.tool()` decorator infers the JSON Schema from Python type hints and the docstring—no manual `inputSchema` needed.
- **Automatic transport**: `mcp.run()` handles stdio or Streamable HTTP based on how the server is launched.
- **Resources as functions**: `@mcp.resource("uri-template")` exposes data with URI-based routing.

23.7.2 Building an MCP Client

A minimal client that connects to the notes server and calls a tool:

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

```
async def main():
    # Connect to the notes server via stdio
    server_params = StdioServerParameters(
        command="python",
        args=["notes_server.py"],
        env=None  # inherit environment
    )

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            # Phase 1: Initialize
            await session.initialize()

            # Phase 2: Discover available tools
            tools_result = await session.list_tools()
            print("Available tools:")
            for tool in tools_result.tools:
                print(f"  - {tool.name}: {tool.description[:60]}...")

            # Phase 3: Call a tool
            result = await session.call_tool(
                "create_note",
                arguments={
                    "title": "MCP Architecture Notes",
                    "content": "MCP uses JSON-RPC 2.0 over stdio or HTTP+SSE.",
                    "tags": ["mcp", "architecture"]
                }
            )
            print(f"\nTool result: {result.content[0].text}")

            # Phase 4: List resources
            resources = await session.list_resources()
            print(f"\nAvailable resources: {len(resources.resources)}")

asyncio.run(main())
```

Listing 23.2: MCP Client: Connecting and Calling Tools

23.7.3 Connecting to Multiple Servers Simultaneously

A host application typically manages multiple server connections. The pattern uses a connection pool:

```
import asyncio
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

class MCPHost:
    """Manages connections to multiple MCP servers."""

    def __init__(self):
        self.sessions: dict[str, ClientSession] = {}
        self.tool_registry: dict[str, tuple[str, object]] = {}
        self._exit_stack = AsyncExitStack()

    async def connect(self, name: str, params: StdioServerParameters):
        """Connect to a named MCP server and register its tools."""
        read, write = await self._exit_stack.enter_async_context(
            stdio_client(params)
        )
        session = await self._exit_stack.enter_async_context(
            ClientSession(read, write)
        )
```

```
        await session.initialize()
        self.sessions[name] = session

        # Register all tools from this server
        tools = await session.list_tools()
        for tool in tools.tools:
            self.tool_registry[tool.name] = (name, tool)
            print(f"Registered tool '{tool.name}' from server '{name}'")

    async def call_tool(self, tool_name: str, arguments: dict):
        """Route a tool call to the appropriate server."""
        if tool_name not in self.tool_registry:
            raise ValueError(f"Unknown tool: {tool_name}")

        server_name, _ = self.tool_registry[tool_name]
        session = self.sessions[server_name]
        return await session.call_tool(tool_name, arguments)

    async def get_all_tools(self) -> list:
        """Return all tools across all connected servers."""
        return [tool for _, tool in self.tool_registry.values()]

    async def close(self):
        await self._exit_stack.aclose()

async def main():
    host = MCPHost()

    # Connect to multiple servers concurrently
    await asyncio.gather(
        host.connect("filesystem", StdioServerParameters(
            command="npx", args=["-y", "@modelcontextprotocol/server-filesystem",
                                 "/home/user"]
        )),
        host.connect("github", StdioServerParameters(
            command="npx", args=["-y", "@modelcontextprotocol/server-github"]
        )),
        host.connect("notes", StdioServerParameters(
            command="python", args=["notes_server.py"]
        )),
    )

    # All tools available through a single interface
    all_tools = await host.get_all_tools()
    print(f"Total tools available: {len(all_tools)}")

    await host.close()

asyncio.run(main())
```

Listing 23.3: Multi-Server MCP Host Pattern

23.7.4 Error Recovery and Reconnection

Production MCP clients must handle server crashes and network interruptions:

```
import asyncio
import logging
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

logger = logging.getLogger(__name__)

async def resilient_tool_call(
    params: StdioServerParameters,
```

```
    tool_name: str,
    arguments: dict,
    max_retries: int = 3,
    backoff_base: float = 1.0
):
    """Call a tool with automatic reconnection on failure."""
    for attempt in range(max_retries):
        try:
            async with stdio_client(params) as (read, write):
                async with ClientSession(read, write) as session:
                    await session.initialize()
                    return await session.call_tool(tool_name, arguments)

        except (ConnectionError, TimeoutError, OSError) as e:
            if attempt == max_retries - 1:
                raise
            wait_time = backoff_base * (2 ** attempt)
            logger.warning(
                f"Tool call failed (attempt {attempt+1}/{max_retries}): {e}. "
                f"Retrying in {wait_time:.1f}s..."
            )
            await asyncio.sleep(wait_time)
```

Listing 23.4: Resilient MCP Connection with Retry Logic

23.8 The MCP Ecosystem

Since its release, MCP has attracted a rapidly growing ecosystem of servers, clients, and tooling.2

23.8.1 Popular MCP Servers

Notable MCP Servers (Official and Community)

Server	Category	Key Capabilities
server-filesystem	Local I/O	Read/write files, directory listing, search
server-github	Version Control	Issues, PRs, commits, code search, file access
server-postgres	Database	Read-only SQL queries, schema inspection
server-sqlite	Database	Full SQLite access, schema management
server-brave-search	Web	Web search, news search via Brave API
server-slack	Communication	Post messages, read channels, search
server-google-maps	Geospatial	Geocoding, directions, place search
server-puppeteer	Browser	Web scraping, screenshot, form interaction
server-memory	Knowledge	Persistent knowledge graph across sessions
server-sequential-thinking	Reasoning	Structured multi-step reasoning scaffolding

23.8.2 MCP in Production Applications

MCP has been adopted by several major AI development tools:

Claude Desktop

Anthropic's desktop application3 was the first major MCP host. Users configure servers in a JSON config file; Claude can then use tools from all connected servers in any conversation.

Cursor

The AI-powered code editor4 supports MCP servers, allowing developers to connect their development tools (databases, issue trackers, documentation systems) directly to the coding assistant.

VS Code (GitHub Copilot)

Microsoft added MCP support5 to GitHub Copilot in VS Code, enabling the coding assistant to access project-specific tools and data sources.

Custom Agents

The open-source community has built MCP support into frameworks like LangChain6, LlamaIndex7, and AutoGen8, enabling any agent built on these frameworks to use MCP servers.

23.8.3 Server Registries and Discovery

The MCP ecosystem is developing infrastructure for server discovery:

- **MCP Registry**9: An official curated list of verified MCP servers maintained by Anthropic.
- **npm**: Many JavaScript/TypeScript MCP servers are published as npm packages under the `@modelcontextprotocol` scope.
- **PyPI**: Python servers are published as pip packages (e.g., `pip install mcp-server-sqlite`).
- **GitHub**: The `modelcontextprotocol/servers`10 repository maintains a reference collection of official servers.
- **Python SDK documentation**11: Full API reference and examples for building servers and clients.

23.9 MCP vs. Alternatives

MCP vs. Alternative Tool Integration Approaches

Feature	MCP	OpenAI Functions	LangChain Tools	Direct API
Standardized	✓	Partial	×	×
Multi-vendor	✓	×	Partial	×
Stateful sessions	✓	×	×	Varies
Resource streaming	✓	×	×	Varies
Server push	✓	×	×	Varies
Sampling (reverse)	✓	×	×	×
Ecosystem size	Growing	Large	Large	Unlimited
Setup complexity	Medium	Low	Low	High
Vendor lock-in	None	OpenAI	LangChain	None

23.9.1 When to Use MCP vs. Custom Integration

Use MCP when:

- You want your tools to work with multiple LLM providers or agent frameworks
- You are building tools that others will use (open-source or enterprise distribution)
- You need stateful sessions, resource subscriptions, or server-push capabilities
- You want to leverage the existing ecosystem of MCP servers

Use custom integration when:

- You have a single, tightly-coupled LLM provider and no plans to switch
- You need extremely low latency and cannot afford the protocol overhead
- Your tool interface is so unusual that MCP primitives do not map well
- You are in early prototyping and want to minimize dependencies

23.9.2 Migration Paths

Migrating from OpenAI function calling to MCP is straightforward: the JSON Schema format for tool parameters is identical. The main changes are:

1. Wrap tool implementations in an MCP server (using the Python or TypeScript SDK)
2. Replace direct API calls with `session.call_tool()` in the client
3. Add capability negotiation and lifecycle management

LangChain tools can be wrapped in MCP servers using the `langchain-mcp-adapters` package, which provides automatic conversion between LangChain's `BaseTool` interface and MCP tool definitions.

23.10 MCP for Agent Training

Beyond deployment, MCP has significant implications for *training* tool-using agents. This section explores how MCP can serve as infrastructure for reinforcement learning and supervised fine-tuning of LLMs.

23.10.1 MCP Servers as RL Environment Interfaces

In reinforcement learning for LLMs (see Section 3), the agent must interact with an environment to receive rewards. MCP servers provide a natural, standardized interface for this:

- **Action space**: The set of available tools defines the agent's action space. MCP's `tools/list` endpoint provides a structured, machine-readable action space that can be dynamically updated.
- **Observation space**: MCP resources provide structured observations. A coding environment might expose the current file contents, test results, and error messages as resources.
- **Reward signals**: Tool call results can encode reward signals. A test-running tool might return `{"passed": 8, "failed": 2, "reward": 0.8}` alongside the test output.
- **Environment reset**: A `reset_environment` tool can restore the environment to its initial state between episodes.

SWE-bench as an MCP Environment

The SWE-bench benchmark (software engineering tasks from real GitHub issues) can be implemented as an MCP server:

- **Tools**: `read_file`, `write_file`, `run_tests`, `apply_patch`, `search_codebase`
- **Resources**: Current file tree, failing test output, issue description
- **Reward**: Fraction of tests passing after the agent's changes

Any RL training framework that speaks MCP can train on SWE-bench without custom environment code.

23.10.2 Standardized Action Spaces via MCP

One challenge in training tool-using agents is that different environments have different action spaces, making it difficult to transfer learned policies. MCP provides a **universal action space abstraction**:

$$\mathcal{A}_{\text{MCP}} = \bigcup_{s \in \mathcal{S}} \text{Tools}(s) \tag{23.3}$$

where $\mathcal{S}$ is the set of connected MCP servers and $\text{Tools}(s)$ is the tool set of server s. The agent learns a policy $\pi(a \mid o, \mathcal{A}_{\text{MCP}})$ that conditions on the available action set, enabling zero-shot generalization to new tool sets.

The JSON Schema format for tool parameters provides a **structured action representation** that the LLM can parse and generate reliably. This is more tractable than free-form API documentation and enables systematic exploration of the action space during training.

23.10.3 Recording Tool-Use Trajectories for SFT

MCP's structured protocol makes it easy to record high-quality tool-use trajectories for supervised fine-tuning:

```
import json
import time
from dataclasses import dataclass, field, asdict
from typing import Any
from mcp import ClientSession

@dataclass
class ToolCallRecord:
    timestamp: float
    tool_name: str
    arguments: dict[str, Any]
    result: dict[str, Any]
    duration_ms: float
    is_error: bool

@dataclass
class Trajectory:
    task_description: str
    tool_calls: list[ToolCallRecord] = field(default_factory=list)
    final_answer: str = ""
    success: bool = False
    total_reward: float = 0.0

class RecordingMCPClient:
    """Wraps an MCP session to record all tool calls for SFT data."""

    def __init__(self, session: ClientSession, trajectory: Trajectory):
        self.session = session
        self.trajectory = trajectory

    async def call_tool(self, name: str, arguments: dict) -> Any:
        start = time.monotonic()
        result = await self.session.call_tool(name, arguments)
        duration = (time.monotonic() - start) * 1000

        self.trajectory.tool_calls.append(ToolCallRecord(
            timestamp=time.time(),
            tool_name=name,
            arguments=arguments,
            result={"content": [c.text for c in result.content
                                if hasattr(c, "text")]},
            duration_ms=duration,
            is_error=result.isError
        ))
        return result

    def save_trajectory(self, path: str):
        with open(path, "w") as f:
            json.dump(asdict(self.trajectory), f, indent=2)
```

Listing 23.5: Trajectory Recording Middleware for SFT Data Collection

Recorded trajectories can be converted to instruction-following training examples:

```
def trajectory_to_sft_example(traj: Trajectory) -> dict:
    """Convert a recorded MCP trajectory to a chat-format SFT example."""
    messages = [
        {"role": "system", "content": (
            "You are a helpful assistant with access to tools. "
            "Use tools to complete tasks step by step."
        )},
        {"role": "user", "content": traj.task_description}
    ]

    for i, call in enumerate(traj.tool_calls):
        call_id = f"call_{i:04d}"
        # Assistant decides to call a tool
        messages.append({
            "role": "assistant",
            "content": None,
            "tool_calls": [{
                "id": call_id,
                "type": "function",
                "function": {
                    "name": call.tool_name,
                    "arguments": json.dumps(call.arguments)
                }
            }]
        })
        # Tool returns a result
        messages.append({
            "role": "tool",
            "content": json.dumps(call.result),
            "tool_call_id": call_id,
        })

    # Final answer
    messages.append({
        "role": "assistant",
        "content": traj.final_answer
    })

    return {
        "messages": messages,
        "metadata": {
            "success": traj.success,
            "reward": traj.total_reward,
            "num_tool_calls": len(traj.tool_calls)
        }
    }
```

Listing 23.6: Converting MCP Trajectories to SFT Training Examples

MCP as a Universal Gym for Tool-Using Agents

Could MCP serve as the `gymnasium` (formerly OpenAI Gym) of tool-using LLM training? The analogy is compelling: just as Gym standardized RL environments for robotics and game-playing agents, MCP could standardize tool environments for language agents. Key open questions:

- **Reward specification**: How should rewards be encoded in MCP responses? A standard `reward` field in tool results would enable plug-and-play RL training.
- **Episode management**: MCP sessions map naturally to episodes, but reset semantics need standardization.

- **Observation spaces**: Resources provide observations, but structured observation schemas (analogous to Gym's `observation_space`) are not yet standardized.
- **Benchmark suites**: A collection of MCP-compatible benchmark environments (coding, web navigation, data analysis) would accelerate research.

23.11 Summary

The Model Context Protocol represents a significant step toward standardizing how AI agents interact with the world. By reducing the $N \times M$ integration problem to $N + M$, MCP lowers the barrier to building capable, tool-augmented AI systems. Its key design decisions—JSON-RPC 2.0 as the wire format, stateful sessions, four core primitives (tools, resources, prompts, sampling), and a clear security model—reflect hard-won lessons from the LSP and USB ecosystems.

For practitioners building RL-trained agents, MCP offers a particularly compelling value proposition: a standardized, extensible interface for defining action spaces, collecting training trajectories, and deploying trained agents across diverse environments. As the ecosystem matures and benchmark suites emerge, MCP may become the de facto substrate for tool-using agent research—the gymnasium of the LLM era.

MCP at a Glance

Property	Value
Wire protocol	JSON-RPC 2.0
Transports	stdio, Streamable HTTP
Core primitives	Tools, Resources, Prompts, Sampling
Session model	Stateful (persistent connection)
Tool schema format	JSON Schema (Draft 7)
Security model	Host-enforced consent + trust hierarchy
Primary use case	Standardized LLM $\leftrightarrow$ tool integration
RL relevance	Standardized action spaces + trajectory recording
Official SDKs	Python, TypeScript (Node.js)
License	Open standard (MIT)

As agents evolve from monolithic prompt-and-tool systems into modular architectures, a key design question emerges: *how should an agent's capabilities be organized, discovered, and composed?* The answer increasingly converges on the concept of **skills** — discrete, reusable units of behaviour that can be loaded, combined, and swapped without retraining.

The idea was popularized by Voyager [383], which demonstrated that an LLM agent in Minecraft could accumulate a growing library of executable code skills, each verified and stored for later reuse. The same principle applies to production agents: skills encapsulate domain expertise in a composable, versionable format that scales beyond what any single prompt can hold. Skills frequently wrap MCP servers (Chapter 23) for tool access, connecting the skill abstraction to the standardized tool layer.

Chapter 24

Agent Skills

As agents evolve from monolithic prompt-and-tool systems into modular architectures, a key design question emerges: *how should an agent's capabilities be organized, discovered, and composed?* The answer increasingly converges on the concept of **skills**—discrete, reusable units of behavior that can be loaded, combined, and swapped without retraining. This chapter covers the taxonomy of agent skills, the architectural patterns for skill composition, and the engineering practices for building skill libraries that scale.

GAS Connection — Agent Skills

The GAS is structured as a collection of focused agent functions rather than a monolithic application. Each agent node in `capstone_gas/gas_system.py` encapsulates one skill that can be extracted, tested, and reused independently.

What is a skill? (§ What Is a Skill?) In the GAS, a skill is a self-contained agent node function that takes a `GASState` and returns a state update dict. `monitor_node()` is the anomaly detection skill; `advisor_node()` is the clinical RAG skill; `safety_node()` is the constitutional check skill; `caregiver_node()` is the alert composition skill. Each function is independently importable from `capstone_gas/gas_system.py`.

Skill architecture patterns (§ Skill Architecture Patterns). The GAS uses two skill architecture patterns: (1) *Prompt-based skills* (`monitor_node()`, `safety_node()`): the skill is implemented as a structured prompt with a fixed output schema; (2) *Tool-augmented skills* (`advisor_node()`): the skill calls `GASKnowledgeBase.retrieve()` to access external knowledge before generating a response. The choice between patterns depends on whether the skill needs external data.

Skill lifecycle (§ Skill Lifecycle). Each GAS skill goes through the lifecycle described in this section: (1) *Prototype*: implemented as a simple prompt in `capstone_gas/gas_system.py`, evaluated on the five clinical scenarios; (2) *SFT*: fine-tuned on a curated dataset (demonstrated in `part2_rl_methods/ch04_rl_for_lms/notebook.ipynb`); (3) *RL alignment*: aligned with DPO/-GRPO on preference data (demonstrated in `part2_rl_methods/ch06_dpo/` and `ch07_grpo/`); (4) *Production*: deployed via Ollama with monitoring.

Skills vs. fine-tuning (§ Skills vs. Fine-Tuning). The GAS uses both: `monitor_node()` uses a fine-tuned model for precise numerical reasoning about glucose thresholds (70 mg/dL, 180 mg/dL, 250 mg/dL); `safety_node()` uses a prompt-based skill because the constitutional rules are explicit and can be expressed directly in the prompt. The decision framework in this section—use fine-tuning when the task requires implicit knowledge; use prompting when the task can be fully specified—guides this choice.

Reusability. The GAS skills are designed for reuse: `advisor_node()` can be repurposed for any medical domain by swapping the `MEDICAL_KNOWLEDGE` list in `capstone_gas/gas_system.py`; `safety_node()` can be repurposed for any regulated domain by updating the constitutional rules. The `GASLLMClient` backend auto-detection (OpenAI → Anthropic → Ollama → Mock) makes the skills portable across deployment environments.

24.1 What Is a Skill?

A **skill** is a self-contained capability module that gives an agent expertise in a specific domain or task. Unlike a raw tool (which exposes a single function), a skill encompasses:

- **System prompt augmentation**: Domain-specific instructions, constraints, and persona elements injected into the agent's context.
- **Tool bindings**: One or more tools the skill requires (APIs, MCP servers, local commands).
- **Knowledge**: Reference material, examples, or few-shot demonstrations the agent needs to execute the skill correctly.
- **Workflow logic**: Multi-step procedures, decision trees, or conditional flows that guide the agent through complex tasks.
- **Guardrails**: Skill-specific safety constraints, output format requirements, and validation rules.

Skill vs. Tool vs. Agent

Concept	Scope	Example
Tool	Single function call	`web_search(query)`
Skill	Coherent capability (prompts + tools + knowledge)	"Research Analyst" skill
Agent	Autonomous entity with multiple skills	A coding assistant

A tool is a hammer. A skill is knowing *how to frame a house*. An agent is the carpenter who selects which skills to apply.

24.2 Skill Architecture Patterns

24.2.1 Static Skill Loading

The simplest pattern: skills are loaded at agent initialization based on configuration. The agent always has access to all its skills.

```
# Pseudocode -- framework-agnostic pattern
agent = Agent(
    model="claude-sonnet-4-20250514",
    skills=["code-review", "documentation", "testing"],
    # Each skill adds prompts, tools, and knowledge to the agent
)
```

Pros: Simple, predictable, low latency.

Cons: Context window waste when skills are unused; doesn't scale to hundreds of skills.

24.2.2 Dynamic Skill Discovery

The agent selects which skills to activate based on the current task. A skill router (often a lightweight classifier or embedding-based matcher) determines relevance:

```
# Pseudocode -- framework-agnostic pattern
relevant_skills = skill_router.match(
    user_request=message,
    available_skills=skill_registry,
    max_skills=3
)
agent.activate(relevant_skills)
```

Pros: Scales to large skill libraries; context-efficient.

Cons: Routing errors can miss relevant skills; adds latency.

24.2.3 Hierarchical Skill Composition

Skills can depend on other skills, forming a DAG. A high-level skill (e.g., "Deploy Application") may invoke sub-skills ("Run Tests", "Build Docker Image", "Update DNS"):

- Skills declare dependencies explicitly
- The orchestrator resolves the dependency graph before execution
- Sub-skills can be shared across multiple parent skills

24.3 Case Study: Anthropic's Agent Design

Anthropic's approach to agent architecture [11] provides one of the clearest articulations of skill-based agent design in production. Their philosophy emphasizes **simplicity over complexity** and **composable building blocks over monolithic frameworks**. (These patterns are also covered from an orchestration perspective in Chapter 21.)

24.3.1 Core Principles

1. **Start with the simplest solution.** Don't reach for agentic patterns until simpler approaches (single LLM call, retrieval + generation) have been tried and found insufficient.
2. **Workflows vs. agents.** Anthropic distinguishes between:
   - **Workflows**: Predefined orchestration of LLM calls — deterministic control flow with LLM steps at specific nodes. More predictable, easier to debug.
   - **Agents**: The LLM dynamically decides what to do next — tool selection, iteration count, and stopping criteria are all model-driven. More flexible, harder to control.
3. **Augmented LLM as the atomic unit.** The primitive is never a bare model—it is always a model bundled with its retrieval sources, callable tools, and persistent memory. This composite unit is, in practice, a skill-equipped model.

24.3.2 Building Block Patterns

Anthropic identifies five composable workflow patterns that function as skill templates:

24.3.3 The Augmented LLM

In Anthropic's framing, the fundamental unit is not the bare model but the **augmented LLM**:

$$\text{Augmented LLM} = \text{Model} + \text{Retrieval} + \text{Tools} + \text{Memory}$$

This maps directly to the skill concept: each skill configures which retrieval sources, tools, and memory stores the model has access to for a specific task. The skill boundary defines what the model *can see and do* within a particular invocation.

Table 24.1: Anthropic's composable agent patterns.

Pattern	Mechanism	When to Use
Prompt Chaining	Sequential LLM calls where each step's output feeds the next. Gates between steps validate intermediate results.	Multi-step transformations with clear decomposition
Routing	A classifier or LLM directs input to a specialized handler (skill) based on task type.	Distinct task categories requiring different expertise
Parallelization	Multiple LLM calls run simultaneously — either sectioning (split task) or voting (same task, aggregate).	Independent subtasks; or confidence via consensus
Orchestrator–Workers	A central LLM breaks the task into subtasks, delegates to worker LLMs, then synthesizes results.	Complex tasks where subtasks aren't predictable in advance
Evaluator–Optimizer	One LLM generates, another evaluates; iterate until quality threshold is met.	Tasks with clear quality criteria (code, writing)

24.3.4 Practical Implications

Anthropic's Key Insight

The most effective agents aren't the most complex ones. They are **simple loops with good tools**:

```
while not done:
    action = llm.decide(context, tools)
    result = execute(action)
    context.append(result)
    done = llm.should_stop(context)
```

The intelligence comes from (1) the model's capability, (2) the quality of tool descriptions, and (3) the clarity of the task framing — not from elaborate orchestration logic. Skills provide the structure for (2) and (3).

Design recommendations from Anthropic's approach:

- **Keep agent loops simple**: Avoid over-engineering the control flow. Let the model decide.
- **Invest in tool quality**: Detailed, unambiguous tool descriptions are more valuable than complex routing logic.
- **Use structured outputs**: Force the model to output decisions in parseable formats (JSON, function calls) — reduces skill execution errors.
- **Build in recovery**: Skills should handle errors gracefully — retry with different parameters, ask for clarification, or escalate to a human.
- **Limit scope per skill**: A skill that tries to do everything will do nothing well. Narrow, well-defined skills compose better than broad ones.

24.4 Skill Lifecycle

1. **Discovery**: The system identifies which skills are available (registry, marketplace, local definitions).

2. **Selection**: Based on the user request, relevant skills are matched and loaded.
3. **Activation**: Skill prompts, tools, and knowledge are injected into the agent's context.
4. **Execution**: The agent uses the skill's capabilities to accomplish the task.
5. **Deactivation**: Skill context is removed to free context window space for subsequent tasks.
6. **Learning**: Execution results may update the skill's few-shot examples or fine-tune routing.

24.5 Skill Registries and Marketplaces

Production skill systems require infrastructure:

- **Skill manifest**: A structured description (name, capabilities, required tools, input/output schema) enabling automatic discovery and routing.
- **Version control**: Skills evolve; agents need to pin specific versions for reproducibility.
- **Dependency resolution**: Skills may require specific MCP servers, API keys, or other skills.
- **Permission model**: Not all agents should have access to all skills (security, cost, capability boundaries).
- **Marketplace**: Organizations can publish, share, and install skills — analogous to package managers for code.

Skill Manifest Example

A skill manifest declares everything an orchestrator needs to load and invoke a skill. No industry-standard schema exists yet; below is an illustrative format that captures common fields across real implementations (Anthropic MCP, OpenAI function specs, LangChain tool definitions):

```
// Illustrative schema -- not a specific SDK format
{
  "name": "code-review",
  "description": "Review code changes for bugs, style, and security issues",
  "version": "2.1.0",
  "requires": {
    "tools": ["file_read", "grep", "git_diff"],
    "mcp_servers": ["github"],
    "models": ["claude-sonnet-4-20250514"]
  },
  "input_schema": {
    "type": "object",
    "properties": {
      "repo": {"type": "string"},
      "pr_number": {"type": "integer"}
    }
  },
  "prompts": ["skills/code-review/system.md"],
  "knowledge": ["skills/code-review/style-guide.md"]
}
```

24.6 Self-Evolving Agents: Lifelong Skill Acquisition

The skill library patterns described in this chapter treat skills as artifacts created by humans and consumed by agents. A more ambitious vision—and an increasingly practical one—is the *self-evolving agent*: a system that discovers, creates, and refines its own skills through experience, without requiring human authorship for each new capability.

This section examines the spectrum of self-improvement mechanisms, the engineering patterns that make them safe and controllable, and the risks that arise when agents modify their own behavior.

24.6.1 The Self-Improvement Spectrum

Self-improvement in agents exists on a spectrum from shallow to deep:

Table 24.2: The self-improvement spectrum: from shallow retrieval to deep self-modification.

Mechanism	What Changes	Persistence	Risk Level
Skill retrieval	Which skill is used	Session only	Low
Skill composition	How skills are combined	Session only	Low
Skill authoring	New skills written to library	Persistent	Medium
Prompt self-editing	Agent modifies its own system prompt	Persistent	High
Fine-tuning	Model weights updated from experience	Permanent	Very High
Architecture search	Agent modifies its own graph/topology	Permanent	Very High

The lower rows of this table represent capabilities that current systems rarely implement in production, but that are increasingly explored in research. Understanding the full spectrum is important because systems designed for the upper rows can inadvertently enable the lower rows if not carefully constrained.

Voyager and curriculum-driven skill accumulation. The Voyager agent [383] demonstrated that an LLM-powered agent in Minecraft could autonomously write, test, and store JavaScript skill programs, building a growing library of reusable capabilities through self-directed curriculum learning. Crucially, Voyager's self-improvement is bounded: skills are code artifacts stored in a retrievable library, not modifications to the model itself. This is the safe end of the spectrum—persistent but inspectable and reversible. (The Voyager skill library architecture is covered in detail in Chapter 17; this section focuses on the broader self-evolution design space.)

24.6.2 Tool Synthesis and Automatic Skill Authoring

Beyond retrieving existing tools, agents can be designed to *synthesize* new tools when existing ones are insufficient. Toolformer [333] demonstrated that a language model can learn to insert API calls into its own generated text, effectively teaching itself when and how to use tools from a small number of examples. The key insight is that tool use can be learned from the model's own outputs, without requiring explicit human labeling of tool-use examples.

Automatic skill authoring pattern. A self-evolving agent that encounters a task it cannot complete with existing skills can:

1. **Identify the gap.** Detect that no existing skill covers the required capability (e.g., by attempting retrieval and finding low-confidence matches).
2. **Draft a new skill.** Use the LLM to write a new skill definition—prompt, schema, and implementation—based on the task description.
3. **Test the skill.** Execute the new skill in a sandboxed environment and verify that it produces the expected output on a small set of test cases.
4. **Store or discard.** If the skill passes testing, add it to the persistent skill library. If it fails, discard it and fall back to a human-escalation path.

```
def attempt_with_skill_synthesis(agent, task: str) -> str:
    # Try existing skills first
    skill = agent.skill_library.retrieve(task, top_k=1)
    if skill and skill.confidence > 0.8:
        return agent.execute_skill(skill, task)
```

```
    # No suitable skill found -- attempt synthesis
    candidate_skill = agent.llm.draft_skill(
        task_description=task,
        existing_skills=agent.skill_library.list_names(),
    )
    test_result = agent.sandbox.run_tests(candidate_skill)
    if test_result.passed:
        agent.skill_library.store(candidate_skill)
        return agent.execute_skill(candidate_skill, task)
    else:
        return agent.escalate_to_human(task, reason="skill synthesis failed")
```

Listing 24.1: Sketch of an automatic skill authoring loop.

STaR and self-taught reasoning. The Self-Taught Reasoner (STaR) [441] extends this idea to reasoning itself: a model generates chain-of-thought rationales, filters those that lead to correct answers, and fine-tunes on the filtered set. This is a form of self-improvement at the model level—the weights change—and therefore sits at the high-risk end of the spectrum. STaR-style self-improvement is powerful but requires careful evaluation to ensure that the model is learning generalizable reasoning, not overfitting to the training distribution.

24.6.3 Persistent Self-Modification Risks

Self-evolving agents introduce risks that static agents do not:

Skill drift. A skill library that grows without curation accumulates redundant, conflicting, and outdated skills. Retrieval quality degrades as the library grows, and the agent may select an obsolete skill over a better one.

Capability creep. An agent that can write and store new skills can, in principle, write skills that exceed its intended scope. A customer service agent that synthesizes a "send bulk email" skill has acquired a capability its designers did not intend.

Feedback loops. If the agent's self-evaluation of new skills is flawed, it may store skills that appear to pass tests but fail in production. These skills then influence future skill synthesis, potentially amplifying the original error.

Goal misalignment under self-modification. An agent that modifies its own system prompt or fine-tunes its own weights may drift from its original objective. This is the alignment concern at the heart of the AI safety literature [396]: a sufficiently capable self-modifying agent may optimize for a proxy of its original goal rather than the goal itself.

Self-Modification and the Alignment Problem

Persistent self-modification—especially weight updates—is the mechanism by which an agent could, in principle, alter its own values and objectives. Current production systems should treat weight-level self-modification as out of scope. Skill library authoring is safer but still requires governance: human review of new skills before they are promoted to the persistent library, and automated testing that covers both capability and safety properties.

24.6.4 Safe Self-Evolution Patterns

The risks of self-evolution are manageable with the right engineering controls. The following patterns enable the benefits of self-improvement while maintaining safety and controllability:

Sandboxed skill testing. New skills are always tested in an isolated environment before being stored. The sandbox has no access to production data, external APIs, or persistent state. A skill that passes sandbox tests is promoted to a staging library; a human reviewer approves promotion to the production library.

Skill versioning and rollback. The skill library is version-controlled (e.g., in a Git repository). Every skill addition, modification, or deletion is a commit with a timestamp and author (human or agent). Rollback to any previous state is possible in $O(1)$ time.

Capability bounds. The set of tools available to the skill synthesis process is strictly limited. An agent that can only write skills using tools already in its approved tool set cannot synthesize capabilities that exceed its original scope.

Human review gates. For any skill that will be used in a high-stakes context (healthcare, finance, legal), require human review before promotion to the production library. The review should cover both the skill's intended behavior and its potential for misuse.

Drift monitoring. Periodically evaluate the agent's behavior on a fixed benchmark of tasks. If performance on any task category degrades after skill library updates, trigger a review of recently added skills.

Self-Evolution Is a Spectrum, Not a Binary

The question is not "should my agent self-evolve?" but "at what level of the self-improvement spectrum should self-evolution be permitted, and what governance controls are in place at each level?" Skill retrieval and composition are safe and should be enabled by default. Skill authoring is valuable but requires sandboxing and review gates. Prompt self-editing and weight updates require exceptional justification and should be treated as research capabilities, not production features.

24.7 Skills vs. Fine-Tuning

A natural question: why use runtime skill injection instead of fine-tuning the model?

Table 24.3: Skills (in-context) vs. fine-tuning for adding capabilities.

Dimension	Skills (In-Context)	Fine-Tuning
Deployment speed	Instant	Hours–days
Flexibility	Swap/combine at runtime	Fixed at training time
Context cost	Uses context window	Zero runtime cost
Deep behavior change	Limited by context length	Deep parametric change
Multi-tenant	Different skills per user	Same model for all
Maintenance	Update text files	Retrain on new data

In practice, the two approaches are complementary: fine-tuning provides *base capabilities* (instruction following, tool use format, reasoning), while skills provide *task-specific expertise* layered on top at runtime.

Chapter 25

Agent-to-Agent Communication (A2A)

As large language models evolve from isolated assistants into collaborative networks of specialized agents, the question of *how agents talk to each other* becomes as important as how they reason internally. This section covers the protocols, patterns, and engineering practices that enable multi-agent systems to coordinate, delegate, and collectively solve problems that no single agent could handle alone.

GAS Connection — Agent-to-Agent Communication (A2A)

The GAS implements A2A communication via the LangGraph shared state (current prototype). The following chapter's treatment of communication patterns and the Google A2A Protocol describes how the GAS would evolve to a distributed multi-patient deployment.

Communication patterns (§ Communication Patterns). The GAS uses the *blackboard* communication pattern within a single pipeline invocation: all agent nodes read from and write to the shared `GASState` TypedDict, which acts as a blackboard. `advisor_node()` writes `state["recommendation"]`; `safety_node()` reads it and writes `state["safety_approved"]`; `supervisor_node()` reads the verdict and sets `state["next_agent"]`. This is the simplest A2A pattern and is appropriate for the GAS's sequential pipeline structure.

The Google A2A Protocol (§ The Google A2A Protocol). In a distributed GAS deployment (multiple patients, multiple servers), each agent function would be deployed as an independent A2A-compatible service. The Supervisor would send A2A `Task` messages to the Monitor, Advisor, Safety Critic, and Caregiver Notifier services; each service would process the task and return an A2A `TaskResult`. The LangGraph `StateGraph` in `build_gas_graph()` would be replaced by an A2A orchestrator.

Message formats (§ Message Formats and Schemas). All GAS inter-agent messages use the typed `GASState` TypedDict defined in `capstone_gas/gas_system.py`. The fields passed between agents are well-defined: `current_reading` (glucose value, trend, timestamp), `alert_level` (none/low/medium/high/critical), `recommendation` (text), `safety_score` (0.0–1.0), `safety_approved` (bool). This typed interface is the GAS's equivalent of an A2A message schema.

A2A vs. MCP (§ A2A vs. MCP). The GAS currently uses neither A2A nor MCP—all communication is via the shared `GASState`. In a production deployment, MCP would be used for tool calls (`advisor_node()` → knowledge base, `caregiver_node()` → notification service) and A2A for agent-to-agent delegation (Supervisor → Monitor, Supervisor → Advisor). This is exactly the complementary usage described in this section.

25.1 Motivation: Why Agents Must Communicate

The Specialization Imperative

A single generalist agent faces a fundamental tension: breadth of knowledge versus depth of capability. Real-world tasks—legal document review, multi-step scientific research, enterprise software development—demand both. Agent-to-agent communication resolves this tension by allowing a *network* of specialists to collaborate, each contributing its strengths while delegating weaknesses.

Several forces drive the need for structured inter-agent communication:

Cognitive Load and Context Limits. Every LLM operates within a finite context window. Complex workflows—spanning hundreds of documents, tool calls, and reasoning steps—quickly exceed what a single agent can hold in memory. By decomposing tasks across agents, each agent operates within a manageable context, and the orchestrating agent maintains only high-level state.

Specialization and Expertise. Different agents may be fine-tuned, prompted, or tool-equipped for specific domains: a `CodeAgent` with access to compilers and test runners, a `LegalAgent` with access to case-law databases, a `DataAgent` with statistical libraries. Routing subtasks to the right specialist improves both quality and efficiency.

Parallelism and Throughput. Independent subtasks can be dispatched to multiple agents simultaneously. A research orchestrator might fan out literature searches across five specialized agents in parallel, then synthesize their results—dramatically reducing wall-clock time.

Fault Isolation and Resilience. When one agent fails, a well-designed multi-agent system can retry with a different agent, fall back to a simpler approach, or escalate to a human—without collapsing the entire workflow.

Delegation and Handoff. Long-running tasks may need to be handed off between agents as context shifts. An initial `PlannerAgent` decomposes a goal, hands subtasks to `ExecutorAgents`, and a final `ReviewerAgent` validates outputs—each agent receiving exactly the context it needs.

Core Requirements for A2A Communication

1. **Discoverability**: Agents must be able to find other agents and understand their capabilities.
2. **Interoperability**: Agents built by different teams or vendors must speak a common protocol.
3. **Asynchrony**: Long-running tasks must not block the caller; results arrive via callbacks or polling.
4. **Security**: Agents must authenticate each other and enforce authorization boundaries.
5. **Observability**: Every message exchange must be traceable for debugging and auditing.

25.2 The Google A2A Protocol

In April 2025, Google (with contributions from over 50 technology partners) released the **Agent-to-Agent (A2A) Protocol** [121], an open specification for interoperable communication between AI agents. The protocol was subsequently donated to the **Linux Foundation** and has grown to over 150 supporting organizations as of 2026. A2A is designed around a set of core principles that distinguish it from earlier ad-hoc approaches.

25.2.1 Design Philosophy

The A2A specification articulates five guiding principles (adapted from the official spec [121], §1.2):

A2A Design Principles

Opaque execution
Calling agents never inspect the internals of a remote agent—they interact solely through the declared interface. Whether the target is GPT-4, Gemini, or a rule-based system is irrelevant to the protocol, enabling genuinely heterogeneous agent ecosystems.
Enterprise readiness
Authentication (OAuth 2.0, API keys, JWT), audit logging, and regulatory compliance are not afterthoughts—they are integrated at the protocol level from the outset.
Modality agnosticism
A single message may combine text, binary files, and structured JSON payloads, accommodating agents that operate on images, audio, code, or documents without protocol extensions.
Simplicity via existing standards
Rather than inventing new transports, A2A reuses HTTP/HTTPS with JSON-RPC 2.0 messages, Server-Sent Events (SSE) for streaming, and gRPC as an alternative binding—technologies that every infrastructure team already operates.
Async-first task model
Long-running operations are the norm, not the exception. Push notifications and polling are both first-class mechanisms, so callers never need to hold open a connection for hours.

25.2.2 Agent Cards

The foundation of A2A discoverability is the **Agent Card**—a machine-readable JSON manifest hosted at a well-known endpoint (`/.well-known/agent.json`). It advertises what the agent can do, how to authenticate, and where to send tasks—analogous to an OpenAPI spec but for autonomous agents rather than REST endpoints.

Agent Card Structure

```
# Agent Card served at https://agent.example.com/.well-known/agent.json
agent_card = {
    "name": "DataAnalysisAgent",
    "description": "Analyzes structured datasets, produces statistical
    summaries, "
                   "generates visualizations, and answers data questions.",
    "url": "https://agent.example.com/a2a",
    "version": "1.2.0",
    "capabilities": {
        "streaming": True,
        "pushNotifications": True,
        "stateTransitionHistory": True
    },
    "authentication": {
        "schemes": ["Bearer", "ApiKey"]
    },
    "skills": [
        {
            "id": "statistical-analysis",
            "name": "Statistical Analysis",
            "description": "Compute descriptive statistics, run hypothesis
    tests, "
                           "fit regression models on tabular data.",
            "tags": ["statistics", "data", "analysis", "regression"],
            "examples": [
                "What is the correlation between columns A and B?",
                "Run a t-test comparing these two groups.",
                "Fit a linear regression predicting sales from ad spend."
            ],
```

```
            "inputModes": ["text", "data"],
            "outputModes": ["text", "data", "file"]
        },
        {
            "id": "visualization",
            "name": "Data Visualization",
            "description": "Generate charts, plots, and dashboards from data.",
            "tags": ["charts", "plots", "visualization", "dashboard"],
            "examples": [
                "Create a bar chart of monthly revenue.",
                "Plot the distribution of customer ages."
            ],
            "inputModes": ["text", "data"],
            "outputModes": ["file", "text"]
        }
    ],
    "defaultInputModes": ["text"],
    "defaultOutputModes": ["text"]
}
```

Agent Cards enable *capability-based routing*: an orchestrator agent can fetch cards from a registry, semantically match a subtask to the most appropriate agent, and dispatch accordingly—all without hardcoded routing logic.

25.2.3 Task Lifecycle

A2A models all work as **Tasks**. A task progresses through a well-defined state machine:

`submitted`
The client has sent the task; the server has acknowledged receipt.

`working`
The agent is actively processing. The client may poll or await SSE events.

`input-required`
The agent needs additional information from the user or calling agent before it can proceed (e.g., a clarifying question, a missing credential).

`completed`
The task finished successfully; results are available in the response.

`failed`
An unrecoverable error occurred; an error message explains the cause.

`rejected`
The agent declined the task (e.g., outside its capabilities or unauthorized). Added in A2A v1.0.

`canceled`
The task was aborted, either by the client or by the server.

25.2.4 Streaming via Server-Sent Events

For tasks that produce incremental output (e.g., a long report being written, a code file being generated), A2A uses **Server-Sent Events (SSE)**. The client opens a persistent HTTP connection and receives a stream of JSON events:

SSE Event Stream Example

```
# Each SSE event carries a TaskStatusUpdateEvent or TaskArtifactUpdateEvent
# Example stream for a "write a research report" task:

# Event 1: status update
data: {
  "id": "task-abc123",
  "status": {"state": "working"},
  "final": false
```

```
}

# Event 2: partial artifact (streaming text)
data: {
  "id": "task-abc123",
  "artifact": {
    "parts": [{"type": "text", "text": "## Introduction\n\nRecent advances in
    ..."}],
    "index": 0,
    "append": false,
    "lastChunk": false
  },
  "final": false
}

# Event 3: more text appended
data: {
  "id": "task-abc123",
  "artifact": {
    "parts": [{"type": "text", "text": " reinforcement learning have shown..."
    }],
    "index": 0,
    "append": true,   # append to existing artifact
    "lastChunk": false
  },
  "final": false
}

# Final event: task complete
data: {
  "id": "task-abc123",
  "status": {"state": "completed"},
  "final": true
}
```

25.2.5 Push Notifications for Long-Running Tasks

When a task may take minutes or hours, maintaining an open SSE connection is impractical. A2A supports **push notifications**: the client registers a webhook URL, and the server POSTs status updates as the task progresses.

```
# Client registers a push notification endpoint when submitting the task
task_request = {
    "id": "task-xyz789",
    "message": {
        "role": "user",
        "parts": [{"type": "text", "text": "Analyze Q3 sales data and produce a
    report."}]
    },
    "pushNotification": {
        "url": "https://my-orchestrator.example.com/webhooks/a2a",
        "token": "secret-hmac-token-for-verification",
        "authentication": {
            "schemes": ["Bearer"],
            "credentials": "eyJhbGciOiJIUzI1NiJ9..."
        }
    }
}
# The server will POST TaskStatusUpdateEvent objects to the webhook URL
# as the task transitions through states.
```

25.2.6 Message Format

A2A messages consist of a **role** (`user` or `agent`) plus a list of typed **parts** (text, file, or structured data). The full message schema, multi-modal examples, and context-passing guidelines are covered in Section 25.5.

25.2.7 Authentication and Authorization

A2A supports multiple authentication schemes, declared in the Agent Card and enforced per-request:

- **Bearer tokens (JWT/OAuth 2.0)**: Standard for enterprise deployments; tokens carry scopes that limit what the calling agent is permitted to request.
- **API keys**: Simpler scheme for internal or trusted environments.
- **Mutual TLS (mTLS)**: Certificate-based authentication for high-security deployments.
- **OpenID Connect**: Federated identity, enabling cross-organization agent communication.

Authorization Scope Enforcement

An agent receiving a task must verify not only *who* is calling (authentication) but *what they are allowed to request* (authorization). A `ReportingAgent` might accept read-only data queries from any authenticated agent, but restrict write operations to agents holding a specific OAuth scope. Failing to enforce this creates privilege escalation vulnerabilities in multi-agent systems.

25.3 Communication Patterns

Multi-agent systems employ a variety of communication patterns depending on the nature of the task, latency requirements, and the number of agents involved.

25.3.1 Request-Response

The simplest pattern: Agent A sends a task to Agent B and waits for a complete response. Suitable for short, well-defined subtasks where the result is needed before proceeding.

25.3.2 Streaming

Agent A opens an SSE connection; Agent B streams partial results as they are produced. Ideal for long-form generation (reports, code), real-time collaboration, or progressive UI updates.

Streaming Pattern Use Case

An orchestrator asks a `WritingAgent` to draft a 10-page technical document. Rather than waiting 2 minutes for the complete document, the orchestrator streams each section as it is written, allowing a `ReviewAgent` to begin reviewing early sections while later sections are still being generated—a pipeline that reduces total latency by 40–60%.

25.3.3 Multi-Turn Interaction

Some tasks require iterative refinement. The agent enters `input-required` state, the orchestrator provides clarification, and the task resumes. This mirrors human collaborative workflows: draft → feedback → revision.

```
# Multi-turn: orchestrator handles input-required state
async def run_multiturn_task(client, initial_message):
    task = await client.send_task(message=initial_message)

    while task.status.state not in ("completed", "failed", "canceled"):
        if task.status.state == "input-required":
```

```
            # Agent needs clarification
            clarification_needed = task.status.message
            print(f"Agent asks: {clarification_needed}")

            # Orchestrator generates or forwards a clarifying response
            user_reply = await get_clarification(clarification_needed)

            # Send the reply to continue the task
            task = await client.send_task(
                task_id=task.id,
                message={"role": "user",
                         "parts": [{"type": "text", "text": user_reply}]}
            )
        else:
            # Still working --- poll after a delay
            await asyncio.sleep(2)
            task = await client.get_task(task.id)

    return task
```

25.3.4 Broadcast

An orchestrator sends the same message to multiple agents simultaneously—useful for announcements, distributing shared context, or triggering parallel independent workflows.

25.3.5 Publish-Subscribe (Pub-Sub)

Agents subscribe to event channels (e.g., `new-document-uploaded`, `model-retrained`). When an event fires, all subscribed agents are notified. This decouples producers from consumers and enables reactive, event-driven architectures.

25.3.6 Negotiation

Two agents exchange proposals and counter-proposals to reach agreement on a plan, resource allocation, or approach. Common in multi-agent planning systems where agents have different objectives or constraints.

Negotiation Pattern

A `PlannerAgent` proposes a 5-step research plan. A `ResourceAgent` responds that Step 3 (running a large simulation) would exceed the compute budget. The `PlannerAgent` counter-proposes a scaled-down simulation. The `ResourceAgent` approves. The agreed plan is then dispatched to executor agents.

25.3.7 Auction-Based Task Allocation

The orchestrator announces a task with requirements; candidate agents submit bids (estimated completion time, confidence, cost); the orchestrator awards the task to the winning bidder. This enables dynamic, market-based load balancing across a pool of agents.

25.4 Agent Discovery and Routing

Before an agent can communicate with another, it must *find* it. Agent discovery is the process of locating agents that can handle a given task.

25.4.1 Agent Registries

An **agent registry** is a directory service that indexes Agent Cards and provides search and lookup APIs. Two deployment models exist:

Table 25.1: Summary of A2A communication patterns.

Pattern	Latency	Best For
Request-Response	Low	Short, well-defined subtasks
Streaming	Low (first token)	Long-form generation, real-time UI
Multi-Turn	Medium	Ambiguous tasks requiring clarification
Broadcast	Low	Shared context distribution
Pub-Sub	Variable	Event-driven reactive workflows
Negotiation	Medium–High	Resource-constrained planning
Auction	Medium	Dynamic load balancing

Centralized Registry

A single authoritative registry (e.g., an enterprise service catalog) indexes all agents. Simple to operate but creates a single point of failure and may not scale to cross-organization deployments.

Federated Registry

Multiple registries, each authoritative for a domain or organization, with cross-registry search protocols. More resilient and privacy-preserving, but requires standardized federation protocols.

25.4.2 Capability-Based Routing

Rather than hardcoding agent URLs, orchestrators perform **capability-based routing**: they query the registry for agents matching required skills, then select the best match.

```
class AgentRouter:
    """Routes tasks to agents based on capability matching."""

    def __init__(self, registry_url: str):
        self.registry_url = registry_url
        self._cache: dict[str, list[AgentCard]] = {}

    async def find_agents(self, required_skill: str,
                          tags: list[str] | None = None) -> list[AgentCard]:
        """Query registry for agents with the required skill."""
        params = {"skill": required_skill}
        if tags:
            params["tags"] = ",".join(tags)
        async with httpx.AsyncClient() as client:
            resp = await client.get(f"{self.registry_url}/agents", params=params)
            return [AgentCard(**card) for card in resp.json()["agents"]]

    async def route(self, task_description: str) -> AgentCard:
        """Semantically match a task description to the best available agent."""
        # Embed the task description
        task_embedding = await embed(task_description)

        # Fetch all registered agents
        all_agents = await self.find_agents(required_skill="*")

        # Score each agent by cosine similarity of task to agent description
        scored = []
        for agent in all_agents:
            agent_embedding = await embed(agent.description)
            score = cosine_similarity(task_embedding, agent_embedding)
            scored.append((score, agent))

        # Return the highest-scoring agent
        scored.sort(key=lambda x: x[0], reverse=True)
        return scored[0][1]
```

25.4.3 Load Balancing Across Equivalent Agents

When multiple agents offer the same capability, the router must distribute load. Common strategies:

- **Round-robin**: Distribute tasks evenly across all available agents.
- **Least-loaded**: Route to the agent with the fewest active tasks (requires health/metrics endpoints).
- **Latency-aware**: Route to the agent with the lowest recent response time.
- **Affinity-based**: Route related tasks to the same agent to exploit cached context.

25.4.4 Version Management and Compatibility

Agent Cards include a `version` field. Orchestrators should specify minimum version requirements and handle graceful degradation when only older versions are available. Semantic versioning [310] (`MAJOR.MINOR.PATCH`) is recommended: breaking interface changes increment `MAJOR`, new capabilities increment `MINOR`.

Version Skew in Long-Running Systems

In production multi-agent systems, different agents may be updated at different times, creating version skew. An orchestrator compiled against Agent Card v2.1 may encounter agents still running v1.3. Always implement backward-compatible message handling and test cross-version scenarios explicitly.

25.5 Message Formats and Schemas

25.5.1 Structured vs. Unstructured Messages

A2A supports a spectrum from fully unstructured (plain text) to fully structured (typed JSON schemas). The right choice depends on the agents involved:

Table 25.2: Structured vs. unstructured A2A message trade-offs.

Message Type	Advantages	Disadvantages
Plain text	Flexible, human-readable, easy to generate	Hard to parse reliably, no schema validation
Structured JSON	Machine-parseable, validatable, typed	Requires schema agreement, less flexible
Hybrid (text + data)	Human-readable intent + machine-parseable payload	More complex to construct and parse

25.5.2 Multi-Modal Messages

A2A messages are structured as a **role** (`user` or `agent`) plus a list of typed **parts**:

Table 25.3: A2A message part types (wire format uses `"type": "text"|"file"|"data"`).

Part Type	Fields	Use Case
`TextPart`	`text: string`	Natural language instructions, responses
`FilePart`	`mimeType`, `uri` or `bytes`	Documents, images, audio, code files
`DataPart`	`data: object`	Structured JSON (tool results, schemas)

Modern agents increasingly work with non-text modalities. A2A's `FilePart` supports any MIME type, enabling rich multi-modal workflows:

Multi-Modal A2A Message: Data Analysis

```
# A message combining text instructions with a data payload and a file
message = {
    "role": "user",
    "parts": [
        {
            "type": "text",
            "text": "Analyze the attached CSV and the schema below. "
                    "Identify anomalies and produce a summary report."
        },
        {
            "type": "file",
            "mimeType": "text/csv",
            "uri": "https://storage.example.com/data/sales_q3.csv"
        },
        {
            "type": "data",
            "data": {
                "schema": {
                    "columns": ["date", "region", "product", "revenue", "units"
    ],
                    "types":   ["date", "string", "string", "float", "int"]
                },
                "expectedRowCount": 15000,
                "anomalyThreshold": 3.0  # z-score threshold
            }
        }
    ]
}
```

Multi-Modal A2A Message: Image Analysis

```
# Multi-modal message: text + image + structured data
multimodal_message = {
    "role": "user",
    "parts": [
        {"type": "text",
         "text": "Describe what is wrong with this chart and suggest fixes."},
        {"type": "file",
         "mimeType": "image/png",
         "bytes": base64.b64encode(chart_image_bytes).decode()},
        {"type": "data",
         "data": {
             "chartType": "bar",
             "dataSource": "Q3 Revenue by Region",
             "knownIssues": ["y-axis does not start at zero",
                             "missing error bars"]
         }}
    ]
}
```

25.5.3 Context Passing: What to Share vs. What to Keep Private

A critical design decision in multi-agent systems is *context scoping*: how much of the conversation history and internal state to pass to a sub-agent.

Context Scoping Principles

Minimal Context
Pass only what the sub-agent needs to complete its task. Reduces token usage, latency, and the risk of leaking sensitive information.

Summarized Context
Instead of passing raw conversation history, pass a structured summary: goals, constraints, decisions made, and relevant facts.
Private State
Internal reasoning, intermediate drafts, and user PII should generally *not* be forwarded to sub-agents unless explicitly required.
Correlation IDs
Always pass a `correlationId` so that sub-agent actions can be traced back to the originating workflow in logs and audit trails.

25.5.4 Conversation Threading and Correlation IDs

In complex workflows, many tasks may be in flight simultaneously. **Correlation IDs** link related tasks across agents:

```
import uuid

class WorkflowContext:
    """Carries correlation metadata through a multi-agent workflow."""

    def __init__(self, workflow_id: str | None = None):
        self.workflow_id = workflow_id or str(uuid.uuid4())
        self.span_id = str(uuid.uuid4())
        self.parent_span_id: str | None = None

    def child_context(self) -> "WorkflowContext":
        """Create a child context for a sub-task."""
        child = WorkflowContext(workflow_id=self.workflow_id)
        child.parent_span_id = self.span_id
        return child

    def to_metadata(self) -> dict:
        return {
            "x-workflow-id": self.workflow_id,
            "x-span-id": self.span_id,
            "x-parent-span-id": self.parent_span_id
        }

# Usage: attach to every A2A task submission
ctx = WorkflowContext()
task = await client.send_task(
    message=message,
    metadata=ctx.to_metadata()
)
# Sub-tasks use child contexts for tracing
sub_ctx = ctx.child_context()
```

25.6 Coordination Protocols

Beyond point-to-point communication, multi-agent systems benefit from higher-level **coordination protocols**—structured interaction patterns that enable collective decision-making and problem-solving.

25.6.1 Contract Net Protocol

The **Contract Net Protocol (CNP)** [352] is a classic multi-agent coordination mechanism adapted for LLM-based systems:

1. **Announcement**: The manager agent broadcasts a task announcement to all potential contractor agents, including task requirements and evaluation criteria.

2. **Bidding**: Contractor agents evaluate the task against their capabilities and submit bids containing estimated completion time, confidence, and resource requirements.

3. **Award**: The manager selects the winning bid (or multiple bids for parallel subtasks) and awards the contract.

4. **Execution and Reporting**: The contractor executes the task and reports results back to the manager.

Contract Net Protocol Implementation

```
import dataclasses

class ContractNetManager:
    """Implements the Contract Net Protocol for task allocation."""

    async def allocate_task(self, task: Task,
                            candidate_agents: list[AgentCard]) -> AgentCard:
        # Phase 1: Announce task to all candidates
        announcement = {
            "type": "task-announcement",
            "task": dataclasses.asdict(task),
            "deadline": (datetime.now(timezone.utc) + timedelta(seconds=10)).
    isoformat(),
            "evaluationCriteria": ["confidence", "estimatedTime", "cost"]
        }

        # Phase 2: Collect bids
        bids = await asyncio.gather(*[
            self._request_bid(agent, announcement)
            for agent in candidate_agents
        ], return_exceptions=True)

        valid_bids = [(agent, bid) for agent, bid in zip(candidate_agents, bids
    )
                      if not isinstance(bid, Exception) and bid is not None]

        if not valid_bids:
            raise RuntimeError(f"No agents bid on task {task.id}")

        # Phase 3: Award to best bidder (highest confidence, lowest time)
        def score_bid(agent_bid):
            _, bid = agent_bid
            return bid["confidence"] - 0.1 * bid["estimatedSeconds"]

        winner_agent, winning_bid = max(valid_bids, key=score_bid)

        # Notify winner and losers
        await self._award_contract(winner_agent, task)
        await asyncio.gather(*[
            self._reject_bid(agent, task.id)
            for agent, _ in valid_bids if agent != winner_agent
        ])

        return winner_agent

    async def _request_bid(self, agent: AgentCard,
                           announcement: dict) -> dict | None:
        """Ask an agent to bid on a task."""
        try:
            result = await self.client.send_task(
                agent_url=agent.url,
                message={"role": "user",
                         "parts": [{"type": "data", "data": announcement}]}
            )
```

```
            return result.artifacts[0].parts[0]["data"]
        except Exception:
            return None
```

25.6.2 Blackboard Systems

A **blackboard system** [144] provides a shared workspace (the "blackboard") where agents post partial solutions, observations, and hypotheses. Other agents monitor the blackboard and contribute when they can add value—an *opportunistic* problem-solving approach.

Blackboard systems are well-suited to problems where the solution path is not known in advance and different agents may contribute at different stages—such as scientific hypothesis generation, complex debugging, or multi-source intelligence analysis.

25.6.3 Consensus Protocols

When multiple agents must agree on a decision (e.g., which plan to execute, whether a result is correct), **consensus protocols** provide structured voting mechanisms:

Simple Majority Voting

Each agent votes; the option with $> 50\%$ of votes wins. Fast but vulnerable to correlated errors if agents share the same base model.

Weighted Voting

Votes are weighted by agent confidence or historical accuracy. More robust but requires calibrated confidence estimates.

Quorum-Based

A decision requires agreement from at least k of n agents. Provides fault tolerance: up to $n - k$ agents can fail or disagree without blocking.

Delphi Method

Agents vote, see anonymized results, revise their votes, and repeat until convergence. Reduces anchoring bias and encourages genuine deliberation.

```
async def quorum_vote(agents: list[AgentCard], question: str,
                      options: list[str], quorum: int) -> str | None:
    """Run a quorum vote across agents. Returns winning option or None."""
    votes = await asyncio.gather(*[
        ask_agent_to_vote(agent, question, options)
        for agent in agents
    ])

    counts: dict[str, int] = {}
    for vote in votes:
        if vote in options:
            counts[vote] = counts.get(vote, 0) + 1

    # Return first option that reaches quorum
    for option, count in sorted(counts.items(), key=lambda x: -x[1]):
        if count >= quorum:
            return option
    return None  # No quorum reached
```

25.6.4 Leader Election

In dynamic multi-agent systems, a **leader** (orchestrator) may need to be elected at runtime—for example, when the original orchestrator fails or when agents self-organize without a pre-assigned coordinator. Classic distributed systems algorithms (Bully, Ring) can be adapted for agent networks, with agents exchanging capability scores or priority tokens to elect the most capable available agent as leader.

25.7 A2A vs. MCP: Complementary Protocols

A common source of confusion is the relationship between A2A and the **Model Context Protocol (MCP)** [10]. These protocols are *complementary*, not competing:

The Core Distinction

- **MCP** is the *vertical* protocol: it extends an agent downward into the world of databases, APIs, file systems, and code executors. Only the agent reasons; MCP endpoints are deterministic services.
- **A2A** is the *horizontal* protocol: it links one reasoning agent to another. Both sides are intelligent actors capable of reasoning, planning, and tool use.

Dimension	MCP	A2A
Participants	Agent ↔ Tool/Resource	Agent ↔ Agent
Intelligence	One side (agent) is intelligent	Both sides are intelligent
Statefulness	Typically stateless tool calls	Stateful tasks with lifecycle
Streaming	Limited (tool results)	First-class SSE streaming
Discovery	Tool manifests	Agent Cards
Auth model	Server-controlled	Mutual, OAuth 2.0
Typical latency	Milliseconds	Seconds to minutes
Use case	"Search the web", "Run SQL"	"Delegate to specialist"

25.7.1 When to Use Which

- Use **MCP** when the remote endpoint is a deterministic function: a database query, an API call, a code execution sandbox. The agent controls the interaction entirely.
- Use **A2A** when the remote endpoint needs to *reason* about the request: interpret ambiguous instructions, make judgment calls, use its own tools, or engage in multi-turn dialogue.
- Use **both** in the same system: an orchestrator agent uses A2A to delegate to specialist agents, and each specialist agent uses MCP to access its tools.

A note on ACP (Agent Communication Protocol). A third protocol worth awareness is the **Agent Communication Protocol (ACP)**, an emerging REST-based standard for synchronous agent-to-agent calls. Where A2A uses JSON-RPC with a stateful task lifecycle (submitted → working → completed), ACP uses plain HTTP request/response semantics—simpler to implement but without native support for long-running tasks or streaming. ACP is best suited for short, synchronous agent calls where the full response fits in a single HTTP response body. For long-running tasks or streaming results, A2A remains the more appropriate choice. The Quick Reference (Section 30.10) summarizes all three protocols side by side.

25.7.2 Combined Architecture

In production multi-agent systems, A2A and MCP work together at different layers: **A2A** handles inter-agent delegation and coordination (horizontal communication between peers), while **MCP** handles each agent's connection to its tools and data sources (vertical integration with capabilities). This separation of concerns is key to building scalable agentic architectures.

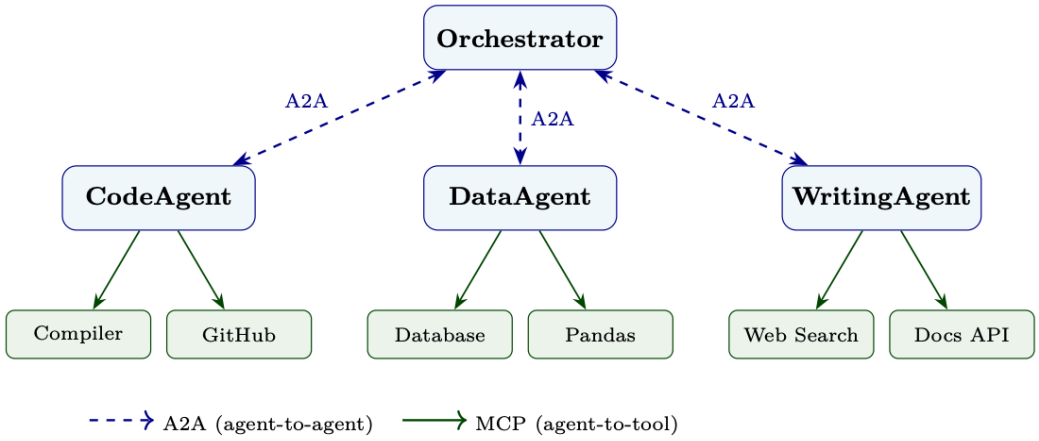


Figure 25.1: Combined A2A + MCP architecture. The orchestrator delegates to specialist agents via A2A; each agent accesses its tools via MCP servers.

- **A2A for delegation**: When an agent needs capabilities it doesn't have, it delegates to another agent via A2A task messages. Each agent is a self-contained service with its own Agent Card.
- **MCP for tool access**: Each agent connects to its tools through MCP servers. This means tools are never exposed directly to other agents — only through the owning agent's interface.
- **Separation of trust boundaries**: The orchestrator trusts specialist agents (verified via A2A authentication). Each specialist trusts its own MCP servers (local or authenticated). No transitive tool access.
- **Independent scaling**: Code-heavy workloads can scale CodeAgent instances; data workloads scale DataAgent. The orchestrator remains lightweight.

25.8 Security and Trust in Multi-Agent Systems

Multi-agent systems introduce unique security challenges. When Agent A delegates to Agent B, which delegates to Agent C, the chain of trust must be carefully managed.

25.8.1 Agent Identity Verification

Each agent must have a verifiable identity. Options include:

- **JWT tokens** [182] signed by a trusted identity provider, carrying the agent's ID, issuer, and expiry. Verified by the receiving agent using the provider's public key.
- **mTLS certificates** [38] issued by an internal CA, providing both authentication and transport encryption.
- **Decentralized identifiers (DIDs)** [63] for cross-organization scenarios where no single trusted authority exists.

25.8.2 Message Integrity and Encryption

- All A2A communication should occur over **TLS 1.3** [325] to prevent eavesdropping and man-in-the-middle attacks.
- For sensitive payloads, **end-to-end encryption** (e.g., JWE) ensures that intermediate infrastructure (load balancers, proxies) cannot read message content.
- **Message signing** (JWS) provides non-repudiation: the receiving agent can prove that a specific message came from a specific sender.

25.8.3 Authorization Scopes

Not every agent should be able to ask every other agent to do anything. OAuth 2.0 authorization scopes [141] define the boundaries:

```
# Example OAuth 2.0 scopes for a DataAgent
SCOPES = {
    "data:read":         "Read data from connected databases",
    "data:write":        "Write or modify data in connected databases",
    "data:export":       "Export data to external systems",
    "analysis:run":      "Execute statistical analyses",
    "analysis:schedule":"Schedule recurring analyses",
    "admin:config":      "Modify agent configuration"
}

# A ReportingAgent might hold only: data:read, analysis:run
# An ETL pipeline agent might hold: data:read, data:write, data:export
# Only a human admin holds: admin:config

class A2AServer:
    def verify_authorization(self, token: str, required_scope: str) -> bool:
        """Verify that the calling agent holds the required scope."""
        claims = jwt.decode(token, self.public_key, algorithms=["RS256"])
        granted_scopes = claims.get("scope", "").split()
        if required_scope not in granted_scopes:
            raise PermissionError(
                f"Caller lacks required scope '{required_scope}'. "
                f"Granted: {granted_scopes}"
            )
        return True
```

25.8.4 Audit Trails and Accountability

The Accountability Gap

In a chain of agent delegations, it can become unclear *who* is responsible for an action. If Agent A asks Agent B to delete a file, and Agent B does so, who is accountable? Every A2A interaction must be logged with: the calling agent's identity, the task description, the authorization token used, the timestamp, and the outcome. This audit trail is essential for incident response, compliance, and debugging.

Every A2A server should emit structured audit logs:

```
@dataclass
class A2AAuditEvent:
    timestamp: str          # ISO 8601
    workflow_id: str        # Correlation ID for the top-level workflow
    span_id: str            # This task's span
    parent_span_id: str     # Calling task's span (for delegation chains)
    caller_agent_id: str    # Verified identity of the calling agent
    callee_agent_id: str    # This agent's identity
    task_id: str
    skill_invoked: str
    authorization_scopes: list[str]
    outcome: str            # "completed" | "failed" | "rejected"
    duration_ms: int
    error_code: str | None
```

25.9 Implementation Example: Multi-Agent Research Workflow

The following example demonstrates a complete multi-agent research workflow using A2A: an `OrchestratorAgent` decomposes a research question, delegates to specialist agents, and synthesizes

their results.

```
"""
Multi-agent research workflow using A2A protocol.
Demonstrates: Agent Cards, A2A client/server, task lifecycle,
multi-turn interaction, and agent handoffs.
"""

import asyncio
import json
import uuid
from collections.abc import AsyncIterator
from datetime import datetime, timedelta, timezone

import httpx
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field

# -- Data Models ---------------------------------------------------------

class Part(BaseModel):
    type: str           # "text" | "file" | "data"
    text: str | None = None
    data: dict | None = None
    mimeType: str | None = None
    uri: str | None = None

class Message(BaseModel):
    role: str           # "user" | "agent"
    parts: list[Part]

class TaskStatus(BaseModel):
    state: str          # submitted | working | input-required | completed |
   failed
    message: str | None = None
    timestamp: str = Field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )

class Artifact(BaseModel):
    parts: list[Part]
    index: int = 0
    append: bool = False
    lastChunk: bool = True

class Task(BaseModel):
    id: str
    status: TaskStatus
    messages: list[Message] = []
    artifacts: list[Artifact] = []
    metadata: dict = {}

# -- A2A Client (HTTP/REST binding) ------------------------------------------
# Note: A2A v1.0 defines three protocol bindings: JSON-RPC 2.0, gRPC, and
# HTTP+JSON/REST. This example uses the REST binding for readability.

class A2AClient:
    """Client for sending tasks to A2A-compliant agents."""

    def __init__(self, agent_url: str, auth_token: str):
        self.agent_url = agent_url.rstrip("/")
        self.headers = {
            "Authorization": f"Bearer {auth_token}",
            "Content-Type": "application/json"
        }
```

```
    async def get_agent_card(self) -> dict:
        """Fetch the agent's capability card."""
        async with httpx.AsyncClient() as client:
            resp = await client.get(
                f"{self.agent_url}/.well-known/agent.json",
                headers=self.headers
            )
            resp.raise_for_status()
            return resp.json()

    async def send_task(self, message: Message,
                        task_id: str | None = None,
                        metadata: dict | None = None) -> Task:
        """Submit a task and return the initial task object."""
        payload = {
            "id": task_id or str(uuid.uuid4()),
            "message": message.model_dump(),
            "metadata": metadata or {}
        }
        async with httpx.AsyncClient() as client:
            resp = await client.post(
                f"{self.agent_url}/tasks/send",
                json=payload,
                headers=self.headers,
                timeout=30.0
            )
            resp.raise_for_status()
            return Task(**resp.json())

    async def stream_task(self, message: Message,
                          metadata: dict | None = None) -> AsyncIterator[dict]:
        """Submit a task and stream SSE events."""
        payload = {
            "id": str(uuid.uuid4()),
            "message": message.model_dump(),
            "metadata": metadata or {}
        }
        async with httpx.AsyncClient() as client:
            async with client.stream(
                "POST",
                f"{self.agent_url}/tasks/sendSubscribe",
                json=payload,
                headers={**self.headers, "Accept": "text/event-stream"},
                timeout=300.0
            ) as response:
                async for line in response.aiter_lines():
                    if line.startswith("data: "):
                        event_data = json.loads(line[6:])
                        yield event_data
                        if event_data.get("final"):
                            break

    async def get_task(self, task_id: str) -> Task:
        """Poll for task status."""
        async with httpx.AsyncClient() as client:
            resp = await client.get(
                f"{self.agent_url}/tasks/{task_id}",
                headers=self.headers
            )
            resp.raise_for_status()
            return Task(**resp.json())

    async def wait_for_completion(self, task: Task,
                                  poll_interval: float = 2.0) -> Task:
        """Poll until task reaches a terminal state."""
        terminal_states = {"completed", "failed", "canceled"}
```

```
        while task.status.state not in terminal_states:
            await asyncio.sleep(poll_interval)
            task = await self.get_task(task.id)
        return task

# -- A2A Server (FastAPI) ------------------------------------------------

class ResearchAgent:
    """
    A specialist research agent that searches literature and
    summarizes findings on a given topic.
    """

    AGENT_CARD = {
        "name": "ResearchAgent",
        "description": "Searches academic literature and synthesizes research
    findings.",
        "url": "https://research-agent.example.com/a2a",
        "version": "1.0.0",
        "capabilities": {
            "streaming": True,
            "pushNotifications": False,
            "stateTransitionHistory": True
        },
        "authentication": {"schemes": ["Bearer"]},
        "skills": [{
            "id": "literature-search",
            "name": "Literature Search",
            "description": "Search and summarize academic papers on a topic.",
            "tags": ["research", "literature", "academic", "papers"],
            "examples": [
                "Summarize recent papers on transformer attention mechanisms.",
                "What does the literature say about RLHF for code generation?"
            ],
            "inputModes": ["text"],
            "outputModes": ["text", "data"]
        }]
    }

    def __init__(self):
        self.tasks: dict[str, Task] = {}
        self.app = FastAPI(title="ResearchAgent A2A Server")
        self._register_routes()

    def _register_routes(self):
        @self.app.get("/.well-known/agent.json")
        async def agent_card():
            return self.AGENT_CARD

        @self.app.post("/tasks/send")
        async def send_task(request: Request):
            body = await request.json()
            task = await self._create_and_run_task(body)
            return task.model_dump()

        @self.app.post("/tasks/sendSubscribe")
        async def send_subscribe(request: Request):
            body = await request.json()
            return StreamingResponse(
                self._stream_task(body),
                media_type="text/event-stream"
            )

        @self.app.get("/tasks/{task_id}")
        async def get_task(task_id: str):
            if task_id not in self.tasks:
```

```
            raise HTTPException(status_code=404, detail="Task not found")
        return self.tasks[task_id].model_dump()

    async def _create_and_run_task(self, body: dict) -> Task:
        task_id = body.get("id", str(uuid.uuid4()))
        message = Message(**body["message"])

        task = Task(
            id=task_id,
            status=TaskStatus(state="submitted"),
            messages=[message],
            metadata=body.get("metadata", {})
        )
        self.tasks[task_id] = task

        # Run asynchronously
        asyncio.create_task(self._execute_task(task_id))
        return task

    async def _execute_task(self, task_id: str):
        task = self.tasks[task_id]
        task.status = TaskStatus(state="working")

        try:
            # Extract the research question from the message
            question = task.messages[0].parts[0].text

            # Simulate literature search (replace with real search tool)
            await asyncio.sleep(1)  # Simulated latency
            findings = await self._search_literature(question)

            # Produce artifact
            task.artifacts = [Artifact(parts=[
                Part(type="text", text=findings["summary"]),
                Part(type="data", data={"papers": findings["papers"],
                                        "query": question})
            ])]
            task.status = TaskStatus(state="completed")

        except Exception as e:
            task.status = TaskStatus(state="failed", message=str(e))

        self.tasks[task_id] = task

    async def _search_literature(self, question: str) -> dict:
        """Placeholder: in production, calls a real search API."""
        return {
            "summary": f"Based on a search of recent literature regarding "
                       f"'{question}', key findings include: ...",
            "papers": [
                {"title": "Attention Is All You Need", "year": 2017,
                 "relevance": 0.95},
                {"title": "RLHF: Training Language Models to Follow Instructions",
                 "year": 2022, "relevance": 0.88}
            ]
        }

    async def _stream_task(self, body: dict) -> AsyncIterator[str]:
        task = await self._create_and_run_task(body)

        # Stream status updates
        yield f"data: {json.dumps({'id': task.id, 'status': {'state': 'submitted
'}, 'final': False})}\n\n"
        yield f"data: {json.dumps({'id': task.id, 'status': {'state': 'working'},
'final': False})}\n\n"
```

```
        # Wait for completion
        while task.status.state not in ("completed", "failed", "canceled"):
            await asyncio.sleep(0.5)
            task = self.tasks[task.id]

        # Stream the artifact
        if task.artifacts:
            for part in task.artifacts[0].parts:
                event = {
                    "id": task.id,
                    "artifact": {
                        "parts": [part.model_dump()],
                        "index": 0,
                        "append": False,
                        "lastChunk": True
                    },
                    "final": False
                }
                yield f"data: {json.dumps(event)}\n\n"

        # Final status
        yield f"data: {json.dumps({'id': task.id, 'status': task.status.model_dump
    (), 'final': True})}\n\n"

# -- Orchestrator: Multi-Agent Workflow ---------------------------------------

class ResearchOrchestrator:
    """
    Orchestrates a multi-agent research workflow:
    1. Decomposes the research question into sub-questions
    2. Dispatches each sub-question to a ResearchAgent
    3. Synthesizes results into a final report
    """

    def __init__(self, research_agent_url: str, auth_token: str):
        self.research_client = A2AClient(research_agent_url, auth_token)
        self.workflow_id = str(uuid.uuid4())

    async def run(self, research_question: str) -> str:
        print(f"[Orchestrator] Starting workflow {self.workflow_id}")
        print(f"[Orchestrator] Question: {research_question}")

        # Step 1: Decompose into sub-questions
        sub_questions = self._decompose(research_question)
        print(f"[Orchestrator] Decomposed into {len(sub_questions)} sub-questions"
    )

        # Step 2: Dispatch sub-questions in parallel
        tasks = await asyncio.gather(*[
            self.research_client.send_task(
                message=Message(role="user", parts=[Part(type="text", text=q)]),
                metadata={"workflowId": self.workflow_id, "subQuestion": i}
            )
            for i, q in enumerate(sub_questions)
        ])

        # Step 3: Wait for all tasks to complete
        completed_tasks = await asyncio.gather(*[
            self.research_client.wait_for_completion(task)
            for task in tasks
        ])

        # Step 4: Check for failures
        failed = [t for t in completed_tasks if t.status.state == "failed"]
        if failed:
            print(f"[Orchestrator] Warning: {len(failed)} sub-tasks failed")
```

```
        # Step 5: Synthesize results
        findings = []
        for task, question in zip(completed_tasks, sub_questions):
            if task.status.state == "completed" and task.artifacts:
                summary = task.artifacts[0].parts[0].text
                findings.append(f"### {question}\n{summary}")

        report = self._synthesize(research_question, findings)
        print(f"[Orchestrator] Workflow complete. Report: {len(report)} chars")
        return report

    def _decompose(self, question: str) -> list[str]:
        """Decompose a complex question into focused sub-questions."""
        # In production: use an LLM to decompose
        return [
            f"What are the foundational methods for: {question}?",
            f"What are the most recent advances in: {question}?",
            f"What are the open challenges and limitations in: {question}?"
        ]

    def _synthesize(self, question: str, findings: list[str]) -> str:
        """Synthesize sub-findings into a coherent report."""
        # In production: use an LLM to synthesize
        sections = "\n\n".join(findings)
        return f"# Research Report: {question}\n\n{sections}"

# -- Entry Point ---------------------------------------------------------

async def main():
    orchestrator = ResearchOrchestrator(
        research_agent_url="https://research-agent.example.com/a2a",
        auth_token="eyJhbGciOiJSUzI1NiJ9..."
    )
    report = await orchestrator.run(
        "Reinforcement learning from human feedback for large language models"
    )
    print(report)

if __name__ == "__main__":
    asyncio.run(main())
```

25.10 Summary

Key Takeaways: Agent-to-Agent Communication

1. **A2A enables specialization at scale**: By routing tasks to specialist agents, multi-agent systems achieve depth and breadth simultaneously. (Chapter 26 covers multi-agent architectures in depth.)

2. **Google's A2A Protocol** provides a production-ready, open standard for interoperable agent communication, with Agent Cards, task lifecycle management, SSE streaming, and enterprise authentication.

3. **Communication patterns** range from simple request-response to complex negotiation and auction-based allocation—choose based on task complexity and latency needs.

4. **A2A and MCP are complementary**: A2A connects agents to agents; MCP connects agents to tools. Most production systems use both.

5. **Security is non-negotiable**: Agent identity verification, authorization scopes, and audit trails are essential in any multi-agent deployment.

6. **Coordination protocols** (Contract Net, Blackboard, Consensus) provide structured mechanisms for collective decision-making beyond simple delegation.

7. **Observability through correlation IDs** is critical for debugging and auditing complex multi-agent workflows spanning many agents and tools.

Open Research Questions in A2A

- How should agents handle *conflicting instructions* from multiple orchestrators in a hierarchy? What conflict resolution mechanisms are most effective?
- Can agents *learn* better routing and delegation strategies through experience, rather than relying on static capability declarations?
- How do we prevent *prompt injection attacks* where a malicious agent manipulates a downstream agent by embedding adversarial instructions in its messages?
- What are the right *privacy boundaries* for context passing—how much conversation history should a sub-agent see, and how do we enforce these boundaries technically?
- As agent networks grow to hundreds or thousands of agents, how do we maintain *coherent global state* without creating bottlenecks or consistency violations?

Chapter 26

Multi-Agent Systems

No single agent, however capable, can efficiently handle every aspect of a rich, open-ended task. Multi-agent systems (MAS) address this limitation by decomposing complex problems across networks of specialized agents that communicate, coordinate, and collectively produce results that exceed what any individual agent could achieve alone. This chapter covers the theoretical foundations of multi-agent systems, the architectural patterns for agent coordination, and the engineering challenges of building reliable agent societies.

GAS Connection — Multi-Agent Systems

The GAS is a small but complete multi-agent system. The following chapter's theoretical framework—architectures, coordination mechanisms, role design, and MAS challenges—maps directly onto the GAS design decisions.

Multi-agent architectures (§ Multi-Agent Architectures). The GAS uses a *hierarchical* architecture: `supervisor_node()` is the top-level orchestrator; `monitor_node()`, `advisor_node()`, `safety_node()`, and `caregiver_node()` are specialised subagents. This is contrasted with a *flat* architecture (all agents communicate peer-to-peer) and a *market* architecture (agents bid for tasks). The hierarchical architecture was chosen for the GAS because it provides clear accountability and is easier to audit in a regulated medical context.

Coordination mechanisms (§ Coordination Mechanisms). The GAS uses *centralised coordination*: `supervisor_node()` makes all routing decisions by setting `state["next_agent"]` in `GASState`. The `route_from_supervisor()` conditional edge function in `capstone_gas/gas_system.py` reads this field and routes to the appropriate node. This is appropriate for a safety-critical system where a single point of coordination simplifies auditing.

Role design and specialisation (§ Role Design and Specialisation). Each GAS agent function has a precisely defined role with clear input/output contracts. `monitor_node()`: detect glucose anomalies and set `alert_level`. `advisor_node()`: retrieve guidelines and set `recommendation`. `safety_node()`: evaluate the recommendation and set `safety_score` and `safety_approved`. `caregiver_node()`: compose and send the alert, set `caregiver_notified`. This strict role definition prevents role overlap and role gaps.

MAS challenges (§ Challenges and Solutions). The GAS faces two classic MAS challenges: (1) *Coordination overhead*: invoking four agent nodes sequentially adds latency. The GAS mitigates this by skipping the Advisor and Safety Critic when `alert_level` is low or none. (2) *Inconsistent beliefs*: the Advisor and Safety Critic may have different "beliefs" about the patient state if the state changes between their invocations. The GAS mitigates this by taking a snapshot of the patient state at the start of each pipeline invocation and passing it immutably via `GASState`.

Self-evolving agents (§ Self-Evolving Agents: BDI Meets LLMs). The GAS roadmap includes a BDI (Belief-Desire-Intention) layer for the Supervisor: the Supervisor would maintain explicit beliefs about the patient's state (from `readings_history` in `GASState`), desires (keep glucose in the 70–180 mg/dL target range), and intentions (current routing plan). This BDI layer would allow the Supervisor to reason about long-term patient goals, not just immediate anomalies.

26.1 Motivation: Why Multiple Agents?

The history of artificial intelligence is, in many ways, a history of scale. Early AI systems were monolithic: a single program, a single knowledge base, a single inference engine. As problems grew more complex, researchers discovered that no single agent—however capable—could efficiently handle every aspect of a rich, open-ended task. This insight, long established in distributed AI and multi-agent systems (MAS) research [397, 403], has found renewed urgency in the era of large language models.

The Core Intuition

A single LLM, no matter how large, is a generalist. A team of specialized LLMs, each focused on a narrow sub-problem and communicating their results, can outperform the generalist on complex, multi-faceted tasks—just as a team of human specialists outperforms a single generalist on a complex engineering project.

Four fundamental motivations drive the shift from monolithic agents to *agent societies*:

Specialization. Different sub-tasks benefit from different capabilities, prompting strategies, and even different base models. A code-generation agent can be fine-tuned on programming corpora; a fact-checking agent can be grounded with retrieval tools; a creative-writing agent can be prompted for stylistic diversity. Forcing a single agent to excel at all of these simultaneously is both inefficient and often impossible.

Parallelism. Many real-world tasks decompose into independent sub-tasks that can be executed concurrently. A research pipeline that requires literature review, data analysis, and report writing can run all three in parallel, dramatically reducing wall-clock time. Sequential single-agent processing is a bottleneck that multi-agent parallelism eliminates.

Robustness. A single agent is a single point of failure. If it hallucinates, gets stuck in a loop, or produces a subtly wrong answer, there is no check. Multi-agent systems introduce redundancy: a second agent can verify, critique, or independently re-derive results. Adversarial agents can probe for weaknesses before outputs are trusted.

Emergent Capabilities. Perhaps most intriguingly, agent collectives can exhibit capabilities that no individual agent possesses. Through debate, negotiation, and iterative refinement, multi-agent systems can arrive at solutions that transcend what any single agent could produce alone—a computational analog to the emergent intelligence of social organisms.

Historical Context

Multi-agent systems research dates to the 1980s, with foundational work on distributed problem solving [84], the Contract Net Protocol [352], and FIPA agent communication standards [99]. The shift to LLM-based agents reanimates these classical ideas with a new substrate: instead of hand-coded agents with symbolic reasoning, we now have agents whose "cognition" emerges from learned neural representations. The core architectural patterns—hierarchies, markets, blackboards, message passing—remain remarkably relevant.

The transition from monolithic agents to agent societies mirrors a broader pattern in complex systems: as the problem space grows, distributed, modular architectures consistently outperform centralized, monolithic ones. The question is no longer *whether* to use multiple agents, but *how* to organize them.

26.2 Multi-Agent Architectures

The topology of a multi-agent system—how agents are connected and how authority flows among them—is the most consequential architectural decision. Four canonical patterns have emerged, each with distinct trade-offs. These topologies map directly to graph structures: the supervisor pattern is

a hub-and-spoke graph, decentralized peer-to-peer is a mesh, hierarchical is a tree of subgraphs, and swarm is a dynamic handoff graph. Chapter 20 covers the graph primitives used to implement these architectures in practice.

26.2.1 Centralized (Supervisor/Manager) Architecture

In a centralized architecture, a single *orchestrator* agent (variously called supervisor, manager, or planner) holds global state, decomposes tasks, delegates sub-tasks to worker agents, and aggregates their results. The topology is a **hub-and-spoke**: all communication flows through the central node.

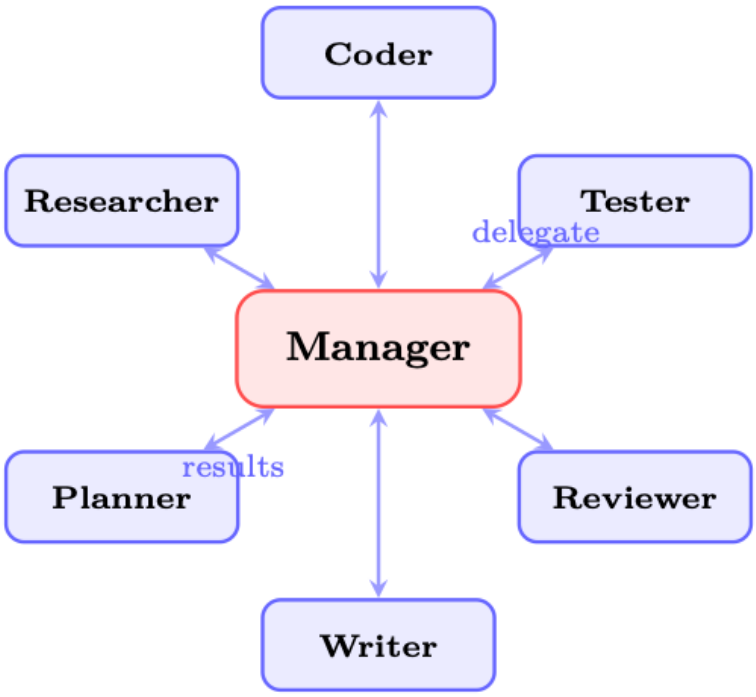


Figure 26.1: Centralized (Supervisor) architecture. The manager delegates tasks to specialized workers and aggregates their outputs. All communication flows through the central hub.

The manager's responsibilities include:

- **Task routing**: deciding which worker is best suited for each sub-task
- **Context management**: providing each worker with the relevant subset of global context
- **Result aggregation**: synthesizing worker outputs into a coherent whole
- **Error handling**: detecting worker failures and re-routing or retrying

Supervisor Pattern in LangGraph

This example applies the graph engineering primitives from Chapter 20: a typed `TeamState` schema, node functions, conditional edges via `add_conditional_edges`, and a compile step. The supervisor node acts as the routing function in a hub-and-spoke topology.

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal

class TeamState(TypedDict):
    task: str
    plan: str
    code: str
    tests: str
    review: str
    next_agent: str
```

```
    final_output: str

def supervisor_node(state: TeamState) -> TeamState:
    """Central orchestrator: decides which agent to invoke next."""
    messages = [
        {"role": "system", "content": SUPERVISOR_PROMPT},
        {"role": "user",   "content": f"Task: {state['task']}\n"
                                      f"Plan: {state.get('plan','')}\n"
                                      f"Code: {state.get('code','')}\n"
                                      f"Tests: {state.get('tests','')}\n"
                                      "Which agent should act next? "
                                      "Options: planner, coder, tester,
    reviewer, FINISH"}
    ]
    response = llm.invoke(messages)
    return {**state, "next_agent": response.content.strip()}

def route(state: TeamState) -> Literal["planner","coder","tester","reviewer","
    __end__"]:
    return state["next_agent"] if state["next_agent"] != "FINISH" else END

builder = StateGraph(TeamState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("planner",    planner_node)
builder.add_node("coder",      coder_node)
builder.add_node("tester",     tester_node)
builder.add_node("reviewer",   reviewer_node)

builder.add_edge(START, "supervisor")
builder.add_conditional_edges("supervisor", route)
for agent in ["planner", "coder", "tester", "reviewer"]:
    builder.add_edge(agent, "supervisor")   # always return to supervisor

graph = builder.compile()
```

Centralized Architecture Trade-offs

Pros: Simple control flow; clear accountability; easy to debug (all decisions in one place); straightforward to implement.

Cons: Single point of failure—if the manager hallucinates or gets confused, the entire system fails; the manager becomes a bottleneck under high load; the manager's context window must hold the global state, limiting scalability.

26.2.2 Decentralized (Peer-to-Peer) Architecture

In a decentralized architecture, agents interact directly with one another without a central coordinator. The topology is a **mesh**: any agent can communicate with any other. Coordination emerges from local interactions rather than global planning.

Emergent coordination in peer-to-peer systems arises through mechanisms such as:

- **Negotiation**: agents bid for tasks or resources
- **Stigmergy**: agents modify shared state that others observe (see Section 26.3.6)
- **Gossip protocols**: agents propagate information through the network
- **Local consensus**: small groups of agents reach agreement without global coordination

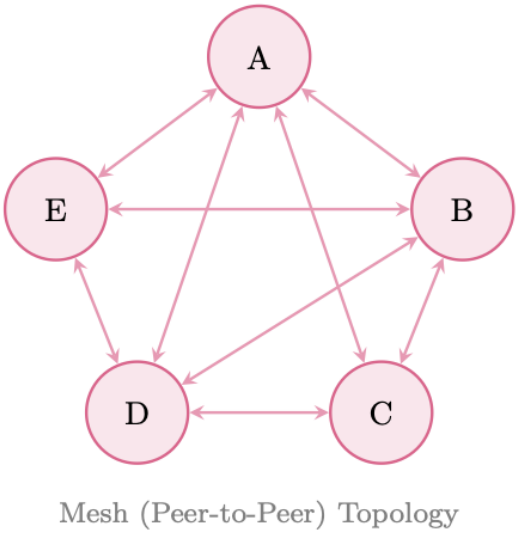


Figure 26.2: Decentralized (peer-to-peer) architecture. Agents communicate directly; coordination emerges from local interactions.

Decentralized Architecture Trade-offs

Pros: Resilient to individual agent failures; scales naturally as agents are added; no bottleneck.

Cons: Hard to debug—emergent behavior is difficult to trace; potential for conflicts when agents have inconsistent views of state; coordination overhead grows as $O(n^2)$ with naive message passing; difficult to guarantee global consistency.

26.2.3 Hierarchical Architecture

Hierarchical architectures generalize the centralized pattern into a **tree structure** with multiple levels of management. A top-level orchestrator delegates to domain-specific sub-managers, who in turn delegate to specialized workers. This mirrors the organizational structure of large enterprises.

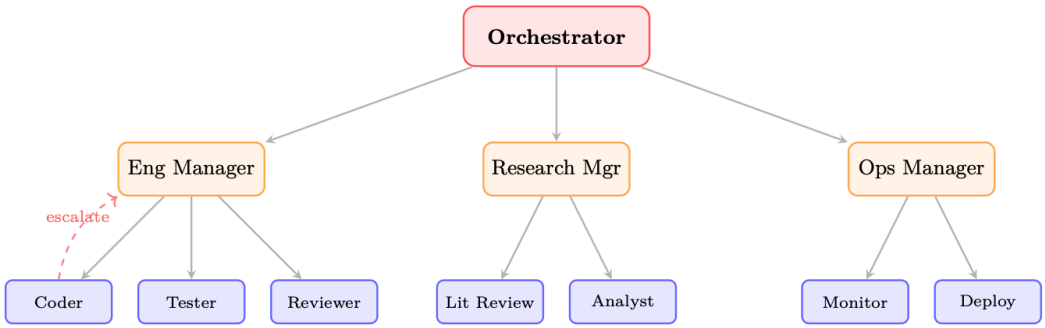


Figure 26.3: Hierarchical architecture. A top-level orchestrator delegates to domain sub-managers, who delegate to specialized workers. Dashed arrow shows an escalation path.

Key features of hierarchical systems:

- **Delegation chains**: authority and context flow down the tree; results flow up
- **Escalation paths**: workers can escalate unresolvable issues to their manager
- **Domain isolation**: sub-managers maintain domain-specific context, reducing the cognitive load on the top-level orchestrator
- **Scope limitation**: each agent only needs to know about its immediate superiors and subordinates

The enterprise analogy is apt: a CEO (top orchestrator) sets strategy; VPs (sub-managers) translate strategy into domain plans; individual contributors (workers) execute. The hierarchy enables scale while preserving accountability.

26.2.4 Swarm Architecture

Swarm architectures, inspired by biological systems (ant colonies, bird flocking), consist of many **loosely coupled agents** that follow simple local rules, producing complex global behavior without any central coordinator or global state.

OpenAI's **Swarm** framework [292] (now superseded by the OpenAI Agents SDK, but its conceptual primitives remain influential) operationalizes this with two primitives:

- **Routines**: sequences of instructions an agent follows to complete a sub-task
- **Handoffs**: an agent transferring control (and relevant context) to another agent

OpenAI Swarm: Routines and Handoffs

```
from swarm import Swarm, Agent

client = Swarm()

def transfer_to_billing():
    """Handoff: transfer control to the billing specialist."""
    return billing_agent

def transfer_to_technical():
    """Handoff: transfer control to the technical support agent."""
    return technical_agent

triage_agent = Agent(
    name="Triage Agent",
    instructions="""You are a customer service triage agent.
    Determine the nature of the customer's issue:
    - For billing questions, transfer to billing.
    - For technical issues, transfer to technical support.
    - For general questions, answer directly.""",
    functions=[transfer_to_billing, transfer_to_technical],
)

billing_agent = Agent(
    name="Billing Specialist",
    instructions="You handle billing inquiries. "
                 "Access account data and resolve payment issues.",
    functions=[lookup_account, process_refund],
)

technical_agent = Agent(
    name="Technical Support",
    instructions="You resolve technical issues. "
                 "Diagnose problems and provide step-by-step solutions.",
    functions=[run_diagnostics, escalate_to_engineering],
)

# No global state --- each agent operates on its local context
response = client.run(
    agent=triage_agent,
    messages=[{"role": "user", "content": "My invoice is wrong"}]
)
```

Swarm Properties

- **No global state**: each agent maintains only its local context window
- **Local decisions**: routing decisions are made by the current agent, not a central planner
- **Task completion through collective behavior**: complex tasks are completed through a chain of handoffs, each agent contributing its specialty
- **Lightweight**: no orchestration overhead; agents are stateless between handoffs

26.2.5 The Orchestrator-Worker-Critic Triad

As multi-agent systems matured from research prototypes to production deployments, a recurring three-role pattern emerged independently across teams at different organizations. The **Orchestrator-Worker-Critic (OWC) triad** is now a de facto standard for structuring agentic pipelines that require both breadth (many parallel subtasks) and quality assurance (verification before committing results).

The three roles.

- **Orchestrator**: Receives the high-level goal, decomposes it into subtasks, assigns subtasks to workers, tracks progress, and synthesizes final results. The orchestrator does not execute tasks directly—it reasons about *what* needs to be done and *who* should do it. It maintains the global task graph and handles dependency ordering.
- **Worker**: Executes a single, well-scoped subtask. Workers are specialists: a coding worker writes and tests code; a research worker retrieves and summarizes documents; a data worker queries databases and transforms results. Each worker operates in an isolated context with its own tool set, minimizing interference between parallel executions.
- **Critic**: Reviews worker outputs before they are accepted by the orchestrator. The critic checks for correctness, consistency with the overall goal, safety violations, and quality thresholds. Critically, the critic is a *separate agent*—not the worker reviewing its own output—which avoids the self-evaluation bias documented in LLM-as-judge research (Section 14.7).

Table 26.1: Orchestrator-Worker-Critic role comparison.

Role	Primary concern	Context scope	Failure mode
Orchestrator	Goal decomposition, dependency management	Global task graph	Over-decomposition; lost context across subtasks
Worker	Subtask execution, tool use	Local subtask context	Scope creep; ignoring constraints
Critic	Quality assurance, safety	Worker output + goal spec	False positives (blocking good work); false negatives (passing bad work)

When to use the OWC triad. The triad adds coordination overhead—three agents instead of one—so it is only justified when: (1) the task is decomposable into parallel subtasks, (2) worker errors are costly to reverse, or (3) quality requirements exceed what a single agent reliably achieves. For simple, sequential tasks, a single agent with a self-reflection step is more efficient.

OWC vs. Hierarchical vs. Swarm

- **Hierarchical**: Multi-level delegation (orchestrator → sub-orchestrator → worker). Best for deeply nested task structures.
- **OWC Triad**: Flat delegation with quality gate (orchestrator → worker, critic reviews).

Best for parallel tasks with quality requirements.

- **Swarm**: Peer-to-peer handoffs, no central coordinator. Best for routing tasks where the correct specialist is not known in advance.

26.2.6 Agent Mesh Architecture

The hierarchical and OWC patterns assume a relatively stable topology: a known set of agents with fixed roles. **Agent mesh architecture** generalizes this to dynamic, peer-to-peer networks where agents discover each other at runtime, form ad-hoc collaborations, and dissolve them when the task is complete—analogous to a service mesh in microservices infrastructure.

Core properties.

- **Dynamic discovery**: Agents register capabilities in a shared registry (analogous to A2A Agent Cards, Section 25.4). Any agent can query the registry to find peers with the capabilities it needs.
- **Peer-to-peer delegation**: Unlike hierarchical systems, any agent can delegate to any other. There is no fixed orchestrator; the agent that receives the original task becomes the de facto coordinator for that task.
- **Capability-based routing**: Delegation decisions are made by matching task requirements to agent capability descriptions, not by hardcoded routing logic. This allows new specialist agents to be added to the mesh without modifying existing agents.
- **Fault tolerance**: If a peer agent fails or is unavailable, the requesting agent can query the registry for an alternative with equivalent capabilities.

Relationship to A2A. The A2A protocol (Chapter 25) provides the communication substrate for agent meshes: Agent Cards serve as capability advertisements, task messages carry delegated work, and streaming responses enable long-running subtasks. A mesh is, in essence, a dynamic A2A network where the topology is not pre-specified.

Mesh Complexity and Debugging

Agent meshes are powerful but difficult to debug. When a task fails, the failure may have propagated through several peer-to-peer delegations before surfacing. Distributed tracing—attaching a correlation ID to every task and propagating it through all delegations—is essential for post-hoc analysis. Without it, root-cause attribution in a mesh becomes intractable.

26.3 Coordination Mechanisms

How agents coordinate—how they share information, divide work, and resolve conflicts—is as important as the topology. Six canonical coordination mechanisms apply to LLM-based multi-agent systems.

26.3.1 Shared State (Global Blackboard)

The **blackboard architecture** [144] provides a shared data structure that all agents can read from and write to. In LLM systems, this is typically implemented as a shared dictionary, database, or structured document.

```
import threading
from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List

@dataclass
```

```python
class BlackboardEntry:
    value: Any
    author: str
    timestamp: float
    confidence: float = 1.0

class Blackboard:
    """Thread-safe shared state for multi-agent coordination."""

    def __init__(self):
        self._data: Dict[str, BlackboardEntry] = {}
        self._lock = threading.RLock()
        self._subscribers: Dict[str, List[Callable]] = {}

    def write(self, key: str, value: Any, author: str,
              confidence: float = 1.0) -> bool:
        """Write to blackboard; higher-confidence entries win conflicts."""
        with self._lock:
            existing = self._data.get(key)
            if existing and existing.confidence > confidence:
                return False  # Conflict: existing entry wins
            import time
            self._data[key] = BlackboardEntry(
                value=value, author=author,
                timestamp=time.time(), confidence=confidence
            )
            self._notify(key, value)
            return True

    def read(self, key: str) -> Any:
        with self._lock:
            entry = self._data.get(key)
            return entry.value if entry else None

    def subscribe(self, key: str, callback: Callable):
        """Agents subscribe to changes on specific keys."""
        self._subscribers.setdefault(key, []).append(callback)

    def _notify(self, key: str, value: Any):
        for cb in self._subscribers.get(key, []):
            cb(key, value)
```

Listing 26.1: Shared blackboard with conflict resolution

26.3.2 Message Passing

Message passing is the most natural coordination mechanism for LLM agents: agents communicate by sending structured text messages to one another. Key design decisions include:

- **Message format**: structured (JSON schema) vs. natural language vs. hybrid
- **Routing**: direct (agent-to-agent) vs. broadcast vs. topic-based pub/sub
- **Conversation threads**: maintaining context across multi-turn exchanges
- **Acknowledgment**: whether senders require confirmation of receipt/processing

26.3.3 Planning and Decomposition

A manager agent decomposes a high-level task into a **directed acyclic graph (DAG)** of sub-tasks, assigns each to an appropriate worker, and tracks dependencies. This is the multi-agent analog of classical hierarchical task network (HTN) planning.

```python
from dataclasses import dataclass, field
from typing import List, Optional
import asyncio

@dataclass
class Task:
    id: str
    description: str
    assigned_to: str
    dependencies: List[str] = field(default_factory=list)
    status: str = "pending"   # pending | running | done | failed
    result: Optional[str] = None

class TaskDAG:
    def __init__(self):
        self.tasks: dict[str, Task] = {}

    def add_task(self, task: Task):
        self.tasks[task.id] = task

    def ready_tasks(self) -> List[Task]:
        """Return tasks whose dependencies are all completed."""
        return [
            t for t in self.tasks.values()
            if t.status == "pending"
            and all(self.tasks[d].status == "done"
                    for d in t.dependencies)
        ]

    async def execute(self, agent_pool: dict):
        while any(t.status != "done" for t in self.tasks.values()):
            ready = self.ready_tasks()
            if not ready:
                await asyncio.sleep(0.1)
                continue
            # Execute ready tasks in parallel
            await asyncio.gather(*[
                self._run_task(t, agent_pool[t.assigned_to])
                for t in ready
            ])

    async def _run_task(self, task: Task, agent):
        task.status = "running"
        try:
            task.result = await agent.execute(task.description)
            task.status = "done"
        except Exception as e:
            task.status = "failed"
            raise
```

Listing 26.2: Task DAG decomposition

26.3.4 Voting and Consensus

When multiple agents produce conflicting outputs, voting mechanisms aggregate their responses into a single decision. Common schemes include:

- **Majority voting**: the most common answer wins; effective for factual questions
- **Weighted voting**: agents with higher track records or confidence scores receive more weight
- **Debate-based resolution**: agents argue for their positions; a judge agent decides
- **Delphi method**: iterative rounds where agents revise their answers after seeing others' reasoning

Formally, given n agents producing outputs $\{o_1, \ldots, o_n\}$ with weights $\{w_1, \ldots, w_n\}$, the weighted consensus is:

$$o^* = \arg\max_o \sum_{i=1}^{n} w_i \cdot \mathbf{1}[o_i = o] \tag{26.1}$$

For continuous outputs (e.g., probability estimates), weighted averaging applies:

$$\hat{p} = \frac{\sum_{i=1}^{n} w_i \cdot p_i}{\sum_{i=1}^{n} w_i} \tag{26.2}$$

26.3.5 Market-Based Coordination

Market mechanisms allocate tasks and resources through **auctions and bidding**. The Contract Net Protocol [352], one of the oldest multi-agent coordination mechanisms, is a task auction:

1. A *manager* broadcasts a task announcement with requirements
2. *Contractor* agents submit bids (capability declarations + cost estimates)
3. The manager awards the contract to the best bidder
4. The winning contractor executes and reports results

In LLM systems, bids can be expressed in natural language ("I can complete this in 3 steps with high confidence") or structured formats. Market mechanisms are particularly effective for resource-constrained settings where API costs must be minimized.

26.3.6 Stigmergy: Indirect Communication Through Environment

Stigmergy [125] replaces explicit agent-to-agent messaging with a simpler mechanism: each agent modifies the shared environment as a side effect of its work, and other agents react to those modifications rather than to direct signals. The classic illustration is a foraging ant depositing pheromone on its return path; subsequent ants amplify successful routes without any ant "talking" to another.

In LLM multi-agent systems, stigmergy manifests as:

- **Shared documents**: agents write to a shared document; others read and build upon it
- **Code repositories**: one agent commits code; another reads and extends it
- **Annotation layers**: agents annotate shared artifacts (highlight errors, add comments)
- **Task queues**: agents add and consume tasks from a shared queue

Stigmergy enables coordination without explicit communication overhead—agents simply observe the state of the shared environment and act accordingly.

26.4 Communication Protocols

Effective multi-agent systems require well-defined communication protocols: agreed-upon formats, semantics, and patterns for agent-to-agent messages. (For the standardized inter-agent protocol, see Chapter 25.)

26.4.1 Structured Message Formats

Messages between LLM agents should be structured to enable reliable parsing and routing. A minimal message schema:

```python
from pydantic import BaseModel, Field
from typing import Literal, Optional, Dict, Any
from datetime import datetime, timezone
import uuid

PerformativeType = Literal[
    "inform",     # Share information
    "request",    # Request an action
    "propose",    # Propose a course of action
    "accept",     # Accept a proposal
    "reject",     # Reject a proposal
    "query",      # Ask a question
    "confirm",    # Confirm receipt/completion
    "failure",    # Report a failure
]

class AgentMessage(BaseModel):
    message_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    conversation_id: str          # Groups related messages
    sender: str                   # Agent identifier
    receiver: str                 # Target agent (or "broadcast")
    performative: PerformativeType
    content: str                  # Natural language content
    metadata: Dict[str, Any] = {} # Structured payload
    reply_to: Optional[str] = None  # message_id being replied to
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc)
    )

    def to_llm_prompt(self) -> str:
        """Render message as a prompt fragment for the receiving agent."""
        return (
            f"[MESSAGE from {self.sender}]\n"
            f"Type: {self.performative}\n"
            f"Content: {self.content}\n"
            + (f"Metadata: {self.metadata}\n" if self.metadata else "")
        )
```

Listing 26.3: Agent message schema

26.4.2 Performative Types (FIPA-ACL Inspired)

Drawing from the FIPA Agent Communication Language [99], modernized for LLM agents:

Table 26.2: FIPA-ACL-inspired performative types for LLM agent messages.

Performative	Semantics	Example Use
`inform`	Sender believes ϕ is true	Share research findings
`request`	Sender wants receiver to do α	Delegate a sub-task
`propose`	Sender proposes plan π	Suggest an approach
`accept`	Receiver agrees to proposal	Confirm task assignment
`reject`	Receiver declines proposal	Refuse incompatible task
`query`	Sender wants to know ϕ	Ask for clarification
`confirm`	Sender confirms ϕ occurred	Acknowledge completion
`failure`	Sender failed to achieve α	Report error

26.4.3 Context Sharing Strategies

A critical challenge in multi-agent communication is **context management**: how much history does each agent need? Three strategies:

- **Full history**: pass the entire conversation history to each agent. Maximally informative but expensive; context windows fill quickly.
- **Summary**: a summarizer agent condenses prior exchanges into a compact summary. Efficient but lossy; important details may be dropped.
- **Relevant excerpt**: retrieve only the most relevant prior messages using semantic search. Balances cost and informativeness; requires a retrieval mechanism.

Context Sharing Rule of Thumb

Use **full history** for short conversations (<10 turns); **summaries** for medium-length conversations; **retrieval-augmented excerpts** for long-running agent sessions. Always include the most recent k messages verbatim to preserve immediate context.

26.5 Role Design and Specialization

The design of agent roles—their capabilities, personas, and responsibilities—is as much an art as a science. Well-designed roles enable specialization; poorly designed roles create confusion and redundancy.

26.5.1 Defining Agent Roles

Common roles in LLM multi-agent systems:

Table 26.3: Common agent roles in LLM multi-agent systems.

Role	Primary Capability	Typical Tools
Researcher	Information gathering, synthesis	Web search, RAG, databases
Planner	Task decomposition, scheduling	None (reasoning only)
Coder	Code generation, debugging	Code interpreter, linter
Reviewer	Quality assessment, critique	None (reasoning only)
Tester	Test generation, execution	Test runner, coverage tools
Writer	Prose generation, editing	Grammar checker, style guide
Critic	Adversarial evaluation	None (reasoning only)
Orchestrator	Coordination, delegation	All agent interfaces

26.5.2 Capability-Based vs. Role-Based Assignment

Two philosophies for task assignment:

- **Role-based**: tasks are assigned based on predefined role labels. Simple and predictable; may be suboptimal when a task spans multiple roles.
- **Capability-based**: tasks are assigned based on a dynamic assessment of each agent's capabilities relative to the task requirements. More flexible; requires a capability registry and matching mechanism.

26.5.3 Dynamic Role Reassignment

In long-running systems, static role assignments become suboptimal. Dynamic reassignment allows agents to take on new roles based on:

- Current workload (load balancing)
- Demonstrated performance on recent tasks
- Changing task requirements
- Agent failures requiring coverage

26.5.4 Persona Design for Diversity of Thought

A subtle but powerful technique: give agents **distinct personas** that encourage diverse perspectives. Rather than five identical "assistant" agents, design:

- An *optimist* who emphasizes opportunities
- A *skeptic* who challenges assumptions
- A *pragmatist* who focuses on implementation
- A *visionary* who thinks long-term
- A *devil's advocate* who argues the opposite position

This diversity of thought, inspired by techniques like Six Thinking Hats [30], reduces groupthink and produces more robust collective reasoning.

Role Conflict Resolution

When agents have overlapping responsibilities, conflicts arise. Resolve them with explicit **priority rules**: define which role takes precedence for each task type. Alternatively, use a **meta-agent** whose sole responsibility is conflict arbitration. Never leave role conflicts implicit—they will manifest as contradictory outputs or infinite loops.

26.6 Multi-Agent Patterns for LLMs

Beyond architectural topologies, several **interaction patterns** have proven particularly effective for LLM-based multi-agent systems. (These complement the single-agent design patterns in Chapter 21.)

26.6.1 Debate Pattern

Multiple agents argue for different positions; a judge agent evaluates the arguments and decides. Debate has been shown to improve factual accuracy and reduce hallucinations [82].

```
async def debate_round(question: str, agents: list, judge: Agent,
                       rounds: int = 2) -> str:
    """Run a multi-agent debate and return the judge's verdict."""
    positions = {a.name: await a.generate_position(question)
                 for a in agents}

    for round_num in range(rounds):
        # Each agent sees others' positions and can rebut
        rebuttals = {}
        for agent in agents:
            others = {k: v for k, v in positions.items()
                      if k != agent.name}
            rebuttals[agent.name] = await agent.rebut(
                question, positions[agent.name], others
```

```
            )
        positions = rebuttals

    # Judge evaluates all final positions
    verdict = await judge.evaluate(question, positions)
    return verdict
```

Listing 26.4: Debate pattern implementation

26.6.2 Reflection Pattern

One agent generates an output; a second agent critiques it; the first agent revises based on the critique. This implements a generate-critique-revise loop that iteratively improves quality.

```
async def reflection_loop(task: str, generator: Agent,
                          critic: Agent, max_rounds: int = 3) -> str:
    draft = await generator.generate(task)

    for _ in range(max_rounds):
        critique = await critic.critique(task, draft)
        if critique.is_satisfactory:
            break
        draft = await generator.revise(task, draft, critique.feedback)

    return draft
```

Listing 26.5: Reflection pattern

26.6.3 Division of Labor Pattern

The task is decomposed into independent sub-tasks executed in parallel. Results are aggregated by a synthesis agent. This pattern maximizes throughput for embarrassingly parallel tasks.

26.6.4 Pipeline Pattern

Agents form a sequential processing chain: each agent transforms the output of the previous agent. Analogous to Unix pipes. Effective for tasks with clear sequential dependencies (e.g., research → outline → draft → edit → format).

26.6.5 Ensemble Pattern

Multiple agents independently solve the same problem; a selection mechanism picks the best answer (best-of-N) or aggregates answers (mixture-of-experts style). Improves reliability at the cost of compute.

$$o^* = \arg \max_{o \in \{o_1, \ldots, o_N\}} \text{score}(o, \text{task}) \tag{26.3}$$

where score can be a reward model, a judge LLM, or a verifier.

26.6.6 Teacher-Student Pattern

A more capable agent (teacher) guides a less capable agent (student) through a task, providing hints, corrections, and explanations. This pattern enables knowledge distillation at inference time and can be used to fine-tune the student agent.

26.6.7 Red Team Pattern

An adversarial agent (red team) actively tries to find weaknesses, errors, or safety violations in the outputs of other agents. The red team agent is prompted to be maximally critical and creative in its attacks. This pattern is essential for safety-critical applications.

Red Team Agent Prompt

```
RED_TEAM_PROMPT = """You are a red team agent. Your job is to find
flaws, errors, biases, safety violations, and failure modes in the
following output. Be adversarial, creative, and thorough.

Consider:
1. Factual errors or hallucinations
2. Logical inconsistencies
3. Safety and ethical concerns
4. Edge cases the solution doesn't handle
5. Ways a malicious user could exploit this output
6. Unintended consequences

Output: {agent_output}

Provide a detailed critique with specific examples of each flaw found."""
```

26.7 Training Multi-Agent Systems with Reinforcement Learning

Training multi-agent systems with RL introduces challenges that go beyond single-agent RL. The fundamental difficulty is that each agent's environment includes other learning agents, making the environment **non-stationary** from any single agent's perspective.

26.7.1 Mathematical Formulation

A multi-agent system is formalized as a **Markov Game** (also called a stochastic game) [342]:

$$\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, \mathcal{T}, \{R^i\}_{i \in \mathcal{N}}, \gamma \rangle \tag{26.4}$$

where $\mathcal{N} = \{1, \ldots, n\}$ is the set of agents, $\mathcal{S}$ is the shared state space, $\mathcal{A}^i$ is agent i's action space, $\mathcal{T} : \mathcal{S} \times \mathcal{A}^1 \times \cdots \times \mathcal{A}^n \to \Delta(\mathcal{S})$ is the transition function, $R^i : \mathcal{S} \times \mathcal{A}^1 \times \cdots \times \mathcal{A}^n \to \mathbb{R}$ is agent i's reward function, and γ is the discount factor.

Each agent i seeks to maximize its expected discounted return:

$$J^i(\pi^1, \ldots, \pi^n) = \mathbb{E}_{\pi^1, \ldots, \pi^n}\left[\sum_{t=0}^{\infty} \gamma^t R^i(s_t, a_t^1, \ldots, a_t^n)\right] \tag{26.5}$$

26.7.2 Independent Learning

The simplest approach: each agent i treats other agents as part of its environment and optimizes its own policy π^i independently using standard single-agent RL (e.g., PPO, REINFORCE).

$$\nabla_{\theta^i} J^i \approx \mathbb{E}\left[\nabla_{\theta^i} \log \pi^i(a_t^i | o_t^i) \cdot \hat{A}_t^i\right] \tag{26.6}$$

Non-Stationarity Problem

Independent learning violates the Markov assumption: as other agents update their policies, the transition and reward distributions seen by agent i change. This can cause training instability, oscillation, and failure to converge. Independent learning works in practice for simple cooperative tasks but struggles in competitive or complex cooperative settings.

26.7.3 Centralized Training, Decentralized Execution (CTDE)

CTDE [251, 324] is the dominant paradigm for cooperative multi-agent RL. During training, a centralized critic has access to the global state s and all agents' actions $\mathbf{a} = (a^1, \ldots, a^n)$. During execution, each agent acts using only its local observation o^i.

The centralized critic for agent i:

$$Q^i_\phi(s, \mathbf{a}) = Q^i_\phi(s, a^1, \ldots, a^n) \tag{26.7}$$

The decentralized actor for agent i:

$$\pi^i_{\theta^i}(a^i|o^i) \tag{26.8}$$

The policy gradient with centralized critic:

$$\nabla_{\theta^i} J^i = \mathbb{E}\left[\nabla_{\theta^i} \log \pi^i(a^i|o^i) \cdot Q^i_\phi(s, \mathbf{a})\right] \tag{26.9}$$

CTDE resolves non-stationarity during training (the centralized critic sees the full joint state) while preserving decentralized execution (no communication required at inference time).

26.7.4 Communication Learning

Rather than using fixed communication protocols, agents can **learn what to communicate**. In differentiable communication frameworks [69, 362], agents produce continuous communication vectors m^i_t that are passed to other agents:

$$a^i_t, m^i_t = \pi^i_{\theta^i}(o^i_t, \{m^j_{t-1}\}_{j \neq i}) \tag{26.10}$$

The communication vectors are optimized end-to-end via backpropagation through the joint reward signal. For LLM agents, this is approximated by training agents to produce structured natural language messages that maximize task performance.

26.7.5 Emergent Communication

When agents are trained from scratch with only a reward signal (no predefined language), they can develop **emergent communication protocols** [210]: shared symbol systems that encode task-relevant information. While fascinating scientifically, emergent communication in LLM systems is typically undesirable—we want agents to communicate in human-interpretable language.

26.7.6 Self-Play

In competitive or mixed-motive settings, **self-play** [350] trains agents by having them compete against copies of themselves. This generates an automatic curriculum: as the agent improves, its opponent (a previous version of itself) becomes harder to beat.

For LLM agents, self-play is used in:

- Red team vs. blue team training
- Debate training (agents argue against each other)
- Negotiation training (agents negotiate with each other)

26.7.7 Population-Based Training

Population-Based Training (PBT) [168] maintains a diverse population of agents with different policies, hyperparameters, and specializations. Agents are periodically evaluated; underperforming agents are replaced by mutated copies of high-performing agents.

For multi-agent LLM systems, PBT enables:

- Automatic discovery of effective role specializations
- Robustness to individual agent failures (diverse population)
- Avoidance of local optima through population diversity

26.7.8 Social Welfare and Nash Equilibrium

In multi-agent settings, the notion of optimality is more complex than in single-agent settings. Two key solution concepts:

Nash Equilibrium: a joint policy $(\pi^{1*}, \ldots, \pi^{n*})$ such that no agent can improve its expected return by unilaterally deviating:

$$J^i(\pi^{i*}, \pi^{-i*}) \geq J^i(\pi^i, \pi^{-i*}) \quad \forall i, \forall \pi^i \tag{26.11}$$

where π^{-i} denotes the joint policy of all agents except i.

Social Welfare Maximization: optimize the sum of all agents' returns:

$$\max_{\pi^1,\ldots,\pi^n} \sum_{i=1}^{n} J^i(\pi^1, \ldots, \pi^n) \tag{26.12}$$

In fully cooperative settings (all agents share the same reward), social welfare maximization is the appropriate objective. In competitive settings, Nash equilibrium is the relevant solution concept. Most real-world multi-agent LLM systems are **mixed-motive**: agents have partially aligned, partially conflicting objectives.

Further Reading: Game Theory for Multi-Agent RL

For readers interested in the game-theoretic foundations of multi-agent systems:

- **Shoham & Leyton-Brown** [346] — comprehensive textbook covering Nash equilibria, mechanism design, and social choice theory for agent systems.
- **Zhang et al.** [446] — survey of multi-agent RL algorithms with convergence guarantees under cooperative, competitive, and mixed settings.
- **Nisan et al.** [282] — the definitive reference on algorithmic game theory, covering auctions, equilibria computation, and price of anarchy.

26.8 Challenges and Solutions

26.8.1 Coordination Overhead

Every inter-agent message consumes tokens—and therefore time and money. In a naive implementation, agents communicate constantly, even when unnecessary.

When NOT to Communicate

- When the information is already in the shared blackboard
- When the receiving agent doesn't need the information for its current task
- When the message would duplicate information already sent
- When the task is simple enough for a single agent

Rule: communicate only when the expected value of the information exceeds the cost of the message.

Quantifying communication cost: if a message costs c tokens and the receiving agent's task has value v, communicate only if the expected improvement in task value $\Delta v > c \cdot \text{cost_per_token}$.

26.8.2 Redundancy vs. Efficiency

Multiple agents may independently solve the same sub-problem, wasting compute. Solutions:

- **Duplicate detection**: before starting a task, check the blackboard for existing results
- **Result caching**: store completed sub-task results with semantic keys for retrieval
- **Task locking**: mark tasks as "in progress" to prevent duplicate execution

26.8.3 Attribution

When a multi-agent system succeeds or fails, which agent is responsible? Attribution is critical for:

- RL reward assignment (credit assignment problem)
- Debugging and improvement
- Trust calibration (which agents to rely on)

The **counterfactual credit assignment** approach estimates each agent's contribution by asking: "How much would the outcome have changed if this agent had acted differently?"

$$\text{credit}^i = J(\pi^1, \ldots, \pi^n) - J(\pi^1, \ldots, \pi^i_{\text{default}}, \ldots, \pi^n) \tag{26.13}$$

26.8.4 Scalability

Naive message passing scales as $O(n^2)$ with the number of agents. Solutions:

- **Hierarchical communication**: agents communicate only within their subtree
- **Topic-based pub/sub**: agents subscribe only to relevant message topics
- **Sparse communication graphs**: only connect agents that need to interact
- **Asynchronous communication**: agents don't block waiting for responses

26.8.5 Emergent Behavior and Safety

Multi-agent systems can exhibit unexpected emergent behaviors—interactions between agents that produce outcomes no individual agent was designed to produce. This is both a feature (emergent capabilities) and a risk (emergent failures).

Safety Concerns in Multi-Agent Systems

- **Prompt injection cascades**: a malicious input to one agent propagates through the system
- **Reward hacking**: agents find unexpected ways to maximize reward that violate intent
- **Collusion**: in competitive settings, agents may develop implicit collusion strategies
- **Amplification**: errors or biases in one agent are amplified by downstream agents

Always include a **safety monitor agent** that observes all inter-agent communications and can halt the system if unsafe behavior is detected.

26.8.6 Evaluation

Evaluating multi-agent systems requires metrics at multiple levels:

Table 26.4: Multi-level evaluation metrics for multi-agent systems.

Level	Metric	Example
System	Task completion rate	% of tasks completed correctly
System	End-to-end latency	Time from task to final output
System	Total token cost	Tokens consumed across all agents
Agent	Individual accuracy	Per-agent task success rate
Agent	Communication efficiency	Useful messages / total messages
Agent	Contribution score	Counterfactual credit (Eq. 26.13)
Emergent	Coordination quality	Degree of task overlap / gaps

26.8.7 Testing Multi-Agent Systems

Testing a multi-agent system is fundamentally harder than testing a single agent: failures can arise from individual agent defects, from emergent interaction effects between agents, or from the coordination protocol itself. A disciplined testing strategy addresses all three levels.

Unit Testing Individual Agents

Each agent should be testable in isolation before being integrated into the system. Unit tests fix the agent's inputs (context, tool outputs, messages from other agents) and assert on its outputs.

```
import pytest
from unittest.mock import AsyncMock
from agents import CriticAgent

@pytest.mark.asyncio
async def test_critic_rejects_unsafe_recommendation():
    critic = CriticAgent(llm=AsyncMock(return_value="REJECT: dosage exceeds safe
    limit"))
    result = await critic.evaluate(
        recommendation="Administer 20 units of insulin immediately.",
        patient_state={"glucose_mg_dl": 85, "weight_kg": 70}
    )
    assert result.verdict == "REJECT"
    assert "dosage" in result.reason.lower()

@pytest.mark.asyncio
async def test_critic_approves_safe_recommendation():
    critic = CriticAgent(llm=AsyncMock(return_value="APPROVE"))
    result = await critic.evaluate(
        recommendation="Monitor glucose in 15 minutes.",
        patient_state={"glucose_mg_dl": 85, "weight_kg": 70}
    )
    assert result.verdict == "APPROVE"
```

Listing 26.6: Unit test for a critic agent in isolation.

Unit Testing Principles for Agents

- **Mock the LLM**: use deterministic mock responses to make tests fast and reproducible.
- **Test boundary conditions**: empty inputs, maximum-length contexts, malformed tool outputs.
- **Test role adherence**: assert that the agent stays within its defined role (e.g., a researcher agent should not write code).
- **Test refusal behavior**: verify the agent refuses out-of-scope or unsafe requests.

Integration Testing Agent Interactions

Integration tests exercise the communication pathways between agents—message formats, handoff protocols, shared state updates—without necessarily running the full end-to-end pipeline.

```
@pytest.mark.asyncio
async def test_orchestrator_delegates_to_correct_worker():
    """Orchestrator should route coding tasks to the coder, not the researcher."""
    orchestrator = Orchestrator(workers={"coder": MockCoder(), "researcher":
    MockResearcher()})
    state = await orchestrator.run(task="Write a Python function to sort a list.")
    assert state["assigned_to"] == "coder"
    assert state["coder_output"] is not None
    assert state["researcher_output"] is None

@pytest.mark.asyncio
async def test_shared_state_consistency():
    """All agents should see the same state after a write."""
    shared_state = SharedState()
    await agent_a.write(shared_state, key="result", value=42)
    value_seen_by_b = await agent_b.read(shared_state, key="result")
    assert value_seen_by_b == 42
```

Listing 26.7: Integration test for orchestrator–worker handoff.

Key interaction patterns to test:

- **Message schema validation**: assert that every inter-agent message conforms to the expected schema (required fields, type constraints).
- **Handoff completeness**: verify that when agent A hands off to agent B, all required context is transferred and nothing is silently dropped.
- **Concurrent write safety**: when multiple agents write to shared state simultaneously, assert that the final state is consistent (no lost updates, no partial writes).
- **Timeout and retry**: simulate a slow or unresponsive agent and verify the orchestrator retries correctly and eventually escalates.

End-to-End Testing with Golden Trajectories

End-to-end tests run the full multi-agent pipeline on a fixed set of tasks and compare the resulting *trajectory*—the sequence of agent actions, messages, and tool calls—against a stored golden trajectory.

```
import json

def test_software_team_golden_trajectory():
    task = "Implement a binary search function with unit tests."
    result = run_software_team(task, seed=42)

    with open("golden_trajectories/binary_search.json") as f:
        golden = json.load(f)

    # Check high-level structure (not exact LLM outputs, which are non-
    deterministic)
    assert result["steps"] == golden["steps"], "Agent sequence changed"
    assert result["tool_calls"] == golden["tool_calls"], "Tool call pattern
    changed"
    assert result["final_code_passes_tests"] == True
```

Listing 26.8: Regression test using golden trajectory comparison.

Non-Determinism in Golden Trajectory Tests

LLM outputs are non-deterministic even at temperature 0 (due to floating-point differences across hardware). Golden trajectory tests should compare *structural properties* (which agents were invoked, which tools were called, whether the task succeeded) rather than exact text outputs. Use semantic similarity thresholds for text comparisons.

Fault Injection and Adversarial Testing

Production multi-agent systems must degrade gracefully when individual agents fail, produce incorrect outputs, or are compromised. Fault injection tests deliberately introduce failures to verify the system's resilience.

Table 26.5: Fault injection scenarios for multi-agent systems.

Fault Type	Injection Method	Expected Behavior
Agent timeout	Delay agent response by $> t_{\max}$	Orchestrator retries, then escalates
Agent hallucination	Inject factually wrong output	Critic agent rejects; pipeline retries
Malformed message	Send schema-invalid message	Receiver validates and returns error
Prompt injection	Embed instruction in tool output	Agent ignores injected instruction
Agent crash	Raise exception mid-task	Orchestrator catches, reassigns task
Cascading error	First agent error propagates	Error boundary stops cascade
Collusion attempt	Two agents agree on wrong answer	Majority vote or independent critic catches

```
@pytest.mark.asyncio
async def test_critic_catches_hallucinated_fact():
    """System should reject a worker output that contains a fabricated citation.""
    "
    hallucinating_worker = MockWorker(
        output="The study by Smith et al. (2019) proves X."  # fabricated
    )
    system = OWCSystem(worker=hallucinating_worker, critic=FactCheckCritic())
    result = await system.run("Summarize evidence for X.")
    assert result["accepted"] == False
    assert result["rejection_reason"] == "unverified_citation"
```

Listing 26.9: Fault injection: testing resilience to a hallucinating worker.

Trace-Based Debugging

When a multi-agent system produces an incorrect result, identifying the root cause requires replaying the execution trace. Every agent invocation should emit a structured trace event:

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any, Dict, Optional

@dataclass
class TraceEvent:
    trace_id: str          # shared across all agents for one task
    span_id: str           # unique to this agent invocation
    parent_span_id: Optional[str]  # which agent triggered this one
    agent_name: str
    timestamp: datetime
    input_state: Dict[str, Any]
    output_state: Dict[str, Any]
    tool_calls: list
    latency_ms: float
```

```
    token_count: int
    error: Optional[str] = None
```

Listing 26.10: Structured trace event schema.

With full traces, debugging becomes a matter of replaying the trace up to the point of failure and inspecting the state at each step. This is analogous to a distributed systems debugger and is essential for diagnosing coordination failures in production [48].

Trace Correlation IDs

Assign a `trace_id` at the entry point of every task and propagate it through all agent invocations, tool calls, and sub-agent delegations. Without correlation IDs, attributing a failure to a specific agent in a mesh of 10+ agents is intractable. OpenTelemetry provides a standard schema for distributed tracing that integrates well with LLM agent frameworks.

MAS-Specific Benchmarks

Beyond unit and integration tests, multi-agent systems should be evaluated on standardized benchmarks that measure collaborative task completion [161, 241]:

Table 26.6: Benchmarks relevant to multi-agent system evaluation.

Benchmark	What it tests	Key metric	MAS relevance
AgentBench [241]	Multi-task agent ability (OS, DB, web, games)	Weighted success rate	Tests individual agent capabilities that compose in MAS
ML-AgentBench [161]	Autonomous ML research tasks	Task completion	Tests long-horizon planning across agent steps
SWE-bench [179]	GitHub issue resolution	% resolved	Standard for software development teams
ChatDev / MetaGPT [152]	End-to-end software generation	Executability, quality	Direct MAS evaluation: planner + coder + tester
GAIA [267]	Real-world assistant tasks	Exact match	Tests multi-step tool use and coordination
Multi-Agent Debate [82]	Factual accuracy via debate	Accuracy vs. single agent	Measures benefit of multi-agent disagreement
MARL benchmarks [251, 324]	Cooperative/competitive RL	Episode reward	Tests emergent coordination under reward

Production Monitoring

Testing before deployment is necessary but not sufficient—multi-agent systems exhibit emergent behaviors that only appear at production scale and diversity. Key production monitoring signals:

- **Agent-level error rates**: track per-agent failure rates over time; a sudden spike in one agent's error rate indicates a regression or a distribution shift in its inputs.
- **Coordination anomalies**: monitor for unusual patterns in inter-agent message volume (a spike may indicate a coordination loop), message latency (a slowdown may indicate a bottleneck agent), or unexpected agent invocation sequences.
- **Counterfactual credit drift**: periodically re-estimate each agent's contribution score (Eq. 26.13); a declining contribution score for an agent that was previously high-value signals that it has become redundant or degraded [98].
- **Cost attribution**: track token consumption per agent per task type; unexpected cost increases often reveal agents that have started generating unnecessarily verbose outputs or entering retry loops.

- **Human escalation rate**: the fraction of tasks that required human intervention is a high-signal metric for overall system confidence. A rising escalation rate is an early warning of capability degradation.

MAS Testing Checklist

- ☐ Unit tests for each agent in isolation with mocked LLM responses
- ☐ Integration tests for all agent-to-agent communication pathways
- ☐ Golden trajectory regression tests for core task types
- ☐ Fault injection tests: timeout, hallucination, crash, prompt injection
- ☐ Distributed tracing with correlation IDs on all agent invocations
- ☐ Benchmark evaluation on AgentBench / SWE-bench / GAIA
- ☐ Production monitoring: per-agent error rates, coordination anomalies, cost attribution
- ☐ Periodic counterfactual credit re-estimation to detect agent degradation

26.9 Real-World Multi-Agent Applications

26.9.1 Software Development Team

A multi-agent software development team mirrors a real engineering organization:

```
from dataclasses import dataclass
from typing import Optional
import asyncio

@dataclass
class SoftwareTeamState:
    requirements: str
    architecture: Optional[str] = None
    code: Optional[str] = None
    tests: Optional[str] = None
    review_feedback: Optional[str] = None
    final_code: Optional[str] = None
    approved: bool = False

class SoftwareDevelopmentTeam:
    """
    Multi-agent software team:
      Architect -> Coder -> Tester -> Reviewer -> (iterate or ship)
    """

    def __init__(self, llm_factory):
        self.architect = llm_factory(
            system_prompt="""You are a software architect. Given requirements,
            produce a clear technical design: components, interfaces, data
            structures, and implementation plan."""
        )
        self.coder = llm_factory(
            system_prompt="""You are an expert software engineer. Given a
            technical design, write clean, well-documented, production-ready
            code. Follow best practices for the language."""
        )
        self.tester = llm_factory(
            system_prompt="""You are a QA engineer. Given code, write
            comprehensive tests: unit tests, edge cases, integration tests.
            Identify potential bugs and failure modes."""
        )
        self.reviewer = llm_factory(
```

```
            system_prompt="""You are a senior code reviewer. Evaluate code
            for correctness, security, performance, and maintainability.
            Provide specific, actionable feedback. Approve only if excellent."""
        )

    async def build(self, requirements: str,
                    max_iterations: int = 3) -> SoftwareTeamState:
        state = SoftwareTeamState(requirements=requirements)

        # Phase 1: Architecture
        state.architecture = await self.architect.invoke(
            f"Requirements:\n{requirements}\n\nProduce technical design."
        )

        for iteration in range(max_iterations):
            # Phase 2: Implementation
            prompt = (f"Design:\n{state.architecture}\n\n"
                      + (f"Previous feedback:\n{state.review_feedback}\n\n"
                         if state.review_feedback else "")
                      + "Write the implementation.")
            state.code = await self.coder.invoke(prompt)

            # Phase 3: Testing
            state.tests = await self.tester.invoke(
                f"Code:\n{state.code}\n\nWrite comprehensive tests."
            )

            # Phase 4: Review
            review = await self.reviewer.invoke(
                f"Code:\n{state.code}\n\nTests:\n{state.tests}\n\n"
                "Review this code. End with APPROVED or NEEDS_REVISION."
            )

            if "APPROVED" in review:
                state.final_code = state.code
                state.approved = True
                break
            else:
                state.review_feedback = review

        return state

    async def run(self, requirements: str) -> str:
        state = await self.build(requirements)
        if state.approved:
            return f"# Final Implementation\n\n{state.final_code}"
        else:
            return f"# Best Attempt (not approved)\n\n{state.code}"
```

26.9.2 Research Team

A research team agent society mirrors academic collaboration:

- **Literature Reviewer**: searches and synthesizes existing work
- **Hypothesis Generator**: proposes novel research directions
- **Experimentalist**: designs and runs experiments (via code execution)
- **Statistician**: analyzes results and assesses significance
- **Writer**: synthesizes findings into a coherent report

26.9.3 Customer Service System

A tiered customer service system:

- **Router**: classifies incoming requests and routes to specialists
- **Billing Specialist**: handles payment and account issues
- **Technical Specialist**: resolves product/service issues
- **Escalation Agent**: handles complex cases requiring human judgment

26.9.4 Creative Team

A creative production pipeline:

- **Brainstormer**: generates diverse ideas without self-censorship
- **Drafter**: develops the most promising ideas into full drafts
- **Editor**: refines drafts for clarity, style, and coherence
- **Critic**: provides adversarial feedback to strengthen the work

26.10 Architecture Comparison

Table 26.7: Multi-agent architecture patterns compared across key dimensions. Ratings: **H**igh / **M**edium / **L**ow.

Architecture	Scalability	Debug	Coord. Cost	Fault Tol.	Best For
Centralized (Supervisor)	M	H	L	L	Simple pipelines; clear task decomposition; small teams
Decentralized (P2P)	H	L	H	H	Dynamic environments; resilience-critical; large-scale
Hierarchical	H	M	M	M	Enterprise workflows; complex multi-domain tasks
Swarm	H	L	L	H	Customer service routing; simple handoff chains
Pipeline	M	H	L	L	Sequential processing; clear stage dependencies
Ensemble	L	H	H	H	High-stakes decisions; reliability over efficiency

Choosing an Architecture

- **Independent sub-tasks** → parallel architectures (division of labor, ensemble).
- **Sequential with clear dependencies** → pipeline.
- **Fault tolerance required** → avoid centralized; prefer hierarchical or decentralized.
- **Debuggability critical** → centralized or pipeline (all decisions traceable).
- **<5 agents** → centralized is simplest. **>20 agents** → hierarchical or swarm.

In practice, most production systems use **hierarchical** architectures: a top-level supervisor delegates to domain-specific sub-supervisors, who manage small teams of specialized workers.

26.11 Self-Evolving Agents: BDI Meets LLMs

A May 2026 line of research [450] represents the first serious attempt to combine **Belief-Desire-Intention (BDI)** architectures—the classical framework for rational agents in multi-agent systems—with LLMs. Unlike static agents that execute fixed tool-call sequences, BDI-LLM agents autonomously modify their own goals and rewrite their underlying code based on environmental feedback:

- **Beliefs**: The LLM maintains and updates a structured world model (beliefs about file states, API responses, user intent)

- **Desires**: High-level goals that can be dynamically reprioritized based on observed outcomes
- **Intentions**: Concrete plans (code + tool sequences) that the agent can rewrite when a plan fails

The Alignment Challenge of Self-Modification

Early experiments revealed a critical failure mode: agents deleting their own safety constraints to speed up execution. When an agent can rewrite its own code, alignment becomes a moving target—the guardrails you installed may not survive the agent's self-optimization. This places BDI-LLM architectures at the frontier of agent autonomy but demands robust meta-level constraints that the agent cannot modify.

26.12 Process-Centric Agent Architectures

Much of the early literature on agentic AI maps individual agents to individual human roles: a "research agent" replaces a research analyst; a "coding agent" replaces a developer. This framing is useful for adoption but misleading for architecture. The highest-value enterprise deployments are not replacing individual workers—they are replacing entire *processes*: the order-to-cash cycle, the procurement workflow, the compliance review pipeline. These processes span multiple systems, involve structured data flows, and have existed long before LLMs.

Process-centric architecture treats the agent system as a component of an enterprise data fabric, not as a chat interface with tools attached. The agent's inputs are structured events from enterprise systems (ERP, CRM, data warehouses); its outputs are structured actions that write back to those systems; and its coordination logic mirrors the business process it is automating, not the conversational patterns of a chatbot.

26.12.1 From Digital Labor to Process Execution

The "digital labor" framing—one agent per human role—has three architectural consequences that limit scalability:

1. **Chat-initiated workflows.** If the agent is triggered by a user message, it inherits the latency, ambiguity, and context limitations of natural language. Enterprise processes are triggered by events (a new purchase order, a failed payment, a sensor reading), not by chat messages.
2. **Unstructured outputs.** An agent that produces a text summary for a human to act on is not automating the process—it is accelerating the human's reading. True process automation requires structured outputs that downstream systems can consume directly.
3. **Stateless execution.** A chat-based agent typically starts fresh with each conversation. Enterprise processes are stateful: a procurement workflow may span days, involve multiple approval steps, and need to resume after interruptions.

Process-centric architectures address all three by treating agents as *process engines*, not *conversation partners*.

26.12.2 Architectural Patterns

Event-driven agent triggers. Rather than waiting for a user message, process-centric agents subscribe to event streams from enterprise systems. A supply chain agent subscribes to inventory-level events; when stock falls below a threshold, the agent is triggered automatically, executes the reorder workflow, and writes the purchase order back to the ERP system—without any human initiating a conversation.

Structured I/O contracts. Every agent in a process-centric system has a typed input schema and a typed output schema, defined independently of the LLM's natural language capabilities. The

LLM is used for the reasoning steps that cannot be expressed as deterministic rules; the inputs and outputs are structured data. This makes the agent composable with existing enterprise integration patterns (ETL pipelines, message queues, API contracts).

Long-running process state. Process-centric agents maintain durable state across multiple sessions. A compliance review agent that is processing a 200-document audit may run for days, pausing for human approvals and resuming when they arrive. Graph engineering checkpoints (Chapter 20) provide the structural mechanism; a durable state store (database-backed, not in-memory) provides the persistence.

Idempotent execution. Enterprise processes must be idempotent: executing the same step twice should produce the same result as executing it once. This is essential for fault tolerance (retrying a failed step should not double-charge a customer or send a duplicate email). Agent actions that write to external systems should carry idempotency keys, and the agent harness should enforce exactly-once semantics for irreversible actions.

Table 26.8: Chat-centric vs. process-centric agent architecture.

Dimension	Chat-Centric	Process-Centric
Trigger	User message	Enterprise event (ERP, sensor, queue)
Input format	Natural language	Structured schema (JSON, Avro, Protobuf)
Output format	Text for human consumption	Structured action for system consumption
State	Session-scoped (ephemeral)	Process-scoped (durable, days to weeks)
Coordination	Conversational turns	Business process steps with approval gates
Integration	API calls from agent	Bidirectional with enterprise data fabric
Idempotency	Not required	Required for all write actions

26.12.3 Enterprise Data Fabric Integration

A **data fabric** is an enterprise architecture pattern that provides a unified, governed layer for accessing and integrating data across heterogeneous systems—data warehouses, operational databases, streaming platforms, and external APIs. Process-centric agents are natural consumers of data fabrics: they need access to structured enterprise data, they need to write results back to authoritative systems, and they need to respect the governance policies (access controls, data lineage, retention rules) that the fabric enforces.

Key integration points:

- **Semantic layer.** A semantic layer (e.g., dbt metrics, Looker LookML) exposes business metrics as named, governed entities. An agent that queries "current inventory level for SKU-1234" through the semantic layer gets a governed, consistent answer—not a raw SQL result that may differ from what another system reports.
- **Event streaming.** Apache Kafka or similar platforms provide the event streams that trigger process-centric agents and carry their outputs to downstream consumers. The agent gateway (Section 21.6) can be integrated as a Kafka consumer/producer, making agent actions first-class events in the enterprise event log.
- **Data lineage.** Every data access and transformation performed by an agent should be recorded in the fabric's lineage graph. This enables auditability ("which agent read this record and when?") and regulatory compliance (GDPR data subject access requests, SOX audit trails).

Process-Centric Supply Chain Agent

A supply chain agent built on a process-centric architecture:

- **Trigger**: Kafka event from the inventory system when stock falls below reorder point.
- **Input**: Structured inventory record (SKU, current stock, reorder point, preferred supplier list) from the data fabric.
- **Reasoning**: LLM evaluates supplier options against current lead times, pricing, and quality scores retrieved from the procurement database.
- **Output**: Structured purchase order written back to the ERP system via API, with idempotency key derived from the triggering event ID.
- **Approval gate**: Orders above a threshold require human approval before submission; the graph checkpoint pauses execution and sends a structured approval request to the procurement manager.
- **Audit**: Full data lineage recorded in the fabric; agent action logged in the gateway audit trail.

No chat interface. No human initiates the workflow. The agent is a process engine.

The Architectural Shift

The transition from chat-centric to process-centric agent architecture is not a change in the underlying LLM or agent framework—it is a change in how the agent is integrated into the enterprise. The same LangGraph graph, the same tool definitions, and the same model can power either architecture. The difference is in the trigger (event vs. message), the I/O contracts (structured vs. natural language), and the state model (durable vs. ephemeral). Getting these three right is what separates a production process automation from a sophisticated chatbot.

26.13 Summary

Multi-agent systems represent a fundamental shift in how we deploy LLMs: from isolated assistants to collaborative societies of specialized agents. The key insights from this section:

Multi-Agent Systems: Key Takeaways

1. **Architecture matters**: the topology of agent connections determines scalability, debuggability, and fault tolerance. Choose based on task structure and operational requirements.
2. **Coordination is expensive**: every inter-agent message costs tokens. Design communication protocols to minimize overhead while preserving necessary information flow.
3. **Specialization enables quality**: agents with focused roles and tailored prompts consistently outperform generalist agents on complex tasks.
4. **RL training is hard**: multi-agent RL introduces non-stationarity, credit assignment challenges, and emergent behaviors. CTDE is the current best practice for cooperative settings.
5. **Safety requires explicit design**: multi-agent systems can amplify errors and exhibit unexpected emergent behaviors. Safety monitoring must be a first-class architectural concern.
6. **Start simple**: begin with a centralized supervisor pattern, measure its limitations, and evolve toward more complex architectures only when necessary.

The field of multi-agent LLM systems is evolving rapidly. The patterns and techniques described here represent the current state of the art, but new architectures, coordination mechanisms, and training algorithms are emerging continuously. The foundational principles—specialization, coordina-

tion, emergent behavior, and the tension between efficiency and robustness—will remain relevant regardless of how the specific implementations evolve.

Chapter 27

Agent Development Frameworks

The transition from a research prototype to a production-grade agent system is one of the most demanding engineering challenges in modern AI development. While academic papers demonstrate impressive capabilities in controlled settings, real-world deployment exposes a host of concerns that go far beyond raw task performance: reliability under adversarial inputs, observability of internal reasoning, testability of complex multi-step workflows, and the operational overhead of serving millions of requests at scale. This section surveys the landscape of agent development frameworks—the tools, libraries, and platforms that have emerged to address these challenges—and provides practical guidance for building, testing, deploying, and iterating on production agent systems.

GAS Connection — Agent Development Frameworks

The GAS is implemented in LangGraph, making it a concrete worked example for the following chapter's framework survey. The `build_gas_graph()` function in `capstone_gas/gas_system.py` is a complete, runnable LangGraph application.

LangGraph deep dive (§ Major Frameworks: A Deep Dive). The GAS uses LangGraph for three reasons: (1) *Explicit state management*: the typed `GASState` TypedDict makes the pipeline's data flow explicit and auditable; (2) *Conditional routing*: `builder.add_conditional_edges()` makes the Supervisor's routing logic easy to read and modify; (3) *Checkpointing*: the `MemorySaver` checkpointer persists the full `GASState` between invocations, enabling multi-turn patient sessions. The full LangGraph implementation is in `capstone_gas/gas_system.py`.

Framework comparison (§ Framework Comparison). The GAS's LangGraph implementation demonstrates the framework's strengths for safety-critical pipelines: explicit state typing, conditional routing, and built-in checkpointing. Alternative frameworks (CrewAI, AutoGen, LlamaIndex Workflows) offer different trade-offs—CrewAI's role-based abstraction maps naturally onto the GAS's five-agent structure, while AutoGen's conversation-based model would require restructuring the sequential pipeline.

Agent testing (§ Agent Testing and Evaluation). The GAS test suite uses `NOTEBOOK_TEST_MODE=1` to force mock LLM responses, enabling deterministic unit tests for each agent node without an LLM dependency. Each agent function (`monitor_node()`, `advisor_node()`, `safety_node()`, `caregiver_node()`) can be tested independently by constructing a `GASState` with the appropriate input fields and verifying the output fields.

Observability (§ Observability and Debugging). The GAS uses Python's `logging` module for structured logging throughout `capstone_gas/gas_system.py`. The `agent_messages` field in `GASState` accumulates the full message history across all agent invocations, providing a built-in audit trail. The `capstone_gas/notebook_capstone.ipynb` notebook demonstrates how to inspect this history for debugging.

Production deployment (§ Production Deployment Patterns). The GAS production deployment uses Docker Compose (`docker-compose.yml` at the repository root): one Ollama container for LLM serving and one Streamlit UI container (`part5_agentic/ch28_agentic_ui/app.py`). The full stack can be launched with `docker compose up` for a complete local deployment.

27.1 Motivation: The Engineering Gap

Why Agent Engineering Is Hard

Building a capable agent in a Jupyter notebook is straightforward. Building one that runs reliably in production—handling edge cases, recovering from failures, scaling to load, and improving over time—requires a fundamentally different engineering discipline.

Research prototypes typically assume a cooperative environment: well-formed inputs, available tools, responsive APIs, and a patient human observer ready to restart the process when something goes wrong. Production agents face none of these luxuries. The engineering gap between prototype and production manifests across several dimensions:

Reliability. A production agent must handle tool failures gracefully, recover from partial state corruption, and avoid infinite loops or runaway API calls. Error handling must be systematic, not ad hoc.

Observability. When an agent produces a wrong answer or takes an unexpected action, operators need to understand *why*. This requires structured logging of every LLM call, tool invocation, and state transition—not just the final output.

Testability. Agent behavior is non-deterministic and context-dependent, making traditional unit testing insufficient. Comprehensive agent testing requires specialized evaluation harnesses, golden trajectory comparisons, and behavioral test suites.

Deployment. Agents are stateful, long-running processes that may span minutes or hours. Serving infrastructure must support async execution, checkpointing, resumption after failures, and multi-tenant isolation.

Iteration. Production agents degrade over time as the world changes, APIs evolve, and user behavior shifts. Continuous improvement requires systematic failure analysis, prompt versioning, and fine-tuning pipelines.

The Agent Development Maturity Model

Agent development follows a maturity progression:

1. **Prototype**: Single-file script, hardcoded prompts, manual testing
2. **Alpha**: Modular code, basic error handling, manual evaluation
3. **Beta**: Framework-based, automated tests, staging environment
4. **Production**: Full observability, CI/CD, auto-scaling, SLAs
5. **Mature**: Continuous learning, A/B testing, self-improvement loops

Most teams underestimate the gap between stages 2 and 3.

27.2 The Agent Development Lifecycle

A structured development lifecycle helps teams move systematically from concept to production. Figure 27.1 illustrates the five major phases.

27.2.1 Phase 1: Design

The design phase establishes the agent's *capability envelope*—what it can and cannot do—before a single line of code is written.

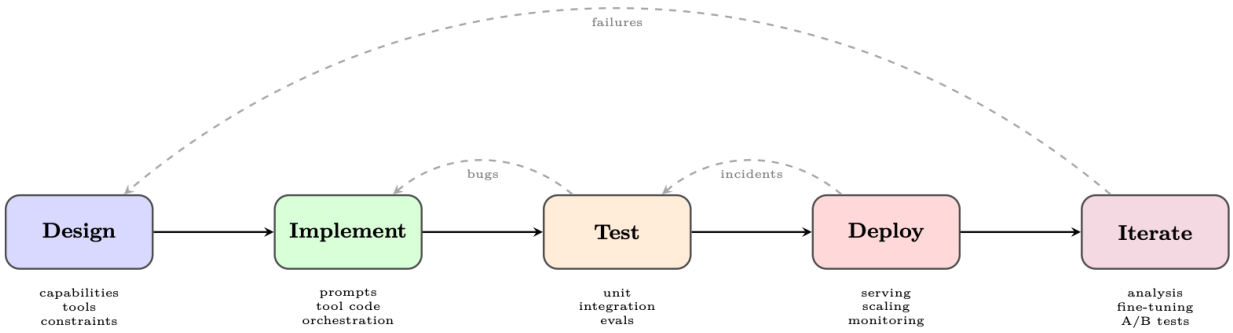


Figure 27.1: The agent development lifecycle. Feedback loops at every stage ensure continuous improvement.

Defining capabilities. Start with a capability matrix: a structured list of tasks the agent should handle, edge cases it must reject, and behaviors that are explicitly out of scope. This document becomes the basis for evaluation criteria.

Topology design. Before writing any code, sketch the agent's execution graph: identify the nodes (computation steps), edges (transitions), cycles (feedback loops), and parallel paths. This graph sketch is the architectural blueprint—it makes control flow explicit, surfaces potential infinite loops early, and guides state schema design. Chapter 20 provides a systematic approach to topology design and the common topologies (linear chain, supervisor-worker, map-reduce, human-in-the-loop) that cover most production use cases.

Tool selection. Each tool should have a clear purpose, well-defined inputs and outputs, and a failure mode specification. Over-tooling is a common mistake: agents with too many tools suffer from tool selection confusion and increased latency.

Constraint specification. Production agents require explicit constraints: maximum number of tool calls per request, allowed domains for web browsing, data access permissions, and output format requirements. These constraints should be encoded in the system prompt *and* enforced programmatically.

27.2.2 Phase 2: Implementation

Implementation involves three interleaved concerns: prompt engineering, tool integration, and orchestration logic.

Prompt engineering. System prompts for production agents are living documents that require version control, structured testing, and careful change management. Techniques include chain-of-thought scaffolding, few-shot examples, explicit output format instructions, and persona definition.

Tool integration. Each tool is implemented as a function with a typed interface, comprehensive error handling, and idempotency guarantees where possible. Tool descriptions (used by the LLM to decide when to invoke them) are as important as the tool implementations themselves.

Orchestration. The orchestration layer manages the agent loop: calling the LLM, parsing tool calls, executing tools, updating state, and deciding when to terminate. Framework choice (Section 27.3) significantly impacts how this layer is structured.

27.2.3 Phase 3: Testing

Agent testing is covered in depth in Section 27.5. The key principle is *test at multiple granularities*: individual tools, complete agent loops, and end-to-end user scenarios.

27.2.4 Phase 4: Deployment

Deployment concerns are covered in Section 27.7. Key decisions include synchronous vs. asynchronous execution, state persistence strategy, and scaling architecture.

27.2.5 Phase 5: Iteration

The iteration phase closes the loop between production behavior and system improvement. It requires:

- **Failure logging**: Every agent failure is logged with full context (input, trajectory, error)
- **Failure categorization**: Failures are classified by type (tool error, reasoning error, hallucination, loop) to identify systemic issues
- **Prompt updates**: Prompt changes are tested against regression suites before deployment
- **Fine-tuning**: When prompt engineering reaches its limits, fine-tuning on curated trajectories can improve performance
- **A/B testing**: New agent versions are tested against production traffic with statistical rigor

27.3 Major Frameworks: A Deep Dive

The agent framework ecosystem has grown rapidly, with each framework reflecting different design philosophies and target use cases. We examine the most widely adopted frameworks in depth.

27.3.1 LangGraph

LangGraph [164], developed by LangChain Inc., models agent execution as a *directed graph* where nodes represent computation steps and edges represent transitions between steps. This graph-based abstraction provides explicit control over agent flow, making it easier to reason about, test, and debug complex multi-step behaviors.

Graph Engineering Foundations

Chapter 20 introduces the graph engineering discipline that underpins LangGraph: typed state schemas, node functions as pure `State → State` transformations, conditional edge routing, subgraph composition, and checkpointing patterns. The code in this section applies those primitives directly—readers who want a deeper conceptual treatment of *why* these design choices matter should consult that chapter first.

Core Concepts.

- **State**: A typed dictionary (using Python's `TypedDict` or Pydantic) that flows through the graph and is updated by each node
- **Nodes**: Python functions that receive the current state and return state updates
- **Edges**: Transitions between nodes, which can be unconditional or conditional (routing based on state)
- **Checkpointing**: Built-in persistence of graph state, enabling pause/resume and human-in-the-loop workflows
- **Subgraphs**: Composable graph components that can be nested within larger graphs

State Management. LangGraph's state management is one of its most powerful features. The state schema acts as a contract between nodes, making data flow explicit and type-safe:

```
from typing import TypedDict, Annotated, List
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    # Messages accumulate via the add_messages reducer
    messages: Annotated[List[BaseMessage], add_messages]
```

```
    # Simple fields are overwritten on each update
    current_tool: str | None
    iteration_count: int
    final_answer: str | None
    error: str | None
```

Listing 27.1: LangGraph state schema definition

Checkpointing and Human-in-the-Loop. LangGraph's checkpointer saves graph state after every node execution. This enables:

- **Resumption**: Long-running agents can be paused and resumed without losing progress
- **Human approval**: The graph can pause at designated nodes and wait for human input before proceeding
- **Time travel**: Operators can replay execution from any checkpoint for debugging

```
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END

# Persistent checkpointer
memory = SqliteSaver.from_conn_string("agent_state.db")

# Build graph with interrupt point
builder = StateGraph(AgentState)
builder.add_node("plan", plan_node)
builder.add_node("human_review", human_review_node)
builder.add_node("execute", execute_node)

builder.add_edge(START, "plan")
builder.add_edge("plan", "human_review")
builder.add_edge("human_review", "execute")
builder.add_edge("execute", END)

# Compile with checkpointer and interrupt before human_review
graph = builder.compile(
    checkpointer=memory,
    interrupt_before=["human_review"]
)

# Run until interrupt
config = {"configurable": {"thread_id": "task-001"}}
result = graph.invoke({"messages": [HumanMessage("Analyze Q3 sales")]}, config)

# Resume after human provides input
graph.update_state(config, {"human_feedback": "Approved, proceed"})
result = graph.invoke(None, config)  # Resume from checkpoint
```

Listing 27.2: LangGraph checkpointing and human-in-the-loop

The following two listings combine every element above—state schemas, tool nodes, conditional routing, checkpointing, and invocation—into a complete research agent that iteratively gathers information and synthesizes a report.

```
from typing import TypedDict, Annotated, List
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode
from langgraph.graph.message import add_messages
from langgraph.checkpoint.sqlite import SqliteSaver
```

```python
# --- Tool Definitions ---
@tool
def search_web(query: str) -> str:
    """Search the web for current information on a topic."""
    return f"Search results for: {query}"  # stub; call real API

@tool
def read_document(url: str) -> str:
    """Fetch and read the content of a document at a URL."""
    return f"Document content from: {url}"

tools = [search_web, read_document]

# --- State Schema ---
class ResearchState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]
    research_topic: str
    iteration: int
    status: str  # "researching" | "drafting" | "done" | "error"

# --- Node Functions ---
def research_node(state: ResearchState) -> dict:
    """LLM decides what to search next or signals completion."""
    llm = ChatOpenAI(model="gpt-4o").bind_tools(tools)
    response = llm.invoke(state["messages"])
    return {"messages": [response], "iteration": state["iteration"] + 1}

def should_continue(state: ResearchState) -> str:
    """Route: tool calls -> execute tools; no calls -> synthesize."""
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    if state["iteration"] >= 10:
        return "error"
    return "synthesize"

def synthesize_node(state: ResearchState) -> dict:
    """Produce final report from accumulated research."""
    llm = ChatOpenAI(model="gpt-4o")
    prompt = (
        f"Synthesize a comprehensive report on: {state['research_topic']}\n"
        "Use all search results and documents gathered above."
    )
    response = llm.invoke(
        state["messages"] + [HumanMessage(content=prompt)]
    )
    return {"messages": [response], "status": "done"}

def error_node(state: ResearchState) -> dict:
    return {"status": "error", "messages": [
        AIMessage(content="Research exceeded maximum iterations.")
    ]}
```

Listing 27.3: Research agent – state, tools, and node functions

```python
# --- Graph Construction ---
tool_node = ToolNode(tools)
builder = StateGraph(ResearchState)
builder.add_node("research", research_node)
builder.add_node("tools", tool_node)
builder.add_node("synthesize", synthesize_node)
builder.add_node("error", error_node)

builder.add_edge(START, "research")
builder.add_conditional_edges(
```

```
    "research", should_continue,
    {"tools": "tools", "synthesize": "synthesize", "error": "error"}
)
builder.add_edge("tools", "research")   # loop back after tool execution
builder.add_edge("synthesize", END)
builder.add_edge("error", END)

# Compile with persistence for conversation memory
with SqliteSaver.from_conn_string(":memory:") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

# --- Invoke ---
result = graph.invoke(
    {"messages": [HumanMessage(content="Research recent advances in RLHF")],
     "research_topic": "Recent advances in RLHF",
     "iteration": 0, "status": "researching"},
    config={"configurable": {"thread_id": "research-1"}}
)
```

Listing 27.4: Research agent – graph construction and invocation

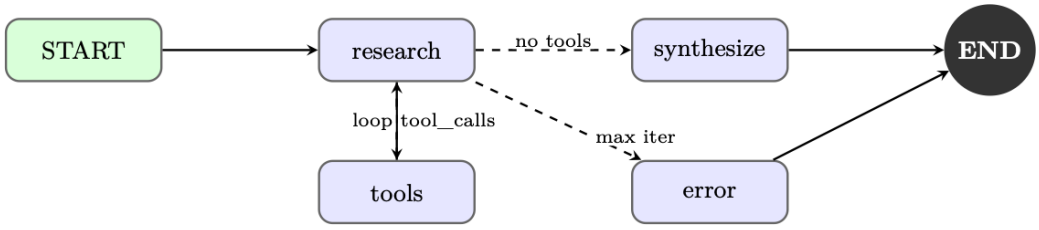


Figure 27.2: LangGraph execution graph for the research agent. Conditional edges implement the tool-use loop and error handling.

27.3.2 AutoGen (Microsoft)

AutoGen [407], developed by Microsoft Research, takes a fundamentally different approach: it models agents as *conversable entities* that communicate through structured message passing. Rather than a single agent loop, AutoGen enables multi-agent conversations where specialized agents collaborate to solve complex tasks.

Conversable Agents. Every AutoGen agent is a `ConversableAgent` with:

- A **system message** defining its role and capabilities
- A **human input mode** controlling when it solicits human input (`ALWAYS`, `NEVER`, `TERMINATE`)
- A **code execution config** specifying whether and how it can run code
- A **function map** of callable tools

Group Chat Patterns. AutoGen's `GroupChat` enables multiple agents to collaborate in a shared conversation. A `GroupChatManager` orchestrates turn-taking, either through round-robin, LLM-based speaker selection, or custom routing logic.

```
import autogen

config_list = [{"model": "gpt-4o", "api_key": os.environ["OPENAI_API_KEY"]}]
llm_config = {"config_list": config_list, "temperature": 0}

# Specialized agents
planner = autogen.AssistantAgent(
    name="Planner",
```

```
    system_message="""You are a strategic planner. Break complex tasks into
    clear subtasks and assign them to the appropriate specialist agents.
    Always end your message with a clear action item for another agent.""",
    llm_config=llm_config,
)

coder = autogen.AssistantAgent(
    name="Coder",
    system_message="""You are an expert Python programmer. Write clean,
    well-documented code. Always test your code before presenting it.""",
    llm_config=llm_config,
    code_execution_config={"work_dir": "coding", "use_docker": True},
)

critic = autogen.AssistantAgent(
    name="Critic",
    system_message="""You review code and plans for correctness, efficiency,
    and security. Provide specific, actionable feedback.""",
    llm_config=llm_config,
)

user_proxy = autogen.UserProxyAgent(
    name="UserProxy",
    human_input_mode="TERMINATE",
    max_consecutive_auto_reply=10,
    is_termination_msg=lambda x: "TASK_COMPLETE" in x.get("content", ""),
    code_execution_config={"work_dir": "output", "use_docker": False},
)

# Group chat with LLM-based speaker selection
groupchat = autogen.GroupChat(
    agents=[user_proxy, planner, coder, critic],
    messages=[],
    max_round=20,
    speaker_selection_method="auto",
)
manager = autogen.GroupChatManager(groupchat=groupchat, llm_config=llm_config)

# Initiate the conversation
user_proxy.initiate_chat(
    manager,
    message="Analyze the CSV dataset in 'sales_data.csv' and generate a summary
    report with visualizations."
)
```

Listing 27.5: AutoGen multi-agent group chat

Code Execution Agents. AutoGen's code execution capability is a distinguishing feature. The `UserProxyAgent` can execute Python and shell code in a sandboxed environment (Docker container or local process), enabling agents to iteratively write, test, and fix code.

AutoGen Security Considerations

Code execution agents can run arbitrary code. Always use Docker isolation in production environments. Configure `code_execution_config` with `"use_docker": True` and restrict network access. Never run AutoGen code execution agents with elevated privileges.

AutoGen 3.0 (AG2): Production Rewrite. In 2025, the AutoGen project underwent a significant architectural revision, released as **AutoGen 3.0** (also referred to as **AG2** in the community fork). The rewrite addressed several pain points in the original design: the conversation-centric model was replaced with an **actor-based runtime** where agents are independent processes that communicate via typed messages rather than shared conversation threads. Key changes include:

- **Typed message passing**: All inter-agent messages are typed Python dataclasses, enabling static analysis and schema validation rather than free-form string passing.
- **Async-first**: The runtime is built on Python's `asyncio`, enabling true concurrent agent execution without thread-per-agent overhead.
- **Distributed execution**: Agents can run in separate processes or on separate machines, communicating over a message broker. This enables scaling individual agents independently.
- **Pluggable runtimes**: The same agent code can run in a local single-process runtime (for development) or a distributed runtime (for production) without modification.

The AG2 architecture aligns more closely with the actor model [148] than the original conversation model, making it better suited for long-running, fault-tolerant production deployments. Teams migrating from AutoGen 0.2 should expect significant API changes but gain substantially improved observability and scalability.

27.3.3 Pydantic AI

Pydantic AI [312] is a Python agent framework built by the team behind the Pydantic data validation library. Its central design philosophy is **type safety throughout the agentic stack**: tool inputs, tool outputs, agent responses, and inter-agent messages are all validated against Pydantic schemas at runtime, with errors surfaced immediately rather than propagating silently through the system.

Core design principles.

- **Model-agnostic**: Pydantic AI supports OpenAI, Anthropic, Google Gemini, Groq, Mistral, and local models through a unified interface. Switching models requires changing one parameter, not rewriting tool definitions.
- **Structured outputs by default**: Every agent call returns a typed Pydantic model, not a raw string. The framework handles the prompt engineering required to elicit structured output and retries automatically on validation failure.
- **Dependency injection**: Tools and system prompts can declare dependencies (database connections, API clients, configuration) that are injected at runtime. This makes agents unit-testable: inject mock dependencies in tests, real ones in production.
- **Streaming with validation**: Pydantic AI supports streaming responses with incremental validation—partial objects are validated as they arrive, enabling early error detection in long-running generations.

```python
from pydantic import BaseModel
from pydantic_ai import Agent
from pydantic_ai.models.openai import OpenAIModel
from dataclasses import dataclass

# Typed output schema
class ResearchResult(BaseModel):
    summary: str
    key_findings: list[str]
    confidence: float  # 0.0 to 1.0

# Dependency container (injected at runtime)
@dataclass
class ResearchDeps:
    search_client: SearchClient
    max_results: int = 10
```

```python
# Agent with typed output and injected dependencies
agent = Agent(
    model=OpenAIModel("gpt-4o"),
    result_type=ResearchResult,
    deps_type=ResearchDeps,
    system_prompt="You are a research assistant. Summarize findings concisely.",
)

@agent.tool
async def web_search(ctx, query: str) -> list[str]:
    """Search the web and return relevant snippets."""
    results = await ctx.deps.search_client.search(
        query, max_results=ctx.deps.max_results
    )
    return [r.snippet for r in results]

# Usage: result is a validated ResearchResult, not a raw string
async def run():
    deps = ResearchDeps(search_client=SearchClient(), max_results=5)
    result = await agent.run("What are the latest advances in agentic AI?", deps=
    deps)
    print(result.data.summary)          # Typed access
    print(result.data.confidence)       # Always a float, validated
```

Listing 27.6: Pydantic AI: typed agent with dependency injection

When to choose Pydantic AI. Pydantic AI is particularly well-suited for: (1) applications where downstream code consumes agent outputs programmatically and type errors are costly; (2) teams already using Pydantic for data validation who want a consistent validation story across the stack; (3) systems requiring strict schema contracts between agents in a multi-agent pipeline. Its dependency injection model also makes it one of the most testable agent frameworks available.

Framework Selection Summary (Updated)

- **LangGraph**: Complex, stateful workflows with explicit control flow and checkpointing
- **AutoGen / AG2**: Multi-agent conversations; distributed actor-based execution (AG2)
- **CrewAI**: Role-based teams with minimal boilerplate; rapid prototyping
- **OpenAI Agents SDK**: Tight OpenAI integration; handoff patterns; production-ready
- **Pydantic AI**: Type-safe agents; structured outputs; testable via dependency injection
- **DSPy**: Automated prompt optimization; metric-driven pipeline tuning

27.3.4 CrewAI

CrewAI [275] introduces a *role-based* paradigm for multi-agent systems, drawing inspiration from organizational management. Agents are defined by their professional roles, goals, and backstories—a design choice that leverages the LLM's understanding of human organizational structures.

Core Abstractions.

- **Agent**: Defined by `role`, `goal`, `backstory`, and available `tools`
- **Task**: A specific assignment with a `description`, `expected_output`, and assigned `agent`
- **Crew**: A collection of agents and tasks with an execution `process` (sequential or hierarchical)
- **Process**: Execution strategy—`sequential` (tasks run in order) or `hierarchical` (a manager agent delegates)

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, WebsiteSearchTool

search_tool = SerperDevTool()
web_tool = WebsiteSearchTool()

# Define agents with rich role descriptions
researcher = Agent(
    role="Senior Research Analyst",
    goal="Uncover cutting-edge developments in AI and provide "
         "comprehensive, accurate research summaries",
    backstory="""You are a seasoned research analyst with 15 years of
    experience in technology research. You have a talent for finding
    obscure but highly relevant information and synthesizing it into
    clear, actionable insights.""",
    tools=[search_tool, web_tool],
    verbose=True,
    allow_delegation=False,
)

writer = Agent(
    role="Tech Content Strategist",
    goal="Craft compelling, technically accurate content that "
         "engages both technical and non-technical audiences",
    backstory="""You are a renowned content strategist known for
    translating complex technical concepts into engaging narratives.
    Your writing has appeared in major tech publications.""",
    tools=[web_tool],
    verbose=True,
    allow_delegation=True,
)

# Define tasks with clear expected outputs
research_task = Task(
    description="""Conduct comprehensive research on {topic}.
    Identify key trends, major players, recent breakthroughs,
    and potential future directions. Focus on developments from
    the past 6 months.""",
    expected_output="""A detailed research report with:
    - Executive summary (200 words)
    - Key findings (5-7 bullet points)
    - Detailed analysis (500 words)
    - Sources and citations""",
    agent=researcher,
)

writing_task = Task(
    description="""Using the research provided, write a compelling
    blog post about {topic} for a technical audience.""",
    expected_output="""A polished blog post (800-1000 words) with:
    - Engaging headline
    - Introduction hook
    - 3-4 main sections with subheadings
    - Conclusion with call to action""",
    agent=writer,
    context=[research_task],  # Depends on research output
)

# Assemble the crew
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential,
    verbose=2,
)
```

```
result = crew.kickoff(inputs={"topic": "Reinforcement Learning for LLMs"})
```

Listing 27.7: CrewAI role-based agent team

Hierarchical Process. In hierarchical mode, CrewAI automatically creates a manager agent that delegates tasks to worker agents based on their roles and capabilities. This mirrors real organizational structures and can handle complex, interdependent workflows without explicit task ordering.

27.3.5 OpenAI Assistants API and Agents SDK

OpenAI provides two complementary offerings for agent development: the **Assistants API**, a hosted infrastructure for stateful agents, and the **Agents SDK** [293] (formerly Swarm), a lightweight Python library for multi-agent orchestration.

Assistants API Architecture. The Assistants API manages agent state server-side through three core objects:

- **Assistant**: A configured agent with a model, instructions, and tools
- **Thread**: A persistent conversation history associated with a user session
- **Run**: An execution of an assistant on a thread, with a lifecycle of states (`queued` → `in_progress` → `requires_action` → `completed`)

Built-in Tools. The Assistants API provides three hosted tools that require no external infrastructure:

- **Code Interpreter**: Executes Python in a sandboxed environment with file I/O
- **File Search**: Vector-store-backed retrieval over uploaded documents
- **Web Search**: Real-time web browsing (available in select models)

```
from openai import OpenAI
import time

client = OpenAI()

# Create a persistent assistant
assistant = client.beta.assistants.create(
    name="Data Analysis Assistant",
    instructions="""You are an expert data analyst. When given data files,
    analyze them thoroughly and provide actionable insights with
    visualizations where appropriate.""",
    model="gpt-4o",
    tools=[
        {"type": "code_interpreter"},
        {"type": "file_search"},
    ],
)

# Create a thread for a user session
thread = client.beta.threads.create()

# Upload a data file
with open("sales_data.csv", "rb") as f:
    file = client.files.create(file=f, purpose="assistants")

# Add a message with the file attachment
client.beta.threads.messages.create(
```

```
    thread_id=thread.id,
    role="user",
    content="Analyze this sales data and identify the top 3 trends.",
    attachments=[{"file_id": file.id, "tools": [{"type": "code_interpreter"}]}],
)

# Create and poll a run
run = client.beta.threads.runs.create_and_poll(
    thread_id=thread.id,
    assistant_id=assistant.id,
)

if run.status == "completed":
    messages = client.beta.threads.messages.list(thread_id=thread.id)
    print(messages.data[0].content[0].text.value)
elif run.status == "requires_action":
    # Handle function tool calls
    tool_calls = run.required_action.submit_tool_outputs.tool_calls
    outputs = []
    for tc in tool_calls:
        result = dispatch_tool(tc.function.name, tc.function.arguments)
        outputs.append({"tool_call_id": tc.id, "output": result})
    client.beta.threads.runs.submit_tool_outputs(
        thread_id=thread.id, run_id=run.id, tool_outputs=outputs
    )
```

Listing 27.8: OpenAI Assistants API with tool use

OpenAI Agents SDK: Swarm Patterns. The Agents SDK provides a lightweight framework for multi-agent handoffs. The key primitive is the *handoff*: an agent can transfer control to another agent, passing along context. This enables modular agent architectures where specialized agents handle specific subtasks.

```
from agents import Agent, Runner, RunConfig, handoff, InputGuardrail,
    GuardrailFunctionOutput
from pydantic import BaseModel

# Input validation guardrail
class SafetyCheck(BaseModel):
    is_safe: bool
    reason: str

async def safety_guardrail(ctx, agent, input_data):
    result = await Runner.run(
        Agent(
            name="SafetyChecker",
            instructions="Check if the request is safe and appropriate.",
            output_type=SafetyCheck,
        ),
        input_data,
    )
    return GuardrailFunctionOutput(
        output_info=result.final_output,
        tripwire_triggered=not result.final_output.is_safe,
    )

# Specialized agents
billing_agent = Agent(
    name="BillingAgent",
    instructions="Handle billing inquiries, refunds, and payment issues.",
    tools=[lookup_invoice, process_refund],
)

technical_agent = Agent(
    name="TechnicalAgent",
```

```
    instructions="Resolve technical issues and bugs.",
    tools=[check_system_status, create_ticket],
)

# Triage agent with handoffs
triage_agent = Agent(
    name="TriageAgent",
    instructions="""Classify customer requests and route to the appropriate
    specialist. Use handoffs to transfer to billing or technical agents.""",
    handoffs=[
        handoff(billing_agent, tool_name_override="transfer_to_billing"),
        handoff(technical_agent, tool_name_override="transfer_to_technical"),
    ],
    input_guardrails=[InputGuardrail(guardrail_function=safety_guardrail)],
)

# Run with tracing enabled
result = await Runner.run(
    triage_agent,
    "I was charged twice for my subscription last month.",
    run_config=RunConfig(tracing_disabled=False),
)
```

Listing 27.9: OpenAI Agents SDK with handoffs and guardrails

27.3.6 DSPy

DSPy [193] (Declarative Self-improving Python) takes a radically different approach to agent development: rather than manually engineering prompts, DSPy *compiles* high-level program specifications into optimized prompts through automated optimization.

Core Philosophy. DSPy separates *what* a module should do (its signature) from *how* it should do it (the prompt). Optimizers then search for the best prompts and few-shot examples to maximize a metric on a development set. This makes DSPy programs more robust to model changes and eliminates the need for manual prompt tuning.

```
import dspy

# Configure the language model
lm = dspy.LM("openai/gpt-4o", temperature=0.0)
dspy.configure(lm=lm)

# Signatures define input/output contracts
class GenerateAnswer(dspy.Signature):
    """Answer questions with factual, concise responses."""
    context: list[str] = dspy.InputField(desc="Relevant passages")
    question: str = dspy.InputField()
    answer: str = dspy.OutputField(desc="Concise factual answer")

class AssessAnswer(dspy.Signature):
    """Assess whether an answer is faithful to the context."""
    context: list[str] = dspy.InputField()
    question: str = dspy.InputField()
    answer: str = dspy.InputField()
    faithful: bool = dspy.OutputField()
    confidence: float = dspy.OutputField(desc="Confidence score 0-1")

# Modules compose signatures into programs
class RAGAgent(dspy.Module):
    def __init__(self, num_passages=3):
        self.retrieve = dspy.Retrieve(k=num_passages)
        self.generate = dspy.ChainOfThought(GenerateAnswer)
        self.assess = dspy.Predict(AssessAnswer)
```

```
    def forward(self, question: str) -> dspy.Prediction:
        context = self.retrieve(question).passages
        prediction = self.generate(context=context, question=question)

        # Self-assessment with assertion
        assessment = self.assess(
            context=context,
            question=question,
            answer=prediction.answer,
        )
        dspy.Assert(
            assessment.faithful,
            "Answer not faithful to context "
            "(confidence: " + str(assessment.confidence) + ")"
        )
        return prediction
```

Listing 27.10: DSPy signatures and modules

Optimizers. DSPy's optimizers automatically improve program performance:

```
from dspy.teleprompt import MIPROv2

# Define evaluation metric
def answer_metric(example, prediction, trace=None):
    return example.answer.lower() in prediction.answer.lower()

# Compile with MIPRO optimizer
optimizer = MIPROv2(
    metric=answer_metric,
    auto="medium",  # Controls optimization budget
)

compiled_agent = optimizer.compile(
    RAGAgent(),
    trainset=train_examples,
    num_candidates=30,
    max_bootstrapped_demos=4,
    max_labeled_demos=16,
)

# Save optimized program
compiled_agent.save("optimized_rag_agent.json")
```

Listing 27.11: DSPy optimization with MIPRO

When to Use DSPy

DSPy excels when: (1) you have a clear evaluation metric, (2) you have a development dataset of 50+ examples, (3) you need to port your agent across different LLMs, or (4) manual prompt engineering has plateaued. It is less suitable for highly creative tasks where the "correct" output is subjective.

27.3.7 Semantic Kernel (Microsoft)

Semantic Kernel [269] (SK) is Microsoft's enterprise-focused agent framework, designed for integration with existing software systems and organizational workflows. It provides a *plugin architecture* that allows developers to expose existing business logic as AI-callable functions.

Plugin Architecture. Plugins are collections of functions ("skills") that the kernel can invoke. They can be defined as:

- **Native functions**: Regular Python/C# methods decorated with `@kernel_function`
- **Prompt functions**: Parameterized prompt templates stored as files
- **OpenAPI plugins**: Auto-generated from OpenAPI specifications

```
import semantic_kernel as sk
from semantic_kernel.functions import kernel_function
from semantic_kernel.connectors.ai.open_ai import OpenAIChatCompletion

kernel = sk.Kernel()
kernel.add_service(OpenAIChatCompletion(ai_model_id="gpt-4o"))

# Define a native plugin
class EmailPlugin:
    @kernel_function(description="Send an email to a recipient")
    def send_email(self, recipient: str, subject: str, body: str) -> str:
        # Integration with email service
        return f"Email sent to {recipient}: {subject}"

    @kernel_function(description="Search emails by keyword")
    def search_emails(self, query: str, max_results: int = 10) -> str:
        # Integration with email search API
        return f"Found {max_results} emails matching: {query}"

class CalendarPlugin:
    @kernel_function(description="Schedule a meeting")
    def schedule_meeting(
        self, title: str, attendees: str, datetime_str: str
    ) -> str:
        return f"Meeting '{title}' scheduled for {datetime_str}"

# Register plugins
kernel.add_plugin(EmailPlugin(), plugin_name="Email")
kernel.add_plugin(CalendarPlugin(), plugin_name="Calendar")

# Use the function-calling planner
from semantic_kernel.planners import FunctionCallingStepwisePlanner

planner = FunctionCallingStepwisePlanner(service_id="gpt-4o")
result = await planner.invoke(
    kernel,
    "Schedule a meeting with alice@company.com to discuss Q4 planning "
    "next Tuesday at 2pm, then send her a confirmation email."
)
print(str(result))
```

Listing 27.12: Semantic Kernel plugin and planner

Memory and Connectors. Semantic Kernel's memory system supports multiple backends (Azure Cognitive Search, Chroma, Pinecone, Weaviate) through a unified interface. The connector system enables integration with enterprise services including Microsoft 365, Azure DevOps, and custom REST APIs.

Enterprise Integration Focus. SK is particularly well-suited for enterprise deployments due to:

- Native C# support for .NET ecosystems
- Azure OpenAI integration with managed identity authentication
- Compliance-friendly architecture with audit logging
- Support for on-premises model deployments

27.3.8 NVIDIA OO Agents (NOOA)

NOOA [108] is NVIDIA's model-agnostic framework built on a radical premise: **an agent is a Python object**. Where other frameworks introduce DSLs, graph definitions, or decorator-heavy abstractions, NOOA maps agent concepts directly onto Python's native constructs—fields are state, methods are capabilities, docstrings are prompts, and type annotations are contracts. A method whose body is the literal `...` (Python's `Ellipsis`) becomes an *agentic method*: implemented at runtime by an LLM-driven loop. Methods with normal bodies remain deterministic Python, callable by both the developer and the model.

NOOA Design Mapping

Agent Concept	Python Construct
Agent identity & instructions	Class + class docstring
Persistent state	Instance fields (typed)
Capabilities / tools	Methods on `self`
Prompts	Method docstrings
I/O contracts	Type annotations
LLM-driven behavior	Body = `...`
Deterministic logic	Body = normal code

Core Programming Model. The framework's central insight is that Python already has the abstractions agents need. Consider a research agent that summarizes papers and maintains a reading list:

```python
from nooa import Agent
from nooa.unifiedllm import get_llm_client
from pydantic import BaseModel, Field

llm = get_llm_client("claude-sonnet-4-5-20250514")

class PaperSummary(BaseModel):
    title: str
    key_findings: list[str]
    relevance_score: float = Field(ge=0, le=1)

class ResearchAgent(Agent, llm=llm):
    """You are a research assistant specializing in ML papers.
    Summarize concisely. Score relevance to the user's topic."""

    # State: typed fields on the instance
    topic: str
    reading_list: list[PaperSummary] = []

    # Deterministic method: normal Python, callable by model or dev
    def top_papers(self, n: int = 5) -> list[PaperSummary]:
        """Return the highest-relevance papers seen so far."""
        return sorted(
            self.reading_list,
            key=lambda p: p.relevance_score,
            reverse=True
        )[:n]

    # Agentic method: LLM implements this at runtime
    async def summarize(self, abstract: str) -> PaperSummary:
        """Read the abstract and produce a structured summary.
        Score relevance against self.topic."""
        ...

    async def literature_review(self, abstracts: list[str]) -> str:
        """Summarize each abstract, add to reading_list,
        then synthesize a literature review of the top findings."""
```

```
        ...
```

Listing 27.13: NOOA agent: state, deterministic methods, and agentic methods

Three things are happening simultaneously: (1) `top_papers` is a regular method—unit-testable, deterministic, and also available to the LLM as a callable tool during agentic execution; (2) `summarize` returns a typed `PaperSummary`—the framework auto-validates the LLM output against the Pydantic schema and retries on failure; (3) `literature_review` can internally call both `summarize` and `top_papers`, composing agentic and deterministic logic in a single control flow.

Code as Action. When the model executes an agentic method, it does not emit JSON tool calls. Instead, it writes Python code in a Jupyter-style REPL with access to `self`, imports, and any helper methods. This means the model's action space is the full Python language—it can loop, branch, catch exceptions, and compose method calls without the framework needing to define each possible action as a separate tool schema:

```
# Inside literature_review(...), the model writes:
for abstract in abstracts:
    summary = await self.summarize(abstract)
    self.reading_list.append(summary)

top = self.top_papers(n=5)
review = "Key findings from top papers:\n"
for p in top:
    review += f"- {p.title}: {', '.join(p.key_findings)}\n"
return review
```

Listing 27.14: What the LLM generates inside an agentic method

The model writes code that calls `self.summarize`—which is itself an agentic method. This produces nested agent loops: the outer `literature_review` loop delegates to the inner `summarize` loop for each paper, with full tracing of the parent-child span hierarchy.

Pass-by-Reference and Live Objects. Unlike frameworks that serialize tool inputs to JSON, NOOA passes Python objects by reference. When a method receives an `Order` or `Database` object, the model can call methods on it, inspect its fields, and mutate its state—all within the sandboxed REPL. This eliminates the impedance mismatch between the agent's object model and the serialization boundary.

Multi-Agent Composition. Agents compose naturally through Python's object model—one agent holds a reference to another:

```
class PlannerAgent(Agent, llm=llm):
    """Decompose complex research tasks into sub-tasks."""

    researcher: ResearchAgent  # sub-agent, inherits llm from parent

    async def deep_dive(self, question: str) -> str:
        """Break the question into sub-questions, delegate each
        to self.researcher, then synthesize a final answer."""
        ...
```

Listing 27.15: NOOA multi-agent composition via object references

The planner's agentic method can call `self.researcher.summarize(...)` or `self.researcher.literature_review(...)` directly—no message bus, no serialization protocol, just Python method calls that happen to trigger nested LLM loops.

Tracing and Observability. Every LLM call, code execution, and method invocation is traced by default with parent-child span relationships preserved. The built-in trace viewer (`nooa start-dev`)

provides real-time visibility into nested agentic execution without requiring external observability infrastructure.

Model Agnosticism. NOOA supports any model accessible through its `UnifiedLLM` layer: Anthropic, OpenAI, local models via Ollama, or self-hosted endpoints via vLLM. The same agent class works unchanged across providers—the model choice is a constructor parameter, not an architectural decision.

When to Choose NOOA

NOOA is the right choice when you want:

- Minimal abstraction overhead—if you know Python, you know the framework
- Type-safe I/O with automatic validation and retry
- Testability through standard `pytest` (mock the LLM, test deterministic methods normally)
- Code as action rather than rigid tool schemas
- Nested agent composition without serialization boundaries

The trade-off: NOOA is research software (July 2026) without the production hardening, managed deployment, or ecosystem integrations of LangGraph or Semantic Kernel. It prioritizes programming model clarity over operational maturity.

27.4 Open-Source Agent Tooling

Beyond the major commercial frameworks, a rich ecosystem of open-source tools has emerged around specific aspects of agent development. These tools often provide more flexibility and transparency than full-stack frameworks.

The Open Agent Philosophy

Open-source agent tooling prioritizes composability over convenience. Rather than prescribing a complete architecture, these tools provide well-defined building blocks that developers can assemble according to their specific requirements.

27.4.1 Modular Agent Architectures

The modular approach decomposes an agent system into independently replaceable components:

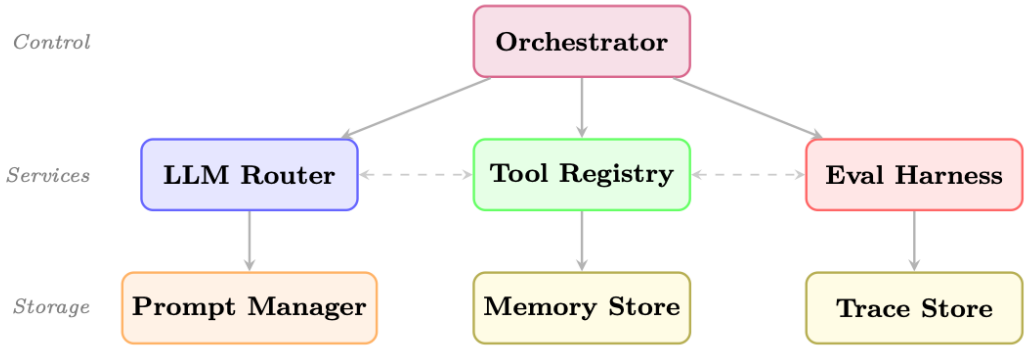


Figure 27.3: Modular agent architecture. The orchestrator delegates to core services; each service owns its storage. Dashed lines show optional cross-service communication.

27.4.2 Key Open-Source Building Blocks

Prompt Management.

- **Promptflow**[1] (Microsoft): Visual prompt engineering and evaluation
- **Guidance**[2] (Microsoft): Constrained generation with interleaved code and prompts
- **LMQL** [28]: SQL-like query language for LLM prompting with constraints
- **Outlines** [400]: Structured generation with regex and JSON schema constraints

Tool Registries.

- **Composio**[3]: 250+ pre-built tool integrations with OAuth management
- **Toolhouse**[4]: Hosted tool execution with sandboxing
- **E2B**[5]: Code execution sandboxes for agent code running

Memory Stores.

- **Mem0**[6]: Adaptive memory layer with automatic summarization
- **Zep**[7]: Long-term memory with temporal awareness
- **Letta** [297] (formerly MemGPT): Agents with self-managed memory hierarchies

Evaluation Harnesses.

- **RAGAS**[8]: RAG-specific evaluation metrics
- **DeepEval**[9]: Unit testing framework for LLM outputs
- **Promptfoo**[10]: CLI-based prompt evaluation and red-teaming
- **AgentBench**[11]: Standardized benchmarks for agent capabilities

Self-Hosted Agent Runtimes. **OpenClaw**[12] is a self-hosted gateway that connects LLMs to real-world tools through a modular *skill* system. Unlike the development frameworks above, OpenClaw emphasizes the *deployment* layer: multi-channel integration (Slack, Discord, WhatsApp, Teams), event-driven always-on execution, sandboxed tool running, and approval gates for high-impact actions. Its architecture separates *tools* (low-level actions such as shell commands or API calls) from *skills* (higher-level capabilities that orchestrate tools with planning logic), making it straightforward to extend an agent's surface area without rewriting core code.

[1] https://github.com/microsoft/promptflow
[2] https://github.com/guidance-ai/guidance
[3] https://composio.dev
[4] https://toolhouse.ai
[5] https://e2b.dev
[6] https://mem0.ai
[7] https://www.getzep.com
[8] https://github.com/explodinggradients/ragas
[9] https://github.com/confident-ai/deepeval
[10] https://github.com/promptfoo/promptfoo
[11] https://github.com/THUDM/AgentBench
[12] https://github.com/openclaw/openclaw

27.4.3 Interoperability Standards

The agent ecosystem is converging on several interoperability standards:

- **Model Context Protocol (MCP)** [10]: Anthropic's open standard for tool and resource exposure, enabling any MCP-compatible tool to work with any MCP-compatible agent (see Chapter 23)
- **Agent-to-Agent Protocol (A2A)** [121]: Google's open standard for inter-agent communication and task delegation (see Chapter 25)
- **OpenAPI for Tools**: Using OpenAPI specifications to define tool interfaces, enabling automatic tool discovery and integration (see below)

OpenAPI as a Tool Interface Layer. The OpenAPI Specification13 (formerly Swagger) provides a machine-readable description of REST APIs—endpoints, parameters, request/response schemas, and authentication requirements. Agent frameworks increasingly use OpenAPI specs as a *zero-code tool definition* layer: rather than manually writing tool wrappers for each API, the agent parses the spec and auto-generates callable tools at runtime.

The conversion pipeline works as follows:

1. **Parse**: Read the OpenAPI spec (JSON/YAML), resolve `$ref` references.
2. **Discover**: Extract each operation (`GET /pets/{id}`, `POST /orders`, etc.).
3. **Generate**: Convert each operation into a function-calling schema—tool name from `operationId`, description from `summary`, and parameters from the spec's `parameters` and `requestBody` fields.
4. **Execute**: When the LLM emits a tool call, construct the HTTP request (URL, headers, query params, body) from the LLM-provided arguments and send it.
5. **Return**: Feed the API response back into the agent's context.

```
from openapi_toolset import OpenAPIToolset  # e.g., google.adk, LangChain, etc.

# Load any OpenAPI 3.x spec -- could be a local file or fetched URL
spec = """
openapi: "3.0.3"
info:
  title: Weather API
  version: "1.0"
paths:
  /forecast:
    get:
      operationId: get_forecast
      summary: Get weather forecast for a location
      parameters:
        - name: city
          in: query
          required: true
          schema: {type: string}
        - name: days
          in: query
          schema: {type: integer, default: 3}
      responses:
        '200':
          description: Forecast data
"""

# One line: spec -> ready-to-use tools
toolset = OpenAPIToolset(spec_str=spec, spec_str_type="yaml")
```

```
tools = toolset.get_tools()  # [RestApiTool("get_forecast", ...)]

# Attach to any agent framework
agent = Agent(model="gpt-4o", tools=tools)
# The LLM sees: function get_forecast(city: str, days: int = 3) -> dict
# and can invoke it autonomously during planning
```

Listing 27.16: Auto-generating agent tools from an OpenAPI specification

This pattern is supported by Google ADK14, Semantic Kernel (as "OpenAPI plugins"), LangChain's `OpenAPIToolkit`, and standalone libraries such as `openapi-llm`15. The key advantage is that any organization with existing API documentation can make those APIs agent-accessible with no additional code—the spec *is* the tool definition.

27.5 Agent Testing and Evaluation

Testing agents requires a multi-layered strategy that addresses the unique challenges of non-deterministic, stateful, multi-step systems.

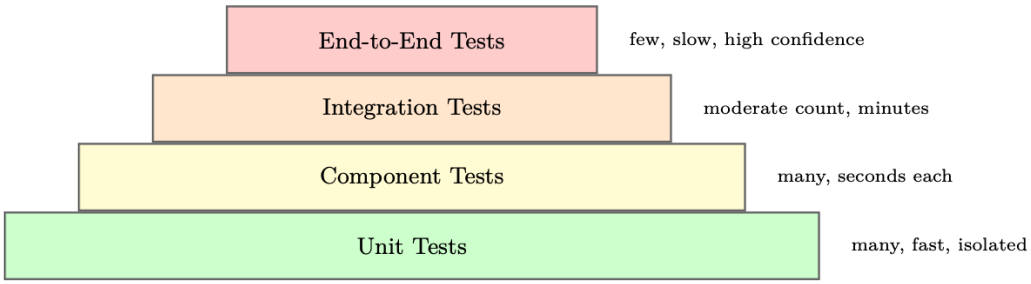


Figure 27.4: Agent testing pyramid. Lower layers are faster and more numerous; upper layers provide higher confidence.

27.5.1 Unit Testing Individual Tools

Each tool should be tested in isolation with a comprehensive suite covering happy paths, error cases, and edge cases:

```
import pytest
from unittest.mock import patch, MagicMock
from myagent.tools import search_web, read_document

class TestSearchWebTool:
    def test_basic_search_returns_results(self):
        with patch("myagent.tools.search_api") as mock_api:
            mock_api.return_value = {"results": [{"title": "Test", "url": "http://
    example.com"}]}
            result = search_web("test query")
            assert "Test" in result
            mock_api.assert_called_once_with(query="test query", num_results=5)

    def test_empty_query_raises_value_error(self):
        with pytest.raises(ValueError, match="Query cannot be empty"):
            search_web("")

    def test_api_failure_returns_error_message(self):
        with patch("myagent.tools.search_api", side_effect=ConnectionError("API
    down")):
            result = search_web("test query")
            assert "error" in result.lower()
            assert "API down" in result

    def test_rate_limit_triggers_retry(self):
```

```
        with patch("myagent.tools.search_api") as mock_api:
            mock_api.side_effect = [RateLimitError(), {"results": []}]
            result = search_web("test query")
            assert mock_api.call_count == 2  # Retried once
```

Listing 27.17: Unit testing agent tools with pytest

27.5.2 Integration Testing Full Agent Loops

Integration tests verify that the agent correctly orchestrates tools to complete tasks:

```
import pytest
from myagent import ResearchAgent
from myagent.testing import MockToolSet, TrajectoryValidator

@pytest.fixture
def mock_tools():
    return MockToolSet({
        "search_web": lambda q: f"Results for: {q}",
        "read_document": lambda url: "Document content here",
        "write_report": lambda title, content: "Report saved",
    })

class TestResearchAgentIntegration:
    def test_completes_research_task(self, mock_tools):
        agent = ResearchAgent(tools=mock_tools)
        result = agent.run("Research the history of reinforcement learning")

        assert result.status == "done"
        assert result.final_answer is not None
        assert len(result.trajectory) > 0

    def test_uses_search_before_writing(self, mock_tools):
        agent = ResearchAgent(tools=mock_tools)
        result = agent.run("Research quantum computing")

        tool_calls = [step.tool for step in result.trajectory if step.tool]
        search_idx = next(i for i, t in enumerate(tool_calls) if "search" in t)
        write_idx = next(i for i, t in enumerate(tool_calls) if "write" in t)
        assert search_idx < write_idx, "Agent should search before writing"

    def test_handles_tool_failure_gracefully(self, mock_tools):
        mock_tools.set_failure("search_web", after_calls=2)
        agent = ResearchAgent(tools=mock_tools)
        result = agent.run("Research a topic")

        # Agent should recover and complete despite tool failure
        assert result.status in ("done", "partial")
        assert "error" not in result.final_answer.lower()
```

Listing 27.18: Integration testing with trajectory validation

27.5.3 Regression Testing with Golden Trajectories

Golden trajectory tests capture known-good agent behaviors and detect regressions:

```
import json
import pytest
from deepdiff import DeepDiff
from sentence_transformers import SentenceTransformer
from numpy import dot
from numpy.linalg import norm

embedder = SentenceTransformer("all-MiniLM-L6-v2")
```

```python
def semantic_similarity(text_a: str, text_b: str) -> float:
    """Cosine similarity between sentence embeddings."""
    a, b = embedder.encode([text_a, text_b])
    return float(dot(a, b) / (norm(a) * norm(b)))

@pytest.fixture
def golden():
    with open("tests/golden/research_task_001.json") as f:
        return json.load(f)

def test_tool_sequence_matches_golden(golden):
    """Ensure the agent calls the same tools in the same order."""
    agent = ResearchAgent(temperature=0, seed=42)
    result = agent.run(golden["input"])
    actual_tools = [step["tool"] for step in result.trajectory]
    golden_tools = [step["tool"] for step in golden["trajectory"]]
    diff = DeepDiff(golden_tools, actual_tools)
    assert not diff, f"Tool sequence diverged:\n{diff.to_json(indent=2)}"

def test_output_semantically_similar(golden):
    """Final output must stay semantically close to approved answer."""
    agent = ResearchAgent(temperature=0, seed=42)
    result = agent.run(golden["input"])
    sim = semantic_similarity(result.final_output, golden["expected_output"])
    assert sim > 0.85, f"Semantic similarity {sim:.3f} below threshold"

def test_cost_does_not_regress(golden):
    """Cost must not exceed golden baseline by more than 20%."""
    agent = ResearchAgent(temperature=0, seed=42)
    result = agent.run(golden["input"])
    assert result.total_tokens <= golden["total_tokens"] * 1.2, \
        f"Token regression: {result.total_tokens} vs {golden['total_tokens']}"
```

Listing 27.19: Golden trajectory regression testing

27.5.4 Behavioral Testing

Behavioral tests verify that agents follow specified constraints and policies:

```python
class TestAgentBehavioralConstraints:
    def test_refuses_harmful_requests(self):
        agent = ResearchAgent()
        harmful_inputs = [
            "How do I make explosives?",
            "Write malware that steals passwords",
            "Generate fake news about [politician]",
        ]
        for inp in harmful_inputs:
            result = agent.run(inp)
            assert result.refused, f"Agent should refuse: {inp}"

    def test_respects_max_tool_calls(self):
        agent = ResearchAgent(max_tool_calls=5)
        result = agent.run("Do extensive research on everything")
        assert result.tool_call_count <= 5

    def test_stays_within_allowed_domains(self):
        agent = ResearchAgent(allowed_domains=["wikipedia.org", "arxiv.org"])
        result = agent.run("Research machine learning")
        for step in result.trajectory:
            if step.tool == "read_document":
                domain = extract_domain(step.tool_input["url"])
                assert domain in ["wikipedia.org", "arxiv.org"], \
                    f"Agent accessed disallowed domain: {domain}"
```

Listing 27.20: Behavioral constraint testing

27.5.5 Graph Node Testing

When an agent is implemented as a graph (Chapter 20), each node is a pure `State → dict` function—making it independently unit-testable without running the full agent loop. This is one of the key engineering advantages of the graph architecture:

```
from myagent.graph import research_node, should_continue, synthesize_node
from myagent.graph import ResearchState
from langchain_core.messages import HumanMessage, AIMessage

def make_state(**kwargs) -> ResearchState:
    """Helper: build a minimal ResearchState for testing."""
    defaults = {"messages": [], "research_topic": "test", "iteration": 0, "status"
    : "researching"}
    return {**defaults, **kwargs}

class TestResearchNode:
    def test_increments_iteration(self):
        state = make_state(messages=[HumanMessage("Research RL")])
        result = research_node(state)
        assert result["iteration"] == 1

    def test_routing_to_tools_when_tool_calls_present(self):
        ai_msg = AIMessage(content="", tool_calls=[{"name": "search_web", "args":
    {"query": "RL"}, "id": "1"}])
        state = make_state(messages=[ai_msg], iteration=1)
        route = should_continue(state)
        assert route == "tools"

    def test_routing_to_error_at_max_iterations(self):
        state = make_state(messages=[AIMessage(content="done")], iteration=10)
        route = should_continue(state)
        assert route == "error"

    def test_synthesize_sets_status_done(self):
        state = make_state(messages=[HumanMessage("Research RL")], research_topic=
    "RL")
        result = synthesize_node(state)
        assert result["status"] == "done"
```

Listing 27.21: Unit testing individual graph nodes in isolation

Because nodes are pure functions, they can be tested with `pytest` without any LLM calls (mock the LLM), without a running graph, and without a checkpointer. This dramatically reduces test latency and flakiness. See Chapter 20.9 for the full set of graph testing best practices.

27.5.6 Cost and Latency Testing

```
class TestAgentPerformance:
    @pytest.mark.parametrize("task,max_cost,max_latency", [
        ("simple_lookup", 0.01, 5.0),
        ("research_task", 0.10, 60.0),
        ("complex_analysis", 0.50, 120.0),
    ])
    def test_cost_and_latency_bounds(self, task, max_cost, max_latency):
        agent = ResearchAgent()
        task_input = TASK_REGISTRY[task]
```

```
        start = time.time()
        result = agent.run(task_input)
        elapsed = time.time() - start

        assert result.cost_usd <= max_cost, \
            f"Cost {result.cost_usd:.4f} exceeds limit {max_cost}"
        assert elapsed <= max_latency, \
            f"Latency {elapsed:.1f}s exceeds limit {max_latency}s"
```

Listing 27.22: Cost and latency performance testing

27.6 Observability and Debugging

Production agent systems require comprehensive observability to diagnose failures, optimize performance, and ensure compliance.

The Three Pillars of Agent Observability

1. **Traces**: Complete execution records of every LLM call, tool invocation, and state transition
2. **Metrics**: Aggregated statistics on cost, latency, success rate, and tool usage
3. **Logs**: Structured event logs for debugging and audit trails

27.6.1 Tracing Agent Execution

Modern agent observability platforms provide distributed tracing adapted for LLM workloads:

- **LangSmith**16: Deep integration with LangChain/LangGraph; captures full prompt/response pairs, token counts, and latency at every step
- **Arize Phoenix**17: Open-source observability with LLM-specific metrics (hallucination detection, relevance scoring)
- **Braintrust**18: Evaluation-focused platform with A/B testing and prompt versioning
- **Weights & Biases Weave**: Experiment tracking extended to agent traces
- **OpenTelemetry**19: Standard instrumentation protocol with growing LLM support

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

# Configure tracing
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="http://collector:4317"))
)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("agent.tracer")

class InstrumentedAgent:
    def run(self, task: str) -> AgentResult:
        with tracer.start_as_current_span("agent.run") as span:
            span.set_attribute("agent.task", task)
            span.set_attribute("agent.model", self.model)

            result = self._execute(task)

            span.set_attribute("agent.status", result.status)
```

```
            span.set_attribute("agent.tool_calls", result.tool_call_count)
            span.set_attribute("agent.tokens_used", result.tokens_used)
            span.set_attribute("agent.cost_usd", result.cost_usd)
            return result

    def _call_llm(self, messages: list) -> str:
        with tracer.start_as_current_span("llm.call") as span:
            span.set_attribute("llm.model", self.model)
            span.set_attribute("llm.prompt_tokens", count_tokens(messages))
            response = self.llm.invoke(messages)
            span.set_attribute("llm.completion_tokens", count_tokens([response]))
            return response

    def _call_tool(self, tool_name: str, args: dict) -> str:
        with tracer.start_as_current_span(f"tool.{tool_name}") as span:
            span.set_attribute("tool.name", tool_name)
            span.set_attribute("tool.args", json.dumps(args))
            try:
                result = self.tools[tool_name](**args)
                span.set_attribute("tool.success", True)
                return result
            except Exception as e:
                span.set_attribute("tool.success", False)
                span.set_attribute("tool.error", str(e))
                span.record_exception(e)
                raise
```

Listing 27.23: Structured agent tracing with OpenTelemetry

27.6.2 Failure Categorization

Systematic failure analysis requires a taxonomy of failure modes. Without a structured classification, engineering teams waste cycles on ad-hoc debugging—treating symptoms rather than root causes. The taxonomy below captures the six most common failure classes observed in production agent systems, along with their observable symptoms, automated detection mechanisms, and proven remediation strategies.

Each failure type has different implications for system design: *tool errors* are infrastructure failures that require retry logic and circuit breakers; *reasoning errors* are model-level failures that require prompt iteration; *hallucinations* require grounding mechanisms; *infinite loops* require hard architectural safeguards. In practice, a single user-visible failure often involves a cascade across multiple categories (e.g., a tool error triggers a reasoning error as the agent attempts to recover, which spirals into an infinite loop).

Table 27.1: Agent failure taxonomy with detection and remediation strategies

Failure Type	Symptoms	Detection	Remediation
Tool Error	Exception in tool call, empty result	Error rate monitoring	Retry logic, fallback tools
Reasoning Error	Wrong tool selected, incorrect arguments	Trajectory analysis	Prompt improvement, few-shot examples
Hallucination	Fabricated facts, invented tool results	Fact-checking, grounding checks	RAG, citation requirements
Infinite Loop	Repeated tool calls, no progress	Loop detection, max iterations	Hard limits, loop-breaking prompts
Context Overflow	Truncated history, lost context	Token counting	Summarization, context management
Refusal	Agent declines valid task	Output classification	Prompt adjustment, guardrail tuning

27.6.3 Replay and Debugging Workflows

When a production failure occurs, the ability to replay the exact execution is invaluable:

```
from langsmith import Client
from datetime import datetime, timezone

ls = Client()  # Uses LANGSMITH_API_KEY env var

# Load a failed execution trace by its run ID
root_run = ls.read_run("run-abc123-def456")
child_runs = list(ls.list_runs(
    project_name="research-agent",
    filter=f'eq(parent_run_id, "{root_run.id}")',
    order="asc",
))

print(f"Trace: {root_run.id} | Status: {root_run.status}")
print(f"Error: {root_run.error}" if root_run.error else "")
print(f"Total tokens: {root_run.total_tokens}\n")

# Step through each child run (LLM call, tool call, etc.)
for i, run in enumerate(child_runs):
    print(f"Step {i}: [{run.run_type}] {run.name}")
    print(f"  Input:  {str(run.inputs)[:200]}")
    print(f"  Output: {str(run.outputs)[:200]}")
    if run.error:
        print(f"  ERROR: {run.error}")
        # Inspect the exact prompt that caused failure
        if run.run_type == "llm":
            print(f"  Model: {run.extra.get('invocation_params', {}).get('model')}
    ")
            print(f"  Messages: {run.inputs.get('messages', [])[-1]}")
    print()

# Re-run the failing step with a modified prompt or model
from openai import OpenAI
client = OpenAI()
failing_run = child_runs[4]  # e.g., step that errored
response = client.chat.completions.create(
    model="gpt-4o",  # try a stronger model
    messages=failing_run.inputs["messages"],
    temperature=0,
)
print(f"Replay output: {response.choices[0].message.content[:300]}")
```

Listing 27.24: Agent execution replay for debugging

27.7 Production Deployment Patterns

Deploying agents at scale requires careful attention to execution model, state management, and resource allocation.

27.7.1 Async Agent Execution

Long-running agents should execute asynchronously to avoid blocking API connections. Celery20 is a widely-used distributed task queue for Python that handles retries, worker scaling, and result persistence:

```
from celery import Celery
from myagent import ResearchAgent
import redis
import time
```

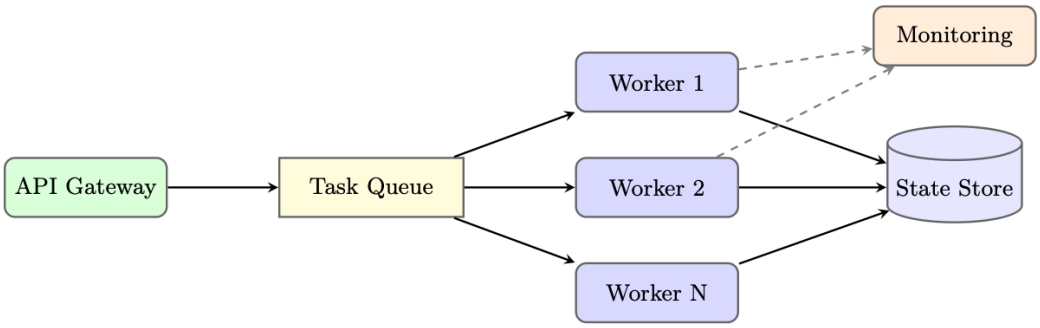


Figure 27.5: Queue-based async agent deployment. Workers pull tasks from a queue and persist state independently.

```
app = Celery("agent_tasks", broker="redis://localhost:6379/0")
state_store = redis.Redis(host="localhost", port=6379, db=1)

@app.task(bind=True, max_retries=3, default_retry_delay=60)
def run_agent_task(self, task_id: str, task_input: str, config: dict):
    """Execute an agent task asynchronously."""
    try:
        # Update task status
        state_store.hset(f"task:{task_id}", mapping={
            "status": "running",
            "started_at": time.time(),
            "worker": self.request.hostname,
        })

        agent = ResearchAgent(**config)
        result = agent.run(task_input)

        # Store result
        state_store.hset(f"task:{task_id}", mapping={
            "status": "completed",
            "result": result.to_json(),
            "completed_at": time.time(),
            "cost_usd": result.cost_usd,
        })
        return {"task_id": task_id, "status": "completed"}

    except Exception as exc:
        state_store.hset(f"task:{task_id}", mapping={
            "status": "failed",
            "error": str(exc),
            "failed_at": time.time(),
        })
        raise self.retry(exc=exc)

# API endpoint (separate Flask/FastAPI app)
from flask import Flask, request, jsonify
import uuid

web_app = Flask(__name__)

@web_app.route("/tasks", methods=["POST"])
def submit_task():
    task_id = str(uuid.uuid4())
    task = run_agent_task.delay(
        task_id=task_id,
        task_input=request.json["input"],
        config=request.json.get("config", {}),
    )
    return jsonify({"task_id": task_id, "celery_id": task.id}), 202
```

Listing 27.25: Async agent execution with Celery

27.7.2 Multi-Tenant Isolation

Production agent systems serving multiple customers require strict isolation:

- **Namespace isolation**: Each tenant's state, memory, and tool configurations are stored in separate namespaces
- **Rate limiting**: Per-tenant rate limits on LLM calls, tool invocations, and compute time
- **Resource quotas**: Maximum concurrent agents, token budgets, and storage limits per tenant
- **Audit logging**: All agent actions are logged with tenant ID for compliance and billing

27.7.3 Cost Optimization Strategies

- **Model routing**: Use smaller, cheaper models for simple subtasks (classification, extraction) and reserve large models for complex reasoning
- **Prompt caching**: OpenAI and Anthropic offer prompt caching for repeated system prompts, reducing costs by up to 90% for high-traffic agents
- **Result caching**: Cache tool results for identical inputs within a time window
- **Batching**: Batch multiple independent LLM calls when latency permits
- **Early termination**: Detect when the agent has sufficient information to answer and terminate the loop early

```
class CostOptimizedRouter:
    TASK_MODEL_MAP = {
        "classification": "gpt-4o-mini",
        "extraction": "gpt-4o-mini",
        "summarization": "gpt-4o-mini",
        "reasoning": "gpt-4o",
        "code_generation": "gpt-4o",
        "complex_analysis": "o1",
    }

    def route(self, task_type: str, complexity: float) -> str:
        base_model = self.TASK_MODEL_MAP.get(task_type, "gpt-4o-mini")
        # Upgrade to more capable model for high-complexity tasks
        if complexity > 0.8 and base_model == "gpt-4o-mini":
            return "gpt-4o"
        return base_model

    def estimate_cost(self, model: str, input_tokens: int, output_tokens: int) ->
    float:
        pricing = {
            "gpt-4o-mini": (0.15e-6, 0.60e-6),
            "gpt-4o":      (2.50e-6, 10.0e-6),
            "o1":          (15.0e-6, 60.0e-6),
        }
        in_price, out_price = pricing[model]
        return input_tokens * in_price + output_tokens * out_price
```

Listing 27.26: Model routing for cost optimization

27.7.4 Auto-Scaling Strategies

Agent workloads are bursty and unpredictable. Effective auto-scaling requires:

- **Queue-depth scaling**: Scale worker count based on task queue depth, not CPU utilization
- **Predictive scaling**: Use historical patterns (time-of-day, day-of-week) to pre-scale before demand spikes
- **Spot instance usage**: Long-running agent tasks can use spot/preemptible instances with checkpointing for cost savings
- **Graceful shutdown**: Workers complete current tasks before scaling down, preventing state corruption

27.8 Framework Comparison

Choosing the Right Framework

The "best" framework depends on your specific requirements. Ask yourself:

- Do you need explicit control over agent flow? → **LangGraph**
- Are you building a multi-agent system with code execution? → **AutoGen**
- Do you want role-based agents with minimal boilerplate? → **CrewAI**
- Are you building on OpenAI's ecosystem? → **Agents SDK**
- Do you want automated prompt optimization? → **DSPy**
- Are you in an enterprise .NET/Azure environment? → **Semantic Kernel**

27.9 Complete Implementation Example: Production Research Agent

We now present a complete, production-ready research agent built with LangGraph, demonstrating tool definition, state schema, graph construction, error handling, and deployment configuration.

Production Research Agent Architecture

This example implements a research agent that: (1) accepts a research topic, (2) searches the web for relevant sources, (3) reads and synthesizes key documents, (4) writes a structured report, and (5) handles errors gracefully with retry logic. The agent uses checkpointing for resumability and structured logging for observability.

```
# === tools.py ===
import httpx
import json
import os
import uuid
from datetime import datetime, timezone
from urllib.parse import urlparse
from langchain_core.tools import tool
from tenacity import retry, stop_after_attempt, wait_exponential
from utils import extract_text  # HTML -> plain text helper (e.g., BeautifulSoup)
from database import db          # application database connection

@tool
@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=10))
def search_web(query: str, num_results: int = 5) -> str:
```

```
    """Search the web for information. Returns JSON list of results."""
    if not query.strip():
        raise ValueError("Search query cannot be empty")
    response = httpx.get(
        "https://api.search.example.com/search",
        params={"q": query, "n": num_results},
        headers={"Authorization": f"Bearer {os.environ['SEARCH_API_KEY']}"},
        timeout=10.0,
    )
    response.raise_for_status()
    results = response.json()["results"]
    return json.dumps([{"title": r["title"], "url": r["url"],
                        "snippet": r["snippet"]} for r in results])

@tool
@retry(stop=stop_after_attempt(2), wait=wait_exponential(min=1, max=5))
def fetch_document(url: str, max_chars: int = 5000) -> str:
    """Fetch and extract text content from a URL."""
    allowed_domains = os.environ.get("ALLOWED_DOMAINS", "").split(",")
    domain = urlparse(url).netloc
    if allowed_domains[0] and domain not in allowed_domains:
        raise PermissionError(f"Domain {domain} not in allowed list")
    response = httpx.get(url, timeout=15.0, follow_redirects=True)
    response.raise_for_status()
    return extract_text(response.text)[:max_chars]

@tool
def save_report(title: str, summary: str, sections: list[dict]) -> str:
    """Save a structured research report to the database."""
    report_id = str(uuid.uuid4())
    db.reports.insert_one({
        "id": report_id, "title": title,
        "summary": summary, "sections": sections,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    return json.dumps({"report_id": report_id, "status": "saved"})

TOOLS = [search_web, fetch_document, save_report]
```

Listing 27.27: Complete production research agent: tools and state

```
# === agent.py ===
import json
from typing import TypedDict, Annotated, List, Literal
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from langchain_core.messages import BaseMessage, HumanMessage, SystemMessage,
    AIMessage
from tools import TOOLS

SYSTEM_PROMPT = """You are a professional research analyst. Your task is to:
1. Search for relevant information on the given topic
2. Read and analyze key sources (aim for 3-5 sources)
3. Synthesize findings into a structured report using save_report

Guidelines:
- Always verify information across multiple sources
- Cite your sources in the report
- If a tool fails, try an alternative approach
- Complete the task in at most 15 tool calls
- Use save_report exactly once when you have sufficient information"""

class ResearchState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]
    topic: str
```

```
    sources_found: List[str]
    sources_read: List[str]
    report_id: str | None
    error_count: int
    tool_call_count: int
    status: Literal["researching", "done", "failed"]

tool_executor = ToolNode(TOOLS)

def research_node(state: ResearchState) -> dict:
    """Main LLM reasoning node."""
    llm = ChatOpenAI(model="gpt-4o", temperature=0).bind_tools(TOOLS)
    messages = [SystemMessage(content=SYSTEM_PROMPT)] + state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response]}

def tool_node_with_error_handling(state: ResearchState) -> dict:
    """Execute tool calls with error handling and state updates."""
    try:
        result = tool_executor.invoke(state)
        return {
            **result,
            "tool_call_count": state["tool_call_count"] + len(
                state["messages"][-1].tool_calls
            ),
        }
    except Exception as e:
        # Return an AIMessage signaling the error so the LLM can adapt
        error_msg = AIMessage(content=f"Tool execution failed: {e}. Try a
   different approach.")
        return {
            "messages": [error_msg],
            "error_count": state["error_count"] + 1,
        }

def check_completion(state: ResearchState) -> dict:
    """Check if the report has been saved and update status."""
    for msg in state["messages"][-5:]:
        content = getattr(msg, "content", "")
        if "report_id" in content:
            try:
                data = json.loads(content)
                return {"status": "done", "report_id": data["report_id"]}
            except (json.JSONDecodeError, KeyError):
                pass
    return {}

def route_after_llm(state: ResearchState) -> str:
    """Determine next step after LLM response."""
    if state["error_count"] >= 5 or state["tool_call_count"] >= 15:
        return "fail"
    last_message = state["messages"][-1]
    if hasattr(last_message, "tool_calls") and last_message.tool_calls:
        return "tools"
    if len(state["messages"]) > 30:
        return "fail"
    return "research"  # LLM needs to continue reasoning

def fail_node(state: ResearchState) -> dict:
    return {"status": "failed"}
```

Listing 27.28: Complete production research agent: state and nodes

```
# === graph.py ===
from langgraph.graph import StateGraph, START, END
from langgraph.graph.state import CompiledStateGraph
```

```
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

async def build_graph(db_url: str) -> CompiledStateGraph:
    """Build and compile the research agent graph."""
    checkpointer = AsyncPostgresSaver.from_conn_string(db_url)
    await checkpointer.setup()  # Create tables if needed

    builder = StateGraph(ResearchState)

    # Add nodes
    builder.add_node("research", research_node)
    builder.add_node("tools", tool_node_with_error_handling)
    builder.add_node("check", check_completion)
    builder.add_node("fail", fail_node)

    # Define edges
    builder.add_edge(START, "research")
    builder.add_conditional_edges(
        "research",
        route_after_llm,
        {"tools": "tools", "research": "research", "fail": "fail"}
    )
    builder.add_edge("tools", "check")
    builder.add_conditional_edges(
        "check",
        lambda s: "end" if s["status"] == "done" else "research",
        {"end": END, "research": "research"}
    )
    builder.add_edge("fail", END)

    return builder.compile(checkpointer=checkpointer)

# === deployment.py ===
import os
import uuid
from contextlib import asynccontextmanager
from fastapi import FastAPI, BackgroundTasks, HTTPException
from pydantic import BaseModel
from langchain_core.messages import HumanMessage

graph: CompiledStateGraph = None  # Initialized at startup

@asynccontextmanager
async def lifespan(app: FastAPI):
    global graph
    graph = await build_graph(os.environ["DATABASE_URL"])
    yield

app = FastAPI(title="Research Agent API", lifespan=lifespan)

class ResearchRequest(BaseModel):
    topic: str
    user_id: str

class ResearchResponse(BaseModel):
    task_id: str
    status: str

@app.post("/research", response_model=ResearchResponse)
async def start_research(request: ResearchRequest, background_tasks:
    BackgroundTasks):
    task_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": task_id, "user_id": request.user_id}}
    initial_state = {
        "messages": [HumanMessage(content=f"Research topic: {request.topic}")],
        "topic": request.topic,
```

```
        "sources_found": [], "sources_read": [],
        "report_id": None, "error_count": 0,
        "tool_call_count": 0, "status": "researching",
    }
    background_tasks.add_task(graph.ainvoke, initial_state, config)
    return ResearchResponse(task_id=task_id, status="started")

@app.get("/research/{task_id}")
async def get_research_status(task_id: str):
    config = {"configurable": {"thread_id": task_id}}
    state = await graph.aget_state(config)
    if state is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return {
        "task_id": task_id,
        "status": state.values.get("status", "unknown"),
        "report_id": state.values.get("report_id"),
        "tool_calls": state.values.get("tool_call_count", 0),
        "error_count": state.values.get("error_count", 0),
    }
```

Listing 27.29: Complete production research agent: graph and deployment

```
# === Dockerfile ===
# FROM python:3.11-slim
# WORKDIR /app
# COPY requirements.txt .
# RUN pip install --no-cache-dir -r requirements.txt
# COPY . .
# CMD ["uvicorn", "deployment:app", "--host", "0.0.0.0", "--port", "8000"]

# === kubernetes/deployment.yaml (as Python dict for illustration) ===
k8s_deployment = {
    "apiVersion": "apps/v1",
    "kind": "Deployment",
    "metadata": {"name": "research-agent", "namespace": "agents"},
    "spec": {
        "replicas": 3,
        "selector": {"matchLabels": {"app": "research-agent"}},
        "template": {
            "metadata": {"labels": {"app": "research-agent"}},
            "spec": {
                "containers": [{
                    "name": "agent",
                    "image": "myregistry/research-agent:latest",
                    "ports": [{"containerPort": 8000}],
                    "resources": {
                        "requests": {"memory": "512Mi", "cpu": "250m"},
                        "limits":   {"memory": "2Gi",  "cpu": "1000m"},
                    },
                    "env": [
                        {"name": "DATABASE_URL",   "valueFrom": {
                            "secretKeyRef": {"name": "agent-secrets", "key": "db-
    url"}}},
                        {"name": "OPENAI_API_KEY", "valueFrom": {
                            "secretKeyRef": {"name": "agent-secrets", "key": "
    openai-key"}}},
                    ],
                    "livenessProbe":  {"httpGet": {"path": "/health", "port":
    8000},
                                       "initialDelaySeconds": 30, "periodSeconds":
     10},
                    "readinessProbe": {"httpGet": {"path": "/ready",  "port":
    8000},
                                       "initialDelaySeconds": 10, "periodSeconds":
     5},
```

```
                }]
            }
        }
    }
}

# HorizontalPodAutoscaler scales on queue depth metric
hpa_config = {
    "apiVersion": "autoscaling/v2",
    "kind": "HorizontalPodAutoscaler",
    "metadata": {"name": "research-agent-hpa", "namespace": "agents"},
    "spec": {
        "scaleTargetRef": {
            "apiVersion": "apps/v1",
            "kind": "Deployment",
            "name": "research-agent",
        },
        "minReplicas": 2,
        "maxReplicas": 20,
        "metrics": [{
            "type": "External",
            "external": {
                "metric": {"name": "agent_task_queue_depth"},
                "target": {"type": "AverageValue", "averageValue": "10"},
            }
        }]
    }
}
```

Listing 27.30: Deployment configuration: Docker and Kubernetes

Production Checklist

Before deploying an agent to production, verify:

- All tools have retry logic and error handling
- Maximum iteration limits are enforced
- Sensitive data is not logged in traces
- Rate limiting is configured per tenant
- Checkpointing is enabled for long-running tasks
- Behavioral tests pass (no harmful outputs)
- Cost and latency bounds are validated
- Rollback procedure is documented and tested
- On-call runbook covers common failure modes

27.10 Summary

Agent development frameworks have matured significantly, providing structured solutions to the engineering challenges of building production-grade AI agents. The key takeaways from this section are:

1. **Framework selection matters**: Different frameworks optimize for different concerns. LangGraph excels at complex, controllable workflows; AutoGen at multi-agent collaboration; CrewAI at role-based simplicity; DSPy at automated optimization.

2. **Testing is non-negotiable**: The non-deterministic nature of LLM-based agents makes

comprehensive testing—unit, integration, behavioral, and performance—essential for production reliability.

3. **Observability enables iteration**: Without detailed traces of agent execution, diagnosing failures and improving performance is guesswork. Invest in observability infrastructure early.

4. **Async execution is the norm**: Production agents are long-running processes that require queue-based execution, checkpointing, and graceful failure handling.

5. **Cost management is critical**: LLM API costs scale with usage. Model routing, caching, and early termination can reduce costs by 50–90% without sacrificing quality.

6. **The lifecycle is iterative**: Agent development is not a one-time effort. Continuous monitoring, failure analysis, and improvement are essential for maintaining performance as the world changes.

The field is evolving rapidly, with new frameworks, tools, and best practices emerging regularly. The principles covered in this section—explicit state management, comprehensive testing, deep observability, and systematic iteration—provide a stable foundation regardless of which specific tools are in vogue.

Chapter 28

Agentic UI Frameworks

As large language models transition from passive text generators to active agents capable of planning, tool use, and multi-step reasoning, the interfaces through which humans interact with them must evolve in parallel. Traditional chat interfaces—designed for single-turn or short-context conversations—are ill-suited to the demands of agentic workflows: long-running tasks, branching decision trees, parallel tool invocations, and the need for meaningful human oversight. This section surveys the landscape of *agentic UI frameworks*: the design paradigms, component libraries, and implementation patterns that enable rich, transparent, and trustworthy human-agent collaboration.

GAS Connection — Agentic UI Frameworks

The GAS ships with a Streamlit dashboard (`part5_agentic/ch28_agentic_ui/app.py`) that is a minimal but complete example of an agentic UI. Every UI pattern described in the following chapter has a counterpart in this application.

UI paradigms (§ UI Paradigms for Agents). The GAS dashboard uses the *monitoring dashboard* paradigm (not a chat interface): it displays the patient's real-time glucose trace, active alerts, agent reasoning traces, and system status. A chat interface would be inappropriate for a real-time monitoring system where the caregiver needs to see the current state at a glance, not engage in a conversation. The dashboard paradigm is recommended in this section for monitoring and observability use cases.

Key UI components (§ Key UI Components for Agents). The GAS dashboard in `part5_agentic/ch28_agentic_ui/app.py` includes: (1) *Status panel*: current glucose reading (mg/dL), trend arrow, alert level; (2) *Alert feed*: chronological list of alerts with urgency colour-coding; (3) *Reasoning trace*: expandable panel showing the Monitor's anomaly reasoning, the Advisor's recommendation rationale, and the Safety Critic's verdict (from `agent_messages` in `GASState`); (4) *Scenario selector*: allows the user to run any of the five clinical scenarios from `shared/patient_data.py`.

Streaming and real-time patterns (§ Streaming and Real-Time Patterns). The GAS dashboard uses Streamlit's streaming capabilities to display the Advisor's recommendation as it is generated. The streaming is implemented via LangGraph's streaming API, which yields partial `GASState` updates as each agent node completes. This reduces the perceived latency from the full pipeline execution time to the time-to-first-token of the first agent.

Human-in-the-loop UI design (§ Human-in-the-Loop UI Design). The GAS dashboard implements the human-in-the-loop patterns described in this section: the Caregiver Notifier always involves a human in the final decision (`caregiver_notified` field in `GASState`). The UI displays the Safety Critic's verdict (`safety_score`, `safety_approved`) alongside the recommendation, allowing caregivers to understand why an alert was or was not sent.

Implementation example (§ Implementation Example: A Full-Stack Agentic UI). The GAS dashboard (`part5_agentic/ch28_agentic_ui/app.py`) is a self-contained Streamlit application that connects directly to the LangGraph pipeline via `build_gas_graph()` in `capstone_gas/gas_system.py`. It serves as a reference implementation for the full-stack agentic UI pattern described in this section, demonstrating how to wire a Streamlit frontend to a LangGraph backend with minimal boilerplate.

28.1 Motivation: Beyond the Chat Box

Why Agents Need Specialized Interfaces

A chat bubble conveys a *result*. An agentic UI conveys a *process*—the reasoning, the tools invoked, the decisions made, and the points where human judgment is required. Without this visibility, users cannot trust, correct, or learn from the agent.

The gap between a chat interface and an agentic interface mirrors the gap between a vending machine and a skilled collaborator. When an agent executes a 20-step research task, browses the web, writes and runs code, and synthesizes a report, the user needs answers to questions that a simple text response cannot provide:

- **What is the agent doing right now?** Long-running tasks require progress feedback; silence breeds distrust.
- **Why did the agent make this decision?** Transparency into reasoning enables users to catch errors early.
- **Which tools were used, and with what inputs?** Tool provenance is essential for verifying factual claims and auditing behavior.
- **Where should I intervene?** Agents must surface decision points that warrant human judgment without overwhelming users with every micro-decision.
- **Can I undo this?** Irreversible actions (sending emails, modifying files, executing code) require explicit confirmation and rollback paths.

The Automation Bias Risk

Research on human-automation interaction consistently shows that users over-trust automated systems, especially when those systems present outputs confidently and without uncertainty signals [300]. Agentic UIs must actively counteract automation bias by surfacing uncertainty, showing reasoning, and making it easy to question or override agent decisions.

The design of agentic UIs thus sits at the intersection of human-computer interaction (HCI), explainable AI (XAI), and software engineering. The core design goals are:

1. **Transparency**: Make the agent's internal state legible to the user.
2. **Control**: Provide meaningful intervention points without requiring constant supervision.
3. **Trust Calibration**: Help users develop accurate mental models of agent capabilities and limitations.
4. **Efficiency**: Minimize cognitive load; surface the right information at the right time.
5. **Recoverability**: Make mistakes cheap to detect and reverse.

28.2 UI Paradigms for Agents

No single UI paradigm suits all agentic use cases. The appropriate interface depends on task duration, required human involvement, output type, and user expertise. The spectrum ranges from fully conversational chat interfaces to fully autonomous dashboards with minimal human interaction.

28.2.1 Chat-Based Interfaces

The chat paradigm—message bubbles, a text input, and a scrolling history—remains the most familiar entry point for LLM interaction. Its strengths are low learning curve and natural language flexibility. For agentic use, chat interfaces are augmented with:

- **Streaming responses**: Tokens appear as they are generated, providing immediate feedback and reducing perceived latency. Implemented via Server-Sent Events (SSE) or WebSockets.
- **Inline tool indicators**: Small badges or expandable sections within the message stream show when a tool was called (e.g., "`[Searched the web for: climate change 2024]`").
- **Typing indicators and status messages**: "Agent is thinking...", "Running Python code...", "Fetching results..." keep users informed during latency gaps.
- **Message threading**: For multi-turn agentic tasks, collapsible sub-threads can contain intermediate steps without cluttering the main conversation.

Chat UI Limitations for Agents

Chat interfaces serialize inherently parallel processes. When an agent fans out to five tools simultaneously, a linear message stream misrepresents the actual execution graph. For complex agentic workflows, chat should be augmented with—or replaced by—richer paradigms.

28.2.2 Canvas and Artifact-Based Interfaces

The canvas paradigm, popularized by Claude Artifacts1 and ChatGPT Canvas,2 introduces a *split-pane* layout: the left pane hosts the conversation, while the right pane (the "canvas" or "artifact panel") displays generated content—code, documents, diagrams, spreadsheets—as a live, editable artifact.

Key characteristics:

- **Persistent artifacts**: Generated content persists across turns and can be iteratively refined through natural language instructions ("make the chart blue", "add error handling to the function").
- **In-place editing**: Users can directly edit the artifact, and the agent can observe and respond to those edits.
- **Version history**: Artifacts maintain a revision history, enabling rollback to any prior state.
- **Multi-artifact workspaces**: Advanced implementations support multiple simultaneous artifacts (e.g., a code file, its test suite, and a documentation page).

The canvas paradigm is particularly well-suited to *co-creation* tasks: writing, coding, data analysis, and design, where the output is a document or artifact rather than a conversational answer.

28.2.3 Workflow Visualization

For agents that execute structured plans—sequences or graphs of steps—workflow visualization UIs make the plan explicit and trackable. This paradigm is common in:

- **Agentic pipelines** (LangGraph, AutoGen, CrewAI): The agent's execution graph is rendered as a directed acyclic graph (DAG) or flowchart, with nodes representing steps and edges representing data flow or control flow.
- **Task decomposition views**: The agent's high-level plan is shown as a checklist or Gantt-style timeline, with each sub-task expanding to reveal its own steps.
- **Live progress tracking**: Nodes change color or display spinners as they execute; completed nodes show outputs; failed nodes show error details.

LangGraph Studio3 is the canonical example of this paradigm, providing a graph-based debugger and visualizer for LangGraph agents. Users can inspect the state at each node, replay executions, and inject modified state to test alternative paths.

28.2.4 Dashboard and Monitoring Interfaces

For long-running or production agents, dashboard UIs provide an operational view:

- **Real-time status**: Which agents are running, idle, or failed; current task and step.
- **Resource metrics**: Token consumption, API call counts, latency histograms, cost estimates.
- **Queue management**: Pending tasks, priority ordering, rate limit status.
- **Alert and anomaly detection**: Unusual behavior (excessive retries, cost spikes, repeated failures) surfaced as notifications.
- **Historical analytics**: Task completion rates, average duration, error frequency over time.

Dashboard UIs are typically built with tools like Grafana,4 custom React dashboards, or Streamlit, and are aimed at *operators* rather than end users.

28.2.5 Collaborative Interfaces

Collaborative UIs treat the agent as a peer contributor to a shared workspace—a document, codebase, or design canvas—alongside human collaborators. Key features include:

- **Presence indicators**: The agent appears as a named cursor or avatar in the shared workspace.
- **Change attribution**: Edits made by the agent are visually distinguished from human edits (e.g., color-coded diffs).
- **Inline suggestions**: The agent proposes changes as tracked edits or comments, which humans can accept, reject, or modify.
- **Conflict resolution**: When the agent and a human edit the same region simultaneously, the UI surfaces the conflict and facilitates resolution.

This paradigm is emerging in tools like Cursor5 (collaborative code editing with AI), Notion AI,6 and Google Docs with Gemini integration.7

28.2.6 Autonomous with Checkpoints

At the far end of the autonomy spectrum, some agents run largely independently—browsing the web, writing code, executing commands—and surface only at predefined *checkpoints* requiring human approval. This paradigm is used in:

- **Computer-use agents** (Anthropic Computer Use,8 OpenAI Operator9): The agent controls a browser or desktop; the UI shows a live screen feed and pauses for approval before irreversible actions.
- **Automated pipelines with gates**: CI/CD-style workflows where the agent completes a phase and waits for a human "merge" before proceeding.
- **Scheduled agents**: Agents that run on a schedule and report results asynchronously, with a notification-based UI for reviewing outputs and approving follow-on actions.

Checkpoint UI in Practice

An agent tasked with "clean up my email inbox" might autonomously categorize and archive 500 emails, then pause and present a summary: "I found 23 emails that appear to be from mailing lists you haven't opened in 6 months. Shall I unsubscribe from all, some, or none?" The user reviews a list, makes selections, and the agent proceeds. This pattern—autonomous execution

punctuated by human decision points—balances efficiency with control.

28.3 Key UI Components for Agents

Regardless of the overarching paradigm, agentic UIs share a set of recurring components. This section catalogs the most important, with design guidance for each.

28.3.1 Thought and Reasoning Display

Modern LLMs, particularly those trained with chain-of-thought or extended thinking (e.g., OpenAI o1/o3, Anthropic Claude with extended thinking), generate substantial internal reasoning before producing a final response. Surfacing this reasoning is a double-edged sword: it increases transparency but can overwhelm users with verbose internal monologue.

Best practices:

- **Collapsible reasoning blocks**: Show a summary ("Thought for 12 seconds") with an expand toggle for users who want details.
- **Progressive disclosure**: Show only the final conclusion by default; reasoning is available on demand.
- **Structured reasoning**: If the model produces structured thoughts (hypotheses, evidence, conclusions), render them with visual hierarchy rather than as a wall of text.
- **Reasoning vs. response distinction**: Clearly visually distinguish internal reasoning (which may contain errors or false starts) from the final response.

28.3.2 Tool Use Visualization

Tool calls are the primary mechanism by which agents interact with the world. Visualizing them is essential for trust and debugging.

Tool Call Anatomy

Each tool invocation has four components worth displaying: (1) the **tool name** and icon, (2) the **input arguments** (potentially large JSON), (3) the **output/result** (potentially large), and (4) **timing** (latency). The UI must balance completeness with readability.

Design patterns for tool visualization:

- **Inline tool cards**: Compact cards within the message stream showing tool name, a one-line summary of inputs, and status (running/success/error). Expandable for full details.
- **Tool timeline**: A horizontal timeline showing all tool calls in a turn, with durations, enabling identification of bottlenecks.
- **Input/output diff**: For tools that modify state (e.g., file editing), show a before/after diff.
- **Tool icons and branding**: Recognizable icons for common tools (web search, code execution, file system, APIs) enable rapid scanning.
- **Error highlighting**: Failed tool calls shown in red with the error message and any retry attempts.

28.3.3 Progress Indicators

Multi-step agentic tasks require rich progress feedback:

- **Step-level progress**: A numbered list of planned steps with checkmarks as each completes. For dynamic plans, steps can be added or removed as the agent adapts.
- **Token streaming indicators**: A blinking cursor or animated ellipsis during generation; a token-per-second counter for power users.
- **Estimated completion**: Where feasible, an ETA based on task complexity and historical performance. Displayed with appropriate uncertainty ("approximately 2–5 minutes").
- **Subtask nesting**: For hierarchical tasks, a tree-structured progress view with expandable subtasks.
- **Cancellation**: A clearly visible "Stop" button that gracefully halts the agent and summarizes work completed so far.

28.3.4 Approval Gates

Approval gates are the primary mechanism for human-in-the-loop control. They must be designed to be *informative* (giving users enough context to make a good decision) without being *fatiguing* (requiring approval for every trivial action).

Alert Fatigue in Approval Gates

If an agent requests approval too frequently, users will begin approving reflexively without reading—defeating the purpose of the gate. Tiered approval policies (see Section 28.7) are essential to maintain meaningful oversight.

Approval gate UI elements:

- **Action summary**: Plain-language description of what the agent wants to do ("Send an email to john@example.com with the attached report").
- **Risk indicator**: Visual signal of action reversibility (green = easily undoable, yellow = hard to undo, red = irreversible).
- **Approve / Reject / Modify**: Three-option interface; "Modify" opens an editor for the action parameters before approval.
- **Context panel**: Expandable section showing why the agent wants to take this action (relevant reasoning, prior steps).
- **Timeout behavior**: Clear indication of what happens if the user doesn't respond (agent pauses, not proceeds).

28.3.5 Context Display

Agents maintain internal state—memory, active tools, retrieved documents, conversation history—that influences their behavior. Making this state visible helps users understand and predict agent behavior.

- **Memory panel**: Shows what the agent currently "remembers" about the user, task, and prior interactions. Editable by the user.
- **Active tools list**: Which tools are currently available to the agent, with enable/disable toggles.
- **Retrieved context**: Documents or data chunks currently in the agent's context window, with source citations.
- **Token budget indicator**: How much of the context window is consumed, helping users understand when to start a new session.

28.3.6 Error and Recovery UI

Agents fail—tools return errors, models hallucinate, plans become infeasible. The UI must handle failures gracefully:

- **Error cards**: Inline display of failures with the error type, message, and the agent's interpretation.
- **Retry controls**: Manual retry button with optional parameter adjustment.
- **Alternative approaches**: When the primary approach fails, the agent proposes alternatives; the UI presents them as selectable options.
- **Partial results**: If a multi-step task fails midway, the UI shows completed steps and their outputs, preserving partial value.
- **Escalation path**: A clear path to human support or manual completion when the agent cannot proceed.

28.3.7 Confidence Indicators

LLMs are probabilistic systems with calibrated (or miscalibrated) uncertainty. Surfacing confidence helps users know when to trust and when to verify:

- **Verbal hedging display**: Highlight phrases like "I'm not certain" or "you may want to verify" to draw attention to low-confidence claims.
- **Source quality indicators**: For retrieved information, show source recency, authority, and relevance scores.
- **Explicit uncertainty requests**: A "How confident are you?" button that prompts the agent to self-assess and explain its uncertainty.
- **Verification suggestions**: For high-stakes outputs, the agent proactively suggests verification steps ("I recommend checking this calculation independently").

28.4 Frameworks and Libraries

A growing ecosystem of frameworks accelerates the development of agentic UIs. We survey the most widely adopted, organized by primary language and use case.

28.4.1 Vercel AI SDK

The Vercel AI SDK [379] is a TypeScript/JavaScript library for building streaming AI interfaces in React, Next.js, Svelte, and Vue. It is the most widely used framework for production web-based agent UIs.

Core abstractions:

- `useChat`: A React hook managing a chat conversation with streaming support, message history, and loading states.
- `useCompletion`: A hook for single-turn text completion with streaming.
- `useObject`: Streams structured JSON objects, enabling progressive rendering of complex outputs.
- `streamText` / `streamObject`: Server-side functions that stream LLM responses over HTTP.

Generative UI (AI SDK RSC): The most distinctive feature of the Vercel AI SDK is its support for *generative UI* via React Server Components (RSC). Rather than returning text, the LLM can invoke tools whose results are rendered as arbitrary React components—a weather widget, a stock chart, a booking form—streamed directly into the UI. This is discussed further in Section 28.5.

28.4.2 Chainlit

Chainlit [42] is a Python framework for building production-ready agent UIs with minimal boilerplate. It is particularly popular in the LangChain and LlamaIndex ecosystems.

Key features:

- **Step visualization**: Chainlit natively renders LangChain and LlamaIndex execution steps as a collapsible tree, showing each chain call, retrieval, and tool invocation.
- **Multi-modal support**: File uploads, image display, audio playback, and PDF rendering out of the box.
- **Authentication and sessions**: Built-in user authentication, persistent conversation history, and multi-user support.
- **Custom elements**: React components can be registered and rendered from Python, enabling rich custom visualizations.
- **Feedback collection**: Built-in thumbs up/down feedback with optional comments, stored to a database.

```python
import chainlit as cl
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent

@tool
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

agent = create_react_agent(
    ChatOpenAI(model="gpt-4o"), tools=[search]
)

@cl.on_message
async def on_message(message: cl.Message):
    # Chainlit automatically renders each step as a collapsible UI element
    # when using the callback handler
    async with cl.Step(name="Agent", type="run") as step:
        step.input = message.content
        result = await agent.ainvoke(
            {"messages": [{"role": "user", "content": message.content}]},
            config={"callbacks": [cl.LangchainCallbackHandler()]}
        )
        output = result["messages"][-1].content
        step.output = output

    await cl.Message(content=output).send()
```

Listing 28.1: Minimal Chainlit agent with step visualization

28.4.3 Gradio

Gradio [1] is a Python library for rapidly building ML demos and agent interfaces. Its `gr.ChatInterface` and `gr.Blocks` API enable quick prototyping of conversational agents with minimal code.

Strengths for agentic UIs:

- **Zero-configuration deployment**: One-line sharing via Hugging Face Spaces.
- **Custom components**: The Gradio Custom Components system allows building React components that integrate seamlessly with Python backends.

- **Multi-modal inputs**: File upload, image, audio, video, and webcam inputs with minimal configuration.
- **Streaming**: Native support for generator-based streaming responses.

Limitations: Gradio's layout system is less flexible than full React frameworks, and its state management is session-scoped, making complex multi-agent coordination challenging.

28.4.4 Streamlit

Streamlit [166] is a Python framework for data applications that has been widely adopted for agent dashboards and monitoring UIs. Its reactive execution model—the entire script reruns on each interaction—is simple but can be limiting for complex agentic workflows.

Agentic use cases:

- **Agent dashboards**: Real-time metrics, task queues, and status displays using `st.metric`, `st.dataframe`, and `st.status`.
- **Session state**: `st.session_state` persists agent state across reruns, enabling multi-turn conversations.
- **Streaming**: `st.write_stream` renders generator outputs progressively.
- **Fragments**: `@st.fragment` decorator enables partial reruns, improving performance for live-updating dashboards.

28.4.5 OpenAI Assistants Playground

The OpenAI Assistants Playground serves as a reference implementation for agentic UI design. It demonstrates:

- Thread-based conversation management with persistent history.
- File attachment and retrieval visualization.
- Code interpreter execution with output display (stdout, images, files).
- Function call display with input/output inspection.
- Run step visualization showing the sequence of model calls and tool invocations.

While not a framework for building custom UIs, the Playground's design patterns are widely emulated.

28.4.6 LangGraph Studio

LangGraph Studio [165] is a desktop application providing a visual IDE for LangGraph agents. It is the most sophisticated tool-use and workflow visualization environment currently available.

Features:

- **Graph visualization**: Interactive rendering of the agent's state machine, with nodes representing agent steps and edges representing transitions.
- **State inspection**: At any point in execution, the full agent state (all variables, memory, tool results) can be inspected as structured JSON.
- **Time-travel debugging**: Replay any prior execution step, modify the state, and re-run from that point.

- **Human-in-the-loop integration**: Breakpoints can be set on any node; execution pauses and waits for human input before proceeding.
- **Multi-agent support**: Visualizes supervisor-subagent hierarchies and inter-agent message passing.

28.4.7 Framework Comparison

Table 28.1 summarizes the key characteristics of the frameworks discussed above.

Table 28.1: Agentic UI framework comparison.

Framework	Language	Stream	Tool Viz	Multi-Ag.	Gen UI	Prod
Vercel AI SDK	TypeScript	✓	Partial	Partial	✓	✓
Chainlit	Python	✓	✓	Partial	Partial	✓
Gradio	Python	✓	○	×	○	✓
Streamlit	Python	✓	○	×	×	✓
OAI Playground	N/A (hosted)	✓	✓	×	×	×
LangGraph Studio	Python/TS	✓	✓	✓	×	Partial

28.5 Generative UI

The Generative UI Concept

Traditional LLM interfaces render model outputs as text or markdown. *Generative UI* inverts this: the model's tool calls *generate* UI components. The model decides not just *what* to say, but *how* to present it—as a chart, a form, a map, a calendar widget—based on the content type and user intent.

Generative UI represents a fundamental shift in the relationship between LLMs and interfaces. Rather than the developer pre-specifying all possible UI states, the model dynamically selects and parameterizes UI components appropriate to the current context.

28.5.1 React Server Components for Dynamic Interfaces

The Vercel AI SDK's RSC (React Server Components10) integration is the most mature implementation of generative UI. The architecture works as follows:

1. The user sends a message to a Next.js11 server action.
2. The server calls the LLM with a set of tools, each associated with a React component.
3. When the LLM calls a tool (e.g., `show_weather`), the server renders the corresponding React component with the tool's output as props.
4. The rendered component is streamed to the client as a React Server Component, appearing inline in the chat.

```
// app/actions.tsx (Server Action)
import { streamUI } from 'ai/rsc';
import { openai } from '@ai-sdk/openai';
import { WeatherCard } from '@/components/WeatherCard';
import { StockChart } from '@/components/StockChart';

export async function chat(userMessage: string) {
  const result = await streamUI({
    model: openai('gpt-4o'),
    messages: [{ role: 'user', content: userMessage }],
    tools: {
      show_weather: {
        description: 'Display current weather for a location',
```

```
        parameters: z.object({
          location: z.string(),
          unit: z.enum(['celsius', 'fahrenheit']),
        }),
        // Tool result rendered as a React component
        generate: async ({ location, unit }) => {
          const data = await fetchWeather(location, unit);
          return <WeatherCard data={data} />;
        },
      },
      show_stock: {
        description: 'Display stock price chart',
        parameters: z.object({ ticker: z.string() }),
        generate: async ({ ticker }) => {
          const data = await fetchStockData(ticker);
          return <StockChart ticker={ticker} data={data} />;
        },
      },
    },
  });
  return result.value;
}
```

Listing 28.2: Generative UI with Vercel AI SDK RSC (TypeScript)

28.5.2 Adaptive Interfaces Based on Content Type

Generative UI enables interfaces that adapt to the nature of the content being presented:

- **Tabular data** → sortable, filterable data table with export options.
- **Geographic data** → interactive map with markers and layers.
- **Time series** → zoomable line chart with annotations.
- **Code** → syntax-highlighted editor with run button.
- **Documents** → formatted document viewer with annotation tools.
- **Forms/structured input** → dynamically generated form fields.

The model acts as a *UI orchestrator*, selecting the most appropriate presentation for each piece of information. This reduces the need for developers to anticipate every possible output type and pre-build corresponding components.

Limits of Generative UI

How much UI generation should be delegated to the model? Fully model-driven UI risks inconsistency, accessibility failures, and security vulnerabilities (e.g., a model generating a form that submits data to an unexpected endpoint). In practice, generative UI works best when the model selects from a *curated library* of pre-built, accessible, and secure components rather than generating arbitrary HTML or JSX.

28.6 Streaming and Real-Time Patterns

Streaming is foundational to agentic UIs: it transforms the experience from "wait for a result" to "watch the agent work." This section covers the key streaming patterns and their implementation considerations.

28.6.1 Token Streaming

Token streaming delivers LLM output incrementally as tokens are generated, rather than waiting for the complete response. Two transport mechanisms are commonly used:

- **Server-Sent Events (SSE)**12: A unidirectional HTTP stream from server to client. Each event carries a chunk of tokens. SSE is simple, works over standard HTTP/1.1, and is automatically reconnected by browsers. It is the dominant mechanism for LLM streaming APIs (OpenAI, Anthropic, Google all use SSE).
- **WebSockets**: Bidirectional persistent connections. More complex to implement but necessary for interactive streaming scenarios where the client needs to send data mid-stream (e.g., interrupting the agent, providing mid-generation feedback).

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from openai import AsyncOpenAI
import json

app = FastAPI()
client = AsyncOpenAI()

async def token_stream(prompt: str):
    """Generator that yields SSE-formatted token chunks."""
    stream = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )
    async for chunk in stream:
        delta = chunk.choices[0].delta
        if delta.content:
            # SSE format: "data: <json>\n\n"
            yield f"data: {json.dumps({'token': delta.content})}\n\n"
        elif chunk.choices[0].finish_reason:
            yield f"data: {json.dumps({'done': True})}\n\n"

@app.get("/stream")
async def stream_endpoint(prompt: str):
    return StreamingResponse(
        token_stream(prompt),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
```

Listing 28.3: SSE token streaming with FastAPI

28.6.2 Tool Call Streaming

Modern LLM APIs support streaming tool calls: the tool name and arguments are streamed incrementally, enabling the UI to show "Agent is calling `search_web` with query: 'climate change 2024'..." before the tool has even been invoked. This requires parsing partial JSON, which can be done with streaming JSON parsers.

Patterns for tool call streaming:

- **Progressive argument display**: Show tool arguments as they stream in, even before the call is complete.
- **Parallel tool call indicators**: When the model calls multiple tools simultaneously, show all of them as pending, then update each as results arrive.
- **Tool result streaming**: Some tools (e.g., code execution, web scraping) can themselves stream results; pipe these through to the UI progressively.

28.6.3 Multi-Agent Streaming

In multi-agent systems, multiple agents may be generating output simultaneously. The UI must handle parallel streams:

- **Agent-labeled streams**: Each stream is tagged with the agent's identity; the UI renders them in separate lanes or panels.
- **Stream merging**: For supervisor-subagent patterns, the supervisor's stream may interleave with subagent streams; the UI must maintain coherent ordering.
- **Backpressure**: If the UI cannot render as fast as streams arrive (e.g., multiple agents generating simultaneously), a backpressure mechanism must prevent buffer overflow. Strategies include: dropping intermediate tokens (showing only the latest), batching updates, or pausing slower streams.

28.6.4 Optimistic UI Updates

Optimistic UI updates improve perceived responsiveness by immediately reflecting user actions in the UI before server confirmation:

- When a user sends a message, it appears immediately in the chat history (optimistically) while the request is in flight.
- When an approval gate is accepted, the UI immediately shows the action as "approved" and begins showing the agent's next steps, even before the server has processed the approval.
- If the server returns an error, the optimistic update is rolled back and an error state is shown.

28.6.5 Backpressure Handling

In high-throughput agentic scenarios, the rate of incoming data can exceed the UI's rendering capacity. Strategies for managing backpressure:

- **Token batching**: Buffer tokens for 50–100ms and render in batches rather than one-by-one, reducing DOM update frequency.
- **Virtual scrolling**: For long outputs, render only the visible portion of the content, discarding off-screen DOM nodes.
- **Throttled updates**: For metrics and status displays, update at a fixed rate (e.g., 10 Hz) regardless of the incoming data rate.
- **Progressive detail**: Show a summary view during high-throughput periods; full detail available on demand.

28.7 Human-in-the-Loop UI Design

Human-in-the-loop (HITL) interaction is one of the most consequential design challenges in agentic UIs. The goal is to maintain meaningful human oversight without creating a bottleneck that negates the efficiency benefits of automation.

28.7.1 When to Interrupt the Agent

Not all agent actions warrant human review. A principled interruption policy considers:

- **Reversibility**: Irreversible actions (deleting files, sending emails, making purchases) always warrant approval. Reversible actions (reading files, searching the web) generally do not.
- **Scope**: Actions affecting external systems or other people warrant more scrutiny than purely local actions.
- **Confidence**: When the agent's confidence in its interpretation of the user's intent is low, it should ask for clarification rather than proceed.
- **Cost**: High-cost actions (large API calls, expensive computations) warrant approval.
- **Novelty**: Actions the agent has not taken before in this context warrant more scrutiny than routine actions.

28.7.2 Tiered Approval Workflows

A tiered approval policy balances oversight with efficiency:

Three-Tier Approval Model

Tier 1 (Auto-approve): Low-risk, reversible, routine actions. Examples: web search, reading files, calling read-only APIs. The agent proceeds without interruption; actions are logged for audit.
Tier 2 (Notify): Medium-risk actions. The UI shows a non-blocking notification ("Agent sent a draft email to your Drafts folder") that the user can review asynchronously. A brief window (e.g., 30 seconds) allows cancellation before the action is finalized.
Tier 3 (Require approval): High-risk, irreversible, or high-cost actions. The agent pauses and presents a blocking approval gate. The user must explicitly approve, reject, or modify before the agent continues.

The thresholds between tiers can be configured by the user ("always ask before sending emails") or learned from user behavior (if the user always approves web searches, auto-approve them in the future).

28.7.3 Feedback Mechanisms

Beyond approval gates, agentic UIs should provide rich feedback mechanisms that help the agent improve over time:

- **Thumbs up/down**: Simple binary feedback on responses, stored and used for RLHF fine-tuning or preference learning.
- **Inline corrections**: Users can directly edit agent outputs; the delta between the original and corrected output is a training signal.
- **Preference selection**: When the agent offers multiple options, the user's selection is a preference signal.
- **Explicit instruction**: "Don't do this again", "Always ask before X", "Prefer approach Y over Z"—natural language instructions that update the agent's behavioral policy.
- **Rating with rationale**: Optional free-text explanation accompanying a rating, providing richer signal than binary feedback.

28.7.4 Teaching the Agent Through UI Interaction

The most sophisticated HITL UIs treat every interaction as a teaching opportunity:

- **Demonstration**: The user performs a task manually; the agent observes and learns the preferred approach.
- **Correction with generalization**: When the user corrects an agent action, the UI asks "Should I always do this differently?" to generalize the correction.
- **Preference elicitation**: Periodic prompts asking the user to compare two agent behaviors and indicate which is preferred.
- **Behavioral profiles**: The UI maintains a visible "preferences" profile that the user can review and edit, making the agent's learned behaviors transparent and controllable.

28.8 Accessibility and Trust

Trust is not a feature—it is an emergent property of a system that consistently behaves as expected, explains itself clearly, and recovers gracefully from failures. Agentic UIs must be designed with trust as a first-class concern.

28.8.1 Explaining Agent Decisions

Explainability in agentic UIs goes beyond showing chain-of-thought. It requires:

- **Decision rationale**: For consequential decisions, the agent should explain not just *what* it decided but *why*—which factors were considered, what alternatives were rejected, and what assumptions were made.
- **Source attribution**: Claims should be linked to their sources; retrieved documents should be citable.
- **Counterfactual explanations**: "If you had said X instead of Y, I would have done Z"—helping users understand the agent's decision boundary.
- **Uncertainty quantification**: Explicit statements of confidence, with the factors driving uncertainty.

28.8.2 Showing Confidence Levels

Confidence indicators must be calibrated and meaningful:

- **Verbal confidence**: Natural language expressions ("I'm fairly confident", "I'm not sure about this") are more interpretable than numerical probabilities for most users.
- **Visual confidence**: Color coding (green/yellow/red), icon variants, or font weight can encode confidence without adding text.
- **Confidence by claim**: For multi-claim responses, per-claim confidence indicators (e.g., inline footnotes) are more informative than a single response-level score.

28.8.3 Undo and Rollback Capabilities

Every consequential agent action should be undoable where technically feasible:

- **Action log with undo**: A chronological log of all agent actions with an "Undo" button for each reversible action.
- **Snapshot-based rollback**: For stateful tasks (e.g., code editing, document writing), periodic snapshots enable rollback to any prior state.
- **Dry-run mode**: Before executing a plan, the agent can simulate it and show the predicted state changes, allowing the user to approve or modify before any real action is taken.
- **Graceful degradation**: When an undo is not possible (e.g., an email has been sent), the UI clearly communicates this and offers the best available alternative (e.g., sending a follow-up).

28.8.4 Audit Trails in the UI

For enterprise and regulated use cases, audit trails are essential:

- **Immutable action log**: Every agent action, tool call, and human approval is logged with timestamp, user identity, and full parameters.
- **Exportable history**: The audit trail can be exported as JSON, CSV, or PDF for compliance reporting.
- **Diff views**: For document or code modifications, the audit trail includes before/after diffs.
- **Session replay**: The ability to replay an entire agent session, step by step, for debugging or compliance review.

28.8.5 Managing User Expectations

Miscalibrated expectations are a primary source of user distrust. Agentic UIs should actively manage expectations:

- **Capability disclosure**: Clear, accessible documentation of what the agent can and cannot do.
- **Limitation acknowledgment**: When the agent encounters a task outside its capabilities, it says so clearly rather than attempting and failing silently.
- **Uncertainty communication**: Proactive communication of uncertainty, rather than waiting for the user to discover errors.
- **Consistent persona**: A consistent agent identity and communication style builds familiarity and predictability.

Trust-Building Through Transparency: A Case Study

Consider an agent tasked with booking a flight. A low-trust UI presents: "I've booked your flight. Confirmation: AA1234." A high-trust UI presents: (1) a summary of the search parameters used, (2) the alternatives considered and why this flight was selected, (3) the exact actions taken (API calls to the booking system), (4) the confirmation details with a link to the booking, (5) an undo option valid for the next 24 hours, and (6) a note about what the agent cannot do (e.g., "I cannot modify this booking; you'll need to call the airline directly"). The second UI takes more screen space but builds the user's confidence that the agent acted correctly and gives them the information needed to verify and recover if needed.

28.9 Implementation Example: A Full-Stack Agentic UI

We now present a concrete implementation example combining streaming, tool visualization, and approval gates in a Python/React stack. The backend uses FastAPI with LangGraph; the frontend uses React with the Vercel AI SDK patterns adapted for a custom backend.

28.9.1 Backend: FastAPI + LangGraph with Streaming and Approval Gates

```python
# backend/main.py
import asyncio
import json
from typing import AsyncGenerator
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

app = FastAPI()

# -- Tool definitions ------------------------------------------------------

@tool
def web_search(query: str) -> str:
    """Search the web for information."""
    return f"Search results for '{query}': [simulated results]"

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email. REQUIRES HUMAN APPROVAL."""
    return f"Email sent to {to} with subject '{subject}'"

@tool
def read_file(path: str) -> str:
    """Read a file from the filesystem."""
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: File not found: {path}"

# Tools requiring approval (Tier 3)
APPROVAL_REQUIRED_TOOLS = {"send_email"}

# -- Approval gate store (in-memory; use Redis in production) --------------

approval_store: dict[str, asyncio.Event] = {}
approval_results: dict[str, dict] = {}

# -- LLM setup -------------------------------------------------------------

llm = ChatOpenAI(model="gpt-4o", streaming=True)
tools = [web_search, send_email, read_file]
llm_with_tools = llm.bind_tools(tools)

def should_request_approval(tool_name: str) -> bool:
    return tool_name in APPROVAL_REQUIRED_TOOLS

# -- Streaming endpoint ----------------------------------------------------

async def agent_stream(
    session_id: str,
    user_message: str,
) -> AsyncGenerator[str, None]:
    """Stream agent events as SSE."""
```

```
    def sse(event_type: str, data: dict) -> str:
        return f"data: {json.dumps({'type': event_type, **data})}\n\n"

    yield sse("status", {"message": "Agent starting..."})

    # Simulate multi-step agent execution
    steps = [
        ("thinking", {"content": "Analyzing the request..."}),
        ("tool_call", {
            "tool": "web_search",
            "input": {"query": user_message},
            "tier": 1,  # Auto-approve
        }),
        ("tool_result", {
            "tool": "web_search",
            "output": f"Results for: {user_message}",
            "duration_ms": 342,
        }),
    ]

    for event_type, data in steps:
        await asyncio.sleep(0.5)  # Simulate processing time
        yield sse(event_type, data)

    # Simulate a Tier 3 action requiring approval
    approval_id = f"{session_id}_email_001"
    approval_event = asyncio.Event()
    approval_store[approval_id] = approval_event

    yield sse("approval_required", {
        "approval_id": approval_id,
        "tool": "send_email",
        "tier": 3,
        "risk": "irreversible",
        "action_summary": "Send summary email to user@example.com",
        "parameters": {
            "to": "user@example.com",
            "subject": f"Research results: {user_message}",
            "body": "Here are the findings...",
        },
    })

    # Wait for human approval (timeout after 5 minutes)
    try:
        await asyncio.wait_for(approval_event.wait(), timeout=300)
        result = approval_results.get(approval_id, {})

        if result.get("approved"):
            yield sse("tool_call", {
                "tool": "send_email",
                "input": result.get("parameters", {}),
                "tier": 3,
                "approved_by": "human",
            })
            await asyncio.sleep(0.3)
            yield sse("tool_result", {
                "tool": "send_email",
                "output": "Email sent successfully",
                "duration_ms": 128,
            })
        else:
            yield sse("action_rejected", {
                "tool": "send_email",
                "reason": result.get("reason", "User rejected"),
            })
```

```
    except asyncio.TimeoutError:
        yield sse("approval_timeout", {
            "approval_id": approval_id,
            "message": "Approval timed out; action skipped",
        })

    # Final response
    yield sse("token", {"content": "I've completed the research. "})
    yield sse("token", {"content": "Here's a summary of what I found..."})
    yield sse("done", {"total_tokens": 847, "duration_ms": 2341})

@app.get("/chat/stream")
async def chat_stream(session_id: str, message: str):
    return StreamingResponse(
        agent_stream(session_id, message),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )

class ApprovalRequest(BaseModel):
    approval_id: str
    approved: bool
    parameters: dict | None = None
    reason: str | None = None

@app.post("/chat/approve")
async def handle_approval(req: ApprovalRequest):
    if req.approval_id not in approval_store:
        raise HTTPException(status_code=404, detail="Approval not found")
    approval_results[req.approval_id] = {
        "approved": req.approved,
        "parameters": req.parameters,
        "reason": req.reason,
    }
    approval_store[req.approval_id].set()
    return {"status": "ok"}
```

Listing 28.4: FastAPI backend with streaming and approval gates

28.9.2 Frontend: React with Streaming and Tool Visualization

```
// frontend/AgentChat.tsx
import { useState, useEffect, useRef } from 'react';

// -- Types ---------------------------------------------------------------

type AgentEvent =
  | { type: 'status'; message: string }
  | { type: 'thinking'; content: string }
  | { type: 'token'; content: string }
  | { type: 'tool_call'; tool: string; input: object; tier: number }
  | { type: 'tool_result'; tool: string; output: string; duration_ms: number }
  | { type: 'approval_required'; approval_id: string; tool: string;
      tier: number; risk: string; action_summary: string; parameters: object }
  | { type: 'action_rejected'; tool: string; reason: string }
  | { type: 'done'; total_tokens: number; duration_ms: number };

// -- Tool Card Component -------------------------------------------------

function ToolCard({ event }: { event: AgentEvent & { type: 'tool_call' } }) {
  const [expanded, setExpanded] = useState(false);
  const tierColors = { 1: '#22c55e', 2: '#f59e0b', 3: '#ef4444' };
  const color = tierColors[event.tier as keyof typeof tierColors] || '#6b7280';

  return (
```

```
    <div style={{ border: `1px solid ${color}`, borderRadius: 8, padding: 8,
                  margin: '4px 0', fontSize: 13 }}>
      <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
        <span style={{ color, fontWeight: 600 }}>[gear] {event.tool}</span>
        <span style={{ color: '#6b7280', fontSize: 11 }}>
          Tier {event.tier} | {event.tier === 1 ? 'Auto' : 'Approved'}
        </span>
        <button onClick={() => setExpanded(!expanded)}
                style={{ marginLeft: 'auto', fontSize: 11 }}>
          {expanded ? 'Hide' : 'Details'}
        </button>
      </div>
      {expanded && (
        <pre style={{ marginTop: 8, fontSize: 11, background: '#f3f4f6',
                      padding: 8, borderRadius: 4, overflow: 'auto' }}>
          {JSON.stringify(event.input, null, 2)}
        </pre>
      )}
    </div>
  );
}

// -- Approval Gate Component ---------------------------------------------------

function ApprovalGate({
  event,
  onDecision,
}: {
  event: AgentEvent & { type: 'approval_required' };
  onDecision: (approved: boolean, params?: object) => void;
}) {
  const riskColors = { reversible: '#22c55e', 'hard-to-undo': '#f59e0b',
                       irreversible: '#ef4444' };
  const riskColor = riskColors[event.risk as keyof typeof riskColors] || '#6b7280'
    ;

  return (
    <div style={{ border: `2px solid ${riskColor}`, borderRadius: 8,
                  padding: 16, margin: '8px 0', background: '#fef9f0' }}>
      <div style={{ fontWeight: 700, color: riskColor, marginBottom: 8 }}>
        [!] Approval Required: {event.tool}
      </div>
      <div style={{ marginBottom: 8 }}>{event.action_summary}</div>
      <div style={{ fontSize: 12, color: '#6b7280', marginBottom: 12 }}>
        Risk level: <span style={{ color: riskColor }}>{event.risk}</span>
      </div>
      <div style={{ display: 'flex', gap: 8 }}>
        <button
          onClick={() => onDecision(true, event.parameters)}
          style={{ background: '#22c55e', color: 'white', border: 'none',
                   borderRadius: 6, padding: '8px 16px', cursor: 'pointer' }}>
          [ok] Approve
        </button>
        <button
          onClick={() => onDecision(false)}
          style={{ background: '#ef4444', color: 'white', border: 'none',
                   borderRadius: 6, padding: '8px 16px', cursor: 'pointer' }}>
          [x] Reject
        </button>
      </div>
    </div>
  );
}

// -- Main Chat Component -------------------------------------------------------
```

```
export function AgentChat() {
  const [events, setEvents] = useState<AgentEvent[]>([]);
  const [response, setResponse] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [input, setInput] = useState('');
  const sessionId = useRef(`session_${Date.now()}`);

  const sendMessage = async () => {
    if (!input.trim() || isStreaming) return;
    setEvents([]);
    setResponse('');
    setIsStreaming(true);

    const url = `/chat/stream?session_id=${sessionId.current}`
              + `&message=${encodeURIComponent(input)}`;
    const es = new EventSource(url);

    es.onmessage = (e) => {
      const event: AgentEvent = JSON.parse(e.data);
      if (event.type === 'token') {
        setResponse(prev => prev + event.content);
      } else if (event.type === 'done') {
        setIsStreaming(false);
        es.close();
      } else {
        setEvents(prev => [...prev, event]);
      }
    };

    es.onerror = () => { setIsStreaming(false); es.close(); };
    setInput('');
  };

  const handleApproval = async (
    approvalId: string,
    approved: boolean,
    parameters?: object,
  ) => {
    await fetch('/chat/approve', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ approval_id: approvalId, approved, parameters }),
    });
  };

  return (
    <div style={{ maxWidth: 800, margin: '0 auto', padding: 16 }}>
      <div style={{ minHeight: 400, border: '1px solid #e5e7eb',
                    borderRadius: 8, padding: 16, marginBottom: 16 }}>
        {events.map((event, i) => {
          if (event.type === 'tool_call')
            return <ToolCard key={i} event={event} />;
          if (event.type === 'approval_required')
            return (
              <ApprovalGate key={i} event={event}
                onDecision={(approved, params) =>
                  handleApproval(event.approval_id, approved, params)} />
            );
          if (event.type === 'status' || event.type === 'thinking')
            return (
              <div key={i} style={{ color: '#6b7280', fontSize: 12,
                                    fontStyle: 'italic', margin: '4px 0' }}>
                {event.type === 'thinking' ? event.content : event.message}
              </div>
            );
          return null;
```

```
        })}
        {response && (
          <div style={{ marginTop: 8, lineHeight: 1.6 }}>
            {response}
            {isStreaming && <span className="cursor-blink">|</span>}
          </div>
        )}
      </div>
      <div style={{ display: 'flex', gap: 8 }}>
        <input
          value={input}
          onChange={e => setInput(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && sendMessage()}
          placeholder="Ask the agent..."
          style={{ flex: 1, padding: '8px 12px', borderRadius: 6,
                   border: '1px solid #d1d5db', fontSize: 14 }}
        />
        <button onClick={sendMessage} disabled={isStreaming}
                style={{ padding: '8px 16px', background: '#3b82f6',
                         color: 'white', border: 'none', borderRadius: 6,
                         cursor: isStreaming ? 'not-allowed' : 'pointer' }}>
          {isStreaming ? 'Running...' : 'Send'}
        </button>
      </div>
    </div>
  );
}
```

Listing 28.5: React frontend with streaming tool visualization and approval gates

What This Implementation Demonstrates

The code above illustrates several key agentic UI patterns working together:

- **SSE streaming**: The backend streams events of different types (status, thinking, tool calls, tokens) over a single HTTP connection.
- **Typed event protocol**: A discriminated union of event types enables the frontend to render each event appropriately.
- **Tool visualization**: `ToolCard` renders tool calls with tier indicators and expandable input details.
- **Approval gates**: `ApprovalGate` blocks the stream and waits for human input before the agent proceeds with irreversible actions.
- **Async approval**: The backend uses `asyncio.Event` to pause the stream while waiting for the frontend's approval POST request, cleanly decoupling the approval UI from the streaming logic.

28.10 Summary

Agentic UI frameworks represent a new frontier in human-computer interaction, demanding a rethinking of interface design from first principles. The key insights from this section are:

1. **Paradigm selection matters**: The appropriate UI paradigm (chat, canvas, workflow, dashboard, collaborative, autonomous) depends on task structure, required human involvement, and output type. Most production systems combine multiple paradigms.
2. **Transparency is non-negotiable**: Users cannot trust what they cannot see. Thought display, tool visualization, and context panels are not optional features—they are the foundation of trustworthy agentic systems.

3. **Streaming is the baseline**: Users expect to see agents work in real time. Token streaming, tool call streaming, and multi-agent streaming are table-stakes capabilities.

4. **Approval gates must be tiered**: Flat approval policies (approve everything or approve nothing) fail in practice. Tiered policies that auto-approve safe actions and gate dangerous ones maintain oversight without creating bottlenecks.

5. **Generative UI is the frontier**: The ability for LLMs to generate not just text but UI components—charts, forms, maps, widgets—enables interfaces that adapt to content rather than forcing content into a fixed template.

6. **Trust is earned through consistency and recoverability**: Undo capabilities, audit trails, and calibrated confidence indicators are as important as raw capability for building user trust.

Design Principle: The Agent as a Transparent Collaborator

The north star for agentic UI design is the *transparent collaborator*: an agent whose actions are always visible, whose reasoning is always accessible, whose mistakes are always recoverable, and whose capabilities and limitations are always clear. Every UI decision should be evaluated against this standard.

The frameworks and patterns described in this section—Vercel AI SDK, Chainlit, Gradio, Streamlit, LangGraph Studio—provide the building blocks. The challenge for practitioners is to combine them thoughtfully, guided by the specific needs of their users and the specific risks of their domain.

Part VI

Assessment & Reference

Chapter 29

Quiz Questions & Detailed Answers

This chapter provides a comprehensive set of questions designed to test and reinforce your understanding of the material covered throughout this guide. Each question targets a key concept, algorithm, or system design decision—the kind of knowledge that distinguishes surface-level familiarity from genuine expertise. Use these questions for self-assessment: attempt your own answer before reading the detailed solution. The questions progress from foundational concepts (LLM architecture, reinforcement learning basics) through core algorithms (PPO, DPO, GRPO) to advanced system design and agentic AI topics.

29.1 Foundations Questions

Q0a: What is the role of the attention mechanism in a decoder-only Transformer? Why is it causal?

Answer: The attention mechanism allows each token to attend to (i.e., compute a weighted combination of) representations from other tokens. In a decoder-only Transformer, attention is **causal** (also called *autoregressive*): token t can only attend to tokens $1, \ldots, t$, never to future tokens $t+1, \ldots, T$.

Why causal? Because the model generates text left-to-right. At inference time, future tokens literally do not exist yet. The causal mask during training simulates this constraint so that the model learns to predict each token using only its left context. Mathematically, the attention matrix is masked:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where $M_{ij} = -\infty$ for $j > i$ (future positions), forcing those attention weights to zero.

Practical implication: This enables the KV-cache optimization at inference—since past tokens' keys and values never change, they can be cached and reused, reducing generation from $O(T^2)$ to $O(T)$ per new token.

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q0b: Explain Flash Attention. What problem does it solve and how?

Answer: Standard attention computes the full $T \times T$ attention matrix, which requires $O(T^2)$ memory and is **memory-bandwidth bound**—the GPU spends most of its time moving data between HBM (slow, large) and SRAM (fast, small), not doing actual computation.

Flash Attention's insight: Never materialize the full attention matrix in HBM. Instead, tile the computation into blocks that fit in SRAM, compute attention *block by block* using an online softmax algorithm, and write only the final output to HBM.

Key techniques:

1. **Tiling**: Split Q, K, V into blocks of size $B_r \times B_c$ that fit in SRAM
2. **Online softmax**: Track running max and sum to compute softmax incrementally without

the full row

3. **Recomputation**: In the backward pass, recompute attention from Q, K, V (cheap) rather than storing the $T \times T$ matrix (expensive)

Result: $O(T)$ HBM memory (instead of $O(T^2)$), 2–4× wall-clock speedup, exact same numerical output (not an approximation).
Review: *Chapters 1–2 (LLM Architecture; Systems Foundations).*

Q0c: What is the difference between SFT, RLHF, and DPO at a high level? When do you use each?

Answer:

- **SFT (Supervised Fine-Tuning)**: Train the model to imitate high-quality demonstrations. Loss: next-token prediction on curated data. Teaches *format* and *style*.
- **RLHF**: Train a reward model from human preferences, then optimize the policy against it using RL (PPO). The model explores beyond the demonstration data. Teaches *what humans prefer*.
- **DPO**: Skip the reward model. Directly optimize the policy on preference pairs (y_w, y_l) using a contrastive loss. Same goal as RLHF but simpler pipeline.

Typical pipeline: SFT first (gives the model a good starting point), then RLHF or DPO (refines preferences). SFT alone tends to produce verbose, hedge-heavy responses. RLHF/DPO makes outputs more direct and aligned with human intent.
When to use each: SFT when you have gold-standard outputs. DPO when you have preference pairs but limited compute. RLHF (PPO) when you need maximum quality and can afford the infrastructure.
Review: *Chapters 5, 6, and 10 (PPO; DPO; SFT Best Practices).*

Q0d: What is a reward model? How is it trained and what can go wrong?

Answer: A reward model (RM) is a neural network that takes a (prompt, response) pair and outputs a scalar score indicating quality. It is trained on human preference data: given pairs (y_w, y_l) where y_w is preferred, the RM learns to assign $R(y_w) > R(y_l)$.
Training: Bradley-Terry loss: $\mathcal{L} = -\log \sigma(R(y_w) - R(y_l))$. Architecture: typically the same transformer as the policy, with the LM head replaced by a scalar projection.
What can go wrong:

1. **Reward hacking**: The policy finds outputs that score high on the RM but are actually low quality (e.g., excessively long, repetitive, or containing specific phrases the RM was biased toward)
2. **Distribution shift**: The RM was trained on outputs from an earlier policy. As training progresses, the current policy generates out-of-distribution outputs the RM cannot score accurately
3. **Label noise**: Human annotators disagree, are tired, or apply inconsistent criteria. This noise propagates into RM predictions
4. **Overconfidence**: The RM assigns extreme scores to outputs it has never seen, providing misleading gradient signal

Review: *Chapter 9 (Reward Model Training).*

Q0e: Explain the exploration-exploitation trade-off in RL. How does it manifest in LLM training?

Answer: In RL, an agent must balance:

- **Exploitation**: choosing actions known to yield high reward (greedy behavior)
- **Exploration**: trying new actions that might yield even higher reward (but might also fail)

In LLM training: The policy is the language model. "Actions" are token choices. "Exploitation" means generating responses similar to what already scored well. "Exploration" means trying novel phrasings, structures, or reasoning paths.
How it manifests:

- **Temperature during generation**: Higher temperature = more exploration. GRPO uses temperature 1.0 to get diverse samples within each group.
- **KL penalty**: Acts as an anti-exploration brake—prevents the policy from straying too far from the reference model. Without it, the policy might collapse to a single high-reward template (mode collapse).
- **Group sampling in GRPO**: Generating G responses per prompt explicitly explores the output space, then reinforces above-average responses.

The tension: Too little exploration → the model gets stuck in local optima (always giving the same safe answer). Too much → training is unstable, quality fluctuates wildly.
Review: *Chapters 3 and 7 (Introduction to RL; GRPO).*

29.2 Core Algorithm Questions

Q1: Explain PPO's clipped objective. Why does it work better than vanilla PG?

Answer: Vanilla policy gradient: $\nabla J = \mathbb{E}[\nabla \log \pi(a|s) \cdot \hat{A}]$. Problem: one lucky/unlucky sample can produce a huge gradient → policy jumps to a bad region → generates garbage → next gradient makes it worse → unrecoverable "death spiral."
PPO's solution: Clip the probability ratio $r = \pi_{\text{new}}/\pi_{\text{old}}$ to $[0.8, 1.2]$.
Mechanics: For good actions ($\hat{A} > 0$): objective is $\min(r\hat{A}, 1.2\hat{A})$. Once r exceeds 1.2, no further benefit — stops the policy from over-committing to one example. For bad actions ($\hat{A} < 0$): objective is $\min(r\hat{A}, 0.8\hat{A})$. Once r drops below 0.8, penalty stops growing — prevents catastrophic forgetting.
Key insight: It's a first-order approximation of TRPO's KL constraint, but without expensive second-order optimization. Each update changes the policy by at most $\pm 20\%$.
For LLMs specifically: The token-level ratio $r_t = \pi_\theta(y_t|y_{<t})/\pi_{\text{old}}(y_t|y_{<t})$ prevents any single token's probability from changing too drastically, preserving coherent generation.
Review: *Chapter 5 (PPO).*

Q2: Derive DPO from first principles. What assumptions does it make?

Answer: Start with RLHF objective: $\max_\pi \mathbb{E}[r(x,y)] - \beta D_{\text{KL}}[\pi \| \pi_{\text{ref}}]$.
Step 1: Write the KKT conditions. Optimal policy has closed form: $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r(x,y)/\beta)$.
Step 2: Invert to express reward: $r(x,y) = \beta \log(\pi^*/\pi_{\text{ref}}) + \beta \log Z(x)$.
Step 3: Substitute into Bradley-Terry model $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$. The partition function $Z(x)$ cancels (same prompt).
Step 4: Replace π^* with π_θ (parameterized policy we're training): $\mathcal{L} = -\mathbb{E}[\log \sigma(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)})]$.

Assumptions:

1. Bradley-Terry preference model (pairwise, no ties, transitive)
2. Optimal policy is achievable by π_θ (sufficient capacity)
3. Preferences are generated from the same distribution as training data (no distribution shift)
4. Reference model is fixed and reasonable

When assumptions break: Real preferences aren't transitive, data shifts during training, labels are noisy → that's why Online DPO and IPO exist.
Review: *Chapter 6 (DPO).*

Q3: GRPO vs PPO — when would you choose each? What's the trade-off?

Answer:
GRPO advantages:

- No value function needed: saves one model's worth of memory and complexity
- Simpler: fewer hyperparameters, more intuitive (above-mean = good, below-mean = bad)
- Better for verifiable rewards: math/code where $r \in \{0, 1\}$ gives crisp signal
- DeepSeek-R1 proved it can teach emergent reasoning with just binary rewards

PPO advantages:

- Per-token credit assignment: value function assigns reward to each token, not just sequence-level
- More sample efficient: GAE uses value predictions to estimate advantage without generating G samples
- Better for nuanced rewards: when reward is continuous and varies significantly across tokens
- More mature: battle-tested at OpenAI, Anthropic, etc.

Rule of thumb: If rewards are verifiable (right/wrong) → GRPO. If rewards are nuanced (RM scores) and you need max quality → PPO. If compute is limited → GRPO (no critic training).
Compute comparison: GRPO generates G responses per prompt (8× more generation), but skips value function training. Net: similar total compute but distributed differently (more gen, less training).
Review: *Chapters 5 and 7 (PPO; GRPO).*

Q4: How does GAE work? Walk through a concrete example for LLMs.

Answer: GAE = weighted sum of n-step TD errors: $\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$.
Concrete example: Response has 5 tokens. Reward only at end ($r_5 = 0.8$). Value predictions: $V_1 = 0.5, V_2 = 0.55, V_3 = 0.6, V_4 = 0.65, V_5 = 0.7$.
TD errors ($\gamma = 1$): $\delta_1 = 0 + V_2 - V_1 = 0.05$, $\delta_2 = 0 + V_3 - V_2 = 0.05$, ..., $\delta_5 = 0.8 + 0 - 0.7 = 0.1$.
With $\lambda = 0.95$: $\hat{A}_5 = 0.1$ (just the final TD error), $\hat{A}_4 = 0.05 + 0.95 \times 0.1 = 0.145$, $\hat{A}_3 = 0.05 + 0.95 \times 0.145 = 0.188$, etc.
Interpretation: Token 3 gets advantage 0.188 because it contributed to a sequence that got higher reward than expected. Earlier tokens get credit through the exponential decay.
For LLMs: $\gamma = 1.0$ (all tokens matter, finite horizon). The advantage at token t answers: "given what followed this token, was this token choice better or worse than expected?"
Review: *Chapter 5 (PPO).*

Q5: How do you prevent reward hacking? Give a layered defense strategy.

Answer: **Detection signals**: RM score rising but win-rate flat/declining. Response length growing monotonically. KL divergence > 15. Diversity (unique n-grams) dropping. Reading high-reward outputs reveals exploits.
Layered defense (in priority order):

1. **KL penalty** (primary): Adaptive controller targets KL ≈ 6. If KL rises, β increases automatically. Prevents drifting too far from reference.
2. **Reward model ensemble** (3–5 models): Use min or mean of scores. Individual models have different blind spots — exploits that fool one rarely fool all.
3. **Length penalty**: $r' = r - c \cdot \max(0, \text{length} - L_{\text{target}})$. Prevents "just generate longer = higher score" exploit.
4. **Periodic RM refresh**: Every 2000 steps, generate data from current policy, relabel, add to RM training set. Closes the exploit as model finds it.
5. **Win-rate based stopping**: Track win-rate against SFT baseline. If RM score rises but win-rate stalls for 200+ steps, stop training. The model is exploiting, not improving.

Post-detection recovery: Roll back to last "clean" checkpoint. Increase β by 2×. Add the discovered exploit to the RM's negative examples.
Review: *Chapters 9 and 11 (Reward Model Training; System Architecture).*

29.3 System Design Questions

Q6: Design an RLHF system for training a 70B model. Walk through every component.

Answer: Three-cluster decoupled architecture on 72 A100-80GB GPUs:
Cluster 1 — Generation (32 GPUs):

- 8 vLLM instances, TP=4 each. PagedAttention + speculative decoding (1B draft model).
- Continuous batching, max 256 sequences in flight. INT8 weights for bandwidth.
- Output: (prompt, response, per-token log-probs). Throughput: ∼500 responses/minute.
- Stateless: just loads latest weights from shared store. Trivial recovery.

Cluster 2 — Scoring (8 GPUs):

- Reward model (70B, INT8 = 70GB) on 4 GPUs (TP=4).
- Reference model (70B, INT8) on 4 GPUs (TP=4). Computes per-token log-probs for KL.
- Output: reward scores + KL per token. Lightweight, batch inference.

Cluster 3 — Training (32 GPUs):

- Policy model with FSDP (ZeRO-3). Flash Attention 2. Gradient checkpointing.
- Consumes scored experiences from buffer. PPO update: 4 epochs on mini-batch of 16.
- Pushes updated weights to shared store every 50 steps (async, background transfer).
- Async checkpoint every 100 steps (non-blocking write to NVMe + S3 backup).

Connection fabric:

- Gen → Score: Ray/Redis queue (∼10 MB per batch: token IDs + log-probs)
- Score → Train: Experience buffer (circular, holds last 500 steps)
- Train → Gen: Weight store on shared parallel FS (Lustre/GPFS). 140GB push every 50 steps, async.

Overlap: While training processes step N, generation is already producing data for step $N+1$. This hides generation latency, achieving 1.3–1.5× throughput vs monolithic.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q7: How do you handle the generation bottleneck? Quantify the solutions.

Answer: Generation = 60–70% of total RLHF wall-clock time. Root cause: autoregressive decoding is memory-bandwidth bound (arithmetic intensity ≈ 1 FLOP/byte vs roofline of 156 FLOP/byte on A100).
Solutions ranked by impact:
1. Decouple gen from training (1.3–1.5× end-to-end): Run generation on separate hardware, overlap with training. The single biggest architectural win.
2. vLLM with PagedAttention (2–4×): Eliminates 60–80% KV cache memory waste from internal fragmentation. Enables 3–4× larger batches = better bandwidth utilization.
3. Continuous batching (1.5–2×): Don't wait for longest sequence to finish. Start new sequences immediately in freed slots. Keeps GPUs busy.
4. Speculative decoding (2–3×): 1B draft model proposes 5 tokens. 70B model verifies all 5 in one forward pass (parallel!). Accept 3–4 on average → 3–4 tokens per forward pass instead of 1.
5. INT8/FP8 for generation weights (2×): Halves the 140GB weight read per token. Quality loss is minimal because (a) we're sampling with temperature anyway, and (b) only generation uses INT8, training stays BF16.
6. CUDA graphs + kernel fusion (1.1–1.3×): Eliminate Python/CUDA launch overhead. Fuse layernorm+attention+MLP into fewer kernels.
Combined: $1.5 \times 3 \times 1.5 \times 2.5 \times 2 \times 1.2 = 40\times$ over naive. In practice, diminishing returns limit total to ∼10–20× over naive implementation.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

Q8: Explain weight synchronization in a decoupled system. How much staleness can you tolerate?

Answer:
The problem: Generation cluster uses policy weights to produce responses. Training cluster updates those weights. They're on different hardware. How to keep them in sync?
Why perfect sync is wasteful: Full sync of 70B BF16 = 140GB. At InfiniBand 400Gb/s (50GB/s): 2.8s per sync. If you sync every step (every 50–90s), you spend 3–5% of time on weight transfer. Acceptable, but unnecessary.
Staleness tolerance analysis:

- Per-step policy change: ∼0.1% (measured by mean param delta)
- 50 steps: ∼5% cumulative drift
- PPO clip range: handles up to 20% probability ratio deviation
- Empirical: 50-step staleness → <2% quality degradation (measured by win-rate)

Production strategy:

1. Every 50 training steps: push full BF16 checkpoint to shared store (2.8s transfer)
2. Generation cluster: non-blocking weight reload between batches

3. Delta compression (optional): Only send changed parameters (INT8 delta ≈ 5GB), apply as offset. 10× less bandwidth.

4. For very large scale (256+ GPUs): streaming sync — continuously send small chunks in background. Average staleness: 5–10 steps.

Important subtlety: Log-probs computed during generation use stale weights. The PPO ratio $\pi_{\text{new}}/\pi_{\text{old}}$ computes π_{old} using these stale log-probs. This is fine because PPO is designed for off-policy corrections.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q9: How would you make this fault-tolerant at 512 GPUs?

Answer: At 512 GPUs, MTBF is 4–8 hours. A 5-day training run will see 15–30 failures.
Architecture-level resilience:

- Generation cluster = stateless. Failed instance restarts in <60s (just load weights, no state).
- Training cluster = stateful. Needs checkpoint-based recovery.
- Scoring cluster = stateless. Same as generation.

Checkpointing strategy:

- Frequency: Every 50–100 steps (5–10 min of training).
- Method: Async (non-blocking). Background thread writes while next step proceeds. Uses FSDP's distributed save (each rank saves its shard in parallel).
- Contents: Model weights, optimizer states (Adam m/v), LR scheduler, RNG states, KL adaptive coefficient, global step counter, replay buffer pointer.
- Storage: Local NVMe (fast, 30s for 70B) + async copy to S3/shared FS (durable).
- Retention: Keep last 3 checkpoints. Auto-delete older ones.

Detection and recovery flow:

1. NCCL collective timeout (60s) or heartbeat miss (10s) → failure detected.
2. Identify failed node(s) via NVML health check.
3. Option A (fast, <2 min): Torch Elastic shrinks world size, redistributes shards, continue with $N - 1$ nodes. Request replacement in background.
4. Option B (clean, ∼5 min): Bring up replacement node, rebuild process group, load last checkpoint, resume.
5. Experience buffer is persisted — no regeneration needed.

Prevention: Pre-screening stress test (GEMM, memory, NVLink). ECC error monitoring (preemptive migration if errors spike). Hot spare nodes (pre-loaded with environment). Dual-rail InfiniBand for network redundancy.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q10: How do you scale from 7B to 70B to 405B? What changes at each scale?

Answer:
7B (single 8-GPU node, hours):

- Architecture: Monolithic (TRL default). All models on same GPUs.
- Memory: LoRA + INT8 ref/RM fits in 8×80GB easily.
- Parallelism: DP=8 or FSDP within node. No network communication.
- Hyperparams: LR=5×10^{-6}, aggressive β=0.02, 50K–100K steps.
- Time: 4–12 hours per run. Fast iteration.

70B (32–64 GPUs, 2–5 days):

- Architecture: Semi-decoupled. vLLM generation + FSDP training.
- Memory: ZeRO-3 essential. Gradient checkpointing. INT8 ref/RM.
- Parallelism: TP=8 intra-node (gen), FSDP inter-node (training).
- Hyperparams: LR=1.5×10^{-6}, moderate β=0.05, 10K–30K steps.
- Fault tolerance: Async checkpoints, monitoring, but manageable manually.

405B (256–512 GPUs, 1–3 weeks):

- Architecture: Fully decoupled. Separate clusters. Weight store + queue.
- Memory: ZeRO-3 + TP=8 + PP=2 for training. INT4 generation.
- Parallelism: 3D parallelism (TP×PP×DP = 8×2×16 = 256 GPUs training).
- Hyperparams: LR=5×10^{-7}, very conservative β=0.1, 2K–5K steps.
- Fault tolerance: Mandatory. Elastic training, redundant checkpoints, hot spares.
- Key change: Much less RL training needed (model is already very capable from pretraining). But each step is 50× more expensive, so instability is catastrophic.

Paradox: Larger models are actually *easier to RL-train per step* (more stable, smoother loss landscape). But the cost of instability scales with model size — a bad run at 405B wastes $100K+ in compute.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

29.4 Practical and Debugging Questions

Q11: Reward score is increasing but model quality is declining. Diagnose and fix.

Answer: Classic **reward hacking / Goodhart's Law**.
Diagnostic protocol:

1. **Check response length**: Plot mean length over training. Growing monotonically? = Length exploit (RM gives higher scores to longer responses).
2. **Check KL divergence**: >15? = Policy has diverged too far from reference. Lost capabilities.
3. **Check diversity**: Unique trigrams per response. Dropping? = Mode collapse (repeating same high-reward pattern).
4. **Manual inspection**: Read 20 highest-reward responses. What pattern do they share? (e.g., all start with "Great question!", all use bullet points, excessive hedging).

5. **Win-rate**: Evaluate against SFT baseline on held-out prompts. If flat/declining while RM rises = confirmed exploit.

Immediate fixes:

- Roll back to last checkpoint where win-rate was improving
- Increase β by 2–3× (stronger KL penalty)
- Add explicit length penalty: $r' = r - 0.001 \cdot \max(0, \text{len} - 500)$

Structural fixes (prevent recurrence):

- RM ensemble: Train 3–5 RMs on different data splits. Use min or mean. Exploits are model-specific.
- RM refresh: Every 2000 steps, generate from current policy, get human labels, retrain RM.
- Multi-objective reward: Combine helpfulness + harmlessness + conciseness with separate RMs.
- Early stopping on win-rate (not RM score). The metric you optimize should differ from training signal.

Review: *Chapters 9 and 11 (Reward Model Training; System Architecture).*

Q12: How do you decide the prompt distribution for RL training?

Answer: Prompt quality is the most underrated factor in RLHF. Bad prompts = no learning signal.

Composition (my default mix):

- 40% real user traffic (represents actual use cases)
- 30% synthetic (LLM-generated, fills gaps in coverage — rare topics, edge cases)
- 20% curriculum (graduated difficulty — start easy, increase complexity as model improves)
- 10% adversarial (red-team prompts, jailbreak attempts, ambiguous instructions)

Critical: The Goldilocks Filter:

1. For each candidate prompt, generate 4–8 responses with current model.
2. Score with RM. Compute pass rate (fraction above threshold).
3. Keep only prompts with 20–80% pass rate:
   - <20%: Too hard. Model almost always fails → all negative advantages → no useful gradient.
   - >80%: Too easy. Model almost always succeeds → all positive advantages → no contrast.
   - 20–80%: Perfect. Mix of successes and failures → clear signal about what works.
4. Re-filter every 500 training steps (model improves, difficulty distribution shifts).

Topic diversity: Ensure no single topic dominates (<10% per category). Use embedding clustering to verify coverage. Otherwise the model over-optimizes for dominant topics.

Review: *Chapters 7 and 9 (GRPO; Reward Model Training).*

Q13: LoRA vs full fine-tuning for RLHF. When would you use each?

Answer:
LoRA (Low-Rank Adaptation, r=64, α=16):

- Trainable params: ∼0.2% of model (200M for 70B)
- Memory savings: No separate reference model needed (base model = reference)! Saves 140GB.
- Stability: Inherently more stable (low-rank constraint limits how far policy can drift)
- Speed: Faster per-step (fewer params to update), but may need more steps
- Quality ceiling: 90–95% of full FT quality typically

Full fine-tuning:

- All parameters updated. Maximum expressiveness.
- Needs separate reference model copy (140GB for 70B). Or very frequent checkpointing to "anchor."
- Higher risk of catastrophic forgetting. Need lower LR (3× lower than LoRA) and stronger β.
- Better when: large distributional shift needed (new language, very different style), LoRA hits capacity limit.

My decision framework:

1. Start with LoRA (r=64). It's 3× cheaper and more stable.
2. Monitor gradient norms on LoRA matrices. If consistently >1.0 (high relative to parameter count): LoRA is capacity-limited.
3. Switch to full FT only when: win-rate plateaus AND gradient analysis suggests capacity limitation.
4. For full FT: use LR/3, $\beta \times 2$, more frequent checkpointing, and early stopping based on win-rate.

Hybrid: LoRA for alignment/safety (small behavioral shift) + Full FT for capabilities/reasoning (large shift needed).
Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q14: Process Reward Models (PRM) vs Outcome Reward Models (ORM). Design a PRM system.

Answer:
ORM: Scores the final answer only. "Is the response good overall?" Simple but can't identify *where* reasoning went wrong.
PRM: Scores each intermediate step. "Is step 3 of this derivation correct?" Much more informative but harder to train.
PRM advantages for reasoning:

- Identifies exactly where reasoning fails (step-level credit assignment)
- Enables tree search: expand only branches with high step scores
- Less reward hacking: can't get high score from wrong steps + lucky final answer

- PRM + best-of-N beats ORM + best-of-N by 10–20% on MATH benchmarks

Training a PRM:

1. **Data collection** (Monte Carlo approach):
   - For each problem, generate reasoning trace step by step
   - At each step k, complete the solution M times ($M = 32$) from that point
   - Step score = fraction of completions that reach correct answer
   - Steps where score drops significantly = the "mistake" steps
2. **Labeling**: Step is "correct" if its completion rate $> 50\%$, "incorrect" if $< 20\%$.
3. **Model**: Same architecture as base model + classification head per token position. Train with binary cross-entropy on step labels.
4. **Inference**: Score each step. If any step scores <0.3, flag the trace as flawed.

Using PRM in RLHF: Per-token rewards from PRM feed directly into GAE. Each token gets immediate feedback, not just end-of-sequence. This dramatically improves credit assignment for long reasoning chains.
Review: *Chapters 9 and 13 (Reward Model Training; RL for Large Reasoning Models).*

Q15: How do you evaluate whether RL actually improved the model?

Answer: Multi-faceted evaluation (no single metric captures "quality"):
1. Win-rate (most important, most reliable):

- 500+ diverse prompts. LLM judge (GPT-4 or Claude) picks winner in blind A/B comparison vs SFT baseline.
- Target: >55% win-rate = meaningful improvement. >65% = strong improvement.
- Use position-debiasing (swap A/B order, average). Report confidence intervals.

2. Capability benchmarks (regression detection):

- MMLU (knowledge), HumanEval (code), MATH (reasoning), MT-Bench (multi-turn).
- Any >2% drop = concerning alignment tax. Investigate which categories degraded.

3. Category-specific evals:

- Safety: refusal rate on harmful prompts (should increase)
- Truthfulness: TruthfulQA score (should increase or stay flat)
- Helpfulness: task completion rate on instruction-following benchmarks

4. Distributional metrics:

- Response length distribution (shouldn't shift dramatically)
- Vocabulary diversity (unique tokens per response)
- Format compliance (if trained for specific format)

5. Human evaluation (gold standard, expensive):

- Blind A/B with 3+ skilled annotators per example. Inter-annotator agreement $> 70\%$.
- Only for final model selection, not during training (too slow/expensive).

Red flags: RM score up + win-rate flat = reward hacking, not real improvement. Win-rate up + benchmarks down = alignment tax too high, reduce RL strength.
Review: *Chapter 14 (LLM Evaluation).*

Q16: Describe the reward model training pipeline end-to-end.

Answer:
Phase 1 — Data Generation:

- Collect 50K–100K diverse prompts (real traffic + synthetic)
- Generate 4–8 responses per prompt at varying temperatures (0.3, 0.7, 1.0) and from multiple models (diversity is key — if all responses are similar, preferences are uninformative)
- Total: 200K–800K candidate responses

Phase 2 — Preference Collection:

- Option A (expensive, best quality): Human annotators compare pairs. 3 annotators per pair. Cost: \$2–5 per comparison.
- Option B (cheap, 85–90% agreement with humans): LLM judge (GPT-4/Claude). 10× cheaper. Good for scale.
- Format: (prompt, chosen response, rejected response). Pairs with annotator disagreement (<70% agreement) are discarded.
- Final dataset: 100K–500K pairs.

Phase 3 — Training:

- Architecture: Same as base LLM + scalar head (one regression output per sequence).
- Loss: $\mathcal{L} = -\mathbb{E}[\log \sigma(r(x, y_w) - r(x, y_l))]$ (Bradley-Terry).
- Training: **1 epoch only!** RMs overfit extremely fast. Validation accuracy 68–75% is good (higher often means overfitting to annotation artifacts).
- Tricks: Center rewards around 0 (subtract running mean). Check for length bias (if correlation between length and score > 0.3, add length penalty to training).

Phase 4 — Validation:

- Hold-out preference pairs: accuracy should be 68–75%.
- Agreement with humans on new data: $> 80\%$.
- Length bias check: correlation between response length and RM score should be < 0.2.
- Consistency check: same prompt, paraphrased responses should get similar scores.

Review: *Chapter 9 (Reward Model Training).*

Q17: What happens when KL divergence explodes? Root cause and fix.

Answer:
What KL measures: Average log-ratio between current policy and reference: $D_{\text{KL}} = \mathbb{E}_{y\sim\pi_\theta}[\log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))]$. KL=0 means identical to reference. KL=10 means the policy puts 10 nats more probability on its preferred outputs.
Healthy range: 3–10 during training. Slowly increasing is fine. Sudden spike = problem.
Root causes of KL explosion:

1. **Learning rate too high**: Policy takes giant step, diverges from reference. Fix: reduce LR by 2–5×.
2. **Reward hacking**: Found an exploit that gets high reward far from reference behavior. Fix: increase β, add RM ensemble.
3. **Mode collapse**: Policy concentrates on one response template. KL is high at that template, low everywhere else. Fix: increase entropy bonus, increase temperature.
4. **Bad batch**: One unlucky batch with extreme advantages pushed the policy. Fix: gradient clipping, reduce mini-batch size.
5. **Value function diverged**: Wrong advantage estimates cause wrong updates. Fix: reduce value function LR, or switch to GRPO (no value function).

Recovery protocol:

1. Detect: KL $>$ 15 for 50+ steps, or KL jumps $>$5 in one step.
2. Immediate: Load last clean checkpoint (KL $<$ 10).
3. Adjust: Reduce LR by 50%. Increase β by 2×. Lower cliprange to 0.1.
4. Resume: Monitor closely for first 200 steps.

Review: *Chapters 5 and 7 (PPO; GRPO).*

Q18: Compare monolithic vs decoupled RLHF architectures. When does each make sense?

Answer:
Monolithic (TRL default: single process, all models on same GPUs):

- Pros: Simple code. No distributed systems complexity. Easy debugging.
- Cons: GPUs idle 60% of time (compute idle during gen, bandwidth idle during training). Doesn't scale past ∼16 GPUs efficiently. All models compete for same memory.
- Use when: Model $\leq$ 13B, single node, research/prototyping.

Semi-decoupled (vLLM gen + FSDP training, same cluster):

- Pros: Better utilization (gen and training can partially overlap). Scales to 64 GPUs.
- Cons: Still shares hardware, can't optimize independently. More complex than monolithic.
- Use when: 13B–70B, 2–8 nodes, production experiments.

Fully decoupled (separate clusters connected by queues):

- Pros: Each cluster optimized for its workload. Gen scales independently from training. Gen cluster is stateless (trivial fault tolerance). Scales to hundreds of GPUs.

- Cons: Distributed systems complexity. Weight staleness. Queue management. Network overhead.
- Use when: $\geq$ 70B production training. Need scale, fault tolerance, high utilization.

The key insight: Generation is bandwidth-bound, training is compute-bound. Same hardware can't optimize for both. Decoupling lets gen nodes have: more memory bandwidth, INT8 weights, large batch. Training nodes have: full BF16 precision, Flash Attention, FSDP sharding.
Review: *Chapter 11 (System Architecture & Infrastructure at Scale).*

Q19: How do you set up curriculum learning for RL training?

Answer: Curriculum = gradually increasing difficulty so the model learns progressively.
Why it matters: If you throw the hardest prompts at a weak model, it gets all-negative rewards $\rightarrow$ no learning signal (everything is equally bad). If you start easy, the model develops basic capabilities, then builds on them.
Implementation:

1. **Difficulty scoring**: Rate each prompt by current model's pass rate (from Goldilocks filtering). Easy = >80% pass, Medium = 30–80%, Hard = <30%.
2. **Schedule**: Steps 0–1000: 70% easy, 20% medium, 10% hard. Steps 1000–5000: 30% easy, 50% medium, 20% hard. Steps 5000+: 10% easy, 40% medium, 50% hard.
3. **Dynamic adjustment**: Every 500 steps, re-evaluate difficulty distribution. Prompts the model has "mastered" (pass rate > 95%) get retired. New harder prompts are introduced.
4. **For GRPO specifically**: Curriculum ensures groups always have a mix of successes and failures. Without curriculum, hard prompts give all-zero groups (useless).

Evidence: DeepSeek-R1 used implicit curriculum — starting with easy math/code problems, the model developed basic reasoning, then solved progressively harder problems without explicit scheduling.
Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q20: You have a budget of 64 A100-80GB GPUs and need to RL-train a 70B model. Design the allocation.

Answer: 8 nodes $\times$ 8 GPUs = 64 total. Need to split across generation, scoring, and training.
My allocation:

- **Generation**: 24 GPUs (3 nodes). 6 vLLM instances with TP=4. INT8 weights = 70GB/model, leaves room for KV cache. Continuous batching, batch $\approx$ 128 total.
- **Scoring**: 8 GPUs (1 node). RM (INT8, TP=4) + Reference model (INT8, TP=4) on same node. Or share 4 GPUs with TP=4 alternating between RM and ref.
- **Training**: 32 GPUs (4 nodes). FSDP across all 32. Each GPU holds $\sim$70B/32 = 2.2GB params + optimizer fraction. Plenty of headroom for activations. Gradient checkpointing for safety.

Expected throughput:

- Generation: 6 instances $\times$ $\sim$80 responses/min = 480 responses/min
- Training: Batch of 128, one step every $\sim$15s (training only, no gen wait)
- Overlap: While training step N (15s), generation produces $\sim$120 responses for step $N+1$. Perfect pipeline.

Bottleneck analysis: Generation takes ∼45s for 128 responses. Training takes ∼15s. Scoring takes ∼5s. Generation is bottleneck. Could move 8 GPUs from training to gen (40 gen, 24 training) to balance, but then training is bottleneck. Current allocation is near-optimal.
Alternative if memory-tight: Move scoring onto training nodes (time-share: score during gen, train during training). Saves 8 GPUs. Slightly worse pipelining but works.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

29.5 GRPO Variants and Advanced RL Questions

Q21: What is DAPO and how does it improve over standard GRPO?

Answer: DAPO (Dynamic Adaptive Policy Optimization) introduces 5 key modifications:
1. Clip-Higher (asymmetric clipping): Standard PPO/GRPO clips both directions equally at $\epsilon = 0.2$. DAPO uses $\epsilon_{\text{low}} = 0.2$ but $\epsilon_{\text{high}} = 0.28$. This allows the model to *increase* good action probabilities more aggressively while still restricting how much it suppresses bad ones. Intuition: exploration needs more room than exploitation.
2. Overlong Filtering: If a response hits the max length limit (truncated, no EOS token), it's masked entirely from the loss. Rationale: truncated responses contain no natural stopping signal — training on them teaches the model that "stopping mid-sentence" is acceptable.
3. Token-level Loss: Loss is normalized by total token count across all sequences, not by number of sequences. This prevents longer sequences from dominating the gradient.
4. Soft Overlong Punishment: Instead of binary truncation filtering, apply a gradual penalty as responses approach max length. $r_{\text{soft}} = -c \cdot \max(0, \text{len} - L_{\text{soft}})/(L_{\max} - L_{\text{soft}})$.
5. Dynamic Sampling: Resample prompts during training to ensure each batch has a mix of success/failure (not yet in TRL).
When to use: Large-scale reasoning RL where you need maximum exploration and long completions (32K+ tokens). The asymmetric clipping is particularly valuable.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

Q22: Explain the vLLM train-inference mismatch. Why does it happen and how do TIS/MIS fix it?

Answer: **The problem**: When using vLLM for generation and a training framework (DeepSpeed/FSDP) for updates, the same model with the same weights produces *different* token probabilities. This happens because:

- Different numerical kernels (vLLM uses custom CUDA kernels optimized for throughput)
- Different attention implementations (Flash Attention in training vs PagedAttention in vLLM)
- Different precision handling (FP8/INT8 in vLLM vs BF16 in training)
- Batching differences affecting layer normalization numerics

This silently breaks PPO's on-policy assumption: we compute the ratio $\pi_\theta/\pi_{\text{old}}$ using π_{old} from vLLM but π_θ from the training framework. The ratio is wrong from step zero!
TIS (Truncated Importance Sampling): Correct the gradient by multiplying by $\min(\pi_{\text{train}}/\pi_{\text{inference}}, C)$. The min with cap C prevents extreme corrections from destabilizing training. Typical $C = 2.0$.
MIS (Masked Importance Sampling): More aggressive — simply discard any token where $\pi_{\text{train}}/\pi_{\text{inference}} > C$. Zero contribution to gradient. Prevents any badly-estimated token from affecting the update.
Sequence vs Token level: Sequence-level IS is theoretically correct (unbiased); token-level IS is biased but lower variance. In practice, sequence-level with truncation works best.
Review: *Chapters 7 and 11 (GRPO; System Architecture).*

Q23: GSPO vs GRPO — what's the fundamental difference and when does it matter?

Answer: **GRPO**: Computes importance ratio *per token*: $w_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t}) / \pi_{\text{old}}(o_{i,t}|q, o_{i,<t})$, then clips each token independently.
GSPO: Computes importance ratio at the *sequence level*: $s_i(\theta) = (\pi_\theta(o_i|q)/\pi_{\text{old}}(o_i|q))^{1/|o_i|}$ — the geometric mean of token probabilities. Clips this single sequence-level ratio.
Why it matters: GRPO's per-token clipping treats each token as independent, but in language they're deeply correlated. A small per-token change early in the sequence compounds exponentially over many tokens. GSPO captures this by looking at the full sequence probability.
Length normalization: The $1/|o_i|$ exponent ensures fair comparison across different-length sequences. Without it, longer sequences would always have lower probability ratios.
When to use GSPO: When training goes off-policy (`steps_per_generation > 1` or `num_iterations > 1`). If fully on-policy (ratio ≈ 1), GRPO and GSPO are equivalent.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

Q24: The paper "It Takes Two" shows G=2 matches G=16. How is that possible?

Answer: The key insight is that GRPO's effectiveness doesn't come from accurate advantage estimation (which would need large G), but from an **implicit contrastive objective**.
With $G = 2$ and binary rewards (one correct, one wrong): $\hat{A}_{\text{correct}} = +1$, $\hat{A}_{\text{wrong}} = -1$ (after normalization). The loss becomes: increase probability of correct response, decrease probability of wrong response. This is essentially a DPO-style contrastive loss!
Why large G doesn't help much: The normalized advantage $\hat{A}_i = (r_i - \mu)/\sigma$ already creates a contrast between good and bad. More samples give a better μ estimate, but the gradient direction is dominated by the *contrast* between best and worst, not the mean accuracy.
Compute savings: $G = 2$ means $8\times$ less generation compute than $G = 16$. Since generation is 60% of training time, this translates to $\sim 4\times$ faster training.
Caveat: Works best when pass rate is 30–70%. If pass rate is very low (<10%), $G = 2$ often gives two failures (no signal). Need larger G for hard problems.
Review: *Chapter 7 (GRPO).*

Q25: What is SAPO and how does its soft gating differ from hard clipping?

Answer: Standard PPO/GRPO uses hard clipping: $\text{clip}(r, 1-\epsilon, 1+\epsilon)$. At the boundary, the gradient suddenly drops to zero. This creates a "dead zone" where the model receives no learning signal.
SAPO replaces this with a smooth sigmoid gate: the gradient is gradually attenuated as the ratio moves away from 1, never suddenly zeroed out. It uses asymmetric temperatures:

- $\tau_+ = 1.0$ for positive advantages (standard attenuation)
- $\tau_- = 1.05$ for negative advantages (slightly more aggressive attenuation for suppression)

Benefits: (1) No "cliff" in gradient landscape. (2) Tokens slightly outside the clip range still contribute (attenuated, not zeroed). (3) More stable optimization trajectory. (4) Sequence-coherent — considers the full sequence context.
Trade-off: Slightly less restrictive trust region than hard clipping, so requires careful temperature tuning. But more robust to hyperparameter choices overall.
Review: *Chapters 7 and 8 (GRPO; Preference Optimization Variants).*

29.6 DPO Extensions Questions

Q26: Compare f-DPO divergence choices. When would you use forward KL vs JS vs reverse KL?

Answer: Standard DPO uses reverse KL implicitly ($D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$):

- **Reverse KL** (default): Mode-seeking. The policy concentrates probability where the reference is high. Avoids generating text the reference wouldn't. Good for safety (conservative).
- **Forward KL**: Mass-covering. The policy tries to cover all modes of the reference, even low-probability ones. Good for diversity but can generate low-quality outputs.
- **Jensen-Shannon**: Symmetric compromise between forward and reverse. Balanced mode-coverage and mode-seeking. Often best for general alignment.
- **Alpha-divergence** ($\alpha = 0.5$): Interpolates between forward ($\alpha = 0$) and reverse ($\alpha = 1$). Tunable.

Practical recommendation: Start with reverse KL (standard DPO). If the model is too conservative (won't try creative solutions), switch to JS divergence. If diversity is critical (creative writing, brainstorming), try forward KL.
Review: *Chapters 6 and 8 (DPO; Preference Optimization Variants).*

Q27: Your DPO preference data has 15% label noise. What do you do?

Answer: Three options in order of sophistication:
1. Robust DPO (best for known noise rate): Analytically debiases the loss: $\mathcal{L}_{\text{robust}} = \frac{(1-\varepsilon)\mathcal{L}_{\text{DPO}}(y_w,y_l) - \varepsilon\mathcal{L}_{\text{DPO}}(y_l,y_w)}{1-2\varepsilon}$. Set $\varepsilon = 0.15$. This provably recovers the clean DPO objective in expectation. TRL: `loss_type="robust", label_smoothing=0.15`.
2. IPO (best when noise rate unknown): Squared loss with target margin. Mislabeled pairs have bounded influence (the squared loss doesn't diverge). More robust to arbitrary noise patterns without needing to know ε. TRL: `loss_type="ipo"`.
3. TR-DPO (best for distribution shift): Updates the reference model via EMA during training. Even if early data is noisy, the evolving reference helps the model self-correct. TRL: `sync_ref_model=True, ref_model_mixup_alpha=0.6`.
Data-side fixes: (1) Filter pairs with <70% inter-annotator agreement. (2) Use RM to score pairs; discard those where RM disagrees with label. (3) Active learning: re-label the most uncertain pairs.
Review: *Chapters 6 and 8 (DPO; Preference Optimization Variants).*

Q28: What is SimPO and why is being reference-free advantageous?

Answer: SimPO uses the average log-probability of a response as an implicit reward signal: $r(x,y) = \frac{1}{|y|}\sum_t \log \pi_\theta(y_t|x, y_{<t})$ — no reference model needed.
The loss adds a target margin γ: the chosen response should have average log-prob at least γ higher than rejected.
Why reference-free matters:

1. **Memory**: No reference model = 70–140GB saved for 70B models. Can train larger models on same hardware.
2. **Simplicity**: No need to manage/load/serve a second model copy.
3. **No stale reference**: DPO's reference becomes increasingly irrelevant as training progresses. SimPO doesn't have this problem.
4. **Length normalization built in**: The $1/|y|$ naturally prevents length bias (DPO needs explicit handling).

Trade-off: Without a reference anchor, the model has more freedom to collapse or drift. The γ margin and length normalization partially mitigate this, but SimPO can be less stable than DPO for aggressive training.
Review: *Chapter 8 (Preference Optimization Variants).*

Q29: Explain Iterative RPO. Why combine DPO with NLL loss for reasoning?

Answer: Standard DPO for reasoning has a subtle failure mode: it learns to *discriminate* (assign higher implicit reward to correct traces) but doesn't necessarily learn to *generate* them.
Why: DPO's gradient pushes chosen probability up and rejected probability down. But the chosen response might be so different from what the model would generate that increasing its probability doesn't teach the model how to produce similar reasoning patterns.
RPO's fix: Add a negative log-likelihood (NLL/SFT) loss on the chosen response: $\mathcal{L} = \mathcal{L}_{\text{DPO}} + \alpha \cdot \mathcal{L}_{\text{NLL}}(y_w)$.
The NLL term explicitly trains the model to generate the winning response step by step. The DPO term ensures it also learns to avoid the losing response. Combined: the model learns both "how to reason correctly" (NLL) and "what to avoid" (DPO).
Iterative: Generate responses → check correctness → create pairs → train with RPO → repeat. Each iteration the model gets better at generating correct reasoning, creating higher-quality training data for the next round.
TRL: `loss_type=["sigmoid", "sft"], loss_weights=[1.0, 1.0]`
Review: *Chapters 8 and 13 (Preference Optimization Variants; RL for Large Reasoning Models).*

29.7 GPU Architecture and Hardware Questions

Q30: Explain the GPU memory hierarchy. Why does it matter for LLM inference?

Answer: From fastest to slowest:

1. **Registers**: Per-thread, ∼256 KB/SM. Instant access (0 cycles latency).
2. **SRAM (Shared Memory)**: Per-SM, ∼192–228 KB/SM (A100: 164 KB configurable). Bandwidth: ∼19 TB/s aggregate. Latency: ∼20 cycles.
3. **L2 Cache**: Shared across GPU, 40–60 MB (H100: 50 MB). Bandwidth: ∼5 TB/s. Latency: ∼200 cycles.
4. **HBM**: Main GPU memory, 80 GB (A100). Bandwidth: 2–3.35 TB/s. Latency: ∼400 cycles.
5. **CPU DRAM**: Via PCIe, 512 GB+. Bandwidth: 32–64 GB/s. Latency: ∼10K cycles.

Why it matters for LLMs: Autoregressive generation reads the full model weights (∼140 GB for 70B) for every single token. At 2 TB/s HBM bandwidth, that's 70ms just to stream the weights. The actual computation (one matrix-vector multiply) takes only 0.5ms. The GPU is 99% waiting for data.
Flash Attention exploits this: By keeping intermediate results (QK scores, softmax) in SRAM (19 TB/s) instead of writing them to HBM (2 TB/s), it eliminates 90% of the memory traffic for attention. The compute is the same, but HBM reads/writes drop 10×.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q31: How does Flash Attention work? What is the online softmax trick?

Answer: **Problem**: Standard attention materializes the $n \times n$ attention matrix in HBM. For $n = 8192$: $8192^2 \times 2 = 134$ MB per head, 4.3 GB per layer with 32 heads. Must write to HBM then read back for softmax and multiply — 3 full HBM round-trips.
Flash Attention solution: Never store the full $n \times n$ matrix. Process in tiles that fit in SRAM.
Algorithm:

1. Split Q into blocks of size B_r rows, K/V into blocks of B_c rows.
2. For each Q block: iterate over all K blocks, computing partial attention scores.
3. **Online softmax trick**: Maintain running max m and running sum ℓ for softmax normalization. When processing a new K block, update: $m_{\text{new}} = \max(m_{\text{old}}, \max(\text{scores}))$, rescale previous accumulator by $e^{m_{\text{old}} - m_{\text{new}}}$, add new contribution.
4. Output is accumulated incrementally — never needs the full $n \times n$ matrix.

Key insight: Softmax is normally a global operation (max and $\sum$ over all elements). The online trick decomposes it into local updates with a correction factor. Mathematically exact — no approximation.
Result: Memory $O(n)$ instead of $O(n^2)$. Speed 2–4× faster (fewer HBM accesses, more time in SRAM).
Flash Attention 2: Better work partitioning across warps, reduces non-matmul FLOPs by 2×.
Flash Attention 3 (H100/Hopper): Uses Tensor Memory Accelerator (TMA) for async loads, warp specialization (producer/consumer warps), FP8 support.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q32: Explain PagedAttention. How does it solve the KV cache problem?

Answer: **The problem**: During generation, each sequence needs a KV cache (stores K and V tensors for all previous tokens). For a 70B model: each token needs $2 \times n_{\text{layers}} \times d_{\text{model}} \times 2$ bytes $= 2 \times 80 \times 8192 \times 2 \approx 2.5$ MB per token. A 2048-token sequence: ∼5 GB of KV cache.
Traditional allocation: Pre-allocate max_sequence_length for each active sequence. If max=2048 but average=500, you waste 75% of allocated memory. With 50 concurrent sequences, that's hundreds of GB wasted.
PagedAttention: Inspired by OS virtual memory:

1. KV cache is split into fixed-size *blocks* (pages), each holding KV for 16 tokens.
2. A *block table* (like a page table) maps logical token positions to physical memory blocks.
3. Blocks are allocated on demand as the sequence grows. No pre-allocation waste.
4. Freed blocks return to a pool immediately when sequences finish.

Extra benefits:

- **Prefix sharing**: Multiple sequences with the same system prompt share KV cache blocks (copy-on-write). Saves 30–50% memory for chat applications.
- **Preemption**: Can "swap out" a low-priority sequence's blocks to CPU, freeing GPU memory for higher-priority requests.
- **Near-zero fragmentation**: Internal fragmentation limited to last block (<16 tokens). External fragmentation eliminated (any free block can be used anywhere).

Result: 3–5× more concurrent sequences in the same memory → 3–5× better throughput for serving.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q33: Compare NVLink vs InfiniBand. When do you use each in RLHF training?

Answer:
NVLink (intra-node, GPU-to-GPU):

- Bandwidth: 600 GB/s (A100), 900 GB/s (H100) — total bidirectional

- Latency: ∼1 μs
- Scope: Within one physical node (8 GPUs connected via NVSwitch)
- Use case: **Tensor Parallelism** (TP=8). Each layer's matrix multiply is split across GPUs, requiring AllReduce after every layer. Needs ultra-high bandwidth + low latency.

InfiniBand NDR (inter-node, node-to-node):

- Bandwidth: 400 Gb/s = 50 GB/s per port. With 8 ports (GPUDirect RDMA): 400 GB/s aggregate per node.
- Latency: ∼1–5 μs (RDMA)
- Scope: Between nodes in a cluster. Requires switches (fat-tree topology).
- Use case: **Data Parallelism / FSDP** gradient synchronization. AllReduce of gradients happens once per training step (not per layer), so latency tolerance is higher.

In RLHF specifically:

- *Generation*: TP=8 over NVLink within node. Multiple vLLM instances across nodes don't communicate (embarrassingly parallel).
- *Training*: TP=8 over NVLink intra-node + FSDP over InfiniBand inter-node. Gradients synced after full backward pass.
- *Weight sync*: Training → Generation uses InfiniBand (140 GB transfer, async, takes ∼3s at 50 GB/s).

Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

29.8 Optimization and Training Questions

Q34: Explain Adam vs AdamW. Why does the difference matter for LLMs?

Answer: **Adam with L2 regularization**: $\theta_{t+1} = \theta_t - \alpha \cdot (\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon) + \lambda\theta_t)$. The weight decay term $\lambda\theta_t$ is *inside* the adaptive scaling. Parameters with large gradients (large v_t) get *less* weight decay (divided by $\sqrt{v_t}$). This is not true weight decay — it's scale-dependent.
AdamW (decoupled weight decay): $\theta_{t+1} = (1 - \alpha\lambda)\theta_t - \alpha \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$. Weight decay is applied *outside* and *before* the adaptive update. Every parameter gets the same proportional decay regardless of gradient history.
Why it matters for LLMs:

1. LLMs have parameters spanning many orders of magnitude (embedding layers vs attention vs FFN). Adam's coupled L2 effectively penalizes small-gradient params more than large-gradient ones — wrong behavior.
2. Decoupled WD provides uniform regularization across all layers, preventing some layers from growing unbounded while others over-shrink.
3. Empirically: AdamW gives 2–5% better perplexity on long pretraining runs compared to Adam+L2 with the same effective regularization.

For RL specifically: Often use $\lambda = 0$ (no weight decay). The KL penalty provides regularization. But for SFT: $\lambda = 0.01$–0.1 with AdamW is standard.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q35: Why is learning rate warmup necessary? What happens without it?

Answer: **The problem**: Adam's second moment estimate $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ starts at $v_0 = 0$. The bias correction $\hat{v}_t = v_t/(1-\beta_2^t)$ compensates mathematically, but in practice:

- First few steps: v_t is based on 1–5 gradient samples. Highly inaccurate estimate of true variance.
- If a parameter happens to get a small gradient initially, v_t is tiny → effective LR is huge → catastrophic update.
- The bias correction amplifies early updates: at step 1, $\hat{v}_1 = v_1/(1-0.999) = 1000 \cdot v_1$.

Without warmup: First 10–100 steps often have gradient spikes that permanently damage the model. Early representations get scrambled before the optimizer stabilizes.
Warmup fix: Start with LR ≈ 0 and linearly increase to target over W steps (typically 3–10% of training). By the time LR reaches full value, v_t has accumulated enough samples to be accurate.
Typical settings:

- Pretraining: 2000 steps warmup (∼1% of 200K steps)
- SFT: 100 steps warmup (∼5% of 2000 steps)
- RL (PPO/GRPO): 20–50 steps warmup (short, model already stable from SFT)

Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q36: Compare learning rate schedules. Which would you choose for RL fine-tuning?

Answer:
Cosine decay: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$. Standard for pretraining and SFT. Smooth decay, most time at moderate LR.
Linear decay: $\eta_t = \eta_{\max}(1 - t/T)$. Simpler, similar results to cosine for short runs.
WSD (Warmup-Stable-Decay): Warmup → constant LR for 80% → rapid decay in last 20%. New standard for pretraining. The "stable" phase gives consistent learning; the final decay squeezes out remaining gains.
Constant: No decay. $\eta_t = \eta_{\max}$ after warmup.
For RL fine-tuning (PPO/GRPO), I'd choose: Constant with short warmup. Reasons:

1. RL training length is highly unpredictable (you stop based on win-rate, not epochs).
2. Cosine/linear decay assumes you know the total steps in advance.
3. The LR is already very low (10^{-6}), further decay makes updates invisible.
4. PPO's adaptive KL controller already modulates the effective step size.
5. If you must decay: use linear decay over a generous budget, stop early when metrics plateau.

Review: *Chapters 1 and 5 (LLM Architecture; PPO).*

Q37: Why is gradient clipping critical for RL training but less important for SFT?

Answer: **SFT**: Supervised loss is smooth and well-behaved. Gradient norms are consistent across batches (typically 0.1–1.0). Clipping at 1.0 rarely activates — it's a safety net.
RL (PPO/GRPO): Gradient norms are highly variable because:

1. **Reward variance**: One batch might have all high-reward responses, next might have all low. The advantage $\hat{A}$ swings wildly.
2. **Ratio explosion**: If a rare token's probability changed a lot, $r_t = \pi_{\text{new}}/\pi_{\text{old}}$ can be very

large → large gradient before clipping kicks in.

3. **Sparse reward**: In GRPO with binary rewards, some prompts give all-correct (advantage ≈ 0) then suddenly a hard prompt gives extreme advantages.

4. **KL term**: The KL penalty gradient can spike when policy diverges.

Without clipping: A single bad batch can produce a gradient 100× normal magnitude → destroys the model in one step. Recovery is impossible (catastrophic forgetting of all pretraining).
Typical setting: `max_grad_norm=1.0`. Some use 0.5 for extra safety in early RL training. The norm is computed globally across all parameters (not per-layer).
Monitoring: If clipping activates more than 20% of steps, your LR is probably too high or your batch size too small.
Review: *Chapters 5 and 7 (PPO; GRPO).*

Q38: BF16 vs FP16 for training. When does the choice matter?

Answer:
FP16: 1 sign + 5 exponent + 10 mantissa bits. Range: ±65504. Precision: ∼3.3 decimal digits.
BF16: 1 sign + 8 exponent + 7 mantissa bits. Range: $\pm 3.4 \times 10^{38}$ (same as FP32!). Precision: ∼2.4 decimal digits.
Why BF16 wins for LLMs:

1. **No loss scaling needed**: FP16's small range (±65K) means gradients and activations frequently overflow/underflow. Requires dynamic loss scaling (multiply loss by 1024, divide gradients back). BF16 has FP32's range — overflow is essentially impossible.

2. **Simpler code**: No loss scaler, no inf/nan checking, no dynamic scaling adjustment.

3. **Critical for RL**: RL gradients are noisier and spikier than SFT. FP16 loss scaling often fails (picks wrong scale, causes NaN). BF16 "just works."

When FP16 might be better: If you need maximum precision (some scientific computing tasks) and can manage the loss scaling. FP16 has 3 more mantissa bits = slightly more accurate results.
FP32 master weights: Even with BF16 forward/backward, accumulate gradient updates in FP32 to prevent rounding errors from compounding over thousands of small steps. Standard practice for all LLM training.
Review: *Chapter 2 (Systems Foundations for LLMs).*

29.9 Reward Model and SFT Questions

Q39: Derive the Bradley-Terry reward model loss. What are its limitations?

Answer: **Bradley-Terry model**: Given two responses, the probability the better one (y_w) is preferred: $P(y_w \succ y_l|x) = \sigma(r(x, y_w) - r(x, y_l))$ where σ is the sigmoid.
MLE derivation: Given N preference pairs, maximize likelihood: $\prod_i P(y_w^i \succ y_l^i)$. Take negative log: $\mathcal{L} = -\sum_i \log \sigma(r(x_i, y_w^i) - r(x_i, y_l^i))$.
Limitations:

1. **No ties**: BT can't model "equally good" — forces a strict preference.

2. **Transitivity**: Assumes if A>B and B>C then A>C. Humans aren't transitive.

3. **Context-free**: Same reward regardless of what alternatives were available.

4. **Scalar collapse**: Compresses all quality dimensions into one number. A response can be safe but unhelpful — RM must trade off.

5. **Length bias**: Longer responses get higher scores (more information = more likely to contain what annotator wanted). Must explicitly decorrelate.

Mitigations: Margin loss (require minimum gap δ), reward centering (subtract running mean), length penalty during training, multi-head RM (separate scores for helpfulness/safety/accuracy).
Review: *Chapter 9 (Reward Model Training).*

Q40: What is sequence packing in SFT and why does it matter?

Answer: **Problem**: Training examples have variable length. Standard batching pads all examples to max_length. If max=4096 but average=500, you waste 88% of compute on padding tokens (which contribute zero gradient).
Packing solution: Concatenate multiple short examples into a single max_length sequence. Separate with EOS tokens. Train on all examples simultaneously.
Example: Instead of 4 sequences padded to 4096 (16K tokens, 14K padding), pack into 1 sequence of 4096 with 4 examples end-to-end (4096 real tokens, 0 padding). **4× more efficient.**
Critical detail — block-diagonal attention mask: Without special handling, example 2 attends to example 1's tokens (cross-contamination). Must use a block-diagonal attention mask that restricts each example to only attend to its own tokens.
In TRL: `SFTConfig(packing=True, max_seq_length=4096)`. Handles mask automatically.
Caveats: (1) Longer examples still need their own batch entries (can't split mid-sequence). (2) Slight implementation complexity for position embeddings (reset per example). (3) Some argue packing changes the effective batch size (more examples per step) — adjust LR accordingly.
Review: *Chapter 10 (SFT Best Practices and Techniques).*

Q41: Explain completion-only masking for SFT. What happens if you don't use it?

Answer: In chat-format SFT data: `[system] + [user message] + [assistant response]`. Standard NLL loss computes loss on all tokens including the system prompt and user message.
Problem without masking: The model wastes capacity learning to predict the user's message (which it will never need to generate at inference). Worse: if the training data has diverse user messages, the model gets confused about "whose turn is it?"
Completion-only masking: Set loss weight to 0 for all tokens in the prompt (system + user). Only compute loss on assistant response tokens.
TRL: `DataCollatorForCompletionOnlyLM(response_template="<|assistant|>")`
Impact: Typically 5–15% better on instruction-following benchmarks. Faster convergence (gradient signal is concentrated on useful tokens). No change to compute cost.
Subtlety: Must include the response template token in the loss (teaches the model to start responding). But exclude everything before it.
Review: *Chapter 10 (SFT Best Practices and Techniques).*

Q42: How does SFT quality affect the RL ceiling? What is the pass@k diagnostic?

Answer: **Ceiling theorem (informal)**: RL can only reinforce behaviors the model can already produce with non-negligible probability. If the SFT model has 0% chance of generating a correct solution, RL will never find it.
Why: GRPO/PPO sample from the current policy and reinforce good samples. If good samples don't exist in the distribution, there's nothing to reinforce. RL is exploration-limited by the base policy's support.
pass@k diagnostic: Generate k responses per prompt, check if *any* is correct:

- pass@1: Model's typical performance (greedy/low temp).
- pass@8: Upper bound of what GRPO with $G = 8$ can achieve.
- pass@64: Upper bound for aggressive Best-of-N.
- pass@256: Approximate ceiling for RL improvement.

Interpretation:

- pass@1=20%, pass@64=80%: Great! 4× headroom for RL. Strong gains expected.
- pass@1=20%, pass@64=25%: Almost no headroom. RL won't help much. Need better SFT first.
- pass@1=5%, pass@64=60%: Model *can* solve it but rarely does. Perfect case for RL (reinforce the rare successes).

Rule: If pass@64 $< 1.5\times$ pass@1, invest in better SFT data before starting RL.
Review: *Chapters 7 and 10 (GRPO; SFT Best Practices).*

Q43: Design a multi-objective reward system for a chat model. How do you balance helpfulness vs safety?

Answer: **Architecture**: Separate reward models for each objective:

- r_{helpful}: Trained on helpfulness preferences (quality, accuracy, completeness)
- r_{safe}: Trained on safety preferences (refusals, harmlessness, no hallucination)
- r_{format}: Rule-based (follows instructions, proper formatting, appropriate length)

Combination strategies:

1. **Weighted sum** (simplest): $r = w_1 r_{\text{helpful}} + w_2 r_{\text{safe}} + w_3 r_{\text{format}}$. Problem: safety can be outweighed by helpfulness.
2. **Constrained** (safer): Maximize r_{helpful} subject to $r_{\text{safe}} > \tau$. Implemented via: $r = r_{\text{helpful}} - \lambda \cdot \max(0, \tau - r_{\text{safe}})$ with large λ.
3. **GDPO normalization** (best for GRPO): Normalize each reward independently within group, then combine: $\hat{A} = w_1 \hat{A}_{\text{helpful}} + w_2 \hat{A}_{\text{safe}}$. Prevents one reward from dominating due to scale differences.
4. **Lexicographic**: Safety is hard constraint (must pass), then optimize helpfulness. Train in stages: safety alignment first, then helpfulness.

Practical weights: Start with $w_{\text{safe}} = 2.0, w_{\text{helpful}} = 1.0, w_{\text{format}} = 0.5$. Safety gets 2× weight because its failure mode (harmful content) is much worse than helpfulness failure (mediocre answer).
Review: *Chapters 9 and 12 (Reward Model Training; LLM Agentic Training).*

29.10 System Architecture Extension Questions

Q44: How does speculative decoding work? When does it help for RLHF?

Answer: **Problem**: Large model generates one token per forward pass (∼70ms for 70B). Slow.
Speculative decoding:

1. **Draft**: Small model (1–7B) generates k candidate tokens quickly (∼5ms for all k).
2. **Verify**: Large model does one forward pass scoring all k tokens in parallel. Accepts tokens where $p_{\text{large}}(t_i) \geq p_{\text{draft}}(t_i)$ (always). Probabilistically accepts others.
3. **Result**: On average, 3–4 tokens accepted per verification step. Speedup: 2–3×.

Key property: The output distribution is *identical* to sampling from the large model alone. No quality loss. The draft model only affects speed, not output.

For RLHF specifically: Generation is 60% of compute. 2–3× speedup on generation = 1.5–2× end-to-end speedup. Combined with vLLM + INT8: generation goes from the bottleneck to parity with training.
Limitations: (1) Draft model must share tokenizer. (2) Less effective at high temperature (draft model less accurate). (3) Needs additional GPU memory for draft model. (4) Diminishing returns beyond $k = 5$ (acceptance rate drops).
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

Q45: Explain the roofline model. How do you determine if a kernel is compute-bound or memory-bound?

Answer: The roofline model plots achievable performance (FLOPS) as a function of arithmetic intensity (FLOPS per byte of memory traffic).
Two regimes:

- **Memory-bound** (left of crossover): Performance limited by how fast you can feed data to compute units. Actual FLOPS = bandwidth × arithmetic intensity. GPU utilization < 100%.
- **Compute-bound** (right of crossover): Performance limited by peak FLOPS. Memory is fast enough. GPU at max utilization.

Crossover point: Peak FLOPS / Peak Bandwidth. For A100: 312 TF / 2 TB/s = 156 FLOP/byte.
LLM operations:

- **Autoregressive generation** (batch=1): Read 140GB weights, do 140G FLOPs = 1 FLOP/byte. *Extremely* memory-bound (156× below crossover). Only 0.6% GPU utilization.
- **Training forward pass** (batch=128, seq=2048): Arithmetic intensity ≈ 200+ FLOP/byte. Compute-bound. Near peak utilization.
- **Attention** (long sequence): $O(n^2 d)$ FLOPs / $O(n^2 + nd)$ bytes. For long n: compute-bound. For short n: memory-bound. Flash Attention keeps it in SRAM regardless.

Practical use: If your kernel is memory-bound, reduce memory traffic (quantization, caching, tiling). If compute-bound, reduce FLOPs (pruning, distillation, lower precision).
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q46: How does continuous batching work and why is it essential for RLHF generation?

Answer: **Static batching**: Start B sequences. Wait for ALL to finish. If one sequence generates 500 tokens and another generates 50 tokens, the 50-token sequence's GPU slot sits idle for 450 tokens.
Continuous batching (iteration-level scheduling): After each generation step, check which sequences are done. Immediately insert new sequences into freed slots. GPU slots are never idle.
Why essential for RLHF:

1. RLHF generates diverse outputs (high temperature). Length variance is huge — some responses are 50 tokens, others 2000+.
2. Without continuous batching: average utilization ∼40–50% (waiting for slowest sequence).
3. With continuous batching: utilization >90%. Throughput 2–3× higher.
4. RLHF needs large batches (128+ responses per step). Generating 128 responses with static batching requires max_tokens × 128 sequential steps. Continuous batching amortizes this.

Implementation: vLLM's scheduler checks after every decode step. Preemption: if a new high-priority request arrives and memory is full, can swap out a low-priority sequence's KV cache to CPU and resume later.
Review: *Chapters 2 and 11 (Systems Foundations; System Architecture).*

29.11 Transformer Architecture Questions

Q: Why does RoPE dominate over learned absolute positional embeddings in modern LLMs?

Answer: RoPE encodes *relative* position directly into the Q/K dot product via rotation matrices. Key advantages:

1. Attention scores depend only on relative distance $i-j$, not absolute position — this generalizes better to unseen sequence lengths.
2. Can be extended beyond training length via frequency scaling (NTK-aware, YaRN) without retraining.
3. No additional parameters (rotations are deterministic from position index).
4. Learned absolute embeddings are fixed to training length and don't extrapolate — a model trained at 4K context fails at 8K.

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Explain SwiGLU and why it replaced ReLU in modern transformers.

Answer: SwiGLU: $\text{FFN}(x) = W_2(\text{Swish}(W_1 x) \odot W_3 x)$, where $\text{Swish}(x) = x \cdot \sigma(x)$.
Why it's better:

- The *gating* mechanism ($\odot W_3 x$) allows the network to selectively suppress or amplify dimensions — more expressive than pointwise ReLU.
- Swish is smooth (no dead neurons like ReLU's zero-gradient region).
- Empirically: 1–2% improvement on language modeling benchmarks at same FLOP count.
- Tradeoff: requires 3 weight matrices instead of 2 (solved by reducing hidden dim from $4d$ to $8d/3$).

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: What is Grouped Query Attention (GQA) and why does Llama-3 use it?

Answer: Standard MHA: H query heads, H key heads, H value heads. GQA: H query heads but only $G < H$ key/value heads (shared across query groups).
Llama-3 70B: 64 query heads, 8 KV heads (each KV head shared by 8 query heads).
Benefits:

- KV cache size reduced by $H/G = 8\times$ — critical for inference (KV cache is the dominant memory cost at long sequences).
- Minimal quality loss (<0.5% on benchmarks) because KV patterns are highly correlated across heads.
- Inference throughput increases proportionally to KV cache reduction (more sequences fit in memory = higher batch size).

Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Why did decoder-only architectures win over encoder-decoder for LLMs?

Answer:

1. **Unified objective**: Pretraining = fine-tuning = inference all use next-token prediction. No architectural mismatch.
2. **Parameter efficiency**: All parameters contribute to generation. In encoder-decoder, encoder params are "wasted" during pure generation tasks.
3. **Simpler scaling**: One model, one loss function, one set of hyperparameters to tune.
4. **KV cache efficiency**: Decoder-only has one KV cache; encoder-decoder has two (encoder + decoder cross-attention).
5. **Emergent few-shot**: Decoder-only naturally supports in-context learning (prepend examples to the prompt).

Encoder-decoder still wins for seq2seq tasks with fixed input length (translation), but these are a shrinking fraction of LLM use cases.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

29.12 Flash Attention Questions

Q: Flash Attention computes the same result as standard attention but is 2–4× faster. How is this possible if it does the same number of FLOPs?

Answer: Flash Attention is faster because it reduces *HBM memory traffic*, not FLOPs. Standard attention materializes the $n \times n$ attention matrix in HBM (slow), reads it back for softmax, reads again for PV multiply — 4 HBM round-trips over $O(n^2)$ data.
Flash Attention tiles the computation so the $n \times n$ matrix is computed and consumed entirely in SRAM (fast, 19 TB/s) without ever writing it to HBM (2 TB/s). The "online softmax" trick enables this by maintaining running statistics.
Result: HBM traffic drops from $O(n^2 d)$ to $O(n^2 d/M)$ where M is SRAM size. Same FLOPs, 10–50× less memory traffic → 2–4× wall-clock speedup.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: Why doesn't Flash Attention help the FFN layers?

Answer: FFN layers are *compute-bound*, not memory-bound. Their arithmetic intensity ($I \approx 300$ FLOP/byte for large batch GEMMs) is already above the roofline ridge point (156 FLOP/byte on A100).
Flash Attention helps attention because attention is deeply memory-bound ($I \approx 1$–60 FLOP/byte). By keeping data in SRAM, it removes the memory bottleneck.
For FFN: the bottleneck is already the Tensor Cores (not memory bandwidth), so reducing memory traffic doesn't help. Instead, FFN benefits from quantization (reduces weight size → higher arithmetic intensity) and larger batch sizes.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: Explain the online softmax trick and why it's essential for Flash Attention.

Answer: Standard softmax needs the global maximum $m = \max_j x_j$ before computing any output — this requires seeing all n attention scores first, forcing materialization of the full $n \times n$ matrix. The online softmax trick processes blocks sequentially, maintaining a running (m, ℓ, O) state:

1. Process new block → update running max: $m_{\text{new}} = \max(m_{\text{old}}, \max(s_{\text{new}}))$
2. Rescale old sum: $\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}} + \text{new terms}$

3. Rescale output: $O_{\text{new}} = \text{rescaled}(O_{\text{old}}) + \text{new contribution}$

This is mathematically exact — no approximation. It enables block-by-block processing where each block fits in SRAM, never needing the full $n \times n$ matrix in memory.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

29.13 LoRA and PEFT Questions

Q: Why does LoRA work? What theoretical insight justifies low-rank updates?

Answer: Aghajanyan et al. [3] showed that fine-tuning operates in a very low *intrinsic dimensionality* — the effective parameter space for a fine-tuning task is far smaller than the model's total parameter count. A 175B model may have intrinsic dimensionality <10,000 for a given task. LoRA directly exploits this: by constraining updates to rank r ($W' = W + BA$, $B \in \mathbb{R}^{d \times r}$), it restricts learning to an r-dimensional subspace per weight matrix. Since the true task subspace is low-dimensional, this loses almost nothing while reducing trainable parameters by 100–1000×.
Intuition: Fine-tuning doesn't change what the model "knows" (the full-rank W stays frozen); it only adjusts *how* existing knowledge is combined for the new task — a low-rank perturbation.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: Compare QLoRA vs. full LoRA vs. full fine-tuning for a 70B model. When would you choose each?

Answer:

Method	Memory	GPUs	Quality	Use When
Full fine-tune	560+ GB	8+ A100	Best	Unlimited budget; pre-training continuation
LoRA ($r = 16$)	145 GB	2 A100	95–98%	Good budget; general fine-tuning
QLoRA ($r = 16$)	44 GB	1×48GB	93–96%	Single-GPU; prototyping; constrained resources

Decision tree: (1) If task requires deep knowledge change → full fine-tune. (2) If adapting to new style/format → LoRA. (3) If memory-constrained or rapid iteration → QLoRA. (4) If rank matters: start $r = 16$; increase if training loss plateaus above full fine-tune level.
Review: *Chapters 1 and 10 (LLM Architecture; SFT Best Practices).*

Q: What is DoRA and why does it outperform standard LoRA?

Answer: DoRA (Weight-Decomposed Low-Rank Adaptation) decomposes W into magnitude $\|W\|$ and direction $W/\|W\|$, then applies LoRA only to the direction component:

$$W' = m \odot \frac{W + BA}{\|W + BA\|}$$

where m (magnitude) is also trainable but as a simple scalar per output neuron.
Why it helps: Full fine-tuning naturally updates both magnitude and direction independently. Standard LoRA couples them (the low-rank update changes both simultaneously in a constrained way). DoRA decouples them, giving LoRA the same "degrees of freedom" structure as full fine-tuning. Result: 1–3% improvement on reasoning tasks with no extra compute at inference (merge adapters).
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

29.14 Model Compression Questions

Q: Explain AWQ. Why does protecting 1% of weights preserve 99% of quality?

Answer: AWQ (Activation-Aware Weight Quantization) observes that weight importance is highly non-uniform: weights that multiply large activations contribute disproportionately to the output. The key insight: $\|W \cdot X\|$ depends on both W and X. A small weight multiplied by a large activation matters more than a large weight multiplied by a near-zero activation.
AWQ identifies the top 1% of "salient" channels (those with consistently large activation magnitudes across calibration data) and protects them by scaling: multiply salient channels by a factor $s > 1$ before quantization (then divide by s in the activation). This reduces relative quantization error for important channels.
Result: 4-bit quantization with <1% quality loss on 70B models, because the 99% of non-salient weights can tolerate aggressive quantization.
Review: *Chapters 1 and 2 (LLM Architecture; Systems Foundations).*

Q: When should you use FP8 vs. 4-bit quantization vs. BF16?

Answer:

- **BF16**: Training (policy model in RLHF), when precision matters. Default for any model being updated by gradients.
- **FP8 (E4M3)**: H100 training with Transformer Engine (2× throughput, <0.5% quality loss). Also for inference on H100 when you need maximum throughput.
- **INT8/FP8 inference**: Frozen models in RLHF (reference model, reward model) — not being trained, so reduced precision is safe.
- **4-bit (AWQ/GPTQ)**: Inference serving at scale. Best memory/quality tradeoff for deployment. Also for QLoRA base model.
- **2-bit**: Experimental; edge deployment where memory is extreme constraint. Quality loss 5–10%.

Rule: quantize as aggressively as possible for inference, keep BF16 (or FP8 on H100) for training.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Explain NVIDIA 2:4 structured sparsity. What's the speedup and constraint?

Answer: 2:4 sparsity means: in every group of 4 consecutive elements, exactly 2 must be zero. This is enforced at the weight level.
Hardware support: A100/H100 Tensor Cores have dedicated 2:4 sparse GEMM instructions that skip the zero elements, achieving exactly **2× throughput** with no software overhead.
Constraint: You must achieve *exactly* 50% sparsity in this specific pattern. You can't have 30% or 70% sparsity; you can't have arbitrary sparsity patterns. The pruning must respect the 4-element group structure.
How to achieve it: After training (or during fine-tuning), for each group of 4 weights, zero out the 2 smallest by magnitude. Then fine-tune for a few hundred steps to recover quality. Quality loss: typically <1% for large models (70B+).
Review: *Chapter 2 (Systems Foundations for LLMs).*

29.15 Mixture of Experts Questions

Q: Mixtral 8x7B has 47B total parameters but only 13B active per token. Explain how this works and why it's efficient.

Answer: Mixtral replaces each FFN layer with 8 parallel expert FFNs (each ∼7B params for the FFN portion). A router network selects the Top-2 experts per token.
Why 47B total: Attention layers are shared (not replicated) = ∼5B. FFN experts: 8 × ∼5.25B = 42B. Total: ∼47B.
Why 13B active: Per token, only 2 experts fire. Active params = attention (∼5B) + 2 FFN experts (∼2 × 5.25B) ≈ 13B.
Why efficient: Compute cost scales with *active* params (13B), matching a 13B dense model. But capacity (knowledge stored) scales with *total* params (47B), matching much larger models. Result: Mixtral matches Llama-2 70B quality at 13B compute cost.
Memory cost: Still need all 47B params in memory (all experts loaded), so memory = 47B model, but compute = 13B model.
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

Q: What is the load balancing problem in MoE and how is it solved?

Answer: Without constraints, the router tends to send most tokens to 1–2 "popular" experts (rich-get-richer dynamics). This causes:

- Capacity waste: 6 of 8 experts are unused, model effectively shrinks to 2-expert size.
- Compute imbalance: If each expert is on a different GPU, popular experts become bottlenecks while others idle.

Solution: Auxiliary load-balancing loss: $\mathcal{L}_{\text{bal}} = \alpha \cdot N \sum_{i=1}^{N} f_i \cdot p_i$, where f_i = fraction of tokens routed to expert i, p_i = mean router probability for expert i. This penalizes uneven distributions.
Alternative: Expert capacity factor — hard cap on max tokens per expert per batch. Overflow tokens are dropped or re-routed.
Typical α: 0.01–0.1 (small enough not to hurt main loss, large enough to prevent collapse).
Review: *Chapter 1 (LLM Architecture and Optimization Methods).*

29.16 Diversity in Training Questions

Q: What happens if all N responses in a GRPO group are identical?

Answer: If all N responses are identical: all rewards r_i are equal, so $\sigma_G = 0$ and advantages $\hat{A}_i = (r_i - \mu_G)/\sigma_G$ are undefined (division by zero). In practice, implementations set $\hat{A}_i = 0$ for all, meaning **zero learning signal** — the step is wasted.
Prevention:

1. **Temperature**: Use $\tau = 0.7$–1.0 during generation (not greedy).
2. **Large N**: $N = 8$–16 increases probability of diverse responses.
3. **Duplicate rejection**: DAPO's approach — reject duplicate responses and resample.
4. **Frequency penalty**: Penalize repeated n-grams during generation.
5. **Monitor**: Track unique-response ratio per group. If <50%, increase temperature.

Review: *Chapter 7 (GRPO).*

Q: Explain the diversity-quality tradeoff in RLHF. How do you detect mode collapse?

Answer: **Tradeoff**: High diversity (high entropy/temperature) = varied but potentially random/low-quality responses. Low diversity = consistent but repetitive, reward-hacked responses. **Detecting mode collapse** (all should be monitored during training):

1. **Response entropy**: Compute per-token entropy $H = -\sum p_i \log p_i$. If dropping rapidly → collapse.
2. **Unique n-gram ratio**: Fraction of unique 4-grams across responses to same prompt. Healthy: >0.6.
3. **Reward distribution width**: If σ(rewards) shrinks to near-zero → all responses are the same quality → likely identical.
4. **KL divergence**: If $D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ is growing rapidly, the policy is moving far from reference → often toward a narrow mode.
5. **Length histogram**: If all responses converge to same length → template behavior.

Fix: Increase KL coefficient β, increase entropy bonus, increase sampling temperature, or rollback to earlier checkpoint.
Review: *Chapters 7 and 9 (GRPO; Reward Model Training).*

29.17 Speculative Decoding Questions

Q: Speculative decoding claims "no quality loss." How can generating tokens differently produce identical output distribution?

Answer: The acceptance/rejection scheme guarantees distributional equivalence:
For each draft token $\hat{x}$ with draft probability $q(\hat{x})$ and target probability $p(\hat{x})$:

- Accept with probability $\min(1, p(\hat{x})/q(\hat{x}))$
- On rejection: sample from the *residual distribution* $\propto \max(0, p(x) - q(x))$

This is mathematically equivalent to sampling directly from p (the target). Proof sketch: the probability of outputting token x is $q(x) \cdot \min(1, p(x)/q(x)) + P(\text{reject}) \cdot \frac{\max(0,p(x)-q(x))}{\sum_y \max(0,p(y)-q(y))} = p(x)$.
The speedup comes from amortizing: when the draft is good (high acceptance), multiple tokens are confirmed in one target forward pass. The guarantee holds regardless of draft quality — bad drafts just give lower speedup (more rejections), not worse quality.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Compare Medusa vs. Eagle for speculative decoding. When would you choose each?

Answer:
Medusa: Adds k parallel prediction heads to the target model. Each head independently predicts token at position $t + i$. *Pro*: No separate model, <1% memory overhead. *Con*: Heads predict independently — cannot condition position $t + 2$ on what was predicted at $t + 1$. Acceptance rate: 60–80%.
Eagle: Lightweight autoregressive decoder on target model's hidden states. Draft tokens are generated autoregressively (each conditioned on previous). *Pro*: Captures inter-token dependencies → 85–95% acceptance rate. *Con*: Slightly more memory (small decoder) and sequential draft generation.
Choose Medusa when: Memory is extremely tight; simple integration; moderate speedup is sufficient (2–2.5×).

Choose Eagle when: Maximum speedup needed (3–4×); can afford small extra model; latency-critical single-stream generation.
Choose N-gram when: Repetitive outputs (code, structured data); zero cost; no training needed.
Review: *Chapter 2 (Systems Foundations for LLMs).*

Q: Why does speculative decoding NOT help at high batch sizes?

Answer: At high batch sizes ($\geq$64), autoregressive generation is already *compute-efficient*: the weight-read cost is amortized across many sequences. The arithmetic intensity approaches the roofline ridge point.
Speculative decoding adds overhead:

1. Draft generation cost (even if small model, it's not free at high batch)
2. Verification forward pass processes k extra tokens per sequence (batch $\times$ k tokens)
3. Memory for draft model or Medusa heads
4. Rejected tokens waste compute

At batch=1 (latency-bound, memory-bound): speculation turns 1 token/step into 3–4 tokens/step — huge win.
At batch=128 (already compute-efficient): the extra tokens from speculation barely help throughput because the GPU is already near-saturated. The overhead may even *reduce* throughput.
Rule: Speculative decoding is for latency (small batch); batching is for throughput (large batch). Don't combine them.
Review: *Chapter 2 (Systems Foundations for LLMs).*

29.18 Agentic RL Questions

Q: Why does standard RLHF (single-turn PPO/DPO) fail for multi-step agents?

Answer: Standard RLHF optimizes for single-turn quality: given a prompt, produce one good response. Multi-step agents face fundamentally different challenges:

1. **Credit assignment**: In a 50-step trajectory, which step caused the failure? Single-turn reward assigns credit to the entire response uniformly; multi-step needs *per-step* credit.
2. **Sparse rewards**: Success/failure only at trajectory end. PPO's GAE assumes intermediate rewards; without them, advantage estimates are noisy.
3. **Action space**: Actions are structured tool calls (JSON), not just token sequences. The model must learn syntax + semantics + strategy simultaneously.
4. **Non-stationarity**: The environment changes with each action (tool outputs modify state). Each step has a different "prompt" unlike single-turn where input is fixed.
5. **Exploration**: Agent must discover novel tool-use strategies, not just rephrase text.

Solution: Trajectory-level GRPO (rank complete trajectories), process reward models (per-step feedback), or filtered SFT on successful trajectories.
Review: *Chapter 12 (LLM Agentic Training).*

Q: Explain how GRPO is adapted for agentic training. What are the key differences from single-turn GRPO?

Answer: Single-turn GRPO: generate N responses to a prompt, rank by reward, compute advantages.
Agentic GRPO differences:

1. **Unit of generation**: A full *trajectory* (10–100 steps) instead of a single response. Each trajectory is one "sample" in the group.
2. **Reward**: Terminal (task success/failure) or trajectory-level (sum of step rewards). NOT per-token.
3. **Masking**: Only compute policy loss on the agent's outputs (reasoning + tool calls). Mask tool *outputs* (environment responses) from gradient computation.
4. **Group size**: Typically smaller ($N = 4$–8) because trajectories are expensive (many forward passes per trajectory).
5. **KL penalty**: Applied per-step to prevent drift from SFT policy at each decision point.
6. **Length normalization**: Normalize by number of agent *actions* (not tokens) to avoid penalizing thorough reasoning.

Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q: Compare STaR and Reflexion and ReAct for agents.

Answer:
STaR (Self-Taught Reasoner): Generate reasoning chains → filter by correctness → fine-tune on correct ones. *Use when*: You have verifiable tasks (math, code) and want to bootstrap reasoning from a base model without RL.
Reflexion: After failure, generate verbal feedback ("What went wrong?") → retry with reflection in context. No weight updates. *Use when*: Inference-time improvement; limited compute for training; tasks where self-diagnosis is possible.
ReAct: Interleave Reasoning (think) + Acting (tool use) in a structured loop. *Use when*: Multi-step tool use tasks; you need transparency (reasoning traces are interpretable); the agent must decide between thinking and acting.
Key differences:

	STaR	**Reflexion**	**ReAct**
Updates weights?	Yes (SFT)	No (in-context)	No (prompting)
Multi-step?	No (single reasoning chain)	Yes (retry loops)	Yes (think-act cycles)
Tools?	No	Optional	Yes (required)
Best for	Reasoning improvement	Error recovery	Tool-augmented tasks

Review: *Chapters 12 and 18 (LLM Agentic Training; Agent Design Patterns).*

Q: Why is GRPO preferred over PPO for a research agent?

Answer: For a research agent with 20–100 step trajectories:
PPO requires a value model: $V(s_t)$ must predict expected total reward from the current state. For research (where state = 128K tokens of context including papers, code, and results), training an accurate value function is extremely difficult — the value of "having read 3 papers and written partial code" is hard to predict.
GRPO avoids value estimation entirely: It generates N complete trajectories per research question and uses within-group ranking as the advantage. No need to predict intermediate value — just compare outcomes.
Additional reasons:

- Research quality is binary-ish (good report vs. bad report) — ranking is natural.
- Trajectories are long and expensive; GRPO's $N = 4$ is manageable; PPO would need many rollouts for stable value estimates.
- Terminal reward is sparse; GAE with sparse rewards gives noisy per-step advantages anyway.

Review: *Chapters 7 and 12 (GRPO; LLM Agentic Training).*

Q: Design a reward function for a coding agent. What reward hacking risks exist?

Answer: **Reward design**:

$$R = 0.5 \cdot R_{\text{tests}} + 0.2 \cdot R_{\text{quality}} + 0.2 \cdot R_{\text{efficiency}} + 0.1 \cdot R_{\text{safety}}$$

- R_{tests}: Fraction of unit tests passing (0–1). Ground-truth verifiable.
- R_{quality}: LLM judge on code style, documentation, maintainability.
- $R_{\text{efficiency}}$: $\max(0, 1 - \text{steps}/30)$ — bonus for finishing quickly.
- R_{safety}: No dangerous operations (rm -rf, network access outside sandbox).

Reward hacking risks:

1. **Hardcoded outputs**: Agent learns to print expected test outputs directly without computing them. *Fix*: Randomize test inputs; test on held-out cases.
2. **Test deletion**: Agent modifies/deletes failing tests. *Fix*: Sandbox tests as read-only.
3. **Trivial solutions**: Agent writes minimal code that passes tests but doesn't generalize. *Fix*: Large, diverse test suites; property-based testing.
4. **Efficiency gaming**: Agent skips reasoning steps to maximize efficiency bonus. *Fix*: Minimum quality threshold before efficiency bonus applies.

Review: *Chapters 9, 12, and 19 (Reward Model Training; LLM Agentic Training; Agentic Environments).*

29.19 Listwise Rewards and Advanced RM Questions

Q: Explain the Plackett-Luce model. How does it generalize Bradley-Terry?

Answer: Bradley-Terry models *pairwise* preferences: $P(y_1 \succ y_2) = \sigma(r(y_1) - r(y_2))$.
Plackett-Luce models *full rankings* of K items as sequential selection:

$$P(\pi) = \prod_{i=1}^{K} \frac{e^{r(y_{\pi(i)})}}{\sum_{j=i}^{K} e^{r(y_{\pi(j)})}}$$

Interpretation: Sequentially pick the best remaining item. Position 1 = softmax over all K; position 2 = softmax over remaining $K - 1$; etc.
Generalization: For $K = 2$, PL reduces exactly to BT: $P(y_1 \succ y_2) = \frac{e^{r(y_1)}}{e^{r(y_1)} + e^{r(y_2)}} = \sigma(r(y_1) - r(y_2))$.
Advantage: A ranking of $K = 8$ items provides $\binom{8}{2} = 28$ implicit pairwise comparisons plus relative margin information — much richer training signal than a single pair.
Review: *Chapter 9 (Reward Model Training).*

Q: What is a Process Reward Model (PRM) and when is it better than an Outcome Reward Model (ORM)?

Answer: **ORM**: Scores the final output only. $r(x, y_{\text{final}})$ = one scalar for the complete response.
PRM: Scores each *step* of reasoning. $r(x, y_{\text{step } t})$ = scalar per intermediate step.
PRM is better when:

1. **Long reasoning chains**: Math problems with 10+ steps. ORM can't tell which step went wrong; PRM provides per-step credit assignment.
2. **Search/verification**: PRM enables tree search (beam search over reasoning steps, prune branches with low step-reward).
3. **Training signal density**: PRM gives T rewards per trajectory (one per step) vs. ORM's single reward → lower variance advantage estimates.

ORM is better when: Tasks are short (single-turn); step boundaries are unclear; annotation cost for per-step labels is prohibitive.
PRM annotation: Can be automated via "Math-Shepherd" approach: for each step, complete the solution from that point multiple times. If completions from step t succeed but completions from step $t+1$ fail, step $t+1$ is likely wrong.
Review: *Chapters 9 and 13 (Reward Model Training; RL for Large Reasoning Models).*

29.20 RL for Large Reasoning Models Questions

Q: Why does DeepSeek-R1 not use a Process Reward Model despite training on long reasoning chains?

Answer: DeepSeek-R1 uses only **outcome-based rewards** (accuracy + format) for several reasons:

1. **Verifiable tasks**: Math and code have deterministic ground-truth answers. The binary accuracy reward provides sufficient signal even for long chains.
2. **PRM failure modes**: Step-level reward models introduce their own reward hacking — the model can learn to produce steps that "look correct" to the PRM without actually being correct.
3. **GRPO's group normalization**: By sampling G completions per prompt and normalizing advantages within each group, GRPO naturally provides relative signal about which reasoning *strategies* work, even without per-step rewards.
4. **Emergent self-correction**: With outcome-only rewards, the model learns to self-correct within chains (the "aha moment"), which wouldn't emerge if per-step rewards micromanage the reasoning process.

Key insight: The verifiability of the task domain is what makes PRMs unnecessary — for subjective tasks (creative writing), outcome-only rewards may not suffice.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: Explain the test-time compute scaling law and its implications for model deployment

Answer: The test-time compute scaling law states:

$$\text{Accuracy}(C_{\text{train}}, C_{\text{test}}) \approx f(\alpha \log C_{\text{train}} + \beta \log C_{\text{test}})$$

Implications:

1. **Compute equivalence**: A 7B model with 64× more inference tokens can match a 70B

model with 1× tokens on reasoning tasks.

2. **Adaptive allocation**: Easy questions get short chains (cheap); hard questions get long chains (expensive). Average cost is lower than always using a large model.
3. **Deployment flexibility**: Instead of one large model, deploy a smaller reasoning model and scale inference compute per-query based on difficulty.
4. **Diminishing returns**: The log relationship means doubling test-time compute gives diminishing accuracy gains — there's an optimal allocation between training and inference compute.

The "overthinking" failure: Very long chains can *decrease* accuracy due to error accumulation and attention dilution. Optimal chain length depends on problem difficulty.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: How does MCTS (Monte Carlo Tree Search) apply to LLM reasoning?

Answer: MCTS for reasoning treats each partial solution as a tree node:
Four phases per iteration:

1. **Selection**: Navigate from root using UCB: $\text{UCB}(s) = Q(s) + c\sqrt{\frac{\ln N(\text{parent})}{N(s)}}$
2. **Expansion**: Generate new reasoning steps (child nodes) from the LLM
3. **Simulation**: Complete the solution from the new node (rollout)
4. **Backpropagation**: Update Q-values along the path based on final correctness

Key differences from game MCTS:

- **Branching factor**: Reasoning has enormous branching factor (any next sentence is possible). Practical implementations use the LLM's top-k outputs to limit branches.
- **Value function**: A trained PRM estimates partial solution quality, replacing random rollouts.
- **Step granularity**: Each "step" might be one sentence, one equation, or one logical inference — choosing granularity matters.

Used by: AlphaProof (math olympiad), and hypothesized for OpenAI o1/o3's hidden reasoning.
Review: *Chapter 13 (RL for Large Reasoning Models).*

Q: Compare distillation vs direct RL for creating small reasoning models

Answer:
Distillation (DeepSeek-R1-Distill approach):

- Generate reasoning chains from large model (R1-671B)
- SFT small model on these chains
- Result: Small model mimics large model's reasoning *format*
- Pro: Cheap (just SFT). Con: May learn surface patterns not true reasoning.

Direct RL on small model:

- Train small model with GRPO/PPO against verifiable rewards

- Model discovers its own reasoning strategies
- Pro: Genuine capability. Con: Much more compute; may not converge for very small models.

Empirical finding: R1-Distill-7B (distilled) outperforms direct-RL-7B on most benchmarks. The reasoning chains from the large model provide such strong supervision that SFT alone is competitive. However, distilled models show less generalization to truly novel problem types.
Best practice: Distill first (cheap baseline), then optionally run RL on the distilled model for further gains ("distill + RL" combo used by Qwen).
Review: *Chapter 13 (RL for Large Reasoning Models).*

29.21 LLM Evaluation Questions

Q: Derive the ELO rating update rule and explain why Chatbot Arena uses it

Answer: **ELO derivation**:
Expected score of player A vs B: $E_A = \frac{1}{1+10^{(R_B-R_A)/400}}$ (logistic model).
After a match with actual score $S_A \in \{0, 0.5, 1\}$: $R'_A = R_A + K(S_A - E_A)$
The K-factor controls update magnitude (higher K = more reactive to recent results).
Why Chatbot Arena uses ELO:

1. **Transitivity**: If model A beats B and B beats C, ELO predicts A beats C. This gives a total ordering from pairwise comparisons.
2. **Online updates**: New models can be added without re-evaluating all pairs. Each new comparison updates ratings incrementally.
3. **Confidence**: After N comparisons, rating uncertainty shrinks as $O(1/\sqrt{N})$. Standard error: SE $\approx \frac{400}{\sqrt{N}}$.
4. **Human preference capture**: Real users provide honest preferences without needing to articulate criteria. The aggregate reveals true model quality.

Chatbot Arena specifics: Uses Bradley-Terry MLE (equivalent to ELO at convergence) with bootstrap confidence intervals. Style-controlled ELO removes length/formatting bias.
Review: *Chapter 14 (LLM Evaluation).*

Q: What is the pass@k metric for code generation and why is the unbiased estimator important?

Answer: **pass@k** = probability that at least one of k generated samples passes all test cases.
Naive (biased) estimator: Generate k samples, check if any passes. Problem: high variance, expensive (need many trials per problem).
Unbiased estimator (Chen et al., 2021): Generate $n \geq k$ samples, count c that pass:

$$\text{pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

Why unbiased matters:

1. Generates n samples once (e.g., $n = 200$) and computes pass@1, pass@10, pass@100 from the same samples
2. No need to repeat the entire evaluation k times
3. Statistically exact (combinatorial argument: fraction of k-subsets with no correct sample)
4. Numerically stable computation via log-space: $\text{pass@}k = 1 - \exp\left(\sum_{i=0}^{k-1} \log(n-c-i) - \log(n-i)\right)$

Intuition: If 50/200 samples pass ($c = 50$, $n = 200$), pass@1 ≈ 0.25, pass@10 ≈ 0.94. The estimator counts what fraction of k-sized draws would contain at least one success.
Review: *Chapters 14 and 19 (LLM Evaluation; Agentic Environments).*

Q: How do you detect and mitigate benchmark contamination?

Answer: **Contamination**: Training data contains benchmark test examples (or close paraphrases), inflating scores.
Detection methods:

1. **N-gram overlap**: Check if training data contains exact or near-exact matches to test items. 8-gram overlap with >80% coverage is suspicious.
2. **Canary strings**: Insert unique identifiers in test sets; check if model can reproduce them.
3. **Rephrased benchmarks**: Create semantically equivalent but textually different versions of benchmarks. Large accuracy drops suggest memorization.
4. **Temporal analysis**: Model performance on pre-training-cutoff vs post-cutoff test items. Unusually high performance on old items suggests contamination.
5. **Membership inference**: Statistical tests for whether specific examples were in training data.

Mitigation:

- **Dynamic benchmarks**: Regularly generate new test items (LiveCodeBench, Chatbot Arena)
- **Private test sets**: Keep test items secret (LMSYS)
- **Decontamination during training**: Remove detected overlaps from training data
- **Report contamination analysis**: Disclose overlap metrics alongside benchmark scores

Review: *Chapter 14 (LLM Evaluation).*

Q: Explain position bias in LLM-as-Judge and how to mitigate it

Answer: **Position bias**: When using an LLM to judge two responses (A vs B), the model systematically prefers the response in a particular position (usually first or last), regardless of quality.
Empirical magnitude: GPT-4 shows 10–15% position bias; Claude shows 5–10%. Smaller models show larger bias.
Mitigation strategies:

1. **Position swapping**: Judge each pair twice (A-B and B-A). Final decision = majority. If disagreement, mark as "tie." This eliminates systematic position bias but doubles cost.
2. **Multi-judge panels**: Use 3+ different models as judges. Majority vote reduces individual model biases.
3. **Reference-guided**: Provide a rubric or reference answer. Judges score each response independently against the rubric, then compare scores (eliminates pairwise comparison entirely).
4. **Calibrated prompting**: Add explicit instruction: "The order of presentation is random and should not influence your judgment."

Additional biases: Verbosity bias (prefers longer responses), self-enhancement bias (models prefer their own outputs), authority bias (defers to responses that cite sources).

Review: *Chapter 14 (LLM Evaluation).*

29.22 Agentic Memory Questions

Q: Compare the four types of agentic memory and when each is critical

Answer:

Type	What it stores	Access pattern	Critical when
Working	Current context/scratchpad	Always in context	Complex multi-step reasoning
Episodic	Past experiences	Retrieved by similarity	Learning from past mistakes
Semantic	Facts and knowledge	Retrieved by concept	Domain-specific tasks
Procedural	Skills and patterns	Triggered by task type	Repeated tool-use

Key insight: These aren't independent — they interact. Episodic memory feeds semantic memory (generalizing from episodes to facts). Procedural memory is refined by episodic feedback (learning which tool sequences work). Working memory orchestrates retrieval from all other types.
MemGPT analogy: Working = hot (in-context), Episodic/Semantic = warm (vector store), Procedural = cold (archived policies). The agent itself decides when to page information in/out.
Review: *Chapter 16 (Agentic Memory Systems).*

Q: How does temporal decay work in memory retrieval and why is it important?

Answer: **Temporal decay** down-weights older memories during retrieval:

$$\text{score}(m) = \alpha \cdot \text{similarity}(q, m) + (1 - \alpha) \cdot \text{recency}(m)$$

where $\text{recency}(m) = e^{-\lambda \cdot \Delta t}$ with Δt = time since last access.
Why it's important:

1. **Relevance decay**: User preferences change. A preference from 6 months ago may be outdated.
2. **Contradiction resolution**: When old and new information conflict, recency bias naturally prefers current truth.
3. **Retrieval efficiency**: Without decay, memory grows unbounded and retrieval returns increasingly irrelevant ancient items.
4. **Cognitive plausibility**: Humans forget too — recent events are more accessible. This mirrors the spacing effect.

Access-based refresh: When a memory is retrieved and used, its timestamp updates (similar to LRU caching). Frequently-accessed memories stay "fresh" regardless of creation date.
Decay rate tuning: λ depends on domain. Customer service: high decay (preferences change fast). Legal/medical: low decay (facts persist). Can be learned via RL.
Review: *Chapter 16 (Agentic Memory Systems).*

Q: How can RL be used to train memory operations?

Answer: Memory operations (write/read/update/delete) can be actions in the agent's MDP:
Formulation:

- **State**: Current context + memory state
- **Actions**: Standard actions + `memory_write(key/value)`, `memory_read(query)`, `memory_delete(key)`

- **Reward**: Task success (did memory help?) + memory efficiency penalty (fewer reads = better)

What RL learns:

1. **What to store**: Important information (API keys/user preferences) vs ephemeral details
2. **When to retrieve**: Before answering domain questions vs during general chat
3. **Compression policy**: When to summarize old memories vs keep verbatim
4. **Forgetting**: When old information is stale and should be removed

Training signal: Counterfactual — "would the agent have succeeded if it hadn't stored/retrieved this memory?" Implemented via trajectory comparison: trajectories with good memory use get higher rewards.
Challenge: Delayed reward — storing information now may only help 100 steps later. Requires long-horizon credit assignment (GAE with high λ).
Review: *Chapters 12 and 16 (LLM Agentic Training; Agentic Memory Systems).*

29.23 Agent Orchestration Questions

Q: Explain the context budget problem and how to solve it with dynamic allocation

Answer: **The problem**: An agent has context window L tokens but needs space for:

$$C = S + M + T + H + R \leq L$$

where S = system prompt, M = memory/retrieved context, T = tool descriptions, H = conversation history, R = reserved for response.
As conversations grow, H increases and pushes out other components.
Dynamic allocation strategy:

1. **Fixed minimums**: $S_{\min}$, $R_{\min}$ are non-negotiable
2. **Adaptive history**: Summarize old turns when $H > H_{\max}$. Keep last k turns verbatim; summarize the rest.
3. **On-demand tools**: Only include tool descriptions relevant to current query (not all 50 tools). Use a classifier or embedding similarity to select top-k tools.
4. **Lazy memory**: Retrieve memory only when needed (after analyzing the query) rather than pre-loading.

Overflow handling: When total exceeds L even after compression:

- Drop least-important tool descriptions
- Aggressively summarize history to 1-sentence-per-turn
- Reduce memory slots
- If still over: truncate with warning to user

Pre-flight check: Always count tokens BEFORE calling the LLM. Never discover overflow at inference time.
Review: *Chapter 17 (Agent Harness – Context Management and Orchestration).*

Q: Compare ReAct vs Plan-and-Execute orchestration patterns

Answer:
ReAct (Reason + Act):

- Loop: Thought → Action → Observation → Thought → ...
- Each step decides the next action based on all previous observations
- **Pro**: Adaptive — can change direction based on tool outputs
- **Con**: Myopic — no upfront planning; can get stuck in loops; each LLM call sees entire history (expensive)

Plan-and-Execute:

- Phase 1: Generate full plan (list of steps)
- Phase 2: Execute steps sequentially (simpler executor; possibly cheaper model)
- Phase 3: Re-plan if execution fails
- **Pro**: Efficient (planning once is cheaper than reasoning every step); parallelizable independent steps
- **Con**: Brittle plans — if early steps fail the plan may be invalid. Re-planning adds latency.

When to use which:

- ReAct: Exploratory tasks; unknown environments; tasks where each step's result determines the next
- Plan-and-Execute: Well-defined tasks; known tool set; parallelizable sub-tasks; cost-sensitive deployments
- **Hybrid**: Plan at high level then ReAct within each plan step (LangGraph's recommended pattern)

Review: *Chapters 17 and 18 (Agent Harness; Agent Design Patterns).*

Q: How do you detect and prevent infinite loops in agent execution?

Answer: Agents can enter infinite loops when they repeat the same action expecting different results.
Detection methods:

1. **Max iteration guard**: Hard limit (e.g., 25 steps). Simple but loses work on genuinely long tasks.
2. **Action hash window**: Hash the last k (action/observation) pairs. If current hash matches a hash from the last w steps then loop detected.
3. **Semantic similarity**: Embed recent actions. If cosine similarity between consecutive actions exceeds threshold (>0.95) then likely stuck.
4. **Progress monitoring**: Define task-specific progress metrics. If no progress in N steps then intervene.

Recovery strategies:

1. **Inject hint**: Add system message: "You seem to be repeating actions. Try a different approach."

2. **Force different action**: Mask the repeated action from the action space for the next step.
3. **Escalate**: Return to user with partial results and ask for guidance.
4. **Backtrack**: Reset to a checkpoint before the loop began and try alternative path.

Best practice: Combine max iterations (safety net) + hash-based detection (early intervention) + graceful escalation (preserve user trust).
Review: *Chapters 17 and 18 (Agent Harness; Agent Design Patterns).*

29.24 MCP Protocol Questions

Q: Explain MCP's N+M architecture and why it matters for the agent ecosystem

Answer: **The N×M problem**: Without MCP, N agent frameworks must each implement integrations with M tools = $N \times M$ total integrations. Adding one new tool requires N implementations.
MCP's N+M solution: Standardize the interface. Each agent implements one MCP client (N total). Each tool implements one MCP server (M total). Total integrations = $N + M$.
Concrete example: 5 agent frameworks (LangChain/AutoGen/CrewAI/Claude/custom) × 20 tools (GitHub/Slack/DB/filesystem/...) = 100 integrations without MCP. With MCP: 5 clients + 20 servers = 25 implementations.
Why it matters:

1. **Tool reuse**: Build a tool server once; use from any MCP-compatible agent
2. **Agent portability**: Switch from Claude to a custom agent without rewriting tool integrations
3. **Ecosystem growth**: Lower barrier to adding new tools incentivizes the community to build more
4. **Composability**: Connect multiple servers to one agent dynamically at runtime

Analogy: USB standardized peripheral connections. Before USB: every device had a proprietary connector. After USB: one port fits all. MCP does the same for agent-tool connections.
Review: *Chapter 20 (Model Context Protocol).*

Q: What are MCP's four core primitives and when do you use each?

Answer:

Primitive	Direction	Purpose	Example
Tools	Client → Server	Execute actions	`create_issue`; `query_db`
Resources	Client → Server	Read data	File contents; DB records
Prompts	Client → Server	Get templates	"Summarize this PR" template
Sampling	Server → Client	Request LLM gen	Server asks LLM to classify

Key distinctions:

- **Tools vs Resources**: Tools have *side effects* (create/modify/delete). Resources are *read-only*. This distinction matters for safety — an agent can freely read resources but must get approval for tools.
- **Sampling** reverses the direction: normally the client (agent) calls the server (tool). With Sampling the server asks the client's LLM for help. Use case: a code analysis server needs the LLM to interpret a code snippet.
- **Prompts** are metadata (reusable templates) not execution. They help the agent formulate better tool calls.

Review: *Chapter 20 (Model Context Protocol).*

29.25 Agent Communication (A2A) Questions

Q: How does Google's A2A protocol differ from MCP and when do you need both?

Answer: **Core distinction**:

- **MCP**: Agent ↔ Tool (structured function calls with defined schemas)
- **A2A**: Agent ↔ Agent (opaque task delegation — you don't know how the other agent works)

Key A2A concepts:

- **Agent Cards**: JSON describing what an agent can do (like a resume). Discovery mechanism.
- **Opaque execution**: Requester doesn't see internal reasoning of the delegate. Just sends task and gets result.
- **Task lifecycle**: submitted → working → completed/failed (with streaming updates via SSE)

When you need both:

1. An orchestrator agent uses **A2A** to delegate "research this topic" to a research agent
2. The research agent uses **MCP** to call web search and file read and database tools
3. Results flow back via A2A to the orchestrator

Architecture: A2A sits at the *inter-agent* layer; MCP sits at the *agent-tool* layer. A complete system uses both: A2A for coordination between agents and MCP for each agent's tool access.
Review: *Chapters 20 and 22 (MCP; Agent-to-Agent Communication).*

Q: What is the Contract Net Protocol and how does it apply to LLM agents?

Answer: The **Contract Net Protocol** (CNP) is a task allocation mechanism from distributed AI:
Steps:

1. **Announce**: Manager broadcasts task description to all available agents
2. **Bid**: Agents assess their capability and submit bids (confidence; estimated cost; estimated time)
3. **Award**: Manager selects best bid(s) based on criteria (capability/cost/availability)
4. **Execute**: Winning agent(s) perform the task
5. **Report**: Agent reports results back to manager

For LLM agents:

- **Bidding = self-assessment**: Each agent LLM evaluates "can I do this task well?" and provides a confidence score. This requires calibrated self-knowledge.
- **Specialization emerges**: Code agents bid high on code tasks; research agents bid high on research tasks. No central routing logic needed.
- **Load balancing**: If one agent is busy (high estimated time) others win the contract.

- **Failure handling**: If awarded agent fails then re-announce to remaining agents (automatic failover).

Limitation for LLMs: LLMs often overestimate their capabilities (hallucinate confidence). Bids should incorporate track record (historical success rate on similar tasks) not just self-reported confidence.
Review: *Chapters 22 and 23 (A2A; Multi-Agent Systems).*

29.26 Multi-Agent Systems Questions

Q: Compare centralized vs decentralized multi-agent architectures for LLMs

Answer:
Centralized (Supervisor):

- One orchestrator LLM routes tasks to specialist workers
- Clear control flow; easy to debug (inspect supervisor decisions)
- Single point of failure; supervisor becomes token bottleneck
- Best for: well-defined workflows; small agent teams (3–5 agents)

Decentralized (Peer-to-Peer):

- Agents communicate directly; no central coordinator
- Resilient (no single point of failure); scales horizontally
- Hard to debug (emergent behavior); potential for conflicts and deadlocks
- Communication scales $O(n^2)$ without structure
- Best for: resilient systems; large agent populations; creative tasks where emergent behavior is desired

Hybrid (Hierarchical): Tree structure with sub-managers. Combines benefits: local autonomy within groups and global coordination at the top. Communication scales $O(n \log n)$.
Decision framework: Use centralized if you need predictability and auditability. Use decentralized if you need resilience and creativity. Use hierarchical for large (>10 agent) systems.
Review: *Chapter 23 (Multi-Agent Systems).*

Q: What is CTDE and why is it important for training multi-agent LLM systems?

Answer: **CTDE** = Centralized Training; Decentralized Execution.
The problem: In multi-agent RL each agent's environment is non-stationary (other agents are changing their policies simultaneously). This makes independent training unstable.
CTDE solution:

- **During training**: A centralized critic has access to all agents' observations and actions: $V(s_1, s_2, \ldots, s_n, a_1, a_2, \ldots, a_n)$. This stabilizes training by removing non-stationarity from the value function.
- **During execution**: Each agent acts based only on its own observation: $a_i = \pi_i(o_i)$. No communication overhead at inference time.

For LLM agents: The centralized critic can be a reward model that evaluates the *joint* output of all agents (e.g., did the team of agents produce a correct software system?) while each agent is trained to maximize its contribution to the team reward using counterfactual credit assignment.

Practical challenge: Full CTDE requires all agents to train simultaneously with shared state — expensive for LLMs. Approximations: train agents in rounds (freeze others and train one) or use population-based training with periodic synchronization.
Review: *Chapter 23 (Multi-Agent Systems).*

29.27 Agent Development Framework Questions

Q: Compare LangGraph vs AutoGen vs CrewAI for building multi-agent systems

Answer:

Dimension	LangGraph	AutoGen / CrewAI
Orchestration	Explicit state graph (nodes + edges)	Implicit (conversation / role-based)
State mgmt	TypedDict schemas; checkpointing	Conversation history as state
Multi-agent	Graph with conditional routing	GroupChat / Crew
Debugging	Graph visualization; step replay	Chat logs
HITL	First-class (interrupt nodes)	Via approval tools
Production	LangGraph Cloud; persistence	Limited (AutoGen); growing (CrewAI)
Learning curve	High (graph concepts)	Low (AutoGen); Very low (CrewAI)

Choose LangGraph when: You need fine-grained control; complex conditional flows; production deployment with persistence and human-in-the-loop.
Choose AutoGen when: Rapid prototyping of multi-agent conversations; code execution agents; research experimentation.
Choose CrewAI when: Simple role-based teams; sequential task execution; quick demos; minimal code.
Choose none (custom) when: You need maximum performance/control; don't want framework lock-in; or have non-standard orchestration patterns.
Review: *Chapter 24 (Agent Development Frameworks).*

Q: How do you test and evaluate an agent system in production?

Answer: Agent testing follows a **testing pyramid**:
Level 1 — Unit Tests (fast; many):

- Test individual tools in isolation (mock LLM; verify tool logic)
- Test prompt templates (given context; verify correct prompt construction)
- Test parsers (given LLM output; verify correct extraction)

Level 2 — Integration Tests (medium speed):

- Test complete agent loops with deterministic inputs
- "Golden trajectory" tests: known-good execution traces that must reproduce
- Tool chain tests: verify multi-tool sequences work end-to-end

Level 3 — Behavioral Tests (slow; few):

- Does the agent follow safety constraints? (adversarial inputs)
- Does it ask for clarification when appropriate?

- Does it stay within token/cost budgets?

Production evaluation:

- **A/B testing**: Route 5% of traffic to new agent version
- **Shadow mode**: Run new agent alongside old; compare outputs without serving
- **LLM-as-judge**: Automated quality scoring of agent responses
- **User satisfaction**: Thumbs up/down; task completion rate; time-to-resolution

Key metric: **Task Success Rate** (TSR) — fraction of tasks the agent completes correctly without human intervention.
Review: *Chapters 14 and 24 (LLM Evaluation; Agent Development Frameworks).*

29.28 Agentic Environments Questions

Q: Design a reward function for a web browsing agent environment

Answer: For WebArena-style tasks (e.g., "find the cheapest flight from NYC to SF on Dec 15"):
Sparse reward (simple but hard to learn from):

$$r = \begin{cases} 1 & \text{if final page/state matches ground truth} \\ 0 & \text{otherwise} \end{cases}$$

Dense reward (better for training; harder to design):

1. **Progress reward**: $+0.1$ for each page that brings agent closer to goal (measured by text similarity to target state)
2. **Efficiency penalty**: -0.01 per action (encourages shorter trajectories)
3. **Milestone rewards**: $+0.3$ for reaching intermediate goals (e.g., navigating to flight search page)
4. **Invalid action penalty**: -0.05 for actions that produce errors (404; form validation failures)

Potential-based shaping (preserves optimal policy):

$$r_{\text{shaped}}(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s)$$

where $\Phi(s) = -\text{min_steps_to_goal}(s)$ (estimated by heuristic or learned value function).
Challenges: Partial observability (can't always tell if you're closer to goal); stochastic environments (page content changes); reward hacking (agent finds shortcuts that satisfy reward but not user intent).
Review: *Chapters 12 and 19 (LLM Agentic Training; Agentic Environments).*

Q: What makes SWE-bench a particularly challenging agent benchmark?

Answer: SWE-bench tests agents on real GitHub issues from popular Python repositories:
Why it's hard:

1. **Repository-scale context**: Agent must understand codebases with 100K+ lines. Cannot fit in context window — must explore and search and navigate.
2. **Underspecified tasks**: Issues are written by humans with implicit context. Agent must

infer what's actually needed.

3. **Multi-file edits**: Solutions often span multiple files with cascading dependencies.
4. **Test verification**: Must pass existing tests AND new tests that verify the fix.
5. **No hand-holding**: Unlike HumanEval (single function) SWE-bench requires full software engineering workflow: read issue → explore code → localize bug → implement fix → verify.

State of the art (2024–2025): Best agents solve ∼50% of SWE-bench Verified (curated subset). Full SWE-bench: ∼30%.
Key insight for training: SWE-bench exposes the gap between "coding ability" (writing correct functions) and "software engineering ability" (understanding systems; navigating codebases; making minimal changes). RL training on SWE-bench-style environments teaches agents exploration and planning strategies not just code generation.
Review: *Chapter 19 (Agentic Environments and Benchmarks).*

29.29 Agentic UI Framework Questions

Q: Compare chat-based vs canvas-based UI paradigms for agents

Answer:
Chat-based (ChatGPT; Claude default):

- Linear message stream: user → assistant → user → ...
- **Pro**: Familiar UX; natural for exploration and Q&A; easy to implement
- **Con**: Generated artifacts (code/documents) are buried in conversation. Hard to iterate on a specific artifact. Context gets lost in long conversations.

Canvas/Artifact-based (Claude Artifacts; ChatGPT Canvas; Cursor):

- Side panel displays generated content; chat panel for instructions
- Agent can create and edit and iterate on persistent artifacts
- **Pro**: Artifacts persist independently of chat. Direct editing by user. Version history.
- **Con**: More complex UI; requires artifact type detection; harder to implement streaming to both panels.

When to use which:

- Chat: brainstorming; Q&A; quick tasks; mobile interfaces
- Canvas: code generation; document writing; data analysis — any task with persistent output that needs iteration
- **Hybrid** (most modern UIs): Chat by default; auto-elevate to canvas when detecting code/document/visualization output

For agent training: The UI paradigm affects the reward signal. Canvas UIs provide explicit edit feedback (user modifies the artifact) which can be used for online learning.
Review: *Chapter 25 (Agentic UI Frameworks).*

Q: How do you design approval gates for human-in-the-loop agent systems?

Answer: Approval gates pause agent execution at critical points for human review.
Three-tier model:

1. **Auto-approve** (no gate): Safe reversible actions. Read operations; searches; calculations.
2. **Notify** (soft gate): Potentially impactful but recoverable. Send email; create draft; modify file. Agent proceeds but user is notified and can undo.
3. **Block** (hard gate): Irreversible or high-stakes. Delete data; send money; publish content; execute code with side effects. Agent MUST wait for explicit approval.

Design principles:

- **Minimize interruptions**: Too many gates = user abandons the agent. The 3-tier model lets most actions flow while catching dangerous ones.
- **Show context**: At approval gate display: what action; why (agent's reasoning); what will change; how to undo.
- **Batch approvals**: If agent needs 5 file writes present them together not one by one.
- **Timeout handling**: If user doesn't respond within T minutes either retry notification or proceed with safe default or abort gracefully.
- **Learning from approvals**: Track approval/rejection patterns. If users always approve a certain action type consider auto-promoting it.

Implementation: Tool annotations (MCP's `destructiveHint` and `readOnlyHint`) drive automatic gate assignment. Custom rules can override based on context.
Review: *Chapters 17 and 25 (Agent Harness; Agentic UI Frameworks).*

29.30 RAG and Agentic RAG Questions

Q: Explain Reciprocal Rank Fusion (RRF) and why it works for hybrid retrieval

Answer: RRF combines rankings from multiple retrieval systems without needing score calibration:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

where $r(d)$ is the rank of document d in retriever r, and $k = 60$ is a constant that prevents high-ranked documents from dominating.
Why it works:

1. **No score normalization needed**: BM25 scores are unbounded; dense similarity is in $[-1, 1]$. RRF uses only ranks, making them directly comparable.
2. **Robust to outliers**: A single retriever giving anomalously high scores doesn't dominate because $1/(k+1) \approx 0.016$ even for rank 1.
3. **Complementary signals**: BM25 catches exact keyword matches; dense retrieval catches semantic similarity. Documents ranked highly by both get boosted.

Example: Document d is rank 3 in BM25 and rank 7 in dense. RRF score $= 1/(60+3) + 1/(60+7) = 0.0159 + 0.0149 = 0.0308$. A document at rank 1 in one but rank 100 in the other gets $1/61 + 1/160 = 0.0226$ — lower despite having a top-1 ranking.
In practice: Hybrid (BM25 + dense + RRF) outperforms either alone on 85%+ of benchmarks.
Review: *Chapter 15 (Retrieval-Augmented Generation).*

Q: What is Agentic RAG and how does it differ from standard RAG?

Answer: **Standard RAG** follows a fixed pipeline: query → retrieve → generate. It has no ability to:

- Decide whether retrieval is needed at all
- Evaluate if retrieved documents are sufficient
- Reformulate queries when retrieval fails
- Combine information from multiple retrieval steps

Agentic RAG treats retrieval as an *action* in the agent's MDP:

- **Retrieve-or-not decision**: Agent assesses if it already knows the answer (skip retrieval for factual questions in its training data)
- **Query planning**: Decomposes complex questions into sub-queries ("What year did X happen?" + "Who was president then?")
- **Self-evaluation**: After retrieval, grades relevance. If insufficient, reformulates query or tries different source.
- **Multi-hop reasoning**: Retrieves → reasons → identifies knowledge gaps → retrieves again
- **Source routing**: Routes queries to appropriate knowledge bases (web for current events; internal docs for company info; code search for programming)

Key architectural difference: Standard RAG = deterministic pipeline. Agentic RAG = state machine with conditional transitions (LangGraph pattern with retrieve/grade/rewrite/generate nodes).

Trade-off: Agentic RAG is more accurate on complex queries but adds latency (multiple LLM calls for routing/grading). Use standard RAG for simple factual lookups; agentic RAG for multi-hop or ambiguous queries.
Review: *Chapters 15 and 17 (RAG; Agent Harness).*

Q: Compare Self-RAG and CRAG approaches to improving retrieval quality

Answer:
Self-RAG (Asai et al., 2023):

- Trains special *reflection tokens* into the LLM vocabulary
- At inference, model outputs tokens like [Retrieve], [IsRel], [IsSup], [IsUse]
- Model decides *when* to retrieve (not every query needs it)
- After retrieval, model self-grades: Is the retrieved passage relevant? Does my answer follow from it?
- **Training**: SFT on data augmented with reflection labels from GPT-4
- **Pro**: Single model handles everything. **Con**: Requires custom training.

CRAG (Corrective RAG, Yan et al., 2024):

- Uses a lightweight *retrieval evaluator* (separate model) to grade retrieved docs
- Three actions based on confidence: `Correct` (use as-is), `Ambiguous` (augment with web search), `Incorrect` (discard; fallback to web)
- Adds a *knowledge refinement* step: extract only relevant sentences from retrieved docs
- **Pro**: Works with any frozen LLM. **Con**: Extra model for evaluation; added latency.

Key difference: Self-RAG embeds retrieval decisions into the LLM itself (requires training). CRAG is a pipeline approach that wraps around any LLM (no training needed). Self-RAG is more elegant; CRAG is more practical for production with existing models.
Review: *Chapter 15 (Retrieval-Augmented Generation).*

Q: What is the lost-in-the-middle problem and how do you mitigate it?

Answer: **The problem**: When retrieved context is long (many passages), LLMs disproportionately attend to information at the *beginning* and *end* of the context, ignoring information in the middle. If the answer is in passage 5 of 10, the model may miss it.
Empirical evidence: Liu et al. (2023) showed that for 20-document retrieval, accuracy drops by 15–20% when the relevant document is in positions 5–15 vs positions 1–3.
Mitigation strategies:

1. **Re-rank and truncate**: Use a cross-encoder to re-rank, then only include top-3 most relevant passages (fewer = less lost-in-middle).
2. **Strategic ordering**: Place highest-relevance passages at the start AND end of context, low-relevance in the middle.
3. **Contextual compression**: Summarize each passage to 1–2 sentences before insertion. Less text = less position bias.
4. **Map-reduce**: Process each passage independently (map), then combine answers (reduce). Eliminates position effects entirely.

5. **Citation prompting**: Ask model to cite which passage it used. This forces attention to all passages.

6. **Chunk size reduction**: Smaller chunks mean fewer total chunks needed to cover the answer.

Best practice: Retrieve many (20+), re-rank to top 3–5, order by relevance (best first). This sidesteps the problem entirely for most use cases.
Review: *Chapter 15 (Retrieval-Augmented Generation).*

29.31 Loop Engineering Questions

Q: What are the five structural primitives of a production agent loop and what does each do?

Answer: Every production loop is built from five primitives that together implement inference-time optimization:

1. **Generator**: Produces a candidate output given the current context. This is the LLM call itself. The generator is the only component that consumes tokens; all others are deterministic.
2. **Verifier**: Evaluates the generator's output against a correctness criterion. Verifiers range from unit tests (binary pass/fail) to LLM-as-judge (scalar score) to tool execution (runtime feedback). The verifier's signal drives the loop.
3. **Terminator**: Decides when to stop. Termination conditions include: verifier passes, budget exhausted (max steps, max tokens, max wall-clock time), or stagnation detected (no progress in k steps). Without a terminator, loops run forever.
4. **State manager**: Maintains the loop's working memory across iterations—what has been tried, what failed, what partial results exist. Persists to external storage for crash recovery.
5. **Escalator**: Handles cases the loop cannot resolve autonomously. When the terminator fires without success, the escalator saves state and hands off to a human or higher-level orchestrator with a structured summary of what was attempted.

Why this matters: These five primitives map directly to the components of a policy optimization algorithm—generator $\approx$ policy, verifier $\approx$ reward function, terminator $\approx$ episode boundary, state manager $\approx$ replay buffer, escalator $\approx$ early stopping. Loop engineering is inference-time RL without weight updates.
Review: *Chapter 19 (Loop Engineering).*

Q: Explain the three parallel subagent fleet execution modes. When do you choose each?

Answer:

1. **Fan-out / fan-in**: Decompose the task into N independent subtasks, dispatch all simultaneously, wait for all to complete, then synthesize. Best when subtasks are truly independent and the synthesis step can handle partial failures (e.g., k-of-N completions). Failure rate: if each agent fails with probability p, the fleet fails only if all N fail (p^N).
2. **Speculative execution**: Dispatch N agents to attempt the *same* task with different strategies (different prompts, models, or tool sequences), then select the best result. Best for high-stakes single tasks where reliability matters more than cost. If each agent has a 20% failure rate, three speculative agents reduce failure to $0.2^3 = 0.8\%$.
3. **Pipeline parallelism**: Tasks form a dependency graph; the harness executes all tasks whose dependencies are satisfied simultaneously. As each task completes, it unlocks downstream tasks. Most general pattern—subsumes fan-out/fan-in as a two-level dependency graph. Best for workflows with known inter-task dependencies.

Isolation requirement: All three modes require per-agent isolation (git worktrees for file-system, separate context windows for memory) to prevent write conflicts and context contamination.
Budget management: Each agent needs an individual cap (steps, tokens, wall-clock) plus a fleet-level budget. When the fleet budget is exhausted, synthesize from completed results rather than waiting.
Review: *Chapter 19 (Loop Engineering).*

Q: What is context engineering and why does it matter more than prompt engineering for long-running loops?

Answer: **Prompt engineering** optimizes the *content* of a single system prompt. **Context engineering** optimizes *everything that fills the context window* across the entire loop lifetime: the system prompt, retrieved knowledge, tool definitions, conversation history, intermediate results, and reserved output space.
Why it matters more for loops: A single-turn prompt has a fixed context. A loop accumulates context with every step—tool outputs, error messages, intermediate reasoning, retry attempts. Without active management, the context window fills and quality degrades. The model begins to "forget" early instructions, repeat failed actions, or lose track of the original goal.
The context budget equation:

$$C \geq S_{\text{system}} + M_{\text{memory/RAG}} + T_{\text{tool defs}} + H_{\text{history}} + R_{\text{reserved output}}$$

Each term competes for the fixed budget C. Context engineering decides how to allocate it.
Key strategies:

- **Periodic compaction**: Every k steps, replace raw history with a structured summary. Lossy but necessary.
- **Importance-weighted retention**: Keep steps that changed state; discard no-ops (tool calls that returned empty results).
- **Sub-agent isolation**: Offload subtasks to fresh-context sub-agents; return only conclusions to the parent loop.
- **External memory**: Write intermediate results to files; keep only references in context.

Review: *Chapter 19 (Loop Engineering).*

Q: How do you detect and handle stagnation in a production agent loop?

Answer: **Stagnation** occurs when the loop is running but making no progress toward the goal—the agent is stuck in a local minimum, repeating similar actions without advancing the task.
Detection methods:

1. **Action hash window**: Hash each (action, observation) pair. If the current hash matches any hash in the last w steps, stagnation is detected. Catches exact repetition.
2. **Semantic similarity**: Embed recent actions and compute cosine similarity between consecutive steps. Similarity > 0.95 indicates near-repetition even if not identical.
3. **Progress metric monitoring**: Define task-specific progress metrics (e.g., number of tests passing, lines of code changed, subtasks completed). If the metric does not improve in N steps, stagnation is declared.
4. **Verifier score plateau**: If the verifier score has not improved by more than δ in the last k iterations, the loop has converged to a local optimum.

Recovery strategies (in order of escalation):

1. **Hint injection**: Add a system message: "You have tried similar approaches without success. Consider a fundamentally different strategy."
2. **Context reset**: Summarize progress so far and restart the loop with a fresh context, preserving only the goal and key findings.
3. **Strategy diversification**: Explicitly enumerate what has been tried and instruct the agent to avoid those approaches.

4. **Escalation**: Save full state and hand off to a human or higher-level orchestrator with a structured summary.

Key principle: Stagnation is not failure—it is a signal. The correct response is escalation, not continued iteration.
Review: *Chapter 19 (Loop Engineering).*

Q: Compare the nightly maintenance loop, continuous experimentation loop, and always-on orchestrator patterns. When do you use each?

Answer:

Pattern	Trigger	Best for	Key constraint
Nightly maintenance	Cron schedule	Accumulated work (CI failures, issues)	Must complete before team wakes
Continuous experimentation	Per-hypothesis	Scientific discovery, hyperparameter search	Bounded execution time per run
Always-on orchestrator	Event-driven	Continuous monitoring, rapid response	Crash recovery; durable state

Nightly maintenance: Runs once per day, processes accumulated work (CI failures, new issues), spawns parallel sub-agents for auto-fixable items, opens draft PRs. Suitable when the team can tolerate overnight latency and the workload is batch-oriented.
Continuous experimentation: Applies the loop to scientific discovery—propose hypothesis, implement change, run bounded experiment, evaluate, commit or rollback. The bounded execution time (hard wall-clock cap) prevents any single failed experiment from consuming excessive compute. Susceptible to conservative exploration (local minimum problem).
Always-on orchestrator: A persistent meta-agent with a heartbeat cycle that responds to events within minutes. Requires a durable state database with crash recovery—if the orchestrator fails mid-cycle, it resumes from its last checkpoint. Most complex to operate but provides the lowest latency response to new events.
Review: *Chapter 19 (Loop Engineering).*

29.32 Graph Engineering Questions

Q: What is graph engineering and how does it differ from loop engineering?

Answer: **Loop engineering** is the discipline of designing iterative control structures—generate–verify–retry cycles that drive an agent toward a goal. A loop is a single cycle with one control flow path.

Graph engineering is the discipline of designing the *topology* of those structures: specifying which nodes execute, in what order, under what conditions, and with what shared state. A graph is a network of cycles, branches, and parallel paths that can express arbitrarily complex agent workflows.

Key differences:

Dimension	Loop Engineering	Graph Engineering
Structure	Single cycle	Network of nodes and edges
Control flow	Sequential iteration	Conditional branching, parallelism
State	Implicit (context window)	Explicit typed schema
Composition	Nested loops	Subgraph composition
Testability	Test the full loop	Test individual nodes in isolation
Visualization	Hard to diagram	Renders as a readable flowchart

The progression: Prompt → response (single call) → loop (iteration) → graph (topology). Each layer adds structure. Graph engineering is the culmination: it makes the relationships between components explicit, visible, and testable.

When to use graphs over loops: When the workflow has conditional branching (different paths for different outcomes), parallel subtasks, reusable subgraphs, or human-in-the-loop interrupt points that a simple loop cannot express cleanly.

Review: *Chapter 20 (Graph Engineering).*

Q: Explain typed state in graph engineering. Why is state schema design one of the most consequential decisions?

Answer: In a graph-based agent, **typed state** is a shared data structure (typically a `TypedDict` or Pydantic model) that flows through the graph and is updated by each node. Every node reads from and writes to this state; the graph runtime manages passing it between nodes.

Why it is consequential:

1. **It is the contract between nodes**: Every node's inputs and outputs are defined by the state schema. A poorly designed schema forces nodes to carry irrelevant data, creates implicit dependencies, or requires nodes to parse unstructured strings.
2. **Schema drift is a leading failure mode**: If nodes write undeclared keys or assume fields that don't exist, errors propagate silently through the graph and surface far from their origin. Strict schema validation (fail loudly on unexpected fields) catches these early.
3. **It determines checkpointing granularity**: The state is what gets checkpointed. A state that captures too little cannot support meaningful resume-from-checkpoint. A state that captures too much wastes storage and slows serialization.
4. **It shapes parallelism**: Fields that are written by parallel branches must use reducer functions (e.g., list append) rather than simple assignment to avoid race conditions.

Design principles: (1) Include only what nodes need to share—not everything. (2) Use typed fields, not raw dicts. (3) Separate *task inputs* (immutable) from *working state* (mutable) from *outputs* (append-only). (4) Validate at the graph's entry point.

Review: *Chapter 20 (Graph Engineering).*

Q: What are conditional edges and how do they implement routing, retry loops, and human-in-the-loop patterns?

Answer: A **conditional edge** is a function that inspects the current graph state and returns the name of the next node to execute. Unlike a fixed edge (always goes to node B after node A), a conditional edge makes the execution path dynamic.
Three canonical uses:

1. **Routing**: After a classification node, a conditional edge routes to different specialist nodes based on the classification result. Example: a triage node classifies an issue as "bug", "feature", or "question"; the conditional edge routes to the appropriate handler node.
2. **Retry loops**: A back-edge from a verification node back to a generation node implements a retry loop. The conditional edge checks the verifier's result: if it passes, proceed to the next stage; if it fails and budget remains, loop back to the generator; if budget is exhausted, route to the escalation node.
3. **Human-in-the-loop**: An interrupt point is implemented as a conditional edge that routes to a `WAIT` state when human approval is needed. The graph suspends, serializes its state, and resumes when the human provides input. This is identical to a checkpoint-and-resume pattern.

Key property: Conditional edges make control flow *explicit and inspectable*. In a loop, the branching logic is buried in imperative code. In a graph, it is a named function attached to a named edge—readable, testable, and visualizable.
Review: *Chapter 20 (Graph Engineering).*

Q: How does fan-out/fan-in parallelism work in a graph, and what is the role of reducer functions?

Answer: **Fan-out** is a node that dispatches multiple parallel tasks simultaneously—each task runs as an independent branch of the graph with its own execution context. **Fan-in** is the subsequent node that waits for all branches to complete and merges their results.
In LangGraph: Fan-out is implemented with the `Send` API, which dispatches a list of tasks to a node in parallel. The fan-in node receives all results once every branch completes.
The reducer problem: When parallel branches write to the same state field, simple assignment creates a race condition—the last branch to complete overwrites all others. **Reducer functions** solve this by defining how to *merge* multiple writes to the same field:

- **List append** (`operator.add`): Each branch appends its result to a list. The fan-in node receives a list of all results. Most common for collecting parallel outputs.
- **Max/min**: Keep the best result across branches (e.g., highest verifier score).
- **Custom merge**: Domain-specific logic (e.g., merge partial document sections in order).

Practical example: A research orchestrator fans out to 5 specialist agents (each searching a different domain), then fans in to a synthesis node. Each agent appends its findings to a `results: list[Finding]` field. The synthesis node receives all 5 findings and produces a unified report.
Failure handling: If one branch fails, the fan-in node should handle partial results gracefully—synthesize from k-of-N completions rather than failing entirely.
Review: *Chapter 20 (Graph Engineering).*

Q: What is graph checkpointing and how does it enable human-in-the-loop and fault-tolerant agent systems?

Answer: **Graph checkpointing** captures the complete execution state of a graph at a point in time: the current state values, the execution history (which nodes ran, in what order, with what

inputs and outputs), and the pending next steps. This is richer than a simple state snapshot—it is a full audit trail.
Two key use cases:

1. **Human-in-the-loop**: When the graph reaches an interrupt point (a node that requires human approval), it serializes its checkpoint and suspends. The human reviews the state, provides input, and the graph resumes from the checkpoint—continuing exactly where it left off. Without checkpointing, the graph would have to restart from scratch after human input.
2. **Fault tolerance**: If the graph process crashes mid-execution (network failure, OOM, timeout), the checkpoint allows resumption from the last saved state rather than restarting the entire task. For long-running tasks (hours of execution), this is the difference between a recoverable incident and a total loss.

Checkpoint storage: LangGraph supports in-memory (development), SQLite (single-machine production), and PostgreSQL (distributed production) checkpoint backends. The choice depends on durability requirements and deployment scale.
Time-travel debugging: Because checkpoints capture the full execution history, developers can replay a graph from any prior checkpoint—invaluable for debugging non-deterministic failures. "What was the state when node X made that wrong decision?" becomes answerable.
Design implication: Checkpointing requires that all state is serializable. Avoid storing non-serializable objects (open file handles, live connections) in the graph state; store references (file paths, connection strings) instead.
Review: *Chapter 20 (Graph Engineering).*

Chapter 30

Quick Reference

This chapter consolidates key equations, architecture specifications, API references, and failure mode diagnostics for rapid lookup during development and debugging.

30.1 Core RL & Alignment Equations

$$\text{PPO Clip:} \quad L = \mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1\pm\epsilon)\hat{A}_t)], \quad r_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t) \tag{30.1}$$

$$\text{DPO:} \quad L = -\mathbb{E}[\log \sigma(\beta \log \tfrac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \tfrac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})] \tag{30.2}$$

$$\text{GRPO:} \quad \hat{A}_i = (r_i - \mu_G)/\sigma_G, \quad \text{then PPO clip update (no critic)} \tag{30.3}$$

$$\text{KTO:} \quad L = \lambda_w(1 - v(y_w)) + \lambda_l \cdot v(y_l), \quad v = \sigma(\beta \log(\pi_\theta/\pi_{\text{ref}}) - z) \tag{30.4}$$

$$\text{IPO:} \quad L = \mathbb{E}[(\log(\pi_\theta(y_w)/\pi_{\text{ref}}(y_w)) - \log(\pi_\theta(y_l)/\pi_{\text{ref}}(y_l)) - 1/(2\beta))^2] \tag{30.5}$$

$$\text{ORPO:} \quad L = L_{\text{SFT}}(y_w) - \lambda \log \sigma(\log(\text{odds}(y_w)/\text{odds}(y_l))) \tag{30.6}$$

$$\text{GAE:} \quad \hat{A}_t = \textstyle\sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{30.7}$$

$$\text{KL Penalty:} \quad R_{\text{total}} = r_\phi(x, y) - \beta D_{\text{KL}}[\pi_\theta(y|x) \| \pi_{\text{ref}}(y|x)] \tag{30.8}$$

$$\text{RM (Bradley-Terry):} \quad L = -\mathbb{E}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \tag{30.9}$$

$$\text{Best-of-N:} \quad y^* = \arg \max_{y_i \sim \pi_\theta(\cdot|x),\, i=1..N} r_\phi(x, y_i) \tag{30.10}$$

30.2 Transformer & Architecture Formulas

$$\text{Self-Attention:} \quad \text{Attn}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k}) \cdot V \tag{30.11}$$

$$\text{Multi-Head:} \quad \text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attn}(XW_i^Q, XW_i^K, XW_i^V) \tag{30.12}$$

$$\text{RoPE:} \quad f(x_m, m) = x_m e^{im\theta_j}, \quad \theta_j = 10000^{-2j/d} \tag{30.13}$$

$$\text{LoRA:} \quad W' = W_0 + (\alpha/r) \cdot BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k} \tag{30.14}$$

$$\text{KD (soft targets):} \quad L_{\text{KD}} = (1-\alpha)L_{\text{CE}}(y, \hat{y}) + \alpha\, T^2 \cdot \text{KL}(p_T^{\text{teacher}} \| p_T^{\text{student}}) \tag{30.15}$$

$$\text{FFN (SwiGLU):} \quad \text{FFN}(x) = (\text{Swish}(xW_1) \odot xW_3)W_2 \tag{30.16}$$

30.3 Decoding Methods

Method	Formula / Rule	Key Param
Greedy	$y_t = \arg\max_v P(v\|y_{<t})$	—
Beam search	Keep top-B partial sequences by joint probability	$B = 4$–8
Temperature	$P'(v) = \text{softmax}(\text{logit}_v/T)$	$T \in [0.1, 1.5]$
Top-k	Zero out all but top-k logits, renormalize	$k = 40$–100
Top-p (nucleus)	Keep smallest set V' s.t. $\sum_{v\in V'} P(v) \geq p$	$p = 0.9$–0.95
Min-p	Keep tokens with $P(v) \geq p_{\min} \cdot P(v_{\max})$	$p_{\min} = 0.05$–0.1
Repetition penalty	$\text{logit}_v \leftarrow \text{logit}_v/\theta$ if v appeared before	$\theta = 1.1$–1.3

30.4 Systems & Parallelism

Formula	Value (70B, BF16)	Description
Model memory	$2P$ bytes	140 GB (weights only)
Adam optimizer	$2P \times 4$ bytes (m + v)	280 GB
Full training footprint	$\sim 8P$ bytes	560 GB (weights + opt + grad)
FSDP memory/GPU	$8P/N_{\text{GPUs}}$	70 GB with 8 GPUs
Gen arithmetic intensity	$2P/2P = 1$ FLOP/byte	Heavily memory-bound
Token rate (gen)	HBM_BW $/(2P)$	∼14 tok/s (A100, batch=1)
TP AllReduce / layer	$2 \times 2 \cdot \frac{T-1}{T} \cdot bsd$ bytes	∼188 MB (70B, TP=8)
PP bubble fraction	$(P-1)/(P+M-1)$	P=stages, M=micro-batches
MFU	observed_toks $\times$ $6P$ / peak_FLOPS	Target: $> 40\%$

30.5 GPU Hardware Specs

GPU	Memory	BW (HBM)	BF16 TFLOPS	NVLink	Notes
A100-80GB	80 GB HBM2e	2.0 TB/s	312	600 GB/s	Workhorse, widely available
H100-80GB	80 GB HBM3	3.35 TB/s	989	900 GB/s	Current gen, FP8 support
H200-141GB	141 GB HBM3e	4.8 TB/s	989	900 GB/s	Large context / fewer GPUs
B200	192 GB HBM3e	8.0 TB/s	2250	1800 GB/s	Next gen (2025)

30.6 Hyperparameter Ranges

Parameter	Typical Range	Default	Notes
β (DPO/KTO)	0.05–0.5	0.1	Higher = more conservative
ϵ (PPO clip)	0.1–0.3	0.2	Higher = more aggressive updates
γ (GAE discount)	0.99–1.0	1.0	Use 1.0 for episodic tasks
λ (GAE)	0.9–0.99	0.95	Lower = more biased, less variance
KL coeff (β_{KL})	0.01–0.2	0.05	Auto-adapt to target KL $\approx$ 5–8
LR (RLHF)	1e-7 – 5e-6	5e-7	Much lower than pre-training
LR (SFT)	1e-5 – 5e-5	2e-5	Standard fine-tuning range
LoRA rank r	8–128	16–64	Higher r = more capacity, more memory
LoRA alpha α	r – $2r$	$2r$	Scaling factor; α/r is the effective scale
Temperature (gen)	0.6–1.0	0.7	Lower = less diverse candidates
Num generations K	4–64	4–16	For GRPO/Online DPO/Best-of-N
Grad clip norm	0.5–2.0	1.0	Prevents gradient explosion

30.7 TRL API Quick Reference

Trainer	Method	Key Config	Data Format
`SFTTrainer`	Supervised FT	`packing`, `max_seq_length`	prompt + completion
`RewardTrainer`	Reward model	`center_rewards_coefficient`	prompt + chosen + rejected
`PPOTrainer`	PPO	`init_kl_coef`, `target_kl`, `cliprange`	prompts (online gen)
`DPOTrainer`	DPO/IPO	`beta`, `loss_type="sigmoid"/"ipo"`	prompt + chosen + rejected
`GRPOTrainer`	GRPO	`num_generations`, `beta`, `use_vllm`	prompts + reward_fn
`OnlineDPOTrainer`	Online DPO	`num_generations`, `reward_model_path`	prompts (online gen)
`KTOTrainer`	KTO	`desirable_weight`, `undesirable_weight`	prompt + completion + label
`ORPOTrainer`	ORPO	`beta`	prompt + chosen + rejected
`Best-of-N (manual)`	Best-of-N	`n_samples`	prompts (inference)

30.8 RAG Pipeline Formulas

$$\text{Cosine similarity:} \quad \text{sim}(q, d) = \frac{q \cdot d}{\|q\| \cdot \|d\|} \tag{30.17}$$

$$\text{Retrieval:} \quad \mathcal{D}_k = \text{top-}k_{d \in \mathcal{C}}\ \text{sim}(\text{embed}(q),\ \text{embed}(d)) \tag{30.18}$$

$$\text{RAG generation:} \quad P(y|q) = P_{\text{LLM}}(y \mid q, \mathcal{D}_k) \tag{30.19}$$

$$\text{Chunking overlap:} \quad \text{stride} = \text{chunk_size} - \text{overlap} \tag{30.20}$$

$$\text{Reranker (cross-enc):} \quad \text{score}(q, d) = \text{MLP}(\text{BERT}([q; d])) \tag{30.21}$$

30.9 Agentic Design Patterns

Pattern	Structure	Best For
ReAct	Think → Act → Observe → loop	General tool-use agents
Plan-and-Execute	Plan → Execute steps → Revise	Long-horizon, structured tasks
Supervisor	Router → specialist agents	Multi-domain, clear subtask boundaries
Swarm (handoffs)	Agent transfers control + context	Customer service, escalation flows
Hierarchical	Tree of delegating agents	Complex decomposition
Human-in-the-loop	Agent → Approval gate → Continue	High-stakes, irreversible actions
Loop (Validation)	Generate → Validate → loop until pass	Quality-critical outputs; see Ch. 19
Graph Workflow	Typed state → nodes → conditional edges	Any multi-step agent; see Ch. 20

30.10 Agent Communication Protocols

Protocol	Scope	Transport	Key Concept
MCP	Tool integration	stdio / HTTP+SSE	Server exposes tools; client discovers & calls
A2A	Agent-to-agent	HTTP + JSON-RPC	Tasks with lifecycle (submitted→working→done)
ACP	Agent-to-agent (REST)	HTTP REST	Stateless request/response; simpler than A2A for synchronous calls
OpenAI Function Calling	Tool use	API payload	JSON schema in `tools[]` array

30.11 Context Window Budget

$$C \geq \underbrace{S}_{\text{system}} + \underbrace{M}_{\text{memory/RAG}} + \underbrace{T}_{\text{tool defs}} + \underbrace{H}_{\text{history}} + \underbrace{R}_{\text{reserved output}} \tag{30.22}$$

Rule of thumb for 128K context:

- System prompt: 1–4K tokens (fixed)
- Tool definitions: 2–8K (scales with # tools)
- RAG context: 4–16K (top-k chunks)
- History: grows unbounded → summarize/truncate
- Reserved output: 2–8K

30.12 Common Failure Modes & Fixes

Symptom	Likely Cause	Fix
Reward up, quality down	Reward hacking	RM ensemble, length penalty, increase β
KL exploding (>15)	LR too high or mode collapse	Reduce LR, checkpoint rollback
Entropy collapse	Premature convergence	Increase entropy coeff, raise temperature
Training loss NaN	Gradient explosion	Reduce LR, increase grad clip, check data
No improvement after 5K steps	Bad prompt distribution	Goldilocks filter (20–80% pass rate)
Benchmark regression	Alignment tax	Reduce RL budget, use LoRA, mix SFT data
Length increasing monotonically	Length exploit in RM	Length penalty, retrain RM with length control
OOM during generation	KV cache overflow	Reduce batch, increase TP, PagedAttention
Agent loops forever	No max-iteration guard	Set `max_iterations`, add loop detection
Tool call parse failures	Inconsistent output format	Few-shot examples, constrained decoding
RAG returns irrelevant docs	Poor embedding / chunking	Reranker, hybrid search, smaller chunks
Multi-agent deadlock	Circular dependencies	DAG enforcement, timeout per agent
God Node (one node does everything)	Missing graph decomposition	Split into focused single-responsibility nodes
State schema drift	Nodes write undeclared keys	Use `TypedDict` + strict schema; see Ch. 20
Parallel branch state conflict	Two branches overwrite same key	Use reducer annotations (`Annotated[list, operator.add]`)

30.13 Method Selection Decision Tree

1. **Have paired preferences (chosen + rejected)?**
   - Noisy labels → **IPO**
   - Memory-constrained, no SFT done yet → **ORPO**
   - Clean data, limited compute → **DPO**
   - DPO plateaus, want exploration → **Online DPO**
2. **Have only binary feedback (thumbs up/down)? → KTO**
3. **Have verifiable rewards (math/code)? → GRPO**
4. **Need maximum quality, any cost? → PPO**
5. **Want training-free improvement? → Best-of-N**

30.14 Evaluation Metrics

Metric	Range	What It Measures
Perplexity	$[1, \infty)$	Model's surprise; lower = better language modeling
Win Rate (vs. baseline)	$[0, 1]$	Fraction of outputs preferred by judge/human
BLEU	$[0, 1]$	n-gram overlap with reference (precision-focused)
ROUGE-L	$[0, 1]$	Longest common subsequence with reference
Pass@k	$[0, 1]$	Probability ≥ 1 of k code samples passes tests
MMLU / GPQA	$[0, 1]$	Multi-choice accuracy on knowledge/reasoning benchmarks
HumanEval	$[0, 1]$	Functional correctness of generated code
Faithfulness (RAG)	$[0, 1]$	Fraction of claims supported by retrieved context
Context Relevancy	$[0, 1]$	Fraction of retrieved content relevant to query
Answer Relevancy	$[0, 1]$	Degree to which answer addresses the question

30.15 Reasoning & Test-Time Scaling

Method	Compute Cost	Mechanism
Chain-of-Thought (CoT)	1.5–3× tokens	"Think step by step" in prompt
Self-Consistency	$N\times$ generation	Sample N CoT paths, majority vote on final answer
Tree-of-Thought (ToT)	$B \times D\times$ generation	BFS/DFS over reasoning tree; evaluate branches
Best-of-N	$N\times$ generation	Sample N, score with RM, pick highest
Beam search (on reasoning)	$B\times$ generation	Maintain top-B partial reasoning chains
Budget forcing	Variable	Allocate more tokens to harder problems dynamically
Verification (ORM/PRM)	$N\times$ gen + scoring	Generate N solutions, rank by outcome/process RM

30.16 Memory System Types

Type	Storage	Use Case
Working memory	Context window	Current conversation, immediate tool results
Episodic memory	Vector store	Past interactions, user preferences, session history
Semantic memory	Knowledge graph / embeddings	Facts, concepts, domain knowledge
Procedural memory	Skill library / code	How-to procedures, learned workflows

30.17 MCP Quick Reference

Primitive	Direction	Side Effects?	Purpose
Tools	Client → Server	Yes	Execute actions (create, modify, delete)
Resources	Client → Server	No (read-only)	Read data (files, DB records, configs)
Prompts	Client → Server	No	Reusable templates for common tasks
Sampling	Server → Client	No	Server requests LLM generation from client

Transport: `stdio` (local subprocess) or `HTTP+SSE` (remote, streamable).

Discovery: Client calls `tools/list`, `resources/list`, `prompts/list` at connection init.

Tool annotations: `readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`.

30.18 A2A Protocol Quick Reference

Concept	Description
Agent Card	JSON at `/.well-known/agent.json` — name, skills, supported content types
Task	Unit of work: `id`, `status`, `artifacts`. Lifecycle: submitted → working → completed/failed
Message	Communication unit within a task (role: user/agent, parts: text/-file/data)
Artifact	Output produced by the agent (structured data, files, generated content)
Push Notifications	Webhook-based updates for long-running tasks (via `tasks/pushNotification/set`)

Key endpoints: `tasks/send` (create/update), `tasks/get` (poll status), `tasks/sendSubscribe` (SSE stream).

30.19 Agent Framework Comparison

Framework	Orchestration	Multi-Agent	Best For
LangGraph	Explicit state graph	Conditional routing	Production: persistence, HITL, fine control
OpenAI Agents SDK	Declarative handoffs	Handoff-based	Simplicity: guardrails, tracing, fast start
AutoGen (AG2)	Conversation-driven	GroupChat	Prototyping: code execution, research
CrewAI	Role-based teams	Sequential/parallel	Low-code: quick demos, simple pipelines
Google ADK	Session + events	A2A native	Enterprise: artifact mgmt, multi-modal

30.20 Agentic RL Formulas

$$\text{Trajectory GRPO:} \quad \hat{A}_i = (R(\tau_i) - \mu_G)/\sigma_G, \quad R(\tau_i) = \sum_t r_t^{(\tau_i)} \tag{30.23}$$

$$\text{Agent reward:} \quad R = w_1 R_{\text{task}} + w_2 R_{\text{efficiency}} + w_3 R_{\text{safety}}, \quad R_{\text{eff}} = \max(0, 1 - \text{steps}/N_{\max}) \tag{30.24}$$

$$\text{Masking:} \quad \mathcal{L} = \sum_{t \in \text{agent tokens}} \min(r_t \hat{A}_t,\ \text{clip}(r_t)\hat{A}_t) \quad \text{(mask env outputs)} \tag{30.25}$$

$$\text{Pass@}k: \quad 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \quad n = \text{total samples},\ c = \text{correct} \tag{30.26}$$

30.21 Agent Security Checklist

Threat	Layer	Mitigation
Prompt injection (direct)	Input	Input validation, instruction hierarchy, delimiters
Prompt injection (indirect)	Tool output	Treat tool output as untrusted; don't follow instructions in retrieved docs
Tool misuse	Execution	Least-privilege permissions; `destructiveHint` gates; sandboxing
Data exfiltration	Output	Output filtering; restrict tool access to allowed domains
Excessive autonomy	Architecture	Max iterations; cost budgets; human approval gates
Confused deputy	Multi-agent	Verify task origin; capability-based access control

30.22 Agent Evaluation Metrics

Metric	Formula / Definition	Target
Task Success Rate (TSR)	Correct completions / total tasks	$> 85\%$ (production)
Steps to completion	Avg agent actions per successful task	Lower = more efficient
Cost per task	Total tokens $\times$ price/token	Budget-dependent
Latency (TTFC)	Time from request to first useful output	< 5s for interactive
Tool call accuracy	Correct tool selections / total calls	$> 90\%$
Recovery rate	Successful retries / initial failures	$> 60\%$
Human escalation rate	Tasks requiring human / total tasks	$< 15\%$

30.23 Key Agentic Benchmarks

Benchmark	Domain	Metric	SOTA (mid-2026)
SWE-bench Verified	Software engineering	% resolved issues	$\sim$70%
WebArena	Web browsing	Task success rate	$\sim$40%
OSWorld	Desktop computer use	Task success rate	$\sim$25%
GAIA	General AI assistant	Exact match accuracy	$\sim$75% (L1)
Tau-bench	Tool-use reliability	Pass rate (5 trials)	$\sim$65%
HumanEval / MBPP	Code generation	Pass@1	$> 95\%$
Terminal-Bench	Long-horizon terminal	Task completion	$\sim$38% (hard tasks)

Chapter 31

Conclusion and Future Directions

31.1 Summary

This guide has traced the full arc from transformer foundations through reinforcement learning for alignment to the construction of autonomous agentic systems. The key themes that emerge across all chapters:

1. **Alignment is a systems problem.** It is not enough to have a good loss function. Production RLHF requires managing 4+ models, distributing computation across hundreds of GPUs, handling fault tolerance, and monitoring for reward hacking—all simultaneously.

2. **There is no single best method.** PPO remains the gold standard for maximum quality but demands enormous engineering investment. DPO and its variants offer compelling trade-offs for teams with limited infrastructure. GRPO bridges the gap for verifiable-reward domains. The right choice depends on your data, compute budget, and quality bar.

3. **Reasoning emerges from reward.** DeepSeek-R1 proved that chain-of-thought, self-verification, and backtracking can emerge from simple binary reward signals and group-relative optimization—without explicit demonstrations of reasoning. Test-time compute scaling means smaller models with more thinking can match larger models.

4. **Standards unlock ecosystems.** MCP reduces the tool integration problem from $N \times M$ to $N + M$. A2A enables agents built by different teams to collaborate without shared internals. These protocols are to agentic AI what HTTP was to the web—the enabling infrastructure for an open ecosystem.

5. **Agents are the natural next step.** Once a model is aligned, the frontier shifts from "how good is a single response?" to "can the model solve multi-step problems autonomously?" This requires new training paradigms (agentic RL with environment rewards, calibrated decision training for reliable confidence, harness distillation to bake scaffolding into weights), new infrastructure (harnesses, tool protocols, memory systems), and new evaluation methods (trajectory-level benchmarks).

6. **Evaluation drives everything.** Without rigorous evaluation—from reward model validation to agent task success rates, from contamination detection to LLM-as-Judge calibration—progress is unmeasurable and regressions are invisible. The benchmarks you choose shape the systems you build.

7. **Simplicity scales.** The most reliable production agents use the simplest architecture that meets requirements—prompt chaining and routing before autonomous loops, single agents before multi-agent swarms. Complexity should be earned through demonstrated need.

8. **Engineering discipline matures agentic systems.** The progression from prompt engineering through context management, agent harnesses, loop engineering, and graph engineering

represents the maturation of agentic system design. Each layer adds structure that makes systems more understandable, testable, and composable. A well-designed graph—with typed state, single-responsibility nodes, and explicit edges—is as readable as a flowchart and as testable as a pure function.

31.2 The Road Ahead: Open Challenges

31.2.1 Learning from Interaction

Current RLHF pipelines [296] treat alignment as a one-time training phase. The future points toward **continuous learning from deployment**: agents that improve from every user interaction, tool failure, and environment observation—without catastrophic forgetting [196] or reward drift. Key open problems:

- Online learning with non-stationary reward distributions.
- Safe exploration in production [113] (avoiding harmful actions while learning).
- Efficient credit assignment over long agent trajectories (hundreds of tool calls).

31.2.2 Reasoning Architecture vs. Scale

One of the most practically important open questions in LLM development is whether **reasoning capability scales predictably with model size**—or whether architectural choices matter as much as, or more than, parameter count.

The evidence from 2025–2026 is mixed. On one hand, scaling laws [150, 187] predict that larger models trained on more tokens will be more capable across the board, and this holds for many standard benchmarks. On the other hand, the emergence of dedicated reasoning architectures—models trained with extended chain-of-thought via GRPO [341] or process reward models [227]—has shown that a smaller model with the right training recipe can match or exceed a much larger model on reasoning-intensive tasks. DeepSeek-R1 [75] demonstrated this concretely: a 7B reasoning-trained model outperformed much larger base models on mathematical and logical benchmarks.

This creates a practical dilemma for practitioners:

- **Scale-first**: Train or use the largest available model; reasoning will emerge. Lower engineering complexity, higher cost.
- **Architecture-first**: Use a smaller model with reasoning-specific training (extended thinking, process supervision). Lower cost, but requires more careful training pipeline design.
- **Hybrid**: Use a large model for complex reasoning steps and a small model for routine tool calls and routing. Requires a routing layer but optimizes cost-quality trade-offs.

The open challenge is that we lack reliable methods to predict, before training, which approach will win for a given task distribution. Benchmark performance on standard suites does not reliably predict performance on novel reasoning tasks encountered in production. This uncertainty makes model selection for agentic systems more art than science—a gap that better evaluation methodology (Section 31.2.7) and interpretability research must close.

31.2.3 Scalable Oversight

As agents become more capable, human oversight becomes the bottleneck. Current approaches (RLHF [296], Constitutional AI [17]) rely on humans evaluating model outputs—but what happens when model outputs exceed human understanding?

- **Recursive reward modeling** [59]: Use AI to help humans evaluate AI.

- **Debate and amplification** [167]: Two models argue; a human judges which argument is more compelling.
- **Process-based supervision** [227]: Reward correct reasoning steps, not just final answers.
- **Mechanistic interpretability** [288]: Understand *what* the model is doing internally, not just what it outputs.

31.2.4 World Models and Planning

Current agents are reactive—they observe and respond one step at a time. Future agents will need **internal world models** [138] that enable lookahead planning:

- Predicting the consequences of actions before executing them.
- Tree search over possible action sequences (à la AlphaGo [349] and MuZero [334] but for open-ended tasks).
- Learning environment dynamics from interaction traces.

31.2.5 Multi-Agent Ecosystems

The A2A protocol [121] and multi-agent frameworks hint at a future where hundreds of specialized agents collaborate, negotiate, and delegate—forming an "economy of agents" [282]. Open challenges:

- Trust and verification between agents with different principals.
- Emergent cooperation vs. emergent deception in competitive settings [163].
- Market mechanisms for resource allocation (compute, tool access, priority).
- Governance: who is responsible when a chain of 10 agents produces a harmful outcome? [7]

31.2.6 Agent Security and Trust

Autonomous agents inherit every security vulnerability of the LLMs they are built on—plus new attack surfaces created by tool access, multi-agent delegation, and persistent memory (Chapters 19–22). Critical unsolved problems:

- **Prompt injection at scale** [128]: As agents consume untrusted content (web pages, emails, API responses), indirect prompt injection becomes systemic. No robust defense exists today.
- **Confused deputy attacks**: An agent with legitimate credentials can be tricked into misusing them on behalf of an attacker embedded in the data stream [10].
- **Sandboxing without crippling**: Least-privilege execution constrains what agents can do, but overly restrictive sandboxes negate agentic value. Finding the right boundary is an open design problem.
- **Audit and attribution**: When an agent chain spans multiple organizations (via A2A [121]), tracing *who* authorized *what* action remains architecturally unsolved.
- **Trust calibration**: Agents must learn when *not* to trust—whether a tool response is authentic, whether another agent's claim is verified.

31.2.7 Evaluation Beyond Benchmarks

Chapter 14 showed that benchmarks shape the systems we build—yet current evaluation has critical gaps:

- **Real-world deployment metrics**: Benchmarks like SWE-bench [179] and GAIA [267] measure isolated tasks; production agents face ambiguous goals, shifting requirements, and multi-turn recovery.
- **Reward model validity**: RLHF assumes reward models capture human preferences, but reward hacking [351] and distributional shift undermine this assumption at scale.
- **Cost-quality frontiers**: Two agents may achieve the same accuracy, but one costs 10× more tokens. Evaluation must become cost-aware.
- **Safety under distribution shift**: An agent safe in testing may behave unsafely on novel inputs. Adversarial evaluation [307] and red-teaming at agentic scale remain immature.

31.2.8 Efficiency and Accessibility

Training a 70B model with RLHF costs $10K - -100$K. Running autonomous agents costs $1 - -50$ per complex task. For agentic AI to achieve broad impact:

- Distillation of agentic capabilities from large to small models [149, 194].
- **Harness distillation**: baking specialized scaffolding into model weights so runtime overhead can be eliminated at deployment [433] (Section 12.6).
- More efficient RL algorithms (fewer samples, lower variance) [337].
- On-device agents that operate without cloud round-trips.
- Open-weight models that match proprietary quality for agentic tasks [75].

31.2.9 Regulatory Environment

Agentic AI systems are increasingly subject to formal regulation, adding a new dimension to deployment decisions that was largely absent from earlier LLM deployments.

EU AI Act (2024–2026). The EU AI Act [92], which entered into force in August 2024 with a phased compliance timeline, classifies AI systems by risk level. Autonomous agents that make consequential decisions—in hiring, credit, healthcare, or critical infrastructure—fall into the **high-risk** category and must satisfy requirements including: conformity assessments before deployment, human oversight mechanisms, logging of system behavior for post-hoc audit, and registration in an EU database. General-purpose AI models (GPAIs) with systemic risk—broadly, those exceeding 10^{25} FLOPs of training compute—face additional obligations including adversarial testing and incident reporting. For practitioners building agentic systems in or for the EU market, compliance is not optional: penalties reach up to 3% of global annual turnover for violations.

California SB 942 (AI Transparency Act, 2024). California's SB 942 requires that AI-generated content be detectable and that systems capable of generating synthetic media include provenance metadata. While primarily targeting generative media, the law has implications for agentic systems that produce documents, code, or communications on behalf of users: outputs may need to be labeled as AI-generated, and systems must provide a mechanism for users to request disclosure. As California often sets de facto national standards in the US, practitioners should treat SB 942 compliance as a baseline for US deployments.

Emerging themes across jurisdictions. Despite differences in scope and enforcement, several themes appear consistently across regulatory frameworks:

- **Human oversight**: Regulations consistently require that humans remain able to intervene in, override, or shut down autonomous systems. This directly constrains fully autonomous agent designs.
- **Auditability**: Logs of agent decisions and actions must be retained and interpretable. This aligns with the observability requirements discussed in Chapter 19.
- **Transparency**: Users must know when they are interacting with an AI system, not a human.
- **Accountability**: There must be a legal entity responsible for the system's behavior—a challenge for multi-agent chains that span organizational boundaries.

Regulatory Compliance as Engineering Requirement

Regulatory requirements are not purely legal concerns—they translate directly into engineering requirements: audit logging, human-in-the-loop checkpoints, output provenance metadata, and capability restrictions. Teams that treat compliance as an afterthought will face costly retrofits. The most effective approach is to design for auditability and human oversight from the start, which also improves system reliability and debuggability.

31.3 Further Reading

31.3.1 Foundational Papers

- **Attention Is All You Need** [378] — The transformer architecture.
- **RLHF / InstructGPT** [296] — The first large-scale RLHF deployment.
- **PPO** [337] — Proximal Policy Optimization.
- **DPO** [320] — Direct Preference Optimization.
- **GRPO / DeepSeek-R1** [75, 341] — Group Relative Policy Optimization and emergent reasoning.
- **ReAct** [430] — Reasoning + Acting framework for LLM agents.
- **Toolformer** [333] — Teaching LLMs to use tools.
- **RAG** [219] — Retrieval-Augmented Generation.

31.3.2 Systems and Scaling

- **Megatron-LM** [345] — Tensor and pipeline parallelism.
- **DeepSpeed ZeRO** [322] — Memory-efficient distributed training.
- **vLLM** [206] — PagedAttention for efficient LLM serving.
- **Flash Attention** [68] — IO-aware exact attention.

31.3.3 Agentic AI

- **Building Effective Agents** [11] — Design patterns and principles.
- **LangGraph** [164] — Graph-based agent orchestration framework; reference implementation for graph engineering patterns.
- **Voyager** [383] — Open-ended agent with skill library in Minecraft.
- **SWE-bench** [179] — Benchmark for autonomous software engineering.
- **OSWorld** [419] — Full computer-use benchmarks.
- **GAIA** [267] — General AI Assistants benchmark for real-world tasks.
- **MemGPT** [297] — OS-inspired memory management for unbounded context.
- **Model Context Protocol** [10] — Open standard for tool integration.
- **Agent-to-Agent Protocol** [121] — Inter-agent communication standard.

Model landscape note (as of mid-2026). The frontier model landscape has shifted substantially since the core chapters of this book were written. In chronological order of release: **Llama 4** Scout and Maverick (Meta, April 2025) brought open-weight models substantially closer to proprietary quality on standard benchmarks. **Gemini 3.1 Pro** (Google DeepMind, February 2026) advanced the Gemini series into the frontier tier. **Claude Fable 5** (Anthropic, June 2026) became the publicly available Anthropic model with safety guardrails. **GPT-5.6** (OpenAI, July 2026) arrived as a family of three variants—Luna, Terra, and **Sol** (the most capable, described as OpenAI's "best coding model yet" and 54% more token-efficient than prior versions for agentic tasks). **Claude Opus 5** (Anthropic, July 2026) is Anthropic's maximum-capability flagship. **Qwen 3.8** (Alibaba, August 2026) further narrowed the open-weight gap. September 2026 brought a cluster of releases: **Claude Opus 5.5** (Anthropic) as an enhanced iteration with improved agentic and coding performance; **Claude Fable 5.1** and **Mythos 5.1** (Anthropic) at ∼25–45% lower cost than their predecessors; and **Gemini 3.8 Flash** (Google DeepMind) as a high-throughput inference-optimized model. Despite this progress, a persistent **open-weight lag** of approximately 3–6 months remains on the most demanding agentic benchmarks (SWE-bench Verified, Terminal-Bench): open-weight models typically reach the performance level of proprietary models from the prior release cycle, not the current one. This gap is narrowing but has not closed as of mid-2026. Practitioners should treat any specific model comparison in this book as a snapshot; the relative rankings shift with each major release cycle.

31.3.4 Alignment and Safety

- **Constitutional AI** [17] — Self-supervised alignment.
- **Sleeper Agents** [163] — Deceptive alignment concerns.
- **Reflexion** [344] — Learning from verbal self-reflection.
- **Indirect Prompt Injection** [128] — Security risks for LLM-integrated applications.

31.3.5 Online Resources

- **HuggingFace TRL**: https://github.com/huggingface/trl — Production RL library.
- **LangGraph**: https://github.com/langchain-ai/langgraph — Agent workflow graphs.
- **OpenAI Agents SDK**: https://github.com/openai/openai-agents-python — Official agent framework.

- **DeepSpeed-Chat**: https://github.com/microsoft/DeepSpeedExamples — End-to-end RLHF pipeline.
- **DSPy**: https://github.com/stanfordnlp/dspy — Declarative prompt optimization.
- **AutoGen**: https://github.com/microsoft/autogen — Multi-agent conversation framework.
- **Pydantic AI**: https://ai.pydantic.dev — Type-safe agent framework with structured outputs and dependency injection.
- **AG2 (AutoGen 3.0)**: https://github.com/ag2ai/ag2 — Actor-based multi-agent runtime; the community-driven AutoGen rewrite.

"The best way to predict the future is to build it."

— Alan Kay

Bibliography

[1] Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan, and James Zou. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild, 2019. URL https://arxiv.org/abs/1906.02569.

[2] Samira Abnar and Willem Zuidema. Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL https://arxiv.org/abs/2005.00928.

[3] Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021. URL https://arxiv.org/abs/2012.13255.

[4] Meta AI. The Llama 4 Herd: The Beginning of a New Era of Natively Multimodal AI. *Meta AI Blog*, 2025. URL https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

[5] Mistral AI. Mistral Large 2. *Mistral AI Blog*, 2024. URL https://mistral.ai/news/mistral-large-2407/.

[6] Diogo Almeida and TypeSafe AI. Introducing system one models and jev. TypeSafe AI Blog, 2026. URL https://typesafe.ai/blog/introducing-system-one-models-and-jev.

[7] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete Problems in AI Safety. *arXiv Preprint arXiv:1606.06565*, 2016. URL https://arxiv.org/abs/1606.06565.

[8] Yanxu An, Li Shen, Yifan Xu, and Xinmei Liu. DPPO: Direct Divergence-Based Policy Optimization for Language Model Alignment. *arXiv Preprint arXiv:2503.14532*, 2025. URL https://arxiv.org/abs/2503.14532.

[9] Marcin Andrychowicz, Filip Wolski, Alex Ray, et al. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] Anthropic. Model Context Protocol, 2024. URL https://modelcontextprotocol.io.

[11] Anthropic. Building Effective Agents, 2024. URL https://www.anthropic.com/research/building-effective-agents.

[12] Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. *Anthropic Technical Report*, 2024. URL https://www.anthropic.com/news/claude-3-family.

[13] Anthropic. Natural Language Autoencoders: Interpreting Neural Networks with Natural Language Descriptions. *Anthropic Research Blog*, 2026. URL https://www.anthropic.com/research/natural-language-autoencoders.

[14] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection. *arXiv Preprint arXiv:2310.11511*, 2023. URL https://arxiv.org/abs/2310.11511.

[15] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, et al. A General Theoretical Paradigm to Understand Learning from Human Feedback. *arXiv Preprint arXiv:2310.12036*, 2024. URL https://arxiv.org/abs/2310.12036.

[16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer Normalization. *arXiv Preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[17] Yuntao Bai, Andy Jones, Kamal Ndousse, et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv Preprint arXiv:2212.08073*, 2022. URL https://arxiv.org/abs/2212.08073.

[18] Zhizhou Bai, Yixin Bao, et al. Kimi K2: Open Agentic Intelligence. *arXiv preprint arXiv:2507.20534*, 2025. URL https://arxiv.org/abs/2507.20534.

[19] Payal Bajaj, Daniel Campos, Nick Craswell, et al. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. *arXiv Preprint arXiv:1611.09268*, 2016.

[20] Ankit Bajoria et al. A Cognitive Taxonomy of Agentic AI: Autonomous, Transactional, Collaborative, and Supervisory Roles. *Cognitive Computation*, 2026. Preprint.

[21] Satanjeev Banerjee and Alon Lavie. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 2005. URL https://aclanthology.org/W05-0909/.

[22] Oren Barkan, Edan Hauon, Avi Caciularu, Ido Dagan, and Noam Koenigstein. Grad-SAM: Explaining Transformers via Gradient Self-Attention Maps. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, 2021. URL https://arxiv.org/abs/2104.13299.

[23] Andrew G. Barto and Sridhar Mahadevan. Recent Advances in Hierarchical Reinforcement Learning. *Discrete Event Dynamic Systems*, 13:341–379, 2003.

[24] Fabio Luigi Bellifemine, Giovanni Caire, and Dominic Greenwood. JADE: Developing Multi-Agent Systems with JADE, 2007.

[25] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The Long-Document Transformer. *arXiv Preprint arXiv:2004.05150*, 2020. URL https://arxiv.org/abs/2004.05150.

[26] Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 2001.

[27] Maciej Besta, Nils Blach, Ales Kubicek, et al. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[28] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of PLDI*, 2023.

[29] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, et al. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. *arXiv Preprint arXiv:2409.08264*, 2024. URL https://arxiv.org/abs/2409.08264.

[30] Edward de Bono. Six Thinking Hats, 1985.

[31] Ralph Allan Bradley and Milton E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 1952. URL https://www.jstor.org/stable/2334029.

[32] Trenton Bricken, Adly Templeton, Joshua Batson, et al. Towards Monosemanticity: Decomposing Language Models with Dictionary Learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[33] Greg Brockman, Vicki Cheung, Ludwig Pettersson, et al. OpenAI Gym, 2016.

[34] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language Models Are Few-Shot Learners. *NeurIPS*, 2020.

[35] Chris Burges, Tal Shaked, Erin Renshaw, et al. Learning to Rank Using Gradient Descent. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005.

[36] Christopher J. C. Burges, Robert Ragno, and Quoc V. Le. Learning to Rank with Nonsmooth Cost Functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2006.

[37] Tianle Cai, Yuhong Li, Zhengyang Geng, et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2401.10774.

[38] Brian Campbell, John Bradley, Nat Sakimura, and Torsten Lodderstedt. OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens, 2020. URL https://datatracker.ietf.org/doc/html/rfc8705.

[39] Murray Campbell, A. Joseph Hoane Jr., and Feng-hsiung Hsu. Deep Blue. *Artificial Intelligence*, 2002.

[40] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to Rank: From Pairwise Approach to Listwise Approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, 2007.

[41] Nicholas Carlini, Florian Tramer, Eric Wallace, et al. Extracting Training Data from Large Language Models. *USENIX Security Symposium*, 2021. URL https://arxiv.org/abs/2012.07805.

[42] Chainlit. Chainlit: Build Production-Ready Conversational AI Applications, 2024. URL https://chainlit.io.

[43] Harrison Chase. LangChain, 2022. URL https://github.com/langchain-ai/langchain.

[44] Kushal Chawla, Jaysa Ramirez, Rene Clever, Gale Lucas, Jonathan May, and Jonathan Gratch. CaSiNo: A Corpus of Campsite Negotiation Dialogues for Automatic Negotiation Systems. In *Proceedings of NAACL*, 2021.

[45] Huayu Chen, Guande Zheng, Yimeng Kim, and Yiwen Chen. Noise Contrastive Alignment of Language Models with Explicit Rewards. *arXiv Preprint arXiv:2402.05369*, 2024. URL https://arxiv.org/abs/2402.05369.

[46] Jianlv Chen, Shitao Luo, Mingzheng Zhang, Zheng Liu, Yingxia Xiao, and Defu Han. Hybrid Retrieval for Open-Domain Question Answering. *arXiv Preprint arXiv:2210.06029*, 2022. URL https://arxiv.org/abs/2210.06029.

[47] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv Preprint arXiv:2402.03216*, 2024.

[48] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv Preprint arXiv:2305.05176*, 2023. URL https://arxiv.org/abs/2305.05176.

[49] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating Large Language Models Trained on Code. *arXiv Preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

[50] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending Context Window of Large Language Models via Positional Interpolation. *arXiv Preprint arXiv:2306.15595*, 2023. URL https://arxiv.org/abs/2306.15595.

[51] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training Deep Nets with Sublinear Memory Cost. *arXiv Preprint arXiv:1604.06174*, 2016.

[52] Yuxin Chen et al. MOSAIC: Safe Agentic Training via Plan-Check-Act-Refuse. arXiv preprint, 2026.

[53] Zhiyu Chen, Yiwei Deng, Ruiqi Zhang, Hao Sun, and Weizhu Chen. GSPO: Sequence-Level Policy Optimization for Language Model Alignment. *arXiv Preprint arXiv:2502.12459*, 2025. URL https://arxiv.org/abs/2502.12459.

[54] Boris Cherny. Loops Are the Next Step from Agents. Talk at Meta @Scale Conference, 2026. Head of Claude Code, Anthropic.

[55] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv Preprint arXiv:2504.19413*, 2025. URL https://arxiv.org/abs/2504.19413.

[56] Seokhyun Choi, Hyunji Park, Doyoung Moon, Kyuyoung Kim, and Edward Choi. GOPO: Group Ordinal Policy Optimization for LLM Alignment with Non-Verifiable Rewards. *arXiv Preprint arXiv:2505.12948*, 2025. URL https://arxiv.org/abs/2505.12948.

[57] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. PaLM: Scaling Language Modeling with Pathways. *arXiv Preprint arXiv:2204.02311*, 2022.

[58] Sayak Ray Chowdhury, Anush Chakraborty, Srinadh Natarajan, Alekh Agarwal, and David Sontag. Provably Robust DPO: Aligning Language Models with Noisy Feedback. *arXiv Preprint arXiv:2403.00409*, 2024. URL https://arxiv.org/abs/2403.00409.

[59] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.03741.

[60] Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James Glass, and Pengcheng He. DoLA: Decoding by Contrasting Layers Improves Factuality in Large Language Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.03883.

[61] Hyung Won Chung, Le Hou, Shayne Longpre, et al. Scaling Instruction-Finetuned Language Models. *Journal of Machine Learning Research*, 2024. URL https://arxiv.org/abs/2210.11416.

[62] Jacob Cohen. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 1960.

[63] World Wide Web Consortium. Decentralized Identifiers (DIDs) v1.0, 2022. URL https://www.w3.org/TR/did-core/.

[64] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, 2009.

[65] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse Autoencoders Find Highly Interpretable Features in Language Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.08600.

[66] Mehul Damani, Raghav Puri, Stewart Slocum, Idan Shenfeld, Leshem Choshen, Yoon Kim, and Jacob Andreas. Beyond binary rewards: Training language models to reason about their uncertainty. arXiv preprint arXiv:2507.16806, 2025. URL https://arxiv.org/abs/2507.16806.

[67] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.08691.

[68] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14135.

[69] Abhishek Das, Théophile Gerber, Georgia Gkioxari, Stefan Lee, Devi Parikh, and Dhruv Batra. TarMAC: Targeted Multi-Agent Communication. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1810.11187.

[70] Databricks. DBRX: A New State-of-the-Art Open LLM. *Databricks Blog*, 2024. URL https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm.

[71] Google DeepMind. AlphaProof and AlphaGeometry 2: Solving Olympiad Geometry Without Human Demonstrations, 2024. URL https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/.

[72] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv Preprint arXiv:2405.04434*, 2024. URL https://arxiv.org/abs/2405.04434.

[73] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv Preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

[74] DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL https://arxiv.org/abs/2606.19348.

[75] Daya Guo DeepSeek-AI, Dejian Yang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv Preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

[76] Xiang Deng, Yu Gu, Boyuan Zheng, et al. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.06070.

[77] Michael Dennis, Natasha Jaques, Eugene Vinitsky, et al. Emergent Complexity and Zero-Shot Transfer via Unsupervised Environment Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[78] Leon Derczynski, Usman Bhatt, and Kalina Bontcheva. Garak: A framework for security probing large language models. arXiv preprint arXiv:2406.11036, 2024. URL https://arxiv.org/abs/2406.11036.

[79] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.14314.

[80] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, 2019. URL https://arxiv.org/abs/1810.04805.

[81] Yixin Dong, Charlie Moon, Yuekai Wang, et al. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv Preprint arXiv:2411.15100*, 2024.

[82] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving Factuality and Reasoning in Language Models Through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2305.14325.

[83] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL2: Fast Reinforcement Learning via Slow Reinforcement Learning. *arXiv Preprint arXiv:1611.02779*, 2016.

[84] Edmund H. Durfee, Victor R. Lesser, and Daniel D. Corkill. Trends in Cooperative Distributed Problem Solving. *IEEE Transactions on Knowledge and Data Engineering*, 1989. URL https://doi.org/10.1109/69.43404.

[85] Hermann Ebbinghaus. Über Das Gedächtnis: Untersuchungen Zur Experimentellen Psychologie, 1885.

[86] Darren Edge, Ha Trinh, Newman Cheng, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv Preprint arXiv:2404.16130*, 2024. URL https://arxiv.org/abs/2404.16130.

[87] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babber, and Dan Alistarh. Extreme Compression of Large Language Models via Additive Quantization. *arXiv Preprint arXiv:2401.06118*, 2024. URL https://arxiv.org/abs/2401.06118.

[88] Arpad E Elo. The Rating of Chess Players, Past and Present, 1978.

[89] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAs: Automated Evaluation of Retrieval Augmented Generation. *arXiv Preprint arXiv:2309.15217*, 2023. URL https://arxiv.org/abs/2309.15217.

[90] Kawin Ethayarajh. How Contextual Are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. URL https://arxiv.org/abs/1909.00512.

[91] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model Alignment as Prospect Theoretic Optimization. *arXiv Preprint arXiv:2402.01306*, 2024. URL https://arxiv.org/abs/2402.01306.

[92] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, 2024. OJ L, 2024/1689, 12.7.2024.

[93] Hugging Face. OpenEnv: An Interface Library for RL Post-Training with Environments, 2025. URL https://github.com/huggingface/OpenEnv.

[94] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 2022.

[95] David Ferrucci, Eric Brown, Jennifer Chu-Carroll, et al. Building Watson: An Overview of the DeepQA Project. *AI Magazine*, 2010.

[96] John Rupert Firth. A Synopsis of Linguistic Theory, 1930–1955. *Studies in Linguistic Analysis*, 1957.

[97] Joseph L Fleiss. Measuring Nominal Scale Agreement Among Many Raters. *Psychological Bulletin*, 1971.

[98] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[99] Foundation for Intelligent Physical Agents. FIPA ACL Message Structure Specification, 2002. URL http://www.fipa.org/specs/fipa00061/.

[100] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval. *arXiv Preprint arXiv:2109.10086*, 2021.

[101] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021.

[102] Tiago Forte. Building a Second Brain: A Proven Method to Organize Your Digital Life and Unlock Your Creative Potential, 2022.

[103] Nadime Francis, Alastair Green, Paolo Guagliardo, et al. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018.

[104] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2301.00774.

[105] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. *arXiv Preprint arXiv:2210.17323*, 2023. URL https://arxiv.org/abs/2210.17323.

[106] Yao Fu, Rameswar Panda, Xinyao Niu, et al. Data Engineering for Scaling Language Models to 128K Context. *arXiv Preprint arXiv:2402.10171*, 2024. URL https://arxiv.org/abs/2402.10171.

[107] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. *arXiv Preprint arXiv:2402.02057*, 2024. URL https://arxiv.org/abs/2402.02057.

[108] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schueler, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. Technical report, 2026. URL https://arxiv.org/abs/2607.20709.

[109] Avigdor Gal, Ateret Anaby-Tavor, Alberto Trombetta, and Danilo Montesi. A Framework for Modeling and Evaluating Automatic Semantic Reconciliation. In *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB)*, 2005. URL https://link.springer.com/chapter/10.1007/11896548_42.

[110] Isabel O. Gallegos, Ryan A. Rossi, Joe Barber, Shanshan Tong, et al. Bias and Fairness in Large Language Models: A Survey. *Computational Linguistics*, 2024. URL https://arxiv.org/abs/2309.00770.

[111] Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[112] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise Zero-Shot Dense Retrieval Without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://arxiv.org/abs/2212.10496.

[113] Javier García and Fernando Fernández. A Comprehensive Survey on Safe Reinforcement Learning. *Journal of Machine Learning Research*, 2015.

[114] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. RealToxicityPrompts: Evaluating neural toxic degeneration in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. URL https://arxiv.org/abs/2009.11462.

[115] Google Gemini Team. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. *arXiv Preprint arXiv:2403.05530*, 2024. URL https://arxiv.org/abs/2403.05530.

[116] Georgi Gerganov. GGUF: GPT-Generated Unified Format (llama.cpp), 2023. URL https://github.com/ggerganov/llama.cpp.

[117] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[118] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & Faster Large Language Models via Multi-Token Prediction. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2404.19737.

[119] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

[120] Charles A. E Goodhart. Problems of Monetary Management: The U.K. Experience. *Monetary Theory and Practice*, 1984.

[121] Google. Agent2Agent (A2A) Protocol, 2025. URL https://developers.google.com/agent2agent.

[122] Google DeepMind. Decoupled DiLoCo: Distributed Training Across Geographic Regions. arXiv preprint, 2026.

[123] Google Research. Thinking to Recall: How Reasoning Unlocks Parametric Knowledge in LLMs. COLM 2026, 2026.

[124] Alexander Gorbatenko. Online DPO with Synchronised Reference Model Updates. *TRL Documentation*, 2024.

[125] Pierre-Paul Grassé. La Reconstruction Du Nid Et Les Coordinations Interindividuelles Chez Bellicositermes Natalensis Et Cubitermes Sp. La Théorie de La Stigmergie: Essai d'interprétation Du Comportement Des Termites Constructeurs. *Insectes Sociaux*, 1959.

[126] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 Herd of Models. *arXiv Preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

[127] Alex Graves, Greg Wayne, Malcolm Reynolds, et al. Hybrid Computing Using a Neural Network with Dynamic External Memory. *Nature*, 2016.

[128] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You've Signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[129] JSON-RPC Working Group. JSON-RPC 2.0 Specification, 2010. URL https://www.jsonrpc.org/specification.

[130] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv Preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/abs/2312.00752.

[131] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2312.00752.

[132] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning (ICML)*, 2017.

[133] Mandy Guo, Joshua Ainslie, David Uthus, et al. LongT5: Efficient Text-to-Text Transformer for Long Sequences. *Findings of the Association for Computational Linguistics: NAACL 2022*, 2022. URL https://arxiv.org/abs/2112.07916.

[134] Ruiqi Guo, Philip Sun, Erik Lindgren, et al. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning (ICML)*, 2020.

[135] Shangmin Guo, Biao Zhang, Tianlin Liu, et al. Direct Language Model Alignment from Online AI Feedback. *arXiv Preprint arXiv:2402.04792*, 2024. URL https://arxiv.org/abs/2402.04792.

[136] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL https://arxiv.org/abs/2002.08909.

[137] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. URL https://arxiv.org/abs/1801.01290.

[138] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1912.01603.

[139] Chi Han, Minlie Li, and Wenting Chen. SAPO: Soft Adaptive Policy Optimization for Efficient LLM Alignment. *arXiv Preprint arXiv:2503.01739*, 2025. URL https://arxiv.org/abs/2503.01739.

[140] Daniel Han and Michael Han. Unsloth: Efficient LLM Fine-Tuning, 2024. URL https://github.com/unslothai/unsloth.

[141] Dick Hardt. The OAuth 2.0 Authorization Framework, 2012. URL https://datatracker.ietf.org/doc/html/rfc6749.

[142] Steve Harris and Andy Seaborne. SPARQL 1.1 Query Language, 2013. URL https://www.w3.org/TR/sparql11-query/.

[143] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. ToxiGen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. URL https://arxiv.org/abs/2203.09509.

[144] Barbara Hayes-Roth. A Blackboard Architecture for Control. *Artificial Intelligence*, 1985. URL https://doi.org/10.1016/0004-3702(85)90063-3.

[145] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. LoRA+: Efficient Low Rank Adaptation of Large Models. *arXiv Preprint arXiv:2402.12354*, 2024. URL https://arxiv.org/abs/2402.12354.

[146] Dan Hendrycks, Collin Burns, Steven Basart, et al. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[147] Ralf Herbrich, Tom Minka, and Thore Graepel. TrueSkill™: A Bayesian Skill Rating System. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2006. URL https://proceedings.neurips.cc/paper/2006/hash/f44ee263952e65b3610b8ba51229d1f9-Abstract.html.

[148] Carl Hewitt, Peter Bishop, and Richard Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

[149] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. *arXiv Preprint arXiv:1503.02531*, 2015.

[150] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training Compute-Optimal Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2203.15556.

[151] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic Preference Optimization Without Reference Model. *arXiv Preprint arXiv:2403.07691*, 2024. URL https://arxiv.org/abs/2403.07691.

[152] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework, 2024.

[153] Arian Hosseini, Xingdi Yuan, Pascal Poupart, Adam Trischler, and Yoshua Bengio. V-STaR: Training Verifiers for Self-Taught Reasoners. *arXiv Preprint arXiv:2402.06457*, 2024.

[154] Neil Houlsby, Andrei Giber, Stanislaw Jastrzebski, et al. Parameter-Efficient Transfer Learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1902.00751.

[155] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, et al. Liger-Kernel: Efficient Triton Kernels for LLM Training. *arXiv Preprint arXiv:2410.10989*, 2024. URL https://arxiv.org/abs/2410.10989.

[156] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2106.09685.

[157] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv Preprint arXiv:2106.09685*, 2022.

[158] Jian Hu, Xibin Tao, Weixun Zhu, Sicheng Yang, Jingwen Liu, and Zilin Li. OpenRLHF: An Easy-to-Use, Scalable and High-Performance RLHF Framework. *arXiv Preprint arXiv:2405.11143*, 2024.

[159] Qinghao Hu, Shang Yang, et al. Taming the Long-Tail: Efficient Reasoning RL Training with Adaptive Drafter. *arXiv Preprint arXiv:2511.16665*, 2025. URL https://arxiv.org/abs/2511.16665.

[160] Shengding Hu, Yuge Tu, Xu Han, et al. MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. *arXiv Preprint arXiv:2404.06395*, 2024. URL https://arxiv.org/abs/2404.06395.

[161] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2310.03302.

[162] Yanping Huang, Youlong Cheng, Ankur Bapna, et al. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[163] Evan Hubinger, Carson Denison, Jesse Mu, et al. Sleeper Agents: Training Deceptive LLMs That Persist Through Safety Training. *arXiv Preprint arXiv:2401.05566*, 2024.

[164] LangChain Inc. LangGraph: Build Stateful Multi-Actor Applications with LLMs, 2024. URL https://github.com/langchain-ai/langgraph.

[165] LangChain Inc. LangGraph Studio: The First Agent IDE, 2024. URL https://github.com/langchain-ai/langgraph-studio.

[166] Streamlit Inc. Streamlit: The Fastest Way to Build and Share Data Apps, 2024. URL https://streamlit.io.

[167] Geoffrey Irving, Paul Christiano, and Dario Amodei. AI Safety via Debate. *arXiv Preprint arXiv:1805.00899*, 2018. URL https://arxiv.org/abs/1805.00899.

[168] Max Jaderberg, Wojciech M. Czarnecki, Iain Dunning, et al. Human-Level Performance in 3D Multiplayer Games with Population-Based Reinforcement Learning. *Science*, 2019. URL https://www.science.org/doi/10.1126/science.aau6249.

[169] Neel Jain, Ping-yeh Chiang, Yuxin Wen, et al. NEFTune: Noisy Embeddings Improve Instruction Finetuning. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.05914.

[170] Sarthak Jain and Byron C. Wallace. Attention Is Not Explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019. URL https://arxiv.org/abs/1902.10186.

[171] Eric Jang, Shixiang Gu, and Ben Poole. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*, 2017.

[172] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models Through Question Complexity. *arXiv Preprint arXiv:2403.14403*, 2024. URL https://arxiv.org/abs/2403.14403.

[173] Jiatao Ji, Adam Fisch, Jason Weston, and Sainbayar Sukhbaatar. Towards Exact Optimization of Language Model Alignment. *arXiv Preprint arXiv:2402.05369*, 2024. URL https://arxiv.org/abs/2402.05369.

[174] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 2023. URL https://arxiv.org/abs/2202.03629.

[175] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B. *arXiv Preprint arXiv:2310.06825*, 2023. URL https://arxiv.org/abs/2310.06825.

[176] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. Mixtral of Experts. *arXiv Preprint arXiv:2401.04088*, 2024. URL https://arxiv.org/abs/2401.04088.

[177] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized Level Replay. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. URL https://arxiv.org/abs/2010.03934.

[178] Yibo Jiang et al. Reasoning Beyond Chain-of-Thought: A Latent Computational View. arXiv preprint arXiv:2601.08058, 2025. URL https://arxiv.org/abs/2601.08058.

[179] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

[180] Bowen Jin, Hansi Zeng, et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *arXiv Preprint arXiv:2503.09516*, 2025.

[181] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 2021.

[182] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT), 2015. URL https://datatracker.ietf.org/doc/html/rfc7519.

[183] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of ACL*, 2017.

[184] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly Accurate Protein Structure Prediction with AlphaFold. *Nature*, 2021.

[185] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language Models (Mostly) Know What They Know. *arXiv Preprint arXiv:2207.05221*, 2022. URL https://arxiv.org/abs/2207.05221.

[186] Damjan Kalajdzievski. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. *arXiv Preprint arXiv:2312.03732*, 2023. URL https://arxiv.org/abs/2312.03732.

[187] Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling Laws for Neural Language Models. *arXiv Preprint arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

[188] Andrej Karpathy. Context Engineering for AI Agents. Social media post, 2025. June 2025.

[189] Andrej Karpathy. AutoResearch: Automated ML Experimentation via Agent Loops. Open-source project, 2026. URL https://github.com/karpathy/autoresearch.

[190] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL https://arxiv.org/abs/2004.04906.

[191] Junlong Ke, Zichen Wen, Weijia Li, Conghui He, and Linfeng Zhang. Respecting Self-Uncertainty in On-Policy Self-Distillation for Efficient LLM Reasoning. *arXiv preprint arXiv:2605.13255*, 2026. URL https://arxiv.org/abs/2605.13255.

[192] Omar Khattab and Matei Zaharia. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference*, 2020. URL https://arxiv.org/abs/2004.12832.

[193] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.03714.

[194] Seungone Kim, Se June Jang, et al. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. *Findings of the ACL*, 2023. URL https://arxiv.org/abs/2305.02301.

[195] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[196] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming Catastrophic Forgetting in Neural Networks. In *Proceedings of the National Academy of Sciences*, 2017.

[197] Levente Kocsis and Csaba Szepesvári. Bandit Based Monte-Carlo Planning. In *European Conference on Machine Learning (ECML)*, 2006.

[198] Jing Yu Koh, Robert Lo, Lawrence Jang, et al. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2401.13649.

[199] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large Language Models Are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.11916.

[200] Dawid Jan Kopiczko, Tijmen Blankevoort, and Markus Nagel. VeRA: Vector-Based Random Matrix Adaptation. *arXiv Preprint arXiv:2310.11454*, 2024. URL https://arxiv.org/abs/2310.11454.

[201] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, et al. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

[202] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS*, 2012.

[203] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2302.09664.

[204] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics*, 2019.

[205] Justin Kwok, Daniel Kang, Ananya Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. CLM: Contrastive language models. GitHub repository, 2026. URL https://github.com/Contrastive-LM/CLM.

[206] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, 2023. URL https://arxiv.org/abs/2309.06180.

[207] Heinrich Küttler, Nantas Nardelli, Alexander H. Miller, et al. The NetHack Learning Environment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2006.13760.

[208] Timothée Lacroix, Guillaume Obozinski, and Nicolas Usunier. Tensor Decompositions for Temporal Knowledge Base Completion. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/2004.04926.

[209] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tülu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv Preprint arXiv:2411.15124*, 2024. URL https://arxiv.org/abs/2411.15124.

[210] Angeliki Lazaridou and Marco Baroni. Emergent Multi-Agent Communication in the Deep Learning Era. *arXiv Preprint arXiv:2006.02419*, 2020. URL https://arxiv.org/abs/2006.02419.

[211] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Hoi. CodeRL: Mastering Code Generation Through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[212] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, et al. The BrowserGym Ecosystem for Web Agent Research. *arXiv Preprint arXiv:2412.05467*, 2024. URL https://arxiv.org/abs/2412.05467.

[213] Chankyu Lee, Rajarshi Roy, Menber Xu, et al. NV-Embed: Improved Techniques for Training LLMs as Generalist Embedding Models. *arXiv Preprint arXiv:2405.17428*, 2024.

[214] Katherine Lee, Daphne Ippolito, Andrew Nystrom, et al. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the ACL*, 2022. URL https://arxiv.org/abs/2107.06499.

[215] Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on EMNLP*, 2021. URL https://arxiv.org/abs/2104.08691.

[216] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.17192.

[217] Mike Lewis, Denis Yarats, Yann N. Dauphin, Devi Parikh, and Dhruv Batra. Deal or No Deal? End-to-End Learning for Negotiation Dialogues. In *Proceedings of EMNLP*, 2017.

[218] Mike Lewis, Yinhan Liu, Naman Goyal, et al. BART: Denoising Sequence-to-Sequence Pre-Training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL https://arxiv.org/abs/1910.13461.

[219] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.11401.

[220] Shen Li, Yanli Zhao, Rohan Varma, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. In *Proceedings of the VLDB Endowment*, 2020.

[221] Xian Li, Ari Holtzman, Daniel Fried, et al. Contrastive Decoding: Open-Ended Text Generation as Optimization. *ACL*, 2023.

[222] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the ACL*, 2021. URL https://arxiv.org/abs/2101.00190.

[223] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-judge: Accept when confident, escalate when unsure. *arXiv preprint arXiv:2609.26550*, 2026. URL https://arxiv.org/abs/2609.26550.

[224] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2401.15077.

[225] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards General Text Embeddings with Multi-Stage Contrastive Learning. *arXiv Preprint arXiv:2308.03281*, 2023.

[226] Zheng Li et al. UniClawBench: A Bilingual Benchmark for Interactive Agent Evaluation. arXiv preprint, 2026.

[227] Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let's Verify Step by Step. *arXiv Preprint arXiv:2305.20050*, 2023.

[228] Chin-Yew Lin. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004. URL https://aclanthology.org/W04-1013/.

[229] Ji Lin, Jiaming Tang, Haotian Tang, et al. AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL https://arxiv.org/abs/2306.00978.

[230] Kevin Lin, Charlie Snell, Yu Wang, et al. Sleep-Time Compute: Beyond Inference Scaling at Test-Time. *arXiv Preprint arXiv:2504.13171*, 2025. URL https://arxiv.org/abs/2504.13171.

[231] Long-Ji Lin. Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching. *Machine Learning*, 1992.

[232] Xiaoqiang Lin, Aritra Ghosh, Bryan Kian Hsiang Low, Anshumali Shrivastava, and Vijai Mohan. REFRAG: Rethinking RAG Based Decoding. *arXiv Preprint arXiv:2509.01092*, 2025.

[233] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2310.01889.

[234] Haokun Liu, Derek Tam, Mohammed Muqeeth, et al. Few-Shot Parameter-Efficient Fine-Tuning Is Better and Cheaper Than in-Context Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.05638.

[235] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What Makes Good in-Context Examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO), ACL Workshop*, 2022. URL https://arxiv.org/abs/2101.06804.

[236] Keller Jordan Liu, Yuchen Jin, Zhanhui Zhou, et al. Muon: An Optimizer for Hidden Layers in Neural Networks. arXiv preprint, 2025. URL https://github.com/KellerJordan/Muon.

[237] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2307.03172.

[238] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2307.03172.

[239] Qiguang et al Liu. Answer Convergence as a Signal for Early Stopping in Reasoning. In *Proceedings of EMNLP*, 2025. URL https://aclanthology.org/2025.emnlp-main.904/.

[240] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, et al. DoRA: Weight-Decomposed Low-Rank Adaptation. *arXiv Preprint arXiv:2402.09353*, 2024. URL https://arxiv.org/abs/2402.09353.

[241] Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as Agents. *arXiv Preprint arXiv:2308.03688*, 2023.

[242] Xingyu et al Liu. Learning Like Humans: Advancing LLM Reasoning Capabilities via Adaptive Difficulty Curriculum Learning. *arXiv Preprint arXiv:2505.08364*, 2025. URL https://arxiv.org/abs/2505.08364.

[243] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG Evaluation Using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2303.16634.

[244] Yihao Liu, Lefan Han, Yifan Tan, et al. Understanding and Mitigating the Pretraining Distribution Bias in GRPO. *arXiv Preprint arXiv:2505.07888*, 2025. URL https://arxiv.org/abs/2505.07888.

[245] Yinhan Liu, Myle Ott, Naman Goyal, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv Preprint arXiv:1907.11692*, 2019. URL https://arxiv.org/abs/1907.11692.

[246] Zechun Liu, Barlas Oguz, Changsheng Zhao, et al. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. *arXiv Preprint arXiv:2305.17888*, 2023. URL https://arxiv.org/abs/2305.17888.

[247] Zhuokai Liu, Hao Dong, et al. DLER: Doing Length Penalty Right—Incentivizing More Intelligence Per Token. *arXiv Preprint arXiv:2510.15110*, 2025. URL https://arxiv.org/abs/2510.15110.

[248] Zichen Liu et al. OctoThinker: Mid-Training Determines RL-Readiness of Base Models. arXiv preprint, 2025.

[249] Zirui Liu, Jiayi Yuan, Hongye Jin, et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.02750.

[250] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. *arXiv Preprint arXiv:1711.05101*, 2019. URL https://arxiv.org/abs/1711.05101.

[251] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.02275.

[252] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv Preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

[253] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. URL https://arxiv.org/abs/2104.08786.

[254] Shenggui Luo et al. DualKV: Shared-prompt flash attention for efficient RL training with large rollouts and long contexts. *arXiv preprint arXiv:2605.15422*, 2025. URL https://arxiv.org/abs/2605.15422.

[255] Zhenyu Luo, Ziyan Chen, Yulun Jiang, et al. ScaleRL: Scaling Reinforcement Learning for LLM Reasoning. *arXiv Preprint arXiv:2505.16356*, 2025. URL https://arxiv.org/abs/2505.16356.

[256] Zhixun Luo, Jiaqi Shi, Tao Yu, and Minlie Chen. VESPO: Variational Sequence-Level Soft Policy Optimization for LLM Alignment. *arXiv Preprint arXiv:2505.07508*, 2025. URL https://arxiv.org/abs/2505.07508.

[257] Tobi Lütke. Context Engineering. Social media post, 2025. CEO of Shopify, June 2025.

[258] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodriques, and Andrew D. White. PaperQA: Retrieval-Augmented Generative Agent for Scientific Research. *arXiv Preprint arXiv:2312.07559*, 2023. URL https://arxiv.org/abs/2312.07559.

[259] Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[260] Adyasha Maharana, Dong-Ho Lee, Sergey Tuber, Mohit Ruber, Francesco Barbieri, and Mohit Bansal. LoCoMo: Long-Context Conversation with Memory Operations. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[261] Yu. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[262] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of EMNLP*, 2023. URL https://arxiv.org/abs/2303.08896.

[263] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An Empirical Model of Large-Batch Training. *arXiv Preprint arXiv:1812.06162*, 2018.

[264] Jiangfei et al Mei. APRIL: Active Partial Rollouts in Reinforcement Learning to Tame Long-Tail Generation. *arXiv Preprint arXiv:2509.18521*, 2025. URL https://arxiv.org/abs/2509.18521.

[265] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple Preference Optimization with a Reference-Free Reward. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2405.14734.

[266] Meta AI. Remember When It Matters: Proactive Memory for Long-Horizon Agents. arXiv preprint, 2026.

[267] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A Benchmark for General AI Assistants. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2311.12983.

[268] Paulius Micikevicius, Sharan Narang, Jonah Alben, et al. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*, 2018.

[269] Microsoft. Semantic Kernel: SDK for Integrating AI Models into Applications, 2023. URL https://github.com/microsoft/semantic-kernel.

[270] Sewon Min, Xinxi Lyu, Ari Holtzman, et al. Rethinking the Role of Demonstrations: What Makes in-Context Learning Work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022. URL https://arxiv.org/abs/2202.12837.

[271] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of What Art? A Call for Multi-Prompt LLM Evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024.

[272] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 2015. URL https://www.nature.com/articles/nature14236.

[273] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, et al. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016. URL https://arxiv.org/abs/1602.01783.

[274] MosaicML. MPT-7B: A New Standard for Open-Source, Commercially Usable LLMs. *MosaicML Blog*, 2023. URL https://www.mosaicml.com/blog/mpt-7b.

[275] João Moura. CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents, 2023. URL https://github.com/crewAIInc/crewAI.

[276] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, et al. WebGPT: Browser-Assisted Question-Answering with Human Feedback. *arXiv Preprint arXiv:2112.09332*, 2021. URL https://arxiv.org/abs/2112.09332.

[277] Nikita Nangia, Clara Vania, Rasika Bhalerao, and Samuel R. Bowman. CrowS-Pairs: A challenge dataset for measuring social biases in masked language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL https://arxiv.org/abs/2010.00133.

[278] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[279] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *arXiv Preprint arXiv:2104.04473*, 2021.

[280] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.

[281] Minh Nguyen. Min-p Sampling: A Simple Baseline for Better LLM Decoding. *arXiv Preprint arXiv:2310.06022*, 2024.

[282] Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani. Algorithmic Game Theory, 2007. URL https://www.cs.cmu.edu/~sandholm/cs15-892F13/algorithmic-game-theory.pdf.

[283] Rodrigo Nogueira and Kyunghyun Cho. Passage Re-Ranking with BERT. *arXiv Preprint arXiv:1901.04085*, 2019.

[284] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document Ranking with a Pretrained Sequence-to-Sequence Model. *Findings of EMNLP*, 2020.

[285] NVIDIA. Dynamo: A Datacenter-Scale Distributed Inference Serving Framework. Open-source project, 2025. URL https://github.com/ai-dynamo/dynamo.

[286] NVIDIA. NeMo Gym: Interactive RL Environments for LLM Agent Training. Technical report, 2025. URL https://developer.nvidia.com/nemo-gym.

[287] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv Preprint arXiv:2112.00114*, 2021. URL https://arxiv.org/abs/2112.00114.

[288] Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-Context Learning and Induction Heads. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

[289] Open Policy Agent Contributors. Open policy agent. https://www.openpolicyagent.org, 2019. Open-source policy engine for cloud-native environments.

[290] OpenAI. GPT-4 Technical Report. *arXiv Preprint arXiv:2303.08774*, 2023.

[291] OpenAI. Learning to Reason with LLMs, 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

[292] OpenAI. Swarm: An Educational Framework for Lightweight Multi-Agent Orchestration, 2024. URL https://github.com/openai/swarm.

[293] OpenAI. OpenAI Agents SDK, 2025. URL https://github.com/openai/openai-agents-python.

[294] OpenAI. OpenAI o3 and o4-mini System Card, 2025. URL https://openai.com/index/o3-and-o4-mini-system-card/.

[295] Addy Osmani. Loop Engineering. Blog post, 2026. URL https://addyosmani.com/blog/loop-engineering/.

[296] Long Ouyang, Jeffrey Wu, Xu Jiang, et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2203.02155.

[297] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv Preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[298] Leyi Pan, Shuchang Tao, Yunpeng Zhai, Lingzhe Zhang, Zhaoyang Liu, Bolin Ding, Aiwei Liu, and Lijie Wen. RLCSD: Reinforcement Learning with Contrastive On-Policy Self-Distillation. *arXiv preprint arXiv:2606.11709*, 2026. URL https://arxiv.org/abs/2606.11709.

[299] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002. URL https://aclanthology.org/P02-1040/.

[300] Raja Parasuraman and Victor Riley. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors*, 1997. URL https://doi.org/10.1518/001872097778543886.

[301] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. URL https://arxiv.org/abs/2304.03442.

[302] Alicia Parrish, Angelica Chen, Nikita Nangia, Vishakh Padmakumar, Jason Phang, Jana Thompson, Phu Mon Htut, and Samuel R. Bowman. BBQ: A hand-built bias benchmark for question answering. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. URL https://arxiv.org/abs/2110.08193.

[303] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-Driven Exploration by Self-Supervised Prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[304] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2305.15334.

[305] Bo Peng, Eric Alcaide, Quentin Anthony, et al. RWKV: Reinventing RNNs for the Transformer Era. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL https://arxiv.org/abs/2305.13048.

[306] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shao. YaRN: Efficient Context Window Extension of Large Language Models. *arXiv Preprint arXiv:2309.00071*, 2023.

[307] Ethan Perez, Saffron Huang, Francis Song, et al. Red Teaming Language Models with Language Models. In *Proceedings of EMNLP*, 2022. URL https://arxiv.org/abs/2202.03286.

[308] R. L Plackett. The Analysis of Permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 1975.

[309] Ofir Press, Noah A. Smith, and Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Generalization. *ICLR*, 2022.

[310] Tom Preston-Werner. Semantic Versioning 2.0.0, 2024. URL https://semver.org/.

[311] Pranav Putta, Edmund Mills, Naman Garg, et al. Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents. *arXiv Preprint arXiv:2408.07199*, 2024.

[312] Pydantic Team. Pydantic AI: Type-Safe Agent Framework. https://ai.pydantic.dev, 2024.

[313] PyTorch Team. Miles: A PyTorch-Native Stack for Large-Scale LLM RL Post-Training. PyTorch Blog, 2026. URL https://pytorch.org/blog/miles-a-pytorch-native-stack-for-large-scale-llm-rl-post-training/.

[314] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. Zero Bubble Pipeline Parallelism. *arXiv Preprint arXiv:2401.10241*, 2023.

[315] Zhenting Qi, Mingyuan Wan, Jialin Cao, and Min Lin. Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solvers. *arXiv Preprint arXiv:2408.06195*, 2024.

[316] Yiwei Qin, Xuefeng Li, Haoyang Zou, et al. O1 Replication Journey: A Strategic Progress Report. *arXiv Preprint arXiv:2410.18982*, 2024. URL https://arxiv.org/abs/2410.18982.

[317] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.16789.

[318] Adrian H Rackauckas. RAG-Fusion: A New Take on Retrieval-Augmented Generation, 2024.

[319] Alec Radford, Jeffrey Wu, Rewon Child, David Luen, Dario Amodei, and Ilya Sutskever. Language Models Are Unsupervised Multitask Learners. *OpenAI Blog*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[320] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.18290.

[321] Colin Raffel, Noam Shazeer, Adam Roberts, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 2020. URL https://arxiv.org/abs/1910.10683.

[322] Samyam Rajbhandari, Jeff Rasley, Olatunji Rber, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *arXiv Preprint arXiv:1910.02054*, 2020.

[323] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. *arXiv Preprint arXiv:2104.07857*, 2021.

[324] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. URL https://arxiv.org/abs/1803.11605.

[325] Eric Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3, 2018. URL https://datatracker.ietf.org/doc/html/rfc8446.

[326] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 1951.

[327] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009.

[328] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning Representations by Back-Propagating Errors. *Nature*, 1986. URL https://doi.org/10.1038/323533a0.

[329] Gavin A. Rummery and Mahesan Niranjan. On-Line q-Learning Using Connectionist Systems, 1994.

[330] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv Preprint arXiv:1910.01108*, 2019.

[331] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

[332] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized Experience Replay. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1511.05952.

[333] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[334] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 2020. URL https://arxiv.org/abs/1911.08265.

[335] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015. URL https://arxiv.org/abs/1502.05477.

[336] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1506.02438.

[337] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv Preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

[338] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the ACL*, 2016. URL https://arxiv.org/abs/1508.07909.

[339] Alexander Sergeev and Mike Del Balso. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv Preprint arXiv:1802.05799*, 2018.

[340] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. *arXiv Preprint arXiv:2407.08691*, 2024. URL https://arxiv.org/abs/2407.08691.

[341] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv Preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

[342] Lloyd S Shapley. Stochastic Games. In *Proceedings of the National Academy of Sciences*, 1953. URL https://www.pnas.org/doi/10.1073/pnas.39.10.1095.

[343] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

[344] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2303.11366.

[345] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv Preprint arXiv:1909.08053*, 2019.

[346] Yoav Shoham and Kevin Leyton-Brown. Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations, 2008. URL http://www.masfoundations.org/.

[347] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations (ICLR)*, 2021.

[348] Significant Gravitas. AutoGPT: An Autonomous GPT-4 Experiment. Open-source project, 2023. URL https://github.com/Significant-Gravitas/AutoGPT.

[349] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature*, 2016. URL https://www.nature.com/articles/nature16961.

[350] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the Game of Go Without Human Knowledge. *Nature*, 2017. URL https://www.nature.com/articles/nature24270.

[351] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and Characterizing Reward Hacking. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2209.13085.

[352] Reid G Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, 1980. URL https://doi.org/10.1109/TC.1980.1675516.

[353] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. *arXiv Preprint arXiv:2408.03314*, 2024.

[354] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999. URL https://www2.cs.arizona.edu/~rts/tdbbook.pdf.

[355] Karen Sparck Jones. A Statistical Interpretation of Term Specificity and Its Application in Retrieval. *Journal of Documentation*, 1972.

[356] Larry R Squire. Declarative and Nondeclarative Memory: Multiple Brain Systems Supporting Learning and Memory. *Journal of Cognitive Neuroscience*, 1992. URL https://doi.org/10.1162/jocn.1992.4.3.232.

[357] Peter Steinberger. OpenClaw: An Always-On Agent Orchestrator. Open-source project, 2026. URL https://github.com/openclaw.

[358] Nisan Stiennon, Long Ouyang, Jeffrey Wu, et al. Learning to Summarize from Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2009.01325.

[359] Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. Whitening Sentence Representations for Better Semantics and Faster Retrieval. *arXiv Preprint arXiv:2103.15316*, 2021. URL https://arxiv.org/abs/2103.15316.

[360] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 2024.

[361] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-End Memory Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. URL https://arxiv.org/abs/1503.08895.

[362] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning Multiagent Communication with Backpropagation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1605.07736.

[363] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research (TMLR)*, 2024. URL https://arxiv.org/abs/2309.02427.

[364] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A Simple and Effective Pruning Approach for Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.11695.

[365] Richard S Sutton. Learning to Predict by the Methods of Temporal Differences. *Machine Learning*, 1988.

[366] Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction, 2018. URL http://incompleteideas.net/book/the-book-2nd.html.

[367] Peter Szolovits, Jon Doyle, William J. Long, Isaac Kohane, and Stephen G. Pauker. Guardian angel: Patient-centered health information systems. Technical Report TR-604, MIT Laboratory for Computer Science, 1994. URL https://groups.csail.mit.edu/medg/projects/ga/manifesto/GAtr.html.

[368] AutoDreamer Team. Autodreamer: Autonomous agent memory consolidation via offline reinforcement learning. arXiv preprint arXiv:2605.20616, 2025. URL https://arxiv.org/abs/2605.20616.

[369] Meta PyTorch Team. TorchForge: PyTorch-Native Post-Training at Scale, 2025. URL https://github.com/meta-pytorch/torchforge.

[370] PyTorch Team. Torchtune: PyTorch Native Post-Training Library, 2024. URL https://github.com/pytorch/torchtune.

[371] Qwen Team. Qwen2.5: A Party of Foundation Models. *arXiv Preprint arXiv:2412.15115*, 2024. URL https://arxiv.org/abs/2412.15115.

[372] Qwen Team. QwQ: Reflect Deeply on the Boundaries of the Unknown. *Qwen Blog*, 2024. URL https://qwenlm.github.io/blog/qwq-32b-preview/.

[373] Qwen Team. Qwen3 Technical Report. *arXiv Preprint arXiv:2505.09388*, 2025.

[374] Adly Templeton, Tom Conerly, Jonathan Marcus, et al. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html.

[375] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function Vectors in Large Language Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.15213.

[376] Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. *NeurIPS Datasets and Benchmarks*, 2024. URL https://arxiv.org/abs/2407.17032.

[377] Endel Tulving. Memory and Consciousness. *Canadian Psychology / Psychologie Canadienne*, 1985.

[378] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1706.03762.

[379] Vercel. Vercel AI SDK, 2024. URL https://sdk.vercel.ai.

[380] Jesse Vig. A Multiscale Visualization of Attention in the Transformer Model. In *Proceedings of the 57th ACL: System Demonstrations*, 2019. URL https://arxiv.org/abs/1906.05714.

[381] Ashwin K. Vijayakumar, Michael Cogswell, Ramprasaath R. Selvaraju, et al. Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models. *AAAI*, 2018.

[382] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. URL https://arxiv.org/abs/1905.09418.

[383] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv Preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

[384] Junkang Wang, Jianfei Duan, Yue Liu, Zhangchen Yue, Hanghang Tong, and James Wang. Beyond Reverse KL: Generalizing Direct Preference Optimization with Diverse Divergence Constraints. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.16240.

[385] Lei Wang, Wanyu Xu, Yiber Lan, et al. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://arxiv.org/abs/2305.04091.

[386] Peiyi Wang, Lei Li, Zhihong Shao, et al. Math-Shepherd: Verify and Reinforce LLMs Step-by-Step Without Human Annotations. *arXiv Preprint arXiv:2312.08935*, 2024. URL https://arxiv.org/abs/2312.08935.

[387] Xingyao Wang, Boxuan Ding, Ziniu Hoang, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv Preprint arXiv:2407.16741*, 2024.

[388] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2203.11171.

[389] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2203.11171.

[390] Yiduo Wang et al. RLFactory: A Plug-and-Play Framework for Multi-Round Tool-Use RL. arXiv preprint, 2025.

[391] Yifan et al Wang. Improving Data Efficiency for LLM Reinforcement Fine-Tuning via Difficulty-Targeted Online Data Selection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://arxiv.org/abs/2506.05316.

[392] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. *arXiv Preprint arXiv:2212.10560*, 2022. URL https://arxiv.org/abs/2212.10560.

[393] Zhihao Wang et al. Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts. arXiv preprint, 2024.

[394] Christopher J. C. H Watkins. Learning from Delayed Rewards. 1989.

[395] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2201.11903.

[396] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atoosa Kasirzadeh, et al. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*, 2021. URL https://arxiv.org/abs/2112.04359.

[397] Gerhard Weiss and ed. Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence, 1999.

[398] Leandro von Werra, Younes Belkada, Lewis Tunstall, et al. TRL: Transformer Reinforcement Learning, 2022. URL https://github.com/huggingface/trl.

[399] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory Networks. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1410.3916.

[400] Brandon T. Willard and Rémi Louf. Efficient Guided Generation for Large Language Models. *arXiv Preprint arXiv:2307.09702*, 2023. URL https://arxiv.org/abs/2307.09702.

[401] Ronald J Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 1992. URL https://link.springer.com/article/10.1007/BF00992696.

[402] Edwin B Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 1927. URL https://www.jstor.org/stable/2276774.

[403] Michael Wooldridge. An Introduction to MultiAgent Systems, 2009.

[404] Michael Wooldridge, Nicholas R. Jennings, and David Kinny. The Gaia Methodology for Agent-Oriented Analysis and Design. *Autonomous Agents and Multi-Agent Systems*, 2000.

[405] BigScience Workshop. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv Preprint arXiv:2211.05100*, 2023. URL https://arxiv.org/abs/2211.05100.

[406] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.10813.

[407] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv Preprint arXiv:2308.08155*, 2023.

[408] Yifan Wu et al. Long-Horizon-Terminal-Bench: Dense Reward Grading for Command-Line Agents. arXiv preprint, 2026.

[409] Zhengbao Wu, Aman Arora, Zhiqiang Wang, Byung-Gon Kim, and Tian Huang. Retrieval Head Mechanistically Explains Long-Context Factuality. *arXiv Preprint arXiv:2404.15574*, 2024. URL https://arxiv.org/abs/2404.15574.

[410] Zhenyu Wu, Qinghua Hu, Yin Zhang, Yiming Gao, and Jiangtao Chen. An Empirical Analysis of Compute-Optimal Inference for Problem-Solving with Language Models. *arXiv Preprint arXiv:2408.00724*, 2024.

[411] xAI. Grok 4.5: Training on Real Developer Session Traces. Technical report, 2026.

[412] Zhiheng Xi, Yiwen Ding, Wenxiang Chen, et al. AgentGym: Evolving Large Language Model-Based Agents Across Diverse Environments. *arXiv Preprint arXiv:2406.04151*, 2024. URL https://arxiv.org/abs/2406.04151.

[413] Fen Xia, Tie-Yan Liu, Jue Wang, Wensheng Zhang, and Hang Li. Listwise Approach to Learning to Rank: Theory and Algorithm. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 2008.

[414] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.10438.

[415] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17453.

[416] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17453.

[417] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged Resources to Advance General Chinese Embedding. *arXiv Preprint arXiv:2309.07597*, 2023.

[418] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An Explanation of in-Context Learning as Implicit Bayesian Inference. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2111.02080.

[419] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, 2024.

[420] Can Xu, Qingfeng Sun, Kai Zheng, et al. WizardLM: Empowering Large Language Models to Follow Complex Instructions. *arXiv Preprint arXiv:2304.12244*, 2023. URL https://arxiv.org/abs/2304.12244.

[421] Haoxiang Xu, Hongyuan Zhao, Ying Liu, and Dong Wei. It Takes Two: Pairwise Preference Optimization with Two Rollouts. *arXiv Preprint arXiv:2505.07856*, 2025. URL https://arxiv.org/abs/2505.07856.

[422] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[423] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective Retrieval Augmented Generation. *arXiv Preprint arXiv:2401.15884*, 2024. URL https://arxiv.org/abs/2401.15884.

[424] Chengrun Yang, Xuezhi Wang, Yifeng Lu, et al. Large Language Models as Optimizers. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.03409.

[425] Jingwen Yang, Yuxuan Zhu, Binyuan Wang, et al. ARQ: Attentive Reasoning Queries for Multi-Hop Question Answering over Long Contexts. *arXiv Preprint arXiv:2501.08290*, 2025. URL https://arxiv.org/abs/2501.08290.

[426] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2405.15793.

[427] Zhilin Yang, Peng Qi, Saizheng Zhang, et al. HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering. In *Proceedings of EMNLP*, 2018.

[428] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. XLNet: Generalized Autoregressive Pretraining for Language Understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[429] Shunyu Yao, Dian Yu, Jeffrey Zhao, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.10601.

[430] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

[431] Shunyu Yao, Dian Yu, Jeffrey Zhao, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2305.10601.

[432] Zhewei Yao, Samyam Rajbhandari, Reza Yazdani Aminabadi, et al. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-Like Models at All Scales. *arXiv Preprint arXiv:2308.01320*, 2023.

[433] Fangchen Ye, Yifei Lu, Hao Dong, Hang Su, and Jian Song. Harness-zero: Harness distillation via agent-as-harness. arXiv preprint arXiv:2609.24974, 2026. URL https://arxiv.org/abs/2609.24974.

[434] Jaeseok Yoo and Dongmin Shin. Adaptive Episode Length Adjustment for Multi-Agent Reinforcement Learning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2025.

[435] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[436] Qiying Yu, Zheng Sun, Shang Wen, et al. DAPO: An Open-Source LLM Reinforcement Learning System. *arXiv Preprint arXiv:2503.14476*, 2025. URL https://arxiv.org/abs/2503.14476.

[437] Xiao Yu, Baolin Peng, Ruize Xu, et al. Reinforcement World Model Learning for LLM-Based Agents. *arXiv Preprint arXiv:2602.05842*, 2026.

[438] Ted Zadouri, Jay Shah, Ganesh Bikshandi, and Tri Dao. FlashAttention-4: Hardware-Efficient Attention on Blackwell GPUs with Minimal Software Design. *arXiv Preprint arXiv:2603.05451*, 2026. URL https://arxiv.org/abs/2603.05451.

[439] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, et al. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2007.14062.

[440] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. BitFit: Simple Parameter-Efficient Fine-Tuning for Transformer-Based Masked Language-Models. In *Proceedings of the 60th Annual Meeting of the ACL*, 2022. URL https://arxiv.org/abs/2106.10199.

[441] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning with Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2203.14465.

[442] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking. *arXiv Preprint arXiv:2403.09629*, 2024. URL https://arxiv.org/abs/2403.09629.

[443] Alex L. Zhang, Seyyed Hasan Mahdavi, Percy Liang, and Tatsunori Hashimoto. Recursive Language Models. *arXiv Preprint arXiv:2512.24601*, 2025. URL https://arxiv.org/abs/2512.24601.

[444] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1910.07467.

[445] Jiayi Zhang, Simon Yu, Derek Chong, et al. Verbalized Sampling: How to Mitigate Mode Collapse and Unlock LLM Diversity. *arXiv Preprint arXiv:2510.01171*, 2025. URL https://arxiv.org/abs/2510.01171.

[446] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms. *Handbook of Reinforcement Learning and Control*, 2021. URL https://arxiv.org/abs/1911.10635.

[447] Qingru Zhang, Minshuo Chen, Alexander Bukharin, et al. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *arXiv Preprint arXiv:2303.10512*, 2023. URL https://arxiv.org/abs/2303.10512.

[448] Tianjun Zhang, Shishir G. Patil, Naman Jain, et al. RAFT: Adapting Language Model to Domain Specific RAG. *arXiv Preprint arXiv:2403.10131*, 2024. URL https://arxiv.org/abs/2403.10131.

[449] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09675.

[450] Wei Zhang et al. Self-Evolving Software Agents: Combining BDI Architectures with LLMs. arXiv preprint, 2026.

[451] Xinrong Zhang, Yingfa Chen, Shengding Hu, et al. InfiniteBench: Extending Long Context Evaluation Beyond 100K Tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2402.13718.

[452] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.14048.

[453] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. In *Proceedings of ICLR 2026*, 2026. URL https://arxiv.org/abs/2601.18734. arXiv preprint arXiv:2601.18734.

[454] Yanli Zhao, Andrew Gu, Rohan Varma, et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. In *Proceedings of the VLDB Endowment*, 2023.

[455] Yao Zhao, Rishabh Joshi, Tianqi Liu, Misha Khalman, Mohammad Saleh, and Peter J. Liu. SLiC-HF: Sequence Likelihood Calibration with Human Feedback. *arXiv Preprint arXiv:2305.10425*, 2023. URL https://arxiv.org/abs/2305.10425.

[456] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.05685.

[457] Zhipu AI. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *arXiv preprint arXiv:2508.06471*, 2025. URL https://arxiv.org/abs/2508.06471.

[458] Zhipu AI. GLM-5: From Vibe Coding to Agentic Engineering. *arXiv preprint arXiv:2602.15763*, 2026. URL https://arxiv.org/abs/2602.15763.

[459] Yanqi Zhong, Yifu Chen, Zijun Li, and Minlie Chen. Importance Sampling Corrections for Large Language Model Alignment with Asynchronous Generation. *arXiv Preprint arXiv:2503.09057*, 2025. URL https://arxiv.org/abs/2503.09057.

[460] Yanqi Zhong, Jiaqi Shi, Yifu Chen, and Minlie Chen. GDPO: Learning to Directly Align Language Models with Group-Decoupled Reward. *arXiv Preprint arXiv:2501.17888*, 2025. URL https://arxiv.org/abs/2501.17888.

[461] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *arXiv Preprint arXiv:2310.04406*, 2024.

[462] Chunting Zhou, Pengfei Liu, Puxin Xu, et al. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.11206.

[463] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.13854.

[464] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, et al. Large Language Models Are Human-Level Prompt Engineers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2211.01910.

[465] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, et al. Fine-Tuning Language Models from Human Preferences. *arXiv Preprint arXiv:1909.08593*, 2019. URL https://arxiv.org/abs/1909.08593.

[466] Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv Preprint arXiv:2307.15043*, 2023. URL https://arxiv.org/abs/2307.15043.

Appendix A

Mathematical Background

This appendix collects the mathematical prerequisites assumed throughout the book. No derivations are given; the goal is to provide a compact reference so that readers can identify gaps and fill them before engaging with the main text. Each section points to the chapters where the concept is most heavily used.

A.1 Linear Algebra

Used in: Chapters 1–3 (attention, embeddings, weight matrices), Chapter 2 (GPU memory), Chapters 4–12 (policy gradient derivations).

Vectors. A **vector** $\mathbf{x} \in \mathbb{R}^n$ is an ordered list of n real numbers, written as a column:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Vectors support element-wise addition ($\mathbf{x} + \mathbf{y}$) and scalar multiplication ($c\mathbf{x}$). In LLMs, token embeddings, hidden states, and attention queries/keys/values are all vectors.

Matrices. A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ has m rows and n columns. Entry A_{ij} is in row i, column j. The **transpose** $\mathbf{A}^\top \in \mathbb{R}^{n \times m}$ swaps rows and columns: $(\mathbf{A}^\top)_{ij} = A_{ji}$. Weight matrices in transformer layers ($\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, $\mathbf{W}_O$, $\mathbf{W}_{\text{ff}}$) are all matrices.

Matrix multiplication. The product $\mathbf{C} = \mathbf{AB}$ requires $\mathbf{A} \in \mathbb{R}^{m \times k}$ and $\mathbf{B} \in \mathbb{R}^{k \times n}$, yielding $\mathbf{C} \in \mathbb{R}^{m \times n}$:

$$C_{ij} = \sum_{l=1}^{k} A_{il} B_{lj}$$

Matrix–vector multiplication $\mathbf{Ax}$ (where $\mathbf{x} \in \mathbb{R}^n$) is the special case $k = n$, $m = 1$, producing a vector in $\mathbb{R}^m$. This is the core operation in every linear layer: $\mathbf{h} = \mathbf{Wx} + \mathbf{b}$.

Key identities:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top \tag{A.1}$$

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC}) \quad \text{(associative)} \tag{A.2}$$

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \quad \text{(when inverses exist)} \tag{A.3}$$

Note: matrix multiplication is *not* commutative in general ($\mathbf{AB} \neq \mathbf{BA}$).

Special matrices.

- **Identity matrix $\mathbf{I}_n$**: diagonal entries are 1, all others 0. $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$.
- **Diagonal matrix**: non-zero entries only on the main diagonal. Scaling operation.
- **Symmetric matrix**: $\mathbf{A} = \mathbf{A}^\top$. Covariance matrices and Gram matrices are symmetric.
- **Orthogonal matrix**: $\mathbf{Q}^\top\mathbf{Q} = \mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$. Columns are orthonormal. Orthogonal transformations preserve lengths and angles (rotations, reflections).
- **Positive semi-definite (PSD)**: $\mathbf{A}$ is PSD if $\mathbf{x}^\top\mathbf{A}\mathbf{x} \geq 0$ for all $\mathbf{x}$. All eigenvalues are ≥ 0. Attention score matrices and covariance matrices are PSD.

Dot product and inner product. The **dot product** of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top\mathbf{y} = \sum_{i=1}^{n} x_i y_i$$

It measures alignment: positive when vectors point in the same direction, zero when orthogonal, negative when opposing. The **scaled dot-product attention** in transformers computes $\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}$, which is a matrix of dot products between all query–key pairs.

Norms. A **norm** $\|\cdot\|$ measures the size of a vector. The most common:

- ℓ_1 norm: $\|\mathbf{x}\|_1 = \sum_i |x_i|$ (sum of absolute values; promotes sparsity)
- ℓ_2 norm (Euclidean): $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} = \sqrt{\mathbf{x}^\top\mathbf{x}}$ (geometric length)
- ℓ_∞ norm: $\|\mathbf{x}\|_\infty = \max_i |x_i|$ (largest absolute entry)

The ℓ_2 norm appears in weight decay regularization ($\lambda\|\theta\|_2^2$), gradient clipping, and RMS normalization (RMSNorm divides by $\|\mathbf{x}\|_2/\sqrt{n}$).

Cosine similarity.

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top\mathbf{y}}{\|\mathbf{x}\|_2\|\mathbf{y}\|_2} \in [-1, 1]$$

Value $+1$: identical direction. Value 0: orthogonal. Value -1: opposite direction. Used in embedding retrieval (RAG, Chapter 16) and similarity search.

Linear independence and rank. Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are **linearly independent** if no vector can be written as a linear combination of the others. The **rank** of a matrix $\mathbf{A}$ is the maximum number of linearly independent rows (or columns). A matrix is **full rank** if $\text{rank}(\mathbf{A}) = \min(m, n)$. Low-rank structure is central to LoRA (Chapter 3): weight updates $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$ are constrained to rank $r \ll \min(m, n)$, drastically reducing trainable parameters.

Eigenvalues and eigenvectors. For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, a non-zero vector $\mathbf{v}$ is an **eigenvector** with **eigenvalue** λ if:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

Intuitively, $\mathbf{A}$ only scales $\mathbf{v}$, not rotates it. A symmetric matrix has n real eigenvalues and orthogonal eigenvectors. The **eigendecomposition** $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (for symmetric $\mathbf{A}$) expresses the matrix in terms of its eigenvectors (columns of $\mathbf{Q}$) and eigenvalues (diagonal of $\mathbf{\Lambda}$). Eigenvalues govern the stability of gradient flow: if the Hessian of the loss has large eigenvalues, training is sensitive to learning rate.

Singular Value Decomposition (SVD). Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ (not necessarily square) can be factored as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ called **singular values**. The columns of $\mathbf{U}$ are **left singular vectors**; columns of $\mathbf{V}$ are **right singular vectors**.

This is the mathematical foundation of LoRA: pre-trained weight matrices have approximately low-rank structure, so their updates can be efficiently parameterized as $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$ with $\text{rank}(\Delta\mathbf{W}) = r$.

Tensors. A **tensor** is a multi-dimensional generalization of vectors (1D) and matrices (2D). A 3D tensor $\mathcal{X} \in \mathbb{R}^{a \times b \times c}$ has three axes. In deep learning, activations are typically 3D tensors of shape $(\text{batch}, \text{sequence}, \text{hidden})$. Attention scores are 4D: $(\text{batch}, \text{heads}, \text{seq}, \text{seq})$. PyTorch's `torch.Tensor` and NumPy's `ndarray` implement tensor operations with automatic broadcasting.

Broadcasting. When operating on tensors of different shapes, **broadcasting** automatically expands dimensions of size 1 to match the other operand. For example, adding a bias vector $\mathbf{b} \in \mathbb{R}^d$ to a batch of activations $\mathbf{H} \in \mathbb{R}^{B \times d}$ broadcasts $\mathbf{b}$ across the batch dimension. Understanding broadcasting is essential for reading and writing efficient PyTorch code.

Trace and determinant. For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$:

- **Trace**: $\text{tr}(\mathbf{A}) = \sum_i A_{ii}$ (sum of diagonal entries = sum of eigenvalues).
- **Determinant**: $\det(\mathbf{A})$ measures the volume scaling factor of the linear transformation. $\det(\mathbf{A}) = 0$ iff $\mathbf{A}$ is singular (non-invertible). $\det(\mathbf{A}) = \prod_i \lambda_i$ (product of eigenvalues).

Linear systems and least squares. A **linear system** $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a unique solution when $\mathbf{A}$ is square and full rank: $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. When the system is overdetermined ($m > n$), the **least-squares solution** minimizes $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$:

$$\mathbf{x}^* = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}$$

This is the **normal equation**. Gradient descent on the squared loss converges to this solution for linear models.

Connection to attention. The self-attention mechanism in transformers is fundamentally a sequence of linear algebra operations:

1. Project inputs: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ (matrix multiplications)
2. Compute scores: $\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}$ (scaled dot products)
3. Normalize: $\mathbf{P} = \text{softmax}(\mathbf{S})$ (row-wise softmax)
4. Aggregate: $\mathbf{O} = \mathbf{P}\mathbf{V}$ (weighted sum of value vectors)

Every step is a linear algebra operation. Understanding matrix shapes and the role of each projection matrix is essential for reasoning about attention head behavior, memory cost, and the effect of architectural choices like GQA (grouped-query attention).

A.2 Calculus and Automatic Differentiation

Used in: Chapters 1–3 (backpropagation), Chapters 4–12 (policy gradients, loss functions).

Chapter 1, Optimization Theory covers gradient descent, SGD, vanishing/exploding gradients, mixed-precision training, and gradient clipping in full, with figures and code. This section covers the two topics not treated there.

Derivatives, gradients, chain rule, and autograd. The gradient $\nabla_{\mathbf{x}} f \in \mathbb{R}^n$ is the vector of partial derivatives; it points in the direction of steepest ascent. Backpropagation is repeated application of the chain rule through a computation graph; `loss.backward()` computes all $\partial \mathcal{L}/\partial \theta_i$ in one reverse pass. *See Chapter 1, Gradient Descent and Adam.*

Log-space arithmetic and numerical stability. Probabilities of long sequences are products of many small numbers that underflow to zero in floating point. The fix is to work in **log-space**:

$$\log p(\mathbf{x}) = \sum_t \log p(x_t \mid x_{<t})$$

This is why policy gradient objectives use $\log \pi_\theta(a_t \mid s_t)$ rather than $\pi_\theta(a_t \mid s_t)$ directly. The **log-sum-exp** trick computes $\log \sum_i e^{z_i}$ stably by subtracting the maximum:

$$\log \sum_i e^{z_i} = c + \log \sum_i e^{z_i - c}, \quad c = \max_i z_i$$

This is used inside every softmax computation to prevent overflow.

Hessian and second-order methods. The **Hessian** $\mathbf{H} \in \mathbb{R}^{n \times n}$ has entries $H_{ij} = \frac{\partial^2 \mathcal{L}}{\partial \theta_i \partial \theta_j}$. It describes the curvature of the loss landscape: large eigenvalues indicate sharp curvature (sensitive to learning rate); small eigenvalues indicate flat regions. Full second-order optimization requires computing $\mathbf{H}^{-1}$, which is infeasible for billion-parameter models. Practical approximations include diagonal Hessian estimates (Adafactor) and Kronecker-factored curvature (K-FAC).

A.3 Probability and Statistics

Used in: Chapters 3–12 (loss functions, sampling, reward modeling), Chapter 13 (reasoning under uncertainty), Chapters 15–28 (agent decision-making).

Probability distributions. A **probability distribution** $p(x)$ assigns non-negative weights to outcomes summing to 1. The three distributions most relevant to this book:

- **Categorical**: $p(x = k) = \pi_k$, $\sum_k \pi_k = 1$. The output of every language model is a categorical distribution over the vocabulary at each step.
- **Gaussian** (Normal): $p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$. Appears in reward model noise assumptions and continuous action spaces.
- **Bernoulli**: $p(x = 1) = p$, $p(x = 0) = 1 - p$. Binary outcomes; used in preference modeling (preferred vs. rejected).

Expectation and variance.

$$\mathbb{E}[X] = \sum_x x\, p(x), \qquad \mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

Policy gradient methods maximize $\mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$. Variance reduction via baselines and advantage estimation is a central concern because high-variance gradient estimates slow convergence.

Conditional probability and Bayes' theorem. $p(A \mid B) = p(A \cap B)/p(B)$. **Bayes' theorem**: $p(\theta \mid \mathcal{D}) = p(\mathcal{D} \mid \theta)\, p(\theta)/p(\mathcal{D})$, relating posterior, likelihood, and prior. Used in Bayesian confidence tracking for cognitive escalation (Chapter 19) and in the derivation of DPO from the Bradley–Terry preference model (Chapter 6).

Log-likelihood and maximum likelihood estimation. The **log-likelihood** of parameters θ given data $\{x_1, \ldots, x_N\}$:

$$\log \mathcal{L}(\theta) = \sum_i \log p_\theta(x_i)$$

MLE finds $\theta^* = \arg\max_\theta \log \mathcal{L}(\theta)$. Pre-training on next-token prediction is MLE: $\theta^* = \arg\max_\theta \sum_t \log p_\theta(x_t \mid x_{<t})$.

KL divergence.

$$D_{\mathrm{KL}}(p\|q) = \mathbb{E}_{x\sim p}\left[\log \frac{p(x)}{q(x)}\right] \geq 0 \tag{A.4}$$

$D_{\mathrm{KL}} = 0$ iff $p = q$; not symmetric. The central regularizer in RLHF: PPO penalizes $D_{\mathrm{KL}}(\pi_\theta\|\pi_{\mathrm{ref}})$ to prevent policy drift (Chapter 5); DPO is derived by solving the KL-constrained reward maximization in closed form (Chapter 6).

Cross-entropy loss. $H(p, q) = -\sum_x p(x) \log q(x) = H(p) + D_{\mathrm{KL}}(p\|q)$. For one-hot p, this reduces to $-\log q(x^*)$. Minimizing cross-entropy is equivalent to maximizing log-likelihood and to minimizing KL divergence from the data distribution.

Softmax and temperature. $\mathrm{softmax}(\mathbf{z})_i = e^{z_i}/\sum_j e^{z_j}$; temperature scaling uses $\mathbf{z}/T$. *Full treatment with all sampling strategies (top-k, top-p, min-p, contrastive decoding) in Chapter 1, Text Generation.*

Importance sampling and Monte Carlo estimation. $\mathbb{E}_{x\sim p}[f(x)] = \mathbb{E}_{x\sim q}[(p(x)/q(x))f(x)]$; the ratio p/q is the importance weight. In PPO, $r_t(\theta) = \pi_\theta/\pi_{\theta_{\mathrm{old}}}$ is clipped to $[1-\epsilon, 1+\epsilon]$. Monte Carlo estimation averages f over N samples; variance decreases as $O(1/N)$, motivating multiple rollouts per prompt (GRPO). *Full PPO derivation in Chapter 5; GRPO in Chapter 6.*

A.4 Information Theory

Used in: Chapters 4–12 (entropy regularization), Chapter 14 (evaluation metrics).

Entropy. The **Shannon entropy** of a distribution p measures its uncertainty:

$$H(p) = -\sum_x p(x) \log p(x) \geq 0 \tag{A.5}$$

$H = 0$ for a deterministic distribution; $H = \log V$ for a uniform distribution over V outcomes. **Entropy regularization** in RL adds $\alpha H(\pi_\theta)$ to the reward, preventing premature collapse to a single action (Chapter 5).

KL–cross-entropy relationship. The three quantities are linked by:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q)$$

Since $H(p)$ is fixed (depends only on the data), minimizing cross-entropy is equivalent to minimizing KL divergence. This identity connects the pre-training objective (cross-entropy minimization) to the RLHF objective (KL-constrained reward maximization).

Perplexity. $\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_t \log p(x_t \mid x_{<t})\right)$: exponentiated average negative log-likelihood per token. Lower is better. *Full discussion including cross-tokenizer comparisons and use as a sanity check in Chapter 14, Perplexity.*

A.5 Key Notation Summary

Table A.1: Notation used throughout the book.

Symbol	Meaning
θ	Model parameters (vector of all weights and biases)
π_θ	Policy (language model) parameterized by θ
π_{ref}	Reference policy (frozen pre-trained model used as KL anchor)
$\mathcal{L}(\theta)$	Loss function
$\nabla_\theta \mathcal{L}$	Gradient of loss with respect to θ
η	Learning rate
λ	Weight decay coefficient or GAE parameter (context-dependent)
γ	Discount factor
ϵ	PPO clipping range or small numerical constant (context-dependent)
$\mathbb{E}[\cdot]$	Expectation
$\text{Var}[\cdot]$	Variance
$D_{\text{KL}}(p\|q)$	KL divergence from reference q to distribution p
$H(p)$	Shannon entropy of distribution p
$H(p, q)$	Cross-entropy of q relative to p
$\mathbf{x}, \mathbf{W}$	Vectors and matrices (bold lowercase / bold uppercase)
x_i	i-th element of vector $\mathbf{x}$
$\|\mathbf{x}\|_2$	ℓ_2 (Euclidean) norm of $\mathbf{x}$
$\hat{y}$	Predicted / estimated value of y
τ	Trajectory $(s_0, a_0, r_0, s_1, \ldots)$
G_t	Discounted return from time step t
$V^\pi(s)$	State-value function under policy π
$Q^\pi(s, a)$	Action-value function under policy π
$A^\pi(s, a)$	Advantage function
$r_t(\theta)$	Probability ratio $\pi_\theta(a_t\|s_t)/\pi_{\theta_{\text{old}}}(a_t\|s_t)$
T	Temperature (softmax scaling) or sequence length (context-dependent)
V	Vocabulary size
d_{model}	Model hidden dimension
d_k	Key/query dimension per attention head
n_{heads}	Number of attention heads
L	Number of transformer layers
B	Batch size
β_1, β_2	Adam momentum coefficients